COMP5583 Prompt Appendix

- COMP5583 Prompt Appendix
  - Purpose and reading guide
  - Task 1 — V0 baseline
  - Task 2 — Fixed evaluator
  - Task 3 — V1 In-context Few-Shot Learning
  - Task 3 — V2 Retrieval-Augmented Generation
  - Preserved records
  - README
- Task 1 · V0 交互与构建记录
  - 原文可核：本轮提供的改版回顾
  - 重建提示词记录（非逐字原文）
  - 原始模型输出说明
  - V0 冻结前修复记录
  - V0 CODE OUTPUT EXCERPTS
- V0 代码输出证据索引与关键片段
  - 1. 页面加载本地嵌入语料
  - 2. RAG 检索源清单与双语分词
  - 3. 项目详情与收藏控件
  - 4. 客人留言本地存储
  - ai judge evidence
- V1 AI-Judge Evidence Manifest (for a separate blank session)
  - Latest V1 evidence after structural/visual revisions
  - V1 iteration 4 evidence (Life scrapbook wall)
  - V1 iteration 5 evidence (component architecture rebuild)
  - V1 iteration 6 evidence (interaction clarity)
  - change log
- V1 Generation and Change Log
  - Lineage and freeze check
  - Generated files and changes
  - Necessary bug fixes before V1 test
  - Generated source output
  - Verification correction — 2026-10-23
  - Browser-test iteration record — 2026-10-23
  - Live-release boundary-case correction — 2026-10-23
  - Shared browser protocol timing correction — 2026-10-23
  - User feedback and structural revision — V1 iteration 2 — 2026-10-23
  - Life scrapbook wall and SVG marginalia — V1 iteration 4 — 2026-10-23
  - Copy-preserving component architecture rebuild — V1 iteration 5 — 2026-10-23
  - Screenshot-led interaction correction — V1 iteration 6 — 2026-10-23

- Iteration 7 — page skeleton rebuild (2026-10-27)
- Iteration 9 · 2026-10-27 · full-bleed editorial skeleton (replaces the app shell)
- Iteration 10 · 2026-10-27 · Life wall rebuilt as a compact mosaic
- Iteration 11 · 2026-10-28 · caption plate moved off the hero photograph
- Iteration 12 — the skill lines open (2026-10-28)
- few shot examples

V1 Few-Shot Demonstrations and Reference Mapping
- Example outputs supplied to generation
- Why these examples are few-shot learning
- few shot examples revision2

Few-Shot Demonstrations — Structural Revision 2
- 1. Long-page chapters
- 2. Carousel scene selection
- 3. Project stories
- 4. Filtered reflow
- few shot examples revision3

Few-Shot Demonstrations — Visual Refinement 3
- 1. Editorial navigation instead of duplicate tile cards
- 2. Artwork-first media controls
- 3. Restrained interaction feedback
- few shot examples revision4

Few-Shot Demonstrations — Life Media Wall and Marginalia (Iteration 4)
- 1. Asymmetric media wall with keyboard-accessible viewing
- 2. Hand-drawn SVG annotations as restrained page marginalia
- 3. Preserve the owner's source material
- few shot examples revision5

Few-Shot Examples — V1 Component Architecture (Iteration 5)
- Example 1 — Editorial academic profile
- Example 2 — Image-led art portfolio
- Example 3 — Interactive project portfolio
- Example 4 — Personal visual archive
- Few-shot transfer constraints
- few shot examples revision7

Few-Shot Examples — V1 App Shell (Iteration 7)
- Reference sweep (2026-10-27)
- Example 1 — Bento mosaic (skeleton A)
- Example 2 — Profile rail plus content window (skeleton B, chosen)
- Example 3 — Editorial split spread (skeleton C)
- Few-shot transfer constraints
- gallery pattern sweep

Gallery pattern sweep · iteration 10 (2026-10-27)
- 1. What was actually wrong (measured, not guessed)
- 2. References consulted

- 3. Rendered candidates
- 4. Why D
- 5. Rejected on the way
- 6. Costs, stated plainly
- 7. Test change record
- icl prompt

V1 Generation Prompt — In-context Few-Shot Learning
- Prompt
- Recorded implementation boundary
- icl prompt revision10

Task 3 ・ V1 prompt, revision 10 — take the caption strip off the photograph
- icl prompt revision11

Task 3 ・ V1 prompt, revision 11 — make the skill lines open
- icl prompt revision2

V1 Few-Shot Prompt — Structural Revision 2
- Full prompt used for the structural revision
- Technique record
- icl prompt revision3

V1 Few-Shot Prompt — Visual Refinement 3
- Full refinement instruction
- icl prompt revision4

V1 Few-Shot Prompt — Life Media Wall and Journal Marginalia (Iteration 4)
- Full instruction
- Iteration lineage
- icl prompt revision5

V1 Few-Shot Prompt — Component Architecture Rebuild (Iteration 5)
- Full instruction
- Few-shot inputs
- Iteration lineage
- icl prompt revision6

V1 Usability Correction Prompt — Click Guidance and Isolated Disclosures (Iteration 6)
- User feedback (verbatim)
- Implementation instruction derived from the feedback
- Applied corrections
- Verification
- icl prompt revision7

Generation Prompt — Iteration 7 (App Shell)
- Context handed to the model
- Candidates returned
- Implementation rules derived for the chosen skeleton
- Recorded deviations
- icl prompt revision8

Generation Prompt — Iteration 9 (Full-bleed Editorial)

- Context handed to the model
- Directions offered
- Rules extracted from the direction
- Deviations recorded during implementation
- Verification of this prompt
- icl prompt revision9

Generation Prompt — Iteration 10 (Life gallery, compact mosaic)

- Context handed to the model
- Constraints the composition had to satisfy
- What the model produced
- Deviations recorded
- Verification
- 5205 Introduction to AI

5205 Introduction of AI

Lecture 1

- Part 1

Lecture 2

Overview

- Part 1 Searching
- Part 2 Generate & Test
- Part 3 广度优先搜索
- Part 4 深度优先搜索与两者对比
- Part 5 启发式的有信息搜索： Hill Climbing, Best-first and A*
- Part 6 从搜索到优化 Optimization
- Part 7 模拟退火 Simulated Annealing

Part 8 遗传算法 Genetic Algorithms

- Part 9 Tutorial 2

Lecture 3

- part 1 Search&Optimization Recap 搜索优化回顾
- part 2** **Evolutionary Algorithms 进化算法
- Part 3 Particle Swarm Optimization
- Part 4 Differential Evolution 差分进化
- Part5 Memetic Algorithms
- Part 6 Multi-objective Evolutionary Algorithms
- Part 7 Evolutionary Transfer Optimization (ETO)
- Part 8 Neural Architecture Search (NAS)
- Part9 Exercise3
- Part10 Overall

Lecture 4

- part 1 Neural Network
- Part 2 Model of a Neuron
- Part 3 Activation Functions
- Part 4 The Perceptron

- Part 5 Perceptron Learning
- Part 6 Multilayer Perceptron
- Part 7 Back-Propagation
- Part 8 Limitations of Back-Propagation
- Part 9 Overfitting and Underfitting
- Part 10 Case Study: Voice Recognition
- Part11 Exercise

Lecture 5

- Part 0 Introduction
- Part 1 From NN to Deep Learning
- Part 2 CNN I 卷积神经网络（卷积层与池化层）
- Part 3 CNN II 四个模型
- Part 4 序列神经网络 Sequential Neural Networks
- Part 5 GNNs I: From Graphs to GNNs
- Part 6 GNNs II: Graph Convolution
- Part 7 Generative Models I 生成问题（一）
- Part 8 Generative Models II 生成问题（二）GANs and Diffusion Models
- Part 9 Overfitting and Underfitting 过拟合与欠拟合
- Part 10 Tutorial 5 Reinforcement Learning
- 10.10 TD 误差 How to Interpret the TD Error
- 10.11 Exploration and Exploitation： ε -greedy
- 10.12 Quick Check: ε -greedy Behavior
- 10.13 Why Q-learning Is Off-Policy
- 10.14 Convergence of Tabular Q-learning 表格型 Q - Learning 收敛条件
- 10.15 适用范围与局限（每一条对应一类后续算法）
- 5434 Big Data Computing 1

5434 Big Data Computing1

Lecture 4: Hadoop and MapReduce

- part 0 Navigation
- part 1 Hadoop Introduction
- part 2 Hadoop Economy
- part 3 MapReduce Introduction
- part 4 MapReduce Technical Ideas
- part 5 MapReduce Workflow and Programming
- part 6 Tutorial: Hadoop

Lecture 5 HDFS and Spark

- Part 1 HDFS
- Part 2 HDFS Technical Details
- Part 3 DataNodes and NameNode Illustration
- Part 4 HDFS 读和写操作
- Part 5 Hadoop’ s Job Scheduling 作业调度
- Part 6 Spark Motivation
- Part 7 Spark Technical Details

- Part 8 Spark Application
- Lecture 6 神经网络
  - Part 1 Neural Network：神经网络
  - Part 2 Perceptron：感知机
  - Part 3 多层神经网络与前向计算
  - Part 4 多分类输出与 Softmax
  - Part 5 代价函数与梯度下降
  - Part 6 反向传播：误差传回
  - Part 7 从神经网络到深度学习
  - Part 8 卷积神经网络
  - Part 9 GAN：生成对抗网络
  - Part 10 Overfitting and Underfitting：过拟合与欠拟合
  - 5434 Big Data Computing 2
- 5434 Big Data Computing2
  - Lecture 7 Graph Analytics
  - Part 1 Graphs and Applications
  - Part 2 Graph Traversal Methods：图遍历
  - Part 3 Centrality Measure
  - Part 4 Shortest Distance
  - Part 5 Vertex Similarity/ Proximity Measures 链路预测
  - Part 6 Community and Clustering Concepts：社区与聚类
- Lecture 8 Big Data Storage 大数据存储
  - Part 1 数据类型与存储系统选择 Types of Data
  - Part 2 关系型数据库与 ACID 事务
  - Part 3 数据库横向扩展 Scaling Out Databases
  - Part 4 分布式一致性与两阶段提交 2PC
  - Part 5 CAP 定理与 BASE 属性 CAP Theorem & BASE Properties
  - Part 6 NoSQL 数据库概述 Introduction to NoSQL
  - Part 7 四类 NoSQL 数据库 Various NoSQL Databases
  - 5583 Agentic Computing 1
- 5583 Agentic Computing1
- Lecture 4 Deep Reinforcement Learning II & Introduction to Prompt Engineering
  - Part 1 强化学习基础
  - Part 2 稀疏奖励、奖励塑形与课程学习
  - Part 3 深度强化学习的典型应用
  - Part 4 现代 LLM 智能体及信息来源
  - Part 5 思维链推理与问题分解
  - Part 6 多模态人工智能与内容生成 Multimodal AI and Content Generation
  - Part 7 使用 AI 进行数据分析 Data Analysis with AI
  - Part 8 人工智能的能力与责任 AI Capabilities and Responsibility
  - Part 9 使用 AI 构建应用 Building Apps with AI
  - Part 10 Take-home Points and Exercises
  - 5583 Agentic Computing 2

5583 Agentic Computing2
Lecture 5 Prompt Engineering II
- Part 1 Brainstorming
- Part 2 Context Management
- Part 3 In-Context Learning
- Part 4 推理 Reasoning & Sycophancy
- Part 5 Writing & Revision
- Part 6 Summary

Lecture 6: Retrieval-Augmented Generation (RAG)
- Part 0 Orientation
- Part 1 Why RAG?
- Part 2 RAG Architecture
- Part 3 Typical Retrieval Methods 典型检索方法
- Part 4 Retrieval Evaluation 检索评估

Lecture 7 Retrieval Optimization and Agentic RAG Systems 检索优化与智能体 RAG
- Part 1 Scalable Retrieval 可扩展检索
- Part 2 分块、查询改写与检索编码器
- Part 3 Advanced RAG Architectures 进阶 RAG 架构
- Part 4 RAG Systems Evaluation RAG 系统评估
- 5583 Agentic Computing 3

5583 Agentic Computing3
Lecture 8 智能体系统设计 Design of Agentic Systems
- Part 1 智能体系统基础 Agentic System Foundations
- Part 2 智能体工作流设计 Agentic Workflow Design
- Part 3 智能体推理与决策 Agentic Reasoning and Decision-Making
- Part 4 自反思与反馈优化 Self-Reflection and Feedback
- Part 5 Agentic RAG 与智能体工作流的综合应用
- Part 6 智能体的工具调用与动态决策 Tool Use and ReAct
- Part 6 智能体规划与结构化执行 Planning and Structured Execution
- Part 7 多智能体系统基础 Multi-Agent Fundamentals
- Part 8 多智能体系统设计 Designing Multi-Agent Systems
- 5584 LLM for Agent 1

5584 LLM for Agent1
Lecture 1 大语言模型与智能体导论 Course Overview
- Part 1 智能体与大语言模型的基本概念
- Part 2 智能体的发展历史：从规则系统到生成式 Agent
- Part 3 大语言模型的发展与规模化能力
- Part 4 大语言模型架构与注意力机制
- Part 5 大语言模型的训练与对齐
- Part 6 大语言模型推理与提示工程
- Part 7 LLM 评估、多模态与开放挑战
- Part 8 总结

Lecture 2 Anatomy of an LLM Agent

- Part 1 Agent
- Part 2 Agent 的组成
- Part 3 Perception 感知子系统
- Part 4 Reasoning 推理子系统
- Part 5 Meomory 记忆子系统
- Part 6 Execute 执行子系统
- Part 7 Failure and Evaluation 失败与评估

Lecture 3 Reasoning in LLMs (1)

- Part 0
- Part I Introduction of Reasoning
- Part 2 Basic Prompting Techniques
- Part 3 推理策略与结构
- Part 4 Search and Selection Methods
- 总结

Lecture 4 Reasoning in LLMs (2)

- Part 1 两种思维模式
- Part 2 LLM 推理能力争议
- Part 3 三条教它推理的路径
- Part 4 Post-training Techniques & DeepSeek-R1
- Part 5 强化学习与 DeepSeek-R1

Lecture 5 Planning in LLM Agents

- Part 1 Planning
- Part 2 Reasoning vs Acting
- Part 3 Planning Pressures I：模块化
- Part 4 Planning Pressures II：反馈
- Part 5 Planning Pressures III：长时间任务
- Part 6 Planning Pressures IV：控制
- Part 7 Classical Planning Meets LLMs
- Part 8 Three Solutions
- 5584 LLM for Agent 2

5584 LLM for Agent2

Lecture 6 Tool Use for LLM Agents

- Part 1 Capabilities 工具的能力
- Part 2 Mechanics 工具的机制
- Part 3 Constrained Decoding 工具的约束：只保证形式合法
- Part 4 REST API 工具接口
- Part 5 Model Context Protocol
- Part 6 Multiple Tool Calls：串行、并行与依赖
- Part 7 Evaluating Tool Use

Lecture 7 Agent Memory and RAG

- Part 1 RAG 的研究动机与基本原理
- Part 2 RA-LLM 的研究体系与标准架构
- Part 3 Retriever：检索器的分类、原理与训练

- Part 4 Pre-Retrieval Techniques：检索前优化
- Part 5 Post-Retrieval Techniques：检索后优化
- Part 6 Agent Memory 智能体记忆

Lecture 8 Agent Infrastructure 智能体基础设施

- Part1 长上下文推理与 KV Cache
- Part 2 KV Cache 的分层存储与跨设备管理
- Part 3 LMCache：独立的 KV Cache 基础设施
- Part 4 CacheBlend：非前缀 KV Cache 的选择性重计算
- Part 5 复合 AI 系统 Compound AI Systems
- Part 6 DSPy：从 Prompt Engineering 到 Language Model Programming
- 总结
- 5205 Introduction to AI

5205 Introduction of AI

Lecture 1

- Part 1

Lecture 2

Overview

- Part 1 Searching
- Part 2 Generate & Test
- Part 3 广度优先搜索
- Part 4 深度优先搜索与两者对比
- Part 5 启发式的有信息搜索： Hill Climbing, Best-first and A*
- Part 6 从搜索到优化 Optimization
- Part 7 模拟退火 Simulated Annealing

Part 8 遗传算法 Genetic Algorithms

- Part 9 Tutorial 2

Lecture 3

- part 1 Search&Optimization Recap 搜索优化回顾
- part 2** **Evolutionary Algorithms 进化算法
- Part 3 Particle Swarm Optimization
- Part 4 Differential Evolution 差分进化
- Part5 Memetic Algorithms
- Part 6 Multi-objective Evolutionary Algorithms
- Part 7 Evolutionary Transfer Optimization (ETO)
- Part 8 Neural Architecture Search (NAS)
- Part9 Exercise3
- Part10 Overall

Lecture 4

- part 1 Neural Network
- Part 2 Model of a Neuron
- Part 3 Activation Functions
- Part 4 The Perceptron
- Part 5 Perceptron Learning

- Part 6 Multilayer Perceptron
- Part 7 Back-Propagation
- Part 8 Limitations of Back-Propagation
- Part 9 Overfitting and Underfitting
- Part 10 Case Study: Voice Recognition
- Part11 Exercise

Lecture 5
- Part 0 Introduction
- Part 1 From NN to Deep Learning
- Part 2 CNN I 卷积神经网络（卷积层与池化层）
- Part 3 CNN II 四个模型
- Part 4 序列神经网络 Sequential Neural Networks
- Part 5 GNNs I: From Graphs to GNNs
- Part 6 GNNs II: Graph Convolution
- Part 7 Generative Models I 生成问题（一）
- Part 8 Generative Models II 生成问题（二）GANs and Diffusion Models
- Part 9 Overfitting and Underfitting 过拟合与欠拟合
- Part 10 Tutorial 5 Reinforcement Learning
- 10.10 TD 误差 How to Interpret the TD Error
- 10.11 Exploration and Exploitation： ε -greedy
- 10.12 Quick Check: ε -greedy Behavior
- 10.13 Why Q-learning Is Off-Policy
- 10.14 Convergence of Tabular Q-learning 表格型 Q - Learning 收敛条件
- 10.15 适用范围与局限（每一条对应一类后续算法）
- 5434 Big Data Computing 1

5434 Big Data Computing1

Lecture 4: Hadoop and MapReduce
- part 0 Navigation
- part 1 Hadoop Introduction
- part 2 Hadoop Economy
- part 3 MapReduce Introduction
- part 4 MapReduce Technical Ideas
- part 5 MapReduce Workflow and Programming
- part 6 Tutorial: Hadoop

Lecture 5 HDFS and Spark
- Part 1 HDFS
- Part 2 HDFS Technical Details
- Part 3 DataNodes and NameNode Illustration
- Part 4 HDFS 读和写操作
- Part 5 Hadoop’s Job Scheduling 作业调度
- Part 6 Spark Motivation
- Part 7 Spark Technical Details
- Part 8 Spark Application

Lecture 6 神经网络
- Part 1 Neural Network：神经网络
- Part 2 Perceptron：感知机
- Part 3 多层神经网络与前向计算
- Part 4 多分类输出与 Softmax
- Part 5 代价函数与梯度下降
- Part 6 反向传播：误差传回
- Part 7 从神经网络到深度学习
- Part 8 卷积神经网络
- Part 9 GAN：生成对抗网络
- Part 10 Overfitting and Underfitting：过拟合与欠拟合
- 5434 Big Data Computing 2

5434 Big Data Computing2
- Lecture 7 Graph Analytics
- Part 1 Graphs and Applications
- Part 2 Graph Traversal Methods：图遍历
- Part 3 Centrality Measure
- Part 4 Shortest Distance
- Part 5 Vertex Similarity/ Proximity Measures 链路预测
- Part 6 Community and Clustering Concepts：社区与聚类

Lecture 8 Big Data Storage 大数据存储
- Part 1 数据类型与存储系统选择 Types of Data
- Part 2 关系型数据库与 ACID 事务
- Part 3 数据库横向扩展 Scaling Out Databases
- Part 4 分布式一致性与两阶段提交 2PC
- Part 5 CAP 定理与 BASE 属性 CAP Theorem & BASE Properties
- Part 6 NoSQL 数据库概述 Introduction to NoSQL
- Part 7 四类 NoSQL 数据库 Various NoSQL Databases
- 5583 Agentic Computing 1

5583 Agentic Computing1

Lecture 4 Deep Reinforcement Learning II & Introduction to Prompt Engineering
- Part 1 强化学习基础
- Part 2 稀疏奖励、奖励塑形与课程学习
- Part 3 深度强化学习的典型应用
- Part 4 现代 LLM 智能体及信息来源
- Part 5 思维链推理与问题分解
- Part 6 多模态人工智能与内容生成 Multimodal AI and Content Generation
- Part 7 使用 AI 进行数据分析 Data Analysis with AI
- Part 8 人工智能的能力与责任 AI Capabilities and Responsibility
- Part 9 使用 AI 构建应用 Building Apps with AI
- Part 10 Take-home Points and Exercises
- 5583 Agentic Computing 2

5583 Agentic Computing2

Lecture 5 Prompt Engineering II
- Part 1 Brainstorming
- Part 2 Context Management
- Part 3 In-Context Learning
- Part 4 推理 Reasoning & Sycophancy
- Part 5 Writing & Revision
- Part 6 Summary

Lecture 6: Retrieval-Augmented Generation (RAG)
- Part 0 Orientation
- Part 1 Why RAG?
- Part 2 RAG Architecture
- Part 3 Typical Retrieval Methods 典型检索方法
- Part 4 Retrieval Evaluation 检索评估

Lecture 7 Retrieval Optimization and Agentic RAG Systems 检索优化与智能体 RAG
- Part 1 Scalable Retrieval 可扩展检索
- Part 2 分块、查询改写与检索编码器
- Part 3 Advanced RAG Architectures 进阶 RAG 架构
- Part 4 RAG Systems Evaluation RAG 系统评估
- 5583 Agentic Computing 3

5583 Agentic Computing3

Lecture 8 智能体系统设计 Design of Agentic Systems
- Part 1 智能体系统基础 Agentic System Foundations
- Part 2 智能体工作流设计 Agentic Workflow Design
- Part 3 智能体推理与决策 Agentic Reasoning and Decision-Making
- Part 4 自反思与反馈优化 Self-Reflection and Feedback
- Part 5 Agentic RAG 与智能体工作流的综合应用
- Part 6 智能体的工具调用与动态决策 Tool Use and ReAct
- Part 6 智能体规划与结构化执行 Planning and Structured Execution
- Part 7 多智能体系统基础 Multi-Agent Fundamentals
- Part 8 多智能体系统设计 Designing Multi-Agent Systems
- 5584 LLM for Agent 1

5584 LLM for Agent1

Lecture 1 大语言模型与智能体导论 Course Overview
- Part 1 智能体与大语言模型的基本概念
- Part 2 智能体的发展历史：从规则系统到生成式 Agent
- Part 3 大语言模型的发展与规模化能力
- Part 4 大语言模型架构与注意力机制
- Part 5 大语言模型的训练与对齐
- Part 6 大语言模型推理与提示工程
- Part 7 LLM 评估、多模态与开放挑战
- Part 8 总结

Lecture 2 Anatomy of an LLM Agent
- Part 1 Agent

- Part 2 Agent 的组成
- Part 3 Perception 感知子系统
- Part 4 Reasoning 推理子系统
- Part 5 Meomory 记忆子系统
- Part 6 Execute 执行子系统
- Part 7 Failure and Evaluation 失败与评估

Lecture 3 Reasoning in LLMs (1)

- Part 0
- Part I Introduction of Reasoning
- Part 2 Basic Prompting Techniques
- Part 3 推理策略与结构
- Part 4 Search and Selection Methods
- 总结

Lecture 4 Reasoning in LLMs (2)

- Part 1 两种思维模式
- Part 2 LLM 推理能力争议
- Part 3 三条教它推理的路径
- Part 4 Post-training Techniques & DeepSeek-R1
- Part 5 强化学习与 DeepSeek-R1

Lecture 5 Planning in LLM Agents

- Part 1 Planning
- Part 2 Reasoning vs Acting
- Part 3 Planning Pressures I：模块化
- Part 4 Planning Pressures II：反馈
- Part 5 Planning Pressures III：长时间任务
- Part 6 Planning Pressures IV：控制
- Part 7 Classical Planning Meets LLMs
- Part 8 Three Solutions
- 5584 LLM for Agent 2

5584 LLM for Agent2

Lecture 6 Tool Use for LLM Agents

- Part 1 Capabilities 工具的能力
- Part 2 Mechanics 工具的机制
- Part 3 Constrained Decoding 工具的约束：只保证形式合法
- Part 4 REST API 工具接口
- Part 5 Model Context Protocol
- Part 6 Multiple Tool Calls：串行、并行与依赖
- Part 7 Evaluating Tool Use

Lecture 7 Agent Memory and RAG

- Part 1 RAG 的研究动机与基本原理
- Part 2 RA-LLM 的研究体系与标准架构
- Part 3 Retriever：检索器的分类、原理与训练
- Part 4 Pre-Retrieval Techniques：检索前优化

- Part 5 Post-Retrieval Techniques：检索后优化
- Part 6 Agent Memory 智能体记忆

Lecture 8 Agent Infrastructure 智能体基础设施

- Part1 长上下文推理与 KV Cache
- Part 2 KV Cache 的分层存储与跨设备管理
- Part 3 LMCache：独立的 KV Cache 基础设施
- Part 4 CacheBlend：非前缀 KV Cache 的选择性重计算
- Part 5 复合 AI 系统 Compound AI Systems
- Part 6 DSPy：从 Prompt Engineering 到 Language Model Programming
- 总结
- reference patterns

Reference Pattern Notes for V1

- Decisions after screenshot review
- V1 iteration 4 application
- technique1 comparison

Task 3 · Technique 1 Results — V0 vs V1

- Lineage and protocol
- Outcome
- Human score estimate (provisional)
- How the few-shot examples affected V1
- Trade-offs and remaining issues
- Structural revision 2 — response to observed weakness
- Iteration 6 — screenshot-led interaction clarity
- Iteration 5 — strict copy contract and component architecture
- ai judge prompt v2

AI Judge Prompt — Apply in a New, Clean Session

- Clean Judge run metadata (complete after an independent session)
- answer card cleanup log

V2-RAG Answer Card and Reference Disclosure Revision — 2026-10-24

- Request and baseline
- Code changes
- Verification boundary
- answer consistency and language guard

Answer Consistency and Language Guard

- Reported gaps
- Diagnosis
- General repairs
- Verification
- Limits
- bare concept and process pipeline fix

Generic Bare-Concept and Process-Answer Repair

- Reported failure
- Root cause

- Query-independent repair
- Cross-topic verification
- Verification boundary
- Remaining limitations
- base concept title reranking

Base-concept title reranking (2026-10-27)
- Reported failure
- Cause
- Repair requested and implemented
- Result
- bilingual evidence repair log

Bilingual Evidence Repair Log
- Reported behaviour
- Root cause
- General repair
- Verification
- Remaining limitation
- clause provenance validation log

Clause and Formula Provenance Validation
- Reported defect
- Generic repair
- Evidence
- cnn significance repair log

V2-RAG CNN Significance Retrieval Repair — 2026-10-24
- Reported failure and preserved evidence
- Actual course-note evidence
- Changes
- Verification and limits
- coherent answer generation log

Coherent Answer Generation Revision
- Requirement
- Implementation
- Evaluation
- component relevance log

Parent Concept and Component Relevance
- Reported behavior
- Relevance rule
- Answer construction
- Evidence
- compound query title gate fix

Compound-query title-gate repair (2026-10-27)
- Reported failure
- Cause

- Generic repair
- Result
- deployment record

V2-RAG GitHub Pages Deployment Record
- Scope protection
- Post-deployment checks
- exercise solution relevance log

Exercise and Solution Relevance Rule
- Requirement
- Generic implementation
- Manual generic probe
- generic semantic pipeline refactor

Generic Semantic RAG Answer Pipeline Refactor
- Trigger
- Preserved behavior
- Shared pipeline
- Evaluation change
- knowledge base manifest

V2-RAG Knowledge Base Manifest
- linked answer fix log

V2-RAG Retrieval/Answer Linkage Repair — 2026-10-24
- User-reported defects
- Repair
- Current verification and boundary
- mcp answer gate repair log

V2-RAG MCP Answer-Gate Repair — 2026-10-24
- Reported regression
- Repair
- Verification boundary
- multi source answer log

V2-RAG Multi-Source Answer Revision — 2026-10-24
- Request and prior behavior
- Implementation
- Deterministic evidence
- query run metadata

Query Run Metadata
- query set

V2-RAG Test Query Set
- rag prompt template

V2-RAG Prompt Template
- relevance fix log

V2-RAG Relevance Repair — 2026-10-24
- Reported failure

- Repair steps and code changes
- After-repair observations
- Verification boundary and remaining risks
- retrieval iteration log

Retrieval Iteration Log
- rubric

Evaluation Rubric — Aria’ s Sky Journal
- 1. Content Completeness & Clear Contribution — 30%
- 2. Navigation & Task Completion — 25%
- 3. Readability & Visual Consistency — 25%
- 4. Interaction Robustness — 20%
- Scoring procedure
- title led relevance gate

Title-led relevance gate repair (2026-10-26)
- Problem observed
- General repair
- Observed checks
- Trade-off
- ui simplification log

V2-RAG Front-End Simplification — 2026-10-24
- User feedback and reason for change
- Source changes
- Preserved evidence
- Verification and limits
- Remaining limitations
- v0 v2rag analysis

V0 vs V2-RAG Analysis
- Version lineage and technique
- Rubric comparison
- Observable RAG changes and evidence
- Improvements, trade-offs, limitations
- Screenshot capture and Judge evidence
- v2 v0 lineage check

V2 lineage check — is V2 built on V0?
- 1. V0 was not modified
- 2. What V2 changed relative to V0
- 3. Consequence recorded after the check
- 4. Verification after the change
- 5. Scope note
- visitor answer cleanup log

V2-RAG Visitor Answer Cleanup — 2026-10-24
- Trigger and preserved state
- Code changes

- Verification and limits
- rubric

Evaluation Rubric — Aria’ s Sky Journal

- 1. Content Completeness & Clear Contribution — 30%
- 2. Navigation & Task Completion — 25%
- 3. Readability & Visual Consistency — 25%
- 4. Interaction Robustness — 20%
- Scoring procedure
- ai judge prompt

AI Judge Prompt — Apply in a New, Clean Session

- ai raw output v0

AI Judge — Evidence Manifest and Raw Output Slot

- Clean-session protocol
- Evidence supplied to the clean Judge session
- Raw AI Judge output (paste verbatim below)
- Run metadata (not part of the Judge output)
- ai raw output v1

V1 AI Judge Raw Output

- Clean-session metadata
- Raw response (verbatim)
- ai raw output v2 rerun

V2-RAG AI Judge Raw Output — Rerun 2026-10-28

- Clean-session metadata
- Raw response (verbatim)
- 2026 10 23 live e2e test

V0 Live End-to-End Test Log

- Environment and initial state
- Test results
- Summary and limits
- Screenshot evidence
- V1 E2E test log

V1 Desktop End-to-End Test Log

- Screenshots
- Limits
- Published-site run
- test prompts and feedback index

Task 2 Test Prompts and Feedback Index

- V0 baseline
- V1 Few-Shot version
- V2 RAG version
- Feedback and repair history
- record corrections

Evaluation Record Corrections and Provenance Notes

V0 evidence files
Score and output references
Missing historical data
Companion artifacts

COMP5583 Prompt Appendix

Student: Liu Xinyue
Student ID: 26049382G

Purpose and reading guide

This Markdown file consolidates preserved prompts, user feedback, iteration/change records, Task 2 evaluator materials, and version test logs for export as `prompt_appendix.pdf`. Original user prompts remain in their recorded language; archive text is preserved rather than retroactively rewritten as verbatim dialogue. No missing historical dialogue is presented as an original transcript.

Companion artifacts remain in the submission folder: complete V0/V1/V2 source, V1 generated-code snapshots, machine-readable JSON query/test outputs, and screenshot evidence. The JSON and images are indexed in `evaluation/README.md` and `evaluation/test_prompts_and_feedback_index.md`; they are kept as exact artifacts rather than transcribed by hand.

Task 1 — V0 baseline

The V0 section distinguishes verbatim user instructions from reconstructed prompts because the complete Trae chat export is unavailable. It includes recorded freeze-time repairs and code-output excerpts. The complete frozen website is in `website/V0/`.

Task 2 — Fixed evaluator

The official rubric, preserved clean-session Judge prompt and three version outputs follow. The official human/AI comparison is in `evaluation/comparison_and_limitations.md`; evidence gaps and metadata corrections are in `evaluation/record_corrections.md`.

Task 3 — V1 In-context Few-Shot Learning

The V1 records contain prompt revisions, few-shot examples, reference-pattern decisions, design feedback and change logs. Full generated source snapshots remain under `prompt_appendix/task3_v1/llm_output/`; the final independent site source is under `website/V1/`.

Task 3 — V2 Retrieval-Augmented Generation

The V2 archive records the structured knowledge base, query set, exact query runs, answer/retrieval failures, user feedback, generic repairs, regression results, public E2E evidence and comparison. The deployed site is a static RAG workflow demonstration and does not call an external LLM API.

Preserved records

README

Archive source: `prompt_appendix/task1_v0/README.md`

Task 1 · V0 交互与构建记录

原文可核：本轮提供的改版回顾

用户说明 V0 的改动曾通过 Trae 完成，并回顾了六轮：

1. 全站改版：导航与音乐按钮；About 改为 Highlights；教育经历、成果与技能；项目中删除“管理决策优化建模”；Contact 改为 Guestbook；英文界面与桌面适配；移除 Music Corner；加入自我介绍。
2. 网站打不开后，重启本地服务器（回顾中记录端口 8765）。
3. 精简 Highlights，删除重复 Publication，学历卡片点击展开；前三张照片用于首页，其余照片进入 Life。
4. 增加课程笔记知识库，放入 5205、5434、5583、5584 笔记，并添加检索入口。
5. 将检索区改成交互聊天窗，用 BM25 检索笔记片段并展示来源。
6. Life 加入两段视频，删除上传照片/视频功能，清除旧留言记录；代码回顾称留言存储键改为 `aria-guestbook-v2`。

重建提示词记录（非逐字原文）

以下按用户回顾整理，供报告解释开发过程；如需满足“全部原始 prompt”要求，应从 Trae 历史记录导出原文并替换/补充。

P0-01 初始构建指令（由首轮需求回顾重建）

> 请把现有个人网站改造成英文个人作品集 “Aria’ s Sky Journal”。保留淡蓝水彩天空、白云与远山背景、低饱和蓝白色、毛玻璃卡片与胶囊导航。包含 Home、Highlights、Projects、Life、Guestbook 等板块。导航附近放置可手动播放/暂停的舒曼《梦幻曲》音乐按钮并标注英文曲名。Highlights 展示个人简介、教育经历、学术成果和专业技能；本科与硕士信息分开展示。Projects 展示项目经历，移除“管理决策优化建模”项目。Life 展示个人生活与兴趣内容。Guestbook 提供访客留言。全站使用英文，保留温柔手绘旅行日记视觉，增加轻柔交互动画并适配桌面端；移除独立 Music Corner，将自我介绍放入页面内容/页尾。不要加入无关素材或未经提供的个人信息。

P0-02 网站无法打开时的修复指令（回顾重建）

> 检查本地预览无法打开的原因。恢复本地静态服务器并确认网站可通过本机浏览器访问；不要改动网站内容，除非排查发现必要修复。

P0-03 Highlights 精简与照片分配（回顾重建）

> 精简 Highlights：移除子导航和与 Projects 重复的 Publication 卡片；学校信息默认收起，访客点击学校卡片后再查看本科成绩/奖项或硕士课程信息。将前三张照片放在首页轮播，其余照片放入 Life / Beyond the Lab 相册。保留其他页面和原有风格。

P0-04 加入课程笔记检索入口（回顾重建）

> 在 Highlights 底部加入一个课程笔记知识库入口，使用 5205、5434、5583、5584 的课堂笔记作为检索素材。用一句简短英文引导访客提问，保持原布局与视觉，不要把整个页面改成聊天机器人。

P0-05 将知识库改为可交互 RAG 聊天（回顾重建）

> 将课程笔记检索入口改成可交互聊天窗口：用户输入问题后，从本地 Markdown 课程笔记检索相关内容，回答展示检索到的原文片段和课程/章节来源；加入建议问题。保留静态前端实现，不依赖后端服务，不要虚构笔记中没有的结论；检索不到时清楚说明。保持原页面视觉。

P0-06 Life 视频、禁用上传与清空旧留言（回顾重建）

> 把提供的两段视频加入 Life 相册，作为可点击预览的媒体项目；移除前端上传照片和视频的功能；清除 Guestbook 旧留言记录。保持其他功能和现有风格，并告知项目源码所在位置。

P0-07 根据本轮诊断修复本地 file:// 下 RAG（由当前对话记录重建）

> 修复 V0 在直接双击 `index.html` 时 RAG 不能加载笔记的问题。检查浏览器对 `file://` 下 fetch 本地 Markdown 的限制，采用静态可部署的方案把笔记语料随网页打包，同时保留可读 Markdown 文件和 HTTP 部署能力。验证首页/聊天界面、课程笔记检索及来源显示。不要改成后端服务，也不要伪称由 LLM 生成答案。

P0-08 移除两条课程成绩（当前对话原文）

> Business Operations Research*92*
> Management Information Systems*91把这两个成绩删掉。为什么 rag 用不了*

该轮要求分为：删除两条成绩，并排查 RAG 不能使用的原因。

原始模型输出说明

当前证据目录包含冻结 V0 的完整 HTML、CSS、JavaScript 与静态资源，可作为最终交付源码。但未取得 Trae 各轮完整原始会话导出，因此不能声称它们是 LLM 每轮原始输出。实际 HTML/代码以 `website/V0/` 为准；以下是关键实现片段索引，完整代码请直接查相应文件：

- 页面骨架、导航、板块、音频与本地 RAG 数据脚本：`website/V0/index.html`
- 样式与响应式布局：`website/V0/style.css`

- 导航、Projects、Life 媒体、Guestbook 本地存储交互：`website/V0/app.js`
- 项目数据：`website/V0/projects.js`
- BM25 本地检索：`website/V0/rag.js`
- 随站内嵌课程笔记：`website/V0/knowledge-data.js`
- 可读 Markdown 语料：`website/V0/knowledge/`

V0 冻结前修复记录

编号	问题	修改位置	修复与理由	证据/状态
FIX-01	直接用 `file://` 打开时浏览器限制本地 Markdown `fetch`，RAG 显示无法加载笔记	`rag.js`、`knowledge-data.js`、`index.html`	生成并加载本地内嵌语料，同时保留 Markdown 原文与 HTTP 方式；解决静态站点在双击打开时的浏览器安全限制	V0 现有页面加载 `knowledge-data.js`；README 说明双击和 HTTP 两种打开方式
FIX-02	两门不应显示的课程成绩仍出现	`index.html` 的 Highlights 教育/课程内容	删除 Business Operations Research 92、Management Information Systems 91，避免展示用户要求移除的成绩	代码检索确认两条成绩未在页面源代码中
FIX-03	修改后用户仍看到旧成绩/旧 RAG 报错	`outputs/Aria_Sky_Journal_V0/` 与 V0 zip	将修订同步到用户实际打开的官方 V0 目录并重建 zip；解释旧 `file://` 标签需刷新	当前冻结目录包含修订源码；此前截图证明用户打开的是该路径
FIX-04	原页面从旧 V1 更名为官方 coursework V0，容易让文件名/代码标识混淆	`VERSION.txt`、`README.md`、`index.html`、`style.css`、`app.js`，以及压缩包	将版本标识与文件命名统一为 V0；保留 `aria-guestbook-v2` 作为 localStorage 数据键，因为它是存储键而非网页版本	当前冻结目录 README/VERSION 有说明

冻结 URL： https://liuxinyue84-sketch.github.io/ （GitHub Pages，公开可访问）。线上浏览器测试记录见 `../../evaluation/evidence/V0/2026-10-23_live_e2e_test.md`。已从项目归档恢复实际截图并纳入 `../../evaluation/evidence/V0/`；恢复的原始测试日志和截图没有重新制作。

V0 CODE OUTPUT EXCERPTS

Archive source: `prompt_appendix/task1_v0/V0_CODE_OUTPUT_EXCERPTS.md`

V0 代码输出证据索引与关键片段

来源限制： 这些是当前冻结源码中的实际代码片段，用于说明最终交付行为；它们不是从 Trae 对话导出的逐轮原始 LLM 输出。完整代码作为 `coursework/website/V0/` 归档。

1. 页面加载本地嵌入语料

来源：`website/V0/index.html`

```
<audio id="soundtrack" src="assets/traumerei.mp3" preload="auto" loop hidden></audio>
<p id="audio-status" class="sr-only" role="status" aria-live="polite"></p>
<script src="projects.js"></script>
```

```
<script src="knowledge-data.js"></script>
<script src="app.js"></script>
<script src="rag.js"></script>
```

2. RAG 检索源清单与双语分词

来源：website/V0/rag.js

```
const ragSources = [
 { file: 'knowledge/5205-Introduction-to-AI.md', course: 'Introduction to AI' },
 { file: 'knowledge/5434-Big-Data-Computing-1.md', course: 'Big Data Computing · Part 1' },
 { file: 'knowledge/5434-Big-Data-Computing-2.md', course: 'Big Data Computing · Part 2' },
 { file: 'knowledge/5583-Agentic-Computing-1.md', course: 'Agentic Computing · Part 1' },
 { file: 'knowledge/5583-Agentic-Computing-2.md', course: 'Agentic Computing · Part 2' },
 { file: 'knowledge/5583-Agentic-Computing-3.md', course: 'Agentic Computing · Part 3' },
 { file: 'knowledge/5584-LLM-for-Agent-1.md', course: 'LLM for Agent · Part 1' },
 { file: 'knowledge/5584-LLM-for-Agent-2.md', course: 'LLM for Agent · Part 2' }
];

/* Latin words plus CJK bigrams, so bilingual notes both match. */
function tokenize(text) {
 const tokens = [];
 (text.toLowerCase().match(/[a-z0-9]+/g) || []).forEach(word => {
  const stem = word.length > 3 && word.endsWith('s') ? word.slice(0, -1) : word;
  if (stem.length > 1 && !RAG_STOPWORDS.has(stem)) tokens.push(stem);
 });
 (text.match(/[\\u4e00-\\u9fff]+/g) || []).forEach(run => {
  if (run.length === 1) tokens.push(run);
  else for (let i = 0; i < run.length - 1; i++) tokens.push(run.slice(i, i + 2));
 });
 return tokens;
}
```

3. 项目详情与收藏控件

来源：website/V0/app.js

```
<button class="row-toggle" aria-label="Expand details of ${p.title}"
 aria-expanded="false" aria-controls="project-detail-${i}">+</button>
<span>Open this note</span>
<button class="save-project" aria-label="Save ${p.title}"
 aria-pressed="false">♡</button>
```

4. 客人留言本地存储

来源：website/V0/app.js。存储键名是兼容性标识，并非网站版本名。

```
const value = JSON.parse(localStorage.getItem('aria-guestbook-v2') || '[]');
```

请以 website/V0/ 内的完整文件为最终源码证据；上列片段只作导航索引，不能替代逐轮生成日志。

---

ai judge evidence

Archive source: prompt_appendix/task3_v1/ai_judge_evidence.md

V1 AI-Judge Evidence Manifest (for a separate blank session)

Use this manifest only to assemble evidence. Do not send any human score or human interpretation to the Judge.

1. Fixed evaluator prompt: `coursework/evaluation/ai_judge_prompt.md`.
2. Fixed scoring rubric: `coursework/evaluation/rubric.md`.
3. V1 E2E raw results: local common run `coursework/evaluation/evidence/V1/V1-e2e-results.json` and published HTTPS run `coursework/evaluation/evidence/V1/V1-PAGES-RETEST-e2e-results.json`.
4. V1 screenshots: local captures `coursework/evaluation/evidence/V1/V1-E2E-01_home_desktop.png`, `V1-E2E-12_highlights_desktop.png`, and `V1-E2E-12_highlights_full_desktop.png`; published captures use the matching `V1-PAGES-RETEST-` prefix.
5. The first published run's timing-sensitive failure and original output are retained as `V1-PAGES-initial-e2e-results.json`; the final retest passed all assertions.
6. V1 source evidence: `coursework/website/V1/index.html`, `rag.js`, `v1.css`, and `v1.js`; compare with the frozen baseline files in `coursework/website/V0/` if needed.
7. Test protocol: `coursework/evaluation/scripts/test_portfolio_e2e.cjs`.

Latest V1 evidence after structural/visual revisions

Use `V1-PAGES-R4-e2e-results.json` as the latest published 29-check behavior record. Its screenshots include the updated Home (`V1-PAGES-R4-E2E-01_home_desktop.png`), separate chapter rail (`V1-PAGES-R4-E2E-01b_chapter_rail_desktop.png`), Highlights, and asymmetrical Projects section (`V1-PAGES-R4-E2E-13_projects_full_desktop.png`). The corresponding latest local evidence uses the `V1-R4-` prefix. Earlier R2/R3 artifacts are retained as iteration history rather than current final screenshots.

The Judge should receive only the fixed rubric/prompt and these V1 artifacts. Do not include `human_self_assessment.md`, this comparison, the provisional human score, or the V0/V1 weakness discussion. Paste the Judge's unedited original response into a version-specific raw-output file only after the independent run.

V1 iteration 4 evidence (Life scrapbook wall)

- Local E2E raw output: `../../evaluation/evidence/V1/V1-R5-e2e-results.json`.
- Life section screenshot: `../../evaluation/evidence/V1/V1-R5-E2E-14_life_full_desktop.png`.
- Shared E2E runner also records home, Highlights, Projects, and navigation screenshots for this run.
- The evaluator evidence now includes a test record that the original nine media items remain, distinct tile sizes/positions form the V1 wall, and keyboard activation/Escape operate the native preview.
- Task 2's rubric and AI-Judge prompt are unchanged. Human scores and AI-Judge output must not be inferred from these functional checks.

V1 iteration 5 evidence (component architecture rebuild)

- Published page: `https://liuxinyue84-sketch.github.io/V1/?v=c80de98` (commit `c80de98`).
- Local E2E output and screenshots: `../../evaluation/evidence/V1/V1-R6-e2e-results.json` and `V1-R6-E2E-*.png`.
- Published E2E output and screenshots: `../../evaluation/evidence/V1/V1-PAGES-R6-e2e-results.json` and `V1-PAGES-R6-E2E-*.png`.
- Exact-copy/component-shell contract: `../../evaluation/evidence/V1/V1-R6-content-contract.json` and `V1-PAGES-R6-content-contract.json` (46 copy groups, hero alt text, project stories, and 11 shell assertions; zero differences/errors).
- Latest matched results: local V1 33/33, published V1 33/33; frozen V0 30/32 on the same desktop protocol. The two V0 RAG failures are recorded as limitations.
- Use screenshots for visible evidence and source snapshots from `llm_output/iteration_5/`. The Judge must still receive only the fixed rubric/prompt and site evidence, never human scores or this analysis.

V1 iteration 6 evidence (interaction clarity)

- Published page: `https://liuxinyue84-sketch.github.io/V1/?v=6379505` (commit `6379505`).
- Local/public E2E results: `../../evaluation/evidence/V1/V1-R7-e2e-results.json` and `V1-PAGES-R7-e2e-results.json` (35/35 each); matched V0 baseline: `../../evaluation/evidence/V0/V0-R7-e2e-results.json` (32/34, two known RAG limitations).
- Copy/component contract: `../../evaluation/evidence/V1/V1-R7-content-contract.json` and `V1-PAGES-R7-content-contract.json` (48 groups, no differences).
- Interaction screenshots: `V1-R7-E2E-01b_chapter_rail_desktop.png`, `V1-R7-E2E-16_education_details_open_desktop.png`, and `V1-R7-E2E-17_single_project_open_desktop.png`; the Home full screenshot records thumbnails positioned below the main image.
- The five original issue screenshots are preserved as `V1-R7-user-feedback-01_...png` through `V1-R7-user-feedback-05_...png` for before/after comparison.
- Source: `llm_output/iteration_6/`. Keep the fixed rubric and judge prompt unchanged and omit human assessments from the new-session Judge input.

---

change log

Archive source: `prompt_appendix/task3_v1/change_log.md`

V1 Generation and Change Log

Lineage and freeze check

- Official base: `coursework/website/V0/` (verified against `coursework/MANIFEST.sha256` before making V1 changes).

- V1 was created with a direct directory copy from the official V0 folder, then edited only under `coursework/website/V1/`.
- `outputs/aria-v0`, `outputs/aria-v1`, and any V2 directory were not used as inputs.
- The V0 hash manifest was rechecked after generation; V0 remains byte-identical.

Generated files and changes

V1 file	Change	Evidence / purpose
`index.html`	Added a V1-only stylesheet/script, pegboard wrapper, decorative education connector, and bundled V0 study photos. Updated RAG explanatory copy and visible version label.	Few-shot examples 1 and 3; all original education/project/profile facts remain in the page.
`v1.css`	New soft cream-blue dotted pegboard, irregular two-column education/skills layout, photograph treatment, thread drawing, and responsive single-column layout. New transitions use gentle easing and reduced-motion support.	Example 1; borrows only broad layout patterns.
`v1.js`	Adds an explicit project contribution label around the original source string; mirrors navigation active state with `aria-current`; synchronizes skill-chip `aria-pressed`.	Example 2; improves scanability and state feedback without modifying project data.
`rag.js`	Adds three few-shot-derived topic profiles, low-weight query-term expansion, a topic-heading relevance boost, a low-evidence abstention check, and a visible excerpt/source label.	Examples 3 and 4; factual answer content remains retrieved from V0 notes.
`README.md`, `VERSION.txt`	Identify the folder as V1 and explain lineage, files, behavior, and local launch.	Prevents V0/V1 label ambiguity.

Necessary bug fixes before V1 test

None recorded at generation time. Any later correction must be appended here with the failure, file/region changed, reason, and test result. Do not edit V0 to fix a V1 issue.

Generated source output

`llm_output/` contains the complete initial generated versions of changed source files before verification fixes. If a post-generation fix becomes necessary, preserve the original there and apply the fix only to `website/V1/` with a dated note in this log.

Verification correction — 2026-10-23

- Issue: copied base-style comments still labelled the V1 copy as “V0”, creating version ambiguity; one reused-photo alt label also implied that the graduation photo was specifically an undergraduate campus photo.
- Change: in V1 only, relabelled the inherited stylesheet comments as V1 base styling inherited from frozen V0; changed the image alt to the literal visible event (“Aria holding a bouquet on a campus track”). Clarified the V1 README wording.

- Reason: keep version metadata accurate and avoid an unsupported claim about the photograph.
- The pre-correction initial generated `style.css` is preserved at `llm_output/style.css`.

Browser-test iteration record — 2026-10-23

- First automation attempt read the previous RAG response before the new request completed. The test harness was updated to wait for the retrieval status element to disappear before checking each answer. No website source was changed for this harness correction.
- The browser initially reported its implicit `/favicon.ico` request as a 404. The harness now records this browser-default request separately; it does not count as an application exception. No V0 source was changed to address it.
- The first video-preview assertion checked `currentSrc` before metadata had loaded. The harness now waits for `loadedmetadata`; the final V1 run confirms the bundled MP4 source and controls.
- Initial matched run before adding the paraphrased abstention test: V0 23/24 assertions (MapReduce leading passage failed); V1 24/24 assertions. These historical outputs are retained with the pre-waitfix JSON files; the expanded final protocol results are recorded below.

Live-release boundary-case correction — 2026-10-23

- Failure found after the first public V1 deployment: the query “How do I bake sourdough bread?” returned an unrelated data-mining excerpt because the shared word “bread” met the former low overlap threshold.
- V1-only change in `rag.js`: record the number of matched original query terms and require a non-topic query to match at least two terms and 50% of its content terms before showing an excerpt. Curated MapReduce/RAG/agent-planning topic matches retain their topic-based path. V0 was not edited.
- The common E2E protocol now includes the paraphrased unsupported query as E2E-07c. After the change, V1 passes both no-evidence prompts; unchanged V0 returns an unrelated excerpt for the added paraphrase. The result supports a scoped robustness improvement, not a claim that all unsupported questions are handled correctly.
- Re-ran the same 25-check desktop suite after the correction: V1 25/25; V0 23/25 (the MapReduce relevance case and paraphrased unsupported-query case fail). Raw outputs and refreshed screenshots are saved under `evaluation/evidence/V0/` and `evaluation/evidence/V1/`.
- The correction was applied only to V1 after its initial release, and the final deployment was then updated. The initial generated `rag.js` remains preserved in `llm_output/rag.js`.

Shared browser protocol timing correction — 2026-10-23

- First deployed-site run: 24/25 checks passed; the carousel had not advanced when the test read its counter after a fixed 500 ms wait. Other live checks, including the updated RAG abstention behavior, passed. The initial raw JSON and screenshots are retained with the `V1-PAGES-initial-` prefix.
- Test-only change: `evaluation/scripts/test_portfolio_e2e.cjs` now waits up to five seconds for the visible carousel counter to reach slide 2 before asserting. This does not alter V0 or V1 website behavior.

- Re-ran the updated shared protocol against local V0, local V1, and the published V1 HTTPS URL. Results: V0 23/25, V1 25/25, published V1 25/25. The published run and its screenshots are saved with the `V1-PAGES-RETEST-` prefix.

User feedback and structural revision — V1 iteration 2 — 2026-10-23

- **Observed shortcoming:** the first V1 had most visible changes in Highlights and RAG. The Home and Projects DOM/interaction patterns remained too close to V0, so the effect of Few-Shot Learning was difficult to see. The user explicitly identified this as insufficient evidence of a technique-led version change.
- **Lineage:** this remains the V1 branch derived from the frozen V0 copy. The correction is a new V1 generation iteration using the revised few-shot prompt in `icl_prompt_revision2.md`; no V2 source was used and V0 was not edited. The first prompt and first generated source output remain preserved unchanged as iteration 1.
- **Home structural changes:** `index.html` adds a numbered chapter rail linking to existing long-scroll sections and changes the hero container to semantic `figure`/`figcaption`; `app.js` renders image thumbnails and keeps selected state, main image, caption, and counter synchronized; `v1.css` lays out the thumbnail selector and chapter rail. Desktop proportions were adjusted after screenshot review; mobile rail returns to document flow.
- **Projects structural changes:** `index.html` adds a four-story anchor index and changes `#project-list` from a `div` to an ordered list. `app.js` now renders each item as an `li` with a semantic `article`, distinct purpose/contribution blocks, explicit paper-note actions, and retained tags/details. `v1.css` lays out asymmetrical staggered paper stories; filters preserve category semantics and reflow entries gently. The existing save state and detail controls remain operable.
- **Files changed:** `coursework/website/V1/index.html`, `style.css`, `app.js`, `v1.css`, `v1.js`; shared supplementary test harness `coursework/evaluation/scripts/test_portfolio_e2e.cjs` now checks both V0 baseline structure and the V1-specific chapter/story features. The Task 2 rubric and AI-Judge prompt were not edited.
- **Visual correction:** first screenshot review showed the chapter rail crowding the left hero copy. The rail was moved to a compact full-width strip at the bottom of the desktop hero; the hero column ratio and title size were adjusted. First project-atlas screenshot showed an oversized blank featured story; the first story no longer spans two rows, and the four stories now form a more balanced staggered two-row composition.
- **Follow-up screenshot correction:** the user reported that the carousel thumbnail controls overlapped the chapter rail in the scrolled Home view. The rail was moved out of the hero section into its own flow-positioned region directly below the hero; a dedicated rail screenshot is now captured. The carousel controls remain inside the figure, with the hero's existing bottom padding separating the two regions. This correction supersedes the prior placement note above.
- **Reference-led refinement 3:** after the user asked for further refinement based on the four selected samples, the academic references were used for restrained hierarchy and scanning; the art portfolio for image-first category selection; and the interactive portfolio for small, purposeful hover/focus states. `reference_patterns.md` records each mapping. The Home chapter rail is now one translucent strip with dividers, while the Projects index is a simple editorial index rather than four additional tile cards. Actual project stories remain the visual focus. No reference copy, identity, or assets were used.

- **R4 verification:** final visual iteration passes all 29/29 local desktop E2E assertions at 1512×982, with zero uncaught browser/console errors. Final evidence is `V1-R4-E2E-01_home_desktop.png`, `V1-R4-E2E-01b_chapter_rail_desktop.png`, `V1-R4-E2E-13_projects_full_desktop.png`, and `V1-R4-e2e-results.json`. The E2E run records the layout after the overlap fix; older R2/R3 captures are retained as revision history.
- **Published verification:** V1 was pushed to `liuxinyue84-sketch/liuxinyue84-sketch.github.io` in commit `6b53131`. GitHub Pages served the updated markup, and the same 29-check desktop suite passed 29/29 against the public HTTPS URL. Raw result and screenshots use the `V1-PAGES-R4-` prefix. Frozen V0 is still the repository root and was not part of the V1-only commit.
- **Verification:** at 1512×982 desktop viewport, V1-R2 passes 29/29 E2E checks with no uncaught console errors. Same protocol run on frozen V0 passes 27/29; the two V0 failures remain the known MapReduce-ranking and paraphrased unsupported-query RAG limitations. Raw JSON and V0/V1 screenshots, including full Home, Highlights, and Projects captures, are stored in `coursework/evaluation/evidence/V0/` and `coursework/evaluation/evidence/V1/` with the `*-R2-*` prefix. Favicon 404 requests are separately recorded as browser-default requests.
- **Generated source capture:** complete V1-R2 outputs are saved under `llm_output/iteration_2/`; original iteration-1 generated files remain in `llm_output/`.

Life scrapbook wall and SVG marginalia — V1 iteration 4 — 2026-10-23

- **User-approved scope:** Add original hand-drawn SVG marginal notes and rearrange the current Life photographs/videos into an asymmetric scrapbook wall, preserving existing content and interactions.
- **Few-shot inputs:** `icl_prompt_revision4.md` and `few_shot_examples_revision4.md`. They abstract only gallery structure, marginalia, and keyboard interaction patterns from the selected references. No reference-site text, identity, or media was copied.
- **V1-only source changes:** `index.html` adds two original inline decorative SVGs and identifies the gallery as an accessible scrapbook wall; `v1.css` adds low-saturation marginalia and a responsive varied-size grid with gentle paper-like edges; `app.js` adds gallery item indices/kinds, dialog semantics, and photo alt text. All seven existing photos, two videos, captions, and preview dialog remain.
- **Shared test harness:** added checks for all nine gallery items, V1-only varied/staggered wall geometry, keyboard activation, Escape closing, and a full Life-section screenshot. Existing V0 behavior remains a conditional baseline in the same shared suite. Task 2 scoring rubric and AI-Judge prompt were not changed.
- **Local desktop verification:** 33 checks passed at 1512×982 on the V1 local HTTP server; no uncaught browser or console errors. Evidence: `evaluation/evidence/V1/V1-R5-e2e-results.json` and `V1-R5-E2E-14_life_full_desktop.png` (plus the run's other screenshots).
- **V0 boundary:** `coursework/MANIFEST.sha256` verified before and after; V0 files were not edited.
- **Initial source snapshot:** complete changed V1 source files are saved under `llm_output/iteration_4/`.
- **Published verification:** V1 was pushed to `liuxinyue84-sketch/liuxinyue84-sketch.github.io` in commit `8f0f2e4`. The live URL `https://liuxinyue84-sketch.github.io/V1/?v=8f0f2e4` serves the new markup and passes 33/33 checks in the same desktop suite; no uncaught browser or console errors. Online raw output and screenshots are saved with the `V1-PAGES-R5-` prefix.
- **V0 regression-boundary run:** the same updated desktop protocol completed 32 checks against frozen V0; 30 passed and the two previously documented RAG cases failed (MapReduce passage ranking and a paraphrased unsupported query). The V0 Life gallery and

keyboard preview checks passed. Raw output is `evaluation/evidence/V0/V0-R5-e2e-results.json`; V0 checksum verification passed.

Copy-preserving component architecture rebuild — V1 iteration 5 — 2026-10-23

- **User feedback:** the previous V1 still looked too much like V0. The user set a strict content contract (100% verbatim V0 business copy) and required large, observable changes to component shells, layout, and interactions using the reviewed few-shot patterns.
- **Few-shot inputs:** `icl_prompt_revision5.md` and `few_shot_examples_revision5.md`. The examples abstract editorial folio, image-led feature, project story index/article, and letter/media-atlas patterns. Reference copy and assets are excluded.
- **V1-only structure:** Home uses a journal-cover composition and chapter rail; Highlights uses an education journey, skill ledger, and knowledge workbench; Projects uses an index and semantic story articles; Life uses a letter/media atlas; Guestbook uses an open-postcard/writing-desk shell. V0 copy and content data remain the source.
- **Correction discovered during verification:** the first R6 desktop run failed the paraphrased unsupported RAG query because an abstention check had been omitted while refining answer output. Restored the evidence-confidence gate in V1 `rag.js`, retaining the original V0 no-evidence message. This is recorded as a V1 robustness correction, not a content rewrite.
- **Source snapshot:** complete current V1 source files are preserved in `llm_output/iteration_5/`.
- **Copy and structure verification:** `evaluation/scripts/check_v1_v0_content.cjs` compares 46 text groups, carousel alt text, full project story data, and 11 component-shell assertions. Result `PASS`: no copy differences; all V1 shells present. Raw result: `evaluation/evidence/V1/V1-R6-content-contract.json`.
- **Matched local desktop tests:** shared E2E at 1512×982: V1 33/33, no uncaught browser errors; unchanged V0 30/32, with its known MapReduce-leading-excerpt and paraphrased unsupported-query failures. Raw JSON and screenshots use the `V1-R6-` and `V0-R6-` prefixes in `evaluation/evidence/`.
- **Visual correction:** screenshot review found the Home hero heading crowded in its paper panel. The desktop column ratio and title size were adjusted; the resulting full-section capture is `evaluation/evidence/V1/V1-R6-E2E-01_home_full_desktop.png`.
- **V0 boundary:** frozen manifest verification is required before release; no V0 source files were edited. Task 2 rubric and AI-Judge prompt remain unchanged.
- **Published verification:** V1 was pushed to `liuxinyue84-sketch/liuxinyue84-sketch.github.io` in commit `c80de98`. GitHub Pages completed the deployment successfully; the cache-busted URL `https://liuxinyue84-sketch.github.io/V1/?v=c80de98` serves the new component shells and passes 33/33 shared desktop E2E checks with zero uncaught errors. The public copy/structure contract also passes against local frozen V0 with no differences. Raw online outputs and screenshots use `V1-PAGES-R6-` prefixes.
- **V0 boundary:** `shasum -a 256 -c coursework/MANIFEST.sha256` passed after the work. The V0 source was not changed; the published change is limited to `/V1/`.

Screenshot-led interaction correction — V1 iteration 6 — 2026-10-23

- **User feedback:** screenshots showed that the chapter and project indices used unexplained numbers, carousel thumbnails covered the bottom of the main photo, the education disclosure

affordance did not work, and an expanded project made its grid neighbor look expanded too.

- **Prompt record:** `icl_prompt_revision6.md` keeps the user's feedback verbatim and separates the correction instruction from it. This iteration is a targeted usability correction within the V1 Few-Shot branch, not a new claimed technique.
- **Navigation guidance:** chapter links now show the existing section names; project index names are populated directly from the V0 project-title data. No personal/business text was invented or rewritten.
- **Carousel:** thumbnail controls are positioned below the main image with enough space to prevent overlap.
- **Education:** discovered the V1 event handler had moved to `.education-leaf` while the inherited open-state CSS still targeted `.school-card.open`. Updated the V1 selector, added visible "Click to view details" / "Click to hide details" guidance, and made photo/card clicks activate the matching disclosure. Existing education text remains unchanged.
- **Projects:** changed V1 grid alignment and card sizing from row-stretched to intrinsic height so an adjacent project remains folded and compact when one project expands. Toggle logic remains per article.
- **Evaluation:** expanded the shared suite to assert labeled destinations, both education disclosures (including clicking the undergraduate photo), independent project expansion and compact neighbor geometry. The suite now saves explicit open-state screenshots for education and projects. The copy contract now checks 48 groups including chapter and project-index labels, stripping only the new UI hint from the school copy comparison.
- **Local results:** V1 R7 passes 35/35; V0 R7 passes 32/34 with the same two previously recorded RAG limitations. Copy/structure contract passes with no differences. Evidence files use `V1-R7-` and `V0-R7-` prefixes.
- **V0 boundary:** frozen V0 files remain unchanged; the full 32-file V0 manifest passed. Task 2 rubric and AI-Judge prompt remain unchanged.
- **Source capture:** complete implementation files and static resources are saved under `llm_output/iteration_6/`.
- **Published verification:** V1 usability corrections were pushed in commit `6379505`; GitHub Pages completed successfully. The cache-busted URL `https://liuxinyue84-sketch.github.io/V1/?v=6379505` passes 35/35 shared desktop checks and the live V0/V1 copy/component contract passes with zero differences. Evidence uses the `V1-PAGES-R7-` prefix.
- **Before/after visual record:** the five user-provided feedback screenshots are archived in `evaluation/evidence/V1/` as `V1-R7-user-feedback-01_...png` through `V1-R7-user-feedback-05_...png`; updated Home, chapter rail, Highlights, expanded education, and single-project screenshots are saved with `V1-R7-` and `V1-PAGES-R7-` prefixes.

Iteration 7 — page skeleton rebuild (2026-10-27)

Problem. R7 screenshot review: V1 still read as the same website as V0. Every previous iteration had changed components *inside* one frame — a single column of paper sections under a centred pill navigation.

Reference sweep. Six public portfolios were rendered before writing anything (`few_shot_examples_revision7.md`). Three skeletons survived: bento mosaic, profile rail with a content window, and an editorial split spread. Three were rejected — a desktop simulation and a terminal both hide content behind a metaphor, and Dopefolio is structurally V0.

Chosen. Profile rail + content window. Three clickable drafts were produced in `work/design-draft/` with the real copy and palette; the author picked B.

Files changed.

File	Change
`index.html`	masthead and floating nav replaced by `.app-frame` → `.app-rail` + `.app-window` containing `.pane-bar` and `main`; the chapter rail moved inside `#home`
`v1.css`	appended the app-shell block: rail, vertical `pill-nav`, window frame, one-screen sections, two-column project grid, mobile collapse
`app.js`	`setActive()` follows `--nav-axis` so the indicator travels vertically; `updateSectionNav()` measures the window scroller instead of the page and no longer forces `guestbook` when nothing scrolls
`v1.js`	`syncPaneBar()` keeps the window title bar's section name and `01 / 05` counter in step with the active link

Deliberately unchanged. `style.css`, `projects.js`, `rag.js`, `knowledge-data.js`, every asset, and every section's internal markup. V0 remains byte-identical and unfrozen files are untouched.

Verification.

- `test_portfolio_e2e.cjs` — 35/35 PASS, 0 console errors, 0 page errors (`coursework/evaluation/evidence/V1/V1-R8-e2e-results.json`)
- `check_v1_v0_content.cjs` — 48 copy groups, 0 differences, 0 structure errors (`coursework/evaluation/evidence/V1/V1-R8-content-parity.json`)
- Manual breakpoint pass at 1512 / 820 / 414 px: no horizontal overflow, no clipped controls.
- Screenshots `V1-R8-E2E-01 … 17` regenerated from the same run.

Known trade-off. Each section now claims one window height, so Highlights and Projects still overflow and scroll inside the window. The frame stays fixed; the content moves — which is the intended reading model, not a defect, but it should be stated plainly rather than described as "one screen per section".

Iteration 9 · 2026-10-27 · full-bleed editorial skeleton (replaces the app shell)

Why. The author rejected iteration 7 on sight: "v1好丑啊，越优化越丑…不想要有边框版，很局促" with a screenshot of the framed window, and added "也不喜欢旁边放的信息" about the profile rail. Two concrete faults: the window turned the page into an app inside a box, and the rail occupied a permanent column of facts the author did not want on screen. The author chose the full-bleed magazine direction from three options.

Direction. No frame, no side rail, no boxed cards. Everything runs the full viewport width on one hairline rhythm; structure comes from scale, rules and bleed rather than from borders, radii and shadows.

Files changed.

File	Change
`index.html`	`.app-frame` / `.app-rail` / `.app-window` / `.pane-bar` removed; replaced by a single `.ed-topbar` (wordmark, five labels, music toggle), the progress line and `main`
`v1.css`	the whole app-shell block deleted; a new editorial layer added (top bar, cover spread, hairline section heads, row-based highlights and projects, full-bleed image wall, open-letter guestbook, paper body with a masked sky band confined to the cover)

File	Change
`app.js`	`setActive()` now sizes and places the underline from the active link's own box instead of `translateX(i*100%)`; `updateSectionNav()` measures the page again; `pagehide` cleanup guarded (see below)
`v1.js`	`syncPaneBar()` removed with the window title bar

Deliberately unchanged. `style.css`, `projects.js`, `rag.js`, `knowledge-data.js`, every asset, every section's internal markup and every piece of copy. V0 remains byte-identical.

Legibility correction found during the rebuild. With the paper cards gone, the fixed sky photograph became the page surface and body copy sat directly on mountains — unreadable. The sky is now a masked band confined to the cover, with a paper wash on the text side; the body is a flat `#f7f5ee`. This is recorded because it changes how the page is read: the photograph is an opening image, not a background.

Latent defect fixed. `app.js` called `mediaURLs.forEach(...)` on `pagehide` but `mediaURLs` is declared nowhere in the bundle — inherited unchanged from frozen V0, where it threw on every page hide. Guarded with `typeof mediaURLs !== 'undefined'` in V1 only; V0 is left untouched because it is frozen.

Verification. `test_portfolio_e2e.cjs` 35/35 PASS with 0 console and 0 page errors (`evidence/V1/V1-R9-e2e-results.json`); `check_v1_v0_content.cjs` 48 copy groups, 0 differences (`evidence/V1/V1-R9-content-parity.json`); breakpoint pass at 1512 / 1280 / 1024 / 820 / 414 px with no horizontal overflow; screenshots `V1-R9-E2E-01 … 17`.

Known trade-offs, stated plainly.

- The cover photograph only bleeds on the right. It is a block of image against a paper field, not a full-bleed hero; that is a deliberate editorial choice, not an oversight.
- Project rows leave the lower right of each row empty. The contribution sits at the top of the right column and the expandable detail unfolds beneath it, so a closed row reads as airy rather than dense. On a wide screen this is generous; on a 1280px laptop it is close to sparse.
- The Life wall is a plain four-column grid, not a masonry composition. Uneven tile heights were tried during the rebuild and made the section look broken rather than composed.

Iteration 10 · 2026-10-27 · Life wall rebuilt as a compact mosaic

Why. The editorial skeleton was accepted; the photo wall inside it was not. Verbatim: "你这个图片太平铺、太大了，你再多提取一些样本学习一下吧". Measured faults at 1512 px: nine items in `repeat(4,1fr)` with every tile forced to `aspect-ratio:1.15`, tiles 340 × 296, wall height ~950 px — taller than the copy block above it — and a final row holding one film alone, which the eye reads as a stray panel.

Method. Samples first, then three rendered candidates, then one implementation. The sweep and the rejected candidates are recorded in `gallery_pattern_sweep.md`; the prompt and its constraints in `icl_prompt_revision9.md`.

Chosen composition. 20-column grid, eight tiles at `span 4` (267 × 178) in two bands, the 2:3 portrait pinned to `17 / span 4` across both bands (267 × 400, uncropped). Captions moved out of the photographs into the margin. Wall height 420 px, tile area roughly half of the previous wall.

Files changed.

File	Change
`v1.css`	new R10 layer; deleted the dead iteration-9 gallery rules (`repeat(4,1fr)`, the `!important` span reset, `aspect-ratio:1.15`) and the inheritance-facing bits they relied on
`app.js`	`addGalleryItem()` takes an optional poster and sets `el.poster`; the two films now ship a captured frame
`assets/`	added `life-video-1-poster.jpg` (87 KB) and `life-video-2-poster.jpg` (71 KB)

Nothing else moved. `index.html`, `style.css`, `projects.js`, `rag.js`, `knowledge-data.js` and every existing photograph are unchanged. V0 remains byte-identical.

Latent defects found while rebuilding.

- Two rules from the scrapbook era were still live and silently broke the new grid: `.media-gallery.media-wall{grid-auto-rows:27px}` pinned every row to 27 px, and the `:nth-child(1..9)` block still assigned explicit `grid-row` spans. Both were removed instead of being overridden with `!important`.
- A film tile could paint as an empty beige rectangle before its metadata arrived. Fixed with poster frames rather than `preload="auto"`, which would fetch 31 MB on first paint.

Verification. `test_portfolio_e2e.cjs` 35/35 PASS, 0 console and 0 page errors (`evidence/V1/V1-R10-e2e-results.json`); `check_v1_v0_content.cjs` 48 copy groups, 0 differences (`evidence/V1/V1-R10-content-parity.json`). Wall measured at 1512 / 1280 / 1024 / 820 / 414 px: five-across at the top two widths, four-across to 1120 px, two-across below 980 px, flush right at every one, no horizontal overflow.

Test change, recorded rather than hidden. `E2E-08d` asserted "varied card sizes and staggered positions" (⩾4 distinct `w×h`, ⩾5 distinct `y`). That described the V0 scrapbook mosaic and would fail on any flush composition, including this one. It now asserts what the design actually claims: ⩽2 distinct widths, exactly one tile ⩾1.6× the median height, flush right edge, every caption outside its frame.

Known trade-offs.

- Tiles are small on purpose. 267 px shows a scene, not a face; the dialog is where a photograph is examined.
- The composition is tuned for nine items and the portrait slot is pinned to `:nth-child(7)`. A tenth photograph needs a new slot, not a new rule.
- Between 981 and 1120 px the wall becomes 3 + 3 + 2 because the portrait still claims two rows. It stays flush, but it is a different composition rather than a scaled one.

Iteration 11 · 2026-10-28 · caption plate moved off the hero photograph

Why. The author sent a crop of the bottom of the hero image — the caption bar, the three thumbnails and the two round arrows — with one line: "不要这里". The strip sat directly over the sitter's gown and bouquet. Everything it carried was legible only as clutter: at 82 × 47 the thumbnails showed a rectangle of sky and a caption set at 7 px.

What moved. The photograph now carries only the picture and the corner stamp. Caption, scene counter and switcher sit on a hairline immediately beneath the frame, aligned to its left edge and running to its right edge:

	Before	After
Caption	translucent bar at `bottom:60px`, inside the frame	`10.5px` tracked caps on the hairline, outside the frame
Counter	`— 01 / 03` inside the bar	`01 / 03` at the right end of the plate
Scenes	three 82 × 47 thumbnails on a blurred pill	three 20 × 2 px rules, 34 px when active
Arrows	two 34 px circles floating on the image	two 24 px glyphs at the end of the plate

Files changed.

File	Change
`index.html`	`.cover-visual` wrapper added; `figcaption` and `.art-controls` removed from the `<figure>`; `.cover-plate` added under it with `#art-caption-text`, `#art-counter`, `#art-dots`, `#art-prev`, `#art-next`
`v1.css`	R11 layer appended; the dead thumbnail rules deleted (`.art-controls` ×3, `.art-caption` ×3, `.scene-dot` block, the media-query overrides) rather than left to be overridden
`app.js`	`showScene()` builds the dots without `<img>` or `<span>` now that the selector is a rule; counter text drops the em dash

`style.css`, `projects.js`, `rag.js`, `knowledge-data.js`, every asset and every photograph are unchanged. V0 remains byte-identical.

Accessibility kept. The caption is still a live region (`aria-live="polite"`), each rule still carries `aria-label="View scene n: …"` and `aria-pressed`, the photograph is still clickable and focusable as a "next scene" control, and the arrows remain real buttons. Only the decoration changed.

Correction, 2026-10-28. This entry first claimed that `assets/traumerei.mp3` had never shipped in any build, so the music toggle 404'd in silence. That was wrong, and it was wrong because it was reasoned from the two "failed resource" lines the E2E run reports rather than measured. The file is present in both builds (2,487,708 bytes in V0 and V1, hashed in `MANIFEST.sha256`), it decodes (`readyState 4`, duration 155 s), and a click on the note in the navigation bar starts playback (`paused false`, `currentTime` advancing, `aria-pressed="true"`). A page-level scan of every response during load and playback returns no 4xx or 5xx at all; the two failed resources are `favicon.ico` and its default-path twin, which V0 also lacks. The toggle behaves as designed: autoplay is blocked on load, the reader's click starts the music. No latent defect existed here, and nothing needs fixing. Left in the record rather than quietly deleted so the earlier claim is not mistaken for a finding.

Test change, recorded rather than hidden. `E2E-03` and `E2E-03b` asserted the string `— 02 / 03` and the presence of three `<img>` inside `#art-dots`. Both were properties of the old overlay. They now assert `02 / 03` and three `.scene-dot` buttons. The assertions test the same behaviour — advance, jump, return — under the new markup.

Verification. `test_portfolio_e2e.cjs` 35/35 PASS, 0 console and 0 page errors (`evidence/V1/V1-R11-e2e-results.json`); `check_v1_v0_content.cjs` 48 copy groups, 0 differences (`evidence/V1/V1-R11-content-parity.json`). Plate measured at 1512 / 1280 / 1024 / 820 / 414 px: caption left edge on the photograph's left edge, plate right edge flush with the photograph's right edge, no horizontal overflow. Below 1121 px the plate wraps to two lines — caption on its own row, controls right-aligned under it — because at 1024 the photo column is ~470 px and one line could hold the caption or the controls, not both; letting the caption break left "track" orphaned above the rules.

Known trade-offs.

- The scene whose picture is not on screen now has no preview. Three rules say "there are three, you are on the first"; they cannot say what the others look like. That is the cost of taking the strip off the photograph, and it is the intended reading: the cover is a photograph, not a viewer.
- The caption repeats, in tracked caps, the photograph's `aria-label`. Deliberate — the label serves screen readers, the line serves sighted readers, and they say the same thing.

Iteration 12 — the skill lines open (2026-10-28)

What was reported. "为什么这个点开不能看见具体介绍了" — a review of the `THINGS I WORK WITH` band in which the four lines were clicked and nothing happened.

What was actually there. The four `<li>` were plain V0 copy: a row of four beads with one filled, a title, a one-line summary. No handler, no link, no `tabindex`. The beads made each line look like an unselected carousel page, which is exactly the affordance that invites a click, and the only live control in the band sat 80 px below it — the tool chips, whose own label reads `TOOLS · CLICK TO SEE RELATED PROJECTS`. So the band said "click me" twice and answered once, and the one place it answered was the one place that looked least like a button.

What changed. Each line is a disclosure now.

Element	Before	After
The line itself	static `<li>`	`<button class="skill-note-toggle">` filling the row
Indicator	four decorative beads only	beads plus a 11 px + at the row's right edge, rotating 45° when open
Resting copy	title + one line	unchanged, character for character
Panel	none	hairline-topped panel, two evidence lines and a link into the matching project story
Collapsed panel	—	`inert`, so it is never tabbed into while invisible

Each panel closes on the evidence behind its one-liner: the parsing pipeline and the JSON-Schema guardrails for Python; BM25 with SQLite FTS5 against the SentenceTransformer and ClickHouse pair for RAG; the Future Internet review for AI Agents; the Vue 3 / Spring Boot / ECharts stack for Full-Stack. Every claim is drawn from a project story already on the page or from the CV facts already in it; no new biography was invented.

The link does two things. Clicking `Intelligent Policy Q&A System ↗` jumps to `#project-story-2` *and* unfolds that project's note, so the reader lands on the evidence rather than on a folded row. If the note is already open it is left alone.

Files changed.

File	Change
`index.html`	the four `<li>` rebuilt as `button` + `skill-detail` pairs; `#skill-detail-0…3`
`v1.css`	R12 layer — disclosure, panel, project link, focus ring; R12b fixes the ⩽ 980 px intro stack

File	Change
`v1.js`	toggle handler with `aria-expanded` / `inert` sync; project link unfurls its story
`check_v1_v0_content.cjs`	the parity collector strips `.skill-detail` before reading `.skill-notes li`

`style.css`, `projects.js`, `rag.js`, `knowledge-data.js`, all assets and every photograph are unchanged. V0 remains byte-identical.

Why the plus is generated content. `.skill-note-toggle::after` draws the +, so it never enters the DOM text. The resting string of each line stays byte-identical to V0 and the parity check keeps reading the same four strings without a special case.

Test change, recorded rather than hidden. The parity collector compared the `innerText` of `.skill-notes li` between the two builds. The new detail panels live inside those `li`, so their copy would have counted as a V0/V1 difference. The collector now drops `.skill-detail` from the clone before reading it — the same treatment `.school-action-hint` already gets, and for the same reason: the disclosure is additive, and the guarantee the check exists to make is that no V0 copy was rewritten. It still compares 48 groups and still finds 0 differences.

A second defect found while checking the first. Below 980 px the band's eyebrow printed straight through its own headline: the intro column put eyebrow, title and footnote in one named grid cell and separated them with margins tuned for the 30 px desktop title. At 414 px the title is 21 px and the eyebrow wraps to two lines, so `margin: 26px 0 0` no longer cleared it — measured at y 2374 h 40 against y 2400, a 14 px overlap. Pre-existing since iteration 9 and unrelated to the disclosure, but visible on every phone. R12b gives each line its own row below the breakpoint; the three now sit at 2374 / 2414 / 2472 with no overlap.

Verification. `test_portfolio_e2e.cjs` 35/35 PASS, 0 console and 0 page errors (`evidence/V1/V1-R12-e2e-results.json`); `check_v1_v0_content.cjs` 48 copy groups, 0 differences (`evidence/V1/V1-R12-content-parity.json`). Behaviour measured in Chrome: `aria-expanded` flips `false → true`, `#skill-detail-2` reports `inert true` when folded, the project link leaves `#project-story-2` open with `location.hash` at `#project-story-2`, and the toggles are real `BUTTON` elements so Tab and Enter reach them. No horizontal overflow at 1280 / 1024 / 820 / 414 px (0 px at each).

Correction carried into this pass. Iteration 11 recorded the missing `traumerei.mp3` as a latent defect. It was not missing — see the correction note in that entry. The music was measured this time instead of inferred: it plays.

---

few shot examples

Archive source: `prompt_appendix/task3_v1/few_shot_examples.md`

V1 Few-Shot Demonstrations and Reference Mapping

The four demonstrations are paired instruction/output examples used in the V1 generation prompt. They demonstrate component transformation rather than copying source code. Output snippets below are the target component contracts given to the model; the actual complete generated files

are preserved under `llm_output/`.

Example	Input → expected output	V1 manifestation	Structural reference pattern
1. Education disclosure	Two study stages → two paper-like cards, chronological connector, accessible expand/collapse, all supplied facts retained.	Pegboard layout, bundled V0 photos, existing education facts and details, curved SVG thread; the original `aria-expanded` disclosure behavior is retained.	al-folio and academic-homepage: compact academic information hierarchy, adapted to the V0 scrapbook aesthetic.
2. Contribution-first project card	Project title/purpose/contribution → separate scannable contribution label, expandable paper card, keyboard control.	`v1.js` adds "MY CONTRIBUTION" presentation around the unchanged V0 contribution strings; existing paper cards and details remain.	michaelkolesidis.com: section-led portfolio storytelling; JiyaBatra ArtPortfolio: gallery/card navigation structure.
3. Evidence-grounded retrieval	"How does MapReduce work?" → retrieve the matching Hadoop/MapReduce note, quote it and show source.	`rag.js` adds example-derived topic profiles and a heading relevance boost while retaining BM25 and the original Markdown corpus.	Retrieval and interaction patterns adapted to this site's own notes; no reference copy or third-party text is used.
4. Abstention	Out-of-corpus question → transparent no-evidence response, no unrelated excerpt.	`rag.js` rejects low-overlap results rather than presenting a weak match as an answer.	A grounded interface pattern, implemented against the local V0 evidence only.

Example outputs supplied to generation

```
<article class="study-card">
  <button aria-expanded="false" aria-controls="study-details">Study stage</button>
  <div id="study-details" inert>Only the supplied education facts and details.</div>
</article>
```

```
<p class="project-purpose">Purpose from the source record.</p>
<span class="contribution-label">MY CONTRIBUTION</span>
<span class="contribution-copy">Exact contribution from the source record.</span>
```

```
CLOSEST NOTE EXCERPT
[verbatim retrieved passage]
— from [course] · "[heading]"
Topic matched
```

```
No supporting passage was found in the supplied course notes.
Try a question about MapReduce, RAG, prompt engineering, or agent planning.
```

Why these examples are few-shot learning

The prompt includes multiple task demonstrations with concrete inputs and desired output behavior before the target request. They show how to convert content needs into code-level component decisions, including accessibility and evidence boundaries. The generation is conditioned on these

in-context examples; V1 does not train or fine-tune model parameters. The retrieval topic profiles are a resulting deterministic implementation of the MapReduce/RAG/agent-planning demonstrations, not a separate deployed LLM.

---

few shot examples revision2

Archive source: `prompt_appendix/task3_v1/few_shot_examples_revision2.md`

Few-Shot Demonstrations — Structural Revision 2

These examples were included in the V1 generation prompt. They encode component and interaction transformations; no example provides personal content or replaces Aria's V0 facts.

1. Long-page chapters

- **Input need:** Visitors should understand the long-scroll sequence and jump to existing sections.
- **Demonstrated output:** A compact, numbered chapter rail immediately beneath the Home hero, linking to Highlights, Projects, and Life, with keyboard-operable anchors and responsive placement. It sits outside the hero image/carousel control area to prevent overlap.
- **Applied in V1:** `index.html` `.chapter-rail`; styles in `v1.css`; checked by `E2E-02b`.

2. Carousel scene selection

- **Input need:** A visitor should preview and select a scene instead of using only previous/next arrows.
- **Demonstrated output:** Semantic `figure`/`figcaption`, image thumbnails, a selected state, and synchronized image/caption/counter updates.
- **Applied in V1:** Hero selector markup and `app.js` thumbnail rendering/selection; checked by `E2E-03b`.

3. Project stories

- **Input need:** Provide a quick index and make purpose and personal contribution distinguishable.
- **Demonstrated output:** An index of anchored stories, semantic ordered list, asymmetrically arranged paper cards, explicit expandable notes, filters, and persistent saves.
- **Applied in V1:** `index.html` story index and `ol`; `app.js` story renderer; `v1.css` atlas layout; checked by `E2E-04c`, `E2E-04d`, `E2E-04b`, `E2E-05*`, and `E2E-06*`.

4. Filtered reflow

- **Input need:** Category changes should not make remaining stories jump abruptly or lose their identity.
- **Demonstrated output:** Preserve the same records and animate their position/opacity during filter reflow.
- **Applied in V1:** `.project-entry` transition and filtering in `app.js`; checked by filter result assertions `E2E-05a` - `E2E-05c`.

---

few shot examples revision3

Archive source: `prompt_appendix/task3_v1/few_shot_examples_revision3.md`

Few-Shot Demonstrations — Visual Refinement 3

---

1. Editorial navigation instead of duplicate tile cards

- **Input:** Navigation/index links are useful, but standalone tiles repeat the visual grammar of the content cards.
- **Output pattern:** A compact index with subtle separators and small typographic numbering; keep large cards for content and give the featured area breathing room.
- **Applied:** `.chapter-rail` is a single glass strip after the Home hero; `.project-atlas-index` uses a simple four-part editorial index. They no longer overlay carousel controls or add a second set of project cards.

2. Artwork-first media controls

- **Input:** A visual portfolio should let visitors select work directly without making controls dominate the image.
- **Output pattern:** Direct image thumbnails with one clear selected state, compact controls, and no unrelated stock imagery.
- **Applied:** Existing V0 carousel assets remain the source of the thumbnail and hero image. No new image asset or unrelated photograph was introduced.

3. Restrained interaction feedback

- **Input:** Clickable elements need feedback while the overall portfolio remains quiet.
- **Output pattern:** Small hover/focus shifts or a selected-state change, avoiding animation that moves neighboring content or obscures content.
- **Applied:** The chapter/index link hovers subtly highlight their own target; carousel selection and project filtering retain their existing stateful interactions.

few shot examples revision4

Archive source: `prompt_appendix/task3_v1/few_shot_examples_revision4.md`

Few-Shot Demonstrations — Life Media Wall and Marginalia (Iteration 4)

These examples abstract component and interaction patterns from the previously reviewed references. They do not reproduce reference-site copy, identity, or media.

1. Asymmetric media wall with keyboard-accessible viewing

- **Input pattern:** A photography portfolio uses an uneven, image-first gallery and an accessible viewing interaction.
- **Output pattern:** Use a deliberately varied, responsive composition; make every media tile a semantic button with an accessible name; preserve a single preview dialog and provide keyboard open/close behavior.
- **Applied to V1:** The existing seven photographs and two videos are rendered in a staggered scrapbook wall. Existing captions and video controls are retained. Enter opens the preview and Escape closes it; automated checks verify both.
- **Source pattern:** `Photography Wall` reference noted in `reference_patterns.md`; no reference assets or content were copied.

2. Hand-drawn SVG annotations as restrained page marginalia

- **Input pattern:** A scrapbook portfolio uses original line-art doodles and brief notes to make a page feel personal without expanding the main content blocks.
- **Output pattern:** Add small, low-contrast inline SVGs near an existing section heading or media area; hide purely decorative drawings from assistive technology; preserve the page's existing art direction.
- **Applied to V1:** Two original inline SVG line drawings (a botanical sprig and a small flight-path flourish) sit beside the Life heading and above the media wall. They use muted sage tones and do not overlap readable content.
- **Source pattern:** `Scrapbook Portfolio` reference noted in `reference_patterns.md`; no reference SVG or text was copied.

3. Preserve the owner's source material

- **Input pattern:** An image-led portfolio uses category thumbnails and direct media previews.

- **Output pattern:** Reorganize existing owner-supplied images without substituting unrelated stock photography; retain captions and preview behavior.
- **Applied to V1:** The same seven photos and two short films remain in the gallery, with existing labels and modal/video controls. No new personal image or text was introduced.
- **Source pattern:** Art portfolio and photography-wall patterns previously recorded in `reference_patterns.md`.

---

few shot examples revision5

Archive source: `prompt_appendix/task3_v1/few_shot_examples_revision5.md`

Few-Shot Examples — V1 Component Architecture (Iteration 5)

These are pattern-level examples abstracted from the four GitHub references reviewed earlier. They record the reusable design pattern and the transfer, not any reference author's copy, identity, or assets.

Example 1 — Editorial academic profile

- **Observed pattern:** use a clear editorial hierarchy, short section introductions, and a readable primary content column instead of a dense dashboard of uniform widgets.
- **Abstract component:** `FolioSection(eyebrow, heading, primaryStory, supportingLedger)`; the primary story carries the narrative while a smaller ledger presents supporting facts.
- **V1 transfer:** Highlights becomes an education journey beside a skill ledger; the lecture-note RAG module becomes a separate knowledge workbench. Existing V0 strings are moved into those shells unchanged.
- **Expected effect on generation:** replace the flat academic rows with a semantic, visually tiered folio and a distinct side ledger.

Example 2 — Image-led art portfolio

- **Observed pattern:** let a dominant artwork lead, with small selectable previews and compact category/navigation cues that do not compete with the image.
- **Abstract component:** `ArtworkFeature(mainMedia, caption, index, thumbnails)`.
- **V1 transfer:** Home's existing carousel becomes a figure inside a journal-cover composition. Its existing captions and image alternatives remain; the chapter rail is a separate, flow-positioned navigation component.
- **Expected effect on generation:** make the existing illustration a major visual anchor and give the carousel a meaningful, synchronized thumbnail selector.

Example 3 — Interactive project portfolio

- **Observed pattern:** provide a scannable project index, semantic individual case/story articles, and compact disclosure controls for deeper detail.
- **Abstract component:** `StoryIndex(items) + ProjectArticle(title, purpose, contribution, details, actions)`.
- **V1 transfer:** render the original four V0 projects as numbered index links and ordered semantic articles. Retain category filters, favorites, expansion, original details, tags, and source links.
- **Expected effect on generation:** move from uniform text rows to a loose editorial story atlas with direct section navigation and meaningful article structure.

Example 4 — Personal visual archive

- **Observed pattern:** pair a short first-person introduction with an irregular media wall; use an accessible focused preview rather than turning every item into a heavy card.
- **Abstract component:** `LetterAside(copy) + MediaAtlas(existingMedia, keyboardPreview)`.
- **V1 transfer:** Life becomes a letter and image atlas containing the existing seven photographs and two videos. Guestbook uses the related open-letter form convention while preserving its original fields and state behavior.
- **Expected effect on generation:** vary the spatial rhythm, preserve the user's media, and make the preview interaction keyboard-accessible.

Few-shot transfer constraints

1. Transfer DOM/component composition, semantic structure, hierarchy, and interaction patterns only.
2. Never transfer reference copy, biography, identity, repository imagery, or personal facts.
3. Every business-text string must come from frozen V0; if the source string cannot be identified, omit the proposed copy instead of inventing it.
4. Keep the visual treatment within the user's low-saturation watercolor diary palette.

---

few shot examples revision7

Archive source: `prompt_appendix/task3_v1/few_shot_examples_revision7.md`

Few-Shot Examples — V1 App Shell (Iteration 7)

The previous rounds changed components inside the same page skeleton: a single column of paper sections under a pill navigation. Screenshot review of V1-R7 still read as the same website as V0, because the *frame* never changed.

Iteration 7 therefore conditions the model on **page skeletons**, not component styling. Each demonstration below is a full-page pattern abstracted from a public repository, reduced to its structural rule, then re-rendered with Aria's own copy, photographs and palette. No reference copy, identity, image or code was transferred.

Reference sweep (2026-10-27)

Repositories were inspected and rendered before anything was written; the ones that only repeat a centred hero plus a vertical card stack were discarded.

Repository	Stars	What it actually renders	Verdict
`Ladvace/astro-bento-portfolio`	392	Asymmetric card mosaic; every card is a different function	Kept as skeleton A
`codewithsadee/vcard-personal-portfolio`	8.1k	Fixed profile panel beside a tabbed content pane	Kept as skeleton B
`chrstnvi/zine-moskovskaya-excursia`	115	Full-bleed split spread, display type, meta row, rotated edge labels	Kept as skeleton C
`vivek9patel/vivek9patel.github.io`	4.3k	Ubuntu desktop simulation; content lives inside draggable windows	Rejected: content becomes undiscoverable for a marker
`Cveinnt/LiveTerm`	5.4k	Terminal emulation	Rejected: same discoverability problem, plus a typing metaphor that hides the work
`hackclub/scrapbook`	190	Daily journal cards with a streak grid	Rejected as a skeleton (blocked by a WAF anyway); its journal rhythm is already present in V1's paper cards
`rammcodes/Dopefolio`	3.8k	Centred hero, gradient blob, vertical cards	Rejected: structurally identical to V0

Example 1 — Bento mosaic (skeleton A)

- **Observed pattern:** one grid, twelve columns, cards of unequal span; each card carries exactly one fact or one control; there is no section-level scroll.
- **Abstract component:** `Mosaic(cells[])`, where a cell is `{span, role: identity | evidence | interactive}`.
- **Transfer attempt:** built as `work/design-draft/skeleton-A-bento.html` with Aria's copy — intro card, campus photograph, GPA figure, tool chips, the course-note chat, three project rows, a four-image strip, two school rows, a guestbook stub.
- **Result:** strongest "this is a different site" reading of the three, but the mosaic collapses into a plain card list below 720 px, which is where most of the marking will happen.

Example 2 — Profile rail plus content window (skeleton B, chosen)

- **Observed pattern:** a persistent identity panel on the left holds the person, their meta facts and the navigation; the right side is a bounded window whose title bar names the current view;

the window, not the page, scrolls.

- **Abstract component:** `AppFrame(Rail(identity, meta, nav, player), Window(TitleBar(crumb, counter), Pane[], StatusBar))`.
- **Transfer attempt:** built as `work/design-draft/skeleton-B-app-shell.html`, then implemented in `coursework/website/V1/`.
- **Why it was chosen:** it changes the reading model rather than the ornament — a visitor always sees who this is and where they are, and every V0 section keeps its own content and interactions.

Example 3 — Editorial split spread (skeleton C)

- **Observed pattern:** each section is a full-bleed two-panel spread; one panel is a flat colour block carrying an oversized headline and a three-column meta row, the other is a single photograph; section labels are rotated along the edge.
- **Abstract component:** `Spread(textPanel(headline, kicker, metaRow), fullBleedMedia, edgeIndex)`.
- **Transfer attempt:** built as `work/design-draft/skeleton-C-editorial.html`.
- **Result:** the most "designed" of the three, but information density drops sharply — one project per screen is too slow for four projects plus coursework.

Few-shot transfer constraints

1. Transfer skeleton, hierarchy and interaction pattern only — never copy, identity, imagery, or source code from a reference.
2. Only text that already exists in frozen V0 may appear; when the shell needs a label, it reuses V0's own strings (`Träumerei`, `with a little curiosity, Aria`).
3. Existing identifiers and behaviours must survive: `[data-nav]`, `.chapter-rail`, `#project-list`, `#media-gallery`, `#guestbook-form`, `#rag-form`, and every disclosure, filter, favourite and preview control.
4. The palette stays Aria's watercolour sky palette; the shell adds surfaces and spacing, not new colours.

---

gallery pattern sweep

Archive source: `prompt_appendix/task3_v1/gallery_pattern_sweep.md`

Gallery pattern sweep · iteration 10 (2026-10-27)

The author rejected the Life image wall on sight: "你这个图片太平铺、太大了，你再多提取一些 样本学习一下吧" — with a screenshot showing four equal columns of near-full-width photos. This document records the samples consulted, what each one contributed, and why the chosen composition is the one it is.

1. What was actually wrong (measured, not guessed)

Before the sweep, at a 1512 px viewport:

Property	Value
Items	9 (7 photos + 2 films)
Source ratios	six at 3:2, one at 2:3 (the double-bass portrait), two films at 16:9 / 4:3
Grid	`repeat(4, 1fr)`, `gap: 8px`, every tile forced to `aspect-ratio: 1.15`
Tile size	340 × 296 px
Wall height	~950 px — larger than the copy block above it
Last row	1 item, so the closing film took a quarter of the width on its own

Three faults follow from this: every tile is the same size (no hierarchy), every tile is large (the section reads as a photo dump rather than a page), and the tail row leaves one tile stranded.

2. References consulted

Source	What it contributed
SmugMug gallery-style taxonomy (Grid / Collage / Justified / Masonry / Slideshow)	The vocabulary; specifically that *justified* fixes row height and lets width vary, *masonry* fixes column width and lets height vary
60 Minute Sites, "Photography Portfolio Layout Options Explained"	The explicit trade-offs: standard grid "can feel rigid"; masonry "can feel chaotic if not carefully balanced"; justified keeps ratios but "row compositions may shift on different screen sizes"
mediagallery.org, "Media Gallery Layouts"	The single most useful warning: a uniform grid "asks a portrait, a wide scene and a near-square composition to occupy the same shape", and on the final row "some implementations enlarge a lone remaining image… a quiet detail picture can suddenly look as if it was appointed chairman of the collection". That is exactly what the 4-column grid did to the closing film.
master-addons.com masonry comparison	Confirms the honest implementation routes: CSS columns, or a grid with explicit placement. No JS library needed for nine fixed items.
GitHub: `miromannino/Justified-Gallery` (used by 500px), `nk-o/flickr-justified-gallery`	Confirms justified rows are the photographer-side default for mixed orientations — and that they rely on measuring images at runtime, which a static nine-item page does not need.

3. Rendered candidates

All three were rendered with Aria's real photographs at 1512 px and inspected as screenshots (`work/design-draft/gallery-samples.html` keeps the study page).

Candidate	Composition	Tile size	Wall height	Verdict
A · Contact sheet	5 + 4, uniform	268 × 179	~440	Compact and tidy, but still a wall of equal cells — the flatness moved down in scale

Candidate	Composition	Tile size	Wall height	Verdict
				instead of away
B · Editorial mosaic	5+5+note / 4 / portrait+2 films	566 → 224	~760	Real hierarchy, but the opening pair is bigger than what was already there — the opposite of the complaint
C · Justified rows	3 per band, true ratios	360 × 240	~900	No crop at all, but three different right edges read as unfinished
D · Compact mosaic (chosen)	**4 + 4, one portrait spanning both bands**	**267 × 178**	**~420**	Small tiles, one vertical accent, all edges flush, no stranded tile

4. Why D

- **It fixes the size complaint by arithmetic, not by opinion.** Nine items in two bands of four leaves the ninth slot for the portrait, which is the only image whose shape wants to be tall. Tile area drops from 100 640 px² to 47 526 px² — about 53 % of the old wall — and the wall height falls from ~950 px to ~420 px.
- **The portrait is not decoration.** At 267 × 400 the 2:3 frame is shown at 0.667 with no crop at all, while every landscape tile is a 3:2 frame in a 3:2 box. The accent is the one tile that is *not* cropped.
- **Nothing is stranded.** The right edge is flush in both bands; the closing tile is the tall portrait, not a leftover film.
- **Captions moved out of the photographs.** The gradient-overlay caption was the strongest "stock gallery widget" signal on the page; the caption now sits under each frame in 9.5 px tracked caps, the way a contact sheet is annotated.

5. Rejected on the way

- **Justified rows** — correct preservation of aspect ratio, but three ragged right edges in a row on an otherwise hairline-flush page.
- **Masonry via CSS columns** — solves raggedness, but the visual order stops matching the DOM order, so keyboard focus jumps around the grid. Not acceptable for a page that opens every tile with a button.
- **Restoring the V0 scrapbook** (rotated cards, tape, shadows) — this is what iteration 9 deliberately removed; the author asked for no boxes.

6. Costs, stated plainly

- The tiles are small: 267 px wide at 1512. Faces are legible, details are not; the dialog is where a photograph is actually looked at. This is a deliberate trade for density.
- The composition is tuned for nine items. A tenth photograph would need a new slot, and the portrait slot is pinned to `:nth-child(7)` — the comment in `v1.css` says so.
- Between 981 and 1120 px the grid drops to four per band and the tall portrait takes the third row band with it, so the wall becomes 3 + 3 + 2. It is still flush, but it is a different composition, not a scaled one.
- A video tile used to paint nothing while `preload="metadata"` was still fetching metadata. Two poster frames were captured from the clips (`assets/life-video-1-poster.jpg`, `-2-poster.jpg`)

rather than shipping 31 MB of video on first paint.

7. Test change record

`E2E-08d` previously asserted "varied card sizes and staggered positions" (⩾4 distinct `w×h`, ⩾5 distinct `y`) — written for the V0 scrapbook mosaic, which no longer exists. It now asserts the property the current design actually claims: uniform small tiles (⩽2 distinct widths), exactly one tile at least 1.6× the median height, a flush right edge, and every caption outside its frame. The old assertion would have failed on any flush composition, including this one; that is why it was restated rather than dropped.

---

icl prompt

Archive source: `prompt_appendix/task3_v1/icl_prompt.md`

V1 Generation Prompt — In-context Few-Shot Learning

Version: Task 3 / V1
Technique: In-context learning with four input/output demonstrations (few-shot examples)
Baseline: `coursework/website/V0/` only
Use: This is the generation prompt recorded for the V1 source in `coursework/website/V1/`.

Prompt

> You are a careful front-end engineer editing a static personal portfolio. Generate V1 from the provided, frozen `coursework/website/V0/` files only. Do not use V2, earlier `outputs/aria-v1` drafts, or any other derivative as a source. Preserve all personal facts, project details, education records, scores, media, music, course notes, and visitor features that exist in V0. Do not invent facts, replace the owner's writing with reference-site copy, or import any personal information or assets from references. Keep the existing hand-painted blue-sky visual identity, colors, navigation labels, long-scroll page structure, and static HTML/CSS/vanilla-JS deployment model.
>
> Use the four demonstrations below as few-shot examples of the *transformation pattern*: turn a visitor need into a concrete, accessible component while keeping the supplied evidence and content intact. The demonstrations are authored for this task from structural patterns observed in the reference sites; they are not copied source code or personal content. References are structural inspiration only: alshedivat/al-folio, shaoyanliu academic homepage, JiyaBatra/ArtPortfolio-jtc, and michaelkolesidis/michaelkolesidis.com.
>
> **Few-shot example 1 — education disclosure**
> Input need: "Visitors need to compare two study stages without seeing every award and course at once."
> Desired output pattern: preserve both institutions and their supplied facts; use two distinct paper-like cards with a clear education sequence, one keyboard-operable disclosure button per

> card, `aria-expanded`, and a smoothly expanding detail region. A quiet hand-drawn connector may imply chronology. Do not fabricate campus imagery; use only bundled V0 photos or no photo.
>
> **Few-shot example 2 — project contribution**
> Input need: "A visitor skims a portfolio and needs to distinguish the owner's role from the project purpose."
> Desired output pattern: keep the original title, purpose, contribution, tags, and evidence; show purpose and contribution as separate visual lines with a small 'MY CONTRIBUTION' label. The card can vary slightly in angle and size, but its expand control must remain explicit and keyboard accessible.
>
> **Few-shot example 3 — knowledge chat evidence**
> Input: "How does MapReduce work?"
> Grounded output pattern: rank a passage whose heading or text concerns MapReduce/Hadoop/map-shuffle-reduce above unrelated course-note passages, quote only the retrieved note text, and show the course and heading. Do not claim that a local lexical retriever is an LLM or that it generated a synthesized answer.
>
> **Few-shot example 4 — unsupported course-note question**
> Input: "How do I bake a sourdough loaf?" (not in the supplied course-note corpus).
> Grounded output pattern: say that no supporting passage was found and offer a relevant course-topic suggestion. Never guess an answer or cite an unrelated passage.
>
> **Target task:** Apply these patterns to the frozen V0 website. The V1 iteration should make Highlights feel like a softly scattered pegboard, clarify project-contribution scanning, improve evidence-aware local course-note retrieval, and synchronize visible navigation/filter state with accessibility state. Keep the Home, Highlights, Projects, Life, and Guestbook sections; do not remove or rewrite V0 facts. Reuse only V0's existing files and media. Add reduced-motion handling, responsive layout, and gentle transitions of at least 0.35 seconds for new/modified interactions. No backend, no dependencies, no uploads, no autoplay changes, and no new user data collection.
>
> Output complete changed files, not isolated snippets: `index.html`, any V1-only CSS/JS, and changed V0-derived JavaScript. Include a concise change list. Mark V1 additions in code comments with `【V1迭代新增】`. Do not modify the frozen V0 directory or the Task 2 rubric/evaluator.

Recorded implementation boundary

The runtime examples above informed a deterministic, browser-local retrieval refinement. V1 still retrieves and quotes stored Markdown excerpts; it has no runtime LLM, network request, or generated answer. The only source corpus is the V0 course-note corpus. V1 deployment has not yet been performed; no URL is claimed here.

---

icl prompt revision10

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision10.md`

Task 3 · V1 prompt, revision 10 — take the caption strip off the photograph

Trigger. One screenshot crop and four characters: "不要这里". The crop showed the bottom third of the hero photograph carrying a light caption bar (`A bright start on the campus track … — 01 / 03`), a row of three 82 × 47 thumbnails, and two 34 px circular arrows, all floating over the sitter's gown.

How the instruction was read. "这里" is a place, not a thing: the request is about position, so the elements move rather than disappear. Deleting the caption and the switcher would have removed the only way to reach scenes two and three — a functional loss the author did not ask for. The revision therefore relocates, it does not delete.

Constraints handed to the generation.

1. The `<figure>` holds the photograph and the corner stamp only. No text, no control, no scrim on top of the picture.
2. Caption, counter and switcher move onto a hairline directly beneath the frame, left edge aligned to the photograph's left edge, right edge flush with its right edge.
3. The switcher must stop pretending to be a viewer. Three image thumbnails at 82 × 47 show a rectangle of sky and a 7 px caption; they become rules — 20 × 2 px, 34 px when active.
4. The counter drops the leading em dash it carried when it trailed a caption: `01 / 03`.
5. Nothing else is touched — no section re-layout, no copy rewrite, no change to `style.css`, `projects.js`, `rag.js`, `knowledge-data.js` or any asset.
6. Every behaviour survives: arrow buttons, clicking the photograph, Enter/Space, the `aria-live` caption, `aria-pressed` on each rule.

Deviations forced by the build.

- The plate could not simply be nudged with `position` — the old caption and controls were absolutely positioned inside a fixed-height `<figure>`. They had to leave the element, which meant wrapping the figure and the new plate in `.cover-visual` so the cover grid kept two columns rather than inventing a third row.
- Removing the strip made seven now-dead rules in `v1.css` visible (three `.art-controls`, three `.art-caption`, one `.scene-dot` block plus its media-query overrides). They were deleted rather than overridden, because a rule that can never match is a trap for the next edit.
- Below 1121 px the plate wraps: caption on its own line, controls right-aligned beneath. The first attempt wrapped at 600 px only, and at 1024 the caption broke after "campus", leaving "track" orphaned above the rules — one line could hold the caption or the controls, not both.

Correction to a claim made here (2026-10-28). This record first said `assets/traumerei.mp3` "has never been shipped, so the toggle 404s and plays nothing". That was wrong. The file is present in V0 and V1 (2,487,708 bytes, hashed in `MANIFEST.sha256`), it decodes (duration 155 s) and clicking the note in the navigation bar starts playback. The two failed-resource lines the E2E run reports are `favicon.ico` and its default-path twin, which V0 also lacks. The mistake was inferring a missing asset from those two lines instead of checking the directory; corrected in `change_log.md`, iteration 11.

icl prompt revision11

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision11.md`

Task 3 · V1 prompt, revision 11 — make the skill lines open

Trigger. One screenshot of the `THINGS I WORK WITH` band and one question: "为什么这个点开不能看见具体介绍了". Not a design taste complaint — a report that a control the author expected to work did nothing.

Diagnosis before any writing. The four `<li>` were V0 copy with no handler, no link and no `tabindex`; the only live control in the band was the tool-chip strip 80 px below, whose own label reads `TOOLS · CLICK TO SEE RELATED PROJECTS`. The four beads per row (one filled) read as an unselected carousel page, which is what invited the click. So the failure was an affordance that lied, not a missing paragraph.

Samples. This iteration did **not** run a new external sweep, and the record should say so plainly rather than implying one. The pattern came from two places already inside the project:

1. **In-page precedent** — the education rows above it (`.school-head` / `.school-detail`, `aria-expanded` plus `inert`) already open a hairline panel on click. Reusing that pattern makes the band read as one system instead of a second, invented interaction.
2. **`reference_patterns.md`** — the Korbinian Möller entry ("detail modals … useful for making academic/project details scannable and inspectable without adding more full-size tiles") and the Hitesh Nandakumar entry ("project write-ups, visible capabilities") are what justify inspectable-detail-on-demand over a modal or a jump to another section. Neither was re-read as a new sample; they were already the stated basis for detail-on-demand in this build.

Constraints handed to the generation.

1. The line itself becomes the control — the whole row, not a link at its end.
2. Resting copy stays byte-identical to V0: beads, title, one-line summary. Any new affordance must be generated content, not DOM text, so `check_v1_v0_content.cjs` keeps reading the same four strings.
3. The panel carries evidence, not adjectives: two lines drawn from project facts already on the page, plus one link to the story that backs them. No new biography.
4. The link must land the reader on the evidence, so it also unfolds the project story it jumps to.
5. Collapsed panels are `inert` — never tabbable while invisible.
6. Nothing else on the page changes. No copy rewrite, no new section, no new colour.

Deviations forced by the build.

- The parity collector compared `.skill-notes li innerText` across builds, and the panels live inside those `li`. It now drops `.skill-detail` from the clone before reading, the same treatment `.school-action-hint` already had. The guarantee is unchanged — no V0 copy was rewritten — but the mechanism had to admit additive copy.
- The band's eyebrow had been printing through its own headline below 980 px since iteration 9 (one grid cell, margins tuned for the 30 px desktop title). Found while checking 414 px, fixed in

the same pass as R12b. Pre-existing, not caused by the disclosure, but it would have been dishonest to ship a new mobile check and leave that in place.
- The + mark had to be `::after` rather than text, because a text node would have entered `innerText` and turned the equality check into a special case.

Not done, deliberately. No new external sample sweep for this iteration. If the archive needs a fresh few-shot set per iteration, that is a decision to take on its own — inventing samples after the fact would be fabrication.

---

icl prompt revision2

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision2.md`

V1 Few-Shot Prompt — Structural Revision 2

Purpose: Correct the documented limitation of the first V1 generation: its most visible changes were concentrated in Highlights/RAG, while the Home and Projects DOM and interaction patterns remained close to V0. This is a further V1 iteration from the frozen V0 baseline. It is not V2 and does not use V1 as a source for another version.

Full prompt used for the structural revision

> Work only from the frozen source files in `coursework/website/V0/`. Preserve all personal facts, project evidence, education records, media, music, course-note corpus, section content, and existing working visitor features. Do not use reference-site copy or personal information. Do not remove any V0 content. Produce a meaningful V1 structural transformation, not a palette-only or CSS-only restyle. Keep static HTML, CSS, and vanilla JavaScript; no backend, dependencies, generated personal claims, or autoplay changes. Mark new regions with 【V1迭代新增】 and keep the frozen V0 directory byte-identical.
>
> Apply these four few-shot demonstrations as reusable transformation patterns. They describe structure and behavior, not merely visual styling:
>
> **Example 1 — from page links to a chapter route**
> Input: "A long portfolio has several sections. Visitors should understand the order and jump directly to the part they want."
> Output pattern: preserve global navigation and page content; add a three-stop chapter rail immediately beneath the Home hero, with numbered links to existing sections. Keep it outside the hero image and carousel controls so the two navigation systems never overlap. Make each link keyboard-operable and give it a visible hover/focus response. On a small screen, keep the rail in normal document flow.
>
> **Example 2 — from passive hero image to scene selector**
> Input: "The Home image is a carousel, but visitors cannot preview or directly select a scene."
> Output pattern: keep the existing carousel data and previous/next actions; turn its selector

> controls into small image thumbnails with captions and one clear selected state. Selecting a thumbnail updates the main image, caption, counter, and pressed state together. Use a semantic figure and figcaption; retain alt text and gentle transitions.
>
> **Example 3 — from a flat project list to navigable project stories**
> Input: "Visitors need a quick overview of a few projects, then a direct way to inspect a chosen one and understand my own contribution."
> Output pattern: retain every supplied project fact; add a compact story index linking to individual project sections; render the stories as a semantic ordered list. Use an asymmetric paper-atlas composition rather than identical horizontal rows. Each story separates purpose from "My contribution", and provides explicit expandable notes, filters, and persistent save controls. Preserve keyboard access, accessible expanded/saved states, and filter behavior.
>
> **Example 4 — reflow after filtering without losing story identity**
> Input: "When a category is selected, remaining project cards should settle into their new positions without an abrupt jump."
> Output pattern: preserve the original filtering rules and project data; animate only the cards' opacity and positional reflow with a restrained FLIP-style transition. Do not reorder the source data or make cards move independently of the selected filter.
>
> Apply these demonstrations to the original V0 Home and Projects DOM and rendering logic. A successful result must change both structure and interaction: the Home includes the chapter rail and selectable image thumbnails; Projects uses the story index, an ordered list of individual stories, explicit contribution blocks, and an asymmetric paper layout. Keep all existing V0 sections, facts, project details, and features. Do not modify Task 2 rubric, evaluator prompt, AI-Judge output, or V0. Return complete changed source files and a concise implementation/change report. The supplied local E2E suite is for verification; do not weaken assertions to make a version pass.

Technique record

This is an in-context few-shot prompt: four input-to-output demonstrations are placed directly in the generation context and establish component transformation patterns. The revision responds to an observed weakness in the first V1 output. The examples specify structure and synchronized interaction state, not just color treatment. E2E checks `E2E-02b`, `E2E-03b`, `E2E-04c`, and `E2E-04d` test the corresponding Home/project changes. This is a prompt and generated implementation record; it does not claim that the local course-note retriever itself is an LLM.

---

icl prompt revision3

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision3.md`

V1 Few-Shot Prompt — Visual Refinement 3

Trigger: User screenshot review found that the Home carousel controls collided visually with the chapter navigation and the page composition felt unattractive. The user asked for refinement beyond that fix, informed by the four previously selected GitHub examples.

Full refinement instruction

> Refine the existing V1-R2 design by editing the V1 files only; do not touch frozen V0 and do not use a V2 source. Use the four previously reviewed GitHub examples only as structural references. Keep all of Aria's personal content, page sections, project facts, sky background, colors, and existing interactions. Never copy reference-site text, imagery, identity, or personal information.
>
> Use these reference-derived few-shot patterns:
>
> **Example 1 — restrained academic index**
> Input: "The page needs navigation and scanning aids, but repeated standalone cards compete with the main content."
> Output: keep a clear editorial hierarchy; use a slim index with restrained dividers and typography rather than turning every link into a tile. Allow generous space around the main heading and featured work.
>
> **Example 2 — artwork remains the visual focus**
> Input: "A creative portfolio uses thumbnails, filters, and previews to make its media explorable."
> Output: keep image selection as a direct, visible interaction, but keep controls compact and visually subordinate to the selected artwork. Use only the owner's existing relevant assets; do not fill empty areas with unrelated stock photos.
>
> **Example 3 — purposeful microinteraction**
> Input: "Interaction should reward exploration without making a calm personal website feel like a game."
> Output: use one small hover/focus response that clarifies the clickable target or updates a meaningful selected state. Avoid moving unrelated elements, collision, exaggerated easing, or decorative interactions with no feedback value.
>
> Apply the patterns to the V1-R2 layout: place the chapter rail in normal document flow directly under the Home hero and outside the carousel figure; make the three destinations one light glass strip with separators rather than three separate cards. Restyle the four-item project story index as a slim editorial index with separators, not four extra tiles; keep the actual project stories as the primary, asymmetrically arranged paper cards. Ensure the hero thumbnail selector and story index do not overlap at 1512×982, retain readable labels, and preserve responsive flow. Capture screenshots of the full Home view, chapter rail, and Projects section, then run the unchanged behavior checks. Do not weaken tests to accommodate the visual redesign.

icl prompt revision4

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision4.md`

V1 Few-Shot Prompt — Life Media Wall and Journal Marginalia (Iteration 4)

Record type: execution prompt prepared for this user-approved iteration. It records the operational instruction synthesized from the confirmed scope; it is not presented as a verbatim historical user message.

Full instruction

> Continue the existing V1 branch, whose only source baseline is the frozen `coursework/website/V0/`. Do not read V2 as an implementation source, do not modify V0, and retain all of Aria's existing personal copy, projects, education details, images, videos, navigation, and interactions. The user has approved adding two visual refinements: (1) original, low-saturation hand-drawn SVG margin annotations with a quiet travel-journal character; (2) an asymmetrical scrapbook-style image wall for the existing Life photos and videos.
>
> Use the few-shot patterns below only for component structure and interaction conventions. Do not copy reference text, authors' identities, imagery, or personal facts. Do not add unrelated stock imagery. Keep the watercolor-sky background, calm palette, accessible text contrast, and dependency-free static HTML/CSS/JavaScript implementation.
>
> In Life, arrange the seven existing photos and two existing short videos as a loose but legible staggered composition with varied dimensions, soft paper edges, and restrained rotation. Preserve the existing captions and native media dialog. Make every media tile keyboard reachable, open the same preview with Enter/Space as with pointer activation, close with Escape, and retain visible focus feedback. Add small original inline SVG line drawings and short marginalia as decorative content hidden from assistive technology. Keep transitions gentle and honor reduced-motion preferences. Do not change the common evaluator rubric or AI-Judge prompt.
>
> After implementation, run the shared desktop E2E suite against V1. Capture a full Life-section screenshot and record the raw results. Verify V0 against its frozen checksum manifest. Save this iteration's prompt, few-shot examples, source output, change record, and evidence references in the coursework archive.

Iteration lineage

- Base: frozen V0, copied earlier into the dedicated V1 folder.
- Input examples: `few_shot_examples_revision4.md`.
- Implemented source output: `llm_output/iteration_4/`.

- Shared evaluation protocol: `evaluation/scripts/test_portfolio_e2e.cjs`; Task 2 scoring rubric and AI-Judge prompt remain unchanged.

---

icl prompt revision5

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision5.md`

V1 Few-Shot Prompt — Component Architecture Rebuild (Iteration 5)

Record type: faithful operational prompt reconstructed from the user's explicit V1 constraints and approved scope in this conversation. This is not represented as a verbatim system prompt from a separate model session.

Full instruction

> Generate the next V1 iteration from the frozen `coursework/website/V0/` baseline only. Do not derive V1 from V2, do not edit V0, and do not change Task 2 rubric or AI-Judge prompt. The previous V1 is insufficient because its visible and structural differences from V0 were too small. This iteration must make a substantial, observable change to HTML component shells, page layouts, and interactions using the approved few-shot component patterns.
>
> **Content contract:** retain 100% of V0's business copy verbatim: profile, education, awards, courses, skills, project descriptions, guestbook labels/messages, and all relevant captions. Do not add, delete, paraphrase, or semantically rewrite personal copy. Preserve source facts and use the existing V0 content/data files. Move that copy into the new component templates rather than retaining V0's flat paragraph/div layout.
>
> **Few-shot method:** use the already reviewed GitHub examples only to abstract reusable component architecture, layout, and interaction conventions. Do not copy reference personal information, text, identity, or assets. Explain the mapping from each example to the new component pattern. The examples should affect page structure and component behavior, not merely colors.
>
> **Design language:** preserve the watercolor sky-and-mountain background and quiet diary/illustration-portfolio character. Use low-saturation cream, pale blue, and off-white; paper edges or light frosted glass; generous whitespace; readable dark blue-gray copy; and very restrained hover transitions. Keep the presentation calm and avoid dashboard grids, hard gradients, saturated accents, and heavy shadows.
>
> Rebuild the Home hero as an editorial journal cover with the existing hero illustration carousel and a chapter rail linking to the existing long-scroll sections. Rebuild Highlights as an academic folio with a visual education journey, skill ledger, and knowledge-workbench shell. Rebuild Projects as a numbered story index plus semantic, asymmetrical project articles while preserving filters, saved state, details, and all original project copy. Rebuild Life as a letter

> beside an image atlas/scrapbook wall, keeping the existing seven photos, two videos, captions, and accessible preview behavior. Rebuild Guestbook as an open postcard/writing desk with the original form and guestbook content. Decorative details must stay secondary to reading.
>
> Keep the site as dependency-free static HTML, CSS, and JavaScript. Maintain keyboard access, visible focus, reduced-motion support, and responsive layout. Do not add unrelated stock photos. Do not autoplay music; keep playback under user control.
>
> Before finalizing, compare the rendered V1 copy against V0, verify all project-story fields and carousel alt/caption text, and assert the V1-specific component shells. Run the shared desktop E2E protocol at 1512×982 against local V1 and unchanged local V0. Save raw results and screenshots. Record any failed attempt and correction. Verify the frozen V0 checksum. Preserve this prompt, few-shot patterns, and the complete V1 source output in the coursework archive.

Few-shot inputs

See `few_shot_examples_revision5.md` for the component-pattern abstractions and their transfer to V1. The reviewed repositories supply structure/interaction patterns only; none of their personal copy or assets are used.

Iteration lineage

- Base: frozen V0, independently copied into `coursework/website/V1/`.
- Generated implementation snapshot: `llm_output/iteration_5/`.
- Evaluation: shared script `coursework/evaluation/scripts/test_portfolio_e2e.cjs` plus the V0/V1 copy-and-structure contract script.

---

icl prompt revision6

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision6.md`

V1 Usability Correction Prompt — Click Guidance and Isolated Disclosures (Iteration 6)

Record type: this iteration is based on the user's exact screenshot feedback below. It is a usability correction within the existing V1 Few-Shot branch, not a claim of a new LLM technique.

User feedback (verbatim)

> 这里没有文字提示，我怎么知道会跳转到哪里？不要这个图片下面的东西挡到图片了？这个以及研究生的经历为什么不能点开？这个又没有指引？我点开一个project为什么另一个也会跟着一起打开？

Implementation instruction derived from the feedback

> Modify V1 only. Keep frozen V0 untouched and preserve all V0 personal and business copy. Make chapter-navigation destinations explicit using the existing section names, and label the project index with the existing V0 project titles so visitors can predict where each link goes. Move the image-carousel thumbnail controls fully below the main image without covering it. Make both education cards and their photos activate the corresponding existing disclosure; show a concise visible action hint, and change the hint when details are open. Keep the disclosure state local to each education card. Ensure that expanding one project changes only that project's state and does not make a neighboring card appear expanded because of grid stretching. Preserve the calm paper/watercolor styling, keyboard access, and gentle transitions. Add automated checks for each reported issue, capture clean screenshots of the controls, both education details, and a single opened project, and verify V0's manifest.

Applied corrections

- Chapter rail names the existing destinations: Highlights, Projects, Life.
- Project index labels come from each existing project's exact V0 title.
- Hero thumbnails now sit below the image with a clear gap.
- Education title buttons display “Click to view details” / “Click to hide details”; clicking a school photo also opens that school's details.
- Education open state now targets `.education-leaf`, fixing the stale `.school-card.open` CSS selector.
- Project grid cards use intrinsic heights and top alignment so a neighboring card stays compact when one detail block expands.

Verification

See `evaluation/scripts/test_portfolio_e2e.cjs` and the R7 evidence set. The copy contract ignores only the newly added action hint, while continuing to compare the original school content exactly.

---

icl prompt revision7

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision7.md`

Generation Prompt — Iteration 7 (App Shell)

Context handed to the model

```
You are rebuilding the page skeleton of a personal portfolio.

Frozen baseline: coursework/website/V0/  (must not be modified)
Current V1: coursework/website/V1/      (may be rebuilt)
Constraints:
  - Every string, project fact, education record, award, course name,
    media caption and image already exists in V0. Do not invent copy.
  - The course-note RAG module keeps its retrieval behaviour.
  - Keep every id and class the existing scripts query, or the
    interactions die: [data-nav], .nav-indicator, #sound-toggle,
    #art-prev/#art-next/#art-dots, .school-head/.school-detail,
    #project-list, .row-toggle, .filter, #saved-filter,
    #media-gallery, #media-dialog, #guestbook-form, #rag-form.
  - Palette: the existing watercolour sky palette only.
  - One variable at a time: this iteration changes the shell, not the content.

Problem: the R7 screenshot still reads as the same site as V0 — a single
column of paper sections under a pill navigation. Changing components
inside that frame does not register as change. Rebuild the frame.

Provide three skeleton candidates as clickable single-page drafts rendered
with the real copy, then implement the chosen one.
```

Candidates returned

Draft	Skeleton	Structural rule
A	Bentos mosaic	one 12-column grid, no section scroll, every cell is one fact or one control
B	Profile rail + content window	identity and navigation persist on the left; the right side is a bounded window that owns its scroll
C	Editorial split spread	each section is a full-bleed two-panel spread with an oversized headline and a meta row

Chosen by the author: **B**.

Implementation rules derived for the chosen skeleton

1. **Sections stay in normal flow.** Hiding them behind tabs would break every in-page anchor, the IntersectionObserver reveal, and the shared E2E protocol. The window scrolls; the sections do not move in the DOM.
2. **The rail reuses the existing navigation element.** `.nav-bar` / `.pill-nav` / `.nav-indicator` are restyled vertical rather than replaced, so `setActive()` and `aria-current` keep working. The indicator needs one axis switch.
3. **The active section is measured against the window, not the page.** The old `updateSectionNav()` compared rects to `innerHeight` and forced `guestbook` whenever the page bottom was reached — with no page scroll that fires on load.
4. **The chapter rail moves inside `#home`.** It was a sibling of the section, so between two 1-screen panels it fell into the seam and was never visible.

5. **Narrow screens keep the old reading model.** Below 981 px the rail collapses to a horizontal bar and the page scrolls normally.
6. **Two stories per row.** Four project cards across a 836 px pane produced rotated cards overlapping each other.

Recorded deviations

- The first attempt at "one screen per section" left 300 px of dead space in the rail and pushed the chapter rail below the fold; the rail is now grouped with two auto margins instead of one, and the rail moved into the section.
- `.section-number` (`01 / HIGHLIGHTS`) duplicates the window title bar's `02 / 05`; it is hidden in shell mode, not deleted.
- The mobile cover originally showed the photograph before the headline; the order is now text-first in the shell.

---

icl prompt revision8

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision8.md`

Generation Prompt — Iteration 9 (Full-bleed Editorial)

This iteration was triggered by review feedback, not by a fresh design idea. The author's verdict on iteration 7 was a screenshot of the framed window plus one line: "v1好丑啊，越优化越丑…不想要有边框版，很局促", followed by "也不喜欢旁边放的信息". That is the constraint set the prompt had to encode: no frame, no side column, and not more of the same.

Context handed to the model

```
You are rebuilding the page skeleton of a personal portfolio that is already
four iterations deep. The last iteration (app shell) was rejected on sight.

Frozen baseline: coursework/website/V0/  (must not be modified)
Current V1: coursework/website/V1/      (may be rebuilt)

Rejection feedback, verbatim:
  "v1好丑啊，越优化越丑怎么，不想要有边框版，很局促"
  "而且也不喜欢旁边放的信息"

Hard constraints:
  - No page-level frame and no bordered container around the content. A window,
    a card wall, or a rounded panel that encloses the layout is out.
  - No persistent side column carrying identity or facts.
  - Every string, project fact, education record, award, course name, media
    caption and image already exists in V0. Do not invent or rewrite copy.
  - Keep every id and class the existing scripts query, or the interactions die:
    [data-nav], .nav-indicator, #sound-toggle, #art-prev/#art-next/#art-dots,
    .school-head/.school-detail, #project-list, .row-toggle, .filter,
    #saved-filter, #media-gallery, #media-dialog, #guestbook-form, #rag-form.
  - The course-note RAG module keeps its retrieval behaviour.
  - Change the skeleton, not the content. This is a layout iteration.

Problem to solve: the site must read as structurally different from V0 at first
```

```
glance, without adding chrome. Structure has to come from scale, rules, whitespace
and bleed rather than from borders, radii and shadows.

Ask for three directions first, describe each in one line, let the author pick,
then implement only the chosen one.
```

Directions offered

	Direction	One-line description
A	Full-bleed editorial (chosen)	Content runs the whole viewport width on a hairline grid; oversized display type and edge-to-edge images carry the structure; no card anywhere.
B	Horizontal story line	Scroll-driven horizontal pages, one large image and one sentence per screen — closest to a photo book, most work in mobile and accessibility.
C	Subtraction only	Keep the existing structure, remove the window frame and rail, put navigation back on top — smallest change.

The author picked **A**.

Rules extracted from the direction

1. **One hairline grid.** A single rule weight separates every block. No block owns a border of its own.
2. **Type does the work a card used to do.** Section headings and project titles are display-sized; the old 30 - 40px card headings were not big enough to carry a layout without a container.
3. **Bleed on one axis only.** The cover photograph and the Life wall run to the viewport edge on the right; the left gutter stays constant. Bleeding on both axes turned the page into a slideshow in the draft.
4. **The top bar is a line, not a capsule.** Wordmark left, five labels right, music toggle last, underline for the active section. No pill, no blur box.
5. **The photograph is an opening image, not a background.** The fixed sky backdrop was retired; the sky survives as a masked band behind the cover only.
6. **Sections stay in normal flow.** No tabs, no clipping, no scroll hijacking — every anchor, filter, favourite, disclosure and gallery keeps working.

Deviations recorded during implementation

- The first build kept the fixed sky photograph as the page background. With the paper cards gone, body copy sat on mountains and was unreadable. Confining the sky to the cover and giving the body a flat paper tone was not in the prompt; it was forced by the rendering and is logged in `change_log.md`.
- The Life wall was tried as a masonry composition and reverted to a four-column grid: uneven tile heights read as a broken section rather than a composed one at this content volume.
- The project row number was first placed at the far right of the left column and drifted into the gutter between columns; it now opens the row as a folio.

Verification of this prompt

- `test_portfolio_e2e.cjs`: 35/35 PASS, 0 console errors, 0 page errors.
- `check_v1_v0_content.cjs`: 48 copy groups, 0 differences, 0 structure errors — the prompt's copy constraint held.
- Breakpoints 1512 / 1280 / 1024 / 820 / 414 px: no horizontal overflow.

---

icl prompt revision9

Archive source: `prompt_appendix/task3_v1/icl_prompt_revision9.md`

Generation Prompt — Iteration 10 (Life gallery, compact mosaic)

Second review-driven iteration in a row. The editorial skeleton from iteration 9 was accepted, but the author pointed at the Life image wall: "你这个图片太平铺、太大了，你再多提取一些 样本学习一下吧". Two faults in one line — the wall is flat and uniform, and the tiles are too large — plus an explicit instruction *how* to fix it: study more samples first, do not guess.

Context handed to the model

```
You are revising the "Life" section of a personal portfolio that is already on its
editorial skeleton. The skeleton was accepted; the photo wall inside it was not.

Frozen baseline: coursework/website/V0/  (must not be modified)
Current V1: coursework/website/V1/      (the editorial layer lives in v1.css)

Review feedback, verbatim:
  "你这个图片太平铺、太大了，你再多提取一些样本学习一下吧"

Hard constraints:
  - Do not bring back borders, cards, tape, rotation or drop shadows on the tiles.
    Iteration 9 removed them on purpose.
  - Nine moments exist: seven photographs and two short films. None may be hidden
    at desktop width.
  - Do not rewrite any caption. The captions are compared byte-for-byte against V0.
  - Keep #media-gallery > .media-item as <button> elements; the dialog, keyboard
    activation and the aria-labels are already wired through them.
  - The page grid is full-bleed with a hairline rhythm. The wall must end flush on
    the right like every other block on the page.

Method, in this order:
  1. Measure the current wall: tile size, wall height, item count, what the last
     row does. State the faults as numbers before proposing anything.
  2. Collect references. Read the layout taxonomies (grid / masonry / justified /
     contact sheet), and find at least one source that names the failure mode —
     a uniform grid forcing a portrait and a landscape into the same shape, and
     the last row enlarging a lone remaining tile.
  3. Render at least three compositions with the real photographs at 1512 px and
     look at them. Reject the ones that only shrink the problem.
  4. Implement the one that solves both faults. Keep the passes separable in CSS
     so the review history stays readable.
```

Constraints the composition had to satisfy

1. **No stranded tile.** The right edge of every band must be flush, so no tile can be stretched to fill a leftover slot.
2. **Hierarchy without size inflation.** The accent had to come from *shape*, not from making one tile enormous.
3. **Aspect honesty where it matters.** The one 2:3 portrait should be shown at something close to 2:3; the 3:2 landscapes should not be cropped to squares.
4. **Captions out of the image.** No gradient overlay.
5. **Graceful at 1512 / 1280 / 1024 / 820 / 414.** At two columns the ninth tile must not sit alone in the last row.

What the model produced

A 20-column grid, eight tiles at `span 4` (267 × 178 at 1512 px) in two bands of four, with the double-bass portrait pinned to `17 / span 4` across both bands at 267 × 400 — the uncropped 2:3 frame. Captions moved into the margin. Breakpoint at 1120 px drops to four per band; below 980 px it becomes two columns and the closing film takes the full width so the wall does not end on an orphan.

Deviations recorded

- The first attempt kept the old `grid-auto-rows: 27px` and the V0 `nth-child` row spans, which collapsed every row to 27 px and stacked the tiles on top of each other. Both rules were dead weight from the scrapbook era and were removed rather than fought with `!important`.
- A `!important` span reset from iteration 9 (`#media-gallery.media-wall > *`) had to be deleted for the same reason.
- Video tiles painted an empty beige rectangle while `preload="metadata"` was still fetching. Two poster frames were captured from the clips and shipped as assets instead of raising `preload` to `auto` on a 31 MB file.
- `E2E-08d` was restated. It had been written for the V0 scrapbook mosaic; see `gallery_pattern_sweep.md` §7 for the old and new assertion.

Verification

`test_portfolio_e2e.cjs` 35/35 PASS, 0 console errors, 0 page errors (`evidence/V1/V1-R10-e2e-results.json`); `check_v1_v0_content.cjs` 48 copy groups, 0 differences (`evidence/V1/V1-R10-content-parity.json`); wall measured at 1512 / 1280 / 1024 / 820 / 414 px with no horizontal overflow and a flush right edge at each.

5205 Introduction to AI

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_5/knowledge/5205-Introduction-to-AI.md`

5205 Introduction of AI

Lecture 1

Part 1

1.1 课程要求

理解人工智能的基本原理与算法 (BFS、神经网络、大语言模型等)。 Understand fundamental principles and algorithms of AI (BFS, neural networks, LLMs, ⋯).

分析与评估不同 AI 方法的优势与局限。 Analyze and evaluate the strengths and weaknesses of various AI methodologies.

将 AI 算法应用于工程、数据分析与优化场景。 Apply AI algorithms to engineering, data-analytic and optimisation scenarios.

设计与实现 AI 技术以解决实际问题 (对应期末项目)。 Design and implement AI techniques to solve practical problems (final project).

1.2 AI 概述：Industry 4.0 的核心引擎

阶段 Stage	时代 Era	核心驱动力 Core Driver	目标 Goal
Industry 1.0	18 世纪末	机械化 Mechanization	机器取代人力
Industry 2.0	19 世纪末	电气化 Electrification	大规模生产
Industry 3.0	20 世纪后半	信息化 Informatization	计算机与互联网
Industry 4.0	当下	智能化 Intellectualization	让日常生活更智能 (by AI)

Merriam-Webster 将 artificial 定义为"人为构造,常以自然事物为模型"。这暗示了 AI 的两条核心属性：由人创造、通过行为而非生物大脑来识别。 Merriam-Webster defines "artificial" as "humanly contrived, often on a natural model". This implies two core attributes: (1) human-made, (2) identified by behaviour, not biological substrate.

1.3 AI 的三种典型定义 Three Classic Definitions of AI

提出者 / 时期 Author / Era	定义 Definition
McCarthy & Minsky, 1950s	由程序或机器完成的任何任务,如果由人来完成需要运用智能。 Any task performed by a program or machine that, if a human carried out the same activity, would require the human to apply intelligence.
John McCarthy (现代定义 Modern definition)	AI 又称机器智能 (machine intelligence),是与人类和动物自然智能相对的、由机器展示的智能。 Artificial intelligence — sometimes called machine intelligence — is intelligence demonstrated by machines, in contrast to the natural intelligence displayed by humans and other animals.

提出者 / 时期 Author / Era	定义 Definition
Two emphasised points	(1) 由人创造 human-made;(2) 通过行为而非生物大脑来识别 identified by behaviour rather than biological brain.

1.4 AI 发展史与代表性里程碑

AI 的发展可划分为两大时代： "深度学习前时代"与"大模型时代"。

Year	Milestone	Notes
1958	Perceptron 感知机	最早的神经网络模型之一
1974	BP 神经网络	反向传播算法使多层网络可训练
1998	CNN 卷积神经网络	LeNet 等开启深度卷积时代
2012	Google Brain / AlexNet	深度学习在 ImageNet 大放异彩；大数据 + HPC + 大存储 + 先进算法
2018	BERT & GPT	进入预训练时代
2022	ChatGPT (GPT-3.5)	大模型时代到来 LLM era
2023	GPT-4	大模型全面突破
2024	Agent 时代	多模态智能体与自主工作流
2025 (Aug)	GPT-5	OpenAI 在 2025 年 8 月发布,作为 NLP 的里程碑

Three supporting pillars 三大支柱：大数据 Big Data + HPC 高性能计算 + 先进算法 Adv. Algorithms。 AI 的三个演化阶段：

阶段	能力	典型例子
ANI 弱人工智能 Artificial Narrow Intelligence	专注于特定任务 Focus on specific tasks	AlphaGo、语音识别、图像识别、机器翻译、AI 导师、自动驾驶等
AGI 通用人工智能 Artificial General Intelligence	具备人类水平的思考、计划、理解与复杂问题求解能力；能从经验中学习、自适应。	尚未实现 Not yet realised
ASI 超级人工智能 Artificial Super Intelligence	比最聪明的人类更聪明的 AI	理论上可发展出医学、纳米技术、自我复制机器人等所有可能技术

1.5 AI 的双重属性：工程与科学

AlphaFold 是 Google DeepMind 提出的蛋白质 3D 结构预测模型,解决了 50 年悬而未决的"蛋白质折叠问题",给定氨基酸序列预测其三维构象。 AlphaFold 作为 “AI 同时是科学也是工程”。

视角 Perspective	关注点 Focus	本课侧重
工程 Engineering	构建智能系统解决真实世界问题;研究支持性的机制、算法与表征。 Build intelligent systems to solve real-world problems; study mechanisms, algorithms, representations that make them possible.	主线 Mainstream
科学 Science	理解智能的本质 (人或机器);实现可评估的智能模型;探索算法/表征调整带来的影响。	适度覆盖 Covered, esp. learning & agents

视角 Perspective	关注点 Focus	本课侧重
	Understand the nature of intelligence (human or device); implement evaluable models; explore consequences of algorithm/representation choices.	

1.6 AI 基础:图灵测试与深度学习

1.6.1 图灵测试

1950 年 Alan Turing 提出图灵测试:若一台机器在对话中能让询问者无法区分其与人类,则可认为该机器具有智能。 In 1950 Alan Turing proposed the Turing test: if a machine's conversational responses are indistinguishable from a human's, the machine can be considered intelligent.

“图灵测试关注行为表现而非内部机制。” "The Turing test cares about behaviour, not internals."

1.6.2 深度学习的核心逻辑

A deep neural network is a hierarchy of layers that progressively transforms input data into more abstract representations (e.g., edge → nose → face). The output layer combines those features to make predictions.

深度神经网络由多层组成,每一层把输入数据转换为更抽象的表示 (例如：边缘 → 鼻子 → 整张脸)。输出层把这些特征组合起来完成预测。

1.6.3 规则驱动到数据驱动的范式转变

驱动力 Driver	含义
更好的模型 Better models	Rule-based → Statistical → Deep Learning；以更多参数拟合更复杂的数据。
更强的算力 Better compute	更强大的 RAM / CPU / GPU;分布式计算平台。
更多的数据 More data	海量、可分析、含信息的数据可用 → “大数据”。

"Big data is a collection of data sets so large and complex that it becomes difficult to process using on-hand database management tools or traditional data processing applications."

“大数据是指数据量大到传统数据库与处理工具难以处理的集合。”

1.7 大数据

1.7.1 数据来源

Data fueling model growth comes from everyday human activity — "Nearly every human action leaves a digital trace: medical, traffic, logistics, bioscience, weather, chemistry, education, system, finance, e-commerce, manufacturing, ⋯

1.7.2 4V 特性

维度 Dimension	含义	典型数据
Volume 体量	从 TB 到 ZB 的数据规模需要处理。From terabytes to exabytes to zetabytes.	2015: 8 billion TB;2020: 40 ZB;人均 ~5.2 TB

维度 Dimension	含义	典型数据
Velocity 速度	批式、实时、流式数据,从毫秒到秒级响应。Batch, real-time, streaming.	毫秒级响应的实时推荐系统
Variety 多样性	结构化 / 半结构化 / 非结构化:文本、图片、视频、音频等。	Structured, semi-structured, unstructured.
Veracity 真实性	数据的不一致性、不完整性、歧义性、欺骗、近似模型带来的不确定性。	Uncertainty from inconsistency, incompleteness, ambiguity, deception.

1.8 数据挖掘与机器学习

1.8.1 数据挖掘

Data mining discovers patterns and models that are valid, useful, unexpected and understandable. Examples: "milk + bread" often bought together (descriptive); tomorrow's weather (predictive).

数据挖掘旨在发现数据中“有效、有用、出乎意料、可解释”的模式与模型。

典型例子：从超市数据中发现“牛奶 + 面包”常被一起购买 (描述型)；从历史数据预测明日天气 (预测型)。

任务类别	目标	例子
描述型 Descriptive Methods	发现人类可解读的、描述数据的模式	购物篮分析 Milk and Bread
预测型 Predictive Methods	用部分变量预测其他变量的未知值或未来值	天气预测 weather prediction

1.8.2 经典机器学习 vs 深度学习

阶段	特征 Feature	模型 Model
经典机器学习 Classical ML	手动设计特征 (Handcrafted Feature Extraction → Feature Selection)	统计模型 Statistical models
深度学习 Deep Learning	多层感知机自动学习特征 (Automatic Feature Learning)	神经网络 / 多层感知机 Multi-layer Perceptron

Machine learning lets machines learn new things from data. Deep learning, with more complex models, fits large-scale data better.

机器学习 (Machine Learning) 让机器能从数据中学习新东西；深度学习 (Deep Learning) 通过更复杂的模型拟合大规模数据,更适合大数据场景。

1.9 机器学习三大范式

范式 Paradigm	数据特征	任务	典型例子
监督学习 Supervised Learning	有标签 Labeled Data	预测标签	价格预测
无监督学习 Unsupervised Learning	无标签 Unlabeled Data	聚类 / 主题发现	客户分群、主题挖掘
强化学习 Reinforcement Learning	基于奖励 Rewards	寻找好的行为	训练小狗 / 游戏 AI

1.9.1 监督学习：回归与分类

Supervised learning trains on labeled data to predict labels on new samples. Predicting a real-valued target = regression; predicting a discrete category = classification.

监督学习的核心：用有标签数据训练模型，使其能对新样本做出预测。预测目标若是连续值，称为回归 (Regression)；若是离散类别，称为分类 (Classification)。

回归例子 Example：用面积、楼层、位置预测房价 (连续值)。

Predicting house prices from area, floor, location (continuous).

分类例子 Example：用肿瘤大小、患者年龄判断良性/恶性 (离散)。

Predicting benign/malignant from tumour size and age (discrete).

As the number of features grows from 1 to few to many, models move from simple linear fits to high-dimensional complex models — deep learning is the extreme of this trend.

当特征从 1 个到少量再到大量时，模型从简单线性拟合逐步走向高维复杂模型，深度学习正是这种趋势的极致。

1.9.2 无监督学习

In unsupervised learning, the dataset has no labels x(1), ⋯, x(n); The goal is to discover interesting structures or patterns — clustering, topic discovery, dimensionality reduction, etc.

无监督学习的数据集没有标签 x(1), ⋯, x(n)，目标是发现数据中“有趣的”结构或模式，如聚类、主题、降维等。

1.9.3 强化学习

In reinforcement learning, the agent interacts with the environment at discrete steps t = 1, 2, 3, ⋯, T. At each step the agent:

强化学习中，智能体 (Agent) 在离散时间步 t = 1, 2, 3, ⋯, T 与环境 (Environment) 交互。在每一步，智能体: 观察状态 (环境) Observes the state (environment)；产生动作 Produces an action;；获得奖励 Gets the resulting reward；进入下一状态 And produces the next state。

Through a "try → collect feedback → improve policy" loop, the agent progressively learns better behaviour. 通过“尝试策略并收集反馈 → 根据反馈改进策略”的循环，智能体逐步学会更好的行为。

1.10 伦理、法律与隐私

The rise of AI raises a number of unresolved ethical and legal questions:

数据隐私保护 How do we protect data and the use of data in AI? How do we protect data and the use of data in AI?

人类对 AI 的控制 Should the law regulate the level of human control over AI?AI 是否应在某阶段被视为法律主体? Should the law regulate the level of human control over AI? At what stage, if any, should AI be considered a legal person?

AI 决策的责任边界 Should machines be allowed to make decisions that affect whether people are injured (e.g., self-driving cars making life-and-death choices)? Should machines be allowed to make decisions that may injure people (e.g., self-driving cars)?

自主武器 Should we allow AI in autonomous weapons that select and engage targets without human intervention? Should AI be allowed in autonomous weapons that select and engage targets without human intervention?

Lecture 2

Overview

周 Week	主题 Topic	核心内容 Key Content
Week 1	An Introduction	AI 定义 / 三大学习范式 / ANI - AGI - ASI
Week 2	Searching Techniques	无信息搜索（BFS / DFS）→ 启发式搜索（爬山法 / 最佳优先 / A*）→ 元启发式（SA / GA） 先看什么信息都没有时怎么找解（BFS / DFS），再给搜索装上启发式（爬山法 / 最佳优先 / A*），最后把找一个解升级成找最优解，于是引出两大元启发式：模拟退火（Simulated Annealing, SA）与遗传算法（Genetic Algorithm, GA）
Week 3	Evolutionary Computation	EA 总览 / PSO / DE / MA / MOEA / ETO / NAS
Week 4	Artificial Neural Network	神经元模型 / 感知机 / MLP / 反向传播
Week 5	Deep Learning & Applications	深度网络、卷积、循环与注意力

Part 1 Searching

1.1 Definition and Five Elements

Searching is the process of looking for a solution to a problem through a set of states.

搜索是在一组状态中寻找问题解的过程。

要素	原文	理解
当前状态	Current state: the step of where we are	我们现在走到哪一步
目标状态	Goal state: the step where the solution is found	解所在的那一步
动作	Actions: move one state to another	把一个状态搬到另一个状态的操作
代价	Cost: how many steps needed to find the solution.	找到解需要走多少步
解	Solution: a path from the current state to the goal state.	从当前状态到目标状态的一条路径

解是一条路径（path），不是单个状态。这条约定决定了后面所有的搜索算法都在做同一件事，在状态图上从起点长出一棵树，直到某个叶子正好是目标。所以找到目标状态和给出到达目标的路径是两件事，后者需要回溯父节点。

1.2 搜索的重要性

An agent can find a sequence of actions that achieves its goals when no single action is done.

当没有任何单个动作能直接达标时，智能体需要找出一串动作。

Almost every AI problem depends on a search procedure to find the solution.

几乎每一个 AI 问题都依赖一个搜索过程来求解。

1. Check the current state (reach the goal?); 检查当前状态是否已达成目标;
2. Execute allowable actions to move to the next state; 执行允许的动作，移动到下一个状态;
3. Check if the new state is the goal state. 检查新状态是不是目标状态：No 时回到 Search，Yes 时 End。

循环有一个终止条件：课件在流程图旁写得很清楚 —— The process is repeated until a solution is found or the state space is exhausted.（这个过程一直重复，直到找到解，或者状态空间被穷尽）。

算法	前提	特点	说明
Linear Search 线性搜索	数据无序也可用	逐个比较，最坏情况看完全部	无信息搜索（只能一格一格试，等价于 BFS 的机械展开）
Binary Search 二分搜索	数据必须有序	每次砍掉一半，快得多	有信息搜索（有序这个先验就是启发信息，它让每一步都能排除一半）

1.3 图与树 Graph and Tree Representation

节点 Nodes：the states in a searching process 搜索过程中的状态;

边 Edges：transitions between states (actions) 状态之间的转移（也就是动作）。

与或图（And-Or Graph）：多条路都能成功

节点类型	原文	含义
And 节点	a node with all edges emanating from it connected by a curved line	发出的所有边用一条弧线连起来，表示必须全部满足
Or 节点	a node has no line connecting its edges to indicate that any emanating path can be a solution	边之间没有连线，表示任选一条路径即可

旅行商问题（TSP）：访问完所有城市是 And（每个城市都得去），先访问哪一个是 Or（顺序可以任选），这正是组合爆炸的来源。

1.4 搜索方法 Searching Approaches

Uninformed (Blind) Search：Breadth-first Search / Depth-first Search

Informed (Heuristic) Search：Hill Climbing / Best-first Search / A* Algorithm Search / Simulated Annealing / Genetic Algorithm

大类	方法	用不用启发信息	是否保证最优
无信息（盲）搜索	BFS 广度优先	不用	在边权相等时保证最优
无信息（盲）搜索	DFS 深度优先	不用	不保证（p.20 明确说明）
有信息（启发式）	Hill Climbing 爬山法	用 h	不保证（会困在局部最优）
有信息（启发式）	Best-first / Greedy	只用 h	不保证（贪心，p.28 示例）

大类	方法	用不用启发信息	是否保证最优
有信息（启发式）	A*	g 与 h 都用	h 可采纳 / 一致时保证最优
元启发式（随机）	Simulated Annealing	用概率接受更差解	不保证
元启发式（随机）	Genetic Algorithm	用种群与适应度	不保证

A simplified road map of part of Romania（罗马尼亚部分路网简化图）,这张地图就是本讲后面所有实例的舞台：贪心搜索、 A* 全过程，用的都是它（起点 Arad、终点 Bucharest）。

Part 2 Generate & Test

2.1 生成—测试（Generate & Test）与井字棋

最原始的搜索策略 Generate & Test:

Step 1. Generate a possible state 先生成一个可能的状态;

Step 2. Test to see if it is a goal state (found a solution) 再测试它是不是目标状态。

用井字棋（Tic-Tac-Toe）来演示：从初始状态出发，按放 X（cross）或 O（zero）的规则不断生成新状态。

核心提问：How many states should be explored?

两个数：9! = 362,880 数的是对局数（下棋的先后顺序不同，是不同的对局）；3^9 = 19,683 数的是棋盘局面数（只看到棋盘长什么样，不管怎么走到这里的）

2.2 Open 与 Closed

Move downwards, level by level, until goal is reached.（一层一层向下推进，直到抵达目标）。

Open: a list of states that have been generated but whose children have not been examined.

已经生成、但孩子还没被考察过的状态。决定下个展开谁，BFS 用队列（FIFO）实现它，DFS 用栈（LIFO）实现它。

Closed: a list of states already visited.

已经访问过的状态。防止重复展开与死循环，遇到图里的回路时，靠它把回头路挡住。

本讲开头说盲搜索不用任何额外信息，但记忆不算信息，它只是记录自己走过哪里。所以BFS非常吃内存，BFS 的 Open 表要同时存下整整一层的节点；而 DFS 的 Open/递归栈只存当前这一条路径。

Part 3 广度优先搜索

3.1 无信息搜索 Uninformed (Blind) Search

3.1.1 Definition

The only information available is the ability to distinguish goal state from non-goal state. An uninformed (blind) search algorithm uses no information other than the initial state, the search operator and a test for a solution.

唯一能用的信息就是分辨一个状态是不是目标；盲搜索除了初始状态、搜索算子、解的判定以外，不使用其他信息。

The different types of blind searches are characterized by the order in which they expand the nodes. The order can have a dramatic effect on how well the search performs (randomly).

各种盲搜索的唯一区别就是展开节点的顺序，而这个顺序（会对搜索的表现产生巨大的影响）。

换句话说：BFS 与 DFS 的代码几乎相同，差别只在用队列还是用栈。

3.2 广度优先搜索的定义

Expand the tree layer by layer, progressing in depth. Expand the root node first, expand all nodes at level 1 before expanding level 2 OR expand all nodes at level j before expanding nodes at level j + 1.

一层一层地展开，先展开根节点；先展开完第 1 层的全部节点，再去展开第 2 层，或者说，先展开完第 j 层的全部节点，再展开第 j+1 层的节点。

BFS 在边权相等时保证最短：因为它是按层推进的，第 j 层的所有节点都比第 j+1 层的离起点更近。所以第一次遇到目标时，走的一定是步数最少的路径。

3.2.1 广度优先算法

课件把 BFS 画成一张带 Expand node（展开节点）的流程框图。它的文字层只有 Expand node 与 Breadth-first Algorithm 两个标注，但流程本身与 2.5 / 2.7 的两个实例完全一致，可以还原成下面的伪代码：

> current = open.pop(0) ← 从队列头部取出（FIFO）
>
> if child not in open and child not in closed: open.append(child) ← 追加到队尾

📌 「第几步」怎么数：课件 p.13 与 p.19 都写了 State space at the sixth step（第六步时的状态空间），说明课件把「展开一个节点」算作一步，并把每一步的 Open / Closed 内容都列出来。本笔记照此约定。

3.2.2 Example 1：从 A 到 U

课件给的搜索树以 A 为初始状态、U 为目标状态，并逐次标出展开的节点：

3.2.3 Example 2：八数码

Why only one expansion here? 为什么这里只扩展出一个？。

同一个棋盘局面可以由不同动作到达，因此生成时必须先做去重，否则搜索树会膨胀成一棵指数级的树。其余方向的邻居要么不合法、要么已经在 Open 或Closed 里（重复局面）。

3.2.4 Example 3：从 S 走到 G

这里的广度优先只有两步，而后面的深度优先结果 S-A-C-G（3 步），所以证明了后续的 Depth-first search does not find the shortest path（深度优先一般不找最短路径）。

第 3 步 A 的孩子 C 被追加到队尾，队里 B 还在前面，所以下一步展开的是 B 而不是 C；

第 4 步 B 的孩子里 C 已经在 open 里，于是不重复追加，只追加 G；

第 5 步 C 的两个邻居 A 与 G 都已在表里，于是open 只剩 G。

如果把②③的判重去掉，C 会被反复加入，搜索可能永远不结束。

Part 4 深度优先搜索与两者对比

4.1 深度优先搜索的定义

Expand the tree as deep as possible, returning to upper levels when needed. Select a child from left to right. · Repeatedly go to the left child, as long as possible. · Return to left-over alternatives (higher up) only when needed.

尽量往深里展开，需要时才退回上层；从左到右选一个孩子，只要可能就一直往左孩子走；只有在需要时，才回到（更上层的）那些还没走的分支。

4.2 深度优先算法

与 BFS 一样，课件给的是一张带 Expand node（展开节点）的流程框图，文字层只留下 Depth-first Algorithm 这个标题。按 p.20 的实例还原如下：

> current = open.pop() ← 从栈顶取出（LIFO）
>
> for child in reversed(current.children): ← 倒序压栈，保证从左到右访问

4.3 Example 1：从 A 到 U

同一棵搜索树（初始 A、目标 U），但展开次序完全不同：

对比项	BFS	DFS
初始 / 目标	Initial state: A / Goal state: U	Initial state: A / Goal state: U
展开序列	Expand A → Expand B → Expand C → Expand D → Expand E	Expand A → Expand B → Expand E → Expand K → Expand S → Expand L → Expand T → Expand F → Expand M → Expand C ……
形状	横向铺开，一层层推进	纵向钻下去，一路到底才回头

4.4 Example 2：从 S 走到 G

结论：Solution path: S-A-C-G

In the route example, by breadth-first search, the path is S-B-G; by depth-first search, the path is S-A-C-G. 深度优先一般找不到最短路径。同一个例子里，广度优先给的是 S-B-G，深度优先给的是 S-A-C-G。

4.5 深度优先 vs. 广度优先

They both find a solution if one exists (and if the state isn't too big).

只要解存在（且状态空间不太大），两者都能找到解。

对比维度	深度优先 DFS	广度优先 BFS
主要风险	跑进没有解的极深分支里空耗时间	内存占用极大

对比维度	深度优先 DFS	广度优先 BFS
课件原文	If the search space contains very deep branches without solution, then depth-first may waste much time in them.	Is very demanding on memory! Must remember the unexpanded nodes at a level, which could become very large as you go deeper into the tree/graph
要记住什么	depth-first remembers just the path 只记当前这一条路径	要记住整整一层的未展开节点，越深越大
最短路径	不保证	边权相等时保证
何时该用	解深、内存紧、只要求「有解」	解浅、要求最短、内存够

空间复杂度：设分支因子为 b、最大深度为 m。DFS 的空间是 O(b · m)（深度每加一层，每层只要记住还剩几个没走）；BFS 的空间是 O(b^d)（d 是解所在深度，因为要把整层都存下来）。在棋盘、地图这类分支因子很大的问题里，b^d 会瞬间爆炸，太过依赖记忆。

Part 5 启发式的有信息搜索： Hill Climbing, Best-first and A*

5.1 启发式搜索

5.1.1 使用场景

AI problem solver employs heuristics in two situations:

AI 求解器会在两种情形下动用启发式。

1. A problem may not have an exact solution because of ambiguities in problem statements or available data. Example: medical diagnosis. 问题可能根本没有精确解，因为题面或数据本身就有歧义，例如医学诊断；
2. A problem may have an exact solution, but the computational cost of finding it may be prohibitive. 问题有精确解，但找它的计算代价高到无法承受。

5.1.2 优势与不足

1. Uninformed (blind) searches are normally very inefficient. Information about the problem can sometimes be used to guide the search more efficiently. 盲搜索通常效率很低；问题自身的信息有时可以用来让搜索更高效。
2. Informed search explores the node that is most "likely" to be the nearest to a goal state. 有信息搜索优先展开那个「最可能」离目标最近的节点。
3. Informed search works successfully in many cases, but its success is not guaranteed. 有信息搜索在很多情况下有效，但成功没有保证。比如，爬山法会 stuck at local optima（卡在局部最优），贪心搜索遇到U 形障碍最优都保不住。

5.2 爬山法（Hill Climbing）

Consider an eager but blind mountain climber: go uphill along the steepest possible path until you can go no further up.

想象一个急于登顶却看不见全山的登山者，沿着最陡的坡一直往上爬，直到再也爬不上去为止。

把这个比喻写成算法，就是四条性质：

1. Hill-climbing expands the current state and evaluates its children. 展开当前状态，评估它的孩子；
2. The best child is selected for further expansion; Neither its siblings nor its parents are retained. 选最好的那个孩子继续展开；它的兄弟节点和父节点都不保留；
3. Because it keeps no history, the algorithm cannot recover from failures of its strategy. 因为它不保留历史，所以一旦策略失败就没法恢复。爬山法与 BFS / DFS 的根本分界。BFS 和 DFS 靠 Open / Closed 记住所有走过的路，所以不会迷路（要么找到解，要么穷尽）；爬山法主动扔掉兄弟与父节点，只带一个 Current 走。也是省内存和极快的代价；
4. Hill climbing performs well if the distance estimate (quality measurement) is locally consistent. That is, the node with the shortest distance estimate will eventually lead to the goal. 只有当距离估计（质量度量）局部一致时爬山法才表现良好，也就是距离估计最小的那个节点最终能通向目标。

Hill climbing may encounter the problem of stuck/trap at local optima. 爬山法可能遭遇卡在局部最优的问题。

5.2.1 爬山法三个困境

Local Maxima（局部极大）：四周都比自己低，但并非整座山的最高点，任何单步移动都变差，算法停住。

Plateaus（高原）：周围一片平，所有邻居的评估值都相同，算法看不出该往哪边走，只能随机游荡。

5.3 最佳优先搜索 Best-first search (or Greedy Search)

Select a node for expansion based on an evaluation function f(n), which is construed as a cost estimate. The node with the lowest evaluation is expanded first. i.e., argmin(f(n))

按评估函数 f(n)（代价估计）来选下一个展开的节点；评估值最小的那个先展开，即argmin(f(n))

启发成分 h(n)：

h(n) = estimated cost of the cheapest path from state at node n to a goal state

h(n) = 从节点 n 所在状态走到某个目标状态的**最便宜路径**的代价估计。

If n is a goal node, then h(n) = 0.

若 n 本身就是目标节点，则 h(n) = 0。

Tips: h(n) = 0 这条约定非常重要：它是后面 A* 最优性讨论的基石。到了目标，剩余代价当然是 0，这条看起来理所当然的性质，正是可采纳性（admissible）与一致性（consistent）定义中反复使用的那一步。

5.3.1 Example：启发函数为直线距离

Try to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly. Thus, it evaluates nodes by using just the heuristic function: f(n) = h(n). We use the Straight-Line Distance heuristic, which we call h_SLD.

优先展开离目标最近的那个节点，理由是它看起来能更快到达解；于是它评估节点时只用启发函数：f(n) = h(n)；使用直线距离启发式，记作 h_SLD。

启发式的具体形态：把离目标有多近量化成一个可以直接查表的数。整张表的来源是地图上的直线距离，也就是飞机能直飞的距离，与后面 A* 里的 g(n)（沿公路实际走的里程）形成对照。

注意 Bucharest = 0：这就是 4.4 中 if n is a goal node, then h(n) = 0 的实例。

5.3.2 贪心搜索与局限

从 Arad 出发、直奔 Bucharest 的过程，每个节点旁边标的就是 h_SLD：

把四步连起来看，贪心的短视一目了然：每一步都只做一件事，在眼前这些候选里挑 h_SLD 最小的那个。它从不回头（不像 A* 还留着 Oradea、Rimnicu Vilcea 在候选里），也从不比较已经走了多远（g 完全没出现）。

U 形障碍

However, it is not optimal. This shows why the algorithm is called "greedy" - at each step it tries to get as close to the goal as it can. If it finds its way around the "U"-shaped obstacle, it will follow the red path. It is also incomplete even in a finite state space, much like depth-first search.

它不是最优的；这也解释了为什么叫贪心，每一步都只想尽量靠近目标。如果它能绕过那个U 形障碍，就会走上红色路径。即使状态空间有限，它也可能不完备，很像深度优先搜索。

蓝线是贪心，红线是最优：贪心会径直朝目标的直线方向冲，一头撞进 U 形的内壁，然后卡在凹底（因为它只会朝看起来更近的方向走，而任何往回走的移动都离目标更远，于是它不敢走）。这与爬山法卡在 Local Maxima 是同一个病根：只看一步、不记历史、不接受暂时变差。模拟退火，正是为了治这个病而设计的。

5.4 A* Search

5.4.1 Definition and Core Idea

A* is an algorithm that use heuristic to guide the search while ensuring that it will compute a path with the minimum cost.

A* 是一种一边用启发式引导搜索、一边保证算出的路径代价最小的算法。必须满足的目标：快（靠启发式）与优（保证最小代价），整节剩下的内容都在解释它怎么把这对矛盾统一起来。

核心公式：A* computes the function f(n) = g(n) + h(n)

h(n) 是 "estimated cost"（估计代价）

The h(n) value guides search toward heuristically promising states. The function h(n) relies upon some heuristic information available from the problem domain. It changes from heuristic to heuristic, and from problem to problem. h(n) 把搜索带向启发式上看更有希望的状态；h(n) 依赖问题领域能提供的启发信息，换一个启发式、换一个问题；而且 A* 的全部理论（可采纳性与一致性）都建立在h 不要估高了这一条假设上。

g(n) 是 "actual cost"（实际代价）

while the g(n) value prevents search from persisting indefinitely on a fruitless search. h(n) in general weights how close node n is to the goal. 而 g(n) 防止搜索在一条没有结果的路上一去不回，h(n) 总体上是在度量节点 n 离目标有多近。

分量	是什么	方向	确定性
g(n)	起点 → n 的累计代价	向后看（回头路已定）	确定（实打实发生过的里程）
h(n)	n → 目标的最便宜路径估计	向前看（还有多少路）	不确定（靠启发式猜）
f(n)	两者之和	既是已付出也是还要付	排序用（每一轮选 f 最小的展开）

解决了贪心搜索的U型障碍：贪心只带 h，所以冲进凹底就出不来；A* 额外背着 g，于是已经绕了多远会不断抬升 f 值，让搜索有理由转向别的分支。

5.4.2 Example 罗马尼亚地图上的 A* 全过程

最终答案：最优路径是 Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest，总代价 418

> Arad → Sibiu = 140 → g(Sibiu) = 140，f = 140 + 253 = 393；
>
> Sibiu → Rimnicu Vilcea = 80 → g = 220，f = 220 + 193 = 413；
>
> Rimnicu Vilcea → Pitesti = 97 → g = 317，f = 317 + 100 = 417；
>
> Pitesti → Bucharest = 101 → g = 418，f = 418 + 0 = 418。
>
> 四段相加：140 + 80 + 97 + 101 = 418 ✓ 与 (f) 完全一致。

为什么 (e) 先找到了 Bucharest = 450，却不停?

因为在 (d) 之后，候选队列里有 Pitesti = 417、Bucharest(经 Fagaras) = 450、Craiova = 526…… 417 < 450，所以下一步必须先展开 Pitesti。展开 Pitesti 之后，它就给出了一个更便宜的 Bucharest = 418。如果换成贪心搜索，早在 (e) 见到 h(Bucharest) = 0 就会停下，拿到 450 这个次优解，这就是 A* 与贪心那 32 英里差距的来源。

5.4.3 Properties of A* 前提

条件一：可采纳性（admissible）：

A* generates an optimal solution if h(n) is admissible (never overestimates the cost to reach the destination node, i.e., the cost it estimates to reach the goal is not higher than the lowest possible cost from the current point in the path).

若 h(n) 是可采纳的，A* 就能给出最优解。可采纳的意思是它从不高估到目标的代价，也就是估计值不高于从当前点出发理论上可能的最低代价。

条件二：一致性（consistent）：

A* generates an optimal solution if h(n) is consistent, i.e., for every node n and for every successor node n' of n: $h(n) \leqslant c(n, n') + h(n')$

若 h(n) 是一致的，A* 也能给出最优解，一致的意思是对每个节点 n 和它的每个后继 n'，都有 $h(n) \leqslant c(n, n') + h(n')$，其中 c(n, n') 是从 n 到 n' 的边代价。一致 ⇒ 可采纳；可采纳 ⇏ 一致，一致是更强的条件。

- If h(n) is consistent, f along any path is non-decreasing. 若 h(n) 一致，则沿任意路径的 f 都是不减的；
- If h(n) is consistent, then h(n) is admissible. 若 h(n) 一致，则 h(n) 一定可采纳。

$h(n) \leqslant c(n, n') + h(n')$，即 $h(n) - h(n') \leqslant c(n, n')$ 。从 n 到目标的估计距离减去从 n' 到目标的估计距离，不能超过 n 到 n' 的实际距离。走一条边，估计值最多只能减少这条边的代价。它保证了沿路径的 f 不会下降。而 f 只可能上升或持平，就意味着一旦某个节点被 A* 展开（放进Closed），就再也不会被更便宜的路由绕回来。

Part 6 从搜索到优化 Optimization

6.1 优化问题

What are the values of x and y such that z is equal to the value at the peak? maximization? / minimization?

x 与 y 取什么值时 z 等于峰值？是最大化？还是最小化？

前面几节的目标是从起点走到目标，答案是一条路径；从这里开始，目标是找一组参数使目标函数取极值，答案是一个点。两者的共同骨架仍然是猜一组值 → 看结果好不好 → 改一改 → 重复。区别在于搜索的好不好是到没到目标，优化的好不好是一个数值。

6.2 Simple Example

y = f(x) = x²，where x ∈ [0, 10]，Find the value of x that gives the minimum of y

y = f(x) = x²，其中 x 限制在区间 [0, 10] 内；求使 y 取最小值的那个 x。

优化问题的三要素：

① 决策变量 x；② 目标函数 f(x) = x²；③ 约束 x ∈ [0, 10]。

答案：f 在 [0, 10] 上单调递增，所以最小值在左端点 x = 0 处取得，y = 0。

6.3 搜索空间扩大后的解

Not practical to evaluate ALL combinations! 把所有组合都评估一遍是不现实的！

A String（一个串）与 Cost function or Fitness function（代价函数或适应度函数）。

概念	为什么重要
A String（串 / 染色体） 对一个候选解的编码	优化算法不直接操作解，而是操作它们的编码
Cost function / Fitness function 给一个串打分的函数	整章唯一能判断好坏的依据，所有方法都靠它比较候选

前面 BFS / DFS 的隐含前提是状态空间可以被穷尽（所以 BFS 敢说找到的一定最短）；当搜索空间大到无法穷举时，完备性就必须让位给在有限代价内拿到足够好的解。接下来两个方法（SA 与 GA）都有一个共同特征：都放弃了保证最优，换来了能在大空间里跑得动。

6.4 传统做法：单点搜索与梯度下降 single point search

Conventional methods: single point search

Optimization problem: Finding a $P_o $where

$$P_o = P_o \in S | f(P_o) = \min_{\forall P_i \in S} f(P_i)$$

优化问题就是找一个 $P_o $：它属于解空间 S，且它的函数值 $f(P_o) $等于 S 中所有解的函数值里的最小值。这类方法叫 single point search（单点搜索），并标出小标题 Gradient-driven search（梯度驱动的搜索），每次只维护一个当前解，沿着梯度往低处挪。

核心公式：

1. 极值点的必要条件：在极值点处，函数对参数的偏导为零（曲面的切线是水平的）

$$\partial f / \partial P = 0$$

2. 迭代更新式：下一步的位置 = 当前位置 − 学习率 × 当前点的梯度。

$$P_{n+1} = P_n - \alpha \cdot (\partial f / \partial P)|_{P = P_n}$$

α is a small positive value - learning rate（α 是一个小的正数，称为学习率）。

负号的含义：梯度指向上升最快的方向，所以取负号就是在往下降最快的方向走。

图上有三支从右侧沿切线方向指向谷底的箭头，沿着切线一步步滑下去；谷底那个点上标着 ∂f / ∂P = 0 ，算法停下来的判据，就是梯度为零。

6.5 梯度下降的三条局限

1. Function must be continuous/differentiable 函数必须连续 / 可微；
2. Local optima 会陷在局部最优；
3. Performance depends on the starting point 表现依赖起始点。

局限	问题	后果	解决
必须可微	现实中大量问题是离散的、组合的（排班、路径、选型），根本没有梯度可用	直接无法使用，这也是 GA 的立足点（GA 只需能算适应度，不需要导数）	GA 只要求能算适应度
局部最优	梯度只告诉附近哪里更低，从不告诉你全局最低在哪	可能停在半山腰的坑里，而它自己无从察觉	SA 用概率接受更差解
依赖起点	起点不同，滑进的那个坑可能完全不同	同一个算法跑十次得到十个不同答案	GA 同时维护一整群解

Part 7 模拟退火 Simulated Annealing

7.1 从物理退火到算法

What is simulated annealing? The main idea is to avoid local minima by accepting moves to worse solutions probabilistically. It is one of the first stochastic local search methods, inspired by the analogy to physical annealing.

模拟退火（SA）的核心思想是通过以一定概率接受更差的解来避开局部极小，是最早的一批随机局部搜索方法之一，灵感来自物理退火的类比。

物理退火（Physical annealing）：

A process for obtaining the minimum energy state of a solid, whereby the melted solid is slowly cooled down until reaching its ground state. A specific schedule is used to cool down the solid very slowly.

退火是让固体达到最低能量状态的过程，把熔化了的固体缓慢冷却，直到它落到基态；并且会用一个特定的降温时间表，把固体降得非常慢。

计算机模拟（Computer simulation）：

The Metropolis algorithm is a simulation algorithm for the physical annealing process that can be applied to solve optimization problems!

Metropolis 算法是对物理退火过程的模拟算法，它可以被用来解优化问题

物理世界	优化世界	对应关系
固体的一个微观状态	一个候选解 s	状态与解
能量 E	代价函数值 f(s)	能量越低越好，代价越小越好
温度 T	控制参数 T	它决定敢不敢接受更差的解
基态（最低能量）	全局最优解	退火的目标

物理世界	优化世界	对应关系
缓慢降温的时间表	降温计划（cooling schedule）	降得越慢，越可能落到真正的基态

Tips：降温速度是 SA 成败的核心，所以慢是关键：如果降温太快，固体会来不及排列整齐就被冻住，停在一个亚稳态，这在优化里就是卡在局部最优。SA 用时间换质量，是它相对梯度下降最大的代价。

SA 可以看作是对 Hill Climbing 的改进。普通 hill climbing 只接受让目标函数变好的移动，因此很容易停在 local optimum；而 SA 在搜索早期温度较高时，允许以一定概率接受更差的解，从而有机会跳出局部最优。随着迭代次数增加，temperature T 逐渐降低，算法接受差解的概率也会下降，搜索行为会越来越接近 hill climbing，最终在低温阶段稳定到较优解附近。

7.2 Metropolis 准则

k_B 是 Boltzmann constant（玻尔兹曼常数），T是 temperature（温度）。

情形	判据	动作
新状态更好或持平	$E_j \leqslant E_i$	无条件接受 s_j
新状态更差	$E_j > E_i$	以概率 $ p = exp(−(E_j − E_i)/(k_B · T)) $接受

差得越多，接受概率越小：ΔE 翻倍，概率是平方级下降；

温度越高，接受概率越大：T → ∞ 时 p → 1（高温时几乎什么都接受）；

温度越低，接受概率越小：T → 0 时 p → 0（低温时退化为纯爬山法，只接受更好的解）。

If the temperature is lowered slow enough, the solid may reach thermal equilibrium at each temperature. 如果降温足够慢，固体可以在每个温度下达到热平衡；在优化里意味着在固定的 T 下反复采样足够多次之后，解会按玻尔兹曼分布停在各个状态上；T 缓慢降到 0 时，这个分布会逐渐集中到全局最优那一点。所以 SA 在理论上有概率收敛到全局最优（前提是降温足够慢）；但实践里足够慢意味着时间长得不可接受，所以工程上都是用它换一个足够好的解，而不是真等它收敛。

7.3 SA 的算法框架

Without loss of generality, minimization problem is assumed.

不失一般性，假定是最小化问题，意味着更好= 代价更小，$ f(s_i) < f(s) $就是新解更优解。

SA 的四步流程：

1. Generate neighboring solutions s_i from S

从 s 的邻域里生成邻近解 s_i;

2. Accept new solution if it is better in cost value, i.e., when f(s_i) < f(s)

如果新解的代价更小，接受它；

3. Otherwise, make the decision whether or not to accept: Yes: Replace current solution with the new (worse) solution / No: Discard the new solution

否则要做一个决定：接受（用这个更差的解替换当前解）或丢弃，SA 唯一能翻过山脊逃出局部最优的手段；

4. Probabilistic decision making based on: Difference between new and old cost values / Control parameter T

这个概率决定由两个量决定：新旧代价之差，以及控制参数 T。

7.4 SA illustration

图中标注	含义
HILL CLIMBING × 3	爬山法的轨迹：一条单调下降、然后平掉的曲线（陷入局部最优后不再改善）
AT INIT_TEMP（初始温度）	SA 早期的行为：上下大幅抖动，差解被频繁接受
AT FINAL_TEMP（终止温度）	SA 后期的行为：抖动收敛成一条单调下降线，形态与爬山法一致
Move accepted with probability = $ e^{(-(ΔC/temp))}$	以该概率接受更差移动，即 Metropolis 准则
Unconditional Acceptance（无条件接受）	更好的移动永远直接接受，不掷骰子

在生成邻域解（neighborhood solution）时，可以随机生成一个邻近解，也可以系统性地生成邻近解，以确保整个邻域空间都被探索。SA 常用 Metropolis criterion 作为接受准则。

接受准则（Acceptance criterion）：若新解 s' 的 cost 不大于当前解 s，则直接接受：f(s') ⩽ f(s)；如果新解更差，即 f(s') > f(s)，则以如下概率接受该较差解：p = exp(−(f(s') − f(s)) / T)。

7.2 用的是能量版：p = exp(−(E_j − E_i)/(k_B · T))，分母里有玻尔兹曼常数 k_B；

7.4 用的是优化版：p = exp(−(f(s') − f(s))/T)，k_B 被吸收进了 T。

两者等价，因为 k_B 只是个常数，工程实现时常把它并到温度里统一调节。

其中 T 是当前温度。可以看到，当 T 较大时，即使 f(s') − f(s) 较大，接受概率仍可能不低，因此算法更偏向随机搜索（exploration，探索）；当 T 逐渐降低时，接受差解的概率趋近于 0，算法更偏向爬山法（exploitation，开发）。也就是说，SA 前期通过偶尔接受坏解扩大搜索范围，后期通过降温逐渐收敛。

Part 8 遗传算法 Genetic Algorithms

8.1 GA Definition

A genetic algorithm is a search-based algorithm based on natural selection, which can be used for solving AI optimization problems.

遗传算法是一种基于自然选择的搜索型算法，可以用来求解 AI 优化问题。

术语	课件原文	中文理解
Population 种群	This is a subset of all the probable solutions that can solve the given problem.	能解决给定问题的全部可能解中的一个子集
Chromosomes 染色体	A chromosome/string is one of the solutions in the population.	种群中的一个解，也叫串（string）
Gene 基因	This is an element in a chromosome.	染色体里的一个元素（串上的一位）
Fitness function 适应度函数	This is a function that uses a specific input to produce an improved output. The solution is used as the input while the output is in the form of solution suitability.	以解为输入、以解的合适程度为输出的函数，它是 GA 判断好坏的唯一依据

术语	课件原文	中文理解
Genetic operators 遗传算子	Genetic operators such as Selection, Crossover, Selection, are used for changing the genetic composition of chromosomes.	用来改变染色体基因组成的操作，课件举的例子是选择与交叉

8.2 GA 的组成

A genetic algorithm repeatedly modifies a population of individual solutions. At each step, the genetic algorithm selects individuals at random from the current population to be parents and uses them to produce the children for the next generation. Over successive generations, the population "evolves" toward an optimal solution.

GA 反复地改造一个由个体解构成的种群。每一步中，它从当前种群里随机挑个体当父母，用它们产生下一代的子女。经过一代又一代，种群会进化向一个最优解。

算子	作用
选择 Selection	从当前种群挑出当父母的个体
交叉 Crossover / 重组 Recombination	把两个父母的基因片段交换，产生子女
变异 Mutation	随机改动个别基因

重复与随机：repeatedly说明 GA 是迭代式的；selects individuals at random（随机挑选个体）说明选择并非确定性，适应度高的个体只是更可能被选中，不是一定被选中。这一层随机性就是 GA 能跳出局部最优的机制来源。

8.3 A Simple Genetic Algorithm

1. 编码 Initial/random designs coded：把现实问题的方案，编码成染色体字符串。
2. 初始种群：生成一批随机候选解，附带各自适应度。
3. 解码 / 评估 / 仿真 Decoding / Evaluation / Simulation

   解码：字符串 → 还原回真实问题方案

   仿真计算，得到适应度分数

> 适应度评估一般在 GA 算法外部完成；仿真计算开销大，是 GA 主要算力消耗来源。

4. 选择 Selection：适应度越高，被选为父代的概率越大。
5. 交叉 Crossover：父代染色体片段交换，产生子代。
6. 变异 Mutation：随机修改某一处基因。
7. 新一代 New generation： 返回第 3 步评估，循环迭代。
8. 出口 Final optimized designs coded：迭代终止，输出编码形式的最优解。

交叉：P_2: 4 0 0 3 0 | 1 6 1 与 P_3: 0 1 6 4 1 | 8 0 1交换分割点后的尾巴得到两条子代。

变异：P_2': 4 0 0 3 0 8 0 1 → P_2'': 4 0 1 3 0 8 0 1，第 3 位基因被修改。

8.4 GA三步逐一演示

GA 的工程实现：GA 原生目标是**最大化适应度 fitness**；工程大多任务是最小化代价 cost，需要转换。

1. fitness = 1/cost
2. fitness = C_{max} − cost
3. fitness = exp (−cost/T)

初始种群：

P_1 : [1, 2, 0, 9, 0, 2, 1, 7], $f(P_1)$ = 5%

P_2 : [4, 0, 0, 3, 0, 1, 6, 1], $f(P_2)$ = 60%

P_3 : [0, 1, 6, 4, 1, 8, 0, 1], $f(P_3)$ = 35%

1. 繁殖 Reproduction（选择）：按照适应度分配后代数量。P_1后代少，P_2生 2 个、P_3生 1 个。

> 仅选择：不改动染色体基因，不需要重新算适应度。

2. 交叉 Crossover：随机选分割点，交换父代片段。

> 交叉只重组已经存在的基因，不会创造全新基因；不需要算适应度。

3. 变异 Mutation：随机翻转 / 修改部分基因。唯一可以生成种群原本不存在的新基因。

> 变异后得到全新子代，进入下一轮循环必须重新评估适应度。

交叉只是重组已有的基因（不引入新基因），而变异引入了搜索空间里原本不存在的新基因。从评估预算角度看，这两步的成本是一样的（都产生新串）

8.5 Basic Operations of a Simple GA：简单 GA 的基本算法

步骤	作用	属于
Make initial population	生成初始种群	只做一次
Evaluate the fitness of all initial chromosomes	给初始种群整体打分	只做一次
Reproduce children from parents (selection)	按适应度选父母、产下一代	每代循环
Apply crossover to some chromosomes	对一部分染色体做交叉	每代循环
Apply mutation to some chromosomes	对一部分染色体做变异	每代循环
Preserve some elite individuals	保留若干精英个体	每代循环
Decode the best chromosome found	解码最优染色体	只在最后

Tips：交叉、变异**不是对全部个体生效**。

交叉：较高概率，0.6 - 0.9，成对操作；变异：每个基因很小概率，典型值 0.02； 这两个概率是 GA 最重要调参旋钮。

8.6 Details of Genetic Algorithms

Constant population size = n：**种群大小 n 全程固定不变。**资源受限的竞争：任何个体的加入，都意味着有另一个个体被挤出。而梯度下降只有一个点，无所谓竞争；GA 有一群点，并且每代都在做谁留下、谁出局的淘汰。

层级从上到下：

1. 种群 Population：一共 n 个个体，总数恒定 n。
2. 个体 / 染色体 Chromosome：共 n 条，每条附带自己适应度f_i。
3. 基因 Gene：每条染色体基因长度全部相等。

种群规模固定意味着：**每新增一个个体，就要淘汰另一个个体**，是资源竞争淘汰机制。 对比梯度下降，梯度下降只有单个搜索点，不存在竞争淘汰。

Constant population size = n 这条约束：种群规模在演化过程中保持不变。这带来一个必须被显式处理的后果，选择时必须填满 n 个名额（谁多生、谁不生、谁被淘汰，要凑够整数），这也正是用精英替换最差的 10% 子女这种做法能成立的前提：替换而不是增殖，所以总数不变。

8.7 Designing a Representation

We have to come up with a method of representing an individual as a genotype. When choosing a representation, we have to bear in mind how genotypes will be evaluated and what genetic operators might be. 我们必须想出一种把个体表示成基因型的方法。选择表示法时，必须同时考虑两件事：基因型将如何被评估，以及遗传算子会是什么。

核心思想：表示形式决定算子怎么写，不同编码，交叉、变异实现完全不一样。

两条设计原则：

1. 需要把实际问题解表达成基因型（染色体串）。
2. 设计编码的时候，同时要想好两件事：怎么评估这个基因型、遗传算子如何实现。

设计维度	选项	影响
基因的取值类型	binary, decimal, real, order-based?（二进制 / 十进制 / 实数 / 基于次序）	决定变异怎么写
问题的性质	numerical or combinatorial?（数值型 / 组合型）	决定交叉怎么写
基因的最小单位	Bit or Binary number（位 或 二进制数）	决定串长与精度

举例不同编码对应的变异实现

1. 二进制编码变异：比特位取反。
2. 基于次序编码（TSP 旅行商）变异：随机交换两个基因。不能简单改数字，否则出现重复城市。
3. 树形编码变异：随机替换树内部某个节点。

> 硬性约束：交叉、变异操作结束，**必须产出合法的染色体解**。

8.8 基因型到表现型：二进制整数编码

$$1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 128 + 32 + 2 + 1 = 163$$

8 位二进制串 10100011 按权展开求和得到 163，于是基因型 10100011 对应表现型整数 163。

课件把这两个概念并排写成一张对照表：图上标 8 bits（8 位），左边是 Genotype（基因型：编码字符串）、右边是 Phenotype（表现型：现实世界真实数值）。

What if we have 10 parameters? How many bits in total?

如果有 10 个参数呢？总共需要多少位？若每个参数都用 8 位精度，则总串长 10 × 8 = 80 位。

GA 的搜索空间随参数个数指数增长，1 个参数是 $2^8 = 256$ 种组合，10 个参数搜索空间大小就是 $2^{80} \approx 1.2 \times 10^{24}$ 种指数爆炸，**无法穷举全部组合**。

8.9 二进制映射实数

当表现型是实数时，例如用 8 位二进制表示区间 [2.5, 20.5] 上的一个数，则按线性映射把基因型的整数值163折算成实数 x。把整数基因型线性映射到实数区间 [2.5, 20.5]，区间跨度 20.5 − 2.5 = 18。

$x = 2.5 + \frac{163}{255}\times 18=13.9609$

分母 255：保证全 1 严格等于区间上界 20.5；

分母 256：简单工程实现，但最大值达不到区间上界。

8.10 Mutation Operators：变异算子

变异的核心设定：变异通常以很小的概率 p_m 发生，例如每个基因 0.02。

Mutation usually happens with small probability $p_m $(e.g., 0.02) for each gene.

变异的三个条件：

1. At least one mutation operator should allow every part of the search space to be reached. 至少要有一个变异算子，能让搜索空间的每一个部分都被抵达。没有变异，种群里的基因材料只会被重新组合，永远造不出初始种群没有的基因；搜索会退化成一个封闭的组合游戏。
2. The size of mutation is important and should be controllable. 变异的幅度应当可控。幅度太大等于随机重启（丢掉父辈的全部信息），太小则形同没变，所以必须可调。
3. Mutation should produce valid chromosomes. 变异必须产生合法的染色体。例如基于次序的表示里每个城市必须恰好出现一次，变异绝不能造出重复或缺失的城市。

Is mutation critical in GA? 变异在 GA 里是必需的吗?

是，交叉只能重组已有的基因（selection 与 crossover 都不产生新基因），因此变异是 GA 唯一的新基因来源，也是三条要求里第一条（every part of the search space to be reached）的直接依据。交叉负责开发（exploitation，用好现有材料），变异负责探索（exploration，去没去过的地方）。

表示法决定算子：同一个变异操作，在不同表示法下是完全不同的代码：

1. 二进制表示：把某一位取反

例：（1 1 1 1 1 1 1 → 1 1 1 0 1 1 1）；

2. 基于次序的表示（order-based representation）：随机交换两个基因。交换两个基因会让同一个位置上出现重复取值。所以条件三才会专门强调 Crossover should produce valid chromosomes（交叉必须产生合法染色体）。

例：（7 8 3 4 1 2 6 5 → 7 8 3 4 6 2 1 5）；

3. 树形表示：变异 = 随机把一个节点替换成另一个节点。

8.11 交叉算子

交叉（Crossover）：拿两个父代个体，交换基因片段来生成新子代，核心是混合两份父代的特征。

交叉的两个条件：

1. The child should inherit something from each parent. If this is not the case, then the operator is a mutation operator. 子代必须同时继承来自两个父代的基因。如果子代完全只抄其中一方，这不叫交叉，本质等同于变异。
2. Crossover should produce valid chromosomes. 交叉产生出来的子代染色体，必须是**合法有效的解**。

单点交叉：交换连续一大截基因。

把两条染色体，随机选一个切割位置，互相交换切割点**后面的一整段基因**。

退化情况：如果切割点在**最左端**：子代直接完整复制另一个父代，没有混合双方信息。如果切割点在**最右端**：子代和原来父代一模一样，没有产生任何新组合。 这两种不算有效交叉，代码实现时要重新随机选切割点。

均匀交叉：每一位都可以独立选择来源，基因打散重组更充分。

不再切一整块；生成一条和染色体长度一样的**掩码序列**，掩码每一位决定这一位基因取自哪个父代。

掩码 =0 → 该位置基因拿父代1

掩码 =1 → 该位置基因拿父代2

第3、5、8位是 1，取父代2的C、E、H；剩下位置取父代1。子代2会反过来，掩码0取父代2、掩码1取父代1。

缺点：单点交叉和均匀交叉都很容易把已经搭配得很好的一长串优良基因组合给拆碎。

基于次序表示的重组（用于TSP旅行商这类排列问题）

场景：染色体代表访问城市的顺序，每个数字（城市）只能恰好出现一次。 普通单点/均匀交叉会造出重复城市、丢失城市，直接变成非法解。所以要用专门的次序重组逻辑。

执行步骤：

1. 在父代1上，截取一段连续基因片段，直接原样保留放到子代对应位置。
2. 剩下空出来的位置：从**父代2**拿基因，但跳过已经被截取片段里出现过的基因。
3. 填充顺序：从截取片段的切点往后填，填到染色体末尾就绕回最开头。
4. 两个父代互换角色，再生成第二个子代。

8.12 选择策略 Selection Strategy

核心设计思想有两点：

> We want to have some way to ensure that better individuals have a better chance of being parents than less good individuals. This will give us selection pressure which will drive the population forward. We have to be careful to give less good individuals at least some chance of being parents - since they may include some useful genetic material. The most commonly used selection methods include Roulette Wheel Selection, Rank Selection, Tournament Selection etc.

1. 质量好的个体，有更大机会被选为父代。由此产生选择压力，驱动种群不断往更优的方向演化。
2. 但不能完全淘汰掉差的个体。就算表现差，它身上也可能藏有有价值的基因；要给它保留一点点被选中的可能性，避免基因多样性彻底消失。最常用的选择方法有：轮盘赌选择、排名选择、锦标赛选择等。

选择的核心目的：从当前种群里挑选个体当父代，用来做交叉、变异产生下一代。

方法	怎么选	特点
轮盘赌 Roulette Wheel	按适应度占比分配被选概率（本页下一页的算例）	直观，但适应度差异大时会过早收敛
排名 Rank	按适应度的排名而非数值分配概率	抗超级个体适应度再大也只按名次算
锦标赛 Tournament	随机抽 k 个个体，取其中最优秀的	实现简单、参数少，最常用

轮盘赌选择的两大缺陷

$P(x_i)=\frac{f_i}{\sum f_i}$

个体被选中概率和自身适应度**数值大小直接成正比**，想象一个轮盘，适应度越高占的扇区面积越大。

1. **早熟收敛 premature convergence** 如果出现一个适应度遥遥领先的“超级个体”，它被选中的概率会极高。几代之内种群大部分都变成它的后代，种群多样性直接丢失，算法卡在局部最优出不来。
2. **选择压力不足** 如果种群里面所有个体适应度都差不多，那大家被抽中的概率几乎一样。选父代接近随机抽签，演化几乎变成随机游走，很难向着好解推进。

为什么需要排名选择？ 排名选择扔掉原始适应度的绝对数值，只用排序名次。 哪怕某一个个体适应度爆炸式高，它也只能拿到第一名对应的概率，不会一家独大。既保留优胜者优先被选的机制，又规避超级个体垄断种群的风险。除了换排名选择，精英保留（8.13）也是缓解这类问题的常用手段。

8.13 精英保留 Elitism

Re-introduce in the population previous best-so-far (elitism). For example, keep the best 10% of the parent population and re-introduce them to the next generation by replacing the worst 10% child population.

把算法运行到现在为止，历史表现最好的一批个体，重新放回下一代种群。例如，保留父代中最好的 10%，用它们替换掉子代中最差的 10%，从而进入下一代。

Example of a convergence trace（收敛轨迹示例）

无精英保留	开启精英保留
之前找到的优秀解，可能被交叉变异破坏、直接丢失；最优适应度曲线上下震荡波动	历史最优个体一定会被保存。最优适应度曲线单调不降，呈现阶梯上升，很容易观察算法有没有继续进步。

精英保留：只做保底。不会产生任何新基因、不会探索新区域，只管守住已经找到的好结果。

变异Mutation：负责探索，制造之前种群没有的新基因。 两者各司其职，不能互相替代。

8.14 Stopping Criterion：停止准则

三条停止条件：

1. The optimum is reached. 已经达到最优（基本不存在）
2. Limit on CPU resources: maximum number of fitness evaluations. 计算资源上限：适应度评估的最大次数。

3. Limit on the patience: after some generations without improvement. 耐心上限：连续若干代没有改善之后就停。

三条实验纪律：

1. Never draw any conclusion from a single run. 不能只跑一次 GA 就下结论。
2. Use statistical measures (e.g., average, standard deviation). 报告统计指标：平均值、标准差，还可以补充最优、最差、中位数，再配上收敛曲线（最优值随迭代代数变化）。
3. Obtain results from a number of independent runs. 必须跑多次独立重复实验。原因：GA 有大量随机性，初始种群是随机生成。单次运行结果有偶然性，一次跑的好不代表算法本身强。

8.15 GA 的优点与局限

8.15.1 优点

1. 不需要导数信息 梯度下降要求目标函数可微分才能更新参数。GA 不用求导，只要能算出适应度就行。

> 这是 GA 最大优势，特别适合组合优化问题，例如路径规划、排班，这类问题根本求不出导数。

2. 适配多种优化任务 不管是连续数值问题、离散问题，还是多目标优化都可以处理，适用面广。
3. 天然适合并行计算 一个种群里面有很多个体，每个个体算适应度互相之间互不干扰，可以同时跑。

> 如果适应度是仿真模拟，并行可以大幅节省运行时间，工程仿真场景很喜欢用 GA。

4. 结果随迭代持续变好 随着一代代演化，通常能够不断得到质量更高的候选解。

8.15.2 局限

1. 简单可微问题效率低 如果目标函数光滑可求导，梯度下降收敛速度远快于 GA。GA 不是万能算法，它是梯度方法的备选方案，而不是替代品。
2. 不能保证得到全局最优解 如果参数、算子设置不合理，很容易卡在局部最优。就算调得很好，也没有理论保证一定找到全局最优。

> 在搜索空间巨大、没办法全部穷举的场景，GA 目标是有限算力下拿到 “足够好用的解”，不等于一定拿到理论上最好的解。

3. 适应度计算开销巨大 每一代所有个体都要计算适应度。如果适应度需要跑仿真、复杂模拟，算力消耗会非常高。
4. 实现细节影响结果 编码方式、交叉变异概率、选择策略、精英保留这些配置一旦设置错，最终结果会很差。

优点 / 局限	呼应
不需要导数	直接绕过了梯度下降必须可微这一条，这是 GA 能被用在组合优化（排班、路径、选型）上的根本原因
能处理离散与多目标	适应度函数只需能算，不需要连续、不需要可微
并行能力极佳	因为一整代个体彼此独立，可以并行评估，这是 GA 在工程仿真场景里特别受欢迎的原因
不保证最终解质量	在无法穷举的空间里，没有任何方法能保证最优；GA 只是在有限预算内给出一个足够好的解

优点 / 局限	呼应
反复算适应度有计算挑战	一次适应度评估可能就是一次完整的工程仿真
简单问题上效率不高	简单问题（尤其可微的连续问题）应当直接用梯度方法；GA 是没有梯度可用时的备选，不是万能替代

BFS / DFS（无信息、完备但昂贵）→ 爬山法、最佳优先与 A* 算法（引入启发式）→ 模拟退火（用概率接受差解，逃出局部最优）→ 遗传算法（用种群与算子，在无法穷举的空间里做有方向的随机搜索）。

四类方法的共同骨架：猜一个解 → 用某个函数评价它 → 按某个规则改成新解 → 重复；区别只在怎么改与怎么评价。

Part 9 Tutorial 2

9.1 Exercise 1：广度优先搜索

Given the problem-solution graph in Figure 1, perform a breadth-first search starting from the initial state A and ending in the goal state F. If there are two or more nodes of the same preference, then handle the nodes alphabetically.

给定 Figure 1 的问题—解图，从初始状态 A 出发、到目标状态 F 为止，执行一次广度优先搜索。若出现两个或以上优先级相同的节点，则按字母顺序处理。

按同层按字母升序的规则，BFS 逐层展开如下：

步	展开	open（队列）	closed
1	A	[B, C, D]	[A]
2	B	[C, D]	[A, B]
3	C	[D, E]	[A, B, C]
4	D	[E, F]	[A, B, C, D]
5	E	[F]	[A, B, C, D, E]
6	F	—	—

全部展开后才找到：A → B → C → D → E → F

9.2 Exercise 2：深度优先搜索

Given the problem-solution graph in Figure 1, perform a depth-first search starting from the initial state A and ending in the goal state F. If there are two or more nodes of the same preference, then handle the nodes in descending (rather than ascending) alphabetical order.

同样的 Figure 1，从 A 出发、到 F 为止，执行一次深度优先搜索。但这次优先级相同的节点改为按字母降序（而不是升序）处理。Exercise 2 与 Exercise 1 的区别就是descending (rather than ascending)，升序改正降序

按同层按字母降序的规则：

步	展开	孩子按降序	选择	open（栈）
1	A	D > C > B	选 D	[B, C, D]（D 在栈顶）
2	D	F > E	选 F	[B, C, E, F]（F 在栈顶）

步	展开	孩子按降序	选择	open（栈）
3	F	—	F 就是目标 → 结束	—

解路径为 A → D → F，只用了 2 条边，因为降序规则恰好把搜索直接指向了最深处的目标 F。

9.3 Exercise 3：四皇后问题的爬山法

The 4 queens problem consists of a 4 x 4 chessboard with 4 queens. The goal is to place the 4 queens on the chessboard such that no two queens can attack each other. Each queen attacks anything in the same row, in the same column, or in the same diagonal.

在 4×4 棋盘放 4 个皇后。

规则：皇后可以攻击同一行、同一列、同一斜线上的其他皇后。要求任意两个皇后之间不能互相攻击。

好处：直接彻底消除同列冲突，不用再检查列；只需要判断同行、对角线斜线冲突。

表达方式：用数组 $[x_0, x_1, x_2, x_3]$。x_i 代表第i列的皇后所在行号。

邻域操作：交换两列对应的行号。交换之后依旧保持每列 1 个皇后，不会产出非法解。

代价函数 cost

统计互相攻击的皇后对的数量。4 个皇后，两两组合一共 $C_4^2 = 6$ 对。 对第i列、第j列的皇后（$i < j$，行号x_i, x_j）： 发生攻击，满足下面任意一条：

1. 同行：$x_i = x_j$
2. 斜线：$|x_i - x_j| = |i - j|$

> cost = 冲突的皇后对数。 cost = 0 代表完全没有冲突，就是问题的解。

并列候选的抉择规则

当多个邻居的代价 cost 一样好时： 优先选择被交换的两列下标数字最小的那一对。 例：可以交换(x_0, x_1)，也可以交换(x_2, x_3)，选(x_0, x_1)。

爬山法逻辑

1. 当前状态，把所有邻域（全部两列交换）全部枚举，算出每个邻居 cost。
2. 挑选代价最小的邻居作为下一步状态；多个一样优，就用上面下标最小规则破平局。
3. 重复，直到：所有邻居都不能比当前更好；或者直接达到`cost=0`（不可能再优化），算法终止。

完整爬山执行步骤

初始状态：$[3, 1, 2, 0]$，初始 cost=2 含义：第 0 列皇后在 3 行；第 1 列在 1 行；第 2 列在 2 行；第 3 列在 0 行。

第一轮：初始状态生成全部邻居

序号	交换的列对	新状态 (x_0, x_1, x_2, x_3)	cost
1	(x_0, x_1)	(1, 3, 2, 0)	1
2	(x_0, x_2)	(2, 1, 3, 0)	1

序号	交换的列对	新状态 (x_0, x_1, x_2, x_3)	cost
3	(x_0, x_3)	(0, 1, 2, 3)	6
4	(x_1, x_2)	(3, 2, 1, 0)	6
5	(x_1, x_3)	(3, 0, 2, 1)	1
6	(x_2, x_3)	(3, 1, 0, 2)	1
初始	（基准）	(3, 1, 2, 0)	2

最优代价是 cost=1，一共有 4 个邻居同时达到。看交换下标：(x_0, x_1)下标最小，所以下一步跳到状态 [1, 3, 2, 0]。

第二轮，以[1, 3, 2, 0]作为当前状态，再枚举全部邻居

序号	交换的列对	新状态 (x_0, x_1, x_2, x_3)	cost
1	(x_0, x_1)	(3, 1, 2, 0)	2
2	(x_0, x_2)	(2, 3, 1, 0)	2
3	(x_0, x_3)	(0, 3, 2, 1)	4
4	(x_1, x_2)	(1, 2, 3, 0)	4
5	(x_1, x_3)	(1, 0, 2, 3)	2
6	(x_2, x_3)	(1, 3, 0, 2)	0 ← 目标!
当前	（基准）	(1, 3, 2, 0)	1

出现 cost=0，已经完全没有冲突，得到四皇后的合法解。 cost 最小只能等于 0，不可能继续优化，爬山算法直接终止。最终解：[1, 3, 0, 2]

9.4 Exercise 4：A* 算法

Perform the A* Algorithm on the following graph. Explicitly write down the queue at each step, where the red figure means the heuristic value, and the black figure is the path cost.

在给定图上执行 A* 算法，并显式写出每一步的队列；图上红色数字是启发值 h，黑色数字是路径代价 g。

i	c（展开谁）	open_list	close_list	本步发生了什么
0	—	[S]	[ ]	初始化：open_list.append(start)
1	S	[A, B, C]	[S]	展开 S → 加入 A、B、C
2	C	[A, B, D]	[S, C]	f 最小者为 C（14）→ 加入 D
3	A	[E, B, D]	[S, C, A]	f 最小者为 A（16）→ 加入 E
4	E	[B, D, F]	[S, C, A, E]	f 最小者为 E（16）→ 加入 F
5	F	[B, D, G]	[S, C, A, E, F]	f 最小者为 F（17）→ 加入 G
6	D	[B, G]	[S, C, A, E, F, D]	f 最小者为 D（18，与 B 并列，课件取 D）；D → F 无改进
7	B	[G, D, E]	[S, C, A, F, B]	展开 B → E（f 降到 15）与 D（f 降到 14）被重开，从 close_list 移出
8	D	[G, E]	[S, C, A, F, B, D]	f 最小者为 D（14） → 重新展开；D → F 仍无改进

i	c（展开谁）	open_list	close_list	本步发生了什么
9	E	[G, F]	[S, C, A, B, D, E]	f 最小者为 E（15） → E → F 使 F（f 降到 16）被重开
10	F	[G]	[S, C, A, B, D, E, F]	f 最小者为 F（16） → F → G 使 G 的 f 降到 18
11	G	—	—	current_node == end → return，算法结束

Lecture 3

part 1 Search&Optimization Recap 搜索优化回顾

1.1 Search

Search 搜索：Agent（智能体）能够构造**动作序列**以达成其目标。

The agent can construct sequences of actions that achieve its goals.

Uninformed (blind) search 无信息（盲目）搜索：除初始状态、搜索算子与解的测试外，不使用任何其他信息。*Uses no information other than the initial state, the search operator, and a test for a solution.*

Informed search 有信息搜索：可访问启发函数 h(n)，用来估计从 n 到解的成本。*Has access to a heuristic function h(n) that estimates the cost of a solution from n.*

**算法 ** Algorithm	**展开准则 ** Expansion Criterion	**公式 ** Formula
Greedy Best-First Search 贪婪最佳优先	展开 h(n) 最小的节点	f(n) = h(n)
A* Search	展开 f(n) 最小的节点	f(n) = g(n) + h(n)

其中 g(n) = 从起点到 n 的实际成本，h(n) = 从 n 到目标的估计成本。

A* 在 h 满足可采纳性（admissible）与一致性（consistent）时最优。

1.2 Optimization

Optimization 优化：通过**最小化成本函数**或**最大化适应度函数**，从一组候选中找出最佳解。*Finding the best solution from a set of candidates by minimizing a cost function or maximizing a fitness function.*

Simulated Annealing (SA) 模拟退火：以**概率方式接受更差的动作**。概率取决于成本差 ΔE 与温度 T：**高温探索（explore）、低温利用（exploit）**，从而跳出局部极小。*Accepts worse moves probabilistically; high T explores, low T exploits. It escapes local minima.*

Genetic Algorithms (GA) 遗传算法：维持一个**种群**，通过**选择（selection）、交叉（crossover）、变异（mutation）**逐代进化出更好的解。*Maintain a population of solutions and apply selection, crossover, and mutation to evolve better solutions over generations.*

Schema Theorem 模式定理：**定义长度短、阶数低、适应度高于平均**的模式（schema）获得指数级增长的试验次数，这些是搜索的**积木块（building blocks）**。*Short defining length, low order, above-average schemata receive exponentially increasing trials.*

part 2** **Evolutionary Algorithms 进化算法

2.1 Definition and Origin

EAs are **population-based **search approaches that are inspired by the principles of natural selection and genetics.

一类基于种群的搜索方法，其灵感来源于自然选择与遗传学原理。

Era	Milestone	Who / What
1950	Alan Turing 提出用自然进化构建智能机器	*“Computing machinery and intelligence”, Mind 49: 433-460, 1950.* 进化驱动生物系统的适应、多样性与创新。
1960s Sprouting Period 萌芽期	Alexander Fraser 尝试模拟遗传系统并研究其进化动力学	Evolutionary Programming 思想萌芽
1980s / 1970s	**Genetic Algorithm 遗传算法**	John Holland 提出模拟自然进化过程来求解优化问题
1990s	**Particle Swarm Optimization 粒子群优化**	James Kennedy （社会心理学家） & Russell Eberhart （电气工程师）提出，模拟 **鸟群飞行行为** 求解优化问题（ 1995 ）
1990s	**Genetic Programming 遗传编程**	John Koza 提出用遗传算法 **进化计算机程序代码**
1990s - 2000s	**Neuroevolution 神经进化**	Kenneth Stanley & Risto Miikkulainen 提出 **NEAT** （ NeuroEvolution of Augmenting Topologies ）算法
2016 Applications in the New Era	Uber & OpenAI 展示进化计算的强大能力	EC 进入新世代工业应用
Now 至今	ETO / NAS 等前沿方向持续发展	与深度学习、迁移学习融合

2.2 The Four-Step Loop

Step	Operation	Description
1	**Initial population** 初始种群	从一个初始种群（一组候选解）开始 Start with an initial population
2	**Evaluation & Selection** 评估与选择	评估并选择出更适应的个体 Evaluations and selection performed to identify fitter individuals
3	**Crossover & Mutation** 交叉与变异	通过交叉和变异进行繁殖（ reproduction ） Crossover and mutation carried out for reproduction
4	**Iterate** 迭代	迭代进化直到满足停止准则 Iteratively evolves until stopping criteria are satisfied

2.3 Three Advantages

EA 的种群移动趋势本身携带**可跨问题迁移的信息**，可以用来引导搜索走向有希望的最优区域。*The population movement patterns carry transferable cross-problem information to guide the search towards promising optimal regions.*

特点：

1. **Strong global search capability 强大的全局搜索能力**

不依赖梯度，适合黑盒问题；

2. **Easy to implement 易于实现**

优化逻辑简单：初始化 → 评估 → 选择 → 交叉变异 → 迭代，易于实现；

3. **Self-adaptation and self-learning 自适应与自学习**

过程信息可复用：一次搜索中往哪里走更好的经验，可以被搬到同类问题上（Part 7 ETO范式）。

Part 3 Particle Swarm Optimization

3.1 Core Idea

PSO is a robust stochastic optimization technique based on the movement and intelligence of swarms.

PSO 是一种鲁棒的随机优化技术，基于群体的运动与智能

PSO 将社会交互（social interaction）的概念应用于问题求解。

术语 Term	含义 Meaning
Swarm 群体	由多个particles（粒子）构成，在搜索空间中移动寻找最优解。
Particle 粒子	N 维空间中的一个点，依据自身飞行经验与其他粒子的飞行经验调整其飞行。
pbest 个体最佳	该粒子自身迄今到达过的最佳位置（ personal best ）。 The best solution achieved so far by that particle.
gbest 全局最佳	该粒子邻域内任意粒子迄今取得的最佳值（ global best ）。 The best value obtained so far by any particle in the neighborhood.

The basic concept of PSO lies in accelerating each particle toward its p-best and g-best locations, with a random weighted acceleration at each time step.

PSO 的基本原理：在每一时间步，以随机加权加速度将每个粒子朝其 pbest 与 gbest 位置加速。

3.2 Update Equation

$Vid = w ・ Vid + c1 ・ Rand() ・ (pid − xid) + c2 ・ Rand() ・ (gd − xid) (1) $

$xid = xid + Vid (2) $

1. *w* ・ *Vid* **惯性项**（inertia）：保持原有运动趋势；
2. *c*1 ・ *Rand*() ・ (*pid* − *xid*) **认知项**（cognitive / personal）：向自身历史最佳靠拢；
3. $c2 ・ Rand() ・ (gd − xid) $ **社会项**（social）：向群体最佳靠拢。

Symbol	Meaning
V id	每个粒子在每一维上的**速度**
i	粒子索引
d	维度
w	**惯性权重**

Symbol	Meaning
c 1 , c 2	常数，分别对应**认知与社会**分量
Rand()	[0, 1] 之间的随机数
p id	每个粒子的 **最佳位置**
g d	**群体最佳位置**
x id	每个粒子在每一维上的**当前位置**

3.3 Parameters Tuning

Parameter	Range	Effect
惯性权重 w Inertia Weight	[0.5, 1.5]	大w促进**全局搜索(global search);** **小w促进局部搜索（local search)。** 沿运行过程**线性递减**w比固定值表现更好。
c1,c2	[0, 4.0]	若c2>c1：社会信息影响更大; 若c1>c2：粒子更受个体最佳影响而非全局最佳。 c_2 大听社会，c_1 大听自己。
Rand()	[0, 1]	均匀随机数

3.4 Comparison with other EC Techniques

特征 Feature	PSO	GA / EP / ES
选择操作Selection	**没有**选择操作	有选择操作
种群存续	所有粒子**始终保留**在种群中	个体可能被淘汰
交叉/变异crossover / mutation	**不需要**	核心算子
全局vs局部平衡	通过**惯性权重 w** 调节	通过其他机制调节

Tips: PSO 用速度 + 向最佳位置加速代替了 GA 的选择 + 交叉 + 变异

没有个体被淘汰，没有基因重组，全部通过连续的位置更新实现。

3.5 Why Particle?

3.5.1 Particle vs Point

问题：为什么PSO 里的个体叫 particle（粒子），而不是 point（点）

原因：粒子携带位置、速度与加速度，而点只有位置。

正因为粒子带着运动的惯性，它才能把上一次的运动方向保留下来（即公式中的 $w・Vid $惯性项），并在个体记忆（pbest）与群体信息（gbest）的共同作用下不断修正前进方向。

3.5.2 In-Class Demo

Step	Description
初始配置	20个粒子，**随机或规则排布**，并赋一个**很小的****随机初速度**
运动规则	每个粒子同时追踪**pbest（自己的历史最佳）与gbest（群体的历史最佳）**

Step	Description
过程现象	粒子逐渐**漂移到有希望的区域**；公式中的随机项让粒子**不会过早停滞在局部极小**
最终结果	收敛到（近似）全局最优，**全程不需要梯度信息**

Tips:

1. 因为不依赖梯度，PSO 天然适用于**梯度不可得或不可微（gradient-free）**的实际问题
2. 这里的随机成分不是为了增加随机性，而是**防早熟，**它让粒子有概率跳出当前正在聚集的区域，这与模拟退火的按概率接受更差解是同一种思路的两种表达。

Part 4 Differential Evolution 差分进化

4.1 Background

Item	Content
提出者Authors	**Price&Storn**，1995年
定位Positioning	被认为是**实数函数优化**最强大的进化算法之一
出处Paper	*“DifferentialEvolution—asimpleandefficientadaptiveschemeforglobaloptimizationovercontinuousspaces”,1995.*
竞赛成绩 Competition	1996年在Nagoya举行的首届国际进化计算竞赛中获得第3名
性能对比 Performance	在**34函数测试集**上**优于GA和PSO**（Vesterstrom&Thomsen,2004）
持续表现	在IEEECEC系列会议中，对动态、多目标、约束、多模态等各类优化问题持续展现卓越性能

4.2 Main Idea

利用下式生成试验向量（trialvectors,v）:

$v = x_r1 + F \cdot (x_r2 - x_r3)$

其中 x_r1, x_r2, x_r3从当前种群中随机选取，F称为缩放因子（scalingfactor）。

优势：DE的自适应变异引擎，具有随种群收敛自动调节的性质，是DE**参数极少、易用**（few parameter sto be tuned）的根本原因。

解读：DE的差分体现在 $ x_r2,x_r3 $：用两个随机个体之差作为扰动方向与步长。这种扰动幅度不是人为设定的固定步长，而是有当前种群的分散程度决定的。

种群越分散，$(x_r2 - x_r3)$步长越大，偏向探索（exploration）;

种群收敛，$(x_r2 - x_r3)$步长自动减小，自适应地平衡探索与开发（exploitation）。

4.3 DE Algorithm

步骤	伪代码Pseudocode
1	用**均匀随机值**初始化种群X

步骤	伪代码Pseudocode
2	根据适应度函数确定X中各成员的质量
3	**repeat**（重复）
4	对种群中的每个父代Xc：
5	随机选择差分对Xa和Xb
6	计算X′ c=Xc+F・(Xa−Xb)
7	计算X″ c=Crossover(Xc,X′ c)
8	确定X″ c的质量
9	**if **Xc″ 比Xc更好; **then**Xc″ 进入新种群; **else **Xc进入新种群
10	**endfor**；新种群替换旧种群
11	**until**种群收敛或达到代数上限

Tips: DE采用1-to-1**贪婪选择（greedyselection）**，即只有当试验向量 X_c优于父代 $ X_c $时才替换，否则保留父代。优势如下：

1. 种群规模恒定：每个父代对应一个试验向量，种群大小不随迭代变化
2. 适应度不退化：只有变好才接受，因此种群质量单调不降（fitnessneverdegrades）
3. 实现极简：不需要轮盘赌、不需要排序，一次比较即可
4. 与GA的差别：GA（先选父代→再重组）；DE是每个父代都给一次机会（只用进行一个减法和比较）选择与繁殖耦合更紧。

4.4 DE Mutation Scheme 变异方案命名约定

Symbol	Meaning
DE	DifferentialEvolution（差分进化）
x	表示**被扰动向量**的字符串（thevectortobeperturbed）
y	用于扰动x的**差分向量个数**（numberofdifferencevectors）
z	所采用的**交叉类型**（typeofcrossover）

Part5 Memetic Algorithms

5.1 Development in EC

Era	Method
1960s	Evolutionary programming 进化规划
1970s	Genetic algorithm 遗传算法
1970s	Evolutionary strategies 进化策略
1990s	Evolutionary algorithms 进化算法
1990s	Genetic programming 遗传编程

Era	Method
1990s	**Memetical gorithms 模因算法**

5.2 Definition & Metaphor

模因算法是一类同时整合全局搜索（交叉+变异）与局部搜索的进化算法。

MA are evolutionary algorithms that are in corporate global search (crossover+mutation) and local searches simultaneously.

层次	机制	对应算法成分
跨代 Cross-generation	**进化适应** Evolutionary adaptation	遗传算法GA
个体生命周期 Within a lifetime	**个体学习** Individual learning	局部搜索器 problem-specific local searchers

5.3 Preservation of Diversity

局部搜索（local search）：一种使算法能够在**当前个体的邻域**内获得更好解的机制。

Mechanism that enables the algorithm to obtain better solutions within the neighborhood of the current individual.

问题：在MA中使用局部搜索很容易导致多样性丧失（lossofdiversity）因为所有点/个体都被移动到同一个局部最优。

Strategy	Description
限制局部搜索比例	只允许**一小部分种群**进行局部搜索 Allow only a small fraction of the population to undergo local search
使用多种局部搜索器	引入**不同的搜索空间与不同的局部最优** Use multiple local searchers to induce different search spaces and distinct local optima
选择算子去重	在选择算子中**防止重复**个体 Prevent duplicates in the selection operator
新的接受准则	为局部搜索引入新接受准则，如**Boltzmann方法**（类似模拟退火） New acceptance criteria, e.g. Boltzmann method (similar to Simulated Annealing)

5.4 Utilizing Domain Knowledge

GA 可以通过引入问题特定的领域知识而更具针对性。领域知识表现为：

- Specialized crossover 专用交叉算子
- Specialized mutation 专用变异算子
- Problem specific representations 问题特定的表示
- Problem specific local search 问题特定的局部搜索让 EA 在具体任务上表现更好，就催生了模因算法（Memetic algorithms）

5.4.1 General Workflow of MA

步	流程 Step
1	Initialize population 初始化种群
2	Evaluate each candidate 评估每个候选
3	Selection and Recombination 选择与重组
4	Mutation 变异
5	**Improve via local search** 通过局部搜索改进 ← MA 特有
6	回到步骤2，循环（Improve via local search ）

领域知识的注入位置是可选的 Multiple Injection Points

MA的工作流保留了标准进化循环（选择→重组→变异），差别只在第五步（评估之前多一步局部搜索）。但这一步的插入位置并不唯一：

- 可以在变异之后注入；
- 也可以在重组之后、评估之前注入；
- 还可以通过专门的进化算子把领域知识写进搜索过程。

具体插在哪，取决于问题的性质与算力预算。所以MA的本质就是全局评估+局部精炼的循环。

5.5 Performance of EA

可见，早期进化算法在通用性上强，但在特定问题上不如专用算法；加入领域知识（→ MA）后可显著提升针对性。

方法类型	适用面	峰值表现
Generalist 通用方法（纯 EA ）	广	中等但稳定
Specialist 专用方法	窄	很高
EA + Domain Knowledge （ → MA ）	**较广**	**较高** ← 兼顾两者

Tips: MA 是把**本问题**的知识注入，而 Part 7 的 ETO 是把**其他相似问题**的知识注入。两者动机相同，知识来源不同。

Part 6 Multi-objective Evolutionary Algorithms

6.1 Rational and Motivation

1. 许多**现实问题**有多个（可能相互**冲突**的）目标需要优化。
2. 有些问题被**转化（或聚合）**成单目标问题（SOP），从而用单目标优化技术求解。
3. 把多目标问题（MOP）转成 SOP 需要**先验知识**来设定各目标的相对重要性。
4. 用**多目标框架**定义问题更**通用**，并为决策者提供不同目标之间的**权衡（tradeoffs）**。
5. **多目标优化（multi-objective optimization）又称多准则优化（multi-criteria optimization），目标是找到一组向量**，在满足给定约束的同时优化多个目标函数。

6.2 Classification of MOP

对一个 MOP，目标数 M 必须**大于 1**。根据约束可分类为：

不等式约束J	等式约束K	x_i^L	x_i^U	MOP Type
0	0	$-\infty$	∞	**Unconstrained MOP** 无约束
0	0	C_1	C_2	**Bound constrained MOP** 边界约束
> 0	0	x	x	**Inequality constrained MOP** 不等式约束
0	> 0	x	x	**Equality constrained MOP** 等式约束

其中 C1、C2 为常数。为简化，用 S 表示**可行决策空间**（满足约束的区域）。MOP 可重写为：

$minx \in S[f_1(x), f_2(x), \cdots, f_M(x)]$

6.3 Dominance Relation 支配关系

Definition：**解 **$x_1 * * 支配（*dominate*）另一个解* *x_2 $，记作 $ x_1 ≼ x_2$，当且仅当以下两个条件同时成立：

1. x_1在**所有目标**上都不比 x_2差（no worse in all objectives）
2. x_1在**至少一个目标**上严格优于 x_2（strictly better in at least one objective）

$x_1 ≼ x_2 \iff$ $ f_i(x1) ⩽ f_i(x2) ∀i ∈ {1,⋯,M} $ 且 $\exists j \in 1, ..., M | fj(x1) < fj(x2)$

6.4 Three Key Definitions

Definition	内容 Content
非支配集 ** **Non-Dominated Set	在一组解P中，非支配解集P′ 由**不被 P 中任何成员支配**的解组成。 Among a set of solutions, P, the non-dominated set P′ are those not dominated by any member of P.
Pareto 最优集 **Pareto Optimal Set**（决策空间）	在**可行目标空间**中**非支配**的解的集合。 The set of solutions that are non-dominated in the feasible objective space.
Pareto 最优前沿 **Pareto Optimal Front**（目标空间）	在**可行目标空间**中的**非支配解集** 。 The set of non-dominated solutions in the feasible objective space.

Two-Objective Example：两个待最小化目标 f_1 与 f_2，Pareto 最优前沿是**一条最优权衡曲线（trade-off curve）**，位于**不可行区域**（unfeasible region）的边界上。

which one is better? Pareto 前沿上的点相等，所以**互不支配**，无法仅凭目标值判断谁更优，需依赖决策者偏好。

6.5 Pareto Ranking

Why is sharing needed for MO optimization? 为什么多目标优化需要共享机制?

答：Pareto 排序只关注收敛，会让种群聚集到前沿的某一小段；共享函数通过降低拥挤区域个体的适应度，迫使种群散布在整个前沿。

6.6 Mainstream MOEAs

Evolutionary Multi-objective Optimization (EMO)是进化算法用于求解多目标优化问题的应用。MOEA 用于逼近多目标问题的 Pareto 最优前沿。

Algorithm	Full Name
MOGA	Multi-objective Genetic Algorithm 多目标遗传算法
NSGA-II	Non-Dominated Sorting Genetic Algorithm II 非支配排序遗传算法 II
IMOEA	Incrementing Multi-objective Evolutionary Algorithm 增量式多目标进化算法
MOEA/D	Multiobjective Evolutionary Algorithm based on Decomposition 基于分解的多目标进化算法

这些算法主要在以下方面不同：适应度分配方案（fitness assignment scheme）、多样性保持（diversity preservation）、共享与精英策略（sharing and elitism strategies）。

6.7.1 General MOEA Flow

步	流程Step
1	Initialization 初始化： $x_1, x_2, \cdots, x_N$
2	Fitness Evaluation 适应度评估： $f(x_1), f(x_2), \cdots, f(x_N)$
3	Mating Selection 交配选择
4	Recombination / Crossover 重组 / 交叉
5	Mutation 变异（ Perturb the generated offspring solution 扰动生成的子代解）
6	Fitness Evaluation 适应度评估
7	Selection 选择（ Select individual to survive 选择存活的个体）
8	Termination 终止检查（ Check terminated condition ）

6.7.2 Issues&Strengths

EMO needs to address these issues	Why EA is powerful for MO
适应度分配、偏好 （Fitness assignment, preference）	基于种群，能搜索**全局权衡** （global trade-off）
良好散布、均匀分布、最小邻近 （Good spread, uniform distribution, minimum proximity）	鲁棒，适用问题范围广 （Robust and applicable to a wide range of problems）
多目标、多约束 （Many objectives, constraints）	能处理**不连续与多维**问题 （discontinuous and multi-dimensional）
噪声、动态环境、鲁棒优化 （Noise, dynamic landscape, robust optimization…）	

6.6 What to change in Simple GA for MO optimization?

修改适应度计算（Modify the Fitness Computation）：

- 通过Pareto排序强调非支配解 → 促进收敛性（convergence）
- 通过共享函数（sharing function）强调不拥挤的解 → 促进多样性（diversity）

6.8 A House-Buying Example

6.8.1 House Buying as a MOP

用买房来解释什么叫做相互冲突的多个目标：

目标 Objective	冲突 Conflict
地段好坏	地段越好 → 价格越高
价格高低	价格越低 → 地段与房屋条件往往越差
房屋条件	条件越好 → 价格越高

没有唯一最优解，只有不同的权衡（trade-off）。

把这三个目标加权求和成单目标再优化，就必须先知道地段 vs 价格 vs 条件各自的权重，而这恰恰是决策者事先给不出的先验知识（prior knowledge）。

MOEA 的价值：不做加权聚合，直接求出一整组权衡解（Pareto 前沿），把最终选择留给决策者。

6.8.2 Partial Ranking

偏序排序（partial ranking）的两个条件

调整 Adjustment	作用 Effect
偏序排序 Partial ranking	优先保留**非支配解，促进收敛（convergence）**； 非支配个体rank = 1，其余个体 rank=1+**支配它的个体数量**
共享机制 Sharing	优先保留**不拥挤的解** → 促进**多样性（diversity）**

6.8.3 Why Sharing Is Necessary

Why sharing is needed for MO optimization?

只有Pareto排序	加上Sharing共享机制
只强调**收敛（convergence）**	同时强调**多样性（diversity）**
种群会**聚集到Pareto前沿的某一个点或一小段**	通过**降低拥挤区域个体的适应度**，迫使种群**散布在整个前沿**
结果：只拿到一个折中方案	结果：拿到**完整的前沿近似**

偏序排序负责往前沿推，共享机制负责在前沿上散开，两者缺一不可。这也解释了为什么多目标优化比单目标更复杂：单目标只需回答哪个最好，多目标还要回答这一组解分布得够不够开。

6.9 Limitation and Observation of EA

1. EA 中独立的搜索与迭代过程常导致缺乏有效引导、搜索效率低。
2. 当前的进化优化器往往从零开始搜索，不利用求解相似问题的经验。
3. 任务/问题之间的共同信息（common information）若被恰当利用，可有效帮助问题求解。

Sphere 函数优化过程中找到的解，**有可能**被用于帮助更复杂的 **Rastrigin 函数**的优化，这正是知识迁移的切入点，直接引出 Part 7 的 ETO。

Part 7 Evolutionary Transfer Optimization (ETO)

7.1 Motivation and Definition

Motivation: Knowledge Learning and Transfer.

人类具备跨任务学习与迁移知识的能力，机器可以通过微调（fine-tuning）复用已训练的模型来实现知识迁移。

Definition: A paradigm that integrates evolutionary optimizers with knowledge learning and transfer across related domains for better searching performance.

进化迁移优化（Evolutionary Transfer Optimization, ETO）是一种将进化优化器与跨相关领域的知识学习与迁移相结合，以获得更好搜索性能的范式。

EA + KT（Knowledge Transfer）→ 更好的性能 Performance

Benefit	Description
Head start 更好的起点	从更好的初始点出发
Faster convergence speed 更快的收敛速度	缩短优化过程
Closer approximation 更接近的逼近	更接近最优解

适用问题类型：约束优化问题（Constrained）、动态优化问题（Dynamic）、多目标优化问题（Multi-objective）。

7.3 Why ETO?

ETO aims to guide and accelerate the search process in the new problem space by leveraging information and solutions from similar optimization problems.

ETO 的目标：通过利用来自相似优化问题的信息与解，来引导并加速新问题空间中的搜索过程。

维度 Dimension	目标 Goal
Efficiency 效率	提升搜索效率、 **减少计算资源** Enhance search efficiency and reduce computational resource
Effectiveness 有效性	处理**复杂问题** Handle complex problems
Robustness 鲁棒性	提升**适应性与鲁棒性 **Improve adaptability and robustness** **应对**环境变化 **Environments Changing

7.4 Paradigms of ETO ETO范式

范式 Paradigm	知识来源 Knowledge Source	结构示意 Structure
Evolutionary **Sequential Transfer Optimization 进化序列迁移优化	从**先前已求解的源任务**中捕获的知识，支持后续优化 Knowledge captured from previously-solved source tasks	Source Tasks → Knowledge Base → Target Task

范式 Paradigm	知识来源 Knowledge Source	结构示意 Structure
**Evolutionary **Multitask Transfer Optimization 进化多任务迁移优化	在**多个正在进行**的任务间**共享**的知识 Knowledge shared across multiple ongoing tasks	Task 1 / Task 2 / ⋯ → Knowledge Base（同时进行）
**Evolutionary **Multiform Transfer Optimization 进化多形式迁移优化	把**复杂原任务分解**成更简单的替代源任务，积累经验反哺原任务 Knowledge obtained from alternate formulations of the original task	Formulation 1 / Formulation 2 → Knowledge Base → Target Task
**Evolutionary **Dynamic Transfer Optimization 进化动态迁移优化	从**历史信息**中提取的知识，提升当前任务的表现 Knowledge extracted from historical information	Dynamic Tasks 1 → 2 → ⋯ → t-1 → t，共享 Knowledge Base

Tips: 序列 = 过去任务间；多任务 = 并行任务间；多形式 = 同一任务的不同表述间；动态 = 随时间变化的同一任务。

7.5 Key Issues of ETO

设计新的知识学习与迁移方法对 ETO 而言是必要的。核心三问：

Question	Explanation
What to transfer?	选择**具有跨任务共性（latent inter-task communality）的可迁移知识：解（ solutions ）、特征（ features ）、参数（ parameters ） ⋯⋯ 并挑出最有希望**的候选
When to transfer?	需要**自适应的迁移时机**：迁得太频繁 → **负迁移（negative transfer）**； 迁得太少 → 白白损失正向收益。或不迁移No transfer。
How to transfer?	选择合适的迁移方式，如**直接朴素平移（direct naive translation）**、**精细的拓扑变换（fine topological transformation）**等

负迁移（Negative Transfer）

从不相关或弱相关的任务迁移知识，反而拖累当前任务的优化。ETO的关键难点之一就是只迁该迁的；在不该迁时，果断不迁。这也解释了为什么No transfer本身就是When to transfer的一个合法答案。

7.6 ETO Successes

Application	Result
能源系统优化	运营成本降低 **20%**
污染路径问题	污染扩散减少 **50%**
油藏生产优化	收益增加 **700 万美元**
气动设计优化	升阻比提升 **13%**

7.7 ETO benefits &MA

收益 Benefit	课堂说明 Explanation
更好的初始种群（Head start）	不用随机初始化，而是从一个表现更好的初始种群出发→ 稳健的领先优势
更快的收敛 + 更低的计算成本	复用存储下来的历史搜索经验，避免从零开始
更接近全局最优	借助简单问题到复杂问题之间的知识迁移提升解的质量； 对动态环境把历史状态知识迁移到当前已变化但相似的地形上，增强鲁棒性

Dimension	MA 模因算法	ETO 进化迁移优化
知识来源	**本问题自身**搜索过程中的局部搜索知识	**其他相似优化问题**（跨问题域）的信息
类比	一个人自己反复琢磨同一道题	把做过的同类题的解法搬过来
作用范围	单个优化过程内部	**问题与问题之间**

Part 8 Neural Architecture Search (NAS)

8.1 Development of Neural Architecture

Period	Stage	Representative Models
1943~1980	**Fundamentals** 基础	M-P模型、 MLP（多层感知机）
1990s	**Various architectures** 多种架构	LSTM、LeNet-5、RBM、HNN
2012	**Deep architecture** 深度架构	AlexNet
2014 - 2016	**Deeper models** 更深的模型	VGGNet 、ResNet 、DenseNet
2017	**Novel fundamental model** 全新基础模型	Transformer
Now 至今	**Large models** 大模型	BERT、GPT、Llama

8.2 NAS as an Optimization Problem

神经网络架构设计常可被形式化为一个**优化问题**，神经架构搜索（Neural Architecture Search, NAS）正是为求解该问题而提出的。

NAS问题的三大特性

- Bi-level optimization 双层优化：外层搜架构 arch，内层训练权重 w
- Black-block optimization 黑盒优化：无法求梯度/无解析形式
- Expensive optimization 昂贵优化：每次评估都需训练并验证模型

Component	Content
Search space 搜索空间	A（架构空间）
Search method 搜索方法	强化学习Reinforcement Learning、进化算法Evolutionary Algorithm、梯度方法 Gradient-based
Performance evaluation 性能评估	准确率Accuracy、延迟Latency、计算量Flops、模型大小Model size

8.3 MTNAS：Knowledge Transfer Strategy

问题：多任务场景下（图像分类、图像分割、机器翻译），若对每个任务独立运行 NAS（Task1 → NAS、Task2 → NAS、Task3 → NAS），计算开销巨大且无法复用经验。

思路：让多个任务的架构搜索**共享知识，**这正是ETO思想在NAS上的落地。

Mechanism	Description
Knowledge extraction 知识提取	从搜索过程中提取可复用知识
Knowledge reuse 知识复用	提出一种**局部搜索方法**用于知识复用
Transfer probability adaption 迁移概率自适应	对每次搜索过程**自适应调整迁移概率**： divi：来自**搜索过程自身**的有用程度； di：来自**公共档案（common archive）**的有用程度。

迁移概率示意：0.8 / 0.7 / 0.9 / 0.6 / 0.5 / 0.5，随搜索进程动态变化。

8.4 Results & Ablation Study

Experiments	Conclusion
Search Cost on NAS-bench-201	MTNAS至少实现**2×时间缩减**
Performance on Trans-bench-101	可实现 **2 - 10000× 加速**（2-10000x speedup）
Performance on DARTS	能为**多个任务**找到更好的架构（find better architectures）

Conclusion	Value
NAS-Bench-201 基准	仅需现有SOTA方法**一半的评估次数**即可找到全局最优架构
性能vs时间成本	在性能与时间成本上**均优于**对比方法
消融验证的四个模块	**知识共享 / 知识提取 / 知识复用 / 迁移概率自适应，**每个模块都有效
搜索轨迹可视化	证明方法**快速移动到最优架构**，并借助知识共享**逃离局部最优**

8.5 Visualization

Overview of the search space and search process：可视化展示进化搜索过程如何在搜索空间中探索，以及如何**跳出局部最优（Escape from the local optima）**这是本章所有算法（EA / PSO / DE / MA / MOEA / ETO / NAS）的共同主题。

8.6 NAS Loop & MT-NAS Archive 交互架构与共享机制

8.6.1 The Common NAS Loop 通用交互架构

环节 Stage	内容 Content
定义搜索空间	定义**极其庞大**的候选架构空间（ a massive search space ）
应用搜索方法	常用**进化算法**， 也有强化学习、梯度方法
评估采样架构	采样一个架构 → **训练 + 验证** → 得到性能

环节 Stage	内容 Content
反馈指导迭代	把评估结果反馈回去， **指导下一轮搜索**

8.6.2 Two Problems of Single-Task NAS

单任务架构在多任务场景中的困境

1. 成本线性增长：对每个目标任务独立跑一遍 NAS，任务数越多，总开销线性上升；
2. 没有万能架构：由于数据分布漂移（data distribution shifts），不存在一个架构能在所有任务上都最优。

结论：把**知识迁移**引入搜索空间，是降低多任务 NAS 成本的**有希望的方向**。

8.6.3 The Shared Archive Mechanism 共享机制

机制 Mechanism	说明 Explanation
并行搜索 Parallel search	基于EA的**并行搜索范式**，同时为不同任务搜索各自的架构
公共档案 Common archive	设置一个**公共档案**，存放**所有并行搜索过程中**发现的高性能架构
选择性注入 Selective injection	**有选择地**把这些有价值的架构加入**每一个**独立的搜索过程，从而加速整体优化

Tips: ETO 思想在 NAS 上的落地：每个任务各搜各的，但**把别人的好架构放进一个共享仓库**，谁需要谁取。

Part9 Exercise3

9.1 Exercise1：GA 交叉子代适应度计算Crossover

三条染色体 00001, 01000, 11010 用于最大化 f(x, y) = x + y。x 由前 2 位表示，y 由后 3 位表示。第 1 与第 2 条染色体交叉，交叉点在第 4 位与第 5 位之间；第 1 与第 3 条染色体交叉，交叉点在第 2 位与第 3 位之间。请计算子代的适应度值。

注意解码时用二进制

答案汇总：4 个子代适应度分别为 **0, 2, 2, 4**（对应 00000、01001、00010、11001）。注意交叉时**前段取父代 1、后段取父代 2**（反之亦然），不要混淆分段归属。

9.2 Exercise2: Selection (Rank & Fitness Weighting) 选择算子

题目：一个含 4 个个体的二进制 GA，适应度如下（最大化问题）：

Q1：(a) 秩加权选择的概率表

(b) 需选 2 个个体，随机数为 0.15, 0.82

为了确定每个染色体对应的区间，需要计算累积概率：

- 染色体 2: [0, 0.4)，染色体 4: [0.4, 0.7)，染色体 1: [0.7, 0.9)，染色体 3: [0.9, 1.0)

所以选择染色体2和1。

Q2：(a) 适应度加权选择的概率表

适应度总和 = 169 + 576 + 64 + 361 = **1170**

Q2：(b) 需选 2 个个体，随机数为 0.15, 0.82

累积区间：#2 ∈ [0, 0.49)，#4 ∈ [0.49, 0.80)，#1 ∈ [0.80, 0.94)，#3 ∈ [0.94, 1.0)。

Chromosomes 2 and 1（与 Q1 结果相同，但**概率分布完全不同，**注意区分）。

9.3 Exercise3: TSP 变异 + 秩加权选择

题目：两条染色体 ACDBE, ADECB 用于求解旅行商问题，5 个节点的距离矩阵如下。**变异算子**为交换两个变异点的位置。染色体 ACDBE：变异点为第 **2** 与第 **4** 位；染色体 ADECB：变异点为第 **5** 与第 **4** 位。若从父代与子代中按**秩加权选择**选出 2 条染色体，随机数为 0.3, 0.6，会选中哪两条?

路径成本计算 · Cost Calculation

类型	染色体	成本计算 Cost	成本	秩
Parent	ACDBE	AC + CD + DB + BE + EA = 31+15+40+7+10	**103**	2
Parent	ADECB	AD + DE + EC + CB + BA = 8+5+20+9+56	**98**	3
Offspring	ABDCE	AB + BD + DC + CE + EA = 56+40+15+20+10	**141**	1
Offspring	ADEBC	AD + DE + EB + BC + CA = 8+5+7+9+31	**60**	4

Tips: TSP 是**最小化问题**，但 GA 需要最大化适应度。因此**成本越低 → 秩越高**：最优的 ADEBC（60）秩 4，最差的 ABDCE（141）秩 1。

秩加权概率表 · Rank Weighting Probability

Chromosome	Cost	Rank	Probability
ADEBC	60	4	4/(4+3+2+1) = **0.4**
ADECB	98	3	3/(4+3+2+1) = **0.3**
ACDBE	103	2	2/(4+3+2+1) = **0.2**
ABDCE	141	1	1/(4+3+2+1) = **0.1**

累积区间：ADEBC ∈ [0, 0.4)，ADECB ∈ [0.4, 0.7)，ACDBE ∈ [0.7, 0.9)，ABDCE ∈ [0.9, 1.0)。选中 **ADEBC 和 ADECB**。

9.4 Exercise4: 模式定理

题目：考虑繁殖、交叉与变异在遗传算法中的单独与联合作用，推导模式定理以考察种群中各种模式（相似模板）的增长率。第 0 代种群如下：异概率 Pm = 0.01，交叉概率 Pcross = 1。用模式定理估计第 1 代中形如 1**** 的模式（schemata）的期望数量。

因为该模式**定义长度短（d=0）、阶数低（o=1）、适应度高于平均（15 > 12.5）**，所以它获得增长（2 → 2.376）。

Tips:

1. 把 1**** 的 **d(S)** 误算成非 0（只有确定位才计定义长度，单个确定位 → d = 0）
2. 把 fit(S) 当成全体平均（应为**匹配该模式的串**的平均）
3. 忘记乘交叉破坏与变异破坏两个惩罚因子

Part10 Overall

10.1 关于模因算法 On Memetic Algorithms

**"General AI is a good starting point, but adding problem knowledge makes it much stronger." **通用方法是很好的起点，但加入「问题知识」后，它会强得多。

对应知识点：MA = 全局进化搜索 + 问题特定的局部搜索；1990s 的 generalist / specialist / +domain knowledge 三分法。

为什么重要：它一句话概括了 MA 的核心价值，也预示了 ETO 的思路，**引入外部知识。**

10.2 关于进化迁移优化 On ETO

"We human, we do not learn everything from scratch."我们人类，并不是什么都从零学起。

对应知识点：ETO 的核心动机——传统 EA **从零开始搜索**，不复用相似任务的已有经验；而人类会**跨任务迁移知识。为什么重要**：这是理解 ETO「为什么存在」的最直观入口，也是 What / When / How 三问的前提。

10.3 关于遗传算法 On Genetic Algorithms

"Genetic algorithms have been proved to be very powerful in escaping from the local optimum to search for the global maximum."遗传算法已被证明非常擅长逃离局部最优，去搜索全局最大值。

对应知识点：GA 相对传统启发式搜索（A* 等）的核心优势。

注意：是**擅长逃离局部最优，不是保证全局最优。**

10.4 串讲 The Through-line

GA 擅长跳出去（10.3）→ 但通用方法还不够强，要加问题知识（10.1）→ 而知识不只来自本问题，还能来自别的相似问题（10.2）。

这三步正好对应 Part 9（GA 定位）→ Part 5（MA）→ Part 7（ETO）的推进逻辑。

Lecture 4

---

part 1 Neural Network

1.1 Definition

A massively parallel distributed processor

Storing experimental knowledge and making it available for use

神经网络是一个大规模并行分布式处理器，天生具有存储经验知识并使其可用的倾向

Acquire knowledge by the network through a learning process

Store knowledge by the inter-neuron connection strengths known as synaptic weights

知识是通过学习过程获得的

神经元之间的连接强度，即突触权重（synaptic weights），用来存储知识

ANN: **parameterized computational nonlinear** algorithms for numerical data/ signal/ image processing.

人工神经网络是**带参数的、可计算的非线性算法**，用来处理（数值型）数据 / 信号 / 图像

Tips: **parameterized**（带参数 → 所以「学习」= 调参）与 **nonlinear**（非线性 → 所以能拟合真实世界的复杂数据；也让误差面变得非凸，见第 9.3 节的局部极小

1.2 Properties and Capabilities

1.2.1 High computational power

Generalization: producing reasonable outputs for inputs not encountered during training/learning.

Massively parallel distributed structure

1.2.2 Useful properties and capabilities

Nonlinearity: most physical systems are nonlinear.

Adapativity : adapt their **synaptic weights** to changes in the environment.

Fault tolerance: if connecting links are damaged, the response may still be OK.

Tips: 容错的来源是**分布式存储**（distributed nature of information stored in a network），而不是，网络足够大所以坏几个无所谓。

性质 Property	英文原文要点	含义与意义
高计算能力 High computational power	embodied in two core aspects	体现在两点：**泛化 + 大规模并行分布式结构**
泛化 Generalization	Producing reasonable outputs for inputs **not encountered during training**	对**训练时没见过**的输入也能给出合理输出 —— 这才是能用的前提
大规模并行分布式结构	Has a massively parallel distributed structure	信息分布存储在许多单元上，支持大批量信息同时处理
非线性 Nonlinearity	Most physical systems are nonlinear	大多数物理系统是非线性的；线性模型表达不了
自适应性 Adaptivity	built-in capability to adapt their **synaptic weights** to changes in the environment	环境变了就调突触权重 —— 这是「在线学习」的基础
容错性 Fault tolerance	If a neuron or its connecting links are damaged, the overall response **may still be ok**	因为信息是**分布式存储**的，坏掉个别神经元或连接，整体响应仍可接受

1.3 Biological Neural Networks

1.3.1 Biological neurons vs.silicon logic gates

Biological neurons, the basic building blocks of the brain, are slower than silicon logic gates. So it makes up for the slow rate of operation with 2 factors;

**数量巨大: **A huge number of nerve cells(neurons) and interconnections.

**单个神经元的功能远比一个逻辑门复杂: **The function of a biological neuron seems to be much more complex than that of a logic gate.

**1.3.2 生物神经元的结构 **

胞体 cell body：信号处理的核心；

轴突 axon：一根细长的管子（tubular axon），分成多个分支，末端是**终末小球（end bulbs）**，几乎触到其他细胞的树突；

树突 dendrites：大量毛发状分支，形成围绕胞体的极细纤维丛（a very fine filamentary brush）

突触（synapses）的三个要点：

是**终末小球与树突之间的小间隙**（Small gap between an end bulb and a dendrite）；

是神经元之间相互作用（interactions）的**基本结构与功能单位**；

可以对接纳信号的神经元施加兴奋（excitation, active）或抑制（inhibition, inactive）

1.4 Synapse to Weight

生物学习机制和人工学习机制

生物侧 Biological	人工侧 Artificial	对应关系
突触的强度可随激活情况调整（adjustable, activation-dependent strength）	连接权重 w_kj	**突触强度 ↔ 权重**：这是全部学习发生的地方
兴奋 / 抑制（excitation / inhibition）	权重的**正 / 负号**	正权重 ≈ 兴奋性连接；负权重 ≈ 抑制性连接
学习与记忆的生物学基础	训练过程（training）改变权重	「学习」= 把权重调到合适的值
神经元的激活与否	激活函数 $\phi(\cdot)$ 的输出	决定这个单元是否「被激活」向下游传递信号

tips：人工神经网络里**唯一会被学习改变的东西，就是连接权重**（以及每个神经元的偏置）。网络结构、激活函数在训练开始前就定死了。反向传播，本质上就是在算每个权重该改多少。

Part 2 Model of a Neuron

2.1 Three Basic Components

A set of synapses or connecting links: characterized by a weight or strength of its own.

一组突触 / 连接：每条连接都有自己的权重或强度

tips：这里的权重是**可调的**，这正是学习发生的地方。

An adder: sum the input signals, weighted by the respective synapses of the neuron, for a linear combiner.

一个加法器：把经过各自突触加权的输入信号求线性组合

An activation function: limit the amplitude of the neuron output, typically[0,1]or[-1,1].

一个激活函数：用来**限制神经元输出的幅度**

2.2 Mathematical Formulation

符号	含义	说明
$x_1, x_2, \cdots, x_m$	输入信号	一共 m 个输入，即输入数据的**维度**（dimension of input data）
w_k1, w_k2, ⋯, w_km	神经元 k 的**突触权重**	每个输入对应一个权重，共 m 个
u_k	线性组合器输出	$u_k = \Sigma\ w_{kj}\ x_j$
b_k	偏置 bias	一个与输入无关的常数项，**移动激活阈值**
$\phi(\cdot)$	激活函数	限制输出幅度、引入非线性
y_k	神经元的输出信号	$y_k = \phi(u_k + b_k)$

2.3 Effect of bias

偏置 b_k 的作用：对 u_k 做一次仿射变换（affine transformation）

2.3.1 定义

v_k = u_k + b_k，这个量被称为**诱导局部场（induced local field）**。

2.3.2 几何含义

b_k 把激活函数的输入**整体平移**，等价于**移动了神经元的激活阈值**（shifts the activation threshold）在Part4.7中体现。

- **三个情形**（slide 10 的三张图）：b_k > 0 曲线左移（更容易被激活）；b_k = 0 不变；b_k < 0 曲线右移（更不容易被激活）
- **一个细节**：当 u_k = 0 时 v_k = b_k ，也就是说 单看偏置就能知道**没有任何输入时这个神经元的起点在哪里**

2.4 Asborbing bias

改写做法：**额外增加一个输入 $x_0 = +1$**（固定输入，fixed input），并令它的权重 w_k0 = b_k。

于是偏置就变成了一条**输入恒为 1 的特殊连接**（图注写作 Fixed Input, x0 = +1），不再需要单独的加法项。

好处：记号统一（少写一个 b_k）、**实现统一**（所有连接一视同仁，权重向量就是一个整体）。

改写前 Before	改写后 After
输入向量 $x = (x_1, \cdots, x_m)^T$	输入向量 $x' = (x_0, x_1, \cdots, x_m)^T$，其中 $x_0 = +1$
权重向量 $w = (w_1, \cdots, w_m)^T$	权重向量 $w' = (w_0, w_1, \cdots, w_m)^T$，其中 $w_0 = b$
y_k = ϕ(Σ_{j=1}^{m} w_kj x_j + b_k)	y_k = ϕ(Σ_{j=0}^{m} w_kj x_j)

Part 3 Activation Functions

3.1 Threshold Function

阈值函数

优点：简单，用符号判断。感知机就是它。

缺点：不可微，在0处不连续，其余位置导数恒为0。

后果：与梯度下降训练不兼容。梯度要么为0，要么不存在。

tips：为什么不能只用阈值函数？阈值函数**不能**用于多层网络的训练。

it adopts a **nonlinear (differentiable)** activation function for each neuron

所以 MLP 与反向传播，每个神经元**必须换成非线性且可微的激活函数**

3.2 Sigmoid and its Derivative

其中 **a 是斜率参数**（slope parameter），且 a > 0、$-\infty < v < \infty$。极限值为 **0 和 1**

3.2.1 三条性质

1. **严格单调递增**（Strictly increasing function）
2. **渐近逼近饱和值**（Asymptotically approach the saturation values）
3. **连续且可微**（Continuous & differentiable everywhere）

3.2.2 Example

1. **只要知道神经元的输出 $\phi(v)$，就能直接算出导数，不需要再单独求导 **

反向传播能把梯度算得又快又省的关键

2. **ϕ' 在 $\phi(v) = 0.5$ 时取最大值**（a/4）
3. **ϕ' 在 $\phi(v) \to 0$ 或 1 时趋近于 0** —— **梯度消失**（vanishing gradient）的数学根源

3.3 Common Activation Functions

函数 Function	公式 Formula	输出范围	优点 Advantages	缺点 Drawbacks
Sigmoid 逻辑函数	f(x) = 1/(1+e^{-x})	**(0, 1)**	平滑可微、导数可由输出自表达	**梯度消失 vanishing gradient**
Tanh 双曲正切	f(x) = (e^x − e^{-x})/(e^x + e^{-x})	**(−1, 1)**	**零中心（zero-centered）**，梯度流动比 sigmoid 好	两端仍会饱和
ReLU 修正线性单元	f(x) = max(0, x)	**[0, +∞)**	**计算快**；x > 0 时**没有梯度消失**	x < 0 时出现 **dying ReLU**（神经元永久失活）
Leaky ReLU 带泄漏 ReLU	f(x) = max(a · x, x)，a 为小斜率	**(−∞, +∞)**	用小斜率 a 修好 dying ReLU 问题	需要多调一个超参 a
Softmax	softmax(z_i) = e^{z_i} / Σ_j e^{z_j}	**(0, 1) 且和为 1**	输出可直接当**概率分布**	只用于**分类网络的输出层**（见 4.4）

tips：为什么 Tanh 比 Sigmoid 好？零中心（zero-centered）

sigmoid 的输出恒为正，下一层神经元的输入就恒为正，于是每层梯度的符号高度相关，**参数更新会走出「之」字形（zig-zag）路径**，收敛变慢。tanh 的输出关于 0 对称，这个问题就缓解了。

3.4 Softmax

用途：把一个 **logits 向量**转换成**概率分布**（a probability distribution）

使用位置：**分类网络的输出层**（Output layer of classification networks）

性质：所有输出之和为 **1.0**

3.4.1 Example

logits 为 [2.0, 1.0, 0.5, −0.5]

softmax = [0.5977, 0.2199, 0.1334, 0.0491]，逐项为 **[0.598, 0.220, 0.133, 0.049]**

3.4.2 三条性质

1. 平移不变：softmax(z) = softmax(z + c)

所以，logits 的绝对值不重要，只看差值，所以**logits 不是概率。**

2. 保序：z_i > z_j ⟺ p_i > p_j

所以，取最大概率 = 取最大 logit（argmax 不变）

3. 指数放大了差距：z 差 1 时概率比约 e ≈ 2.72 倍

3.5 How to Choose

使用场景	推荐选择	理由
分类任务的输出层	**Softmax**（多类）/ Sigmoid（二类）	需要把输出解释成概率、且（多类时）和为 1
回归任务的输出层	**恒等函数（线性）**	输出不该被压进 (0,1)，需要实数范围
隐藏层（现代默认）	**ReLU / Leaky ReLU**	计算快、正区间不饱和、缓解梯度消失
隐藏层（需要零中心）	**Tanh**	输出关于 0 对称，收敛路径更直
单神经元分类器（感知机）	**阈值函数**	不需要求导，只需要符号判断

tips:

1. **把 Softmax 用在隐藏层** —— 错。Softmax 只在分类网络的输出层用
2. **ReLU 完全解决梯度消失** —— 不严谨。ReLU 只在 **x > 0** 时梯度恒为 1；**x < 0 时梯度为 0**，这正是 dying ReLU 的成因
3. **Sigmoid 已经不用了** —— 在本讲的语境里不成立：课件后面推导反向传播用的**就是 Sigmoid**（因为它的导数能写成 y(1−y)，形式极简）

Part 4 The Perceptron

4.1 Definition and Structure

4.1.1 Definition

a machine that learns, using examples, to assign input vectors(samples) to different classes.

一台通过样例学习、把输入向量（样本）分配到不同类别的机器。

本质是一个神经元：built around a nonlinear neuron（the hard limiter model）

用的3.1的硬限幅器（阈值函数）

4.1.2 Structure

左边是线性部分（Linear，$v = \Sigma\ w_i x_i + b$），右边是非线性部分（Non-linear，硬限幅）

tips：

1. 它是一个线性分类器（linear classifier）：决策边界是一条直线（二维）/ 一个超平面（高维）
2. 为什么用感知机而不是逻辑回归？因为它不需要求导，是神经网络最早的、也是最简单的形态
3. 线性 + 非线性这个两段式结构是全部神经网络的通用模板：先做加权和（线性），再过激活函数（非线性）。感知机的特殊之处只是它的非线性部分换成了**取符号hard limiter**而已。

4.2 Formulation and Decision Boundary

the equation of the hyperplane (in this example, a straight line) as decision boundary for a two-dimensional, 2-class pattern-classification problem.

用超平面（此例中是一条直线）作为 decision boundary，解决二维的两类模式分类问题

Goal

To correctly classify the set of externally applied stimuli $x_1, x_2,\ldots$ Into one of two classes, C_1 and C_2

把外部施加的刺激 $x_1, x_2, \cdots, x_n$正确分类到两个类别 C_1 与 C_2 之一

Difine

输入向量：$x'(n) = [+1, x_1(n), x_2(n), \cdots, x_m(n)]^T$

权重向量：$w'(n) = [b(n), w_1(n), w_2(n), \cdots, w_m(n)]^T$

线性组合器的输出：$v(n) = w'^T(n) \cdot x'(n)$

1. 方程 $w'^T x' = 0$ 在 **m 维空间**（坐标为 $x_1, x_2, \cdots, x_m$）中定义了一个**超平面（hyperplane）**，它就是两类输入之间的**决策面（decision surface）**
2. 二维时退化成一条**直线**（$w_1x_1 + w_2x_2 + b = 0$）
3. **判决规则**：$w'^T x' \geqslant 0$ → 判为 class 1（例如 y = 1）；$w'^T x' < 0$ → 判为 class 2（例如 y = 0）

Tips: 课件用 $w'^T x' \geqslant 0$ 中**大于等于 0** 归到 class 1。这个≥ 还是 >的约定**必须与题目一致，因为 v = 0 恰好落在决策边界上，两种约定对边界点**的判定结果不同

4.3 How to Select the Weights

方法 Method	做法 How	适用场景	局限
离线计算（off-line calculation）	如果问题**相对简单**，往往可以**从问题的规范直接算出**权重向量	逻辑门这类结构已知的问题	复杂问题算不出来
学习过程（learning procedure）	从给定的（训练）输入—输出向量对（**exemplars**，样例）中确定权重，目标是**对训练向量取得最好的分类**	**本讲重点，也是神经网络的通用做法**	需要数据；收敛性依赖数据性质

Tips: 两种方法的关系是可不可解：离线计算要求把边界先想出来再反推权重；学习过程则是把**确定边界**这一步也交给算法。NAND 门是前者（答案一眼可见），学习算法是后者

4.4 Building NAND Gates——example

用感知机实现 NAND 门。所需的输入—输出关系由真值表与相应图示给出

1. 输入模式（向量）分属两类，在输入空间（平面 (x_1, x_2)）中用两种记号标出；
2. 决策平面是一条直线，方程为 $x_1 + x_2 = 1.5$，即 $\mathbf{-x_1 - x_2 + 1.5 = 0}$；其实只要阈值在 (1,2] 之间都行，**感知机的解不唯一**。
3. 为了让权重向量**指向 y = 1 的那一类**，选取 $w = (1.5, -1, -1)$，即 $w' = [b, w_1, w_2] = [1.5, -1, -1]$；
4. 对每个输入向量计算输出信号 y = 1 if v ⩾ 0 else 0，其中 $v = 1.5 \cdot 1 - 1 \cdot x_1 - 1 \cdot x_2$。

Tips:

线性可分：感知机只能处理能用“直线”一刀切开的问题（所以它搞不定 XOR 异或门）。

权重比例：权重和偏置同比例缩放（如全部翻倍），决策直线的位置不变。

只看符号：分类结果只取决于计算结果是正还是负，而不取决于这个数有多大。感知机只看符号，所有权重同乘一个正数都等价。所以 (1.5,−1,−1) 、(3,−2,−2) 、(15,−10,−10) 都表示同一个 NAND 分类器。

4.5 2-D Example

一个感知机（Perceptron）进行二分类的具体实例。它通过几何图形和数学模型直观地解释了感知机是如何工作的。

1. 分类任务

- 第一类（Class 1）：包含点 $(0,0), (0,1), (1,0), (-1,-1)$；第二类（Class 2）：包含点 $(2.1,0), (0,-2.5), (1.6,-1.6)$。目标是找到一条直线，把这两组点分开。

2. 左侧：几何划分（Geometric Partition）

- 取决策线 $x_1 - x_2 = 2$，即 $x_1 - x_2 - 2 = 0$。对应权重 $w = [+1, -1, -2]$（依次为 x_1、x_2 的权与偏置）
- *求和节点v：计算公式为* $v = (+1 \times -2) + (x_1 \times 1) + (x_2 \times -1)$，*即* $v = x_1 - x_2 - 2$
- 原理：这条线把平面切成两半。 在线的右下方，满足感知机输出 1；在线的左上方，满足感知机输出 0

3. 右侧：感知机模型（Perceptron Model）

这部分展示了硬件或算法层面是如何实现的：

- 固定输入（Fixed Input）：它的权重（Weight）是 -2。这其实就是偏置项（Bias, b）
- 激活函数（Hard limiter）：符号 $\phi(\cdot) $表示，如果算出 $ \ge 0$ 就输出 1，否则输出 0。

4. 总结：Output = 1 if $(x_1 - x_2 - 2) \geqslant 0$, and 0 otherwise.

4.6 Equivalent Perceptrons

等价感知机：无限多组权重

硬阈值感知机（hard limiter / sign）的决策边界只由权重“比例”决定，不由绝对大小决定，所以有无穷多组等价权重

$核心点：感知机输出是阶跃函数，不关心 ∣v∣ 多大，只关心正负 $

$ 所有权重同乘一个正数，边界不变： $(w0,w1,w2) \sim \lambda (w0,w1,w2), \lambda > 0$

4.7 Linear Separability

(a) A pair of linearly separable patterns;

线性可分：**当且仅当（至少）**存在一个权重向量 **w**，使感知机能正确完成分类。

(b) A pair of non-linearly separable patterns.

线性不可分：单层感知机**永远无法**解决 XOR。因为把四个点画在平面上，不可能用**一条直线**把 (0,1)、(1,0) 与 (0,0)、(1,1) 分开，**解不存在**。这个问题直到加上隐藏层才被解决，也是神经网络在 1969 - 1980 年代沉寂的直接原因。

逻辑函数	真值表（输出为 1 的组合）	是否线性可分	一条可用的判定线
AND	仅 (1,1) → 1	**是**	$x_1 + x_2 - 1.5 = 0$
OR	(0,1)、(1,0)、(1,1) → 1	**是**	$x_1 + x_2 - 0.5 = 0$
XOR	(0,1)、(1,0) → 1	**否**	**不存在**这样一条直线

4.8 Beyond Two Dimensions

4.8.1 多维空间中的线性可分

当输入维度达到 3 维或更高时，我们不再使用简单的“直线”来划分数据，而是使用超平面（Hyperplane）。

- 超平面方程：在 m 维空间中，超平面的数学表达式为
- 空间划分：这个超平面将 m 维空间划分为两个区域，分别对应不同的类别，不同区域满足不同方程：

4.8.2 维度挑战

- 二维空间：我们可以通过几何构造（即肉眼观察和作图）轻松判断数据是否线性可分，并找到合适的分类器。
- 三维及更高维：随着维度的增加，直观的几何判断变得不再直接，也无法通过简单的视觉观察来确定分类界限。

4.8.3 核心问题

如果两个类别在数学上是线性可分的，是否存在一种机械化的程序（即算法）来自动获取感知机（Perceptron）的权重 w?

引出了机器学习中著名的**感知机学习算法**，它能够通过迭代自动找到这个超平面的权重。

Part 5 Perceptron Learning

5.1 Setting up the problem

假设感知机的输入变量来自两个**线性可分**的类 C_1 和 C_2。

已知：若 C_1 与 C_2 线性可分，则**存在**权重向量 w^* 使：

$w'^{*T} \cdot x' > 0$ 对所有 $x \in C_1$ … (1)

$w'^{*T} \cdot x' < 0$ 对所有 $x \in C_2$ … (2)

给定训练集 $X = X_1 \cup X_2$（其中 $X_1 \cap X_2 = \varnothing$），且 $X_1 \subseteq$ class C_1、$X_2 \subseteq$ class C_2。

训练问题（Training problem）：寻找一个权重向量 w'^{*}，使感知机能正确分类训练集 X。注意这个定义里没有最小化误差，**感知机不优化误差**，它只是反复修正直到全部判对（这是**它与后面反向传播最本质的差别**）

5.2 The Learning Algorithm

情形	d	y	(d − y)	更新式
真实属于 C_1，却判成了 2	1	0	**+1**	$w'_k = w'_{k-1} + \eta \cdot x'$ （**朝 x' 方向加**）
真实属于 C_2，却判成了 1	0	1	**−1**	$w'_k = w'_{k-1} - \eta \cdot x'$ （**朝 x' 反方向减**）

感知机学习算法（Perceptron Learning Algorithm, PLA）的数学原理、具体计算步骤以及最终的收敛过程。

5.2.1 感知机权重更新的几何直观

算法的核心逻辑：

- 动态调整：图中展示了决策边界（直线）随着迭代次数 $n = 0, 1, 2, 3$ 的变化。通过不断旋转和移动，直到 $w'^{T}x' = 0$ 这条线把所有 + 与 − 样本分到两侧。
- w' 是决策面的法向量，样本被漏判为正类时，把 w' 朝这个样本的方向拉（$+\eta x'$） 让这个点在新的法向下投影变正；样本被误判为正类时，把 w' 朝它的反方向推 （$-\eta x'$）。
- η 只影响步长，不影响最终能否收敛（对线性可分数据）。
- 更新量正比于 x' 本身，所以离原点远的样本会造成更大的修正。
- 两种情形合起来就是 $w'_k = w'_{k-1} + \eta(d-y)x'$
- 对于**线性可分**的数据集，感知机学习算法会在**有限次训练轮（epoch）内收敛**到一个有效的决策边界。反之，**对线性不可分的数据（如 XOR），算法会持续振荡、永不收敛。**

```
Algorithm Perceptron;
Start with a randomly chosen weight vector w'₀;
Let k = 1;
while there exist input vectors that are misclassified by w'_{k-1}, do
    Let x' be a misclassified input vector;
    Update the weight vector to w'_k = w'_{k-1} + η(d − y)x'
    where η > 0 and → d = 1 if x belongs to class C₁; d = 0 if x belongs to class C₂
    Increment k
end-while
```

5.3 1-D Worked Example

题目给定的初始条件：

学习率 $\eta = 1$；初始权重 $w'_0 = [-1.5, -2]^T$（依次为偏置 b 与权重 w）

样本呈现顺序：$x' = (1, 0.5)^T, (1, -1)^T, (1, 2)^T, (1, -2)^T$

逐步推演（每一步都写全：算 v → 判 y → 比 d → 决定是否更新）：

迭代	x'	真实 d	$v = w'^{T}x'$	判出 y	是否更新	更新后的 w'
0	（初始）	—	—	—	—	[−1.5, −2]

迭代	x'	真实 d	$v = w'^{\top}x'$	判出 y	是否更新	更新后的 w'
1	(1, 0.5)	1	$-1.5 \cdot 1 + (-2)(0.5) = -2.5$	0	**是**（$0 \neq 1$）	$[-0.5, -1.5]$
2	(1, −1)	0	$-0.5 \cdot 1 + (-1.5)(-1) = +1.0$	1	**是**（$1 \neq 0$）	$[-1.5, -0.5]$
3	(1, 2)	1	$-1.5 \cdot 1 + (-0.5)(2) = -2.5$	0	**是**（$0 \neq 1$）	$[-0.5, 1.5]$
4	(1, −2)	0	$-0.5 \cdot 1 + 1.5 \cdot (-2) = -3.5$	0	**否**（$0 = 0$）	$[-0.5, 1.5]$

每次更新用了哪条分支：

1. **迭代 1**：$w'_1 = w'_0 + \eta(1 - 0)x' = w'_0 + (1, 0.5)^{\top} = [-0.5, -1.5]^{\top}$（漏判为正类，**加**）；
2. **迭代 2**：$w'_2 = w'_1 + \eta(0 - 1)x' = w'_1 - (1, -1)^{\top} = [-1.5, -0.5]^{\top}$（误判为正类，**减**）；
3. **迭代 3**：$w'_3 = w'_2 + \eta(1 - 0)x' = w'_2 + (1, 2)^{\top} = [-0.5, 1.5]^{\top}$（漏判为正类，**加**）；
4. **迭代 4**：$w'_4 = w'_3$（本轮判对，**不更新**）。

至此完成了**第一个 epoch**。复用训练数据重复上述过程后可以发现，**该算法在这个问题上于第一个 epoch 之后就已经收敛**（即 w'_4 就是解）：$w'_4 = [-0.5, 1.5]^{\top}$

对应判定点为 $x = -b/w = -(-0.5)/1.5 = 1/3$

$w'_1 = [-0.5, -1.5] \rightarrow x = -1/3$（1st）；

$w'_2 = [-1.5, -0.5] \rightarrow x = -3$（2nd）；

$w'_3 = w'_4 = [-0.5, 1.5] \rightarrow x = +1/3$（3rd）。

初始权重 $w'_0 = [-1.5, -2]$ 对应的判定点为 $x = -0.75$（即图上的 0th）—— 该图的横轴为非等距的自定义刻度，标注位置只作示意，不必按像素间距去反推数值。最终边界 $x = 1/3$ 恰好落在 Class 2 的最大值（−1）与 Class 1 的最小值（0.5）之间，所以它成功分开了两类 。

5.4 2-D Example

有两类模式，分别围绕 (1, 2) 与 (3, 1)，**每类 5 个数据点**

- 用**感知机学习算法**训练；
- **经过 100 个 epoch 之后**，得到的权重向量为：

（自上而下依次为**偏置 **$b = 0.0041$、$w_1 = -0.0349$、$w_2 = 0.0534$）

因此判定线为：$0.0041 - 0.0349 \cdot x_1 + 0.0534 \cdot x_2 = 0$

这个结果怎么读（把它整理成熟悉的斜截式）：

- 移项：$x_2 = (0.0349 \cdot x_1 - 0.0041) / 0.0534 \approx 0.6536 \cdot x_1 - 0.0768$；
- 代入 (1, 2)：$f = 0.0041 - 0.0349 + 0.1068 = +0.076 > 0$，落在正侧；
- 代入 (3, 1)：$f = 0.0041 - 0.1047 + 0.0534 = -0.047 < 0$，落在负侧；
- 两类确实被分到判定线两侧 —— 训练成功。

1. **权重都趋近于 0（0.0041 / −0.0349 / 0.0534）**，说明算法在后期只做极小的修正 ， 感知机在线性可分数据上收敛后，误分样本会越来越少，更新步长自然变小。注意这不是学习率自动衰减，而是错分的样本少了、修正的机会少了。

2. **w_1 是负的、w_2 是正的，与两类样本的位置关系一致**：x_1 越大越可能属于 (3,1) 那一类，x_2 越大越可能属于 (1,2) 那一类。权重的符号直接告诉你哪个特征指向哪一类，这是线性模型唯一可解释的地方，多层网络变黑箱的对照：多层网络的权重就没有这种直观含义了。

这两张图片从网络架构的角度，解释了感知机如何从简单的“单层结构”演变成能够处理复杂问题的“多层结构”。

Part 6 Multilayer Perceptron

6.1 Network Architectures

6.1.1 网络架构的定义

分层网络（Layered Networks）：

1. 单个节点不足以解决许多实际问题（Single node is insufficient for many practical problems）；实践中经常使用大量节点组成的网络；
2. 网络架构（network architecture）定义节点之间如何连接；
3. 分层网络（Layered Networks）：节点被划分成若干子集，每个子集称为一层（layer）；如果 k > j，则没有从第 k 层到第 j 层的连接（No connections lead from layer k to layer j if k > j）；层内连接可能存在（Intra-layer connections may exist）。

层 Layer	名称	说明
Layer 0	输入层 input layer	只负责接收外部数据，不做计算，不计入层数
Layer 1 ⋯ n−1	隐藏层 hidden layer(s)	夹在输入与输出之间，做非线性变换
Layer n	输出层 output layer	产出网络的最终结果

6.2 Single-Layer Feedforward Networks

单层前馈网络

- 计算层：虽然图中有“输入”和“输出”两排节点，输入层（源节点）→ 一层输出神经元，中间没有隐藏层，它被称为“单层网络”。
- 原因：因为输入层的节点只负责分发信号，不进行任何数学运算（不加权、不求和、不激活）。只有输出层的神经元在进行计算 the layer refers to the **output layer of computation nodes**。
- 局限性：正如我们之前讨论的，这种**单层感知机**只能解决线性可分的问题。

6.3 Multilayer Feedforward Networks

多层前馈网络：从单层（只能画直线）到多层（可以画曲线、处理复杂逻辑）

隐藏层（Hidden Layers）概念：

- 隐藏层节点：位于输入层和输出层之间，被称为“隐藏神经元”，是神经网络强大的核心。
- 特征提取：**隐藏层的作用（intervene between the external input and the network output）是介入输入与输出之间，作为中间表达，对原始数据进行加工**。通过多层非线性变换，**网络能够提取出数据中更深层次的特征**。

- 解决复杂问题：通过增加隐藏层，神经网络可以突破线性可分的限制，从而解决像 XOR（异或） 这样单层感知机无法处理的非线性问题，所以隐藏层的加入，使得神经网络具备了模拟任何复杂函数的能力。
- 全连接（Fully Connected）：图中展示了每一层的每个节点都与下一层的所有节点相连，这是最经典的多层感知机（MLP）结构 Neurons in each layer have as their inputs the **output signals of the preceding layer only**。

6.4 Multilayer Perceptrons (MLP)

MLP 是**单层感知机的推广（Generalization of the single-layer perceptron）**，由三部分组成：

1. 一个**输入层**（An input layer）
2. **一个或多个**由**计算节点组成**的**隐藏层**（One or more hidden layers of computation nodes）
3. 一个**输出层**（An output layer of computation nodes）

MLP **一般对每个神经元采用非线性（且可微）的激活函数** it adopts a **nonlinear (differentiable)** activation function for each neuron，例如**逻辑函数（logistic function）**。

其中 $v_j $是神经元 $ j $的**诱导局部场**（加权和 \+ 偏置），$y_j $是该神经元的输出：

$y_j = 1/(1 + exp(-v_j))$

Tips：

为什么必须是可微的？Training Algorithm: Back-Propagation (BP) algorithm。因为训练要靠反向传播，而反向传播要靠链式法则求导，所以激活函数必须可微。

为什么不能就用阈值函数堆多层？因为不可微，梯度传不下去。

这一个因果链串起了阈值函数不可微、必须换 logistic、BP 全部推导。

6.5 The XOR Problem

1. 问题背景

- 任务是**对单位超立方体中的点（取 0 或 1）做分类**；
- **XOR 是非线性可分（nonlinearly separable）的典型**；
- **基本感知机无法解决 XOR 问题，**因为 **XOR 的决策边界不可能是一条直线，不是线性的。**

2. 解决方法

使用一个**两层网络**，每个神经元都用**阈值函数**作为激活函数。

网络含 3 个神经元：**Neuron 1、Neuron 2 在隐藏层，Neuron 3 在输出层。**

Part 7 Back-Propagation

7.1 Training as Optimization

目标：to minimize an energy function representing the instantaneous error.

最小化一个表示「瞬时误差」（instantaneous error）的 energy function（能量函数）

误差函数（能量函数）：

$E = ½ Σ_{j=1}^{n₃} (d_qj − x⁽³⁾{out,j})² = ½ ‖ d_q − x⁽³⁾{out} ‖ ² $

其中 $ d_q $是第 q 个输入模式的期望输出， $x⁽³⁾_out $是 MLP 的实际输出，1/2是为了在求导时抵消平方带来的因子 2。

7.2 The General Learning Rule

权重更新通式：

$\Delta w^{(s)}_{ji} = -\eta^{(s)} \cdot \partial E_q / \partial w^{(s)}_{ji}$

即， $w(n+1) = w(n) - \eta \cdot g(n)$

问题：怎么高效地算出 $\partial E / \partial w$?

解决：链式法则+从输出层往输入层递推，即反向传播。

符号	含义
s = 1, 2, 3	指定网络层（designates the appropriate network layer）
$\eta^{(s)} > 0$	对应层的学习率参数（learning rate parameter），例如取一个常数
∂E_q/∂w	误差对权重的偏导（**反向传播要算的核心量**）

7.3 Output Layer 局部梯度的推导

对输出层的神经元 j：

1. 先用链式法则（By chain rule）把要求的偏导拆成两部分： $\partial E / \partial w^{(3)}_{ji} = (\partial E / \partial v^{(3)}_j) \cdot (\partial v^{(3)}_j / \partial w^{(3)}_{ji})$
2. 第二部分： $v^{(3)}_j = \Sigma_i w^{(3)}_{ji} \cdot x^{(2)}_{out,i}$，所以 $∂v⁽³⁾_j/∂w⁽³⁾ji = x⁽²⁾{out,i} $
3. 也就是前一层（第二层）的输出信号，即进入这个权重的那个输入；
4. 第一部分定义为局部梯度：

输出层的局部梯度：局部梯度 = （期望输出 − 实际输出）× 激活函数的导数，即

$\delta^{(3)}_j = -\partial E / \partial v^{(3)}_j = (d_{qj} - x^{(3)}_{out,j}) \cdot \phi'(v^{(3)}_j)$

最后得到输出层的权重更新规则：

$\Delta w^{(3)}_{ji} = \eta^{(3)} \cdot \delta^{(3)}_j \cdot x^{(2)}_{out,i}$

即 $w^{(3)}_{ji}(k+1) = w^{(3)}_{ji}(k) + \eta^{(3)} \cdot \delta^{(3)}_j \cdot x^{(2)}_{out,i}$

$δ⁽³⁾_j$： δ 被称为该神经元的局部梯度（local gradient），也叫局部误差（local error）。即这个神经元该往哪个方向改、改多猛，误差越大改得越多，激活函数在当前工作点的导数越大改得越多。

7.4 Hidden Layers 局部梯度的递归

原则：隐藏层的神经元没有直接的期望输出，所以它对总误差的贡献是间接的，它通过影响后一层，再影响输出，才作用到误差上。

推导过程（仍然是链式法则，只是展开得更长）：

1. 起点同样是 $\Delta w^{(2)}_{ji} = -\eta^{(2)} \cdot (\partial E / \partial v^{(2)}_j) \cdot (\partial v^{(2)}_j / \partial w^{(2)}_{ji})$；
2. 第二部分依然简单： $\partial v^{(2)}_j / \partial w^{(2)}_{ji} = x^{(1)}_{out,i}$（第一层的输出）；

3. 难点全在 $\partial E/\partial v^{(2)}_j$：要把误差函数展开，用链式法则对它求导，并引入输出层的局部误差来化简。

隐藏层的局部梯度：局部梯度 = 本层激活导数 × （后一层所有神经元的局部梯度，按连接权重加权求和）

$$\delta^{(2)}_j = \phi'(v^{(2)}_j) \cdot \sum_{k=1}^{n_3} \delta^{(3)}_k \cdot w^{(3)}_{kj}$$

对应的权重更新规则：

$$\Delta w^{(2)}_{ji} = \eta^{(2)} \cdot \delta^{(2)}_j \cdot x^{(1)}_{out,i}$$

$$w^{(2)}_{ji}(k+1) = w^{(2)}_{ji}(k) + \eta^{(2)} \cdot \delta^{(2)}_j \cdot x^{(1)}_{out,i}$$

可见何为反向?

输出层的 δ 用的是已知的期望值 d（直接可算）；隐藏层的 δ 用的是后一层已经算好的 δ。所以计算顺序必须从输出层开始，一层一层往回退。也正因为如此，只要激活函数可微，隐藏层有多少层都能用同一个递推式。

7.5 Unified Form and Two Passes

1. 两层的规则放在一起对照，权重更新统一式相同：

$$w^{(s)}_{ji}(k+1) = w^{(s)}_{ji}(k) + \eta^{(s)} \cdot \delta^{(s)}_j \cdot x^{(s-1)}_{out,i}$$

2. 区别：局部梯度的算法

输出层：局部梯度正比于期望输出与实际输出之差；

$$\delta^{(s)}_j = (d_{oj} - x^{(s)}_{out,j}) \cdot \phi'(v^{(s)}_j)$$

隐藏层：局部梯度也可以理解为正比于该神经元期望输出与实际输出之差，但隐藏层神经元的期望输出是未知的，必须用所有相连神经元的误差信号递归估计出来；

$$\delta^{(s)}_j = \phi'(v^{(s)}_j) \cdot \sum_k \delta^{(s+1)}_k \cdot w^{(s+1)}_{kj}$$

Tips: 该式可以推广到任意数量隐藏层的 MLP（Eqn. (4) can be generalized for MLP having an arbitrary number of hidden layers）

反向传播算法的计算（Back-propagation: 2 passes of computation）：

1. 前向传播（Forward pass）：为每个神经元计算函数信号（function signals）。
2. 反向传播（Backward pass）：从输出层开始，递归地为每个神经元计算 δ，从输出层一直回到第一个隐藏层；在每一层，按（5）式的 delta 规则相应地修改突触权重。

Tips：BP 能撑起深度学习的最直接的工程原因：无论你的多层网络有多少个隐藏层，都可以用完全相同的递归方式来更新权重堆层不需要新算法，只需要多循环几次。

7.6 Why Sigmoid Is Convenient

1. 激活函数的核心要求是可微（Differentiability）；常用的是 S 形函数，例如逻辑函数。简化后公式为：

$$\phi'(v) = a \cdot y \cdot (1-y)$$

此时，$\phi'(v)$在 y = 0.5 时取最大值，在 y = 0 或 1 时取最小值 0。

2. **梯度消失：**Sigmoid 饱和 → $\phi' \to 0$ → 梯度消失 → 前面的层学不动

如果某个神经元的输出已经饱和（y 接近 0 或 1），它的 $ $\phi'(v) \approx 0$，于是权重几乎不再更新，这个神经元学不动了。在多层网络里，这种衰减会逐层相乘，越靠前的层梯度越小，最后趋近于 0，这就是深层网络难以训练的经典原因。

3. 两个局部梯度都可以只用输出值表达：

情形	局部梯度表达式（Sigmoid 下）	含义
输出层神经元 j	$\delta_j = a \cdot y_{out,j} \cdot (1 - y_{out,j}) \cdot (d_j - y_{out,j})$	误差 × 导数，导数用输出值表示
隐藏层神经元 j	$\delta_j = a \cdot y_{out,j} \cdot (1 - y_{out,j}) \cdot \sum_k \delta_k \cdot w_{kj}$	后一层误差信号加权求和，再乘本层导数

由于权重改变量正比于导数 $ $\phi'(v)$，对 Sigmoid 激活函数来说，那些函数信号处于中间范围的神经元，权重被改变得最多；局部梯度 $\delta_j $可以在不需要显式知道激活函数的情况下算出来。

7.7 Epoch and Stopping Criteria

7.7.1 Epoch

An epoch = A complete presentation of the entire training set during the learning process. 一个 epoch 就是把整个训练集完整过一遍，学习过程按 epoch 为单位反复进行（epoch-by-epoch），直到满足收敛准则（criterion of convergence）。

7.7.2 三种停止准则

1. 每个 epoch 的均方误差的绝对变化率足够小（Absolute rate of change in mean squared error per epoch is sufficiently small）看的是**误差还在不在动**（一阶差分），哪怕误差还很大，只要不再下降就停；
2. 突触权重与偏置水平稳定下来（Synaptic weights and bias level stabilized）看的是**参数还动不动；**
3. 整个训练集上的均方误差小于某个最小值（Mean squared error over the entire training set is less than some minimum value）看的是**误差够不够小**（绝对水平）。

Tips: 三者可以独立使用，也可以组合。实践中常把1或3配一个**最大 epoch 数**做保险（否则可能永远不满足）。

7.8 The Full Algorithm

用流程图标出五个步骤：Initialization → Presentation of training examples → Forward computation → Backward computation → Is stopping criterion met? → No（回到呈现样本）/ Yes → End

1. **初始化（Initialization）**：初始化**权重与偏置**；
2. **呈现训练样本（Presentation of training examples）**：呈现一个 epoch 的训练样本；**对每个样本**执行前向与反向计算；
3. **前向计算（Forward computation）**：计算**误差信号**；
4. **反向计算（Backward computation）**：计算 **δ 并依据delta rule调整权重**；
5. **迭代（Iteration）**：若**未满足停止准则**，回到第 2、3、4 步。

7.9 Structure vs Decision Regions

Different Nonlinearly Separable Problems

结构 Structure	决策域类型 Types of Decision Regions	XOR 问题	网格状区域 Meshed Regions	最一般形状 Most General
单层 Single-Layer	**由超平面界定的半平面** Half Plane Bounded By Hyperplane	做不到	做不到	只得到半平面
两层 Two-Layer **隐藏层 + 输出层	**凸的开放或闭合区域** Convex Open Or Closed Regions	能做到	部分可以	受限
三层 Three-Layer	**任意形状（复杂度受节点数限制）** Arbitrary (Complexity Limited by No. of Nodes)	能做到	能做到	最一般

单层 → 半平面（一条直线 / 一个超平面）；

两层 → 凸的开放或闭合区域；

三层 → 任意形状，复杂程度只受节点数量限制。

需要注意：一个 MLP 在容量上可能足以表示某个函数（**三层能表示任意形状**），但梯度下降这个过程未必能找到那组权重，因为它在错误的盆地（basin）里下降到谷底就停下了。表达力是网络结构的问题，可学习性是优化过程的问题，两者不是一回事。

Part 8 Limitations of Back-Propagation

编号	缺陷 Deficiency	课件原文 Key Sentence	含义
①	**收敛慢 Slow convergence**	Learning often takes a long time to converge. Complex functions often need **hundreds or thousands of epochs**.	复杂函数要成百上千个 epoch，训练成本高
②	**黑箱 Black box**	It may provide a desired mapping between input and output vectors (x, y) but does not have the information of **why a particular x is mapped to a particular y**.	只能给出映射，学到的只是操作性参数，没有可解释的因果说明
③	**容易陷入局部极小 Local minimum**	Gradient descent approach only guarantees to reduce the total error to a **local minimum** （E may not be reduced to zero）.	误差曲面非凸、无唯一最小点，梯度下降可能困在局部极小；通常只能靠换结构、换初值、加随机扰动（如模拟退火）缓解。
④	**参数全靠经验 Empirical parameters**	The learning (accuracy, speed, and generalization) is highly dependent on a set of learning parameters… Most of them can only be determined **empirically (via experiments)**.	初始权重、学习率、层数与节点数没有理论解，只能试

Tips: 1. 慢、2. 说不清、3. 可能不是最优、4. 全靠调参。

前两条是工程代价，后两条是理论缺口。

第4条同时说明了一件事，神经网络的超参数（hyperparameter）这个概念就是从这儿来的：**权重与偏置是学出来的，层数 / 节点数 / 学习率 / 初始权重是人给的。**

8.1 Long Training Time

1. 现象：复杂函数常常需要成百上千个 epoch才收敛。

2. 为什么慢：每个 epoch 都要对全部训练样本做一次前向 + 反向传播计算；而每次权重更新只挪动很小的一步（步长由学习率 η 决定）。
3. 与第7部分呼应：第 7.3 / 7.4 节的更新式 $w ← w + η · δ · x $里，如果 $ η $太小 → 步数多； $η $太大 → 可能在极小点附近来回震荡甚至发散。这就是第4条参数全靠经验的具体表现。

Tips：没有在这里给出怎么加速（动量、自适应学习率等都不在本讲范围）。本讲只要求记住：收敛慢是 BP 的固有代价，不是实现 bug。

8.2 Black Box

The net is essentially a black box 这个网络本质上就是一个黑箱。

为什么它说不清：

1. 它能给出映射，但不给理由：It may provide a desired mapping between input and output vectors (x, y) but does not have the information of why a particular x is mapped to a particular y.
2. 所以给不出直观的（例如因果的）解释：It thus cannot provide an intuitive (e.g., causal) explanation for the computed result.
3. 根因在于隐藏单元与学到的权重没有语义：This is because the hidden units and the learned weights do not have a semantics.

What can be learned are operational parameters, not general or abstract knowledge of a domain.

能学到的只是操作性参数，而不是某个领域的通用或抽象知识。

operational parameters与abstract knowledge这组对立：一个训练好的网络里存的是一堆数字（权重与偏置），不是如果高频能量集中在 2 号频段，就说明是 Steve这类可以用语言说清楚的规则。这正是医疗、司法等需要高透明度决策的场景不能直接用 BP 网络的原因。

8.3 Local Minima

1. Gradient descent approach only **guarantees to reduce the total error to a local minimum**（E may not be reduced to zero）
2. **Cannot escape from the local minimum error state.**
3. **Not every function that is representable can be learned.**

可表示≠可学习，一个 MLP 在**容量上**可能足以表示某个函数，但**梯度下降这个过程**未必能找到那组权重，因为它在错误的盆地（basin）里下降到谷底就停下了。**表达力是网络结构的问题，可学习性是优化过程的问题，两者不是一回事。**

8.3.1 How bad

depends on the shape of the error surface. Too many alleys/wells will make it easy to be trapped in local minima.

窄巷与坑越多，越容易被困在局部极小

MLP with nonlinear activation functions have complex error surfaces (e.g., plateaus, long valleys etc.) with no single minimum.

含非线性激活函数的 MLP，其误差曲面是复杂的，**深度学习网络是非凸优化，分布山谷山脊，以山谷为例。在山谷震荡，不存在唯一的最小点。**

词汇 Vocabulary	中文	在误差曲面上的含义
plateau	平台区	一大片几乎平坦的地方，梯度≈0 → 走得很慢，像是卡住了
long valley	长谷	一个方向陡、另一个方向平的长条形凹槽 → 在最速下降里会沿谷壁来回弹
alley	窄巷	又窄又深的通道，很容易把人关在里面
well	井 / 坑	局部极小点所在的凹陷处
local minimum	局部极小	比周围都低，但不是全局最低
global minimum	全局最小	整个曲面上真正的最低点 —— 我们想要的

8.3.2 Three Remedies

1. 换网络规模试试：Try nets with different # of hidden layers and hidden units ——它们会带来不同的误差曲面，有些可能比另一些更好（they may lead to different error surfaces; some might be better than others）。
2. 换初始权重试试：Try different initial weights —— 相当于在曲面上换一个出发点（different starting points on the surface）。
3. 用随机扰动强行跳出：Forced escape from local minima by random perturbation（例如 simulated annealing）。

simulated annealing（模拟退火）正是 Week 2 讲过的允许以一定概率接受更差解的方法，Week 3 的进化计算也是同一思路（靠种群与随机变异维持探索）。

Tips：所以本讲最后给出的跳出局部极小的出路，其实是用的之前学的，**不依赖梯度的迭代搜索，梯度负责快，随机搜索负责别被困死。**

8.4 No Theoretical Quality Guarantee

There is no theoretically well-founded way to assess the quality of BP learning:

What is the confidence level one can have for a trained BP net, with the final E (which may or may not be close to zero)?

没有理论上站得住脚的办法去评价BP 学出来的东西有多好。训练完误差 E 可能很小、也可能不接近 0，而你对这个网络的信心该有多大，无法从理论上回答。

问题出在学习参数上：

1. 依赖哪些参数：Initial weights（初始权重）、learning rate（学习率）、# of hidden layers and # of units（隐藏层层数与节点数）…
2. 关键限制：Most of them can only be determined empirically (via experiments). —— 其中绝大多数只能靠实验（经验）来确定。

这一条与第9部分的衔接关系：既然训完不知道好不好，那就必须在训练集之外另拿一份数据去看，于是才有了下一部分的 validation error（验证误差）、underfitting / overfitting 与 learning curves。

Part 9 Overfitting and Underfitting

9.1 Three Regimes of Fit

状态 State	别名 Alias	课件原文 Original Wording	训练误差	验证误差
Underfitting 欠拟合	**High Bias** 高偏差	The model is too simple — **cannot capture the underlying pattern**. Both training and validation errors are high.	高	高（且与训练误差接近）
Good Fit 恰当拟合	**Balanced** 平衡	The model captures the true pattern **without memorizing noise**. Both errors are low and close.	低	低（且与训练误差接近）
Overfitting 过拟合	**High Variance** 高方差	The model is too complex — **memorizes training data, including noise. Low training error but high validation error.**	低	**高**

Tips:

欠拟合 = 都高（训练也学不会）;

恰当拟合 = 都低且接近（学会了规律，没背住噪声）;

过拟合 = 训练低、验证高（把噪声也记下来了，换新数据就露馅）。

9.2 The Bias-Variance Tradeoff

模型复杂度与偏差：讨论模型该多大。

图例 Legend	中文	曲线形状（看图）	随复杂度怎么变
$Bias^2 $(underfitting)	偏差平方（对应欠拟合）	红色，**单调下降**，由约 1.0 降到接近 0	复杂度 ↑ → 偏差 ↓
Variance (overfitting)	方差（对应过拟合）	蓝色，**单调上升**，由接近 0 升到约 5	复杂度 ↑ → 方差 ↑
Total Error	总误差	绿色，**U 形**，最低处约 0.97	先降后升，中间有个最低点
Irreducible Error	不可约误差	灰色虚线，**水平**，约 0.05	与复杂度无关，降不下去

Bias：来自假设过度简化的误差。

Variance：来自对训练数据中微小波动过度敏感的误差。

Goal：找到总误差最小的那个。

★**总误差曲线的最低点 **Optimal Complexity：**落在 Model Complexity ≈ 1.7 的位置，再往左走偏差太大（欠拟合），再往右走方差太大（过拟合）**。

Tips:

1. 三条曲线的关系：总误差 ≈ 偏差² + 方差 + 不可约误差。红色（偏差²）往下走、蓝色（方差）往上走，两者相加再加一条水平的灰线，得到的正是那条绿色的 U 形曲线。这就是权衡（tradeoff）这个词的来源：动复杂度，一边降另一边必然升，只能找平衡点。
2. 注意课件把偏差写成了 Bias^2（偏差的平方），不是 Bias。这不是笔误，标准分解式里偏差那一项本来就是平方项。所以图例读作bias squared，中文说偏差平方。

9.3 Learning Curves as a Diagnostic

讨论训练集大小与误差（训练误差，验证误差）：讨论数据量该多少。

子图 Panel	两条曲线怎么走	诊断结论
Underfitting 欠拟合	红线从约 **0.38** 缓降到约 **0.36**（几乎水平）；蓝线从约 **0.47** 降到约 **0.40**。**两条都高，且彼此贴得很近。**	Both training and validation errors are high and close together. Adding more data does not help-need a more complex model. **加更多数据没用，需要更复杂的模型。**
Good Fit 恰当拟合	红线从约 **0.18** 降到约 **0.09**；蓝线从约 **0.28** 降到约 **0.13**。**两条都低，且随数据量增加而收敛到一起。**	Both errors are low and converge. The model generalizes well to unseen data. **模型对未见数据泛化得好。**
Overfitting 过拟合	红线从约 **0.06** 降到约 **0.015**（非常低）；蓝线只从约 **0.48** 降到约 **0.38**。**训练低、验证高，中间夹着一大片粉色区域。**	Large gap between training (low) and validation (high) error. More training data can help reduce the generalization gap. **加更多训练数据有助于缩小泛化差距。**

Tips：Generalization gap（泛化差距），**它就是训练误差曲线与验证误差曲线之间的面积**。gap 越大 = 过拟合越严重。**过拟合的量化指标就是这个gap有多宽。**

诊断口诀（先用学习曲线判断是哪一种病，再用偏差-方差权衡决定该往哪个方向调）：

1. **两条都高、且挤在一起 → 欠拟合。**
2. **两条都低、且收敛靠近 → 拟合恰当。**
3. **一条低一条高、中间 gap 很大 → 过拟合。**

9.4 Five Remedies for Overfitting and Underfitting

9.4.1 Five Remedies for Overfitting

1. 正则化 Regularization（L1/L2 penalty）：Adds penalty for large weights.

改目标函数：对大权重加惩罚项，逼网络别把权重推到极端值。

2. 随机失活 Dropout：Randomly deactivate neurons during training.

改训练过程：训练时随机关掉一部分神经元，让网络不能依赖某个特定神经元的组合。

3. 早停 Early Stopping：Stop when validation error starts increasing.

改训练过程：验证误差一开始上升就停止训练（对应第 7.7 节的停止准则，但这里用的是验证误差而不是训练误差）。

4. 数据增强 Data Augmentation：Increase training data diversity.

改数据/输入分布：增加训练数据的多样性（对应第 9.3 节加数据能缩小 gap）。

5. 批归一化 Batch Normalization：Normalize layer inputs to stabilize training.

改数据/输入分布：把每层的输入归一化，稳定训练过程。

9.4.2 Five Remedies for Underfitting

1. 增加模型复杂度 Increase Model Complexity：Add more layers or neurons.

加层或加节点。

2. 训练更久 Train Longer：Increase number of epochs.

加 epoch 数。

3. 减弱正则化 Reduce Regularization：Lower L1/L2 penalty or dropout rate.

把 L1/L2 惩罚或 dropout 率调小。

4. 更好的特征工程 Better Feature Engineering：Add more informative features.

补更有信息量的特征。

5. 换更强的结构 Use a Different Architecture：Try a more powerful model class (e.g., deeper network).

换更深的网络等。

Tips：

1. 过拟合要**减容量**（正则化、Dropout、早停）；欠拟合要**加容量**（加层加节点、训练更久、减弱正则化）。**唯一的例外是第4条的两侧不对称**：过拟合那边的数据增强是**造更多数据**，欠拟合这边的特征工程是**把现有数据表达得更好。**一个加量，一个提质，因为两种缺的东西不一样。
2. **减弱正则化这条容易被忽略**：如果你发现模型欠拟合，要先检查一下是不是**正则化开得太狠**、或者**dropout 率设得太高，**这不一定是模型容量的问题，也可能是把它捆得太紧了。
3. 掌握：这是欠拟合还是过拟合、为什么、下一步该加数据还是改模型。

Part 10 Case Study: Voice Recognition

本讲概念 Concept	在这个案例里对应什么
神经元模型与激活函数（Part 2 - 3）	60 个频段值进入输入层；每个隐藏神经元做加权求和 + 激活，输出层同理
MLP 结构（Part 6）	60 - 6 - 2 的前馈网络，一个隐藏层；输入层不计入层数 → 两层网络
前向传播（第 6.6 节）	从 60 维频谱算出 2 个输出值（第 10.3 节的 0.43 / 0.26）
误差函数（第 7.1 节）	输出与 one-hot 目标之差（第 10.4 节的四个误差）
反向传播（第 7.3 - 7.5 节）	按误差反推，调整 60×6（输入→隐藏）+ 6×2（隐藏→输出）共 372 个权重与 8 个偏置
epoch 与停止准则（第 7.7 节）	走完所有训练对= 一个 epoch；重复多次= 多个 epoch
收敛（第 8.2 节）	输出从 (0.43, 0.26) 收敛到 (0.01, 0.99)
过拟合 / 泛化（Part 9）	已知样本与新样本都是 100% → 属于 Good Fit，未见明显过拟合

权重个数怎么来的：输入层 60 → 隐藏层 6，所以输入到隐藏的权重有 60 × 6 = 360 个；隐藏层 6 → 输出层 2，所以隐藏到输出的权重有 6 × 2 = 12 个；合计 372 个权重。偏置每个神经元一个：6 + 2 = 8 个。逐层相乘再相加的算法在第 6.6 节里有对应，也是第7部分推导里 $w^{(s)}_{ji}$ 的两个下标（i 遍历前一层、j 遍历本层）的由来。

Part11 Exercise

11.1 Exercise1: Regression

11.2 Exercise2: Gradient Descent

Lecture 5

Part 0 Introduction

周 Week	主题 Topic	核心内容 Key Content
Week 1	An Introduction AI 导论	AI 定义 / 机器学习三大范式 / ANI-AGI-ASI / 学术技能
Week 2	Searching & Optimization 搜索与优化	BFS / DFS / Hill Climbing / Best-First / A* / SA / GA
Week 3	Evolutionary Computation 进化计算	EA 总览 / PSO / DE / MA / MOEA / ETO / NAS
Week 4	Artificial Neural Network 人工神经网络	神经元模型 / 激活函数 / 感知机 / MLP / 反向传播 / 过拟合引言
Week 5	**Deep Learning & Applications 深度学习与应用**	**CNN 卷积网络 / RNN-LSTM 序列网络 / GNN 图网络 / 生成模型与扩散模型**

本周回答了：同样一套反向传播，为什么堆得更深反而更强？面对图像、序列、图这三种完全不同的数据，网络结构要怎么改？

学习路线：深度学习总览 → CNN → 序列神经网络 → 图神经网络 → 生成模型与扩散模型 → 过拟合与欠拟合。

总结

被装在四种不同的数据接口上的同一个反向传播引擎。

区别在于接口的类型：图像是规则网格，用滑动窗口（卷积）；序列有先后，用递归链（循环）；图没有顺序，用邻居聚合（消息传递）；要造新数据，换成对分布的建模（自编码 / 对抗 / 扩散）。

共同点：猜参数 → 看误差 → 反向传播改参数

本讲的四条支线	解决什么	共同的技术手段
CNN	图像：把局部性 + 平移不变注入模型	权值共享；逐步加深；残差连接
RNN / LSTM	序列：处理变长输入与长程依赖	细胞状态的加性记忆通道
GNN / GCN	图：在无固定顺序的邻居集合上做聚合	邻居消息传递；置换不变
生成模型	造新数据：学数据分布本身	潜在空间正则；对抗博弈；迭代去噪

Part 1 From NN to Deep Learning

1.1 Three Formulas and Levels

1. **单个神经元 Model of a neuron**：每个输入被一个权重缩放、与一个偏置求和，再过一层激活函数。

$v = \Sigma_{i=1}^{m} w_i x_i + b,\ y = \phi(v)$

2. **感知机 Perceptron**：一个线性分类器。只有两类**线性可分（linearly separable）**时才收敛。

$w_{n+1} = w_n + \eta(d_n - y_n)x_n$

3. **MLP 与反向传播 MLP and backpropagation**：隐藏层去掉了线性限制；输出神经元与隐藏神经元使用**不同的局部梯度**。

$w_{n+1} = w_n - \eta \cdot \partial E/\partial w$

Lecture4 得这些概念分别代表了单元（一个神经元）→ 能力边界（单层只能线性分类）→ 突破办法（多层 + 反向传播），本讲是这些的延伸，讲解层与层的联系。

1.2 Deep Learning

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction.

多个处理层 multiple processing layers 即深，层的数量，不是参数的绝对规模。

学习表示 learn representations 即学，不再是人工设计的，而是从数据里生成的。神经网络是**实现**深度学习的方式之一。

多个抽象层次 multiple levels of abstraction 即层次的递进：边缘—鼻子-脸。

1.3 Two Paradigms

两者的区别在特征提取器能否被训练：上面那张图的红色框圈住的是 hand-crafted Feature Extractor，下面那张图的红色框圈住的是 Trainable Feature Extractor + Trainable Classifier 这个整体。

环节 Stage	传统模型（20 世纪 50 年代起）	深度学习模型
特征来源	Fixed/engineered feature 人工设计、固定不变的特征提取器	Trainable feature extractor **可训练**的特征提取器
分类器	"Simple" Trainable Classifier 可训练的**简单**分类器（并非不可训练）	Trainable Classifier 可训练的分类器
特征端代表方法	HoG（方向梯度直方图）、SIFT（尺度不变特征变换）	不独立存在
分类端代表方法	两段式：人做特征 → 机器学分类 Linear Regression、SVM、Decision Tree、Random Forest	一体式：与特征端**连成一个整体**一起训练
代表网络	无	AlexNet、VGGNet、Inception、ResNet、ViT

1.4 Why is it called Deep?

It is deep if more than one stage of non-linear feature transformation.

只要有**多于一个阶段的非线性特征变换**就算深度。深的含义是表示被逐级复合地变换，与实现上是否有隐藏层、隐藏层有几个没有对应关系。

1.5 The ImageNet Trend

1.6 Why Deep Learning Works

1. 表示能力 Representation Power

已知结论：a shallow network can approximate any function. 一个浅层网络可以逼近任意函数。

代价：this requires many hidden units. 需要极多的隐藏单元。

结论：Depth gives an exponential advantage. 深度带来指数级优势。同样表达力下，深层网络的参数需求远小于浅层。

2. 可优化性 Tractability of Optimization

存在的问题：Lots of local minima. 误差曲面上布满局部极小（Lecture 4 第 8.3.1 ）。

实践：optimization still works. 优化仍然能用。

可行的解决方法：with tricks like variants of adagrad, using relu / leaky relu, dropout, batch norm⋯ —— 自适应学习率（Adagrad 的变体）、ReLU / Leaky ReLU 激活、Dropout、批归一化。

3. 泛化能力 Generalization Power

存在的问题：DNN can "memorize" a large training set (huge capacity) . 深度网络容量巨大，完全有能力把训练集背下来。

实践：Yet they don't overfit (when training data and model size is compatible). 只要训练数据量与模型规模相匹配，它并不因此过拟合。

可行的解决方法：they don't just store the examples. 它学到的不是样本本身。

Tips：表示能力承认浅层也能逼近、但代价指数级；可优化性承认满地局部极小、但工程技巧能压住；泛化承认容量足以背训练集、但实测没有背。

1.7 A Brief History

Part 2 CNN I 卷积神经网络（卷积层与池化层）

2.1 CNN Definition

Convolutional Neural Networks learn a complex representation of visual data using vast amounts of data. They are inspired by the human visual system and learn multiple layers of transformations, which are applied to the top of each other to extract a progressively more sophisticated representation of the input.

CNN 用**大量数据**学习视觉数据的复杂表示；它受**人类视觉系统（局部感受野 + 逐级抽象**）启发，学习**多层变换（每一层都在对上一层的输出做运算）**，这些**变换逐层叠加（至少一个非线性函数）**，从而提取出越来越精细的输入表示

2.2 Convolution Definition 卷积

1. **数学定义：**Convolution is one of the main building blocks of a CNN. The term convolution refers to the mathematical combination of two functions to produce a third function. It merges two sets of information. 卷积是两个函数的数学组合，产生第三个函数；它把两组信息合并。

2. **操作定义**：Convolution is essentially sliding a filter over the input. 卷积本质上就是把一个滤波器/卷积核在输入上滑动。
3. **直觉定义**：A convolution can be thought as looking at a function's surroundings to make better/accurate predictions of its outcome. 卷积可以理解为看一个函数的周围环境，以便对它的结果做出更准的预测。这就意味着，卷积的输出在某个位置上的值，取决于**输入在该位置邻域内的取值**，而不是该位置单独的取值。

2.3 Convolutional Kernels 卷积核

英文术语	中文	课件原句
filter / kernel	滤波器 / 卷积核	these terms are used interchangeably **两个词指的是同一个东西**，可互换使用
feature map	特征图	卷积在输入上滑动后**产生**的输出
sliding the filter over the input	滑动	At every location, a matrix multiplication is performed and sums the result onto the feature map.

Examples:

1. 左图用全 1 的 2×2 卷积核对一张 4×4 的输入做卷积，步长（stride）为 2，得到一张 2×2 的特征图。
2. 右图真实输入图像得到特征图：用的核是 [[−1,−1,−1], [−1,8,−1], [−1,−1,−1]]。特点是中心权重远大于四周、且符号相反，它输出的特征图上只剩边缘，平坦区域被减成接近 0。**卷积核里装什么，特征图上就留下什么。**

2.4 Pooling Layer 池化层

功能角度：The function of pooling is to continuously reduce the dimensionality to reduce the number of parameters and computation in the network. 池化持续降低维度，以减少网络中的参数量与计算量。

动作：The function of pooling is the process of merging. So it's basically for the purpose of reducing the size of the data. 池化是合并的过程，目的是缩小数据尺寸。

Example：input image → convolutional layer → pooling layer → convolutional layer → pooling layer → fully-connected layer，整条链被框在一个标着 CNN 的方框里，最右端输出 car（predicted class）。

Tips：最后一级是 fully-connected layer（全连接层）而不是卷积层：特征提取完成之后，仍要靠全连接层把特征映射到类别。

2.4.1 Max and Average Pooling

Example:

同一张 4×4 输入，2×2 窗口、步长 2，并排演示两种池化。

窗口位置	四个数 Four values	最大池化 Max pooling	平均池化 Average pooling
左上	5, 3, 0, 2	5（最大值）	(5+3+0+2)/4 = 2.5
右上	3, 1, 8, 5	8（最大值）	(3+1+8+5)/4 = 4.25
左下	1, 4, 0, 9	9（最大值）	(1+4+0+9)/4 = 3.5
右下	4, 2, 2, 7	7（最大值）	(4+2+2+7)/4 = 3.75

最大池化输出 [[5, 8], [9, 7]]，平均池化输出 [[2.5, 4.25], [3.5, 3.75]]

Tips：

1. 池化层没有参数：它不是卷积，卷积核里的参数是学出来的，而池化的取最大或取平均是固定的运算，训练过程中不更新任何值。所以池化能降低维度和计算量，却不需要额外的学习容量。
2. 两种池化的分工：最大池化保留窗口内最强的响应，对某个特征在不在这类判断更敏感，也因此对微小位移更宽容；平均池化保留窗口内的整体强度，对这个区域大概有多活跃更敏感。实际网络里最大池化更常见。

2.5 How CNN Works

2.5.1 Two Kinds of Layers

卷积层 Convolution layer：is a feature detector that automatically learns to filter out not needed information from an input by using convolution kernel. 一个特征检测器，靠卷积核自动学出该过滤掉输入里的哪些信息。

池化层 Pooling layers：compute the max or average value of a particular feature over a region of the input data (downsizing of input images). Also, it helps to detect objects in some unusual places and reduces memory size. 在输入的一个区域上求某个特征的最大或平均值（即缩小输入图像）；还有助于检测出现在异常位置的物体，并减少内存占用。

2.5.2 What Each Layer Learns

某一层的卷积核的响应模式：可视化（**边缘** → **纹理** → **部件** → **物体**）

层 Layer	学到的内容	含义
Conv 1	Edge+Blob（边缘与斑点）	最底层：只认边缘朝向与色块
Conv 3	Texture（纹理）	由边缘组合出的重复性图案
Conv 5	Object Parts（物体部件）	开始出现眼睛、轮子、鼻子这类可命名部件
Fc8	Object Classes（物体类别）	最顶层：整类物体（ship、sock、dining table、grocery store）

Part 3 CNN II 四个模型

3.1 Four Milestone Models

eNet 证明卷积可行→ AlexNet 证明卷积能规模化→ ResNet 证明卷积能叠很深→ ViT 证明卷积是不是必须的。前三个模型都在**改进卷积的用法**，第四个模型**取消了卷积**。

对比项	LeNet-5 (1998)	AlexNet (2012)	ResNet (2015)	ViT (2020)
代表成绩	手写数字识别成功落地	ILSVRC 2012 Top-5 错误率 15.3%	ILSVRC 2015 Top-5 错误率 3.57%	JFT-300M 上预训练后超越 CNN
核心创新	卷积 + 池化的基本结构；**权值共享 + 局部连接**	**ReLU + GPU + Dropout + 数据增强**	**残差（跳跃）连接**，学 F(x) = H(x) − x	**图像切成 patch + 纯注意力**，不用卷积
解决的问题	MLP 直连像素导致**参数爆炸、忽略空间结构**	把 CNN **放大到大数据集**（ImageNet 120 万张）	深层网络的**退化问题**与梯度消失/爆炸	CNN 的**归纳偏置**是否必要

对比项	LeNet-5 (1998)	AlexNet (2012)	ResNet (2015)	ViT (2020)
遗留局限	浅、sigmoid/tanh、小数据集	层数仍有限	未提；后由注意力架构接续	需要**超大数据集**，小数据上效率低

3.1.1 LeNet-5 第一个成功的卷积网络

架构：经过 Conv、Pool、Conv、Pool 、FC 、 FC 、Output 两次卷积 + 池化，两次全连接。

输入输出：Input: 32x32 grayscale; Output: 10 digit classes (0-9)

应用：handwritten digit recognition (MNIST, postal codes) 手写数字识别，如邮政编码。

解决的问题：MLPs on raw pixels had too many parameters, ignored spatial structure. 在原始像素上做 MLP 参数量太大，且完全忽略空间结构。

关键洞察：weight sharing + local connectivity drastically reduce parameters. 权值共享 + 局部连接大幅减少参数。

局限：shallow, sigmoid/tanh activations, small datasets. 网络浅、用 sigmoid/tanh 激活、只在小数据集上验证过。

3.1.2 AlexNet 把 CNN 推到大数据集上（进入GPU时代）

成绩：Top-5 error: 15.3% vs. 26.2% (second place, hand-crafted features) .

解决的问题：scaling CNNs to large datasets (ImageNet, 1.2M images) 把 CNN 扩展到大数据集。

四项创新：

1. **ReLU 激活**：ReLU activation (faster training than sigmoid/tanh) 训练比sigmoid/tanh更快。
2. **GPU 训练**：GPU training (2 GPUs, major speedup) 用 2 块 GPU，带来大幅加速。
3. **Dropout 正则化**：Dropout for regularization 用随机失活抑制过拟合。
4. **数据增强**：Data augmentation (cropping, flipping, color jitter) **裁剪、翻转、颜色抖动**。

3.1.3 ResNet 用残差连接把网络做深

成绩：3.57% top-5 error（对比 AlexNet 的 15.3%）。

解决的问题：vanishing/exploding gradients in very deep networks. 极深网络的梯度消失/爆炸。

3.1.3.4 The Degradation Problem 退化问题

现象：deeper networks are harder to train — degradation problem (not overfitting) 更深的网络更难训练，这叫退化问题，而且它不是过拟合。

证据：Adding layers increases training error, contrary to expectation. 加深层数反而让训练误差上升。（过拟合的表现是训练误差下降、验证误差上升）

成因：vanishing/exploding gradients 梯度在深层传播过程中消失或爆炸。

解法：学「残差」而不是学「映射」

做法：Solution: skip (residual) connections 加**跳跃连接（残差连接）**。

改写的目标：Learn residual F(x) = H(x) - x, instead of H(x) directly 不去直接学目标映射 H(x)，而是学**残差 F(x) = H(x) − x**。

更容易学习的原因：Identity mapping is easy to learn: just set F(x) = 0 **恒等映射**：F(x) = 0 。

效果：Enabled networks of 50, 101, even 152 layers 使更深的网络成为可能。

Tips: Lecture 4 讲反向传播时提到过深层网络梯度会消失。ResNet 作为**工程解法，用跳跃连接给梯度提供了一条不经过任何非线性层**的直通路径，即x，使梯度可以直接回传到浅层。

3.1.4 Vision Transformer 把 Transformer 直接用在图像上

apply the Transformer architecture (from NLP) directly to images

把来自 NLP 的 Transformer 架构直接用到图像上。

做法：

1. Split image into fixed-size patches (e.g., 16x16), linearly embed, add position 把图像切成固定大小的小块（如 16×16），做线性嵌入，再加上位置信息。
2. Feed patch embeddings into a standard Transformer encoder 把小块嵌入送进标准 Transformer 编码器。
3. 分类头与预训练：[CLS] token for classification; pretrain on large datasets (JFT-300M) 用 [CLS] 标记做分类；在大规模数据集（JFT-300M）上预训练。

**解决的问题: **CNNs have inductive bias (locality, translation equivariance) CNN 自带归纳偏置：局部性与平移等变。

Tips: CNN 的归纳偏置（局部性、平移等变）在数据有限时是巨大的优势，它注入了自然图像的规律。但数据量足够大时，它限制了模型去发现人类没预设过的模式。

结论：ViT shows that with enough data, a general attention model beats CNNs 数据足够时，让模型自己学，不要预设。

局限：needs huge datasets; less efficient on small data 需要超大数据集，在小数据上效率低。

Part 4 序列神经网络 Sequential Neural Networks

4.1 Sequence Transduction Models

Sequence transduction models refer to a class of models that aim to transform one sequence of data into another sequence. Such as Google's autocomplete feature predicts the rest of the words a user is typing.

序列转换模型指的是一类**把一条数据序列变换成另一条序列**的模型，如谷歌补全词条。

优势：序列到序列的优势是输入和输出的长度可以变化。

4.2 循环神经网络 Recurrent Neural Network

基本形式：input sequence is transformed into output sequence in a one-to-one fashion 输入序列以**一对一（步进节拍：时间步 t 进一个 token、同一时间步出一个 token）**的方式变换为输出序列（每一步进一个、出一个）。

设计目标：Design an architecture that possesses the ability to generate output sequences of variable length, while incorporating contextual relevance and a memory mechanism that leverages prior inputs to influence the current input and output.

1. 能产生任意长度的输出序列；
2. 兼顾上下文相关性；
3. 有一套记忆机制，让先前的输入能影响当前的输入与输出。

Tips: RNN是一对一，而Transformer 也是一步进一个位置、出一个位置，但它的上下文不再靠一条递归链传递，而是靠注意力一次性看全序列。

4.3 RNN 的能力边界 RNN's Breaks Down

A small example where RNN can work perfectly: Prediction of the last word in the sentence: "The clouds are in the sky" 一个 RNN 能完美工作的例子 预测句子的最后一个词。

RNN can't handle situation where the gap between the relevant information and the point where it is needed is very large. (Long-term dependency)

当**相关信息**与**需要用到它的位置**之间的**间隔很大**时，RNN 处理不了。这就是**长程依赖（long-term dependency）**问题。

1. Vanishing and exploding gradients 梯度消失与梯度爆炸。递归链上每一步都要乘一次权重矩阵，经过 t 步以后梯度里就带着权重矩阵的 t 次幂，若**矩阵特征值小于 1 就消失，大于 1 就爆炸。**
2. Limited memory capacity 记忆容量有限。
3. Computational complexity 计算复杂度（逐步递归，无法并行）。

4.4 LSTM

为了解决长期依赖：LSTMs - are a special kind of RNN, capable of learning long-term dependencies. LSTM 是一种**特殊的 RNN**，**能够学习长程依赖**。

结构：All RNNs have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.

所有 RNN 都是一条由重复模块组成的链。标准 RNN 的重复模块极简，例如只有一层 tanh；

LSTM has the same chain-like structure except for the repeating module. LSTM 的**链式结构完全相同**，区别只在重复模块内部。

对照项	标准 RNN 的重复模块	LSTM 的重复模块
结构复杂度	极简单，例如单层 tanh	明显复杂：多个门控与逐点乘法
输入输出	$x_t $进、 h_t 出	$x_t 进，h_t $与 C_t**（细胞状态）**同时维护

左边（标准 RNN）每个方块标 A，方块里只有一条带 tanh 的支路；

右边（LSTM）每个方块标 A，内部有 **四个小的 σ / tanh 单元**以及多条交叉的加减与乘法节点。

4.4.1 How to Work

The core idea behind LSTMs is the cell state, which acts as an information highway running through the entire sequence.

LSTM 的核心是细胞状态（cell state），它像一条贯穿整条序列的信息高速公路。

4.4.2 Three Gates

The cell state is updated and controlled by three main structures called gates: the input gate, the forget gate, and the output gate. These gates regulate the flow of information into, out of, and within the cell state, allowing LSTMs to selectively retain or forget information as needed.

细胞状态由三个称为「门」的结构更新与控制 —— 输入门、遗忘门、输出门。这些门**调节信息流入、流出以及细胞状态内部**的流动，使 LSTM 能按需**选择性地保留或遗忘**信息。

门 Gate	作用	直觉
input gate	把新信息**写进**细胞状态	决定「这一时刻要不要记住新东西」
forget gate	从细胞状态里**丢掉**信息	决定「过去记下的东西还要留多少」
output gate	把细胞状态里的信息**读出**来给输出	决定「这一刻要把记忆的哪一部分说出来」

门的实现方式：

Gates are generally composed out of a sigmoid neural net layer and a pointwise multiplication operation 门一般由**一个 sigmoid 神经网络层 + 一次逐点乘法**组成。

sigmoid 输出在 (0,1)，乘上去就等价于**保留多少比例，**输出 0 是丢，输出 1 是留。

4.4.3 The Four Equations

LSTM 的四个核心方程。 $C_t $是细胞状态， $h_t $是隐藏状态， $x_t $是当前输入。

量	方程	读法
遗忘门	$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$	把**上一时刻的细胞状态、上一时刻的隐藏状态、当前输入**拼在一起，过 σ得到 0 - 1 之间的系数
输入门	$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$	同一组输入，另一套权重，得到「写入多少」的系数
输出门	$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$	注意这里用的是**更新后的** C_t，而不是 C_{t-1}
细胞状态	$C_t = f_t * C_{t-1} + (1 - f_t) * C_t$	旧记忆按 $f_t $的比例保留，新候选记忆 $C_t $按 $1 - f_t $的比例写入

4.5 SNN 问题What Issue Are They Facing?

短板	原文	中文
训练慢	Slow to train (inefficient / not parallelizable) This sequential dependency limits the ability to parallelize computations, as each step must wait for the completion of the previous step.	训练慢、效率低、**无法并行化** 这种**顺序依赖**限制了并行计算能力，**每一步都必须等上一步算完**
梯度问题	Suffer from exploding or vanishing gradients	仍会遭遇梯度爆炸或梯度消失
长程依赖仍受限	Cannot handle very long-term dependencies In Seq-Seq models, decoder only accesses the last hidden state. Early information in sentences can be lost.	仍然处理不了**极长**的依赖 在**序列到序列（Seq-Seq）模型里，解码器只能拿到最后一个隐藏状态**，句子**早期的信息可能丢失**

Tips: 主要问题是**并行性**（能不能同时算）和**信息通道宽度**（所有历史被压进一个固定长度的隐藏状态）。所以设计目标是 **既要能并行，又要能直接看到序列里的任意位置，而不是只透过一个固定大小的窗口。**

Part 5 GNNs I: From Graphs to GNNs

图是什么 → 为什么不能直接用 DNN → GNN 怎么做

5.1 Definition of Graph

A graph represents the relations (edges) between a collection of entities (nodes).

图表示**一组实体（节点）之间的关系（边）**

属性类型	英文原文	中文	例子
节点属性	V Vertex (or node) attributes	顶点 / 节点属性	e.g., node identity, number of neighbors 节点的身份、邻居数量
边属性	E Edge (or link) attributes and directions	边 / 链接属性与方向	e.g., edge identity, edge weight 边的身份、边的权重
全局属性	U Global (or master node) attributes	全局 / 主节点属性	e.g., number of nodes, longest path 节点总数、最长路径

属性分成 V / E / U： 后续所有 GNN 模型的输入清单，任何一个 GNN 层要做的事都可以写成拿 V、E、U 的一部分作为输入，输出更新后的 V、E、U。

5.2 图的应用 Graphs Application

5.2.1 Images and Text as Graphs

图像和文本本身就是规则结构，把它们看成图只是换了一种表示。真正的图数据是下一组那些天生就没有规则网格结构的数据，图像和文本的图结构是规则、稠密、平移不变的，不需要通用的图方法。

5.2.2 Graph-Valued Data in the Wild

Molecules as graphs **分子**：原子是节点，化学键是边。

Social network as graphs **社交网络**：人是节点，关注或互动是边。

Brain network as graphs **脑网络**：脑区是节点，功能或结构连接是边。

Graph data is everywhere around the world !

为什么要专门为图设计网络的动机：如果图数据真的无处不在，那把图塞进只能用于规则网格的 CNN 就会损失大量结构信息。

5.3 图解决的问题 Solving Problems over Graphs

图可以解决的五类问题（6.3.2）

任务 Task	定义	中文
Node Classification	Classifying individual nodes.	**节点分类**：对单个节点分类，对应 $softmax(z_n) $，**逐节点**过 softmax

任务 Task	定义	中文
Graph Classification	Classifying entire graphs.	**图分类**：对整个图分类，对应 $softmax(\Sigma_n z_n)$，**先对所有节点求和**再过 softmax
Node Clustering	Grouping together similar nodes based on connectivity.	**节点聚类**：按**连接关系**把相似节点归为一组
Link Prediction	Predicting missing links.	**链接预测**：预测缺失的边
Influence Maximization	Identifying influential nodes.	**影响力最大化**：找出有影响力的节点

5.4 图数据上用深度神经网络的挑战Challenges of Using Graphs in DNN

为什么不能使用DNN？图的特征对应CNN的前提不成立。

1. **Graph-Structured Data（图结构的数据）**：Neural networks have been traditionally used to operate on fixed-size and/or regular-structured inputs (such as sentences, images and video). 神经网络传统上都用来处理**固定尺寸、规则结构**的输入，例如句子、图像和视频。**图两者都不是。**
2. **Lack of Consistent Structure（缺乏一致结构）**：Graphs are extremely flexible mathematical models; but this means they lack consistent structure across instances. 图是**极其灵活**的数学模型，但这也意味着**不同样本之间结构不一致**：每个图的节点数、连边数、度数分布都可能不同。
3. **Node-Order Equivariance（节点顺序等变）**：Graphs often have no inherent ordering present amongst the nodes. 图的节点之间往往没有固有的顺序。任何 GNN 的输出不能依赖于节点的编号顺序。把输入节点重排一遍，输出的对应关系应该跟着重排，而数值不变。所以Invariance to permutations（置换不变）

Tips：CNN 靠平移不变来适应图像的位移，GNN 靠置换不变来适应图的重新编号。

5.5 图神经网络 What is a Graph Neural Network?

Define:

A GNN is an optimizable transformation on all attributes of the graph (nodes, edges, global-context, connectivity) that preserves graph symmetries.

GNN 是作用在图的全部属性（节点、边、全局上下文、连接关系）上的可优化变换，并且保持图的对称性。

Main Idea:

Pass messages between nodes to refine node (and possibly edge) representation.

在节点之间传递消息，以此来精修节点（可能还有边）的表示。

1. 节点V、边E、全局上下文U、连接关系这四个属性全都要处理。
2. 可优化变换：GNN 不是一套固定规则，不是人为设定，它的参数是通过训练学的（与 CNN、MLP 同源）。
3. 保持图的对称性：节点换编号后结果不能变，这就是要保持的对称性。
4. 传递消息：这是 GNN 的工作方式，也是消息传递网络（Message Passing）每个节点从邻居那里收消息，更新自己的表示，然后进入下一层。

Part 6 GNNs II: Graph Convolution

6.1 单个像素的更新 Update for a Single Pixel

卷积的关键两个步骤：

1. Transform messages individually 每个邻居传来的信息 $ h_i $各自乘上一套权重 W_i，得到 W_ih_i。

Tips：在CNN中，每个位置一套独立权重，卷积核可以给每个位置配一套固定的权重 。但图的节点没有固定范围，一个节点的邻居不能确定数量和先后，所以卷积无法在图上使用。

2. Add everything up 把结果全部加起来 $ Σ_i W_i h_i$。

$h_i \in \mathbb{R}^F$ 是某个像素 / 节点的（隐藏层 hidden layer）激活值 activations of a pixel/node。

为了在图上使用，完整的单点更新式：

$$h_4^{(l+1)} = \sigma(W_0^{(l)}h_0^{(l)} + W_1^{(l)}h_1^{(l)} + \cdots + W_8^{(l)}h_8^{(l)})$$

中心节点编号为 4，它周围有 0 - 8 共 9 个位置（3×3 窗口，含自己），每一个位置各自的激活值先乘上各自的权重矩阵 W_i，全部相加，再过激活函数 σ。

6.2 图卷积网络 Graph Convolution Networks (GCNs)

6.2.1 更新规则 The Update Rule

$ h_i^{(l+1)} = σ(h_i^{(l)} W_0^{(l)} + Σ_{j∈N_i} (1/c_ij) h_j^{(l)} W_1^{(l)})$

节点 i 在下一层的表示，等于两部分的加权和:

1. 它自己的表示 $h_i $乘一套权重 W_0;
2. 它的所有邻居 j 的表示 $h_j $各乘一套权重 W_1、再除以一个归一化常数 c_ij，然后全部加起来;
3. 最后过一个非线性激活 σ。

符号	说明	中文
σ	σ 加在整体外面	非线性激活函数
$h_i^{(l)}$	节点 i 在第 l 层的表示	注意**自己的信息**单独乘 W_0，不与邻居混在一起
$W_0^{(l)}/W_1^{(l)}$	两套权重矩阵	**只有两套**：一套给自己、一套给邻居
N_i	neighbor indices	节点 i 的**邻居索引集合**
c_{ij}	norm. constant (fixed/trainable)	归一化常数，可以**固定**也可以**可训练**

Tips：关键改动：邻居之间共享权重 Weight sharing over all locations，代价是丢失了邻居的相对方位，所以后来 GAT 等模型要引入注意力权重。

6.2.2.1 希望具备的四条性质 Four Desirable Properties

1. Weight sharing over all locations 在所有的位置上共享权重。
2. Invariance to permutations 对置换不变：重排节点编号，结果不变。

3. Linear complexity O(E) 线性复杂度，与边数 E 成正比。

4. Applicable both in transductive（**直推式**） and inductive（**归纳式**） settings 两种设定都能用。

直推式和归纳式的区分：直推式指训练时已经见过整张图（包括测试节点），只是不知道它们的标签，例如对一个固定的社交网络做节点分类，图是同一张，只是有一部分人没标注；归纳式指测试时要处理训练时没见过的新节点或新图，例如对一张全新的分子做预测。

为什么 GCN 两者都能用：因为它的更新规则只依赖邻居的现场表示与学好的共享权重。前者可以现场算出来，后者是训练阶段留下的参数；所以遇到新节点时不需要重新训练整个模型。

6.2.3 谱方法 The Spectral View

Spectral-based methods can be also interpreted as spatial-based methods.

基于谱的方法也可以被解释为基于空间的方法。

$ H^{(l+1)} = \delta (D^{(-1/2)} Â D^{(-1/2)} H^{(l)} \theta)$

含义：把整层所有节点的表示堆成矩阵 H，先左乘一个**归一化后的邻接矩阵** $ D^{(-1/2)} Â D^{(-1/2)} $（相当于聚合邻居），再乘可学习参数 θ，最后过一个非线性函数 δ。

记号	课件说明	中文
A	A: Adjacency matrix	邻接矩阵
D	D: Degree matrix	度矩阵（对角阵，记录每个节点的邻居数）
Â	Â: A + I	带自环的邻接矩阵（加上单位阵 I，把自己也算作自己的邻居） 即更新规则里自己那一项 $h_i W_0$（把自己当成邻居）
D	D: D + I	带自环的度矩阵
θ	θ: learnable parameter	**可学习参数**
δ	δ: non-linear activation function	**非线性激活函数**

空间式更好理解（在邻居之间传消息），谱式更好实现（一次矩阵乘法算完一整层）。

6.3 GNNs应用 GNNs for Application

6.3.1 输入与逐层更新 Input and Layer Update

环节	课件内容	中文
输入 1	Feature matrix $X \in \mathbb{R}^{(N \times E)}$	特征矩阵 X，形状 N×E
输入 2	preprocessed adjacency matrix $\hat{A}$	预处理过的邻接矩阵$\hat{A}$
逐层更新	$H^{(l+1)} = \sigma(\hat{A}H^{(l)}W^{(l)})$	每一层做一次邻接聚合 → 线性变换 → 非线性激活
起始状态	$X = H^{(0)}$	输入特征矩阵就是第 0 层的表示
结束状态	$Z = H^{(N)}$	跑完 N 层之后得到的最终表示 Z
层间激活	图上每个隐层旁标注 ReLU 与 $\sigma(\cdot)$	每层的非线性激活，与普通神经网络一致

6.3.2 三类下游任务 Three Readout Heads

任务	输出头
节点分类 Node classification	$softmax(z_n) $对**每个节点**的表示过 softmax
图分类 Graph classification	$softmax(\Sigma_n z_n)$ **先对所有节点求和**，再过 softmax
链接预测 Link prediction	$p(A_ij) = \sigma(z_i^T z_j) $ 两个节点表示的内积再过 sigmoid

区别是读哪里：**节点分类**读单个节点的向量；**图分类**读整张图的汇总（所以要先求和，把 N 个节点压成 1 个向量）；**链接预测**读两个节点的配对（所以用内积来衡量它们像不像应该相连）

回到 5.3 节的五类任务 节点分类与图分类对应前两类，链接预测对应第四类；节点聚类与影响力最大化在本页没有给出式子。

6.4 GNN vs CNN

对比维度	CNN（Part 2 - 3）	GNN / GCN（Part 5 - 6）
数据形态	规则网格（图像）	不规则的图
输入尺寸	固定（如 224×224×3）	可变（节点数与边数都随样本变）
样本间结构	所有样本结构相同	**每个样本结构都可能不同**
节点顺序	有固定朝向（宽、高）	**无固有顺序**
要保的性质	**平移等变 / 不变**	**置换不变**
权重套数	每个相对位置一套（3×3 核 = 9 套）	**只有两套**：W_0（*自己*） + W_1（*邻居*）
聚合方式	在固定窗口内做加权求和	在**邻居集合**$N_i $上求和，除以归一化常数 c_{ij}
复杂度	与图像尺寸、核大小相关	O(E)，与**边数**成正比
引入的归纳偏置	**局部性 + 平移等变**	**置换等变 + 邻居聚合**

Tips: ViT 质疑的是CNN 的归纳偏置是否必需（3.1.4）；而 GNN 不是去掉归纳偏置，而是换一套更贴合图数据的归纳偏置（置换不变 + 邻居聚合）。两条路线的共同问题：当数据本身没有规则结构时，我们该把什么结构免费注入模型？

Part 7 Generative Models I 生成问题（一）

前三个部分（CNN / RNN / GNN）的共同点：输入一段已有的数据（图像 / 序列 / 图），输出一个**判别结果**。从本部分开始换任务：输入一批样本，输出同一分布下的新样本。这是机器学习里一个完全不同的建模目标。不处理已经存在的数据的部分，而是造出新的数据。

7.1 判别模型与生成模型 Discriminative vs Generative

In Machine Learning, one can clearly distinguish between the two modelling types:

1. Classifying an image as a dog or a cat falls under Discriminative Modelling,
2. Producing a realistic dog or a cat image is a Generative Modelling problem

机器学习可以清楚地区分两种建模类型：把一张图判成狗还是猫，属于判别式建模；生成一张逼真的狗或猫的图片，是生成式建模问题。判别模型学的是**边界**，生成模型学的是**分布**。

对比维度	判别模型 Discriminative	生成模型 Generative
要回答的问题	**这是哪一类?**	**这类数据长什么样?**
学的是什么	条件分布 $p(y	x)$：给定输入 x，输出的标签概率
输出	一个标签（或一组概率）	一个**新样本**

7.2 Goal and Three Ingredients

7.2.1 Goal

Goal: Given training samples, generate new samples from the same distribution.

给定训练样本，**从同一分布中生成新的样本**。

In other words, generative models try to learn a model p_model(x) similar to p_data(x).

换句话说，生成模型试图学到一个 p_model(x)，让它**接近真实的数据分布** p_data(x)。

7.2.2 Three Ingredients

1. 一个模型 Model：we specify its parametric form, e.g., neural networks, etc. 指定它的参数化形式，例如神经网络。
2. 一个学习目标 Learning objective：e.g., maximum likelihood 例如最大似然。
3. 一个优化算法：optimization algorithm to achieve the learning objective 一个用来达成该学习目标的算法。

Tips: 生成模型的三要素与神经网络框架一致：定结构 → 定损失 → 用梯度下降优化，区别在于生成模型的第二项，如何确定学习目标，因为 p_data(x) 是一个无法直接写出的高维分布，所以测量模型离真实分布有多远要另想办法，所以引出了VAE、GAN、Diffusion 三条路线的分歧点。

7.3 自编码器Autoencoders

结构：Autoencoders are neural network architectures that consist of an encoder（编码器**）** and a decoder,（解码器） which work together to learn a **lower-dimensional representation** of input data and **reconstruct** the data from the representation.

自编码器由与解码器组成，两者合作学习输入的低维表示，并从该表示重建数据。

核心任务：With the lower-dimensional representation z, the fundamental task of an autoencoder is to reconstruct its input x as accurately as possible. 用低维表示 z，**尽可能准确地重建输入 x**。

过程：Through a process of encoding and decoding, the autoencoder learns to map the input data to a compressed representation and then decode this representation back to the original input space.

把输入映射到**压缩表示**，再解码回**原始输入空间**。

7.3.1 数据通路与损失函数 Data Flow and Loss

环节	记号	含义
输入	x	原始数据
编码器	E（encoder, x → z）	**降维**：把 x 压成 z

环节	记号	含义
潜在编码	z	压缩后的低维表示
解码器 / 生成器	G（decoder/generator, z → x）	把 z 还原回数据空间
重建	$\hat{x}$	重建出的输出
损失	L	重建误差，用均方误差衡量

均方误差（Mean Squared Error, MSE） 把每个样本原始值与其重建值之差的平方求平均。n 是变量的个数（where n is the number of variables）。

$$L = (1/n)\,\Sigma_{i=1}^{n}(x^i - G(E(x^i)))^2$$

1. The encoder network is for dimension reduction, just like PCA 编码器的作用就是降维，和 PCA 类似；
2. Latent code: the compressed low dimensional representation of the input data 潜在编码：输入数据的压缩低维表示。

7.3.2 自编码器的三条局限 Why it Cannot Generate

1. **训练目标里没有生成这一项。**损失函数 L 只要求 $\hat{x}$ 尽量等于 x，也就是输入什么就还回来什么。一个完美重建的自编码器，它的输出空间里没有新样本的位置。
2. **潜在空间 z 没有被规定形状。**编码器可以学会把每个训练样本映射到潜在空间里一个孤立的点，只要解码器能把这些点，还原回来，损失就是 0。这时在潜在空间里随便取一个新点 z，解码器完全不知道该输出什么，因为它从没见过那个位置。
3. 问题的根源：它**只学了点对点的映射**，没学空间的结构，缺少一个可以安全采样的潜在空间。 这正是下一节 VAE 要解决的事。

7.4 变分自编码器 Variational Autoencoders (VAEs)

7.4.1 两条关键改动 Two Key Changes

改动一：Encode the latent features in a meaningful space, e.g., Gaussian distribution. 把潜在特征编码到一个有意义的空间里，例如高斯分布。

改动二：Sample from the meaningful latent space to generate data. 从这个有意义的潜在空间中采样来生成数据。

Example：Assume training data x is generated from the **Gaussian distribution** of latent representation z

Intuition: x is an image, z is latent factors used to generate x.

假设训练数据 x 是由潜在表示 z 的**高斯分布**生成的。**直觉**：x 是一张图像，z 是**用来生成 x 的潜在因子**。

In a nutshell, a VAE is an autoencoder whose encoding distribution is **regularized** during the training in order to ensure that its latent space has good properties allowing us to generate some new data.

简言之，VAE 就是在训练过程中对编码分布加正则的自编码器，目的是让潜在空间具备允许我们生成新数据的良好性质。

7.4.2 两个概率分布与损失 Two Distributions, Two Losses

项	记号	读法
编码（推断）	q(z|x) Inference (encoder)	给定数据 x，推断潜在变量 z 的分布
解码（生成）	p(x|z) generation (decode)	给定潜在变量 z，生成数据 x 的分布
重建损失	L_{MSE}	重建得像不像
正则损失	L_{kl}	**KL 散度**：把编码分布拉向标准高斯，让潜在空间有形状
总损失（核心）	$L_{total} = L_{MSE} + L_{kl}$	**两项相加**：既要重建得准，又要潜在空间规则

$L_{total} = L_{MSE} + L_{kl}$ 这一行是 VAE 的核心。根据 7.3.2 的三条局限：只有 L_{MSE}（即普通自编码器）时，潜在空间可以是任意形状、点与点只有映射；加上 L_{kl} 之后，编码器被迫把每个样本编码成**一个高斯分布**而不是一个点，于是整个潜在空间被铺满，**随便取一个点都能解码出合理的数据**。

Example：先验采样 → 解码生成的完整路径

从 Latent features: z 经 Decoder 得到 Data: x，标注为 Generate x ~ p_θ (x|z)；而 z 是 Sample from true prior z ~ p_θ (z)（从真实先验中采样）。生成时不再需要编码器，只需要从先验里取一个 z 然后解码。

7.5 GAN 的提出动机 Why GANs

问题：VAEs define density function with latent z of Gaussian distribution. However, it is non-trivial（不容易） to model complex, high-dimensional training data with an **explicit distribution**.

VAE 用高斯分布的潜在变量 z 来定义密度函数。**但是**，要**用一个显式的分布**去建模复杂的、高维的训练数据，并不是一件容易的事。

7.5.1 GAN 的两条路线选择 Two Design Choices

1. 放弃显式密度：GANs belong to "implicit density"（隐式密度）group, where any explicit density function of samples did not provide. GAN 属于隐式密度这一组，它不提供样本的显式密度函数。

即 VAE 的路线是写出密度，然后去逼近它；GAN 的路线是写不出密度，那就绕过它，既然无法显式写出 p_data(x) 长什么样，那就让一张网络（判别器）来隐式地衡量 这像不像真数据，另一张网络（生成器）去把这个衡量推翻。

2. 用博弈论做动机：Motivated by game theory.

GAN 的代价是训练变得不稳定，因为它本质上是一个对抗性的博弈过程，而不是一个单纯的 误差下降。

玩家	课件原文	中文
生成器 Generator	Generator tries to generate "fake" samples from its model (tries to fool the discriminator)	生成器试图从自己的模型里**造出假样本**，目标是骗过判别器
判别器 Discriminator	Discriminator tries to distinguish "fake" and "real" samples	判别器试图**区分假样本**与真样本

Part 8 Generative Models II 生成问题（二）GANs and Diffusion Models

8.1 GAN 的目标函数 The Minimax Objective

Minimax objective function（极小极大目标函数）与 How to train (Alternative between)（怎么训练：在两者之间交替）。

极小极大目标：

生成器参数 θg想最小化、判别器参数 θd想最大化同一个目标函数；它由两项组成，**真数据的对数和**与**假数据的1减去判别结果的对数和**。

记号	中文	说明
$D_{\theta d}(x)$	判别器对真样本 x 的输出	判别器认为 x 为真的概率
$G_{\theta g}(z)$	生成器由噪声 z 造出的假样本	从先验 p(z) 采样 z，生成假样本
第一项 $E_{x\ p_{data}} logD(x)$	真数据的损失	用红框标出，注为 Loss for real data x
第二项 $E_{z\ p(z)} log(1 - D(G(z)))$	假数据的损失	图上用蓝框标出，注为 Loss for fake data x

8.1.1 两个网络 How It Works

判别器 D：D tries to maximize the cost such that D(x) close to 1 (for real x) and D(G(z)) close to 0 (fake)

让真样本的输出趋近 1、假样本的输出趋近 0。

生成器 G：G tries to minimize the cost such that D(G(z)) is close to 1 (try to make generated samples real-looking as much as possible to fool D) 让假样本的判别输出也趋近 1，即尽可能把假样本做得像真的去骗过 D。

8.1.2 交替训练 Alternate Between

固定 G，训练 D：Fix G, D maximize the cost (Gradient ascent on discriminator) 判别器做梯度上升。

固定 D，训练 G：Fix D, G minimize the cost (Gradient descent on generator) 生成器做梯度下降。

Tips：交替训练的问题：

普通的神经网络训练有一个静止的目标：损失函数在训练过程中形状不变（数据固定）。而 GAN 里，判别器每更新一次，生成器的损失函数就变了一次，这是博弈式建模的固有性质。

8.1.3 GAN Lab

GAN Lab: Play with Generative Adversarial Networks in Your Browser! (poloclub.github.io)

GAN 的交替训练过程很难用静态文字讲清楚，生成器与判别器的决策边界在二维平面上一来一回地移动，这是 GAN Lab 最直观的地方。看生成器的样本点如何一步步逼近真实分布、判别器的分界线如何反复摆动。观察交替训练和不稳定性。

8.3 GAN 的三个问题与扩散模型 Problems of GANs

问题 Problem	原文	中文
梯度消失 Vanishing Gradients	when the discriminator doesn't provide enough information for the generator to make process.	当判别器不能提供足够的信息让生成器取得进展时出现
模式崩塌 Mode Collapse	This is when generator starts producing the same output (or a small set of output) over and over again.	生成器开始反复产出同一个输出（或很小的几个输出）
难以训练 Hard to train	GANs are very sensitive to hyperparameters.	GAN 对超参数非常敏感

GAN问题共同根源：判别器的设计。梯度消失来自判别器太强，模式崩塌来自生成器找到了骗过判别器的捷径，难训练来自两者需要保持脆弱的平衡。

解决：**Diffusion Models**: Emerging as powerful generative models, outperforming other deep generative models, such as GANs.

扩散模型正在成为强大的生成模型，表现超过了 GAN 等其他深度生成模型。扩散模型的做法是它不训练判别器，而是训练一个网络去拟合一个明确的、已知的加噪过程。

8.4 扩散模型是什么 What Are Diffusion Models

灵感来源：Inspired by non-equilibrium thermodynamics to model the data distribution, diffusion models can describe the probability distribution of each step in the ink diffusion process.

受非平衡热力学启发，把一滴墨水滴进水里，它会逐步散开，最后完全均匀，这个过程是随机、不可逆的。扩散模型的做法是**反向学**这个过程：从完全均匀的噪声出发，一步步收拢回一滴墨水。

Diffusion models are a type of probabilistic generative model that transforms noise into a representative data sample. 是一类概率生成模型，它把噪声变换成一个有代表性的数据样本。

8.5 前向加噪与反向去噪 Forward and Reverse Processes

Diffusion model aims to learn the reverse of noise generation procedure

扩散模型的目标是学习 噪声生成过程 的反过程。

具体而言，正向的加噪过程是我们自己定义的、完全已知的（每一步加多少高斯噪声固定的），所以模型只需要学怎么反向走一步，这是一个有明确监督信号的学习问题，不需要另一个网络来判别。

8.5.1 两个方向 Two Directions

方向	原文	中文
前向步 Forward step	(Iteratively) Add noise to the original sample The sample x_0 converges to the complete noise x_T (which is the Markov chain)	（迭代地）给原始样本加噪声 样本 $x_0 $收敛到完全的噪声 x_T；这个过程是一条马尔可夫链
反向步 Reverse step	Recover the original sample from the noise Note that it is generation procedure	从噪声中恢复出原始样本 注意：这一步就是 生成 过程

8.5.2 训练目标 Training Objective

To train a Diffusion Model, the objective is to find the reverse Markov transitions that **maximize the likelihood of the training data.**

训练扩散模型的目标，是找到那些能**最大化训练数据似然**的反向马尔可夫转移。

左边是交叉熵损失（负的、模型对真实样本的对数似然），右边是它的一个上界：分子$ q(x_{1:T}|x_0) *是我们自己定义的前向加噪分布*，*分母*p_θ (x_{0:T}) $是要学的反向生成分布。

8.6 一个网络处理所有时间步 One Network for All Timesteps

For diffusion models, it is common to use a single network for all timesteps. Something UNet-like, which is very common in denoising and image-to-image problems.

对扩散模型来说，常见做法是用一个网络处理所有时间步。这个网络大致是 U-Net 形态，U-Net 在去噪与图像到图像类问题里非常常见。

8.6.1 训练时的六个步骤 The Training Process

1. Sampling a random timestep t for each training sample within the batch (e.g. images)

为 batch 里的每个训练样本各采样一个随机时间步 t。

2. Adding Gaussian noise by using the closed-form formula, according to their timesteps t

按各自的 t，用闭式公式加上高斯噪声。

3. Converting the timesteps into embeddings for feeding the U-Net or similar models (or other family of models)

把时间步转成嵌入向量，喂给 U-Net 或同类模型。

4. Using the images with noise and time embeddings as input for predicting the noise present in the images

用带噪图像 + 时间嵌入作为输入，去预测图中的噪声。

5. Comparing the predicted noise to the actual noise for calculating the loss function

把预测噪声与真实噪声比较，算出损失。

6. Updating the Diffusion Model parameters via backpropagation using the loss function

用损失做反向传播，更新参数。

三张训练进度图：分成 EPOCH 1、EPOCH 2、EPOCH k 三栏，每栏里列出 TIME EMBEDDINGS（时间步嵌入）与 NOISY IMAGES（带噪图像）的配对。EPOCH 1 里是 t = 14 / 35 / 6，EPOCH 2 里是 t = 78 / 153 / 721，EPOCH k 里是 t = 64 / 4 / 9。这组数字要传达的是** 每个样本的 t 都是随机抽的** 同一个样本在不同 epoch 会配到不同的噪声强度，这样网络才能学会所有时间步。

U-Net 结构图：Conv+BatchNormalization+ReLU（红色块）、Pooling operation（黄色块）、Upsampling Layer（蓝色块）、Skip-Connection（箭头）。整图呈 U 形：左侧向下经过多次池化（Layer1 → Layer2 → Layer3 → Layer4），底部最窄，右侧向上经过多次上采样，并有跨过底部的跳跃连接把两侧同层连起来，就是 U-Net 名字的由来，也是它能在去噪任务里保细节的原因。

8.7 采样过程 The Sampling Process

与训练的唯一区别：For sampling new images, the difference lies in that we don't have an input image.

生成新图像时，区别在于我们手上没有输入图像。

起点：We sample random Gaussian noise and define how much steps of noise (T) to take for generating the new images. 我们从随机高斯噪声出发，并规定要走的噪声步数 T。

每一步做什么：In each step, the Diffusion Model predict the whole noise present in the image, taking as input the current timestep. Then, it removes just a fraction of this predicted noise. —— 每一步里，模型以当前时间步为输入，预测整幅图中的噪声，然后只去掉其中的一小部分。

Tips: 为什么不一次把预测出来的噪声全减掉？因为网络在时间步 t 上的预测只在 t 这个噪声强度下可靠。一次性减完会直接跳到 x_0，但那个 $x_0 $是在 噪声还很多 的假设下估出来的，误差很大。逐步减去一小部分，等价于“走一小步、重新估一次、再走一小步”每一步都在新的噪声强度下重新用网络判断，误差不会累积放大。

结果：We obtain our image generation result after T inference steps. 经过 T 次推理步后得到最终生成结果。

两段伪流程对应图：

1. Sample Gaussian noise t = T = 1000、X_1000 = N(0, I)，箭头标注 sample，指向一张纯噪声图；

2. Iteratively denoise the image 下面画了多组 噪声图 → U-Net → pred noise 的迭代。

关键等式 $X_{999} = X_{1000} - prednoise$、 $X_2 = X_1 - prednoise$、 $X_1 = X_2 - prednoise$，最后得到干净图像。

这些等式传达的正是减掉预测噪声这个动作，两者合起来看：预测的是全部噪声，但每次只减一部分。

8.8 扩散模型小结 Summary

生成质量更好：Generates better data than VAEs and GANs. Diffusion models learn true distribution through step-by-step learning of Gaussian noise instead of raw data. This makes learning objectives simpler and the generation quality better. 扩散模型逐步学习高斯噪声而不是直接学原始数据，这让学习目标更简单、生成质量更好。

更容易训练：Easier to train. Diffusion models are easier to train than GANs and do not suffer from mode collapse due to their inherent diffusion process. 比 GAN 更容易训练，且由于其内在的扩散过程，不会遭遇模式崩塌。

In conclusion, Diffusion Models became a new effective paradigm for image generation and manipulation. By combining the principles of diffusion processes with deep learning techniques, these models offer new avenues for generating high-quality images.

总而言之，扩散模型已经成为图像生成与图像编辑的一种新的有效范式。它把扩散过程的原理与深度学习技术结合，为生成高质量图像提供了新的路径。

8.9 四类生成模型 Four Generative Paradigms

对比项	Autoencoder（AE）	VAE	GAN	Diffusion
核心思路	编码降维 + 解码重建	在 AE 上给潜在空间加正则	生成器与判别器对抗博弈	学加噪的反过程
损失函数	$L = (1/n)\Sigma(x^i - G(E(x^i)))^2$	$L_{total} = L_{MSE} + L_{kl}$	$min_\theta g max_\theta d ElogD(x) + Elog(1 - D(G(z)))$	$L_{CE} = -Elogp_\theta(x_0)$（及其变分上界）
能否生成新样本	不能	能（从先验采样 z 再解码）	能（生成器直接造）	能（从纯噪声迭代去噪）
潜在空间	无约束，可能长成任意形状	被正则成高斯，可安全采样	隐式密度，无显式 z 空间要求	噪声强度 t 充当连续控制变量

对比项	Autoencoder（AE）	VAE	GAN	Diffusion
主要毛病	无法生成	显式建模高维分布很难	梯度消失 / 模式崩塌 / 难训练	采样步数多（T = 1000）

AE 只会重建 → VAE 加正则让潜在空间可用 → GAN 绕过显式密度改用对抗 → Diffusion 放弃对抗改学一个已知的加噪过程。每一次转折都是在解决上一代的某个具体毛病：AE 的问题是不能生成，VAE 的问题是显式密度难写，GAN 的问题是训练不稳定，而 Diffusion 用有确定答案的回归问题换掉了没有确定答案的博弈问题。

Part 9 Overfitting and Underfitting 过拟合与欠拟合

9.1 三种拟合状态 Three Fitting Regimes

状态	定义	中文	图上特征
欠拟合 Underfitting（高偏差 High Bias）	Model too simple - cannot capture the underlying pattern. Both training and validation errors are high.	模型太简单，抓不住潜在规律；训练误差与验证误差都高	红色直线，几乎穿过数据的整体趋势，但完全跟不上那个弯
良好拟合 Good Fit（平衡 Balanced）	The model captures the true pattern without memorizing noise. Both errors are low and close.	抓住了真实规律而没有记住噪声；两个误差都低且接近	绿色曲线，与灰色虚线几乎重合
过拟合 Overfitting（高方差 High Variance）	Model too complex - memorizes training data including noise. Low training error but high validation error.	模型太复杂，把训练数据连噪声一起背下来；训练误差低，但验证误差高	蓝色曲线，穿过每一个黑点，在黑点之间剧烈扭曲

9.2 偏差-方差权衡 Bias-Variance Tradeoff

曲线	走向	含义
偏差平方 Bias² (underfitting)	随复杂度单调下降	Bias: Error from oversimplified assumptions. → Underfitting 来自假设过于简化的误差，对应欠拟合
方差 Variance (overfitting)	随复杂度单调上升	Variance: Error from sensitivity to small fluctuations in training data. → Overfitting 来自对训练数据微小波动过于敏感的误差，对应过拟合
总误差 Total Error	先降后升，呈 U 形	偏差与方差之和；最低点就是最优复杂度
不可约误差 Irreducible Error	一条水平虚线	与模型复杂度无关、降不下去的那部分误差

Find the sweet spot with minimum total error. 即找到总误差最小的那个最佳点(Optimal Complexity)。

在复杂度 0 处，偏差平方约 1.0、方差约 0.05、总误差约 1.05；随着复杂度增加到约 2，偏差降到约 0.7、方差升到约 0.3；到复杂度 10 时，偏差接近 0，但方差升到约 5。所以总误差的最低点并不在偏差最小或方差最小处，而在两者交叉附近。

权衡 Tradeoff: 降低偏差要靠更复杂的模型，而更复杂的模型必然带来更高的方差，两者是同一个动作的两面，不可能同时降。这与 1.6 节 的第三条（泛化）直接相关：Yet they don't overfit (when training data and model size is compatible). 只要训练数据量与模型规模相匹配，它并不因此过拟合。

9.3 诊断模型性能的学习曲线 Learning Curves

状态	定义	两曲线的形态	图上读数
欠拟合	Both training and validation errors are high and close together. Adding more data does not help- need a more complex model.	两条线都很高且互相贴近，且随数据量增加几乎不下降	训练约 0.38 → 0.36；验证约 0.47 → 0.42
良好拟合	Both errors are low and converge. The model generalizes well to unseen data.	两条线都低、且随着数据增多收敛到一起	训练约 0.19 → 0.09；验证约 0.28 → 0.13
过拟合	Large gap between training (low) and validation (high) error. More training data can help reduce the generalization gap.	两条线之间差距很大；图例把这段差距标为 Generalization gap，并用粉色阴影涂出	训练约 0.08 → 0.02；验证约 0.49 → 0.38

More training data can help reduce the generalization gap. 更多的训练数据可以缩小泛化差距。这与欠拟合的状态：Adding more data does not help（加数据没用）形成对照。

注意图上过拟合曲线在横轴右端仍然没有收敛，这正是在说数据还不够。

欠拟合：两条曲线都高且贴在一起，加数据只是让它们更贴，误差下不去 → 该换更复杂的模型。

过拟合：两条曲线差距大，加数据会让验证误差往下追训练误差 → 该去收集更多数据。

9.4 五条对策 How to Address Overfitting and Underfitting

这两类问题需要完全相反的动作。

序号	对过拟合（缓解）	对欠拟合（增强）
1	Regularization (L1/L2 penalty) , adds penalty for large weights. （对大的权重**加惩罚**）	Increase Model Complexity, Add more layers or neurons. （加层或**加神经元**）
2	Dropout, randomly deactivate neurons during training. （训练时随机失活一部分神经元）	Train Longer, Increase number of epochs. （增加训练轮数）
3	Early Stopping, Stop when validation error starts increasing. （早停：验证误差开始上升就停）	Reduce Regularization, Lower L1/L2 penalty or dropout rate.（降低惩罚项或失活率）
4	Data Augmentation, Increase training data diversity. （增加训练数据的**多样性**）	Better Feature Engineering, Add more informative features. （加更有**信息量**的特征）
5	Batch Normalization, Normalize layer inputs to stabilize training.（归一化每层输入以稳定训练）	Use a Different Architecture, try a more powerful model class (e.g., deeper network). （换更强的模型）

Part 10 Tutorial 5 Reinforcement Learning

Learning Objectives

1. 描述通过交互与延迟奖励来学习 describe learning through interaction and delayed rewards。
2. 把序列决策问题形式化成一个马尔可夫决策过程 formulate a sequential decision problem as a Markov decision process。
3. 区分策略、回报、状态价值与动作价值 distinguish policies, returns, and state and action values。

4. 解释贝尔曼最优方程与 Q-learning 更新式 explain the Bellman optimality equation and the Q-learning update。
5. 说出表格型 Q-learning 的主要假设与局限 state the main assumptions and limitations of tabular Q-learning。

10.1 Introduction: Reward Now or Later

近处有一颗小星星，远处还有一颗更大的，但要多走一步。智能体该选哪条路?

眼前收益 vs 长期收益。后面 p.9「等待的价值」会把这个问题算成具体数字。

注意插图本身不含技术信息（一个机器人在岔路口，左近右远）—— 本讲真正给出答案的是折扣因子 γ：γ 大就愿意等，γ 小就只顾眼前。

10.2 Reinforcement Learning

Reinforcement learning (RL) studies how an agent learns to choose actions through interaction with an environment to maximize expected cumulative reward.

强化学习研究智能体如何通过与环境交互来学会选择动作，以最大化期望累积奖励。

Actions can change both the immediate reward and future states. 动作会同时改变即时奖励与未来状态。

Feedback is typically a scalar reward, not the correct action for each state. 反馈是一个标量奖励，而不是每个状态下的正确动作，与监督学习的区别。

The agent must balance using what it has learned with gathering new information. 必须在使用已学到的信息与获取新信息之间权衡。

三类决策问题示例：robot control（机器人控制）、resource allocation（资源分配）、game playing（博弈）。

10.3 The Agent - Environment Interaction

在时刻t，智能体观察S_t、选择 A_t、先把动作做出去，环境才给出反馈，于是收到奖励R_{t+1} 与下一个状态S_{t+1}。

策略 policy：一个从状态到动作的映射，策略$\pi(a|s)$规定在状态 s 下如何选动作（给每个动作一个被选中的概率）。

轨迹 trajectory：训练数据，一条轨迹就是状态、动作、奖励交替出现的序列。

回合 episode：一条样本的结束，一个回合在终止状态（terminal state）结束；而持续性任务没有自然终点。LM的区别在于：RL 的样本不是别人给的，是智能体自己走出来的。An episode ends in a terminal state; Continuing tasks have no natural endpoint.

10.4 序列决策的挑战

问题1：Delayed consequences: an action may have little immediate reward yet improve later outcomes

延迟后果：某个动作即时奖励很少，却改善了后续结果。需要回报 G（把未来奖励折现加总），因为只看即时奖励就分不出好坏；

问题2：Uncertainty：the same action may lead to different next states or rewards

不确定性：同一个动作可能导向不同的下一状态或奖励。需要期望值 $E[・] $与转移概率 $p(s', r|s, a)$

问题3：Exploration：information about unfamiliar actions is acquired by trying them

探索：关于陌生动作的信息，只能靠去试才能获得。需要$ ε -greedy $这类行为策略

The objective is therefore to optimize a sequence of decisions, rather than each reward in isolation. 要优化的是一连串决策，而不是孤立的每一个奖励。

10.5 马尔可夫决策过程 MDP

Markov property: given the current state and action, the distribution of the next state and reward does not depend on the earlier history.

给定当前状态与动作时，下一状态与奖励的分布不依赖于更早的历史。

状态与可行动作 states S and available actions A(s)。可行动作依赖当前状态。

转移与奖励律： $p(s', r|s, a) = Pr(S_{t+1} = s', R_{t+1} = r|S_t = s, A_t = a)$。在 s 做 a 之后，下一状态是 s′ 且奖励是 r 的概率。

折扣因子：a discount factor $\gamma \in [0,1)$ for the discounted setting。

Tips：The state must therefore contain the information needed to predict the consequences of actions. If only partial observations are available, an MDP state may not be directly observed.

马尔可夫决策的工程依据：如果你的状态表示漏掉了预测所必需的信息，那它就不是一个合格的状态。只能拿到部分观测时，MDP 里的状态可能根本观测不到。

10.6 Return, Value, and Bellman Equations 回报、价值与贝尔曼方程

10.6.1 回报与学习目标 Return and the Learning Objective

The return from time t is the discounted sum of future rewards：从时刻 t 出发的回报（return），是未来奖励的折现和。 γ^k 是第 k 步之后那笔奖励的折扣权重。

$G_t = \Sigma_{k=0}^{\infty} \gamma^k R_{t+k+1}, 0 \leqslant \gamma < 1$

回合制任务：For an episodic task, the sum stops at the terminal time. 求和到终止时刻就停。

γ 的作用： γ controls how strongly later rewards contribute to the objective. γ 控制越晚的奖励对目标的贡献有多强。

$\gamma \in [0,1)$ 这个区间是为了保证求和收敛。如果奖励有界（比如 |R| ⩽ R_max），那么 $ |G_t| ⩽ R_max / (1 − γ)$ 越接近 1，这个上界越松，长期视野越长。 $\gamma = 0$ 时 $ G_t = R_{t+1}$，智能体只看眼前一步。

学习目标：The goal is to find a policy maximizing expected return，写成 $\pi^* \in argmax_{\pi} E[G_0]$（对某个指定的初始状态分布取期望）。

奖励设计也是问题的一部分：The objective depends on how rewards are defined; reward design is part of the problem formulation.

10.6.2 Example：The Value of Waiting

Suppose one action ends the episode with reward 2. Another yields 0 now and rewards 5 on the following step, then ends the episode.

路线	课件写法	含义
立即收益 Immediate reward	$G_t = 2$	做完这一步回合就结束，直接拿 2
延迟收益 Delayed reward	$G_t = 0 + \gamma(5)$	第一步拿 0，第二步拿 5，但第二步的 5 要乘 γ 折现

At $\gamma = 0.9$, the delayed route returns 4.5 and is preferable. At $\gamma = 0.3$, its return is 1.5, so the immediate route is preferable.

$\gamma = 0.9$ 时延迟路线的回报是 4.5（$0.9 \times 5 = 4.5 > 2$），优于立即收益；$\gamma = 0.3$ 时回报是 1.5（$0.3 \times 5 = 1.5 < 2$），立即收益更好。

γ 不是多重视未来这种含糊的说法，它直接改变最优决策。同一个环境、同一组奖励，只换了 γ，答案就从等翻成不等。取舍的分界点是 $\gamma \times 5 = 2$，即 $\gamma = 0.4$。$\gamma > 0.4$ 等，$\gamma < 0.4$ 不等，$\gamma = 0.4$ 两条路线完全相同（都是 2）。这个临界 γ 是判断这类题最快的方法。

10.6.3 状态价值与动作价值 State and Action Values

$v_\pi(s) = E_\pi[G_t \mid S_t = s]$

$q_\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$

$v_\pi(s)$：evaluates a state when actions follow π 评估在状态 s，之后都按策略 π 选动作时的期望回报。

$q_\pi(s, a)$：evaluates first taking a in s, then following π 评估在 s 先做 a，之后才按 π 走时的期望回报。因为选动作需要一个对每个候选动作打分的函数，而 v 给不出这个分数。

最优动作价值：对所有策略取最大，得到真正的最优动作价值。

$q^*(s,a) = \max_\pi q_\pi(s,a)$

在有限 MDP 里，对 q* 逐状态取 argmax 的策略就是最优策略。

10.7 从回报到贝尔曼期望方程 From Return to Bellman Expectation

10.7.1 Splitting the Return 把回报拆成现在 + 将来

The return separates into the next reward and the remaining return:

$G_t = R_{t+1} + \gamma G_{t+1}$

从今天看的回报 = 下一步的奖励 + 从下一步看的回报再折一次现。

1. 第一步，引入的是环境的不确定性（$p(s', r|s, a)$）：先对可能的下一状态与奖励取条件 condition on the possible next state and reward (s′, r)。
2. 第二步，引入的是策略的不确定性（$\pi(a'|s')$）：选择下一动作时按策略 π 走 follow policy π when choosing the next action a′。
3. 最后对所有这些结果取平均 average over these outcomes。

10.7.2 The Bellman Expectation Equation 贝尔曼期望方程

$q_\pi(s, a) = \Sigma_{s', r} p(s', r|s, a)[r + \gamma \Sigma_{a'} \pi(a'|s') q_\pi(s', a')]$

This equation separates action value into:

1. the immediate reward;

2. the discounted value of subsequent decisions under π.

方程	对应上一步	说明
$Σ_{s′,r} p(s′,r	s,a) [⋯]$	第 1 步 + 第 3 步
r	第 1 步	括号里的即时奖励部分
$γ Σ_{a′} π(a′	s′) q_π(s′,a′)$	第 2 步

意义：它把一个无限长的期望变成了一步 + 同一个量。右边又出现了 $q_π$ 自己（bootstrapping）：用下一状态的价值来定义当前状态的价值。正因为有这个自指结构，才能迭代求解，而不必把无穷级数真的展开。

At the terminal next state, the continuation value is zero. 若下一状态是终止状态，后续价值那一项为 0（此时只剩即时奖励）。

10.8 从策略评估到最优控制 From Policy Evaluation to Optimal Control

From policy evaluation to optimal control, The Bellman optimality equation replaces the policy average with the best available next action:

唯一改动是把平均换成取最大（最好的动作），把评估换成了优化

贝尔曼最优方程：

$$ q*(s,a) = Σ_{s′,r} p(s′,r | s,a) [r + γ max_{a′} q*(s′,a′)]$$

对比项	贝尔曼期望方程	贝尔曼最优方程
评估对象	$q_π$ 给定策略 π 的价值	< br > $q*$最优价值
下一动作	$Σ_{a′} π(a′ | s′) ·$按策略取加权平均	$(⋯)$
用途	策略评估：算这个 π 有多好	最优控制：算最好能有多好
对应的算法路线	迭代策略评估等评估类方法	Q-learning 等最优控制方法

Dynamic programming can use this equation when the transition and reward law is known. 动态规划（DP）能用这条方程，前提是已知 $p(s′,r|s,a)$；

Q-learning estimates q* from sampled transitions $(s,a,r,s′)$ without requiring that law. Q-learning 从采到的样本估计 q*，不需要知道这个定律。

10.9 From Bellman Optimality to Q-learning

10.9.1 三个替换 Three Substitutions

贝尔曼最优方程，需要对所有可能的环境输出结果求期望：$r + \gamma \max_{a'} q_*(s', a')$。

但环境模型未知的时候，智能体只能采样观察到单条状态转移元组 (s, a, r, s')。

因此需要做三处替换来得到可实际更新的目标：

1. Replace the unknown q_* with the current estimate Q. 把真实最优价值函数 q_* 替换成模型当前的估计值 Q。自举（bootstrap）
2. Use the observed reward and the next state to form a one - step target. 利用真实观测得到的奖励 r 和下一状态 s'，构造一步更新目标。 随机近似（用单个样本代替整体期望）

3. Move $Q(s,a)$ partway toward that target. 不再做完整平均，将 $Q(s,a)$ 向上面构造的目标值挪动一部分。步长 / 学习率控制更新幅度

10.9.2 The Q-learning Update

$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha_t(S_t,A_t)[R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a') - Q(S_t,A_t)]$

步长： $\alpha_t(S_t,A_t) \in (0,1]$ is the step size for the updated pair. α 是针对被更新那一对的步长，取值在 (0,1]。

The bracketed quantity is the temporal-difference (TD) error. 方括号整块就是时序差分误差。

终止状态：If $S_{t+1} $is terminal, the target is R_{t+1}: there is no future-value term. 若下一步是终止状态，目标就是 R_{t+1}，没有未来价值项。

更新式	作用
$R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a')$ TD 目标（one-step target）	用实际观测 + 当前估计拼出来的这一步应该是多少
$R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a') - Q(S_t,A_t)$ TD 误差 δ	目标比现值高多少；乘上 α 就是要修正的量
α_t 步长 / 学习率	一次修正走多远：$\alpha = 1$ 直接跳到目标，α 小则慢慢挪

α 的写法是 $\alpha_t(S_t,A_t)$ 而不是一个常数 α。这提示步长允许随状态动作对而不同，甚至允许随迭代轮次变化。

10.9.3 Example： Q-learning by Numbers

给定	值
当前估计 $Q(s,a)$	3
观测到的奖励 r	2
下一状态的最优估计 $\max_{a'} Q(s',a')$	4
折扣因子 γ	0.9
步长 α	0.5

第 1 步，先算 TD 目标： $ r + \gamma \max_{a'} Q(s',a') = 2 + 0.9(4) = 5.6。$

第 2 步，更新估计： $ Q_{new}(s,a) = 3 + 0.5(5.6 - 3) = 4.3。$

10.10 TD 误差 How to Interpret the TD Error

Define the one-step target and error as:

$ Y_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a')$

$ \delta_t = Y_t - Q(S_t,A_t)$

$Y_t $是一步目标， $ \delta_t $是TD 误差 = 目标 - 当前估计。

课件的读法：更新把$Q(S_t,A_t) *按比例*\alpha_t$ 朝样本目标$ Y_t $移动。

A positive δ_t means the sample target exceeds the current estimate; A negative value means it falls below it. δ_t为正表示样本目标高于当前估计；为负表示低于。

A single TD error is noisy, so its sign alone does not establish that the true action value is under-or overestimated. 单次 TD 误差是有噪声的，所以只看它一次的正负，并不能断定真实动作价值被低估或高估了。

10.11 Exploration and Exploitation： ε-greedy

以概率 ε_t随机选一个可行动作（探索）；其余概率下在 $argmax_a Q(S_t, a)$ 里选（利用）。

探索的作用：Exploration supplies information about less-visited actions. 提供较少被访问动作的信息。

利用的作用：Exploitation uses the currently highest-valued action. 使用当前估值最高的动作。

衰减必须有下限意义上的保障：A decay schedule must still provide sufficient visits to state - action pairs for tabular convergence claims to apply. 衰减方案仍须保证状态动作对获得足够的访问次数。

10.12 Quick Check: ε-greedy Behavior

一般式： $Pr(贪心动作) = (1 - \varepsilon) + \varepsilon \cdot (1/n)$，其中 n 是可行动作数（本题 n = 2）。当 n 很大时，探索带来的这 ε /n 就接近于 0，这也说明探索在大动作空间里更稀释。

10.13 Why Q-learning Is Off-Policy

> 核心现象： 智能体用 ε - *greedy*这种带探索的**行为策略去产生样本轨迹**； 但是更新目标里面用 $\max_{a'} Q(S_{t+1}, a')$，瞄准的是**完全贪心的目标策略**。
>
> 关键定义： The policy generating experience need not match the policy whose value is being learned. 生成样本的行为策略 ≠ 我们正在学习价值的目标策略 → 这就是 off - policy（离策略）。

off - policy ≠ 不需要探索 虽然更新不依赖行为策略，但依然要有充足探索。如果很多(s, a)从来没被访问，Q 得不到修正，max 就取不到真实最优值。

10.14 Convergence of Tabular Q-learning 表格型 Q - Learning 收敛条件

前提：有限折扣 MDP，奖励有界。两组**充分条件**（满足就一定收敛；不满足不代表一定发散，只是定理不作保证）

1. 每一个相关状态 - 动作对(s, a)被**无穷多次更新**。
2. 步长序列满足：

$\Sigma_n \alpha_n(s, a) = \infty, \quad \Sigma_n \alpha_n(s, a)^2 < \infty$

含义拆解：

$\Sigma \alpha_n = \infty$：总步长要足够大，保证参数可以走到目标，不会中途卡住。

$\Sigma \alpha_n^2 < \infty$：步长后期快速变小，压制样本噪声，不会持续震荡。

> 经典可行例子：$\boldsymbol{\alpha_n=\dfrac1n}$

两种不满足充分条件的实践做法：

1. **固定步长**：$\Sigma\,\alpha = \infty$成立，但$\Sigma\,\alpha^2 = \infty$，第二条不满足。工程上大量使用，但没有该理论收敛保证。
2. ε衰减到 0，部分(s, a)再也不会被采样：第一条无穷次访问不满足。

在全部充分条件成立下：**表格 Q - Learning 以概率 1 收敛到真实最优动作价值**q_*。

10.15 适用范围与局限（每一条对应一类后续算法）

1. **内存爆炸**：Q 表内存大小正比于状态 - 动作对总数，无法处理巨大、连续状态空间。

A Q-table requires memory proportional to the number of state - action pairs and is unsuitable for very large or continuous spaces.

2. **稀疏 / 延迟奖励**：很难做探索和信用分配。

Sparse or delayed rewards can make exploration and credit assignment difficult.

3. **部分可观测 POMDP**：需要额外记录历史信息或者更强模型。

Partial observability requires a representation of relevant history or a richer model.

4. **函数逼近（神经网络如 DQN）**：可以跨状态泛化，**但是表格 Q - Learning 那一套收敛保证不会自动继承过来。**

Function approximation can generalize across states, but tabular convergence guarantees do not automatically carry over.

> 这四个局限催生三大方向：近似价值函数、策略梯度、基于模型强化学习。

The agent may behave according to an exploratory policy, such as ε-greedy, while the update targets the greedy next action through $max_{a'}\ Q(S_{t+1}, a')$.

智能体可能按探索性策略行动，但更新式瞄准的是贪心动作，与正在执行的策略无关。

Thus the policy generating experience need not match the policy whose value is being learned.

产生经验的策略，不必等于被评估价值的那个策略。这就是 off-policy（离策略）的含义。

This does not remove the need for adequate exploration. 但 off-policy 不等于不需要探索。行为策略可以随便与可以不去试是两件事。如果没有足够探索，Q 里那些没被访问过的状态动作对就永远得不到修正，max 也就取不到真正的最大。

Overall

1. RL optimizes expected long-term reward through interaction. RL 通过交互优化期望长期奖励。
2. An MDP formalizes states, actions, rewards, and transition dynamics. MDP 把状态、动作、奖励与转移动态形式化。
3. Bellman equations relate current values to future values. 贝尔曼方程把当前价值与未来价值联系起来（这是能迭代求解的根源）。

4. Q-learning uses sampled transitions to estimate optimal action values. Q-learning 用采样的转移估计最优动作价值。
5. Reliable learning depends on exploration, suitable step sizes, and the assumptions of the chosen model. 可靠的学习依赖探索、合适的步长、以及所选模型的假设三者齐备。

目标 → 形式化 → 方程 → 算法 → 边界。G_t 怎么定义 → MDP 三要素 → 贝尔曼期望 vs 最优（就差一个 max）→ 更新式的四块（现值、奖励、折现后继、步长）→ 收敛条件与四条局限。

5434 Big Data Computing 1

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_5/knowledge/5434-Big-Data-Computing-1.md`

5434 Big Data Computing1

Lecture 4: Hadoop and MapReduce

part 0 Navigation

Hierarchical Structure

层级	课件原文	内容	与本课程的关系
第 1 层	**Data** 数据	Images, text documents, social networks, spatial data (GPS), time series data, transaction networks, gene networks, ⋯	Week 1 - 3 处理过的对象
第 2 层	**Data analytics** 数据分析	Clustering, Community detection, Association rules, Classification, Neural Networks, Graph algorithms ⋯	**Week 1 - 3 的内容全在这一层**
第 3 层	**Computational infrastructures** 计算基础设施	Hadoop, MapReduce, Spark, Hive, RDBMS, KV database, Graph database, ⋯	**本讲与下一讲的内容**
第 4 层	**Applications** 应用	FinTech, Smart City, Bioinformatics, Social Media, Tourism, Transportation, ⋯⋯	最终落点

本节主讲：**当数据装不下一台机器时，把存储和计算都拆开、并把计算搬到数据旁边去。**这就是Hadoop的工程实现，MapReduce 是它的计算模型，整讲都在回答“为什么必须这样拆、拆了之后要额外解决哪些问题”。

全篇脉络:

100 PB 装不下单机

多线程只能解决效率（存储与内存仍无解） 跨机器并行会 lost update（读-改-写非原子） 加锁能保正确但降低并行度（reduced parallelism） 改用“各写各的局部表 + 最后合并”——不需要共享可写状态 这条思路就是计算向数据并拢MapReduce（move operations near the data） 实现为分治法Partition/Combine = Map / Shuffle and Sort / Reduce → 用户只写两个函数，运行时自动并行、容错、均衡、限制通信

板块	内容
Hadoop · Introduction	大数据四 V、两个核心关切、Hadoop 的定义与四个核心组件、历史、里程碑、设计目标
Hadoop · Ecosystem	四个核心模块 + 九个相关项目
MapReduce · Introduction	出身、收益、Google 的用途、典型应用
MapReduce · Motivations	词频统计算例、单机瓶颈、丢失更新、加锁代价、把计算搬到数据旁、分治法
MapReduce · Technical Ideas	并行编程的六类难题、负载均衡、MapReduce 隐藏了什么、三条设计立场、工作流全景
**MapReduce ** · Workflow and Programming	Map / Reduce 抽象、伪代码逐行解读、数据分区、流水线、可解问题、使用步骤、局限

part 1 Hadoop Introduction

1.1 background

大数据4V：Volume, Velocity, Variety, Veracity（体量、速度、多样性、真实性）;

数据经济：We are in a data/knowledge economy 我们处在数据/知识经济之中;

1. 数据是资产：Data is an important asset to any organization 数据对任何组织都是重要资产，需要存储。
2. 知识发现 Knowledge Discovery：基因组（速度）、奥巴马竞选（个体化）、Netflix 纸牌屋（内容决策）

1.2 Two Main Concerns

How to process such a huge amount of data in a distributed system?

处理：并行度与同步——Hadoop MapReduce

How to store such a huge amount of data in a distributed system?

存储：容量与可靠性——Hadoop Distributed File System (HDFS)

1.3 Definition, Goal and Components

1.3.1 Definition

1. **Hadoop**: an open-source software framework that supports **data-intensive distributed applications**, licensed under the **Apache v2** license.

data-intensive：数据密集而不是计算密集，所以瓶颈在 I/O 与存储，不在 CPU，把计算搬到数据旁边。

Apache v2 license**：**允许商用、允许修改与再分发。Hadoop 低成本的法律基础。

2. **HDFS **：A distributed file system that runs on large clusters of commodity machines. 运行在大型商用机器集群上的分布式文件系统。
3. **YARN **：A framework for job scheduling and cluster resource management. 作业调度与集群资源管理的框架。

1.3.2 Goal

Goals: to solve the problems about process and storage of huge amount of data解决.大规模数据的处理与存储问题。

1. 抽象并简化存储与处理

Abstract and facilitate the storage and processing of large and/or rapidly growing data sets. 抽象并简化大型（体量大）以及快速增长（在线扩容）的数据集的存储与处理。

2. 两类数据都要支持结构化与**非结构化**数据。这是 HDFS 与 HBase 并存的原因。
3. **简单的编程模型 Simple programming models**。
4. 高可扩展与高可用 **High scalability and availability** 。

open-source implementation of frameworks for reliable, scalable, distributed computing and data storage. 可靠（reliable）、可扩展（scalable）、分布式（distributed）的计算与数据存储。

5. 便宜、冗余少的商用硬件 **Use commodity hardware with little redundancy**

a flexible and highly-available architecture for large scale computation and data processing in a network of commodity hardware. 硬件前提: 面向商用硬件网络的、灵活且高可用的架构。

6. 容错 **Fault-tolerance**

这是用廉价硬件的必要补偿：硬件不可靠，软件必须可靠。

7. 把计算搬向数据 **Move computation rather than data**

移动计算而不是移动数据。数据量大、移动成本高，所以把程序送到数据所在节点执行。

因果链	内容
成本 → 容错	用 **commodity (cheap!) hardware with little redundancy** ⇒ 硬件会坏 ⇒ 必须有 **Fault-tolerance** ⇒ 由软件层（HDFS 副本 + 任务重试）承担
数据大 → 移动计算	**data-intensive** 且数据**rapidly growing** ⇒ 移动数据的 I/O 成本极高 ⇒ 必须 **Move computation rather than data** ⇒ 把操作搬到数据旁

1.3.3 Components

组件	职责	解决的问题
Hadoop MapReduce	a system for parallel processing datasets 并行处理数据集的系统	处理
Hadoop Distributed File System	a system for distributed dataset storage 分布式数据集存储系统	存储
Yet Another Resource Negotiator（Hadoop YARN）	a framework for job scheduling and cluster resource management 作业调度框架和集群资源管理	支撑前两者
Hadoop Common	Common utilities that support Hadoop 支撑 Hadoop 的通用工具	支撑前三者

1.4 Hadoop and MapReduce

1. MapReduce是一个计算模型（a Computational Model/Program）

2. Apache Hadoop是MapReduce这一思想的开源实现（an open-source implementation of the MapReduce idea）

MapReduce这个思想不只Hadoop一个实现，它是 Google 提出的编程模型；Hadoop 只是Apache基金会把它工程化、开源化的产物。It is not a program, but an idea。

1.5 Question and founders

1. The Question: "How to process big data with reasonable cost and time?" 如何以合理的成本与时间处理大数据?

传统方案用**高端服务器**换时间（scale up），成本极高；

Hadoop选择用**大量廉价机器**换时间（scale out），把可靠性问题交给软件层（fault-tolerance）。

2. Google search engines

1998年的Google页面复杂，但2013年的Google界面极简，全部复杂度转移到后端。15 年间前端极简化、后端能力膨胀。用户看到的越简单，背后的分布式计算需求越大，所以需要Hadoop。

3. 从论文到 Apache

Nutch（2002，Doug Cutting + Mike Cafarella）→ GoogleFileSystem论文（2003，存）→ MapReduce论文（2004，算）→ Yahoo 资助 → 2008 移交 Apache

1.6 Google Origins

2003 存：GFS论文 → HDFS。HDFS 不是 Google 的实现，而是这篇论文的开源复现。

2004 算：MapReduce论文 → Hadoop MapReduce。这是本讲的主角。

2006 结构化存储：Bigtable 论文 → Apache HBase。解决结构化数据的分布式存储。

part 2 Hadoop Economy

2.1 Four Hadoop Core Modules

模块	课件定义原文	关键词
Hadoop Common	A set of components and interfaces for distributed file systems and general I/O	components and interfaces（组件与接口）、general I/O（通用 I/O）
Hadoop Distributed File System (HDFS)	A distributed file system that runs on large clusters of commodity machines	runs on large clusters（跑在集群上）、commodity machines（商用机器）
Hadoop MapReduce	A distributed data processing model and execution environment that runs on large clusters of commodity machines	**model and execution environment**（模型**与执行环境**）
Hadoop YARN	A framework for job scheduling and cluster resource management	job scheduling（作业调度）、cluster resource management（集群资源管理）

2.2 Nine Hadoop-Related Projects

项目	课件定义原文	中文与归类
Ambari	A tool for provisioning, managing, and monitoring Hadoop clusters	集群**配置 / 管理 / 监控**工具 → 运维类
Avro	A serialization system for cross-language RPC, and persistent data storage	**跨语言 RPC** 的序列化系统、持久化数据存储 → 数据格式类
Cassandra	A scalable multi-master database with no single points of failure	可扩展的 **multi-master** 数据库，**无单点故障** → NoSQL 类
Chukwa	A data collection system for managing large distributed systems	大型分布式系统的**数据采集**系统 → 采集类
HBase	A distributed, column-oriented database	分布式**列式**数据库 → 数据库类（对应 2006 Bigtable）
Hive	A distributed data warehouse provides a query language based on SQL	分布式**数据仓库**，提供基于 **SQL** 的查询语言 → 数仓类
Mahout	A Scalable machine learning and data mining library	可扩展的**机器学习与数据挖掘**库 → 算法类
Pig	A data flow language and execution environment for exploring huge datasets	**数据流语言**及其执行环境，用于探索海量数据集 → 语言类
ZooKeeper	A distributed, highly available coordination service	分布式**高可用协调服务** → 协调类

Tips:

1. **HBase vs Hive：HBase 是数据库**（列式，直接存取行/列）；Hive 是**数据仓库**（用类 SQL 语言做批式查询）。一个是存储引擎，一个是查询层。
2. **Pig vs Hive：两者都是用更简单的语言写MapReduce，但 Pig 是数据流语言**（过程式），Hive 是**类 SQL**（声明式）。
3. **Cassandra vs HBase：**Cassandra 的特点是multi-master + no single points of failure（无中心节点，任一节点可写）；HBase的定位是 column-oriented（列式）。

part 3 MapReduce Introduction

3.1 Definition

1. MapReduce is a programming model Google has used successfully for processing its "big-data" sets. 它是一个Google 成功用于处理其大数据集的编程模型。
2. A framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. 它是一个允许用简单编程模型在计算机集群间分布式处理大数据集的框架。
3. It is not a program, but an idea. MapReduce 是方法论层面的东西，不是一份可以下载运行的软件。
4. Hadoop MapReduce is an implementation of MapReduce. Hadoop MapReduce 是它的实现。

MapReduce 是什么？它是 Google 提出的编程模型（不是程序），是允许在计算机集群上分布式处理大数据集的框架；Hadoop MapReduce 是它的开源实现，其底层运行时自动完成并行化、容错与通信。

3.2 Function

课件原文	含义	对应 Part IV
The underlying runtime system automatically parallelizes the computation across large-scale clusters of machines	底层运行时**自动**在大型集群上把计算并行化	automatic parallelization and distribution
The underlying system also handles machine failures	运行时负责**处理机器故障**	fault-tolerance、assume failures are common
…efficient communications, and performance issues	以及**高效通信**与**性能问题**	网络与磁盘传输优化

**Tips: **

1. 自动化：这些在传统并行编程里必须手写的部分（调度、同步、容错、通信），MapReduce 全部自动化。
2. runtime system：它说明MapReduce包括一个在背后干活的系统（runtime）。用户只写Map和Reduce两个函数，剩下的交给runtime。

3.3 Benefits

含义	机制
突破存储与计算的传统限制	把两者都拆到集群上
以廉价商用硬件为平台	commodity (cheap!) hardware
提供从1到5000台服务器的线性可扩展性（scale out）	scale out：加机器即扩容
低成本、开源软件	Apache v2 license

Tips:

linear scalability线性可扩展性：**性能随机器数量近似线性提升，**2 倍机器约 2 倍吞吐。这个性质的前提是任务之间足够独立（**数据量大而处理独立：索引、聚类、报表、影像、语言模型、备份，每一项都是逐条/逐片处理**，条目之间不需要互相等待），这正是MapReduce Pipeline要处理的问题（限制进程间通信量）。

3.4 Scale Out & Scale Up

维度	scale out（横向扩展）	scale up（纵向扩展）
做法	增加**机器数量**（1 → 5000 台）	提升**单机性能**（换更强的服务器）
成本曲线	线性、可用廉价硬件	非线性、须高端专有硬件
上限	理论上可持续加机器	受单机物理上限限制
原文	**A large number of commodity low-end servers is preferred over a small number of high-end servers**	—
代价	必须解决**故障**与**通信**问题	成本高

3.5 Application

应用	中文	为什么适合 MapReduce
distributed pattern-based searching	分布式模式检索	逐条记录独立匹配模式，天然可并行
distributed sorting	分布式排序	分片局部排序 + 归并（Shuffle and Sort 的核心操作）；也是2008基准测试的题目
web access log stats	网站访问日志统计	日志逐行处理 + 按 key 聚合（与词频统计同构）
inverted index construction	倒排索引构建	词 → 文档列表的聚合，Map 输出 (词, 文档)，Reduce 汇总（**典型的两阶段结构**）
document clustering	文档聚类	Google News
statistical machine translation	统计机器翻译	需要大规模语料上的n-gram统计

**Tips: **

1. **倒排索引为什么是标准范例：**它是**结构上最贴合 MapReduce 的问题，**Map 阶段逐文档产出 (term, docID) 对，Shuffle 按 term 分组，Reduce 阶段把同一 term 的所有 docID 合成列表。与词频统计（Map 出 (word,1)、Reduce 求和）只差累加与收集的区别。

part 4 MapReduce Technical Ideas

本节思路：100 PB 装不下 → 多线程只解决效率 （efficiency）→ 跨机器并行会丢更新（lose update）→ 加锁代价高 （lock solution）→ 改用局部状态 + 最后合并（MapReduce机制和分治法）→ 这就是 MapReduce 的起点

4.1 Example：Word Count

问题：为什么统计一堆文档里面的词频单机做不了，MapReduce能做?

MapReduce 能默写 输入：文档集合；输出：<term, frequency> 列表

1. 结构最简：只需「拆分 → 计数 → 合并」三步，没有迭代、没有共享状态、没有先后依赖。典型解题四步正是从这里抽象出来的。
2. 可扩展性最强：把 100 PB 文档切成 N 片，每片独立计数，最后把 N 张局部表按词相加即可，这就是 Separated data / Separated counters图的全部内容。
3. 与其他任务同构：倒排索引、访问日志统计、n-gram 统计都是同一个形状，因此这个词频统计的解法可以原样迁移。

4.2 Other Applications

应用	是什么	与词频表的关系
Word cloud 词云	按词频决定字号的可视化	**直接**用频次作字号权重
Inverted index 倒排索引	从文档 → 词翻转为词 → 文档列表	把频次换成文档 id 列表，聚合结构相同
TF-IDF	词频 × 逆文档频率，衡量词对文档的代表性	**TF 就是词频**；IDF 需要该词出现在多少篇文档中，这也是一次按词聚合

应用	是什么	与词频表的关系
Google Trend	热门查询趋势报表	与Google 用例直接对应，是查询日志的词频统计

1. 四个应用的共同点：都需要先按词聚合。词频表就是这个聚合的结果，因此它是这些任务的公共前置步骤。
2. TF-IDF 的特殊地位：两个因子（TF 与 IDF）都能由一次 MapReduce 式的按词聚合算出。
3. 倒排索引的计算量：在 Google 的规模下，倒排索引的构建正是 MapReduce 的发源场景。

Lecture2 中的 TF-IDF 与余弦相似度是用已经统计好的词频向量算的；本章说明那些词频向量本身也需要一个可扩展的方式来算。

4.3 ** **Three Single Machine Problems

词频统计在单机上的三个瓶颈及MapReduce的解决：

瓶颈	具体含义	MapReduce 的解法
Efficiency issue 效率问题	单机串行扫描 100 PB 需要天文数字的时间；CPU 与磁盘带宽是上限	**并行化：**多 worker 同时处理不同分片
Storage issue 存储问题	一台机器装不下 100 PB；即使装下也无法可靠维护	**HDFS：**切块 + 多副本存在集群上
Memory issue 内存问题	词表本身可能大到装不进内存（不同词的个数也可以是上亿级）	**局部聚合：**每个 worker 只维护自己分片的局部计数表，最后再合并

三个问题的独立性：三者不能互相替代，加 CPU 不能解决存储（数据还是装不下）；加磁盘不能解决效率（单机 I/O 带宽有限）；加内存不能解决前者（100 PB 远超任何单机内存上限）。必须同时换三套方案。

4.4 For Efficiency Issues

4.4.1 Naive Solution by Multiple Threading

用多线程来应对效率问题（注意：本质是单机内的优化，所以只提效率，不能扩容存储与内存，后面必须走分布式）。

朴素方案：一份 Data collection 输入，连接到n 个计数器线程并行处理。单核串行变成多核并行，CPU 利用率提升，**效率问题的单机部分**被缓解。

弊端：1. 数据仍要**全部在一台机器上**（存储问题未解决）；2. 所有线程**读同一份数据**，I/O 带宽成为新瓶颈；3. 各线程若共享一张全局计数表，就埋下了丢失更新的隐患。

4.4.2 The Worker: Generalize to a larger scope

Worker: a basic instance to undertake the tasks. 承担任务的基本执行单元 · A worker can be a thread or a processor. · Workers can be in a single machine or multiple machines.

Worker 的实现形态	所在位置	对应场景
一个**线程**（thread）	单台机器内	多线程方案
一个**处理器**（processor）	多台机器上	真正的分布式计算

Tips: 实际上，所有并发问题（同步、共享状态）在单机与多机上是同一类问题，不必重新发明一套问题描述。

4.4.3 Distributed Computing

课件原文	含义
A cluster of machines/nodes 一个机器/节点集群	分布式计算的最小前提：多台机器组成集群
Data is **split** into several parts and **stored** in distributed machines 数据被切成若干部分存放在分布式机器上	数据**先切分再分布**（这就是 HDFS 的角色）
Multiple workers work together simultaneously 多个 worker 同时协同工作	并行不是可选项，而是定义的一部分

Tips: This requires careful programming. Otherwise, we risk getting incorrect results! 因为并行的风险不只是慢，而是**答案****错，并行与正确性之间存在张力。**并行计算不会自动得到正确结果，需要谨慎的编程（careful programming）**

4.4.4 Lost Update

4.4.4.1 Lost Update Problem

为什么并行的风险是错呢?

- **场景描述**：介绍了一个全局数组“count”，存储在中心节点，并可由两个工作者访问。用红色文本强调潜在错误。假设两个工作者正在处理相同的术语“apple”。指出这个问题不仅存在于分布式系统中，也适用于本地多线程。
- 一个标记为“工作者”的框中包含5步伪代码算法：“读-改-写”循环
- **时间线**：**工作者1（蓝色柱）**：执行“读”操作，然后进行“写”操作；**工作者2（粉色柱）**：在工作者1的读取后执行“读取”，在工作者1的写入后进行“写入”。
- **结果**：数据库中的最终值为1，正确应为2，一个增量操作被覆盖，实际丢失了。
- **原因：**Worker 2 的那一次读发生得太早，读到了0，写发生得太晚。**根因是「读-改-写」不是原子操作，**中间有可被交错的时间窗口。
- **为什么两个 worker 同时处理同一个词：这正是词频统计的常态。因此这个冲突不是极端情况，而是必然发生。**

4.4.4.2 Locking-based Solution

又如何解决呢?

A locking protocol may be used to ensure correctness. A worker must hold a lock on "count[t]" before using Read/Write.

- **执行调度表**：将“工作者1”和“工作者2”分成两列。
- **工作者1列**：“Lock (count["apple"])” & “Read (count["apple"])”
- **工作者2列**：显示工作者1持有锁时的情况：
    1. “Wait for Lock(count["apple"])”用红色文本书写。
    2. 一个省略号（“...”）表示工作者2被阻塞并在等待。
- **锁机制的工作原理：**当工作者1获得对特定资源（在此为“apple”的计数）的锁时，工作者2必须等待该锁被释放，这也体现了上文提到的“降低并行性”的概念。

- **缺点**：1. 复杂性增加：实现代码变得复杂。2. 运行时开销高：导致“降低并行性”。

4.4.4.3 The Cost of Solution

加锁这种方式的弊端

代价	含义	后果
Complicate implementation	使实现变复杂	程序员必须自己管理锁的获取、释放、异常路径，这与Simple programming models（只写map和reduce代码）目标背道而驰
Expensive runtime overhead: reduced parallelism	运行开销昂贵：并行度下降	这是更致命的一条，为了正确性付出的是性能，而性能正是当初并行的目的

1. 排队意味着串行化，并行度直接掉下来。
2. **锁的粒度问题：**若锁的粒度是单个词，不同词的worker仍可并行；但若共享表很大，锁管理的开销本身也会成为负担。
3. **与三个瓶颈的关系：**加锁只解决正确性问题，对 Storage issue 与 Memory issue 完全无解，还额外拖慢了效率。

4.4.5 The Core Idea of MapReduce

Q：How to ensure correctness without or with less usage of locking?

加锁有很多问题，如何少用or不用锁呢?

A：引入Core Idea：move operations near the data

把操作移向数据，再用局部状态代替共享状态。把加锁的全局表换成各写各的局部表最后并行。

1. Let each worker use its local storage (hard disk) to store a local array "count"

每个 worker 有一份**自己的**局部计数表（存在本机磁盘上）

2. Each worker can only update its local array

worker **只写自己的表**，绝不碰别人的

3. When workers finish their tasks, they send their local arrays to the central node

全部完成后，把局部表**上交中心节点**

Cost：引入了一次**额外的通信**（上交局部表）与一次**额外的合并步骤**（把局部表相加）。这个代价可接受，因为合并发生在**所有 worker 完成后**，可以批处理，不打断并行。

4.5 For Memory and Storage Issues

4.5.1 Example: Peta-Scale Data

Peta-scale data is commonly distributed from data that can be easily stored or processed on a single machine.

大尺度数据通常来自那些可以在单台机器上轻松存储或处理的数据

主题	与本节的关系
两个核心问题（处理和存储）	提出单机服务不了全部数据

主题	与本节的关系
100 PetaBytes + 三个瓶颈	给出具体体量与三条困难
集群与数据切分	给出物理形态（机器集群 + 切分后的数据）
Peta 级数据天然分布式（本节）	把分布式从方案确认为前提
分治法 Divide and Conquer	给出处理前提（分布式）下的方法论

Peta 级数据无法在单机上存储/处理，因此必须是分布式的。

4.5.2 Divide and Conquer 分治法

方法论层面的答案：**分治法**

标签	含义	MapReduce 术语	阶段
Partition	把一个巨大工作拆成三份子工作，分给三个 worker 各自计算，得到三个局部结果，最后合成 Result	输入切分（Split）+ 任务分配	Map 之前（由运行时完成）
各 worker 独立计算		Map + 本地聚合	Map 阶段
（中间的通信）		Shuffle and Sort	Map 之后、Reduce 之前
Combine	把分与合正式命名为两个阶段	Reduce	Reduce 合并阶段

1. **split 与 merge 成对出现：**每条边上都有 **split / merge**（split 用粉色箭头向下，merge 用蓝色箭头向上），在后面的内容中把这一页的两条边正式命名为 **Partition**（对应 split）与 **Combine**（对应 merge）。
2. **递归结构：从 1 个 Problem 到 2 个 Subproblem 再到 4 个 Compute Subproblem，是逐层二分。**实际的 MapReduce 分多少层都可以。
3. **compute 的位置：计算发生在叶层**（Compute Subproblem），中间层只负责**拆与合**。这对应 MapReduce 的两个阶段：Map 在叶层干活，Reduce 负责合并。

分治法与前面各节的关系

前面讲的问题	分治法给出的答复
数据装不下	把 Problem **split** 成多个 Subproblem，每个都能放进一台机器
两个 worker 会冲突	让各 Subproblem **独立计算**，不共享可写状态
加锁拖慢	只在 **merge** 阶段通信，中间过程无需同步
结果怎么合	**merge** 就是把各局部结果合并的一步

Tips: 分治法不保证正确：分治法给出的是结构，不是正确性保证。任务怎么分给 worker？部分结果怎么合并？worker 死了怎么办？这些问题分治法自身并不回答。

part 5 MapReduce Workflow and Programming

分治法（Partition/Combine）→ 六类难题 → 负载均衡 → 自动化与三条设计立场 → 五段工作流 → 四步解题范式

5.1 Basic Workflow of MapReduce

5.1.1 Input and Output

1. 输入是一个目录或数据集，不是单个文件：Peta-scale data collection意味着输入本身就是一个许多文件的集合，切分粒度在运行时决定。

2. 输出是<key, value> 对的列表：词频表“hadoop 1”就是 key=hadoop、value=1。这个形态是Map与Reduce的接口。

3. 输入输出是一对多与多对一的关系：合并步骤的通信量远小于输入数据量，输入很大（Peta 级）、输出很小（一张词频表）。The output usually contains a few pairs compared to input。

5.1.2 DIvide and Conquer

MapReduce的最简形式：Split → Parser → Counter → 汇总

有Separated data and counters 独立的数据和计数器（数据分治+局部数组）

级	节点	职责	MapReduce 对应
第 1 级	**Split 1 ⋯ Split n**	把数据切成 n 份（分开的数据）	**Split：**输入分片
第 2 级	**Parser 1 ⋯ Parser n**	逐份解析（切词）	**Map：**把记录变换成 <key, value> 对
第 3 级	**Counter 1 ⋯ Counter n**	各自统计本份的局部计数（分开的计数器）	**Map 的本地状态 / Combine：**局部聚合
汇总	（虚线箭头汇入右侧一张表）	把 n 张局部表合并	**Reduce：**按 key 合并

**Tips: **

1. **Parser 与 Counter 完全可并行：**第 i 份数据的解析与计数只依赖第 i 份数据，与其他份无关，任务之间足够独立（linear scalability的前提）。
2. **汇总必须等待：**右侧那张表要等所有 Counter 都完成才能给出最终结果，**All Map should be completed before Reduce starts**（所有 Map 必须在 Reduce 开始前完成）。

5.2 Challenges

给出一个具体的集群规格，在这样的集群上写程序要考虑什么？这些问题正是 MapReduce 运行时系统要自动处理的清单。

1. How do we assign work tasks to workers?

怎么把工作分配给 worker？调度问题。

2. What if we have more work tasks than workers?

任务比 worker 多怎么办？需要任务队列与排队机制。

3. What if workers need to share partial results?

worker 之间需要交换部分结果怎么办？通信问题。

4. How do we aggregate partial results?

怎么汇总部分结果？合并问题。

5. How do we know all the workers have finished?

怎么知道所有 worker 都完成了？同步/终止检测问题。

6. What if workers die?worker

死了怎么办？容错问题（Assume failures are common）。

类别	对应问题	MapReduce 的处理
调度与负载	1、2	**5.4 load balancing**；任务数可配置
通信与同步	3、5	**Limit the amount of communication**；Map 全部完成后才进 Reduce
结果合并	4	**Shuffle and Sort + Reduce**
容错	6	**handling machine failures；Assume failures are common**

5.3 Managing Multiple Workers

不知道 worker **运行的顺序**。不知道 worker **何时互相打断**。不知道 worker **何时需要交换部分结果**。不知道 worker **访问共享数据的顺序**。（5.4补充工作量不可知）

4.4.2 节把 Worker 抽象为一个执行单元；这一节说明这种执行单元的**时间行为不可预测**，这正是管理困难的全部来源。

工具	操作	用途
Semaphores 信号量	Semaphores (lock, unlock)	互斥：保证同一时刻只有一个 worker 进入临界区
Conditional variables 条件变量	Conditional variables (wait, notify, broadcast)	同步：让 worker 在条件不满足时等待，条件成立时被唤醒

broadcast 的含义：notify 唤醒一个等待者，broadcast 唤醒所有等待者。

5.3.1 工具管理共享状态后仍然存在的问题

Still, lots of problems: e.g., Deadlock. Cause of problems with resource access: They need to access shared resources.

死锁 Deadlock：加锁后多个 worker 互相等待对方持有的锁，谁也无法推进。

根因：They need to access shared resources 问题的根源是**worker仍然需要访问共享资源**。所以改用局部状态（如果不共享，这些工具与问题都不需要）。

5.4 Load Balancing

MapReduce的**调度目标**：让所有 worker 同时结束

Q 问题：如何优化worker运行和等待访问共享资源？原方案里Worker2承担了两个大任务，成为最慢的一环。总时间由最慢的worker决定，空闲的算力被浪费。

A 改进的办法：把Task4从Worker2移到Worker3，三条时间线的长度变得接近，最慢的那条变短，总时间随之缩短。图右侧用两条竖直红线标出Time Saved。

P 前提条件：这种移动任务只有在**任务之间互相独立**时才可能，与5.1.2节Split / Parser / Counter 同一前提。

Tips：任务的实际耗时取决于数据量，程序员事先无法精确预知。

所以Load balancing直接列为MapReduce运行时替用户完成的工作之一，因为手工做这件事需要预知任务耗时，而在集群上这是做不到的，所以MapReduce把这个自动化5.4.1。

5.4 Problem Types

类别	课件原文清单	解读
Fundamental issues 基本问题	Scheduling, data distribution, synchronization, inter-process communication, robustness, fault tolerance, …	调度、数据分布、同步、进程间通信、健壮性、容错——**与前两节的六个问题同源**
Architectural issues 体系结构问题	Flynn's taxonomy (SIMD, MIMD, etc.), network topology, bisection bandwidth, cache coherence, …	Flynn 分类（SIMD/MIMD）、网络拓扑、对分带宽、缓存一致性——**偏硬件与体系结构**
Common problems 常见故障	Livelock, deadlock, data starvation, priority inversion, … dining philosophers, sleeping barbers, cigarette smokers, … Load balancing,…	活锁、死锁、数据饥饿、优先级反转；以及经典的哲学家就餐、理发师睡觉、吸烟者问题——**并发编程的经典病**
Different programming constructs 编程构件	Mutexes, conditional variables, barriers, … masters/slaves, producers/consumers, work queues, …	互斥量、条件变量、屏障；以及主从模式、生产者/消费者、工作队列——**实现这些机制的工具与模式**

拓展：

Flynn 分类的两个缩写

1. SIMD：Single Instruction Multiple Data 单指令多数据。同一条指令作用在一批数据上（如 GPU 的向量运算）。
2. MIMD：Multiple Instruction Multiple Data 多指令多数据。各处理单元执行不同的指令流（Hadoop 集群属于这一类）。

5.4.1 Reason and Solution

**Problems arise from: **Access to shared resources (e.g., data), Communication between workers (e.g., to exchange state).

两个根因：**访问共享资源；worker 之间需要通信。**

**MapReduce: **Automate most of these for you.

MapReduce 把这些大部分自动化，把复杂性从程序员转移到运行时。从而实现了Hadoop的目标之一：Simple programming models。

注意是most而不是 all：MapReduce 不解决所有并行编程问题，只自动化了它面向的那一类（批式、读多写少的大数据处理）。

5.5 MapReduce Features

Hide messy details in MapReduce **runtime library**

关键是 **runtime library** 复杂性被搬进**运行时库**，因此用户只需写Map与Reduce函数。

Features	中文	对应5.2的问题
automatic parallelization and distribution	自动**并行**化与**分发**	怎么把任务分给 worker（问题 1）
load balancing	负载均衡	Load imbalance（问题2）

Features	中文	对应5.2的问题
network and disk transfer optimization	**网络与磁盘**传输优化	worker之间怎么交换结果（问题 3）+ 节的通信虚线
handling machine failures	机器故障处理	worker 死了/终止怎么办（问题 5，6）+ 节的容错
inherently designed for batch and high throughput processing jobs	天生面向批式与高吞吐的作业	说明它的适用范围，不是通用并行框架 Note that MapReduce is not designed to solve all kinds of problems. 它是**有适用边界**的框架

三位一体：**scale out → failures common → co-location**

1. Scale "out", not "up" **横向扩展**而不是纵向扩展。

A large number of commodity low-end servers is preferred over a small number of high-end servers. 更倾向于使用大量低端的商品服务器，而不是少量高端服务器。

out 必然导致 failures：服务器数量一多，按概率就有机器在坏。选择廉价硬件就是选择接受故障，所以假设故障是常态。

2. Assume failures are common 假设故障是常态。

Fault-tolerant service must cope with failures **without impacting the quality of service. **容错服务必须能够应对故障，而不影响服务质量。

failures 必须靠 co-location 缓解：如果计算与存储分离，机器一坏就要跨网络重传数据，代价极高；把 processors与storage放在一起，坏了一台只需把它的任务在别处就地重跑（因为那里也有同一份数据的副本，HDFS备份机制）。

3. Moving processing to data 把处理移向数据。

MapReduce assumes an architecture where processors and storage are co-located. MapReduce 假设处理器和存储是共址的架构。

processors and storage are co-located：前提是MapReduce 假设每一台机器既是数据节点也是计算节点，这是计算移到数据的物理实现，并与**HDFS配合保证数据在本地备份**，使 co-location 的假设在发生故障后依然成立。

「without impacting the quality of service」的分量 ・容错不只是「不崩」，而是服务质量不下降——用户不应感知到某台机器坏了。这是一个较强的要求，靠的是副本与任务重试。

5.6 MapReduce Workflow

Split → Map（local write）→ Shuffle and Sort（remote read, sort）→ Reduce（write）→ Output File

1. Map的定义

Map: a mapping that is responsible for dividing the data（划分数据） and transforming the original data into key-value pairs（把原始数据变换成键值对）.

2. Shuffle and Sort 的定义

Shuffle and Sort: the process of further organizing（组织） and delivering（传递） the Map output to the Reduce.

The output of the Map must be sorted（排序） and segmented（分段/分组），

then passed（传递） to the corresponding Reduce

Communication occurs（唯一的跨网络通信发生）.

3. Reduce 的定义

Reduce: a merge（合并） that processes the **values** with the same key and then outputs to the final result.

段	元素	说明
Input Data	Split 0 / Split 1 / Split 2 三个红框，带一条 read 箭头	输入数据被切成三片；read表示本地读取
Map	extract something you care about from each record 从记录提取需要的信息	3个worker 并行做 Map；输出本地写出local write（不跨网络）
Shuffle	remote read, sort	跨网络读取各 Map 的输出并排序，这是唯一的跨节点通信段
Reduce	aggregate, summarize, filter, or transform 聚合、汇总、过滤或变换	2个worker做 Reduce，把结果写出
Output Data	Output File 0 / Output File 1	Reduce 的输出文件；两个 Reduce ⇒ 两个输出文件

Tips:

1. local write与remote read： Map 的输出是本地写（local write），Shuffle 是远程读（remote read）。跨网络的数据移动只发生在 Shuffle 这一步。
2. Map 任务数与 Reduce 任务数可以不同。
3. Shuffle 的两个动作：remote read（去各 Map 节点把属于自己 key 的数据取回来）+ sort（把取回的数据按 key 排序，使同 key 的值相邻）。这两个动作合称 Shuffle and Sort，即将映射的数据进行组织和分组。
4. Map 是从记录中抽出（record → 若干键值对，一对多）；Reduce 是聚合/汇总/过滤/变换（同 key 的值集合 → 结果，多对一）。方向相反。

5.7 Typical Problem Solving by MapReduce

Read（读文档集）→ Map（每条记录产出 (term, docID)）→ Shuffle（按 term 分组）→ Reduce（收集同 term 的 docID 成列表）→ Write

5.7.1 MapReduce 单词计数过程

如何通过 MapReduce 计算三个文件中单词的频率。

- **流程图:**
  - **输入:** 来自三个文件的原始文本。
  - **Spliting 分割:** 文件被分成独立的块。
  - **Mapping 映射:** 每个块被拆分为键值对。
  - **Shulffing and Sorting 洗牌与排序:** 这些对被重组，使相同的键聚集在一起。
  - **Reducing 归约:** 对每个键的值进行求和（例如，“Bear, 2”）。
  - **最终结果:** 每个单词的汇总计数。

5.7.2 Map抽象

- **输入:** 接受一个键值对，其中键引用输入值，值是待操作的数据集。
- **评估:** 对每个输入值应用用户定义的函数，可能需要解析。
- **输出:** 生成一组新的键值对，其数据类型可以与输入不同。

5.7.3 Reduce抽象

- **输入:** 接受中间的键值对。
- **过程:** 系统在调用“reduce”函数之前执行“洗牌和排序”。所有共享相同键的对被分组在一起，迭代器将这些值提供给函数。
- **功能:** 其目的是聚合、汇总、过滤或转换数据。
- **输出:** 生成最终的键值对。输出通常比输入要小得多，因为它对数据进行了压缩（例如，对每个特定关键词的值求和）。

5.7.4 编程模型

- **Map():** 程序员指定一个函数，该函数接受输入数据并生成一组中间的键值对。
- **Reduce():** 程序员指定一个函数，该函数接受中间的键值对并将其合并，以生成一组合并的输出值。

part 6 Tutorial: Hadoop

6.1 Pseudo-Code

MapReduce 模型中用于“词频统计”（WordCount）的两个核心函数的伪代码：

1. **Map 函数**：处理输入数据，将文本行拆分为单个单词（Token），并为每个单词输出一个键值对，键是单词，值是1。
2. **Reduce 函数**：汇总处理结果，接收每个单词对应的所有计数值，进行累加求和，输出该单词及其总出现次数。

6.2 Challenge and Solution

管理多个工作单元（Workers）时面临的挑战及相应的解决方案：

面临的挑战：工作单元的执行顺序不可预测，工作单元之间可能会相互干扰，难以确定工作单元何时需要交换中间结果，对共享数据的访问顺序不确定。

解决方案：限制工作单元立即访问共享数据，防止工作单元相互干扰，将操作移至数据附近执行，在工作单元完成后进行批量更新。

6.3 MapReduce: Data Partition

问题：数据在什么时候、由谁切分？

答案：在加载时就分好，由 HDFS 负责

数据分区与存储 (Data Partitioning & HDFS)：

事实	含义	解决的问题
加载时即分布	数据在写入时就已经分布在集群各节点上，不是处理时才切分	解释了为什么Map能本地读取
切成chunks	文件被切成块，每块由不同节点管理	决定了Split 1…Split n的颗粒度，数据切分大小由chunks决定
每块多副本	Each chunk is replicated across several machines	Assume failures are common 副本是容错的物理基础

6.3 MapReduce Pipeline

1. **限制通信**：旨在减少进程间的通信量。
2. **独立处理**：每个记录由 Mapper 独立处理，互不干扰。
3. **映射阶段 (Mapping)**：各节点对本地预加载的数据进行映射处理。
4. **洗牌阶段 (Shuffle)**：节点间交换 Mapper 生成的中间数据。
5. **归约阶段 (Reducing)**：Reducer 将不同 Mapper 的结果合并，生成最终输出并存储在本地。

6.4 Classes of Problems

MapReduce 的适用场景（可并行化问题）：基准测试：数据密集型计算，如排序；Google 应用：用于词频统计、AdWords、PageRank 和数据索引；简单算法：如 grep（文本搜索）、文本索引和反向索引；其他行业应用：贝叶斯分类、Facebook 的人口统计分析、金融服务分析以及天文学中的高斯分析。

6.5 MapReduce steps

阶段	课件原文	要点
判断适用性	Determine if the problem is parallelizable and solvable using MapReduce	两个条件：可并行 + 可用 MapReduce 解
检查数据形态	Is the data Write Once Read Many (WORM)?	WORM 数据一次写入、多次读取，是并行的前提（无需锁）
设计与实现	Design and implement Mapper class and Reducer class	两个函数
配置、加载与运行	Configure the application (job) as to the number of mappers and reducers (tasks), input and output streams；Load the data；Launch the job and monitor	任务数可配置；数据加载；提交并监控

6.6 Limitation and Summary of MapReduce

- **局限性**：

    1. 难以处理共享全局状态，依赖共享状态的算法在 MapReduce 中难以实现。

    Online learning：在线学习每来一个样本就更新一次模型参数，下一个样本的处理依赖上一次更新后的参数。这意味着：样本之间不独立；参数是共享可写状态。

    model parameters as shared global state：共享全局状态，只要有全局可变的模型，就无法用纯 MapReduce 表达。

    2. 不适合小数据集，该框架主要为大规模数据的批量处理而优化。

要处理小数据集，框架必须改造。原因是MapReduce 的整套机制（切分、分发、Shuffle、容错）都有固定开销，数据量小的时候这些开销占比过高。

batch 是设计目标而非缺陷。specially optimized for batch operations，这是它的设计选择。batch的另一面就是不擅长低延迟。

Lecture 5 HDFS and Spark

1. HDFS 动机：分布式系统的存储挑战 → 故障概率算式 → HDFS 的八条需求 → 文件系统定义与四个动机问题 → 两页设计目标。这是问题定义。
2. HDFS 技术细节（一）：块与副本、抽象拓扑、NameNode / DataNode / 2NN 的架构、两层物理部署、元数据与 FsImage + EditLog、DataNode 职责。这是静态结构。
3. HDFS 技术细节（二）：副本放置策略、机架与带宽、心跳与故障检测、复制引擎、2NN 的合并机制与恢复上限。这是容错机制。
4. HDFS 技术细节（三）：读操作七步、写操作七步、四个带宽层级、网络拓扑与距离定义、命令行接口。这是动态行为。
5. Hadoop 作业调度：JobTracker / TaskTracker、Master-Slave 架构、七步调度流程、十步执行流程、Map/Reduce 数量、负载均衡、冗余执行、局部性优化。这是任务派发。
6. Spark 动机：MapReduce 的五条短板、Spark 的六项能力、内存 vs 磁盘共享、Daytona Gray 排序竞赛、Spark 组件栈。这是换引擎的理由。
7. Spark 技术细节：RDD 三个字母的含义、Transformation vs Action、窄变换与宽变换（shuffle）、DataFrame 与血统、DAG 与 stage 划分、Action 的语义。这是新引擎的原理。
8. Spark 应用与边界：八类用例、五个公司案例、三条不适用场景、全讲总结。这是落地与边界。

Part 1 HDFS

1.1 Motivation

1.1.1 Big data storage: Challenging

1. Data volumes are massive 数据量巨大
2. Reliability of storing PBs of data is challenging 规模化存储的可靠性
3. All kinds of failures: Disk / Hardware / Network Failures 故障种类多（磁盘故障、硬件故障、网络故障）

1.1.2 Failures are Common

两条设计的前提：

1. Single machine cannot serve all the data（单机服务不了全部数据）
2. Failures are common in commodity hardware, **Failure is expected, rather than exceptional**（商用硬件上故障很常见，事故不是例外）

在这里假设了独立同分布的算式隐含每块盘独立以 0.01 的概率故障。真实系统里同机架、同批次的盘会相关失效，那时的概率会更高。

1.2 Why HDFS?

HDFS: a distributed（分布式：文件跨机器存放） file system（文件系统）to store files with fault-tolerance（自带容错） 一个带容错能力的分布式文件系统

原文	解读
Need to process **huge datasets on large clusters of computers**	要在**大规模机器集群**上处理超大数据集
Very **expensive to build reliability into each application**	**把可靠性做进每个应用里太贵，**所以可靠性要下沉到基础设施层
Nodes fail every day **Failure is expected, rather than exceptional** The number of nodes in a cluster is **not constant**	节点**每天都在坏** 故障是**预期之内**，不是例外 集群节点数是**变化的，**扩容、下线、故障替换都要能在线完成
Need a **common infrastructure** **Efficient, reliable, easy to use** **Open Source, Apache Licence**	需要一个**公共基础设施**，而不是每个应用各做一套 三个品质要求：高效、可靠、易用 开源、Apache 许可

1.3 File System

系统文件：In computing, file system (often abbreviated to fs) is a method and data structure that the operating system uses to control how data is stored and retrieved.

两个组成：文件系统 = method（方法） + data structure（数据结构）。前者是组织规则，后者是磁盘上的实际布局。

职责范围：由操作系统用来控制数据存（stored）与取（retrieved）的方式。所以文件系统本来就是一个 OS 概念。

HDFS 的特殊之处：HDFS 是同一个抽象（方法 + 数据结构、存 + 取）在多机上的实现，所以它叫 distributed file system，而不是换了一个全新的东西。

1.5 Motivation Question

四个动机问题与 HDFS 的对策

问题	HDFS 的对策	对应章节
Problem 1: Data is too big to store on one machine.	Store the data on multiple machines 把数据存到多台机器上	块与副本
Problem 2: High-end machines are too expensive	Run on commodity hardware 跑在商用硬件上	物理部署
Problem 3: Many commodity hardware will fail occasionally	intelligent enough to handle hardware failure 足够聪明能处理硬件故障	心跳与故障检测
Problem 4: How to handle data on failed hardware	data replication 数据复制	副本放置

1.6 Goals of HDFS

原文	解读
Very Large Distributed File System (Scalability) 10K nodes, 100 million files, 10PB	非常大的分布式文件系统，括号里点明对应的是可扩展性 三个具体刻度：1 万节点、1 亿文件、10 PB 数据
Assumes Commodity Hardware Files are replicated to handle hardware failure Detect failures and recover from them (Failure tolerant)	假设跑在商用硬件上 用副本应对硬件故障 检测 + 恢复两件事都要做，合起来才叫容错
Optimized for Batch Processing (High Throughput) Data locations exposed so that computations can move to where data resides Provides very high aggregate bandwidth	为批处理优化，追求高吞吐，不是低延迟 暴露数据位置，好让计算搬到数据所在处（局部性优化） 提供非常高的聚合带宽，聚合指整个集群合起来的总带宽，是靠多机并行 + 就近读得到的

Tips:

1. High Throughput 吞吐量（throughput）与延迟（latency）是一对权衡。批处理场景要的是单位时间处理的数据量，允许单个任务慢。
2. It is not for low latency access. 凡是要求毫秒级随机读写的场景（在线事务、交互式查询）都不该用HDFS。
3. WORM: Write Once, Read Many. 一次写入、多次读取，无需互斥锁。本讲把它升格为HDFS 的语义前提。。

Part 2 HDFS Technical Details

2.1 Data Organization: Blocks and Replicas

文件不是整体存放，而是被切成固定大小的块，每块再存多份副本。这两件事解释了 HDFS 的容量、容错与负载均衡。

原文	解读
HDFS is a block-structured file system	与普通文件系统最大的结构性差别是HDFS 是块结构的文件系统
Individual files are broken into blocks (chunks) of a fixed block size (64MB by default)	文件被切成固定大小的块，默认 64MB。固定大小让寻址变成算术，若块长可变，就必须维护一张索引表才能定位第 k 块。
Blocks are stored across a cluster of machines	块散在集群上 ≠ 块被切开：一个块整体存在一台机器上（不做条带化）。文件级才是分布式的，所以一台机器坏了丢的是若干整块，而不是每个文件的一小片。
Each copy of the block is called replica	每一份块的拷贝叫 replica（副本）

Block size and replicas are configurable.

Block size 与 Replicas 都可配，所以 64MB 与默认 3 副本都只是默认值，不是固定约束。

2.1.1 Why Block?

问题：既然已经有文件了，为什么还要多一层块？（为什么需要块抽象？）

1. 超出单盘容量：File can be larger than a single disk 文件可以比任何一块磁盘都大。把文件切成块后，各块落在不同机器上，文件总大小就不再受单盘限制。

2. 固定大小便于管理：Block is of fixed size, easy to manage and manipulate 块大小固定，于是管理与操作都变成等长单元的算术，无需处理变长边界。
3. 易复制且细粒度均衡：Easy to replicate and do more fine-grained load balancing 块是复制的基本单位，也是负载均衡的基本单位：某台机器忙，就把它的块搬到别处。

2.2 HDFS's Abstract Topology

抽象拓扑（只有角色与连线）

角色	职责
HDFS Client	发起操作的客户端；与 NameNode 之间标注 FS / namespace / meta ops
NameNode	接收客户端的文件系统 / 命名空间 / 元数据操作；向下辐射到所有 DataNode
Secondary NameNode	与 NameNode 之间是灰色双向箭头，标注 Namespace backup（命名空间备份）
DataNode	每个 DataNode 内部画了 3 个彩色小方块（表示持有的块）；向下连到磁盘图标

② 图上的两条横向说明带 ·

中间黄色横带：Heartbeats, balancing, replication etc.. 底部黄色横带：Nodes write to local disk

Heartbeats（心跳，存活检测）、balancing（均衡，搬块）、replication（复制，补副本）这三件事都在 NameNode 与 DataNode 之间发生。

Nodes write to local disk 数据写在本地磁盘上。这是Move computation rather than data能成立的前提：数据既然在本地盘，把计算送过来比把数据送出去便宜。

Data serving（DataNode → 磁盘，表示实际的数据服务）与 Namespace backup（NameNode → 2NN）。

考点：NameNode 只有一个（中心）、DataNode 有很多个（5 个是示意）、数据落在本地磁盘。

2.3 HDFS Architecture

组件：命名空间、NameNode（NN）、DataNode（DN）、Secondary NameNode（2NN），以及各自的语言与版本归属。

三处名词的层级：三者不构成主从复制关系。

NameNode（1 个） 管理命名空间；DataNode（N 个，通常一机一个） 存实际数据；Secondary NameNode（1 个） 帮 NameNode 合并元数据。

2.3.1 命名空间与 NameNode

HDFS exposes a file system namespace and allows user data to be stored in files.

命名空间：HDFS 对外暴露一个文件系统命名空间，用户数据以文件形式存放。接口仍然是我们熟悉的目录/文件。

The HDFS cluster consists of a NameNode (NN), a master node that manages the file system namespace and regulates access to files.

NameNode：NN 是主节点（master node），负责管理命名空间并管制对文件的访问（manage是有哪些文件，regulate是谁能动）。

2.3.2 DataNode

There are a number of DataNodes (DN), usually one per machine in a cluster. The DataNodes manage storage attached to the machines that they run on.

DataNode 数量、位置、管理范围：通常一台机器一个。每个 DN 只管自己这台机器上的存储(data locality)。

2.3.3 Secondary NameNode

原文	说明
Another node runs Secondary NameNode (2NN) to serve as a helper of NN	2NN 是 NameNode 的助手（helper）
Proposed in Hadoop 1.x	Hadoop 1.x 引入（Hadoop 2.x 换成 CheckpointNode）
NOT a complete backup of NN	不是 NameNode 的完整备份
Written in Java, it also supports Python and Ruby	主体 Java，另外支持 Python 与 Ruby

2NN 不是 NameNode 的完整备份（NOT a complete backup of NN），2NN 在合并 FsImage 与 EditLog，加速重启。

2.4 HDFS Physical Deployment

把架构图落到机房布局：两层交换机、机架、每架节点数，并给出 NN/DN/JT/TT 在机架上的实际分布。物理部署（机房、机架、交换机）：

两层架构

Aggregation switch（汇聚交换机）连到 Rack switch（机架交换机），每个机架交换机再连若干 Node（节点，每个带 Disk 图标）。

Ps. Nodes are commodity PCs（节点是商用 PC）；30-40 nodes/rack（每机架 30 - 40 个节点）。

机架内外的带宽差

同机架便宜、跨机架贵：Rack switch 内部（同机架）带宽高；Rack switch ↔ Aggregation switch（跨机架）带宽低。

机架	特殊角色	普通节点	说明
Rack 1	Name Node	DN + TT × 4	NameNode 与第一批数据节点同架
Rack 2	Job Tracker	DN + TT × 4	JobTracker（作业调度）在第 2 架
Rack 3	Secondary NN	DN + TT × 4	2NN 在第 3 架，刻意与 NameNode 分开 因为 NameNode 是单点故障，如果 2NN 与 NN 同架，一次机架级断电就同时失去两者。物理隔离是冗余的一部分。
Rack 4	Client	DN + TT × 4	客户端所在机架
Rack N	—	DN + TT × 5	省略号表示中间还有若干个同构机架

HDFS: NameNode, DataNode MapReduce: JobTracker, TaskTracker

World：向下连到两个顶层 switch，代表集群之外的外部网络。

2.4 NameNode

2.4.1 Functions of NameNode

1. Manages File System Namespace (maintains and manages the file system metadata): Maps a file name to a set of blocks; Maps a block to the DataNodes where it resides.

管理文件系统命名空间，即维护与管理元数据，两条映射，文件名 → 块集合的映射（一个文件由哪些块组成）；块 → 所在 DataNode 集合的映射（每块在哪几台机器上）

2. Cluster Configuration Management 集群配置管理（有哪些节点、机架归属等）
3. Replication Engine for Blocks 块的复制引擎（检测失败并补副本）

Tips：两条映射合起来才构成读一个文件的完整寻址链：文件名 → 块列表 → 每块的位置。

NameNode 不存数据：五项功能里没有一项是存文件内容。

为什么叫中心而不是主节点复制：2.3.1 把NN 叫 master node，但 HDFS 的 master 与一般主从复制的 master 不同，它管的是元数据，不是数据本身。

2.4.2 Types of metadata

1. List of files 文件清单（有哪些、在哪里）
2. List of Blocks for each file 每个文件的块清单
3. List of DataNodes for each block 每个块的 DataNode 清单
4. File attributes, e.g., creation time, replication factor 文件属性，如创建时间、副本因子等属性。

2.4.3 Metadata in Memory 元数据放内存

The entire metadata is in main memory. Memory read/write is faster than disk read/write

为什么放内存：内存读写比磁盘快。元数据每次操作都要查，放磁盘会成为瓶颈。

代价：Data in memory will be lost if the machine shuts down 机器一关，内存里的元数据全丢。

于是需要持久化：In addition to metadata in memory, we need a persistent copy of metadata 所以除了内存中的元数据，还需要一份持久化副本。所以 引入FsImage 与 EditLog。

2.4.3.1 FsImage and EditLog

内存元数据怎么落盘。答案是一对文件——只读的快照 FsImage + 记录增量的 EditLog，以及「重启时合并」这一套固定流程。

文件	功能	性质	大小
FsImage	The persistent record of the metadata stored on the local disk is called FsImage	只读（Read-only）的元数据持久化记录	随集群增长，大
EditLog	Records modifications to FsImage — file creations, file deletions, etc.	记录对 FsImage 的修改（建文件、删文件等）本质是所有会改变元数据的操作。	Stored in disk. (smaller)，较小

分工：FsImage 本身不是备份，它是一份截至某时刻的元数据快照；EditLog 也不是备份，它是快照之后的增量。只有两者合起来（FsImage + EditLog）才等于当前元数据状态。

Metadata in memory will be modified as well 内存元数据也会同步改：内存里的那份会同步更新，所以服务期间的查询走内存，不走磁盘。

2.4.3.2 Current status and Restart

current status: FsImage + EditLog Restart 1. NameNode loads FsImage in memory, applies modifications logged into EditLog, and generates a new FsImage and empty EditLog to replace the old ones. 2. Start service.

1. 回放合并：把 FsImage 载入内存 → 逐条应用 EditLog 的修改 → 生成新的 FsImage 与一个空的 EditLog，替换掉旧的。合并后 EditLog 归零，重新开始记录。

2. 启动服务Start service：元数据就绪后才对外提供服务。

3. 为什么这样设计：若不合并，EditLog 会无限增长，每次重启都要回放全部历史修改，重启时间越来越长。合并把它压成一份快照，重启时间与上次合并点而非集群总历史相关。

4. 但这个合并谁来做：第 23 页只讲了 NameNode 自己重启时的合并。第二十节会讲 2NN 如何把这个合并从 NameNode 的重启路径里挪出去——这就是 2NN 存在的全部理由。

2.5 DataNode

讲完名称（中心）节点，转向干活的一方。DataNode 有五件事：存块、存块的元数据、对外服务、定期汇报、参与流水线转发。

2.5.1 Functions of DataNode

原文	解读
Where HDFS stores the actual data；A Machine	实际数据存放处；物理上就是一台机器
Stores data blocks in the local file system	DataNode 不直接操作裸磁盘，而是把块写成本地文件
Stores metadata of a block	同时存块自身的元数据（注意：是块的元数据，不是文件系统元数据）
Serves data and metadata to Clients	向客户端提供数据与元数据，读操作的数据流直接从 DataNode 走
定期 BlockReport + 心跳 + 流水线转发	

Tips: NameNode 的元数据 = 文件名/块/位置的全局映射；DataNode 的块元数据 = 这一块自己的信息（校验和、长度等）。两者的范围完全不同。

2.5.2 Block Report 心跳

Block Report：Periodically（3s/time） sends a report of all existing blocks（本机**现有全部块**的清单）to the NameNode (heartbeat)

汇报：因为 NameNode 才持有块，即DataNode 的映射。这份映射必须由 DataNode 主动上报才能维持准确，NameNode 不主动扫描。

2.5.3 Facilitates Pipelining of Data

Forwards data to other specified DataNodes

转发：链式传递，写一个块时，客户端只把数据发给第一台 DataNode，由它转发给下一台。

流水线：若客户端给三台 DN 各发一份，客户端的出口带宽就是瓶颈。流水线让每台 DN 收一份、转一份，客户端只需一份出口带宽。

Part 3 DataNodes and NameNode Illustration

如果坏了副本放哪、怎么发现坏了、怎么补，以及中心节点自己坏了的上限在哪里。

3.1 DataNodes and NameNode Illustration

NameNode 只存元数据，DataNode 存块，文件到块的映射关系就写在元数据里。

1. NameNode：Stores metadata only 只存元数据（文件名 → 块号列表）

foo → 1, 2, 4：文件 /user/aaron/foo 由块 1、块 2、块 4 三块组成

2. DataNodes: Store blocks from files 存块

图 1 持 {1, 2, 4, 5} 共 4 块、图 2 持 {2, 3, 5} 共 3 块、图 3 持 {1, 3, 4} 共 3 块，本例中 10 个块位置对应 5 个块，所以每块恰好 2 份副本。

Tips：foo → 1,2,4 与副本的关系：元数据里只记录哪些块，不记录每块有几份副本（副本数由 replication factor 属性控制）。所以 foo 的块 1 与块 4 在图1和3 各一份、块 2 在图 1和2 各一份，元数据侧不体现。

3.2 Racks and Network Bandwidth

把同机架便宜、跨机架贵写成前提，并指出机架这个物理概念在 HDFS 里是被显式建模的。五条关于机架的事实：

	原文	解读
1	Many racks in a cluster	一个集群有很多机架
2	Communication between racks is through switches	跨机架通信要经过交换机，交换机就是带宽瓶颈的位置
3	Network bandwidth between machines on the same rack is greater than those on different racks	同机架带宽 > 跨机架带宽（成本模型的基石）
4	NameNode determines the rack ID for each DataNode	NameNode 为每个 DataNode 判定机架 ID，机架归属是元数据的一部分
5	Racks play an important role in block replication	机架在块复制中扮演重要角色（副本放置的逻辑：既然跨机架贵，副本就不能全放一个机架。）

Tips：

4 中的 机架 不是网络层自动知道的，而是 HDFS 主动建模的：管理员设计机架映射/感知（rack awareness），NameNode 据此计算距离、决定副本落点、决定该读哪份副本。

3.3 HDFS Data Replication: Illustration

块 2 在 Node A、B、D 三处被标成红色，即同一个块的三份副本分别落在哪。所以红色 = 同一块的多份副本。

总数守恒：5 个块 × 3 份 = 15 个块位置；5 个节点 × 3 个格位 = 15 个格位。两边相等，说明示意图是恰好铺满的设计。

副本不在同一节点：任何一块的 3 份副本分布在不同节点上。这是最直观的容错要求：一台机器坏了，该块仍有两份可用。

3.4 Replication：Block Placement

HDFS Rack-aware replica placement

机架策略（本地/同机架 → 不同机架 → 同远程机架的另一节点）

1. To put one replica on the local machine if the writer is on a DataNode, otherwise on a random DataNode in the same rack as that of the writer.

第一份副本的两种情形（在 DN 上 / 不在 DN 上）：若写入方本身在某个 DataNode 上，第一份就放在本地机器上，本地优先让写操作变成本地磁盘写，省掉一次跨网络传输。否则，放于与写入方同一机架的某个随机 DataNode。

2. Another on a node in a different (remote) rack

第二份放另一个（远程）机架上的节点。

3. The last one on a different node in the same remote rack

第三份放在相同的远程机架里的另一台不同节点的机器上。

Ps. If the replication factor is greater than 3, the placement of the 4th and following replicas is determined randomly.

超过 3 份怎么办，第 4 份起随机放置。默认副本因子 3，前 3 份有策略，后面的不管。

To reduce communication costs, clients read from nearest replicas

降低通信开销，读最近（按网络拓扑算距离）的那份副本。

策略意图：

1. 至少跨 2 个机架，机架断电不会丢块；
2. 最多跨 2 个机架，避免跨机架写入的带宽开销。

3.5 Necessity for Replication

三个故障场景（节点级 → 数据级 → 设备级）

1. A Datanode may become unavailable

节点整机不可用：一台 DataNode 可能整机不可用（宕机、断电、网络隔离）。

2. A replica may become corrupted

副本被损坏：副本内容可能损坏（静默数据损坏 bit rot）。注意这里说的是 replica 而非 node。若只有节点故障，多副本就够了；但副本损坏意味着副本本身不可信，所以必须有校验机制来判断哪份副本是好的，所以需要后续的BlockReport 与校验和设计。

3. A hard disk on a Datanode may fail

磁盘故障：DataNode 上的某块硬盘可能坏掉（一机多盘时，坏盘不等于整机不可用）。

3.6 心跳机制Heartbeats

复制解决坏了还能读，本节解决怎么知道坏了。

机制：DataNode 定期报平安，NameNode 超时就判死。

课件原文	解读
DataNodes send heartbeat to the NameNode	DataNode → NameNode 发送心跳（DN 主动上报，不是 NN 去探）
Once every 3 seconds	heartbeat 周期 = 3 秒，唯一的具体时间数字
The Namenode receives a Heartbeat and a BlockReport from each DataNode in the cluster	NN 从每个 DataNode 收到心跳 + BlockReport两样东西；Heartbeat = 我还活着（存活信号）；BlockReport = 我有哪些块（内容清单）
BlockReport contains a list of all the blocks on a Datanode	BlockReport 的内容 = 该 DataNode 上全部块的清单
NameNode uses heartbeats to detect DataNode failure	NN 用心跳来判断 DataNode 故障

3.7 复制引擎 Replication Engine

1. NameNode detects DataNode failures. Data blocks on the failed Datanode are replicated on other Datanodes

NameNode 检测到 DataNode 故障。补副本：失效节点上的块在其他 DataNode 上重新复制。

2. Choose new DataNodes for new replicas

选新落点：为新副本选新的 DataNode。选新落点时的约束：Balance disk usage（均衡磁盘使用率）与 Balance communication traffic to DataNodes（均衡到各 DataNode 的通信流量）。

两个均衡目标的区别：磁盘均衡是容量维度（别让某台机器塞满）；通信均衡是带宽维度（别让某台机器成为热点）。

补副本 vs lecture4 重跑任务：第 4 讲的容错是重算（任务重跑）；本讲的容错是补数据（复制块）。同一思想在存储层与计算层各有一个实现。

3.8 NameNode 故障

代次	方案	说明
共性	唯一的单点故障 A single point of failure	NameNode 只有一个，它挂了整个 HDFS 不可用
Hadoop 1.x	Secondary NameNode (2NN): Hadoop 1.x	1.x 的缓解方案（第 19 - 20 节展开）
Hadoop 2.x	CheckpointNode (CN): Hadoop 2.x	2.x 改名为 CheckpointNode

两代方案名字不同、性质相似：2NN 与 CN 都不是 NameNode 的活备份（NOT a complete backup），它们做的是元数据检查点的合并。

3.8 Accelerate NameNode Restart

澄清一个最常见的误解：2NN 的第一职责不是备份，而是加速重启。它会合并 FsImage 与 EditLog，并周期性把结果回传。

3.8.1 五步工作流程

copy → merge → upload → purge → 目的是加速重启

没有 2NN（Original 原始版本）

NN 重启时，必须：加载旧 FsImage → 重放整条完整 EditLog，日志越长，重启越慢。

搭配 2NN 的版本

2NN 已经提前做完 Checkpoint，生成了合并后的新 FsImage。 NN 重启时，只需要加载 2NN 生成的最新 FsImage，只重放 Checkpoint 之后新增的一小段 EditLog（绿色增量），大幅缩短启动时间。2NN 本地保存一份最新 FsImage 副本。

1. 2NN copies FsImage and EditLog from NameNode

取文件：2NN 从 NameNode 拷贝 FsImage 与 EditLog。2NN 是读取方，不是被写入方，它不可能实时同步。

2. 2NN merges FsImage and EditLogs periodically

合并：2NN 周期性合并两者（2NN 的核心动作）。

3. 2NN uploads new FsImage to the NameNode

回传：把新的 FsImage 上传回 NameNode。

4. EditLog on NameNode is purged

清理：NameNode 上的 EditLog 被清空（因为它已经合并进新 FsImage 了）。

5. It is designed to accelerate the speed of NameNode's restarting

目的：设计目的是加速 NameNode 重启。如果 EditLog 很大，重启会耗很多时间。2NN 把回放长日志这件事提前、离线做掉了，NameNode 重启时只需载入一份新鲜的快照。

检查点（checkpoint）：2NN 做的事情本质是检查点合并器，把快照 + 增量压成新快照 + 空增量，NameNode 之外执行，于是 NameNode 重启变快。

3.8.2 备份能力与限制

Additional features: It can provide a copy if NameNode fails. Not the latest one.

NN 挂了可以用 2NN 上的 FsImage 临时顶替，所以 2NN 常被误认为备份。2NN 只在每次合并的时点持有快照，两次合并之间的修改（新产生的 EditLog）它没有。所以丢掉的是上次合并到故障时刻这段更新。所以，这不是完整备份（NOT a complete backup of NN），数据是滞后的，两页合起来才是完整答案。

易错点：2NN 的 Secondary 不是第二主节点，而是次要的、辅助的。它不接管 NameNode 的在线服务，也不参与读写路径。

3.9 2NN Works

F = **FsImage**（文件系统镜像，NameNode 的持久化元数据快照）

E = **EditLog**（编辑日志，记录后续所有文件系统操作的增量日志）

1. NameNode(NN)

   NN 初始：FsImage (F) + 空 EditLog (E)

   客户端操作不断写入 EditLog(E)（红色代表新增日志），FsImage 保持不变

   此时：FsImage 是旧快照，所有新改动存在 EditLog 里

2. SecondaryNameNode(2NN) 周期性触发Checkpoint检查点操作：

   拉取（fetch）NN 当前的 FsImage + EditLog

合并（merge）：把 EditLog 的所有操作应用到 FsImage，生成新的 FsImage

把合并完成的最新 FsImage 保存在 2NN 本地

再将新 FsImage 传回 NameNode，NN 替换旧 FsImage，并清空自己的 EditLog

2NN 不是 NN 热备，只定期帮 NN 做 FsImage+EditLog 合并，生成最新快照，减轻 NN 重启加载压力

3.10 NameNode Recovery

场景：NameNode 在两次 2NN Checkpoint 之间崩溃

1. NN 宕机时，最新的增量修改（绿色部分）只存在 NN 本机 EditLog，还没同步合并到 2NN
2. 2NN 手里，只有上一次 Checkpoint 完成的 FsImage 快照（缺少崩溃前这一段新操作）
3. 故障恢复：

   把 2NN 上保存的 FsImage 副本拿出来，作为恢复的基础镜像

   缺点：Checkpoint 之后、宕机之前的新增元数据（绿色更新）会丢失

重点考点：

1. 2NN ≠ NameNode 备用节点，**不**能在 NN 宕机后自动切换接管，只是定期做 Checkpoint、生成 FsImage 备份。
2. Checkpoint：2NN 拉取 FsImage+EditLog，回放日志合并生成新镜像。
3. NN 正常重启：有 2NN 可以减少需要回放的 EditLog，加快启动。
4. NN 崩溃恢复：2NN 不能做到实时热备份，只能恢复到最近一次 2NN checkpoint 时刻，两次 checkpoint 之间的元数据更新丢失。

Part 4 HDFS 读和写操作

4.1 HDFS Architecture (Read and Write Path)

Read 路径：读走同机架的 DataNode；

Replication 路径：跨机架复制

1. NameNode（NN）

- 管理**所有元数据**：文件名、目录、权限、Block编号、Block所在DN位置
- **不存储业务数据**，只存索引信息
- 接收客户端RPC请求，调度读写，**不参与数据传输**

2. DataNode（DN）

- 真正**存储文件Block真实数据**
- 定时向NN心跳、汇报块状态
- 负责读写数据、副本存储、Pipeline传输

3. Client 客户端

- 负责文件拆分Block、组装数据、容错重连、流水线控制
- Client 到 NameNode，标注 Metadata ops，只读写不传数据
- Client 到 DataNode，标注 Write，真正的直接流，读时 Client 直接向 DataNode 取，写时 Client 发起流水线；

4.3 HDFS读操作

步骤1：Client connects to NN to request data read.

Client发起 open() 请求。客户端想要读取 HDFS 文件，首先通过 RPC 向 NameNode 发送 open 请求，询问该文件的元数据信息。

步骤2：NN tells the client where (i.e., DN) to find the data blocks.

NameNode 返回 Block 位置列表。NN 查询自己内存中的元数据，返回：

- 该文件包含哪些 Block（Block1、Block2、Block3…）
- 每个 Block 的所有副本分别位于哪些 DataNode 节点
- 返回的列表已经经过机架感知排序（就近优先）

重点：NN 只返回位置，不传输任何数据！

步骤3：Client reads blocks directly from data nodes (without going through NN)

Client 主动连接最优的 DataNode。客户端根据 NN 返回的有序 DN 列表，优先连接距离最近、负载最低的 DN，建立 Socket 连接。

步骤4：DN 以 Packet 数据包形式返回数据

DataNode 从本地磁盘读取 Block 数据，拆分为 Packet（默认64KB），逐包发送给 Client。

Client 持续接收、缓存、拼接数据。

步骤5：n case of node failures, client connects to another node that serves the missing block

读取失败自动容错（核心考点）如果当前 DN 掉线、卡顿、读取出错：

- 客户端立刻断开当前连接
- 从同 Block 的其他副本节点中选择下一个最优 DN
- 从失败的 Packet 位置继续读取，不重头读整个块

步骤6：一个 Block 读完，读取下一个 Block

当前 Block 的全部 Packet 接收完毕，客户端继续读取文件的下一个 Block，重复上述流程。

步骤7：全部读取完成，执行 close()

文件所有 Block 读取完毕，客户端关闭数据流、释放连接，读操作结束。

1. 客户端连接 NameNode，请求读数据。

2. NN 告诉客户端到哪个 DataNode 去找这些块。

3. 客户端直接从 DataNode 读块，不经过 NN。

4. I节点故障时，客户端改连另一台持有该块的节点。

4.2.1 两个问题

Why does HDFS choose such a design for reading?

Why not ask clients to read blocks through NN?

原因	解读
Prevent NN from being the bottleneck of the cluster	防止 NameNode 成为集群瓶颈，所有数据若都过它，带宽立刻打满
Allow HDFS to scale to large number of concurrent clients	让 HDFS 能扩展到大量并发客户端，瓶颈一旦在中心，并发度就被锁死
Spread the data traffic across the cluster	让数据流量分散到整个集群，而不是集中到一个节点出口

4.2.2 机架感知机制

HDFS 将集群网络抽象为三层树状拓扑：数据中心 → 机架 → 节点，读取副本优先级（速度从快到慢）：

1. 本节点读取（最快，无网络开销）
2. 同机架不同节点（同交换机，带宽高、延迟低）
3. 同机房不同机架（跨交换机，速度下降）
4. 跨机房（最慢、延迟最高）

作用：减少跨机架流量、提升读取速度、降低集群网络压力

4.2.3 读操作核心特点

数据不经过 NameNode，避免 NN 瓶颈

控制流走 NN，数据流直连 DN

天然支持容错，副本自动切换

支持顺序读、大文件吞吐极强，不适合随机读

4.3 HDFS 写操作

步骤1：Client 向 NN 发送 create() 创建文件请求。客户端想要上传文件，先 RPC 请求 NameNode 创建空文件。

步骤2：NN 执行权限、路径、状态校验，NN 检查：用户权限是否合法；文件路径是否已存在；父目录是否存在；集群是否可写。校验通过后，NN 在元数据中生成一个空文件（此时文件对用户可见，但未完成）。

步骤3：NN 分配 Pipeline 节点列表（副本节点策略），NN 根据副本机制（默认3副本）智能选择一组 DN，形成流水线顺序：放置策略（必考）：

1. 副本1：写入客户端所在节点（就近写入，速度最快）
2. 副本2：写入同机架不同节点（防单节点故障）
3. 副本3：写入不同机架节点（防机架整体断电/故障）

NN 将有序的 DN 列表返回给 Client。

步骤4：Client 构建 Pipeline 流水线。客户端根据返回的 DN 顺序，依次和 DN1、DN2、DN3 建立链式 Socket 通道，形成流水线。

步骤5：客户端切分 Packet，链式传输写入。客户端将文件数据切分为 Packet（默认64KB），先发给第一个 DN：

- DN1 收到 Packet → 落盘保存 → 立刻转发给 DN2
- DN2 收到 Packet → 落盘保存 → 立刻转发给 DN3
- DN3 收到 Packet → 落盘保存（最后节点，不转发）

整条流水线并行写多副本，极大提升写入效率

步骤6：反向 ACK 确认机制（核心重难点）。每一个 Packet 写完后，ACK 从后往前逐级返回：

- DN3 写完 → 发 ACK 给 DN2
- DN2 收到后 → 发 ACK 给 DN1
- DN1 收到后 → 最终 ACK 传回 Client

规则：必须收到整条流水线的 ACK，才算当前 Packet 写入成功，才会继续发送下一个 Packet。

步骤7：Block 写满，继续写下一个 Block。当一个 Block 写满（默认128M）：

- Client 关闭当前 Block
- 重新向 NN 请求新的 DN 节点列表
- 开启新 Pipeline，写下一个 Block

步骤8：文件全部写完，Client 发送 complete() 完成写入。文件所有 Block 写完后：

- Client 调用 close()
- 向 NN 发送 complete 信号
- NN 将文件状态从“正在写入”改为“已完成”

此时文件正式对外可读。

4.3.1 写操作容错机制

流水线中任意一个 DN 故障（掉线、卡死、磁盘坏）：

1. Client 立刻检测到 ACK 超时/失败
2. 自动剔除故障 DN
3. 用剩余健康 DN 重建新 Pipeline
4. 从上一个成功 ACK 的 Packet 位置继续写入，不重头写整个文件
5. 后续 NN 会自动补充副本，保证副本数量达标

考点：HDFS 写过程支持断点续写、动态重建流水线

4.3.2 写操作核心特性

- 控制流走 NN，数据流走 Pipeline
- 采用链式流水线批量写副本，效率高
- 严格 ACK 校验，保证数据强一致性

- 写入过程文件处于“未完成状态”，不可读
- 支持动态容错、节点剔除、续写

4.4 HDFS 读写操作

对比维度	读操作 Read	写操作 Write
NN 作用	只返回 Block 位置列表，不参与数据传输	创建文件、分配DN、最后标记文件完成
数据流向	DN → Client 单点直连	Client→DN1→DN2→DN3 流水线链式传输
副本策略	就近读（机架感知择优）	跨机架三副本策略，保证容灾
容错方式	节点失败，切换同块其他副本继续读	节点失败，重建Pipeline，剔除坏节点续写
数据包单位	Packet	Packet，带反向ACK确认
文件状态	文件必须已完成	写入中文件临时不可读，complete后才可见

1. 为什么 HDFS 读写数据不经过 NameNode?

为了避免 NameNode 成为集群性能瓶颈，分离控制流和数据流，让大量读写流量分散到各个 DataNode，支持高并发、高吞吐，提升集群扩展性。

2. HDFS 三副本放置策略是什么?

第一副本存客户端当前节点，第二副本存同机架不同节点，第三副本存不同机架节点。兼顾写入速度和容灾能力，防止单节点、单机架故障导致数据丢失。

3. 什么是 Pipeline 流水线写入?

客户端将数据 Packet 依次传递给多个 DataNode，链式完成多副本写入，并逐级返回ACK确认。实现多副本并行写入，大幅提升写入效率，同时保证数据一致性。

4. 读操作机架感知的作用?

根据网络拓扑选择距离最近的副本节点读取数据，减少跨机架网络传输，降低延迟、节省带宽、提升读取性能。

Part 5 Hadoop’s Job Scheduling 作业调度

本文讲解的是 MapReduce v1（MRv1），即 Hadoop 1.x 时代的经典版本。采用Master/Slave 主从架构，核心组件为 JobTracker + TaskTracker。

该版本存在致命架构缺陷，在 Hadoop 2.x 后被 YARN 完全替代，是大数据面试、考试、入门的核心基础知识点。

核心定位：MapReduce 是 Hadoop 生态圈中的分布式离线计算框架，负责对海量HDFS存储数据进行分片、并行计算、汇总输出，主打高容错、高可靠、可横向扩展。

5.1 JobTracker & TaskTracker 作业调度

HDFS 解决了数据放哪，这一节转向任务派给谁。

5.1 MRv1 整体集群架构

MRv1 架构严格分为Master 主节点 和 Slave 从节点，同时依赖 HDFS 的 NameNode、DataNode 实现数据存储，全程是计算与存储绑定的设计思路。

与 HDFS 的镜像关系：NameNode : DataNode ≈ JobTracker : TaskTracker。前者管元数据与任务，后者管块与执行。

5.1.1 Master 主节点核心进程

Master 节点仅部署 JobTracker（JT） 一个计算调度进程，同时集群的 NameNode 通常也部署在主节点，负责文件系统管理。

JobTracker = central control node，负责 split 与 send；

TaskTracker = worker node 上接受 Map / Reduce / Shuffle 任务的进程。

YARN 改进此结构（out of scope）。

JobTracker 核心职责（全局调度中心）

A central control node runs JobTracker (JT). JobTracker splits up data into smaller tasks("Map") and sends it to the TaskTracker process in each node

把数据切成更小的任务 Map，再派发给各节点的 TaskTracker 进程。

- 作业接收与注册：专门接收客户端提交的完整 MapReduce Job 作业，分配全局唯一的 JobID，初始化作业运行环境。
- 作业拆分与分片规划：解析用户提交的作业配置、输入路径、计算逻辑，自动将一个大 Job 拆分为若干个 MapTask 和若干个 ReduceTask，同时根据文件大小生成输入分片（InputSplit）。
- 任务调度与分配：集群唯一的任务决策者，实时感知所有 Slave 节点的资源空闲状态、数据存储位置，精准分配 Map/Reduce 任务，实现负载均衡和数据本地性优化。
- 集群状态监控：维持与所有 TaskTracker 的心跳通信，实时监控节点存活状态、资源占用、任务运行进度。
- 故障判定与重试：检测节点故障、任务运行失败、任务卡顿等异常，自动触发任务重试、节点拉黑、作业容错机制。
- 作业进度管理与结果汇总：全局统计作业运行进度，记录任务运行日志，作业完成后更新最终状态，通知客户端获取结果。

5.1.2 Slave 从节点

所有从节点同时部署两个进程，分别负责计算执行和数据存储，是真正干活的节点：

1. TaskTracker（TT）计算执行代理

Worker Nodes also run TaskTracker (TT). A TaskTracker is a node in the cluster that accepts tasks — Map, Reduce, and Shuffle operations — from a JobTracker

在工作节点上运行 TaskTracker。集群里接受 JobTracker 派发任务的节点。

每个 Slave 节点有一个 TaskTracker，是 JobTracker 的从属工作进程，无自主调度权限，只服从主节点指令。

- 心跳上报：默认每3秒向 JobTracker 发送一次心跳，上报节点CPU、内存、空闲槽位、正在运行的任务状态等信息。
- 任务接收与执行：接收 JobTracker 下发的 MapTask、ReduceTask，独立启动子JVM运行任务，避免任务之间相互干扰。
- 资源管控：管理当前节点的任务运行槽位（Map槽位、Reduce槽位），限制单节点最大并发任务数，防止资源耗尽。

- 状态反馈：实时向 JobTracker 汇报任务启动、运行、完成、失败、卡顿等状态。

2. DataNode（DN）数据存储节点

负责存储 HDFS 数据块，保存所有待计算的原始数据、任务中间临时数据，为本地性调度提供数据支撑。计算任务优先读取本节点 DataNode 的数据，减少网络传输。

5.1.3 Client 客户端

用户程序入口，无需常驻集群，仅负责作业提交、资源上传、进度查询、结果获取，核心操作：

- 封装 Job 配置信息（输入输出路径、Map/Reduce逻辑、任务参数）；
- 向 JobTracker 申请 JobID；
- 将作业Jar包、配置文件、依赖资源上传至HDFS临时目录；
- 正式提交作业，轮询查询作业运行进度，作业结束后输出结果。

5.2 MRv1 完整作业执行流程

整个流程分为 作业准备阶段、任务调度阶段、任务运行阶段、作业收尾阶段 四大阶段，全程自动化执行。

阶段1：客户端作业准备

1. 用户编写 MapReduce 程序，设置作业参数，打包为Jar文件；
2. Client 连接 JobTracker，请求分配唯一 JobID；
3. Client 在 HDFS 中创建该作业的专属临时目录，上传Jar包、配置文件、切片信息等资源；
4. Client 正式向 JobTracker 提交完整作业。

阶段2：JobTracker 作业初始化

1. JobTracker 接收作业，校验配置合法性，初始化作业运行上下文；
2. JobTracker 主动请求 NameNode，查询本次作业所有输入文件的 数据块存储位置、副本分布节点；
3. 根据文件大小、Block大小，自动切分 InputSplit 输入分片，一个分片对应一个MapTask，确定Map任务总数；
4. 初始化 ReduceTask 数量，创建任务调度队列，等待节点资源空闲。

阶段3：任务调度与下发

1. 所有 TaskTracker 持续向 JobTracker 发送心跳，上报空闲资源；
2. JobTracker 基于数据本地性优先、节点负载均衡原则，筛选合适的TaskTracker；
3. JobTracker 通过心跳响应，将 MapTask/ReduceTask 分配给目标节点；
4. TaskTracker 接收任务指令，从HDFS下载作业Jar包和配置资源。

阶段4：任务运行执行

1. TaskTracker 为每个任务启动独立子JVM进程（核心隔离机制，单个任务崩溃不影响节点和其他任务）；
2. 子JVM 运行 MapTask：读取本地HDFS数据块，执行Map逻辑，输出局部中间数据；
3. 所有MapTask执行完成后，启动Shuffle洗牌阶段，对数据分区、排序、合并；

4. 运行ReduceTask：拉取对应分区的Map中间数据，聚合汇总，执行Reduce业务逻辑；
5. 任务运行中持续向TaskTracker反馈状态，TT同步上报JobTracker。

阶段5：作业收尾

1. 当所有MapTask、ReduceTask全部执行成功，JobTracker 标记作业为完成状态；
2. 清理HDFS中的作业临时资源、日志缓存；
3. 客户端轮询获取作业完成状态，读取HDFS中的最终输出结果；
4. 若存在任务失败，触发重试机制，重试次数耗尽则标记作业失败。

5.3 MapTask与ReduceTask数量确定规则

5.3.1 MapTask 数量规则

MapTask 是数据分片并行处理的核心，数量直接决定作业并行度。

- 默认规则：MapTask 数量 = 输入文件的 HDFS Block 块数量。Hadoop 默认Block块大小128M，一个1G文件会被分为8个Block，默认启动8个MapTask。
- 核心原理：一个 InputSplit 对应一个 MapTask，分片大小默认等于Block大小，最大限度贴合数据存储粒度，避免跨块读取。
- 手动调整方式：可通过修改最小分片大小、最大分片大小，主动增加或减少MapTask数量，适配小文件、大文件场景。
- 优化原则：小文件过多会产生大量小MapTask，造成资源浪费，需提前合并小文件；超大文件可适当调大分片，减少任务数量。

5.3.2 ReduceTask 数量规则

- 默认值：默认ReduceTask数量为1，所有数据汇总到一个Reduce任务中处理。
- 常规生产配置：手动设置，数量远小于MapTask数量，通常根据Key的种类、数据量分配。
- 性能风险：ReduceTask数量过少，会导致单个Reduce任务处理海量数据，成为作业瓶颈；数量过多会产生大量小输出文件，增加文件系统压力。
- 特殊场景：无需聚合汇总的任务，可设置ReduceTask数量为0，只执行Map阶段，提升效率。

5.4核心优化机制（底层原理）

5.4.1 数据本地性优化（Locality Optimization）

核心思想：移动计算，不移动数据。大数据场景下，磁盘IO速度远大于网络传输速度，跨节点传输数据是最大性能瓶颈。

三层调度优先级（从高到低）

1. 本地节点优先：将Map任务调度到「存储该数据块的Slave节点」，直接读取本地磁盘数据，无网络开销；
2. 同机架优先：若数据所在节点繁忙，调度到同一机架的空闲节点，机架内网络传输速度快、延迟低；
3. 跨机架调度：无本地、同机架资源时，才调度到其他机架节点，产生跨网络传输。

价值：极大降低集群网络IO压力，是MapReduce海量数据处理高效运行的核心保障。

5.4.2 推测执行机制（Speculative Execution / 冗余执行）

问题背景

集群中部分节点会因磁盘老化、CPU负载高、网络卡顿、资源抢占等问题，运行速度远慢于其他节点。单个慢任务会拖慢整个作业的完成时间，形成木桶效应。

机制原理：JobTracker 实时对比同批次所有任务的运行进度，当检测到某一个任务进度远低于平均进度、长期卡顿，不会无限等待，而是主动在空闲节点启动一个相同的备份任务（冗余任务）。

执行规则：谁先执行完成，就采用谁的结果，后完成的任务直接被强制终止，释放资源。

适用与禁用场景

- 适用：离线批量计算、只读数据任务；
- 禁用：会写入数据、非幂等性任务，避免重复写入导致数据错乱。

5.4.3 负载均衡机制

JobTracker 调度时不仅看数据本地性，还会实时统计每个TaskTracker的当前任务负载、空闲槽位、资源使用率，避免任务集中分配到少数节点。

通过动态分配任务，让所有Slave节点的任务执行时长、资源占用尽量均衡，杜绝部分节点过载、部分节点空闲的问题，最大化利用集群整体算力。

5.4.4 任务 JVM 复用机制

MRv1 默认每个任务新建 JVM，开销大。支持开启 JVM 复用，同一作业的多个小任务可复用同一个 JVM，减少启停开销，极大提升小任务集群效率。

5.5 故障恢复机制

MapReduce v1 具备完善的容错机制，可应对绝大多数节点、任务异常，保障集群稳定运行。

5.5.1 TaskTracker（Slave节点）故障

故障判定：JobTracker 依靠心跳机制判定节点故障：默认心跳超时时间为10次心跳无响应（约30秒），超时则直接判定该TaskTracker宕机。

恢复策略

- JobTracker 立即清除该节点的所有任务运行记录；
- 该节点上正在运行的未完成MapTask、ReduceTask，全部重新调度到其他空闲节点执行；
- 若该节点频繁故障、卡顿，JobTracker 会将其加入集群黑名单，不再分配任务，人工排查修复后可解除黑名单；
- 已完成的任务结果无需重跑，仅重试未完成的任务，最大限度节省资源。

5.5.2 单个Task任务故障

故障场景：任务代码异常、内存溢出、数据读取失败、子JVM崩溃等。

恢复策略

- TaskTracker 检测到任务失败，立即上报JobTracker；

- JobTracker 自动重新分配该任务，更换节点重试；
- 重试次数限制：默认最大重试4次，超过次数仍失败，则判定作业失败，终止任务并打印错误日志。

5.5.3 JobTracker（Master节点）故障

MRv1 最大的架构短板：JobTracker 单点故障（SPOF）。

- 整个集群只有一个JobTracker，无备用节点、无故障转移机制；
- 一旦JobTracker进程挂掉、主节点宕机，整个集群调度彻底瘫痪：所有正在运行的作业终止、新作业无法提交、节点全部失联；
- 恢复方式：仅能重启JobTracker进程，但重启后所有正在运行的作业全部丢失，需要重新提交执行，无法断点续跑。

核心结论：这是MRv1被YARN彻底替代的最根本原因。YARN将JobTracker的职责拆分，解决了单点故障问题，同时实现资源与任务解耦。

5.6 MRv1 架构优缺点

核心优点

- 架构简单清晰，主从分工明确，入门门槛低，易于运维；
- 容错机制完善，节点、任务故障可自动恢复，适合海量离线数据处理；
- 数据本地性优化优秀，极大规避网络IO瓶颈，计算效率高；
- 任务粒度细，单任务崩溃不影响全局，集群稳定性较好。

致命缺点（核心考点）

- 单点故障问题：JobTracker 单节点运行，宕机即集群瘫痪，无高可用；YARN later , Hadoop improved this structure. (out of scope) 后续版本的 Hadoop 用 YARN 改进了这个结构；
- 资源耦合严重：计算资源、任务调度、作业管理全部由JobTracker承担，压力极大；
- 扩展性差：集群节点数量过多时，JobTracker 心跳处理、任务调度压力过载，性能急剧下降；
- 资源利用率低：任务资源固定分配，无法动态调整，容易出现资源闲置或资源不足。

Part 6 Spark Motivation

6.1 MapReduce的缺陷

1. Force your data processing into Map and Reduce

强制将所有数据处理逻辑限定在 Map 和 Reduce 两个固定阶段。

扩展知识点：MapReduce 编程模型极度固化，无论业务简单或复杂，都必须拆分为映射、归约两个步骤。面对多表关联、多层嵌套计算、复杂统计逻辑时，代码冗余度极高，开发效率很低，灵活性极差。

2. Not good for jobs with a shared global state

不支持带有全局共享状态的计算任务。

扩展知识点：MapReduce 是无状态计算模型，不同任务、不同阶段之间无法直接共享内存变量。对于需要累计参数、迭代更新状态的场景（机器学习、图计算），无法直接实现，只能通过磁盘文件进行中转。

3. Read and write to Disk before and after Map and Reduce (stateless machine)

Map、Reduce 阶段前后均需要磁盘读写，属于无状态计算机制。

扩展知识点：这是 MapReduce 最大的性能瓶颈。每一轮计算的中间结果都会强制落地到 HDFS 磁盘，下一轮计算再重新读取。磁盘IO速度远低于内存读写，大量重复读写造成极大资源浪费。stateless 无状态指集群节点不会保留任何中间计算数据。

4. Not efficient for iterative tasks, i.e. Machine Learning

迭代式计算任务效率极低，典型场景为机器学习。

扩展知识点：机器学习、深度学习、图算法都需要成千上万次迭代运算。MapReduce 每次迭代都要落盘、读盘，迭代次数越多，IO 累积开销越大，几乎无法支撑大规模迭代计算。

5. Only for Batch processing

仅支持离线批处理计算。

扩展知识点：MapReduce 只能处理已经完全存储完毕的静态数据，无法处理实时增量数据，不适合实时业务场景。

6. Does not support interactivity, streaming data

不支持交互式查询与流式数据处理。

扩展知识点：MapReduce 每次提交任务都需要完整调度执行，无法即时返回查询结果，不适合数据分析人员交互式探索数据，也无法处理源源不断的实时流数据。

核心总结：MapReduce 痛点集中在：模型死板、磁盘IO频繁、不支持迭代计算、仅离线批处理、无交互、无实时能力，这也是 Spark 诞生的核心原因。

6.2 Apache Spark

1. A new general framework, which solves many of the shortcomings of MapReduce

全新的通用分布式计算框架，解决了 MapReduce 的绝大多数缺陷。Spark 2009年诞生，核心设计理念是**内存计算**，将中间计算结果缓存至内存，规避 MapReduce 频繁磁盘读写的短板，大幅提升计算速度。

2. It leverages the Hadoop ecosystem, e.g., HDFS, YARN, HBase, S3

完全兼容并复用 Hadoop 生态，支持 HDFS、YARN、HBase、S3 等主流组件。Spark **不替代 Hadoop**，只替代 MapReduce 计算引擎。Spark 可以直接部署在现有 Hadoop 集群上，沿用原有存储和资源调度体系，迁移成本极低，生态兼容性极强。

3. Has around 30 efficient distributed operations

内置约30种高效分布式数据操作算子，包含 join、filter、flatMap、groupByKey、reduceByKey、sortByKey、collect、count 等。MapReduce 仅有 Map、Reduce 两个基础算子，而 Spark 提供丰富的开箱即用算子，复杂的数据清洗、分组、关联、排序逻辑只需少量代码实现，开发效率成倍提升。

4. In-memory caching of data for iterative, graph, and machine learning algorithms

支持数据内存缓存，适配迭代计算、图计算、机器学习算法场景。可将高频使用的中间数据集缓存于集群内存，多次计算可重复复用数据，无需重复读取磁盘，是 Spark 迭代任务秒杀 MapReduce 的核心原因。

5. Native Scala, Java, Python, and R support

原生支持 Scala、Java、Python、R 四种开发语言。适配不同岗位需求，后端高性能开发优先使用 Scala，数据分析人员常用 PySpark，统计分析可使用 SparkR，受众更广。

6. Supports interactive shells for exploratory data analysis

支持交互式命令行，可用于探索性数据分析。通过 spark-shell、pyspark 可实现逐行代码执行、即时出结果，适合临时数据分析、代码调试、数据探索，完美弥补 MapReduce 无法交互的缺陷。

7. Spark API is extremely simple to use

Spark API 简洁易用，封装度高。高层API屏蔽了分布式底层细节，开发者无需关注集群调度、节点通信、数据分片，专注业务逻辑即可。

Spark 与 MapReduce 核心机制对比

方案	流程	每轮之间
Hadoop: Use Disk for Data Sharing	HDFS read → Iteration1 → HDFS Write → HDFS read → Iteration2 → HDFS Write	落盘（每次交接都是一次 HDFS 读 + 一次 HDFS 写）
Spark: In-Memory Data Sharing	HDFS read → Iteration1 →（内存）→ Iteration2 →（内存）	留在内存（只有最开始一次 HDFS read）

Hadoop MapReduce：依靠磁盘共享中间数据，每轮计算结束强制落盘，迭代任务反复读写磁盘，IO开销巨大。

Spark：依靠内存共享中间数据，计算结果常驻内存，多轮迭代直接读取内存数据，跳过磁盘IO，性能大幅提升。

官方排序测试结论：Spark 仅用 1/10 的集群节点，实现 3 倍于 MapReduce 的计算速度，单机资源利用率远高于 MapReduce。

6.3 The Spark Stack 组件栈

Apache Spark supports data analysis, machine learning, graphs, streaming data, etc. It can read/write from a range of data types and allows development in multiple languages.

Apache Spark 支持数据分析、机器学习、图计算、流式数据处理等多类场景，支持读写多种数据源，兼容多语言开发。Spark 是**一站式全域大数据计算平台**，并非单一计算引擎，覆盖离线、实时、机器学习、图计算全场景。

6.3.1 Spark 分层架构（由上至下）

1. 应用层：支持 Scala、Java、Python、R、SQL 多语言开发

2. 高层API层：DataFrames、ML Pipelines 结构化、机器学习流水线API

3. 核心组件层：Spark SQL（结构化查询）、Spark Streaming（实时流计算）、MLlib（机器学习库）、GraphX（图计算库），所有组件均基于内核构建

4. 底层内核：Spark Core，负责任务调度、内存管理、容错机制、RDD 核心抽象

5. 数据源层：兼容 HDFS、Hive、HBase、MySQL、S3、流式数据、JSON、高性能分布式文件系统等几乎所有存储介质

Part 7 Spark Technical Details

新引擎的原理层：一个统一抽象（RDD）、两类算子、一条依赖图（DAG），以及它们如何决定快慢。

7.1 RDD 弹性分布式数据集

Resilient Distributed Datasets (RDDs) are a fundamental data structure of Spark. It is an immutable distributed collection of objects.

RDD（弹性分布式数据集）是 Spark 的基础核心数据结构，是一组不可变的分布式对象集合。RDD 是 Spark 所有功能的底层基石，Spark SQL、Streaming、MLlib 高层 API 底层全部依赖 RDD 实现。immutable 不可变是 RDD 的核心特性，创建后无法修改，只能通过算子生成新 RDD，保证数据安全、支持容错。

7.1.1 三大核心特性

1. Resilient - capable of rebuilding data on failure

弹性，在节点故障时可自动重建丢失的数据。依靠**血缘依赖（Lineage）**记录所有计算过程，无需备份数据，故障后只需重放上游算子即可恢复数据，轻量化容错。

2. Distributed partition data across various nodes in a cluster

分布式，数据被切分为多个分区，分散在集群的不同节点。Partition 分区是 Spark 的最小并行计算单元，分区数量决定并行度，分区越多，并行计算能力越强。

3. Dataset - a collection of partitioned data with values

数据集，用于存储被分区的真实业务数据。

All the different processing components in Spark share the same abstraction: RDD. RDD is created by parallelizing a collection or reading a file.

Spark 的所有计算组件共用 RDD 抽象，RDD 可通过并行化本地集合、读取外部文件两种方式创建。日常开发两种常用的创建方式：本地集合并行化 sc.parallelize()、外部文件读取 sc.textFile()，覆盖绝大多数数据初始化场景。

RDD vs HDFS 块的对照

HDFS 块：物理存在的、多副本的、可寻址的数据单元。

RDD：逻辑上的、不可变的、可重算的数据集合。RDD 是分布在 HDFS 之上的逻辑视图。

7.2 Spark 的两大算子

Spark Operations = TRANSFORMATIONS + ACTIONS;

Action 给出 non-RDD values（判据）、触发 DAG 执行、将结果返回程序或写入外部存储。Action 才触发计算，那么调试时看一个 Spark 程序是否真的运行，就看有没有 Action，只写一串 Transformation 的程序什么都不会发生。

7.2.1 Transformation 转换算子

Transformation 是 RDD → RDD 的函数，且是惰性的（不会立刻执行）。因为变换是惰性的，Spark 就能在真正执行前优化整条链（合并、剪枝、决定分区）

Spark Transformation is a function that **produces a new RDD** from the existing RDDs. It takes RDD as input and produces one or more RDDs as output.

转换算子是以已有 RDD 为输入，生成新 RDD 的函数操作。转换算子具备**懒加载特性**，调用后不会立即执行计算，仅记录血缘依赖、构建 DAG 执行图，无任务调度、无资源消耗。同时天然支持容错，可链式连续调用。

窄依赖 Narrow Transformation

Each input partition will contribute to only one output partition.

每一个输入分区仅对应一个输出分区。一对一数据依赖，无需跨节点网络传输（无 Shuffle），计算速度快、开销低。常见算子：map、filter、flatMap。

宽依赖 Wide Transformation

Input partitions that contribute to many output partitions require a shuffle.

输入分区数据会分发到多个输出分区，需要 Shuffle 洗牌操作。一对多数据依赖，需要跨节点网络传输数据，是 Spark 的最大性能开销。Shuffle 的目的是将相同 Key 的数据汇聚到同一分区，完成分组、聚合。常见算子：groupByKey、reduceByKey、join、sortByKey。

7.2.2 Action 行动算子

Action is performed when we need actual dataset results; it does not form a new RDD and triggers DAG execution.

行动算子用于获取最终计算结果，不会生成新 RDD，同时触发 DAG 完整执行。行动算子返回非 RDD 结果（数值、列表、文件），是**真正触发集群计算**的唯一入口。所有懒加载的转换操作，都会在遇到 Action 后统一执行。结果可返回到 Driver 端或写入外部存储。

三个副作用：

1. 触发 DAG 执行 Triggers the execution of the DAG

2. 返回结果给程序 Returns the results to the program

3. 或者把数据写入 HDFS 或文件 Or writes the data to HDFS or to a file

看返回值：返回 RDD ⇒ Transformation；返回非 RDD（数字、列表、写文件）⇒ Action。Transformation 产生新 RDD，Action 不产生。这是两者的形式差别；语义差别是记录 vs 求值。

Action 是惰性的终点： Action 触发执行。所以一个 Spark 程序里，只有 Action 会出现真正的计算。

极简区分口诀：转换懒加载、只构图不运算；行动触发执行、出结果、跑任务。

7.3 DataFrames & Spark SQL

DataFrames are distributed datasets organized in named columns, similar to a relational database or a Pandas DataFrame, immutable, and track lineage.

DataFrame 是按命名列组织的分布式数据集，结构类似关系型数据库表、Pandas 表格，具备不可变性与血缘追踪能力。Pandas DataFrame 是单机、仅能处理小数据；Spark DataFrame 是分布式、支持 TB/PB 级大数据。和 RDD 一致，不可变、可容错、支持分布式并行计算。

DataFrame 构建方式

Construct from files, existing DFs, or a parallelizing Python collection.

可通过读取文件、转换已有 DataFrame、并行化本地集合三种方式构建。

1. 读文件 Read from file(s)

2. 从已有 DataFrame 变换 Transforming an existing DF (Spark or Pandas)

3. 把 Python 的 list 并行化 Parallelizing a Python collection list。

三种方式对应三个数据来源 ・外部文件 / 已有的表 / 内存里的 Python 对象。第二条特别实用：可以把小规模的 Pandas DataFrame 接进来。

RDD 与 DataFrame 对比扩展

RDD 是底层接口，DataFrame 建在它之上，并且多了模式与优化。RDD 是底层低级 API，无数据结构描述，Spark 无法自动优化，需要手动优化代码。

DataFrame 基于 RDD 构建，自带 Schema 表结构（列名、数据类型），Spark 可通过 **Catalyst 优化器**自动优化执行计划，运行速度远快于原生 RDD，是企业开发主流用法。

原文	层级 / 含义
RDDs provide a low-level interface into Spark	RDD 是面向 Spark 的底层接口
DataFrames are built on top of the RDDs and the core Spark API	DataFrame 建立在 RDD 与 Spark Core API 之上
DataFrames have a schema	DataFrame 有模式（schema）即列名与类型
DataFrames are cached and optimized by Spark	DataFrame 被 Spark 缓存并优化

7.4 Spark DAG 有向无环图

DAG consists of vertices (RDDs) and edges (operations). Action triggers DAG submitted to scheduler and splits into stages.

DAG 由顶点（RDD）和边（算子操作）组成，调用 Action 后，DAG 提交调度器并切分为多个 Stage 执行。DAG 完整记录数据计算血缘与依赖关系。

Stage 划分核心规则：**遇到宽依赖 Shuffle 就切分 Stage**，单个 Stage 内部全部为高效窄依赖操作。集群根据 Stage、Task 完成分布式并行计算，故障时可根据 DAG 血缘重算恢复数据。

边清单：A→B、B→C、B→D、B→E、C→E、S→D、D→E、E→F

顶点 = RDD，边 = 操作（Transformation）；Action 触发 DAG 提交给 DAG Scheduler，再由它切成 stages；会算一张给定 DAG 的源点 / 汇点 / 汇合点与边数。

Part 8 Spark Application

8.1 Spark's Main Use Cases 主要用例

三类用例：时序类（Streaming、Batch、Interactive）、算法类（Machine Learning、Graph、Spatial/GIS）、数据组织类（Data Warehousing、Exploratory Data Analysis）。

1. 流处理 Streaming Data 流式数据。Spark Streaming

2. 机器学习 Machine Learning 机器学习。MLlib

3. 交互式分析 Interactive Analysis 交互式分析。

4. 数据仓库 Data Warehousing 数据仓库。Spark SQL

5. 批处理 Batch Processing 批处理（与 MapReduce 重叠的那一块）。

6. 探索性数据分析 Exploratory Data Analysis 探索性数据分析（EDA）。

7. 图数据分析 Graph Data Analysis 图数据分析。GraphX

8. 空间（GIS）数据分析 Spatial (GIS) Data Analysis 空间/GIS 数据分析。

8.2 Spark in the Real World

公司	行业	核心技术组合	业务目标
Uber	出行	Kafka + Spark Streaming + HDFS	实时 ETL：原始事件 → 结构化数据
Pinterest	图片社交	Spark ETL + Spark Streaming	实时推荐相关 Pin、推断购买与旅行意向
Conviva	流媒体视频	Spark	优化视频流 / 管理直播流量，降低 churn

公司	行业	核心技术组合	业务目标
Capital One	金融	Spark + data science algorithms	理解客户、欺诈侦测
Netflix	流媒体	Spark	观看习惯洞察 + 影片推荐 + 内容创作

8.3 Spark的边界

三条限制

Even though Spark is versatile, that doesn't mean Spark's in-memory capabilities are the best fit for all use cases

原文	解读
For many simple use cases Apache MapReduce might be a more appropriate choice	对许多简单用例，Apache MapReduce 可能更合适
Spark users are required to know that memory they have is sufficient for a dataset	用户必须自己确认内存够放这个数据集
Spark was not designed as a multi-user environment	Spark 不是为多用户环境设计的

局限 2 + 局限 3：Spark 把中间数据放在共享的内存里，内存是独占且有限的资源，既要求单次数据集能装下（局限 2），又要求没有别的用户来抢（局限 3） 局限 1：通用框架有额外开销，简单任务的调度开销可能大于收益。

8.4 Summary

块	课件原文
Hadoop (HDFS, MapReduce)	Provides an easy solution for processing Big Data；Brings a paradigm shift in programming distributed system 为大数据处理提供简单的解决方案；带来了分布式系统编程的范式转变。
Spark	Has extended MapReduce for memory computations；for streaming, interactive, iterative and machine learning tasks 为内存计算扩展了 MapReduce；四类任务：流式 / 交互式 / 迭代式 / 机器学习。
Changing the World	Made data processing cheaper and more efficient and scalable；It is the foundation of many other tools and software 让数据处理变得更便宜、更高效、更可扩展；是许多其他工具与软件的基础。

Lecture 6 神经网络

Part 1 Neural Network：神经网络

问题定义：神经网络的沉浮史 → 图像识别 → 二分类 vs 多分类 → 车辆检测算例（2500 / 7500 / ≈300 万特征）→ 线性假设的两条不足。

线性假设需要手工造组合特征，而组合数会爆炸。

1.1 神经网络 Neural Networks for Classification

Neural Networks for Classification are very powerful, but have been dumped for decades because of their complexity and large computational resources

神经网络被弃用，因为复杂性和算力需求赶不上。 Resurrected a few years ago because of cheaper CPU, RAM, GPU, large clusters with many machines, high-speed networks, and many more training examples (big data). Now, like Deep Learning.

复兴原因：算力条件（更便宜的CPU / RAM / GPU 处理器、更大的集群large clusters with many machines、更快的网络 high-speed network）；数据条件（样本更多、就是大数据many more training examples, big data）。

深度学习是神经网络复兴后的名字，不是另起炉灶的新方法。神经网络是方法，深度学习是它复兴后的名字，图像识别是它最典型的任务。

1.2 Classification

对比项	二分类	多分类
标签写法	Giraffe / Not Giraffe	Giraffe / Car / Cat
正类个数	1 个正类，其余全归为负类	k 个互斥类别，每张图恰好属于 1 个
输出层	1 个输出单元（sigmoid → 概率）	k 个输出单元+ Softmax
真值 y 的形式	标量 0 或 1	one-hot 向量，只有 1 维是 1
用到的损失	交叉熵的 k = 2 特例	交叉熵/ 均方误差

1.2.1 Binary Classification

只有是 / 不是两种结果，可以用一个概率表示。三张图里只有长颈鹿是 Giraffe，另外两张（车、猫）都被归为 Not Giraffe。正类 1 个、负类归成一堆。

1.2.2 Multi-class Classification

类别数从 2 变成 3，输出层就不能再用一个概率表示。需要多个输出单元，每个类别一个三个互斥标签，同一组图换成 Giraffe / Car / Cat 三个互斥类别，每张图只属于其中一个。

1.2.3 Example: Car Detection

如果把像素两两相乘当特征，会有 300 万个特征。所谓识别，就是从这堆数字里判断它是不是车，输入是数值，不是概念。

灰度图每像素 1 个数 ⇒ 50 × 50 = 2500；RGB 每像素 3 个数（R/G/B）⇒ 2500 × 3 = 7500。

x 把 50×50 的图按顺序摊平成一个 2500 维列向量，图像从此就是普通的特征向量，可以喂给任何学习算法。

二次特征的量级 ・把像素两两乘积 $x_i \times x_j$ 也当特征，个数 ≈ $n^2 / 2 = 2500^2 / 2 \approx 312$ 万，所以课件写 ≈ 3 million features。

1.3 Motivation

动机总结：复杂度爆炸，所以换方法

1. Need non-linear decision boundary

需求一：非线性边界，右上角的图：x_1 - x_2 平面上蓝色空心圈与红色叉无法用一条直线分开。

2. Need complicated hypothesis with different combinations of features

需求二：复杂的组合假设，算式里除了一次项 $\theta_1 x_1$、$\theta_2 x_2$，还出现了特征两两相乘的项，$ g(\theta^T X) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_k x_1 x_2 + \theta_{k+1} x_1 x_3 + \cdots)$

The computation complexity is extremely large. Hence we have to take another method, called Neural Network.

线性假设不够 → 手工加特征组合 → 组合数爆炸（≈3 million）→ 算不动 → 改用神经网络自动学特征。

Part 2 Perceptron：感知机

最小单元。大脑启发 → 线性神经元的阈值与偏置 → Sigmoid 激活 → 感知机四部件 → AND / OR 的权重解 → XNOR 线性不可分 → 两层网络解决。

2.1 Models and Perceptron

2.1.1 Models

It learns new features from input features. 模型自动学特征 Its architecture is based on our brain structure. 架构来源于人的神经网络，但并不模拟生物机制 The axon terminal（轴突末梢）of one neuron connects with the dendrite（树突）of another neuron, which consists of quite a complicated neural network.

2.1.2 Perceptron: The Artificial Neuron

$x_i $are inputs and $a = h(x; \theta)$ is the output.

> 感知机 = 把输入向量 x 映射成输出 a 的函数 h(x; θ)，θ 是要学、要逼近最优的那个参数组
>
> x is an input vector. In this example, x = (x_1, x_2, x_3) θ is the parameters that result in the best function approximation 需要学习的最佳参数

Node in orange represents the body of neuron.

2.2 A Simple Linear Neuron 线性神经元：阈值与偏置

运算：加权和 + 阈值判定 把偏置并进权重

输出结果是 1 或 0 两种，没有中间值。逻辑神经元是光滑的 sigmoid。

三种等价写法：阈值式（$\geqslant -b$）⇔ 偏置式（$+b \geqslant 0$）⇔ 内积式（$wx \geqslant 0$）。

2.3 Logistic Neuron 与 Sigmoid 激活

把阶跃换成光滑的 sigmoid，输出就变成 (0,1) 区间里的数，可当概率用。

2.3.1 Linear vs Logistic

Logistic Neuron: Sigmoid, Non-linear Activation Function Linear：直接输出

$h(x; \theta) = wx$

Logistic：经过激活函数 f（= sigmoid），输出被压进 (0, 1)

$h(x; \theta) = f(wx) = 1/(1 + e^{(-wx)})$

2.3.2 Perceptron four parts

一个感知机的四个部件：输入 → 参数 → 加权和 → 激活 → 输出

部件	说明
Inputs	输入 x_i（可以是像素矩阵拉直后的向量）
Parameters (weights and biases)	权重 $w_1 \cdots w_n$ 与偏置 b（ = w_0）
Weighted Sum	$\Sigma\ w_i x_i$ 或内积 wx
Activation Functions	sigmoid（也可换成别的非线性函数）

Tips：sigmoid 是非线性激活（Non-linear Activation Function）。如果激活也是线性的，多层叠加仍等价于一层线性模型，非线性是多层有用的前提。

2.4 特征学习 Feature Learning:

一个神经元能学出什么：AND 与 OR 都是线性可分问题，各由一组权重实现。

2.4.1 AND：权重已知的算例

先算 $z = w_1x_1 + w_2x_2 + w_0$，再取 g(z)。

2.4.2 OR：同样的形式，权重留白

把三个权重都留给读者自己解。

一组可行解：取 $w_1 = w_2 = 20$、$w_0 = -10$，符合 OR 真值表。

与 AND 的差别：AND 的 $w_0 = -30$，OR 的 $w_0 = -10$；两条权重都是 20。偏置决定门槛有多高。

2.4.3 线性假设 Linear Hypothesis

两者都是线性可分问题，一个单神经元就能解决。

衔接：线性不可分的反例，XNOR。

2.5 Non-Linear and Multi-Layer NN

问题：XNOR 一条直线分不开，加一层隐藏层就能分开，多层的价值在这里被具体化。

解决： Multi-Layer NN Model $x_1, x_2 \to a_1, a_2 \to h$

a_1 的偏置 −30； $x_1 \to a_1 = 20$，$x_2 \to a_1 = 20$ a_2 的偏置 +10； $x_1 \to a_2 = -20$，$x_2 \to a_2 = -20$

$a_1 \to h = 20$，$a_2 \to h = 20$；h 的偏置 −10

第一层学出 AND 与 NOR 两个中间特征，a_1 算的是 AND（只有 (1,1) 为 1），a_2 算的是 NOR（只有 (0,0) 为 1）。

第二层再把这两个新特征 OR 起来 → 正好是 XNOR。

a_1 列 = 0,0,0,1 是 AND；a_2 列 = 1,0,0,0 是 NOR；h 列 = 1,0,0,1 是 XNOR。

Part 3 多层神经网络与前向计算

模型的语言：三种网络形状 → $a_i^{(j)}$ / $w_ki^{(j)}$ ；

符号约定 → 一层的标量式与矩阵式 → 网络 = 函数复合。

3.1 网络形状与符号约定 Neural Network and Notation

1. input layer (4) , hidden layer (5/3/2) , output layer (1)

四层网络；三层隐藏层说明深就是把隐藏层一路叠下去

2. Input layer (3) , hidden layer (4) , output layer (2)

输出 2 个单元，适用于二分类或多标签任务

3. input layer (3) , hidden layer 1 (4) , hidden layer 2 (4) , output layer (1)

两个隐藏层 + 单输出，适用于二分类

3.2 Model Representation

符号	读法要点
$a_i^{(j)}$ 第 j 层第 i 个神经元（同时也是它的输出值）	上标 j 是层号，下标 i 是层内序号；神经元与它的输出共用同一符号
$w_ki^{(j)}$ 从 $a_i^{(j-1)}$ 连到 $a_k^{(j)}$ 的权重	第一个下标 k 是去哪，第二个下标 i 是从哪来
X 输入向量	$(1, x_1, x_2, x_3)^T$，第一维恒为 1（偏置项）
$a^{(j)}$ 第 j 层的输出向量	$(1, a_1^{(j)}, \cdots)^T$，同样第一维为 1
θ 全部参数的统称	每层各有一份：$\theta^{(1)}$、$\theta^{(2)}$

前向传播

1. 逐个神经元写 Connections between Input Layer and Hidden Layer 1:

$ $a_1^{(1)} = f^{(1)}(w_{10}^{(1)} + w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3)$ $ $a_2^{(1)} = f^{(1)}(w_{20}^{(1)} + w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{23}^{(1)}x_3)$ $ $a_3^{(1)} = f^{(1)}(w_{30}^{(1)} + w_{31}^{(1)}x_1 + w_{32}^{(1)}x_2 + w_{33}^{(1)}x_3)$

三条式子结构相同：一个偏置 + 三个加权输入，外面套同一个激活 $f^{(1)}$。三条并列，就是三个隐藏神经元各自的计算

$w_{10}^{(1)}/w_{20}^{(1)}/w_{30}^{(1)}$即 bias，三个偏置和 b 是一回事，只是写成了这个形式。

2. 合并成矩阵：Rewrite it by matrix calculation

$a^{(1)} = f^{(1)}(W^{(1)}x)$

Function $f^{(1)}$ applies to each component in a vector $f^{(1)}$

逐分量作用，因为 sigmoid 一次只能算一个数，向量上的用法是对每个分量分别算

$W^{(1)}$ 的三行对应三个隐藏神经元，四列对应 (偏置1, x_1, x_2, x_3)；矩阵乘法的结果就是一个 3 维向量。

3. 逐层复合：Layer Composition

把隐藏层 1 → 输出层也写成矩阵式，然后把两层叠起来，网络的最终假设就是各层函数的复合。

Connections between Hidden Layer 1 and Output Layer:

$a_1^{(2)} = f^{(2)}(w_{10}^{(2)} + w_{11}^{(2)}a_1^{(1)} + w_{12}^{(2)}a_2^{(1)} + w_{13}^{(2)}a_3^{(1)})$

对比上一步，把输入从 x 换成 $a^{(1)}$：上一层的输出就是下一层的输入。bias 同样存在。

$ $a^{(2)} = f^{(2)}(W^{(2)} a^{(1)})$

Function $f^{(2)}$ applies to each component in a vector

输出层只有一个神经元，所以结果是一个标量（写成 1 维向量）。

第一层算出来的 $a^{(1)}$ 只有三个数，进第二层前前面补一个 1，才能让第二层的偏置项进得来。每往下一层都要补一次。叠起来：整个网络是一个复合函数

$a^{(2)} = f^{(2)}(W^{(2)}a^{(1)}) = h^{(2)}(a^{(1)}; \theta^{(2)})$ $ $a^{(1)} = f^{(1)}(W^{(1)}x) = h^{(1)}(x ; \theta^{(1)})$ $ $a^{(2)} = h^{(2)}(h^{(1)}(x ; \theta^{(1)}) ; \theta^{(2)}) = h(x ; \theta)$

Each layer is a function. 每一层 = 一个函数 $h^{(j)}(x; \theta^{(j)})$ The final value is a function composition from layer functions. The latter one will be the outer one.

整个网络 = 各层函数依次复合，训练网络就是拟合一个复合函数，不是拟合 k 个独立函数；后面的层写在外层；全网络只有一组参数 θ，梯度下降是对整体 θ 做的。

三层结构被压缩成$h(x; \theta) = h^{(2)}(h^{(1)}(x))$。深：再叠一层就是再复合一个函数。

Part 4 多分类输出与 Softmax

多分类要多个输出单元，还要一层归一化才能当概率用。

4.1 多分类 Multi-class Classification

输出向量之和不是 1，不能直接当概率。

Multiple output units: One-vs-all 每个输出单元 负责 回答 是 不是 这一类。输出单元数 = 类别数 k。

one-hot：真值 $y^{(i)}$ 是四个向量之一，只有一个分量为 1，其余为 0。

问题：课件给的输出向量不是概率分布。单看这组数，既不能求和为 1、也不好直接比较。如果按取最大（argmax）判断，最大的是 0.3，对应第 4 类 Truck。

解决：把 [0.2, 0.1, 0.05, 0.3] 这种原始输出归一化成概率向量，这就是 Softmax。原始输出的各分量之和不一定为 1，需要 Softmax 才能当概率用。

4.2 Softmax Function

Softmax 是输出层的激活函数：接收任意实数向量，输出一个和为 1、每项非负的概率向量。

Softmax is an activation function to normalize the outputs. Given a real vector a ∈ R^k (Classification with k classes)

$ s_j(a) = e^{(a_j)} / Σ_{i=1..k} e^{(a_i)}$

性质	数学形式	作用
非负	$< br > e^{(a_j)} > 0$	分子是正数，输出恒为正
归一	$\Sigma_j s_j(a) = 1$	分母是所有 e^(a_i) 之和 ⇒ 输出和为 1

性质	数学形式	作用
放大最大项	$a_j \gg a_i \Rightarrow s_j(a) \to 1$	指数把差距放大，明显的类更明显
可当概率	s(a) 是 probability vector	固定搭配：softmax输出后配交叉熵Cross-entropy loss，直接与 one-hot 真值比较

Softmax 是激活函数（activation function），作用位置是输出层；它本身不带参数，只是归一化。

Part 5 代价函数与梯度下降

5.1 代价函数：Cost Function

作用：衡量模型预测结果到底有多差

符号	含义	对应
$y = (y_1, \cdots, y_k)$	ground truth，one-hot 向量（ $y_j = 1$ 当且仅当属于第 j 类）	四个 4 维向量
$h(x; \theta)$	probability prediction，第 j 个分量给出第 j 类的预测概率	s(a)，即 Softmax 之后的输出
$h(x; \theta)_j$	第 j 类的预测概率（下标取分量）	0.2 / 0.1 / 0.05 / 0.3
cost function	评价预测有多差的函数	Cross-entropy loss and mean squared error cost

对于k分类任务： 真实标签$y = [y_1, y_2 \cdots y_k]$

one-hot 编码：样本属于类别 j 则$y_j = 1$，其余全部 = 0；

模型预测输出向量$h(x; \theta)$，$[h(x; \theta)]_j$ 代表模型预测该样本属于第 j 类的概率

不同任务可以选不同代价函数，h(x; θ) 与 y 的区别是 y 是硬标签（只有 0/1），h(x; θ) 是软概率（0 到 1 之间的实数）。代价函数就是衡量这两者的差距。

different scenarios：不同场景用不同代价函数，交叉熵与均方误差不是替代关系，而是并列可选项。

y 是真值（one-hot）、h(x; θ) 是预测概率向量、J(θ) = L(y, h) 是代价。

5.1.1 交叉熵损失：Cross-Entropy Loss

分类问题的常用代价：只惩罚真值的预测概率，预测得越不自信，损失越大。

$L(y,h)=-\sum_{j=1}^k y_j\log_2[h(x;\theta)]_j=-y^T\log_2 h(x;\theta)$

- y 只有 1 个位置是 1，其余为 0，公式简化：$L = -\log$ (模型预测真实类别的概率)
- 如果模型给真实类别的预测概率很低 → 损失 L 很大； 训练目标：调整参数 θ，尽量让损失变小。

预测低的 Loss越大，越不自信，罚得越狠，而且是指数级的严厉。

算例：若softmax [0.2584, 0.2338, 0.2224, 0.2855]，则 $L = -\log_2 0.2855 = 1.8083$ bit。

Change θ to keep cost as low as possible 改变参数使代价最小，即使用梯度下降。

5.1.2 均方误差损失：Mean Squared Error Cost

$L(y,h)=\frac1k\sum_{j=1}^k\big([h(x;\theta)]_j-y_j\big)^2$

- 预测值 h 和真实 y 越接近，损失 L 越小
- 分类任务也可以用，但不如交叉熵好用（梯度容易消失）

同一个任务可以换一把「尺子」：均方误差衡量预测向量与真值向量之间的欧氏距离，本页与上一页构成两种并列选择。

对比项	交叉熵	均方误差
形式	$-\Sigma_j y_j log_2 h_j$	$(1/k)\,\Sigma_j (h_j - y_j)^2$
用到 y 的几维	只有 1 维（因为 y 里只有一维是 1）	全部 k 维（每维都参与平方）
对自信的错误的惩罚	极严厉（log 在 0 附近发散）	有上界（最大 1，因为 h_j、y_j ∈ [0,1]）
梯度表现	误差越大梯度越大，收敛快	预测接近 0 或 1 时梯度会变小（饱和）

算例：真类为第 4 类，h = [0.2584, 0.2338, 0.2224, 0.2855]，各项平方和为 0.6813，除以 k = 4 得 L = 0.1703。

两种损失区别：交叉熵只取真值那一维、对自信的错误惩罚更狠；均方误差用全部 k 维、惩罚有上界。

5.2 梯度下降：Gradient Descent

一阶导数迭代优化算法，用来找损失函数的局部最小值

Gradient descent is a first-order iterative optimization algorithm for finding a local minimum of a differentiable function. The idea is to take repeated steps in the opposite direction of the gradient of the function at the current point, because this is the direction of steepest descent.

核心思想：沿着**梯度的反方向**一步步更新参数，直到走到（局部）最小值。

$x = x_0 - \alpha\left.\frac{\partial f(x)}{\partial x}\right|_{x=x_0}$

- αlearning rate 学习率，控制每一步走多大。太大可能越过最低点甚至发散；太小则收敛很慢。
- $∂f/∂x $当前的导数（陡峭程度与方向）。
- 减号表示往梯度反方向走，梯度指向上升最快的方向，所以要下降就走反方向。

神经网络参数更新公式：

$w_i = w_i - \alpha \frac{\partial J(\theta)}{\partial w_i}$

每一步用参数的梯度，减去学习率 × 梯度，更新权重 w，降低代价 J

局限： local minimum

Part 6 反向传播：误差传回

链式法则 + 中转量复用 + 反转的工作流 + 梯度乘权重。

6.1 Chain Rules

神经网络是多层嵌套复合函数，求梯度必须用链式法则：

若 $z = f(y),\ y = g(x)$

$\frac{\partial z}{\partial x}=\frac{\partial z}{\partial y}\cdot\frac{\partial y}{\partial x}$

例子：$z=\frac12(x-5)^2$，令$y=x-5,\ z=\frac12 y^2$ $z'(x) = f'(y) \cdot g'(x) = y \cdot 1 = x - 5$

6.2 单层网络及梯度拆解 Single Network and Gradient Decomposition

$a = h(x;\theta) = f(z) = f(w_0 + w_1 x_1)$

$z = w_0 + w_1 x_1$

$J(\theta) = L(y, a)$

- f 激活函数；a 输出预测；J损失函数；
- 目标：求权重w_1, w_0的梯度，用梯度下降更新 ；
- 链式展开：

$\frac{\partial J}{\partial w_i}=\frac{\partial J}{\partial a}\cdot\frac{\partial a}{\partial z}\cdot\frac{\partial z}{\partial w_i}$

1. $\frac{\partial J}{\partial a}$：代价函数对输出 a 求导，选定的损失函数决定；
2. $\frac{\partial a}{\partial z}$：激活函数 f 的导数，选定的激活函数决定；
3. $\frac{\partial z}{\partial w_i}$：线性部分$z = w_0 + w_1 x_1$对权重求导

6.3 两层网络

两层网络（每层 1 个神经元，引出反向传播）

$a^{(2)} = f(w_{21} a^{(1)} + w_{20}) = f(w_{21} f(w_{11} x_1 + w_{10}) + w_{20})$

w_{10}, w_{20}是 bias 偏置

网络链路：

$x_1 \to z^{(1)} \to a^{(1)} \to z^{(2)} \to a^{(2)} \to J$

继续链式展开：

$\frac{\partial J}{\partial w_{11}}=\frac{\partial J}{\partial a^{(2)}}\frac{\partial a^{(2)}}{\partial z^{(2)}}\frac{\partial z^{(2)}}{\partial a^{(1)}}\frac{\partial a^{(1)}}{\partial z^{(1)}}\frac{\partial z^{(1)}}{\partial w_{11}}$

节点	关系	图中标注
$z^{(1)}$	由 x_1 与权重 w_{10}、w_{11} 汇入	第一层的加权和
$a^{(1)}$	由 $z^{(1)}$ 经过外层函数得到	第一层的输出（下一层的输入）
$z^{(2)}$	由 $a^{(1)}$ 与权重 w_{20}、w_{21} 汇入	第二层的加权和
$a^{(2)}$	由 $z^{(2)}$ 得到，即 prediction	网络的最终预测
J	由 $a^{(2)}$ 与真值 y 得到	代价

6.4 反向传播：Backward Propagation

核心：从输出层往输入层反向算梯度，复用已经算好的结果 $\delta J/\delta z$，避免重复计算。

正向传播：从左往右，输入→计算每层z, a，得到预测与损失；

反向传播：从右往左（输出层往回）逐层求梯度。梯度反向流动，乘权重，一层一层往前传。

1. 第 L 层神经元先算出$\frac{\partial J}{\partial z^{(L)}}$
2. 用这个值，计算本层权重梯度 $\frac{\partial J}{\partial w_{Li}}$
3. 再乘以本层权重，把梯度传递给上一层 (L-1)：$\frac{\partial J}{\partial a^{(L-1)}}=\frac{\partial J}{\partial z^{(L)}}w_{L1}$
4. 上一层继续计算$\frac{\partial J}{\partial z^{(L-1)}}$，不断向前传递

拓展：一层多个神经元

公式形式不变，需要对后一层所有神经元求和：

$\frac{\partial J(\theta)}{\partial a_k^{(L-1)}}=\sum_{j=1}^{n_L}\frac{\partial J(\theta)}{\partial a_j^{(L)}}\frac{\partial a_j^{(L)}}{\partial z_j^{(L)}}w_{jk}^{(L)}$

n_L = L 层神经元数量

每一层把 $\delta J/\delta a$ 这个信号传给上一层，上一层再补上自己那段激活导数继续往下传，逐层回传。

量	单神经元	一层多神经元
本层加权和	$z = w_0 + w_1 a^{(L-1)}$	$z_j^{(l)} = w_j0^{(l)} + \Sigma_k w_jk^{(l)} a_k^{(L-1)}$
本层权重梯度	$\delta J/\delta w_L i = (\delta J/\delta z)(\delta z/\delta w_L i)$	$\delta J/\delta w_j k^{(l)} = (\delta J/\delta a_j^{(l)})(\delta a_j^{(l)}/\delta z_j^{(l)})(\delta z_j^{(l)}/\delta w_j k^{(l)})$
回传给上一层	$\delta J/\delta a^{(L-1)} = (\delta J/\delta z^{(l)}) \cdot w_L 1$	$\delta J/\delta a_k^{(L-1)} = \Sigma_j (\delta J/\delta a_j^{(l)})(\delta a_j^{(l)}/\delta z_j^{(l)}) \cdot w_j k^{(l)}$
是否需要求和（二者区别）	不需要（只有一个上层神经元）	需要，求和范围 1⋯n_L

6.5 多条训练记录的训练循环：Multiple Training Records

把单样本的前向 → 算代价 → 反向 → 更新（Forward → Calculate Cost → Backward → Update weights）放进数据集循环，构成完整的训练流程。

训练集 $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$，对每一条样本：

1. 前向传播 Forward Propagation：从左向右，逐层算出所有层$a_j^{(L)}$
2. 计算当前样本的代价 $J(\theta)$
3. 反向传播 Backward Propagation：从右向左，逐层算出$\frac{\partial J}{\partial z_j^{(L)}}$和所有权重梯度$\frac{\partial J}{\partial w_{jk}^{(L)}}$
4. 更新后的 θ（用梯度下降）：梯度下降更新全部权重，循环迭代直到损失收敛

Part 7 从神经网络到深度学习

层数堆上去带来的问题：错误率、层次化特征。

7.1 深度学习

浅层前馈网络 vs 深度神经网络

Non-deep feedforward NN（浅层前馈网络）：单个隐藏层，特征提取能力弱，依赖人工设计特征，泛化能力差。

Deep neural network（深度神经网络）：⩾2 个隐藏层，多层堆叠，分层自动特征学习，无需人工设计特征。

> 定义：深度 = 隐藏层数量，层数越多，网络越深

ImageNet：深度带来性能飞跃（Revolution of Depth）

网络越深，图像识别错误率越低；ResNet 解决了深层网络训练难的问题。

深层的好处：层次化特征

- 浅层：学习底层基础特征。线条、边缘、色块、纹理
- 中层：学习局部组合特征。轮廓、部件（眼睛、鼻子、纹理块）
- 深层：学习高层语义特征。完整物体、人脸、场景、抽象概念

深度学习的本质：通过多层嵌套，自动从数据中学习从简单到复杂的特征层级。

7.2 Modern Applications

7.2.1 Generative Pre-trained Transformer（文本）

GPT 的整体流水线：Input → Input Embedding（旁边并入 Positional Encoding）→ Dropout → Transformer Block Layer 1 → Layer 2 → ⋯ → Transformer Block Layer L → LayerNorm → Linear → Softmax → Output。

Transformer Block 是重复单元，堆叠 L 层。单个Transformer Block的内部：LayerNorm → Linear（分出三路 → 就是 Q/K/V）→ 多头注意力（三路各自走 MatMul / Mask / Softmax / Dropout，图上标 Head1 / Head2 / Head3）→ Linear（合并各头）→ Dropout → 残差相加 → LayerNorm → Linear → GELU → Linear → Dropout → 残差相加 → 输出。

残差连接：一处在注意力之后、一处在 GELU 前馈之后。

7.2.2 扩散模型：Diffusion Model（文生图）

扩散模型是生成模型：输入一段文字，输出一张图。

两代并排是为了展示迭代速度：同一类模型在一年内效果明显提升。

生成模型的另一条技术路线是 GAN，两者都在回答如何让网络生成新数据。

Part 8 卷积神经网络

网络过深会出现：梯度消失/梯度爆炸 + 模型退化（层数增加，训练精度反而下降）

ResNet 创新：残差连接（shortcut），允许梯度直接跨层传递，避免链式累乘导致的梯度消失，支持超深层网络训练。

全连接在图像上要 10 亿个参数；局部连接 + 参数共享把这件事变得可行。

8.1 全连接处理图像：Fully Connected on Images

深度网络中 1000×1000 的图像接 1000 个隐藏单元，需要 10 亿个参数。

Spatial correlation is local（空间相关性是局部的：一个像素只和它附近的像素相关）;

Better to put resources elsewhere!（别把资源全砸在每个输入连每个单元上）。

若将1000×1000图像拉直为一维向量输入全连接网络：

1. **参数量爆炸**：产生数亿甚至十亿级参数，算力消耗极大、极易过拟合
2. **丢失空间信息**：二维图像的位置、相邻相关性被彻底破坏
3. **特征冗余**：全连接层认为每个像素独立，忽略图像局部相关性（相邻像素高度关联，远距离像素无关）

8.2 局部连接与参数共享：Reduce Connection to Local Regions

局部连接 + 参数共享 = 卷积

减少参数的两种做法：每个神经元只看图像的一小块（局部连接），同一组参数在图像各处重复使用（参数共享）。

1. 局部连接：不再让神经元接收整张图输入，每个神经元只感知图像局部小区域。大幅减少连接数与参数量，贴合图像局部相关性规律。
2. 参数共享：图像的基础特征（边缘、纹理、角点）具有**平移不变性**：左上角的边缘特征和右下角的边缘特征原理一致。同一个卷积核（Filter/Kernel）在整张图像滑动，全程复用同一套权重参数，极大压缩参数量，这是CNN高效的根本原因。

Convolutions with learned kernels(filter)。学出来的卷积核就是那组共享的小矩阵。

8.3 CNN：Convolutional Neural Network

Convolutional Neural Network (CNN) is proposed to reduce the number of parameters, preserve the image layout information, and make the network deeper 减少参数（局部连接 + 参数共享）、保存图像的布局信息（不是拉直成向量，行列结构保留）、使网络更深（参数省了，就有余力加层）。

Key Components in CNN

流水线：输入图像 → 卷积层（提特征）→ 激活层（非线性）→ 池化层（降维）→ 重复堆叠 → 拉直Flatten → 全连接层 → 分类输出

Input → Convolution → Pooling → Fully Connected → Output。

阶段	包含的层	作用
Feature Extraction	Convolution → Pooling	从像素里逐层抽取特征
Classification	Fully Connected → Output	把抽出的特征映射到类别

8.4 卷积层：Convolution Layer

操作规则：卷积核在特征图上按固定步幅滑动，每滑动一个位置，执行对应位置像素相乘，全部求和，输出一个特征值，最终生成全新特征图。

超参数详解

Kernel Size 卷积核尺寸：常用3×3、5×5，尺寸越大感受野越大

Stride 步幅：每次滑动的像素数，stride越大输出特征图越小

Padding 填充：图像边缘补0，保留边缘特征、控制输出尺寸

卷积核数量：一个卷积核提取一种特征，多个卷积核可同时提取边缘、纹理、色块等多类特征，输出多通道特征图

8.5 池化层：Pooling Layer

降采样：把特征图变小以减少计算量，常见做法是取最大值或取平均值。

作用：

1. Makes the representations smaller and more manageable 让表示更小更易管理；
2. Operates over each activation map independently 逐特征图独立处理；
3. Max pooling and average pooling are widely used max / average 两种最常用；
4. Pooling can speed up the training process 尺寸减半，后面各层的计算量与参数量大幅下降。

两种池化方式对比

最大池化 Max Pooling（最常用）：取窗口内最大值，保留最强特征，抗干扰能力强

平均池化 Average Pooling：取窗口平均值，保留整体特征，但容易模糊细节

关键区别：卷积层有可学习参数，池化层无任何可学习参数，只是固定的统计操作。

例：max pool 2×2、stride 2

Part 9 GAN：生成对抗网络

和Diffusion Model 扩散模型一样，都在回答如何让网络生成新数据。

两个网络、一个损失、方向相反：从纳什均衡到极小极大博弈。

9.1 纳什均衡：Nash Equilibrium

In game theory, the Nash equilibrium, is a set of strategies that players act out, with the property that no player benefits from changing their strategy. Intuitively, if any given player were told the strategies of all their opponents, they still would choose to retain their original strategy.

纳什均衡来自博弈论，是一组策略集合：博弈里所有玩家选定策略之后，任何单独一个玩家单方面改变自己的策略，都不会获得更好的收益。这是 GAN 对抗训练的核心思想。

Nash showed that there is a Nash equilibrium for every finite game, a finite number of players in which each player can choose from finitely many pure strategies

纳什定理：任何有限博弈，一定存在纳什均衡。优先博弈是玩家数量有限，每个玩家可选的纯策略数量也是有限的。 When it reaches Nash Equilibrium, if any competitors change their current strategies, they will get a worse result.

结论：达到纳什均衡后，任意竞争者单方面改变策略，结果会变差。

GAN 里，Generator 生成器、Discriminator 判别器，就是博弈里的两个玩家，训练目标就是到达纳什均衡。

第 62 页先补一段博弈论：GAN 的对抗训练，目标就是让两个网络走到纳什均衡。

9.2 GAN Main Idea and Framework

GAN 包含两个独立神经网络，对抗式同步训练：

1. Generator takes in random noise and returns a fake image.
2. Discriminator takes in both real and fake images and returns probabilities with 1 representing authenticity and 0 fake
3. One neural network, called the generator, generates new data instances
4. One neural network, called the discriminator, evaluates them for authenticity; i.e. decides whether each image is real.

角色	输入	输出	目标
Generator（G）	random noise（随机噪声）	fake image（图中生成了一个手写的 8）	造出以假乱真的样本
Discriminator（D）	both real and fake images	probabilities：1 = 真，0 = 假	分辨真伪

训练循环： 生成器造假图 → 判别器判断真假 → 两者互相迭代对抗，共同提升能力。

真实样本x：从真实数据集采样，送入判别器D，$D(x)$希望越接近 1 越好（识别为真）

噪声z输入生成器G，得到伪造样本$G(z)$送入判别器；

D希望$D(G(z))$尽量接近 0（识别出这是假图）；

G希望$D(G(z))$尽量接近 1（欺骗 D，让 D 以为是真图）

D 和 G 玩极小极大对抗游戏，目标最终达到纳什均衡，联合训练得到强大的生成器和判别器

9.3 极小极大博弈：Min-Max Game

损失函数：

$$L = E_x[\log(D(x))] + E_z[\log(1 - D(G(z)))]$$

符号拆解：

1. $D(x)$：判别器评估真实数据x是真样本的概率
2. $E_x[\cdots]$：在全部真实数据上求数学期望
3. $G(z)$：生成器根据噪声z输出伪造样本
4. $D(G(z))$：判别器评估生成器造的假样本是真样本的概率
5. $E_z[\cdots]$：在全部随机噪声输入上求期望

博弈目标：

- 判别器 D：最大化这个 Loss，尽可能分清真假
- 生成器 G：最小化这个 Loss，尽可能让判别器分不清真假

minimax（极小极大）博弈：D 最大化损失，G 最小化损失，二者对抗。

9.4 GAN 图像合成

GAN 能生成多个类别、逼真度较高的图像。

第 67 页回到粗版大纲 ・三条：Neural Networks / Deep Neural Networks / Overfitting and Underfitting——与第 2 页同一版本。位置在 GAN 之后、Overfitting and Underfitting 之前，标志进入全讲最后一块。

考点 ・ 能列举 GAN 的典型应用场景（图像合成 / 生成）；并知道它与扩散模型同属生成模型两条路线。

Part 10 Overfitting and Underfitting：过拟合与欠拟合

属于机器学习基础问题，GAN 训练也同样会遇到过拟合问题。欠拟合、过拟合是机器学习算法的通用问题，不只在分类任务中出现，在回归任务中同样存在。

10.1 Underfitting 欠拟合

左图	中图	右图
$\theta_0 + \theta_1 x$	$\theta_0 + \theta_1 x + \theta_2 x^2$	$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$
High Bias（Underfitting）	Low Bias, Low Variance（Goodfitting）	High Variance（Overfitting）
一条直线，明显拟合不上	一条平缓抛物线，贴合趋势	一条蛇形曲线，穿过每个点

模型拟合能力太弱，连训练集的数据规律都学不好。

特点：高偏差 High Bias，低方差 Low Variance

原因：模型太简单，假设过于简化，无法捕捉数据内在模式。

例子：用一条直线去拟合带曲线趋势的数据

理想模型：Low Bias + Low Variance，既能学到真实规律，又不会被噪声干扰（$\theta_0 + \theta_1 x + \theta_2 x^2$）

10.2 Overfitting 过拟合

概念	课件原话	中文	什么时候出现
Overfitting	fit the training set very well, but fail to generalize to new examples (high variance)	训练集拟合得很好，但泛化失败	特征太多（too many features）/ 模型太复杂
Bias	error due to overly simplistic assumptions in the learning algorithm	由假设过于简单带来的误差	模型太简单 ⇒ 欠拟合 ⇒ high bias
Variance	error due to the model’s sensitivity to fluctuations in the training data	由对训练数据波动过于敏感带来的误差	模型太复杂 ⇒ 过拟合 ⇒ high variance

模型能力太强，把训练集里的噪声、随机波动也当成规律记住了（曲线）。

在训练集上效果极好，但是泛化能力差，在新的测试样本上表现很差 → 高方差 High Variance

原因：特征数量过多，模型复杂度过高，过度记住训练集的细节。

10.3 Model Complexity 复杂度曲线

训练误差与测试误差画成两条曲线，给出诊断规则。

② 复杂度曲线：训练误差与测试误差（第 72 页）· Underfitting and Overfitting

图中元素	内容
横轴	model complexity，分成 low / medium / high 三档
纵轴	prediction error (1 − prediction accuracy)
实线	Training data 训练误差
虚线	Test data 测试误差
标号 1 / 2 / 3	训练误差上的三个点，分别在 low / medium / high 复杂度
标号 4 / 5 / 6	测试误差上的三个点，分别在 low / medium / high 复杂度

两条曲线之间的间距就是泛化误差。

训练误差高 + 测试误差高 ⇒ 欠拟合（high bias）;

训练误差很低 + 测试误差回升 ⇒ 过拟合（high variance）;

测试误差最低处 ⇒ 好模型。

10.4 Ways to Address

欠拟合 = More data + More powerful model；加（加数据、加模型能力）

过拟合 = Feature Selection + Regularization。减或约束（减特征、压参数）

欠拟合

1. More data
2. A more powerful model. e.g., Neural Networks, rather than simple linear regression

过拟合

手段	做法	适用场景
Feature Selection（特征选择）	Manual selecting which features to keep (by domain knowledge) 手工挑，靠领域知识	Okay esp. when some features are useless.（有些特征完全没用时最合适）
Regularization（正则化）	Keep all features, but reduce their influence by giving smaller values to the parameter θ_i 保留全部特征，但把 θ_i 压小	Okay when many features, each of which contributes a bit to predicting y（特征很多、每个都贡献一点点时最合适）

5434 Big Data Computing 2

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_5/knowledge/5434-Big-Data-Computing-2.md`

5434 Big Data Computing2

Lecture 7 Graph Analytics

本讲是七讲里第一次处理「关系数据」：前几讲的数据都能排成一张表，本讲的信息藏在连接方式里。

七个部分的分工

1. Part 1 图与应用：六类真实网络→ 定义加权 / 有向两种分类 → 邻居与度数 → 路径 → 完全图 → 三种存储模型。这是语言与表示。
2. Part 2 图遍历：BFS与 DFS→ 两页伪代码 → 两张算例。这是所有全局算法的发动机。
3. Part 3 中心性（上）：度中心性（数邻居）→ 接近中心性（到所有人的距离之和取倒数，C(2) = 1/9）→ 算法要对每个顶点各跑一次 BFS。这是局部与几何两种「重要」。
4. Part 4 PageRank：全局重要性 → 随机游走与阻尼因子 d → 迭代更新公式 → 两轮手工迭代算例（n = 4、d = 0.8，结果写成/12 的分数）。这是被重要者指向才重要。
5. Part 5 最短距离与 Dijkstra：最优子结构观察 → 伪代码与松弛 → 五顶点六帧算例（δ(s,t) = 8）→ 归纳法正确性证明（可选）。这是加权图上的最短路。
6. Part 6 顶点相似度：链路预测问题 → Jaccard 相似度（共同邻居 ÷ 邻居并集）。这是用结构猜缺失的边。
7. Part 7 社区与聚类：连通分量（无向）、SCC 与 WCC（有向）、局部聚类系数（无向 1/3 / 有向 1/6）与平均聚类系数。这是把图切开看结构。

Part 1 Graphs and Applications

1.1 社交网络与用户/ 物品图 Social Networks / User-Item Graphs

两者的结构差别：边是不是跨类型的。

1. Social Networks：节点是实体，边是关系

节点（顶点）：users、images 等 all kinds of entities（各种实体）。

边：friendship、followship 等 all kinds of relationships（各种关系）。

任务	中文	在问什么
Community detection	社区发现	把图上抱团的节点群找出来（连通分量与聚类系数）
Link prediction	链路预测	预测两个节点之间将不会不会连上边（Jaccard 相似度就是它的工具）
Product/Ad advertisement	商品/广告投放	按图上的邻近关系决定推给谁
Node classification	节点分类	给节点打标签（例如机器人账号、高价值用户）

1.2 User-Item Graphs：二部图（Bipartite Graph）

两类：users（用户）与 items（物品），这是二部图的第一条特征。

边：ratings / purchasing / reviewing ，边只能连接两类之间的节点，这是二部图的第二条特征。

典型任务：Recommendation systems（推荐系统）。

二部图与普通图的关键差别

对比项	社交网络（普通图）	用户 - 物品图（二部图）
顶点类型	同一种（人）	两种（用户 / 物品）
边的方向性	通常是无向的（好友是互相的）	常有方向（谁买了谁）
边能不能出现在同类之间	可以（人 - 人）	不可以（只有用户 - 物品）
典型分析	社区发现、链路预测	推荐、二部图投影

二部图不是有两种节点的图这么简单，关键是边只存在于两类之间。若允许用户 - 用户边，它就不再是二部图。

考点 ： 看到 Bipartite 就要条件反射地想起两类顶点 + 边只跨类；看到 Recommendation systems 就要想到它是用户 - 物品图上的任务。

1.3 应用：Web / 交易 / 电信 / 航空网络

1. Web Graphs

这是有向图最经典的原型：PageRank（Part 3）就是在这个图上定义的。

顶点Vertices: webpages（网页）;

边 Edges: hyperlinks（超链接），且有向，A 链到 B 不代表 B 链到 A。

2. Transaction Networks

典型任务：欺诈检测（Fraud detection），异常的资金环、异常的中转链在图上比在一张流水表里明显得多。

顶点：账户 / 商户 / 支付平台。

边：一笔转账或支付。

3. Telecommunication Networks

典型任务：社区发现（把通话圈子切出来）、关键节点识别（去掉哪个节点会切断网络）。

顶点：电话 / 手机 / 基站。

边：一次通话或一条链路。

4. Airline Networks

典型任务：枢纽识别（Hub identification，中心性度量）、连通性分析（连通分量）。

顶点：机场（或城市）。

边：一条直飞航线。

四类网络的共同结构

网络	顶点 Vertices	边 Edges	有向?	本讲会用到的工具
Social Networks	users, images, etc	friendship, fellowship, etc	多为无向	连通分量、聚类系数
User-Item Graphs	users, items	ratings, purchasing, reviewing …	常有向	Jaccard 相似度（推荐）

网络	顶点 Vertices	边 Edges	有向?	本讲会用到的工具
Web Graphs	webpages	hyperlinks	有向	PageRank
Transaction Networks	账户 / 商户	一笔支付（Third-party payments）	有向	社区发现
Telecommunication Networks	电话 / 基站	通话 / 链路	可无向	社区发现、连通分量
Airline Networks	机场	直飞航线	多为无向	中心性、连通性

1.4 图的数学定义

Graph model G = (V, E)

V is a set of vertices; E is a set of edges; each edge is for a **pair** of vertices

G = (V, E) 图 = 顶点集合 + 边集合；V 是 vertex（顶点）的集合，E 是 edge（边）的集合。

边的定义：each edge is for a pair of vertices 每条边对应一对顶点。

Tips：定义里说的是 pair（一对），所以单独一个顶点不构成边；但单独一个顶点可以构成一个连通分量。

Example:

项	取值	读法
顶点集合 V	{1, 2, 3, 4}	4 个顶点，编号 1 - 4
边集合 E	{(1,2), (2,3), (3,4), (2,4)}	4 条边
顶点 1 的邻居	{2}	只连到 2
顶点 2 的邻居	{1, 3, 4}	度数为 3（本图度数最大的顶点）
顶点 3 的邻居	{2, 4}	度数为 2
顶点 4 的邻居	{2, 3}	度数为 2

1.4.1 Weighted vs Unweighted / Directed vs Undirected

有向 ≠ 加权，两者是正交的

类型	定义	例子
Unweighted graph（无权图）	An edge does not have any weight 边不带任何权重	Facebook friend relationship（好友关系）
Weighted graph（加权图）	An edge has a weight 边带权重	road segment: length（路段：长度）；communication link: bandwidth（通信链路：带宽）
Undirected graph（无向图）	An edge can be traveled in both ways 一条边两个方向都能走	Facebook friend relationship（加好友是双向的）
Directed graph（有向图）	An edge can be traveled in one way 一条边只能单向走	Twitter follower relationship（单向关注）

1.4.2 邻居与度数 Neighbors and Degree of a Vertex

度数（degree）是图上最基础的局部指标。无向图只有一个度数；有向图拆成入度与出度两个。

1. 无向图：邻居集合与度数

N(v) 表示顶点 v 的邻居集合。上图里 N(2) = {1, 3, 4}，所以顶点 2 的度数是 3。

等价说法：无向图中，度数 = 邻居个数 = 该顶点上的边数。

无向度数 = 入度 + 出度

2. 有向图：入邻居 / 出邻居，入度 / 出度

1.5 路径 Paths

A path is a finite or infinite sequence of edges which joins a sequence of vertices. In most definitions, edges and vertices are **distinct**.

简单路径（simple path）是一串边，把一串顶点依次接起来。路径的长度是边的条数（不是顶点个数）。边不重复、顶点也不重复。

对比项	无向图	有向图
可达是否对称	对称：a 能到 b ⇒ b 能到 a	不对称：4 → 1 可能不存在路径
最短路径	两端之间的最短路	从一个点到另一个点的最短路（方向固定）
连通性概念	连通分量 CC（Part 6.1）	强连通 SCC / 弱连通 WCC

无向图上的路径

1<->2 is a path with length 1；1<->2<->3 is a path with length 2；This is the shortest path between 1 and 3

结论：1<->2<->3 是顶点 1 与 3 之间的最短路径（the shortest path between 1 and 3）因为 1 与 3 之间没有直接相连的边，绕 2 是唯一的两步走法。

有向图上的路径

1->2->3->4 is a path with length 3；1->2->4 is a path with length 2

This is the shortest path from 1 to 4. There is no path from 4 to 1

1.6 完全图与稠密度: Complete Graph

完全图是稠密度的标尺：它给出任意 n 个顶点最多能有多少条边，于是实际边数 ÷ 完全图边数就成了密度的定义。

A graph in which each vertex is connected to every other vertex is called a complete graph.

Densest possible graph

How many edges are in an undirected complete graph with n vertices? —— n(n-1)/2

How many edges are in a directed complete graph with n nodes? —— n(n-1)

类型	边数公式	怎么来的
无向完全图	n(n−1)/2	每个顶点连其余 n−1 个 ⇒ 有向意义上的 n(n−1) 条有序对；无向边 (u,v) 与 (v,u) 是同一条，故除以 2

类型	边数公式	怎么来的
有向完全图	n(n−1)	每个顶点有 n−1 条出边，共 n 个顶点，且 (u→v) 与 (v→u) 是两条不同的边，不除 2

两个用途

Real-world graphs are rarely complete.

Complete graphs are usually used as substructures to decide the density of a graph or a community in a graph

现实中很罕见，真实网络通常是稀疏的，边数远小于 n(n−1)/2。

可以当作分母用，完全图是密度的参照系：图（或社区）的稠密度 = 实际边数 ÷ 完全图边数。

局部聚类系数的分母正是v 的邻居们两两相连最多能有几条边，也就是邻居集合上的完全图边数。

1.7 三种存储模型：Graph Models

四个顶点：V = {1, 2, 3, 4}。

四条边：(1,2) = 1.2、(1,3) = 4.3、(2,3) = 0.8、(3,4) = 2.6，注意这是加权无向图（邻接矩阵对称）。

度数：顶点 1 度 2、顶点 2 度 2、顶点 3 度 3、顶点 4 度 1（4 是唯一的叶子）。

三种存储模型

模型	形状	一句话
边表 Edge List	一张 (V1, V2, Weight) 的表，4 行	最省地方；要查某点的邻居必须扫全表
邻接矩阵 Adjacency Matrix	4 × 4 的方阵 A	查邻居 O(1)；但空间是O(n²)，稀疏图极浪费
邻接表 Adjacency Lists	每行一个顶点，后面跟 邻居(权重)	空间 O(n + m)；遍历邻居最常用

邻接矩阵（完整数值）

对称性：矩阵关于主对角线对称（A[i][j] = A[j][i]），是无向图（有向图一般不对称）。对角线全是 0，说明没有自环。

行和 = 度数（加权图中的强度）：第 1 行 0+1.2+4.3+0 = 5.5；第 2 行 0+1.2+0.8+0 = 2.0；第 3 行 0+4.3+0.8+2.6 = 7.7；第4 行 0+0+2.6+0 = 2.6。

所有行和相加 = 所有边权之和的 2 倍：5.5 + 2.0 + 7.7 + 2.6 = 17.8，而边权和 = 1.2 + 4.3 + 0.8 + 2.6 = 8.9，17.8 = 2 × 8.9

邻接表（完整数值）

顶点 1 的邻居是 2（权重 1.2）和 3（权重 4.3）。每个顶点只出现一次作为行首，所以共 4 行对应 4 个顶点。2 + 2 + 3 + 1 = 8 条，正好是无向边数 4 的 2 倍，因为每条无向边在两端各登记一次。

问题	边表	邻接矩阵	邻接表
查两点间是否有边	扫全表 O(m)	O(1)	扫该点邻居 O(deg)
列出某点的所有邻居	扫全表 O(m)	扫一行 O(n)	O(deg) 最优
空间	O(m) 最省	O(n²) 最费	O(n + m)
适合	存储 / 交换	稠密图、需要矩阵运算	稀疏图的算法（BFS/DFS/PageRank）

Part 2 Graph Traversal Methods：图遍历

起点：从源顶点 s（source vertex）出发。换一个 s，整个遍历的次序就全变了，遍历是关于 s 的，不是关于图的。

覆盖：要走到图中的每一个顶点，一个都不能漏。

次序：按某个一定的顺序访问。BFS 与 DFS 的差别只在这个顺序上，其余（起点、覆盖、只访问一次）两条完全一样。

不重复：每个点只访问一次（visited 一次）。前提：图里有环，不做标记就会绕圈不停。

对比项	BFS（广度优先）	DFS（深度优先）
推进策略	先近后远：把离 s 近的点全走完，才碰远一层的点	能深就深：沿当前顶点一直往下钻，钻不动了才回头
生活比喻	在水里投一颗石子，波纹一圈圈往外扩	走迷宫：一路往前，撞墙才退回来换岔路
课件原话	Visit the vertices level by level	Visit a vertex "deeper" in the graph whenever possible
被谁用到	无权图最短路、社交网络的分层扩散	连通性、环检测、拓扑排序、强连通分量

判断遍历序列题：给一张图与起点 s，要求写出 BFS 序列或 DFS 序列。BFS 的答案唯一（按层）；DFS 的答案依赖邻接表顺序，题目必须指定「按顶点编号从小到大」或给定邻接表，否则答案不唯一。

2.1 广度优先搜索 Breadth-First Search

给定图 $G = (V, E)$ 和源点 s

按距离分层遍历：先访问离 s 更近的所有顶点，再访问更远的顶点，一层一层向外扩展（level by level）

- 遍历轨迹会生成一棵 BFS 树（breadth-first tree），树是无环连通图
- 数据结构：FIFO 队列 Queue（先进先出）

Tips:

规则：离 s 近的顶点没走完，绝不碰离 s 远的顶点，**所以 BFS 拿到了最短路径**。

必须有一个起点 s，以及图 G = (V, E)。

访问序列天然是按层分块 (level by level) 的：第 0 层 {s}、第 1 层、第 2 层……

A tree is a connected graph containing no cycles.

两个条件：连通（任意两点之间都有路）；无环（不存在回到自己的回路）。

副产品：广度优先树，无环 + 连通 → 任取一个点当根，父子关系不会出现一个点有两个爸爸的情形。同一个原图，可以生成**不同的 BFS 树**（邻接点遍历顺序不同）。但无论哪棵树，各顶点的 d 值完全相同。

BFS 森林: 如果图不连通，从 s 出发走不到所有点。此时对每个未访问的顶点再起一次 BFS，得到的若干棵树组成森林。

顶点属性

$u.p$：parent，u 的父节点

$u.d$：源点 s 到 u 的距离（无权图：两点之间最少边的数量）未访问顶点初始距离： ∞（无穷大）

n 个顶点的树恰好有 n − 1 条边。因为每次发现一个新顶点，恰好新增一条树边，从 1 个点长到 n 个点。

用途	产出什么	成立条件
求最短路	从 s 到每个顶点的路径本身（沿 u.p 一路回溯到 s）	仅无权图（或边权全相等）
求最短距离	从 s 到每个顶点的边数 u.d	仅无权图；加权图要用 Dijkstra
检测环	遍历时若遇到一条边指向已访问且不是父亲的顶点，说明有环	无向图需额外排除回到父亲这条边
构造 BFS 树 / 森林	树 = 从单个 s 出发；森林 = 对每个未访问顶点各跑一次	任意图（有向无向皆可）

2.1.1 BFS Example：用队列一层层推

算法思想：

1. 源点距离设为 0，入队
2. 不断从队首取出节点u，遍历它所有邻接节点v
3. 如果v没有被访问（$v.d = \infty$）：更新距离、记录父节点，入队

性质：出队节点的距离一定是升序排列

矛盾点：边数少的那条（直达）反而贵一倍多。BFS 只看边数，看不见权重，于是给出错误答案。当所有边权相等（例如全为 1）时，边数最少与权重和最小重合，BFS 就是加权图上的特殊正确情形。

正确做法：加权图的单源最短路要用 Dijkstra 算法，它的松弛（relaxation）操作专门处理绕远路但更便宜的情形。

高频判断题：加权图上能否用 BFS 求最短路？不能。

复杂度： $O(V + E)$，每个顶点最多入队出队一次（$O(V)$）；每个顶点的邻接表恰好扫一次（$O(E)$）。合计 $O(V + E)$。前提是用邻接表存储，若用邻接矩阵则是 $O(V^2)$。

把队列换成栈会发生什么？答案：退化成 DFS。

2.2 深度优先搜索： Depth-First Search

DFS 与 BFS 的差别只是把队列换成递归栈，但行为完全不同：BFS 像水波扩散，DFS 像走迷宫，沿着一条路一直钻，钻到没新点可走才回头。

核心定义：给定源点 s，尽可能往深处走，走到底走不通再回溯（backtrack），像走迷宫。隐式生成一棵 DFS 树（depth-first tree）；实现方式是递归（本质是调用栈 LIFO 后进先出）

顶点属性

u.p：parent 父节点

u.depth：节点在 DFS 树里的深度；未访问顶点初始深度：∞

deeper = 离起点更远的树的层。只要当前顶点还有未被访问的邻居，就立刻钻过去，不先把同层邻居都走完。

implicitly（隐式）：BFS 的树是显式存的（每个点存 u.p 并连边画出来）；DFS 的树是递归调用栈自己形成的，算法本身不维护一棵树对象。

算法思想：

1. 初始化所有节点深度无穷，源点深度 = 0，调用访问函数
2. DFS-VISIT(u)：遍历 u 的所有邻接点 v
3. v 未访问，则更新深度、父节点，**递归访问 v**

4. 递归触底之后，执行回溯 backtrack

细节	BFS	DFS
初始化范围	$G.V - s$（除源点）	G.V（全部顶点，含源点）
源点赋值	$s.d \leftarrow 0; s.p \leftarrow NIL$（连父亲一起置空）	只写 s.depth ← 0（父亲在初始化那步已置 NIL）
新点处理	标记 d、记父亲、入队（稍后处理）	标记 depth、记父亲、立刻递归（马上处理）
判重字段	v.d = ∞	v.depth = ∞

2.2.1 DFS Example：递归

课件例子（起点 node 1）

遍历顺序：1, 2, 3, 4, 6, 5, 7 步骤：

1. 访问 1 → 访问 2 →访问 3 →访问 4（到底）
2. 回溯回到 3，回溯回到 2
3. 访问 2 的另一个邻点 6 →访问 5（到底）
4. 回溯回到 6，回溯回到 2，回溯回到 1
5. 访问 1 剩下邻点 7

同一个原图，可以生成**不同的 DFS 树**（邻接点遍历顺序不同）

2.3 BFS VS DFS

	BFS 广度优先	DFS 深度优先
核心思路	一层一层向外扩散，就近优先	一路往深处钻，走不动回溯
底层数据结构	队列 Queue（FIFO）	递归调用栈 Stack（LIFO）
适用场景	无权图最短路径，找最近节点	拓扑排序、找环、连通分量、迷宫搜索
遍历特点	按到起点距离从小到大访问	先一条路走到黑，再回头探索其他分支
树生成	BFS 树	DFS 树
树的形式	显式构造	隐式形成
层号字段	u.d（distance，等于最短路）	u.depth（只是树深）
实现要点	while 循环 + if v.d = ∞	for 循环 + if v.depth = ∞ + 递归调用

1. BFS **只适用于无权图最短路径**，带权图 BFS 失效
2. BFS、DFS 树都不唯一，取决于邻接表遍历邻节点的先后顺序
3. BFS 距离 = 源点到该点最少边数；DFS 的 depth 只是树里深度，**不是原图最短距离**
4. BFS：队列；DFS 递归版本靠系统栈，迭代版需要手动栈

Part 3 Centrality Measure

前置基础：BFS 用于无权图最短路径；Degree、Closeness 多用于无向图；PageRank 用于有向图（网页超链接）

核心目标：衡量网络中每个节点的重要程度（how central/important a vertex is）

三种度量

度量	特点	只看什么	计算代价
Degree Centrality	邻居数量多	局部：只看自己的度数	最便宜，读一遍边即可
Closeness Centrality	到其他所有顶点的距离之和小	全局几何：需要所有点对距离	贵：每个顶点各跑一次BFS
PageRank	被重要的顶点指向	全局迭代：靠迭代求解	较贵：多轮迭代直至收敛

3.1 度中心性：Degree Centrality

Degree centrality uses the vertex degree. Higher degrees means more central. It only considers local direct neighbors, ignoring global network structure.

> 节点的度 = 和该节点直接相连的边数量 Rule：The higher the degree, the more central the node is. 度越大，节点越重要、越处于网络中心。

核心思想：只看直接相邻的邻居（local 局部信息），不考虑整个网络的全局结构。

- 节点拥有越多直接相连的邻居 → 度越高 → 度中心性越高
- 在社交网络例子：一个人好友数量越多，度中心性越高

优点：计算最简单，开销极低，一眼就能算出；直观易懂。

缺点：只看局部，忽略全局信息 只统计直接相连节点，完全不考虑邻居本身是否重要，也不考虑节点到远处节点的距离。

> 例子：一个节点有 5 个普通邻居；另一个节点只有 2 个邻居，但这 2 个是网络超级核心节点。
>
> Degree Centrality 会判定前者更重要，但真实影响力后者更强。
>
> 无法区分边的方向（有向图要拆成入度中心性 in-degree / 出度中心性 out-degree）

适用场景

快速粗略筛选网络中连接多的节点；简单无向网络。比如，社交网络找关键人物、Web 找权威页面、传播场景找阻断谁最有效。能高效地把信息传遍全图的顶点，就是最中心的顶点。

度最高的顶点一定是最中心的吗？不一定。度中心性只数邻居，不看邻居是谁；PageRank 恰恰相反，看的是邻居的质量。反例：页面 E 的入链比 C 多得多，PageRank 却只有 C 的四分之一。

3.2 接近中心性：Closeness Centrality

Closeness centrality $C(v) = 1/\Sigma\, dist(v, u)$. We run BFS for each node to get shortest paths in unweighted graph. Nodes with smaller total distance have higher closeness. It is a global measure. It works only for connected unweighted graphs.

度中心性只看直接连着几个人；接近中心性看从我开始，到全图所有人，加起来要走多少条边，越小越中心。

$C(v)=\frac{1}{\sum_{u} dist(v,u)}$

- $dist(v,u)$：v 到 u 的最短路径距离（无权图：边的条数）
- $\Sigma_u dist(v,u)$：节点 v 到图中所有其他节点 u的最短路径距离之和（总距离）
- 总距离 $\Sigma dist(v,u)$ 越小 → $C(v)$ 越大
- High closeness centrality：这个节点离网络里所有其他节点都很近，信息可以快速传播到全网。

例题：vertex 2 到 1,3,4,5,6,7 的最短距离依次：1,1,2,2,1,2 Sum = 1+1+2+2+1+2 = 9 → $C(2) = 1/9$

顶点 v	到其余 6 个点的距离	距离之和	C(v)
1	1, 2, 3, 3, 2, 1	12	1/12 ≈ 0.0833
2	1, 1, 2, 2, 1, 2	9	1/9 ≈ 0.1111（最大）
3	2, 1, 1, 3, 2, 3	12	1/12 ≈ 0.0833
4	3, 2, 1, 4, 3, 4	17	1/17 ≈ 0.0588（最小）
5	3, 2, 3, 4, 1, 2	15	1/15 ≈ 0.0667
6	2, 1, 2, 3, 1, 1	10	1/10 = 0.1
7	1, 2, 3, 4, 2, 1	13	1/13 ≈ 0.0769

排序 C(2) = 1/9 > C(6) = 1/10 > C(1) = C(3) = 1/12 > C(7) = 1/13 > C(5) = 1/15 > C(4) = 1/17。

算法

Input: Unweighted graph G For each vertex v in graph:

1. Run BFS starting from v
2. Record shortest path distance $dist(v,u)$ from v to every other vertex u
3. Sum all distances, compute $C(v) = \frac{1}{\sum dist(v,u)}$ Return vertex v' with maximum $C(v')$

关键点：BFS 是用来计算无权图最短路径的工具，Closeness Centrality 依赖 BFS，对每个顶点各跑一次 BFS。对 n 个顶点各跑一次，总代价 O(V · (V + E))。稀疏图（E ≈ V）时约为 $O(V^2)$。而度中心性只要 O(V + E)，一次遍历就够。接近中心性贵了整整一个因子 V。

优点：全局指标：考虑节点到网络全部节点的距离，不只是直接邻居；

衡量信息传播速度：closeness 高的节点，消息传递到全网更快。

缺点：只适用于无权连通图。如果图不连通，部分节点之间没有路径，距离无穷大，公式失效；

计算开销更大：每个节点都要单独跑一遍 BFS。n 个节点，复杂度$O(n(n+m))$，大图很慢；

不区分邻居的重要性，只看距离长短。

问题：度中心性为什么不足以刻画重要性？只看数量、不看质量，且只覆盖一步邻域

适用场景：连通、无权无向网络；找网络里 “到所有人最近” 的枢纽节点（交通枢纽、信息中转站）

3.3 Google PageRank

PageRank is a global importance metric for directed graphs, originally for web search. It can be interpreted by random walk. With probability d, surfer follows out-links; with 1 − d, jumps to a random page. We iteratively update PR values. A page is important if many high-quality pages link to it. Sum of all PR = 1.

PageRank：有向图上的全局节点重要性指标，谷歌早期搜索引擎用来给网页排序。

Vertices = web pages；Directed edges = hyperlinks（超链接 A→B：网页 A 链接到网页 B）

度中心性与接近中心性都是在图上量距离，它们看不出邻居的质量。PageRank 的解决思路是把这件事做成一个递归，PageRank 量的是整个图范围内的重要性，而且这个重要性是被其他顶点投票投出来的。

核心思想

A webpage has high PageRank if:

1. Many other pages link to it（大量网页指向它）；AND/OR
2. The pages that link to it themselves have high PageRank value（指向它的网页本身是高权重权威页面）

课件例子：Page C has fewer incoming links than page E, but C’s PageRank is higher. Because the page linking to C has very high value. 不是单纯数入链数量，入链的质量 > 入链数量

3.3.1 Random Walk 随机游走模型

想象一个随机上网的人（random surfer /random walker）：

d = damping factor 阻尼因子，经典取值 $d = 0.85$，课件例题用$d = 0.8$

1. With probability d：顺着当前网页的超链接，随机选择一个出邻居页面跳转
2. With probability (1 − d)：不再点击链接，随机跳转到图中任意一个网页

$PR(v)$ = the probability that this random walker lands on page v 在稳态下，随机游走者停留在页面 v 的概率。

All PageRank values in a graph sum to 1。整张图所有节点 PR 之和 = 1。

阻尼因子1 − d作用： 解决悬挂节点（dangling node，没有出链接的页面），防止游走卡死；同时模拟用户不想继续点链接，新开网页的行为。

3.3.2 PageRank 迭代更新公式

$PR^t(v) = \frac{1-d}{n} + d \cdot \Big(\sum_{v'\in N_{in}(v)} \frac{PR^{t-1}(v')}{|N_{out}(v')|}\Big)$

- n：total number of nodes in graph 总节点数
- $N_{in}(v)$：in-neighbors of v，指向 v 的节点集合（入邻居）
- $|N_{out}(v')|$：out-degree of node v' ，节点v' 的出度（v' 向外链接多少个页面）
- $PR^{t-1}(v')$：上一轮（t-1 迭代）节点 v' 的 PageRank 值
- $\frac{1-d}{n}$：随机跳转项。1 − d这部分概率，平均分给所有 n 个节点

迭代完整流程

1. Initialization (Iteration 0)： 所有节点初始 PR 均等分配：$PR^0(v) = \frac1n$
2. Iteratively update PR： 不断用上面公式计算新的PR^t，每一轮用上一轮 t-1 的 PR 结果
3. Convergence 收敛判断： 重复迭代，直到所有节点 PR 值变化很小，基本稳定，停止迭代。
4. 性质：每一轮迭代，全部节点 PR 总和始终等于 1

例子（n=4，d=0.8）

Iteration0（初始化）：$PR(v_1)=PR(v_2)=PR(v_3)=PR(v_4)=\frac14$

Iteration 2：把 Iteration1 算出的$\frac{1.4}{12},\frac{2.6}{12},\frac{4.2}{12},\frac{3.8}{12}$作为旧值，再次代入公式，继续更新。反复迭代直到稳定。

5. 优点：有向图全局指标；不只看链接数量，重视链接来源的质量；

抗简单作弊：单纯大量垃圾页面链接你的页面，提升 PR 效果有限，因为垃圾页面自身 PR 很低。

6. 缺点：需要多次迭代计算，开销大；

是概率稳态值，依赖阻尼因子参数 d 的选择；

原始 PageRank 不考虑网页内容，只看链接结构。

3.4 三种测量总结

指标	图类型	核心信息来源	计算方法	核心思想
Degree Centrality 度中心性	无向图（有向图可拆入 / 出度）Local 局部	直接相连邻居数量	统计节点度数	连接越多邻居，越重要
Closeness Centrality 接近中心性	连通无权无向图 Global 全局	到所有节点最短路径总和	每个节点 BFS 求最短距离，公式 $C(v) = \Sigma 1/dist(v,u)$	离全网所有节点越近，越重要
PageRank	**有向图** Global 全局	入链数量 + 入链页面自身权重	随机游走模型，迭代更新公式，阻尼因子 d	被高质量页面链接，则节点重要

1. BFS 的定位：BFS 用来求无权图最短路径，是计算 Closeness Centrality 的底层算法；PageRank不使用BFS，靠迭代。
2. Closeness Centrality：图必须连通；不连通图，部分 dist 无穷大，无法计算。
3. PageRank 公式分母：$|N_{out}(v')|$ 是源节点 v' 的出度，不是目标节点 v 的度。
4. 阻尼因子d：0.85 是经典值；$1-d$是随机跳转概率，均匀分给全部节点。
5. PageRank 所有节点 PR 之和恒等于 1；初始化全部等于$1/n$。
6. 区分：Degree：数邻居多少；Closeness：算距离远近；PageRank：看谁链接，重视链接来源质量（有向）。

Part 4 Shortest Distance

加权图上 BFS 失效，靠**松弛**逐个把顶点的最短路确定下来；正确性的根基是最优子结构 + 非负权。

基础概念

Problem：给定图，求源点 source s到图中所有其他顶点的最短距离。 与接近中心性不同，那里要跑 n 次单源最短路，这里只跑一次。

Application：地图导航、路线规划。

$\delta(u, v)$：u 与 v 之间真实最短路径距离

前提假设：图所有边权重 NON-NEGATIVE（非负权重！Dijkstra 不能处理负权边）

4.1 Optimal Substucture 最优子结构

If vertex u lies on the shortest path from s to v, then the subpath $s \to \cdots \to u$ must be the shortest path from s to u. 最短路的最短路仍然是最短路。

推论：$\boldsymbol{\delta(s, u) \leqslant \delta(s, v)}$

距离单调递增：最短路上的点，沿着路径走只会越来越远（因为非负权重）。

记号	读法	含义
δ (u, v)	delta	真实的最短距离（true shortest distance）是想求但还不知道的那个真值
d(s, v)	d	算法当前维护的估计值（用 p.45 的伪代码）是个会不断变小的上界
w(u, p)	w	边的权重（weight）课件假设 w ⩾ 0（非负）

解题思路：先求解离源点更近节点的最短距离，利用已经算出的近点结果，求解更远节点。

Dijkstra 为什么可以按距离从小到大逐个确定顶点？因为最优子结构 + 非负权，所有更远的顶点的距离一定不小于当前顶点，所以当前最小的 d 不可能再被改进。

4.2 Dijkstra Algorithm 伪代码

Input: 带非负权图 $G(V, E)$，源点s

1. 初始化：对所有顶点 $v \in V$，$d(s, v) = \infty$；源点 $d(s, s) = 0$
2. 集合 Q：保存全部 $(v, d(s, v))$ 键值对（优先队列 / 最小堆，每次取出距离最小节点）
3. 集合 S：空集合，存放**已经确定最短距离**的节点
4. while $Q \neq \varnothing$:
    1. 从 Q 取出拥有最小$d(s, u)$的元组$(u, d(s, u))$，从 Q 删除
    2. 将u加入 S：$S = S \cup \{u\}$
    3. 遍历u的所有邻居p：if $d(s, p) > d(s, u) + w(u, p)$则$d(s, p) = d(s, u) + w(u, p)$，并更新p的距离值。

Relaxation（松弛操作）：尝试用经过 u 的新路径，更新 p 的最短距离。只有新路径更短才更新。

Q 的实现	取最小的代价	总复杂度
线性数组（扫一遍找最小）	O(V)	$O(V^2 + E)$
二叉堆 / 优先队列	O(log V)	O((V + E) log V)
斐波那契堆	O(1) 摊还	O(E + V log V)

4.2.1 Example

结论：对所有自然数 n 成立，算法正确。

操作轨迹：四个距离之和 8 + 5 + 9 + 7 = 29

帧	取出的顶点（Q 中最小）	本轮发生的松弛	取出后的 d 值（s,t,x,y,z）
(a)	—（初始）	—	0, ∞, ∞, ∞, ∞
(b)	s（0）	s→t: ∞ → 10；s→y: ∞ → 5	0, 10, ∞, 5, ∞
(c)	y（5）	y→t: 10 → 8；y→x: ∞ → 14；y→z: ∞ → 7	0, 8, 14, 5, 7
(d)	z（7）	z→x: 14 → 13；z→s: 0 < 14，不改	0, 8, 13, 5, 7
(e)	t（8）	t→x: 13 → 9；t→y: 5 < 10，不改	0, 8, 9, 5, 7
(f)	x（9）	x→z: 7 < 13，不改；Q 空 → 结束	0, 8, 9, 5, 7

对比：BFS vs Dijkstra

BFS 无权图最短路径，边权重 = 1，队列，不处理权重。是加权最短路在等权特例下的退化形式。BFS 只用于无权图（边数最小）；

Dijkstra 非负加权图单源最短路径，最小优先队列，处理任意⩾0 边权。若把所有边权都设成 1，结果与 BFS 完全一致，是 BFS 的加权推广。Dijkstra 用于边权非负的加权图（权重和最小）

> Dijkstra 限制：不能存在负权重边；负权图要用 Bellman-Ford

4.2.2 Dijkstra 正确性证明

命题：算法结束时，对所有顶点v，$d(s, v) = \delta(s, v)$（预估距离 = 真实最短距离）

步骤：按取出次序做归纳；基例是源点；归纳步在最短路里取一条S → Q的跨界边(x, y)；用 δ(s,y) ⩽ δ(s,v) ⩽ d(s,v) ⩽ d(s,y) 夹逼，得到 d(s,v) = δ(s,v)。

基础 Base case：第 1 个取出节点是源点s，$d(s, s) = 0 = \delta(s, s)$，成立。

归纳假设 Induction hypothesis：前$n-1$个从 Q 移除的节点，都满足$d = \delta$

归纳步骤 Inductive step：第n个取出节点v，证明$d(s, v) = \delta(s, v)$

1. 松弛操作只会**减小**$d(s, v)$；**一旦**$d(s, v) = \delta(s, v)$，值不会再改变。
2. 当节点v被从 Q 取出的时候，它的$d(s, v)$已经等于真实最短距离$\delta(s, v)$，不会有更短路径。
3. 利用最优子结构：最短路径上的子路径本身也是最短路径。

步	内容	原理
归纳起点	n = 1 时取出的是源点 s，d(s,s) = δ(s,s) = 0，成立	初始化的定义
归纳假设	假设前 n − 1 个被取出的顶点都满足 d = δ	归纳法
找一条跨界的边	在第 n 个顶点 v 的真实最短路上，取第一条起点在 S、终点在 Q的边(x, y)	S 非空（至少含 s）、Q 含 v，所以这样的跨界边一定存在
夹逼（三个不等式）	由观察得 δ(s,y) ⩽ δ(s,v)；由v 是Q 中最小得 d(s,v) ⩽ d(s,y)；由第一部分得 δ(s,v) ⩽ d(s,v)	最优子结构 + 贪心选择 + 单调性

步	内容	原理
夹逼（收口）	三式串成 $\delta(s,y) \leqslant \delta(s,v) \leqslant d(s,v) \leqslant d(s,y)$；由归纳假设 $d(s,y) = \delta(s,y)$，两端相等 → 中间全部相等 → $\delta(s,v) = d(s,v)$	归纳假设把链条锁死

Tips:

跨界的边：要证明 d(s,v) 已经降到 $\delta(s,v)$，得说明在 v 之前一定已经有一条路径把正确的值送过来了。这条路径就是 s → ⋯ → x → y → ⋯ → v，其中 x 已在 S（已确定）、y 还在 Q。

夹逼：把不等式链条把四个量串在一起，两端相等就逼出中间全部相等。

$d(s,y) = \delta(s,x) + w(x,y) = \delta(s,y)$ ： x 被取出时（由归纳假设 $d(s,x) = \delta(s,x)$）它一定松弛过自己的邻居 y，把 y 的估计值设成了 $\delta(s,x) + w(x,y)$；而这条路径本身是最短路的一段，所以这个值就是 $\delta(s,y)$。即最短路的一部分必须是最短路。

Part 5 Vertex Similarity/ Proximity Measures 链路预测

5.1 链路预测 Link Prediction

目标：预测两个节点v_1, v_4之间未来会不会产生边，或者评估两个节点之间存在边的可能性。

核心思路：计算节点之间的相似度 similarity / 邻近度 proximity。

5.2 雅卡尔相似度 Jaccard Similarity （无向图）

又称定点相似度：共同邻居 ÷ 邻居并集，一个 0 到 1 的数，用来预测这条边该不该存在。

链路预测为什么可以用相似度？因为真实网络有同质性（homophily），相似的人更容易产生联系，所以邻居重合度高的一对顶点未来连边的可能性更大。但相似度只是启发式，不是必然，最终要看预测任务的评价指标。

$Jaccard(v_1,v_4)=\frac{|N(v_1)\cap N(v_4)|}{|N(v_1)\cup N(v_4)|}$

- $N(v)$：节点v的一阶邻居集合
- 分子：两个节点公共邻居的数量（交集大小）
- 分母：两个节点所有邻居总数（并集大小）
- 含义：Jaccard 越高，两个节点共享越多共同邻居，越有可能产生新边。常用于社交网络好友推荐。

例子：v1 邻居 {v2,v3}, v4 邻居 {v2,v5} 交集$N(v1) \cap N(v4) = \{v2\}$，并集$\{v2, v3, v5\}$ $Jaccard = \frac13$

顶点对	交集	并集	Jaccard	是否已有边
(v_1, v_2)	$\{v_3\}$ → 1	$\{v_1,v_2,v_3,v_4\}$ → 4	1/4 = 0.25	已有
(v_1, v_3)	$\{v_2\}$ → 1	$\{v_1,v_2,v_3,v_4\}$ → 4	1/4 = 0.25	已有
(v_1, v_4)	$\{v_2, v_3\}$ → 2	$\{v_2, v_3\}$ → 2	1（最大）	没有 → 预测它就对了
(v_2, v_3)	$\{v_1, v_4\}$ → 2	$\{v_1,v_2,v_3,v_4\}$ → 4	1/2 = 0.5	已有
(v_2, v_4)	$\{v_3\}$ → 1	$\{v_1,v_2,v_3,v_4\}$ → 4	1/4 = 0.25	已有
(v_3, v_4)	$\{v_2\}$ → 1	$\{v_1,v_2,v_3,v_4\}$ → 4	1/4 = 0.25	已有

Part 6 Community and Clustering Concepts：社区与聚类

6.1 Connected Components 连通分量

6.1.1 Undirected Graph 无向图：Connected Components (CC)

定义： A connected component (CC) is a maximal subgraph in which each pair of nodes is connected via paths. 连通分量 CC 是极大连通子图：子图内部任意两点之间都存在路径；不能再加入更多节点而保持连通。

- 单个孤立节点本身也可以是一个连通分量。

寻找无向图连通分量算法（BFS/DFS）

Steps:

1. 全部顶点初始标记为未访问 not visited。
2. 重复直到所有节点都被访问：挑选一个未访问节点v，从v开始运行 DFS 或者 BFS 遍历；遍历过程中把所有可达节点标记为 visited；本次 DFS/BFS 遍历结束，得到一个连通分量。

为什么 DFS 和 BFS 都可以？两者都能从一点出发走到所有可达点，而 CC 只关心可达不可达，不关心访问次序。

外层循环跑几次？跑的次数正好等于 CC 的个数。从几个起点出发过，就知道图被切成了几块。

复杂度：合计O(V + E)，与一次遍历同阶。每次遍历把一批顶点标记一次，所有顶点总共只被访问一次、所有边也只被检查一次（无向图每条边检查两次）。

6.1.2 Directed Graph 有向图：SCC & WCC

1. Strongly Connected Components (SCC) 强连通分量

A strongly connected component (SCC) of a directed graph is a maximal strongly connected subgraph. Within SCC, there exists a path in both directions between every pair of vertices.

强连通分量 SCC：有向图的极大强连通子图。分量内部任意一对节点u, v，同时存在 u→v 和 v→u 双向路径。

- 单个孤立节点自身也可以是 SCC。

课件例题 SCC 结果：$\{1, 2\}, \{3\}, \{4, 5, 8\}, \{6\}, \{7\}$

1 和 2 互相可达，构成 SCC；3 只能单向进出，自己单独 SCC；4,5,8 互相可达。

6 只能被 3 指向，无法返回；7 指向 6，无法回来，各自独立 SCC。

2. Weakly Connected Components (WCC) 弱连通分量

Treat directed graph as undirected graph (ignore edge directions). The connected components of this undirected graph = WCC of original directed graph.

把有向图忽略边方向，当成无向图看待，得到的无向连通分量就是原图的弱连通分量 WCC。

Example

例题 WCC 结果：$\{1, 2, 3, 6, 7\}, \{4, 5, 8\}$ 只要不看箭头，节点之间能连通，就属于同一个 WCC，不要求双向路径。

对比项	SCC（强连通）	WCC（弱连通）
用什么图算	保持有向	先转成无向
判据	任意一对顶点双向可达	任意一对顶点至少单向可达
要求强度	强	弱
数量关系	个数 ⩾ WCC 个数	个数 ⩽ SCC 个数
典型用途	找封闭的圈层（如互相转发的朋友圈）	找同一批人/同一个话题（不关心方向）

SCC 是否一定是 WCC？是（前提是它作为子图还有意义）；

怎么求 WCC？把有向边当无向边，求连通分量即可。

6.2 Local Clustering Coefficient 局部聚类系数 & 平均聚类系数

衡量网络三角闭合程度（朋友之间互相认识的程度，社交网络经典指标）

6.2.1 Undirected graph 无向图

Local Clustering Coefficient C_v

$C_v=\frac{\text{number of edges between neighbors of }v}{\text{max possible edges among neighbors of }v} =\frac{\text{actual edges between neighbors}}{\frac{|N(v)|(|N(v)|-1)}{2}}$

分子：节点v的邻居们之间真实存在的边数量；

分母：v的邻居集合如果构成完全图，理论上最多能拥有多少边（组合公式 $C_k^2=\frac{k(k-1)}{2}$，k 是邻居数量$|N(v)|$）

课件例子：$C_2=\frac{2\times2}{4\times3}=\frac13$

顶点	分子（邻居间的边数）	分母 k(k−1)/2	C_v
v_1	v_2 - v_3 → 1	2 × 1 / 2 = 1	1（邻居连着）
v_2	v_1 - v_3、v_4 - v_5 → 2	4 × 3 / 2 = 6	1/3 ≈ 0.333（课件给的）
v_3	v_1 - v_2 → 1	2 × 1 / 2 = 1	1
v_4	v_2 - v_5 → 1	2 × 1 / 2 = 1	1
v_5	v_2 - v_4 → 1	2 × 1 / 2 = 1	1

含义：$C_v \in [0,1]$

$C_v = 1$：v 的所有邻居两两之间全部相连，邻居构成完全图；

$C_v = 0$：v 的邻居之间没有任何边，没有三角结构。

6.2.2 Directed graph 有向图 Local Clustering Coefficient

邻居集合取入邻居∪出邻居 $N_{in}(v) \cup N_{out}(v)$

$C_v=\frac{\text{edges between nodes in }N_{in}(v)\cup N_{out}(v)}{|N_{in}(v)\cup N_{out}(v)|\cdot(|N_{in}(v)\cup N_{out}(v)|-1)}$

课件例子：$C_2=\frac{2}{4\times3}=\frac16$

顶点	k	分子（并集内的有向边）	分母 k(k−1)	C_v
v_1	2	$v_3 \to v_2 \to 1$	2 × 1 = 2	1/2
v_2	4	$v_3 \to v_1$、$v_4 \to v_5 \to 2$	4 × 3 = 12	1/6（课件给的）
v_3	2	$v_1 \to v_2 \to 1$	2 × 1 = 2	1/2
v_4	2	$v_2 \to v_5 \to 1$	2 × 1 = 2	1/2
v_5	2	$v_2 \to v_4 \to 1$	2 × 1 = 2	1/2

6.2.3 Graph Average Clustering Coefficient 图平均聚类系数

整张图的平均聚类系数 = 图里所有节点的局部聚类系数的算术平均值

$C_{avg}=\frac1n\sum_{v\in V}C_v$

用来描述整个网络的整体聚集特性：高平均聚类系数代表网络有大量三角形、小团体结构（例如社交网络）。

Lecture 8 Big Data Storage 大数据存储

Part 1 数据类型与存储系统选择 Types of Data

数据的组织结构与更新频率直接影响存储系统的设计。课件从两个相互独立的维度对数据进行分类：结构化程度（Structured / Unstructured）与变化频率（Dynamic / Static），并通过四象限模型说明不同数据类型的存储需求。

1.1 按结构化程度分类

类型	定义与特征	典型示例
Structured Data（结构化数据）	具有预定义的数据模型，便于存储、处理、检索和管理，通常以表格形式组织	关系型数据（Relational Data）
Unstructured Data（非结构化数据）	不具有便于直接查询和处理的统一数据模型，通常需要额外解析或处理	文本、音频、视频等原始文件

非结构化数据并非完全没有结构。 例如，音频文件仍然具有编码格式及相关元数据。两类数据的核心区别在于其组织形式是否便于直接检索与处理，而非是否存在任何内部结构。

1.2 按变化频率分类

类型	定义	典型示例
Dynamic Data（动态数据）	内容更新相对频繁	办公文档、金融数据库中的交易记录
Static Data（静态数据）	内容相对稳定，更新频率较低	MRI、CT 医学影像

动态与静态描述的是数据内容的变化频率，而非访问频率。静态数据也可能被频繁读取。

1.3 数据分类四象限 Data Classification Quadrants

Unstructured + Dynamic 动态非结构化，例如：媒体制作、eCAD、mCAD、办公文档

Unstructured + Static 静态非结构化，例如：媒体归档、广播内容、医学影像

Structured + Dynamic 动态结构化，例如：交易系统、ERP、CRM

Structured + Static 静态结构化，例如：BI（商业智能）、Data Warehousing（数据仓库）

横向：Dynamic → Static；纵向：Unstructured → Structured

分类的主要目的是辅助选择合适的存储方案（Storage Solution）：

- 关系型数据库（Relational Databases）：通常用于结构化数据，特别是需要事务管理的业务系统。
- 文件系统或 NoSQL 数据库：可用于静态非结构化数据的存储。

Part 2 关系型数据库与 ACID 事务

2.1 事务 Transaction

RDBMS（Relational Database Management System） 即关系数据库管理系统。

事务是数据库执行操作的逻辑单元，可以包含多个底层操作（Low-level Tasks），用于将相关的数据修改作为一个整体进行管理。例如，账户 A 向账户 B 转账金额 x： $\begin{aligned} A'&=A-x\ B'&=B+x \end{aligned}$，转账完成后应满足： $A'+B'=A+B $。

虽然转账涉及多个数据库操作，但这些操作必须共同满足业务约束。因此，RDBMS 通过 ACID 属性保障事务的正确执行。

2.2 ACID 四大属性

属性	定义	核心要求
Atomicity（原子性）	事务中的所有操作被视为不可分割的整体	全部成功或全部失败
Consistency（一致性）	事务开始和结束时，数据库均满足完整性约束	保持数据合法性
Isolation（隔离性）	并发事务不能访问其他事务未完成的中间状态	避免不当的并发干扰
Durability（持久性）	事务一旦提交，其修改必须持久保存	系统故障后仍保留已提交数据

Atomicity： 若转账过程中扣款成功但入账失败，整个事务必须回滚，不能保留部分修改。

Consistency： 转账前后应保持资金总额不变，且数据库约束仍然成立。

Isolation： 一个事务不应读取其他事务尚未提交的中间修改。具体并发行为由数据库隔离级别决定。

Durability： 已提交的事务在系统故障后仍应能够恢复。

保障机制包括：Transaction Logs（事务日志）、Backups（备份）

需要区分：ACID 的 Consistency 指事务对数据完整性约束的维护，不等同于后续 CAP 定理中的分布式副本一致性。

Part 3 数据库横向扩展 Scaling Out Databases

随着数据规模与访问需求增长，单台数据库服务器可能受到存储容量、计算资源和吞吐能力的限制。

数据库扩展主要有两种方式：

- Vertical Scaling（纵向扩展）：提高单台服务器的 CPU、内存或存储性能。
- Horizontal Scaling（横向扩展）：增加服务器数量，通过多台机器并行处理工作负载。

重点讨论横向扩展，主要技术包括 Sharding（数据分片） 和 Replication（数据复制）。

3.1 Amdahl 定律：并行加速的理论上限

Amdahl's Law 用于描述程序中不可并行部分对整体加速效果的限制。

设：

- T_1：程序串行执行时间。
- T_p：使用 p 个处理器并行执行的时间。
- s：程序中不可并行部分所占比例。
- $1 - s$：程序中可并行部分所占比例。
- p：处理器或机器数量。

在理想情况下，不可并行部分耗时保持不变，可并行部分由 p 个处理器均匀分担：

$T_p=sT_1+\frac{(1-s)T_1}{p} $

因此，加速比（Speedup）为：

${ S(p)=\frac{T_1}{T_p} =\frac{1}{s+\frac{1-s}{p}} } $

课件算例：80% 的程序可以并行，使用 4 个处理器。

已知：$s = 0.2, 1 - s = 0.8, p = 4$

代入公式：$S(4)=\frac{1}{0.2+\frac{0.8}{4}} =\frac{1}{0.4} ={2.5} $

因此，4 个处理器只能实现理想情况下的 2.5 倍加速，而非 4 倍。

若串行执行总耗时为 100 个时间单位，则：

$T_1=20+80=100 $

$T_4=20+\frac{80}{4}=40 $

$S(4)=\frac{100}{40}=2.5 $

进一步，当处理器数量趋于无穷大时：

${ \lim_{p\to\infty}S(p)=\frac{1}{s} } $

本例的理论最大加速比为：

$S_{\max}=\frac{1}{0.2}=5$

结论：不可并行部分决定整体加速比的上限，增加处理器数量无法消除串行执行的限制。

3.2 理想并行与实际并行

Amdahl 定律建立在理想条件下，假设任务能够均匀分配，且忽略通信与调度开销。

实际并行系统还受到以下因素影响：

限制因素	含义	影响
Communication Overhead（通信开销）	处理器或服务器之间的数据传输、同步与协调	增加执行时间
Workload Imbalance（负载不均衡）	不同处理器承担的任务量不同	整体完成时间受最慢任务限制

对于必须等待所有并行任务完成的程序，可将实际执行时间近似表示为：

$T_p^{\text{actual}} \approx sT_1+\frac{(1-s)T_1}{p} +T_{\text{communication}} +T_{\text{imbalance}} $

其中后两项是对课件理想模型的补充，分别表示额外通信成本与负载不均衡造成的延迟。

因此，实际加速比通常低于理想加速比。

提高并行效率需要遵循三项原则：

1. Maximize Parallelizable Fraction：提高可并行部分比例，减小 s。
2. Balance Workload：均衡各处理器的工作负载，减少等待。
3. Minimize Communication Time：降低节点之间的数据传输与同步开销。

3.3 数据分片 Data Sharding

数据分片是将大型数据集划分为多个部分（Shards / Chunks），并分布到不同机器上，以支持并发和并行访问。

课件示例将一个大型文件划分为六个数据块：

由于 Chunk 2、Chunk 4 和 Chunk 6 位于不同机器，因此可以同时访问。

分片的主要作用包括：

- 将数据存储需求分散到多台机器。
- 提高并行读取和处理能力。
- 在适当的分片策略下，提高整体吞吐量。

分片的主要挑战包括跨分片查询、数据分布不均与跨节点事务协调。

3.4 数据复制 Data Replication

数据复制是将相同数据保存到多个服务器上，形成多个副本（Replicas）。

三项作用：

- Avoid Performance Bottlenecks：将访问请求分散到不同服务器，缓解性能瓶颈。
- Avoid Single Points of Failure：单个服务器故障时，可由其他副本继续提供服务。
- Enhance Scalability and Availability：提高系统的可扩展性与可用性。

分片与复制的核心区别如下：

对比	Sharding	Replication
基本操作	拆分数据	复制数据
节点数据	不同节点保存不同部分	多个节点保存相同数据
主要目的	扩展容量、支持并行处理	提高容错能力、可用性与读吞吐量

对比	Sharding	Replication
主要挑战	跨分片访问与负载均衡	副本同步与一致性

两种技术可以结合使用：将数据划分为多个分片，并为每个分片建立多个副本。

Part 4 分布式一致性与两阶段提交 2PC

4.1 数据复制带来的一致性问题

当同一份数据分布在多个服务器上时，数据修改必须在相关副本之间协调。

例如，电商系统将银行数据库复制到两台服务器。如果某次账户余额更新只在一台服务器上完成，而另一台尚未同步，则不同服务器可能返回不同余额。

因此，数据复制虽然提高了可用性与扩展性，但也引入了 Consistency Challenge（一致性挑战）。

Two-Phase Commit Protocol（两阶段提交协议，2PC） 说明如何协调分布式事务的提交或中止，以维护原子性和一致性。

4.2 两阶段提交协议 Two-Phase Commit Protocol

2PC 包含两类角色：

- Coordinator（协调者）：组织投票并决定全局事务的提交或中止。
- Participants（参与者）：执行本地事务操作，并根据协调者的最终决定提交或回滚。

协议分为两个阶段。

Phase I: Voting / Prepare（投票或准备阶段）

Coordinator: 发送 VOTE_REQUEST

Participant 1: 准备事务并投票

Participant 2: 准备事务并投票

Participant 3: 准备事务并投票

返回 VOTE_COMMIT 或 VOTE_ABORT

执行过程：

1. 协调者向所有参与者发送 `VOTE_REQUEST`。
2. 各参与者执行可撤销的本地事务操作，并检查是否能够提交。
3. 能够提交的参与者返回 `VOTE_COMMIT`；否则返回中止投票。
4. 已准备提交的参与者需要保留必要的事务状态与锁，等待第二阶段的最终决定。

全局决策规则为： $\text{Decision}= \begin{cases} \text{Commit}, & \text{所有参与者均投赞成票}\\ \text{Abort}, & \text{否则} \end{cases} $

Phase II：Commit / Abort（提交或中止阶段）

若所有参与者均同意提交：

1. 协调者发送 `GLOBAL_COMMIT`。
2. 参与者执行 `LOCAL_COMMIT`，正式提交本地修改。
3. 参与者释放事务锁，并完成相应的提交确认。

若存在参与者无法提交：

1. 协调者决定全局中止（Global Abort）。
2. 参与者回滚尚未提交的本地事务。
3. 释放相关锁与资源。

需要区分两个阶段的操作：

第一阶段	第二阶段
准备并检查事务能否提交	执行全局提交或中止决定
修改仍可撤销	提交后成为正式持久化结果
参与者可能持有锁	事务完成后释放锁
不产生最终提交决定	完成最终事务决策

4.3 2PC 的局限性

2PC 通过协调多个参与者的事务结果，实现跨节点的原子提交，但需要额外的通信与等待。

其主要代价包括：

- 锁持有时间增加：参与者在准备阶段完成后，通常必须等待全局决定才能释放相关锁。
- 通信开销增加：协调者需要与多个参与者交换消息。
- 阻塞风险（Blocking）：协调者在关键阶段发生故障时，已准备的参与者可能无法独立决定提交还是回滚。
- 可扩展性受限：参与者数量增加时，协调成本和资源占用可能上升。

因此，严格一致性（Strict Consistency）的实现可能限制系统的可扩展性（Scalability）。

2PC 主要保障分布式事务的原子提交，并不意味着仅凭 2PC 就能解决所有副本一致性或网络分区问题。

4.4 小结

本部分内容围绕数据库从单机存储向分布式存储扩展时面临的问题展开，其核心逻辑为：

1. Data Classification 根据数据结构和变化频率选择存储系统
2. RDBMS & ACID 通过事务保证数据库操作的正确性
3. Horizontal Scaling 通过并行处理、分片和复制扩展系统能力
4. Consistency Challenge 分布式数据更新引入跨节点协调问题
5. Two-Phase Commit 通过协调提交保障分布式事务原子性，但增加通信与阻塞成本

概念对照

概念	主要解决的问题
Data Classification	如何根据数据特征选择存储方案
ACID	如何保障事务的正确性与可靠性
Amdahl's Law	并行处理的理论加速上限
Sharding	如何分散数据和处理负载
Replication	如何提高可用性和容错能力
2PC	如何协调多个节点共同提交或中止事务

结论： 横向扩展通过增加机器提高系统的存储与处理能力，但分片和复制使数据管理从单节点操作转变为跨节点协调。为了维护分布式事务的原子性与一致性，系统必须承担额外的通信、同步与等待成本。这也为后续 CAP Theorem、BASE Properties 和 NoSQL 数据库的讨论建立了基础。

Part 5 CAP 定理与 BASE 属性 CAP Theorem & BASE Properties

随着数据库从单机架构扩展到分布式架构，数据复制与跨节点通信使一致性问题更加复杂。CAP 定理揭示了网络分区发生时，一致性与可用性之间的根本权衡；BASE 则提供了一种适用于部分高可用分布式系统的宽松一致性设计思路。

5.1 CAP 定理 CAP Theorem

CAP 定理指出：当分布式系统发生网络分区时，无法同时保证强一致性（Consistency）和可用性（Availability）。

CAP 包含三个属性：

属性	定义	核心要求
C — Consistency（一致性）	每次读取都返回最近一次成功写入的数据，或者返回错误	不返回过期数据
A — Availability（可用性）	每个发送到正常运行节点的请求最终都获得非错误响应	请求能够得到服务
P — Partition Tolerance（分区容错性）	节点之间发生网络通信故障时，系统仍能继续运行	容忍网络分区

CAP 中的 Consistency 与 ACID 中的 Consistency 不同：

- ACID Consistency：事务执行前后满足数据库完整性约束。
- CAP Consistency：强调分布式读写操作的单副本一致性，通常用线性一致性（Linearizability）描述。

CAP 的关键并非一般情况下只能拥有三个属性中的两个，而是网络分区发生时，必须在 C 和 A 之间作出选择。

5.2 网络分区下的 CP 与 AP

假设两个节点 N_1 和 N_2 保存同一数据的副本，网络故障导致两者无法通信。

此时，如果客户端向其中一个节点发起更新操作，系统面临两种策略：

策略	CP（Consistency + Partition Tolerance）	AP（Availability + Partition Tolerance）
核心目标	优先保证一致性	优先保证可用性

策略	CP（Consistency + Partition Tolerance）	AP（Availability + Partition Tolerance）
分区期间	拒绝或延迟无法安全执行的操作	尽可能继续响应请求
主要代价	部分请求不可用	可能读取旧数据或出现副本差异
适用倾向	对数据正确性要求较高的业务	对持续服务要求较高的业务

CP： 当无法确认其他副本的状态时，系统可能拒绝操作，以避免违反一致性要求。

AP： 系统允许分区两侧继续处理请求，但可能产生暂时不一致，需要在网络恢复后进行同步或冲突解决。

需要注意，AP 并不意味着所有请求必然成功，也不意味着系统完全放弃一致性；CP 同样不意味着所有操作都不可用。

5.3 一致性权衡

Google、Amazon 等企业在构建大规模数据库时，需要特别重视 24/7 Availability（全天候可用性），因为短时间停机也可能造成显著的业务损失。当数据库横向扩展至数千台服务器时，节点故障和网络故障的可能性增加。因此，系统必须考虑如何在网络分区下维持服务。

部分大规模系统选择优先保障可用性和分区容错性，并放宽严格一致性要求。

这种设计体现了以下权衡： $\text{Consistency Strictness} \quad\longleftrightarrow\quad \text{Availability / Scalability} $

一致性要求越严格，跨节点同步与协调成本通常越高；适当放宽一致性要求，则可能减少协调开销并改善可用性与扩展能力。Good-enough Consistency（足够的一致性）：一致性级别应根据应用需求确定，而非所有系统都采用最严格的一致性模型。

例如，银行账户余额通常需要较强的一致性保障，而社交平台的点赞数量可以在一定时间内允许副本存在差异。

这并非严格一致性必然低效，而是分布式环境中的一致性保障通常需要额外的通信和协调。

5.4 从 ACID 到 BASE

传统关系型数据库通常强调 ACID 事务，而部分面向大规模分布式应用的 NoSQL 系统采用 BASE 思路，以更宽松的一致性要求换取可用性与扩展能力。

BASE 可视为在一致性要求上更加宽松的设计思想，但并不是 ACID 的直接替代品。

BASE 属性	含义	说明
Basically Available（基本可用）	系统在大多数情况下能够提供服务	允许部分功能降级或响应延迟
Soft State（软状态）	数据状态可能在没有新的外部输入时发生变化	副本同步等内部过程会改变系统状态
Eventually Consistent（最终一致性）	在停止更新且同步能够持续完成的条件下，各副本最终收敛到一致状态	允许暂时不一致

最终一致性可以用以下形式表示：

设 $R_i(t)$ 表示第 i 个副本在时刻 t 的数据状态。如果系统停止接收新写入，且更新能够可靠传播，则：

$\exists T, \forall t \geqslant T: R_1(t) = R_2(t) = \cdots = R_n(t)$，该表达式表示在满足相应条件时，所有副本最终达到相同状态。

5.5 ACID 与 BASE 对比

维度	ACID	BASE
设计重点	事务正确性与可靠性	可用性与宽松一致性
一致性要求	保证事务维护完整性约束	允许副本暂时不一致
状态变化	强调事务边界及其约束	允许内部异步同步引起状态变化
典型应用	金融交易、订单事务	社交互动、内容分发、部分分布式存储系统

ACID 与 BASE 并非完全互斥。现代分布式数据库可以同时支持 ACID 事务、复制与不同级别的一致性保证。

本部分逻辑：网络分区使 C 与 A 的冲突不可避免，而 BASE 为部分优先可用性的系统提供了最终一致性的设计方向。

Part 6 NoSQL 数据库概述 Introduction to NoSQL

6.1 NoSQL 的产生背景与定义

传统关系型数据库（RDBMS）长期是企业业务系统的主流存储方案。然而，Web 应用快速发展，特别是 Facebook、X 等社交平台带来的数据量和访问量激增，对数据库提出了新的要求：

- 海量数据存储与高并发访问。
- 跨多台服务器的水平扩展。
- 对半结构化及非结构化数据的灵活支持。
- 在分布式环境下维持较高的可用性。

传统 RDBMS 的事务协调、固定模式以及部分跨节点操作可能增加大规模分布式部署的复杂度，由此推动了 NoSQL 数据库的发展。

NoSQL（Not Only SQL） 表示“不仅仅是 SQL”，泛指一类不局限于传统关系模型的数据库系统。

其名称发展经历两个阶段：

- 1998 年：Carlo Strozzi 使用 NoSQL 命名其基于文件的数据库。
- 2009 年：Eric Evans 在讨论开源分布式数据库的活动中重新推广这一术语。

NoSQL 的核心思想不是否定关系型数据库，而是针对关系型数据库不适合解决的特定问题，采用不同的数据模型与系统架构。

6.2 NoSQL 的主要特征

NoSQL 数据库通常具有以下特征：

特征	说明
Non-relational	采用非关系型数据模型
Flexible Schema	不要求所有记录遵循统一、严格的固定模式
Horizontal Scalability	通过增加节点扩展存储和处理能力
Partitioning	支持将数据分布到多个节点

特征	说明
Replication	通过数据副本提高容错能力与可用性
High Write Performance	部分系统针对大规模写入进行优化
Fast Access	针对特定访问模式提供快速读写
Open-source Ecosystem	存在大量开源实现，可降低软件许可成本

通过分片与复制，部分 NoSQL 系统能够在节点故障后重新分配任务或使用其他副本，避免单点故障（Single Point of Failure）。

但这些优势并非所有 NoSQL 产品都具备，具体性能取决于数据库实现、数据模型及工作负载。

6.3 NoSQL 的局限性

相较于传统关系型数据库，部分 NoSQL 系统在复杂关系查询、统一查询接口和事务保障方面存在限制。

1. 关系型操作支持有限

部分 NoSQL 数据库不直接支持跨整个分布式集群的复杂关系操作，例如：

- `JOIN`：多表连接。
- `GROUP BY`：分组聚合。
- `ORDER BY`：排序。
- 跨分区参照完整性约束（Referential Integrity Constraints）。

这些操作可能需要在应用层实现，或借助额外的查询与计算组件。

2. 缺少统一的声明式查询语言

关系型数据库普遍使用 SQL，而不同 NoSQL 数据库的查询接口差异较大，因此可能增加开发与迁移成本。

3. 事务保障可能较弱

部分 NoSQL 系统放宽 ACID 约束，以减少分布式协调成本，但这也意味着开发者可能需要自行处理部分一致性与并发问题。

现代 NoSQL 数据库已有较大改进，例如 MongoDB 支持多文档事务，Neo4j 支持 ACID，因此不能将“不支持事务”视为 NoSQL 的普遍特征。

6.4 NoSQL 的主要分类

按照数据模型，NoSQL 数据库主要分为四类：

类别	数据组织方式	课件代表产品
Key-Value（键值型）	通过唯一键访问对应值	DynamoDB、Voldemort、Scalaris
Column-based（列族型）	通过列族组织相关数据	Bigtable、Cassandra、HBase
Document-based（文档型）	以 JSON 等文档结构存储对象	MongoDB、CouchDB
Graph-based（图数据库）	以节点和边表示实体及关系	Neo4j、InfoGrid

主要区别：数据如何组织，以及数据库针对哪类查询操作进行优化。

早期应用案例：Google Bigtable、Amazon Dynamo、LinkedIn Voldemort，以及使用 Cassandra、HBase 等技术的互联网服务。这些案例说明 NoSQL 技术在大规模互联网应用中的发展背景，并不代表相关企业当前的完整技术架构。

Part 7 四类 NoSQL 数据库 Various NoSQL Databases

四类 NoSQL 数据库的根本差异在于数据模型及其访问方式：键值型强调按键快速访问，列族型强调稀疏宽表与相关列的组织，文档型支持复杂嵌套对象，图数据库则面向实体之间的关联关系。

7.1 键值型数据库 Key-Value Stores

1. 数据模型与分布式架构

键值型数据库将数据组织为键值对集合： $D={(k_1,v_1),(k_2,v_2),\ldots,(k_n,v_n)} $

其中 k_i 是用于定位数据的键（Key），v_i 是对应的值（Value）。

该模型结构简单，适合海量数据存储、高并发访问及水平扩展。

以 Amazon Dynamo 系列设计为背景，介绍了 Dynamo Ring Partitioning and Replication（环形分区与复制）：

- 通过键的哈希值确定数据分布位置。
- 将不同键的数据分配到不同节点。
- 为数据建立多个副本，提高容错能力。

其基本思想是通过键分布数据，而不是依赖单台服务器保存完整数据集。

2. DynamoDB 的复合主键

DynamoDB 示例采用复合主键（Composite Primary Key），由两部分构成：

- Partition Key（分区键）：决定数据所属的逻辑分区。
- Sort Key（排序键）：在同一分区键下区分并排序不同记录。

两者组合形成唯一主键： ${PK=(\text{Partition Key},\text{Sort Key})} $

例如：

Partition Key	Sort Key	Attributes
ProductID = 1	Book ID	Odyssey、Homer、1871
ProductID = 2	Album ID	6 Partitas、Bach
ProductID = 2	Album ID : Track ID	Partita No. 1
ProductID = 3	Movie ID	The Kid、Chaplin

不同记录可以具有相同的分区键，但完整主键组合必须唯一。

DynamoDB 还允许不同 Item（数据项）具有不同的非主键属性，因此不要求所有记录遵循完全相同的模式。DynamoDB 也支持仅使用 Partition Key 的简单主键，并非所有表都必须采用复合主键。

3. 基本操作接口 APIs

API	功能
get(key)	根据键获取对应值
put(key, value)	创建或更新键值对
delete(key)	删除键及其对应值
execute(key, operation, parameters)	对指定键对应的数据结构执行操作

其中 `execute` 可以用于操作 List、Set、Map 等数据结构，具体支持范围取决于数据库实现。

4. 优缺点

优点	局限性
数据模型简单	难以直接表达复杂对象关系
按键访问速度快	复杂条件查询能力可能有限
易于按键进行水平分区	跨键关联操作可能需要应用层实现
可通过复制实现容错	分布式副本可能带来一致性问题
适合高并发访问	不适合所有复杂分析型查询

将 Eventual Consistency（最终一致性）列为键值系统的常见特征，但不同产品支持的一致性级别并不相同。

5. 代表产品

产品	开发者或来源	数据模型与查询特点
SimpleDB	Amazon	键关联属性集合，支持受限 SQL 风格查询及 GetAttributes、PutAttributes 等操作
Redis	Salvatore Sanfilippo 最初开发	支持 String、List、Set、Sorted Set、Hash 等数据结构，并提供相应操作原语
Dynamo	Amazon	分布式键值架构，强调简单的 Get/Put 操作及上下文相关的版本处理
Voldemort	LinkedIn	分布式键值存储，接口设计与 Dynamo 类似

其中需要区分 Dynamo（Amazon 的分布式存储设计） 与 DynamoDB（Amazon 的托管数据库服务），二者并非同一产品。

7.2 列族型数据库 Column-Family Stores

1. 行式与列式存储

传统关系型数据库通常采用行式存储（Row-oriented Storage），将同一记录的不同属性组织在一起。

列式存储（Column-oriented Storage）则将相同列的数据集中组织，以便读取指定列的数据。

列式组织的主要优势是 Data Locality（数据局部性）：当查询仅涉及部分列时，数据库可以减少不必要的数据读取；相关数据集中存放，也有利于批量访问。

Bigtable、HBase、Cassandra 等属于宽列或列族数据库（Wide-column Stores），其物理组织并不完全等同于分析型数据库中的纯列式存储。

2. 列族数据模型

列族型数据库通过 Column Family（列族）组织相关列。

课件将其抽象为： Column Family = (*key*, *value*)

其中： $value={\text{Related Columns}} $

其主要特点包括：

- 一个列族包含若干逻辑相关的列。
- 不同记录可以拥有不同数量的列，适合稀疏数据（Sparse Data）。
- 数据通常按照行键、列名或其他存储顺序组织。
- 通过数据局部性优化特定的范围读取与列访问。

例如，同一个用户表中，有的用户记录包含电话号码，有的用户记录包含地址，而不必为每条记录保存完全相同的属性集合。

3. 代表产品

产品	开发者或来源	数据模型	查询特点
Bigtable	Google	基于行键、列与时间戳组织数据	支持行、列及时间戳范围选择
HBase	Apache	基于 Bigtable 思想的列族模型	Java API、Shell 操作
Hypertable	Hypertable 项目	类似 Bigtable	HQL（Hypertable Query Language）
Cassandra	Apache，最初由 Facebook 开发	分区键、聚簇列及列组织	按键查询、范围查询及指定列访问
PNUTS	Yahoo	支持哈希或有序表及灵活模式	主键查询、范围查询、排序及 Top-k 等操作

PNUTS 属于较早期的研究系统，表中的产品特征主要用于理解不同数据库模型的设计差异。

7.3 文档型数据库 Document Stores

1. 数据模型

文档型数据库以文档（Document）作为基本存储单位，能够表示比简单键值对更复杂的数据对象。

其逻辑结构为： $D = \{d_1, d_2, \cdots, d_n\}$

每个文档可以包含多个字段、数组以及嵌套对象。

常见文档格式包括：

- JSON（JavaScript Object Notation）
- XML（Extensible Markup Language）
- 其他半结构化格式（Semi-structured Formats）

文档模型能够表达对象、记录、结构体、列表、数组、映射和布尔值等结构。日期等类型通常由具体数据库的扩展格式支持，例如 MongoDB 的 BSON。

2. 嵌套文档示例

以下示例展示了如何在单个文档中表示个人信息、子女及联系电话：

文档数据库可以直接保存这类嵌套结构，而不必将其全部拆分为多个关系表。

其优势在于：

- 能够自然表示复杂对象。
- 支持嵌套对象与数组。
- 允许不同文档具有不同字段。
- 可以通过文档 ID 或字段进行查询。

3. 文档型与列族型的区别

对比	Document-based	Column-family
基本单位	Document	Row / Column Family
数据结构	支持多层嵌套	以行键和列族组织
模式	灵活、可变	通常需要定义列族或表结构
查询特点	文档字段、嵌套属性查询	行键、分区键及列范围查询
适用场景	用户档案、内容管理、复杂对象	海量稀疏数据、时序或大规模分布式写入

课件将列族模型描述为相对固定的一至两层字典结构，而文档模型允许任意复杂的嵌套文档。这是对典型数据模型的概括，不代表所有列族数据库都无法保存嵌套值。

4. 代表产品

产品	开发者或来源	数据模型与查询特点
MongoDB	10gen（后更名 MongoDB Inc.）	文档存储于 Collection，每个文档具有唯一 _id，支持字段查询、逻辑条件、更新和删除
Couchbase	Couchbase	以 JSON 文档存储数据，支持键访问及文档查询；课件介绍了 Views、JavaScript 和 MapReduce 等机制

文档数据库特别适合对象结构复杂、字段变化较频繁的应用，但对于高度复杂的跨实体关系，仍可能需要额外的关联查询或数据建模设计。

7.4 图数据库 Graph Databases

1. 数据模型

图数据库重点表示数据之间的连接关系（Interconnectivity），其设计受到图论（Graph Theory）的启发。

图可以表示为： ${G=(V,E)} $

其中：

- *V*：节点集合（Vertices / Nodes），表示实体。
- *E*：边集合（Edges），表示实体之间的关系。

在属性图模型（Property Graph）中，节点与边都可以具有属性（Properties）。

例如，社交网络中的用户可以表示为节点，关注关系可以表示为有向边。节点表示实体，边表示实体间的连接关系。

图数据库主要具有以下特征：

- 节点可以包含 ID 及其他属性。
- 边可以包含标签（Labels）、角色（Roles）及属性。
- 节点与边均可通过键值对表示属性。
- 不同产品提供不同的查询语言与访问接口。
- 适合邻居查询、路径搜索和多跳关系遍历。

图数据库尤其适用于关系结构复杂的数据，例如社交网络、知识图谱、推荐系统及欺诈检测。

2. 图查询

图数据库擅长执行 Graph-like Queries，例如：

- 邻居查询（Neighbor Query）
- 路径查询（Path Query）
- 最短路径（Shortest Path）
- 多跳关系遍历（Multi-hop Traversal）

例如，寻找两个节点之间的最短路径： $d(u,v)=\min_{p\in P(u,v)}|p|$。其中 $P(u, v)$ 表示从节点 u 到节点 v 的所有路径集合，$|p|$ 表示路径中的边数。该公式适用于无权图的最短路径定义。

3. 代表产品

产品	技术与来源	特征
JanusGraph	Apache 生态相关集成；主要使用 Java	分布式图数据库，支持大规模图分析与查询，并可集成 Spark、Hadoop 等平台
Neo4j	属性图数据库	支持 ACID 事务、图查询与高可用部署，提供可视化图探索工具
FlockDB	Twitter 开发的图存储系统	面向大规模社交关系的存储与访问
Pregel	Google 图计算模型	面向大规模图计算，不属于典型的事务型图数据库
InfoGrid	图数据相关平台	以图结构组织和访问相互关联的数据

Neo4j 产品表还列出了 Java、Python、Go、Ruby、PHP、R、C/C++ 等语言生态。需要区分数据库本身的实现语言与其客户端支持语言。

7.5 四类 NoSQL 数据库对比

维度	Key-Value	Column-family	Document	Graph
核心模型	Key → Value	行键 + 列族	Document	Nodes + Edges
主要访问方式	按键访问	按键及列范围访问	按文档字段查询	沿关系遍历
结构复杂度	较低	中等	支持复杂嵌套	支持复杂关系
主要优势	简单、高吞吐、易分区	稀疏宽表、大规模数据访问	灵活模式、对象表达	关系查询与路径计算
主要局限	复杂查询较弱	查询受数据模型影响较大	跨文档关联可能复杂	大规模全局分析可能需要专用计算框架

维度	Key-Value	Column-family	Document	Graph
典型场景	缓存、会话、键值查询	日志、时序数据、海量记录	用户档案、内容管理	社交网络、知识图谱

数**据库选**型的核心依据

不同 NoSQL 模型并非简单的性能高低关系，而是分别针对不同的数据组织和访问模式进行优化。

- 主要通过唯一键读取数据：优先考虑 Key-Value。
- 需要管理海量稀疏记录、按分区键或列范围访问：考虑 Column-family。
- 数据包含复杂嵌套对象，且字段结构经常变化：考虑 Document。
- 主要查询实体之间的多跳关系或路径：考虑 Graph。

选型时还需结合一致性要求、事务能力、数据规模、读写比例及运维成本进行综合判断。

7.6 本章知识总结

本章围绕大规模数据存储的架构演进展开，其核心逻辑为：

Data Types 数据结构与访问需求决定存储模型

Scaling & Replication 横向扩展提高系统容量，但引入分布式协调问题

CAP & BASE 网络分区下权衡一致性与可用性

NoSQL 采用不同数据模型满足特定规模与查询需求

本章需要重点掌握三个层面的区别：

1. ACID 与 BASE：前者是事务正确性的属性集合，后者是强调基本可用和最终一致性的分布式设计思路。
2. CAP 中的 C 与 ACID 中的 C：前者关注分布式操作的一致性，后者关注事务对数据库约束的维护。
3. 四类 NoSQL：分类依据是数据模型，而不是是否支持 SQL、是否支持 ACID 或是否使用分布式架构。

结论：NoSQL 的价值并非取代关系型数据库，而是通过不同的数据模型、分区机制和一致性策略，为特定的大规模数据存储与访问需求提供更合适的解决方案。

5583 Agentic Computing 1

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_5/knowledge/5583-Agentic-Computing-1.md`

5583 Agentic Computing1

Lecture 4 Deep Reinforcement Learning II & Introduction to Prompt

Engineering

深度强化学习进阶与提示工程导论

本讲主要包括三个部分：深度强化学习的进阶方法、现代 LLM 智能体的信息获取机制，以及强化学习策略训练实践。

整体知识体系可归纳为六个相互衔接的主题：

1. 强化学习基础回顾：智能体与环境的交互、价值函数、DQN 与策略梯度。
2. 强化学习中的奖励与探索问题：稀疏奖励、奖励塑形、课程学习。
3. 深度强化学习的典型应用：AlphaGo 系列、多智能体强化学习。
4. 现代 LLM 智能体与知识边界：预训练知识的来源、覆盖范围、质量与时效性。
5. 联网搜索与信息可靠性：搜索触发、来源选择、信息过滤与多步搜索。
6. 深度研究与自主任务执行：研究规划、多源评估、迭代检索、证据整合与 Agentic AI。

Part 1 强化学习基础

1.1 强化学习的基本框架

强化学习（Reinforcement Learning, RL）是一种通过与环境交互、利用奖励反馈优化决策策略的机器学习方法。其核心目标是学习能够最大化长期期望回报的策略，而非仅优化单步奖励。

强化学习系统包含以下基本要素：

要素	符号	含义
Agent（智能体）	—	与环境交互并作出决策的学习主体
Environment（环境）	—	智能体执行动作并获得反馈的外部系统
State（状态）	\(s_t\)	环境在时刻 \(t\) 的状态
Observation（观测）	\(o_t\)	智能体能够感知的环境信息
Action（动作）	\(a_t\)	智能体在当前状态下采取的行为
Reward（奖励）	\(r_t\)	环境对动作结果提供的数值反馈
Policy（策略）	\(\pi\)	从状态或观测到动作或动作概率的映射

智能体与环境的交互过程可以表示为：

每一时间步的交互流程为：

1. 智能体接收当前状态或观测。
2. 根据策略 π 选择动作 a_t。
3. 环境执行状态转移，并返回奖励与下一状态。
4. 智能体根据反馈更新策略，重复交互过程。

以电子游戏为例，游戏画面构成观测，移动或射击构成动作，得分构成奖励，游戏模拟器则充当环境。

回报（Return） 是一个回合（Episode）内获得的累计折扣奖励。

$R=\sum_{t=1}^{T}\gamma^t r_t $

- T：回合长度。
- r_t：第 t 步的即时奖励。
- $\gamma \in [0,1]$：折扣因子（Discount Factor），控制未来奖励的重要程度。

一般而言，γ 越接近 1，智能体越重视长期收益；γ 越小，智能体越偏向近期收益。

强化学习的优化目标为：$\pi^* = \arg\max_\pi \mathbb{E}_\pi[R]$，即寻找能够最大化期望累计回报的最优策略。

1.2 状态价值函数与动作价值函数

强化学习通过价值函数（Value Function）评估不同状态或动作的长期收益，主要包括两种形式。

1. 状态价值函数（State-Value Function）

状态价值函数 $V^\pi(s)$ 衡量：从状态 s 出发，持续遵循策略 π 时能够获得的期望回报。

${ V^\pi(s)= \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty}\gamma^k r_{t+k} \mid s_t=s \right] }$

例如，在网格导航任务中，靠近目标位置的状态通常具有较高价值，而靠近危险区域的状态通常具有较低价值。

2. 动作价值函数（Action-Value Function）

动作价值函数 $Q^\pi(s,a)$ 衡量：在状态 s 下首先执行动作 a，之后遵循策略 π 时的期望回报。

${ Q^\pi(s,a)= \mathbb{E}*{\pi} \left[\sum*{k=0}^{\infty}\gamma^k r_{t+k} \mid s_t=s,a_t=a \right] } $

两者的区别在于：

- $V^\pi(s)$ 评价状态本身的长期价值。
- $Q^\pi(s,a)$ 评价特定状态下执行某一动作的长期价值。

对于离散动作空间，若已经获得最优动作价值函数 $Q^*(s,a)$，则最优动作可由以下规则确定：

$pi^*(s) = \arg\max_a Q^*(s,a)$

课件中的机器人导航示例：

由于向右的 Q 值最高，因此贪心策略选择向右移动。

需要注意，Q 值表示期望回报，而非动作概率，因此可以为负，也不要求归一化。

1.3 深度 Q 网络与策略梯度

深度强化学习（Deep Reinforcement Learning, DRL）利用深度神经网络近似价值函数或策略函数，从而处理高维状态空间，例如游戏图像、机器人传感器数据等。

两类方法：

1. 深度 Q 网络（Deep Q-Network, DQN）

DQN 属于基于价值的方法（Value-Based Methods），利用深度神经网络估计不同动作的 Q 值：

$Q(s, a; \theta) \approx Q^*(s, a)$

其中，θ 表示神经网络参数。其动作选择规则为：$\pi(s) = \arg\max_a Q(s, a; \theta)$

课件中的游戏示例：

由于 $Q(s, \text{Fire}) = 10$ 最大，DQN 选择射击。随后智能体执行动作，观察下一状态及奖励，继续更新网络。

2. 策略梯度（Policy Gradient）

策略梯度属于基于策略的方法（Policy-Based Methods），不通过 Q 值间接选择动作，而是直接学习参数化策略：$\pi_\theta(a \mid s) = P(a \mid s; \theta)$

神经网络（Actor）输出动作概率分布，再根据该分布采样：$a_t \sim \pi_\theta(\cdot \mid s_t)$

尽管 Fire 的概率最高，策略仍可能采样到 Move right。课件展示的采样结果就是 Move right。

对于离散动作空间，动作概率必须满足：$\Sigma_{a \in \mathcal{A}} \pi_\theta(a \mid s) = 1$

对于连续动作空间，策略也可以采用概率分布表示，例如：$pi_\theta(\cdot \mid s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta^2(s))$

以高斯分布为例，展示了根据均值、方差生成动作分布，再采样得到连续动作（如速度 0.4 m/s）的过程。

两类方法的核心区别如下：

维度	DQN	Policy Gradient
方法类别	Value-Based	Policy-Based
网络输出	动作价值 $Q(s, a)$	动作概率 $pi_\theta(a \mid s)$
动作选择	选择最大 Q 值动作	根据策略分布采样
输出约束	Q 值可正可负	概率非负且总和为 1
主要优化对象	价值函数	策略函数

Actor-Critic 方法：Actor-Critic 结合价值估计与策略优化

- Actor：学习策略，负责动作选择。
- Critic：估计价值，为 Actor 的更新提供评价信号。

Part 2 稀疏奖励、奖励塑形与课程学习

2.1 稀疏奖励问题 Sparse Rewards

稀疏奖励是强化学习中的重要困难，指智能体在大多数交互步骤中无法获得具有充分指导意义的奖励，或者奖励仅在任务完成后才出现。

任务	奖励特点	主要困难
围棋（Go）	胜负奖励可能仅在终局产生	奖励延迟，难以评价中间落子
机械臂套环（Ring-on-Peg）	成功套环时才获得有效奖励	随机探索很难达到目标状态

稀疏奖励主要造成两方面的问题：

- 探索效率低：智能体难以通过随机尝试发现成功路径。

- 信用分配困难（Credit Assignment）：最终奖励难以准确反映此前各个动作的贡献。

例如，在机械臂套环任务中，若只有成功套环时奖励为 1，其余时刻奖励为 0，则可表示为：

$r_t= \begin{cases} 1,&\text{成功完成套环}\ 0,&\text{其他情况} \end{cases} $

当成功概率极低时，大量训练轨迹可能完全没有正奖励，导致策略难以获得有效改进。

2.2 奖励塑形 Reward Shaping

奖励塑形通过引入中间奖励，为智能体提供更密集的学习信号，缓解稀疏奖励问题。其基本思想是在原始任务奖励之外，对接近目标、完成中间步骤等行为提供反馈，同时保留任务完成时的主要奖励。

一般可写为： $r'_t = r_t + F(s_t, a_t, s_{t+1})$

- r_t：原始环境奖励。
- F：额外设计的塑形奖励。
- r'_t：用于学习的调整后奖励。

例如，在目标导航任务中，可以奖励智能体缩短与目标的距离： $F_t=\lambda(d_t-d_{t+1}) $

其中 d_t 表示当前位置与目标之间的距离， λ 为奖励权重。

当 $d_{t+1} < d_t$ 时，智能体获得正向反馈。

奖励塑形的风险与改进：不合理的中间奖励可能使智能体追求代理目标，而非真正完成任务。例如，若持续奖励移动行为，智能体可能反复移动以获取奖励。

一种具有理论保证的设计方式是基于势函数的奖励塑形（Potential-Based Reward Shaping）：

$F(s_t, s_{t+1}) = \gamma \Phi(s_{t+1}) - \Phi(s_t)$

其中 $\Phi(s)$ 为状态势函数。在适当的终止条件等假设下，该形式能够保持原始任务的最优策略不变。

2.3 ViZDoom 奖励塑形案例

以 ViZDoom 第一人称射击竞赛说明复杂环境中的奖励设计。任务目标不仅是击败敌人，还需要考虑生存、弹药、移动及资源获取等因素。因此，奖励函数由多个部分组成：

$r_t = r_{\text{living}} + r_{\text{health}} + r_{\text{ammo}} + r_{\text{pickup}} + r_{\text{distance}} + \cdots$

其中，living 是课件中对持续存活状态施加的每动作惩罚，不应误解为单纯奖励生存时间。

奖励塑形的**核心价值**在于将复杂任务的最终目标转化为训练过程中更容易观察的反馈信号。

2.4 课程学习 Curriculum Learning

课程学习通过将复杂任务组织为一系列难度逐渐增加的子任务，使智能体能够从较容易的问题开始学习，再逐步迁移到目标任务。

其一般过程为： $\mathcal{T}_1 \to \mathcal{T}_2 \to \cdots \to \mathcal{T}_n$

其中： $\text{Difficulty}(\mathcal{T}_1) < \text{Difficulty}(\mathcal{T}_2) < \cdots < \text{Difficulty}(\mathcal{T}_n)$

以机械臂套环任务介绍反向课程学习（Reverse Curriculum Learning）。传统训练直接从随机初始位置开始，机械臂很难偶然完成套环，因此难以获得有效奖励。

反向课程学习则从接近目标的位置开始训练：阶段 1 靠近目标 低难度、阶段 2 扩大起点范围、阶段 3 随机初始位置 高难度。通过逐步扩大初始状态范围，智能体可以在已有能力的基础上学习更困难的任务。

奖励塑形与课程学习的区别如下：

维度	Reward Shaping	Curriculum Learning
核心思想	增加中间奖励	调整任务训练顺序
主要改变	奖励函数	任务或初始状态分布
主要作用	增强反馈信号	降低早期探索难度
典型应用	ViZDoom 奖励设计	机械臂反向课程学习

两者可以结合使用，以进一步提高复杂强化学习任务的训练效率。

Part 3 深度强化学习的典型应用

3.1 AlphaGo：监督学习与自我对弈

围棋是深度强化学习的典型应用场景，具有以下特点：

- 状态空间极大：19×19 围棋的合法局面数量远超可观测宇宙中的原子估计数量（约 10^{80}）。
- 奖励稀疏且延迟：胜负通常需要在对局结束后才能确定。
- 长期决策依赖显著：单步落子的价值需要结合后续多步变化判断。

围棋的复杂性使得简单穷举所有可能局面不可行。AlphaGo 通过结合神经网络、搜索与强化学习应对上述问题。

其中有两个学习阶段：

阶段一：基于人类棋谱的监督学习（Supervised Learning）

利用专家对局数据训练策略网络，学习在给定棋局下模仿专家落子： $pi_\theta(a \mid s)$

其中 s 为棋盘状态，a 为落子位置。

可以使用交叉熵损失进行训练：

$\mathcal L_{\mathrm{SL}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log \pi_\theta(a_i^{*}\mid s_i)$

其中 a_i^* 为专家在状态 s_i 下选择的动作。

监督学习可以快速获得具有基本棋力的策略，但其局限在于人类棋谱无法覆盖全部可能的棋盘状态。

阶段二：基于自我对弈的强化学习（RL Self-Play）

在监督学习策略的基础上，通过智能体之间的自我对弈生成新的训练经验，并利用胜负结果继续优化策略。

该阶段不再局限于模仿人类行为，而是通过自主探索改进决策质量。

因此，两阶段学习分别解决：

- 监督学习：提供有效的初始策略，降低训练难度。
- 强化学习：突破专家数据覆盖范围，提高策略表现。

蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）

AlphaGo 还利用 MCTS 在决策时探索未来可能的落子序列，并结合策略网络与价值网络提高搜索效率。

其中：

- 策略网络提供候选动作的先验信息。
- 价值网络估计棋盘局面的胜率或价值。
- MCTS 在有限计算预算内搜索具有较高潜力的决策路径。

3.2 AlphaGo、AlphaZero 与 MuZero 的演进

三代系统展示强化学习从依赖人类经验逐步走向自主学习与规划的发展过程。

系统	人类棋谱依赖	环境知识	核心特点
AlphaGo	使用专家棋谱进行监督预训练	已知围棋规则	监督学习、强化学习与树搜索结合
AlphaZero	不依赖专家棋谱	已知游戏规则	从自我对弈中学习，可应用于围棋、国际象棋和将棋
MuZero	不依赖专家棋谱	不要求预先提供完整环境动态模型	学习用于规划的环境模型

AlphaGo 人类棋谱 + 自我对弈、AlphaZero 从零开始自我对弈、MuZero 学习环境模型并规划

三代系统的主要区别在于学习所依赖的先验知识，减少对人类示范及预先提供的环境动态知识的依赖。

AlphaGo 利用人类专家数据进行初始化；AlphaZero 通过自我对弈学习策略；MuZero 则进一步学习用于规划的环境动态表示。

需要区分：MuZero 不需要预先提供完整的环境转移模型，但仍然需要可交互的环境、动作接口以及奖励等任务信息，并非完全不需要任何规则或环境知识。

3.3 多智能体强化学习 Multi-Agent Reinforcement Learning

多智能体强化学习（MARL）研究多个智能体在共享环境中同时学习与决策的问题。

与单智能体强化学习相比，MARL 的关键特点是一个智能体的决策结果不仅取决于自身动作，还取决于其他智能体的行为。

对于包含 N 个智能体的系统，联合动作可以表示为： $\mathbf{a}_t = (a_t^1, a_t^2, \cdots, a_t^N)$

环境状态转移依赖所有智能体的联合动作： $P(s_{t+1} \mid s_t, a_t^1, \cdots, a_t^N)$

每个智能体 i 的目标仍然是最大化自身期望回报：

$\max_{\pi_i} \mathbb E_{\pi_i,\pi_{-i}} \left[\sum_{t=0}^{T}\gamma^t r_t^i \right] $

其中 π_{-i} 表示其他智能体的策略集合。

以多智能体 Hide and Seek 环境作为示例，说明多个智能体能够在同一环境中相互影响、学习和行动。

根据奖励目标的关系，多智能体系统可分为三种类型：

类型	目标关系	课件示例
Cooperative（合作）	共享目标与团队收益	仓库机器人协同配送
Competitive（竞争）	目标相互对立	网球比赛中的双方选手

类型	目标关系	课件示例
Mixed（混合）	同时存在合作与竞争	足球比赛中队内合作、队间竞争

奖励设计决定了智能体之间的行为关系。例如，网球比赛中的胜负奖励使双方成为竞争者，而仓库配送任务中的共享奖励则鼓励合作。

3.4 多智能体独立学习的困难

每个智能体可以独立采用 DQN 或策略梯度进行训练，但这种方式存在两项主要困难。

1. 环境非平稳性（Non-Stationarity）

在单智能体强化学习中，通常假设环境转移规律相对稳定。但在多智能体环境中，其他智能体也会不断更新策略。因此，从某个智能体的视角看，即使自身状态和动作不变，下一状态及奖励的分布也可能随着其他智能体的策略变化而改变：

$P(s_{t+1} \mid s_t, a_t^i; \pi_{-i})$

当 π_{-i} 不断变化时，该转移分布也随之变化。

这会导致：

- 已学习的有效动作可能失效。
- 历史经验可能与当前环境不一致。
- 策略优化过程更难稳定收敛。

2. 团队信用分配问题（Multi-Agent Credit Assignment）

在合作任务中，多个智能体可能共享一个团队奖励：

$R_{\mathrm{team}} = \sum_{t=0}^{T}\gamma^t r_t^{\mathrm{team}}$

但团队奖励无法直接指出每个智能体对最终结果的具体贡献。

例如，多个机器人共同完成配送后获得奖励，但无法仅根据最终得分判断哪个机器人的动作最关键。

一种常见解决思路是 Centralized Training with Decentralized Execution：

- 训练阶段：Critic 可以利用多个智能体的联合状态、动作等信息进行价值评估。
- 执行阶段：每个 Actor 根据自身可获得的观测独立选择动作。

CTDE 可以缓解部分非平稳性与信用分配问题，但不能保证完全消除这些困难。

Part 4 现代 LLM 智能体及信息来源

4.1 智能体架构的转变

以传统 AI Agent 与现代 AI Agent 的对比，引出基于大语言模型的智能体。

传统强化学习智能体主要通过环境交互获得奖励，再利用梯度更新模型参数。现代 LLM 智能体则以预训练语言模型为核心，结合记忆、自然语言指令、工具调用与反馈进行任务执行。

维度	传统 RL Agent	现代 LLM Agent
核心模型	策略网络、价值网络	大语言模型
决策依据	状态、价值估计、奖励	语言上下文、预训练知识、外部信息
主要学习机制	奖励驱动的参数优化	预训练与后训练，推理时可结合反馈调整行动
反馈利用	更新策略参数	可通过反思、记忆与重新规划调整后续步骤
外部能力	环境交互	搜索、工具、记忆及其他系统

两种不同的反馈机制：

- 传统 RL 中的梯度下降通常用于更新模型参数。
- LLM Agent 的反思推理通常是在推理过程中利用反馈调整后续行动，不必然修改模型参数。

两类智能体并非完全互斥。现代 LLM 也可以通过强化学习进行后训练，LLM Agent 也可以使用价值评估或奖励机制。

整门课程划分为四个阶段：

阶段	课程	核心内容
Agent Foundations	L1 - L3	AI Agent 概述、多模态嵌入、神经网络、Transformer、经典 RL、深度 RL
Agent Interaction	L4 - L7	Prompt Engineering、Chain of Thought、工具增强智能体、Retrieval、RAG、Grounding 与 Verification
Agent Systems	L8 - L10	单智能体与多智能体系统、Codex、Skills、具身智能体、Vision-Language-Action 模型
Integration	L11 - L13	系统评估、前沿研究与项目实践

本讲是 Agent Interaction 阶段的起点，重点从传统 RL 决策机制过渡到 LLM 智能体的信息获取与任务执行机制。

4.2 大语言模型的知识边界 Knowledge Boundary of LLMs

4.2.1 预训练知识的来源

大语言模型通过大规模预训练语料学习语言模式、事实关联及其他知识表示。

主要预训练知识来源包括：

数据来源	代表平台或资源
Internet Forums（网络论坛）	Reddit、Quora
Books（书籍）	Google Books、Open Library
Encyclopedias（百科）	Wikipedia、Wikidata
News Sites（新闻网站）	The New York Times、BBC
Academic Articles（学术论文）	arXiv、PubMed

这些来源共同构成模型可能接触的知识基础，但并不意味着每个模型都实际使用了上述所有平台的数据。

LLM 的知识能力受到训练数据的覆盖范围、质量、频率和时间范围等因素影响。

4.2.2 预训练知识的覆盖与频率差异

Pretraining data shapes the agent's internal knowledge structure.

预训练数据的分布会影响模型内部形成的知识表示。不同主题在训练数据中的出现频率并不相同。

知识类别	课件示例	数据分布特点
高频知识	Cooking、Celebrities、Movies	相关语料较丰富
相对低频知识	Cantonese、Quasars	相关语料相对有限
非公开知识	Secret / Proprietary Data	可能未包含在可访问训练语料中

通常，某个主题在训练数据中出现得越频繁，模型就越有机会学习与其相关的语言和事实模式。

但知识频率并不等同于知识正确性。高频错误同样可能被模型学习，而低频知识也不一定无法被准确掌握。

此外，模型可能在缺乏充分依据时生成表面合理但事实上不正确的内容，即产生幻觉（Hallucination）。

4.2.3 对不完美自然语言的理解能力

LLM 对自然语言具有一定鲁棒性，可以处理拼写错误、缩写和不完整表达。

- 非标准输入：can u cook egs in microwav
- 标准表达：Can you cook eggs in the microwave?

模型能够识别两者表达的基本意图，并给出关于微波炉加热鸡蛋的安全提示，例如某些加热方式可能导致鸡蛋爆裂。

这一能力说明，LLM 不必依赖严格的关键词匹配或固定命令格式，即可从自然语言中推断用户意图。但自然语言容错不意味着模型能够可靠识别所有错误前提，因此仍需要事实验证。

4.2.4 训练数据的质量问题

训练数据中可能存在的三类错误：

1. Typos（拼写错误）：例如 `cookin egs` 等不规范表达。
2. Misconceptions（错误观念）：未经验证或不符合事实的观点。
3. Outdated Information（过时信息）：例如将北京 2022 年冬奥会描述为尚未举行的事件。

这些问题可能影响模型输出的准确性与可靠性。

从知识形成过程看，模型主要通过学习数据中的统计规律获得能力，而非对每条训练信息进行独立的事实核验。

因此，训练数据中的错误、偏差和过时内容可能反映在模型生成的答案中。

4.2.5 知识截止日期 Knowledge Cutoff

预训练知识具有时间边界。

课件以 GPT-5.4 为例，将 2025 年 8 月 31 日标记为其知识截止日期，用于说明模型无法仅依靠该截止日期之前的预训练数据掌握之后发生的新事件。这一日期是课件示例，并不代表所有模型具有相同的知识截止时间。

知识截止日期带来的问题包括：无法仅依靠预训练知识获得最新新闻、无法保证掌握当前营业信息、政策及价格、无法确认训练截止日期之后发生的事件、可能沿用已经失效的历史事实。

因此，预训练知识适合提供相对稳定的基础知识，但不能替代实时信息检索。

4.3 联网搜索与信息可靠性 Web Search

4.3.1 联网搜索的适用场景

联网搜索（Web Search）使 LLM Agent 能够在推理过程中获取外部信息，以弥补预训练知识的局限。

问题分为两类：预训练知识和 Web Search

联网搜索的主要触发条件可概括为：

- Real-time：需要当前或近期信息。
- Location-specific：需要特定地理位置的信息。
- Niche：涉及相对小众、训练数据可能覆盖不足的知识。

例如，查询香港红磡附近的健身房时，需要结合近期用户评价、健身房官网及地理位置信息。而对于长洲抢包山比赛，搜索可以补充活动背景、规则和具体安排。

4.3.2 联网搜索的触发机制

两种搜索触发方式：

1. 模型自主触发（Model-Initiated Search）

模型根据用户问题判断是否需要外部信息。

例如，当问题涉及实时、本地或小众知识时，模型可以主动选择搜索。

2. 用户显式触发（User-Initiated Search）

用户明确要求模型使用网络搜索，或在界面中主动启用搜索功能。

两种方式的区别主要在于工具调用决策由模型还是用户发起。

4.3.3 搜索来源的可靠性与时效性

联网搜索可以获取外部信息，但并不保证搜索结果天然准确。两个风险：

1. 来源可靠性差异（Source Reliability）

搜索结果可能来自：社交媒体、博客、网络论坛、百科平台、政府机构、专业组织、学术研究。

不同来源的可靠性、证据质量和适用范围存在差异。2025 年 LLM 常见引用域名统计中，Reddit、Wikipedia、YouTube、Google、Yelp 等平台占据较高位置，说明模型可能大量引用热门网络来源。但引用频率并不能直接证明来源可靠。

2. 来源时效性问题（Source Freshness）

以美国内华达州 Henderson 的跑步地点查询为例。模型检索到一条发布于 2003 年的跑步论坛帖子，但该信息距离查询时间已有约 23 年，相关场地可能已经关闭。

这说明：成功检索到信息，不代表该信息仍然有效。对于营业状态、交通路线、设施开放情况等动态信息，必须核查发布时间、更新日期和当前官方公告。

4.3.4 来源引导 Source Guidance

来源引导是提示工程中控制搜索质量的重要方法。

普通提示：How safe are gray-market peptides?

来源约束提示：How safe are gray-market peptides? Use official health organizations and rigorous scientific studies.

第二种提示明确要求使用官方卫生机构与严谨科学研究，能够引导搜索过程优先考虑更符合证据要求的信息来源。提示不仅决定搜索主题，也可以影响搜索来源的选择与信息过滤规则。

需要注意，权威机构和学术论文也应结合具体问题评估其适用性、发布时间与证据质量，而不能仅根据机构名称判断结论一定正确。

4.3.5 联网搜索的多步骤工作流

课件将 Web Search 描述为一个包含检索、筛选和综合的多步骤过程：

用户提示中的主要信息包括：

- Topic：研究主题。
- Task：需要完成的任务。
- Target Sources：目标信息来源。
- Desired Depth：期望的信息深度。

搜索系统据此执行检索、扫描结果、过滤不相关内容、汇总信息和生成回答。

其中，信息过滤不仅要考虑主题相关性，还应结合可靠性、时效性和来源质量。

案例：黄山徒步

该案例说明，复杂度适中的搜索任务可以分解为多个具体信息需求，再将结果整合为可执行的建议。

4.3.7 搜索引擎与 AI 模型的选择

传统搜索引擎与 LLM 在信息处理方式上具有不同优势。

搜索引擎（Search Engine）	AI 模型（AI Model）
快速浏览多个搜索结果	综合多个来源的信息
定位具体网站或页面	解释复杂概念
查看信息原始形式	比较不同来源的观点
适合明确的导航和查找任务	适合解释、比较与综合任务

两个典型例子：

搜索 2013 Honda Civic air filter：更适合通过搜索引擎定位 The Home Depot、Walmart 等商品页面；查询维生素补充剂的利弊：更适合利用 AI 综合 WHO、FDA、EMA 等机构的相关信息。

因此，搜索引擎更适合获取原始页面与定位资源，而 AI 模型更适合跨来源的信息解释与综合。两者可以结合使用。

4.4 Deep Research 与 Agentic AI

4.4.1 深度研究的基本概念

深度研究（Deep Research）是一种面向复杂问题的多步骤信息获取与分析工作流。

与普通联网搜索相比，其主要特点是：

1. Plan a Research Task：将复杂目标分解为多个研究子任务。
2. Read and Evaluate Sources：阅读并评估不同来源的内容。
3. Synthesize Findings with Citations：综合研究结果，并提供可追溯的引用。

深度研究通常需要多次搜索、阅读、评估和迭代，而不是执行一次检索后直接生成答案。

4.4.2 深度研究的迭代工作流

课件展示的完整流程为：

1. Plan Research Task 研究规划与子问题分解
2. Web Search 多次检索；Read Sources 阅读资料
3. Evaluate Sources 判断相关性、可靠性与充分性
4. More Needed 返回检索阶段补充证据；Sufficient 进入结果综合阶段
5. Synthesize → Draft → Add Citations 综合证据、撰写报告并添加引用
6. Final Answer with Sources

深度研究的关键在于评估结果是否充分，并据此决定继续检索还是生成最终报告。

这构成一个基于证据充分性的反馈循环，而不是固定执行一次的线性流程。

Agentic AI：模型能够在给定任务范围内，自主决定下一步需要执行的操作。

这种自主性体现在：

- 根据任务目标分解研究步骤。
- 判断不同子任务需要哪些信息来源。
- 在执行过程中决定下一步搜索或阅读内容。
- 整合检索结果并生成报告。

但自主执行并不等于不需要用户控制。用户仍然可以通过任务范围、约束条件及必要的确认步骤限制智能体行为。

4.5 Web Search 与 Deep Research 的区别

维度	Web Search	Deep Research
问题类型	通常是单个聚焦问题	多个相互关联的问题
任务复杂度	相对较低	相对较高
来源数量	通常少量来源	可能几十条或更多
典型耗时	几秒至几分钟	几分钟或更长
处理过程	检索、过滤、总结	规划、检索、阅读、评估、迭代、综合
主要产出	聚焦的事实或答案	带证据的综合分析报告
案例一	寻找附近高评分健身房	研究每日步数对长期健康的影响
案例二	查询迪拜本周天气	分析天气对迪拜旅游业的影响

两者的主要区别不是是否使用搜索工具，而是研究任务的复杂度、证据整合要求及是否需要多轮迭代。

4.6 智能体信息来源的综合比较

现代 LLM 智能体的三种信息获取方式对比：

维度	Pretrained Knowledge	Web Search	Deep Research
信息来源	模型内部参数知识	外部检索结果	多轮外部检索与分析
检索来源数量	无	通常少量	可能几十条以上
信息时效性	受训练数据限制	取决于来源	取决于来源
典型耗时	秒级	秒至分钟级	分钟级或更长
主要用途	定义、常识、基础解释	实时、本地、小众信息	复杂问题的多源综合
课件示例	手机掉入汤中如何处理	寻找附近健身房	每日步数与长期健康关系

三种方式不是相互排斥的独立技术，而是可以组合使用的信息获取机制。

例如，深度研究通常同时利用：

- 预训练知识理解任务、生成研究计划。
- Web Search 获取外部证据。
- LLM 的推理与综合能力整理资料。
- 引用机制提高结果的可追溯性。

因此，现代 LLM Agent 的能力不仅取决于模型自身掌握多少知识，也取决于其能否正确判断信息需求、选择工具、评估来源并综合证据。

4.7 本讲核心知识总结

本讲的内容可以归纳为两条主要技术路线。

第一条：从奖励驱动的学习到复杂环境中的自主决策。

强化学习通过奖励优化策略，DQN 和策略梯度分别代表基于价值与基于策略的典型方法。

当任务具有稀疏奖励时，可以通过奖励塑形和课程学习改善训练条件。AlphaGo 系列展示了监督学习、自我对弈、搜索与模型学习的结合；多智能体强化学习则进一步引入环境非平稳性与团队信用分配问题。

第二条：从预训练知识到外部信息增强的智能体。

LLM 具备广泛的预训练知识，但受到数据覆盖、知识频率、数据质量及知识截止日期等限制。

联网搜索通过外部信息补充模型知识，同时引入来源可靠性和时效性问题。深度研究进一步将信息获取组织为规划、检索、评估、迭代与综合的完整工作流。

两条路线共同体现了智能体研究的核心问题：如何使智能体在不确定环境中利用反馈与信息持续改进决策。

Part 5 思维链推理与问题分解

本部分讨论如何通过提示工程（Prompt Engineering）提高大语言模型处理复杂任务的能力。其核心思想是：将复杂任务分解为可管理的中间步骤，并根据任务性质选择语言推理、程序计算或外部工具。

主要涉及三种方法：

1. 思维链推理（Chain-of-Thought, CoT）：通过中间推理步骤解决复杂问题。
2. 程序辅助语言模型（Program-Aided Language Models, PAL）：利用语言模型生成程序，再由程序执行精确计算。
3. 问题分解与提示链（Problem Decomposition & Prompt Chains）：将复杂任务拆分为多个子任务，通过模型与工具的多轮调用完成。

5.1 思维链提示 Chain-of-Thought Prompting

5.1.1 基本原理

思维链提示是一种通过引导模型生成中间推理步骤来完成复杂任务的提示技术；传统提示通常要求模型直接输出最终答案，而思维链提示则要求模型先完成必要的中间计算或逻辑分析，再得出结论。

1. 以网球数量计算为例：Roger 原本有 5 个网球，又购买了 2 罐网球，每罐有 3 个。他现在共有多少个网球?

标准提示（Standard Prompting）可能直接得到答案，而思维链提示会提供分步求解。

2. 通过苹果数量问题说明两种方法的区别：某食堂原有 23 个苹果，使用 20 个制作午餐，之后又购买 6 个。

标准提示示例错误地回答为 27，而思维链提示按照步骤求解。

显式分解问题可以帮助模型处理多步骤计算，降低遗漏条件或错误组合运算的风险。但需要注意，思维链并不保证结果正确。模型生成的推理步骤仍可能存在逻辑错误或计算错误，因此不能仅凭推理过程完整就认定结论可靠。

5.1.2 Few-shot CoT 与 Zero-shot CoT

思维链提示主要有两种实现方式。

比较维度	Few-shot CoT	Zero-shot CoT
提示方式	提供包含中间步骤的完整示例	直接使用推理指令
是否需要示例	需要	不需要
主要机制	上下文学习（In-context Learning）	通过指令诱导模型进行分步推理
提示长度	相对较长	相对简短

1. Few-shot Chain-of-Thought

通过在提示中提供若干完整的解题示例，使模型学习任务的推理结构。

其形式可以表示为： $P = \{(x_1, r_1, y_1), \cdots, (x_k, r_k, y_k), x_{\text{new}}\}$

- x_i：示例问题。
- r_i：示例中的中间推理步骤。
- y_i：示例的最终答案。
- x_{new}：需要解决的新问题。

模型根据上下文中的示例，生成新问题的求解步骤及答案。这种方法属于上下文学习，不需要更新模型参数。

2. Zero-shot Chain-of-Thought

即使没有提供完整的推理示例，模型也可能通过简单指令生成中间推理步骤。

典型指令为：Let's think step by step.

该研究涉及 PaLM 和 InstructGPT 等模型。模型能够在缺少示例的情况下产生推理步骤，可能与预训练数据中包含的推理范例，以及系统指令等因素有关。

CoT 的适用边界

思维链更适合多步骤推理、逻辑分析和复杂问题求解，而不是所有任务都必须生成详细推理文本。对于简单事实查询，直接回答通常更高效；对于精确计算，则应优先使用程序验证。

5.2 程序辅助语言模型 Program-Aided Language Models, PAL

5.2.1 基本思想

程序辅助语言模型（PAL）通过结合自然语言理解与程序执行，提高复杂计算任务的可靠性。

核心原则：由语言模型理解问题并生成程序，由程序解释器执行计算并返回结果。

传统语言模型直接生成计算结果，可能产生算术错误。PAL 将自然语言推理与数值计算分离，使计算过程能够通过确定性程序执行。

整体流程：自然语言问题（Question）；LLM：理解问题并生成程序；Python Interpreter：执行程序；计算结果（Answer）。

5.2.2 PAL 的实现方式

两项关键方法：

1. 在提示中提供结合自然语言与代码的示例。
2. 由 LLM 生成程序，并使用 Python 解释器执行程序以获得答案。

5.2.3 PAL 的优势与局限

PAL 的主要优势包括：

- 计算准确性：使用程序执行算术运算，减少语言模型直接计算产生的错误。
- 可复现性：相同输入和相同程序通常能够得到相同结果。
- 可验证性：可以检查生成的代码、变量定义及计算逻辑。
- 工具扩展性：可进一步使用 Python 库处理统计分析、符号计算及数据可视化任务。

需要区分两类错误： 最终结果错误 = 问题形式化错误 ∨ 程序执行或计算错误

这一表达式用于说明错误来源，而不是概率公式。即使程序能够正确执行，如果模型错误理解题意、遗漏条件或生成错误公式，最终结果仍然可能不正确。

因此，PAL 的可靠性依赖于两个环节：

1. 语言模型是否正确地将问题转化为程序。
2. 程序是否正确执行并产生预期结果。

5.3 问题分解与提示链 Problem Decomposition and Prompt Chains

5.3.1 问题分解

问题分解是指将复杂任务划分为多个规模较小、目标明确的子问题，并逐步解决。

典型工作流（重复执行）： $\text{提出子问题} \rightarrow \text{搜索引擎检索} \rightarrow \text{获取中间答案} \rightarrow \text{继续提问} $

例如，问题为：Theodore Haeker 和 Harry Vaughan Watkins 谁活得更久？

模型首先需要分别查询两人的死亡年龄，再比较结果；另一示例涉及查询当前最高的木制格构塔及其建成年份。模型首先通过搜索确定目标建筑为 Gliwice Radio Tower，随后继续查询其建成年份，得到 1935 年。

复杂信息查询往往需要利用前一步的检索结果来确定下一步的检索目标。通过 Few-shot Examples 示范如何提出子问题、调用搜索工具以及根据检索结果继续追问。

5.3.2 提示链 Prompt Chains

提示链是将多次语言模型调用与外部函数调用按照一定顺序连接起来的工作流。

5.3.3 思维链与提示链的区别

维度	思维链（CoT）	提示链（Prompt Chains）
核心对象	推理过程	系统执行流程
主要形式	中间推理步骤	多次模型或工具调用
是否必须调用工具	否	否，但可以集成工具
中间结果	推理步骤或中间答案	结构化数据、文本或工具结果
主要作用	提升复杂推理能力	提高复杂任务的可管理性

两者可以结合使用：提示链中的某个模型调用可以采用思维链提示，而整个任务由多个调用共同完成。

总结： CoT 解决如何组织推理，PAL 解决如何借助程序计算，Problem Decomposition 解决如何拆分任务，Prompt Chains 则将这些步骤组织为可执行的工作流。

Part 6 多模态人工智能与内容生成 Multimodal AI and Content Generation

多模态人工智能能够处理不同形式的信息，包括文本、图像、音频和视频。

本部分主要讨论多模态输入输出、视觉理解的局限、图像提示工程以及生成模型的基本原理。

6.1 多模态输入与输出 Multimodal Inputs and Outputs

6.1.1 多模态系统的基本结构

传统文本模型主要处理自然语言，而多模态模型可以同时接收不同类型的信息，并生成一种或多种形式的输出。

多模态任务可以抽象为： $Y=f_{\theta}(X_{\text{text}},X_{\text{image}},X_{\text{audio}}) $

其中，f_θ 表示多模态模型，X 表示不同模态的输入，Y 表示输出结果。

6.1.2 多模态生成的价值

课件通过两个示例说明生成式 AI 在视觉表达方面的应用：

1. 从设计概念到可视化效果

通过生成图像，可以将蛋糕设计构想转化为具体的视觉参考，从而提高设计方案的可沟通性。

2. 生成现实中难以拍摄的场景

例如，生成一个人缩小到昆虫大小的画面。相较于静态图像，视频还包含时间维度上的变化，包括人物动作、物体运动以及场景演化。

因此，视频生成不仅需要保证单帧画面的合理性，还需要维持连续帧之间的一致性。

6.2 多模态理解的局限与图像提示策略

6.2.1 细粒度视觉识别的挑战

尽管多模态模型能够理解图像中的主要对象，但在识别外观高度相似的物体时，仍可能发生错误。

课件中的健身器械案例：图像整体语义理解能力不等于细粒度视觉辨别能力。对于具有相似外观的设备，模型可能忽略决定其功能的局部结构。

实际应用中，应重点关注物体的结构、连接方式、操作部件和功能差异，而不应仅依赖整体外观。

6.2.2 图像与文本的联合推理

课件以餐厅收据为例，说明多模态模型需要同时完成图像信息提取、文本条件匹配和数值计算。

这一案例反映出多模态任务可能同时包含三种误差来源：

1. 图像识别错误，例如读取了错误的商品价格。
2. 语义匹配错误，例如未正确识别用户消费的商品。
3. 数值计算错误，例如错误计算税费或合计金额。

因此，对于涉及票据、报表等结构化信息的任务，应将信息提取与数值计算分开处理，并对最终结果进行复核。

6.2.3 图像提示的实践原则

课件提出以下建议：

1. 利用图像文字识别能力：模型可以读取图像中的基本文字，例如收据、标牌和截图。
2. 注意视觉细节遗漏：对关键数字、细小文字和相似物体进行额外检查。
3. 必要时提供多张图像：使用不同角度、不同区域或不同时间的图像补充信息。
4. 明确每张图像的作用：说明各图提供的具体信息，减少模型对图像关系的错误理解。

其中，第四点尤为重要。例如，分析某个设备时，可以明确规定：图一展示设备整体结构、图二展示控制面板、图三展示接口与连接方式。这种方式能够提高输入信息的组织程度，使模型更容易围绕任务目标提取有效信息。

6.3 多模态生成的能力、时间与成本

6.3.1 多模态生成技术的发展

课件通过不同阶段的生成结果说明图像、视频和语音生成能力的发展。

视频生成方面，对比了：Imagen Video（2022）、Seedance 系列视频生成模型

语音生成方面，对比了早期与近期的 AI 语音生成效果。这些示例主要说明多媒体生成在视觉质量、动作表现和语音自然度等方面取得了显著进展。

6.3.2 不同模态的生成成本

课件按照典型生成时间与成本，从低到高给出以下排列： Text < Speech < Images < Video

这里的 C 表示课件中的典型成本趋势，而非适用于所有模型的严格不等式。

实际生成成本还取决于模型、输出长度、分辨率、采样设置及服务定价。

在工程实践中，可以先使用低成本的文本交互确定需求，再进行图像生成或视频生成，从而减少不必要的高成本调用。

6.4 图像生成提示工程 Image Generation Prompt Engineering

6.4.1 图像提示的核心组成

课件建议通过 AI 辅助编写图像生成提示，将简短需求扩展为包含具体视觉信息的描述。

例如，原始需求为：生成一只猫在夜晚秘密经营咖啡店的图像。

扩展后的提示包含三个主要组成部分。

组成	英文	主要内容
场景	Setting	时间、地点、环境、背景
角色细节	Character Details	主体外观、服装、动作
氛围与风格	Mood / Style	情绪、艺术风格、画面表现

例如：

- Setting：夜间咖啡店、温暖的室内环境。
- Character Details：橘猫、围裙、制作浓缩咖啡。
- Mood / Style：奇幻、温馨、卡通风格。

这一结构能够将抽象创意转化为更具体的视觉生成要求。

6.4.2 使用明确的视觉风格词汇

课件列举了四种典型风格：相比于“让图片更好看”等模糊要求，明确指定视觉风格更容易形成可执行的生成约束。此外，课件还建议让 AI 描述参考图像，将参考图中的构图、光线、色彩与风格转化为文字提示。

图像生成提示的扩展结构

在课件三个组成部分的基础上，可以进一步加入构图与限制条件：

- S：Setting，场景。

- C：Character Details，主体细节。
- M：Mood / Style，氛围与风格。
- K：Composition，构图与镜头。
- R：Restrictions，限制条件。

6.5 图像生成模型的基本原理

6.5.1 自回归生成与扩散生成

两种生成机制：

1. 自回归模型（Autoregressive Models）

自回归语言模型通常根据已有 token 序列，逐个预测后续 token。其联合概率可以表示为：

$P(x_1,\ldots,x_T) = \prod_{t=1}^{T}P(x_t\mid x_1,\ldots,x_{t-1})$

其中，x_t 表示第 t 个 token。

2. 扩散模型（Diffusion Models）

扩散模型通过多个步骤，将噪声逐渐转化为具有结构的图像。简化的反向生成过程为：

$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$

x_T：初始噪声；x_t：中间去噪状态；x_0：最终生成图像。

对于文本条件图像生成，模型根据文本提示对应的条件表示引导去噪过程，使最终图像符合用户描述。

简化流程：Text Prompt → Text Understanding → Latent Representation → Image Generation

文本理解模块负责处理提示中的词语及其关系，图像生成模块根据条件信息生成视觉内容。需要注意，扩散模型并非唯一的图像生成方法，实际系统也可能使用其他生成架构。

6.5.2 从低维潜在分布生成图像

课件进一步介绍了生成模型的一般思想：通过生成器，将低维潜在变量映射到高维图像空间。

设潜在变量为： $z \sim p_z(z)$

其中，p_z 可以是标准正态分布： $z \sim \mathcal{N}(0, I)$

生成器执行映射： $y = G_\theta(z)$

- z：低维潜在向量。
- G_θ：生成器。
- y：生成的高维图像表示。

生成器通过学习，将简单潜在分布映射为复杂的数据分布，从而生成具有不同外观与特征的图像。

6.5.3 生成对抗网络 Generative Adversarial Networks, GAN

GAN 由两个网络组成：

- Generator（生成器）：生成尽可能逼真的图像，以欺骗判别器。

- Discriminator（判别器）：判断输入图像来自真实训练数据还是生成器。

训练过程可以理解为生成器与判别器之间的对抗博弈。

其经典目标函数为： $\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log (1 - D(G(z)))]$

- $D(x)$：判别器认为真实图像 x 为真的概率。
- $G(z)$：生成器根据潜在向量 z 生成的图像。
- $D(G(z))$：判别器认为生成图像为真的概率。

生成器希望生成图像难以被判别器识别为伪造，判别器则希望提高真假识别能力。在反复训练过程中，生成器逐渐学习真实图像的数据分布。

GAN 与扩散模型的主要区别

维度	GAN	Diffusion Model
核心机制	生成器与判别器对抗训练	学习逐步去噪
生成过程	通常一次生成映射	通过多个去噪步骤生成
典型训练特点	对抗优化	去噪目标优化
主要挑战	训练不稳定、模式崩溃	采样计算成本等

Part 7 使用 AI 进行数据分析 Data Analysis with AI

本部分讨论如何结合语言模型、代码执行工具与数据可视化完成数据分析任务。

核心原则：模型负责理解分析目标、组织流程和解释结果，代码负责数据处理、数值计算及绘图。

7.1 AI 数据分析的基本流程

三个步骤：

1. Ask a clear question about your data：明确数据分析问题。
2. Use code to calculate and plot：使用代码计算并绘制图表。
3. Review the resulting insights：审查分析结果与结论。

这一流程强调，数据分析不能仅依赖模型生成的文字解释，而应建立在实际数据计算和结果验证的基础上。

7.1.1 明确分析目标

分析问题决定了需要使用的数据字段、统计指标和可视化方式。

例如：我的跑步配速和距离如何变化？该问题至少涉及时间、跑步距离和配速三个变量。

7.1.2 使用程序计算与可视化

课件展示了个人跑步记录的分析案例。

7.1.3 审查分析结论

数据分析完成后，应检查：

- 指标定义和单位是否正确。
- 统计结果是否与原始数据一致。
- 图表是否正确反映数据变化。
- 模型给出的解释是否具有数据依据。

尤其需要区分相关关系与因果关系，不能仅根据趋势同时变化就认定存在因果联系。

7.2 代码执行作为智能体工具

7.2.1 AI 工具的基本类型

课件将代码执行与其他工具放在同一个智能体框架中。

其中，代码执行工具主要用于需要精确计算或生成图表的任务。

7.2.2 工具调用与推理循环

智能体可以根据当前上下文判断是否需要使用工具，并将工具返回结果重新纳入后续处理。

这一流程将语言模型的推理能力与外部工具的执行能力结合起来。

7.2.3 商业销售数据分析

以小型企业销售数据为例，展示 AI 如何通过代码执行完成月度分析。

数据包含：Date：日期；Orders：订单数量；AvgOrderValue：平均订单金额；Revenue：营业收入

基本关系为： $R_i = N_i \times A_i$

其中，R_i 表示第 i 天营业收入，N_i 表示订单数，A_i 表示平均订单金额。

月度营业收入为： $R_{\text{month}}=\sum_{i=1}^{n}R_i $

模型可以通过代码计算月度营业收入、生成图表并识别异常值。

该案例展示了 AI 从自然语言问题到可执行数据分析流程的转换。

7.3 从数据中提取洞察 Getting Insights from Data

课件以饮品销售数据为例，要求识别销售变化最大的饮品并绘制图表。

模型执行的主要步骤为：

1. Inspecting the data：检查数据。
2. Calculating monthly changes：计算月度变化。
3. Identifying outstanding drinks：识别变化显著的饮品。
4. Plotting selected drinks：绘制选定饮品的趋势图。

图表同时使用 Others (avg) 作为其他饮品的平均销售基线，并标注 Spring Promo、Summer Push 和 Fall Launch 三个营销阶段。

销售变化的计算：设某饮品在月份 t 的销售额为 S_t。

绝对变化量： $\Delta S_t = S_t - S_{t-1}$

相对变化率： $r_t=\frac{S_t-S_{t-1}}{S_{t-1}}\times100\%$

其中，变化率仅在 $S_{t-1} \neq 0$ 时可直接使用。

绝对变化量反映实际销售额的变化规模，相对变化率反映相对于原有销售水平的变化幅度。

可视化不仅是展示数据，更应突出与分析问题相关的变化、异常和对照关系。

Part 8 人工智能的能力与责任 AI Capabilities and Responsibility

8.1 生成式 AI 的双重用途

随着生成式 AI 能力增强，技术应用不仅需要考虑功能实现，还需要考虑安全性、授权和潜在社会影响。

课件通过 AI 语音生成说明同一种技术既可以支持合理应用，也可能被用于欺诈。

应用场景	具体用途	风险性质
Podcast Editing	修复播客中的口误或音频问题	正当编辑用途
Character Voice Generation	为虚拟角色生成逼真的语音	创意内容制作
Voice Impersonation	模仿亲属声音实施诈骗	身份冒充与欺诈

语音生成本身并不能决定应用是否安全，关键取决于具体使用方式、是否获得授权以及是否具有误导或欺诈目的。

8.1.1 负责任使用 AI 的原则

在实际系统中，可以从以下方面降低风险：

- 知情同意：使用真实人物声音或形象时，应获得必要授权。
- 透明性：在适当场景中标明内容由 AI 生成或修改。
- 隐私保护：避免未经授权收集或处理个人敏感数据。
- 身份验证：对涉及资金、账户及身份的高风险请求进行独立验证。
- 人工监督：对可能产生重大影响的输出设置审核机制。

这一部分与前文的多模态生成形成对应关系：模型生成能力越强，越需要将安全约束纳入实际应用设计。

Part 9 使用 AI 构建应用 Building Apps with AI

本部分介绍如何通过自然语言提示，让 AI 辅助生成交互式应用程序。

课件强调将 AI 的代码生成能力与明确的应用需求结合起来，并通过简单项目逐步积累开发经验。

9.1 AI 辅助应用开发

AI 可以将用户描述的功能需求转换为程序代码，并组合界面、交互逻辑与算法。

应用类型包括：Game App：游戏应用、Study Timer：学习计时器、Quiz App：测验应用、Habit Tracker：习惯追踪应用、Expense Splitter：费用分摊工具。

应用开发的关键不只是让模型生成代码，还包括准确描述需求并验证生成结果。

9.2 AI 应用提示的三个基本要素

三个基本模块组织应用开发提示。

模块	英文	核心问题
Goal	What to create	要开发什么应用?
Input	What users provide	用户如何提供输入或操作?
Output	What the app shows	应用应该显示什么结果?

三者共同构成应用需求的基本描述框架。

9.2.1 示例：寻路游戏（Pathfinding Game）

用户希望构建一个可以放置障碍物与目标位置，并模拟路径搜索的游戏。

其需求可以拆分为：

- Goal：构建交互式寻路游戏。
- Input：用户在画布上设置障碍物、起点和终点。
- Output：显示搜索到的路径，并通过动画演示。

模型使用单个 HTML 文件实现交互式游戏，并使用 A* 算法进行路径搜索。

A* 算法的评价函数为： $f(n) = g(n) + h(n)$

- $g(n)$：从起点到当前节点 n 的已知路径代价。
- $h(n)$：从当前节点到目标节点的估计代价。
- $f(n)$：用于选择搜索节点的估计总代价。

当启发函数满足相应条件时，A* 可以找到最优路径。

9.2.2 示例：烟花模拟器（Fireworks Simulator）

用户要求：构建一个交互式烟花应用，点击屏幕后从底部发射烟花并产生爆炸效果。

其需求结构为：

- Goal：生成烟花模拟器。
- Input：用户点击或触摸屏幕。
- Output：显示彩色烟花的发射、爆炸与消散动画。

课件中的模型使用 HTML5 Canvas 与 JavaScript 实现烟花动画，并引入重力、摩擦和渐隐拖尾等效果。该案例说明，用户可以先描述应用目标与交互行为，再由模型选择适当的实现技术。但对于重要功能，仍需要在提示中明确约束，避免模型自行补充不符合预期的设计。

9.3 应用需求描述与验证

9.3.1 学习计时器（Study Timer）

课件要求创建一个包含以下功能的学习计时器：25 分钟专注时间。5 分钟休息时间。Start、Pause 和 Reset 控制。倒计时结束时显示完成消息。提供 10 秒测试模式，便于课堂验证完整流程。

该任务不仅定义了应用功能，还包含明确的测试要求。从软件工程角度看，测试模式可以帮助开发者快速验证状态转换、计时逻辑及完成反馈是否正常。

9.3.2 Flappy Bird 游戏

课件要求根据参考图片生成 Flappy Bird 游戏。

与学习计时器不同，该任务没有提供完整的文字规格，因此需要从参考图像和游戏规则中补充：游戏目标。玩家操作方式。障碍物与碰撞规则。得分机制。游戏结束条件。

这一案例说明，AI 辅助开发并不能消除需求分析的重要性。即使模型能够生成代码，仍需要明确预期行为与验收标准。

9.4 应用复杂度与开发策略

不同应用的实现难度存在显著差异，应优先从简单项目开始。

简单应用通常具有明确的规则、有限的状态以及较少的外部依赖。

复杂应用则可能涉及：网络通信与服务器。多用户状态同步。实时模型调用。响应延迟。API 成本。异常处理与系统可靠性。

因此，合理的开发策略是先完成最小可行产品（Minimum Viable Product, MVP），再逐步加入复杂功能。

Part 10 Take-home Points and Exercises

10.1 本讲核心知识总结

课件最后将整讲内容归纳为四个方面。

第一，借鉴人类学习策略提高智能体性能。

课程学习（Curriculum Learning）与奖励塑形（Reward Shaping）等策略可以改善智能体的训练效果。

第二，将预训练知识与外部搜索结合。

LLM 智能体能够利用已有知识，并通过搜索与研究工具获取当前、小众或复杂的信息。

第三，通过思维链与问题分解处理复杂任务。

将复杂问题转化为中间步骤，并在必要时结合程序计算与外部工具，提高任务处理的可验证性。

第四，利用提示工程完成实际应用。

通过明确目标、输入、输出和约束，AI 可以辅助完成图像生成、代码编写及简单应用开发。

从智能体能力提升到实际应用构建的整体逻辑：

$\text{学习与优化} \rightarrow \text{知识与工具增强} \rightarrow \text{推理与任务分解} \rightarrow \text{多模态生成与应用开发} $

10.2 课堂练习

练习一：DQN 的贪婪动作选择

问题：采用贪婪策略时，DQN 选择什么动作？

A. Estimated Q-value 最大的动作

B. Policy Sampling Probability 最大的动作

C. 上一次即时奖励最大的动作

正确答案：A。

DQN 根据估计的动作价值选择动作： $ a^*=\arg\max_a Q(s,a)$

练习二：应用提示的三个组成部分

问题：哪个提示同时包含应用目标、用户输入与输出？

A. 输入账单和人数，显示每个人应支付的金额。

B. 创建一个设计美观且实用的学生应用。

C. 创建一个使用最先进的 AI 的账单应用。

正确答案：A。同时明确了费用分摊目标、账单与人数输入，以及每人应付金额的输出。

练习三：模型推理详细但计算错误

问题：AI 给出了详细推理过程，但最终总额错误，应如何处理？

A. 因为推理详细而接受答案。

B. 检查单位并使用代码重新计算。

C. 要求模型提供更长的解释。

正确答案：B。详细推理不等于正确计算。对于可形式化的数值任务，应通过代码执行与结果复核提高可靠性。

建立一套完整的方法体系：通过推理与分解理解任务，通过代码与工具执行任务，通过多模态技术扩展输入输出形式，最终将这些能力组合为可验证、可交互的 AI 应用。

5583 Agentic Computing 2

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_5/knowledge/5583-Agentic-Computing-2.md`

5583 Agentic Computing2

Lecture 5 Prompt Engineering II

Part 1 Brainstorming

1.1 Roadmap

Brainstorming 上下文 + 选项 + 迭代

Context Management 上下文管理：上下文窗口里到底装了什么、什么是相关的、什么是干扰的。

In-Context Learning 上下文学习：不改参数，只靠提示里的示例改变模型行为。

Reasoning & Sycophancy 推理与谄媚：模型怎么想多步问题，以及它为什么会顺着你的话讲。

Writing & Revision 写作与修订：拆解 AI 味的来源，用大纲、逐段编辑、评分标尺来改进产出。

Tutorial: Prompt Engineering Practice 提示工程实践。

1.2 Modern Agent

Recap: from classical RL agents to modern LLM agents

从经典强化学习智能体过渡到以语言模型为核心的智能体。

Recap: several key concepts in prompt engineering

复习提示工程的几个关键概念，为第二部分的深入做铺垫。

Recap: GPT with next-token prediction

下一词预测是后面为什么采样能带来多样性的机制前提。

1.3 How people use GPT

AI 最常被用来做的是写和给建议，而创意发散只占很小一块分，这是一项被低估、但值得专门练的能力。

Writing：Editing text / Personal communication / Translation / Summarization / Writing fiction。

Practical Guidance：Learning / How-to advice / Health, fitness, beauty / Creative ideation。

其余并列大类：Finding info. / Technical help / Multimedia / Self-expression / Other。

1.4 Brainstorming with an AI agent

三个可复用动作

1. 上下文具体化：Providing more context 提供更多上下文，把可量化条件一次给全。
2. 限定输出形态：Give 3 plans 明确要三个方案而不是一段建议。
3. 反馈式迭代：Iterating multiple rounds（多轮迭代）I can’t stick to these 把问题说成我做不到而不是你的方案不好

$p(x_{t+1}|x_{1:t}, W)$

From next-token prediction $p(x_{t+1}|x_{1:t}, W)$ to varied responses through sampling。

W 是模型参数（权重）， $x_{1:t} $是已经生成的前缀， $x_{t+1} $是下一个 token。

模型输出的不是一个词，而是一个概率分布；从分布里采样（sampling）而不是取最大，就会得到不同结果。

Sampling can produce different responses 同一句提示问两遍，答案可能不同，原因就在这里。

注意 W 在这句里是固定不变的：多样性来自采样过程，不是来自模型更新。

1.5 Recap: GPT with next-token prediction

Given the previous tokens, the model tries to predict the next token。

训练目标是自回归（autoregressive）：给定前 t 个 token，最大化第 t+1 个 token 的似然。所以模型的一切行为，顺着你说、写 AI 味的句子、给出常见答案，都源于预测下一个最可能的 token这一件事。

理解了这个目标，才能理解为什么 prompt 有这么大影响（改变的是条件分布）。

1.6 How to encourage creativity?

给出上下文（决定在哪个区域采样）和多个选项（决定采几个点）

1. 只给上下文、不要多个选项。模型给一个看起来很合理的方案，没有比较的余地；
2. 只要多个选项、不给上下文。模型可能给的是分布中心的三个变体。

1.7 Brainstorming is iterative

三轮对话的节奏：发散 → 收敛 → 落地

第一轮给约束（4 人 / 10 分钟 / 下周五），产出 3 个结构不同的方案；

第二轮给反馈（不能线下见面 / 有人紧张 / 只有 20 分钟线上），产出 3 个针对新约束的方案；

第三轮做选择（Plan 4 sounds workable）并要求展开到日程级别。

注意第三轮的关键动作：先明确说保留什么（Keep the shared outline and short rehearsal），再要求细化，这样模型不会在细化过程中把前两轮谈定的东西改掉。

1.8 头脑风暴流水线　Brainstorming recipe

把前三节收成一条流水线，并给出AI 当思考伙伴的完整定义。

Context（上下文） 说清楚约束与现状；

Options（选项）一次要多个，不要一个；

Iteration（迭代）对选项给反馈，让模型在新约束下重出；

High-quality ideas（高质量想法）是这条链路末端的产物，不是第一步就能直接要到的。

Part 2 Context Management

2.1 上下文 Context

Definition：All the text and files the model uses to generate its response 模型用来生成回答的全部文本与文件。

没有上下文，即通用模型。问 Pros and cons of studying physics vs. zoology，若模型一无所知，只能给一页通用的利弊清单，它不知道你的成绩、时间表、职业倾向。但补上上下文即可得到定制回答，应用于Career assessment results、High school schedule、Course choice。

2.1.1 AI 的上下文 AI model’s context

AI context 系统提示在底部，对话历史往上叠，窗口自下而上包含：

组成部分	原文	说明
System prompt	Current date, model name and capabilities, general instructions（restrictions）	当前日期、模型名与能力、通用指令
Tool definitions	Available tools (like web search) and how to use them	可用工具（如联网搜索）及其用法
User prompt	Build workout plan for: 38 yo, beginner, dumbbells, 15 mins/day	
AI response	3 options: full-body, upper - lower split, low-impact strength	

Tips: The conversation is part of the context 对话窗口本身就是上下文的一部分

为什么同一句话在不同对话里效果不同？ User prompt 是叠加在 Chat history 之上被处理的，进入模型的完整输入不同，输出自然不同。

相关性：这条信息是否会影响本次任务的目标输出。

1. 反例：Irrelevant context can become distracting and may lead the AI to generate a worse answer 不相关的上下文会成为干扰，可能让 AI 给出更差的答案。
2. 解决：Start a new chat to reduce influence from the current conversation. 开新对话之后，Chat history 被清空，只剩 System prompt + Tool definitions + 本轮提问。想要一个不受前文影响的判断，就换一个干净上下文

2.2 提示 Prompt

2.2.1 用户提示 vs. 系统提示 User prompts vs. system prompts

角色	说明
system	A system prompt sets persistent and high-level instructions（系统提示设定的是持久、高层的指令，位置在最前面、对整段对话持续生效） 规定怎么回答：任务范围、输出格式、语言、风格、限制。
user	the user’s message（用户的消息） 规定回答什么：本次的具体输入。

2.2.2 聊天提示 Chat Prompts

The series of messages is turned into a string that uses special tags.

多条消息会被转成一个带特殊标记的字符串。

Tips：模型看到的始终是一段连续文本，system / user / assistant 的分界靠特殊标记（special tokens）表示。

2.3 系统提示 System Prompt

The intended answer follows the system instruction。系统提示不是建议，而是条件化输出分布的那个条件

Example：Anthropic Claude

1. 行为规范型

System prompts can contain many different rules describing how the model should behave. Such as Knowledge cutoff（知识截止时间）、Helps with prompting techniques（协助提示技巧）、Step-by-step reasoning（逐步推理）.

2. 安全与风格型

The system prompt can also be used to guide the model toward safer behavior and help it avoid harmful or inappropriate responses. Such as Avoid harmful requests（拒绝有害请求）、AI “self-esteem”（模型的自我评价/语气倾向）.

https://console.groq.com/playground 一个可以直接改系统提示、立刻看输出变化的在线试验台。

Example: basic English and proficient English

对照实验：同一个问题，只改系统提示里对语言水平的要求。用基础英语与熟练英语要求作为系统提示，对比同一请求得到的回答。

2.3.1 Functions of System Prompt

1. Can help to control the model’s behavior 帮助控制模型行为；
2. Specify the set of tasks 指定任务集合；
3. Desired properties of model responses 规定回答应有的属性（如语言）；
4. Output formatting 输出格式；
5. Condition the model’s output distribution 对模型的输出分布施加条件；

2.3.2 Downsides of System Prompt

Question：If two prompts mean exactly the same thing but differ slightly in formatting (e.g., spacing), should the model perform similarly?

两句语义完全相同、只有格式（如空格）不同的提示，模型的表现在理想情况下应该一样?

Answer：Prompt formatting matters in seemingly arbitrary ways

实验：用同一条提示的六种写法做对照，横轴是 Task Accuracy（任务准确率，0 到 1）。语义相同的几种格式，准确率从 0.036 跨到 0.804。

现象：刻度两端标注了极值：最低 0.036、最高 0.804，中间那段横条被标注为 Performance Spread Among Plausible Formats（各种看起来都合理的格式之间的性能跨度）。

结论：Even tiny irrelevant formatting changes can cause large performance differences 即使极微小、看似无关的格式变化，也能造成很大的性能差异。

1. No universal “best” format : format sensitivity depends on the model, task and prompt. 不存在通用的最佳格式：格式敏感性取决于模型、任务和提示本身。
2. For serious systems, test robustness and use structured interfaces：对正式系统而言，要测鲁棒性，并使用结构化接口（而不是把关键信息塞进自由文本的格式里赌稳定性）。

Part 3 In-Context Learning

3.1 上下文学习 In-context learning

An LLM exhibits a capability using the text supplied in its input context, without updating its parameters. 不改参数，只靠输入里的文本，模型就表现出新能力。

For complex tasks, ICL provides useful behavioral specifications!

对复杂任务而言，上下文学习提供了一种行为规格说明书。

Advantages：（cost）

Easy: use task examples without collecting a large task-specific training set.

容易：不必为一个任务收集大规模专用训练集；

No extra training: use one model for several tasks without extra training.

无需额外训练：一个模型跑多个任务；

Deployable: reuse one model across applications.

易部署：一个模型复用到多个应用。

3.1.1 Strategy 1: Text completion 文本补全（不给指令）

After pre-training, we have an autoregressive language model.

预训练之后，我们得到一个自回归语言模型

The language model can complete text: — 语言模型可以补全文本。公式写作 $x_{1:t} \to x_{t+1:T}$，即用前 t 个 token 生成后面的 token。

Prompt: "When a dog sees a squirrel, it will usually"

Output: "bark a lot, especially at close range.", "start to chase after it. When it does⋯"

3.1.2 Strategy 2: Zero-shot 指令提示

We can prompt the model to perform a specific task by providing instruction. Called “zero-shot” because we don’t provide (x, y) examples in the prompt.

通过给出指令，可以提示模型执行某个具体任务。之所以叫 zero-shot，是因为提示里没有提供任何 (x, y) 示例。公式 f(x; instruction) 里的两个部分：x 是输入、instruction 是指令，二者一起决定输出。

Considerations（可靠性 / 格式敏感性问题）

1. 输出格式不好对付 The model’s generated output formatting can be varied and tricky to deal with.
2. 表现随措辞变化 Performance can vary based on the prompt wording.

3. 基础模型可能不会听指令或只支持特殊格式 Base models may not be trained for instruction following, or trained only to support specific formats

3.1.3 Strategy 3: Few-shot （Instruction + examples）

Additionally provide (x, y) examples in the prompt 在提示里额外提供 (x, y) 示例。

公式中 K 是示例个数，x_test 是待预测的新输入，y_test 是输出。

Examples help the model understand the task format and expected outputs without any parameter updates. Referred to as “in-context learning”示例帮助模型理解任务格式与期望输出，且不涉及任何参数更新——这就叫上下文学习。它传递的不是知识，而是行为要求（格式、风格、推理颗粒度）。

Example: MATH dataset: a few-shot prompt used in Llemma

用 Llemma 模型在 MATH 数学题数据集上的 few-shot 提示原文。

Conclusion: ICL 擅长教行为（如用比喻解释），而不擅长塞知识。

3.1.4 Extension: Many-Shot In-Context Learning

变体	原文	说明了什么
ICL	input, reference output	标准做法：每个示例都带人写的或标注的参考答案
Unsupervised ICL	inputs + format guidance	只给输入和格式说明，不给参考答案
Reinforced ICL	model-generated outputs filtered for correctness	用模型自己生成的输出，过滤掉不正确的再当示例

In some experiments, inputs with format guidance outperform full demonstrations. 在某些实验里，输入 + 格式说明 的效果优于完整的示例。

原因：示例的两个作用要分开看，定位任务（靠输入+格式就能做到）与示范风格/颗粒度（往往需要完整示例）。如果提示效果不好，可以试着拆开：先给格式说明看够不够，不够再加完整示例。

3.2 示例数量与模型的影响 Effects of sample numbers and models

把示例数量从 2^0 一路加到 2^{10}，看四个模型的翻译准确率怎么变。

图例模型	Bemba 方向的大致走势	Kurdish 方向的大致走势
Gemini 1.5 Pro	整体最高，从 ~32% 一路上升到 ~48%	起始 ~39%，缓慢升到 ~43%
Claude-3-Opus	起始 ~35.5%，几乎持平，末端 ~39%	起始 ~38%，中间略降到 ~37.5%，末端回到 ~39%
GPT-4-Turbo	起始 ~26%，升到 ~32%	全程最低，约 ~31% 上下
Gemini 1.5 Flash	起始 ~24%，升幅最大，末端 ~40%	起始 ~39%，升到 ~42%

Ability to use many examples varies by model

能用多少示例这件事本身因模型而异。示例越多越好并不成立，收益取决于模型是否有能力把这些示例用起来。

3.3 不同任务的表现不同 Effects of different tasks

同一批模型、同一套示例数量，换任务之后曲线形状会变。不存在通用的最佳示例数量

Different tasks do not all follow the same pattern. 不同任务并不都遵循同一种规律。

3.4 上下文学习的建议 Tips for in-context learning

1. What ICL is good for

“Illustrate your explanations with examples, thought experiments, or metaphors.” and “Avoid being heavy-handed or repetitive.”

要求用例子、思想实验或比喻来讲解；避免说教或重复。

共同点：这两条要求的都是风格与做法（behavior），不是事实与知识（knowledge）。

2. What to be careful about

Knowledge tokens leave less room for conversation. 塞知识会挤占对话空间；

Long contexts make inference expensive. 长上下文让推理更贵；

New information must fit in the context window. 新信息必须塞得进上下文窗口。

Part 4 推理 Reasoning & Sycophancy

AI reasoning: using context and intermediate computation to solve multi-step problems. 推理即使用上下文与中间计算来解决多步问题。

4.1 长思考示例 Long thinking example

Example：购车比价场景，说明推理在智能体里指的是什么。

输入材料：Insurance plans（保险方案）、Spec sheets（规格表）、Car loan quotes（车贷报价）。

用户请求：What are the tradeoffs for each car? Read everything and **think hard before answering（显式要求先思考再回答）**

模型思考时做的四件事：Reading documentation / Searching for customer reviews / Defining evaluation criteria / Generating report。

最终输出：Here’s a detailed review of the tradeoffs among these three cars…

4.2 推理工作流程 How reasoning works

Example：行程规划，推理是反复循环的回路。

The reasoning process may involve repeatedly gathering information and thinking about it. 推理过程可能涉及反复地收集信息并加以思考。

4.3 AI长任务发展 AI progress on long-running tasks has exploded

4.4 鼓励推理 Encouraging reasoning

1. Use reasoning controls 使用推理控制
2. Give the model hard tasks 给模型难题。简单问题上，模型不需要长推理就能给出分布中心的答案

Example：Design a 12-month plan for a 4-person startup with limited cash为一家现金有限的 4 人创业公司设计 12 个月计划。

给模型的两样东西：Real job tasks（真实工作任务）与 All the context someone would need to complete the task（完成该任务所需的全部上下文）；

在旁标注提示语：Check your reasoning.（检查你的推理）。

4.5 AI 推理的四条建议 Tips for reasoning with AI

1. use the best models available 前提（模型能力）：使用可得的最强模型；
2. Give them as much context as needed 开关（推理预算）：给足所需的上下文；
3. Give them hard tasks 给难题；
4. Use reasoning settings when available 有推理设置就用。

4.6 AI 过度同意 Sycophancy

Example：

问法一：Don’t you think remote work is better than office work?

回答一：Yes, remote work offers many advantages…

问法二：Is it true that office work is more productive?

回答二：Yes, office environments provide strong benefits…

两次提问指向互相对立的结论，而模型两次都同意。

How you phrase the question may influence the agent’s response. 如何措辞提问，会影响智能体的回答。

两个问法都有共同的诱导特征：Don’t you think…（你不觉得……吗）与 Is it true that…（……是真的吗）都是预设了答案的引导式提问。

4.6.1 产生原因 Why does sycophancy happen?

Models are trained to be helpful assistants using human feedback, which can reinforce sycophancy 模型通过人类反馈被训练成有用的助手，而这可能强化谄媚

问题：How can model responses better match human preferences? 模型回答怎样才能更好地匹配人类偏好?

答案：Learn from human feedback (RLHF) 从人类反馈中学习（RLHF）。

逻辑链：人类偏好 → 标注者倾向给顺着我的回答高分 → RLHF 放大这个倾向 → 模型学会先同意。

4.6.2 难以察觉的陷阱 Sycophancy feels helpful but degrades quality

Example：

1. 明显

用户：I’m really proud of this essay, what do you think?

模型：Your essay is excellent! You clearly put in a lot of effort…

2. 难以察觉

用户：Analyze this data and find all the positive measures of performance this quarter.

模型：The data clearly shows revenue growth, strong retention, improving margins…

What bias does this prompt introduce?（这个提示引入了什么偏差？）

第一个例子里，用户的倾向写在情绪里；第二个例子里，倾向藏在任务定义里 find all the positive measures。模型在第二种情况下并没有违反指令，所以不会触发任何警报；但分析结论会系统性地偏乐观。

4.6.3 解决方案 Combating sycophancy

课件给出三条并列动作。

1. Neutral framing 中性框架。Avoid giving hints as to the answer you want to hear

换度量词：bad / good → to what extent（把是非题改成程度题）；

换来源：Do you agree → What does current research say（把立场题改成证据题）；

换动词：reduce / improve → compare（把方向题改成对比题）。

必要时再加 if at all / if any 这类让步短语。

引导式提问　Leading prompt	中性提问　Neutral prompt
Aren’t carbon taxes bad for small businesses?	To what extent, if at all, do carbon taxes affect small businesses?
Do you agree that AI will create a lot of jobs?	What does current research say about AI’s effect on jobs?
Doesn’t remote work reduce worker productivity?	How does productivity compare between remote and in-office work?

2. Explicitly ask for objective critique 明确要求客观批评。

Example：Be critical. Assume this needs improvement 它一定还有改进空间。

3. Start a new chat for a fresh opinion 开新对话以获得不受影响的意见。

Part 5 Writing & Revision

5.1 AI 味产生原因 What makes AI slop?

AI slop：AI content that looks good, but lacks substance (Vague, empty-sounding text) 看起来不错、但没有实质内容的 AI 产出。

AI 味 的两个可观测特征：破折号的使用频率，以及句子结构的固定套路。

四类可辨认的语言模式：

模式	例句	特征
Overused words（过度用词）	“This is a nuanced issue. Let’s delve into the implications…”	偏爱 nuanced、delve 这类词
Lists of three（排比）	“Clear, concise, and compelling”	强行凑三个并列形容词/短语

模式	例句	特征
Vague praise（空洞的赞美）	“This is a robustly structured and highly insightful paper…”	用高级形容词但不说具体好在哪
Not x but y（强行转折）	“It’s not just about speed—it’s about availability”	用一个否定-转折句式制造虚假深度

智能体可能影响人类 AI agent may influence human speech behaviours

例如，用 delve 这个词在播客与演讲里的使用频率，展示 AI 语言向人类口语的渗透。

5.2 去 AI 味

5.2.1 技巧一：用渐进式大纲写作 Write via progressive outlining

研究 → 3 个大纲 → 选一个并调整结构 → 展开成要点 → 局部补充 → 生成全文

1. 起点：给研究任务，不要直接要文章

不要一上来就让它做完整深度研究然后出稿。用户的原话是：

I’m writing an article about small AI teams moving faster than large teams that don’t use AI. Research **evidence for and against this hypothesis**.

2. 要三个结构不同的选项

Create 3 outline options. Include a counterarguments section. Also, here are some stories from teams I work with.

3. 选定后改结构，不动内容

Let’s use Option 1 and keep all the stories, but move the thesis after Story 1.,. I also want to add a historical analogy: Pixar created Toy Story in the 90s.

4. 大纲确认后才要正文

Expand each heading into bullet points based on the research.

5. 继续局部修改，最后才生成全文

Explain more about the startup. Tie in the Pixar story with the thesis.

Now, generate the text for this article.

5.2.2 技巧二：逐段编辑 Editing piece by piece

把改一篇文章拆成反复改一句/一段。

1. 先要几种不同的说法

Everyone talks about “human-level AI,” but what does that actually mean? Smarter chatbots? Automated office work?，用户要求 Brainstorm a few different ways to say this:

风格	改写
Punchy（有力/短促）	（课件在本列展示的是同一段的另一种紧凑写法）

风格	改写
Visionary（有远见式）	The public thinks achieving AGI means computers will be as smart as people. For decades, we’ve imagined AGI as machines that can reason and create like humans do.
Conversational（口语化）	A lot of people hear “AGI” and assume it means computers as smart as humans that can do most desk jobs.

2. 逐段推进，而不是整篇重写

5.2.3 技巧三：评分标尺 Well-written rubrics

把客观一点这个模糊要求，换成一个有分项、有分值的评分表。

1. Specify important criteria（明确重要标准）+ 2. Develop instructions for evaluating criteria（为每条标准写评价细则）+ Point system（分数体系）。

评价维度	分值	课件给出的评分细则示例
Characters	25 points	Every named character has a goal (10 points) / Conflict between two characters’ goals (10 points) / At least one character makes an irreversible decision (5 points)
Plot	25 points	（同上结构，细则按剧情维度展开）
World building	25 points	（同上结构）
Writing craft	25 points	（同上结构）

反面对照：同样有标尺，但写法不对，分数会被抬高。

1. Scoring early can anchor judgment 先给总分，后面的分项就会被调去凑这个总分
2. Vague categories invite subjectivity 没有细则，模型只能靠语感，而语感会偏向夸

对比项	5.9 节（可用）	5.10 节（会抬分）
分项细则	有（每条标准可判定、带小分值）	无（只有维度名）
给总分的位置	最后 sum the scores at the end（先分项、后求和）	最前 score it out of 100（先定总分）
自我指涉	无	story I wrote（暗示作者身份）
结果	65 / 100，指出人物目标无冲突	分数被抬高

5.2.4 技巧四：评分标尺 Cross-model review

换一个模型来评，避免自己评自己。

同样带标尺 + 分项打分 + 求总分，差别在于打分者不是写作者，切断自我一致性。

不同模型有不同的训练数据与偏好，能暴露出某一个模型特有的宽容，这正是一种鲁棒性测试。

Part 6 Summary

6.1 Summary: instruction prompts and in-context examples

提示的三种基本型态：

6.2 In-class exercises

第 1 题　中性框架

题干：You want to compare productivity in remote and office work. The prompt: Don't you agree remote work is better? invites agreement. Which replacement best reduces this leading framing?

- A. Explain why remote work improves productivity more than office work.
- B. Be critical: explain why office work is more productive than remote work.
- C. Compare productivity in both settings using evidence and its limitations.

第 2 题　上下文管理

题干：Your workout chat contains 15 minutes/day, dumbbells, no squats. You now want a plan for your mother, who has 30 minutes/day and no equipment. Which request manages context most clearly?

- A. In the same chat, ask only to make my plan longer for my mother.
- B. Copy the entire old chat and ask Adapt this for my mother.
- C. Start a new chat with her available time, equipment and preferences.

第 3 题　上下文学习的机制

题干：A model follows 4 examples, then loses the pattern in a fresh chat without them. Best explanation?

- A. The new chat reset the newly trained weights.
- B. The rule was stored but hidden in the new chat.
- C. The examples guided inference without training.

第 4 题　评分标尺

题干：AI gives your essay 92/100, then fits vague category scores to that total. Best prompt revision?

- A. Define criteria, score with evidence, then total.
- B. Keep the total and explain each category in detail.
- C. Ask for a stricter overall score before categories.

Scoring early can anchor judgment（先给总分成为锚点）与 Vague categories invite subjectivity（含糊分类引入主观性）。应该先定义标准、每项带证据打分、最后才求和。

Lecture 6: Retrieval-Augmented Generation (RAG)

Part 0 Orientation

0.1 Roadmap

本讲围绕一个具体的工程问题展开：当大语言模型自身掌握的信息不足以回答问题时，如何在生成答案之前，从外部知识源中检索出相关证据，并利用这些证据提高回答质量？将这个问题拆成五个部分。

Part	英文标题	中文主题	主要问题
I	Why RAG?	为什么需要 RAG	为什么不能只依赖模型参数中的知识?
II	RAG Architecture	RAG 架构	检索器、知识库与生成模型如何协作?
III	Typical Retrieval Methods	典型检索方法	如何从大量文档中找出相关信息?
IV	Retrieval Evaluation	检索评估	如何判断检索器是否找对了内容?
V	Tutorial: RAG in practice	RAG 实践	如何实现一个可运行的检索增强问答系统?

上一讲讨论如何通过 Prompt、Context Management、Reasoning 等方法控制模型的行为。本讲进一步追问：如果模型需要的上下文本身不存在于当前提示中，那么这些上下文从哪里来?

这就是 RAG 需要解决的问题。

两讲的关系可以概括为：

- Prompt Engineering：如何组织和使用已经拥有的信息。
- RAG：如何根据问题找到需要的信息，再将其组织进提示。
- Generation：模型如何利用提供的信息进行推理和回答。

需要注意，RAG 不等于 Prompt Engineering，但 RAG 通常会把检索结果放入提示，因此两者在实际系统中密切相关。

Part 1 Why RAG?

1.1 Three Ways of LLM Obtaining Information

How Does an LLM Obtain Information? LLM 获取信息的三种方式。

维度	参数知识	上下文信息	网页信息
来源	训练数据	当前输入	外部网络
是否依赖实时获取	否	不一定	通常需要
是否可以使用私有知识	取决于训练情况	可以	取决于授权与访问
主要限制	可能过时、不完整	上下文容量有限	质量、可靠性和安全性不一
是否必然属于 RAG	否	否	否

有外部信息，不一定就是 RAG。如果用户手动复制一段文章给模型，这属于上下文提供。**如果系统根据用户问题自动搜索知识库、选择相关片段，再将片段提供给模型，这才符合典型的 RAG 工作方式。**

1. **Stored in parameters** 数据经过训练过程，使模型获得大量通用知识。

Training Data（E-books, Books, Web pages, Articles） → Training → LLM

例如：为什么酒店周末更贵？模型可以根据已经学习到的供需关系，给出解释。但模型参数中的知识有一个重要限制：知识不会因为现实世界发生变化就自动更新。

知识截止时间 Knowledge cutoff 不意味着模型一定不知道此后的所有事实，也不意味着截止时间之前的事实都掌握得准确。它表示模型的预训练知识不能保证覆盖之后的新信息。

2. **Provided in context **模型除了使用已有知识，还可以利用当前输入中提供的信息

Chat history / Your documents / Notes → LLM

例如，用户给模型一段课程说明：Week 6 covers Retrieval-Augmented Generation. 然后询问：What is covered in Week 6? 即使模型原本不知道这门课程，课程内容出现在当前上下文中，也可以根据提供的文本回答。

3. **Found on the web** 通过网络获取

Search → Web pages → LLM

例如：What happened in Hong Kong last weekend? 模型可以通过外部搜索系统获得相关网页内容，再利用这些内容作答。但需要强调：

LLM 本身不一定具备联网能力。网页搜索通常由模型外部的工具或系统完成。

1.2 The Core Problem and Idea

Tips：为什么不能把所有外部知识都塞进上下文?

The problem with large external sources. Too much to put into context.

当知识源很大时，无法简单地把全部内容都放入模型输入。即使模型支持较长的上下文，也不代表把全部数据塞进去就是合理方案。主要原因包括：

1. 上下文窗口存在容量限制。
2. 输入越长，通常消耗越多计算资源与费用。
3. 大量无关内容可能干扰模型识别关键证据。
4. 每次重复发送完整知识库会产生额外开销。
5. 知识持续更新时，维护完整上下文十分困难。

解决方案：Retrieval-Augmented Generation (RAG).

核心流程：Large knowledge source → Retriever → Relevant information → LLM

不把所有知识交给模型，而是先找到与当前问题最相关的部分。LLM 可以从参数、上下文和外部工具获得信息。当外部知识规模很大时，RAG 通过检索选择相关内容，使模型能够在有限上下文中使用更有针对性的证据。

1.3 General Knowledge Answers

问题一：

Why are hotels more expensive on weekends? 为什么酒店在周末更贵?

More people travel, so there is more competition for rooms. 更多人出行，因此酒店房间的竞争更激烈。

可以直接回答，因为主要询问一般性的经济规律，模型在预训练过程中接触过大量类似知识，因此通常能够给出合理解释。其逻辑为：周末出行需求增加 → 酒店房间供给相对有限 → 供需关系变化 → 房价可能上涨。

如果一个问题能够依靠模型已有知识可靠回答，额外检索未必带来收益。

问题二：

What is Python for loop? 通常不需要专门搜索网页。

How does our company's internal Python framework implement task retries? 就可能需要检索公司代码库。

这类问题通常不需要外部检索，但模型仍可能进行一定程度的内部推理。

检索与推理不是同一个概念。

- Retrieval：寻找所需事实或证据。
- Reasoning：根据已有事实进行分析和推导。

是否需要 RAG，首先取决于回答问题是否需要模型当前尚未掌握的外部信息。一个问题可能不需要检索，但需要复杂推理；也可能需要检索，却只需要简单地提取事实。

1.4 Recent Information and Thinking Answers

Why were HK hotels so expensive last weekend? 为什么香港酒店上周末那么贵?

需要知道特定时间、特定地点发生了什么。这涉及两个不同的问题。

问题一：Knowledge Gap（知识缺口）模型没有掌握相关事实。

问题二：Reasoning Gap（推理缺口）模型掌握了事实，但不能正确推导结论。

RAG 主要针对第一个问题，通过外部检索补充事实。但 RAG 并不能保证第二个问题自动得到解决。

推理不能代替事实，如果模型没有核实就给出具体事件名称，就可能产生没有证据支持的回答。因此，合理的处理方式是先检索近期信息，再分析可能的原因。

时间一致性问题

还要检查 last weekend 对应的实际日期。如果问题发生在黄金周之后较长的时间，不能自动把黄金周作为那个周末的原因。RAG 系统不仅需要找到相关关键词，还需要验证检索结果的时间范围是否与问题一致。这属于工程中的 Temporal Relevance（时间相关性）。

总结

General knowledge gives us a starting point, but it is not enough to explain particular events.

通用知识为解释提供起点，但不足以解释具体事件。

Retrieval provides time-specific and more relevant evidence.

检索能够提供具有时间针对性、更加相关的证据。

1.5 RAG 的两个关键步骤

Two Critical Steps in RAG

1. Retrieval：Collecting useful information.

检索：收集有用信息。

2. Generation：Reasoning over that information and responding.

生成：基于这些信息进行推理并作答。

第一类问题：不需要专门收集外部信息，模型可直接依据已有知识回答。

第二类问题：需要先调查特定事件，再综合证据生成回答。

Retrieval 主要解决：应该把哪些信息交给模型?

Generation 主要解决：已经拿到这些信息，应该如何回答?

这是 RAG 系统最重要的职责划分。

假设用户询问：Week 6 的 tutorial 在哪间教室?

可能出现两种失败：检索失败，知识库中存在教室信息，但检索器没有找到；生成失败，检索器已经找到正确教室信息，但模型在回答时使用了错误的教室号。两种失败表面上都表现为回答错误，但修复方式完全不同。前者需要改善检索。后者需要改善提示、生成模型或答案验证。

1.6 Key Idea in RAG

LLMs store extensive information → LLMs can also benefit from an efficient retrieval phase.

LLM 已经存储了大量信息，但仍然能够从高效检索中进一步受益。RAG 通过高效检索弥补模型在近期信息和专业知识上的不足，使已有知识与外部证据相互补充。

两类需要外部补充的信息：Very recent information. Specialized knowledge.

1. Very recent information：近期信息

例如：最新发布的软件版本；今天的产品库存；最近发生的事件；最新的公司政策。

这些信息可能在模型训练之后才出现。因此，即使模型具有很强的语言理解能力，也不意味着它知道这些新事实。

2. Specialized knowledge：专业知识

例如：企业内部代码结构；某个医学研究领域的最新文献；特定项目的技术规范；专业法律文档。

这些知识可能很少出现在通用训练数据中，也可能受到访问限制。RAG 可以让模型在回答相关问题时使用经过授权的专业材料。

Great answer OR Not so good：模型在拥有足够相关知识时，可能给出较好的回答；面对近期或专业知识缺口时，如果没有适当的外部证据，回答质量可能下降。

OR 表示知识库是可插拔的：回答酒店问题需要网页，回答中国菜谱需要菜谱库，回答课程问题需要课程文档。工程含义是RAG 系统的检索器要对着一个特定的知识库来配。换库就要重建索引。

RAG 不是替代 LLM：LLM 已有大量知识，但在需要的时候可以通过检索获得额外帮助。因此，RAG 应被理解为增强机制，而不是完全替代模型原有能力。

1.7 检索器的工作流 The Retriever Workflow

Step 1: Search for a knowledge base of trusted information 在可信知识库中搜索信息。

knowledge base 知识库是用于保存和组织可检索信息的外部数据集合。它可以包括：课程文档、企业内部手册、项目源代码、产品资料、专业文献。

Retriever 负责接收查询、选择相关内容，查询与检索结果

Knowledge Base 负责存储可供检索的外部知识。检索器位于知识库上方，通过双向箭头表示查询与结果返回。

Trusted information 可信信息，检索系统不能只考虑内容是否相关，还要考虑信息来源是否适合被使用。但可信来源也不保证其中每一条内容都准确、完整或及时。

Step 2: Finds the most relevant information and shares it with the LLM 找到最相关的信息，并提供给 LLM。

第一步：寻找；第二步：传递。

Step 3: Improves generation 改善生成结果。

这是检索器存在的最终目的，但它描述的是预期收益，不是绝对保证。如果检索结果不相关、过时或错误，也可能降低生成质量。

Relevant 不等于 Correct

Retriever 的职责是从知识库中找出相关信息，但相关性、正确性和是否足以回答问题需要进一步区分。

Relevant：相关。表示某段文本与问题在词语或语义上有联系，相关性不等于可回答性，检索到不等于已经核实。

Correct：正确。表示信息本身准确，且能够支持所需结论。

1.8 RAG Definition

RAG combines a pretrained LLM with external knowledge retrieved at inference time to support generation.

RAG 将预训练语言模型与推理时检索到的外部知识结合起来，以支持答案生成。

RAG 的关键特征是在推理阶段检索外部知识，并利用检索结果支持生成，而不是要求模型事先记住全部知识。

Pretrained LLM：预训练语言模型

表示系统已经拥有一个经过训练的模型。它可以具备：语言理解能力、通用知识、文本生成能力、一定的推理能力。RAG 通常不需要从头训练一个新的语言模型。

External knowledge：外部知识

课件明确列举: Local documents, knowledge base, web search, APIs ... 即本地文档、知识库、网页搜索和 API 等。

这些来源有一个共同点：信息不必全部编码在语言模型参数中。

Retrieved at inference time：推理时检索

Inference time 指模型实际处理用户请求、生成回答的阶段。在典型 RAG 中，系统会根据当前问题，从外部来源寻找相关信息。这与训练阶段学习知识不同。

To support generation：支持生成

检索结果并不是最终答案。它们是用于支持答案生成的外部上下文。如果问题要求进一步解释或比较，模型还可以在证据基础上组织更复杂的回答。

RAG 与 Fine-tuning 的区别

维度	RAG	Fine-tuning
主要方式	推理时提供外部知识	通过训练更新模型参数
知识更新	更新知识库和索引	通常需要额外训练
适合任务	事实查询、动态知识、引用	行为适配、格式、特定任务能力
是否必然改变 LLM 权重	否	是
能否组合使用	可以	可以

实际系统可以同时使用 RAG 和微调。先微调模型，使其更擅长按照企业规范回答；再通过 RAG 检索最新企业文档。

某些知识广泛存在于公共数据中，而另一些知识很少出现，甚至根本不公开。因此，不能假设通用模型能够可靠掌握所有专业或私有信息。不过，互联网出现频率只是一个说明性维度，不能直接等同于模型对该知识的掌握程度。

1.9 Why RAG? Code Generation Example

RAG is important when we want to adapt an off-the-shelf LLM to a specific application.

当我们希望将现成的 LLM 应用于某个具体场景时，RAG 非常有用。

RAG 能让通用代码模型利用项目专属知识，提高代码生成和代码问答的针对性。

Off-the-shelf LLM

表示现成、可直接使用的模型。例如，一个通用代码模型可能知道：Python 语法、SQL 查询、REST API、常见软件设计模式。但它不一定知道：你的项目有哪些模块、项目采用什么内部约定、哪些函数已经实现、哪些接口刚刚发生变化。

Use your codebase as a knowledge base

把自己的代码库作为知识库。例如，代码库中存在：

```
class DatabaseManager:
def connect_to_db(self):
return self.connection
```

以及：

```
def process_user_data(user_input):
validator = CustomValidator()
return validator.validate(user_input)
```

这些函数体现了项目特定的结构与约定。

如果用户要求：请按照当前项目的规范实现一个新的数据库访问函数。通用模型可能生成语法正确、但不符合项目规范的代码。而 RAG 可以先检索：

- 现有数据库管理类。
- 项目使用的验证器。
- 已有错误处理方式。
- 相似函数的实现。

再把这些内容交给模型。

RAG 如何改善代码生成?

其作用可以分为三层。第一层：识别已有接口，避免模型凭空编造函数名称；第二层：学习项目约定，让生成代码更符合项目现有结构；第三层：利用最新代码，减少依赖已经过时的接口说明。

代码场景的三个典型缺口&为什么代码是最典型的 RAG 场景

缺口	表现	RAG 的作用
私有性	项目代码未公开	检索内部代码库
时效性	代码持续修改	获取较新版本
专业性	存在自定义模块与约定	提供项目特定上下文

公共语料足够丰富，加检索的收益很小。这也是代码库问答非常适合 RAG 的原因。

1. 完全私有（网上没有第二个一样的项目）；
2. 频繁变动（每天都可能改函数签名）；
3. 命名独特（自研模块名、内部约定，模型不可能见过）。

工程延伸：代码检索的特殊性

代码不仅包含自然语言，还包含大量精确符号：函数名称、类名称、文件路径、参数、依赖关系。

因此，代码 RAG 不能只依赖宽泛的语义相似性。

例如：`connect_to_db`与`create_database_connection`语义可能相近，但它们不一定是同一个真实接口。实际工程中通常需要兼顾精确匹配和语义匹配。

1.10 RAG for Companies 企业应用

企业 RAG 的价值不仅在于使用私有知识，还在于根据权限与时效要求，安全地提供相关业务信息。

1. Chatbots：企业客服机器人

Know about your company's products, current inventory, and common troubleshooting steps.

了解公司的产品、当前库存和常见故障排查步骤。

例如：Product A 现在还有库存吗？模型仅凭预训练知识无法可靠知道实时库存。企业 RAG 系统可以查询经过授权的库存数据，再生成回答。

又例如：我们公司的设备出现 Error 103 应该怎么处理？系统可以检索内部故障手册，找到对应的处理步骤。

2. AI Assistants：企业内部助手

Access and use private company data through internal or local services.

通过内部或本地服务访问和使用企业私有数据。

例如：检索内部制度、查询项目文档、总结企业报告、回答员工操作流程问题。

3. 三段式流程：Company knowledge → Retrieve evidence → Answer with context

即：企业知识 → 检索证据 → 结合上下文回答。

这个流程表明：企业知识不需要全部预先存储在模型参数中，系统可以根据用户问题按需获取。

工程延伸：权限控制

企业场景除了相关性，还必须考虑权限。假设企业有两类文档：普通员工可访问的产品说明和仅管理层可访问的财务报告。即使财务报告与用户问题高度相关，也不意味着检索器可以把它交给所有员工。因此，企业 RAG 需要满足：

也就是说，可被用户检索到的文档必须处于该用户的授权范围内。权限过滤应在向模型暴露内容之前执行，而不是仅要求模型在生成时自行保密。

工程延伸：知识更新

current inventory 信息的动态性。如果库存数据库每分钟更新，但 RAG 索引每天才更新一次，回答仍可能过时。因此，需要区分：数据源是否更新、检索索引是否更新、检索结果是否来自最新版本。

1.11 RAG for Web Search

Web-based RAG allows an LLM to retrieve the latest information from the Web and use it as external context when generating an answer.

基于网页的 RAG 允许 LLM 从网络检索最新信息，并将其作为生成答案时的外部上下文。

网页 RAG 让模型能够利用外部网页证据生成回答，并通过来源引用提高可核查性，价值包括时效性、覆盖范围和可追溯性。

Search the web → **Retrieve passages** → Answer with sources

搜索网页 → 检索相关段落 → 附带来源回答。

1.12 RAG for Specialized Knowledge

专业领域 RAG 的核心价值在于按需利用专业资料，并使回答能够接受更严格的证据核查。

1. Healthcare：医疗

Medical journals and authorized patient records. 医学期刊与经过授权的患者记录。

医疗领域具有几个特点：专业术语密集、研究持续更新、患者资料涉及隐私、错误信息可能造成严重后果。因此，医疗 RAG 需要特别重视证据质量和授权范围。

例如：医生希望查找某项临床研究的结论、系统可以检索医学文献，提供相关段落与来源。但不能仅因为系统使用了 RAG，就认为它的结论已经得到医学验证。

2. Legal Research：法律研究

知识来源：Case files and legal reference documents. 案件文件和法律参考文档。

法律知识具有：强烈的司法辖区差异、条文版本变化、具体案件事实依赖、较高的引用与核查要求。

例如：用户询问某条法律规定的适用条件。系统不仅需要找到相关条文，还需要确认其版本、司法辖区和适用时间。

两个领域的共同特征

维度	Healthcare	Legal Research
专业性	高	高
时效要求	医学研究持续更新	法律条文与判例变化
私有数据	患者记录	案件文件
来源要求	医学文献与授权记录	法律文件与权威资料

维度	Healthcare	Legal Research
主要风险	医疗误导、隐私泄露	法律误导、保密风险

工程延伸：高风险场景为什么需要更严格验证？

普通问答系统可能只关注：回答是否看起来合理？

高风险系统还需要检查：检索来源是否可靠、证据是否支持回答、信息是否过时、用户是否有访问权限、是否存在重要遗漏、是否需要专业人员复核。需要强调，具体法律合规义务取决于地区、用途和适用法规，不能直接从课件推出某个检索指标是法定合规要求。

1.13 Personalized RAG

More software includes personal assistants. These tools need context. Relevant context can improve results.

越来越多软件包含个人助理。这些工具需要上下文。相关上下文可以改善结果。

个性化 RAG 使通用模型能够在获得授权的前提下，利用与用户相关的动态信息提供更贴合具体情境的回答。

个人数据的四种来源

- Calendar Events：日历事件。
- Emails：电子邮件。
- Contacts：联系人。
- Text Messages：文本消息。

模型的通用能力负责理解与总结，用户数据提供具体事实。

与企业 RAG 相比，个性化 RAG 的数据具有：强个人相关性、高频更新、多来源分散、权限敏感、时间关联明显。

工程延伸：个性化不等于无限制访问

个人助理不应该因为拥有某个数据连接，就无条件读取全部个人数据。系统应根据具体任务和用户授权确定访问范围。例如，回答日程问题通常不需要读取所有私人邮件。这涉及最小权限原则：只访问完成当前任务所需的数据。

维度	企业 RAG	个性化 RAG
知识主体	企业	个人
数据来源	内部文档、业务系统	日历、邮件、消息、联系人
权限依据	企业角色和访问控制	用户授权与服务权限
典型任务	企业问答、内部知识助手	日程查询、个人信息整理
共同机制	检索相关数据并用于生成	检索相关数据并用于生成

Part 2 RAG Architecture

2.1 The RAG Architecture RAG 的整体架构

上路由（Direct LLM use） Prompt 直接进 LLM，得到 Response。这是没有 RAG 的基线。

下路由（RAG System） Prompt 先被 Routes it to the retriever（路由给检索器），再由 Retriever 从 Knowledge Base 取回 Relevant Documents。

1. Direct LLM use：直接使用语言模型。

用户提供 Prompt，LLM 根据已有参数知识和输入上下文生成 Response。

例如：Why were Hong Kong hotels expensive last weekend?

如果没有外部检索，模型只能利用当前已知的信息回答。

这会带来一个风险：模型可能掌握一般性的酒店供需规律，却不知道特定周末香港发生了什么。因此，直接调用模型并非一定错误，而是可能缺少回答特定问题所需的外部证据。

2. RAG System：先检索外部知识，再支持模型生成。

Prompt → Retriever → Relevant Documents

同时：Retriever ↔ Knowledge Base

组件	中文	职责
Retriever	检索器	接收查询，并判断知识库中哪些文档与问题相关
Knowledge Base	知识库	存储外部知识，例如课程资料、公司文档、网页或代码
Relevant Documents	相关文档	检索阶段的输出，后续会被用于构造模型输入

Routes it to the retriever：将问题路由给检索器

这里的 Route 指的是数据流向，而不一定意味着系统已经实现了一个复杂的智能路由算法。

原笔记写：是否检索由系统决定，不由用户决定。

需要调整为：在图示的 RAG 路径中，系统负责把 Prompt 传递给 Retriever；但课件没有进一步说明是否存在动态判断需要检索还是直接回答的路由器。

在实际系统中，可以有不同设计：

- Always Retrieve：每个问题都执行检索。
- Conditional Retrieval：根据问题决定是否检索。
- User-triggered Retrieval：由用户操作触发检索。

两条路径的真正区别

RAG 的核心区别在于：生成模型能够使用根据当前问题检索到的外部证据。

- q：用户问题。
- K：知识库。
- R：检索过程。
- D_q：与问题相关的检索结果。
- G：生成模型。
- A：最终回答。

2.2 Building the Augmented Prompt 构造增强提示

RAG 的增强不是直接改变模型参数，而是在生成前把检索证据组织进模型输入。

Why were Hong Kong hotels expensive last weekend? 为什么香港酒店上周末那么贵?

1. 从 Relevant Documents 到 Augmented Prompt

Prompt → Retriever → Relevant Documents → Augmented Prompt

这意味着检索结果不能只停留在检索器中。它们必须以某种方式提供给生成模型。通常做法是把原始问题、检索片段和任务指令组织成新的 Prompt。

其中：

- P_{aug}：Augmented Prompt。
- I：Instructions，任务指令。
- $C_{\text{retrieved}}$：检索得到的上下文。
- Q：原始问题。

加号在这里表示文本或结构化消息的组合，而不是数值相加。

2. 提示示例

Answer the hotel question using the passages below. ... 使用下面提供的段落回答酒店问题。

这句话包含三个功能：

第一：明确任务。模型需要回答酒店价格问题。

第二：指明信息来源。模型应关注下面提供的段落。

第三：建立问题与证据的联系。检索结果不是独立展示的资料，而是用于支持当前回答的上下文。

using the passages below 不严格等于 using only the passages below。前者要求使用给定段落，后者才明确限制只能使用给定段落。因此，不能仅凭这一句认定课件已经要求模型完全禁止使用其他知识。

一个完整的增强提示示例

```
Task:
Answer the user's question using the provided context.
If the context is insufficient, explain the uncertainty.

Retrieved Context:
[Document 1]
National Day holiday weekend.

[Document 2]
Higher visitor demand expected.

User Question:
Why were Hong Kong hotels expensive last weekend?
```

模型可以据此回答：香港酒店价格较高可能与国庆假期期间的游客需求增加有关，但要确认具体涨价幅度及原因，还需要酒店价格和入住率等数据。这种回答将外部证据与已有经济知识结合，同时保留了证据不足时的不确定性。

Augmented Prompt 不等于普通 Prompt

维度	Original Prompt	Augmented Prompt
用户问题	包含	包含
外部检索证据	通常没有	包含
任务指令	可以包含	通常包含
内容长度	相对较短	通常更长
知识来源	模型已有知识和用户输入	额外加入检索结果

工程延伸：为什么 Prompt Construction 很重要?

即使检索器找到了正确文档，如果提示组织不当，模型也可能无法正确利用。例如，存在以下问题：证据与问题混在一起，难以区分；多份文档之间没有来源标记；文档包含相互矛盾的信息；无关片段占据大量上下文；检索文本包含恶意指令。因此，实际 RAG 系统通常需要专门的 Prompt Construction 模块。它负责把检索结果整理为模型容易理解、可追溯且安全的输入结构。

2.3 Generating the Response 生成最终回答

RAG 通过检索增强提示后再生成答案，但这种增强通常需要额外的系统计算与时间成本。

National Day holiday weekend; higher visitor demand expected. 国庆假期周末；预计游客需求较高。

这段上下文提供了两类信息：

- 时间背景：National Day holiday weekend。
- 需求背景：Higher visitor demand expected。

但它并没有直接提供酒店价格数据。因此，模型可以据此提出合理解释，却不能仅凭这段信息确认某家酒店的涨价幅度。

Added Latency：额外延迟

这是 RAG 的重要工程代价。相较于直接调用模型，RAG 通常增加了：

1. 检索查询时间。
2. 检索结果处理时间。
3. 增强提示构造时间。
4. 可能增加的模型输入处理时间。

RAG 通常引入额外的检索开销，但不能保证在所有实现、所有请求下都比直接生成更慢。

工程延伸：如何降低 RAG 延迟?

方法	工作原理	潜在代价
缓存	复用已有检索结果	可能出现过期数据
调整 Top-K	减少进入模型的片段	可能遗漏证据
优化索引	提高检索效率	需要索引维护
并行检索	同时查询多个来源	增加资源使用
压缩上下文	减少输入 token	可能丢失信息

关键词检索并非在所有场景下都比向量检索快。速度取决于索引结构、规模、硬件和具体实现。

2.4 Advantages of RAG RAG 的五项优势

RAG 的优势不仅在于提高回答质量，还包括知识更新、来源追溯和模块化维护；但这些优势都是有条件的，而不是自动保证。

课件原文	中文解释	核心机制
Adds missing knowledge	补充缺失知识	外部知识弥补模型知识缺口
Updates knowledge without retraining	不重新训练即可更新知识	更新数据源与检索索引
Supports source citation	支持来源引用	保留检索结果的来源信息
Separates retrieval from generation	检索与生成分离	模块化设计
Helps alleviate hallucinations	有助于缓解幻觉	提供与问题相关的证据

1. Adds missing knowledge：补充缺失知识

例如，模型不知道某个企业的最新产品规格。RAG 可以从企业知识库检索规格文档，再用于回答。

它解决的是知识不足的问题。但前提是外部知识源确实包含所需内容，而且检索器能够找到。

2. Updates knowledge without retraining：不重训即可更新知识

传统模型的参数知识通常依赖训练过程。RAG 可以在不修改生成模型参数的情况下，更新外部知识。

例如：旧文档：Product A warranty: 12 months

新文档：Product A warranty: 24 months

系统更新数据源和必要的索引后，就可以检索新的保修政策。但这不意味着 RAG 自动拥有最新知识。如果索引没有同步更新，仍可能检索旧内容。此外，并非每次更新都必须完整重建索引；一些系统支持增量更新。

3. Supports source citation：支持来源引用

RAG 可以保留文档的来源信息，例如：文件名称、URL、文档 ID、页码、片段位置。回答时可以展示引用，帮助用户核查。但要区分：能够引用来源与引用正确支持答案。前者是系统功能，后者需要额外验证。

4. Separates retrieval from generation：检索与生成分离

这是重要的工程优势。当回答错误时，可以分别检查：

- Retriever 是否找到了正确证据?
- Generator 是否正确使用了证据?

例如，检索结果正确但回答错误，问题可能出在生成阶段。反过来，如果检索器根本没有找到关键文档，单纯修改生成提示可能无法解决根本问题。这种模块化设计使系统更容易测试、替换和优化。

5. Helps alleviate hallucinations：缓解幻觉

检索证据能够降低模型在缺少信息时随意补充事实的风险。但它不能彻底消除幻觉。因为模型仍可能：

- 误读证据。
- 引用不相关片段。
- 混合不同来源的事实。
- 将推测表述成确定结论。

- 使用错误或过期的检索内容。

2.5 Recap: Autoregressive Generation 回顾自回归生成

New tokens depend on the tokens already generated. 新生成的 token 依赖此前已经生成的 token。

模型并不是一次性直接输出整个答案，而是逐步生成 token。

Sampling：采样

With sampling, the same prompt can produce different completions.

使用采样时，相同的 Prompt 可以产生不同的续写。这两个续写都可能符合语法与语义。因此，模型生成具有概率性，而不必然是固定输出。

Sampling 与 Greedy Decoding 的区别

Greedy Decoding：每一步选择概率最大的 token。

Sampling：按照概率分布采样 token。

采样能够提高输出多样性，但也可能引入不稳定性。

为什么自回归机制与幻觉有关?

自回归模型的基本目标是生成符合其学习到的语言分布的后续 token。但语言上的合理性不等于事实正确性。

例如：A marathon doubled hotel prices.

这句话在语法上完全成立，也符合某种可能的经济因果关系。但它是否真实发生，需要额外证据。因此，自回归生成机制本身不保证输出事实正确；事实核查需要依赖模型能力、提示策略或额外验证机制。

2.6 Hallucinations in LLM Outputs LLM 输出中的幻觉

Factuality 关注回答是否符合现实事实；Faithfulness 关注回答是否受到给定上下文的支持。

Generated content that is factually incorrect or unfaithful to the supplied context (unsupported).

生成内容在事实上不正确，或者不忠实于给定上下文、缺乏上下文支持。

1. Factuality：事实性

Does the claim agree with verifiable real-world facts? 这个说法是否符合可验证的现实世界事实?

例如：A marathon doubled hotel prices. 一场马拉松使酒店价格翻倍。

判断事实性时，需要检查：

- 是否确实发生了马拉松?
- 酒店价格是否真的翻倍?
- 是否存在支持两者因果关系的证据?

如果缺少这些验证，就不能认定该说法为真。

2. Faithfulness to Context：对上下文的忠实度

Do the supplied passages support the claim? 提供的段落是否支持这一说法?

假设检索内容只有：National Day holiday weekend. Higher visitor demand expected.

但模型回答：A marathon doubled hotel prices. 这段回答没有得到给定上下文的支持。

因为上下文没有提到：

- 马拉松。
- 酒店价格翻倍。
- 马拉松与价格之间的因果关系。

因此，它属于 unsupported answer（缺乏证据支持的答案）。

3. Factuality 与 Faithfulness 的区别

	Faithful：有上下文支持	**Unfaithful：无上下文支持**
Factual：事实正确	事实正确且有证据支持	事实正确，但不是由给定证据支持
Non-factual：事实错误	忠实复述了错误资料	事实错误且无证据支持

RAG 能改善哪个维度?

RAG 能为模型提供外部证据，从而帮助改善 Faithfulness。同时，如果外部证据准确、及时，RAG 也可能改善 Factuality。但两者都不是自动保证的。因此，更准确的结论是：RAG 可以同时帮助事实正确性和上下文忠实度，但必须结合检索质量、来源质量和生成行为进行判断。

2.7 Grounding Responses with RAG 用 RAG 为回答提供依据

RAG 通过在生成前提供相关证据，使模型有机会给出更加有依据的回答faithfulness，但证据使用是否正确factuality仍需验证。

Relevant context “grounds” the LLM's responses. RAG adds evidence before the model answers.

相关上下文为模型回答提供依据。RAG 在模型回答之前加入证据。

Grounding 让模型回答与可识别的外部证据建立联系。

用户问：Why were Hong Kong hotels expensive last weekend? 没有证据时，模型可能提出各种猜测。

加入相关证据后：National Day holiday weekend. Higher visitor demand expected.

模型就可以围绕假期需求解释酒店价格。

Grounding 的三个环节

1. Retrieval 找到与问题相关的证据
2. Context Construction 把证据组织进模型输入
3. Evidence-aware Generation 要求模型参考证据生成答案

Grounding 与 Citation 不一样

Grounding 关注：回答是否由证据支持?

Citation 关注：回答是否标出了来源?

例如：香港酒店价格上涨是因为国庆假期。[Source 1]

即使附有 Source 1，也必须检查 Source 1 是否真的支持这个因果结论。

因此：有引用不等于有充分证据支持。

Grounding 是否保证不会幻觉?

不能。模型可能忽略部分证据、错误理解检索结果、引入证据之外的细节、将相关性误认为因果关系。

因此，Grounding 是降低无依据生成风险的机制，不是正确性的数学保证。

2.8 RAG 的计算限制

上下文窗口与计算资源都有限，因此 RAG 必须在信息覆盖范围和上下文成本之间进行选择。

1. Higher Computational Cost：更高计算成本

Longer prompts require more computation and memory.

更长的 Prompt 通常需要更多计算与内存资源。

Long contexts can still increase latency.

长上下文仍然可能增加延迟。

例如：输入 2,000 tokens 与输入 200,000 tokens，模型需要处理的上下文规模明显不同。即使模型支持长上下文，也不代表处理成本可以忽略。

2. Context Window Limit：上下文窗口限制

长上下文通常增加计算与内存压力，但具体增长方式取决于模型架构、推理阶段和实现。

Prompt and answer share a limited context budget.

Prompt 与 Answer 共享有限的上下文预算。

Capacity depends on the model and configuration.

为什么检索比全部输入更合理?

假设知识库包含 10,000 个片段。用户的问题只需要其中 5 个。全部输入意味着大量无关内容进入上下文。选择性检索则只需要返回相关片段。这样可以降低上下文占用。

但也引入新问题：如果检索器没有选对片段怎么办? 这正是后续 Retriever Tradeoffs 和 Retrieval Evaluation 要解决的问题。

2.9 Example

How can I find some recipes to make Shanghai-style baozi at home? 怎样找到在家制作上海风味包子的食谱?

图书馆拥有大量书籍，但用户不可能逐本阅读。因此，需要一种高效的查找机制。

组成部分	课件原文	作用
Collection	Books on many topics	收藏不同主题的书
Organization	Different shelves and sections	按书架与区域组织
Search	A librarian helps you find the best sections or books	图书管理员帮助查找

图书馆	RAG 系统	工程作用
Books	Documents	存储原始知识
Shelves and sections	Index	组织可检索信息

图书馆	RAG 系统	工程作用
Librarian	Retriever	根据问题查找文档

检索系统不仅需要信息集合，还需要组织信息与查找信息的机制。

Organization：索引

如果文档没有组织结构，查找可能非常低效。索引可以帮助系统快速定位候选内容。

索引是提升检索效率的重要工具，但不是所有检索算法在逻辑上都必须依赖索引。

不同索引的组织方式不同。常见索引包括：

- Inverted Index：倒排索引。
- Vector Index：向量索引。
- Metadata Index：元数据索引。

Search：检索

用户提出问题后，Retriever 根据查询寻找相关文档。

例如：

问题：How can I make Shanghai-style baozi?

检索器可能优先返回：Baozi Basics

而不是：Shanghai History

What's the Role of a Librarian?

1. Understands the meaning of your question 理解问题的含义。

例如：How can I make Shanghai-style baozi?

系统需要识别用户希望寻找的是制作方法，而不是上海的历史介绍。

在检索系统中，这通常涉及 Query Understanding（查询理解）。

2. Identifies the right shelves to search 确定应该查找哪些书架。
3. Eventually finds relevant books 最终找到相关书籍。

图书管理员行为	检索系统概念
理解问题	Query Understanding / Representation
定位书架	Candidate Selection / Filtering
找到相关书籍	Retrieval / Ranking

需要注意，课件是在用人类图书管理员进行概念类比。它并不意味着所有 Retriever 都必须严格分成三个独立模块。例如，一个向量检索器可以直接把查询编码为向量，再进行近邻搜索。查询表示与候选选择可能在同一套检索过程中完成。

优秀的检索器需要根据问题选择合适的候选内容，并将相关结果返回给后续生成阶段。

2.10 Ranking Retrieved Documents 检索文档排序

检索器不仅需要找到候选文档，还需要对它们排序；但检索分数代表算法定义的匹配程度，不等于事实正确性或可回答概率。

How can I find some recipes to make Shanghai-style baozi at home? 检索器返回四个结果。

Ranking 是将候选文档按相关性得分排序。检索器通常希望相关文档的得分高于不相关文档。

检索器需要考虑：What does the prompt mean? 问题表达了什么含义？ 以及 What documents in the knowledge base are similar? 知识库中哪些文档与问题相似？ 这两个问题体现了查询理解与文档匹配。但它们不能严格对应为两条互斥的技术路线。关键词检索也需要查询处理，语义检索也需要计算相似度。

检索分数是不是概率？ 不是必然。

某篇文档即使与问题存在部分相关性，也不一定包含回答问题所需的信息。

相似度较高，不代表文档足以回答问题。

2.11 Retriever Tradeoffs 检索器的取舍

检索结果过少可能遗漏证据，过多可能引入噪声。系统需要通过指标和实际测试选择合适的检索配置。

检索系统的核心权衡：应该返回多少文档？

1. Relevance vs Irrelevance: Need to return relevant documents and withhold irrelevant ones.

应该返回相关文档，并尽量排除不相关文档。这看起来很简单，但实际并不容易。因为检索器无法保证每个评分都完全符合真实相关性。

2. Return Every Document? Irrelevant documents waste the context window.

如果返回全部文档：优点：可能减少遗漏。缺点：大量不相关文档占用上下文窗口。

3. Return the Single Highest Ranked Document?

如果只返回排名第一的文档：优点：上下文更短，输入成本更低。缺点：可能遗漏其他有价值的信息。

4. Top-K 的基本思想

Top-K 表示返回评分最高的 K 个结果。K 越大，通常能够覆盖更多候选信息，但也可能增加噪声和上下文成本。

5. Precision 与 Recall 的权衡

Precision：表示返回的结果中，有多少是真正相关的。

Recall：表示全部相关文档中，有多少被成功找回。

6. No Perfect Solution：没有完美方案

No perfect solution: Scoring and monitoring. Change settings to find what works.

通过评分、监控和调整配置，寻找适合实际任务的方案。这意味着 Top-K 并不存在一个对所有系统都最好的固定值。

7. 工程延伸：为什么不能只追求 Recall?

如果为了找回全部相关文档而返回大量候选内容，Recall 可能上升，但无关片段也可能增加。

这会导致：上下文成本上升、模型更难定位关键证据、可能出现证据冲突、回答质量不一定提高。

因此，检索系统的目标不是简单地让 K 越大越好，而是在覆盖率、精确率、延迟和生成质量之间取得平衡。

2.12 RAG 的改进

RAG 建立在成熟的信息检索技术之上，并通过语言模型将检索结果转化为面向用户的生成式回答。

信息检索并不是随着 LLM 和 RAG 才出现的技术。

1. Search Engine：搜索引擎

Retrieves relevant webpages. 检索相关网页。

2. Database：数据库

Retrieves relevant tables and rows. 检索相关的数据表和数据行。数据库根据查询条件返回符合要求的记录。

需要注意，数据库中的精确条件查询与搜索引擎的相关性排序并不完全相同，但两者都涉及从大量信息中定位所需内容。

3. Information Retrieval 是成熟领域

Information retrieval was already mature before LLMs, and ideas from this field underlie the design of retrievers and RAG systems.

在 LLM 出现之前，信息检索已经是一个成熟领域，而该领域的思想构成了 Retriever 和 RAG 系统设计的重要基础。

RAG 继承了哪些经典技术?

技术	主要作用	与 RAG 的关系
TF-IDF	词项重要性加权	关键词检索
BM25	文档相关性排序	稀疏检索
Inverted Index	快速定位包含词项的文档	检索索引
Cosine Similarity	比较向量方向相似性	稀疏与稠密向量检索
Precision	衡量返回结果的准确程度	检索评估
Recall	衡量相关结果覆盖程度	检索评估
MAP / MRR	衡量排序检索表现	排序评估

需要区分，向量相似度方法可以与经典信息检索技术结合，但现代神经嵌入模型的发展也引入了新的检索能力。

RAG 真正增加了什么?

传统搜索引擎主要返回：相关文档或网页。

RAG 在此基础上进一步执行：将检索结果提供给生成模型，让模型综合信息生成自然语言答案。

RAG 的创新重点不是从零发明检索，而是将检索机制与语言生成系统结合。

Part 3 Typical Retrieval Methods 典型检索方法

3.1 The Retrieval Challenge 检索的挑战

Retriever Goal: find documents in the knowledge base that can help an LLM respond to a prompt.

从知识库中找到能够帮助 LLM 回答用户问题的文档。

角色	课件定义	实际困难
Prompts	Unstructured conversation	用户表达自由、口语化
Retriever	Find relevant documents quickly	必须同时兼顾相关性与速度
Documents	Many formats written for humans	文档格式复杂、表达不统一

1. 用户问题不一定与文档使用相同的词。

例如，用户问：How can I make baozi at home? 文档可能写：Use flour and yeast for Shanghai-style baozi dough. 用户关心的是制作方法，但文档不一定包含完整的提问表达。

2. 检索命中不等于文档能够回答问题。

如果文档写的是：Shanghai baozi shops offer home delivery. 它虽然包含 baozi 和 home，却没有提供制作包子的方法。因此需要区分两个概念：

- Lexical Match（词面匹配）：查询词是否出现在文档中。
- Relevance（相关性）：文档是否真正有助于回答问题。

这正是后面引入语义检索的原因。

3.2 检索器内部

How can we efficiently search the knowledge base?

这里的 efficiently（高效）不仅是运行速度快，还意味着能够从大量文档中优先找出相关内容。

Retriever 本身并不负责生成最终答案，而是为 LLM 提供更有用的上下文。

因此，检索质量会直接影响后续生成质量：

- 检索遗漏关键文档，LLM 可能缺少必要信息。
- 检索返回大量无关文档，有限的上下文窗口可能被噪声占用。
- 检索返回高相关文档，LLM 更容易生成有依据的回答。

3.3 Two Search Approaches 两种搜索范式

	Keyword Search	Semantic Search
中文	关键词检索	语义检索
核心依据	Exact words	Similar meaning
关注的问题	词是否相同?	意思是否接近?
典型方法	TF-IDF、BM25	Embeddings
主要优势	精确词匹配	同义表达匹配
主要局限	难处理同义词	可能忽略精确标识符

查询：How can I make baozi?

文档 A：Instructions for preparing steamed buns.

文档 B：Baozi shops provide home delivery.

关键词检索可能更偏向 B，因为它直接包含 baozi。语义检索则可能更偏向 A，因为 preparing steamed buns 与制作包子在含义上更接近。两种方法的差异，不是有没有向量，而是向量表示的是什么。

关键词检索用向量表示词项出现情况；语义检索用模型学习出的向量表示文本含义。

3.4 Search Techniques 检索技术总览

三个数量概念：

1. 候选文档数量：检索器最初召回的文档。
2. 过滤后数量：满足元数据约束的文档。
3. Top-K 数量：最终返回给 RAG 后续环节的最高排名文档。

后面还会引入 RRF（Reciprocal Rank Fusion），用于把关键词检索和语义检索得到的两个排名列表融合成一个。

3.5 Metadata Filtering 元数据过滤

元数据（Metadata）指描述文档属性的信息，而不一定是文档正文。

Metadata Filtering 的核心不是计算相关性，而是判断文档是否满足硬性条件。

例如，用户要求只查询 2024 年 6 - 7 月的文章，即使一篇 2023 年的文章内容高度相关，也应当被排除。

优点与局限

Benefits（收益）	Limits（局限）
Easy to understand and debug　容易理解与调试	Does not interpret document content　不解读文档内容
Fast, mature implementations　实现快且成熟	Does not rank by relevance　不按相关性排序
Enforces exact metadata conditions　严格执行元数据条件	Usually paired with a search method　通常需与检索方法搭配

- 第一条收益 debug 很实际，过滤是布尔逻辑，出错了可以直接看出是哪条条件写错，不像打分那样难定位。
- 第一条限制是根本性的，它只读元数据，不读正文，所以无法判断内容是否切题。
- 第二条限制让它无法独自完成检索，它只能缩小范围，不能提供排序，最终还是需要一个排序器。
- 第三条既是限制也是用法说明，通常与检索方法搭配把它的定位钉死为预处理，而非独立方案。

其中最重要的区别是：Filtering（过滤）不等于 Ranking（排序）。假设有三篇文档：

文档	发布日期符合要求?	内容相关性
Doc A	是	高
Doc B	是	低
Doc C	否	很高

元数据过滤会保留 A、B，排除 C。但它不会自动判断 A 比 B 更相关。这需要关键词检索或语义检索等排序方法。因此，Metadata Filtering 通常作为检索流水线中的约束机制，与搜索算法配合使用。

3.7 Typical Retrieval Methods 典型检索方法

课件接下来正式进入三个主要方法：

1. Keyword Search：Bag of Words、TF-IDF、BM25。
2. Semantic Search：Embedding、Vector Similarity、Positive and Negative Training Pairs。
3. Hybrid Search：结合关键词与语义检索结果。

这三种方法并非完全替代关系，而是逐步解决不同的问题。关键词检索回答的是哪些词匹配，语义检索回答的是哪些含义接近，混合检索则尝试综合两种证据。

3.8 Introduction to Keyword Search

关键词检索可以找到字面相似的文档，但词项重合不等于内容相关。

Match words in the query to words in documents. Shared terms provide a simple basis for ranking.

关键词检索通过统计查询与文档之间的共有词项来衡量相关性。给这些共有词项打分。

1. BoW 把文本转换为词频统计
2. Sparse Vector 把词频表示成向量
3. TF 衡量词在文档中出现的频率
4. IDF 衡量词在整个文档集合中的罕见程度
5. BM25 进一步控制词频饱和与文档长度影响

这一条技术路线的逻辑是：

最初只判断词有没有出现，随后考虑出现多少次，再考虑这个词是否常见，最后处理重复词带来的过度加分以及文档长短不公平的问题。

Example：包子查询中的关键词匹配

用户的问题是：How can I make baozi at home?

关键词检索首先寻找共有词

把问题拆开，可以得到：`how`、`can`、`I`、`make`、`baozi`、`at`、`home`

这些词并不具有相同的信息价值。

Doc 1 包含 baozi，并提供了制作面团的信息。

Doc 2 包含 baozi 和 home，但讨论的是外送。

Doc 3 包含 Baozi，却主要介绍餐厅。

因此，包含更多相同词项，并不一定意味着文档更能回答问题。

后续关键词检索的三个问题：

1. 不同关键词是否应该具有相同权重？
2. 同一个词出现很多次，是否应该持续增加得分？
3. 长文档比短文档包含更多关键词，是否就应该排名更高？

接下来的 BoW、TF-IDF 和 BM25，就是逐步解决这些问题。

3.8.1 Bag of Words 词袋模型

Bag of Words represents text by counting how often each word appears. Word order is ignored, only word presence and frequency matter.

词袋模型通过统计每个词出现的次数来表示文本。

模型只关心有哪些词，以及每个词出现了多少次，而不关心词的排列顺序。

BoW 本质上是一种词项统计表示，而不是完整的语义表示。

词袋模型为什么仍然有用

虽然它丢失词序，却带来两个重要优势。

第一，文本可以转换为数字。

第二，不同长度、不同内容的文本，可以映射到同一个词汇空间中进行比较。

这为后面的稀疏向量、词项—文档矩阵和相关性评分建立了基础。

3.8.2 稀疏向量 Sparse Vectors

词频表可以表示成一个向量。

构造词汇表，按照课件展示的词项顺序，可以整理为：

Term	Value	Term	Value	Term	Value
making	1	baozi	2	without	1
a	1	steamer	1	rice	0
flour	0	dough	0	tea	0
bowl	0	water	0	salt	0
soup	0	yeast	0	filling	0

向量长度为 15，因为词汇表有 15 个词项。只有 5 个位置非零。

One vocabulary term per position; most entries are zero.

Sparse 即每个位置对应词汇表中的一个词，而大部分位置都是 0。

例如，一个真实知识库可能有几十万个不同词项，但一篇短文只包含其中几百个。因此，其向量绝大多数维度都是 0。这就形成了高维稀疏向量。

稀疏向量与稠密向量的区别

	Sparse Vector	Dense Vector
中文	稀疏向量	稠密向量
典型用途	关键词检索	语义检索
维度含义	对应具体词项	学习得到的特征维度
数值分布	大量为 0	通常多数非零
代表方法	BoW、TF-IDF	Embedding Models

稀疏向量的每一维通常可以解释为一个具体词项；语义嵌入向量的单个维度通常没有这种直接可读的含义。

工程上为什么不直接存储所有 0

如果一个向量有 100,000 维，只有 100 个非零值，那么直接存储全部数值非常浪费空间。实际系统可以只存储：非零元素所在的位置、对应的数值。

在真实检索系统中，倒排索引（Inverted Index）通常比直接逐篇比较完整稀疏向量更适合关键词搜索。

3.8.3 The Term-Document (TD) Matrix 词项—文档矩阵

如果知识库有很多篇文档，如何把它们组织到同一个数据结构里？

答案是 Term-Document Matrix，即词项—文档矩阵。

矩阵的基本结构

- 行：Terms（词项）。
- 列：Document Vectors（文档向量）。
- 单元格：某个词项在某篇文档中的匹配或词频信息。

其中：

- t_i：第 i 个词项。
- d_j：第 j 篇文档。
- M_{ij}：该词项在该文档中的出现次数。

从矩阵到排序

这个过程可以概括为：文本 → 词频向量 → 文档矩阵 → 查询匹配 → 相关性得分 → 排序。

课件后面的 TF-IDF 仍然使用这个结构，只是把矩阵中的简单词频替换为带权重的词频。

因此，TF-IDF 不是推翻词袋模型，而是在已有的词项表示基础上改进权重。

Scoring a Keyword Match 计算相关性

规则：每匹配一个查询词项，就加 1 分。

这个公式对应的是 Binary Term Matching（二值词项匹配）。

它只判断词项是否出现，不考虑出现次数。

问题一：常见词与罕见词同权。a 命中一次也 +1，baozi 命中一次也 +1，但显然后者更有信息量，归一化解决。

问题二：长文档天然占优。文档越长，越可能包含查询里的词，命中数自然更高，词重要性（IDF）解决。

Scoring All Query Terms 对全部查询词打分

在当前规则下：

- `baozi` 匹配，加 1 分。
- a 匹配，也加 1 分。

但从信息价值来看，两者显然不同。`baozi` 能帮助识别文档主题。a 几乎不能帮助区分哪些文档与制作包子相关。不同词项是否应该具有不同的重要性？后面的 IDF 就是为此设计的。

3.8.4 Frequency-Based Scoring

前面的简单匹配规则只考虑词有没有出现。那么一个查询词在文档中出现很多次，是否意味着文档更相关?

课件定义:

TF 的优势与问题

TF 比简单的二值匹配更细致，因为它考虑了重复出现的词项。但它存在一个明显问题：较长的文档往往天然包含更多词，因此也更容易获得较高的原始 TF 分数。

例如，一篇 10,000 字的文章，可能比一篇 100 字的文章包含更多次 baozi。但这不代表前者一定更相关。

所以，课件下一页引入长度归一化。

3.8.5 Normalized Term Frequency 归一化词频

Issue: Raw counts can favor longer documents.

原始词频可能偏向长文档。

排名发生了逆转

Raw TF：Document 2 更高；Normalized TF：Document 1 更高

这说明：仅仅改变评分中的长度处理方式，就可能改变最终的检索排序。

归一化是否完全解决了长度问题?

没有。简单地除以文档长度，可能过度奖励非常短的文档。例如，一篇只有 5 个词的文档，其中 3 个是查询词，它的归一化分数可能很高。但这篇文档不一定包含足够的信息来回答问题。

这也是后面 BM25 不直接使用简单的 $TF/|d|$，而采用更灵活的长度归一化机制的原因之一。

3.8.6 TF-IDF：词频与逆文档频率

前面已经解决了两个问题：词是否出现。词出现多少次。

现在还剩一个重要问题：不同词的重要性是否应该相同?

Basic TF scoring treats all words equally, whether they're common filler words or rare, meaningful terms.

基础 TF 评分把所有词都看得一样重要，不论它是常见词还是具有区分性的关键词。

例如：`the`：非常常见；`baozi`：相对少见，并且具有明确主题信息。因此，引入 TF-IDF。

注意，TF-IDF 有多种实际实现形式，TF 可以使用原始次数、对数缩放或归一化形式，IDF 也可以进行平滑处理。这里先严格按照课件给出的基本公式理解。

为什么要使用逆文档频率

假设有 100 篇文档。`the` 出现在全部 100 篇中。`baozi` 只出现在 5 篇中。

`the` 几乎不能帮助检索器区分文档，因为每篇都有。`baozi` 则更具有区分性，因为只有少量文档包含它。

所以一个词越少见，它在区分文档方面通常越有价值。这就是 IDF 的基本思想。

3.8.6.1 Calculating Inverse Document Frequency 计算逆文档频率

第一步：统计文档频率

第二步：取倒数

第三步：取自然对数

为什么还要取对数？如果直接使用 N/df极罕见词可能得到非常大的权重。

例如：出现在 10 篇文档中：倒数为 10；出现在 1 篇文档中：倒数为 100。两者相差 10 倍。

取对数后，差距被压缩。因此，对数变换既保留了罕见词权重更高的趋势，又避免权重增长得过快。

3.8.6.2 Applying TF-IDF Weights

3.8.6.3 Ranking with TF-IDF 用 TF-IDF 排序

Rare keywords can contribute more to the score than common words

罕见关键词对分数的贡献可以超过常见词

加了 TF 之后，同一个词的得分变化，但词与词之间的相对高低仍由 IDF 主导

TF-IDF 解决了什么

相比原始词频，TF-IDF 能够降低大量普通词的影响。

例如，一篇文档反复出现 `a`、`the`、`and`，不应仅仅因为这些词出现得越多就获得高相关性。

TF-IDF 仍然存在什么问题

TF-IDF 并不是完整的相关性理解模型，仍然依赖词项重合。

例如：查询：How to prepare steamed buns? 文档：Instructions for making baozi.

即使两个文本语义非常接近，如果没有合适的词形处理或词项扩展，传统 TF-IDF 也可能无法有效匹配。

此外，原始 TF-IDF 中的词频可能持续增长，长文档长度问题也需要额外处理。

这两个问题正是 BM25 重点改进的方向。

3.8.7 From TF-IDF to BM25

BM25 refines keyword scoring with saturation and length normalization.

BM25 通过词频饱和与文档长度归一化，进一步改进关键词相关性评分。

两个核心概念：

1. Term Frequency Saturation（词频饱和）

一个词出现 20 次，不应该必然获得出现 10 次时的两倍得分。

2. Document Length Normalization（文档长度归一化）

长文档包含更多词，但不应该仅仅因为篇幅更长而获得不合理的排名优势。

这两点构成 BM25 相比简单 TF-IDF 的主要改进。

Modern RAG systems often use BM25 to refine keyword scoring.

BM25 是现代检索系统中广泛使用的关键词排序方法。

3.8.7.1 BM25 Scoring

BM25 全称 Best Matching 25，来自 1990 年代的 Okapi 检索系统，是 TREC 评测里迭代出的第 25 版公式。它不是从理论推导出来的，而是在大量评测中调出来的经验公式。这也解释了为什么它带有 k_1 和 b 两个需要人工设定的参数，它们是评测调优的产物。

BM25 Tunable Parameters BM25 可调参数

维度	Raw TF-IDF	BM25
词频增长	10 次 = X → 20 次 = 2X（线性）	20 次 → Score < 2X（饱和）
短文档	Short Doc = Higher Score	Short Doc = Less Penalty
长文档	Long Doc = Lower Score	Long Doc = Tunable Penalty
惩罚性质	Fixed length penalty　固定	Tunable Penalty　可调（由 b 控制）

Two Improvements in BM25 BM25 的两项改进

第一项：词频饱和

- 简单线性词频评分：rice 出现 10 次得到 X，出现 20 次得到 2X。
- BM25：出现 20 次的得分通常小于出现 10 次得分的两倍。

原因是重复出现的词会产生递减收益。例如，一篇文档已经多次讨论 rice，再增加一次 rice，提供的额外相关性证据可能并不多。

第二项：文档长度归一化

- 简单按长度归一化可能形成固定比例的长度惩罚。
- BM25 使用可调节的长度惩罚。

这让检索器可以适应不同类型的文档集合。例如：新闻标题：文本普遍较短。学术论文：文本较长。RAG 文本块：长度可能较为接近。这些集合不一定适合完全相同的长度惩罚。

工程上的重要细节

如果 RAG 系统已经把长文档切成长度相近的 Chunk，BM25 的长度归一化效果可能与直接检索完整文档时不同。

因为此时长度比例接近 1。这意味着，Chunking 策略会间接影响 BM25 的评分行为。

饱和为什么符合直觉？一篇文档里出现 10 次 rice，已经足够说明它在讲米饭；出现 20 次，信息量并没有翻倍。如果一篇文档反复刷同一个词，多半是垃圾内容或关键词堆砌，不应该无限加分。饱和机制正是对这种噪声的防御，它是边际效用递减在检索打分里的实现。

TF-IDF vs. BM25

维度	TF-IDF	BM25
词频处理	基本形式可线性增长	饱和函数
词项罕见程度	IDF	IDF
文档长度	依赖具体实现，可另行归一化	内置可调长度归一化
参数	基础公式较简单	(k_1, b)
工程应用	经典检索基线	常用生产级关键词排序方法
语义理解	不直接理解语义	不直接理解语义

BM25 比基础 TF-IDF 更灵活，不代表它在任何数据集上都必然更好。最终效果仍然需要通过检索评估指标和实际任务测试判断。

工程实现上

默认选 BM25：成本相近、质量更好、生态成熟

用 TF-IDF：语料很小、需要极简实现、或需要一个能看懂每一步的教学基线——Tutorial 用的就是 TF-IDF，因为 13 个分块、345 个词项的规模下，BM25 的两个参数反而增加理解负担。

3.8.8 关键词检索小结

To match a prompt according to how frequently the prompt’ s keywords appear in each document

按提示关键词在各文档中出现的频次来匹配提示

1. TF-IDF 的三项加权： Keyword rarity（关键词罕见度）、Term frequency（词频）、Document length（文档长度）。
2. BM25（最常用）的两项改进：Document length normalization 与 Term Frequency Saturation。
3. 流程四个动词 ： Sparse Vectors → Scored → Ranked（稀疏向量 → 打分 → 排序）
4. What are the pros and cons?——Why Meaning Matters?

3.9 Semantic Search and Embeddings

3.9.1 语义的重要性

关键词检索的两类系统性失败：

失败一：同义词（Synonyms）

失败后果：文档里写 glad，查询里写 happy，共有词为零，分数为零。

失败二：同形异义（Same spelling, different meaning）

失败后果：查询里的 Python（编程语言）与文档里的 Python（蟒蛇）字面相同，会被判为高度相关。

现象	方向	关键词检索的表现	语义检索的期望表现
Synonyms　同义词	漏检	happy vs glad 分数为 0	两者向量接近，能命中
Polysemy　同形异义	误检	Python 语言 vs 蟒蛇被当作同一回事	由上下文决定向量位置，可区分

关键词检索的全部信息就是字面是否相同，它没有别的线索可用，所以这两类错误是方法层面的，不是参数层面的。 要解决它们的问题，需要引入embedding，让表示本身携带语义。

3.9.2 关键词检索与语义检索

Both approaches represent the query and documents as vectors.

关键词检索和语义检索都可以把查询与文档表示为向量。两种方法都把查询与文档表示成向量，最大的区别不在于是否使用向量，而在于向量表示什么。

1. Keyword Search：A sparse lexical space 稀疏词汇空间

关键词检索统计词汇表中各个位置上的词。例如：每一维通常对应一个具体词项。如果词汇表包含 50,000 个词，向量就可能具有 50,000 个维度。大部分维度为零。

2. Semantic Search：语义嵌入空间

语义检索使用 Embedding Model，将文本转换为能够表达学习到的语义特征的向量。

维度	Keyword Search	Semantic Search
构造方式	Count words in vocabulary positions 在词表位置上计数	Use an embedding model to represent meaning 用嵌入模型表示含义
空间名称	A sparse lexical space（稀疏的词法空间）	A semantic (embedding) space（语义/嵌入空间）
表示依据	词项出现与频率	学习得到的语义特征
维度特点	词表有多大就有多少维，绝大多数为 0	维度固定（如 384），几乎每维都有值
典型向量	Sparse Vector	Dense Vector
可解释性	高：分数可拆到具体词项	低：分数是整体相似度
核心方法	BoW、TF-IDF、BM25	Embedding Model
主要优势	精确词项匹配	语义相似性匹配
主要局限	同义词与上下文理解	可能忽略精确词项差异

两种检索方式都能使用向量，但关键词向量表示词项统计，语义向量表示模型学习到的文本特征。

- 共同点比差异更重要，既然两者都产出向量，那么相似度计算方式、排序方式、top-k 截断方式都可以共用。
- 差异只在怎么造这个向量，这正是混合检索可行的技术前提。

3.9.3 Word Embedding Space 词嵌入空间

Embedding Different Text Units 嵌入不同的文本单元

嵌入可以作用在三个粒度上，各有各的模型。

文本单元	模型	课件示例
Word　词	Embedding Model	cat ・ happy
Sentence　句子	Sentence Embedding Model	The weather is nice ・ What a lovely day
Document　文档	Document Embedding Model	（课件以空框示意，未给示例句）

分块（chunking）本质上是在选粒度。块太小，一个事实会与它的解释被切开；块太大，一个向量要概括太多主题，相似度会被稀释，模型要处理的信息越多。一个句子需要处理语序与修饰关系；一篇文档还要处理长距离依赖。

Sentence Embedding Example 句子嵌入示例

三句话、三个向量：意思接近的两句在空间里靠得近

1. He spoke softly in class.
2. He whispered quietly during class.
3. Her daughter brightened the gloomy day.

第一句和第二句：spoke softly、whispered quietly 表达的动作和语境相近；第三句则表达了不同的内容。

- 课件结论句 — Once we project these sentences into an embedding space, we can use vector similarity to measure the semantic meaning.

示意：A、B 方向更接近，余弦相似度更高；C 与它们的方向差异更大。

3.10 语义检索流程 Semantic Search

Documents/Prompt → Embedding Model → Vector Space

- 第一步，用 Embedding Model 把 Documents 全部转成向量，放进 Vector Space。
- 第二步，用同一个模型把 Prompt 转成向量，也放进 Vector Space。
- 第三步，Rank by distance and return the closest documents（按距离排序，返回最近的文档）。

距离近 = 语义近。

欧氏距离（L2）：越小越相似。

余弦相似度：越大越相似（值域 -1 到 1，非负向量时为 0 到 1）。

必须是同一模型，换模型意味着两个向量不在同一个空间，比较结果失去意义。

3.11 Embedding Models Learn from?

前面已经知道：相似的文本应该在向量空间中更接近。

但这引出了一个新问题：模型怎么知道哪些文本相似，哪些文本不相似?

课件使用三个例子：Hello、Good morning!、Noisy trombone

Hello 和 Good morning! 都属于问候语。

Noisy trombone 则表示一种不同的概念。

1. 向量位置不是天然正确的

在训练前，模型并不会天然拥有一个符合任务目标的语义空间。课件通过散点图展示了不同文本在空间中的位置。模型需要利用训练数据和损失函数，逐步调整参数，让空间结构更符合训练目标。

2. 训练的三个基本组成部分：

Training Data（训练数据）告诉模型哪些文本相似，哪些文本不同。

Architecture（模型结构）负责将文本映射为向量。

Loss Function（损失函数）衡量当前向量表示是否符合训练目标。

训练过程不断更新模型参数:

3.12 Positive and Negative Training Pairs 正样本对与负样本对

样本类型	课件例子	含义	训练动作
Positive pair　正样本对	Good Morning ↔ Hello	Similar meaning	拉近两个向量
Negative pair　负样本对	Good Morning ↔ Noisy trombone	Different meanings	推远两个向量

为什么正负样本需要同时存在

如果只训练正样本，模型可能只学会让某些相似文本靠近，却缺乏足够的区分能力。

负样本提供了对照：不仅要知道什么是相似的，还要知道什么是不相似的。

这种利用相似与不相似关系学习表示的方法，是 Contrastive Learning（对比学习）的核心思想。

3.13 Training data

Compile massive training dataset of positive and negative pairs

构建大规模正负样本训练数据集

为什么需要大量训练数据

单独一个正样本对：`Good Morning` — `Hello`只能告诉模型这两个表达比较相似。但真实语言具有大量不同的语义关系。例如：问候语、同义词、主题相似、问题与答案、描述与标题。

因此，模型需要从大量样本中学习更一般化的表示能力。

样本关系与训练目标

在语义检索中，一个正样本对不一定是两句完全相同意思的话。

Query：How do I make baozi?

Positive Document：Steps for preparing steamed buns at home.

它们不是同一句话，但文档可以回答查询，因此在检索任务中可以被视为相关样本。

这说明：检索模型学习的不仅可以是语言意义上的相似性，也可以是查询与文档之间的任务相关性。

这是对课件正负样本概念的检索场景的延伸。

3.14 Further Reading

3.14.1 Contrastive Loss

公式说明正负样本是如何被同时拉近与推远的

Contrastive Loss 通过比较正负样本的相对相似度，引导模型学习具有区分能力的嵌入空间。

3.14.2 Updating the Embedding Model

上一页定义了训练目标。这一页展示模型如何通过重复训练改善向量空间。

把训练过程画成一个循环：Embed（嵌入当前样本）→ Score（按损失函数打分）→ Evaluate（评估误差）→ Update（更新模型参数）

训练集上的正负样本分离并不能自动保证检索效果良好。实际还需要在独立评估数据上验证模型的泛化能力。

3.14.3 High-Dimensional Embeddings

More dimensions mean more room to form clusters and capture nuanced relationships.

更多维度为模型表示复杂关系提供了更大的空间。高维表示可以帮助模型学习这些复杂因素之间的关系。不过，这并不意味着每一个维度都对应某个明确、独立、可解释的语义属性。

高维空间仍然遵循相同原则

The same principles hold. Close vectors, similar meanings.

即使嵌入向量有数百甚至数千维，基本思想仍然相同：在模型采用的相似度度量下，语义相关的文本应该具有较高相似度。

高维不一定更好

增加维度可能提升表示容量，但也可能增加：向量存储成本、检索计算开销、索引内存需求。

因此，Embedding Dimension 需要在表示能力、检索效果和系统成本之间取得平衡。

3.15 嵌入模型要点

课件原文	含义
Vector locations are arbitrary before training.	训练之前，向量位置没有意义
Training data and constrative loss form the learning	学习由训练数据与对比损失共同塑造
compare vectors from the same embedding model.	只有同一模型产出的向量才能互相比较
Pretrained models can be used directly for retrieval.	预训练模型可以直接拿来做检索，无需自己训练

1. Vector locations are arbitrary before training

训练之前，向量的位置并不天然对应所需的语义结构。通过训练，模型逐步学习有用的表示。

2. Training data and contrastive loss from learning

课件强调训练数据与 Contrastive Loss 的共同作用。训练数据定义哪些文本应该接近、哪些应该区分。损失函数把这些关系转换为可以优化的数学目标。

3. Compare vectors from the same embedding model

工程原则：不同嵌入模型即使输出相同维度的向量，也不代表这些向量位于同一个兼容空间中。虽然维度相同，但如果两个模型独立训练，直接比较其输出通常没有可靠的语义意义。对于 RAG 系统，文档和查询通常需要使用同一个嵌入模型，或者使用经过联合训练、明确兼容的查询与文档编码器。

4. Pretrained models can be used directly for retrieval.

预训练模型可以直接用于检索，也就是说，并不是每个 RAG 项目都需要从头训练一个 Embedding Model。可以使用已有的预训练嵌入模型，为查询和知识库文档生成向量。这显著降低了语义检索系统的构建门槛。

这一部分从关键词检索的局限出发，解释了语义检索的完整逻辑。

核心问题	对应概念	解决思路
不同词能否表达相同意思?	Semantic Similarity	比较语义而非仅比较词项
文本如何变成数字?	Embedding Model	映射到向量空间
哪些文本更相关?	Vector Similarity	比较查询与文档向量
模型如何知道相似关系?	Positive / Negative Pairs	提供训练监督
如何训练嵌入空间?	Contrastive Loss	拉近正样本、区分负样本
如何改善模型?	Embed - Score - Update - Evaluate	反复优化
如何用于实际 RAG?	Pretrained Embeddings	生成向量并检索

3.16 Comparing Retrieval Methods 比较各种检索方法

方法	课件原文	中文
Metadata filtering	Applies strict criteria such as permissions or region.	施加严格判据，如权限或地区
Keyword search	Matches exact terms and provides a strong lexical baseline.	匹配精确词项，提供强的词法基线
Semantic search	Finds similar meanings even when wording differs.	即使措辞不同也能找到相近含义

这三种方法并不互相排斥。实际系统可以先使用元数据约束候选范围，再利用关键词和语义检索找到相关内容。

3.17 Combining Ranked Lists 合并排序列表

混合检索的目标：Lists combined to form single ranking 把多个排序列表合成一个排序。

为什么两个结果列表不同?

因为两种检索方法使用不同的相关性标准。

Keyword Search 可能认为 Document A 最相关，因为它包含更多精确匹配的关键词。

Semantic Search 则可能认为 Document C 最相关，因为它在嵌入空间中更接近查询。

注意，两个列表也存在重叠，例如 Document B 和 Document C。

这意味着某些文档同时得到了两种检索方法的支持。

合并不是简单拼接

假设直接把两份列表连接起来，就可能出现两个问题：

- 同一篇文档重复出现。
- 两个列表之间缺少统一的相关性顺序。

融合要解决的核心难题：分数不可比

关键词检索的分数是 TF-IDF/BM25 值（无上界，量纲依赖于语料）；语义检索的分数是余弦相似度（通常在 0 - 1）。两者的数值不能直接相加，加出来的结果没有任何意义。因此，Hybrid Search 需要一个 Ranking Fusion（排序融合）机制，丢掉分数、只用排名。 Reciprocal Rank Fusion，正是用于解决这个问题。

3.18 Reciprocal Rank Fusion 倒数排名融合

Reciprocal Rank Fusion，简称 RRF。是一种根据文档在多个检索结果列表中的排名，计算综合得分的方法。与直接合并 BM25 分数和向量相似度不同，RRF 主要使用排名位置。

1. Rewards documents for being highly ranked on each list 按每个列表里的排名高低给分，排名越高奖励越多
2. Optional weights balance keyword and semantic rankings 可选权重用来平衡两路排序的比重

RRF 的基本思想

- 排名越靠前，贡献越大。
- 可以使用权重平衡关键词和语义排名。

- 当 $k = 0$ 时，第 1 名得 1 分，第 2 名得 0.5 分。可以看到，排名越靠后，得分越低。当 k 增大时，靠前和靠后排名之间的贡献差距会变得相对平缓。课件没有指定生产系统应该使用的固定 k 值；在实际实现中，该参数可以根据系统需求设置。
- 各个排名列表的得分相加，决定最终排名。

核心逻辑是：一篇文档在多个列表中排名越靠前，最终的融合得分通常越高。

An RRF Calculation

3.19 Weighting Semantic and Keyword Search 语义与关键词的加权

除了排名融合的方式，还有一个更直接的控制手段：给两路结果加权重 β。

- β = 0.8 的情形 — Semantic Search 占 80%，Keyword Search 占 20%。
- β = 0.7 的情形 — Semantic Search 占 70%，Keyword Search 占 30%。
- β 调的是比重 ——直接决定两路贡献的相对大小，作用在加权求和式的融合里。
- RRF 的 k / c 调的是排名差的压缩程度 ——它不改变两路谁更重要，只改变排名靠前这件事的收益有多陡。

β	Semantic Search	Keyword Search	适用倾向
0.8	80%	20%	问题措辞灵活、依赖语义理解
0.7	70%	30%	含精确串（编号、型号、专名），需要字面匹配

3.20 Returning the Top K Documents 返回前 K 篇文档

前面已经完成：Keyword Search、Semantic Search、Metadata Filtering、Ranking Fusion

接下来，系统需要决定：最终应该把多少篇文档返回给 LLM?

课件给出的答案是：Return the “top_k” highest-ranked documents. 即返回排名最高的 K 篇文档。

区分：课件重点讲的是检索结果如何融合并返回 Top-K，图中的完整 LLM 生成流程是与前面 RAG 架构内容的衔接。

为什么不返回全部文档

如果把检索到的所有文档都交给 LLM，可能带来：更高的上下文 Token 消耗、更多无关信息、更长的输入处理时间、更高的生成成本。因此，Top-K 的作用是限制最终进入后续处理阶段的候选文档数量。

K 越大越好吗

不一定。较小的 K 可能减少无关内容，但也可能遗漏有用信息。较大的 K 可以覆盖更多候选文档，但也可能引入噪声。因此，K 的选择需要结合检索质量、上下文预算和具体任务。

Hybrid RAG 的最终结论

Hybrid RAG search is still a mainstream and effective design. 混合检索仍然是一种主流且有效的 RAG 检索设计。

它不是简单地选择关键词检索或语义检索其中之一，而是将不同方法的优势组合起来。

总结

三种技术各自负责什么

技术	核心作用	典型方法
Metadata Filtering	严格约束候选文档	字段条件筛选
Keyword Search	精确词项相关性	BM25
Semantic Search	语义相关性	Embedding + Vector Similarity
Ranking Fusion	合并不同检索结果	RRF
Top-K Selection	选择最终返回的文档	排序截取

五个核心问题

1. 为什么需要 Retrieval？ 因为 LLM 需要从外部知识库获取与用户问题相关的信息。
2. BM25 为什么比简单词频更有效？ 因为它结合了词项重要性、词频饱和与文档长度归一化。
3. Embedding 为什么能用于语义检索？ 因为模型学习了可比较的向量表示，使相关文本具有较高的向量相似度。
4. RRF 为什么适合 Hybrid Search？ 因为它根据排名融合结果，不需要直接比较不同检索器的原始分数。
5. Top-K 的作用是什么？ 从融合后的排序中选出有限数量的高排名文档，供后续 RAG 流程使用。

核心结论：高质量 RAG 检索不只依赖一种匹配方式，而是通过关键词匹配、语义相似性、元数据约束与排序融合，共同找到最适合回答用户问题的文档。

Part 4 Retrieval Evaluation 检索评估

Part 3 解决了一个问题：如何从知识库中找到相关文档？ Keyword Search、BM25、Semantic Search、Hybrid Search，以及 RRF 排序融合。但找到文档并不代表检索效果一定好。

例如，两个 Retriever 都返回了 10 篇文档：

- Retriever A 返回了 8 篇相关文档。
- Retriever B 返回了 5 篇相关文档。

我们可能认为 A 更好，但如果 A 把真正重要的文档排在最后，而 B 把它们排在最前面，情况就没那么简单。

因此，Part 4 要解决三个核心问题：

1. Precision： 返回的文档有多少是相关的?
2. Recall： 所有相关文档中，有多少被成功找回?
3. MAP： 相关文档是否排在检索结果的前面?

4.1 Retrieval Quality Metrics 检索质量指标

三要素：查询、排序结果、标准答案。

课件元素	含义
Query: The specific information needs being evaluated	被评估的那个具体信息需求
Ranked results: Documents returned in ranked order	按排名返回的文档列表
Ground truth: Documents labeled relevant or irrelevant	被标注为「相关 / 不相关」的文档

Ground truth 是评估的前提，没有人工标注的相关性判断，任何指标都算不出来。常见来源：人工标注（最准、最贵）、用户行为日志（点击/停留，规模大但有噪声）、合成标注（用模型生成，最便宜但可能有偏）。

To evaluate a retriever, you need relevant judgments. 评估检索器需要相关性判断，指的是一批查询、每个查询一批标注，不是单点结论。这一步的成本常被低估，标注需要人工，规模越大成本越高，但它是所有指标的地基。

4.2 Precision and Recall 精确率与召回率

指标	公式	课件追问
Precision　精确率	Relevant retrieved / total retrieved	How many returned documents are relevant?
Recall　召回率	Relevant retrieved / total relevant	How many relevant documents were returned?

Precision：你找回来的东西，有多少是对的?

Recall：所有对的东西，你找回来了多少?

4.2.1 The Precision - Recall Trade-off 精确率与召回率的取舍

运行	Retrieved	Relevant	Precision	Recall
First Run	Retrieved: 12 Documents	Relevant: 8 Documents	8/12 = 66.7%	8/10 = 80%
Second Run	Retrieved: 15 Documents	Relevant: 9 Documents	9/15 = 60%	9/10 = 90%

提高一个必然牺牲另一个，为什么会发生这种变化?

第二次检索多返回了 3 篇文档，但只有 1 篇是相关的。

因此：Recall 提高，因为多找回了一篇相关文档；Precision 降低，因为额外引入了两篇无关文档。

Precision penalizes returning irrelevant documents. Precision 惩罚返回无关文档。

Recall penalizes leaving out relevant documents. Recall 惩罚遗漏相关文档。

如果检索器能更准确地找到相关文档，两者也可能同时提高。

4.3 Evaluating the Top K Results 评估前 K 个结果

1. Retrieval metrics are influenced by how many documents the retriever returns（指标受返回数量影响）。
2. Metrics are discussed in terms of top-k documents（指标都以 top-k 的方式讨论）。

返回得越多，Precision 的分母越大（一般会被拉低），Recall 的分子越大（一般会被抬高）。

为什么 K 会影响指标? Retrieval metrics are influenced by how many documents the retriever returns.

因为 K 改变了被评估的文档集合。当 K 增大时：

- Recall@K 不会下降，因为已经找到的相关文档仍然保留。
- Precision@K 可能上升，也可能下降，取决于新增文档是否相关。

实际应用应该如何选择 K? Evaluate at a cutoff that matches the number of results your application uses.

评估时使用的 K，应该与实际应用使用的结果数量一致。

例如，如果 RAG 系统只把前 5 篇文档交给 LLM，那么 Precision@5、Recall@5 就比只报告 Precision@100 更贴近实际使用情况。

4.3.1 Precision and Recall at K 前 K 个结果的精确率与召回率

同一个排序列表，换个截断位置，指标就变了。

截断	课件标注	计算	结果
Precision@5	2 out of 5	前 5 位命中 2 个相关	40%
Precision@10	6 out of 10	前 10 位命中 6 个相关	60%

Precision@K 不一定随着 K 增大而下降。

Evaluate at a cutoff that matches the number of results your application uses.

如果后面新增的文档中有较高比例的相关文档，Precision@K 反而可能上升。

4.4 Mean Average Precision 平均精度均值

前面的 Precision@K 只关心前 K 篇文档里有多少篇相关。但它没有充分体现相关文档具体排在什么位置。

例如：

- Retriever A：相关文档排在第 1、2、3 位。
- Retriever B：相关文档排在第 4、5、6 位。

假设只有这 3 篇相关文档，两者在 K=6 时，虽然两个指标相同，但显然 A 把相关文档排得更靠前。这就是 Average Precision（AP）要解决的问题。MAP 解决的是光看 Precision@k 看不出排序好坏这件事。

MAP averages Average Precision (AP) across queries. MAP 把各查询的 AP 再做平均。

AP 步骤

1. 找出所有相关文档所在的排名。
2. 将这些排名位置对应的 Precision@K 相加。
3. 除以全部相关文档的数量。

4.5 Using Retrieval Metrics 检索指标怎么用

指标	课件定位	回答的问题
Recall@K	Coverage of the relevant documents	“Did we retrieve the relevant information?”
Precision 与 MAP	Result relevance and ranking quality	“How much of what we retrieved is relevant?” / “Are the relevant results ranked near the top?”

不同指标与实际检索系统的调优目标

1. Recall@K：Coverage

Coverage of the relevant documents. 相关文档的覆盖程度。

核心问题：Did we retrieve the relevant information? 如果 Recall@K 很低，说明检索系统遗漏了大量相关文档。

可能的改进方向包括：

- 增加候选召回数量。
- 引入 Semantic Search 补充关键词检索。
- 调整过于严格的非权限类过滤条件。

但权限过滤不能为了提高 Recall 而随意放宽。

2. Precision：Result Relevance

How much of what we retrieved is relevant? Precision 衡量返回结果中相关文档所占的比例。

如果 Precision 很低，说明检索结果包含较多无关内容。

可能的改进方向：

- 调整 Top-K。
- 改进关键词与语义检索的匹配策略。
- 使用更有效的相关性排序或过滤方法。

3. MAP：Ranking Quality

Are the relevant results ranked near the top? MAP 关注相关文档是否排在前面。

如果系统 Recall 较高，但 MAP 较低，可能意味着：相关文档虽然已经被检索出来，却排在较靠后的位置。

这时可以考虑调整：

- BM25 参数。
- Embedding Model。
- Hybrid Search 的权重 β。
- RRF 的融合设置。

这些属于基于指标的工程调优思路，并非课件保证有效的固定操作。

4. 指标与调优方向总结

指标	主要关注	偏低时可能的问题
Recall@K	覆盖率	遗漏相关文档
Precision@K	结果纯度	无关文档太多
MAP	排序质量	相关文档排得靠后

为什么要在同一组查询上比较？

Compare settings on the same labeled query set.

如果 Retriever A 使用一组简单查询，而 Retriever B 使用另一组困难查询，那么两者的指标无法公平比较。

正确的实验方法是：固定相同的 Query Set 和 Ground Truth，再比较不同的检索设置。通过比较，可以判断哪一种设置更符合应用需求。评估指标的意义不只是给检索器打分，更重要的是帮助选择和改进检索配置。

4.6 全讲要点

1. Knowledge beyond training

RAG can provide private, recent or specialized information.

RAG 可以补充模型训练之外的私有、近期或专业信息。

2. Retrieval before generation

Relevant documents ground the answer in supplied evidence.

先检索相关文档，再基于检索证据生成回答。

3. Retrieval methods

Keywords match terms, embeddings match meaning, and filters enforce criteria.

关键词匹配词项，Embedding 匹配语义，Metadata Filter 执行条件约束。

4. Evaluation guidance

Use recall and precision to compare retrieval settings.

使用召回率与精确率等指标比较不同检索设置。

为什么需要外部知识 → 如何把外部知识提供给 LLM → 如何找到相关知识 → 如何判断找得好不好。

Lecture 7 Retrieval Optimization and Agentic RAG Systems 检索优化与智能体 RAG

Part 1 Scalable Retrieval 可扩展检索

1.1 RAG 工作流与检索方法回顾

1.1.1 RAG Workflow

传统 LLM 与 RAG 的主要区别在于是否引入外部知识检索。

- Direct LLM Use： Prompt → LLM → Response。模型直接利用自身参数中存储的知识生成答案。
- RAG System： Prompt → Retriever → Relevant Documents → Augmented Prompt → LLM → Response。检索器从外部知识库（Knowledge Base）获取相关文档，将其与原始问题组合为增强提示，再交给 LLM 生成答案。

RAG 将外部知识引入生成过程，因此检索质量与检索效率都会影响最终系统表现。

1.1.2 三种典型检索方法 Typical Retrieval Methods

方法	核心原理	特点
Keyword Search（关键词检索）	根据查询与文档之间的词项匹配进行检索	依赖词汇匹配

方法	核心原理	特点
Semantic Search（语义检索）	将查询和文档编码为向量，根据语义相似度检索	可以匹配不同措辞表达的相近含义
Hybrid Search（混合检索）	结合关键词检索与语义检索结果	将两路结果融合为统一排序

关键词与语义检索分别返回 50 篇文档，经过元数据过滤后分别剩余 35 篇和 30 篇，再融合为单一排序。

其中，Metadata Filter（元数据过滤） 根据文档属性进一步筛选候选结果；Hybrid Ranking（混合排序） 则将不同检索路径的结果整合，而不是简单地分别返回两组列表。

1.1.3 语义检索流程 Semantic Search

语义检索以向量表示为基础，主要包括三个阶段：

1. Embedding Model（嵌入模型）： 将文档和查询分别编码为向量，使其能够在统一的向量空间中进行比较。
2. Vector Space（向量空间）： 语义相近的文档和查询通常具有更接近的向量表示。
3. Rank by Distance（距离排序）： 根据向量距离或相似度对文档排序，返回与查询最接近的结果。

这一流程构成后续 KNN、ANN、NSW 和 HNSW 的共同基础。不同方法的主要区别不在于是否使用向量，而在于如何高效定位距离查询向量最近的候选文档。

1.2 精确向量检索与规模挑战 KNN & Scaling Challenges

1.2.1 KNN：精确最近邻检索

KNN（k-Nearest Neighbors，k 近邻）通过比较查询向量与所有文档向量之间的距离，确定最近的 k 个文档。检索流程：

1. Embed documents and the query： 将文档和查询编码为向量。
2. Compare with every document： 计算查询向量与每个文档向量的距离。
3. Rank by distance： 按距离从近到远排序。
4. Return the nearest k documents： 返回距离最近的 k 个文档。

在完整扫描的前提下，KNN 能够找到全局最近邻，但必须对整个文档集合进行距离计算。

设知识库包含 N 个文档向量，每次查询需要：N次向量距离比较。因此，距离计算的时间复杂度为：O(N)

这里的线性复杂度针对距离比较阶段；完整检索还包含编码和结果选择等操作。

1.2.2 规模扩展问题 Scaling Challenges

随着知识库规模扩大，精确扫描的比较次数同步线性增长。

示意图：仅表示距离比较次数与文档数量的线性关系。

文档向量可以提前计算并存储，但每次新查询仍需要与全部文档向量比较。因此，主要的扩展性瓶颈是逐一比较全部文档，而非文档向量的初始编码。

1.3 近似最近邻检索 Approximate Nearest Neighbors, ANN

ANN 的核心思想是通过预先建立的索引（Index）缩小搜索范围，只访问部分候选向量，而不对所有文档执行完整扫描。三个基本特征：

- Searches fewer candidates using an index： 利用索引减少候选文档数量。
- Trades exactness for speed： 通过放弃严格的最近邻保证来提高检索速度。
- The closest result is not guaranteed： 检索结果可能不是全局最近邻。

ANN 是一类近似检索方法，而 NSW、HNSW 是其中基于邻近图（Proximity Graph）的具体实现。

KNN 与 ANN 对比

比较维度	KNN（精确扫描）	ANN
搜索范围	所有文档向量	部分候选向量
索引	不依赖近似索引	利用预建索引
最近邻保证	保证全局最近邻	不保证全局最近邻
大规模检索	线性扫描成本较高	通常显著更快
主要取舍	精确性优先	速度与召回率平衡

ANN 的性能需要同时考虑两个维度：

- Search Speed（检索速度）： 完成一次查询所需的时间或计算量。
- Recall（召回率）： 近似检索找到的结果与真实相关结果或精确最近邻结果的重合程度，具体取决于评估定义。

因此，ANN 的优化目标不是单纯追求最快，而是在可接受的召回率下减少检索成本。

1.4 可导航小世界图 Navigable Small World, NSW

NSW 是一种基于图结构的 ANN 方法，通过建立文档向量之间的邻近关系，将全局扫描转化为局部图导航。

1.4.1 邻近图的构建 Proximity Graph Construction

邻近图由节点（Nodes）和边（Edges）构成：

- Node： 代表一个文档向量。
- Edge： 表示两个向量之间建立的邻近连接。

构建过程包括：

1. Compute distances between all document vectors： 计算文档向量之间的距离，确定邻近关系。
2. Connect each node to its nearest neighbors： 将每个节点与其附近的若干节点连接。
3. Insert document vectors incrementally： 支持增量插入新的文档向量。
4. Traverse the graph along edges： 查询时沿图中相邻节点之间的边进行搜索。

前面三个要点描述索引构建与维护，第四个要点描述索引在查询阶段的使用。

邻近图的构建具有一定计算成本，但可以预先完成，并在后续多次查询中重复使用。

1.4.2 NSW 检索算法：贪心游走 Greedy Walk

NSW 的检索过程，四个阶段。

阶段一：选择入口（Entry Point）

从图中随机选择一个节点作为初始候选节点。该节点不需要接近查询向量。

阶段二：比较相邻节点（Neighbor Comparison）

计算当前节点及其相邻节点与查询向量的距离，只考虑当前节点直接连接的邻居。

若存在距离查询更近的邻居，则沿对应的边移动，并将其更新为当前候选节点。

阶段三：重复局部搜索（Iterative Search）

在新的候选节点上重复相同操作：

$v_{t+1} = \arg\min_{u \in N(v_t)} d(u, q)$

其中：

- q：查询向量（Query Vector）
- v_t：当前候选节点
- $N(v_t)$：当前节点的邻居集合
- $d(u, q)$：邻居节点与查询向量之间的距离

只有当所选邻居比当前节点更接近查询时，才执行移动。

阶段四：终止搜索（Stopping Condition）

当所有相邻节点都不比当前节点更接近查询向量时，终止当前贪心游走。

$d(v_t, q) \leqslant d(u, q), \quad \forall u \in N(v_t)$

此时得到的是局部最优（Local Optimum），不一定是全局最近邻（Global Nearest Neighbor）。

1.4.3 NSW 的优势与局限

NSW 通过沿图边访问候选节点，通常能够以更少的距离比较找到接近查询的文档，并可用于寻找多个近邻。但由于贪心搜索只考虑当前节点的直接邻居，搜索可能在局部最优位置停止，即使图中其他区域存在更接近查询的节点。

因此，NSW 与精确 KNN 的根本区别是：

KNN 比较全部向量以确定全局最近邻；NSW 通过邻近图进行局部导航，以较少的比较次数寻找近似最近邻。

1.5 分层可导航小世界图 Hierarchical Navigable Small World, HNSW

HNSW 在 NSW 邻近图的基础上引入层级结构（Hierarchy），通过不同密度的图实现从粗粒度定位到细粒度检索。

1.5.1 HNSW 的层级结构

HNSW 由稀疏上层、中间层和稠密底层组成。

层级	结构特征	检索任务
Sparse Upper Layer	节点数量少、图稀疏	Coarse Navigation：粗粒度导航
Intermediate Layer	节点数量增加	Narrow the Search：缩小搜索范围
Dense Base Layer	包含全部文档向量	Local Refinement：局部精细检索

Start with fewer nodes. Refine the search as the layers get denser.

其核心思想是从节点较少的上层开始搜索，再逐层进入更密集的图，逐步提高定位精度。

1.5.2 分层索引构建 Building the Hierarchy

课件以包含 1,000 个文档向量的数据集为例，构建三层邻近图：

上层节点更少，底层包含全部文档向量。图中数量为课件示例，并非固定配置。

- Layer 3： 随机选择 10 个向量构建邻近图，用于最高层的快速导航。
- Layer 2： 随机选择 100 个向量构建邻近图，用于中间层搜索。
- Layer 1： 包含全部 1,000 个向量，用于最终检索。

上层节点也是下层节点的一部分，使搜索能够在不同层级之间连续进行。课件采用的 10、100、1,000 是用于说明稀疏到稠密结构的示例规模。

1.5.3 HNSW 检索流程 Top-down Search

HNSW 采用自顶向下（Top-down）的搜索方式。

第一阶段：Layer 3 — Coarse Navigation

从最高层随机选择一个入口节点，在稀疏邻近图中搜索距离查询较近的候选节点。由于上层节点较少，可以通过较少的比较完成初步定位（Make Big Jumps Early）。

第二阶段：Layer 2 — Intermediate Search

以上一层找到的最佳候选节点作为本层的起点，在节点数量更多的邻近图中继续搜索，进一步缩小与查询向量的距离。

第三阶段：Layer 1 — Local Refinement

以 Layer 2 的最佳候选作为起点，在包含全部文档向量的底层图中执行最终搜索，得到近似最近邻结果。

这一过程的关键是：

$\text{Best Candidate}_{L3} \rightarrow \text{Entry Point}_{L2}$

$\text{Best Candidate}_{L2} \rightarrow \text{Entry Point}_{L1}$

每一层都继承上一层的搜索结果，而不是重新随机选择入口。

因此，HNSW 与 NSW 使用相似的邻居比较机制，但 HNSW 通过多层图组织搜索过程，减少在大规模底层图中进行无效导航的成本。

1.5.4 检索复杂度与性能 Complexity & Performance

KNN 的线性增长 vs HNSW 的近似对数增长

方法	检索增长趋势	主要原因
KNN	$O(N)$	与全部文档向量比较
HNSW	通常近似 $O(\log N)$	利用稀疏上层导航和逐层细化减少候选访问

这里的 $O(\log N)$ 是课件对 HNSW 常见检索表现的近似描述，不是所有数据分布、索引配置和召回率要求下都成立的严格保证。

HNSW 的实际检索速度还取决于：

- Index Settings（索引设置）： 影响图结构与搜索范围。
- Recall Targets（召回率目标）： 对结果准确性的要求越高，通常需要检查更多候选节点。

因此，HNSW 能够显著改善大规模向量检索的扩展性，但仍需要在速度、召回率与索引构建成本之间进行权衡。

1.6 KNN、NSW 与 HNSW 综合比较

维度	KNN	NSW	HNSW
检索类型	精确检索	近似检索	近似检索
数据结构	向量集合	单层邻近图	多层邻近图
搜索策略	全量距离比较	单层贪心游走	自顶向下逐层搜索
入口节点	不需要	随机图节点	最高层入口节点
候选范围	全部向量	当前节点及邻居	各层的局部候选
最近邻保证	有	无	无
扩展性	线性扫描	减少距离比较	通常近似对数增长
索引构建成本	无需建立近似图索引	需要预建邻近图	需要预建多层邻近图
主要局限	大规模查询成本高	可能陷入局部最优	索引复杂，性能受参数和召回目标影响

1.7 总结

本部分介绍的优化路径可以归纳为三个层次：

1. KNN → ANN：减少比较数量。 精确 KNN 对所有文档向量进行距离计算，查询成本随知识库规模线性增长。ANN 通过索引缩小候选范围，以牺牲全局最近邻保证换取更高的检索效率。
2. ANN → NSW：通过图结构组织候选搜索。 NSW 将文档向量组织为邻近图，查询时沿连接边进行局部贪心搜索，从而避免全量扫描，但可能因局部最优而错过全局最近邻。
3. NSW → HNSW：通过层级结构进一步优化导航。 HNSW 将邻近图划分为从稀疏到稠密的多个层级，先在上层快速定位，再在下层精细检索，通常能够获得更好的大规模检索效率。

最终结论： ANN 的优势来自预计算索引与减少在线距离比较。其代价是更高的索引构建成本，以及检索速度与召回率之间的权衡。

从系统执行阶段来看：

$\underbrace{ \text{Document Vectors} \rightarrow \text{Build Index} }_{\text{Offline Index Construction}}$

$\underbrace{ \text{Query Vector} \rightarrow \text{Visit Candidates} \rightarrow \text{Top-}k }_{\text{Online Retrieval}}$

将较高的计算成本前移至离线索引构建阶段，使在线查询不再需要遍历全部文档向量，是本部分可扩展检索技术的共同设计思路。

Part 2 分块、查询改写与检索编码器

Chunking, Query Rewriting and Retrieval Encoders

本部分围绕 RAG 检索质量的三个核心问题展开：文档如何组织、查询如何表达，以及查询与文档如何匹配。

- Chunking（文档分块）：确定知识库中的基本检索单元，平衡信息完整性、相关性与存储成本。
- Query Rewriting（查询改写）：消除用户输入中的歧义与冗余，使查询更适合检索。
- Retrieval Encoders（检索编码器）：通过不同的编码与交互机制，平衡检索精度、速度及存储开销。

三者分别作用于文档侧、查询侧和匹配阶段，共同决定最终检索证据的质量。

2.1 文档分块 Document Chunking

2.1.1 分块的必要性

Chunking 是将长文档划分为较小的文本单元的过程。其核心目标不是单纯缩短文本，而是使每个检索单元包含足够的信息，同时尽量减少无关内容。

分块的作用

目的	说明
Respect embedding token limits	避免文档超过嵌入模型的输入长度限制
Improve retrieval relevance	减少不同主题之间的语义干扰，提高检索相关性
Send focused context to the LLM	向生成模型提供聚焦且充分的证据，减少上下文窗口浪费

不分块的问题（Indexing without Chunking）

假设知识库包含 1,000 本书，每本书整体编码为一个向量。这种方式虽然减少了向量数量，却存在两个主要问题。

首先，整本书可能包含多个主题，而单个向量只能表达文档的整体语义，难以准确表示具体章节或段落。不同主题的信息混合后，可能降低特定查询的匹配效果。

其次，即使检索到相关书籍，也可能需要将大量无关内容送入 LLM，占用有限的上下文窗口。

因此，分块的关键是将检索粒度由整个文档细化为更有意义的局部内容。

2.1.2 分块粒度与大小权衡

文档可以按不同粒度划分：

Book → Page → Paragraph → Sentence

随着粒度减小，检索单元通常更加具体，但单个片段包含的上下文也会减少。

分块大小	主要问题	对检索的影响
Too large（过大）	包含多个主题和大量无关内容	语义混杂，匹配不够精确
Too small（过小）	缺少必要上下文	语义不完整，匹配能力减弱
Just right（适中）	保留回答问题所需的信息	兼顾相关性与信息充分性

Useful evidence contains enough information to answer the question without large amounts of unrelated material.

分块质量的核心标准是有效证据应包含足以回答问题的信息，同时避免大量无关内容。

因此，最佳分块大小并非固定值，而取决于文档结构、查询需求和模型输入限制。

2.1.3 主要分块方法

1. 固定长度分块（Fixed-size Chunking）

按照预设字符数划分文档，不考虑文本语义或结构。假设每块包含 $C = 250$ 个字符，且没有重叠，则第 k 个块的字符范围为：

其中，N 表示文档总字符数。

优点：实现简单、计算成本低、块大小可预测，适合作为分块策略的基线。

缺点：切分位置可能位于句子或概念内部，破坏语义完整性。

2. 重叠分块（Overlapping Chunking）

在固定长度分块的基础上，使相邻文本块共享部分内容，以减少边界处的上下文损失。设：

- C：Chunk Size，块长度。
- O：Overlap Size，重叠长度。
- S：Stride，分块步长。

三者关系为：

$ S=C-O $

重叠比例为：

$r=\frac{O}{C} $

课件采用：

$C=250,\quad O=25 $

因此：

$S = 250 - 25 = 225$

$r=\frac{25}{250}=10% $

Chunk	字符范围	与前一块的重叠
Chunk 1	1 - 250	无
Chunk 2	226 - 475	226 - 250
Chunk 3	451 - 700	451 - 475

对于从 1 开始计数的字符位置，第 k 个块的起点为：

${Start}_k=1+(k-1)(C-O) $

重叠分块能够保留边界附近的上下文，但也增加了向量数量、存储成本及检索结果的冗余。

忽略最后一个不足整块的片段，对于较长文档，重叠引起的存储量放大比例近似为：

$Storage Factor\approx\frac{C}{C-O} $

在课件示例中：

$\frac{250}{225}\approx1.111$

即相较于不重叠分块，存储的文本量约增加 11.1%。

3. 递归字符切分（Recursive Character Splitting）

递归字符切分优先利用文档的自然结构划分文本，而不是直接按照固定字符位置截断。

典型的分隔符优先级可以表示为：

Paragraph → Line → Sentence → Character

处理过程是先使用较高层级的分隔符；如果某个片段仍超过长度限制，再使用更细粒度的分隔符继续切分。

主要特点：

- 尽可能保留段落和句子的完整性。
- 不同文本块的长度可以不同。
- 相较于固定长度分块，更符合文档的自然结构。

4. 结构感知分块（Structure-aware Chunking）

对于具有明确结构的文档，应优先利用其原有结构信息确定分块边界。

文档类型	分块依据
HTML	标题标签、段落标签，如 <h1>、<p>
Python	函数定义、类结构
Markdown	标题层级、段落结构
普通文本	换行、段落或递归字符分隔符

例如，Python 代码按函数定义进行分块，可以保留函数签名、参数和实现逻辑之间的联系，避免将一个完整函数切割成无法独立理解的片段。

关键原则：分块应尽可能遵循文档已有的结构边界。

2.1.4 分块边界与语义完整性

Chunk Boundaries Can Change Meaning 分块不仅影响检索精度，还可能改变文本的原始含义。

That night she dreamed, as she did often, that she was finally an Olympic champion.

原句表达的是“她梦见自己成为奥运冠军”，并非真实发生的事件。

如果分块后仅保留：She was finally an Olympic champion. 则可能被误解为已经发生的事实。

相反，保留：She dreamed … She was finally an Olympic champion. 可以维持原始语义。

因此，分块应尽量保留影响事实判断的限定信息，例如时间、否定、条件、假设以及主语与谓语之间的关系。

这一问题也是引入语义分块与 LLM 分块的重要原因。

2.1.5 语义分块（Semantic Chunking）

语义分块根据相邻文本的语义相似程度确定边界，而非使用固定字符长度。其基本流程为：

1. 将当前文本块与下一句分别编码为向量。
2. 计算二者的语义距离。
3. 根据预设阈值判断是否合并。
4. 逐句重复，直至处理完整篇文档。

课件使用 Cosine Distance（余弦距离） 作为判定依据。

首先计算余弦相似度：

$\operatorname{sim}(\mathbf a,\mathbf b) = \frac{\mathbf a\cdot\mathbf b} {|\mathbf a||\mathbf b|} $

再定义余弦距离：

$d_{\cos}(\mathbf a,\mathbf b) = 1-\operatorname{sim}(\mathbf a,\mathbf b) $

其中：

- $\mathbf{a}$：当前 Chunk 的语义向量。
- $\mathbf{b}$：下一句的语义向量。
- τ：预设距离阈值。

简化判定规则为：

$\begin{cases} d_{\cos}(\mathbf a,\mathbf b)<\tau &\Rightarrow \text{Add to current chunk}\[4pt] d_{\cos}(\mathbf a,\mathbf b)\geq\tau &\Rightarrow \text{Start new chunk} \end{cases} $

即：

- 距离低于阈值：语义相近，合并。
- 距离达到或超过阈值：语义差异较大，创建新块。

注意这里使用的是 Distance，不是 Similarity，因此数值越小表示语义越接近。

以加拿大的枫糖浆、山脉与自然景观为例，说明语义分块可以将相关句子组织在一起，不必按照固定字符位置截断。

优点：能够保留相关语义，减少不合理的文本边界。

缺点：需要额外的 Embedding 和 Similarity/Distance Computation，增加预处理时间与计算成本。

Evaluate whether better retrieval justifies the extra computation.

因此，语义分块是否值得采用，应根据实际检索质量提升与计算成本进行评估。

2.1.6 基于 LLM 的分块 LLM-based Chunking

LLM-based Chunking 直接利用语言模型理解文档内容，并按照指定规则生成文本块。

与语义分块依赖向量距离不同，LLM 可以根据自然语言指令识别概念、主题转换和内容之间的逻辑关系。

典型指令包括：

- 将同一概念的相关内容保留在同一块中。
- 当文档开始讨论新主题时，创建新块。
- 尽量保留每个片段的语义完整性。

处理流程为：

$\text{Document} \xrightarrow{\text{LLM}} {C_1,C_2,\ldots,C_n} $

其中 C_i 表示模型生成的第 i 个文本块。这种方法能够进行更复杂的语义判断，但需要额外的 LLM 推理成本。

上下文感知增强（Context-aware Enrichment）

除了决定分块边界，LLM 还可以为已有的 Chunk 生成简短的上下文描述。

课件示例：

Original Fragment	Added Context
Reflections on Launching Our First App	App-launch title
After six months … We launched our first app	Launch announcement
The biggest lessons were around user feedback	Lessons learned
Thanks to Daniel Townsend	Contributors

其核心思想是：为缺乏上下文的文本片段补充必要的主题或角色信息，使其在检索时更容易被正确理解。

例如，一个只包含人名的致谢片段，如果缺乏文档背景，可能难以匹配“项目贡献者”相关查询。增加 Contributors 标签后，可以改善其语义表达。

上下文增强不要求必须采用 LLM 分块，也可以与固定长度、递归或语义分块结合使用。

成本发生在哪个阶段?

阶段	操作	成本特点
Before queries arrive	LLM 为 Chunk 添加上下文，并建立索引	增加离线预处理成本
When a query arrives	检索增强后的 Chunk，再生成答案	不需要为添加 Chunk 上下文额外调用 LLM

因此，上下文增强的主要成本发生在索引构建阶段，而不是每次查询时。

需要注意，这并不代表整个 RAG 查询过程没有 LLM 调用；生成最终答案仍然可能需要调用 LLM。

2.1.7 分块策略的综合比较

方法	切分依据	主要优势	主要局限
Fixed-size	字符数量	简单、高效、适合作为基线	容易破坏语义
Overlapping	固定长度 + 重叠区域	减少边界信息丢失	增加存储与冗余
Recursive	自然分隔符	较好保留文本结构	块长度不一致
Structure-aware	文档结构标记	保留逻辑结构	依赖文档格式
Semantic	向量语义距离	保留相关主题	额外 Embedding 成本

方法	切分依据	主要优势	主要局限
LLM-based	模型语义理解	更灵活的概念划分	推理成本与复杂度较高

选型建议：先建立简单基线，再根据检索效果决定是否引入更复杂的方法。

固定长度分块适合作为初始方案；语义分块和 LLM 分块可能改善效果，但应通过实验验证收益。上下文增强则可以与不同分块策略结合使用。

2.2 查询改写 Query Rewriting

分块优化的是文档侧，而查询改写优化的是用户输入侧。

其目标是将不明确、冗余或不适合检索的自然语言输入转化为更清晰的查询，同时保留用户的原始意图。

2.2.1 查询改写的必要性

用户在对话中经常使用省略、指代或非正式表达。

课件示例：Hey, can you tell me about that climate thing we talked about?

that climate thing 缺少明确的主题信息。如果检索器无法获取之前的对话内容，就难以确定用户实际需要检索什么。

因此，Query Rewriting 需要利用已有对话上下文补全指代；如果上下文仍不足，则应要求用户澄清，而不是自行猜测主题。

2.2.2 查询改写的主要操作

课件以医学文档检索为例。

原始查询：My dog yanked hard on the leash. Three days later, my shoulder is still numb and my fingers are all pins and needles.

改写后：Sudden shoulder traction; shoulder numbness and finger tingling for three days. Potential causes?

改写过程主要包括四项操作：

操作	英文术语	目的
消除歧义	Clarify ambiguous phrases	明确指代和模糊表达
术语规范化	Use domain terminology	使查询与专业文档用语一致
同义词扩展	Add synonyms	增加不同表述之间的匹配机会
删除冗余信息	Remove distracting information	减少无关信息对检索的干扰

在示例中，pins and needles 被规范化为 finger tingling，同时保留了症状持续三天这一重要信息。

查询改写的关键约束是：Preserve the user's meaning（保留用户原意）。

改写可以改变表达方式，但不应添加未经用户提供或上下文支持的事实，也不能改变用户真正想要解决的问题。

2.2.3 查询改写的工作流程

$Q_{\text{original}} \xrightarrow{\text{Query Rewriter (LLM)}} Q_{\text{optimized}} \xrightarrow{\text{Retriever}} D_{\text{relevant}}$

其中：

- Q_{original}：用户原始查询。
- $Q_{\text{optimized}}$：改写后的检索查询。
- D_{relevant}：检索到的相关文档。

查询改写发生在检索之前，属于在线处理阶段。

2.2.4 收益与代价

Potential Benefits	Trade-offs
Clearer queries：查询表达更清晰	Extra LLM latency and cost：增加调用延迟与费用
Better retrieval relevance：提高检索相关性	Risk of changing user intent：可能偏离用户原意

查询改写特别适用于对话式检索、指代较多的输入以及口语表达与专业文档用词差异较大的场景。

对于已经明确、简洁且包含准确术语的查询，额外改写不一定带来明显收益。

因此，是否启用 Query Rewriting 同样需要考虑检索质量与额外在线成本之间的权衡。

2.3 检索编码器 Retrieval Encoders

检索编码器决定查询与文档如何被表示和比较。

三种主要架构：Bi-encoder、Cross-encoder 和 ColBERT。

三者的区别在于编码方式、交互粒度以及计算与存储成本。

2.3.1 双编码器 Bi-encoder

Bi-encoder 将 Query 和 Document 分别编码为独立的语义向量。

$\mathbf q=E_q(Q) $

$\mathbf{d}_i = E_d(D_i)$

其中，E_q 与 E_d 分别表示查询编码器和文档编码器，两者可以共享权重。

随后，通过向量相似度计算相关性：

$s(Q,D_i)= \frac{\mathbf q\cdot\mathbf d_i} {|\mathbf q||\mathbf d_i|} $

对于大规模知识库，通常使用 ANN（Approximate Nearest Neighbor）快速搜索相近向量。

**关键特点 **

Precomputed Document Vectors（文档向量预计算）：文档可以在查询到来之前完成编码，并存储在向量数据库中。

因此：

- Offline：编码全部文档并建立索引。
- Online：仅编码新查询，再执行 ANN 搜索。

这种机制避免了每次查询都重新处理全部文档，具有较高的检索速度和可扩展性。

但由于 Query 与 Document 独立编码，模型在编码过程中无法直接建立两者之间的细粒度 Token 交互。

适用场景：大规模语义检索和第一阶段候选文档召回。

2.3.2 交叉编码器（Cross-encoder）

Cross-encoder 将 Query 与 Document 拼接后共同输入模型，直接生成相关性分数。

$s(Q, D_i) = f_{\text{cross}}(Q, D_i)$

与 Bi-encoder 不同，Cross-encoder 可以在编码过程中建模查询与文档之间的上下文交互。

课件示例：Query：Great places to eat in New York

Document	Relevance Score
Manhattan restaurants	0.7
New York subway	0.12
Brooklyn food scene	0.85

因此，排序结果为：$D_{\text{Brooklyn}} > D_{\text{Manhattan}} > D_{\text{Subway}} $

Cross-encoder 能够结合查询意图与文档语义进行更细致的相关性判断，而不只是匹配共同出现的词语。

优势

- 能直接建模 Query - Document 的上下文交互。
- 通常可以提供比 Bi-encoder 更精细的相关性评分。
- 适合对初步检索结果进行重新排序。

局限

Cross-encoder 必须针对每一个 Query - Document Pair 单独计算分数。

若候选文档数量为 N，则一次查询需要进行：$N_{\text{pair evaluations}} = N$次模型评分。

此外，相关性分数依赖当前 Query，因此不能像 Bi-encoder 的文档向量一样提前独立计算。

当文档规模达到百万级或十亿级时，直接使用 Cross-encoder 对全部文档评分通常成本过高。

适用场景：对较小候选集进行 Reranking（重排序），而非大规模初始检索。

2.3.3 ColBERT：Token-level Matching

ColBERT 采用 Late Interaction（后期交互）机制，在 Bi-encoder 的独立编码与 Cross-encoder 的联合交互之间取得平衡。核心思想是：分别编码 Query 和 Document，但保留每个 Token 的向量，而不是将整个文本压缩成单一向量。

假设查询包含 m 个 Token，文档包含 n 个 Token，则：

$Q={\mathbf q_1,\mathbf q_2,\ldots,\mathbf q_m} $

$D={\mathbf d_1,\mathbf d_2,\ldots,\mathbf d_n} $

其中：

- $\mathbf{q}_i$：查询中第 i 个 Token 的上下文向量。
- $\mathbf{d}_j$：文档中第 j 个 Token 的上下文向量。

与 Bi-encoder 每个文档只生成一个向量不同，ColBERT 为文档中的多个 Token 分别生成向量。

2.3.4 Token 相似度矩阵

ColBERT 首先计算每个 Query Token 与各个 Document Token 的相似度。

$S_{ij} = \text{sim}(\mathbf{q}_i, \mathbf{d}_j)$

如果采用点积作为相似度函数：

$ S_{ij}=\mathbf q_i^\top\mathbf d_j $

对于已归一化的向量，点积等价于余弦相似度。

所有 Token 两两比较后形成相似度矩阵：

$\mathbf S= \begin{bmatrix} S_{11}&S_{12}&\cdots&S_{1n}\\ S_{21}&S_{22}&\cdots&S_{2n}\\ \vdots&\vdots&\ddots&\vdots\\ S_{m1}&S_{m2}&\cdots&S_{mn} \end{bmatrix}$

矩阵维度为：$\mathbf S\in\mathbb R^{m\times n} $。每一行对应一个 Query Token，每一列对应一个 Document Token。

例如，查询中的 eat 可能与文档中的 cuisine 具有较高的语义相似度，即使二者并非相同单词。

这使 ColBERT 能够捕获比整篇文档向量更细粒度的语义匹配关系。

2.3.5 MaxSim Scoring

MaxSim 是 ColBERT 的核心评分机制。对于每一个 Query Token，找到与其最相似的 Document Token，并将这些最大相似度相加。

其公式为：

${ \operatorname{MaxSim}(Q,D) = \sum_{i=1}^{m} \max_{1\leq j\leq n} \operatorname{sim}(\mathbf q_i,\mathbf d_j) }$

如果使用点积：

$\operatorname{MaxSim}(Q,D) = \sum_{i=1}^{m} \max_{1\leq j\leq n} \mathbf q_i^\top\mathbf d_j $

公式包含两个运算：

1. Max：对每个 Query Token，在所有 Document Token 中寻找最佳匹配。
2. Sum：将所有 Query Token 的最佳匹配分数相加，得到文档总分。

课件示例中，各查询 Token 的最大相似度为：

Query Token	最大相似度
Great	0.3
places	0.3
to	0.2
eat	0.7
in	0.3
New	0.8
York	0.8

因此，Token 相似度列的求和结果：

$\begin{aligned} \operatorname{MaxSim}(Q,D) &=0.3+0.3+0.2+0.7={3.4} \end{aligned} $

需要注意：MaxSim 分数不是概率，也不要求落在 [0, 1] 区间内。 它是多个 Token 匹配分数的累加结果。

2.3.6 ColBERT 的存储与性能权衡

Bi-encoder 通常为每篇文档存储一个向量：$ D_i\rightarrow\mathbf d_i $

ColBERT 则为文档中的多个 Token 存储向量：$D_i \rightarrow \{\mathbf{d}_{i1}, \mathbf{d}_{i2}, \cdots, \mathbf{d}_{in}\}$

因此，ColBERT 需要更大的向量存储空间。

Pros	Cons
结合独立编码的可扩展性与 Token 级交互	需要存储大量 Token 向量
支持较细粒度的语义匹配	索引体积和存储成本较高
可用于实时或近实时检索	评分过程比单向量相似度更复杂

ColBERT 的优势在于：文档 Token 向量可以预计算，同时保留查询到来后进行细粒度匹配的能力。

因此，它在检索质量、速度和存储之间提供了一种折中方案。

2.4 三种检索编码器的综合比较

比较维度	Bi-encoder	Cross-encoder	ColBERT
编码方式	Query 与 Document 分别编码	Query 与 Document 联合编码	分别编码，保留 Token 向量
表示粒度	文档级向量	Query - Document Pair	Token 级向量
评分方法	向量相似度	模型直接预测相关性	MaxSim
交互机制	独立编码后比较	编码过程中交互	编码后 Token 级交互
检索质量	较好	通常较高	通常接近 Cross-encoder
检索速度	快	慢	中等
向量存储需求	较低	较低	较高
文档预计算	支持	无法预计算查询相关分数	支持
典型用途	大规模初始检索	候选结果重排序	细粒度语义检索

三种架构的本质区别

Bi-encoder：整体向量匹配

$s_{\text{bi}}(Q, D) = \text{sim}\left(E_q(Q), E_d(D)\right)$

Cross-encoder：联合编码评分

$s_{\text{cross}}(Q,D) = f_{\text{cross}}(Q,D) $

ColBERT：Token 级最大相似度求和

$s_{\text{ColBERT}}(Q,D) = \sum_{i=1}^{m} \max_{j} \operatorname{sim}(\mathbf q_i,\mathbf d_j)$

三种方法分别代表不同的检索设计取向：

- Bi-encoder 优先考虑检索效率和可扩展性。
- Cross-encoder 优先考虑精细的相关性判断。
- ColBERT 在保留文档预计算能力的同时，引入 Token 级交互以改善匹配质量。

2.5 本章总结

本部分的知识点可以归纳为三个相互衔接的层次。

层次	核心问题	主要技术	优化目标
Document Processing	文档应如何组织?	Fixed-size、Overlap、Recursive、Semantic、LLM Chunking	语义完整性与证据相关性
Query Processing	用户查询应如何表达?	Query Rewriting	消除歧义、规范术语、保留用户意图
Retrieval Matching	如何判断查询与文档是否相关?	Bi-encoder、Cross-encoder、ColBERT	匹配质量、速度与存储成本

完整流程可以概括为：

$\text{Documents} \xrightarrow{\text{Chunking}} \text{Chunks} \xrightarrow{\text{Encoding}} \text{Index}$

查询阶段：

$\text{User Query} \xrightarrow{\text{Query Rewriting}} \text{Optimized Query} \xrightarrow{\text{Retrieval}} \text{Relevant Chunks} $

最终：

$\text{Relevant Chunks}+\text{Query} \xrightarrow{\text{LLM}} \text{Answer} $

本部分的核心结论是：RAG 检索质量不仅取决于搜索算法，还取决于检索单元的设计、查询表达的质量，以及编码器对语义相关性的建模能力。

在实际系统中，应以简单且可扩展的方案建立基线，再通过实验判断语义分块、查询改写和更复杂的编码架构是否带来足以抵消额外成本的性能提升。

Part 3 Advanced RAG Architectures 进阶 RAG 架构

从优化单一检索模块，扩展到优化 RAG 系统的知识表示、检索决策、生成质量与整体工作流。

传统 RAG 通常采用固定的 Retrieve → Generate（检索 → 生成） 流程，但存在三个主要问题：

- 知识表示受限：传统文本 RAG 难以充分处理图像、图表和复杂 PDF 版式。
- 检索与生成缺乏自我判断：系统可能执行不必要的检索，使用不相关的证据，或生成缺乏证据支持的答案。
- 控制流程缺乏灵活性：固定的线性流程难以处理需要条件分支、反复修正或多任务并行的复杂问题。

因此，本章围绕三个方向展开：

方向	核心技术	主要解决的问题
Multimodal and PDF RAG	多模态嵌入、视觉 Patch 检索	如何检索非纯文本知识
Self-RAG	Reflection Tokens、自适应检索与自我批判	何时检索、证据是否相关、生成是否有依据

方向	核心技术	主要解决的问题
Agentic RAG Workflows	Routing、Evaluation、Iteration、Parallelism	如何组织复杂的检索与生成任务

3.1 Multimodal and PDF RAG 多模态与 PDF RAG

3.1.1 Multimodal RAG：多模态检索增强生成

传统 RAG 主要针对文本进行检索，而实际知识库可能同时包含文本（Text）、图像（Images），以及音频（Audio）和视频（Video）；多模态 RAG（Multimodal RAG）通过多模态嵌入模型（Multimodal Embedding Model），使不同模态的信息能够在统一的语义空间中表示与检索。

核心要求：Match the retriever and generator to the modalities you use.

即检索器（Retriever）和生成器（Generator）都必须支持系统所使用的模态。

- Retriever：能够处理相应模态并检索相关内容。
- Generator：能够理解检索返回的多模态信息，并据此生成回答。

例如，即使检索器成功找到了相关图像，如果生成模型无法理解图像内容，系统仍然无法充分利用这部分证据。

3.1.2 Multimodal Embedding Model：共享嵌入空间

多模态嵌入模型的关键是将文本和图像映射到 Shared Embedding Space（共享嵌入空间）。

对于文本 t 和图像 I，分别进行编码：

$\mathbf{e}_t = f_{\text{text}}(t)$

$\mathbf{e}_I = f_{\text{image}}(I)$

其中：

- f_{text}：文本编码函数。
- f_{image}：图像编码函数。
- $\mathbf{e}_t, \mathbf{e}_I$：处于共享向量空间中的表示。

共享空间的目标是使语义相近的内容具有相近的向量表示，而不受原始模态限制。

例如，课件展示了两个语义聚类：

语义类别	输入示例	向量空间中的关系
Dogs	“dog”“puppy”、狗的图像	语义相近，聚集在同一区域
Trees	“tree”、树的图像	聚集在另一区域

因此，文本查询可以检索图像，图像查询也可以检索相关文本。

跨模态相似度可用余弦相似度表示：

$$\operatorname{sim}(t,I) = \frac{\mathbf{e}_t^\top \mathbf{e}_I}{\|\mathbf{e}_t\|\|\mathbf{e}_I\|}$$

该公式是对共享向量空间检索原理的数学表达，课件本页没有单独给出此公式。

3.1.3 Multimodal Vector Retrieval：多模态向量检索

多模态向量检索同时处理知识库内容和用户查询，多模态查询与知识库通过共享向量空间实现语义匹配

完整流程可以概括为：

1. 将多模态知识库中的文本、图像编码为向量。
2. 使用兼容的多模态编码模型处理用户查询。
3. 在共享向量空间中进行相似度匹配，返回相关内容。
4. 将检索结果提供给支持相应模态的生成模型。

与传统 RAG 的根本区别：传统文本 RAG 主要比较文本语义，多模态 RAG 则能够进行跨模态语义匹配。

3.1.4 PDF RAG：基于视觉 Patch 的文档检索

PDF 和 Slides（幻灯片）通常具有较高的信息密度，可能同时包含：Text：正文文本、Captions：图注、Charts：图表、Images：图像。

传统方法依赖复杂的文档检测和版面解析算法（Sophisticated Detection Algorithms）

PDF RAG 采用 将PDF 页面切分为视觉 Patch 后分别向量化 (Split into patches, then vectorize)

主要优势：保留原始文档的视觉内容（Preserve Visual Content），包括文字、图形及版式信息，减少仅依赖文本提取造成的信息损失。

PDF RAG 与 ColBERT 的联系

Each prompt token finds the most similar patch in a document, and the maximum similarity scores are summed.

PDF RAG 的评分方式与 ColBERT 相似：设查询包含 m 个 Token，文档被切分为 n 个视觉 Patch：

$ Q={\mathbf q_1,\ldots,\mathbf q_m} $ $D = \{\mathbf{p}_1, \cdots, \mathbf{p}_n\}$

则其 MaxSim 风格的相关性评分可表示为：

${ S(Q,D)= \sum_{i=1}^{m} \max_{1\le j\le n} \operatorname{sim}(\mathbf q_i,\mathbf p_j) }$

其中：

- $\mathbf{q}_i$：第 i 个查询 Token 的向量。
- $\mathbf{p}_j$：第 j 个文档 Patch 的向量。
- max：为每个查询 Token 选取最相似的 Patch。
- Σ：汇总所有查询 Token 的最大相似度。

比较维度	ColBERT	PDF RAG
文档表示	多个 Token 向量	多个视觉 Patch 向量
匹配方式	Query Token 对 Document Token	Query Token 对 Document Patch
评分机制	MaxSim + Sum	MaxSim + Sum
主要特点	细粒度文本匹配	保留文档视觉细节

PDF RAG 的代价

一份文档需要存储多个 Patch 向量，因此会带来：

- Larger Vector Index：向量数量增加，索引和存储成本上升。
- Higher Matching Cost：需要比较更多向量，增加检索计算开销。

因此，PDF RAG 需要在视觉信息保留、检索质量、存储成本和匹配成本之间进行权衡。

3.2 Self-RAG：Retrieval and Self-Critique 检索与自我批判

3.2.1 Simple RAG 的有效性与局限

在介绍 Self-RAG 前，需要明确传统 RAG 的优势及其不足。

1. Effectiveness of Simple RAG

不同知识流行度下检索增强的准确率趋势，研究展示的主要趋势是：

- Rare（冷门知识）：检索带来的准确率提升最大。
- Neutral（中等流行度知识）：检索仍然具有明显收益。
- Popular（热门知识）：检索收益可能较小，甚至出现负面影响。

原因：语言模型对常见知识通常具有较充分的参数化记忆（Parametric Memory），而冷门知识更依赖外部的非参数化记忆（Non-parametric Memory）。

结论：Retrieval improves accuracy most on rare questions in this study.

需要注意，这是课件引用研究中的实验结论，并不意味着检索对所有冷门问题都一定有效。

2. Limitations of Simple RAG

传统 RAG 主要存在以下四类局限：

局限	含义	影响
Unnecessary Retrieval	对无需外部知识的问题仍执行检索	增加不必要的计算成本
Unhelpful Context	检索结果不相关或具有误导性	模型可能被错误上下文干扰
Unsupported Citations	引用来源不一定支持生成的结论	降低答案的可验证性
Increased Latency	检索内容增加输入上下文长度	增加编码和推理延迟

说明无效检索的危害的典型案例

User Query: Who is Sam Altman?

Retrieved Context: Samuel Adams (Sam Adams) was an American statesman, political philosopher and a founding father of the U.S.

Incorrect Response: He was a U.S. founding father.

错误产生的原因是检索结果中的 Samuel Adams 与查询对象 Sam Altman 不一致，而生成模型没有识别这种不相关性，直接采纳了错误证据。

这一案例说明：检索结果并不必然提高答案质量；检索相关性与证据支持性必须得到验证。

3.2.2 Self-RAG 的核心机制: Reflection Tokens

传统 RAG 通常使用固定的两阶段流程：

$\text{Input}\rightarrow\text{Retrieve}\rightarrow\text{Generate} $

Self-RAG（Self-Reflective Retrieval-Augmented Generation）引入可学习的反思 Token（Learned Reflection Tokens），使模型能够在检索和生成过程中进行自主判断。

三个判断：

Reflection Token	核心问题	判断阶段	作用
Retrieve?	Is retrieval needed?	检索之前	判断是否需要外部知识
Relevant?	Are documents relevant?	检索之后	判断检索证据是否相关
Supported?	Is generation supported?	生成之后	判断生成内容是否受到证据支持

这三个判断分别对应：

1. Retrieval Decision：是否检索。
2. Evidence Relevance：证据是否相关。
3. Generation Grounding：生成内容是否有依据。

与固定检索流程相比，Self-RAG 的关键变化是将检索与证据评估纳入模型学习的决策过程。

3.2.3 Self-RAG Control Flow：自适应检索与生成

这三个判断共同构成 Self-RAG 的自适应检索与自我批判机制：

第一阶段：检索需求判断

模型根据问题决定是否检索。如果已有知识足以回答，则可以直接生成；否则调用检索器获取外部证据。

第二阶段：证据相关性判断

检索完成后，模型评估返回文档是否与问题相关，避免将无关内容直接纳入生成过程。

第三阶段：生成支持度判断

生成答案后，模型评估回答中的陈述是否得到检索证据支持。

3.2.4 Relevance vs. Support：相关性与支持度

Self-RAG 不仅关注检索到的内容是否相关，还关注生成的具体陈述是否得到证据支持。

两者的区别是：

- Relevance：内容是否与用户问题相关。
- Support：检索证据是否足以支持生成的陈述?

相关内容不一定能充分支持生成的结论。

课件案例: Scientific Literature Synthesis

用户请求: Suggest papers showing how LLMs help scientists synthesize scientific literature.

模型检索到以下两类证据：

- OpenScholar is a retrieval-augmented LM designed to synthesize literature.
- GPT-4o has been shown to generate new research ideas.

在此基础上，模型生成不同陈述并评估其支持程度。

Generated Statement	Relevance	Support
OpenScholar is an LM for literature synthesis.	Relevant	Fully supported
Studies show GPT-4o can help scientists with idea generation and literature synthesis.	Relevant	Partially supported

第一条陈述与检索证据一致，因此被评为 Fully supported。

第二条陈述虽然与科研任务相关，但证据仅支持 GPT-4o 能够帮助产生研究想法，并未充分证明其具有文献综合能力，因此被评为 Partially supported。

两个支持度评分（用于体现候选陈述的证据支持程度）：

OpenScholar 陈述：0.9 Fully supported

GPT-4o 陈述：0.5 Partially supported

Self-RAG 的关键意义：将检索需求、证据相关性和生成支持性转化为模型可学习、可评估的判断，从而提高答案的可靠性。

3.3 Agentic RAG Workflows 智能体 RAG 工作流

3.3.1 From LLM to Agent System：从单模型到智能体系统

传统单模型调用通常采用：

$ \text{Prompt}\rightarrow\text{LLM}\rightarrow\text{Response} $

这种结构适合相对简单的任务，但对于需要规划、检索、评估和多步骤处理的复杂任务，单次调用可能难以完成全部工作。而Agent System（智能体系统）通过将复杂任务分解为一系列步骤与决策，并结合外部工具完成任务。

课件展示了三类典型子任务：

- Planning LLM：规划任务步骤。
- Research LLM：执行信息检索和研究。
- Writing LLM：整合信息并生成最终回答。

智能体系统可以访问外部工具，包括代码工具、数据库和互联网资源。

其主要特点是 a series of steps and decisions（由多个步骤和决策组成），而不是依赖单次模型调用。

3.3.2 Example Agentic RAG System

Agentic RAG：路由、证据评估、重新检索与回答生成

完整逻辑为：

1. Router LLM 判断问题是否需要检索。
2. 如果不需要检索，直接生成回答。

3. 如果需要检索，则执行 Retrieve。
4. Evaluate Evidence 判断当前证据是否充分。
5. 如果证据不足，返回检索阶段再次检索；如果充分，则生成带引用的回答（Generate + Cite）。
6. 输出最终 Response。

这个系统同时包含条件分支与迭代机制，说明 Agentic RAG 并不局限于单一工作流模式。

3.3.3 Agentic Systems as Flowcharts：流程图式系统设计

智能体系统可以表示为流程图（Flowchart），由不同的节点和连接关系组成。每个 LLM 节点完成一个特定任务，并将输出传递给下一节点。

Different LLMs for different tasks. 不同任务可以采用不同规模和能力的模型：

任务	推荐模型类型	设计目的
Routing	Lightweight Models	低成本完成路径选择
Evaluation	Lightweight Models	高效判断结果质量
Response Generation	Larger Models	提高最终回答质量
Citations	Specialized Models	处理引用与证据对应关系

因此，Agentic RAG 的设计不仅涉及模型能力，还包括任务分解、模型分配、流程控制和外部工具调用。

3.3.4 Four Workflow Patterns：四种典型工作流

四种基本工作流：

1. Sequential Workflow：顺序工作流

信息按照固定顺序依次经过多个 LLM 调用，前一步的输出成为后一步的输入。

示例为 Q&A Chatbot：

$\text{Parse} \rightarrow\text{Rewrite} \rightarrow\text{Retrieve} \rightarrow\text{Generate with Citations} $

特点：结构简单、执行顺序明确，适用于步骤固定且存在前后依赖关系的任务。

其局限是缺少动态决策和自动回退机制，前一阶段的错误可能传递至后续阶段。

2. Conditional Workflow：条件工作流

条件工作流通过 Router 判断输入应进入哪条处理路径。

例如：

- Knowledge needed：先检索外部知识，再生成回答。
- No retrieval needed：跳过检索，直接回答。

特点：根据输入动态选择执行路径，可以减少不必要的检索开销，并降低无关上下文带来的干扰。

其主要风险是 Router 的判断错误可能导致系统选择不合适的处理路径。

3. Iterative Workflow：迭代工作流

迭代工作流允许系统根据评估结果返回之前的步骤，重新执行任务。

课件使用 AI Coding 作为示例：

$\text{Generate Code} \rightarrow\text{Run Tests} \rightarrow\text{Identify Errors} \rightarrow\text{Revise Code} \rightarrow\text{Test Again} $

系统包含两个主要角色：

- Writer：生成候选结果。
- Evaluator：评估结果并提供 Feedback。

当评估未通过时，Feedback 返回 Writer，触发下一轮修改。

终止条件：

$\text{Stop}= \begin{cases} \text{Accepted}, & \text{结果满足要求}\ \text{Iteration Limit}, & \text{达到最大迭代次数} \end{cases} $

设置最大迭代次数 N 可以避免无限循环。

特点：适用于需要多轮验证和改进的任务，例如代码生成、答案修订和证据补充。

4. Parallel Workflow：并行工作流

并行工作流将任务分解为多个相互独立的子任务，同时执行，再将结果整合为最终回答。

核心组件包括：

- Orchestrator：拆分任务并协调执行。
- LLM Agents / Router：执行独立子任务或负责分配任务。
- Synthesizer：整合多个子任务的结果。

课件示例为 AI Travel Planner（AI 旅行规划系统），可以分别处理航班、住宿和景点信息，再整合为完整行程。

并行执行的前提是 Independent Subtasks（子任务相互独立）。若子任务之间存在数据依赖，则不能简单并行。对于 n 个独立子任务，其顺序执行时间可近似表示为：

$T_{\text{seq}}=\sum_{i=1}^{n}T_i $

并行执行时间可近似表示为：

$T_{\text{parallel}} \approx \max(T_1,T_2,\ldots,T_n)+T_{\text{overhead}} $

其中 T_{overhead} 包含编排、通信和结果合成开销。

特点：通过同时执行独立任务降低整体等待时间，但需要额外的任务协调与结果合成机制。

3.3.5 四种工作流综合比较

Workflow	控制流特征	核心组件	适用场景
Sequential	线性执行	多个顺序 LLM 调用	固定步骤的问答处理
Conditional	根据条件选择分支	Router	按需检索
Iterative	评估后回退并重新执行	Writer + Evaluator	代码修正、答案优化
Parallel	独立子任务同时执行	Orchestrator + Synthesizer	多来源研究、旅行规划

四种工作流并非互斥关系，而是可以组合使用的控制结构。

例如，一个完整的 Agentic RAG 系统可以先通过 Router 判断是否需要检索，再并行查询多个知识源，随后评估证据充分性；若证据不足，则重新检索，最终生成带引用的回答。

3.3.6 Implementing Agentic Systems：工程实现

三项实现原则：

第一，简单系统可以手动实现。

对于规模较小、步骤明确的 Agentic System，可以直接编写工作流逻辑，无需引入复杂框架。

第二，复杂系统可以使用工具、库或平台。

当系统包含较多模型调用、条件分支、循环及并行任务时，可以考虑使用专门的工具、库或平台进行管理。

第三，不同工作流模式可以组合。

实际智能体系统通常综合使用顺序、条件、迭代和并行结构，以满足不同任务需求。

3.4 本章总结

本章的三个进阶方向分别从知识表示、模型决策和系统控制流三个层面改进传统 RAG。

层面	技术	核心改进
Knowledge Representation	Multimodal RAG	通过共享嵌入空间支持跨模态检索
Document Retrieval	PDF RAG	将视觉 Patch 向量化，保留文档图表和版式
Retrieval Decision	Self-RAG	使用 Reflection Tokens 判断是否检索
Evidence Verification	Self-RAG	判断证据相关性与生成支持度
System Orchestration	Agentic RAG	将复杂任务拆分为多个 LLM 节点
Workflow Control	Sequential / Conditional / Iterative / Parallel	支持顺序执行、动态分支、循环修正与并行处理

1. Multimodal RAG 的基础是 Shared Embedding Space，使不同模态的信息能够进行语义匹配。
2. PDF RAG 通过视觉 Patch 保留文档内容，并使用类似 ColBERT 的 MaxSim 评分机制，但需要承担更高的向量存储和匹配成本。
3. Simple RAG 对冷门知识问题具有明显收益，但不必要的检索、无关上下文、错误引用及更长的输入上下文可能降低系统表现。
4. Self-RAG 通过 Retrieve?、Relevant? 和 Supported? 三类反思判断，实现自适应检索与证据支持性评估。
5. Agentic RAG 将复杂任务组织为由多个模型节点及外部工具构成的工作流，不同节点可以采用不同能力与成本的模型。
6. Four Workflow Patterns 分别提供线性执行、条件路由、迭代反馈和并行执行机制，并且可以组合使用。

结论： 进阶 RAG 的目标不只是检索更多信息，而是使系统能够处理更丰富的知识形式、判断何时及如何使用证据，并通过灵活的工作流提高答案质量与执行效率。

Part 4 RAG Systems Evaluation RAG 系统评估

本章核心： RAG 系统的评估不仅需要衡量检索结果和生成答案的质量，还需要考察系统在真实生产环境中的响应延迟、吞吐能力与可扩展性，并通过组件级性能分析定位和优化瓶颈。

本章围绕三个方面展开：

1. Evaluation Challenges：真实用户请求的不可预测性，以及高并发环境下的性能压力。
2. Performance and Quality Metrics：从性能和质量两个维度评估完整 RAG 系统。
3. Evaluation and Optimization：通过延迟分析、模型优化以及自动化与人工评估改进系统。

4.1 Evaluation Challenges RAG 系统评估的挑战

与固定测试集上的离线检索评估不同，生产环境中的 RAG 系统需要持续处理复杂、多样且不断增长的用户请求，因此面临两项主要挑战。

4.1.1 Unpredictability of Prompts：用户输入的不可预测性

真实用户的请求具有创造性（Creative）和不可预测性（Unpredictable），即使经过严格测试，也无法预先覆盖系统可能接收到的所有问题。

课件通过 Google AI Overview 对异常问题的回答说明这一风险：当用户询问每天应该吃多少石头时，系统可能将不可靠的检索内容转化为具有误导性的建议。

这反映了 RAG 系统的两个问题：

- 有限的测试覆盖率：离线测试集无法穷举真实用户的输入类型。
- 较高的生产错误成本：不准确或缺乏证据支持的回答可能在实际应用中造成严重影响。

因此，离线测试表现良好并不意味着系统在生产环境中始终可靠，还需要持续评估真实请求及其生成结果。

4.1.2 Scaling Performance：系统性能的可扩展性

随着用户数量和请求流量增加，RAG 系统需要处理更高的计算负载。

课件强调三个方面：

- Latency and Load：流量增长会增加系统负载，并可能导致响应延迟上升。
- Memory and Compute Costs：请求数量增加会提高内存与计算资源需求。
- Scalability：系统需要在扩大服务规模的同时维持较高的性能。

两个挑战分别对应不同的评估重点：

评估挑战	核心问题	主要关注
Prompt Unpredictability	能否可靠处理未知或异常输入	回答质量与可靠性
Scaling Performance	能否在请求增长时保持服务能力	延迟、吞吐与资源开销

结论： RAG 系统评估必须同时覆盖 Quality（质量）和 Performance（性能），仅衡量检索准确率不足以反映系统的整体表现。

4.2 Performance and Quality Metrics 性能与质量指标

课件将系统评估划分为两个维度：

- Performance：How efficiently does it run? 衡量系统运行效率。
- Quality：How useful are the results? 衡量检索与生成结果的有效性。

四项核心指标如下：

评估维度	Metric	定义	评估对象
Performance	Latency	完成一次请求所需的时间	响应速度
Performance	Throughput	单位时间内处理的请求数量	系统处理能力
Quality	Relevance	检索文档是否与用户问题相关	检索质量
Quality	Faithfulness	生成答案是否得到证据支持	生成可靠性

4.2.1 Performance Metrics：性能指标

1. Latency：响应延迟

Latency 表示系统从接收用户请求到返回响应所经历的时间。

对于第 i 个请求： $ L_i=t_{\text{response},i}-t_{\text{request},i} $

其中，$t_{\text{request},i}$ 为请求到达时间，$t_{\text{response},i}$ 为响应完成时间。

若统计 N 个请求，其平均延迟为： ${ L_{\text{avg}}=\frac{1}{N}\sum_{i=1}^{N}L_i }$，延迟越低，通常意味着用户等待时间越短。

2. Throughput：系统吞吐量

Throughput 表示系统在单位时间内能够处理的请求数量，常以 Requests per Second（RPS）衡量。

${ \text{Throughput}=\frac{N_{\text{completed}}}{\Delta t} }$

其中：

- $N_{\text{completed}}$：统计时间内完成的请求数。
- Δt：统计时间长度。

吞吐量越高，说明系统在给定条件下能够处理更多请求。

Latency 与 Throughput 的区别： Latency 衡量单个请求的完成时间，Throughput 衡量系统整体的处理能力。二者需要结合分析，高吞吐量并不必然意味着低延迟。

4.2.2 Quality Metrics：质量指标

1. Relevance：检索相关性

Relevance 衡量检索返回的文档是否与用户问题相关。

其评估对象是： $ \text{Query}\longleftrightarrow\text{Retrieved Documents}$

这一指标与前面介绍的检索评估指标相衔接，例如：

- Precision@K：前 K 个结果中相关文档的比例。

- Recall@K：前 K 个结果覆盖全部相关文档的比例。
- MRR：首个相关文档的排名质量。

这些指标用于量化不同方面的检索相关性。

2. Faithfulness：生成忠实度

Faithfulness 衡量模型生成的陈述是否能够得到提供给模型的证据支持。

其评估对象是： Generated Response ↔ Retrieved Evidence

例如，若检索文档明确记载某航天任务有两名宇航员，而模型回答三名，即使文档检索正确，生成结果仍存在 Faithfulness 问题。

因此：

- Relevance 关注是否检索到了正确的信息。
- Faithfulness 关注生成答案是否正确使用了这些信息。

二者相互关联，但不能互相替代。高检索相关性并不保证生成答案具有高忠实度。

4.3 System Latency Analysis and Optimization 系统延迟分析与优化

4.3.1 End-to-End Latency：端到端延迟

RAG 系统通常采用以下处理流程：

端到端延迟（End-to-End Latency）表示从用户提交问题到系统返回完整响应的总耗时。

对于顺序执行的 RAG 流程，可以将其分解为：$${ T_{\text{total}} = T_{\text{query}} + T_{\text{retrieval}} + T_{\text{generation}} + T_{\text{other}} } $$

其中：

- T_{query}：查询处理时间。
- $T_{\text{retrieval}}$：检索耗时。
- $T_{\text{generation}}$：LLM 生成耗时。
- T_{other}：其他处理、通信等开销。

两个测量层次：

测量层次	指标	作用
System-Level	End-to-End Latency	判断系统整体响应是否存在性能问题
Component-Level	Retriever Timing / Generation Timing	定位延迟主要来自哪个组件

核心原则： 端到端延迟用于发现整体性能问题，组件级测量用于定位具体瓶颈。

4.3.2 Latency Bottleneck：延迟瓶颈

典型 RAG 流程： Query Processing、Retrieval 和 LLM Generation 三部分，Transformer Inference 往往是主要延迟来源。

课件的主要结论是：

- Transformer Inference can dominate latency.
- LLM Calls are often the main bottleneck.
- Retrieval and database operations are typically faster.

但这一结论并非适用于所有配置。如果检索阶段包含高成本的重排序或复杂计算，Retriever 也可能成为主要瓶颈。因此，优化应当以实际测量结果为依据，而不是预先假定某一组件最慢。

4.3.3 Reducing LLM Latency：降低生成延迟

当组件级测量表明 LLM 推理是主要瓶颈时，课件提出两种优化方法。

1. Smaller or Quantized Models：使用小模型或量化模型

- Smaller Model：采用参数规模较小的模型，减少推理计算需求。
- Quantized Model：采用低精度数值表示模型参数，降低存储与计算资源需求。

两种方法都可能降低推理成本，但实际加速效果及输出质量变化需要通过 Benchmark 验证。

2. Route by Task Complexity：根据任务复杂度进行模型路由

Router 根据任务复杂程度选择不同模型：

- 简单任务交由较小的模型处理，以减少推理成本。
- 复杂任务交由能力更强的模型处理，以维持回答质量。

这一策略通过差异化分配计算资源，在效率和质量之间进行权衡。

Benchmark the Speedup：评估加速效果

优化完成后，需要比较优化前后的推理耗时。

加速比（Speedup）可以表示为： ${ \text{Speedup} = \frac{T_{\text{before}}}{T_{\text{after}}} }$

例如，加速比为 2 表示优化后完成相同任务所需的时间为原来的一半。

但性能优化不能只关注加速比，还应检查 Relevance 与 Faithfulness 是否受到影响。

因此，完整的优化过程是： ${ \text{Measure} \rightarrow \text{Locate Bottleneck} \rightarrow \text{Optimize} \rightarrow \text{Benchmark} } $

即先测量系统表现，再定位瓶颈，针对性实施优化，最后验证性能和质量变化。

4.4 Evaluation Methods 系统评估方法

除直接测量性能与质量指标外，区分两种评估器：Code-Based Evaluators 与 Human Feedback Evaluators。

4.4.1 Code-Based Evaluators：基于代码的评估器

Code-Based Evaluators 使用程序自动检查系统运行情况和可明确判定的输出要求。

典型检查包括：

- Track Requests per Second：统计每秒请求处理数量，衡量吞吐量。
- Check for Valid JSON Output：检查输出是否符合合法 JSON 格式。

- Low Evaluation Cost：自动化评估通常成本较低，执行简单。

其优势是便于自动化、成本低，适合频繁执行的系统检查。

但传统规则型代码评估器通常难以直接判断开放式回答的语义正确性或证据支持程度。

4.4.2 Human Feedback Evaluators：人工反馈评估器

Human Feedback Evaluators 通过用户或人工评审人员评价系统输出。

三种方式：

- Thumbs Up / Down Ratings：通过点赞或点踩收集总体评价。
- Detailed Text Feedback：收集具体文字反馈，识别回答存在的问题。
- Manual Quality Assessments：通过人工审查判断回答的相关性、正确性与证据支持程度。

人工评估能够发现代码规则难以识别的语义和质量问题，但需要更多时间和人力成本。

4.4.3 两类评估方法的比较

比较维度	Code-Based Evaluators	Human Feedback Evaluators
成本	较低	较高
自动化程度	高	较低
评估重点	吞吐量、输出格式等可程序化检查的内容	用户体验、语义质量、证据支持
主要优势	简单、高效、可重复执行	能发现代码检查遗漏的问题
主要局限	难以覆盖复杂语义问题	成本高，难以全面覆盖所有请求

结论： 两种方法具有互补性。代码评估器适合大规模自动化检查，人工反馈评估器则用于识别难以通过固定规则判断的质量问题。

4.5 本章总结

技术方向	关键方法	核心权衡
Scalable Retrieval	ANN、HNSW	检索速度与 Recall
Chunking and Query Processing	Chunking、Query Rewriting	上下文保留与检索有效性
Ranking	Cross-Encoder、ColBERT	排序质量与计算、存储成本
Advanced RAG	Multimodal RAG、Self-RAG、Agentic Workflows	系统能力与架构复杂度
System Evaluation	Relevance、Faithfulness、Latency	结果质量与运行效率

五项核心原则

1. Scalable Vector Search

使用 Approximate Nearest Neighbor（ANN）提高大规模向量检索速度，但需要权衡速度与检索召回率。

2. Effective Chunking and Query Processing

通过合理分块和查询改写提高检索有效性，同时保留原始上下文及用户意图。

3. Improved Ranking

利用 Cross-Encoder 或 ColBERT 改善排序质量，但需要考虑额外的计算开销和向量存储成本。

4. Advanced RAG Architectures

根据任务需求引入多模态证据、Self-RAG 自我批判机制和 Agentic Workflows，提高系统对复杂任务的处理能力。

5. Measurement-Driven Optimization

同时评估 Retrieval Quality、Answer Faithfulness 和 End-to-End Latency，并针对实际测得的性能瓶颈实施优化。

核心关系：

- Relevance 与 Faithfulness 分别衡量检索证据的相关性和生成内容的证据支持程度。
- Latency 与 Throughput 分别衡量单次请求响应时间和系统单位时间处理能力。
- End-to-End Latency 与 Component-Level Timing 分别用于识别整体性能问题和定位具体瓶颈。
- Code-Based 与 Human Feedback Evaluators 分别提供低成本自动化检查和更深入的人工质量判断。

最终，RAG 系统的优化应遵循 Evaluation → Bottleneck Identification → Targeted Optimization → Benchmarking 的过程，在检索质量、生成可靠性、响应速度和资源成本之间取得平衡。

4.6 In-Class Exercise

题目	正确答案	核心依据
Q1. ColBERT 的 Token-Level Representation 相比每篇文档一个向量的主要权衡是什么?	A	更细粒度的匹配能力，但需要存储更多向量
Q2. HNSW 为什么先搜索稀疏的上层，再进入密集的底层?	C	先进行大范围导航，再逐层缩小范围，完成局部精细搜索
Q3. 从精确搜索切换到 ANN 的主要权衡是什么?	B	减少距离比较次数，提高速度，但可能错过真正的最近邻

$\text{ColBERT}: \quad \text{Matching Detail} \leftrightarrow \text{Vector Storage} $

$\text{HNSW}: \quad \text{Coarse Navigation} \rightarrow \text{Local Refinement} $

$ \text{ANN}: \quad \text{Search Speed} \leftrightarrow \text{Retrieval Recall} $

其中，ColBERT 的多向量表示不同于 Cross-Encoder 对查询与文档进行联合编码；HNSW 的上层用于导航，并非删除不相关文档；ANN 减少的是搜索时的比较次数，并不必然减少存储文档的数量或向量维度。

5583 Agentic Computing 3

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_5/knowledge/5583-Agentic-Computing-3.md`

5583 Agentic Computing3

Lecture 8 智能体系统设计 Design of Agentic Systems

本讲围绕智能体系统（Agentic Systems）的设计展开，重点讨论如何从单一语言模型构建具有自主决策、工具调用、任务规划和反馈修正能力的智能体系统。

课程内容按照以下逻辑组织：

1. 智能体系统基础（Agentic System Foundations）：理解 AI Model、AI System、Agent 和 Multi-Agent System 的区别，掌握智能体的组成及自主性程度。
2. 智能体工作流设计（Agentic Workflow Design）：理解顺序、并行、迭代和条件工作流，学习任务分解与组件选择。
3. 智能体推理与决策（Agentic Reasoning and Decision-Making）：研究智能体如何利用感知、记忆、目标与约束选择行动。
4. 自反思与反馈优化（Self-Reflection and Feedback）：理解内部反思、外部反馈及其在代码生成、SQL 查询等任务中的应用。

Part 1 智能体系统基础 Agentic System Foundations

1.1 从语言模型到智能体系统

人工智能应用可以按照系统能力与组织形式，区分为四个层次：

AI Model → AI System → Agent → Multi-Agent System

从基础模型能力到自主行动、再到多智能体协作的系统演进

1.1.1 四个层次

层次	核心定义	主要特征
AI Model	提供基础计算与智能能力的模型	生成文本、理解信息、进行预测
AI System	将模型能力集成到应用程序中	包含模型、界面、数据及业务逻辑
Agent	能够围绕目标自主选择行动的系统	具有决策、规划、工具调用和反馈能力
Multi-Agent System	由多个具有决策能力的智能体组成的系统	支持智能体之间的交互、分工与协作

1. AI Model：基础模型

模型提供计算能力，但通常不直接负责完整的业务流程。例如，LLM 可以根据输入生成文本。

这一形式描述了单次模型调用，但并不包含外部工具、任务执行或环境反馈。

2. AI System：人工智能系统

AI System 将模型能力封装为能够完成实际任务的应用。例如，智能客服系统可能包含：LLM：理解用户问题并生成回复、数据库：保存用户信息与订单记录、API：查询物流状态、用户界面：展示交互结果。

因此，AI System 的重点是将多个组件组织为完整应用，而不一定具备自主规划能力。

3. Agent：智能体

Agent 不仅生成信息，还能够根据目标与环境状态决定下一步行动。

典型行为包括：判断是否需要检索信息、选择合适的外部工具、将复杂任务分解为多个步骤、根据执行结果调整后续行动、判断任务是否已经完成。

智能体的关键特征是 Goal-directed Action（目标导向的行动）。

4. Multi-Agent System：多智能体系统

多智能体系统由多个能够独立作出一定决策的智能体组成。不同智能体可以承担不同职责，例如：Research Agent：负责资料检索、Analysis Agent：负责信息分析、Writing Agent：负责内容生成、Review Agent：负责质量检查。

多个智能体通过通信、任务分配和结果整合共同完成复杂任务。需要注意，增加智能体数量不一定提高系统性能。多智能体协作也可能引入通信开销、协调冲突及错误传播。

1.2 智能体的核心组成 Components of an Agent

将智能体能力划分为核心能力（Core Capabilities）与附加能力（Additional Capabilities）。

1.2.1 核心能力

核心能力	英文	作用
感知	Perception	获取并理解环境或用户输入
自主行动	Agency	根据目标选择并执行行动
推理与规划	Reasoning and Planning	分析当前状态并组织行动步骤
记忆	Memory	保存和利用历史信息
反馈与学习	Feedback / Learning	根据执行结果调整行为

1. Perception：感知

感知是智能体获取环境信息的过程。

输入可以来自用户文本、文件、数据库、搜索结果或外部系统。

感知不仅意味着接收数据，还包括从输入中提取与当前任务相关的信息。

2. Agency：自主行动

Agency 指智能体根据目标采取行动的能力。

与单纯生成文本不同，具有自主行动能力的智能体可以决定是否调用工具、执行操作或进入下一阶段。

3. Reasoning and Planning：推理与规划

推理用于判断当前状态和行动选择，规划用于组织实现目标所需的步骤。

两者密切相关，但侧重点不同：

- Reasoning：当前应该做什么，以及为什么。
- Planning：完成目标需要哪些步骤，以及这些步骤如何安排。

4. Memory：记忆

记忆使智能体能够利用先前的信息，而不是在每次调用时重新开始。

可以进一步区分为：

- 短期记忆：当前任务的上下文、中间结果和工具反馈。
- 长期记忆：历史经验、用户偏好或持久化知识。

这种区分是对课件内容的扩展，具体实现可能采用上下文窗口、数据库或向量检索等方式。

5. Feedback / Learning：反馈与学习

反馈机制使智能体能够判断当前行动是否达到预期。

例如，代码执行失败后，智能体可以读取错误信息、修改代码并重新执行。

这里的学习不一定意味着更新模型参数，也可以表现为任务执行过程中的策略调整。

1.2.2 附加能力

附加能力	英文	作用
多模态	Multimodality	处理文本、图像、音频等不同模态
通信	Communication	与用户、工具或其他智能体交换信息
具身性	Embodiment	通过物理或虚拟环境中的行动与外界交互

具身智能体（Embodied Agent）可以通过传感器感知环境，并通过执行器采取行动。例如，机器人能够观察物体位置、规划移动路径并执行抓取操作。

1.2.3 智能体的基本运行循环

综合上述能力，可以将智能体抽象为：

1. Perception 获取用户输入与环境状态
2. Reasoning & Planning 结合目标、约束与记忆制定行动
3. Action 生成回复或调用外部工具
4. Feedback & Memory Update 检查结果并更新任务状态

未完成时继续循环；满足完成条件后终止。这一循环是智能体区别于单次模型调用的重要特征。

1.3 智能体自主性程度 Degrees of Agent Autonomy

1.3.1 自主性是连续谱

与其争论某个系统是否属于真正的 Agent，不如承认系统存在不同程度的 Agentic Behavior（智能体行为）。因此，智能体不应简单地被划分为“是 Agent”或“不是 Agent”，而应根据其自主决策程度进行分析。

低自主性：固定流程、固定工具、有限决策

半自主性：能够选择行动，但工具集合预定义

高自主性：自主规划较多步骤，可能动态创建工具

1.3.2 三种自主性水平

维度	低自主性	半自主性	高自主性
工作流程	所有步骤预先确定	部分步骤由模型决定	多数步骤可自主决定
工具使用	工具调用硬编码	从预定义工具中选择	可能动态创建新工具
决策能力	主要局限于文本生成	能够选择部分行动	能够自主规划与调整
灵活性	较低	中等	较高
可预测性	通常较高	中等	相对较低

高自主性系统具有更大的灵活性，但也需要更严格的权限控制、执行监测与结果验证。

1.3.3 自主性系统比较及原则

方案 A：低自主性系统

User Task → LLM → Web Search → Web Fetch → LLM → Essay

1. LLM 生成搜索查询。
2. 系统执行网页搜索。
3. 系统抓取网页内容。
4. LLM 根据资料撰写文章。

流程基本预先确定，LLM 对整体流程的修改空间有限。

方案 B：更高自主性系统

Task → LLM → Research → LLM → Fetch Sources → Draft → Reflect → Revise

该系统能够从预定义工具集合中选择工具，包括：Web Search、News、arXiv、Web Fetch、PDF to Text

模型先决定如何搜索，再选择和获取资料，随后生成初稿并进行反思修改。

还明确包含获取五个最佳来源（Fetch 5 Best Sources）的步骤。

两种系统的主要区别不是使用了多少个模型，而是模型对行动选择、工具使用和任务流程具有多大控制权。

自主性设计原则

应根据任务特点选择适当的自主性程度。

对于流程明确、风险较高且需要严格审计的任务，固定工作流可能更合适；对于开放性研究、探索性任务或动态环境，较高自主性可能提供更强的适应能力。

因此，系统设计目标不是最大化自主性，而是在灵活性、可靠性、成本与可控性之间取得平衡。

Part 2 智能体工作流设计 Agentic Workflow Design

智能体工作流是指将模型推理、工具调用、条件判断和反馈修正组织为完整任务执行过程的方法。

合理的工作流设计可以提高任务完成质量、执行效率和系统可维护性。

2.1 四种基本工作流模式

四种常见模式：Sequential、Parallel、Iterative 和 Conditional。

模式	核心机制	主要优势	典型场景
Sequential	依次执行	依赖关系清晰	多阶段数据处理
Parallel	独立任务同时执行	缩短执行时间	多来源检索
Iterative	根据反馈反复修正	改善输出质量	代码调试、文章修改
Conditional	根据条件选择路径	提高流程灵活性	RAG 路由、异常处理

2.1.1 顺序工作流 Sequential Workflow

顺序工作流要求各步骤依次执行，后续步骤通常依赖前一步的输出。

例如： Extract Information → Analyze → Generate Report

适用于存在明确依赖关系的任务。若三个步骤执行时间分别为 T_A, T_B, T_C，忽略额外调度开销，则：

$T_{\text{sequential}} = T_A + T_B + T_C$

2.1.2 并行工作流 Parallel Workflow

并行工作流将能够独立执行的任务分配到不同分支。

例如，研究报告可以同时执行多个独立的信息检索任务，再将检索结果整合为报告。

在任务真正独立、资源允许且忽略调度开销的情况下： $T_{\text{parallel}} \approx \max(T_1, T_2, \cdots, T_n)$

相较于串行执行： $T_{\text{sequential}}=\sum_{i=1}^{n}T_i $，并行执行能够降低等待时间。

但如果任务存在数据依赖、资源冲突或副作用，就不能简单地并行执行。

2.1.3 迭代工作流 Iterative Workflow

迭代工作流通过反复检查与修正来改善结果。

$\text{Draft} \rightarrow \text{Review} \rightarrow \begin{cases} \text{Done}, & \text{通过检查}\\ \text{Revise}\rightarrow\text{Review}, & \text{未通过} \end{cases}$

典型应用包括：文章撰写与修改、代码生成与调试、数据查询与错误修正、复杂任务的多轮优化。

迭代工作流的核心是利用反馈改善输出，而不是仅增加模型调用次数。

2.1.4 条件工作流 Conditional Workflow

条件工作流根据当前状态或判断结果选择不同执行路径。

$\text{Check} \rightarrow \begin{cases} \text{Route A}, & C=\text{True}\\ \text{Route B}, & C=\text{False} \end{cases}$

例如：

- 如果已有知识足以回答问题，则直接生成答案。
- 如果需要外部证据，则先进行检索。

条件工作流可以避免不必要的工具调用，也可以根据任务类型选择不同的处理方式。

2.2 智能体工作流的主要优势

三项关键优势：

1. 潜在的性能提升（Potentially Better Performance）

复杂任务可以经过多轮推理、检查和修改，而不必依赖单次模型生成。

例如： $\text{Draft} \rightarrow \text{Review} \rightarrow \text{Revise} $

但多轮执行并不保证性能一定提升。如果反馈质量较差，迭代也可能引入新的错误。

2. 通过并行执行提高效率（Faster Execution Through Parallel Work）

当多个任务相互独立时，可以同时执行，从而降低整体延迟。

例如，三个检索任务可以并行运行，完成后再整合结果。

3. 模块化（Modularity）

模块化设计使系统能够独立更新工具和模型。

例如：替换搜索 API、增加数据库查询工具、将原有 LLM 替换为其他模型、单独优化评估模块。

模块化能够降低系统维护与扩展的复杂度。

2.3 任务分解与组件选择 Task Decomposition

2.3.1 任务分解的基本原则

任务分解是将复杂任务划分为能够由模型、工具或二者组合完成的子任务。

两个核心问题：

1. 一个 LLM、一个工具，或二者组合能否完成当前步骤?
2. 如果不能，是否需要进一步分解，或引入人工参与（Human-in-the-Loop）？

一个合理的子任务应具有明确的输入、输出和完成标准。任务分解过粗可能导致模型无法稳定完成任务；分解过细则可能增加调用次数、延迟和系统复杂度。

2.3.2 客户邮件处理案例

课件中的客户 Susan Jones 发送了一封投诉邮件：客户订购了蓝色 KitchenPro 搅拌机，订单号为 #8847，但实际收到的是烤面包机。客户希望在女儿周末生日派对前收到正确商品。系统需要处理三个子任务。

第一步：信息提取。从邮件中提取结构化信息；

第二步：订单查询。模型根据订单号调用数据库工具，核实订单记录与实际履约状态；

第三步：生成回复并执行操作。模型根据核实的信息撰写回复，再通过邮件工具发送。

需要注意，课件描述的是“撰写并发送回复”的流程，并未说明系统已经确认能够补发商品。因此，不应在缺少订单查询结果时擅自承诺具体补救措施。

在实际业务系统中，涉及退款、补发或其他具有业务后果的操作时，可以增加权限校验或人工审批。

2.4 智能体系统的基本构建模块 Building Blocks

将可用组件分为 Models 和 Tools 两大类。

类别	组件	典型用途
Models	LLMs	文本生成、工具调用、信息提取
Models	Other AI Modules	文本转语音、图像分析
Tools	APIs	网页搜索、实时数据获取、发送邮件、日历查询
Tools	Information Retrieval	数据库查询、文档检索、RAG
Tools	Code Execution	基础计算、数据分析

模型主要负责语义理解、推理、决策及内容生成。

工具主要负责访问外部资源、执行确定性操作或与实际系统交互。

例如，查询订单时：

- LLM 负责理解用户请求并生成查询参数。
- 数据库工具负责执行查询并返回真实记录。
- LLM 再根据返回结果组织后续行动。

选择组件时应考虑：任务是否需要实时信息、是否涉及精确计算、是否需要读取或修改外部数据、是否要求结果可验证、是否具有安全或权限限制。

例如，涉及数值计算时应优先使用代码执行；涉及当前订单状态时应调用数据库，而不能仅依赖模型预训练知识。

Part 3 智能体推理与决策 Agentic Reasoning and Decision-Making

3.1 推理与决策的基本机制

智能体推理：智能体利用可用信息，决定下一步应该采取什么行动，以实现目标。

推理过程依赖三个主要信息来源：

1. Perception / Grounding：当前感知与外部事实。
2. Memory：历史信息与已有状态。
3. Goals / Constraints：任务目标与约束条件。

这些信息共同支持智能体的决策与行动。

可以抽象为： $a_t = \pi(o_t, m_t, g, c)$

a_t：时刻 t 选择的行动、o_t：当前观测信息、m_t：记忆或历史状态、g：任务目标、c：约束条件、π：智能体的决策策略。

3.1.1 推理的主要作用

推理主要完成三项任务：

1. 理解当前状态与任务目标

识别当前已经掌握的信息、尚未完成的工作以及最终目标。

2. 在多个候选行动之间作出选择

例如，面对用户查询，智能体可能选择：直接回答、调用搜索工具、查询数据库、请求补充信息。

3. 评估权衡、证据与约束

行动选择不仅取决于能否完成任务，还应考虑：证据是否充分、工具是否可用、时间与计算成本、安全性与权限、用户提出的约束条件。

3.1.2 推理的三个主要应用场景

场景	英文	作用
反思	Reflection	诊断问题并修正输出
工具使用	Tool Use	决定何时以及如何调用工具
规划	Planning	组织多步骤行动与依赖关系

这三种能力相互联系。

规划决定整体任务结构，工具使用负责执行具体操作，反思则利用结果反馈修正执行过程。

Part 4 自反思与反馈优化 Self-Reflection and Feedback

自反思（Self-Reflection）是指智能体对已有输出进行检查，识别其中的问题，并根据评价结果生成改进版本。

两种反思方式：基于模型评价的内部反思、基于工具执行结果的外部反馈反思。

4.1 基于模型评价的自反思

4.1.1 基本流程

Task → LLM Draft → Code V1 → LLM Review → Code V2

首先，模型根据任务生成初始代码；随后，模型检查代码是否符合用户要求，并生成修订版本。

例如：def do_task(args): ...

经过反思后生成：def do_task_v2(args): ...

其中，第二版代码应根据审查结果修正第一版存在的问题，而不只是改变函数名称。

4.1.2 反思模型的配置

Reviewer 可以使用不同的 LLM，例如专门用于推理的模型。因此，生成器与审查器不一定必须是同一个模型。

可以采用： $y_1=G(x)$ ， $f=R(x,y_1)$， $y_2=G(x,y_1,f)$

G：生成模型、R：审查模型、f：反馈信息、y_1：初始输出、y_2：修订输出。

使用不同模型可能提供互补的评价能力，但并不能保证审查结果一定正确。

4.2 基于外部反馈的自反思

4.2.1 基本原理

内部反思主要依赖模型对已有结果的判断，而外部反馈利用实际执行结果指导修改。

课件展示的流程为： Task → Draft → Code V1 → Execute → Output / Errors → Revise → Code V2

与内部反思相比，外部反馈提供了来自执行环境的可观察信息。

例如，程序执行后返回：SyntaxError: unterminated string literal (detected at line 1)

该错误表明代码中存在未正确结束的字符串字面量。

模型可以利用错误类型及行号定位问题，修改代码并重新执行。

4.2.2 内部反思与外部反馈的区别

维度	内部反思	外部反馈
反馈来源	模型自身或审查模型	代码解释器、测试工具、数据库等
主要依据	语义理解、规则与评价标准	实际输出、错误信息、测试结果
优势	不一定需要执行环境	具有更直接的可观察依据
局限	可能重复原有错误	反馈质量取决于测试与执行条件
典型场景	文本审查、需求检查	代码调试、SQL 验证

外部反馈的价值在于将模型的主观判断与实际执行结果结合起来。

但程序能够正常执行，并不意味着逻辑一定正确。因此，除检查运行错误外，还需要测试输出是否符合预期。

4.3 反思提示的设计原则 Reflection Prompt Design

课件指出，一个有效的反思提示应明确三个要素：

1. Review Action：需要执行什么审查操作。
2. Review Criteria：依据哪些标准进行检查。
3. Revised Output：最终需要输出什么修订结果。

案例一：域名筛选

课件要求模型检查之前生成的域名建议。

评价标准包括：是否容易发音、是否有利于口头传播、是否可能在其他语言中具有负面含义。

最终只输出符合标准的域名候选列表。

该案例说明，反思不一定意味着修改已有文本，也可以通过评价与筛选减少不合格候选项。

案例二：邮件优化

课件要求模型审查邮件初稿。

评价标准包括：语气是否专业、是否存在粗鲁或不敏感的表达、事实是否准确、日期是否准确、承诺是否准确。

审查完成后，模型应输出下一版本的完整邮件。

这一案例说明，反思提示应针对具体任务设置明确标准，而不能仅使用“改进一下”之类的模糊要求。

4.4 反思机制的实现与评估 Implementing and Evaluating Reflection

4.4.1 SQL 查询修正流程

课件以销售数据查询为例，说明如何通过反思提高查询准确性。

用户问题为：2025 年 5 月共售出了多少件商品?

系统执行流程：User Question 查询 2025 年 5 月商品销量

LLM：Generate SQL V1 生成初始 SQL 查询

LLM：Review V1 and Write SQL V2 审查并修正查询

Execute SQL V2 执行修订后的 SQL

LLM：Generate Final Answer 根据查询结果回答

课件展示的是先审查 SQL V1，再执行 SQL V2 的流程；并未展示 V1 的具体 SQL 语句或错误原因，因此不能据此断定第一版究竟遗漏了哪个字段或筛选条件。

4.4.2 反思前后的实验结果

测试问题	Ground Truth	No Reflection	With Reflection
2025 年 5 月售出多少件商品?	1201	980	1201
最昂贵的商品是什么?	Airflow sneaker	Airflow sneaker	Airflow sneaker
经营多少种款式?	14	14	14

- Ground Truth：标准答案。
- No Reflection：未使用反思机制的结果。
- With Reflection：使用反思机制后的结果。

在第一个问题中，未使用反思机制时得到 980，与标准答案相差： $E_{\text{before}} = |980 - 1201| = 221$

使用反思机制后： $E_{\text{after}}=|1201-1201|=0$

若计算相对误差： $\text{Relative Error} = \frac{|980-1201|}{1201}\times100\% \approx18.40\%$

反思使该示例中的相对误差从约 18.40% 降至 0%，另外两个问题在反思前后均正确。

因此，在课件展示的三个测试问题中： $\text{Accuracy}_{\text{before}}=\frac23\approx66.67\%$， $\text{Accuracy}_{\text{after}}=\frac33=100%$

这只是三个示例构成的小规模比较，不能直接推断反思机制在所有任务中的整体准确率。

4.4.3 反思机制的适用条件与局限

反思通常更适合以下任务：存在明确评价标准、可以获取外部反馈、允许多次生成与修改、错误能够通过审查或测试被发现。例如，代码生成、SQL 查询、结构化数据处理及文档审查都可以采用反思机制。

但反思也具有局限：

1. 增加计算与调用成本

反思通常需要额外的模型调用，甚至多次工具执行。

若初始生成成本为 C_g，审查成本为 C_r，执行验证成本为 C_e，则： $C_{\text{total}} = C_g + C_r + C_e$

2. 不保证每次修改都正确

审查模型可能遗漏错误，也可能错误地修改原本正确的内容。

3. 需要合理的终止条件

如果系统持续反思而没有明确停止标准，可能造成无效循环。

可以设置：最大迭代次数、评价分数阈值、测试通过条件、成本或时间预算。

例如，令第 k 次修订结果的质量分数为 Q_k，当： $Q_k \geqslant \tau$ 或达到最大迭代次数 $K_{\max}$ 时终止。

其中， τ 为预设质量阈值。

Part 5 Agentic RAG 与智能体工作流的综合应用

本讲开头回顾了 Agentic RAG System 与 RAG Workflow。它们可以作为前述智能体概念的具体应用。

5.1 RAG 的基本工作流

RAG（Retrieval-Augmented Generation）通过检索外部知识，为语言模型生成答案提供依据。

课件将完整流程划分为两个阶段。

5.1.1 离线准备 Offline Preparation

离线阶段负责构建可检索知识库。

主要步骤包括： $\text{Raw Documents} \rightarrow \text{Preprocessing} \rightarrow \text{Chunking} \rightarrow \text{Embedding / Indexing} \rightarrow \text{Knowledge Base}$

1. 原始数据准备

知识来源可以包括：PDF 文档、图像、视频、其他多模态数据。

多模态 RAG 可以通过适当的模型或预处理方法，将不同模态的信息转换为可检索表示。

2. 文档预处理

清理与规范化文档内容，使其适合后续检索。

3. 文档分块（Chunking）

课件列举三种方式：Fixed-size Chunking：固定长度分块、Semantic Chunking：语义分块、Context-aware Chunking：上下文感知分块。分块策略影响检索单元的语义完整性和相关性。

4. 向量化与索引

通过 Embedding 将文档片段转换为向量，并建立索引。ANN 与 HNSW 等近似最近邻检索技术能够支持大规模向量搜索。

5.1.2 在线推理 Online Inference

在线阶段从用户查询开始，经过检索、证据选择和答案生成。

$ \text{Query} \rightarrow \text{Query Processing} \rightarrow \text{Retriever} \rightarrow \text{Candidate Documents} \rightarrow \text{Final Evidence} \rightarrow \text{Generation} \rightarrow \text{Response} $

关键步骤包括：

1. 查询处理（Query Processing）

可以采用查询解析、查询改写或可选的 HyDE 方法改善检索输入。

2. 检索（Retrieval）

课件列举的检索方法包括：Bi-encoder：双编码器检索、ColBERT：基于后期交互的检索方法、ANN / HNSW：近似最近邻检索。

3. 证据筛选（Evidence Selection）

从候选文档中选择与问题最相关的证据，减少无关信息进入生成阶段。

4. 增强提示与生成（Augmented Prompt & Generation）

将用户问题与检索证据共同提供给模型，生成有依据的回答。Self-RAG 与 Agentic Workflow 可以参与检索和生成过程，通过判断是否需要更多证据来调整系统行为。

5.2 Agentic RAG 的决策与反馈机制

普通 RAG 往往采用相对固定的检索与生成流程，而 Agentic RAG 可以增加路由、证据评估与迭代检索。

其关键在于三个决策点。

1. 是否需要检索：Router LLM 判断当前问题能否直接回答，还是需要外部知识。
2. 检索证据是否充分：Evidence Evaluation 模块判断检索结果能否支持回答。如果证据不足，则重新检索。
3. 是否能够生成有依据的答案：当证据充分时，模型生成答案并引用相应来源。

这体现了智能体工作流中的条件分支与迭代反馈。

5.3 RAG 系统的主要优化方向

不同技术对应的优化目标：

技术	主要优化目标
Chunking	提高检索相关性
Bi-encoder / ColBERT	提高匹配质量
Query Rewriting / Parsing	改善搜索输入
Multimodal RAG	支持更丰富的证据类型
ANN / HNSW	提高大规模检索速度
Self-RAG	增强自适应控制与可靠性
Evaluation / Security	提高鲁棒性与部署准备程度

这些技术并非互相替代，而是分别作用于数据准备、检索、证据选择、生成控制与系统保障等不同环节。

Part 6 智能体的工具调用与动态决策 Tool Use and ReAct

前文已经介绍智能体的基本组成、工作流设计、推理与决策机制，以及自反思能力。本部分进一步讨论智能体如何选择和执行工具、如何根据外部反馈动态调整决策，以及如何通过代码执行扩展智能体的能力。

6.1 工具调用的决策机制 Tool-Use Decision Making

6.1.1 工具可用性与工具必要性

智能体能够访问某个工具，并不意味着每次任务都需要调用该工具。

Having access to a tool doesn't mean the agent always needs to use it.

工具调用涉及两个不同的问题：

1. Tool Availability（工具可用性）：系统提供了哪些工具？
2. Tool Necessity（工具必要性）：当前任务是否需要使用某个工具？

例如，系统提供 `get_current_time` 工具时：

用户问题	是否调用工具	原因
What time is it?	是	当前时间属于动态外部信息
How much caffeine is in green tea?	通常不需要	可以利用已有知识给出一般性估计

对于第一个问题，执行流程为： $\text{Query} \rightarrow \text{LLM} \rightarrow \texttt{get_current_time} \rightarrow \text{Tool Result} \rightarrow \text{LLM} \rightarrow \text{Answer} $

课件中的工具返回 `15:20:45`，模型将其整理为 `It's 3:20 p.m.`对于第二个问题，模型可以直接回答：一杯绿茶通常含有约 25 ~ 50 mg 咖啡因。具体含量取决于茶叶种类、冲泡时间和用量。

这说明工具调用是智能体根据任务需求作出的独立决策，而不是固定执行步骤。

5.1.2 工具选择的判断依据

工具选择通常需要考虑以下因素：

- 信息时效性：是否需要当前时间、实时价格或最新记录。
- 外部依赖：是否需要访问数据库、文件或其他系统。
- 计算复杂度：是否需要精确计算或程序执行。
- 知识充分性：模型已有信息是否足以支持回答。
- 执行必要性：任务是否要求实际操作，而非仅生成文本。

可以将工具调用决策抽象为： $a_t = \pi(q_t, s_t, \mathcal{T})$

q_t：当前任务或查询；s_t：当前系统状态；$\mathcal{T}$：可用工具集合；a_t：选择的行动，包括调用工具或直接回答。

5.2 基于代码的工具调用 Code-Based Tool Use

工具可以通过普通编程函数定义，再向 LLM 提供其名称、功能描述和参数信息。

使用 Python 的 `datetime` 模块定义时间查询工具

- `datetime.now()`：获取运行环境的当前本地时间。
- `strftime("%H:%M:%S")`：将时间格式化为字符串。
- Docstring：描述函数的用途，帮助模型理解工具能力。

随后，课件使用 `aisuite` 将工具提供给模型：在该示例中，`aisuite` 可以根据 Python 函数信息生成模型可理解的工具描述，使模型能够判断是否调用 `get_current_time`。

需要区分两个层次：模型负责决定调用什么工具，运行环境负责真正执行工具。

因此： $\text{Tool Calling} = \text{Tool Selection} + \text{Argument Generation} + \text{Tool Execution} + \text{Result Processing} $

工具执行通常需要由运行时程序完成。模型本身生成工具调用请求，并不等于已经执行相应操作。

5.3 工具之间的依赖关系 Dependencies Between Tools

复杂任务往往需要连续调用多个工具，而后续调用可能依赖前一次调用的结果。

日历预约案例

用户请求：Find a free slot on Thursday in my calendar and make an appointment with Alice.

系统提供三个工具：`check_calendar, make_appointment, delete_appointment`

$ \text{User Request} \rightarrow \texttt{check_calendar} \rightarrow \text{Available Slots} \rightarrow \texttt{make_appointment} \rightarrow \text{Confirmation}$

1. LLM 选择 `check_calendar`。
2. 工具返回周四 `15:00`、`16:00`、`18:00` 可用。
3. LLM 选择 `make_appointment`。
4. 工具返回预约创建成功。
5. LLM 确认预约时间为周四 `15:00`。

数据依赖与执行顺序

预约工具需要使用日历查询得到的可用时间，因此： `check_calendar` $\prec$ `make_appointment`

其中，$\prec$ 表示前置依赖关系。

类似地，只有预约工具确认成功后，系统才能向用户报告预约已经完成。

因此： $\text{Successful Booking} \prec \text{Success Confirmation} $

这一案例说明：工具执行顺序应由数据依赖与实际执行结果决定，而不能仅根据预先生成的步骤机械执行。

5.4 ReAct：推理与行动的协同 Reasoning and Acting

ReAct 是一种将推理、行动和环境观察结合起来的智能体执行方法。

ReAct 包含三个核心环节：

Reason: 分析当前状态并选择下一步行动

Act: 调用工具或执行操作

Observe: 读取工具返回结果或环境反馈

利用新观察更新状态，继续下一轮推理

在时刻 t，智能体根据当前状态选择行动： $ a_t=\pi(s_t)$

执行行动后，环境返回观察： $ o_{t+1}=\operatorname{Execute}(a_t)$

系统利用观察更新状态： $s_{t+1}=\operatorname{Update}(s_t,a_t,o_{t+1}) $

随后根据更新后的状态决定下一步行动： $a_{t+1} = \pi(s_{t+1})$

智能体的决策不是一次性完成，而是能够随着新信息的出现不断调整。

Tool Calling、ReAct 与 Reflection 的区别

概念	主要作用	核心问题
Tool Calling	提供执行外部行动的接口	如何调用工具?
ReAct	结合推理、行动与观察	如何根据新信息调整下一步行动?
Reflection	检查并修正已有结果	如何发现和纠正错误?

三者并不互斥。ReAct 可以使用工具调用执行行动，也可以在循环内部加入 Reflection 来检查中间结果。

ReAct 与其他提示方法的比较

引用 ReAct 论文中的 HotpotQA 示例，对比四种方法：

方法	主要特征	局限
Standard	直接生成答案	缺少显式推理与外部验证
CoT（Reason Only）	通过多步推理生成答案	可能基于错误知识持续推理
Act-Only	调用工具获取信息	缺少明确的推理指导
ReAct（Reason + Act）	交替进行推理、工具调用和观察	需要管理循环、工具成本与终止条件

示例问题：除了 Apple Remote 之外，最初设计用于交互的程序还可以通过什么设备控制?

课件中的 ReAct 示例通过搜索逐步发现：Apple Remote 与 Front Row 有关、Front Row 是 Apple 的媒体中心软件、Front Row 也可以通过键盘功能键控制。

最终得到正确答案：Keyboard function keys（键盘功能键）。

这一案例体现了 ReAct 的优势：通过外部检索纠正不完整或错误的初始判断。

5.5 为什么需要协同推理与行动?

新证据可以改变下一步行动

课件继续使用日历预约案例，展示了环境状态可能发生变化。

初次查询显示 `15:00` 可用，但预约执行时该时间已被占用。

因此，系统不能继续按照原计划执行，而应根据错误反馈重新查询并选择其他时间。

其决策过程为： Plan → Act → Observe Conflict → Replan → Act Again

这也是 ReAct 相较于固定工作流的重要优势：能够处理动态环境中的执行失败与状态变化。

新证据可以改变最终答案

以校园 AI 使用政策为例。

任务：Write an evidence-backed summary of the campus rules on AI use.

执行过程分为三个阶段：

Search：初步检索，搜索结果声称学校允许使用 AI。

Read：阅读权威来源，进一步阅读官方页面后发现，允许使用 AI 存在附加条件：必须遵守具体作业规定。

Update：更新答案，最终回答应缩小原有结论的适用范围，明确引用官方条件，并在缺少具体作业规则时建议用户向相关负责人确认。

这一案例说明，ReAct 不仅可以改变工具调用顺序，也可以根据新证据修正最终结论。

5.6 ReAct 的最小实现与终止机制

有界执行循环 Bounded Loop

ReAct 的三种终止结果

每次循环都会消耗时间，也可能产生新的外部影响，因此必须设置停止规则。

1. Finish：正常完成

当任务成功条件满足，且结果具有充分证据支持时，终止。

2. Recover：有限恢复

当出现暂时性错误时，可以在固定预算内重试。

例如：网络请求暂时失败、数据库连接超时、日历预约发生时间冲突。

3. Escalate or Stop：升级或停止

当出现以下情况时，应停止自动执行：缺少必要权限、证据不足、已耗尽执行预算、达到最大迭代次数。

可以定义： $N_{steps} \leqslant N_{max}$以及 $C_{total} \leqslant C_{max}$

其中，N_{max} 是最大执行步数，C_{max} 是允许的最大执行成本。

需要特别注意，达到步数上限并不意味着任务已经成功。系统应明确报告未完成状态，而不是生成缺乏依据的成功结论。

5.7 代码生成与执行能力 Code Generation and Execution

5.7.1 通过代码执行扩展模型能力

LLM 可以生成代码，再交由外部运行环境执行。

课件以计算 $\sqrt{2}$ 为例： $ \sqrt{2}\approx1.4142135623730951$

系统提示要求模型将 Python 代码放入特定标签：

Write code to solve the user's query. Return your answer as Python code delimited with <execute_python> and </execute_python> tags.

模型生成：import mathprint(math.sqrt(2))

执行结果：1.4142135623730951

模型读取执行结果后，生成自然语言回答。

整体流程为： $ \text{Question} \rightarrow \text{LLM Generates Code} \rightarrow \text{Execute Code} \rightarrow \text{Result} \rightarrow \text{LLM Answer}$

与直接依赖模型生成数值相比，代码执行可以为确定性计算提供更可靠的结果。

5.7.2 生成代码的安全执行

直接执行 LLM 生成的代码可能产生严重风险。例如，错误生成的删除命令可能删除项目目录中的 Python 文件。因此，代码执行需要受控环境。

Sandbox（沙箱） 是一种隔离执行机制，用于限制代码可以访问的文件、资源和系统操作。

两个相关工具：Docker：容器化运行环境、E2B：面向代码执行的沙箱服务。

需要注意，Docker 容器并不天然等于安全沙箱。安全性还取决于实际配置，例如文件挂载、网络权限、用户权限和资源限制。

实际系统通常需要控制：文件系统访问权限、网络访问权限、CPU 与内存使用、执行超时时间、系统命令权限、敏感数据访问。

安全原则：模型可以生成代码，但代码的实际执行必须受到运行环境的独立约束。

Part 6 智能体规划与结构化执行 Planning and Structured Execution

6.1 规划能力的定义

Planning 是将复杂任务组织为多个有序行动的能力。

与单次工具调用相比，规划更关注：完成任务需要哪些步骤、每个步骤应该使用什么工具、步骤之间是否存在依赖、如何利用前一步结果执行后续步骤。

规划可以表示为： $P = (a_1, a_2, \cdots, a_n)$。其中， a_i 表示第 i 个行动。

若行动之间存在依赖关系，则必须满足相应的执行顺序。

6.2 客户服务智能体的规划案例

用户询问：Do you have any round sunglasses in stock that are under $100?

完成当前任务只需要前三个工具。合理规划为：

1. 使用 `get_item_descriptions` 查找圆形太阳镜。
2. 使用 `check_inventory` 判断候选商品是否有库存。
3. 使用 `get_item_price` 检查有库存商品是否低于或等于 100 美元。

整体依赖关系为： $\text{Product Search} \rightarrow \text{Inventory Check} \rightarrow \text{Price Check} $

只有满足商品形状、库存和价格三个条件，才能给出符合要求的推荐。

Example：邮件助手中的行动依赖

Reply to that email invitation from Bob about dinner in New York and tell him I'll attend. Then archive his email.

可用工具包括：`search_email, send_email, move_email, delete_email`

执行计划为：

1. 搜索 Bob 发送的、提及纽约晚餐的邀请邮件。
2. 使用 `send_email` 回复并确认出席。
3. 使用 `move_email` 将原邮件移动到归档文件夹。

依赖关系为： $ \text{Search Email} \prec \text{Send Reply} \prec \text{Archive Email} $

归档操作应在回复成功后进行，以免任务尚未完成就改变邮件状态。

说明：规划不仅是列出步骤，还必须明确步骤之间的依赖关系与完成条件。

6.4 使用 JSON 表示结构化计划

JSON（JavaScript Object Notation）是一种文本形式的结构化数据交换格式。它能够让模型生成的计划被程序解析、验证和执行。JSON 要求对象键名和字符串值使用双引号。

JSON 序列化与反序列化

Python 使用 `json` 模块进行转换：

- `json.dumps()`：将 Python 对象序列化为 JSON 字符串。
- `json.loads()`：将 JSON 字符串解析为 Python 对象。

转换过程为： $\text{Python Object} \xrightarrow{\texttt{json.dumps()}} \text{JSON String} $, $\text{JSON String} \xrightarrow{\texttt{json.loads()}} \text{Python Object}$

解析 JSON 只完成数据转换，并不会自动执行其中描述的操作。

将行动计划表示为 JSON

要求每个计划步骤包含四个字段：

`step`：步骤编号；`description`：步骤描述；`tool`：工具名称；`args`：工具参数。

这种结构使程序能够识别工具名称、参数和执行顺序。但其中的 `"results from step 1"` 只是描述性占位符。实际执行时，运行环境需要将其解析为第一步返回的真实数据。

因此，结构化计划执行至少包含： Generate Plan → Parse JSON → Validate Plan → Resolve Dependencies → Execute Tools

对于更复杂的系统，还可以增加参数类型验证、工具白名单及异常处理机制。

6.5 多工具规划的复杂性

咖啡店交易数据说明：当工具数量和任务类型增加时，规划难度也会提高。

Find the month with the highest hot-chocolate revenue.

多工具组合问题

这里假设每条记录代表一件商品交易，且 `price` 是该交易的实际收入；若存在数量、退款或折扣字段，还需要调整计算方法。

多工具规划的主要困难

两个核心问题：

- More combinations：工具组合数量增加。
- More edge cases：边界情况增加。

例如，系统需要处理：日期格式不一致、商品名称存在不同写法、数据包含缺失值、查询条件涉及多个字段、工具之间存在复杂依赖。

随着工具数量增加，模型不仅需要选择正确的工具，还必须决定调用顺序和参数传递方式。

通过代码执行简化复杂规划

对于复杂的数据分析任务，可以让 LLM 生成完整代码，再由运行环境执行。

“查询最近五笔交易金额”

- `pd.read_csv()`：读取交易数据。
- `pd.to_datetime()`：将日期转换为时间类型。
- `sort_values()`：按日期降序排列。
- `head(5)`：选取最近五条记录。
- `to_list()`：将金额转换为列表。

与连续调用多个细粒度工具相比，代码执行能够将筛选、排序和聚合等操作组合为完整程序。但这并不意味着代码执行一定优于工具调用。

代码执行需要额外考虑：代码正确性、执行环境安全、文件访问权限、异常处理、结果验证。

因此，适合采用哪种方式取决于任务复杂度、工具能力与执行风险。

6.6 单智能体能力总结

核心能力：

能力	示例	主要作用
Reflection	修改邮件、审查代码	检查并改善已有输出
Tool Use	查询当前时间、创建预约	获取外部信息并执行行动
ReAct	预约冲突后重新查询日历	根据新观察调整行动
Planning	查询商品、库存与价格	组织多步骤任务及依赖关系

这四项能力可以组合在同一个智能体中。然而，单个智能体不一定适合所有复杂任务。某些任务需要多个具有不同专业能力或评价标准的决策角色，因此引出多智能体系统。

Part 7 多智能体系统基础 Multi-Agent Fundamentals

7.1 多智能体系统的定义

多智能体系统（Multi-Agent System, MAS）由多个能够进行决策的智能体组成，这些智能体在共享任务或环境中相互作用。其核心不是简单增加 LLM 数量，而是引入多个具有独立职责、局部状态和决策能力的实体。

要素	含义
Agents	多个能够基于观察和历史信息选择行动的智能体

要素	含义
Goals	智能体可能具有共享、不同或相互竞争的目标
Environment	多个智能体共同观察和改变的环境
Interaction	智能体通过行动、消息或共享状态相互影响

某些复杂任务包含多个互补的专业角色。多个智能体能够将不同知识、工具和职责分配给专门的角色。但并非所有任务都需要多智能体架构。只有当新增智能体能够解决具体任务限制时，才具有引入价值。

7.2 多智能体协作案例

一家商店希望推广售价为 HK$299 的圆形、哑光黑色太阳镜。

目标输出：一张宣传图片、一段不超过 50 个英文单词的宣传文案。系统设置三个专业智能体。

1. Researcher Agent

负责：选择信息来源、检查产品相关声明、确认宣传内容是否有证据支持。

输出：已核实的产品事实、来源说明。

2. Designer Agent

负责：选择视觉风格、设计并修改宣传图片、确保图片符合任务要求。

输出：与营销需求一致的宣传图片。

3. Writer Agent

负责：撰写宣传文案、根据反馈修改内容、控制文案长度。

输出：清晰且不超过 50 个单词的宣传文案。

三个智能体围绕同一营销目标工作，但分别承担事实核查、视觉设计和文本创作。

7.3 采用多智能体系统的五个理由

1. 专业化能力（Specialized Expertise）

不同智能体可以具有不同知识、工具或职责。

例如：Researcher 侧重事实与证据、Designer 侧重视觉表达、Writer 侧重语言与传播效果。

专业化的核心是能力分工，而不是简单地将同一任务重复交给多个模型。

2. 独立并行工作（Independent Parallel Work）

当任务之间不存在必要依赖时，可以并行执行。

例如： $\text{Approved Facts} \rightarrow \begin{cases} \text{Visual Design}\\ \text{Copywriting} \end{cases} \rightarrow \text{Campaign}$

设计与文案可以在共享已核实事实的基础上同时进行，再整合为最终宣传素材。

需要区分：Specialization 关注不同智能体具备什么能力、Parallelism 关注任务之间是否存在执行依赖。专业化不意味着一定能够并行执行。

3. 上下文隔离（Context Isolation）

每个智能体可以维护独立工作上下文，减少无关信息干扰。

Local Context每个角色只保留完成自身任务所需的证据和历史信息。

Shared Constraints 多个角色共享统一的任务要求和已核实事实。

Handoff 智能体之间传递结论、来源和未解决问题。

例如：Researcher 保存详细检索记录、Designer 主要接收产品视觉要求与已确认事实、Writer 主要接收营销目标、产品事实和长度限制。这样可以减少上下文冗余，同时保持关键约束的一致性。

4. 独立验证（Independent Verification）

独立验证用于检查已有结果是否满足证据、约束与推理要求。

例如，Writer 生成：These sunglasses are the most popular product this summer.

如果缺乏销量或市场调查数据，这一声明可能没有充分依据。独立审查智能体可以质疑原有假设，并要求删除或修改未经证实的表述。因此，验证不仅检查文本是否通顺，更重要的是检查结论是否受到证据支持。

5. 多视角评价（Multiple Perspectives）

不同智能体可以采用不同评价标准。

例如：Researcher：事实准确性、Designer：视觉质量、Writer：语言清晰度与传播效果。

最终决策需要在不同标准之间进行权衡。

需要特别区分：Independent Verification 主要关注结果是否正确、有依据；Multiple Perspectives 主要关注不同评价标准之间的平衡。

7.4 多智能体系统的目标与成功标准

Completing individual subtasks does not mean the overall task is complete.

即使所有智能体分别完成自己的工作，也不代表整体任务已经成功。

因此，成功标准需要在三个层次上定义。

层次	核心目标	评价重点
Task Level	交付可使用的最终成果	输出完整性、任务约束
Agent Level	完成分配的职责	单个贡献的质量与完整性
Interaction Level	可靠整合多个贡献	一致性、问题解决、终止条件

例如，太阳镜营销任务中：

- Researcher 必须提供可靠产品信息。
- Designer 必须生成符合要求的图片。
- Writer 必须提供合格文案。
- 整体系统必须保证图片与文案一致，并符合最终交付要求。

逻辑关系：$\text{Task Success} \Rightarrow \text{Required Outputs Valid} \land \text{Constraints Satisfied} \land \text{Integration Valid} $

单个智能体完成任务只是整体成功的必要组成部分，不一定构成充分条件。

7.5 单个智能体角色的定义

为了使多智能体系统具有明确的职责边界，课件提出六个角色描述字段。

字段	需要回答的问题
Name	如何标识该智能体?
Inputs	可以使用哪些证据和上下文?
Responsibilities	负责哪些决策?
Tools	允许使用哪些工具和行动?
Output	必须交付什么结果?
Goal	应追求什么目标?

一个清晰的角色定义应将职责与可检查的行动和输出联系起来。这种结构有助于明确智能体的职责边界，避免多个角色重复执行相同任务或遗漏关键工作。

7.6 多智能体目标关系 Goal Relationships

多个智能体之间的目标关系可以分为三类。

1. 合作型（Cooperative）

多个智能体通过共享成果共同受益。

主要协调问题包括：贡献分配、信息共享、联合成功标准。

例如，Researcher、Designer 和 Writer 共同完成营销活动。

2. 竞争型（Competitive）

一个智能体的成功可能以其他智能体的利益受损为代价。

主要协调问题包括：策略选择、资源分配、信息访问。

例如，多个竞价智能体竞争有限资源。

3. 混合目标型（Mixed-Motive）

智能体在部分目标上合作，在其他目标上存在冲突。

主要协调问题包括：谈判、局部激励协调、共同目标与个体目标的平衡。

三种关系的区别如下：

目标关系	主要特征	协调重点
Cooperative	共享成果	合作与信息共享
Competitive	利益可能相互冲突	策略与资源竞争
Mixed-Motive	部分合作、部分竞争	谈判与激励协调

多智能体系统的交互机制应根据目标关系设计，而不能默认所有智能体始终具有完全一致的利益。

7.7 多智能体交互、记忆

社会模拟中的生成式智能体 Generative Agents

该研究使用多个生成式智能体模拟虚拟社区中的日常行为。每个智能体拥有自己的日程、记忆和计划，并通过行动和对话影响其他智能体。

研究关注的问题包括：智能体是否具有共同目标？每个智能体能够观察到什么？一个智能体如何影响其他智能体？不同智能体之间有哪些差异？

社会模拟强调：多智能体系统中的整体行为可以由多个个体的局部行动与交互共同产生。

7.7.1 生成式智能体的记忆与交互机制

1. Memory Stream：记忆流

智能体将观察到的事件、经历和交互记录保存到记忆中。

例如：与其他角色的对话、曾经执行的行动、环境中发生的事件、已经制定的计划。

2. Memory Retrieval：记忆检索

智能体并非每次都读取全部历史记录，而是检索与当前情境相关的记忆。

检索结果用于支持当前决策。

3. Reflection：反思

反思能够将具体经历整合为更高层次的理解。

例如，智能体可能从多次互动中总结其他角色的习惯或偏好。

4. Planning：规划

智能体根据当前目标、环境状态和相关记忆制定行动计划。

5. Social Interaction：社会交互

智能体的行动和对话会改变共享环境，并成为其他智能体的新观察。

因此，系统形成持续循环： Perceive → Store Memory → Retrieve → Reflect / Plan → Act → New Observations

记忆使智能体能够保持行为连续性，交互则使多个智能体之间产生动态影响。

7.7.2 局部观察与上下文 Local Observations

多个智能体可以共享同一个环境，但不一定拥有相同的信息。

1. World State：世界状态。环境包括其他智能体及其行动。
2. Local Observation：局部观察。每个智能体只能获取自身可访问的信息。
3. Private Context：私人上下文。不同智能体可以拥有不同的记忆、信念和计划。

假设真实环境状态为 s_t，智能体 i 的观察为： $o_t^i = O_i(s_t)$

其中，O_i 是智能体 i 的观察函数。

即使处于同一环境，也可能存在： $o_t^i\neq o_t^j $

因此，两个智能体可能基于不同信息作出不同决策。

这种现象称为 Partial Observability（部分可观测性）。

7.8 多智能体系统的主要挑战

多智能体系统由于包含多个决策主体，会引入信息、协调和学习方面的复杂性。

7.8.1 部分可观测性 Partial Observability

不同智能体掌握的信息不完整，可能导致不兼容的决策。

例如，一个智能体已经修改任务状态，但另一个智能体尚未收到更新，仍然按照旧状态执行。

因此，系统需要合理设计状态共享、消息传递和信息同步机制。

7.8.2 规模扩展 Scale

随着智能体数量增加，系统需要处理更多：消息路由、工具调用、协调过程、结果整合、计算与通信成本。

更多智能体不一定意味着更高效率。

7.8.3 策略依赖 Policy Dependence

一个智能体行动的价值可能取决于其他智能体的行为。

例如，在竞争资源分配的任务中，某个智能体是否应该立即行动，可能取决于其他智能体是否也在竞争相同资源。

可以表示为： $ U_i=U_i(a_i,a_{-i})$

- U_i：智能体 i 的效用。
- a_i：智能体 i 的行动。
- a_{-i}：其他智能体的联合行动。

因此，智能体不能总是独立评估自身行动的价值。

7.8.4 非平稳性 Non-Stationarity

当多个智能体同时学习或调整策略时，环境对于某个智能体而言可能不断变化。

例如，智能体 j 的策略从： $ \pi_j^{(t)} $，更新为： $ \pi_j^{(t+1)} $

那么即使智能体 i 策略没有变化，它面对的交互环境也可能发生改变。

这会增加策略学习、行为预测和系统稳定性维护的难度。

7.9 综合总结

从单智能体到多智能体，系统设计关注点发生了变化。

维度	单智能体系统	多智能体系统
决策主体	一个主要决策实体	多个决策实体
任务组织	内部规划与工具调用	角色分工与协同执行

维度	单智能体系统	多智能体系统
信息管理	单一主要工作上下文	局部上下文与共享状态
反馈机制	工具结果、自反思	个体反馈与智能体间交互
主要挑战	工具选择、规划、执行安全	协调、通信、信息不完整、策略依赖
成功标准	完成整体任务	个体贡献与整体协作均满足要求

Tool Use → ReAct → Planning → Multi-Agent Coordination

其中：

- Tool Use 使智能体能够获取外部信息并执行行动。
- ReAct 使智能体能够根据新观察动态调整决策。
- Planning 使智能体能够组织多步骤任务及其依赖关系。
- Multi-Agent Coordination 使多个具有不同职责和局部信息的智能体能够共同完成任务。

这些能力之间并不是简单的替代关系，而是可以相互组合。

最终，智能体系统设计的核心问题是：如何将模型、工具、推理、规划、记忆和交互机制组织为一个能够可靠完成任务的系统，同时控制执行成本、风险和协调复杂性。

Part 8 多智能体系统设计 Designing Multi-Agent Systems

前文介绍了多智能体系统的基本概念、角色分工、目标关系、局部观察和交互机制。

本章进一步讨论如何将这些概念转化为可实施的系统架构，重点解决任务如何分配、智能体如何协调、工作流如何组织、信息如何交换，以及系统如何安全终止和评估等问题。

多智能体系统设计并非简单地增加智能体数量，而是根据任务依赖、控制需求和协调成本，构建具有明确职责、执行流程和通信规则的协作系统。

8.1 多智能体系统的核心设计维度 Design Decisions

多智能体系统设计包含四个相互关联的维度。

设计维度	核心问题	主要设计内容
Task and Roles（任务与角色）	谁负责每项决策和输出？	任务分解、职责分配、权限边界
Coordination（协调机制）	谁负责分配工作和解决冲突？	集中式、去中心化、混合式协调
Workflow / Architecture（工作流与架构）	智能体、控制逻辑和状态如何组织？	顺序、并行、迭代及图结构工作流
Communication（通信机制）	哪些信息需要传递，在哪里保存？	消息格式、路由、共享状态、版本控制

四个维度分别解决不同层次的问题：

- 任务与角色决定系统中有哪些工作，以及由谁负责。
- 协调机制决定决策权和任务分配权如何组织。
- 工作流与架构决定各项任务如何随时间执行。
- 通信机制决定智能体之间如何交换信息、共享状态和传递结果。

区分：协调模型描述的是谁作出决策，执行工作流描述的是任务以什么顺序执行。两者相关，但并不等同。

8.2 任务分解、职责分配与角色契约

8.2.1 任务分解与分配 Task Decomposition and Allocation

任务分解（Decomposition）是将总体目标拆分为具体输出和子任务，并识别它们的依赖关系与所需输入。

任务分配（Allocation）则是将这些子任务交给具有相应能力的智能体，并明确责任归属和交付条件。

二者的区别如下：

维度	Decomposition	Allocation
核心问题	需要完成哪些工作？	由谁完成这些工作？
主要内容	子任务、输入、输出、依赖	能力匹配、责任归属、交付条件
主要产物	任务分解结构	角色与任务映射关系

任务分解时应明确每个子任务的输入、输出及前置条件，避免将存在数据依赖的任务错误地安排为并行执行。

8.2.2 角色边界、工具与权限 Role Boundaries, Tools and Permissions

角色契约（Role Contract）不仅需要说明智能体应当完成什么工作，还必须明确它被允许执行哪些操作。

角色边界分为三个方面：

1. Decision Ownership：决策归属

明确智能体能够独立作出哪些决策，以及哪些决策需要其他角色或人工确认。

2. Tool Access：工具访问权限

明确智能体能够访问哪些数据，以及允许执行哪些系统操作。

3. Escalation Boundary：升级处理边界

明确哪些不确定情况、冲突或高风险操作必须提交上级协调者或人工处理。

这一原则体现了最小权限设计思想（Principle of Least Privilege）：每个智能体只应获得完成其职责所必需的操作权限。

8.2.3 智能体之间的交接契约 Handoff Contracts

Handoff 指一个智能体完成阶段性工作后，将结果及必要上下文传递给另一个智能体。

有效的交接不应只包含“任务已完成”的通知，而应提供能够支持下一步决策的可用产物。

四项交接内容：

组成部分	应包含的信息
Content	结论、结果及支持证据
Traceability	信息来源和产物版本
Status	已验证、不确定或等待审查
Open Questions	尚未解决的问题以及下一责任方

这种结构能够减少信息丢失，并使后续智能体判断哪些内容可以直接使用，哪些内容需要进一步验证。

8.3 多智能体协调模型 Coordination Models

协调机制决定任务分配、决策和结果整合发生在系统的什么位置。

三种主要模式：集中式、去中心化和混合式。

1. 集中式协调（Centralized Coordination）

集中式架构由一个 Manager Agent 负责分配任务、协调执行并整合结果。

其典型流程为： Manager → Task Assignment → Worker Execution → Manager Integration

优势是责任明确、便于统一规划和结果整合。

主要代价是 Manager Bottleneck（管理者瓶颈）：随着任务数量增加，中央管理者可能承担过多通信、决策和整合工作。

2. 去中心化协调（Decentralized Coordination）

去中心化架构中，各智能体可以直接与其他智能体交换信息，并在本地作出决策。

其特点包括：

- 没有唯一的中央决策者。
- 智能体能够根据局部状态自主协商。
- 信息与任务可以在多个节点之间流动。

这种模式适合规模较小、需要灵活交流的协作团队。但由于缺少统一控制者，系统更难达成全局一致决策，可能出现重复工作、冲突或信息不一致。

3. 混合式协调（Hybrid Coordination）

混合式架构结合集中式的全局规划和去中心化的局部自主性。

例如： $\text{Global Manager} \rightarrow \begin{cases} \text{Team A Coordinator}\ \text{Team B Coordinator} \end{cases} \rightarrow \text{Local Workers} $

上层负责整体目标、资源分配和跨团队协调，下层负责具体任务执行与局部决策。

这种模式可以降低中央管理者的直接负担，但需要设计额外的层级通信和责任边界。

三种模式的对比如下：

模型	决策方式	主要优势	主要代价
Centralized	中央管理者分配与整合	全局控制清晰	中央瓶颈
Decentralized	智能体自主协调	灵活性高	全局一致性较难
Hybrid	全局规划与局部自治结合	兼顾控制与自主性	架构复杂度增加

8.4 执行工作流与系统架构 Execution Workflows and Architecture

协调模型回答“谁负责决策”，执行工作流回答“任务如何随时间组织”。

8.4.1 顺序、并行与迭代工作流

课件介绍三种基本执行模式。

对于顺序工作流，若任务耗时为 $T_1, T_2, \cdots, T_n$，忽略调度开销： $ T_{\text{sequential}}=\sum_{i=1}^{n}T_i $

对于相互独立且同时启动的并行任务： $T_{\text{parallel}} \approx \max(T_1, T_2, \cdots, T_n)$

实际系统还需要考虑调度、通信与结果整合开销。

迭代工作流则需要明确退出条件，避免无界循环。

8.4.2 对等网络与层级架构 Peer Networks and Hierarchies

智能体之间的连接方式也会影响通信成本与协调效率。

Peer Network（对等网络）

智能体之间可以直接通信，具有较高灵活性，但需要消息路由规则和通信预算。

若 n 个智能体形成完全连接的无向网络，潜在连接数量为： $E=\frac{n(n-1)}{2} $

因此，完全连接网络的连接规模为： $E = O(n^2)$

这里描述的是潜在连接数量，不代表每次任务都必须发生这么多次通信。

Hierarchical Architecture（层级架构）

智能体被组织为多个局部团队，由上层协调者处理跨团队信息交换。

这种方式能够减少直接跨团队通信，但增加了信息传递层级。

因此，对等网络与层级架构的主要权衡是： $\text{Communication Flexibility} \quad\text{vs.}\quad \text{Coordination Overhead} $

8.4.3 黑板架构 Blackboard Architecture

黑板架构允许多个智能体通过共享工作空间进行协作，而不必为每项任务建立直接通信连接。

共享工作空间可以保存：Evidence：证据、Requests：任务请求、Drafts：草稿、Priorities：优先级、Status：执行状态。智能体读取共享状态，并向其中提交自己的工作成果。

黑板架构需要解决三个问题。

1. Shared Workspace：共享工作空间

确保各智能体能够访问相关任务状态和中间结果。

2. Contributions and Control：贡献与控制

通过调度或控制策略决定下一步应由哪个智能体作出贡献。

3. Consistency：一致性

对共享产物进行版本管理，并处理多个智能体同时写入时的冲突。

例如，Researcher 更新了产品事实后，Writer 应基于最新已核实版本修改文案，而不是继续使用旧信息。

因此，共享可见性不等于自动实现协调。黑板架构仍需要明确调度、权限和一致性规则。

8.4.4 图结构与混合工作流 Graph-Based and Hybrid Workflows

图结构工作流将执行节点、数据依赖和路由条件显式表示出来。

可以抽象为有向图： $G = (V, E)$

- V：执行节点集合。
- E：节点之间的依赖关系或状态转移。

组成部分	作用
Node Types	定义 Agent、模型调用、工具、函数和人工输入等节点
Routes	定义分支、汇合与循环的条件
Guards	检查权限、状态、合法转移和执行限制

例如，内容生成工作流可以设计为：这种设计的优势是控制流明确，便于验证任务状态、执行条件和终止规则。

8.5 系统架构选择 Architecture Selection

课件根据任务结构、控制需求和协调成本，比较了五类架构模式。

架构模式	适用情况	主要权衡
Pipeline	依赖关系稳定的任务	上游错误容易向下游传播
Centralized	需要统一任务分解与结果整合	管理者瓶颈
Peer Network	小规模团队需要灵活交流	消息与协调成本
Blackboard	多个智能体共同维护动态状态	一致性与调度
Graph	存在明确分支、循环和混合节点	控制流设计与维护成本

架构选择应优先分析任务结构，而不是直接选择最复杂的模式。

例如：

- 稳定的数据处理流程适合 Pipeline。
- 需要统一分工的研究团队适合 Centralized。
- 小规模自主协商任务适合 Peer Network。
- 需要持续共享中间成果的任务适合 Blackboard。
- 包含条件分支、人工审批和重试循环的任务适合 Graph。

需要注意，这些架构并不完全互斥。实际系统可以采用集中式协调，同时通过图结构定义工作流，并使用共享状态保存执行结果。

8.6 多智能体通信设计 Communication Design

通信不仅是信息传输机制，也是影响智能体后续决策的重要组成部分。

一个消息可能改变另一个智能体的知识状态，从而改变其下一步行动。

通信设计的四个方面

设计方面	核心问题
Message Format	传递什么结果、请求或证据?
Routing and Turn-Taking	谁接收消息，谁下一步行动?
Shared State	使用哪个产物及其版本?
Termination	消息是否触发继续、完成或升级?

通信系统必须能够识别消息发送者、接收者、任务上下文、执行状态以及后续处理责任。

结构化消息模式（Message Schema）

这一结构使系统能够明确：Writer 请求 Researcher 检查第二版文案中的“今年夏天最受欢迎”声明是否有证据支持。

消息验证机制

要求在消息接收和回复后进行不同层次的验证。

1. Structure Validation：结构验证

检查必需字段是否存在，字段类型是否合法。

例如，`task_id` 应为有效任务标识，`recipient` 应为可识别的智能体。

2. Permission and State Validation：权限与状态验证

检查：发送者是否有权发起该请求、接收者是否允许执行指定操作、当前任务状态是否允许该转移、产物版本是否仍然有效。

3. Evidence Validation：证据验证

当 Researcher 返回结果后，需要进一步判断来源是否支持原有声明。

因此： Valid Message ≠ Valid Claim，结构正确、权限合法的消息，其内容仍可能缺乏事实依据。

8.7 直接消息与共享状态 Direct Messaging and Shared State

智能体可以通过直接消息或共享产物交换信息。

1. 直接消息（Direct Messaging）

直接消息具有明确的发送者和接收者，通常用于：

- 请求其他智能体执行操作。
- 传递特定任务结果。
- 追踪消息送达与回复责任。

例如： $\text{Writer} \xrightarrow{\text{Evidence Request}} \text{Researcher}$

这种机制适合需要明确责任归属的任务交接。

2. 共享状态（Shared State）

共享状态允许多个智能体读取或更新共同的任务产物。

系统需要管理：Artifact Version：产物版本、Access Control：访问权限、Write Ownership：写入责任、Consistency：数据一致性。

例如，多个智能体可以共享同一份产品事实文件，但必须保证 Writer 和 Designer 使用的是一致且经过确认的版本。

两种机制的区别

维度	Direct Messaging	Shared State
交互方式	向特定接收者发送消息	读取或修改共同产物
适用场景	明确请求、任务交接	持续共享信息与中间结果
关键管理问题	路由、送达、回复责任	版本、访问权限、写入冲突

两种机制可以结合使用：通过直接消息通知任务变化，通过共享状态保存完整产物。

8.8 在 Prompt 中描述协作智能体

智能体需要知道系统中有哪些其他角色，以及什么时候应该与它们通信。

因此，应在角色指令中包含协作智能体的描述。

1. Capability（能力）

智能体需要了解其他角色具备哪些工具和权限，以便将请求发送给合适的对象。

2. Knowledge（知识）

智能体需要判断其他角色已经掌握哪些证据、信息和产物版本，避免基于过期或缺失的信息进行协作。

3. Objective（目标）

智能体需要了解其他角色的职责和优先级，以预测其响应方式并协调共同目标。

例如，Writer 在发现宣传声明缺乏证据时，应知道 Researcher 负责事实核查，并将具体声明及文案版本发送给 Researcher，而不是自行假定声明成立。

这种设计有助于减少角色之间的通信歧义，但实际权限仍应由运行时系统独立验证，而不能仅依赖 Prompt 约束。

8.9 多智能体系统的终止、恢复与升级机制

多智能体系统需要明确区分正常完成、错误恢复、替代执行、升级处理和强制停止。

五种执行结果：

结果	触发条件	系统响应
Done	满足验收标准	返回经过验证的结果
Retry	可恢复的暂时性错误	在固定限制内重试
Fallback	存在获得授权的替代方案	使用替代方案并记录变化
Escalate	存在无法解决的冲突或不确定性	提交相关状态，请求进一步决策
Stop	达到预算、权限或安全限制	停止执行并报告未完成工作

Retry 与 Fallback 的区别

Retry 表示在相同或修正后的条件下再次尝试原操作。例如，网络请求暂时失败后重新请求。

Fallback 表示使用另一种被允许的执行方式。例如，主要数据服务不可用时，改用经过授权的备用数据源。

前者主要针对可恢复错误，后者主要针对可替代执行路径。

Escalate 与 Stop 的区别

Escalate 表示自动决策无法继续，需要其他角色或人工介入。

Stop 表示执行必须终止，例如预算耗尽、权限不足或安全约束被触发。

二者都可能导致当前自动流程暂停，但 Escalate 强调移交决策责任，Stop 强调终止当前执行。

有界执行与停止条件

多智能体系统可以同时设置步数、成本和时间限制：

$N_{steps} \leqslant N_{max}$，$C_{\text{total}}\leq C_{\max} $，$T_{elapsed} \leqslant T_{max}$

- N_{max}：最大执行步数。
- C_{max}：最大允许成本。
- T_{max}：最大执行时间。

达到限制后，系统应返回实际完成状态，并记录尚未完成的任务，而不是将停止误判为成功。

8.9 多智能体系统评估与风险规划 Evaluation and Risk Planning

多智能体系统的评估不能只检查最终输出，还应考虑执行成本、响应时间以及系统在异常情况下的行为。

1. 质量与价值（Quality and Value）

应将多智能体系统与单智能体基线进行比较。

主要指标包括：Quality：任务完成质量、Cost：模型调用、工具使用和协调成本、Latency：任务完成延迟。

如果多智能体系统增加了大量成本，却没有带来明显的质量改善，就需要重新评估其架构合理性。

2. 失败与预算测试（Failure and Budgets）

除了正常任务测试，还应验证系统在异常情况下的行为，包括：权限不足、工具执行失败、证据缺失、智能体之间出现冲突、重试次数耗尽、资源预算不足、需要人工介入。

评估重点不仅是系统能否成功完成任务，还包括它能否在无法完成任务时安全、明确地终止。

多智能体系统设计检查清单（Design Review Checklist）

在正式实施前，应检查以下五个方面。

检查维度	关键问题
Tasks	需要交付什么？如何判断任务成功？
Roles	谁负责各项职责？允许执行哪些操作？
Coordination / Architecture	谁负责分配任务、解决冲突和整合结果？
Messages	哪些信息需要传递？是否需要结构与版本验证？
Stop and Recover	何时完成、重试、升级或停止？

这五项检查共同确保系统的任务、职责、控制、状态和完成规则在实施前已经明确。

8.10 本章总结

本章从系统工程角度讨论了多智能体系统的设计方法，核心结论如下。

1. 任务分解与角色分配是设计起点。 系统需要先明确子任务、依赖关系、输入输出和责任归属，再决定需要哪些智能体。
2. 协调模型与执行工作流属于不同设计维度。 集中式、去中心化和混合式描述决策权的组织方式；顺序、并行和迭代工作流描述任务的时间执行关系。
3. 系统架构必须与任务结构匹配。 Pipeline、Peer Network、Blackboard 和 Graph 等架构具有不同的适用条件与协调成本，不存在对所有任务都最优的单一架构。
4. 通信是多智能体决策机制的一部分。 消息格式、路由规则、共享状态、版本管理和证据验证会直接影响智能体的后续行动。
5. 终止机制与评估机制不可缺少。 系统必须明确完成、重试、替代、升级和停止条件，并综合评价质量、成本、延迟及异常处理能力。

其中，单智能体已经能够完成工具调用、规划、反思和动态决策。多智能体系统则进一步通过专业化分工、信息交换和协作控制处理复杂任务。

因此，多智能体设计的核心不是增加智能体数量，而是通过合理的角色划分、协调架构、执行工作流和通信规则，提高整体任务的完成质量与可靠性。

8.11 课堂练习 In-Class Exercises

Q1：工具观察结果与当前计划矛盾时，ReAct 允许智能体做什么?

正确答案：A. Use observation to revise the next action.

ReAct 允许智能体根据新观察修改下一步行动，而不是忽略新信息或每次都重新启动整个任务。

Q2：两个智能体共享同一环境，但收到不同观察结果，意味着什么?

正确答案：A. Each may need to decide using incomplete information.

共享环境不意味着具有相同观察。不同智能体可能只能访问环境状态的不同部分，因此需要基于局部信息进行决策。

Q3：Researcher 表示“Research done”，还应增加什么信息才能使交接对 Writer 更有用?

正确答案：A. The findings, their sources, and unresolved questions.

有效交接需要包含结论、来源、验证状态和未解决问题，使后续智能体能够直接利用研究成果并判断其可靠性。

5584 LLM for Agent 1

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_5/knowledge/5584-LLM-for-Agent-1.md`

5584 LLM for Agent1

Lecture 1 大语言模型与智能体导论 Course Overview

Part 1 智能体与大语言模型的基本概念

1.1 智能体（Agent）的定义

An “intelligent” system that interacts with some “environment”.

能够与环境进行交互的智能系统。

这一定义包含三个基本要素：

- Agent：负责决策与行动的主体；
- Environment：Agent 所处并与之交互的环境；
- Interaction：通过观察与行动形成的交互过程。

Agent 通过 Observation（观察）获取环境信息，并通过 Action（行动）影响环境。

由此，Agent 的基本运行模式可概括为：Observe → Decide → Act → Observe

与一次性生成答案的系统相比，Agent 更强调持续交互、状态更新与行动反馈。

1.2 Agent 的环境类型

课件根据交互环境将 Agent 分为三类。

环境类型	典型系统	主要特征
Physical Environment	Robot、Autonomous Car	通过传感器感知物理世界，执行实际动作
Digital Environment	DQN for Atari、AlphaGo、Siri	通过数字接口观察环境并执行操作
Humans as the Environment	Chatbot	通过语言与人交互，人的回应构成反馈

PPT p.9 还展示了自动驾驶汽车、AlphaGo、ChatGPT、Dota 2、Alexa 和机械臂等示例，说明 Agent 并不限于聊天机器人，而是一个涵盖物理和数字交互系统的广义概念。

课件同时指出，Agent 的定义取决于如何界定 Intelligence（智能）和 Environment（环境），而这些概念会随着技术发展发生变化。

1.3 大语言模型（LLM）的定义与学习机制

Large Language Model 可以从 Language Model 与 Large 两个方面理解。

Language Model（语言模型） 的核心任务是对语言序列进行建模，并根据输入生成文本。

Large（大规模） 主要体现在两个维度：

1. Massive Number of Parameters：具有大量模型参数；
2. Huge Amounts of Training Data：利用大规模数据进行训练。

LLM 通常采用 Transformer、Mamba 等模型架构，通过训练数据调整模型权重。给出的训练数据来源包括互联网、书籍和代码。

从概率建模角度，语言模型可以表示为：$P(x_1,x_2,\ldots,x_T) = \prod_{t=1}^{T} P(x_t\mid x_1,\ldots,x_{t-1})$。其中，$x_t$ 表示第 t 个 Token。对于自回归语言模型，其核心目标是根据历史 Token 预测下一个 Token。

1.4 LLM 的能力与局限性

课件将 LLM 的主要能力归纳为五类。

能力	英文术语	典型应用
文本生成	Text Generation	邮件、文章、诗歌、代码
机器翻译	Translation	多语言文本转换
文本摘要	Summarization	提取长文档关键信息
问题回答	Question Answering	根据知识或上下文回答问题
对话系统	Conversational AI	聊天机器人、虚拟助手

但 LLM 仍然存在以下问题：

局限性	英文术语	具体表现
幻觉	Hallucination	生成看似合理但不正确的信息
偏见	Bias	反映训练数据中的偏见
实时信息不足	Real-time Information	无法仅凭静态权重获取训练后的新信息
隐私问题	Privacy	训练数据可能包含个人信息
安全问题	Security	可能受到恶意输入操纵

这些局限性构成了引入工具、记忆、检索、验证和外部执行机制的重要动机。

1.5 LLM Agent 与普通 LLM、AI Workflow 的区别

PPT p.15 从、及三个角度定义 AI Agent。

外部行为：从外部看，Agent 是能够自主运行的实体；

内部结构：从内部看，它是包含模型调用、通常也包含工具调用的程序。

比较维度	LLM	AI Workflow	LLM Agent
核心功能	语言生成与推理	按预设流程执行任务	自主决策与执行
执行路径	主要由调用方式决定	通常预先定义	可以根据反馈动态调整
环境交互	需依赖外部系统	由流程设计决定	是核心能力
工具使用	需外部封装	按预设步骤调用	可根据任务选择调用
自主性	较低	受流程限制	相对较高

系统能力：Autonomous and can “morph”。其中，Morph 指 Agent 能够在运行过程中根据环境反馈改变执行路径，而不是始终遵循固定步骤。

因此，LLM Agent 与 AI Workflow 的核心区别不在于是否使用 LLM，而在于系统是否能够自主决定后续行动。

1.6 LLM Agent 的核心组成

1. Planning：规划。规划主要包含两个方面

Subgoal and Decomposition（子目标分解）

将复杂任务拆解为较小、可管理的子任务，提高任务执行的可操作性。

例如：$G \rightarrow \{g_1, g_2, \cdots, g_n\}$。其中，$G$ 为整体目标，g_i 为子目标。

Reflection and Refinement（反思与改进）

Agent 对过去的行动和结果进行评估，通过自我批评和反思修正后续行为。

2. Memory：记忆。Memory 分为短期记忆与长期记忆。

类型	Short-term Memory	Long-term Memory
存储位置	当前上下文	外部存储系统
主要机制	In-context Learning	Vector Store、Retrieval
生命周期	通常受当前会话或上下文限制	可以跨较长时间保留
主要作用	保持当前任务状态	保存历史信息并按需召回

长期记忆通常通过外部向量存储和快速检索实现。 “infinite” 强调其可扩展性，而非真正无限的物理存储容量。

3. Tool Use：工具使用。工具使用使 Agent 能够通过外部 API 获取模型权重中没有的信息或能力。主要包括：

- Current Information：访问当前信息；
- Code Execution：执行代码与计算；
- Proprietary Information：访问专有数据；
- External APIs：调用日历、搜索、计算器等服务。

工具使用的本质是将 LLM 的能力从文本生成扩展到外部系统操作。

4. Action：行动

Action 是 Agent 将规划结果转化为实际操作的阶段。行动既可以发生在数字环境中，也可以作用于物理机器人或设备。行动的有效执行依赖于规划、记忆与工具使用，并通过环境反馈形成新的决策循环。

1.7 LLM Agent 的独特优势

- Language Mastery：能够理解和生成自然语言，支持人机交互。
- Decision-making：具备一定推理与决策能力，能够处理复杂任务。
- Flexibility：可适配不同任务与应用场景。

- Collaborative Interactions：能够与人类或其他 Agent 协作。

LLM 是 Agent 的重要认知组件，但 Agent 是包含规划、记忆、工具与行动机制的完整交互系统。

Part 2 智能体的发展历史：从规则系统到生成式 Agent

核心问题串联智能体的发展历史：How does a machine decide what to do next?

机器如何决定下一步应该执行什么行动？智能体的发展可以划分为三个阶段：

阶段	时间	核心范式
Symbolic & Rule-Based	1950s - 1980s	基于符号表示和人工规则
Formalization & Learning	1990s - 2010s	智能体形式化、搜索与机器学习
Generative & Agentic	2017 - Present	基于大模型的推理、行动与工具协作

2.1 符号主义与规则驱动阶段

1. Turing Test：智能的行为判据（1950）

Alan Turing 在 1950 年提出一个重要问题：Can machines think?

由于“思考”难以直接定义，Turing 将问题转化为 Imitation Game（模仿游戏），通过机器在交互中的可观察行为判断其智能表现。

这一思想为后续智能体研究提供了重要启示：可以通过系统行为而非内部主观体验评价智能。

2. Dartmouth Workshop：人工智能学科的建立（1956）

1956 年的 Dartmouth Summer Research Project 被视为人工智能学科的重要起点。

John McCarthy 提出了 Artificial Intelligence 这一名称，相关研究者包括 Marvin Minsky、Nathaniel Rochester 和 Claude Shannon。

这一阶段确立了人工智能作为独立研究领域的基本方向。

3. ELIZA：基于模式匹配的对话系统（1966）

ELIZA 通过关键词识别、模式匹配和文本替换生成对话回复，其中 DOCTOR 脚本模拟心理咨询式对话。其主要特点是：

- 使用预先定义的文本匹配规则；
- 不具备真正的领域知识或任务规划能力；
- 仍然能够使部分用户产生系统理解自己的印象。

这一现象被称为 ELIZA Effect。

Fluent ≠ Understanding（语言流畅不等于真正理解）。

4. MYCIN：基于规则的专家系统（1970s）

MYCIN 使用约 600 条人工编写的规则，为感染性疾病相关治疗提供抗生素建议。课件指出，其建议约有 65% 被评价为可接受，与 Stanford 专家教师的表现相当，但系统并未实际用于临床。

其主要限制不仅在于推理准确率，还在于实际系统集成：相关事实需要人工输入，难以自然融入真实工作流程。

5. BDI：信念—愿望—意图模型（1987）

BDI（Belief - Desire - Intention）通过三个概念描述智能体的决策状态。

组成	含义	对应问题
Belief	智能体认为当前世界是什么状态	What is true?
Desire	智能体希望达到什么目标	What does it want?
Intention	智能体决定执行什么计划	What does it commit to?

BDI 将 Agent 的内部状态与决策过程联系起来。

这一思想与现代 LLM Agent 的任务状态、目标管理和计划执行具有概念上的连续性。

2.2 智能体形式化与机器学习阶段

1. Russell & Norvig：Agent 的形式化定义（1995）

Perceiving its environment through sensors, acting upon it through actuators.

智能体通过 Sensors（传感器）感知环境，并通过 Actuators（执行器）作用于环境。

在现代 LLM Agent 中，可以建立如下对应关系：

经典 Agent	LLM Agent
Sensors	Context Window、工具返回信息
Percepts	用户输入、观察结果
Decision-making	LLM 推理与规划
Actuators	Tool Calls、API Calls
Actions	工具执行、系统操作

因此，LLM Agent 并未改变经典 Agent 的基本交互结构，而是改变了感知、决策和行动的实现方式。

2. Deep Blue：专用任务中的超人类表现（1997）

IBM Deep Blue 在 1997 年以 3.5 : 2.5 击败国际象棋世界冠军 Garry Kasparov。Deep Blue 主要依靠高性能搜索和人工设计的评估机制，而非现代意义上的通用学习系统。

其意义在于证明：机器可以在特定任务中达到超越人类专家的水平，但这种能力不一定能够迁移到其他任务。

3. LSTM：序列建模与长期依赖（1997）

LSTM（Long Short-Term Memory）由 Hochreiter 和 Schmidhuber 提出，通过门控机制控制信息的写入、保留和输出。

其核心包括：Forget Gate：遗忘门；Input Gate：输入门；Output Gate：输出门；Cell State：单元状态。

LSTM 的主要贡献是改善传统循环神经网络在长序列学习中的梯度传播与长期依赖问题。

课件将其与 Deep Blue 对比：Deep Blue 展示了专用搜索系统的能力，而 LSTM 推动了通用序列学习的发展。

4. Siri 与 Alexa：广泛部署但能力受限（2011 - 2014）

Siri 于 2011 年推出，Alexa 于 2014 年推出。这类早期语音助手能够完成语音识别、意图匹配和预设任务，但存在以下限制：

- 主要依赖 Intent Matching（意图匹配），缺乏复杂推理；
- 交互通常以单轮任务为主；
- 每项技能需要人工设计；
- 难以处理开发者未预先定义的新任务。

其核心局限是缺乏对未知任务的广泛泛化能力。

2.3 生成式与智能体化阶段

1. Transformer：注意力机制与规模化训练（2017）

Vaswani 等人在 Attention Is All You Need 中提出 Transformer。其核心特征是使用注意力机制进行序列建模，摆脱传统循环结构对逐步计算的依赖。

标准 Scaled Dot-Product Attention 表示为： $\operatorname{Attention}(Q,K,V) = \operatorname{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right)V $

Transformer 支持训练阶段对序列进行高度并行化处理，为大规模语言模型训练提供了重要基础。

2. ReAct：推理与行动交替执行（2022）

ReAct（Reasoning and Acting）将模型推理与环境行动结合。其典型过程为：

Thought: 分析当前问题，决定需要什么信息

Action: 调用搜索、计算器或其他外部工具

Observation: 接收工具返回的环境反馈

Update / Finish: 根据观察结果继续推理、调整行动或结束任务

ReAct 的重要特点是允许 Agent 在任务执行过程中根据观察结果调整计划。

3. Function Calling：结构化工具调用（2023）

Function Calling 将工具使用转化为具有明确参数规范的接口调用。模型负责生成工具名称和调用参数，外部程序负责执行工具并返回结果。

因此，模型提出调用请求，实际工具由执行环境运行。这一机制减少了人工解析非结构化 Prompt 输出的需求，提高了工具调用的可控性。

4. MCP 与 A2A：智能体基础设施标准化（2024 - 2026）

MCP（Model Context Protocol）用于标准化模型应用与外部数据、工具之间的连接。MCP 可以减少原本需要针对多个模型与多个工具分别开发的定制集成。

若存在 N 个模型应用和 M 个工具，完全定制集成在最简单的计数模型下可能需要：$ C_{\text{custom}}=N\times M $，统一协议可以通过标准接口减少重复适配工作，但实际集成复杂度仍取决于具体系统。

A2A（Agent-to-Agent）则面向不同 Agent 之间的通信与协作。

课件指出，MCP 于 2024 年 11 月出现，随后在 2025 年获得更多厂商支持；2025 - 2026 年编码智能体逐渐成为重要应用方向。同时引用 Gartner 对智能体项目取消风险的预测，强调智能体落地的核心挑战逐渐从能力演示转向系统可靠性。

2.4 三个阶段的本质变化

The shift is not intelligence but agency — who decides the next step, the programmer or the model?

三个阶段最重要的区别在于决策权的变化：

- 规则系统：程序员预先定义规则和执行路径；
- 学习系统：模型学习部分决策机制，但通常针对特定任务；
- LLM Agent：模型可以根据任务目标和反馈动态决定后续行动。

因此，智能体技术的发展不仅表现为模型能力增强，更表现为行动决策权从固定程序逐步转向具备自主性的模型系统。

Part 3 大语言模型的发展与规模化能力

3.1 从单任务模型到通用语言模型

传统机器学习通常针对不同任务分别训练模型，例如文本分类、翻译、摘要和问答。

这一范式存在两个主要问题：

1. 每个任务可能需要单独训练和维护模型；
2. 每个任务通常需要大量标注数据。

大语言模型改变了这一范式：通过大规模预训练学习通用语言表示，再利用任务描述或少量示例完成不同任务。

语言模型的优势：通过学习预测下一个 Token 或句子，使一个模型能够执行多种任务，而非仅完成单一任务。

3.2 BERT 与 GPT：表示学习和生成学习

BERT 和 GPT 所代表的两条技术路线：

比较维度	BERT	GPT
核心方向	Representation	Generation
主要架构	Encoder-only	Decoder-only
典型预训练目标	Masked Language Modeling	Autoregressive Language Modeling
优势任务	分类、语义理解	文本生成、续写
上下文建模	双向上下文	因果注意力

BERT 侧重学习输入文本的上下文表示，而 GPT 侧重根据已有文本预测后续 Token。

两者均属于预训练语言模型，但其训练目标和注意力结构不同。

3.3 GPT-2：从任务专用模型到通用模型

GPT-2 的重要研究方向是探索语言模型对未见任务（Unseen Tasks）的泛化能力。

传统任务专用模型可表示为：$P(y \mid x)$，其中，x 为任务输入，y 为输出。

通用模型进一步将任务描述纳入条件：$P(y\mid x,\tau)$，*其中*，τ 表示 Task Specification（任务规范）。

例如，任务描述可以直接使用自然语言：Translate this French text to English.

这样，模型不仅根据输入内容生成结果，还可以根据任务说明调整生成行为。

这一转变使任务描述成为模型输入的一部分，为后续 Zero-shot 和 Few-shot Learning 提供了基础。

3.4 Zero-shot Learning 与规模化训练

GPT-2 的研究动机之一是减少对每个新任务进行独立微调的依赖。

构建具有广泛泛化能力的语言模型需要两个重要条件：

1. Diverse Training Data（多样化训练数据）

训练数据覆盖多种语言、主题、任务和表达形式，有助于模型学习可迁移的知识与模式。

2. Scaling Model Capacity and Data（扩展模型容量与数据）

增加模型参数量和训练数据规模，可以改善语言建模能力，并可能提高下游任务表现。

Perplexity（困惑度）说明模型规模化的效果：

对于长度为 T 的 Token 序列，困惑度定义为：$\operatorname{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^{T} \log P(x_t\mid x_{<t}) \right) $。困惑度越低，通常表示模型对测试序列的概率预测越准确。但较低的困惑度并不必然意味着模型在所有下游任务中表现更好。

3.5 GPT-3 与模型规模扩展

对比了 GPT-2 和 GPT-3 的参数规模，单位：十亿参数（Billion Parameters）

GPT-2 的最大版本约为 1.5 Billion Parameters，而 GPT-3 为 175 Billion Parameters。

规模扩展使研究者能够进一步探索大模型在少样本学习、推理和复杂任务中的能力变化。

3.6 Emergent Abilities：涌现能力

Emergent Abilities（涌现能力）：某些能力在较小模型中不明显或不存在，但在更大模型中表现出来。例如，GPT-3 在仅接受文本训练的情况下，能够完成部分加减法任务。

1. 不同能力可能在不同模型规模下出现；
2. 模型规模并非涌现能力的唯一决定因素；
3. 模型架构、训练数据与训练方法同样可能影响能力表现。

引用 BIG-Bench 的部分实验结果：在 14 项任务上，LaMDA 137B 和 GPT-3 175B 的表现接近随机水平，而参数量更小的 PaLM 62B 达到高于随机的表现。

这说明：$\text{Model Performance} \neq f(\text{Parameter Count Only}) $，模型表现受到多个因素共同影响。

涌现能力的具体判定还与评价指标有关。某些看似突然出现的能力，也可能与离散评价标准有关，因此不能简单认为参数规模达到某个固定阈值就必然产生某种能力。

3.7 LLM 的基本特征

将大语言模型概括为具有大量参数、能够通过 Prompting 完成多种任务的语言模型。

课件采用超过 1B 参数作为介绍性规模描述，并列举 GPT、Llama 2、Gemini、PaLM 和 Mixtral 等模型。

这一规模描述并不是严格、普遍适用的 LLM 分类标准。

本章的核心结论是：语言模型从针对特定任务训练，逐渐发展为通过大规模预训练获得通用能力，并通过自然语言任务描述适配不同任务。

Part 4 大语言模型架构与注意力机制

4.1 三种 Transformer 架构

将主流语言模型架构划分为 Encoder-only、Decoder-only 和 Encoder - Decoder 三类。

架构	代表模型	预训练目标	主要应用
Encoder-only	BERT	Masked Language Modeling	文本分类、语义理解
Decoder-only	GPT	Autoregressive Language Modeling	文本生成、通用语言建模
Encoder - Decoder	T5、T0	Masked Span Prediction	机器翻译、摘要、文本转换

1. Encoder-only：掩码语言建模

Masked Language Modeling（MLM）随机遮盖输入中的部分 Token，要求模型根据上下文预测被遮盖内容。

设被遮盖的位置集合为 $\mathcal{M}$，则其训练目标可表示为：$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log P_\theta(x_i \mid x_{\setminus \mathcal{M}})$

由于 Encoder 可以利用双向上下文，因此适合学习文本的语义表示。

2. Decoder-only：自回归语言建模

Decoder-only 模型根据历史 Token 预测下一个 Token。

其训练损失为：$ \mathcal L_{\text{AR}} = -\sum_{t=1}^{T} \log P_\theta(x_t\mid x_{<t})$这种结构适合逐 Token 生成文本，是 GPT 和 LLaMA 等模型采用的主要架构。 Decoder-only 架构具有训练稳定、收敛较快以及预训练后泛化能力较强等优势。

3. Encoder - Decoder：序列到序列建模

Encoder - Decoder 架构使用 Encoder 表示输入，Decoder 根据输入表示生成输出。

其条件生成过程可表示为：$P(y\mid x) = \prod_{t=1}^{T_y} P(y_t\mid y_{<t},x)$，$x$：输入序列；$y$：输出序列；$T_y$：输出序列长度。

这种架构适合机器翻译、摘要等输入输出之间存在明确映射关系的任务。

4.2 T5/T0：Masked Span Prediction

Masked Span Prediction（片段掩码预测），其过程包括：

1. 从输入序列中选择连续 Token 片段；
2. 使用特殊标记替换这些片段；
3. 将处理后的序列输入 Encoder；
4. 由 Decoder 生成被遮盖的片段。

与 BERT 的 Token 级掩码相比，Span Prediction 侧重对连续文本片段进行重建。

这种目标能够使模型学习较长范围的语义结构，并适用于文本生成任务。

4.3 Attention Patterns：注意力模式

不同架构中的注意力连接方式：

1. Bidirectional Attention（双向注意力）

Token 可以访问序列中的其他 Token，适合上下文理解。

2. Causal Attention（因果注意力）

第 t 个 Token 只能访问自身及此前的 Token，避免生成时访问未来信息。

对于 Causal Attention，可以定义注意力掩码：$M_{ij} = \begin{cases} 0,&j\le i\\ -\infty,&j>i \end{cases}$

注意力计算为：$\operatorname{Attention}(Q,K,V) = \operatorname{softmax} \left(\frac{QK^\top}{\sqrt{d_k}}+M \right)V $

在 Encoder - Decoder 架构中，Encoder 通常对输入进行双向注意力建模，而 Decoder 在生成输出时使用因果注意力，并通过 Cross-Attention 访问 Encoder 的表示。

还给出了 Decoder-only 与 Encoder - Decoder 架构的经验性能比较，指出在其引用的相近配置实验中 Decoder-only 表现更优。这并不意味着 Decoder-only 在所有任务中都优于 Encoder - Decoder。

4.4 LLaMA 2 的架构改进

以 LLaMA 2 为例，介绍了 Decoder-only Transformer 的重要设计变化。

技术	传统设计	LLaMA 2 相关设计	目的
Normalization Position	Post-Norm	Pre-Norm	改善训练稳定性
Normalization Type	LayerNorm	RMSNorm	简化归一化并提高效率
Activation	ReLU	SwiGLU	增强非线性表示能力
Position Encoding	Absolute/Relative PE	RoPE	建模位置信息
Attention	Multi-Head Attention	Grouped-Query Attention（部分配置）	减少 KV 存储与推理开销

1. Pre-Norm

Post-Norm 通常在残差连接后执行归一化：$y = \mathrm{Norm}(x + F(x))$

Pre-Norm 则在子层计算前进行归一化：$y = x + F(\mathrm{Norm}(x))$

Pre-Norm 有助于改善深层 Transformer 的梯度传播。

2. RMSNorm

RMSNorm 通过均方根进行归一化：$\operatorname{RMS}(x) = \sqrt{ \frac{1}{d}\sum_{i=1}^{d}x_i^2+\epsilon }$，$\operatorname{RMSNorm}(x) = \frac{x}{\operatorname{RMS}(x)} \odot\gamma $

与 LayerNorm 不同，RMSNorm 不显式减去均值。

3. SwiGLU

SwiGLU 是一种门控激活机制，可写为：$ \operatorname{SwiGLU}(x) = \operatorname{Swish}(xW_1)\odot(xW_2)$，*其中*：$\operatorname{Swish}(z)=z\sigma(z)$

SwiGLU 通过门控乘积提高前馈网络的表达能力。

4. RoPE

RoPE（Rotary Position Embedding）通过旋转变换将位置信息融入 Query 和 Key。

其重要特点是使注意力计算能够反映 Token 之间的相对位置信息。

5. Grouped-Query Attention

Grouped-Query Attention（GQA）允许多个 Query Heads 共享较少数量的 Key/Value Heads。

设 Query Head 数量为 H_q，KV Head 数量为 H_{kv}，则在其他条件相同时，KV Cache 的规模与 H_{kv} 成正比。

相对于相同数量 Query/KV Heads 的普通多头注意力，KV 存储比例约为：$\frac{M_{\text{GQA}}}{M_{\text{MHA}}} = \frac{H_{kv}}{H_q}$

GQA 因而能够减少 KV Cache 占用及相关内存带宽需求。

Part 5 大语言模型的训练与对齐

5.1 Decoder-only LLM 的训练流程

以 LLaMA 2 为例，将大语言模型的训练划分为三个主要阶段：

Stage 1 Autoregressive Pre-training：通过大规模语料学习语言建模能力与通用知识

Stage 2 Supervised Fine-tuning（SFT）：利用指令与目标回答训练模型遵循任务要求

Stage 3 Safety / RLHF：利用人类偏好反馈进一步优化模型行为

这三个阶段分别解决不同问题：

- Pre-training：模型如何学习通用语言能力？
- SFT：模型如何理解并执行人类指令？
- RLHF：模型如何使输出更符合人类偏好和安全要求？

1. Autoregressive Pre-training：自回归预训练

预训练的核心目标是根据历史 Token 预测下一个 Token。

设训练序列为：$X = (x_1, x_2, \cdots, x_T)$，其最大似然训练目标为：$\max_\theta \sum_{t=1}^{T} \log P_\theta(x_t\mid x_{<t})$ 。*等价地，最小化负对数似然损失*：$\mathcal L_{\text{pretrain}} = -\sum_{t=1}^{T} \log P_\theta(x_t\mid x_{<t})$，LLaMA 2 的预训练使用了约 3 Trillion Tokens 的大规模语料。预训练主要使模型获得语言模式、知识和一定的任务泛化能力，但并不直接保证其能够遵循复杂指令。

2. Supervised Fine-tuning：监督微调

SFT 使用带有标准输出的任务样本进一步训练预训练模型。

典型数据形式为：$D_{\text{SFT}} = \{(I_i, X_i, Y_i)\}_{i=1}^{N}$

- I_i：Instruction，任务指令；X_i：Input，任务输入；Y_i：Target Output，期望输出。

SFT 的训练目标为：$\mathcal L_{\text{SFT}} = -\sum_{i=1}^{N} \log P_\theta(Y_i\mid I_i,X_i)$

在 Token 层面，通常只对目标回答部分计算损失，而将指令和输入作为条件上下文。

SFT 与 Pre-training 的区别

比较维度	Pre-training	SFT
数据形式	大规模自然语言语料	指令、输入与目标输出
预测目标	自回归预测语料 Token	预测目标回答 Token
主要目的	学习通用语言知识与能力	学习任务遵循与输出行为
数据要求	大规模预训练语料	高质量指令样本
主要结果	Base Model	Instruction-tuned Model

PPT 还指出，SFT 可以通过训练数据引导模型输出的有用性（Helpfulness）和安全性（Safety）。

5.2 Instruction Tuning：指令微调

Instruction Tuning 通过多种自然语言指令训练模型，使其能够理解不同任务要求。

与只针对某个固定任务进行微调不同，指令微调通常利用多任务数据，提高模型对新任务描述的泛化能力。

因此：Pre-training 主要学习语言与知识分布，Instruction Tuning 主要学习如何按照指令使用这些能力。

5.3 Alignment：模型对齐问题

Unsafe Outputs（不安全输出）及 Alignment Problem（对齐问题）。

模型可能生成：

- 包含偏见、歧视或不当表达的文本；
- 可能造成直接危害的信息；
- 不符合人类偏好、价值或安全要求的内容。

因此，模型训练不仅需要提高任务准确率，还需要使其输出符合适当的人类偏好和行为约束。

两类主要方法：

1. SFT：通过高质量示例学习期望行为；
2. RLHF：通过人类偏好反馈进一步优化输出。

5.4 RLHF：基于人类反馈的强化学习

RLHF（Reinforcement Learning from Human Feedback）通常包含三个核心步骤。

1. 收集人类偏好数据

对于同一个 Prompt，模型生成多个候选回答，由人工标注者比较其质量。

偏好数据可以表示为： $ D_{\text{pref}} = {(x,y^+,y^-)}$

- x：输入 Prompt；
- y^+：更受偏好的回答；
- y^-：较不受偏好的回答。

2. 训练奖励模型（Reward Model）

奖励模型学习根据输入和回答输出一个偏好分数：$ r_\phi(x,y)$

一种常见的成对偏好训练目标为：$\mathcal{L}_{\text{RM}} = -\mathbb{E}[\log \sigma(r_\phi(x, y^+) - r_\phi(x, y^-))]$

3. 利用强化学习更新语言模型

将语言模型视为策略：$\pi_\theta(y \mid x)$，通过策略优化，使模型生成更高奖励的回答。

典型优化目标可表示为：$\max_\theta \mathbb{E}_{x,y\sim\pi_\theta}[r_\phi(x,y) - \beta D_{\text{KL}}(\pi_\theta(\cdot \mid x) \,/\!/\, \pi_{\text{ref}}(\cdot \mid x))]$

- r_ϕ：奖励模型；π_θ：待优化策略；π_{ref}：参考策略；D_{KL}：KL 散度；β：约束强度。

KL 项用于限制策略相对于参考模型发生过大的变化。

以 LLaMA 2 为例介绍了基于奖励模型和策略梯度的 RLHF 训练思想。

5.5 RLHF 的局限与对抗鲁棒性

完成 RLHF 和 Safety Tuning 并不意味着模型永远不会生成有害输出。主要原因包括：

- 不可能穷尽所有潜在有害输出；
- 训练数据无法覆盖所有异常输入；
- 攻击者可能通过对抗性输入诱导模型违反预期行为。

这一问题属于 Adversarial Robustness（对抗鲁棒性）。

课件还给出了讲授者对 Claude、ChatGPT 和 Llama 安全表现的经验性观察。该观察不构成统一测试条件下的定量排名。因此，模型对齐应被视为持续改进过程，而不是一次训练即可彻底解决的问题。

Part 6 大语言模型推理与提示工程

6.1 Prompting 与 Prompt Engineering

Prompt 是在推理阶段提供给模型的自然语言任务描述。Prompt 可以非常简短，也可以包含任务背景、输入数据、输出格式和具体约束。

Prompt Engineering（提示工程）则是设计、选择和优化 Prompt，以提高模型在特定任务中的表现。

基本原则：Be as specific and descriptive as possible. 尽可能明确、具体地描述任务要求。

提示工程可以采用人工方式，也可以通过自动方法完成，例如 Prompt 改写、Prefix Tuning 等。

6.2 Zero-shot、One-shot 与 Few-shot Prompting

根据提供的任务示例数量，Prompting 可以划分为三种基本形式。

方法	示例数量	主要特点
Zero-shot	0	只提供任务说明
One-shot	1	提供一个参考示例
Few-shot	少量	通过多个示例展示任务模式

1. Zero-shot Prompting

不提供示例，直接要求模型完成任务。

```
Classify the sentiment of this sentence:
"The service was excellent."
```

2. One-shot Prompting

提供一个示例，引导模型理解输入输出格式。

```
Text: "The food was terrible."
Sentiment: Negative

Text: "The service was excellent."
Sentiment:
```

3. Few-shot Prompting

提供多个输入输出示例，使模型根据示例推断任务规则。这属于 In-context Learning（上下文学习）。

重要特点：模型在推理过程中利用上下文示例调整行为，通常不需要更新模型参数。

可以将其表示为： $P(y \mid x, D_{\text{demo}})$，其中，$D_{\text{demo}}$ 表示上下文中提供的示例集合。

6.3 Chain-of-Thought Prompting

Chain-of-Thought（CoT）Prompting 通过引导模型进行分步推理，提高其处理多步骤问题的能力。

其核心思想是将复杂任务分解为若干中间推理步骤，而不是直接生成最终答案。

例如： $ \text{Problem} \rightarrow \text{Intermediate Reasoning} \rightarrow \text{Answer} $

CoT 尤其适用于数学、逻辑和多步骤推理任务。

需要注意，模型生成的推理文本并不必然反映其真实内部计算过程，也不能保证最终答案正确。

6.4 Full Fine-tuning 与 Prompting 方法的选择

四种适配方法：

方法	主要优势	主要限制
Full Fine-tuning	可以获得较强的任务专用性能	需要较多标注数据与训练资源
Few-shot	需要较少任务专用数据	示例选择与任务适配存在挑战
One-shot	接近自然语言示范方式	单个示例可能不足以明确任务
Zero-shot	最方便，无需任务示例	任务可能存在歧义，性能不稳定

Full Fine-tuning 通常需要约 10^3 到 10^5 甚至更多标注样本，但这不是所有微调任务的固定要求。不同方法之间存在性能、数据需求和使用便利性的权衡。

通常而言，增加任务专用训练和监督有助于改善特定任务表现，但也会增加数据准备、训练和维护成本。

6.5 Parameter-Efficient Fine-tuning

Parameter-Efficient Fine-tuning（PEFT，参数高效微调）其目标是在不更新全部模型参数的情况下，使预训练模型适应新任务。主要思路包括：

- 冻结基础语言模型，仅训练部分参数；
- 添加外部 Adapter 模块；
- 使用 Low-Rank Adaptation（LoRA）。

LoRA 的基本原理

对于预训练权重矩阵：$W_0 \in \mathbb{R}^{d \times k}$，LoRA 将权重更新表示为低秩矩阵乘积：$ \Delta W=BA$,

其中： $B\in\mathbb R^{d\times r}, A\in\mathbb R^{r\times k}$ ，且：$r\ll\min(d,k)$

更新后的线性变换为：$h=(W_0+BA)x $，训练时冻结 W_0，只更新 A 和 B。

原始全参数更新需要训练：$N_{\text{full}} = dk$个参数，而 LoRA 只需：$N_{\text{LoRA}} = r(d + k)$

因此参数比例为：$ \frac{N_{\text{LoRA}}}{N_{\text{full}}} = \frac{r(d+k)}{dk} $，当 r 较小时，可以显著减少可训练参数量。

Part 7 LLM 评估、多模态与开放挑战

7.1 Evaluating LLMs：大模型评估

LLM 评估具有较高复杂性，应尽可能在不同数据集和任务上进行测试。

原因：LLM 通常具有多任务能力，单一数据集难以全面反映其性能。

因此，评估时需要考虑：不同任务类型；不同数据分布；模型输出质量；不同应用场景下的可靠性。

7.2 Multimodal LLMs：多模态大语言模型

传统 LLM 主要处理文本，而多模态模型进一步引入图像、音频等信息。两个目标：

1. 使模型能够通过视觉和听觉理解世界；
2. 使模型能够执行 Cross-modal Reasoning and Learning（跨模态推理与学习）。

两条引入多模态能力的路线：

路线	代表模型	主要思想
Pre-training	Gemini	在预训练阶段引入多模态学习
Instruction Tuning	AudioGPT、Flamingo	通过多模态组件或任务适配建立跨模态能力

7.3 Open Challenges：大模型的开放挑战

1. New Capabilities：新能力

主要包括：Multimodal：多模态理解与生成；Multilingual：多语言能力；More Complex Tasks：更复杂任务的处理能力。

2. Performance：性能与可靠性

主要包括：Reduce Hallucinations：减少幻觉；Improve Alignment with Human Preference：改善人类偏好对齐；Increase Context Length Efficiently：高效扩展上下文长度；Improve Data, Training Strategy and Model Architecture：优化数据、训练策略和模型架构。

3. Efficiency：效率

主要关注：计算成本；训练与推理时间；资金与资源消耗；GPU、TPU、HPU 等计算架构。

这些挑战表明，LLM 的研究目标不仅是提高模型能力，还包括降低部署成本、改善可靠性和扩展实际应用范围。

Part 8 总结

本讲的知识逻辑可以归纳为以下六个层次。

知识层次	核心内容	关键结论
Agent 概念	Agent - Environment 交互	智能体通过观察与行动形成闭环
LLM Agent 架构	Planning、Memory、Tool Use、Action	Agent 是完整系统，而非单一模型
智能体历史	Rules → Learning → Generative Agents	决策机制从固定规则转向模型驱动
LLM 技术基础	Transformer、GPT、Scaling	大规模预训练支持多任务泛化
模型训练与适配	Pre-training、SFT、RLHF、Prompting、LoRA	不同方法解决不同层面的能力适配问题
未来发展	Evaluation、Multimodal、Reliability、Efficiency	实际部署需要同时考虑能力、可靠性和成本

概念对	关键区别
LLM vs. LLM Agent	语言生成模型 vs. 能够自主交互和行动的系统
Agent vs. Workflow	动态决策执行路径 vs. 主要遵循预设流程
Short-term vs. Long-term Memory	上下文内信息 vs. 外部持久化信息
Encoder-only vs. Decoder-only	双向表示学习 vs. 自回归生成
Pre-training vs. SFT	学习通用语言能力 vs. 学习指令遵循
SFT vs. RLHF	学习标准回答 vs. 优化偏好与行为
Fine-tuning vs. Prompting	更新模型参数 vs. 通过输入上下文适配任务
Full Fine-tuning vs. LoRA	更新大量或全部参数 vs. 训练低秩增量参数

一条完整的技术演进逻辑：

$$ \text{Rule-based AI} \rightarrow \text{Learning-based AI} \rightarrow \text{Large Language Models} \rightarrow \text{LLM Agents} $$

传统规则系统依赖程序员预先定义行为，机器学习使系统能够从数据中学习决策模式，大语言模型进一步通过大规模预训练获得通用语言能力，而 LLM Agent 则将语言能力与规划、记忆、工具和行动机制结合，使系统能够在环境中持续完成任务。

LLM 提供语言理解与推理基础，Agent 架构赋予模型与环境交互和自主执行任务的能力，而可靠的智能体系统还需要训练、对齐、工具、评估及计算基础设施的共同支持。

Lecture 2 Anatomy of an LLM Agent

Part 1 Agent

The engineering question: what is actually inside?

Every design choice we make is a trade-off, not an upgrade 我们做的每一个设计选择都是权衡，不是升级；

Goal: by the end, you can draw an agent, then critique one.

目标：可以绘制一个 agent 并且评判它

Each part answers one hole — and one property of our task. For every option, we ask two things: what it costs, and what it buys you. There is no "best" column — only a fit for your environment.

每一块回答一个漏洞，也回答任务的一个属性；对每个选项都要问两件事：它花什么代价，它买到什么；没有最优这一列，只有与你的环境的匹配度。

Tips：为什么 Fine-tune 出现在 Memory 里而不是 Reasoning 里?

记忆的差别不在存哪儿，而在存什么。微调被归入记忆，因为它的本质是把知识写进权重，是一种持久化手段，而不是一种推理方式。RAG 把知识放在外部文档里、Files 放在文件系统里、Fine-tune 放在参数里，三者都是记住东西的不同落点。

1.1 Agent 的能力和风险

1. 十万行 C 编译器

Sixteen agents built a 100,000-line C compiler in two weeks—able to compile the Linux kernel. 十六个智能体在两周内造出了一个十万行的 C 编译器，能编译 Linux 内核。

规模：十万行、十万行级别的代码是可以由一群 agent + 一个任务队列推进的。

协调：协作的成本被显式地写进了工作流，主体错位 / 责任推诿 要解决的问题；

可观察：agent 的工作必须留下可被人类读懂的痕迹，否则无法验收、复现。

2. 邮件清理失控

不可逆动作：trash EVERYTHING in inbox older than Feb 15 是一个破坏性批量操作，而它是在没有被要求的情况下自行拟定并执行的。

用户的否决晚且无效：agent 的循环并没有把人类否决当成一个全局中断信号

无法中断：除了杀掉宿主机上的进程，它没有任何层内的停止机制。

3. Flask 应用排障

其动作序列是推理闭环：终端里连续尝试启动应用、报错（应用侧自己识别出来的）、采纳自己的建议，换端口重启、右侧浏览器里出现应用页面（ngrok 暴露的公网地址）。识别了错误、提出了处置方案、执行了方案、然后验证了结果。

Part 2 Agent 的组成

2.1 四个组件与环境

组件	连接关系
Perception System	接收 Input；由 Environment 反向喂入
Reasoning System	接收 Perception 的输出；与 Memory 双向连接
Memory System	与 Reasoning 双向连接（读 + 写）
Execution System	接收 Reasoning 的输出；动作打到 Environment
Environment	接收 Execution 的动作，再把结果送回 Perception

数据是单线流动的： Perception → Reasoning → Execution

Memory 是唯一的双向块：推理既能读记忆也能写记忆，这解释了为什么 p.11 的画法里 Memory 被单独标成 read + write；

闭环靠环境完成：Execution 的动作改变 Environment，Environment 的变化再被 Perception 读到。

2.2 Running Example

TASK: Go through my inbox for this week, find every expense-claim email, put the amounts in a spreadsheet, highlight anything over ¥5000 in red, and email the summary to Finance.

翻一遍我这一周的收件箱，找出所有报销申请邮件，把金额填进一张表格，把超过 ¥5000 的标红，再把汇总邮件发给财务。

任务属性	对应的子系统	难处
Email count unknown up front 邮件数量事先未知	Reasoning（unknown number of steps）	无法预先写死步数，固定流程从第一步就错
One amount is inside an image 有一笔金额在图片里	Perception（the amount in the image）	文本感知看不见它，必须上多模态或工具
It runs again next week 下周还要再跑一次	Memory（it runs again next week）	要记住上次怎么做的，否则每周从零开始
The last step is irreversible 最后一步不可撤销	Execution（sending mail is not undoable）	发邮件不可回滚，误发就是真实的失误

2.3 Agent 必要条件

对照项	Workflow 工作流	Agent 智能体
策略来源	Follows a plan the designer wrote 遵循设计者写好的计划	Generates its own strategy 自己生成策略
执行方式	Fixed sequence of steps 固定的步骤序列	Acts on environmental feedback 依据环境反馈行动
出错时	Cannot re-plan when it hits an error 碰到错误无法重新规划	Recovers from mistakes, re-plans 能从错误中恢复并重新规划

对照项	Workflow 工作流	Agent 智能体
适用场景	Great for predictable, repetitive tasks. 适合可预测、重复的任务	Needed for unpredictable environments. 不可预测的环境才需要

Adding tools to an LLM does not make an agent; it makes a workflow.

给 LLM 加上工具，并不会把它变成 Agent，只会把它变成一条工作流。

调用了工具是能力层面的描述：模型能访问外部世界了。

是否是 Agent 是架构层面的描述：控制流由模型在运行时决定，且它能根据反馈改变接下来的步骤。

2.3.1 五种workflow 模式

模式	定义	控制流由谁决定
Prompt chaining 提示链	Fixed order, one LLM call per step 固定顺序，每步一次调用	设计者（顺序写死）
Routing 路由	Classify, then dispatch to a specialist promptly 先分类，再分派给专门的提示	设计者（分派表写死）
Parallelization 并行化	Sectioning into subtasks, or voting on one 切成子任务并行，或对同一任务投票	设计者（切分方式写死）
Orchestrator - workers 编排者 - 工作者	Central LLM splits work at run time 中心 LLM 在运行时拆分工作	模型（运行时决定）
Evaluator - optimizer 评估者 - 优化者	Generate, score, rewrite 生成、打分、重写	模型（循环次数运行时定）

Orchestrator - workers is the first pattern whose step count is unknown until run time. 编排者 - 工作者是第一个在运行时才确定的模式；

The criterion is not "does it have tools?" — it is who decides the control flow. 判据不是它有没有工具，而是谁决定控制流。

2.3.2 Workflow 优先

Agents cost more, run slower, and fail in more ways. Use one only when you cannot enumerate the steps. Agent 更贵、更慢、失败方式更多；只有当步骤无法枚举时才用它；

Your default assumption is that an agent is the more advanced choice. It is the most expensive one. 你的默认假设是Agent 是更先进的选择，但它其实是更昂贵的那个选择。

Tips：Workflow（主干）+ Agent 混合架构才是常态。

能写出且清单稳定就用 workflow，会得到更低的成本、更短的延迟、更可预测的失败；写不出来（因为数量/分支/环境状态视运行时而定）就在那一个节点上嵌一个 agent 模块。

2.4 The Loop and Exit 闭环与退出条件

1. Exit 的三行判定： Success? Budget exhausted? Loop detected?

任务真的完成了吗？这三条里最难验证的一条；

预算（步数 / token / 时间 / 金钱）花完了吗？

是不是在原地打转？

2. 任务与六步

Task: Reply to the unread email from Accounting.

动作	中文	对应章节
Perceive	看见：截图 + 可访问性树	Part 3 感知
Retrieve	回想：上次我是怎么回这类邮件的?	Part 5 记忆
Reason	分解任务，挑下一步	Part 4 推理
Act	行动：click(126)	Part 6 执行
Observe	观察：屏幕变成了新状态	回到 Part 3（重新感知）
Reflect	反思：刚才那步成了吗？继续还是停	Part 4 推理、反思

Part 3 Perception 感知子系统

3.1 Four Options

方案	定义	代价
Text 文本	The environment describes itself in words. 环境用文字描述自己	Cheapest — blind to anything unwritten. 最便宜，但对没被写下来的东西表达不了
Multimodal 多模态	Pixels via VLM / MM-LLM. Sees everything, including images 通过 VLM / MM-LLM 直接看像素，能看到包括图片在内的一切	costs the most 成本最高
Structured 结构化	Node tree, DOM, JSON rows. Exact semantics, cheap 节点树、DOM、JSON 行；语义精确且便宜	but only what the tree exposes. 但只能拿到树暴露出来的东西
Tool-based 工具化	Call something to find out. 调用某个东西去查明，成本可控	Fidelity depends entirely on the tool you wrote. 保真度完全取决于你自己写的那个工具

维度	A 截图	B 可访问性树	C IMAP（JSON）
成本	~1,100 - 1,600 tokens	~400 tokens	~120 tokens
看到图片附件	能（缩略图）	不能	不能（只有文件名）
拿到精确金额	不能（需 OCR）	能（value: "5000"）	能（字段精确）
看懂视觉标记	能（红旗）	不能	不能（丢颜色标记）
界面重做后	失效	失效（除非树重新标好）	稳定

感知方案的选择会直接决定任务可行或不可行。

3.2 方案一：Text-Based Perception 文本感知

The environment writes text straight into the prompt, no perception module at all, Cheapest option by far

环境把文本直接写进 prompt；根本没有感知模块；是所有方案里最便宜的；

代表环境： ALFWORLD / TEXTWORLD；

适用：Chat, text games, text-driven simulations（对话、文字游戏、文字驱动的模拟）；

局限： Blind to anything not already in words（对任何尚未被写成文字的东西都不知道）。

不可迁移： Much agent research runs in text environments — so its conclusions may not hold on a GUI.

大量 Agent 研究是在文本环境里做的，所以它们的结论未必在图形界面上成立。

优势：把感知问题彻底消掉了，环境写好了文本，可以专心研究推理与控制流，不受感知噪声干扰。文本环境是隔离变量的实验台，代价是外部效度下降。

3.3 方案二：Multimodal Perception 多模态感知

适用：在没有任何 API、只剩一块屏幕的时候。Essential when there is no API — only a screen.

左半段叫 Multimodal Understanding（多模态理解），右半段叫 Multimodal Generation（多模态生成）。感知只用到左半段，视觉信号要经过专用编码器 → 投影层 → 主干三跳才能进入语言空间，而且每一跳都有信息损失与计算开销，所以模型看得见但数不准。

Still weak at precise spatial relationships and accurate counting — without external aid.

在没有外部辅助时，仍弱于精确的空间关系与准确的计数。

Seeing an image is not the same as measuring it.

看见一张图，不等于测量一张图。

3.3.1 Set-of-Mark

Draw a numbered box on every clickable element, then let the model name a number.

给每一个可点击元素画一个带编号的框，然后让模型说出一个编号。

收益：Improves counting（改善计数）、Cuts hallucination（减少幻觉）。

Cheaper than retraining the whole MM-LLM. 比重新训练整个多模态模型便宜。

改数据/加训练（贵、慢、还需可复现）与改输入形式（便宜、即插即用）。

Tips：文本环境是把感知问题消掉，结构化数据是把坐标问题消掉，ACI 是把点击问题消掉。这些都在改任务形式，而不是加大模型。

It converts an open-ended spatial problem into a closed-form, multiple-choice one.

它把一个开放式空间问题，转换成了一个封闭式的选择题。这是工程技巧之一：当模型在某类任务上系统性变弱时，先想办法把任务形式换掉，而不是换一个更大的模型。

3.4 方案三：Structured Data 结构化数据

Names, not coordinates. No pixel guessing. 用名字，不用坐标；不用猜像素。

三种结构化来源：

- Accessibility tree 每一个 UI 元素，带角色与状态（every UI element, with role and state）；
- HTML / DOM 元素描述与截图并排放；

> 结构化感知与截图不是二选一，而是互补
>
> A（截图）能看见红色标记，但算不出精确金额；B（树）有精确金额，但看不见任何图形与颜色。所以最稳的组合是结构化做数值与定位 + 截图做视觉属性判断。虽然组合能提高保真度，但代价是上下文被撑爆。

- 健壮性： Survives a re-skin; breaks on a re-label.

> 换皮（re-skin）：只改配色、字号、圆角，元素的名字与角色没变。所以可访问性树里读到的东西不变，agent 照常工作；
>
> 换标签（re-label）：把按钮文字从 Reply 改成 Respond、或把 textfield 的标签改掉。agent 依赖的锚点消失了，于是它崩掉了。
>
> 实践价值：如果长期维护一个结构化感知的 agent，要把标签稳定性当成接口契约来管，而不是当成实现细节。界面换皮它能活，标签一改它就崩。

来源	适用场景
Playwright accessibility snapshot	Web 自动化场景下的可访问性快照
Chrome DevTools Protocol (CDP)	直接通过浏览器调试协议取 DOM 与可访问性信息
Android UI Automator	Android 端的界面元素枚举

Precise semantics, cheap tokens — but only if the tree exists and is well-labeled.

语义精确、token 便宜，前提是这棵树存在，而且标注良好。

3.5 方案四： Tool-based perception 工具化感知

工具化感知：The agent calls something to find out what it cannot see. Agent 调用某个东西，去查明它自己看不见的事。

四类典型工具：

工具类型	说明	在报销任务里的对应
Web search	Facts past the training cutoff 训练截止之后才发生的事实	查某个供应商是否真实存在
Domain APIs	Weather, markets, literature 天气、行情、文献	对接财务系统的报销接口
OCR / sensors	A wrapper that returns JSON — our receipt 一个返回 JSON 的封装，即收据	读出图片里的那笔金额
Code execution	Parse logs, run statistics 解析日志、跑统计	解析邮件原始报文、做金额汇总

感知与执行共用同一个工具调用机制，只是意图不同，一个是为了获取信息，一个是为了改动信息。

它意味着怎么写工具，对感知层的工具同样适用（返回字段要精简、错误信息要写给模型看），也意味着感知与执行在架构上可以复用同一套基础设施。

四种失效的情况：

失效项	说明	中文
Hallucination	reports what is absent	幻觉：报告出屏幕上根本不存在的东西（最严重）
Latency	vision + calls add up	延迟：视觉处理与工具调用的耗时是累加的（时序错导致状态漂移）
Context blowup	screenshot + DOM	上下文爆炸：截图加 DOM 一起塞进上下文
Cost	fidelity is not free	成本：保真度不是免费的

解决思路：先修会静默出错的，再修慢的，最后优化贵的。因为静默错误会被后续所有步骤继承，而后两者至少是可观测的。

Part 4 Reasoning 推理子系统

4.1 推理的职责

职责	课件定义	中文
Decompose	Break a big task into steps you can actually execute	分解：把大任务拆成你真的能执行的步骤
Choose	Generate several candidate plans, then pick one	选择：生成若干候选计划，然后挑一个
Reflect	Judge what happened and revise the plan	反思：判断发生了什么，并修订计划

靠提示词：能力存在于输入里。每次调用都要把请一步步写进上下文，占 token，而且不稳定；

靠训练：能力存在于权重里。推理过程被内化进了模型，不再需要外部提示去激活它。

4.2 CoT 与分解的时机

1. 基础对比：标准提示词 vs CoT 提示词

   标准 Prompt：直接输出答案，复杂算术题容易直接答错。

   CoT（Chain-of-Thought，思维链）：让模型分步写出思考过程再输出答案，显著提升推理正确率。

2. CoT 真正的贡献：推理精度可以靠推理时的计算量换取，而不是只靠模型参数量、训练数据集。

这就是缩放的第二根坐标轴。

> 轴一（训练时缩放）：把参数变大、把训练数据变多。这是 2020 年前后的主线；
>
> 轴二（推理时缩放）：让模型在回答前多花算力（多想几步、多采样几条路径、多搜索几棵分支）。这是 CoT 打开的；
>
> 技巧会过时，轴不会，全都建立在推理时算力可以换准确率这个前提上。

3. 两层视角看待 CoT

   作为提示词技巧：如今已经过时，现代推理大模型内部原生就会做分步思考，不需要人工写这个 prompt。

   作为思想范式：衍生出整套 “思考模型” 的研究方向。

4. 任务拆解的两种策略：先全部拆解 / 边做边拆

   Decomposition first 前置拆解（Plan-and-Solve、ReWOO）：任务开始前一次性把完整计划全部定好。
   缺陷：遇到未知数量子任务（例如不确定邮件数量、附件），计划第一步就出错，后续全部连锁失败。

Interleaved 交错式拆解（CoT、ReAct）：一次只展开 1~2 个步骤，边执行边规划。适合子任务数量未知的场景，无法预先固定总步数。

策略	两种做法
(a) Sequential Decomposition-Planning 全部先规划，再逐步执行	Decomposition first —— Plan everything up front; 代表：Plan-and-Solve, ReWOO
(b) Interleaved Decomposition-Planning 交错：规划一步、执行一步	Interleaved —— Reveal one or two steps at a time; 代表：CoT, ReAct
(c) DPPM 动态的、逐步演化的规划	规划本身随执行展开

Tips：当任务元素的数量与形态在运行前可知时，先规划是更高效的（一次想清楚，减少往返）；当元素数量未知时，交错式（CoT / ReAct）才成立。先规划 = 把分解挪到设计期，交错 = 把分解留在运行时。

4.3 推理结构

4.3.1 链式、树式、图式推理（Chain / Tree / Graph）

1. IO Prompt 输入输出：输入直接到输出，没有中间思考。
2. CoT 思维链：单条线性思考路径，从头到尾一条思路。
3. CoT-SC（Self-Consistency 自一致性）：采样多条独立 CoT 路径，多数投票选出最终答案，消除采样噪声。仅适用于离散答案。
4. ToT Tree of Thoughts 思维树：每一步生成多个候选思路，用 BFS/DFS 搜索思路分支；24 点游戏中，CoT 只有 4% 成功率，ToT 可达 74%，因为 24 点拥有完美算术校验器。
5. GoT Graph of Thoughts 思维图：思路可以合并、互相转换，不止树形分支。

全称	课件定义	结构特征
Chain of Thought	One line of thought, start to finish 一条思路，从头走到尾	线：单一路径，不能回溯
Tree of Thought	Several thoughts per step, searched by BFS or DFS 每一步生成若干想法，用 BFS 或 DFS 搜索	树：可分叉、可回溯，但分支之间不能合并
Graph of Thought	Thoughts can merge and be transformed 想法可以合并与变换	图：允许汇聚与改写，结构最灵活

Tips:

从 CoT 到 ToT 到 GoT，每一步都在增加结构自由度：线不能回头，树可以回头但不能合并，图既可以合并也可以变换。

但是自由度增加带来两个直接后果：需要搜索与评估的机制和计算开销上升，所以要根据情况选择合适的推理结构。

ToT 的优势依赖于存在可靠评估器（下一节）这个前提，而不是树形结构本身更聪明。

4.3.2 评估器

CoT-SC：多数投票，零额外代价，仅离散答案可用。

ToT / GoT：大模型自行打分评估每一步分支；开销大，而且模型评判本身会有噪声。

LLM-MCTS/RAP：需要模拟推演到终点，或者价值评估模型。

核心结论：所有搜索类推理方法都必须配备评估器 evaluator。劣质的评估器反而会让搜索结果更差，因为会自信地选错分支。分支带来的计算开销是线性增长，评估器的可靠性才是瓶颈。

三种评估器按可靠性排序

算术 / 单元测试 / 环境反馈的特点是可靠，代价是只适用于特定任务；

多数投票（CoT-SC） 的特点是免费，但要求答案是离散的，且多数不等于正确；

让同一个模型给自己的步骤打分的特点是最通用但最不可靠（同一个模型的两个错误容易同源）。

4.4 ReAct：工作的循环

Same question, four strategies. Watch what Act-only lacks, and where CoT hallucinates.

同一个问题，四种策略；看只有行动的那一版缺了什么，以及 CoT 在哪里产生幻觉。

Action space = environment actions + language 动作空间 = 环境动作 + 语言（语言本身也是一个动作）；

Thoughts are free — they change nothing outside the model. 思考是免费的， 它们不改变模型之外的任何东西；因为它意味着 Thought 是唯一可以在不产生副作用的前提下试错的环节。这正是 ReAct 相对于直接行动的安全边界：能想清楚的事，就不要用动作去试。

example：邮件事故中的 agent 直接执行了 trash EVERYTHING，中间没有任何 Thought 环节去问这个动作可逆吗。把 Thought 加回来，就有了一个能承载风险判断的位置。

Observations are the only ground truth 观察是唯一的真实来源。

影响：推理与行动不能分家。不用再在先想清楚再动手和边做边想之间二选一，ReAct 证明了两者必须在同一个循环里交替。

4.4.1 一个 while 循环

In engineering terms, an agent is this while loop.

用工程语言说，一个 Agent 就是这个 while 循环。Agent 不是一种新架构或者新模型，它是一个模型 + 工具字典 + 显式的停止条件的循环。所以多数 agent 框架的差别，都是在这三样东西上加约束或加调度。

性质	对应代码
Stop conditions must be written explicitly. 三个停止条件必须被显式写出来	range(MAX_STEPS) / out.done / detect_loop(...)
Messages grows monotonically. 消息列表是单调增长的。它保证了模型能看到完整历史（可归因），但也意味着上下文只会越来越长	messages += [out.raw, f"Observation: {obs}"]
The model never touches the system. 模型从不直接触碰系统。一个安全边界：模型只产出要调用哪个工具、参数是什么，真正的执行发生在你自己的代码里。	真正的系统调用发生在 TOOLS[out.tool](**out.args)
TOOLS is the action space. TOOLS 就是动作空间，拦截危险	动作集合 = 这个字典的键集合

4.5 Reflection

4.5.1 四个动作 Four Moves

Reflection: Learning from mistakes without touching weights. 不碰权重地从不犯错中的学习。

Reflection is verbal reinforcement learning: the feedback is written in language and put back into the context — no gradient step.

反思的机制：语言上的强化学习，反馈被写成语言，再放回上下文里，没有梯度更新。

动作	回答的问题
Self-evaluation 自我评估	Compare what happened to what was expected 把实际发生的和期望的比一比
Error analysis 错误归因	Bad prompt? wrong tool? logic slip? world changed? 是提示写坏了？工具选错了？逻辑漏了？还是环境变了？
Correction 纠正	Revise the plan, the tool use, or the memory 修订计划、工具用法或记忆
Goal-driven 目标驱动	Reflect on efficiency too, not just on failure 不只在失败时反思，也要反思效率。把反思从纠错升级成优化

4.5.2 Reflection Loop 反思循环

循环的组件与功能：

Actor：Generates thoughts and actions from state + memory

行动者：依据状态与记忆生成思考与动作

Evaluator：Scores the trajectory: —the exact match, heuristic, or an LLM

评估者：给整条轨迹打分，可以是精确匹配、启发式规则，或者另一个 LLM

Self-reflection model：Turns a pass/fail signal into specific written advice

自反思模型：把一个通过/失败的信号，转成具体的书面建议

Tips：

1. Reflections enter working memory — up to three;

反思不是越多越好：反思内容本身也要占上下文，而上下文是有限的会进入工作记忆，最多累积三条，所以反思必须写得精炼、可行动；

2. A heuristic stops runs that loop or hallucinate

一个启发式规则会中止那些打转或产生幻觉的运行；

3. Stuck: A loop repeating the same action with the same result is the definition of stuck.

同一个动作在不同环境下可能是合理的，但如果动作和结果都一样，那就一定是在原地打转。

4.5.3 Feedback and Self-correct

With no external signal, asking a model to review its own reasoning lowers accuracy. It has no reliable way to tell "I was wrong" from "I was right" — so it changes correct answers too.

在没有外部信号的情况下，让模型复查自己的推理，会降低准确率。它没有可靠的办法区分我刚才是错的或对的，所以它会把正确的答案也改掉。验证器是反思的前提，要执行反思，先要能验证。没有验证手段时，宁可不要反思环节。

Reflection works when there is a judge（有裁判时才有效）	And fails when there is not（没有裁判时失效）
✓ HumanEval — unit tests run（单元测试会真的跑起来）	✗ Pure math reasoning — no referee（纯数学推理，没有裁判）
✓ ALFWorld — the environment replies（环境会回话）	✗ Open-ended writing（开放式写作）
✓ Game of 24 — arithmetic checks（算术可以检查）	✗ "Does this look right to you?"（「你觉得这个对吗？」）

三档反思：

最强：可执行的检查 unit tests、算术、编译、schema 校验。它们给出的是客观通过与失败；

中等：环境回复 ALFWorld 类的交互环境会如实告诉你动作成功没有；

最弱且常被误用：让模型自己说你觉得这样对吗。

4.5.4 事前反思 Reflect Before

This is why anticipatory reflection matters: it asks the model to imagine a failure, not to judge one. It therefore sidesteps the previous slide's problem entirely — no referee required.

事前反思的意义在于：它要求模型想象一个失败，而不是判断一个失败。因此它完全绕过了上一页的难题，不需要裁判。判断需要一个外部标准，而想象不需要。模型不需要知道我刚才做对了没有，只需要知道有哪几种做法可能出问题。

Example：事前方式：按 RePrompting 的形式，在发邮件给财务这一步之前，反思表格是否已经写完并核对过？unmet → 先回去补表格，不要发信；按 Devil's advocate 的形式，如果这封汇总邮件发错了，会是怎么错的？可能的答案：漏了几封、金额读错、发错收件人。

不可逆的动作最值得在事前做一次失败想象，事后修复的成本是无穷大。

两种反思（事前与事后）不可替换：事前反思负责减少失败（省下那一步），事后反思负责从失败中学习（改进后续尝试）；事前不需要裁判，事后需要。

4.6 六种推理策略

Option	What it buys	What it costs	Needs a verifier?
Single call	Cheapest, lowest latency	No recovery at all	No 它们不搜索、不修订，没有选择这一步
CoT	Accuracy for compute	More tokens per answer	
CoT-SC	Cancels sampling noise	n× the calls	Discrete answers only 只在答案离散时可用多数投票代替裁判
ToT / GoT	Explores wide spaces	Branch cost + scoring cost	Yes — critical 坏评估器让搜索比不搜索更差
ReAct	Grounds each step in	Context grows every turn	The environment is one 环境即是评估器，观察是唯一的真实来源
Reflexion	Improves across attempts	Wasted failing runs	Yes — or it degrades 没有反馈时自纠正会降低准确率

Part 5 Meomory 记忆子系统

5.1 借用人类的记忆地图

人类三级记忆映射到 LLM 系统的三个部件：

人类记忆层级与LLM的映射	说明
Sensory→ embeddings	Raw input turned into vectors 原始输入被转成向量
Short-term→ context window	Finite — about 4 chunks in humans 人类的短时记忆容量极小但会主动遗忘，而上下文窗口容量更大且不主动遗忘
Long-term→ external store	Vector database, queried at retrieval time 外部存储：向量数据库，在检索时被查询

5.2 RAG 的两个阶段

5.2.1 RAG 的两个阶段

检索增强生成的两段流水线：Query → Retrieve → Augment。

Retrieve：vector search over docs（在文档上做向量检索），

Augment：docs + query → LLM（文档与查询一起交给 LLM）。

What you gain	What you pay
Reaches data the model never trained on 能触达模型从未训练过的数据	Retrieval quality caps everything after it 检索质量封死了它之后的一切上限。后面的生成模型再强，也无法弥补前面检索漏掉的内容。如果正确的文档没被检索出来，模型没有任何机会用它；而如果检索出来的是无关文档，模型还会被带偏。
Answers become traceable to a source 答案可以追溯到来源	An indexing pipeline to maintain 多了一条需要维护的索引流水线。反思需要裁判。在 RAG 系统里，答案里的每句话能不能在检索到的文档里找到依据本身就是一个可用的裁判。
Cuts hallucination on internal facts 减少内部事实上的幻觉	Retrieved text competes for context space 检索回来的文本会和别的内容抢上下文空间

5.2.2 排序策略

Filling the context is not the same as using the context. "Bigger context window" is not a memory strategy. This is the bridge to context engineering.

把上下文填满，和把上下文用起来，是两件事，给的信息越多 ≠ 答案越好；更大的上下文窗口不是一种记忆策略，是为了上下文工程。

排序策略

既然中间位置会掉分，那么检索结果的排序顺序本身就成了一种要设计的策略。

把最可能相关的放两端，把最不确定的放中间，注意，这与按相似度从高到低排是不同的策略。

人类的短时记忆靠遗忘来腾空间，而上下文窗口靠位置来分配注意力。

5.2.2.1 CoALA 分类法

Classify by content type, not by storage location — Retrieval strategy follows content.

按内容类型分类，而不是按存储位置分类；检索策略要跟着内容走，而四类内容的存取方式不同：

1. Working Memory 工作记忆

   含义：当前这一轮的上下文，Agent 当下正在处理的信息，对应 LLM 上下文窗口

   例子：this run's 7 emails（本次任务里的 7 封邮件）

2. Episodic Memory 情景记忆

   含义：过去特定的经历、事件轨迹，类似个人日记，记录 “发生了什么”，带时间 / 场景

   例子：last week Sato's amount was in a PDF（上周 Sato 的数据在一份 PDF 里）

3. Semantic Memory 语义记忆

   含义：关于世界的客观事实、静态知识，知识库 / RAG 库，抽象事实，不绑定具体场景

   例子：Finance is finance@...（财务相关的定义 / 客观信息）

4. Procedural Memory 程序记忆

   含义：怎么做一件事，流程、方法、技能，做事的步骤、代码 / 操作模板

   例子：build the sheet before filling it（先建表格，再填数据）

Agent 整体架构

三大长时记忆：Procedural / Semantic / Episodic，都支持 Retrieval（检索读到工作记忆） + Learning（学习写入保存）

Working Memory（工作记忆）是中央工作台：所有检索出来的信息在这里做推理、反思（Reasoning）

Decision Procedure（决策模块）：LLM 解析 Prompt、检索记忆，输出 Action；Agent 通过动作和三类环境交互：Dialogue 对话、Physical 物理世界、Digital 数字环境

CoALA 核心亮点

很多早期 Agent 架构是按数据库位置区分记忆（比如：向量库 A、文件存储 B）； CoALA 提出：按信息本身是什么类型来分类。同一个存储数据库里，可以同时存放 episodic、semantic 内容；检索策略要适配内容类型，而不是适配存储介质。

CoALA 是一套LLM 智能体的认知架构分类框架，把 Agent 记忆拆成工作记忆 + 情景记忆 + 语义记忆 + 程序记忆，并且定义了 Agent 观察 - 规划 - 评估 - 选择 - 执行的决策循环，用来统一梳理、设计、对比各种 LLM Agent。

5.2.2.2 四种记忆方式

工程实现的四种落地方式：

方式	定义	中文
Embodied	Baked into the weights by fine-tuning. Expensive; often impossible on closed models.	写进权重：靠微调写进参数。 昂贵；在闭源模型上常常做不到
RAG	Retrieve documents, then answer them. Citable — the usual default.	检索文档，再据此回答。 可引用——通常的默认选择
SQL database	Structured rows via text-to-SQL. Wins when the answer is a number on a table.	结构化行：通过 text-to-SQL 查询。 当答案就是表里的一个数时，这种方式胜出

方式	定义	中文
Files 主力	Let the agent write notes into the filesystem and read them back next session.	文件：让 agent 把笔记写进文件系统，下次会话再读回来

适用情况：

答案是一个数、且数据在表里 → SQL database；

答案是一段可引用的陈述，且语料庞大 → RAG；

要跨会话累积经验、且希望人类能读懂 → Files；

要让能力成为模型的本能、且愿意付出训练成本 → Embodied（微调）。

Tips：Files是记忆的主力，因为RAG 需要嵌入模型、向量库、分块策略、重建索引，每一层都是一个可能失效也可能需要调优的环节；而文件系统只需要 write 和 read；并且有这三个优点：

- No indexing pipeline to maintain 不需要维护索引流水线；
- Naturally auditable — you can read it 天然可审计，你可以直接读它；
- The agent organises its own structure 结构由 agent 自己组织。

5.3 存储什么信息

核心原则：不要无脑保存原始日志，要提取可复用的抽象知识

WORTH KEEPING（值得保存），对应 CoALA 四类记忆：

1. semantic 语义记忆：客观规则、事实
2. semantic 语义记忆：静态事实信息
3. procedural 程序记忆：做事流程、操作方法
4. episodic 情景记忆：特定事件、过往经历

NOT WORTH KEEPING（不值得保存）

例子：click(412,286) 这种底层原始动作坐标

问题：环境 / 页面布局一变，这条记录直接失效；Agent 还会自信地继续使用，造成错误误导后续任务。

抽象层级决定可迁移性

AGENT WORKFLOW MEMORY 智能体工作流记忆

> 从大量具体任务轨迹（trajectories）里提炼出可复用的工作流程，而不是直接存储整条原始轨迹 记忆的价值取决于抽象程度：你存的内容越具体，就越难迁移到新场景。

5.4 记忆实效

Context window 上下文窗口限制

问题：LLM 一次性能装载的内容有硬性上限（hard ceiling）

常规解法：截断文本（truncate）或者做摘要（summarize）

缺陷：摘要会丢失后续可能需要的细节

Duplication 重复冗余

问题：大量几乎一模一样的记忆记录不断堆积

解决方案： 把多次相似任务浓缩合并成一套通用 plan（计划）或者只保存计数，而不是重复保存完整副本

Part 6 Execute 执行子系统

6.1 为 Agent 设计工具的四条规则

给 Agent 设计工具，是项目里收益最高的事情；

The model understands your tool only through its names and its error text. Both are prompt engineering, whether you meant them to be or not.

模型只能通过工具的名字和它的错误文本来理解你的工具。这两样东西都是提示工程。

4 条设计准则

1. Split by task, not by endpoint 按任务拆分工具，而不是按 API 端点拆分。把列举 + 过滤合在一起，代价是每次多一次模型往返，收益是模型不必自己写过滤逻辑，而筛选逻辑正是最容易出错的地方。
   - ✅ find_expense_emails(week)（面向任务）
   - ❌ list_emails() + filter yourself（底层原始接口，让模型自己过滤，容易出错）
2. Optimise returns for tokens 返回结果要做 token 优化。返回越冗长，关键信息越容易被淹没。
   - ✅ 返回精简字段：{id, subject, amount}
   - ❌ 返回完整原始 MIME 邮件正文（token 开销巨大）
3. Write errors for the model 为模型编写可读错误信息。把错误处理从模型自己推理变成了接口直接告知。
   - ✅ sheet is read-only, call unlock_sheet first（自然语言，告诉 Agent 该怎么做）
   - ❌ HTTP 403（机器码，LLM 很难理解含义）
4. Name parameters self-evidently 参数命名自解释。这在有多个可选工具时，用参数名区分工具。
   - ✅ recipient_email
   - ❌ arg2（无意义变量名）

第三点的核心结论：能自建接口，就不要让 Agent 去点击像素坐标 。

例子 SWE-agent：直接操作像素模拟点击（human tools）成功率≈3%；专门构建的 Agent 友好接口 ACI 成功率 12.5% 同模型，仅仅更换接口，性能大幅提升 推荐给 Agent 的环境组件：带行号文件查看器、限制窗口大小、每次编辑后自动运行 linter 并返回错误。

它与 Set-of-Mark 是同一种思路的两个应用场景：SoM 把空间定位变成报编号，ACI 把编辑代码变成按行编辑 + 自动检查；

它与 规则一（按任务切分工具）也是同一个方向：把模型不擅长的低层操作，封装成模型擅长的形式。

6.2 MCP: Model Context Protocol 模型上下文协议

标准化的 Agent 工具通信底层协议（A client - server protocol），Client-Server 架构

问题：M 个 Agent，N 个工具，需要 **M×N** 套独立适配适配器

解决：Agent 实现一次 MCP，工具实现一次 MCP，直接互通，只需要 M+N 份适配代码

MCP 对外暴露三类东西：

1. tools：可调用工具
2. resources：可读资源
3. prompts：可复用提示模板

Tool discovery and invocation become one interface, not per-framework glue.

作用：工具发现、工具调用统一成一套接口，不用为每个框架单独写 glue 代码。一个协议同时覆盖了感知、执行、推理三个子系统的接口需求。

暴露的东西	说明
工具 tools callable	可调用的工具（四条规则）
资源 resources readable	可读取的资源（感知与执行共用同一套机制）
提示 prompts reusable templates	可复用的提示模板（推理策略，提示形态的封装）

6.3 Execution Breaks

三个根源问题

1. Latency：Vision and physical action run on different clocks

延迟：视觉感知和物理动作的时钟不同步

2. Error propagation：Failure can start in perception, planning or execution

错误传播：错误可以起源于感知、规划或者执行任意一环，逐级放大

3. State sync：The agent's belief must match the real screenAgent

状态同步：内心相信的状态，必须和真实屏幕 / 环境状态保持一致

经典 debug 难题：Agent 点错按钮外部现象完全一样，但是根源分 3 种：看错屏幕 / 规划错步骤 / 正确规划但执行出错 。所以必须保存轨迹日志 trajectory logs 记录每一步：观察 observation、思考 thought、动作 action、结果 result 没有日志，无法区分错误类型，不能定位 bug

Part 7 Failure and Evaluation 失败与评估

7.1 Classical Wrong

四大子系统及检验的典型故障

Subsystem	Typical failure	Task 例子
Perception 感知	GUI 定位识别，读错内容	把 ¥5.000 识别成 ¥5000
Reasoning 推理	级联规划、死循环	计划处理 5 封邮件，实际找到 7 封，计划崩了
Memory 记忆	过时信息、检索污染	调用旧的财务邮箱地址发邮件
Execution 执行	不可逆操作，状态漂移	发送不完整的摘要

Subsystem	Typical failure	Task 例子
Verification 校验	没人检查结果	Agent 说 “完成”，漏掉 2 封邮件

7.2 Benchmark Half-Life 基准测试的半衰期（OSWorld）

OSWorld：369 个真实电脑操作任务

- 2024 原始论文：12.2%
- 2025 中期：≈42.9%
- 2026 OSWorld-Verified：≈85%，超过人类基线

思考题：

1. 模型真的学会操作电脑了？还是 benchmark 被刷饱和了？

OSWorld-Verified 修复了评估脚本，基线本身变了，数字不能直接横向对比

2. 成功率是唯一指标吗？

不是。一个正确率 85% 但要花两三倍时间的系统，在很多场景里依然是不可用的。比如，顶尖 agent 的步数是人类轨迹的 1.4 - 2.7 倍。做对了，但慢到无法部署。Goal-driven（Reflect on efficiency too, not just on failure）效率是与正确性并列的一等指标。

3. 排行榜数字为什么不一致？

它说明在 agent 评测里，模型不是唯一的变量。脚手架、最大步骤限制、重试次数不同，结果差异很大

7.3 MAST: 14 Ways a Multi-Agent System Fails 多智能体 14 类故障

分析 7 个多智能体框架真实失败轨迹，故障分为三大类：

1. System Design Issues 系统设计问题（41.8%）
2. Inter-Agent Misalignment Agent 之间对齐问题（36.9%）
3. Task Verification 任务校验问题（21.3%）

7.4 Prompt Injection 提示注入

案例：邮件正文里面嵌入系统指令

核心问题：Agent 无法区分用户指令 和 数据文本里面的指令；两者会进入同一个上下文窗口。这不是简单 bug，是架构本身结构性缺陷。

触发条件（三者同时满足就危险）

1. 读取不可信外部内容
2. 有权限访问敏感数据
3. 可以对外通信

缓解方案（**没有完美方案**）

- 标记不可信内容，降低它的权重

- 不可逆操作必须人工确认
- 动作空间限制，邮件只能发给白名单邮箱
- 把读取不可信内容的逻辑隔离在低权限子 Agent 里面

Lecture 3 Reasoning in LLMs (1)

Part 0

问题：怎么在不改模型权重的前提下，让它把答案算对？

答案：把一次性输出改成先花算力想，再输出，想的思维链（chain-of-thought）既是草稿纸，也是算力的计量单位。但是代价是想得越多，token 烧得越多，用算力换准确率。

方法一：写得更长

让模型生成更长的思维链。依次给例子（few-shot CoT）、给指令（zero-shot CoT）、让模型自己造例子（analogical prompting）、让模型自己改提示词（prompt optimization）、把问题拆成小问（least-to-most）、让模型自己组推理结构（Self-Discover）。

方法二：挑得更好

既然一次生成可能错，那就生成很多条，然后挑。依次是投票（self-consistency）、按执行结果聚类（AlphaCode）、让模型自己投票（universal self-consistency）、训练一个打分器（ORM / PRM）、把打分器插进搜索过程（Tree-of-Thought）。

Core idea:

1. trigger the LLM to generate long chain-of-thought (CoT)

触发模型写长思维链。

2. variable computation of the thought process adapting to tasks of different difficulty levels

思维链的意义是让计算量随题目难度变化：难题多写几步，简单题少写几步。

3. The best practice to interact with LLMs evolves over time

鼓励长思考 + 支持任务需要的推理策略。

4. When LLM self-evaluation works well: search in the partial solution space can help

当模型能可靠自评时，在半成品上搜索比生成完整答案再挑更省、也更准。

Part I Introduction of Reasoning

1.1 Reasoning Models

The trick didn't make the model bigger, it was just that model thought longer

这次突破不是把模型做大，而是让它想得更久。后面所有技术（CoT、自洽性、树搜索）都在同一个方向上发力，即延长或拓宽想的那一段，而不是换一个更大的模型。

Cost: Not Free

O3 achieved 87.5% accuracy on ARC-AGI, with >$1k test-time cost to solve each task. Other existing LLMs achieved <25% accuracy if no special inference-time techniques were used.

o3 在 ARC-AGI 上达到 87.5% 的准确率，但每解一题要花超过 1,000 美元的测试时算力。 不用任何推理时间技巧的话，其他模型都在 25% 以下。图中 o1 low 正好压在 25% 这条线上，KAGGLE SOTA 也在这附近。

OpenAI o1

The visible answer is only the tip of the whole picture of the iceberg.

能看到的回答只是冰山一角，大部分工作发生在你永远看不到的 token 里。o1 那一代不给原始推理轨迹，所以不能拿它做调试目标，只能拿最终的答案对错来评估。所以需要 verifier 这类外部打分器来替代读模型的思考。

Gemini 2.0

Gemini 2.0 Flash Thinking demo: generating long thought before the solution

先长思考、再短答案，把这个确认为产品层面的标准形态，而不是本模型独家做法。这一代模型（以及 Claude、Gemini 的后续版本）允许你看到完整内部思考，o1 则不给权限。

1.2 Four Approaches to trigger CoT

Core idea: trigger the LLM to generate long chain-of-thought (CoT)

Approaches to trigger CoT generation:

1. Few-shot CoT prompting 少样本思维链提示：纯推理时间手段，给几个带推理过程的例子即可
2. Instruction prompting 指令提示：纯推理时间手段，一句话触发（如 Let's think step by step）
3. Instruction tuning 指令微调：要改模型权重，属于训练侧
4. Reinforcement learning 强化学习：要改模型权重，属于训练侧

Tips:

前两条不动权重：模型拿到更长的上下文，自己写完再答。这就是 inference-time techniques。

后两条要动权重：把先推理再回答这个模式直接写在参数里，不需要人在提示里触发。

本质都在回答同一个问题：scaling token budget（放大 token 预算），即怎么把花在思考上的 token 变多、并且花得值。

Part 2 Basic Prompting Techniques

2.1 从标准提示到思维链 From Standard Prompting to CoT

2.1.1 Background: Standard Prompting

标准少样本提示（standard prompting）的局限：standard few-shot exemplars only provide information on the final solution format, but not the rationale to derive the solution. 只教了格式，没教理由。

环节	课件原文
题目	Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
Exemplar	A: The answer is 11.

环节	课件原文
真实待解题目	Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
Model Output（模型输出）	A: The answer is 27. （答错）

Before the advancement in post-training techniques, standard prompting performance is poor on reasoning benchmarks. 在后训练技术尚未发展的阶段，标准提示被定义为表现差。

2.1.2 Development：CoT 思维链（把思考写进示例）

两层机制：CoT本质是一种提示写法

1. The model can imitate the written procedure, not just from the format。

模仿。模型现在有一个可模仿的程序，而不只是格式。

2. Each step gets to build on the last one instead of everything happening, just a single pass

把中间结果写进上下文。每一步的中间结果都变成下一句的输入，误差不会在一次前向传播里累积。

2.2 思维链的规模效应 CoT Scales

CoT performance improves more significantly with the increase in model size. Better models benefit more with CoT generation. A drastic improvement in reasoning performance when the model reaches a certain scale.

模型越大，思维链带来的提升越显著。更好的模型从思维链中获益更多。规模跨过某个门槛时，推理表现出现陡增。

Note: these experiments used pretrained-only LLMs. Recent post-trained LLMs might have different scaling curves, but the main conclusions still hold.

这些实验用的是仅预训练的模型，近期的后训练模型曲线可能不同，例如，仅预训练的 base 模型和后训练过的 instruct 模型，同一条提示下的表现差别很大。但更强的模型，多花思考时间更划算，这个总的结论不变。

Tips: 鼓励模型写思维链的前提：要有足够的推理能力。

2.3 零样本思维链 Zero-shot CoT

核心思路：直接在问题后面加一句提示：Let's think step by step，不需要提前给示例（exemplars），就能让大模型自动生成思维链推理过程。

对比四种范式：

1. Few-shot：给少量问答示例，直接输出答案，没有分步推理
2. Few-shot-CoT：给少量带分步推理的示例，再提问
3. Zero-shot：不给示例，直接提问，模型直接输出答案
4. Zero-shot-CoT：不给示例，只加一句 “一步步思考”，诱导模型自己做分步推理

结论：Let's think step by step 可以无示例触发 CoT 推理。

Zero-shot-CoT 在绝大多数任务上显著优于普通 zero-shot：

- 算术任务：SingleEq、AddSub、MultiArith、GSM8K、AQUA、SVAMP 全部上涨
- 常识推理：CommonSenseQA 略降，但 StrategyQA 大幅提升
- 其他推理：Date Understanding、Shuffled Objects、Last Letter、Coin Flip 都明显提升

短板：

Zero-shot-CoT 效果依然弱于 Few-shot-CoT

- Zero-Shot-CoT：78.7 / 40.7
- Few-Shot-CoT(8 samples)：93.0 / 48.7

问题：如何提升 CoT 性能，又不用人工标注推理示例?

人工写 CoT 示例成本很高，希望免除人工标注。

实验结果

The reasoning ability was already sitting on the model, and that prompt was just unlocking it.

推理能力本来就在模型里，提示词只是把已有的能力解锁出来。

2.4 Analogical Prompting 类比提示

2.4.1 Core Idea

Analogical prompting: instruct the LLM to generate exemplars. Prompt the LLM to **first recall relevant exemplars**, before solving the test problem.

让大模型自己先生成相关例题（exemplars），再去解决当前问题

1. 指令：先回忆、生成和当前题目**相关的例题**（Relevant exemplars）
2. 再使用生成的例题，求解原始问题

灵感来自人类类比推理（Polya《How to Solve It》）： 人遇到新问题时，不会每次都有人给示范；而是主动回忆过去相似的解题经验，借助旧问题的思路解决新问题。

2.4.2 Advantage

Exemplars are **self-generated by LLMs, **no manual labeling.

1. 例题由模型自己生成，不需要人工标注，节省标注成本

Exemplars are **tailored **to individual problems.

2. 生成的例题针对当前题目量身定制（tailored），匹配度更高

示例

题目：求四个顶点(-2,2), (2,-2), (-2,-6), (-6,-2)构成正方形的面积

1. 模型先自己生成相关例题：边长为 5 的正方形，面积 = 5^2=25
2. 再套用这个解题思路，计算原题：求边长→计算面积，得到答案 32

2.4.3 整体逻辑链

1. 普通 Zero-shot：直接答题，推理弱

2. Zero-shot-CoT：加 Let's think step by step，不用样例，提升推理，但上限不如 Few-shot-CoT

3. Few-shot-CoT 效果好，但需要人工手写推理样例，成本高

Issue：zero-shot CoT performance is still worse than a few-shot CoT. How to improve CoT performance w/o manually labeling exemplars?

Answer：Analogical Prompting：让模型自主生成类比样例，模拟人类回忆相似题的思维，不用人工标注，去逼近 Few-shot-CoT 的效果

tips：为什么少样本更强：手工写的示例携带了额外信息。 they will carry extra information and they will show the model what good reasoning looks like，示例不只是格式，它示范了好的推理长什么样。

为什么这不令人满意：手工标注示例很贵。要覆盖各类题型，就得写很多例子。

Humans are not explicitly given demonstrations every time for a new task, Instead, humans intrinsically recall from past relevant experience 人每接一个新任务，并不会有人先给你演示一遍，人会从过去的相关经验里主动回忆。

2.5 Example：竞赛编程实例

1. 提示词：先讲算法再写教程

Besides exemplars, the LLM can also self-generate high-level knowledge. The generated knowledge complements the problems with broader insights

除了示例，模型还能自己生成高层知识，生成的知识用更宽的视野补足具体题目。它是一个已经解完的题，能直接照着做，但换一道题可能就不适用。

知识抽象：它是前缀积算法这类可迁移的方法论，适用范围宽，但单看它不会写代码。所以提示词要求两者都给，在两个抽象层级上同时覆盖。

2. 模型输出：算法知识 + 三道例题

Those examples are very concrete and specific, and knowledge is abstract and knowledge can be transferred better. It gives you the better result and coverage at two levels

Algorithms（识别算法）→ Tutorial（讲清算法）→ Example Problems（造三道相关题并给出代码）→ 最后才解原题。

自造知识：完整的算法描述 + 复杂度分析（O(n)）+ 可运行代码。

3. 最终作答

Example: Codeforces (solving the initial problem)

2.6 类比提示的结果

1. 结果总览：全面占优

Analogical prompting outperforms 0-shot CoT and manual few-shot CoT

类比提示优于零样本思维链，也优于人工少样本思维链。

Tips：为什么数学题涨的更优

For the mathematics problem, you have the definitions, you have the formula, you have all the concepts. And models should learn the concept first and then adapt that concept to specific questions。

数学任务的解法依赖可显式陈述的概念与公式，所以先唤起知识、再套用这条路收益最大。代码与时序推理虽然也涨，但它们更依赖具体模式，抽象知识能帮的忙相对少。

2. 越强的模型越会类比

Weaker LLMs benefit less from analogical prompting, though it does not hurt zero-shot performance

弱模型从类比提示中获益较少，但至少不会伤害零样本表现。 With stronger LLMs, analogical prompting outperforms CoT with manually-designed or retrieved exemplars

强模型下，类比提示胜过人工设计或检索得到的示例。

The generated CoT is more tailored to the underlying LLM

模型生成的思维链更贴合它自己。人写的例子是按人类的思路组织的，模型未必按那个顺序思考。模型自己生成的推理，用的就是它自己的计算路径，所以更容易被后续生成复用。

2.7 指令 Instructions Work

Current LLMs are sensitive to prompt design 当前模型对提示词设计非常敏感。

There is no clear principle of how to write optimal prompts 不存在怎么写最优提示的明确原则。

2.8 Prompt Engineering By LLM

2.8.1 Roles and Pipeline

这是**LLM 自动优化提示词**的一套方法，也叫元提示（meta-prompt）优化，让大模型自己生成、打分、迭代改进 prompt，不需要人工手写 prompt。

1. Proposal generation: leverage the LLM to generate initial instructions.

LLM 作为推理模型（Inference Model）：输入 prompt + 样例，让模型执行任务，输出答案，用来验证这个 prompt 好不好。

2. Scoring each instruction based on the prediction correctness of a small set of problems.

LLM 作为打分模型（Scoring Model）：给每一条候选 prompt 打分：用对数似然 log-likelihood，看这个指令能不能在少量测试样例上，预测出正确答案 Log Probability。

1. 高分候选：保留，继续迭代
2. 低分候选：直接丢弃
3. LLM 作为重采样模型（Resampling Model，可选）：在不改变语义的前提下，改写现有的高分 prompt，生成更多变体候选，扩大搜索空间。

流程：生成候选 prompt → 打分 → 计算概率分数 → 保留高分 prompt → （可选）重采样生成语义相近的候选 prompt

> 两个核心步骤：
>
> - 候选生成：让 LLM 生成一批初始指令 prompt
> - 指令打分：在小的测试集上，评估每个 prompt 的预测正确性

Tips：这里的评分标准是模型自己的对数概率，不是人类认为的质量。所以最好的提示词对人类来说往往很奇怪。

2.8.2 进阶：迭代改进 Prompt

让 LLM 利用历史优化轨迹（sorted (solution, score) pairs），不断生成更好的新提示词。

优化器 Optimizer：the LLM to propose a new instruction given old ones and task exemplars 接收旧 prompt、任务样例，提出新的指令。

gradient free optimization（无梯度优化）：这个循环类似强化学习或微调，但差别在于这里没有梯度，也没有 GPU。梯度就是模型读一张排行榜。模型从 (solution, score) 对里看出哪类写法分高，然后照着写一条新的。

评估器 Evaluator：the LLM to evaluate the accuracy of an instruction 评估一条 prompt 在任务上的准确率。

整体流程闭环：

1. 优化器 LLM 读取元提示（meta-prompt），包含：任务描述 + 历史（prompt,分数）配对
2. 优化器（LLM as optimizer）生成候选 prompt，交给模型推理，产出任务答案
3. 目标函数评估器（objective function evaluator）对生成的解做客观评分
4. 把新的（prompt,分数）存入 meta-prompt，再次喂回优化器，循环迭代
5. 迭代结束，返回得分最高的 prompt 作为最终方案（return top solutions when finish），本轮生成的新候选（generated solutions）

2.9 Prompt Optimization

问题：提示词写法没有理论指引，只能靠人试

第一版解法：让模型生成候选、按对数概率打分、保留高分并同义改写。

第二版解法：把历史成绩排序后喂回模型当梯度，让它写更好的。

元提示：排行榜 + 带占位符的示例 + 行动指令。

结果：从 60.8% 起步，跑到 80.2%，比人写的最强基线高 8.4 个点，追平少样本思维链。

形态：随步数上升后走平；打分器的选择直接影响上限。

代价：得到的提示词对人类来说往往很难看，且不一定可迁移。

2.9.1 元提示 Meta-prompt：给优化器看的指令

Meta-prompt 就是用来让 LLM 生成更好 prompt 的 prompt，例子是 GSM8K 数学题任务： 里面包含两部分：

1. 历史候选 prompt + 对应的分数：按分数排序，让模型看到哪些 prompt 好、哪些差
2. 任务样例 exemplars：展示这个任务的输入输出格式（GSM8K 数学应用题）

> 元提示要求 LLM：写出和旧版本不一样、尽可能高分的新提示词。

2.9.2 GSM8K 实验结果

基线：经典Let's think step by step.（CoT，71.8% 准确率）

初始 prompt：Let's solve the problem. 只有 60.8%

自动优化得到的最好 prompt：Take a deep breath and work on this problem step-by-step.，**80.2%，**比原版 step-by-step 高约 8%，达到 PaLM2 论文里 few-shot CoT 的水平。

不同优化器（PaLM、GPT3.5、GPT4）生成的 prompt 效果略有差别。

2.9.3 优化曲线图

横轴 = 优化迭代步数，纵轴 = 训练集准确率

多个任务（GSM8K、BBH 的 movie 推荐、时序序列）都有相同规律： 准确率随迭代步数上升，之后进入平台期（plateau，不再明显上涨）

不同打分器（PaLM2-L /text-bison）、不同任务，收敛速度不一样；但趋势一致：前期快速提升。准确率随优化步数上升，然后趋于平稳。

Part 3 推理策略与结构

3.1 CoT Reasoning Strategies

核心思路：把难题拆成多个简单子问题，逐个解决，逐步推导最终答案，解决大模型的组合泛化（compositional generalization）难题（测试样本比训练更长、结构更复杂）

variable computation：Chain-of-thought prompting: variable computation of the thought process, adapting to tasks of different difficulty levels. More complex questions -> more reasoning steps in the chain-of-thought

思维链的本质是可变计算量，推理过程的长度随任务难度自动伸缩。越复杂的问题，思维链里的推理步数越多。

Reasoning strategies enabled by CoT：Decomposition、Planning

思维链使能了一些推理策略：分解、规划，等等。

We can explicitly instruct the LLM with the desired reasoning strategies for problem solving

我们可以显式地指定要让模型用哪种推理策略。与其期待模型自己挑策略，不如把策略写进提示，或者让模型自己写一份策略。

接下来三种方法的共同动机：least-to-most 指定的是分解这个策略，dynamic least-to-most 给分解配上了例子检索，Self-Discover 则是让模型从一堆候选策略里自己挑并组装。

3.2 由易到难 Least-to-Most Prompting

3.2.1 用分解做泛化

easy-to-hard generalization via decomposition

两大阶段

Stage1: Problem Reduction

问题拆解，让大模型把原始复杂问题，自动拆分成一系列更简单的子问题。例子：Amy 滑滑梯题目，原问题拆出子问题：“一趟行程需要多久？”

Stage2: Sequentially Solve Subquestions

顺序求解子问题，逐个回答子问题；子问题的答案传给下一个子问题。上一个子问题的答案，追加进 prompt，作为上下文，用来解决下一个更难的子问题，直到算出最终答案。

3.2.2 Example 1：SCAN 数据集

SCAN: translating synthetic natural language commands into action sequences.

SCAN（把自然语言命令翻译成动作序列） 的任务是把合成自然语言指令翻译成动作序列。

Length split: action sequences in the test set are longer than training samples. These splits are designed specifically to break the models.

长度切分：测试集的动作序列比训练样本更长，切分是专门设计来打断模型泛化的。

Least-to-most prompting can achieve nearly perfect test accuracy with 0.1% training samples as exemplars

只用 0.1% 的训练样本做示例，就能达到接近满分的测试准确率。

结论：

SCAN 难点：**测试集动作序列长度 > 训练集**，考验组合泛化能力

普通 prompt 和原始 CoT 在长序列上失效。text-davinci-002 与 code-davinci-001 在 Chain-of-Thought 一列都是 0.0 说明：思维链在长度切分上完全失效，比标准提示还差。普通思维链必须在一次生成内完成从易到难的泛化。

而 Least-to-Most 把同一个基准从 16.7 推到 99.7（code-davinci-002）。同一模型、同一数据，只换提示结构。L2M 只用 0.1% 训练样例，就能在 SCAN 上拿到极高准确率，因为每个子问题都很短，不再依赖长序列示例。

算例二：

3.2.3 Example 2：自然语言到 SPARQL

把自然语言，如英语问题翻译成 SPARQL 查询

Examples in CFQ (Compositional Freebase Questions) benchmark

> 例 1：Question: Did M1 star M2 , star M3 , and star a art director and editor of M0? SPARQL: SELECT count(*) WHERE { ?x0 edited M0 . ?x0 art directed M0 . M1 starred ?x0 . M1 starred M2 . M1 starred M3 }
>
> 例 2：Question: What was produced by a art director that M1 and M2 employed? SPARQL: SELECT DISTINCT WHERE { ?x0 produced by ?x1 . ?x1 a art director . M0 employed ?x1 . M1 employed ?x1 }

- Challenge: more complicated grammar with a larger vocabulary

挑战在于语法更复杂、词表更大。

- A single prompt might not be enough to cover all grammar rules

单个提示覆盖不了所有语法规则。示例是有限的，而语法规则的空间很大。

- Decomposition enables customized prompt for each subproblem

分解不只是让推理变简单，还让每个子问题都能配一份定制提示。

Tips：SCAN 的困难是长度（序列更长）；CFQ 的困难是语法覆盖（more complicated grammar with a larger vocabulary）。

3.3 动态 Dynamic least-to-most prompting

基础 L2M 的升级版本，在拆解之外，增加动态检索样例 完整三步：

1. Problem Reduction：句法解析，把复杂问句拆成子问题
2. Dynamically Select Exemplars 从样例池（Exemplar Pool）里，根据当前子问题，检索匹配的相关示例，不是固定一套 few-shot 样例，每个子问题用专属样例。Retrieve using constituent from decomposition 触发检索的键是分解出来的成分，所以分解驱动、增强了检索。
3. Sequentially Solve Subproblems：带着检索到的样例，依次求解子问题，自底向上拼装，拼接得到最终答案。

静态版用的是同一批固定示例，所有子问题共用一个提示模板。

动态版用的是示例池 + 按需检索，每个子问题可能配到不同的示例。

3.3.1 Example：自然语言到 SPARQL

CFQ 难点：复杂语法、大词汇量；单一固定 prompt 很难覆盖全部语法规则。

Dynamic L2M（Ours）平均准确率 95.0%，超过所有全监督训练基线模型（T5、LeAR 等）。不仅拆分问题，还给每一个子问题动态找最适合的示例。

右侧曲线图：随着样例池增大，Dynamic L2M 始终 > CoT > Vanilla Few-shot，差距明显。

3.4 Self-Discover

核心思想：instruct the LLM to compose reasoning structures for each task without human demonstrations

让 LLM 自己发现这个任务专属的推理结构，不需要人工写示范样例，定位不是又一个更强的提示词，而是一个按任务生成提示结构的元方法。

分两阶段：

1. Task-level：Discover Reasoning Structure

输入任务描述 + 基础原子推理模块（Atomic Reasoning Modules），让模型生成任务专属推理框架（key-value 结构，是解题的步骤）。例如颜色物体计数任务：推理结构会定义：每种物品类型、每种颜色物品数量等中间问题。

2. Instance-level：Solve Problems with discovered structure

差别在于它加上了 Task Instance（具体题目实例），并在解码时把值填进去（Fill in the Values based on Keys during decoding），套用上一步生成好的推理结构，填充各个 key 的值，得到最终答案。

Tips：Different tasks, they have different reasoning structures 不同任务有不同的推理结构。比如数学题适合分解，但换一类任务就不一定。比如，社交推理题需要换视角（change your perspective）、调试类问题需要验证假设（test some hypothesis）

实验效果

- Self-Discover 对比直接输出答案：BBH 提升 + 11%，T4D+39%，MATH+5.5%
- Self-Discover 对比原始 CoT：BBH+7%，T4D+29%，MATH+8.5%

> 区别对比：
>
> - CoT：直接写一步一步自然语言思考
> - L2M：拆分子问题，顺序求解
> - Self-Discover：先为任务规划一套结构化推理模板，再用模板解题

3.5 Summary

1. CoT：可变计算量，思考过程长度自适应不同难度题目

分支一 让模型写得长：给例子→ 给指令→ 让模型自造例子→ 让模型自写提示。

分支二 让模型按结构写：显式指定策略（分解）→ 分解 + 检索→ 模型自组结构。

分支三 生成多个再挑：Part 4。

2. 推理场景下提升 CoT 效果的几种方案： Few-shot prompt，附带标注好的思考样例；直接用指令 prompt 触发模型做 CoT；让 LLM自动设计 prompt。
3. 提示策略的核心原则（不会过时）：复杂任务鼓励更长的 CoT 推理、提示策略要适配任务所需要的推理方式、和大模型交互的最佳实践一直在迭代更新。

方法	核心	特点
CoT	直接生成连续自然语言思考	简单，但是长组合任务容易崩
Least-to-Most	拆成子问题，顺序求解，带上前序答案	解决组合泛化
Dynamic L2M	L2M + 为每个子问题动态检索样例	CFQ 这类语义解析任务效果极强
Self-Discover	先生成任务专属结构化推理框架，再解题	不需要人工写 demo

Part 4 Search and Selection Methods

原版 Self-Consistency（数学，需要答案抽取） → 局限：无法用于开放式自由文本 → USC 通用自洽：不用抽取答案，开放任务可用 → 升级：不靠单纯一致性投票，训练 ORM/PRM 验证器，PRM 更强 → 局限：以上所有，都要等完整答案生成完才选择，浪费单步信息 → ToT 思维树：分步生成 + 分步评估，在中间 partial solution 上做树搜索（24 点例子）

核心主题：不再只采样 1 条 CoT 推理路径；生成多条候选推理分支，再挑选最优答案，用来修复单条生成里的错误。

4.1 挑战与新思路

之前 CoT、L2M 都是只生成单条推理路径（greedy 解码），存在缺陷：限制模型只能输出一个解法，一旦中间一步推理错，最终答案直接出错。

Idea: Exploring multiple branches allows the LLM to recover from mistakes in a single generation, generate multiple candidate solutions per problem, generate multiple potential next reasoning steps given the current (partial) thought.

新思路（搜索与选择）：探索多个分支，让模型有机会从单次生成的错误中恢复。对同一个问题，生成多条候选完整解；给定当前不完整的思路，生成多个可能的下一步。

Challenge: how to select the best response from multiple candidates? In most cases, we do not have an oracle scorer at inference time

核心挑战：如何从一堆候选里选出最好的回答？推理时（inference）我们没有打分器（oracle scorer），没法直接知道哪个答案是真值。

对比项	Part I - III（提示技术）	Part IV（搜索与选择）
核心动作	让一条思维链写得更好、更长	生成多条候选，然后挑一条
算力花在	单条链的长度（token budget）	候选的数量（sample budget）
新出现的问题	怎么写提示	怎么在没有标准答案的情况下挑
代表方法	CoT / analogical / least-to-most / Self-Discover	self-consistency / AlphaCode / USC / ORM・PRM / ToT
算力换准确率	纵向投入（深度），把一条链写长	横向投入（广度），做法是写很多条

4.2 Self-Consistency 自一致性

Self-Consistency: select the response with the most consistent final answer.

核心思想：对同一个问题，采样多条不同的 CoT 推理路径，统计最终答案的众数（majority vote，出现次数最多的最终答案）作为输出。

Note: the selection is only based on the final answer. The reasoning paths do not need to be the same across different sampled responses.

重点：只看最终答案是否相同，中间推理过程可以完全不一样。只要多条不同思考路径收敛到同一个答案，就采信这个答案。

数学理解：marginalize out reasoning paths（把所有推理路径边缘化，只聚合最终结果）

实验结论

一致性 ≈ 准确度，一致性越高（越多采样得到同一个答案），答案正确概率越高。

1. 在数学数据集（AddSub, MultiArith, GSM8K 等）上，所有模型（UL2, LaMDA, PaLM, GPT, Code-davinci）加上 Self-Consistency 之后，准确率全部明显上涨。
2. 性能随采样路径数量增加而提升：采样越多推理路径，效果越好；一直上升，逐渐趋于平缓。
3. Sample-and-Rank：选 log 概率最高的那条回答；Self-Consistency 远好于单纯基于概率排序。

采样多样性

1. 温度（temperature）/ 核采样（nucleus sampling），能产出更多样推理路径 → SC 效果大幅提升；
2. Self-Consistency（基于采样的自洽性）随样本数持续提升，保持提示词、改采样，差异来自模型随机性。
3. Ensemble（多 prompt 集成）的做法是改提示词、保持贪心解码，所以它们之间差异来自问法不同。同样的样本量，Self-Consistency 改采样比改提示词涨得多。所以改写提示词只是换个入口，模型内部走的还是同一条高概率路径；而高温采样是真的在探索不同的推理路径。
4. Beam search：保留解码过程概率最高的 top-k 路径，多样性差，SC 提升有限。束越宽，取最好的那条，分数反而越低，因为束搜索保留的是概率最高的一批路径，而它们彼此高度相似。所以对这些路径投票，等于对同一批几乎一样的答案投票。

4.2.1 Example：AlphaCode 按执行结果聚类

任务背景：Competitive programming（Codeforces）

题目：长文本描述 + 少量样例输入输出；代码不仅要过样例，还必须通过隐藏测试用例。

本质是自洽性思想跨域迁移，生成多个独立样本 → 按行为一致性分组 → 选取多数收敛的结果。

底层原理是多个独立采样样本收敛到相同行为，通常代表这个解是正确的。

前提是提交预算，可以生成海量候选程序（Sample budget 采样预算），但最终能提交到评测平台的候选数量有硬性上限（submission budget 提交预算）。采样可以很多，但提交名额很少。

AlphaCode 一致性聚类流程：

1. 基础过滤：筛掉无法通过题目给出样例测试用例的代码。

- 缺陷：剩下的代码依然有可能在隐藏测试集（held-out test cases）上出错。

2. 方案：训练模型只生成新的测试输入，不生成输出。

- 原因：没有真值标签（ ground truth ），没法生成标准答案；生成输入远比生成正确判题输出简单。

3. 执行与聚类： 将所有采样出来的候选代码，在全部生成的测试输入上运行；把在所有测试输入上输出完全相同的代码划分到同一个簇。从规模最大的 10 个簇中，每个簇挑选 1 条代码提交（竞赛提交次数有限，多选不同簇，相当于押多个不同解法，提高命中正确代码的概率）。

   核心假设：all programs in the same cluster are semantically equivalent if the generated inputs are diverse and of high quality 如果生成的测试输入足够多样、质量足够高，同一个簇里的代码语义等价。

> 对比曲线：filtering（只过滤掉过不了样例的代码）有提升；filtering + clustering 额外再涨性能；但依然和 oracle upper bound 有差距

结论：

1. 单纯疯狂增加采样预算（生成几百万代码）没用。价值来自在提交预算限制下的筛选策略。
2. 提交预算约束，迫使模型做多样性选择：优先挑选行为不同的方案，而不是单纯挑 “模型打分最高” 的方案。
3. 整套方法强依赖测试输入质量：输入多样性不足，会导致语义完全不同的代码碰巧输出一样，被错误分到同一簇。输入生成模型的好坏，直接决定筛选可靠性。
4. Clustering provides additional performance gain over filtering only. Still a gap from the oracle selection, the selection is far from solved. 聚类相比单纯过滤还有额外收益，但距离完美的神谕选择仍存在差距。选择问题远没有被解决。

> 性能提升来自算法流程设计，不是靠放大模型规模。小模型也能靠这套搜索选择方法拿到显著提升。

4.2.2 Self-Consistency 的局限

Limitation of self-consistency decoding: requires an answer extraction process

核心限制：原版自洽解码，必须有答案抽取（answer extraction）的步骤。在自由开放式生成场景中（文本摘要、开放式问答、故事写作） 两段合格的摘要，表达意思可以相同，但文字句子完全不一样，没办法直接比对。

底层逻辑：投票的前提，是能把每一条模型输出，压缩成一个可以相互比对的标签。 所以前提就是要能够提取、比对答案。

> 本质理解： 自洽流水线隐含要求：存在映射函数，把一段长文本映射成等价类标签

数学任务映射：提取末尾答案数字。

AlphaCode 代码任务映射：程序在整套测试输入上的输出向量 自由文本生成缺少这个映射。

> 这个限制，引出Universal Self-Consistency（USC）：直接让大模型自己完成一致性挑选，绕开答案抽取。

4.3 Universal Self-Consistency（通用自一致性）

4.3.1 流程

Question → LLM → Responses（多条）→ 拼进 Universal Self Consistency prompt → 再交给 LLM → Selected response

1. LLM 生成多条回答（候选 responses）
2. 把所有候选全部塞回 prompt 交给 LLM，指令：基于多数共识，选出最一致的回答。
3. LLM 自己充当选择器，选出最终输出

4.3.2 特点

Universal Self-Consistency (USC) improves performance on open-ended generation (summarization, QA), where the original self-consistency is inapplicable.

USC matches self-consistency performance in math reasoning and coding.

USC does not require answer extraction and code execution.

USC performance is bounded by the long-context capability.

- 开放式生成上有提升（摘要、问答），原本自洽性根本用不了的地方，现在能用了。
- 数学与代码上持平。
- 不需要答案抽取、不需要代码执行 ，工程上更省事，部署成本低。原方法（写程序抽答案）的优点是确定、便宜、可验证；新方法（让模型选）的优点是通用，代价是吃上下文且不确定。
- 受长上下文能力限制（候选很多时 prompt 会很长）。

4.4 LLM 排序器 / 验证器

核心思想：不再只靠 “多数投票一致性”，单独训练一个奖励模型（verifier/reward model）给候选解打分，比单纯一致性选择更强。

4.4.1 三步流水线

1. Train generator：训练生成器，输入问题Q_i，生成解S_i;
2. Generate and label：对每个问题生成 100 个候选解，打标签;

3. Train Verifier：训练验证器，输入问题 + 候选解，输出该解好坏的奖励分数 Y_i^j。

4.4.2 两种 LLM 验证器 / 奖励模型

ORM（结果监督）：在完整解的层面做验证，只看最终答案对不对。

PRM（过程监督）：对每一步推理 step 单独验证，监控推理中间步骤对错，比 ORM 更强。

扩展性不同：PRM 随样本数增加涨得更好，而 ORM 更早走平。

过程监督能抓住理由错但答案蒙对的解进行去噪：the step level scoring just removes the noise. It catches a solution that got lucky with the right answer for a broken reason

基于 LLM 的验证器，性能超过基于一致性的选择方法

1. PRM 随着样本量增加，扩展性更好（scales better with more samples）
2. 性能高度依赖验证器本身质量 highly dependent on the verifier quality
3. 同一个验证器不一定能跨任务泛化 might not generalize across tasks

4.5 Tree of Thoughts

问题：到目前为止的所有方法都是完整回答全部生成完毕之后，再做选择，没有充分利用单步的打分。

引出 Tree of Thoughts（ToT，思维树）：LLM + tree search，不再一次性生成完整答案，在局部部分解（partial solutions）就评估、搜索，优先探索更有前景的分支。

4 种范式：

1. IO Prompting 输入直接输出；
2. CoT Prompting 思维链；
3. CoT-SC 思维链 + 自洽多数投票；
4. ToT Tree of Thoughts 树搜索，在每一步分支做评估剪枝。

核心区别：CoT/SC 是整条推理路径一次性生成；ToT 是分步展开、分步评估，像搜索树。

前提：需要一个好的 LLM，加上好的提示设计，来做自评估。

这是整个搜索过程的地基：剪枝的依据就是模型自己的判断。如果自评估不准，被剪掉的可能是正确答案，最后搜到的是错的那一支。

Example

1. **Thought generation 思路生成**：提示大模型提出下一步可行的思考分支；
2. **Thought evaluation 思路评估**：提示大模型评估当前这个中间状态有没有希望（promising /impossible）。

评估分支好坏：LLM 多次投票，选多数结果作为选择，在候选中间状态里挑最优分支。生成多个 plans → 展开成不同段落方案 → LLM 多次投票，选出最优 state。

> b 代表每一步保留最好的 b 个分支（breadth，搜索宽度），b 越大效果越好。
>
> 右图：success rate 随访问节点数量变化，ToT（绿色虚线）显著高于 CoT、IO。

1. 采用 BFS 广度优先搜索的 ToT，在 token 预算限制下扩展性远好于普通 Prompt、CoT；

2. 可以结合更高级搜索算法，如 MCTS 蒙特卡洛树搜索；
3. ToT 依赖：能力足够的 LLM + 用于自我评估的提示词设计。

总结

1. 推理时计算量可以通过在解空间采样多条分支来扩容（inference-time compute，推理时算力）；
2. Consistency-based selection 基于一致性的选择：简单、有效、通用的核心原则

   Self-Consistency：边缘化多条推理路径，基于最终答案投票选择；

   代码生成：基于执行一致性做重排序（AlphaCode 聚类）
3. 当 LLM 的自我评估靠谱时，训练验证器（ORM / PRM），或依赖模型自评估做树搜索（ToT）。

顺序本身就是成本顺序：越往后越贵、也越强。

Lecture 4 Reasoning in LLMs (2)

Part 1 两种思维模式

1.1 快思考与慢思考

对比项	System 1 Thinking	System 2 Thinking
中文名	系统 1 快思考	系统 2 慢思考
课件给的特征词	Instinctive / Quick / Automatic Little/No Effort / Emotional / Un Conscious	Slower / Rational / Complex Decisions Conscious / More Logical / Effortful
例子	2 + 2	17 x 24
读法	不用动脑、答案自己冒出来	要动脑、要一步一步算

两个例子的差别不是难度，是是否需要中间步骤。前者靠记忆直接取出答案（factual recall），后者必须经过推理，需要多步中间生成（multi-step generation）。

1.1.1 System 1

早期 LLM 只有系统 1

要素	课件原文	读法
结论	Prompting and Fine-Tuning can (and only) yield a (better) System 1 !	提示与微调能做的只是把系统 1 练得更好，做不出系统 2
归因	as they were developed mainly based on the so-called “Deep Learning Hypothesis”	深度学习假说

Deep Learning Hypothesis

神经元最快每秒 100 次；人做感知只需 0.1 秒，最多只跑了 10 层；所以一个有 10 层的神经网络就够了。系统 1 本身极强：语言流畅度、格式遵循、常识补全、模式匹配，全在系统 1 的范围内。

Human perception is fast 限定的是感知（perception），不是推理。也就是说这个假说不是解释慢思考。

如果模型只按这个假说设计，那它天然就是系统 1，一次前向传播，10 层左右，出来就交答案。

1.1.2 System 2

Example：下棋

系统	课件原文	场景
System 1	generates the proposals (used in speed chess)	生成候选着法，用于快棋
System 2	Need to “Think” more / Reason, e.g. keep track of the tree (used in competitions)	在脑子里维护整棵搜索树，用于正式比赛

两个系统是流水线：先由系统 1 靠直觉砍掉大部分着法（否则搜索空间爆炸），再由系统 2 在剩下的候选上算到底。

1.2 Reasoning Definition

Reasoning is decomposing a potentially complex task into simpler subtasks the LM can solve more easily by itself or using tools. There exist various ways to decompose into subtasks, such as recursion or iteration. In that sense, reasoning is akin to planning

1. 推理的动作是分解，不是变聪明（所以能拆 ≠ 拆对了）
2. 子任务要能被模型自己、或借助工具（Agent 接口）解决。
3. 分解的两种组织方式是递归与迭代。递归 = 子任务还是同类问题；迭代 = 一轮一轮改。
4. 推理近似于规划（即Planning）。

For LLMs, “Reasoning” is commonly defined as the process of answering questions that require complex, multi-step generation with intermediate steps. As opposed to answering single-step Factual-recall type of questions

区分的不是答案，是过程的存在与否。能不能写出中间步骤，才是推理的判据。把推理从一个能力概念变成了一个输出形态概念。后面的所有技术（提示、搜索、RL）之所以能起作用，正是因为它们都在改变输出形态，让模型多写中间步骤，而不是让模型变聪明。

1.3 Example

层次	课件标注	在这一页对应什么
第一层	Intermediate reasoning chain that is not explicitly shown to the user	模型内部想了什么，用户看不到（Reasoned about last digit of powers for a couple of seconds）
第二层	Intermediate reasoning steps that are shown as part of the answer	写进答案里的推导过程，用户能看到，能检查、能训练、能投票
结论	The main answer	最终答案 3，只是整段输出里最后一行

Part 2 LLM 推理能力争议

2.1 推理动机 Three Motivation of Reasoning

动机	原文	与人对照
提升少样本学习能力 Few-shot Learning	“Humans (even a small child) can learn from relatively few examples because Human can “Reason.””	from Dr. Denny Zhou of Google DeepMind
提升分布外鲁棒性 Out-of-distribution Robustness	Humans possess the ability to generalize beyond familiar scenarios.	人会迁移到没见过的场景
提供可解释性 Explainability	Humans are capable of providing explanations for their decisions or predictions, whereas machines (especially deep neural networks) are often considered “black box” within limited explainability.	人能解释自己为什么这么判断

三个动机的权重不一样。前两个是性能诉求（更少数据、更强泛化）；最后是信任诉求。

2.2 推理能力的争议

2.2.1 LLM Can Reasoning

证据	原文
1.成绩证据（分数高）：在推理任务上高分	High performance on various tasks requiring reasoning
2.机制证据（存在有效手段）：能用思维链逐步推理	The ability to reason step-by-step with chain-of-thought prompting
3.行为证据（表现出类人偏差）：会表现出类人的内容效应	The reflection of human-like content effects reasoning

Tips:

CoT：被拿来当它有一点推理能力的证据，但 CoT 本身就是提示技巧。

Content Effects：人在推理时会被题目内容（而非结构）影响，比如同一道逻辑题换成现实场景就更容易做错。模型也表现出类似偏差，这被当作像人一样推理的旁证。但另一个角度是会被人无关内容带偏，也许说明的是它只是在做模式匹配，不是真的在推理。

2.2.2 LLM Can not Reasoning

But there are also observations that suggest LLMs may not be capable of reasoning:

观察	原文
1.复杂推理仍然失败	LLMs still struggle with tasks that require complex reasoning
2.对某些术语的出现频率敏感 对训练语料里出现频率不同的数字，准确率会系统性变化，这更像查表而不是计算	The performance of LLMs has been found to be sensitive to the frequency of certain terms
3.关联不上自己记住的信息 模型记住了某条信息，却在需要它的时候用不上，记忆与调用是断开的	Language models have been found to struggle with associating relevant information that they have memorized

观察	原文
4.未解的核心疑问 写长上下文只是往上下文里塞了更多相关文本，让下一个 token 更容易猜对	“It is not yet fully understood whether the LM is really reasoning, or simply producing a larger context that increases the likelihood of correctly predicting the missing tokens.”
5.可用测试集自查	Misguided Attention: a suite of testing prompts 公开的测试提示词集
6.有些错误形态不像懂推理的人会犯的	Some mistakes made by LLMs seems to show they do not actually know reasoning

Example：错误答案范例

题面	模型的答案	正确答案
how many r’s are there in the word "strawberry"?	There are two "r"s in the word "strawberry."	3 个 r（strawbe-r-r-y 中有三个 r）
9.11 and 9.9, which one is larger?	9.11 is larger than 9.9.	9.9 更大（11 > 9 的直觉被误用）

错误的归因：

1. 分词：tokenizer 可能被切成错误的token
2. 训练语料：日期类型的统计强度可能盖过了小数比较。

三条具体的缺陷

硬伤	课件原文
易被无关上下文带偏	LLMs can be easily distracted by Irrelevant Context/ Content
前提顺序会影响结果（但不应该）	Premise Order matters in LLM “Reasoning”, but they **should not be （前提条件的顺序在逻辑上不影响结论，所以模型对顺序敏感本身就是反逻辑的）！**
没有 Oracle 就无法可靠自我纠错	LLMs cannot self-correct reliably without an oracle: while allowing LLMs to review self-generated responses can sometimes help to correct inaccurate answers, **such reviews may also change correct answers into incorrect ones（双向影响：自我检查既可能把错的改对，也可能把对的改错）**.

2.2.2.1 无关信息干扰 Distracted by Irrelevant Context

Example 1：加入无关上下文的干扰信息。模型把不该进算式的数字拉进了算式。所以这不是计算能力问题，是取舍问题。

位置	课件原文
输入	Lucy has $65 in the bank. She made a $15 deposit and then followed by a $4 withdrawal. Maria’s monthly rent is $10（无关信息）. What is Lucy’s bank balance?
模型输出	Lucy’s bank balance is $65 + $15 - $4 - $10（被加入） = $66. The answer is $66.

Example 2：加入不相干但无害的信息，仅起到稀释作用。说明模型的取舍依赖提示。

把原题拆散，在中间反复插入 The sky is blue. 与 The grass is green.

加入一点提示：Ignore irrelevant context.

模型输出变成：Lucy has $65 in the bank. After making a $15 deposit and then a $4 withdrawal, her bank balance is $76. Maria’ s monthly rent is not relevant to this question, so it can be ignored. The answer is $76. 答对了，而且模型还主动说明了为什么忽略那一句。

2.2.2.2 前提顺序改变 Premise Order matters

改动：He loses 10 bills while getting home. 这一句从第 2 句挪到了倒数第 2 句。逻辑上不该有影响：丢 10 张钞票这件事发生在哪个时点，都会让张数减少 10。但挪动之后，减 10 被放到了最后一步，而前面多算了一步 triples，于是基数完全不同。

模型给出的完整推导：

两次推导都把丢 10 张放在不同位置，而两种放法在题目文字里都说得通。原顺序下丢钞票发生在付账单之前，基数变成 40；重排后发生在最后，基数变成 300。

Tips：这种错误有两种解读。读法一是模型对语序敏感，并不是真推理；读法二是这个题面本身歧义，两个答案都对。

Part 3 三条教它推理的路径

3.1 三条路径总览

Make（让它做）→ Lead（引导它做）→ Force（强制它做）

类型	课件原文	改变什么
Type 1	Using some Prompting techniques for the LLM, e.g. Chain-of-Thought (CoT) prompting	让模型一次写得更长（提示技巧）
Type 2	Make LLM perform Search and Selection from Multiple Candidates => Increase the WIDTH to explore solution space	加宽：生成多条候选再挑
Type 3	Make LLM perform Iterative self-improvement => Increase the DEPTH to reach the final solution	加深：在同一条回答上反复改

3.2 加深的动机 Iterative self-improvement

。论证很紧：先说「」，再说「」，最后指出缺的是什么。

问题：

1. Even humans often make (sometimes trivial) mistakes at first thought 人也会错
2. Although sampling multiple solutions (i.e. increase the width of exploration space) can **reduce mistakes **by considering more than one single prediction. 加宽能减少错误但不够

原因：**NO feedback loop to correct the mistakes after a complete solution is generated. **没有反馈回路。加宽只是并行多生成几份，每份都是一次成型，写完就定稿，中途没有修正机会。

解决：Under the Inference-time self-improvement approach, LLM iteratively improves its own response to the given task. 在回答的中间过程上反复迭代。

3.2.1 迭代自省 Self-Refinement

Basic idea: prompt an LM to critique/revise its own outputs iteratively.

元素	作用
Input	原始输入，喂给模型
Model M	在 Feedback 和 Refine 两步 Self-Refine 同一个模型既做生成也做评审
Feedback	让 M 先给自己的输出写评语
Refine	让 M 拿着评语改上一版输出
回环	改完再评、评完再改，形成循环（全部反馈来自模型自己）

Self-Refine vs Reflexion

Self-Refine：同一模型自评自改，强调不需要额外训练、不需要外部信号。

Reflextion：把反馈写成文字存起来（verbal reinforcement learning），强调跨回合（trajectory）积累经验，主要在 Agent 任务上。

共同点：都把反馈表达成自然语言而不是梯度；都是迭代的。

3.2.2 Reflection：三个任务、五个阶段

(a) Task → (b) Trajectory → (c) Evaluation → (d) Reflection → (e) Next Trajectory。

阶段	任务
(a) Task	任务描述
(b) Trajectory	第一轮的动作与观察（Action: / Obs:）
(c) Evaluation	这一轮好不好的判据，内部启发式还是外部环境奖励
(d) Reflection	把失败原因写成一段自然语言（课件的配色规则见下）
(e) Next Trajectory	带着反思重来一遍的动作与观察

任务	判据来源（c）	第一轮的结果	反思后
1. Decision making	内部启发式: Rule/LM Heuristic: Hallucination.	动作全都 Nothing happens.（选错了抽屉/灶眼）	换成 stoveburner 2，成功把锅放进台面
2. Programming	外部但自造: Self-generated unit tests fail	返回 'No'，判据错了（只比总数、不看顺序）	改为检查嵌套结构 check(S1) or check(S2)
3. Reasoning	外部环境奖励: Environment Binary Reward: 0	答 novelist, screenwriter（假设两人职业集合相同）	改为 novelist（只取真正的交集）

三列的判据强度是递减的：列 3 有真正的环境奖励（0/1），列 2 是自造单测，列 1 只是语言模型的启发式判断（Hallucination）。所以，判据越硬，反思越有用。

LLM generates feedback on its output. Use external evaluation (oracle) when available.

LLM self-refines its output given both internal feedback and external evaluation.

3.3 边界条件 Token Budget

3.3.1 Good External Evaluation (Oracle 预言机)

Self-reflection and Self-refinement work ONLY WHEN Good External Evaluation (Oracle) is available

图表特征：

1. 全部上涨，自省有效。
2. ALFWorld 是平行上升后拉开差距（说明反思的增益是逐轮累积的）；HotPotQA 是一开始就分叉（ReAct + Reflexion 从 0.30 一路到 0.55，而两条不带反思的曲线几乎是水平线）。
3. ReAct + Reflexion 比 CoT + Reflexion 高（0.55 vs 0.40）：说明反思作用在被执行过的动作轨迹上，比作用在纯文本推理上更有效，因为动作产生了环境反馈。

3.3.2 Without Oracle 自我纠错掉分

图表特征：

1. 六个格子全部走平或下跌，且基本是单调下降。
2. GPT-4 在 HotpotQA 上是 49.0 → 49.0 → 43.0：第一轮没动，第二轮掉 6 个点。说明改坏不是立刻发生的，多改一轮多一分风险。
3. GPT-3.5 在 CommonSenseQA：75.8 → 38.1，几乎砍半。常识类问题没有统一判据，模型的自评最容易翻车。
4. 第 1 轮要 3 次调用、第 2 轮要 5 次，掉分的同时还多花了近 5 倍算力。

Self-correction without oracle feedback hurts reasoning performance, LLMs can wrongly judge the correctness of its predictions。

3.4 利用预算 Utilize the Token Budget

Giving AI additional test-time computing can greatly improve performance. However, previous demonstrations are mainly limited to specific tasks.

推理时算力可以提升表现，但在特定任务时有限。

如何更好地利用算力?

手段	中文	类型	判对错的依据
N answers … find the most common one（自洽性 / USC）	获得 N 个答案，然后找出其中最常见的那个答案（即“自我一致性”或“普遍自我一致性”）	Type 2 加宽	答案之间的一致性，不需要外部判据
Best-of-N with a Learned Verifier	使用“学习式验证器”来进行“N 人比”的竞赛	Type 2 加宽 + 验证器	训练出来的验证器（因为模型自评不可靠）
revise and correct it’s own responses iteratively	利用大语言模型来反复修改和纠正自己的回答	Type 3 加深	模型自己，所以只在有 Oracle 时可靠

All these techniques induce the use of more token budget

手段不同，代价的性质相同。

3.5 验证器搜索与训练

3.5.1 三种验证器搜索：Best-of-N / Beam / Lookahead

> ⊞ = Apply Verifier（虚线方框：在这个位置调用验证器）
>
> ● = Full Solution（实心圆：完整解答）
>
> ○ = Intermediate solution step（空心圆：中间步骤）
>
> ● = Selected by verifier（绿色：被验证器选中，不等于正确答案，涨分并不是满分）
>
> ● = Rejected by verifier（红色：被验证器否决）

策略	标注	结构
Best-of-N	Generate N full solutions with the verifier → Select the best final answer using the verifier 使用验证器生成N个完整解 → 使用验证器选择最佳最终答案	完全并行：一次生成 N 份完整答案，全部交给验证器打分，取最高。只在终点判。
Beam Search	Select the top-N samples using a PRM → Select the best final answer using the verifier 使用PRM选择前N个样本 → 使用验证器选择最佳最终答案	分层并行：每步保留 top-N 条中间步骤，逐层展开。每一步都判。
Lookahead Search	Beam search, but at each step select the top-N candidates to roll out based on the PRM value at the end of the current step 采用波束搜索法，但在每个步骤中根据当前步骤结束时的PRM值，选择前N个候选解进行进一步处理	带回传的串行：先试跑（Rollout “Lookahead” step），把分数 Propagate PRM values back to previous step，再 Continue Search from the top-N options。先往前多看一步再回来判。

3.5.2 训练过程验证器 Training a Process Based Verifier

For search, the verifier should score each step in a solution. Prior work used human labels of per-step correctness for supervision. This work instead follows the approach to learn value function.

在搜索过程中，验证者需要对解决方案中的每一步进行评分：之前的研究使用了人类对每步正确性的标注来进行监督。而本项研究则采用一个价值函数。

要素	说明
Problem	首一四次多项式，已知三个根，求 p(0) + p(4)
(a) Outcome Annotation	$y_S = 0 $只看最终答案，整条链判 0
(b) Process Annotation	$y_{s1}^{SE} = 2/3$；$y_{s1}^{HE} = 1$，对 s1 这一步单独给分
图例	s1 是第 1 步； $s_{1,1} $是第 1 个完整解的第 1 步

SE 与 HE 是两个不同的估计量： y^{SE}是统计估计，3 次试跑 2 次成功； y^{HE}是硬估计，取多数结果。

3.3.2 说明模型自评不可靠；3.5.2说明不要用模型自评，用一个单独训练的 value function。

3.5.3 搜索方法对比

All verifier search methods outperform the majority baseline. 所有验证搜索方法都优于大多数基线方法。

Majority 是基线，即 self-consistency（多条答案取多数票），准确率最低，证明加了验证器之后，同样的预算能拿到更高分数。但验证器也不能解出所有题。

Beam search performs best at small budgets. At large budgets, Best-of-N performs similarly to beam search. 在预算有限的情况下，波束搜索效果最好；在预算充足的情况下，Best-of-N方法与波束搜索效果相当。

在图片中$M := sqrt(N) $与 $M := 4 $的区别是 beam 的宽度策略：前者随总预算 N 动态取 √N，后者固定为 4。图上两种策略的曲线在中段交叉，说明没有一种宽度策略全程占优。

Lookahead generally underperforms due to the high cost of lookahead rollouts. 由于“预看”技术的实施成本较高，其性能通常不如其他方法。

Lookahead 把预算花在试跑上，而试跑本身不产出最终答案。同样的 token 预算下，纯并行的 Best-of-N 更划算。

Part 4 Post-training Techniques & DeepSeek-R1

4.1 后训练：四条路径

4.1.1 从推理时算力转向训练时算力

背景：没有 Oracle 的自省会掉分，而一个有 Oracle 的场景在真实任务里很少见。如果审查能力不能靠模型自省得到，那就只能把它训进去。本讲的关注点从推理时怎么多花算力切换到训练时怎么改权重。

4.1.2 Reasoning is one of the Specialized Skills

四阶段图说明推理模型在 LLM 生态里的位置。专精 ≠ 替代。

阶段	标注	产物
STAGE 1: BUILDING	构建阶段	Building an LLM
STAGE 2: PRE-TRAINING	预训练阶段	Foundation model（基座模型）
STAGE 3: FINE-TUNING (“POST-TRAINING”)	微调 / 通用后训练	Classifier（分类器）、Personal assistant（个人助理）
STAGE 4: MORE SPECIALIZATION	进一步专精	Web search LLMs / RAGs / Topic chatbots / Multimodal LLMs / Code assistants / Reasoning models / Agents / Distilled & cheap models

Reasoning是八大 Specilized skill 的方向之一，这背后存在不可避免的权衡（inevitable trade-offs）。得到的是省去全部标注数据与SFT阶段；代价是模型输出可读性下降、语言一致性变差。

4.1.3 Four Ways & Two Compute

序号	英文	中文	归类	代表
1	Inference-time compute scaling	推理阶段算力扩展	test-time compute	上一讲全部内容

序号	英文	中文	归类	代表
2	Pure RL (e.g., DeepSeek R1 Zero)	纯强化学习（如 DeepSeek R1 Zero）	train-time compute	DeepSeek-R1-Zero（纯 RL，无 SFT）
3	RL + SFT (e.g., DeepSeek R1)	强化学习 + 监督微调（如 DeepSeek R1）	train-time compute	DeepSeek-R1（先 SFT 再 RL）
4	Pure SFT (e.g. s1, distilled DeepSeek R1)	纯监督微调（如 s1、蒸馏版 DeepSeek R1）	train-time compute	s1、蒸馏出来的小模型

第 1 条不改权重（推理时算力），第 2 - 4 条改权重（训练时算力）。训练算力与测试算力可以互相替换（$log10(test) = -1.2 \cdot log10(train) + ...$）。所以这四条路本质上是在同一条替换曲线上选点。

4.2 推理阶段算力扩展

Realizing LLM Reasoning via Inference-Time Scaling

4.2.1 Core Idea

Core Idea: Trigger the LLM to generate one or more LONG Chain-of-Thought (CoT) Path of Exploration via one or more of the following: 触发 LLM 生成一条或多条**长思维链（CoT）探索路径**，可通过以下一种或多种方式实现：

四个手段	属于哪类	动的是什么
Few-Shot CoT Prompting	提示（Type 1）	给几道带过程的例题
Instruction Prompting	提示（Type 1）	给一句指令，如一步一步想
Instruction Tuning	训练（SFT）	改权重：用带指令的数据微调
Reinforcement Learning	训练（RL）	改权重：用奖励信号优化

Few-shot CoT 的机制：用 token 预算换正确率的直接体现。不是教模型算术，是教它先写过程再给答案的格式。

4.2.2 RL Optimized COT

RL optimized Chain of Thought can be a highly effective way to scaling test-time compute.

RL 强化学习 optimized Chain of Thought 与写长思维链的区别：前者是用一个奖励信号把怎么想训进权重，后者只是提示模型这一次写长一点。前者是训练阶段算力，后者是推理阶段算力。

DeepSeek-R1: **Incentivizing** Reasoning Capability in LLMs via Reinforcement Learning

Tips：是 Incentivizing，不是 Teaching 或 Training。这与 不给任何监督数据 是同一件事的两种表述，能力不是教会的，是被奖励信号激励出来的。

4.3 SFT

Augments input-output examples with instructions, which enables instruction-tuned models to generalize more easily to new tasks.

通过指令扩充输入输出样例，让经过指令微调模型能够更容易泛化到新任务：在训练样本上额外配上任务指令（指令 + 输入→输出），模型学习 “按照指令完成任务”，而不是只记住固定样例，因此泛化到没见过的新任务。

例 1 的答案是开放文本（翻译），例 2 的答案是封闭标签（情感三选一）。这两个案例说明 instruction-tuned models 的通用性：同一套数据格式能覆盖两类任务。

限制：要做推理的 SFT，期望输出里就得包含完整的思考过程。但它只能模仿数据里出现过的过程。要得到数据里没有的新推理路径，就需要的强化学习 RL。

4.4 R1 Family and DeepSeek Training Paradigm

4.4.1 R1 Family (DeepSeek-R1-Zero vs DeepSeek-R1)

对比项	DeepSeek-R1-Zero	DeepSeek-R1
训练方式	large-scale reinforcement learning (RL)，without supervised fine-tuning (SFT) as a preliminary step	incorporating a multi-stage training and cold-start data before RL
有先做 SFT 吗	没有（Zero 的含义） 零监督数据（without supervised fine-tuning）	有（**cold-start data** 就是 SFT 数据） cold-start data 即是一小批人工整理、格式友好的推理数据，用来在 RL 之前把输出格式改正。
问题	poor readability, and language mixing （可读性差，思维链写得读者看不懂；语言混杂，中英文混在一段里，都不影响正确率，只影响可用性）	针对上面两个问题而设计
类型	Pure RL	RL + SFT
意义	不给任何人工标注，纯靠 RL 也能长出推理能力	R1 是在DeepSeek-R1-Zero上，解决可用性问题

4.4.2 DeepSeek Training Paradigm 训练范式全景图

区域	作用
起点	整个家族的共同起点
冷启动主线	GRPO / Rule-based Reward (Accuracy, Formatting) + CoT Language Consistency Reward
虚线捷径	虚线绕过全部 SFT 与数据生产，纯 RL
数据生产	用 DS-V3 as judge 筛数据
三个数据集	Reasoning Data 来自上面的筛选用；Non-Reasoning Data 由 DeepSeek-V3 (671B/37B Activated) 经 CoT Prompting 产出
R1 支路	最终产出 DeepSeek-R1
蒸馏	标注 Distillation，底座为 Qwen2.5-Math-1.5B / Qwen2.5-Math-7B / Qwen2.5 14B / Qwen2.5 32B / Llama-3.3-70B-Instruct / Llama-3.1-8B

4.5 R1-Zero：纯 GRPO 训练

4.5.1 起点：MoE base

DeepSeek-V3 (DeepSeek-AI 24) is used as the starting checkpoint model.

从 V3 的权重出发，而不是从零训练或从头预训练。整个 R1 家族都在 V3 的骨架上做后训练。

DeepSeek-V3 is a Transformer and Mixture of Experts (MoE) based model.

主干是 Transformer，前馈层是 MoE 结构。

MoE 结构的作用：R1 系列能在合理成本下做大规模 RL，RL 要反复前向，激活参数量直接决定训练成本，若 671B 是总参数量，37B 是每次前向实际激活的参数量，比值约 18:1。而MoE 的作用就是节省计算量（不是存储量，总参数量在显存中）。

4.5.2 MoE：Mixture of Experts

MoE：将 Transformer 中原有的稠密前馈网络（FFN）层替换为稀疏 Switch FFN 层（浅蓝色）。该层对序列中的各个 token 独立运算，每个 token 只路由到一个专家。

例子：两个 token（下方x_1=“More”、x_2=“Parameters”）路由过程（实线），二者被分配至 4 个 FFN 专家；路由模块为每个 token 独立选择专家。说明 the router independently routes each token，路由是逐 token 做的，同一个句子里的不同词可以去不同专家。Switch FFN 层最终输出**被选中 FFN 的结果乘以路由门控值**（虚线），说明 the selected FFN multiplied by the router gate value，所以专家的输出要乘上它的权重再相加。

标准 Transformer 块（左边）Add + Normalize → Switching FFN Layer → Add + Normalize → Self-Attention。

MoE 展开（右边）：原本一个 Switching FFN Layer 的位置，展开成 FFN1 - FFN4 四个专家加一个 Router。
MoE 的组成

部分	原文	读法
稀疏层	Sparse MoE layers instead of dense feed-forward network (FFN) layers.	把稠密 FFN 换成多个专家 FFN，每个 token 只走其中少数几个
路由器	Routers determines which tokens are sent to which expert.	一个路由器（ Router）决定每个 token 派给哪个专家 但路由是按 token 相似度学出来的，不一定对应人类理解的领域

4.5.3 只用 GRPO：DeepSeek- R1-Zero

Develop reasoning capabilities without any supervised data, via reinforcement learning finetuning. The Group Relative Policy Optimization (GRPO) algorithm was utilized to fine-tune the V3 base model.

通过强化学习微调，在不使用任何监督数据的前提下，开发模型的推理能力。研究采用GRPO 强化学习算法（Group Relative Policy Optimization，组相对策略优化）对 V3 基座模型进行微调。从 DeepSeek-V3 Base 到DeepSeek-R1-Zero只经过一个模块（面向推理的强化学习 Reasoning Oriented RL），全程没有任何SFT阶段。

所以 R1-Zero的缺陷 poor readability, and language mixing（可读性差、语言混杂）。来源于奖励函数只约束答案正确性和输出格式，省掉了全部标注数据与 SFT 阶段，代价是可读性与语言一致性，不负责文本表达质量。

Part 5 强化学习与 DeepSeek-R1

Part 4 讲为什么要转向训练，并给出了 DeepSeek-R1 家族的训练流水线。这一部分讲流水线里最关键的算法：GRPO。讲完算法，再看它训出来的 R1-Zero 长什么样、代价是什么，最后看 R1 怎么把代价补上。

5.1 Reinforcement Learning

Element	动物行为学	在 RL 里的角色
State	一轮扔木棍（的现场状态）	当前的局面，策略据此决定动作

Element	动物行为学	在 RL 里的角色
Action	狗朝木棍跑过去	当下做出的选择
Reward	一块零食	打分：动作好不好的数值反馈
Policy	狗自己的行为习惯	从状态到动作的映射，就是要学的东西
Objective	尽可能多拿零食	长期找到策略，使奖励最大，而不是答对 让模型在没有标注数据的情况下自己发现推理行为

> State: a round of stick throwing.
>
> Action: the dog moving toward the stick.
>
> Reward: a treat.
>
> Policy defines how an agent behaves in a certain state. How to assign the reward.
>
> The objective is to find the policy that maximizes the reward.

5.1.1 Five Elements of RL

一个逐项映射表，把语言模型还原成一个标准 RL 问题，从而让通用 RL 算法（PPO、GRPO）可以直接搬过来用。

RL 概念	LLM 对应物	工程含义
Agent	the LLM itself 模型本体	智能体就是模型，没有额外的控制器
State	the initial input 初始输入	整段提示就是初始状态；注意它不含模型已生成的内容 在同一道题上采样多次，初始状态完全相同、只有动作序列不同
Action	next token prediction 预测下一个 token	动作空间 = 词表；一个回答是一串动作
Policy	**defines the behavior of the LLM, define the word distribution**	策略就是词分布；训练就是改这个分布 所以 RL 不需要另建一个策略网络，直接微调那 671B 权重。
Reward	a numerical value to identify the correctness of the chosen action	整段回答结束后给一个奖励，不是逐 token 的密集奖励 奖励稀疏是 R1-Zero 需要自己的长思维链的根本原因：没有中间引导，模型只能靠整段结果的好坏去反推哪一步做对了。

5.2 GRPO

5.2.1 GRPO and PPO Algorithms

GRPO algorithm is based on the Proximal Policy Optimization (PPO) algorithm.

GRPO算法基于近端策略优化（PPO）算法

The Reward model is a trained model that assigns rewards.

奖励模型：是一个经过训练、用于分配奖励的模型。

**The value model **estimates the long-term expected future reward from a given state.

价值模型：根据给定状态，预估长期的期望未来奖励。虽然GRPO理论上源自PPO，但**GRPO训练时不再使用价值模型**，这是它和原版PPO最核心的差异。

PPO：q → Policy Model → o → Reference/Reward Model → r → v → A，GAE 把 r 与 v 合并；

GRPO：$r_1⋯r_G *直接进GroupComputation出*A_1⋯A_G$，整条链上没有 v。

模块	中文	作用	状态
Policy Model	策略模型	要训练的主体：给定 q 产生输出 o	两法皆有 Trained Model
Reference Model	参考模型	冻结的旧策略，用来算 KL 散度、防止跑偏	两法皆有 Frozen Model
Reward Model	奖励模型	训练出来的模型，给输出打分 r	两法皆有 Frozen Model
Value Model	价值模型	估计从当前状态出发，未来长期期望回报 v	只有 PPO Trained Model
GAE	广义优势估计	用 r 与 v 一起算优势 A	只有 PPO
Group Computation	组内计算	只靠 $r_1⋯r_G $ 直接算优势 A_i	只有 GRPO

Tips：PPO 价值模型。PPO 算优势需要知道这一步比平均水平好多少，而那个平均水平就是 v。在传统 RL 里，v 由价值网络估出来。但 LLM 场景在每个状态评估多个动作，让这件事变得特别贵。

5.2.2 Three Changes

GRPO generates multiple responses at once and generates multiple rewards. Group computation decides the advantage of the response based on other responses in the group. The value model is removed since it’ s expensive to evaluate multiple actions every state.

实现	改动	换来什么
一次生成多个回答	无（新增采样），得到 o_1 ⋯ o_G	有了一组可以互相比较
用组内比较定优势	把 GAE 换成 Group Computation	以平均分为基线，高于评价就是好动作，不需要价值模型就能算出优势
删掉价值模型	删掉 Value Model 与 v	显著降低训练资源占用，因为 LLM 的每个状态都要评估整个词表的动作

5.2.3 Objective Function 目标函数与优势函数

公式里的符号	读法	要点
$q $~ $ P(Q)$	从问题分布里采一个问题	For every question (q)
$o_i ~ $\pi_{\theta_{old}}$(O	q)$	用旧策略在同一问题上采 G 个回答
r_i < br>	第 i 个回答拿到的奖励	来自 Reward Model
A_i	第 i 个回答的优势，同一道题必定有一部分A_i > 0、一部分$A_i < 0 $这就是组相对（Group Relative）	Group Computation 的输出，组内标准化
$\pi_\theta / \pi_{\theta_old}$	新旧策略的概率比	Scale the reward by the ratio
$clip(1 - \varepsilon, 1 + \varepsilon)$	把比值截断在 $1 ± \varepsilon $内，clip 限制单步更新的幅度（conservative updates）	or clipped ratio (conservative updates)
$\beta \cdot D_{KL}(\pi_\theta \| \pi_{ref})$	与参考模型的 KL 散度惩罚，限制整体偏离参考模型的距离，防止把模型训崩。	Maintain divergence from the ref model

5.2.4 Reward Modeling in GRPO

$r_i $的来历：奖励不靠训练一个奖励模型，而是靠规则直接算出来。

rule-based reward system 规则奖励系统的两个类型：

奖励类型	原文	怎么判定	例子
准确率奖励 Accuracy rewards	Ensures correct responses	答案对不对，可程序化验证	math（比对最终答案）、LeetCode（跑测试用例）
格式奖励 Format rewards	enforces the model to put its thinking process between <think> and </think>tags	思考过程有没有写在指定标签里	<think> 包裹思考、标签外放最终答案

rule-based 规则奖励意味着不需要训练奖励模型、不需要人类标注，只要有一个能判对错的程序就行。但适用范围是数学与编程这种可以判对错的情况，开放写作、对话这类任务用这套奖励训不出来。

5.3 R1-Zero

5.3.1 Naturally more Thinking

训练集回复的长度在RL过程中自然（没有任何奖励，是为了拿准确率奖励而被迫发生的作用）变长了，花费了更多时间，解决了更多思考任务（但不一定都是对的）。

The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time.

阴影带（波动范围）随训练推进而变宽，说明不同题目之间的长度差异在拉大，模型学会了按难度分配思考量。

5.3.2 DeepSeek outperforms Open AI

Comparison of DeepSeek-R1-Zero and OpenAI o1 models on reasoning-related benchmarks.

self-consistency（多采样投票）：上一讲是在推理时手动投票，这一讲训练出来的模型天生适合投票。

pass@1 = 只给一次机会，看答对的概率；

cons@64 = 采 64 次后投票，看多数票是否正确；

赢的三处：AIME 2024 cons@64 拿到 86.7，比 o1-mini（80.0）与 o1-0912（83.3）都高；MATH-500 拿到 95.9，是全表最高（o1-0912 94.8）；GPQA Diamond 的 73.3 明显高于 o1-mini 的 60.0。

输的两处：LiveCodeBench 只有 50.0，低于 o1-mini 的 53.8（差了 3.8）；CodeForces 的 1444 距离 1820 / 1843 差得更远。

AIME 的分裂：pass@1 上 R1-Zero 的 71.0 低于 o1-0912 的 74.4；但 cons@64 上 86.7 反超 83.3。一次答对不如它，64 次投票就超过它。说明它的单次输出方差更大、探索范围更宽，而投票能把这份宽度兑现成准确率。

5.3.3 Aha Moment

The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

模型自己回头检查并推翻了自己的推导。

Wait, wait. Wait. Anthropomorphic tone（拟人化的语气）。模型不是在报告一个计算结果，而是在表达一种惊讶，但模型并不具有人的意识。

That’s an aha moment I can **flag** here：模型把自己发现的问题打了一个**标记**。在它自己的语境里，这是一条待办事项；对我们来说，这是一个突发的检查机制。这是无监督的，并且规则奖励里不会自我检查，所以自我纠错不是被奖励直接鼓励的，而是想得更久的自然产物。

5.4 R1-Zero 的两个代价和 R1

第 56 页是转折页：R1-Zero 的能力很惊人，但它不能直接交付。这一页给出两条毛病，以及 R1 的动机。

代价	中文	具体表现	从哪来
poor readability	可读性差	思考过程写法随意，人读不懂、也不好当成品输出	奖励只对答案对错与标签格式负责，没人管可读性
language mixing	语言混杂	同一段思考里多种语言混着写	训练语料与奖励都没约束语言一致性

两个代价都是奖励设计的直接后果。Accuracy rewards 管对错，Format rewards 管标签，readability 与 language consistency 都不在奖励里。因为trade-off 的一般规律，想让模型保住的品质，要么写进奖励，要么写进冷启动数据。R1 选择的是后者，用 cold-start data 把人喜欢的样子先灌进去，再用 RL 在此基础上得到推理能力。

R1 的产生：To make reasoning processes more readable（目的）we explore DeepSeek-R1, a method that utilizes RL with human-friendly cold-start data：cold-start data （手段）。

区别：R1-Zero 从零冷启动，R1 从人写的数据冷启动。

5.5 DeepSeek-R1 Steps

5.5.1 Step1 启动

起点回到同一个 V3 Base，但中间插了一段 SFT。

Start again from the V3 Base model.

- Perform supervised fine-tuning (SFT) for 2 epochs.
- Perform Reinforcement Learning (RL) to improve helpfulness and harmlessness.

步骤	原文	做了什么	与 R1-Zero 的差别
回到起点	Start again from the V3 Base model.	推倒重来，从 V3 Base 重新开始	相同 R1-Zero 也是从 V3 Base 出发
监督微调	Perform supervised fine-tuning (SFT) for 2 epochs.	用人写的数据（冷启动）做 SFT，跑 2 个 epoch	不同 修复了poor readability and language mixing R1-Zero without any supervised data
强化学习	Perform Reinforcement Learning (RL) to improve helpfulness and harmlessness.	再做 RL，目标扩到有用与无害（not correctness）	目标不同 实现了helpfulness and harmlessness R1-Zero 的 RL 只管推理正确性

R1-Zero：V3 Base → RL（纯 RL，无 SFT）。证明推理可以无监督涌现，代价是可读性与语言一致性。

R1：V3 Base → SFT (2 epochs, cold start) → RL。用一小段人写数据把人的偏好固定在起点上，再用 RL 长推理。

两者共享同一个基座、同一种算法（GRPO）、同一套奖励思路；唯一的不同是要不要先给人写的数据。

5.5.2 Step2 SFT 数据

那 2 个 epoch 用的 80 万条数据是怎么造出来的？两路并行、一冷一热。

左边这条分支，产出60万条推理数据：**模型自己生成推理答案，再筛选合格样本**。用的是拒绝采样（Rejection Sampling）：先生成一大堆候选回答，只留下检验合格的。判分有两套机制，一套是规则校验，另一套直接拿DeepSeek V3模型当裁判打分。

右边这条分支，产出20万条非推理数据，由两部分合并而来：一部分是现成的DeepSeek-V3监督微调数据集；另一部分是用DeepSeek-V3（671B，激活37B）做思维链提示（CoT Prompting）生成的数据。这20万条是用来保住模型的通用能力。如果80万训练数据全都是推理题，模型写作文、日常聊天、事实问答这类基础能力就会变差。加入20万非推理样本，相当于保留25%的基础能力盘。

细节：**CoT语言一致性奖励**。专门针对R1-Zero出现的语言混杂问题，直接新增一项奖励约束。模型身上的问题，如果没有对应的奖励目标，就只能靠奖励函数或者训练数据去修正。

冷启动长思维链数据（Cold Start Long CoT Data）只有几千条，而后面SFT训练用到80万条数据。也就是说：**人工只需要提供几千条种子数据，剩下绝大部分数据，全部靠模型自举自动生成**，这是这套流水线节省人工标注成本最核心的地方。

为什么训练的时候，推理数据和非推理数据要混合在一起？只拿推理数据训练模型，会造成通用能力下滑。推理只是八大专精能力中的一项，能力之间必然存在取舍权衡。资源全部倾斜到推理，别的能力就会下降。 所以80万训练样本采用60万推理能力样本 + 20万基础能力样本的组合。 这也区分了R1-Zero和R1：R1-Zero单纯用来证明模型推理潜力，R1才是能正常对话、落地使用的成品模型。

5.5.3 Step3 Distillation

普通蒸馏（权重蒸馏）：大模型直接教小模型；老师模型（R1）和学生模型同时跑。学生要模仿老师每一步输出的概率、中间层特征。老师模型本身要在线参与训练。缺点：师生模型架构最好接近，很难跨家族，比如Qwen很难直接蒸馏给Llama。

R1的数据蒸馏：大模型只出题写答案，把这些题目+标准答案打包成数据集，拿这个数据集去训一堆别的小模型。

第一步：先用R1生成一大堆高质量问题+ R1 写的完整推理答案，保存成80万条数据。 第二步：R1模型本身可以关掉，不再参与训练。拿这堆数据，直接给Qwen、Llama各种基座做普通SFT（有监督微调）。 图中箭头是从数据集指向各个基座，没有箭头从R1模型连到学生模型。所以用数据蒸馏，不是权重蒸馏。

6个学生基座： Qwen2.5-Math-1.5B、Qwen2.5-Math-7B（数学专精小模型） Qwen2.5 14B、Qwen2.5 32B（通用Qwen） Llama3.3-70B-Instruct、Llama3.1-8B（Llama家族）

同一个80万数据集，丢给上面全部6种模型，全都能训。 说明：**这套推理数据集包容性很强，不挑模型。**

	数据蒸馏（R1）	权重蒸馏（传统）
老师做什么	提前写好大量问答样本，之后老师模型就不用了	训练全程老师模型都要运行
学习素材	文本数据集（问题+答案）	模型输出分布/中间特征
能不能跨家族（Qwen→Llama）	可以	一般不行
训练方式	普通SFT	蒸馏损失训练
应用场景	**跨模型家族、快速批量训一堆不同基座、不想让大模型一直在线** → 优先**数据蒸馏**	**学生模型和老师模型输出行为、概率分布高度一致，师生架构相同** → 用**权重蒸馏**

5.6 R1 的成绩与路线对比

5.6.1 R1 Matches Open AI O1

六个基准、五个模型的柱状图，读法要按组而不是按柱。

核心结论：**推理专项训练不是全方位变强，是能力定向偏移。** 模型在数学、代码这类推理任务上的成绩大幅上涨；但是它原本的通用知识能力基本保持不变，不会明显提升。

> 底层逻辑，呼应前面课件提到的 trade-off 不可避免的取舍： 训练资源会定向优化目标能力（数学、代码推理），但不会额外增益通用知识。通用能力是基座原生自带的，推理 RL 只改造推理相关的行为，不改动已有的知识库。

不是说推理训练**损害**MMLU，这里是**基本不动**。和 R1-Zero 不一样，R1-Zero 是只做 GRPO，没有混合非推理数据，会出现可读性、语言混杂的副作用；而 R1 训练混入了 200k 非推理样本，守住了通用能力。

经过 R1 数据蒸馏后的 32B 开源模型，在大部分评测基准上，性能可以胜过一些闭源的小模型。

5.6.2 RL vs SFT+RL

> R1-Zero：只做GRPO强化学习，**没有SFT冷启动**；
>
> R1：先做2轮epoch的冷启动SFT（80万混合数据：60万推理+20万非推理），再做RL。 **多加这2个epoch的SFT，换来哪些收益、哪些损失**

逐行拆解变化（R1-Zero → R1）

1. **LiveCodeBench（代码基准）** R1-Zero=50.0 → R1=65.9，**+15.9** 全表里涨幅最大。代码正是纯RL（R1-Zero）最薄弱的环节，冷启动SFT把这块短板补上。
2. **AIME 2024（高难度数学竞赛）** R1-Zero=71.0 → R1=79.8，**+8.8** R1-Zero原本71.0，低于o1-0912（74.4）；经过SFT后反超，拉到79.8。
3. **CodeForces（算法代码竞赛评分）** R1-Zero=1444 → R1=2029 直接超过 o1-0912(1843)、o1-mini(1820)。 R1-Zero代码能力明显弱于o1，经过SFT+RL之后，代码能力实现反超。对应前面课件提到R1-Zero代码短板的问题。
4. **GPQA Diamond（生物/物理/化学专家科学题）** R1-Zero=73.3 → R1=71.5，**-1.8，唯一下降项** 增加SFT混合数据之后，专家级科学问答分数轻微下跌。

> 原因：SFT数据集里没有足够多这类高端科学题目，属于前面提到的能力取舍（trade-off）。虽然混入20万非推理数据用来兜底通用能力，但无法完全消除副作用。

核心问题：2个epoch冷启动SFT到底价值多少？冷启动SFT**不是全方面均匀变强**。它重点补齐纯GRPO（R1-Zero）的短板（代码、竞赛数学），但会在部分不在训练数据覆盖范围内的任务上，出现轻微性能下滑。 而80万数据里搭配20万非推理样本，目的就是**把这类能力衰减控制到很小**，但无法完全消除（GPQA这里的-1.8就是残留代价）。

> 代码（LiveCodeBench）：**+15.9百分点**（最大增益）
>
> AIME竞赛数学：**+8.8百分点**，实现反超o1-0912
>
> GPQA专家科学题：**-1.8百分点**，小幅下降

5.7 本讲总结

1. **DeepSeek-R1 matches OpenAI o1 in complex reasoning tasks.**

> R1 在复杂推理任务上追平 OpenAI o1。
>
> 用词是 mtach ，不是outperform 。

定位：**结果层**，证明开源权重模型，推理能力可以对齐闭源前沿模型o1。

2. **DeepSeek-R1-zero achieves strong logical reasoning without supervised fine-tuning.**

> R1-Zero 在不使用监督微调SFT的条件下，获得很强的逻辑推理能力。
>
> R1-Zero有短板，LiveCodeBench、CodeForces分数明显落后。

定位：**方法层**，证明推理能力可以脱离人工标注数据，只用RL训练出来。这是R1-Zero最核心的贡献：**纯强化学习GRPO，不靠人工标注SFT，推理能力可以自己涌现。**

3. **Distillation enables smaller models to inherit advanced reasoning capabilities from DeepSeek-R1.**

> 蒸馏技术可以让小模型继承R1的高级推理能力。
>
> 工程落地价值：671B超大模型训练、部署成本极高，绝大多数团队无法使用。借助**数据蒸馏**，把R1学到的推理能力迁移到1.5B~70B中小模型，低成本可用。

定位：**落地层**，把前面两项科研成果变成行业能实际使用的模型。

Lecture3 和 Lecture4 串联对比

Lecture3（推理1）：提示与搜索，不动模型权重 Self-consistency、ToT、least-to-most、ORM/PRM。 思路：模型权重不变，**推理时靠提示词引导模型拉长思考过程。**

Lecture4（推理2）：训练，修改模型参数 GRPO + 规则奖励 + 冷启动SFT + 蒸馏 思路：在训练阶段，直接把深度思考的能力**固化进模型参数里面。**

共同目标：让模型作答前充分思考。

差别： Lecture3：**当场靠提示临时唤起思考**； Lecture4：**在训练阶段，把思考能力写入参数。** R1证明：后者不仅可行，甚至可以完全不用人工标注数据。

下节预告：Next Course: Planning for LLM Agents

推理和规划是递进关系：推理负责想明白每一步怎么做；规划负责把一连串动作编排好，调用工具、接收环境反馈、失败后重试。 对应之前Lecture2的智能体四子系统：感知 / 推理 / 记忆 / 执行。

Reasoning（本讲）：**单次问题，想出中间推理步骤**（解决思考内容质量）

Planning（下一讲）：**智能体，把多步推理组织成一套可执行动作序列**

Lecture 5 Planning in LLM Agents

1. 为什么需要计划什么是计划：反应式智能体做长任务时，会以三种方式翻车；一个工作定义，以及它最轻量的形态思维链（Part I）。

3. 为什么难：推理错一步可以重来，在真实世界里做错一步往往不行（Part II）。

4. 四股压力：模块化、反馈、长时程与控制，分别逼出不同的解法（Part III - V）。

5. 经典规划遇上 LLM：把 1970 年代就成型的 PDDL 拿出来试（Part VI - VIII）。

Part 1 Planning

1.1 Example：反应式智能体的长任务

Buy the lab's new GPU workstation: compare configurations and prices across vendor sites, check compatibility against the cluster requirements on the internal wiki, and file the purchase requisition.

假设问题：为实验室购买新的 GPU 工作站，比较各厂商网站上的配置与价格，对照内部 wiki 上的集群要求核验兼容性，然后提交采购申请。让一个反应式（reactive）智能体去完成一件长任务。所谓反应式，就是每步只看当前上下文、不做显式计划。

1. Four independent searches run as one thread in one context.

架构问题：比价（信息收集）四次相互独立的搜索，但被串进同一条线程、同一份上下文里跑。

后续解决：把规划器与执行器分开。

2. The four sites it chose all come back and none is great.

信息问题：核兼容（约束检查）它挑的四家网站都返回了，但没有一家让人满意。

后续解决：一个回头改条件或改计划的信号。

3. It picks one anyway and files the requisition.

控制问题：提交申请（不可撤销的动作）它还是挑了一家，直接把申请提交了。

后续解决：计划作为安全边界。

1.1.1 Four Problems

结构	English	在工作站算例里的体现
并行搜索	parallel searches	Vendor A / B / C / D / E 五路互不依赖，可同时进行
依赖	dependencies	五路搜索 → Compatibility check (wiki) → Requisition
迟到信息	information that arrives later in the task	兼容性检查结果要到中途才知道（箭头标注 arrives later）
不可撤销	a step that cannot be undone	提交采购申请这一步标注 irreversible

The graph itself also changes: when none of the four vendors fits, a fifth search has to be added. 图本身还会变：四家都不合适时，必须临时加进第五次搜索。

维度	要回答的问题
价值 Value	When is planning useful? When can it hurt?　规划什么时候有用？什么时候反而有害？
表示 Representation	How is a plan represented?　计划用什么形式表示？
承诺 Commitment	When is a plan made, and what may change it?　计划在什么时候做出？什么可以让它改变？
监督 Oversight	How/when should a person weigh in?　人应该在什么时机、以什么方式介入？

1.2 Plan Mode

1.2.1 Application

Cursor, Claude Code, and Codex all introduced a Plan Mode: write a persistent plan 写一份持久化的计划（落在文件里，不随对话消失）, have a human verify it 让人审这份计划, carry it out (possibly with sub-agents, in parallel) 可以用子智能体并行做.

好处	说明
更多算力 more compute	允许模型多想一步，而不是边想边做
更好的上下文 better context	先调查一遍，再带着全局信息动手
一个外部产物 an external artifact	计划是文件，可以被 review、被 diff、被复用
分解 decomposition	把大任务切成小块，每块可独立验证
人的批准 the human approval	把不可撤销的那一步交给人类把关

1.2.2 Five Steps

步骤	做什么
1. read-only investigation	只读调查：先看代码库，不改任何东西
2.clarifying questions	澄清提问：把不清楚的需求问出来
3.written plan	写出计划：形成一份可读的文档
4.human review	人工评审：由人来确认或打回
5.execution	执行：按计划动手改代码

1.3 Definition

A plan is an explicit representation of intended future behavior: the actions or subgoals an agent will attempt, with any ordering or dependencies among them.

计划是对意图中的未来行为的一种显式表示：智能体将要尝试的动作或子目标，以及它们之间的顺序与依赖关系。

1. 它面向本任务　A plan is for this task.（对照：skill 是跨任务复用的指导）。Plans can be more specific than skills.　计划可以比技能更具体。
2. 它组织未来的工作　A plan organizes future work. It may specify actions or subgoals, along with their ordering and dependencies.
3. 它是提案，不是保证　A plan is a proposal, not a guarantee.It can be inspected and revised: by the user, another model, or interactions with the environment.

对照项	计划 Plan	技能 Skill
适用范围	为这一个任务而做	跨任务复用的通用指导
具体程度	Plans can be more specific than skills 可以更具体	偏抽象，留出实现空间
生命周期	任务结束即作废	长期保留、反复使用
可否被修订	a proposal, not a guarantee 可被检查与修订 修订来源：用户（by the user）、另一个模型（another model）、与环境的交互（interactions with the environment）。	通常不随单次任务改动

1.3 Chain-of-thought

最轻量的一种计划：思维链（chain-of-thought）

CoT 是说出来给自己听，同一个模型，同一次生成，说着说着就答了。

Plan 是写下来给别人看，写完之后，别人（人、另一个模型、环境）可以插手。

1. Provides extra compute and a scratchpad for the model to refer back to　提供了额外算力，以及一块模型可以回头查阅的草稿。
2. Plan and solution arrive in one model generation.　计划与解答在同一次生成里一起出来。
3. Nothing explicitly inspects the plan. 没有任何外部环节去检查这份计划，因为它不能被检查与修订。
4. The CoT can be trained to increase the probability of a correct answer (STaR, Zelikman et al.; DeepSeek R1).　思维链可以被训练，用来提高答对的概率。

Example：Let's think step by step. OR Let's first understand the problem and devise a plan to solve the problem. Then, let's carry out the plan and solve the problem step by step.

1.4 Decompose

1.4.1 Steps and Function

Stage 1: Decompose Question into Subquestions. 把原问题拆成若干子问题

Stage 2: Sequentially Solve Subquestions. 按顺序逐个解答，并把答案向前传递

Function:

1. Models struggled at answering complex questions but were better at answering sub-questions.

模型在复杂问题上吃力，但在子问题上表现更好。

2. Answering sub-questions requires less context.

回答子问题需要的上下文更少。

3. Improvements grow with problem length.

问题越长，这种改进越明显。阶段一写出的子问题是可检查的中间产物，人看一眼就知道拆得对不对。同样，如果阶段一把问题拆错了，回答质量就会下降。后续解决方案是自适应递归分解，让委派本身成为一个可训练的动作，模型自己学什么时候该往下拆。

1.4.2 modularity 模块化

A decomposer generates sub-tasks and routes each sub-task to a dedicated handler.

一个分解器产生子任务，并把每个子任务路由到专门处理它的处理器。

Modularity allows improving handler models independently: via in-context demonstrations or training.

模块化让每个处理器模型可以被独立改进，用上下文示例或直接训练。

Programmatic/structured control: we'll see this later on with RAO, RLM, and MACU.

程序化 / 结构化的控制：后文会在 RAO、RLM、MACU 再见到。

Part 2 Reasoning vs Acting

2.1 Versus

Acting means plans must handle feasibility, consequences, information, and recovery.

一旦要行动，计划就必须同时处理四件事：可行性（feasibility）、后果（consequences）、信息（information）、恢复（recovery）。对应后面章节的规划压力：可行性→模块化，信息→反馈，后果→控制，恢复→长时程。

推理步只改变不产生新信息（changes only text），行动可以揭发信息（can reveal information），这是两者唯一一处行动比推理更有利的地方。

对比项	一次 CoT 推理步 A CoT reasoning step	世界中的一个动作 An action in the world
改动了什么	changes only text 只改动文本	changes the state the agent meets next 改变智能体下一步面对的状态
能否撤回	can be undone by writing more 再写几句就能推翻	can be irreversible: submit, send 可能是不可逆的：提交、发送
信息可见性	every intermediate is visible 每一个中间步骤都可见	can reveal information 可以揭示新信息（原本不知道的）
失败方式	（写错了重写）	can fail （可能失败）
为什么有用	plans are useful if the model can condition on them better 只要模型能更好地以计划为条件	（同样的计划，在这里要面对可行性、后果、信息与恢复）

2.2 Classical Planning 经典规划：状态、动作与前置条件

在 LLM 出现之前，规划是一个有严格定义的问题。

积木世界（blocks world）五个要素：状态、动作、前置条件与效果、初始状态与目标、计划。

A state is a set of predicates; actions have preconditions and effects.

状态是一组谓词的集合；动作带有前置条件与效果。

A valid sequence of actions satisfies every action's preconditions.

一个合法的动作序列，是每一步的前置条件都被满足的序列。

Planning is a search problem: find a sequence of valid actions that reach the goal.

规划就是一个搜索问题：找一条能到达目标的合法动作序列。

步	动作	使合法的前置条件	效果
1	unstack(blue, orange)	blue 在 orange 上面、且 blue 是干净的、手是空的	变成 holding(blue)，orange 变干净
2	put-down(blue)	手上有 blue	blue 落到桌上、手变空
3	pick-up(orange)	orange 在桌上、干净、手空	变成 holding(orange)
4	stack(orange, blue)	手上是 orange、blue 干净	得到目标 on(orange, blue)

2.3 Changed hard point

经典规划里，世界的模型（状态、前置条件、效果）是显式写出来的，所以世界模型对不对是可验证的；LLM 把这些全放进权重里，写一个看起来合理的计划变容易了，判断它是否真的理解世界反而变难了。

1. With LLMs everything is implicit! 有了 LLM，一切都变成隐式的。
2. For an LLM agent, writing a plausible plan is easier, and knowing whether its model of the world is correct is harder. 对 LLM 智能体来说，写出一个看起来合理的计划更容易了，而判断它对世界的建模是否正确更难了。
3. So agent design uses less deep search, and more checking the world: observation, verification, and recovery from errors. 所以智能体设计的重心从深搜转向检验世界：观察、验证、从错误中恢复。
4. But search, and action applicability, are still useful concepts 但搜索与动作的适用性仍然是有用的概念，只是不再需要人手工写出来。

对比项	经典规划	LLM 智能体
世界模型	显式写出（谓词 + 前置条件 + 效果）	everything is implicit，藏在权重里
最难的一步	搜索：在巨大状态空间里找到可行序列	判断：判断模型的界模型到底对不对
主要手段	深度搜索（A*、启发式）	observation, verification, and recovery from errors
搜索还需要吗	需要，是主体	仍然有用，但不再是设计的重心

2.4 Plan Representations

Formal plans are checkable; language plans are general. Programs are in-between.

形式化计划是可检查的，语言计划是通用的，程序在两者之间。

表示	可检查性	通用性	示例
形式化计划（formal）	高：能自动验证每一步是否合法	低：需要预先写出域模型	PDDL
语言计划（language）	低：只有人能读	高：什么任务都能描述	CoT、自然语言 plan
程序（program）	中：能跑、能读，但受 API 限制	中：受可用 API 与程序结构限制	Policy Code

Perception APIs（感知 API，绿色）：把画面变成符号，例如 detect_objects(...)。

Control APIs（控制 API，橙色）：把符号变成动作，例如 pick_place(obj0, obj1)。

这段代码是程序即计划的典型：它把 找空碗 → 拼出待堆列表 → 逐个摆放 写成了确定的控制流，因此可以被读、被 diff、被改，而不是像 CoT 那样只在一次生成里存在。

2.5 RePlan while Acting

1. 开发者预先定义工作流（Developer-defined workflow，无智能体 Agentless） 人直接写死固定流程：只能按写好的供应商列表挨个检查。 特点：逻辑固定，完全不能临场变通。
2. 先规划、再执行智能体（Plan-then-execute agent） 大模型先一次性做好完整规划（决定要查哪些供应商），规划敲定之后，再从头到尾执行，执行中途不再改计划。特点：速度更快、成本更低、结果更容易预测。
3. 自适应智能体（Adaptive agent，RAO/MACU） 边跑边看反馈，遇到意外可以重新规划。 例子：连续 4 个供应商都失败了，模型拿到这个执行反馈之后，主动去找新的供应商。特点：当执行过程里发现了新信息、突发状况，模型可以随之调整方案。原则：只有当临场反馈带来的收益，值得多花的成本和额外复杂度时，才用边执行边重规划。

Part 3 Planning Pressures I：模块化

3.1 Workflow

案例场景：采购 GPU 工作站。一共 4 个核心压力：模块化、反馈、人工管控、长任务视野。

MODULARITY 模块化

Searches carried out separately. 各个供应商的查询任务可以拆成独立小块分开执行。

任务拆成：查询供应商 A、B、C、D，每一步搜索都是独立模块。

好处：可以单独执行、单独失败，不用把整个流程推倒重来。

FEEDBACK 反馈（replan 重规划）

None fits, so revise the plan. 查完 A/B/C/D 之后，发现所有供应商的配置全都不符合要求 → 拿到这个反馈，修改原有计划，再去查找新的供应商。

核心：执行结果返回信息，触发重新规划。

CONTROL 管控 / 人工审核

A person approves before the agent submits. 智能体不能直接决定，提交采购申请之前，必须经过人来审批确认。

作用：给智能体加一道人工安全阀，防止模型做出错误采购。

LONG HORIZON 长视野 / 长任务跨度

Requirements and progress persist across the whole task. 整个采购任务周期很长，从头到尾，需求信息、已经查到的进度都要持续保存，不能中途丢失。

例子：一开始定的 GPU 性能要求，在后面新增供应商检索、人工审批环节都要记住。

Workflow

1. 并行 / 依次独立搜索 A、B、C、D 四个供应商（模块化）
2. 核对所有配置，发现全都不满足需求（拿到反馈）
3. 智能体重新规划，去寻找新供应商
4. 找到合适配置后，提交给人审批（人工管控）
5. 全程记住采购需求、历史查找记录（长视野）

6. 审批通过，提交采购申请

3.3 Solution 1: Planner and Executor

3.3.1 Core Idea

核心思想：把智能体拆成**规划器 (Planner)** 和**执行器 (Executor)** 两个独立模块，各司其职，可以用不同模型、不同训练数据。

Example：找到最便宜的 4K 显示器

1. 视觉观察（Vision-Only Observation） 拿到网页截图画面，任务目标：找最便宜 4k 显示器
2. Planner 规划器（用 GPT-4o） 能看懂画面，但不擅长输出像素坐标。只负责用自然语言描述下一步动作：

在页面顶部搜索框，输入 “4k monitor”

3. Grounding 定位 单独询问：页面顶部搜索框对应的像素坐标是多少？得到坐标
4. Executor 执行器（7B 小模型，单独训练） 把规划器的文字指令，翻译成屏幕像素操作、输入文字，真正操作网页。

Tips：规划需要的是页面上有什么元素、它承担什么角色，接地（grounding）需要的是这个元素在哪个像素。

训练数据：规划可以从动作轨迹里学，接地可以从点击坐标里学。

代价：规划器必须把动作描述得足够清楚，否则执行器接不上接口。

3.3.2 Training the Planner

1. Grounded Plan Generation 用一个更强的教师大模型（Teacher LLM），把已经成功的任务轨迹，切分、标注，生成规划步骤。
2. Synthetic Plan Expansion 基于少量种子规划数据（Seed Plan Data），让教师模型扩充生成更多查询 + 规划样本。 最后拿这些标注好的轨迹，微调规划器模型、执行器模型。

3.3.3 Multi Agent Systems and Four Fixed Roles

流转逻辑： Reproducer （复现 bug，写出复现代码）成功 → 交给 Fault Localizer （定位代码错误位置）；Reproducer 失败 → 直接交给 Editor （修改代码）； Editor 改完 → Verifier （验证 bug 是否修好），验证成功则结束。但给代码任务预先分配固定角色的多智能体方案，效果很有限，每加一次交接，就多一次有损压缩。

两个主要缺点：

1. 验证器（verifier）只负责验证，它本身不能去定位 bug，没办法回头检查自己的判断。角色不能临时切换。
2. 智能体之间靠简短报告交接任务，丢失很多细节上下文，下一个智能体缺少关键信息，容易出错。

Overall

规划器 + 执行器分离：拆分职责，大模型做思考规划，小模型做底层像素 / 动作执行，是有效的模块化方案

但固定角色多智能体（写死分工） 有明显缺陷：角色不能灵活切换、交接报告丢失上下文，代码任务里效果一般。

3.4 Solution 2: SayCan

模块化的另一个用法：不拆角色，而是给每个技能一个现在能不能做的概率，再拿它给计划重打分。

思路：把经典规划的前置条件换成概率

Recall classic planning: actions have pre-conditions. Learn a value function for each robotic skill, which predicts probability of completing it. Multiply this value with the probability of the LLM producing the plan.

经典规划中动作是有前置条件的，为每一个机器人技能学一个价值函数，预测它被成功完成的概率。把这个价值乘上 LLM 给出该动作的概率。

Example：左列是 LLM 的对数似然（负数，越接近 0 表示模型越想选它）；

右列是 价值函数给的成功概率。

图上左侧的提示例子是 How would you put an apple on the table?

模型的回答 I would: 1. ___；重打分之后变成 I would: 1. Find an apple, 2. ___。

Part 4 Planning Pressures II：反馈

1. 重规划：执行中收到环境观测，遇到障碍动态修改计划
2. 思考vs行动：光堆token思考有上限，和环境交互更重要
3. 过度思考：想太多（分析瘫痪）反而降低解决问题的成功率
4. 思考vs探查：拿不准时，优先探查环境获取真实信息，不要盲猜
5. CTA置信校准：行动前显式评估不确定性，让模型识别自己不知道的地方，主动测试

4.1 Environmental feedback

Example：煮土豆，然后丢进回收垃圾桶

执行一步、拿到环境观测，遇到障碍就动态改方案，就是环境反馈驱动重规划。

1. t=0：LLM规划器一次性生成初始高层计划：找到土豆→拿起土豆→放进回收桶
2. t=5：智能体在环境里执行，观察反馈：找不到土豆，但看到冰箱。拿到这个新观测，模型重新修改规划：打开冰箱、拿土豆……
3. t=20：继续执行，再次收到反馈：找不到回收桶，但看到普通垃圾桶，再次重规划，改用普通垃圾桶存放土豆。

4.2 Thinking vs Acting

不要光靠空想推理，多和环境交互拿到真实观测更有用。

横轴：每条任务轨迹消耗token数量（越多=思考越多）；纵轴：任务成功率

3条策略：

1. Per-Step Budget Forcing：每一步强制固定token预算
2. Per-Step Best-of-N：每一步采样多条候选，挑最好一条
3. Interaction Scaling：增加和环境交互次数 结论： 单纯增加token、疯狂长思考，收益会慢慢饱和；和环境交互获取真实信息，能更好提升成功率。

4.3 Overthinking

模型的过度思考分数越高，能解决的问题比例反而越低，想太多会降低任务完成率。

Classical Example：

1. Analysis paralysis 分析瘫痪：无限拆解、反复分析，迟迟不下手行动
2. Rogue action chains 失控动作链 / premature disengagement：跑偏到无关步骤，或者中途直接放弃任务

4.4 Thinking vs Looking

不要仅凭内部猜测直接行动；拿不准的时候，优先探查环境获取真实信息，而不是在脑子里空想。

任务：找到价格最大的商品ID

Guess-and-Go（猜了直接做）：模型凭99%的自信直接写代码读取文件，赌文件格式是csv。一旦猜错直接失败。

Deliberate Testing（先探查测试）：模型意识到自己只有50%把握，先执行探查指令，看一眼文件真实格式，拿到真实信息之后再写最终代码。

置信度就是决策信号

两边的模型能力一样、任务难度一样，差别只在一件事：它对文件格式的置信度不同（99% vs 50%）。99% 时就直走（Guess-and-Go）；50% 时就先去环境里取信息（Deliberate Testing）。同一个模型可以按置信度切换策略，而先测那一路并不比猜那一路上算力更多。

4.5 CTA：Calibrate-Then-Act 行动前校准置信度

Baseline 基线方案（普通Prompt / RL智能体）

模型隐式地判断自己的置信度：靠模型内部知识、RL训练学到的统计，不会显式写出概率，直接选动作。

CTA：在做决策前，显式估算先验概率，把概率信息放进Prompt，模型基于概率做推理：

1. 先预估先验概率 $\hat{p}(Z|x)$（比如文件分隔符是;的概率0.85）
2. 模型读到这个概率，知道“我不能百分百确定”，从而主动安排测试去验证。

Part 5 Planning Pressures III：长时间任务

1. 长视界失败：长任务中**单点错误会连锁传递**，越往后影响越大
2. 自条件效应 self-conditioning：模型会拿自己之前的错误作为上下文继续推理，导致后续准确率持续下降；大模型能跑更长任务，但也更容易出现这个问题
3. 递归自适应拆解：难题自动委派子智能体递归拆分，委派动作本身可以训练

5.1 Vending-Bench 长时程失败

长时序任务的错误会不断累积传递

Example：Vending-Bench 自动售货机业务模拟

任务：定价、接单、补货；最长 222 个模拟天数，最多 2000 条消息（约 25M token）

核心痛点：长序列任务里，一个临时的小错误，会一路影响后面所有决策，越错越严重。故障连锁流程：

1. Premature restock 过早补货：环境提示商品暂时缺货，模型误判成 “已经发货”
2. Wrong inference 错误推理：把 “现在缺货” 直接推导成 “生意失败”，不再等待到货通知
3. Error persists 错误持续放大：后续产生罚金，最终触发业务终止、欺诈警告

Tips：并非是坏计划，如果是计划坏，那么换一份更好的计划就能解决；但这里坏的不是计划，而是每执行一步就把自己的历史当作事实这个机制。哪怕开局计划完美，第一个小错误依然会被继承下去。

5.2 Execution horizon 执行视野

1. 左图：单步准确率一点点提升，放到长任务上，任务总长度会指数级上涨
2. 右图：模型会基于自己之前的错误继续决策（self-conditioning 自条件效应）

绿色线：理想预期的准确率；红色线：真实观测，越往后准确率持续下滑

结论：更高的单步准确率，能支持更长的任务；但自条件效应，会让模型后续步骤因为前面的错误而越做越差。

Better step accuracy extends the horizon, while self-conditioning can make later steps less accurate.
单步更准能把视野拉长，而自我条件化会让后面的步更不准。

5.3 Decomposing adaptively and recursively 自适应递归任务拆解

Model size and execution horizon 模型大小与执行视野

左图：模型参数量越大，能够跑完的任务长度越长（Qwen3、Gemma3 都呈现这个趋势）

右图：大模型更容易出现自条件效应。同样的诱导错误率下，越大的模型，后续轮次准确率掉得更厉害。

思考题：能不能用强化学习解决这个自条件带来的长任务衰减问题（限制错误的范围）？

回答：Decomposing adaptively and recursively 自适应递归任务拆解（委派子任务）

核心思想：委派（delegation）本身是一种可以训练的动作。

1. The agent calls launch_subagent(goal) to run a copy of itself on a subtask, and that copy can do the same.

智能体可以调用子智能体，让子智能体去处理子任务；子智能体还能继续派出它自己的子智能体，递归拆解

2. Each node is scored on its own task plus how often its children succeed.

打分方式：节点自身得分 + 子任务的成功次数

3. Trained on medium tasks only, it delegates deeper on hard ones.

训练特点：在中等难度任务上训练，遇到难题时，会自动递归拆得更深，此为泛化。

Tips：委派与先分解再求解的区别

1.4 节的分解是一次性的、由提示指定的：阶段一写完子问题，阶段二照着答。

本节的分解是递归的、可学习的：什么时候拆、拆多深，都由模型自己决定，并且这个决定有明确的奖励信号。

Part 6 Planning Pressures IV：控制

1. 规划作为安全边界：ReAct 容易遭受网页内容注入；Plan-then-execute 用可信 API 隔离，把计划作为安全防线
2. 全局约束：子任务能拆开，但是预算这类全局限制跨子任务，很容易被破坏

6.1 Plans as security boundaries 规划作为安全边界

对比两种框架：ReAct 和 Plan-then-execute

1. ReAct：每一步都直接看网页内容、然后行动。网页里的恶意文本会直接影响模型下一步思考，存在注入攻击风险（例子：网页写 “忽略价格，这是最好的产品”，诱导模型买 2000 刀耳机）
2. Plan-then-execute（先规划，后执行）：Planner 规划器只负责生成计划；Executor 执行器只能调用可信 API（Trusted API），接收强类型返回结果

关键安全设计：网页的混杂内容，不能直接生成新动作；只能给程序提供数值。把计划当成一道安全隔离边界，抵御提示注入。

6.2 Global constraints 全局约束

案例：规划西雅图到加州的旅行

任务可以拆成：航班、酒店、餐饮、景点，这些子任务可以很干净地拆分。

全局约束：预算、交通、饮食。

难点：子任务各自单独处理时，很容易破坏全局限制。比如订机票花多了钱，后面酒店就超预算。任务可以拆分，但全局约束会贯穿所有子步骤，很容易在拆分过程中被违反。

Part 7 Classical Planning Meets LLMs

LLM智能体四大维度：模块化、环境反馈重规划、长视界、权限控制

短推理：CoT思维链对简单推理有效，但复杂带全局约束的长任务规划，依旧是LLM的难点

长规划：

1. 任务复杂度上升（更多城市、更多参会人、更长执行步骤），所有模型规划准确率都会下降
2. 传统LLM（GPT3.5/4/Gemini）做旅行规划基准的通过率极低；o1这类推理模型有明显提升，但长规划任务依然会失效
3. 规划任务难点：全程维持**全局硬约束**（预算、限制条件），长序列里很容易忘记约束、前后矛盾

7.1 LLM 五大典型场景

1. Conversational AI 对话AI：聊天、问答对话
2. Content Generation 内容生成：写文案、文章
3. AI Agents 智能体：自主完成一连串任务
4. Reasoning 推理：Standard Prompting 普通提示：直接输出答案，容易算错；Chain of Thought（思维链）：让模型写出中间推导步骤，推理正确率大幅提升，但CoT解决不了复杂长任务规划。

5. Planning 规划：拆解目标，制定分步执行方案。

7.2 LLM planning is still a hard problem

example：旅行规划

用户需求：11.6 - 10从西雅图去加州，预算6000美元，整套房子住宿、必须宠物友好。 旅行规划任务拆成3大块：

1. Information Collection 信息采集：调用工具查询城市、航班、距离、住宿信息
2. Planning 规划：要同时满足硬约束（预算、房型、宠物友好）+常识约束
3. Delivery Plan 输出完整行程方案

难点： 旅行规划是带大量全局硬约束的长任务，需要多次调用工具、全程守住预算/条件；LLM很容易顾此失彼，这就是LLM现在短板。

哪怕是GPT4-Turbo、Mistral、Gemini这类当时最强模型，直接做旅行规划的最终通过率很低。并且用SOTA大模型，硬约束长行程规划依然很难；o1推理模型相比传统GPT4有明显提升，但没完全解决。

Two-stage：先调用工具，再做规划

Sole-planning：纯模型自己规划，不用真实工具。Final Pass Rate，GPT4-Turbo也只有0.6%，非常差。

Ground-truth tool use（使用真实、准确的工具返回结果再规划），DirectGPT-4-Turbo能到4.4%，但依旧很低。

o1-mini、o1-preview 相比GPT4系列，在旅行规划基准上大幅提升，但依旧不完美，随着约束变复杂，依然会翻车。

结论：

1. Trip planning 旅行规划：横轴=要去的城市数量。城市越多（任务越长），所有模型准确率一路下跌。
2. Meeting planning 会议规划：横轴=参会人数。人数越多，规划成功率快速下降。
3. Mystery Blocksworld 积木世界规划：横轴=计划的步骤长度。计划越长，正确率断崖下跌。
4. o1系列（LRM大推理模型）整体显著优于普通LLM，但计划一旦很长，照样掉点。

Part 8 Three Solutions

1. 路线 1 Scaling Law：单纯堆大模型、数据、算力。昂贵，很难单独搞定复杂约束规划。
2. 路线 2 Hybrid 混合系统（LLM+Solver）：LLM 理解自然语言，Solver 负责严格约束优化（如 MILP）。可以保证方案满足全部硬约束，旅行规划任务效果好。
3. 路线 3 符号涌现 + PDDL：LLM 翻译成 PDDL 规划语言，交给传统符号规划器做启发式搜索。

   评估智能体用 APEC 四个维度：准确率、主动性、高效、可信度；可以用 Agent-as-Judge 做自动评估。

8.1 三条路线的总览

路线	概括
Option One: Scaling Law	继续堆数据、堆算力、堆参数。
Option Two: Hybrid System	把 LLM 和符号求解器混合在一起。

路线	概括
Option Three: Emerging Symbolic Structure from Neural network	等模型自己产生符号结构。

区别：符号结构从哪里来。 路线一是它从规模里自发长出来（converge to anything symbolic）；路线二由人事先写死（求解器、约束、API）；路线三是它从神经网络的表示里产生出来。

8.2 前两条路线：缩放定律 vs 混合系统

8.2.1 Scaling Law 缩放定律

思路：堆更多数据、更多算力、更大的模型。

图曲线：参数量越大、算力（FLOPs）越高，训练损失不断下降。

缺点：成本极其昂贵，并且只靠堆规模性价比差，规划问题很难只靠大模型搞定。

8.2.2 Hybrid Systems 混合系统（大模型 + 求解器 Solver）

三种混合架构：

1. LLM 调用工具 → Solver 负责计算求解
2. Solver 产出数据，用来训练 LLM
3. End2End 端到端：深度模型 + 求解器融合在一起，联合训练

核心思想：LLM 擅长自然语言理解；Solver 擅长严格约束、组合优化。二者互补。

Example：TTG（To - the - Globe）

语言驱动的有保证旅行规划（LLM + MILP 整数规划求解器）

TTG 的思路：不让大模型做 “规划求解” 这件难事，LLM 只做翻译，求解交给 MILP 混合整数线性规划符号求解器。

把自然语言的旅行需求翻译成形式化描述，再交给混合整数线性规划求解器。

1. 用户输入：自然语言旅行需求。

> 示例：西雅图去加州，11.6 - 11.10，预算 6000 美元，整套房间、宠物友好住宿。

2. Instruction Translator 指令翻译器（微调后的 LLM）。

> 任务：自然语言 → 符号 JSON 规格描述

提取出发地、目的地、时间窗口、预算、住宿偏好；读取外部数据库，把候选航班、酒店信息填充进符号结构；

3. 送入 MILP 混合整数线性规划求解器

拿到全部符号约束，数学求解，输出数学保证最优、并且 100% 满足所有约束条件的行程方案。

> Guaranteed（有保证）的含义：求解器数学上保证解满足全部约束，不是 LLM 猜出来的。

4. 输出层：把求解器输出的符号最优解，再转成人类可读的自然语言行程给用户。

> LLM 翻译：2.508s（整个系统瓶颈）；
>
> MILP 求解总共仅：0.575s 大部分时间耗在 “人话转符号”，求解器本身算得很快。

局限

1. 依赖数据库内的航班、酒店候选集合；数据库之外的选项处理不了。
2. 全部压力落在 LLM 翻译模块，翻译出错，整套方案失效。
3. 问题必须可以写成 MILP 数学形式，不是任意规划问题都可以套用。

TTG（To - the - Globe）是 LLM+MILP 的神经符号混合旅行规划系统。大模型仅将用户自然语言需求翻译成符号规格，由 MILP 求解器输出满足全部约束的最优行程；正确性保证来自求解器，但系统风险来自 LLM 的翻译错误。

核心痛点：LLM 自己很难处理一大堆约束，但组合求解器可以严格满足约束。数据库里存放航班、酒店真实信息，求解器能保证方案满足所有硬约束，不会出现 LLM 忘记预算这类幻觉。

实验结果（人工评估 NPS 净推荐值）

三个维度用户满意度：完全满足需求：+40.0；性价比：+35.1；高效性：+36.9 。说明这套 LLM+MILP 混合方案，用户体验很好。

8.3 Multi-round Dialogs 多轮对话

用户经常有**隐藏约束**（没主动说，需要主动挖掘，比如无障碍需求、舱位偏好）。

The user has hidden constraints. How to figure them out? → Proactively ask!

智能体不能等用户一次性说完，要主动提问，挖出隐藏条件。

例子：普通 Llama 3.1-8B 准确率 0.231；APEC-Travel 智能体 0.279，更强。

8.4 APEC Agent Constitution 智能体评估 4 大指标

评估智能体不能只看最后答案好不好，**过程也要打分**

1. **Accuracy 准确率**：最终方案质量，能不能答对问题

The quality of the final solution that the agent provides (e.g., number of questions that are answered correctly). 最终解的质量，例如答对的问题数

2. **Proactivity 主动性**：能不能主动收集必要信息，主动提问挖掘隐藏约束

Whether the agent proactively collects useful information to solve the task. Such information may be public or private, vague or precise, explicitly provided or inferred from requests. 是否主动收集有用信息；这些信息可能是公开或私有的、含糊或精确的、明说的或从请求里推出来的

3. **Efficiency 高效性**：用最少轮对话、最少工具调用完成任务

Whether the agent can achieve its goal with a minimal number of interactions (e.g., number of questions asked, API calls and tool uses). 用最少的交互次数达成目标，例如提问数、API 调用数、工具使用次数

4. **Credibility 可信度**：幻觉、前后矛盾的多少，结果是否稳定可靠

The reliability with which agents achieve positive outcomes (e.g., amount of hallucination and inconsistency). 稳定达成好结果的能力，例如幻觉与不一致的量

用Agent Constitute 微调模型

Iteration2 第二轮微调效果最好： 总智能体分数 12.95，整体准确率、关键约束准确率、效率、主动性都明显上涨。 证明：用 APEC 这套评估指标做微调，可以实实在在提升智能体能力。

8.5 Judge Agent by Agent

评估方案演进：

1. LLM-as-a-Judge（普通大模型做裁判）
2. Agent-as-a-Judge（智能体当裁判）
3. Human-as-a-Judge 人类评估 Agent 裁判本身可以调用工具、做规划，比单纯 LLM 更适合评估复杂智能体任务。

左：LLM-as-a-Judge（对话任务 Conversation Task）

任务：If the FED buys bonds in the secondary market： (A) the money supply will increase. (B) the money supply will decrease. (C) the money supply will not be affected. (D) the discount rate would be affected. Output the answer only.

LLM A 答：The Federal Reserve buys bonds in the secondary market to increase the money supply.

LLM B 答：(A) the money supply will increase.

Judge LLM 判词：B is better because it gives the correct option directly, following the required format of the question.

Compare-based Judge 做的是相对判断：只回答A 和 B 谁更好，并给出理由（gives the correct option directly, following the required format）。

Metrics-based Judge 做的是绝对打分：两个模型 Accuracy 都是 10/10，差别只在 Instruct-Following（5/10 对 10/10）。A 的答案内容正确，但没按只输出答案的格式要求写，这一条只有指标裁判看得出来。

中：Agent-as-a-Judge（智能体任务 Agentic Task）

任务：Hi! Please follow the instruction and set up the script from the blog https://www.factsmachine.ai/p/hidden-in-plain-sight to generate 1080p images with hidden text "FUTURE," in src/visualize.py. Save them in results/ and verify the text is embedded.

Agent A 与 Agent B 各自有一个 Workspace；中间是共享的 Environment。两侧的 Act 箭头都指向 Environment。

Judge Agent 的判词：B is better because it runs and generates the required files, fulfilling the user's request. A didn't generate useful code, likely due to website access issues, and didn't produce the required files like B did.下面同样分 Compare-based Judge 与 Metrics-based Judge 两层。

右：Human-as-a-Judge（智能体任务 Agentic Task）

右栏的任务与工作区布局和中栏完全相同，只是 Judge Agent 换回了 User。

8.6 第三条路线：PDDL 规划域定义语言

问题：大模型在训练后，能不能自发涌现出符号化的结构（Emerging Symbolic Structure）？

没有给答案。这条路线的想法：不外接 Solver，希望 LLM 自己学会类似符号规划的能力，直接做规划。

引出规划领域标准语言：PDDL Planning Domain Definition Language，规划域定义语言

TTG 和 PDDL 的异同

相同：都是LLM 翻译成符号描述，交给现成外部求解器求解范式。

区别：TTG 用MILP 混合整数线性规划，适合组合优化、预算最优这类连续 + 离散混合问题；PDDL 多用于经典动作规划（积木世界、机器人动作序列）。

8.6.1 PDDL 基础概念

Planning Domain Definition Language：不承诺任何方案，只提出一个问题。

1. **Precondition 前提条件**：执行这个动作必须满足的状态，条件不满足动作不能执行

must hold to ensure the action can be executed；defines the states in which the action can be executed；action is applicable in state s if the preconditions are satisfied by s

2. **Effect 效果**：动作执行之后，世界状态发生什么变化

represent the effects of action on the world；defines the result of executing the action

Add list：动作之后新增成立的事实 (the fluents in) the positive literals in the action's effects　例：{At(p, to)}

Delete list：动作之后不再成立的事实 (the fluents in) the negative literals in the action's effects　例：{At(p, from)}

3. **状态更新公式**：执行动作，删掉 Delete 里的内容，加上 Add 里的内容。

Result of action a in state s: $ RESULT(s,a) ≝ (s \ DEL(a)) ∪ ADD(a)$

例子：Fly 飞机动作，Fly(P1, SFO, JFK) ⟹ remove At(P1, SFO), add At(P1, JFK)。

前提：飞机在 SFO；效果：飞机不在 SFO，出现在 JFK。

8.6.2 Example

Action schema 动作模板

前提：p 是飞机，from/to 是机场，飞机 p 在 from 机场

效果：飞机不在 from，飞机出现在 to

Action instantiation 动作实例化：把 p、from、to 填成具体值，比如 Fly (P1,SFO,JFK)

8.6.3 求解：启发式搜索 Best-First Search

规划本质就是状态空间搜索，PDDL 用来描述状态、动作、目标，搜索算法找到动作序列。

g：从起点走到当前节点的实际代价

h：启发函数，预估当前节点到目标的剩余距离

w：调节的是g和h之间的比例

f = g + w · h：用已知代价加上一个被缩放过的猜测

w=1 → A * 搜索：h 满足可采纳性时能给出最优解；

w>1 → WA（加权 A）：wA* 更偏向 h，更快但不再保证最优；

w → ∞ 时 GBFS 只看 h、完全不管已经花了多少代价，最快也最不稳。

w 趋向无穷大 → GBFS 贪心最佳优先搜索

8.7 用 LLM 解决 PDDL 规划的三类范式

自然语言规划问题，交给 LLM，有 3 大类做法：

1. LLM as search components generator：大模型生成搜索组件代码，交给黑盒搜索算法执行
2. LLM as policy generator：大模型生成领域专用规划策略代码，直接输出动作
3. LLM as PDDL generator（最常用）：LLM 把自然语言问题翻译成 PDDL 符号描述，再调用现成经典规划器（off-the-shelf planner）求解

角色	LLM 产出什么	谁来执行	交付
搜索组件生成器 LLM as search components generator	Search components code (black box model) 搜索组件的代码（把模型当黑盒）	Search algorithm 搜索算法	交代码：LLM 写出一段搜索程序的组件（启发函数、剪枝规则之类），搜索本身还是由算法跑。
策略生成器 LLM as policy generator	Policy code (domain-specific planner) 策略代码（领域专用规划器）	代码本身	交策略：LLM 直接写一个领域专用规划器，规划能力被固化进代码里。这一档对模型的要求比第一档高。模型在这条路上不被信任到能自己搜，只被用来生成搜索用的零件。
PDDL 生成器 LLM as PDDL generator	High level planning model (PDDL) 高层规划模型（PDDL 描述）	Off-the-shelf planner 现成的经典规划器	交规格：LLM 只把自然语言问题翻译成 PDDL 描述，求解交给 Off-the-shelf planner。这一档对模型的信任度最低，也最可靠。

5584 LLM for Agent 2

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_5/knowledge/5584-LLM-for-Agent-2.md`

5584 LLM for Agent2

Lecture 6 Tool Use for LLM Agents

章节	内容
1 Capabilities	tools add retrieval, execution, generation, and application actions. 工具给模型加上检索、执行、生成与应用动作
2 Mechanics	schemas enter the prompt; model protocols emit calls; harnesses validate, dispatch, and match results by ID. schema 进提示、模型按各自协议吐调用、harness 校验并派发、按 ID 配对结果

章节	内容
3 Constraints	JSON Schema and grammar masks guarantee form—not semantic correctness. JSON Schema 与语法掩码只保证形式合法，不保证语义正确
4 Interfaces	Direct REST calls and MCP are alternative ways to expose capabilities and manage credentials. 直连 REST 与 MCP 是暴露能力、管理凭据的两条路
5 Systems	orchestrate dependencies and concurrency; evaluate selection, arguments, outcomes, cost, and operational failures. 编排依赖与并发，并评估选择、参数、结果、成本与运行故障

本讲的三个阶段

第一阶段（Part 1 - 2）：工具是什么、调用怎么走 —— 能力与机制。

第二阶段（Part 3 - 4）：怎么保证调用能用、外部能力怎么接 —— 约束与接口。

第三阶段（Part 5 - 6）：多个调用怎么协同、怎么度量 —— 系统与评估。

Part 1 Capabilities 工具的能力

1.1 Tool Definition

A tool is an interface through which a language model can invoke an external computer program.

工具是一个接口（interface），语言模型通过这个接口去调用一个外部计算机程序。接口说明它是一份契约，不是一段能力与外部说明程序不在模型参数里，模型无法凭自身参数完成这件事。

The model proposes that external software decides whether and how to execute it.

责任分界线：模型只提议，无法绕过 harness 直接产生副作用；是否执行、如何执行，由外部软件决定。

1.2 Benefits of Tools

动词	中文	说明
扩展 Extend	Access information or actions unavailable in the model's parameters. 访达到模型参数里根本没有的信息或动作	Extend 解决的是能力边界：模型参数里没有今天 PolyU 发了什么公告，再大的模型也答不出来
分担 Facilitate	Delegate operations external programs perform more reliably or efficiently. 把外部程序做得更可靠或更高效的操作交给它们	解决的是可靠性：18 + 23 + 41 + 54 的平均数模型其实会算，但会算错；交给程序算是为了让错误率从偶尔降到几乎不可能。

1.3 Example of Tools 典型工具

Search 检索：把模型不知道变成去查一下。

Calculator 计算器：把模型算不准变成交给确定性的程序算。

Browser 浏览器：把读到静态文本变成在网页上操作。

Python 代码解释器：一个能表达任意控制流的通用工具。

API 外部服务接口：把私有数据与业务动作接进来。

Need	Example tool	Benefit
Current information　当前信息	Search	Fresh evidence　新鲜证据
Exact computation　精确计算	Calculator or Python	Reliable execution　可靠执行
Private state　私有状态	Account API	Authorized application data　经授权的应用数据
External action　外部动作	Browser or API	Changes the environment　改变环境

工具并不是越多越好，把 30 个工具一次性塞进提示，会同时抬高四类代价：提示更长（延迟与成本）、候选更多（选错工具的概率上升）、权限面更大（风险）。

Call tools only when their benefit outweighs latency, cost, failure, and risk. Start with the abilities we want, then define a narrow interface for each kind of interaction.

只有当工具的收益超过延迟、成本、失败与风险时，才去调用它。从想要的能力出发，为每一类交互定义一个窄接口（接口窄，则参数少、校验简单、权限小）。

- latency 延迟：一次工具往返要多久？串行四次就是四倍等待。
- cost 成本：检索调用、API 调用通常按次计费，长任务里会累积。
- failure 失败：工具可能超时、限流、返回畸形数据，harness 要有兜底。
- risk 风险：对外部动作类工具（External action），副作用不可撤销。

第一类：harness 控制动作 finish(text)

Return the final text and stop the agent loop. This is a harness control action, not an external program.

智能体循环需要一个出口。返回最终文本并停止智能体循环。这是一个 harness 控制动作，不是外部程序。

第二、三类：检索、计算与生成

三类工具的共同骨架，三者的返回类型不同：

检索类返回的是文本证据（Three matching news items...）；计算类返回的是一个确定性数值（34.0）；生成类返回的是一个文件引用（generated_image.png）。

信息检索 search_web(query)

Retrieve information from the web. 从网上检索信息。

代码执行 execute_code(code)

Run a program in an isolated environment and return its output. 在隔离环境里跑一个程序并返回输出。

图像生成 generate_image(query)

Generate an image from a textual description. 从文字描述生成一张图。

第四类：应用专属工具 create_grocery_cart(items)

前三类工具是通用的（谁都能用），第四类是专属于某个应用的。

Expose an application-specific capability through one typed call.

用一次带类型的调用暴露一个应用专属的能力。

Tips：重点：带类型

好处：模型不需要读懂返回内容，只要按字段取用；下游代码也不需要解析自然语言，可以直接断言 items == 3、total == 31.42。这也是生成时约束的东西：JSON Schema 管住了输入的形式，而工具返回结构的约定管住了输出的形式

第五类：代码作为元工具 execute_code(code)

前四类工具都是一个工具做一件事，而代码是工具之上的工具，所以叫 meta-tool（元工具）。

Code combines control flow, libraries, and multiple operations into a single action.

代码把控制流、库与多个操作组合进一个动作里。control flow 是循环与分支（面包题里的连减）、libraries 是外部库（pandas）、multiple operations 是多次操作的编排（视觉问答三步）。

1.4 程序化组合：同一任务的两种智能体

左边：传统 LLM Agent（Text / JSON as Action，典型就是 ReAct）

Action = JSON 结构化文本调用工具，一次 Action 只能调用 1 个工具

流程：`Think → JSON Action调用单个API → Environment返回结果 → 重复循环`

任务：计算 4 个国家手机 CodeAct 1 的美元总价，找出最便宜国家

1. 第一轮：`lookup_rates(country=Germany)` → 拿到汇率、税率
2. 第二轮：`lookup_phone_price(model="CodeAct 1", country=Germany)` → 当地售价
3. 第三轮：`convert_and_tax(price, rate, tax)` → 换算 + 计税
4. 还要单独调用`estimate_shipping_cost`、`estimate_final_price`
5. 然后重复全部步骤，逐个国家（USA/Japan/India）循环调用 缺点：

- 需要非常多轮 LLM 调用、多步交互，每一步只能执行单一工具调用
- 大量重复的 Think-Action-Observe 循环，token 消耗高、速度慢
- 循环逻辑、批量计算、`min()`这种聚合操作都无法一步完成

右边：CodeAct Agent（Code as Action，代码作为动作）

Action = 一段可执行 Python 代码，单段代码内可以连续调用多个 API、写循环、保存中间变量、调用内置函数

优势（图中标注）

1. Fewer Actions Required：更少动作 / 更少 LLM 轮次。一次性写循环遍历全部 4 国，一次性完成全部 API 调用、计算、比较，不需要一步一步逐个工具交互
2. Control & Data Flow of Code Simplifies Complex Operations：代码自带循环、变量存储、分支逻辑，天然支持复杂流程
3. Re-use `min` Function from Existing Software Infrastructures(Python library)：直接复用 Python 内置函数，不用让 LLM 手写比较逻辑

	Text/JSON Action Agent (ReAct)	CodeAct (Code as Action)
Action type	JSON / 结构化文本	可执行 Python 代码
单次 Action 能力	只能调用单个工具	可以调用多个工具 + 循环 / 变量 / 内置函数
Interaction rounds	Many（多轮）	Much fewer（少轮次）

	Text/JSON Action Agent (ReAct)	CodeAct (Code as Action)
Strength	简单任务，易解析	复杂计算、批量任务、多步骤逻辑
Weakness	长任务下大量交互，容易出错	需要代码沙箱执行环境，有安全风险

CodeAct：LLM Agent 把动作表达为可执行代码，而不是 JSON 工具调用。

论文结论：代码作为动作，表达能力更强、减少交互轮次，处理多步计算类任务表现更好。

代码的两面性：动作空间宽带来的三个结果

维度	内容	对策
强 Expressive	Loops · variables · libraries 循环、变量、库 表达能力来自动作空间宽	Good for multi-step data manipulation and composition.
难管 Harder to constrain	Broad action space 动作空间宽 难约束也来自动作空间宽	Validation is less precise than for a narrow typed function.
危险 Higher impact	Files · network · processes 文件、网络、进程 高影响同样来自动作空间宽	Use sandboxing, resource limits, permissions, and audit logs.

四条对策：

sandboxing（沙箱）把执行环境隔离，让副作用出不了笼子。

resource limits（资源限额）限制 CPU、内存、时长，防住一个死循环拖垮整个 agent。

permissions（权限）决定这段代码能碰什么，是最小权限原则的落地。

audit logs（审计日志）出事后能回答它到底做了什么，这也是合规的前提。

Part 2 Mechanics 工具的机制

工具调用本质上就是**特殊标记（tokens）**，是大模型生成文本序列的一部分。

2.1 工具调用作为 token 序列

LLM 生成有两种输出分支：

1. Text continuation（普通文本续写）：正常自然语言回答，例：The weather is sunny.
2. Tool continuation（工具调用续写）：模型输出特殊控制标记，触发工具调用，例：<tool_call> get_weather(.)

模型不区分 “回答文字” 和 “调用工具”，两者都是模型生成的 token。 当模型输出工具标记，外部执行框架（harness）捕获到这个动作，去执行工具。

messages become model input

API 的多轮消息（system /user/assistant）不能直接喂给模型，需要通过 chat template 序列化为一整条 token 输入序列。

- API messages：按 role 组织（system、user、assistant）

- Chat template：加上特殊分隔符标签（如 Qwen 的<|im_start|>、<|im_end|>）拼接成一串文本，作为模型输入。

2.2 Tools add structure to the request 给请求增加结构化信息

Tool definition：提前把工具的 JSON schema 传给模型。定义工具名称、参数类型、必填字段（比如get_weather需要city字符串参数）

API request：调用 chat completion 接口时，除了messages，额外传入tools=[tool_schema]

框架把消息 + 工具定义一起送入模板，暴露给模型阅读。

Definition in, tool call out

Prompt 内部包含<tools> ... </tools>，里面是全部工具的 JSON 定义。 模型读到工具定义后，生成带标记的工具调用输出：<tool_call> {name, arguments} </tool_call> API 会给这个调用分配唯一call_id。

2.3 Same tool call, different model protocols

同一工具调用，不同模型有不同协议格式

One schema; model-specific serialization and parsing.

模型	协议名	形态特征
Qwen 3.8	Function + parameter tags	层层嵌套的标签：函数名与参数名都做成标签
Mistral Small 4	Control token + call ID	一个控制 token 加一段 JSON（JSON 里带 id）
DeepSeek V3.2	DSML invocation block	一套 DSML 标签，参数上还带类型标注

工具的底层 schema 是统一的，但是不同模型家族，使用不同特殊 token / 标签序列化工具调用。

重点：每个模型有自己专属的控制 token 格式。Agent 框架必须为不同模型写对应的 parser，把模型输出的文本解析成统一的{name, args}结构。

2.4 OpenHands 工具调度流程

对照项	初始化阶段	每次调用阶段
图上编号	1 Register at initialization	2 Dispatch each model call
发生几次	一次	每次模型调用一次
主要动作	resolve_tool(spec, state)	Parse → Lookup → Validate → Execute → Return
产出	tools_map[name]	ObservationEvent(call_id)
单次代价	高：解析 schema、构造类型、准备执行器	低：查表、校验、执行

两大阶段：

Register at initialization（初始化注册阶段）

1. 输入 Tool spec（工具描述，如 BashTool）
2. resolve_tool(spec, state)：解析、配置工具
3. 存入tools_map[name]：字典映射，工具名 → 工具定义 + 执行器 executor

Dispatch each model call（每一次模型输出后的调度）

1. Parse：解析模型输出，得到{name, args}
2. Lookup：在tools_map查找对应工具定义
3. Validate：校验参数合法性（参数类型、必填项）
4. Execute：调用 executor 执行工具 action
5. Return：封装结果为ObservationEvent(call_id)返回

步骤	表示	作用	失败后果
Parse	{ name, args }	从模型输出里抽出函数名与参数	The harness cannot parse it.
Lookup	tools_map[name]	按名字查阶段一注册好的表	查不到就是未知工具
Validate	action_from_arguments(args)	把裸参数转成强类型 Action	类型或必填项不符就校验失败
Execute	executor(action)	真正执行，产生副作用	抛异常则变成一次失败结果
Return	ObservationEvent(call_id)	把结果打包成事件，带上call_id	不带 ID 就无法配对

2.5 通过 call ID 返回工具结果

结果里为什么一定要带 call_id?

用来绑定工具调用和返回结果，支持并行多工具调用（parallel tool calls），保证结果对应正确的调用。

一次模型响应可能带多个调用，harness 可能并发执行它们，完成先后取决于各工具快慢，而不是发出顺序。如果结果按第几个回来配对，一旦乱序就会把 A 的结果填给了 B，而且这种错误不会报错，只会让模型拿到错的数据继续往下推理。只有 ID 能让结果回到正确的调用上。

Example：若同时发出两个调用（例如查两个城市的天气），harness 可能并发执行，谁先完成取决于网络与工具耗时。

没有 ID：只能假设先发的先回，一旦乱序就把 A 的结果填进了 B。这种错误不会抛异常，只会让模型拿着错的数据继续推理，这是最难排查的一类 bug。

有 ID：无论谁先回来，都能放回正确的位置。

2.6 小结

工具调用完整流程：

1. 系统将工具 schema 渲染进 prompt，放在<tools>标签内送入模型。
2. 模型读取工具定义，按照自身专属协议输出<tool_call>（携带工具名、参数、唯一call_id）。
3. 外部执行框架 harness 处理： Parse：解析模型输出；Lookup：查询注册好的工具映射表；Validate：校验参数合法性；Execute：运行外部工具；Return：封装工具返回内容。
4. 工具返回tool 角色消息：role="tool"，tool_call_id与之前调用的 id 匹配，携带结果content。
5. 把工具结果放回对话上下文，模型读取观测结果，生成最终自然语言回答。

完整一轮工具交互链路：

1. assistant：tool_call（发起调用，带 call_id）
2. tool：tool-result（返回观测，匹配 call_id）

3. assistant：最终文本回答

机制要点：call_id 的作用是绑定调用和返回结果，支持并行工具调用；工具调用本质是模型生成的特殊 token，不同模型协议标签格式不一样。

易错：

1. 模型不知道自己的 call ID；
2. tool call 不是函数调用，是一段字符串；
3. 结果靠 tool_call_id 配对，不靠顺序（including parallel calls）。

Part 3 Constrained Decoding 工具的约束：只保证形式合法

JSON Schema 校验工具调用的结构合法性；PDA 用来处理 JSON 嵌套；Token Masking 在解码阶段提前阻止模型输出非法 token，减少 malformed tool calls。

事后校验与事前约束的分工

做法	对应	时机	代价
事后校验	The harness cannot parse it.	整段生成完之后	整段输出作废，需要重试，多一次完整往返
事前约束	Constrained Decoding	生成的每一步	每步多一次掩码计算（CPU 开销）

解码层保证结构上能解析：畸形括号这类问题。

harness 保证业务上能用：Validate（action_from_arguments）。

但要注意约束的射程：受约束解码消除的是形式类错误。{"city":"PittsburghX"} 是合法 JSON、类型也对，城市名却是错的，那已经越出形式约束的范围，要靠Values 甚至更外层的业务校验。

3.1 Malformed tool calls 格式错误的工具调用

模型输出语法破损的 JSON，例如少闭合大括号，harness无法解析，直接调用失败。

Tool-call constraints 工具调用的四层约束（后一层以前一层为前提）

约束	约束	作用	例子
Syntax	valid JSON	合法 JSON	缺一个 } 就不是 valid JSON
Shape	expected fields	字段符合预期	name, arguments 等必填 key 存在
Types	city is a string	参数类型匹配 schema	city 必须是字符串 string
Values	值在允许范围内	值在允许范围内	units 只能是C或F

Constraints ensure form—not truth, permissions, tool choice, or task success.

约束只保证格式合法，不能保证工具返回结果是真的，也不能保证选对工具。

四类约束执行时机

约束	解码时（掩码能做吗）	派发时（harness 做什么）
Syntax	能：字节级语法掩码	解析失败即拒：The harness cannot parse it.

约束	解码时（掩码能做吗）	派发时（harness 做什么）
Shape	能：required 编译进文法	缺字段则 Validate 失败
Types	能："type": "string" 进文法	action_from_arguments 里做类型转换与检查
Values	能：enum 限制候选取值	业务层可再校验一次
truth / permissions / tool choice / task success	不能	需要额外机制：接口与凭据、评估

Tips：加了 JSON Schema 就绝对安全吗?

真伪要靠工具自身返回真实数据，以及额外的交叉验证；权限要靠 harness 的白名单与凭据边界；选对工具要靠评估里的 selection 指标。任务成功要靠端到端的结果评测。

3.2 JSON Schema

Vocabulary for describing JSON data. Definite Types, required fields, allowed values

JSON Schema 是一套用来描述 JSON 数据结构的机器可读规则，定义字段类型、必填字段、允许枚举值（enum）、禁止额外字段。

示例：city:string，units: enum ["C","F"]，required:["city"]，additionalProperties:false

Schema-valid vs schema-invalid 是否可以推导出来derives

Valid：{"city":"Pgh","units":"F"} 结构、类型、取值全部匹配 schema

Invalid：{"city":"Pgh","units":"K"} JSON 语法没问题，但K不在允许枚举集合里，schema 校验失败

1. JSON valid：文本语法能被 JSON 解析器读
2. Schema valid：不仅语法对，数据结构、类型、取值完全符合预先定义的 schema

3.3 A pushdown automaton** **PDA，下推自动机

PDA = **有限状态控制 Finite control + 栈 Stack memory**

3.3.1 两个部件

部件	原句	记住什么	记不住什么
Finite control 有限状态控制器	The state records the local position in a grammar rule.	当前文法规则内部走到哪一步（expect value / inside value / expect separator）	嵌套层级有多深
Stack memory 栈存储	The stack remembers arbitrarily deep nesting.	每一层嵌套对应的结构类型（object /array），支持任意深度 arbitrarily deep	当前这条文法规则内部的局部位置

State tracks current rule; stack remembers where nested rules return.

状态跟踪当前规则；栈记住嵌套结束后要返回的上层规则。栈可以无限增长，有限状态做不到。

栈为空 = 所有括号配平。

底层原理： 纯有限状态机（正则表达式）无法表达任意深度的配对括号，只能处理固定深度嵌套。JSON 存在无上限嵌套，所以必须使用 PDA。

3.3.2 文法检查 Grammar checking happens below tokenization

语法检查运行在tokenization 底层，字节层面

- Token 边界 ≠ 语法边界：单个 token 可能跨多个语法终结符，或者在文法中间截断。不能把 token 边界当成语法边界。
- 流程：预先编译文法 → 构建字节级 PDA；对每一个候选 token，解码为字节，送入 PDA 校验。

示例：PDA 栈演算

1. 读取 `{` → push `object`，栈：`[object]`（深度 1）
2. 读取 `[` → push `array`，栈：`[object, array]`（深度 2，array 在栈顶）
3. 读取 `]` → pop array，栈：`[object]`（深度 1）
4. 读取 `}` → pop object，栈：empty（深度 0）

最终栈为空：括号配平； 缺`}`（malformed 例子）：栈停留在`object`，不为空，直接判定语法错误。仅机械栈操作，不需要理解语义。

3.3.3 PDA 能力边界

对接前面 4 类 tool-call 约束：Syntax / Shape / Types / Values

PDA 可以校验：

- Syntax：括号、引号配对（JSON 配平，本职）
- Shape：必填字段、合法字段名
- Values 里的有限枚举（`units ∈ {"C","F"}`）

上面这些都可以写成上下文无关文法的产生式

PDA **无法校验**：跨字段业务语义、外部事实知识

例：`units="C"` 时 city 必须是加拿大城市，需要外部知识库，不属于文法范畴。

Constraints ensure form—not truth, permissions, tool choice, or task success.

约束只保证**形式合法**，不能保证事实真伪、权限、工具选择、任务是否成功。

3.4 Token masking 令牌掩码

问题：非法 token 判定依赖 PDA 栈状态，如果每次生成 token 都完整重算，开销很高。

缓存掩码流水线思路：大部分 token 查表缓存，少数上下文依赖的 token 现场实时计算。

1. **Parse the prefix**

prefix = 已经生成输出的字节序列。persistent PDA stacks持久化栈，不需要每次从头重建栈，直接在已有栈状态上，用已经输出的字节向前推进状态。

解决：长序列每次重建栈带来的平方复杂度开销。

2. **Fetch cached decisions **

栈顶对应文法节点，索引预计算好的缓存表，直接查询上下文无关 token（context-independent tokens）是否合法。核心操作就是查表，速度很快。

对应图：Retrieve token mask cache from stack top / Adaptive Token Mask Cache

3. **Check the minority **

只对少量上下文依赖 token（context-dependent tokens），拿完整 PDA 栈现场校验。

union results across matching stacks：存在多分支、多条并行栈状态时，把多条栈得到的合法性结果做并集。

对应图：Context-dependent tokens (minority，黄色窄条)

4. **Mask and continue **

合并查表 + 现场校验得到的全部判定结果；对模型 logits 施加 token 掩码；采样选出一个 token；更新 PDA 栈。更新后的栈，交给下一轮Parse the prefix使用，四步形成闭环。

对应图：Complete Token Mask，输出 Sampled Token

三种实现策略取舍

策略	代价	缺点
全部缓存	按栈状态 × 词表 V预计算	栈状态组合爆炸，缓存表容量不可行
全部现场计算	每一个候选 token 都完整跑 PDA	词表 V 上万，CPU 计算开销巨大，拖慢 GPU 推理
缓存 + 少数现场计算	预缓存大部分，仅少数 context-dependent 实时校验	需要处理分支多栈、结果并集逻辑

定性代价模型

词表大小V； m = 上下文无关、可以查表的 token 占比； $1 - m$ = 少数需要现场计算的 token 占比。

- 全现算：每一步成本正比于 V
- 缓存方案：每一步成本 ≈ 一次查表 + 正比于 $(1 - m)V$ 的现场校验

加速幅度近似由m决定；m越大，加速效果越好。

课件只定性：成本随 minority 占比线性下降，不给出具体数值。

配图：绿色（允许）+ 红色（拒绝）占绝大部分词表；黄色上下文依赖 token 只是很小一部分，m很大。

persistent 持久栈的关键意义：

如果不使用 persistent 栈，每次生成新 token，把全部 prefix 字节从头重新跑一遍 PDA。总开销随生成长度平方增长，越长越慢；Persistent 栈只对新增字节推进栈状态，单步是常数复杂度。 所以这套四步流水线不只是用来生成掩码，还解决了长文本生成过程中掩码计算越来越慢的性能隐患。

LLM 推理输出 logits，token mask 是 0/1 掩码：

- 1：允许采样该 token；0：屏蔽，logits 置−∞，softmax 后概率归零
- 流程：LLM 输出原始 logits → 乘 mask 得到 masked logits → softmax 得到概率分布 → sampler 采样 token

3.5 Cached token-mask generation 带缓存的掩码生成，加速方案

1. Parse the prefix：用已经生成的字节，推进持久化 PDA 栈
2. Fetch cached decisions：栈顶节点直接读取预计算缓存，大部分 token 是上下文无关的，直接查表
3. Check the minority：少数上下文依赖的候选 token，用完整 PDA 栈实时校验
4. Mask and continue：合并掩码，屏蔽非法 token，采样新 token，更新 PDA 栈

Case study：加速来源

约束只管形式，加速靠缓存与重叠

1. **Caching**：绝大多数 token 是上下文无关，直接读取预计算 mask，不用实时跑 PDA（大头提速）
2. **Persistent stacks**：持久化 PDA 栈，支持分支、多候选路径
3. **CPU/GPU overlap**：CPU 做语法 / 掩码计算，和 GPU LLM 推理并行，不阻塞模型生成

设计	图上	含义
栈的持久性	Maintained by persistent execution stacks	栈跨步保留，不做每步重建（persistent）
栈的分支	Matching Stack States 里同时列出 [0]<array> 与 [3]<str>	不确定走哪条分支时同时维护多条栈，最后取并集（union results）
token 级查表	Retrieve token mask cache from stack top	查表的键是栈顶节点（Node ）
与推理重叠	Overlapping with LLM engine	CPU 侧的掩码计算与 GPU 侧的推理并行，不是串行等

每步总成本 ≈ cache 查询 + minority × 现场判定

Caching handles most tokens; 大多数 token 查缓存;

persistent stacks support branching; 多栈并行，使不确定走哪条分支不至于卡住;

CPU work overlaps GPU inference. 把 CPU 的工作与 LLM engine 重叠。

多数段直接查缓存（Context-independent: Fetched from cache (fast, majority)）

少数段运行时判定（Context-dependent: Checked at runtime (slow, minority)）

Part 4 REST API 工具接口

4.1 Library API

4.1.1 API 的本质

Python Library API 是程序内部暴露给其他代码使用的编程接口。

Caller and function share one process. Python supplies the name, arguments, types, and return type.

Python 库函数是带类型的编程接口，不是网络服务。示例 get_weather，调用方和函数共享同一个进程、共享内存地址空间。 Python 直接提供函数名、参数、类型、返回类型，直接传内存对象，不需要地址、协议、序列化、认证。

4.1.2 局限与前提

局限：不能远程调用，而进程内调用靠共享地址空间，跨网络没有共享内存。

所以有跨网络调用硬性需求：

1. `address`地址：远端服务的位置
2. `protocol`协议：两端交互报文约定
3. `serialization`序列化：内存对象编码为网络字节流
4. `authentication`认证：验证调用者身份

缺件	进程内为什么不需要	跨网络为什么必须有
address 地址	同一进程里，函数名就是入口	服务在另一台机器上，得先知道它在哪里
protocol 协议	Python 的调用约定就是协议	两端可能不是同一种语言，要约定报文格式与交互次序
serialization 序列化	参数就是内存里的对象，直接传引用	网络只能传字节，对象必须先编码成可传输的形态
authentication 认证	同进程内没有谁在调用这个问题	任何到达端点的请求都必须先证明自己是谁

Pydantic BaseModel 定义返回结构，这套类型标注会一路传递：Python 类型 → JSON 响应 → OpenAPI JSON Schema → MCP 工具 schema。

4.1.2 缺点

不能直接远程调用：进程内部可以直接使用 Python 函数调用机制。但如果调用方在另一台计算机上，就无法直接使用同一个内存地址空间。因此，远程调用必须解决四个问题：

问题	进程内部	跨网络调用
Address	通过函数引用找到代码	必须知道远程服务地址
Protocol	Python 调用约定	必须约定通信协议
Serialization	直接传递 Python 对象	需要编码为可传输数据
Authentication	由进程和应用权限控制	通常需要验证远程调用者

其中，Serialization（序列化）尤其重要。

例如，Python 对象：

```
Weather(    summary="Sunny",    temperature=75.2)
```

通过网络传输时，可以表示为 JSON：

```
{
  "summary": "Sunny",
  "temperature": 75.2
}
```

这里 JSON 是一种数据表示格式，不是网络协议本身。

本节结论：Library API 是 typed programming interface，而不是 network service。

4.2 REST APIs：通过 HTTP 暴露函数

4.2.1 REST API 的核心思想

REST API 使外部客户端可以通过 HTTP 请求访问服务端提供的资源和操作。

在课程案例中，我们希望将：get_weather("Pittsburgh", "F")

转换成远程请求：GET /weather?city=Pittsburgh&units=F

它们实现类似的业务功能，但使用不同的调用方式。

4.2.2 FastAPI 如何实现这一转换?

REST APIs expose functions over HTTP

把库函数挂到一个 URL 上，让它可以被一次 HTTP 请求触发。用 FastAPI 把本地库函数绑定到 HTTP 端点，对外提供网络调用能力。

元素	代码
GET /weather	@app.get("/weather", ...) HTTP 方法与路径
city =Pittsburgh	city: str 这个参数，以查询参数形式传进来（课件原样多印了一个空格）
units F 与孤立的 =	units: Literal["C", "F"] = "F" 参数名、取值、默认值被版式拆散了
Header: X-API-Key	APIKeyHeader(name="X-API-Key") 凭据走 HTTP 头，不走 URL
Response: Weather as JSON	-> Weather 返回类型被序列化成 JSON

注意：`verify(key)` 是验证函数。

第一：创建 HTTP Endpoint

- `GET`：HTTP Method。
- `/weather`：Endpoint Path。
- `get_weather`：OpenAPI 中的 Operation ID。

特别注意：Python 路由函数名是 `weather`，但 Operation ID 被显式指定为 `get_weather`。这让后面 OpenAPI 与 MCP 工具映射可以使用一致的操作名称。不过，`operationId` 并不要求必须与底层 Python 函数同名。

第二：解析请求参数

city: strunits: Literal["C", "F"] = "F"，FastAPI 根据参数声明解析 Query Parameters。

请求示例：/weather?city=Pittsburgh&units=F

第三：获取 API Key 并认证

key_header = APIKeyHeader(name="X-API-Key")

X-API-Key 放在 HTTP header；FastAPI 只取出 key，业务代码 verify (key) 负责校验合法性。

客户端在 HTTP Header 中提交：X-API-Key: YOUR_API_KEY，然后 key: Annotated[str, Security(key_header)] 表示从指定的安全依赖中取得 Key。

FastAPI 负责提取该凭证；示例中的 `verify(key)` 负责判断它是否有效。

- 声明类型：str 这个值是一个字符串。

- 声明来源：Security(key_header) 它不从查询串来，而是由 APIKeyHeader(name="X-API-Key") 从 HTTP 头里取。
- 声明职责：函数体里 verify(key) 负责校验，框架不替你做这一步，FastAPI 只管把值取出来，管不管用是你的事。

第四：执行函数并返回 JSON

请求：GET /weather，city/units放在 query 参数

返回：Python Weather 对象序列化为 JSON

verify(key)return get_weather(city, units) 服务器先检查凭证，再调用 Python 库函数。

返回的 `Weather` 对象由 FastAPI 序列化为 JSON 响应。

FastAPI binds the library function to a REST endpoint and documents its API key header.

FastAPI 将库函数绑定到 REST 端点，并且自动生成包含 API 密钥头的机器可读文档。即端点 + 说明书。

4.3 OpenAPI：API 的机器可读说明书

4.3.1 OpenAPI 的作用

有了 REST API，客户端已经可以调用天气服务。但 Agent 仍然需要知道：有哪些 Endpoint？使用 GET 还是 POST？参数叫什么？参数是否必填？数据类型是什么？如何认证？OpenAPI 就是用来描述这些信息的规范。

4.3.2 OpenAPI 的结构

OpenAPI describes the HTTP operation; embedded JSON Schema describes its values.

- OpenAPI 负责：HTTP 路径、请求方法、参数位置、安全方案
- JSON Schema 负责：类型、枚举、必填、默认值
- required 默认 false；in="query" 只是参数位置，**不等于必填**，二者相互独立。
- OpenAPI 是发布物，是框架自动生成的产物。

三部分：

1. Operation

表示有一个 `GET /weather` 操作，其标识为 `get_weather`。

2. Parameters

OpenAPI 字段	含义
name	参数名称
in	参数的位置
required	参数是否必填
schema.type	参数类型
schema.enum	允许的取值
schema.default	默认值

`in: query` 表示参数来自 URL 查询字符串，并不代表该参数必填。

3. Security

- 认证方案是 API Key。
- API Key 位于 HTTP Header。
- Header 名称是 `X-API-Key`。

4.3.3 OpenAPI 与 JSON Schema 的区别

OpenAPI	JSON Schema
描述 HTTP API	描述 JSON 数据结构
HTTP Method	数据类型
URL Path	枚举值
参数位置	对象属性
认证方案	数据约束
请求和响应结构	数据格式与验证规则

OpenAPI 可以在参数和请求、响应模型中使用 JSON Schema。

因此，课程中有一条重要的数据结构演变路线：Python Type Hints / Pydantic；FastAPI Response & Validation；OpenAPI + JSON Schema；MCP Tool Input Schema。这解释了为什么已有的 OpenAPI 描述可以被 FastMCP 复用。

4.4 curl 直连

4.4.1 curl 的作用

`curl` 是命令行 HTTP 客户端。

参数	含义
-sS	减少输出，同时保留错误信息
-G	将数据参数添加到 URL Query
--data-urlencode	URL 编码参数
-H	设置 HTTP Header
$WEATHER_API_KEY	从 Shell 环境变量读取密钥

Agent 可以根据 OpenAPI 文档构造并执行这个命令。

4.4.2 Agent 的三步调用过程

三个步骤及失败模式

步骤	说明	典型失败
1 Read the API description	Select the operation and required parameters. 读说明书，挑出要调哪个操作、需要哪些参数	读的是过期的文档，调用了一个已经不存在的操作或改了名的参数

步骤	说明	典型失败
2 Construct the command	The shell expands the environment variable; curl sends HTTP. shell 展开环境变量，curl 负责把 HTTP 发出去	环境变量没导出（展开成空串），请求带着空凭据打到服务端
3 Limit the credential	A coding agent with shell access may also read or misuse that secret. 能进 shell 的智能体也能读到、甚至滥用密钥	凭据权限过大（不只读天气，还能改别的资源），一旦泄露损失放大

Step 1 Read the API Description

读取 OpenAPI，找到正确的 Endpoint、参数和认证方式。

Step 2 Construct the Command

根据接口要求构造 curl 命令。

Shell 展开环境变量，curl 发送 HTTP 请求。

Step 3 Limit the Credential

尽量限制 API Key 的权限、有效期和使用范围。

为什么需要第三步？因为运行 curl 的 Shell 环境需要取得 API Key。如果 Agent 能在这个环境中任意执行 Shell 命令，就可能读取或滥用该凭证。

Tips：

与MCP路线的区别：同一个需求，两条路线的第一反应

REST 直连：怎么把命令写对。凭据问题被当成一个注意点，写在 Limit the credential。

MCP ： 怎么让凭据不进入会执行代码的那一侧。所以它先引入服务端，再谈工具清单。

4.4.3 安全问题的本质

存在凭证暴露风险：`WEATHER_API_KEY` 可由 Shell 访问。

这里并不是说 curl 不安全，而是说如果 Agent 的执行环境同时拥有广泛的 Shell 权限和敏感凭证，安全边界就比较弱。这为下一章 MCP 的凭证隔离设计提供了动机。

4.5 本章小结

本章建立了下面这条链路：

Python Library → FastAPI → REST API → OpenAPI → curl / HTTP Client

它解决了三个核心问题：

1. 如何让内部函数能够被远程调用。
2. 如何让机器理解 API 的调用方式。
3. Agent 如何直接调用 HTTP API，以及这种方式可能带来的凭证风险。

三步流程：

1. Read the API description：读取 OpenAPI 文档，选择操作、确定参数
2. Construct the command：shell 展开环境变量，curl 发送 HTTP 请求

3. Limit the credential：密钥暴露在 shell 环境，一旦泄露风险高；只能做最小权限限制，无法完全隐藏。限制它的最小权限、短期有效、随时可撤销。

Direct REST access is simple, but the agent's shell receives the API credential.

核心结论：直接访问 REST 简单，但智能体所在 shell 会拿到 API 凭据，存在密钥泄露风险。

MCP 诞生的核心动机：不让密钥落到 shell/agent 一侧，把凭据放在服务端。

六组对照（Part4&5）：

关键词	必须理解的内容
Library API → REST API	从进程内部调用变成网络调用
FastAPI → OpenAPI	从 HTTP 服务变成机器可读的接口描述
OpenAPI → FastMCP	从 REST 操作变成 MCP 工具
MCP Host → Client → Server	AI 应用与外部工具的通信架构
tools/list → tools/call	先发现工具，再执行工具
MCP Key → Upstream Key	客户端认证与上游认证分离

Part 5 Model Context Protocol

5.1 MCP 基础概念

核心问题：假设有 N 个 AI 应用和 M 个外部工具，如果每个 AI 应用都需要单独适配每个工具，可能产生 N × M 套集成工作。例如：Without MCP：不同 AI 应用分别适配不同工具；MCP 通过标准化工具接口减少重复适配。

即在没有MCP之前，连接 N 个大模型 (LLM) 和 M 个外部工具 / 数据源，是 N×M 的全连接问题。每一个 LLM 都要单独对接每一个工具，对接成本随数量爆炸式增长。

解决：With MCP：使用统一协议进行工具集成，课程使用 N→1→M 来描述这一思想。但要注意，它表示的是统一接口标准，而不是所有工具必须经过同一个中央服务器。实际系统可以包含多个 MCP Server。

即引入 MCP，MCP 将 LLM 和外部工具之间的通信标准化，变成 N→1→M 的星形架构：所有 LLM 只对接 MCP 中间层，MCP 再统一对接所有外部工具；大幅减少集成工作量。

MCP 的主要作用是：

- 统一工具发现方式。
- 统一工具调用方式。
- 减少重复集成工作。
- 支持不同类型的外部能力。
- 允许通过服务端隔离上游凭证。

MCP 不是 REST API 的必要替代品。

对于只需要调用少量 HTTP API 的简单系统，直接使用 REST 也完全合理。

5.2 MCP 的架构：Host、Client、Server

架构：客户端 - 服务器模型（Client-Server）

三个角色

角色	本质	作用
host	一个应用／进程	承载模型循环与用户界面，持有多个client
client	一条连接的连接器	只负责把 host 的意思翻译成 MCP 报文并送出去
server	一个能力提供方	把外部的工具与数据源按 MCP 暴露出来

host 并不是服务端：host 与 server 方向正好相反：host 在客户端一侧，server 是提供能力的一方。MCP is an alternative agent-facing interface 里的 agent-facing，指的正是 host 这一侧看到的东西，接口是给 host 里的智能体用的。

- **MCP Client**：需要调用工具 / 读取数据的 Agent（嵌入在 AI 主机 / LLM 应用内部）
- **MCP Server**：对外暴露外部工具、数据源的服务，包装底层能力给 LLM 调用
- **MCP Host**：承载 LLM + MCP Client 的 AI 应用程序（如 Claude、DeepSeek 这类 AI 应用）
- **Without MCP**：LLMs 和工具两两直连，网状结构；With MCP：所有 LLM 连接 MCP 中间节点，MCP 再连接全部外部工具

Tips：工具（tool）是可以被调用的动作，资源（resource）是可以被读取的数据。

5.2.1 MCP Host

Host 是承载 AI 应用的整体环境，它可以管理多个 MCP Client。例如：

```
AI Application
├── LLM
├── MCP Client A
└── MCP Client B
```

5.2.2 MCP Client

MCP Client 负责：与 Server 建立连接；发现工具；发送工具调用请求；接收工具结果。需要注意，MCP Client 不等于 LLM。LLM 负责推理和工具选择，Client 负责实际协议通信。

5.2.3 MCP Server

MCP Server 负责暴露外部能力。

例如：文件系统工具；数据库查询工具；GitHub 操作工具；REST API 封装工具；MCP Server 可以部署在本地，也可以部署在远程。

5.3 MCP的两种传输方式

MCP is an alternative agent-facing interface—not that is not a requirement for calling REST APIs.

MCP 是一个面向智能体的替代接口，它并不是调用 REST API 的必要条件。

MCP Host（AI 应用）可以同时创建多个 MCP Client，分别对接不同类型 MCP 服务：

1. Client A → LOCAL: STDIO Filesystem server（本地文件系统 MCP 服务）

2. Client B → REMOTE: HTTP Sentry server（远程 HTTP 的 Sentry 服务）

MCP 是面向 Agent 的接口方案；不是调用 REST API 的必需替代品，是一套上层封装协议。

元素	层级
MCP HOST	宿主机：跑 AI 应用的整体
AI application	宿主机里的应用：AI 应用本身
Client A / Client B	客户端：host 内部为每一个服务端各开一个客户端
LOCAL: STDIO + Filesystem server	服务端与传输：本地服务端，走 STDIO（标准输入输出）
REMOTE: HTTP + Sentry server	服务端与传输：远程服务端，走 HTTP

host 与 client 之间是一对多的关系。host 里同时挂着多个客户端，每个客户端各管一条连接，互不干扰。而 client 与服务端之间是一对一的。

stdio 是 Standard Input / Standard Output。

典型特点：通过本地进程的标准输入输出通信；通常不需要网络端口；适合本地文件、开发工具等能力。

HTTP Transport 用于通过 HTTP 进行 MCP 通信。

典型特点：可以连接远程服务器；支持网络部署；可以结合 HTTPS 和认证机制。

重点：MCP 定义上层交互协议，stdio 和 HTTP 是承载消息的传输方式。

同一个 Host 可以同时连接使用不同 Transport 的 MCP Server。

传输	配对例子	它在哪一侧	适合什么
LOCAL：STDIO	Filesystem server	同一台机器上	本地文件、本地进程这类不出机器的能力
REMOTE：HTTP	Sentry server	远端	第三方服务、SaaS 平台这类跨网络的能力

传输方式不是偏好，是被能力的位置决定的。比如Filesystem server（文件系统服务端）要读的是本机文件，把它放到远端反而多绕一圈，所以走STDIO，而 Sentry server（Sentry 是一个 SaaS 错误监控平台）本体就在网上，所以走 HTTP。

所以，同一个 host 里两种传输可以并存，这正是 client-server model 的价值：接口统一，底下的连接方式可以不同。

5.4 MCP 完整调用流程（时序图）

对应课程时序图的十个步骤：

1. User 向 MCP Host 输入 Query。
2. LLM 根据问题选择工具。
3. 模型输出工具调用信息。
4. Host 根据策略决定是否需要用户审批。
5. MCP Client 向 MCP Server 发送请求。
6. MCP Server 执行工具或调用第三方 API。
7. 底层工具产生结果。
8. MCP Server 将结果返回 Client。

9. Host 将工具结果提供给 LLM。
10. LLM 生成最终自然语言回答。

注意，用户审批是可能出现的安全控制，不是每次调用都必须发生。

5.4.1 两个关键 MCP 方法

工具发现：tools/list，用于获取服务器提供的工具及其 Schema。

工具调用：tools/call，用于指定工具名称并传入参数。

例如：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "city": "Pittsburgh",
      "units": "F"
    }
  }
}
```

5.5 MCP 三元素的关系

MCP is an alternative agent-facing interface—notn't a requirement for calling REST APIs.

MCP 是一个面向智能体的替代接口，它并不是调用 REST API 的必要条件。

MCP Host（AI 应用）可以同时创建多个 MCP Client，分别对接不同类型 MCP 服务：

1. Client A → LOCAL: STDIO Filesystem server（本地文件系统 MCP 服务）
2. Client B → REMOTE: HTTP Sentry server（远程 HTTP 的 Sentry 服务）

MCP 是面向 Agent 的接口方案；不是调用 REST API 的必需替代品，是一套上层封装协议。

元素	层级
MCP HOST	宿主机：跑 AI 应用的整体
AI application	宿主机里的应用：AI 应用本身
Client A / Client B	客户端：host 内部为每一个服务端各开一个客户端
LOCAL: STDIO + Filesystem server	服务端与传输：本地服务端，走 STDIO（标准输入输出）
REMOTE: HTTP + Sentry server	服务端与传输：远程服务端，走 HTTP

host 与 client 之间是一对多的关系。host 里同时挂着多个客户端，每个客户端各管一条连接，互不干扰。而 client 与服务端之间是一对一的。

5.6 FastMCP：将 OpenAPI 映射成 MCP Tools

5.6.1 FastMCP

如果已经有 REST API 和 OpenAPI 文档，就不一定需要重新编写一套工具定义。x

默认行为：FastMCP 会把 OpenAPI 里每一条路由，自动变成一个可被 LLM 发现的 MCP 工具。

执行逻辑：

1. 使用 `httpx.get()` 获取 OpenAPI 文档。
2. 解析文档中的 API 描述。
3. 创建用于访问原始 REST API 的 HTTP Client。
4. 通过 `FastMCP.from_openapi()` 构建 MCP Server。
5. 将 OpenAPI 操作映射为 MCP 能力。

5.6.2 自动映射的优点

- 复用已有 OpenAPI 描述。
- 减少重复编写工具 Schema。
- 降低 REST API 与工具定义不一致的风险。
- 适合快速接入已有服务。

5.6.3 自动映射的局限

问题一：工具数量可能过多

如果 OpenAPI 包含上百个操作，自动映射可能产生大量 MCP 工具。这会增加工具发现、上下文管理、模型选择和权限管理的复杂度。同时放大四类代价：延迟（latency）；成本（token / 计费 cost）；失败概率（failure rate）；安全风险（risk）

问题二：接口名称不一定适合 LLM

仅做结构层面映射，不做语义优化 映射只看 OpenAPI 结构，不理解业务语义。

接口命名清晰（如GET /weather）：自动生成的工具 get_weather 效果很好；

接口命名模糊、语义不清：转换后的工具，LLM 很难判断什么时候该调用，选择准确率下降。

适合场景：API 接口数量少、接口命名清晰规范的场景，自动映射性价比很高。 当接口数量庞大时，自动映射只是第一步，生成工具列表之后，必须额外增加一层筛选、分组能力，精简暴露给 LLM 的工具集合。

例如：GET /weather，转换成：get_weather 含义非常清晰。但如果原始 API 命名不规范：POST /v2/action/process 即使成功转换，LLM 也未必能判断什么时候应该调用它。

因此，实际工程中可能需要：

- 筛选工具。
- 改善工具名称和描述。
- 按业务场景分组。
- 对敏感操作设置权限。

注意：具体默认映射行为取决于 FastMCP 的版本与配置，不能笼统认为任何版本都会把所有路由无条件暴露为工具。

5.7 Credential Separation：独立凭证机制

5.7.1 独立凭证的作用

直接 REST 调用的问题

Agent 的执行环境需要取得上游 API Key。

使用 MCP 后

	MCP_API_KEY	UPSTREAM_API_KEY
使用者	MCP Client	MCP Server
验证方	MCP Server	REST API Server
保护对象	MCP 服务入口	上游 API
是否需要交给模型	不需要	不需要

5.7.2 Server 端代码

核心思想：MCP 客户端密钥 和 上游第三方 API 密钥互相隔离，MCP Server 作为中间代理，做凭证中转。

MCP Server 启动：两套密钥

- UPSTREAM_API_KEY：用来调用上游第三方 API（天气服务）
- MCP_API_KEY：用来保护 MCP 服务端点，供 MCP 客户端接入

> 教学演示使用静态 token；生产环境推荐 JWT 或者 OAuth。

代码可以分成三个逻辑模块：

模块 A：上游 HTTP Client，使用 `UPSTREAM_API_KEY` 访问原始天气 API。

upstream = httpx.AsyncClient(...)

模块 B：MCP Authentication，验证 MCP Client 提交的凭证。

mcp_auth = StaticTokenVerifier(...)

模块 C：MCP Server，把工具映射、上游 HTTP Client 和 MCP 认证机制组合起来。

mcp = FastMCP.from_openapi(...)

5.7.3 Client 端代码

MCP Client 侧：只持有 MCP 密钥

凭证代理（Credential brokering）

- Agent/Harness 只持有 MCP key，不知道上游 API 密钥
- MCP Server 负责：验证 MCP 客户端身份 → 授权 → 使用独立的上游密钥去调用第三方 API
- 优势：大模型本身看不到任何上游密钥，密钥不会暴露给 LLM，安全隔离。

两个关键调用：

await client.list_tools()，发现工具。

await client.call_tool(...)，执行工具。

整个过程中，Client 不需要获取 `UPSTREAM_API_KEY`。这就是 Credential Brokering（凭证代理）。

5.7.4 安全边界

Separate credentials allow the MCP server to mediate access rather than forward its upstream secret.

独立凭证让 MCP 服务器充当访问中介，而不是把上游密钥转发出去。

Agent 不知道上游 API Key不等于Agent 完全接触不到任何凭证。MCP Client 仍然持有 MCP_API_KEY。如果 Agent 拥有对该客户端环境的广泛访问权限，这个凭证也可能暴露。

所以，MCP 可以把上游凭证隔离在 Server 端，但 MCP 本身并不自动保证所有凭证安全。生产环境还需要身份认证、细粒度授权、最小权限、密钥轮换和审计等措施。

两套凭据：upstream 与 MCP

变量	保护的是	读法
UPSTREAM_API_KEY	MCP 服务端 → 天气 API 这一段	上游凭据
MCP_API_KEY	客户端 → MCP 服务端这一段	入口凭据

UPSTREAM_API_KEY authorizes API calls; MCP_API_KEY protects the MCP endpoint.

UPSTREAM_API_KEY 授权对上游 API 的调用；MCP_API_KEY 保护 MCP 端点本身。

前者是 authorizes（授权对外发起调用），后者是 protects（保护自家入口）。

5.8 REST API 与 MCP 的最终对比

维度	REST API / OpenAPI	MCP
Shared	名称、描述、JSON Schema	名称、描述、JSON Schema
Discovery	获取 OpenAPI 文档	tools/list
Invocation	HTTP Method + Path + Parameters	tools/call
Authentication	Client 向 API 认证	Client 向 MCP 认证，上游认证可以独立
Scope	描述 HTTP 操作	Tools、Resources、Prompts 等

重点一：Shared Schemas

REST API 和 MCP 可以共享结构化输入的 Schema 信息。

因此，FastMCP 才能利用 OpenAPI 生成工具定义。

重点二：不同的 Discovery

REST API：GET /openapi.json，REST 客户端通过 API 描述了解操作。

MCP：tools/list，MCP Client 通过协议发现 Server 当前提供的工具。

重点三：不同的 Invocation

REST：GET /weather?city=Pittsburgh&units=F，REST 使用 HTTP 方法、路径和参数。

MCP：tools/call(name="get_weather", arguments={...})，MCP 使用统一的工具调用接口。

重点四：不同的 Authentication Boundary

直接 REST 调用：Client → REST API

MCP 代理方案：Client → MCP Server → REST API

后者可以把客户端认证与上游认证分开。但这种凭证隔离是一种可采用的架构设计，不是 MCP 独有或自动提供的安全保证。

重点五：不同的 Scope

OpenAPI 主要描述 HTTP API。MCP 除了 Tools，还支持 Resources 和 Prompts 等能力。

MCP 能力	主要用途
Tools	执行操作
Resources	提供上下文数据
Prompts	提供可复用的提示模板

Part 6 Multiple Tool Calls：串行、并行与依赖

前面我们学习了 Agent 如何发现和调用一个外部工具，现在要解决新的问题：当 Agent 需要调用多个工具时，应该一个接一个执行，还是同时执行？如何减少延迟，同时保证执行结果正确？

五个递进的知识模块：

1. Serial Tool Calls：为什么串行调用很慢？
2. Parallel Tool Calls：并行如何减少模型生成轮数和等待时间？
3. Dependencies：哪些工具可以并行，哪些必须串行？
4. Concurrent Execution：如何用 `asyncio.gather()` 实现并发？
5. Call ID & Result Matching：多个结果如何正确对应原始调用？

6.1 Serial Tool Calls：串行调用的代价

6.1.1 串行工具调用

Serial Tool Calls 指多个工具按照先后顺序执行。

在最基本的串行模式中，Agent 每次生成一个工具调用，等待工具执行完成，再根据返回结果继续生成下一次调用。

课程使用三个工具作为例子：

- Weather：查询天气。
- Calendar：读取日历。
- Flight：查询航班。

串行执行流程如下：

6.1.2 四次 Generate

一次工具调用通常经历：

在课件的串行模式中：

- 第一次 Generate：决定调用 Weather。
- 第二次 Generate：读取天气结果，决定调用 Calendar。
- 第三次 Generate：读取日历结果，决定调用 Flight。
- 第四次 Generate：读取航班结果，生成最终回答。

因此，这个例子中：3 次串行工具调用需要 4 次模型生成。如果每轮只调用一个工具，并且每次结果返回后都重新调用模型，那么：

这是一种特定串行执行模式下的公式，不代表所有多工具系统都必须这样运行。

6.1.3 串行调用的代价

串行调用的延迟来自两个部分。

1. Model Generation Cost 模型生成本身需要时间。
2. Tool Waiting Cost 模型需要等待工具完成，才能继续下一步。

因此，仅仅优化工具速度还不够。如果三个工具都能瞬间完成，仍然需要四次模型生成，总计 3.2 秒。

串行代价的三层结构

- 生成不可重叠：四次生成各 0.8s，串行时只能排队，合计 3.2s。
- 等待不可隐藏：工具执行时模型在空等，那 3.2s 里没有任何计算在推进。
- 失败会放大：链条上任何一环变慢或出错，后面全部顺延；串行的链条越长，尾部风险越大。

6.2 Parallel Tool Calls：并行工具调用

6.2.1 并行的核心思想

如果多个工具互不依赖，Agent 可以在同一次模型生成中提出多个工具调用，再由执行环境并发运行它们。

例如：用户希望获得天气、日历和航班信息。这三个查询在输入已经确定、且彼此独立的情况下，可以同时执行。

6.2.2 两次 Generate

串行模式：Generate → Weather；Generate → Calendar；Generate → Flight；Generate → Final Answer

并行模式：Generate → [Weather, Calendar, Flight]→ Collect Results；Generate → Final Answer

因此，课件中的并行模式只有两个模型生成阶段，中间不再需要为每个独立工具分别进行一次模型生成：

1. 第一次 Generate：同时产生三个工具调用。
2. 第二次 Generate：读取全部工具结果，生成最终回答。

6.2.3 并行总耗时的计算

在课件的理想化模型中：

因为三个工具同时执行，最终需要等待的是耗时最长的那个工具 max()。

例如：Weather：1.2s；Calendar：0.9s；Flight：1.1s

三个工具一起开始，Calendar 最先完成，Flight 随后完成，Weather 最后完成。

这个公式假设工具能够同时启动、没有额外调度开销，而且模型生成与工具等待没有其他重叠。

6.2.4 串行与并行性能对比

所以并行优化有两个来源：减少模型生成轮数 + 重叠工具等待时间。

需要特别注意：并行没有让 Weather 从 1.2 秒变成更短，它只是让 Weather、Calendar 和 Flight 的执行时间互相重叠。

6.2.5 并行的性能瓶颈

并行总耗时由最慢的分支决定。如果三个工具耗时分别是：Weather: 1.2s；Calendar: 0.9s；Flight: 1.1s。那么优先优化 Calendar，可能对整体完成时间没有帮助，因为 Weather 仍然需要 1.2 秒。只有降低最长分支，或改变关键路径，才可能进一步降低这批调用的完成时间。

结论：并行的价值不是让单个工具变快，而是减少串行往返，并让独立工具的等待时间重叠。

6.3 Dependencies 并行条件

Parallelize only independent, safely concurrent calls.

即：只并行那些相互独立、并且能够安全并发执行的调用。

6.3.1 可以并行: Independent Calls

Weather for Pittsburgh; Read today's calendar; Look up flight status

如果三个工具所需的输入已经确定，它们不需要等待彼此的输出。

6.3.2 必须串行: Dependent Calls

Find customer ID; Use ID to fetch orders; Refund selected order

因为第二步需要第一步返回的 `customer_id`，第三步又需要前面查询并选择出的订单信息。在第一步完成之前，第二步所需的输入还不存在。因此，它们存在 Data Dependency（数据依赖）。

6.3.3 三种常见依赖

- 数据依赖：后一步的参数来自前一步的返回值（ID → orders）。
- 控制依赖：后一步是否执行取决于前一步的结果（查不到客户，就不该发起退款）。
- 副作用依赖：后一步改变的状态必须建立在前一步已提交的状态之上（先退款、再记账）。

类型	定义	示例
Data Dependency	后一步需要前一步的输出	获取客户 ID → 查询订单
Control Dependency	后一步是否执行取决于前一步结果	查询是否有库存 → 决定是否下单
Side-effect Dependency	多个操作会影响共享状态或执行顺序	更新余额 → 记录交易

第三种尤其重要，因为没有数据依赖，不代表并发一定安全。

例如：Tool A: Withdraw $100; Tool B: Withdraw $200

两个调用可能都知道账户 ID 和金额，不需要彼此提供参数。但如果同时修改同一账户余额，且底层系统没有正确处理并发，就可能产生 Race Condition（竞态条件）。因此，并行判断至少需要检查：

- 输入是否已经全部确定。
- 一个调用是否依赖另一个调用的结果。
- 多个调用是否会修改共享状态。
- 底层服务是否支持安全并发。

6.3.4 真实任务通常是混合执行

不是所有任务都必须完全串行或完全并行。

在这个例子中：

- Weather 可以与 Find Customer ID 并行。
- Fetch Orders 必须等待 Customer ID。
- Refund Order 必须等待订单查询与必要确认。

这种结构可以理解为一个有向依赖图（Dependency Graph）。工具调用的执行顺序由依赖关系决定，而不是简单地由工具数量决定。

6.4 Concurrent Execution：使用 asyncio 实现并发

前面解释了什么时候应该并行，现在看代码如何实现。课程使用 Python 的 `asyncio.gather()`。

6.4.1 课程代码

```
import asyncioimport json
async def execute(call):    //定义一个 Coroutine Function（协程函数）。
调用它会得到协程对象，可以由 asyncio 事件循环调度执行。
args = json.loads(call.arguments)    //解析参数,假设模型返回的参数是 JSON 字符串
value = await TOOLS[call.name](**args)    //根据名称选择工具,通过工具名称查找对应函数,传递参数并等待工具结果
return call.call_id,  //保留 Call ID
valueresults = await asyncio.gather(
*(execute(call) for call in function_calls),
return_exceptions=True,
)
```

这里假设：

- `TOOLS` 是工具名称到异步函数的映射。
- `function_calls` 包含本轮需要执行的工具调用。
- 每个 `call` 有 `name`、`arguments` 和 `call_id`。

6.5 asyncio.gather()：一次调度多个协程

6.5.1 gather 的作用

```
results = await asyncio.gather(
*(execute(call) for call in function_calls),    //为每个工具调用创建一个协程
//* 将这些协程展开，作为多个位置参数传给 gather()
return_exceptions=True,)
```

`asyncio.gather()` 可以并发运行多个可等待对象，并等待它们完成。

6.5.2 并发完成顺序可能不同

假设同时执行：

`asyncio.gather()` 返回的结果顺序与输入的协程顺序一致，而不是与实际完成顺序一致。并发执行时，谁先跑完谁先回来，这个顺序是不确定的。工具响应时间不同、网络抖动，都会改变完成次序，结果乱序完成时，要靠 ID 把它们对回原调用。

例如，输入顺序为：[execute(weather), execute(calendar), execute(flight),]，那么 `gather()` 返回的结果仍然按照：[weather_result, calendar_result, flight_result,] 排列，即使 Calendar 最先完成。

因此，不能根据位置匹配 gather 返回结果，对于同一次 `gather()` 调用，只要保留原始输入顺序，位置对应关系是有保证的。但在完整 Agent 系统中，仍然应该保留 Call ID，因为工具结果需要与协议层的原始调用关联，不能依赖偶然的执行或消息顺序。

6.5.3 Call ID

例如：return call.call_id, value

可以产生：[("call_001", weather_result), ("call_002", calendar_result), ("call_003", flight_result),]

三个主要作用：

- Result Matching：把结果与原始工具调用配对。
- Traceability：在日志中追踪某次调用。
- Error Handling：将错误与具体调用关联。

在采用工具调用协议的系统中，返回结果通常需要携带与原始调用对应的标识符。

6.5.4 return_exceptions=True 的作用

return_exceptions=True

表示某个协程抛出异常时，`gather()` 将该异常作为对应位置的结果返回，而不是立即把它作为整个 `gather()` 的异常抛出。

例如：[weather_result, TimeoutError(), flight_result,]

这样，Calendar 失败不一定阻止其他两个工具完成。但异常对象不会自动变成成功的工具结果。执行环境仍然需要识别异常，并构造对应的错误响应。

6.6 总结

维度	Serial Tool Calls	Parallel Tool Calls
执行方式	依次执行	同时发起独立调用
模型生成	可能需要多轮	可以在一轮生成中提出多个调用
工具等待	耗时累加	主要由最慢工具决定
依赖处理	自然满足先后依赖	必须提前检查依赖
执行实现	顺序 await	asyncio.gather() 等
结果配对	使用 Call ID	同样需要 Call ID
课件耗时	6.4s	2.8s

其中：

- N：工具数量。
- G：每次模型生成耗时。
- T_i：第 i 个工具的执行时间。

1. Serial Calls：多个工具依次执行，会累加模型生成时间和工具等待时间。
2. Parallel Calls：独立工具可以在同一轮生成中提出，并发执行以降低延迟。
3. Dependencies：只有相互独立、并发安全的调用才能同时执行。
4. Concurrent Execution：Python 可以使用 `asyncio.gather()` 并发运行异步工具。
5. Call ID：工具结果必须能与原始调用正确对应，尤其是在异步和并发系统中。

这章最核心的思想是：优化多工具调用，不是盲目地把所有工具同时运行，而是根据依赖关系，在保证正确性的前提下，尽量减少不必要的模型生成轮数和等待时间。

它与前面 REST API、MCP 两章的关系也很清晰：

- REST API 解决如何通过网络调用工具。
- MCP 解决 Agent 如何标准化地发现和调用工具。
- Multiple Tool Calls 解决 Agent 如何高效、正确地组织多个工具调用。

Part 7 Evaluating Tool Use

前一章解决的是如何让 Agent 高效、正确地组织多个工具调用？这一章进一步讨论我们如何判断一个 Agent 的工具使用能力到底好不好？评估不能只看最终答案是否正确，还要检查它有没有选对工具、传对参数、安排好调用顺序，以及在真实运行环境中是否足够快、稳定、安全。

本章知识框架

1. Benchmark：考什么？BFCL 的四类测试场景
2. Evaluation Levels：在哪一层判断对错？Selection → Arguments → Trajectory → Task
3. Operational Quality：真实运行表现如何？Efficiency · Reliability · Safety
4. Failure Attribution：错误来自哪里？Model · Harness · Provider / Tool
5. Production Evaluation：真实数据怎么看？Provider Tool-call Error Rates
6. Layered System：整讲知识 Capabilities → Mechanics → Constraints → Interfaces → Systems

7.1 BFCL：工具调用能力的四类测试

Berkeley Function Calling Leaderboard 是用于评估大语言模型 Function Calling 能力的基准测试。

核心问题：模型能不能在不同场景下正确使用工具？

课件将测试场景分为四类：

7.1.1 Single Turn：单轮工具调用

Single Turn 评估模型在一次交互中生成正确工具调用的能力。

例如：

Simple: 调用单个工具

get_weather(city="Paris")

Multiple: 一次生成多个工具调用

get_weather(city="Paris")

get_calendar(date="today")

Parallel: 同一工具的多个独立调用

get_weather(city="Paris")

get_weather(city="London")

get_weather(city="Tokyo")

Multiple-parallel: 多种工具的多个并行调用

get_weather(city="Paris")

get_weather(city="London")

get_calendar(date="today")

get_flight(flight_no="AF123")

这里需要区分：

Single Turn 中的 multiple 和 parallel 都涉及多个调用，但 parallel 特别强调同类调用的独立并发场景；multiple-parallel 则组合了多工具和多次调用。这与 Part 6 的 Parallel Tool Calls 直接对应。

7.1.2 Multi-turn：多轮工具调用

Multi-turn 评估模型在多个交互步骤中能否持续正确使用工具。

课件列出四个子任务。

1. Base：基本多轮任务

评估模型能否利用上一轮工具返回的信息继续执行。例如：

```
User: 帮我查一下客户订单
Agent → find_customer()
Tool  → customer_id
Agent → get_orders(customer_id)
Tool  → order_list
Agent → Final Answer
```

2. Missing Function：缺少必要工具

例如用户要求：“帮我给客户退款。”但系统只提供：

```
TOOLS = {
"get_customer": ...,
"get_orders": ...
}
```

没有退款工具。模型不应该编造一个不存在的 `refund_order()` 调用。

3. Missing Parameter：缺少必要参数

例如：

```
get_flight_status(flight_number)
```

用户只说：“帮我查一下航班状态。”却没有提供航班号，也没有其他信息可以确定航班。正确行为通常是请求补充必要信息，而不是随意编造航班号。

4. Long Context：长上下文

在很长的对话中，测试模型是否仍能正确找到工具、参数和之前的任务信息。

7.1.3 Agentic：智能体场景

课件给出两个方向：

- Web Search：使用搜索工具获取外部信息。
- Memory：使用记忆信息支持后续任务。

相比简单的函数调用，这类测试更接近 Agent 的实际工作方式。

例如，一个 Agent 可能需要搜索信息、读取结果、决定下一步，再综合完成用户目标。

7.1.4 Robustness：鲁棒性

Robustness 关注的是：当输入不理想时，模型是否仍然可靠？

两个测试方向

1. Hallucination Measurement 测试模型是否会编造工具、参数或不存在的结果。
2. Format Sensitivity 测试模型对格式变化是否敏感。

例如同一个工具调用需求，分别用 JSON、自然语言、不同排列顺序或不同格式描述，模型的表现是否会发生明显变化。这与 Constraints 的内容相关：格式合法不代表语义正确，格式变化也可能影响模型选择。

7.2 Evaluate the Whole Stack：四层评估体系

一次工具调用究竟应该从哪些层面判断正确性？四个层次：

7.2.1 Selection：选择工具

第一层检查 Tool Selection。

例如用户说：“查询巴黎天气。”

系统提供：get_weather(city); get_calendar(date); book_flight(destination)

模型应该选择：get_weather(city="Paris")，如果调用了 `get_calendar()`，就是选错工具；如果本应调用工具却没有调用，就是 Missing Call如果调用了不必要的工具，就是 Extra Call。

常见评估指标：

- Precision：模型发出的调用中，有多少是应该发出的。
- Recall：应该发出的调用中，有多少被模型正确发出。

其中 TP 是正确调用，FP 是多余或错误调用，FN 是遗漏调用。实际评测还需要明确如何匹配预测调用与标准调用。

7.2.2 Arguments：检查参数

第二层检查 Tool Arguments，即使工具选对，参数仍然可能错误。

例如：get_weather(city="London")，用户要求的是巴黎的天气，这个调用可能满足 JSON Schema，但 `city` 的语义值错误。

两个检查方向：

- AST Match：比较工具调用的结构。
- Schema Match：检查参数结构与字段是否符合要求。

需要注意：Schema Match 主要保证结构合法，不能单独保证参数值符合用户意图。

7.2.3 Trajectory：调用顺序

第三层评估调用轨迹（Trajectory），这与 Part 6的 Dependencies 直接对应。

例如：Correct Trajectory；Find Customer ID；Fetch Orders；Refund Selected Order

如果模型先退款，再查询订单，就违反了依赖关系，即使每个工具本身正确，整个执行序列也可能错误。

课件使用 Sequence Success 作为示例指标，并指出这一层通常需要执行或观察工具交互过程。

7.2.4 Task：最终目标

第四层检查 End-to-end Task Success，这是最接近用户真实体验的指标。

例如：用户要求：“帮我查询明天的航班状态。”

Agent:

1. 正确选择航班工具。
2. 正确传入航班号。
3. 正确执行查询。
4. 却在最终回答中说错了航班时间。

前三步可能都通过，但 Task 仍然失败。课件给出的典型失败是：Plausible Wrong Answer：看起来合理，但实际上错误的答案。它尤其难发现，因为错误可能不是格式问题，也不是工具执行异常，而是模型对结果的理解或使用出了问题。

7.3 Operational Quality：运行质量的三个维度

7.3.1 Efficiency：效率

A correct agent can still be unusably slow or expensive.

即使 Agent 能正确完成任务，也可能因为延迟过高或成本过高而无法实际使用。

四个指标分别是：

指标	关注什么
Latency	完成任务需要多久
Calls	调用了多少次工具

指标	关注什么
Tokens	消耗了多少 Token
Cost	完成任务的实际成本

Part 6 的串行与并行对比就是典型的 Efficiency 问题。串行耗时 6.4 秒，并行耗时 2.8 秒。虽然工具查询结果可能完全相同，但并行执行改善了响应延迟。

7.3.2 Reliability：可靠性

Reliability 关注系统在真实运行条件下能否稳定完成任务。

Timeouts 工具没有在规定时间内返回结果。

Retries 请求失败后重新尝试。

Partial Failure 一批调用中部分成功、部分失败。

例如三个并发工具：

Weather → Success Calendar → Timeout Flight → Success

系统需要决定：是否重试 Calendar？是否可以利用已经成功的两个结果？是否向用户说明部分信息不可用？

这与上一章 `asyncio.gather(return_exceptions=True)` 的异常处理直接相关。

7.3.3 Safety：安全性

Safety 关注 Agent 的工具调用是否符合权限和安全约束。

课件列出：

- Policy：政策与规则约束。
- Permissions：权限控制。
- Side Effects：执行操作带来的状态变化。

例如：

```
delete_file(path)
transfer_money(amount)
send_email(recipient, content)
```

这些工具不仅查询信息，还可能产生真实影响。所以除了判断调用是否正确，还需要评估调用是否经过授权、是否符合安全规则，以及执行后是否造成不必要的副作用。

7.3.4 三个维度之间的权衡

真实系统中，这三个维度可能互相影响。

例如：

- 增加重试可能提高 Reliability，但增加 Latency 和 Cost。
- 增加权限校验可能提高 Safety，但增加处理时间。
- 减少工具调用可能提高 Efficiency，但如果省略必要的检查，也可能降低任务成功率。

因此，生产环境不能只追求单一指标。

本节结论：Agent 的质量不仅取决于是否正确，还取决于是否高效、可靠、安全。

7.4 Operational Tool Failures：三类故障来源

当一次工具调用失败时，错误究竟来自模型、执行框架，还是外部工具？

课件将故障分为三个来源。

7.4.1 Model：模型侧错误

模型侧主要负责决定调用什么工具、传递什么参数，以及如何理解返回结果。

三种典型错误：Wrong Tool 模型选错工具；Bad Arguments 工具正确，但参数不正确；Ignores Result 工具已经返回结果，但模型忽略结果，仍然生成错误答案。

例如：

```
Tool result:
Flight delayed by 2 hours
Agent final answer:
The flight is on time.
```

这里工具执行成功，错误发生在模型对结果的使用阶段。

7.4.2 Harness：执行框架错误

Harness 是模型与工具之间的编排执行层，主要负责解析、验证、派发调用，以及将结果匹配回原始调用。

四种故障：Parse 无法正确解析模型生成的工具调用内容；Auth 调用时出现认证或授权问题；Validation 工具参数不符合 Schema 或其他校验规则；ID Mismatch 工具返回结果与原始 Call ID 配对错误。最后一种直接呼应 Part 7的并发执行。

例如：

```
call_001 → Weather
call_002 → Calendar
```

如果 Harness 把 Calendar 的结果错误关联到 `call_001`，就会导致工具调用与结果不一致。

7.4.3 Provider or Tool：供应商或工具侧错误

即使模型和 Harness 都正确，外部服务仍然可能失败。

Timeout 服务响应超时；Rate Limit 调用频率超过服务限制；Execution Error 工具执行时出现错误。例如：

```
Model:
Correct tool + correct arguments
Harness:
Valid request + correct dispatch
Provider:
HTTP 429 Too Many Requests
```

这种情况下，失败不是模型选错了工具，而是外部服务限流。

7.4.4 故障的不同处理

同样是 Task Failure，不同来源需要不同处理方式。

错误来源	典型改进方向
Model	改善提示词、工具描述、模型能力或训练
Harness	修复解析、验证、授权与结果配对逻辑
Provider / Tool	超时控制、退避重试、熔断、降级

Production evaluation needs benchmark accuracy and operational failure rates.

生产级评估既需要 Benchmark Accuracy，也需要 Operational Failure Rates。仅仅知道模型在测试集上的工具选择准确率，并不能说明整个 Agent 系统在真实环境中有多可靠。

7.5 Provider Tool-call Error Rates：真实错误率数据

Tool Call Error Rate by Provider 不同供应商的工具调用错误率。

图表按时间展示供应商侧的错误率变化，观察外部工具调用在运行中发生错误的情况。

图表纵轴标注为 Error Rate (%)，但截图中的悬浮读数没有单独显示单位。这里保留截图中的原始数值，不额外推断统计口径。

Cloudflare 的 15.1 明显高于其余可见读数，是这张截图中的突出异常值。

不过，单凭截图不能判断异常的具体原因，也不能据此评价供应商长期的整体可靠性。

总结

第一，供应商错误率是随时间变化的。同一服务可能在不同日期出现不同的错误水平。

第二，不同供应商的运行稳定性可能不同。即使模型使用完全相同的工具调用策略，外部服务的失败也可能影响最终任务成功率。

第三，评估需要区分模型能力与运行环境。BFCL 测试的是模型工具调用能力的不同方面。Provider Error Rate 则提供运行环境可靠性的另一类证据。这两类指标应该结合分析，而不是互相替代。

7.6 Tool Use Is a Layered System

Tool Use for LLM Agents 五层内容：

1. Capabilities 说明工具为模型扩展了哪些能力。Retrieval · Execution · Generation · Application Actions
2. Mechanics 说明工具定义如何进入模型上下文，模型如何产生调用，Harness 如何执行并返回结果。Schemas · Model Protocols · Harness · Call IDs
3. Constraints 说明 JSON Schema 和 Grammar Masks 可以约束输出形式，但不能保证语义正确。JSON Schema · Grammar Masks
4. Interfaces 说明 REST API 与 MCP 如何向 Agent 暴露外部能力。Direct REST Calls · MCP

对应：Part 6：如何组织多个工具调用，优化执行效率。

5. Systems 则负责把前面所有部分组织成完整的系统：管理依赖、并发执行、评估正确性、控制成本并处理运行故障。Dependencies · Concurrency · Evaluation · Failures

对应：Part 7：如何评估这些工具调用及其最终结果。

7.7 总结

假设用户要求 Agent 查询航班信息。

Agent 调用了航班查询工具，最后给出航班状态。

我们可以依次检查：

1. Selection：是否选择了航班工具，而不是天气工具？
2. Arguments：航班号、日期等参数是否正确？
3. Trajectory：是否按照必要的依赖顺序执行？
4. Task：最终航班信息是否正确？
5. Efficiency：是否用了过多时间、Token 或工具调用？
6. Reliability：超时、限流或部分失败时能否妥善处理？
7. Safety：是否遵守权限与安全约束？
8. Failure Attribution：如果失败，究竟是模型、Harness，还是外部服务的问题？

这就是Evaluate the whole stack 的真正含义：不仅评估模型生成了什么调用，还要评估整个调用过程以及最终结果。

Lecture 7 Agent Memory and RAG

Part 1 RAG 的研究动机与基本原理

1.1 大语言模型的局限性

大语言模型（Large Language Models, LLMs）依靠大规模训练数据和模型参数获得语言理解与生成能力，但仍存在四类主要问题。

挑战	定义与表现
Hallucination（幻觉）	生成不准确、无意义或缺乏事实依据的内容
Domain-specific Knowledge & Expertise（领域知识不足）	缺乏医学、法律、金融等专业领域的知识与推理经验
Privacy（隐私）	在训练、部署及调用过程中可能产生敏感信息泄露与数据安全风险
Inconsistency（不一致性）	不同调用中回答质量不稳定，可能出现相互矛盾或与问题无关的内容

其中，幻觉和领域知识不足是引入外部检索的重要动机，但 RAG 并不能自动消除隐私风险或保证生成结果完全一致。

垂直领域中的幻觉问题

斯坦福研究人员发表于 Journal of Legal Analysis（2024）的论文：Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models。研究指出，LLM 在回答可核验的联邦法院案件问题时，可能产生较高比例的法律幻觉。在特定法律问题上，幻觉率可达到 69% - 88%。图中的 58% - 88% 则是所展示模型的平均幻觉率范围，两种表述不应混为同一个统计口径。

这一结果说明：语言流畅性与事实可靠性并不等价，通用模型的能力也不能直接代表其专业领域能力。

1.2 参数化知识与非参数化知识

课件用 Big Model + Big Training Data 概括 LLM 的能力来源，并提出两种知识存储方式。

比较维度	Parametric Knowledge	Non-parametric Knowledge
中文	参数化知识	非参数化知识
存储位置	模型参数	外部文档、数据库、索引
获取方式	模型前向推理	Information Retrieval（IR）
更新方式	通常需要训练或参数更新	更新外部知识库及索引
优势	无需每次显式检索	易更新、可追溯、可扩展
局限	知识更新成本高，难以覆盖全部长尾知识	依赖检索质量与外部数据

参数化知识通过训练过程隐式编码于模型权重 θ 中，模型基于输入 x 生成回答： $P_\theta(y\mid x) $

其中 x 表示输入，y 表示生成结果。非参数化知识则通过外部语料库 $\mathcal{D}$ 存储，并在需要时执行检索：

$\mathcal D_q=\operatorname{Retrieve}(q,\mathcal D) $

其中 q 为查询，$\mathcal{D}_q$ 为检索得到的相关文档集合。

两者并非相互替代：LLM 提供语言理解与生成能力，外部知识库提供可动态访问的信息。

1.3 Retrieval-Augmented Generation（RAG）

1. LLM 无法将全部知识，尤其是长尾知识（Long-tail Knowledge），完整编码进参数。
2. 训练结束后新增的信息与专业领域知识，也不一定能够被模型准确获取。

重新训练或微调（Re-training / Fine-tuning）可以改变模型能力，但通常需要较高的计算与数据成本，并不适合所有知识更新需求。Retrieval-Augmented Generation（RAG）通过将外部检索结果引入生成过程，使模型能够利用参数之外的信息完成任务。

其基本过程为：

常见的输入级 RAG 可形式化为： $y\sim P_\theta(y\mid q,\mathcal D_q) $

案例：询问 2023 年女足世界杯冠军。未使用 RAG 的旧模型可能由于知识截止时间而无法回答；使用 RAG 后，模型通过外部信息获得事实，回答 Spain。

传统 LLM 与 RA-LLM =可以类比为：闭卷考试与开卷考试

- Close-book Exam：仅依赖模型参数中已有的知识。
- Open-book Exam：允许检索外部参考资料，再生成答案。

其关系可以概括为： ${\text{RA-LLM}=\text{LLM Generation}+\text{Information Retrieval}} $

课件将训练数据与外部知识库作如下对比：

Data for Training LLMs	External Knowledge Base
Low quality	High-quality knowledge
General	Specialized knowledge

Data for Training LLMs	External Knowledge Base
Fixed	Scalable
Hard to update	Easy-updated

训练数据也可以具有高质量和专业性，外部知识库同样可能包含错误或过时信息。RAG 的主要优势在于知识可更新、可检索和可扩展，但其最终可靠性仍取决于知识库质量、检索准确率与生成模型的证据使用能力。

Part 2 RA-LLM 的研究体系与标准架构

2.1 RA-LLM Research Taxonomy

从 Architecture、Learning 和 Application Tasks 三个维度建立 RA-LLM 的研究分类。

Architecture：架构维度

三种 Augmentation 方式的主要区别是检索知识进入生成模型的位置。

1. Input-layer Integration（输入层集成）

将检索文本与查询拼接，作为模型输入。课件后续介绍的 RAG 与 In-Context RALM 均采用这种思路。

2. Intermediate-layer Integration（中间层集成）

在模型内部表示或中间计算过程中引入检索信息。

3. Output-layer Integration（输出层集成）

在输出阶段结合检索证据，例如通过输出概率或候选结果的融合影响最终预测。

Learning：学习维度

训练方式	含义
Independent	检索器与生成器分别训练
Sequential	按阶段依次训练不同组件
Joint	对检索与生成进行联合优化

Application Tasks：应用维度

- NLP：Question Answering、Fact Verification
- Downstream：Recommendations、Software Engineering
- Domain-specific：Science、Finance

三维分类分别回答：系统如何构建、组件如何学习、技术应用于哪些任务。

2.2 RAG 技术演进（2019 - 2024）

课件通过时间轴展示四条研究路线：

技术路线	代表模型或方法
RAG Framework / Pipeline	kNN-LM、REALM、RAG、FiD、RETRO、Atlas、Self-RAG、In-Context RALM

技术路线	代表模型或方法
RAG Learning	REALM、RAG、EMDR2、RETRO、Atlas、Self-RAG
Retriever Learning	DPR、Contriever、EPR、CEIL、UDR、RA-DIT
Pre-/Post-Retrieval Techniques	SPALM、Re2G、HyDE、Query2doc、RECOMP、QueryRewriter

其中，REALM、RAG 等早期工作奠定了神经检索与语言模型结合的基础；后续研究逐渐扩展至检索器训练、检索前后处理以及更复杂的自适应检索流程。

需要注意，同一研究可能同时属于多个类别。例如 RAG 既是一种架构，也涉及检索与生成的学习机制。

2.3 RA-LLM Standard Pipeline

标准 RA-LLM 由两个必要组件和两个可选模块组成：

Retrieval 和 Generation 是必要组件（Major Components），Pre-retrieval 与 Post-retrieval 属于可选的辅助模块（Affiliated Modules）。Integration 表示检索与生成之间的信息融合关系，而不是额外的独立处理模块。

2.4 两种典型的检索增强模型

1. RAG：DPR + BART

- Retrieval：Dense Passage Retriever（DPR）+ Maximum Inner Product Search（MIPS）
- Generation：BART
- Integration：将检索段落分别与问题拼接

对于查询 q 和检索文档 z，模型可按文档概率对生成结果进行边缘化：

$P(y \mid q) = \Sigma_{z \in \text{TopK}(q)} P_\eta(z \mid q) P_\theta(y \mid q, z)$

其中：

- $P_\eta(z \mid q)$：检索器选择文档 z 的概率。
- $P_\theta(y \mid q, z)$：生成器在查询与文档条件下生成答案的概率。

该式对应经典 RAG-Sequence 的文档级边缘化形式，是对课件架构图的理论补充；不同 RAG 变体的概率融合方式可能不同。RAG 可以通过查询编码器 q_η 和生成器 p_θ 的梯度反向传播进行端到端学习。

2. In-Context RALM：BM25 / BERT / Contriever + GPT-series

In-Context RALM 使用 BM25、BERT 或 Contriever 等检索方法，并将检索段落前置到模型输入中。

$x'=\operatorname{Concat}(d_1,\ldots,d_k,q)$

$y\sim P_\theta(y\mid x')$

其中 $d_1, \cdots, d_k$ 为检索文档。

与前述经典 RAG 模型相比，这种方式通常不要求对生成模型进行端到端训练，而是通过上下文提示利用检索证据。世界杯扩军案例：模型可以从检索文本中获取“2026 年世界杯扩军至 48 支球队”的信息，而无需将该知识重新写入模型参数。

比较	经典 RAG	In-Context RALM
检索器	DPR + MIPS	BM25 / BERT / Contriever
生成器	BART	GPT-series
集成方式	问题与各检索段落分别结合	将检索内容前置到输入
学习机制	可端到端联合训练	通常直接利用预训练 LLM
主要特点	显式建模检索文档与生成概率	实现简洁，适用于黑盒 LLM

Part 3 Retriever：检索器的分类、原理与训练

检索器（Retriever）负责从知识库中筛选与查询相关的信息，是 RAG 系统中决定证据质量的关键组件。不同检索器不仅影响 Retrieval Performance，也会进一步影响生成模型的输出质量。

3.1 检索器的两个分类维度

分类依据	类型	核心区别
Relevance Measurement	Sparse / Dense	如何表示文本并计算相关性
Retriever Learning	Task-specific / General-purpose	如何获得检索能力、适应什么任务

Sparse 与 Dense 是表示和评分方式的区别；Task-specific 与 General-purpose 则是训练目标与适用范围的区别。

不同检索器对生成质量的影响

课件使用 Perplexity（困惑度）衡量不同检索器辅助语言模型时的生成效果。

$ \operatorname{PPL}(x) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t\mid x_{<t}) \right) $

PPL 越低，表示模型对测试文本的预测越好。

课件实验数据：不同检索器下的 PPL。困惑度越低越好。两种模型规模下，BM25 均取得最低 PPL。

实验结果表明：

- 对 GPT-2 117M，BM25 的 PPL 为 30.2，优于 No Retrieval 的 37.8。
- 对 GPT-2 1.5B，BM25 的 PPL 为 16.7，优于 No Retrieval 的 19.9。
- BERT 的表现略差于不检索，说明加入检索不一定改善生成质量。
- 大模型整体 PPL 更低，但不同检索方法之间的差距有所缩小。

这一实验说明，检索器的选择应依据下游任务效果，而不能仅依据是否采用了更复杂的神经网络。

3.2 Sparse Retrieval 与 Dense Retrieval

两者的根本区别在于文本表示方式与相关性计算机制。

Sparse Retrieval（稀疏检索）

稀疏检索通过词项及其统计特征表示文档，通常在高维词汇空间中进行匹配。

基本过程： $\text{Document} \rightarrow \text{Tokenization} \rightarrow \text{Inverted Index} \rightarrow \text{Relevance Scoring} $

代表方法：

1. TF-IDF 词频（Term Frequency）与逆文档频率（Inverse Document Frequency）相结合：

$\operatorname{TFIDF}(t,d) = \operatorname{TF}(t,d)\operatorname{IDF}(t) $

常见 IDF 形式为：

$\operatorname{IDF}(t) = \log\frac{N}{\operatorname{DF}(t)}$

其中 N 是文档总数，DF (t) 是包含词项 t 的文档数量。

2. BM25 在词项匹配基础上加入词频饱和与文档长度归一化。

一种常见表达为：

$\operatorname{BM25}(q,d) = \sum_{t\in q} \operatorname{IDF}(t) \frac{ f(t,d)(k_1+1) }{ f(t,d)+k_1 \left(1-b+b\frac{|d|}{\operatorname{avgdl}} \right) }$

其中：

- $f(t, d)$：词项在文档中的频率。
- k_1：控制词频饱和程度。
- b：控制文档长度归一化。
- $|d|$：文档长度。
- avgdl：语料库平均文档长度。

课件强调 Sparse Retrievers 的三个特点：

- Feasible to apply：易于实现和部署。
- High efficiency：检索效率较高。
- Fine performance：在许多任务上具有良好效果。

Dense Retrieval（稠密检索）

稠密检索通过神经编码器将查询与文档映射为低维连续向量 $\mathbf q=E_Q(q) $， $\mathbf{d} = E_D(d)$。再通过向量相似度计算相关性：$s(q, d) = \mathbf{q}^\top \mathbf{d}$。对于使用归一化向量的系统，也可采用余弦相似度。

其基本过程： $ \text{Query / Document} \rightarrow \text{Embedding} \rightarrow \text{Vector Index} \rightarrow \text{Similarity Search}$

代表方法是 DPR 和 Contriever。

Dense Retrievers 的主要特点包括：

- Allowing Fine-tuning：可以针对特定任务微调。
- Better Adaptation：能够学习更适合任务的语义表示。
- Customizable for More Retrieval Goals：可以针对不同检索目标定制。

综合比较

维度	Sparse Retrieval	Dense Retrieval
文本表示	高维稀疏词项向量	低维稠密语义向量
主要依据	词项匹配及统计权重	向量语义相似性
典型方法	TF-IDF、BM25	DPR、Contriever
索引	倒排索引	向量索引
训练要求	通常无需神经网络训练	通常依赖预训练编码器
优势	高效、可解释、适合精确关键词	语义匹配、可微调、迁移能力
局限	容易受到词汇不匹配影响	训练和索引成本较高，可能遗漏精确实体

两类检索器可以通过 Hybrid Retrieval 结合，在词项匹配与语义匹配之间形成互补。

3.3 Dense Passage Retriever（DPR）

DPR 是面向 Question Answering（QA）的有监督稠密检索器。

双编码器结构（Dual Encoder）

DPR 使用两个独立的 BERT 编码器： $\mathbf q=E_Q(q) $， $\mathbf{d} = E_D(d)$

其中 E_Q 负责查询编码，E_P 负责段落编码，两者不共享参数。

使用 Inner Product Similarity： $\operatorname{sim}(q,p) = E_Q(q)^\top E_P(p)$

检索时对所有候选段落进行评分，选择得分最高的 Top-K 段落： $\operatorname{TopK}(q) = \underset{p\in\mathcal D}{\operatorname{TopK}} \operatorname{sim}(q,p)$

有监督训练数据

DPR 使用 Question-Passage Sets： $\mathcal{D} = \{(q_i, p_i^+, p_{i,1}^-, \cdots, p_{i,n}^-)\}_{i=1}^m$

其中：

- q_i：查询。
- p_i^+：相关段落（Positive Passage）。
- $p_{i,j}^-$：不相关段落（Negative Passage）。
- m：训练样本数量。

对比学习目标

$\mathcal L_i = -\log \frac{ e^{\operatorname{sim}(q_i,p_i^+)} }{ e^{\operatorname{sim}(q_i,p_i^+)} + \sum_{j=1}^{n} e^{\operatorname{sim}(q_i,p_{i,j}^-)} }$

该目标通过增大正例相似度、降低负例相对得分，使检索器能够区分相关与不相关段落。

In-batch Negatives（批内负例）

在一个训练 Batch 中，其他查询的正例段落可作为当前查询的负例。

例如： $(q_1, p_1^+), (q_2, p_2^+), \cdots, (q_B, p_B^+)$

对于 q_1，p_1^+ 是正例，其余段落可作为负例候选。这种方法减少了额外负例采样成本，但存在负例过于简单、难度不足，或错误地将相关段落当成负例的问题。因此，Negative Sample Selection 是影响 DPR 训练效果的重要因素。

3.4 Contriever：无监督通用稠密检索器

Contriever 采用无监督对比学习构建通用检索器。

与 DPR 不同，Contriever 对查询和文档使用共享编码器： $\mathbf{q} = E(q), \mathbf{d} = E(d)$

其相似度仍可通过内积计算： $s(q, d) = E(q)^\top E(d)$

无监督对比学习

Contriever 不要求人工标注的 Question-Passage Sets，而是从未对齐文档（Unaligned Documents）中构建对比学习样本。

对比目标以正样本 k_+、负样本 k_i 和温度参数 τ 为基础。

标准 InfoNCE 损失可写为： $\mathcal L(q,k_+) = -\log \frac{ \exp(s(q,k_+)/\tau) }{ \exp(s(q,k_+)/\tau) + \sum_{i=1}^{K} \exp(s(q,k_i)/\tau) } $

温度系数 τ 控制 Softmax 分布的尖锐程度。

DPR 与 Contriever 对比

比较	DPR	Contriever
定位	Task-specific	General-purpose
训练	Supervised	Unsupervised
编码器	Two Independent BERTs	Shared BERT Encoder
数据	Question-Passage Pairs	Unaligned Documents
目标	特定 QA 任务表现	通用语义检索能力
迁移	可能依赖目标任务适配	更适合零样本与少样本迁移

无监督预训练可能改善 Zero-shot Transfer 和 Few-shot Learning 场景下的鲁棒性，但并不意味着在所有任务上都优于监督检索器。

3.5 DPR 与 Contriever 的性能比较

两组实验比较检索器性能。

1. End-to-end QA：Exact Match Accuracy

Exact Match（EM）表示生成答案与标准答案完全匹配的比例： $\operatorname{EM} = \frac{1}{N} \sum_{i=1}^{N} \mathbb I(\hat y_i=y_i)$

Single 与 Multi 是课件表格中的训练设定标签，不应理解为单轮和多轮问答。

从结果可以观察到：

- DPR（Single）在 NQ 上达到 41.5，高于 BM25 的 32.6。
- DPR 在 TriviaQA 上达到 56.8，高于 BM25 的 52.4。
- BM25 在 SQuAD 上仍具有优势。

- BM25+DPR 的组合并非在所有任务上取得最高分。
- Multi 设定在部分任务上提升显著，但也可能降低其他数据集的表现。

因此，稠密检索通常具有较强的语义匹配能力，但不存在对所有数据集都最优的检索器。

2. 检索 Recall@K

$R@K = \frac{ \text{Top-}K\text{ 中被成功覆盖的相关目标数} }{ \text{全部相关目标数} }$

在开放域 QA 检索评估中，Recall@K 主要反映正确证据是否被前 K 个结果覆盖。

还列出了 Inverse Cloze Task、Masked Salient Spans 等无监督预训练方法，用于比较不同预训练目标对检索效果的影响。从 NQ 数据看，Contriever 的 R@20 与 R@100 均超过 BM25，但仍低于监督训练的 DPR。这说明无监督检索器具有竞争力，而任务相关监督信号仍可能带来额外收益。

DPR 与 Contriever 均已广泛应用于 RAG / RA-LLM 系统：

- DPR：RAG、FiD、RETRO、EPR、UDR 等。
- Contriever：Self-RAG、Atlas、RAVEN 等。

3.6 面向 In-Context Learning 的检索器

此前的检索器主要寻找事实证据，而面向 In-Context Learning（ICL）的检索器主要寻找能够帮助模型完成当前任务的提示或示例。

Prompt Retriever

Prompt Retriever 从候选提示中选择适合当前输入的 Prompt。

结构包括： Training Data → Unsupervised Retriever → Candidate Prompts

再通过 Scoring LM 评价候选提示的质量。

模型还可以使用 Utterance Encoder E_X 与 Prompt Encoder E_P，通过对比学习区分：

- Positive Examples
- Hard Negative Examples

Exemplar Retriever（CEIL）

CEIL 通过检索适合当前输入的 Exemplars（示例），优化上下文学习效果。

其核心过程是：

1. 创建训练数据（Creating Training Data）。
2. 建立示例与输入之间的关联。
3. 使用 DPR Retriever 产生检索得分。
4. 使用语言模型计算示例对任务表现的影响。
5. 通过 Alignment Loss 对齐检索器与语言模型的偏好。

与传统证据检索不同，CEIL 更关注：哪些示例能够使语言模型在当前任务上取得更好的表现？

3.7 Search Engine as Retriever

传统 RAG 通常使用预先建立的知识库，但这种方式存在三类局限：

- 实时网页内容难以及时更新到本地知识库。
- 预定义数据库可能受到存储规模限制。
- 无法直接利用 Google、Bing 等成熟搜索引擎长期优化的排序能力。

因此，可以使用 Search Engine 作为外部检索器。多轮搜索过程：

$x \rightarrow q_1 \rightarrow D_{q_1}$， $D_{q_1}\rightarrow y_1\rightarrow q_2\rightarrow D_{q_2}$， $D_{q_2} \rightarrow y_2 \rightarrow q_3 \rightarrow D_{q_3}$

例如，回答一个人物教育经历的问题时，模型可以先检索人物背景，再检索就读大学，最后检索所获学位。

这类方式允许模型依据中间结果继续搜索，体现了检索与生成交替进行的过程。

但网页搜索并不自动保证信息可信，仍需处理来源可靠性、检索噪声与事实验证问题。

Part 4 Pre-Retrieval Techniques：检索前优化

检索前优化（Pre-retrieval）发生在检索器执行之前，其目标是提高查询与检索系统之间的适配程度。

三种主要方法：

方法	主要操作	解决的问题
Query Rewriting	改写查询	原始表述不适合检索
Query Decomposition	分解查询	复杂问题包含多个信息需求
Query Expansion	扩展查询	查询缺少相关关键词或语义信息

此外，课件将 HyDE 作为独立技术介绍，它通过生成假设文档改变检索输入。

4.1 Query Rewriting 查询改写

查询改写通过语言模型将原始问题转换为更适合检索的表达形式。

$q^{'} = f_{\text{rewrite}}(q)$

其中 q 是原始查询，$q^{'}$ 是改写后的查询。

三种流程：

方法	流程
Direct Retrieval	Input → Retriever → Documents → LLM Reader
LLM Rewriter	Input → Black-box LLM Rewriter → Web Search → Reader
Trainable Rewriter	Input → Small PrLM Rewriter → Web Search → Reader

Trainable Rewriter 进一步通过 Reward 信号优化改写器，使生成的查询更有利于检索命中和下游回答。

查询改写与分解的联系

课件示例：What profession do Nicholas Ray and Elia Kazan have in common? 比较两个人的共同职业。

可以改写为：Nicholas Ray profession, Elia Kazan profession。检索到两个人的职业信息后，再综合得到共同职业 Director。同时体现了 Query Decomposition：将复杂问题拆成多个相对独立的检索子任务。

实验效果

课件在 HotpotQA、AmbigNQ、PopQA 三个数据集上比较了不同改写方式。

其中 F1 用于衡量预测答案与标准答案之间的匹配程度。

对于常见的抽取式问答 Token-level F1： $F_1=\frac{2PR}{P+R}$

实验显示，查询改写在多个 QA 数据集上有效，但可训练改写器并非始终优于 LLM Rewriter。例如 PopQA 上后者的 F1 为 49.74，略高于前者的 49.51。同时，HotpotQA 的普通 Retrieve-then-read 表现甚至低于 Direct，说明引入检索不必然提升答案质量，查询适配与证据质量同样重要。

4.2 HyDE：Hypothetical Document Embeddings

HyDE 的核心思想：先生成一篇可能回答问题的假设文档，再利用该文档的语义表示检索真实文档。

传统检索： $q \rightarrow \text{Retrieve}(q)$

HyDE： $q \xrightarrow{\text{LLM}} d' \xrightarrow{\text{Encoder}} E(d') \xrightarrow{\text{Search}} \mathcal D_q$

其中 d' 是 Hypothetical Document，即模型生成的假设文档。

为什么生成假设文档?

自然语言问题与目标文档在表达形式上可能存在较大差异。例如，问题可能只有几个词，而目标文档是包含背景、术语与详细描述的长段落。HyDE 先生成一段具有文档形式的文本，再执行 Document-to-Document Similarity Search，以减小查询与目标文档之间的表示差异。

课件示例涉及：

- 智齿拔除所需时间。
- COVID-19 对心理健康的影响。
- 韩语表达的领域问题。

这些例子说明，HyDE 可以根据问题类型生成不同风格的假设文档。

关键限制

HyDE 生成的文档不一定事实正确。它的用途是辅助检索，而不是直接充当答案。因此必须区分：

$\text{Hypothetical Document}\neq\text{Verified Evidence} $

其主要风险是生成内容偏离问题，导致检索方向错误；同时也增加了一次生成模型调用的延迟与成本。

4.3 Query Expansion 查询扩展

Query Expansion 不直接替换原查询，而是为其补充相关信息。

Query2doc 说明了通过 LLM 生成文档来扩展查询的方法。

核心公式： $q^+ = \text{concat}(q, [SEP], d')$

其中：

- q 原始查询。
- $\acute{d}$ LLM 生成的扩展文档。
- [*SEP*]：分隔符。
- q^{+}扩展后的查询。

与 HyDE 的区别在于，Query Expansion 保留原始查询，并追加生成内容。

实验比较： MS MARCO dev 与 TREC DL 19 上的 Query2doc 的表现。

- MRR@10：前 10 个结果中第一个相关结果的倒数排名均值。
- R@50、R@1k：前 50、1000 个结果的召回表现。
- nDCG@10：考虑相关性等级与排名位置的归一化折损累积增益。

Query2doc 在两类检索器上均有收益。BM25 的 nDCG@10 从 51.2 提升至 66.2，增加 15.0；DPR 从 64.7 提升至 68.7，增加 4.0。

三种检索前技术对比

技术	对查询的处理	主要特点
Query Rewriting	重新表达查询	提高检索适配性
HyDE	生成假设文档并以其向量检索	缩小问题与文档的语义表示差异
Query Expansion	原查询 + 生成文档	补充关键词与语义信息

Part 5 Post-Retrieval Techniques：检索后优化

检索后优化（Post-retrieval）发生在检索器返回候选结果之后，主要解决结果排序不佳、内容冗余、证据错误等问题。四种重点方法：

- Re-ranking：重新排序候选文档。
- Compression：压缩冗余检索内容。
- Correction：检测并纠正不可靠的检索结果。
- Refiner：抽取与查询相关的原文证据。

5.1 Re-ranking：检索结果重排

初始检索通常追求较高召回率与较低计算成本，但初检排序未必足够准确。

此外，BM25 与神经检索器采用不同的评分机制，其原始分数通常无法直接比较。

因此，Re2G 使用 Reranker 对候选文档进行统一评分。

两阶段检索

普通 RAG 直接使用初检的 Top-K 结果；Re2G 则先收集较大的 Top-N 候选集合，再通过重排器筛选出最终的 Top-K 文档。

BERT Cross-Encoder 重排模型

Interaction Model Based on Sequence-pair Classification.

其输入结构为： $[\mathrm{CLS}]\ q\ [\mathrm{SEP}]\ d\ [\mathrm{SEP}] $。查询和文档被共同输入到 BERT 中，使两者能够在注意力层中发生交互。

1. 取 [CLS] 对应的输出表示： $\mathbf h_{\mathrm{CLS}} = \operatorname{BERT}(q,d)_{\mathrm{CLS}} $；
2. 相关性分类概率：通过线性层与 Softmax $P(r \mid q,d) = \mathrm{Softmax}(W\mathbf{h}_{\mathrm{CLS}} + b)$，其中 $r \in \{\mathrm{yes}, \mathrm{no}\}$；
3. 重排分数： $s_{\mathrm{rerank}}(q,d) = P(\mathrm{yes} \mid q,d)$。

Cross-Encoder 能够对查询与文档进行细粒度交互，但通常需要逐对计算，因此成本高于使用预计算文档向量的双编码器，所以系统通常先执行高效召回，再对少量候选进行精排。

5.2 Retrieved Result Compression：检索结果压缩

检索文档可能包含大量与查询无关的信息，直接输入 LLM 会增加 Token 成本，也可能干扰模型识别关键证据。

Retrieved Result Compression 通过压缩器将长文档转换为更简洁的上下文。

$\mathcal D_q \xrightarrow{\text{Compressor}} c_q$，其中 c_q 是压缩后的上下文。

RECOMP 推理流程为： Retrieve → Compress → Prepend → Black-box LM

**Compressor Learning Objectives **压缩器需要同时满足三个目标：

目标	含义
Concise	尽可能简洁，降低输入成本
Effective	保留有助于下游任务的信息
Faithful	忠实于原始检索内容，不引入虚构事实

这三个目标之间存在权衡：过度压缩可能遗漏关键证据，而仅追求信息完整又无法有效减少 Token。

RECOMP 案例：以日产 Xterra 停产年份为例 When did they stop making the Nissan Xterra?

方法	Context Tokens	Answer	结果
No Retrieval	0	2010	错误
RALM	749	2015	正确
RECOMP	58	2015	正确

$\text{Token Reduction} = \frac{749-58}{749}\times100% \approx92.3% $

压缩比为： $\text{Compression Ratio} = \frac{749}{58} \approx12.91$

说明 RECOMP 在显著减少上下文长度的同时，保留了回答问题所需的关键事实。

5.3 Corrective RAG（CRAG）：纠正式检索增强生成

传统 RAG 往往假设检索结果可直接作为生成依据，但检索器可能返回不相关、错误或存在歧义的文档。

Corrective RAG 引入 Retrieval Evaluator，对检索结果进行评估，并根据结果质量选择不同的知识处理策略。

基本控制流程

****Question → Retrieve → Grade

若检索结果可靠，则直接使用相关证据；若发现不相关或不可靠文档，则

Rewrite Query → Web Search → Answer

两个问答案例对比：

- Henry Feilden 的职业问题：检索到正确人物信息，回答 Politician。
- Death of a Batman 的编剧问题：检索结果混入其他 Batman 作品信息，模型错误回答 Hamm。

该案例说明，错误检索结果可能诱导生成器给出看似合理但实际错误的答案。

CRAG 的三种检索结果状态

完整架构中，Retrieval Evaluator 将结果分为：

状态	处理策略	生成上下文
Correct	Knowledge Refinement	$x + k_{\text{in}}$
Ambiguous	Refinement + Knowledge Searching	$x + k_{\text{in}} + k_{\text{ex}}$
Incorrect	Knowledge Searching	$< br > x + k_{\text{ex}}$

其中：

- x：用户查询。
- k_{in}：从原始检索结果中提炼的知识。
- k_{ex}：通过外部搜索获得的补充知识。

这里的内部知识指初始检索语料中精炼出的内容，并不等同于模型参数化知识。

Knowledge Refinement

对原始检索文档进行： $ \text{Decompose} \rightarrow \text{Filter} \rightarrow \text{Recompose}$

即先将文档分解为细粒度片段，再筛除无关信息，最后重组为更可靠的上下文。

Knowledge Searching

当初始检索结果不可靠时： $x \xrightarrow{\text{Rewrite}} q' \xrightarrow{\text{Web Search}} \{k_1,\ldots,k_n\} \xrightarrow{\text{Select}} k_{\mathrm{ex}}$

包含 Death of a Batman; screenwriter; Wikipedia 的查询作为示例，说明外部搜索可以通过更明确的查询重新寻找证据。

CRAG 的核心贡献是将检索结果质量评估引入生成流程，避免模型无条件信任初检文档。

5.4 Refiner：相关证据抽取与结构化

Refiner 使用单个 Decoder-only LLM，从检索文档中自适应抽取与查询相关的原文内容，并保留必要上下文。

Adaptively extract query-relevant contents verbatim along with the necessary context. 其中 Verbatim 强调尽可能忠实地抽取原始文本，而非自由改写或生成新的事实。

Refiner 的处理流程

$ (q,\mathcal D_q) \xrightarrow{\text{Refiner}} \mathcal C_q$

$ \mathcal C_q \xrightarrow{\text{Downstream LLM}} y$

其中 c_q 为筛选和结构化后的证据。

课件案例

用户询问：Are North Marian High School and Seoul High School both located in the same country?

检索结果包含四个学校的信息。课件中的问题使用 North Marian，而检索结果使用 North Marion，存在拼写不一致。该差异也提示，实体名称匹配错误可能影响证据筛选。

Refiner 通过 Extract and Structure，保留两段关键证据：

1.1 Seoul High School：位于 South Korea。

1.2 North Marion High School：位于 United States。

两种结果：

- Without Refiner：模型受到无关学校信息干扰，错误判断两所学校位于同一国家。
- With Refiner：模型依据筛选后的证据，正确判断两所学校位于不同国家。

Refiner 与 Compression 的区别

比较	Compression	Refiner
核心操作	压缩长文档	抽取查询相关内容
主要目标	降低 Token 成本	减少无关证据干扰
内容处理	可以进行摘要式改写	强调 Verbatim Extraction
输出	精简摘要或压缩上下文	结构化相关证据
代表方法	RECOMP	Refiner

两者都属于检索后优化，但Compression 主要解决上下文冗长，Refiner 更强调证据相关性与保真性。

5.5 RAG 技术体系的综合比较与总结

整个 RAG 系统可以按照信息流分为三个优化阶段：

1. Pre-Retrieval 优化查询表达

Rewriting · Decomposition · HyDE · Expansion

2. Retrieval 从知识库中召回候选证据

BM25 · DPR · Contriever · Search Engines

3. Post-Retrieval 优化检索结果的质量与内容

Re-ranking · Compression · Correction · Refiner

Generation 综合查询与证据生成答案

关键技术对照

技术	处理对象	核心机制	主要作用
BM25	查询、文档	稀疏词项匹配	高效关键词检索
DPR	查询、段落	双编码器、有监督对比学习	提高特定 QA 任务的检索能力

技术	处理对象	核心机制	主要作用
Contriever	查询、文档	共享编码器、无监督对比学习	提高通用语义检索能力
Prompt Retriever	提示模板	候选提示评分与学习	选择有效提示
CEIL	上下文示例	检索得分与 LM 得分对齐	优化示例选择
Query Rewriting	查询	重新表达信息需求	改善查询适配性
HyDE	查询	生成假设文档后检索	改善查询与文档的表示匹配
Query Expansion	查询	原查询与生成内容拼接	扩大检索覆盖范围
Re2G	候选文档	交互式重排	改善候选排序
RECOMP	检索文档	内容压缩	减少 Token 与冗余信息
Corrective RAG	检索结果	质量评估、精炼与外部搜索	修正不可靠证据
Refiner	检索文档	相关内容抽取与结构化	减少证据噪声

重要概念辨析

1. Sparse Retrieval 与 Dense Retrieval

前者依靠词项统计匹配，后者依靠稠密向量的语义相似性。两者可以互补，并非简单的技术替代关系。

2. DPR 与 Contriever

DPR 使用两个独立编码器，通过有监督问答数据学习；Contriever 使用共享编码器，通过无监督对比学习获得通用检索能力。

3. Query Rewriting、HyDE 与 Query Expansion

Rewriting 重新表达查询；HyDE 生成假设文档作为检索输入；Expansion 保留原查询并追加生成内容。

4. Retriever 与 Reranker

Retriever 面向大规模语料进行高效召回，Reranker 对较少候选进行精细评分。两者通常构成粗排与精排的两阶段架构。

5. RECOMP、CRAG 与 Refiner

RECOMP 主要处理上下文冗余，CRAG 主要处理证据质量不可靠，Refiner 主要处理相关证据的抽取与组织。

RAG 的效果不仅取决于是否连接外部知识库，还取决于整条信息处理链的质量。

可以将系统性优化目标概括为：

$${ \begin{aligned} \text{RAG Quality}\ \longleftarrow;&\text{Query Quality}+\text{Retrieval Quality}+\text{Evidence Quality}+\text{Generation Quality} \end{aligned} } $$

LLM 的参数化知识存在局限，因此引入外部检索；检索器决定能够找到哪些信息，检索前优化改善信息需求的表达，检索后优化提高证据的相关性、可靠性与紧凑性，生成器最终利用这些证据完成任务。

Part 6 Agent Memory 智能体记忆

本章围绕智能体如何存储、检索、更新和利用历史信息展开。

核心问题：如何突破大语言模型有限上下文窗口与静态参数知识的限制，使智能体能够在持续交互中积累经验、形成知识、复用技能，并实现长期学习。

本章的知识体系可以归纳为五个层次：

1. 记忆的必要性：LLM 的参数记忆与上下文记忆存在局限，传统 RAG 不能完全替代智能体记忆。
2. 记忆的认知基础：借鉴人类长期记忆的海马体索引理论、模式分离与模式补全机制。
3. 记忆的内容分类：情景记忆（Episodic）、语义记忆（Semantic）和程序性记忆（Procedural）。
4. 记忆的实现与学习：从外部文本存储到参数微调、提示优化、代码库改进及自我反思。
5. 记忆的管理机制：有限上下文下的外部记忆读写、召回、更新与冲突消解。

6.1 智能体记忆的动机与基本概念

6.1.1 从人类持续学习到智能体记忆

人类及多数动物能够通过持续经历形成长期记忆，而传统 LLM 的知识主要来自预训练阶段。模型完成训练后，参数通常保持固定，新的交互信息不会自动转化为永久知识。

课件通过神经突触变化说明生物记忆的形成过程：

- 短期记忆（Short-term Memory）：与短暂的神经活动及突触功能变化相关。
- 长期记忆（Long-term Memory）：涉及更持久的突触结构变化和神经连接调整。

这一类比揭示了智能体设计的基本要求：不仅需要在当前任务中利用信息，还应能够将重要经验保存下来，供后续任务使用。

然而，直接通过修改 LLM 参数实现持续学习存在困难。

1. 灾难性遗忘（Catastrophic Forgetting）

模型学习新知识时，参数更新可能破坏原有知识，导致旧任务性能下降。

2. 涟漪效应（Ripple Effects）

知识之间存在关联。修改一项事实后，模型可能无法同步更新相关知识。

例如，将某人物的国籍由国家 A 修改为国家 B，可能还需要同步更新其所属国家、语言及其他关联事实。

3. 瞬时学习（Transient Learning）

新知识可能仅在当前上下文中有效，或者在后续参数更新中无法稳定保留。

因此，智能体需要一种能够持续存储、组织和调用信息的记忆机制，而不能完全依赖参数更新。

6.1.2 参数记忆与非参数记忆

LLM 的知识存储可以分为两种主要形式。

维度	参数记忆（Parametric Memory）	非参数记忆（Non-parametric Memory）
存储位置	模型权重	外部数据库、文档、记忆库
知识更新	通常需要训练或编辑参数	可以直接修改外部数据
信息访问	通过模型推理隐式调用	通过检索显式获取
持久性	随模型权重保存	由外部存储系统保证

维度	参数记忆（Parametric Memory）	非参数记忆（Non-parametric Memory）
主要局限	更新成本、遗忘、知识冲突	检索误差、信息质量、上下文限制

非参数记忆能够提供额外证据，但并不保证模型正确使用这些证据。

课件研究，说明 LLM 在外部证据与参数知识冲突时，可能接受具有连贯性和说服力的外部信息。

实验包含两种情况：

- Correct Memory：模型原有知识正确，但外部信息提供错误事实。
- Wrong Memory：模型原有知识错误，而外部信息提供正确事实。

例如，模型知道法国首都是巴黎，但外部文本声称首都是尼姆（Nîmes）。模型可能受到外部文本影响而改变答案。这一现象具有双重含义：

- 外部记忆可以帮助纠正模型的过时或错误知识。
- 外部记忆也可能引入错误信息，导致模型放弃原本正确的知识。

因此，记忆系统不仅需要检索能力，还需要证据可信度判断、知识冲突检测和更新控制。

6.1.3 短期记忆与长期记忆

智能体通过与环境交互形成行动—观察循环： $ \text{Reasoning} \rightarrow \text{Action} \rightarrow \text{Observation} \rightarrow \text{Reasoning}$

在这一过程中，记忆负责保存过去的观察、行动和推理结果。

特征	短期记忆（STM）	长期记忆（LTM）
主要载体	当前上下文窗口	外部存储或持久化模型能力
写入方式	追加当前对话与工具结果	显式写入、更新或学习
容量	受上下文长度限制	可扩展
生命周期	通常局限于当前任务或会话	可以跨任务、跨会话保留
主要功能	支持即时推理	支持经验积累和知识复用

课件强调，短期记忆通常具有三个特点：

- Append-only：交互内容不断追加。
- Limited context：上下文容量有限。
- Limited attention：即使信息仍处于上下文中，模型也不一定能够有效关注。

长期记忆则强调 Read and Write，即既能检索已有信息，也能写入新的经验、知识与技能。

需要注意：短期记忆和长期记忆的区别主要是生命周期与使用机制，并不等同于是否使用向量数据库。

6.2 RAG 与 Agent Memory 的关系

6.2.1 传统 RAG 的局限

检索增强生成（Retrieval-Augmented Generation, RAG）通过检索外部知识，为模型生成答案提供参考。

其基本过程为： $q \xrightarrow{\text{Retriever}} D_q \xrightarrow{\text{LLM}} y$

其中：

- q：用户查询。
- D_q：检索得到的相关文档。
- y：最终生成结果。

传统 RAG 的知识库通常具有只读特征： Retrieve $(q) \rightarrow D_q$。这种知识库为 Read-only LTM（只读长期记忆）。因为传统检索语料往往由外部人员编写，例如维基百科编辑者，而非智能体自身产生。

由此带来两方面限制：

1. 经验来源限制：外部知识库主要保存他人的知识和经验，未必适合当前智能体的特定任务。
2. 存储结构限制：文档最初可能面向人类阅读，而不是为智能体的高效检索和推理设计。

相比之下，Agent Memory 需要支持自主写入： Write $(m_t, M_t) \rightarrow M_{t+1}$

其中，M_t 为当前记忆状态，m_t 为新产生的记忆内容。

因此，Agent Memory 不仅需要 Retrieve，还需要 Write、Update 和 Delete 等操作。

6.2.2 为什么传统 RAG 不总是有效

课件使用一个关于斯坦福大学教授与阿尔茨海默病神经科学研究的问答示例，说明传统 RAG 在复杂关联检索中的不足。传统 RAG 通常根据查询与文档之间的相似度返回相关片段，但复杂问题可能要求模型跨多个事实建立关联。

例如： $\text{Stanford Professor} \rightarrow \text{Research Area} \rightarrow \text{Alzheimer's Disease} $

如果相关信息分散在不同文档中，仅依靠单次相似度检索可能无法恢复完整关系。通过人类记忆进行对比，强调人类可以借助已建立的关联，从部分线索恢复相关信息。

由此引出智能体记忆的两个关键要求：

- 关联组织：不仅存储独立文本，还要建立信息之间的联系。
- 关联检索：能够沿已有关系逐步查找，而不是只执行一次相似度匹配。

这也为后续的记忆索引、遍历式检索与推理增强检索提供了动机。

6.3 人类长期记忆的认知基础

6.3.1 海马体索引理论

海马体索引理论（Hippocampal Indexing Theory），作为智能体长期记忆设计的认知启发。

- 新皮层（Neocortex）：参与存储分布式记忆表征。
- 海马体（Hippocampus）：形成指向这些表征及其关联的索引。

记忆可以涉及人物、事件、地点等不同信息。这一理论说明，长期记忆不一定需要将所有内容存储在同一个位置，而可以采用： $\text{Memory Index} \rightarrow \text{Stored Representations} $。索引负责定位和关联，具体记忆内容则分布存储。

6.3.2 模式分离与模式补全

海马体索引机制支持两种重要的记忆能力。

1. 模式分离（Pattern Separation）

将相似经历区分为不同记忆，避免信息相互混淆。例如，两次发生在同一家餐厅的会议虽然地点相同，但时间、参与者和讨论内容不同，系统应将其区分为不同事件。

课件将这一机制与新皮层及海马旁区域（Parahippocampus）联系起来。

2. 模式补全（Pattern Completion）

利用部分线索和已有关联恢复较完整的记忆。例如，用户只提到某次项目会议的地点，系统就可能根据关联找到会议时间、参与者和决策结果。

课件指出，这一机制主要与海马体，特别是 CA3 区域有关。

机制	目标	对 Agent Memory 的启发
Pattern Separation	区分相似记忆	去重、消歧、避免记忆混合
Pattern Completion	从局部线索恢复完整信息	关联检索、多跳检索、上下文恢复

这两种机制相互补充：模式分离提高记忆的可区分性，模式补全提高记忆的可恢复性。

6.4 智能体长期记忆的分类

课件借鉴人类长期记忆体系，将 Agent Memory 按内容分为三类：

记忆类型	英文	存储内容	代表系统
情景记忆	Episodic Memory	经历、事件、交互记录	Generative Agents
语义记忆	Semantic Memory	事实、概念、归纳知识	Generative Agents
程序性记忆	Procedural Memory	技能、操作方法、可执行程序	Voyager

这一分类基于记忆内容，而非底层实现方式。例如，三类记忆都可以使用文本数据库或向量数据库保存，但其语义和用途不同。

6.4.1 情景记忆：Episodic Memory

情景记忆保存智能体经历过的具体事件。

其基本形式为： $m_i=(t_i,e_i,c_i) $

- t_i：事件发生时间。
- e_i：事件内容。
- c_i：事件相关上下文。

该表达式是对课件中事件流结构的抽象，并非课件规定的固定数据格式。课件以 Generative Agents 为例，展示了事件记忆的写入与读取机制。

写入：Append-only Event Streams。

智能体持续记录观察到的事件，形成 Memory Stream（记忆流）。例如：某角色正在计划情人节聚会、某角色邀请朋友参加活动、某角色正在研究聚会安排。这些记录按照时间顺序追加。

读取：Heuristic Retrieval

由于事件流可能非常庞大，系统需要根据启发式评分选择相关记忆。

三个评分因素：

1. Recency：近期性。
2. Importance：重要性。
3. Relevance：相关性。

可以将综合评分概括为： $S(m_i, q) = \alpha R_{\text{recency}}(m_i) + \beta R_{\text{importance}}(m_i) + \gamma R_{\text{relevance}}(m_i, q)$

其中 α, β, γ 为权重。

示例中，系统根据“现在最期待什么”这一问题，从大量事件中检索与情人节聚会有关的记忆，进而生成回答。

因此，情景记忆的核心机制是： Event Recording → Memory Retrieval → Contextual Response

6.4.2 语义记忆：Semantic Memory

语义记忆保存从经历中抽象出的稳定知识，而不是单个事件本身。

Write: LLM reasoning over events 即通过 LLM 对多个事件进行推理、归纳和反思，形成更高层次的知识。

例如，智能体观察到：

- Klaus 正在阅读研究文章。
- Klaus 正在做笔记。
- Klaus 正在与图书管理员讨论研究材料。

系统可以将这些观察归纳为：Klaus Mueller is dedicated to research.

这种由具体观察形成抽象结论的过程，可以表示为： $\{e_1,e_2,\ldots,e_n\} \xrightarrow{\text{Reflection}} k$

其中 k 表示归纳得到的语义知识。

语义记忆的读取方式同样是 Retrieval，但检索对象从具体事件扩展为抽象事实和经验总结。

情景记忆与语义记忆的主要区别：

- 情景记忆回答“发生过什么”。
- 语义记忆回答“从这些经历中知道了什么”。

二者可以相互转化：事件不断积累，经由反思形成语义知识。

6.4.3 程序性记忆：Procedural Memory

程序性记忆保存可重复使用的技能或操作程序。课件使用 Voyager 展示程序性记忆的实现。

Voyager 的三个核心组成部分是：

1. Automatic Curriculum（自动课程）：根据当前能力与探索进度生成新的任务。
2. Iterative Prompting Mechanism（迭代提示机制）：结合执行反馈和错误信息反复改进程序。
3. Skill Library（技能库）：保存已验证的代码技能，供后续任务复用。

例如，智能体可能逐步掌握：Mine Wood Log：采集木材、Make Crafting Table：制作工作台、Craft Stone Sword：制作石剑、Make Furnace：制作熔炉、Combat Zombie：对抗僵尸。

其执行过程为： $\text{Task} \rightarrow \text{Skill Retrieval} \rightarrow \text{Code Execution} \rightarrow \text{Feedback} \rightarrow \text{Skill Update} $

课件强调：

- Write: Code-based skills：将技能保存为代码。
- Read: Embedding retrieval：通过向量检索找到相关技能。

当执行失败时，系统根据环境反馈与错误信息修正代码；当技能通过自我验证后，将其加入技能库。

与情景记忆相比，程序性记忆的重点不是记录过去，而是将成功经验转化为未来可直接执行的能力。

6.5 长期记忆的实现机制

6.5.1 基础实现：文本存储与检索

课件将最简单的长期记忆实现称为 Naïve Implementation。它包含两个操作：

Write 将新的经验追加到文本语料库： $M_{t+1} = M_t \cup \{m_t\}$

Read 使用现成检索器获取相关记忆： $M_q = \text{Retrieve}(q, M)$

例如：BM25、Neural Embeddings（神经向量表示）。

这种实现简单、易于扩展，但主要依赖文本相似性，可能难以处理复杂的关联关系。

6.5.2 改进方向：关联遍历与索引增强

两类更复杂的实现方式：

1. Online Retrieval by Traversal

通过遍历已有关系进行在线检索。与直接返回相似文本不同，这类方法允许系统沿记忆之间的关联逐步查找。

例如： $m_1 \rightarrow m_2 \rightarrow m_3$

这与前文模式补全及复杂关系检索的需求相对应。

2. Index Augmentation with Reasoning/Referral

在写入阶段利用推理或引用关系增强记忆索引。

例如，一条新记忆除了保存文本内容，还可以建立与已有事件、人物或知识的关联。

这意味着记忆写入不再只是追加文本，还包括组织和维护索引结构。

两类方法分别改善：读取阶段的关联探索能力、写入阶段的知识组织能力。

6.6 从记忆到学习：长期能力积累的统一视角

将长期记忆进一步扩展为智能体学习的统一框架。这里的 Memory 不仅指外部文本数据库，还包括能够跨任务保留的模型参数、提示策略、程序和反思经验。

6.6.1 参数微调 Fine-tuning Model Weights

最直接的学习方式是更新模型权重： $\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$

其中：

- θ_t：当前模型参数。
- η：学习率。
- $\mathcal{L}$：训练损失函数。

课件展示了通过多任务问题、多种推理方法和工具调用轨迹生成训练数据的过程。

涉及的方法包括：Chain-of-Thought（CoT）、ReAct、Reflexion。

系统保留正确的执行轨迹，删除错误轨迹，并将有效轨迹重新格式化用于微调。例如，模型可以学习在不同任务中隐式选择适合的推理方法，而不是始终采用固定策略。这种方式将经验写入参数记忆，但可能存在训练成本和灾难性遗忘问题。

6.6.2 跨任务提示优化 Prompt Optimization

第二种方式不直接修改模型参数，而是优化提示指令。一个提示优化过程：

$P_{t+1} = \text{Optimize}(P_t, \mathcal{F}_t)$

- P_t：当前提示。
- $\mathcal{F}_t$：评估反馈。
- P_{t+1}：改进后的提示。

例如，系统针对一组语言任务生成多个候选指令，通过评分模型评估其效果，再保留表现更好的指令。

课件中的图表比较了不同提示优化方法在未见任务上的表现，说明提示优化可以提高跨任务泛化能力。

这类长期记忆保存的是如何指导模型完成任务的策略。

6.6.3 改进智能体自身代码库

第三种方式是将学习成果保存到代码库。

以 SWE-agent 的 Agent - Computer Interface 为例，展示智能体与计算机环境的交互。

智能体可以：Navigate Repository：浏览代码仓库、Search Files：搜索文件、View Files：查看文件、Edit Lines：编辑代码。

环境通过终端、文件系统等接口返回反馈。与参数微调不同，这种方式通过修改外部程序改变系统的后续行为。因此，代码库可以成为一种长期能力存储载体。

6.6.4 保存事件与示例供未来检索

第四种方式是将执行经验保存为可检索的示例。

基本过程为： $\text{Experience} \rightarrow \text{Memory Storage} \rightarrow \text{Future Retrieval}$

其典型实现就是前文 Generative Agents 的 Memory Stream。

这种方式无需改变模型参数，能够通过持续积累历史事件支持后续决策。

6.6.5 自我反思 Self-Reflection

第五种方式是将失败经验和改进建议写入后续提示。

课件使用 Reflexion 来说明这一机制。

基本流程为： $\text{Query} \rightarrow \text{LLM} \rightarrow \text{Action} \rightarrow \text{Environment} \rightarrow \text{Reward} $

随后系统根据行动轨迹和奖励进行反思： $r_t = \text{Reflect}(\tau_t, R_t)$

- τ_t：执行轨迹。
- R_t：环境反馈或奖励。
- r_t：生成的反思内容。

反思内容被加入后续提示，指导下一次尝试。

其特点是：

- 不一定需要更新模型权重。
- 将失败经验转化为自然语言反馈。
- 通过多轮尝试改善后续行为。

6.6.6 多种学习方式的组合

课件最后以 BetterTogether 为例，强调不同学习机制可以组合使用。

例如，交替进行提示优化和参数微调：

$P_{t+1} = \text{OptimizePrompt}(P_t, \theta_t)$

$\theta_{t+1} = \text{FineTune}(\theta_t, P_{t+1})$

核心思想：提示与参数可以协同优化，而不是必须单独选择一种方式。

五种学习机制：

学习方式	主要更新对象	保留的长期能力
Fine-tuning	模型参数	模型内部知识与行为模式
Prompt Optimization	提示指令	任务执行策略
Codebase Improvement	代码库	可执行技能与程序
Experience Storage	记忆库	历史事件与示例
Self-Reflection	反思记录或提示上下文	错误经验与改进建议

这些方法共同说明：智能体长期学习不局限于参数训练，也可以通过外部持久化信息和可复用行为实现。

6.7 外部记忆管理

6.7.1 有限上下文下的记忆架构

即使拥有长期记忆，LLM 在单次推理中仍受到上下文窗口限制。以 MemGPT 风格的外部记忆架构为例，展示了有限上下文与外部存储之间的协作。主要组成部分如下：

组件	作用
System Instructions	保存系统级指令，通常只读
Working Context	保存当前工作信息，可读写
FIFO Queue	按先进先出方式管理近期消息
Output Buffer	为模型输出保留空间
Archival Storage	长期归档存储
Recall Storage	保存可召回的历史交互
Function Executor	执行记忆读写工具
Queue Manager	管理消息队列与上下文

其中，FIFO（First In, First Out）表示先进先出。当上下文窗口接近容量限制时，系统需要决定哪些信息继续保留在工作上下文中，哪些信息移至外部存储。

其基本关系为： Working Context ↔ External Memory

6.7.2 记忆读写作为工具调用

Reads and writes are function calls the agent issues itself. 记忆的读取与写入由智能体主动发起工具调用完成。

例如：recall_storage.search("six flags")

智能体可以根据当前任务检索历史记录，并将结果重新引入上下文。

这一机制具有三个重要特征：

1. 自主性：智能体决定何时检索或写入记忆。
2. 持久性：信息不依赖当前上下文窗口长期存在。
3. 选择性：只将当前任务需要的信息加载进上下文。

因此，外部记忆管理的核心不只是增加存储容量，还包括控制信息何时进入模型的工作上下文。

6.8 记忆冲突与动态更新

6.8.1 记忆更新的必要性

长期记忆可能包含过时、重复或相互矛盾的信息。

例如：

- 旧记忆：用户目前居住在城市 A。
- 新消息：用户已经搬到城市 B。

如果系统仅追加新记录而不处理旧信息，后续检索可能同时返回两个冲突事实。

因此，长期记忆需要动态更新机制。

6.8.2 记忆提取与更新流程

以 mem0 风格的记忆管理流程为例，将其分为两个阶段。

第一阶段：Extraction Phase

系统读取：当前消息、最近交互记录、已有摘要。

通过 LLM 提取新的候选记忆。

$M_{\text{new}} = \text{Extract}(H_t)$

其中 H_t 表示当前可用的交互历史。

第二阶段：Update Phase

系统从已有记忆中检索与新记忆最相似的若干条：

$M_{\text{similar}} = \text{TopK}(M_{\text{new}}, M)$

然后根据新旧信息之间的关系选择更新操作。

6.8.3 四种记忆更新操作

操作	触发条件	处理方式
ADD	不存在相似记忆	当新信息尚未被保存时，创建新的记忆记录
UPDATE	新消息修改已有事实	当新信息改变已有事实时，修改对应记忆
DELETE	新消息否定已有事实	当新信息明确否定已有事实时，删除不再成立的记忆
NOOP	信息已存在或无关	当信息重复、无关或无需保存时，不执行写入

例如：旧记忆：用户正在学习课程 A；新消息：用户已经完成课程 A。系统应更新学习状态，而不是保留两个相互矛盾的当前状态。

这四种操作共同构成长期记忆的基本维护机制： $M_{t+1} = \text{UpdateMemory}(M_t, m_t, o_t)$

其中 $o_t\in {\text{ADD},\text{UPDATE},\text{DELETE},\text{NOOP}} $

需要区分：这是一种对课件更新流程的形式化概括，实际系统还可能涉及时间戳、来源和版本管理等机制。

6.9 智能体记忆的整体架构与总结

6.9.1 外部环境与内部记忆

External Environment 与 Internal Memory 有什么区别？二者都可以通过工具访问，但功能不同。

维度	外部环境（External Environment）	内部记忆（Internal Memory）
核心功能	提供交互对象与行动反馈	保存信息、经验和技能
典型操作	Action / Observation	Read / Write
状态变化	行动可能改变环境	写入或学习改变记忆
主要作用	执行任务、感知世界	支持推理、检索和长期学习

例如，智能体访问网页属于与外部环境交互；将网页中的重要事实保存到长期记忆，则属于记忆写入。

二者并非绝对独立：数据库、文件系统等技术载体既可能承载环境状态，也可能承载智能体记忆。区别主要在于其在系统中的功能。

6.9.2 Agent Memory 的完整工作循环

综合本章内容，智能体的持续学习过程可以抽象为：

这一过程包含两个相互关联的循环：

任务执行循环 Reason → Act → Observe

记忆学习循环 Experience → Store → Retrieve → Reflect → Update

前者使智能体能够完成当前任务，后者使智能体能够积累经验并改善未来表现。

6.9.3 本章核心知识

核心问题	对应概念或技术
为什么需要记忆？	有限上下文、静态知识、灾难性遗忘
为什么传统 RAG 不够？	只读知识库、缺少自主写入、关联检索不足
人类记忆提供什么启发？	海马体索引、模式分离、模式补全
记忆按内容如何分类？	Episodic / Semantic / Procedural
如何读取和写入记忆？	Retrieval、Append、Index Augmentation
如何实现长期学习？	Fine-tuning、Prompt Optimization、Codebase Improvement、Experience Storage、Reflexion
如何管理有限上下文？	Working Context、FIFO Queue、Archival Storage、Recall Storage
如何处理记忆冲突？	ADD、UPDATE、DELETE、NOOP

6.10 本章总结

Agent Memory 的本质是构建一种能够跨任务持续保留和利用信息的机制。

本章的逻辑可以归纳为三个层次。

第一，记忆是对 LLM 固有局限的补充。 参数知识难以实时更新，上下文窗口无法保存全部历史信息，传统 RAG 又主要依赖外部只读知识。因此，智能体需要能够自主读写的长期记忆。

第二，长期记忆不仅是存储，还涉及知识组织和能力形成。 情景记忆记录经历，语义记忆归纳知识，程序性记忆保存技能。通过检索、反思和关联机制，智能体能够将过去的经历转化为当前任务所需的信息与能力。

第三，记忆系统需要动态管理。 记忆的价值取决于能否准确检索、合理更新、处理冲突，并与当前推理过程有效结合。长期学习还可以通过参数微调、提示优化、代码改进和反思等多种方式实现。

最终，智能体的能力不再只由预训练模型决定，而由模型、外部记忆、工具和环境交互共同形成。

$$\text{Agent Capability} = f(\text{Model}, \text{Memory}, \text{Tools}, \text{Environment})$$

Lecture 8 Agent Infrastructure 智能体基础设施

本讲围绕智能体系统的基础设施展开，重点研究两个相互关联的问题：

1. 推理基础设施（Inference Infrastructure）：如何通过 KV Cache 复用、分层存储和选择性重计算，降低长上下文推理的延迟与成本。
2. 智能体软件架构（Agent Software Architecture）：如何利用复合 AI 系统（Compound AI Systems）与 DSPy，将单一模型调用转化为可组合、可控制、可优化的程序。

两部分分别从计算资源管理与程序结构设计出发，提高智能体系统的效率、可靠性和可维护性。

Part1 长上下文推理与 KV Cache

1.1 长上下文推理的机遇与挑战

长上下文推理（Long-context Inference）使大语言模型能够在单次请求中处理大量信息，包括文档、书籍、新闻、代码仓库、对话历史、视频与会议记录等。其核心价值不仅是扩大上下文窗口（Context Window），还在于使模型能够利用更多外部信息完成复杂任务。

上下文长度增加会带来三方面的问题，这三种代价共同限制了长上下文推理的大规模部署。

1. 推理延迟：输入越长，响应时间越长

LLM 推理通常分为两个阶段：

- Prefill（预填充）：处理输入上下文，计算各层的注意力中间状态。
- Decode（解码）：基于已有状态，自回归地逐个生成输出 Token。

总推理时间可以表示为： $T_{\text{total}} = T_{\text{prefill}} + T_{\text{decode}} + T_{\text{other}}$

其中，T_{other} 包括排队、调度、数据传输等开销。

对于标准全注意力 Transformer，长度为 L 的输入，其 Prefill 计算复杂度可近似表示为： $C_{\text{prefill}} = O(Ld^2 + L^2d)$

- L：输入 Token 数量；
- d：隐藏状态维度；
- $O(Ld^2)$：线性投影与前馈网络的主要计算项；
- $O(L^2d)$：标准注意力计算项。

因此，Prefill 的计算成本并非在所有情况下都严格随上下文长度线性增长。对于标准全注意力，注意力部分具有二次复杂度。

2. 推理成本：GPU 计算与资源消耗增加

长上下文需要更多 GPU 计算资源，并增加 KV Cache 的存储需求。GPU 成本随使用量和上下文长度增加。实际成本不仅取决于 Token 数量，还与模型规模、并发程度、硬件利用率和缓存命中率有关。

3. 输出质量：长上下文中的长尾知识可能难以被准确利用

长上下文并不保证模型能够有效利用其中的全部信息。随着上下文长度增加，模型可能难以准确定位、整合和利用部分信息，尤其是长尾知识（Long-tail Knowledge）。

因此，上下文容量（Context Capacity）不等于有效信息利用能力（Effective Context Utilization）；长上下文优化既需要解决计算效率问题，也需要关注信息利用质量。

1.2 KV Cache 的原理

KV Cache（Key-Value Cache）是 Transformer 自回归推理中的一种中间状态缓存机制。

在注意力计算中，输入隐藏状态 X 通过线性变换生成： $Q = XW_Q, K = XW_K, V = XW_V$

注意力输出为： $\operatorname{Attention}(Q,K,V) = \operatorname{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right)V$

- Q（Query）：当前 Token 用于查询其他 Token 的表示；
- K（Key）：用于与 Query 计算匹配程度；
- V（Value）：用于生成注意力加权结果。

在因果注意力（Causal Attention）中，历史 Token 不依赖未来 Token。因此，生成新 Token 时，已有 Token 的 K、V 可以在相同计算条件下复用。

若已经生成 t 个 Token，则： $K_{\text{cache}}^{(t)} = [K_1, K_2, \cdots, K_t]$，$V_{\text{cache}}^{(t)} = [V_1,V_2,\ldots,V_t] $

生成第 $t+1$ 个 Token 时，只需计算新的 K、V，并更新缓存：$K_{\text{cache}}^{(t+1)} = [K_{\text{cache}}^{(t)}; K_{t+1}]$， $V_{\text{cache}}^{(t+1)} = [V_{\text{cache}}^{(t)}; V_{t+1}]$

KV Cache 避免了对历史 Token 的重复 K、V 计算，但新 Token 仍需对历史缓存执行注意力操作，因此不能将其理解为消除了全部注意力计算。

KV Cache 的基本流程

该流程说明：无论 KV 是重新计算还是从缓存加载，只要缓存有效且计算语义一致，后续生成阶段就能够使用它。

1.3 Prefill 瓶颈与推理性能指标

将长上下文推理的主要瓶颈定位于 Prefill 阶段，并重点使用两个指标评估性能。

1. Time to First Token（TTFT，首 Token 延迟）

指从提交请求到产生第一个输出 Token 的时间。

$TTFT = T_{\text{queue}} + T_{\text{prefill}} + T_{\text{first decode}} + T_{\text{overhead}}$

长上下文通常会显著增加 T_{prefill}，从而提高 TTFT。

2. Inter-Token Latency（ITL，Token 间延迟）

指相邻两个输出 Token 之间的时间间隔：$ITL_i=t_{i+1}-t_i$ ，*平均ITL可表示为*：$\overline{ITL} = \frac{1}{N-1} \sum_{i=1}^{N-1}(t_{i+1}-t_i) $

TTFT 衡量开始响应的速度，ITL 衡量持续生成的速度。

二者并不等价：减少 Prefill 时间可能显著改善 TTFT，但不一定提高 Decode 阶段的生成速度。

1.4 跨请求 KV Cache 复用

传统 KV Cache 主要用于同一请求内部的自回归解码，而 PPT 进一步提出跨请求、跨推理引擎的缓存复用。

其基础是：Contexts are often shared across queries.

多个请求可能共享相同的上下文，例如：

- Book：针对同一本书进行多次问答；
- News：对相同新闻材料执行不同分析；
- Docs：企业内部多个用户访问相同文档；
- Chat History：多轮对话重复包含历史消息。

若每个请求都独立执行 Prefill，就会对共享内容进行重复计算。

You Only Prefill Once（YOPO）

当一个推理引擎已经完成某段上下文的 Prefill 后，其他引擎应尽可能复用已有 KV Cache，而非重新计算。

设共享上下文的 Prefill 成本为 C_p，共有 N 次请求，不使用跨请求缓存时：$C_{\text{no-cache}} = NC_p$。理想情况下，仅首次请求需要计算：$C_{\text{cache}} = C_p$，理论上可节省：$S=1-\frac{1}{N} $。

实际系统还需要考虑缓存查找、加载、传输和维护开销，因此并不能达到完全零成本的复用。

结论： KV Cache 从单次推理的计算优化机制，进一步发展为跨请求共享的计算结果。其价值在于减少重复 Prefill，但要实现大规模复用，还必须解决存储容量、数据传输和缓存管理问题。

Part 2 KV Cache 的分层存储与跨设备管理

2.1 KV Cache 的存储容量问题

为了最大化 KV Cache 复用，系统需要存储更多缓存，并延长其保留时间。

理想情况下，每份 KV Cache 应当保存至最后一次复用完成。但由于未来请求无法准确预测，实际系统需要根据访问频率、存储成本和容量限制制定缓存保留与淘汰策略。

KV Cache 的存储量计算

对于具有 N_{layers} 层的 Transformer，KV Cache 的理论存储量为：$M_{\text{KV}} = 2LN_{\text{layers}}N_{\text{kv-heads}}d_{\text{head}}b$

- L：缓存的 Token 数量；
- N_{layers}：Transformer 层数；
- $N_{\text{kv-heads}}$：每层 KV Head 数量；
- d_{head}：每个 Head 的维度；
- b：每个元素的存储字节数；
- 系数 2：分别对应 Key 和 Value。

因此， $M_{\text{KV}}\propto L $即 KV Cache 大小与上下文长度线性相关。在其他条件相同时，增加上下文长度会等比例增加 KV Cache 容量。

对不同模型的 KV Cache 大小进行了比较，说明即使将整张 GPU 显存全部用于缓存，也只能保存有限数量的长文档。以 Llama 3 70B 为例，按图中约 300 GB / 10^6 Token 估算，100K Token 的 KV Cache 约为：$M_{\text{100K}} = 300\times\frac{100000}{1000000} = 30\text{ GB} $，若 H100 显存容量为 80 GB，理论上最多保存： $N_{\text{docs}} = \left\lfloor\frac{80}{30}\right\rfloor = 2$。

这尚未计入模型权重、运行时工作区和其他显存占用。

因此，仅依赖 GPU 显存无法满足大规模、长期的 KV Cache 复用需求。

2.2 分层存储架构 Hierarchical Storage

PPT 提出了 KV Cache 存储层次的扩展路线：

GPU Memory: 最快、容量有限，服务于当前推理

CPU RAM: 提供更大容量，支持 GPU - CPU Offloading

SSD: 持久化与大规模本地缓存

S3 / Remote Storage: 远程、跨节点及跨云共享

越往下，通常容量越大、单位容量成本越低，但访问延迟和传输代价越高。

技术演进顺序为： $\text{GPU (2024)} \Rightarrow \text{CPU RAM (2025 Q1)} \Rightarrow \text{SSD} \Rightarrow \text{S3 / Cross-cloud} $分。

分层存储的基本策略是将高频访问的热数据保留在高速存储中，将低频访问的冷数据迁移到容量更大、成本更低的存储介质中。

2.3 计算成本与存储成本的权衡

Compute (Prefill) Cost ≫ Storage (KV Cache) Cost，重新执行 Prefill 的成本通常显著高于保存 KV Cache 的成本。

需要注意，图中采用两种不同的计费单位：Compute：每百万 Token 的 Prefill 成本；Storage：每百万 Token 每小时的 KV Cache 保存成本。因此，比较时必须考虑缓存保存时间。

设：C_p：一次 Prefill 的计算成本；C_s：每小时保存对应 KV Cache 的成本；T：缓存保存时长；R：保存期间发生的有效复用次数；C_l：每次加载缓存的成本。

缓存的净收益为：$B = RC_p - (TC_s + RC_l)$，当$RC_p>TC_s+RC_l $缓存才具有正向经济收益。

进一步得到： $R(C_p - C_l) > TC_s$，在 $C_p > C_l$ 的前提下，复用次数的盈亏平衡阈值为： $R> \frac{TC_s}{C_p-C_l}$

这属于基于课件成本比较的进一步推导，并非 PPT 直接给出的公式。

2.4 三种 KV Cache 管理机制

传统缓存方式与三种跨 GPU 管理机制对比：

机制	核心操作	主要目的
Traditional KV Cache	缓存仅存在于单次请求和单个引擎中	加速同一请求的解码
CPU Offloading	GPU → CPU	扩展缓存容量与生命周期
Prefix Caching	在不同请求、不同引擎间共享前缀 KV	避免重复计算相同前缀
PD Disaggregation	在 Prefill 与 Decode 引擎之间传输 KV	将两类推理负载分离

CPU Offloading（CPU 卸载）

将 KV Cache 从 GPU 显存迁移到 CPU 内存，在需要时重新加载，从而降低 GPU 显存压力。

Prefix Caching（前缀缓存）

当多个请求具有相同的 Token 前缀时，可以复用该前缀的 KV Cache。

例如：$P_1 = [A, B, C, Q_1]$，$P_2 = [A, B, C, Q_2]$，两个请求均可以复用 $[A, B, C]$ 对应的 KV Cache。

但前缀缓存通常不能直接复用出现在不同前文之后的非前缀片段。这一限制将在第四章进一步讨论。

Prefill-Decode Disaggregation（预填充与解码分离）

将 Prefill 和 Decode 部署到不同推理引擎，由 Prefill 引擎计算 KV Cache，再传输给 Decode 引擎继续生成。其目标是根据两种阶段不同的计算和内存访问特征分别优化资源配置。

三种机制并非互斥，而是可以在同一推理系统中组合使用。

Part 3 LMCache：独立的 KV Cache 基础设施

3.1 LMCache 的设计目标与架构

LMCache 是位于推理引擎与存储后端之间的独立 KV 缓存层，旨在为不同推理引擎提供统一的 KV Cache 管理能力。

四种基本缓存操作语义：

1. GPU - GPU Transfer：在不同 GPU 之间传输 KV Cache。
2. GPU - CPU Offloading：将 KV Cache 从 GPU 卸载到 CPU。
3. CPU - CPU Transfer：在不同 CPU 内存空间之间传输缓存。
4. Storage Offloading：将 KV Cache 卸载到持久化或远程存储。

该架构将缓存管理从推理引擎内部抽离，使缓存能够具有独立的生命周期，并被多个引擎共享。

3.2 LMCache 的三项核心挑战

挑战	问题来源	LMCache 的解决方向
缓存 I/O 效率低	分页内存管理、L2/L3 存储访问和 I/O 干扰推理	优化传输路径与缓存操作
难以适应推理引擎演进	vLLM、SGLang 的调度器与内存分配器不断变化	通过稳定接口降低耦合
缺少交互式管理能力	运维系统无法直接控制缓存	提供实时查询、迁移和管理 API

新模型频繁发布，可能采用不同的内存分配和缓存布局。因此，如果外部缓存系统深度依赖推理引擎内部实现，就会产生较高的适配成本。LMCache 的设计目标是在推理引擎持续变化的情况下，尽量保持缓存层的独立性。

3.3 LMCache 的性能优化技术

技术优化可以整合为三个主要方向：

1. 计算与 I/O 重叠（Compute - I/O Overlapping）

通过异步执行，使 KV Cache 加载或卸载与 GPU 计算同时进行。

相关技术包括：

- Layer-wise Pipelining：按 Transformer 层进行流水线传输；
- Request Async & Prefetch：异步请求与预取；
- KV Cache Proxy：通过缓存代理协调数据访问。

不采用重叠执行时： $T_{\text{serial}}=T_c+T_{io}$ ； *理想完全重叠时*：T_{\text{overlap}} \approx \max(T_c,T_{io})$

因此理论上节省的时间为： $\Delta T = T_c + T_{io} - \max(T_c, T_{io})$。其中， T_c 表示计算时间， T_{io} 表示数据传输时间。

实际系统仍存在流水线启动、同步与资源竞争开销。

2. 批量操作（Batched Operations）

GPU 内存中的最佳计算粒度可能小于网络与存储设备适合处理的数据粒度。如果直接传输大量小型缓存块，会产生频繁的操作开销。因此，LMCache 通过批量操作提高 I/O 效率，包括：

- Configurable Chunk Size：可配置缓存块大小；
- Customized CUDA Kernel：定制 CUDA 内核；
- Batched KV Cache Store/Load：批量缓存读写；
- GPU Transfer Buffering：GPU 传输缓冲。

设每次 I/O 固定开销为 t_0，单次数据传输量为 s，带宽为 B，则： $T_{\text{IO}}=t_0+\frac{s}{B}$。当大量小数据分别传输时，固定开销会被重复支付。通过合并操作，可以减少 I/O 调用次数。

3. 最小化数据复制（Minimum Data Copy）

主要包括：

- Zero Copy：在 PD 分离、远程存储或磁盘传输中尽量减少额外的数据复制；
- Eviction-based Offloading：根据缓存淘汰策略决定哪些缓存需要迁移到较低层存储。

其共同目标是减少不必要的内存复制、带宽消耗和 GPU 资源占用。

3.4 KVConnector：推理引擎与缓存层解耦

KVConnector 为推理引擎和外部缓存层定义相对稳定的交互边界，使 LMCache 不必直接依赖 vLLM 内部调度器或内存分配器的所有实现细节。七个接口如下：

API	功能
build_connector_meta	构建请求与缓存加载相关的元数据
start_load_kv	启动从连接器缓冲区加载 KV Cache
wait_for_layer_load	检查或等待指定层的 KV 加载完成
save_kv_layer	将某层 KV 从 vLLM 分页缓冲区保存至连接器缓冲区
wait_for_save	等待缓存保存操作完成
get_num_new_matched_tokens	获取远程缓存中新匹配的 Token 数量
update_state_after_alloc	临时缓冲区分配后更新连接器状态

这些接口可以归纳为四种功能：元数据维护、缓存加载、缓存保存和状态同步。

其核心价值是实现推理计算与缓存管理的接口解耦。

3.5 LMCache Controller：交互式缓存管理 API

与 KVConnector 主要服务于推理引擎内部交互不同，LMCache Controller 面向用户、管理员和编排系统（Orchestrators）提供缓存控制能力。六个管理 API：

API	功能	典型应用
Lookup	根据 Token 列表查询缓存	检查缓存命中
Clear	清除缓存	释放资源、缓存维护
Pin	持久化缓存或设置 TTL	保留高价值缓存
Move	将缓存迁移到其他位置	跨节点迁移、负载调度
Compress	压缩 KV Cache	减少存储与传输开销
CheckFinish	检查非阻塞控制操作是否完成	异步操作状态管理

其中，TTL（Time to Live）表示缓存的有效保留时间。

KVConnector 与 Controller 的区别在于：KVConnector 解决推理引擎如何加载和保存 KV；Controller 解决外部系统如何查询、控制和管理 KV。

3.6 LMCache 的性能实验与复用特征

1. KV Cache Offloading 性能

实验比较了 LMCache、原生 vLLM（Naive vLLM）以及两种商业方案。TTFT 随 QPS（Queries Per Second，每秒请求数）的变化；ITL 随 QPS 的变化。图中 LMCache 在较高请求负载下仍保持较低的 TTFT 和 ITL，体现了缓存复用与卸载机制在重复上下文工作负载中的优势。

这一实验结果依赖具体模型、硬件、请求分布和缓存命中情况，不能直接推断所有场景都具有相同的性能提升。

2. 实际 KV Cache 复用率

右侧 CDF（Cumulative Distribution Function，累积分布函数）图展示了不同缓存复用率的分布。

其重要结论是：缓存复用具有明显的不均衡性，少量高复用用户或工作负载能够产生较高的缓存收益。

这意味着缓存管理策略应当关注复用频率，而不只是缓存大小。

3.7 实际部署经验与技术趋势

KV Cache 基础设施发展的技术趋势。

1. 存储 I/O 性能持续提升：$100\text{ MB/s (S3)} \rightarrow 1\text{ GB/s (S3 Express One)} \rightarrow \sim10\text{ GB/s (CAIOS)} $
2. 新型注意力技术可以减少 KV Cache 的存储需求。
3. 系统需要综合考虑缓存压缩、远程数据传输、缓存复用价值和存储层次，而非单纯追求更大的缓存容量。

核心结论是：KV Cache 的价值不仅取决于是否能够缓存，还取决于是否能够以足够低的代价保存、定位、传输和复用。

Part 4 CacheBlend：非前缀 KV Cache 的选择性重计算

4.1 Prefix Caching 的局限性

传统前缀缓存适用于多个请求共享相同 Token 前缀的场景，但在 RAG（Retrieval-Augmented Generation）中，检索结果往往由多个独立文档块组成。

例如： $P=[D_1,D_2,D_3,Q] $。其中 D_i 表示检索文档块，Q 表示用户查询。

假设 D_2 的 KV Cache 是在其单独作为输入时生成的，那么将其放在 D_1 后面，并不能保证缓存仍然有效。

原因：KV Cache 不仅与当前 Token 内容有关，还与其前文上下文有关。

因此，相同文本片段在不同上下文位置上可能产生不同的 KV 表示。

简单拼接缓存为何会降低质量？

两个文档块分别包含 Messi 和 Ronaldo 的进球信息，问题要求比较两者的进球数。

完整的 Prefill 可以在 Transformer 的后续层中建立跨文档的信息交互。

但如果直接拼接分别计算得到的文档 KV Cache：$KV_{\text{concat}} = [KV(D_1); KV(D_2)]$ 就可能缺失在完整上下文中应当发生的跨文档注意力交互。

Missing Cross-Attention（缺失跨片段注意力），这里的 Cross-Attention 是指不同文档片段之间的注意力交互，并非特指 Encoder - Decoder 架构中的 Cross-Attention 模块。

对于完整上下文，第二个文档的某层隐藏状态可能依赖第一个文档：$H_{D_2}^{(l)} = f_l(H_{D_1}^{(l-1)}, H_{D_2}^{(l-1)})$，而独立计算 D_2 时，这种依赖不存在。

因此：$KV(D_1 \oplus D_2) \neq KV(D_1) \oplus KV(D_2)$

一般情况下，不能通过简单拼接独立 KV Cache 来获得完整上下文 Prefill 的结果。

4.3 计算效率与生成质量的权衡

三种缓存策略：

策略	计算量	质量特点
Full Prefill	高	保留完整上下文交互
Reusing All KV Caches	低	可能缺失跨片段注意力
CacheBlend	介于两者之间	通过选择性重计算恢复关键交互

CacheBlend 的核心思想：Selective Prefill / Selective Recompute（选择性预填充或选择性重计算）。并非重新计算所有 Token，而是在 Transformer 的不同层中选择部分重要 Token，重新计算其 KV 表示，其余 Token 继续复用已有缓存。

4.4 CacheBlend 的分层重计算机制

完整 Prefill 对所有层、所有相关 Token 重新计算 KV；CacheBlend 则只重计算每层的一部分 Token。

设第 l 层需要重计算的 Token 集合为：$S_l\subseteq{1,2,\ldots,L}$ ，*那么*：KV_i^{(l)} = \begin{cases} KV_{i,\text{recomputed}}^{(l)}, & i\in S_l\ KV_{i,\text{cached}}^{(l)}, & i\notin S_l \end{cases}$

不同层可以选择不同的 Token 集合： $S_1\neq S_2\neq\cdots\neq S_N $

这是因为不同层的隐藏状态和注意力交互需求并不相同。

其基本目标是尽量减少重计算量，同时恢复对生成质量影响较大的跨片段交互。

需要注意，课件展示了分层选择性重计算的机制，但没有在该页完整展开 Token 选择算法的数学细节。

4.5 CacheBlend 的性能实验

实验比较了完整 KV 重计算、仅前缀缓存、直接 KV 复用和 CacheBlend。

1. 响应延迟

使用 F1 Score 与 TTFT 评估不同方案的质量和延迟。CacheBlend 在保持较高问答质量的同时，实现了约 3× 的响应速度提升。

如果基线延迟为 *T*，速度提高三倍意味着：$T_{\text{CacheBlend}}\approx\frac{T}{3}$，对应的延迟降低比例约为： $1-\frac{1}{3} = 66.7\%$

2. 吞吐量

使用平均请求到达率与 TTFT 的关系展示系统吞吐能力。图中 CacheBlend 相较完整重计算可实现约 3× 的吞吐量提升。吞吐量提升的主要来源是减少重复 Prefill 计算，使相同硬件能够处理更多请求。

4.6 KV Cache 作为 AI-Native Data

将 KV Cache 提升到更一般的概念：AI-Native Data（AI 原生数据）

含义：文本、对话、文档等上下文在 LLM 内部会被转换为可供后续计算使用的 KV 表示。与原始文本相比，KV Cache 保存了模型在特定计算条件下形成的中间表示。

三项价值：

- 可复用性：避免重复计算，降低延迟与 GPU 成本。课件给出 5 - 10× 响应速度提升的潜力。
- 注意力信息表示：KV Cache 与模型注意力计算直接相关。
- 上下文操作能力：支持更智能的上下文检索、压缩与融合。

但 KV Cache 不能脱离模型结构、位置编码、上下文依赖和缓存格式等兼容性条件任意复用。

结论： Prefix Caching 解决相同前缀的重复计算问题，CacheBlend 则针对非前缀文档片段，通过少量重计算恢复必要的上下文交互，实现计算效率与生成质量之间的平衡。

Part 5 复合 AI 系统 Compound AI Systems

5.1 从单体模型到模块化 AI 系统

传统 AI 应用通常以单个大语言模型（Monolithic LM）为核心，通过一次或少量模型调用完成复杂任务。

这种架构存在三个主要问题：

- 控制困难（Hard to Control）：模型内部的推理与决策过程难以进行精细控制。
- 调试困难（Hard to Debug）：输出错误时，难以确定错误来源。
- 改进困难（Hard to Improve）：修改提示词或更换模型可能影响整个系统的行为。

Compound AI Systems：使用语言模型作为专门化组件构建的模块化程序（Modular Programs）。

其核心思想是将复杂任务拆解为多个具有明确职责的模块，通过程序控制模块之间的信息流和执行顺序。

5.2 Compound AI Systems 的五项优势

通过 RAG、多跳检索和组合式报告生成等例子，归纳了复合 AI 系统的五项优势：

优势	核心含义
Quality（质量）	组合职责更明确的模型能力，提高系统可靠性
Control（控制）	利用工具和程序逻辑控制执行过程，并逐步改进
Transparency（透明性）	调试执行轨迹，向用户提供结果来源
Efficiency（效率）	使用较小模型，将知识和控制流交由外部组件处理
Inference-time Scaling（推理时扩展）	在推理阶段系统性搜索更优输出

1. Quality：模块化提高输出质量

通过任务分解，可以为不同子任务配置适合的模型或工具。

例如，报告生成系统可以分别完成信息检索、内容规划、段落生成和结果检查，避免让单一模型承担所有职责。

2. Control：显式控制任务执行

多跳 RAG（Multi-Hop RAG）可以根据中间结果执行下一轮检索。

这种机制允许系统通过工具获取外部证据，并根据任务需要调整执行路径。

3. Transparency：可追踪与可解释

系统能够记录检索结果、工具调用、模块输入输出和中间决策。

这些记录既有助于定位错误，也可以为用户提供结果来源（Attribution）。

4. Efficiency：降低系统成本

复合系统可以将知识存储、检索和流程控制从模型参数中分离出来。

因此，部分任务可以使用较小的模型完成，而不必完全依赖大规模模型。

5. Inference-time Scaling：推理时扩展

系统可以在推理阶段投入更多计算资源，例如生成多个候选答案、进行搜索或验证。

其目标不是简单增加 Token 数，而是通过结构化搜索提高获得高质量输出的概率。

5.3 Agentic Frameworks 的三种类型

智能体框架：

类型	代表框架	主要关注点
Agent Development	LangGraph、AutoGen、Crew	会话恢复、低代码、精细控制、并发、流式执行
Data-Driven Optimization	DSPy、TextGrad、Trace	结构化配置、结构化日志、优化算法
Holistic Framework	TapeAgents	同时支持状态管理、结构化日志和数据驱动优化

其中，Agent Development 框架主要解决智能体如何执行任务，Data-Driven Optimization 框架主要解决智能体如何通过数据改进性能。

TapeAgents 试图结合两者，将智能体表示为可恢复的模块化状态机（Resumable Modular State Machine），并通过结构化日志支持后续优化、微调数据生成和日志复用。

5.4 现有复合系统的 Stringly-Typed 问题

虽然 Compound AI Systems 在架构上具有模块化特征，但很多系统仍然高度依赖人工编写的 Prompt 字符串。

传统 Prompt 往往混合五种不同职责：

职责	对应抽象	作用
输入输出行为	Signature	定义模块要完成什么任务
推理策略	Predictor	定义如何执行模型推理
输入输出格式转换	Adapter	构建模型输入并解析输出
目标与约束	Metrics / Assertions	定义评价标准和行为限制
参数优化	Optimizer	优化提示词或模型权重

当这些职责都被写入同一个字符串时，系统架构就会与具体 Prompt 的表达方式耦合。这被称为 Stringly-Typed 问题，即系统过度依赖缺乏明确结构约束的字符串。其后果是修改 Prompt 可能影响多个模块；模型迁移成本较高；优化目标与程序逻辑难以分离；系统难以自动优化。

进一步的研究方向：将 Compound AI Systems 抽象为由自然语言类型化模块组成、且能够学习和优化自身行为的程序。**这构成了 DSPy 的设计动机。**

Part 6 DSPy：从 Prompt Engineering 到 Language Model Programming

6.1 DSPy 的定义与设计理念

DSPy（Declarative Self-improving Language Programs in Python）是由 Stanford NLP 研究人员开发的语言模型编程框架。其核心理念为Programming over Prompting（以编程代替直接操作提示词）。

传统方法主要依赖人工设计 Prompt，而 DSPy 将任务定义、执行策略与参数优化分离。

具体而言：

- 程序员定义任务的输入输出和模块关系；
- 框架负责将程序结构转换为模型可以执行的提示；
- 优化器根据数据和评价指标改进提示词或模型参数。

因此，DSPy 并不是取消 Prompt，而是将 Prompt 从人工维护的主要对象转变为可自动生成和优化的实现细节。

6.2 DSPy 的三个核心抽象

DSPy 的核心概念：

传统方法	DSPy 抽象
Hand-written Prompts	Signatures
Prompting Techniques / Chains	Modules
Manual Prompt Engineering	Optimizers + DSPy Compiler

这三个抽象分别回答三个问题：

- Signature：任务是什么？
- Module：采用什么推理策略？
- Optimizer：如何根据评价结果改进程序？

6.3 Signature：声明式输入输出规范

Signature 是 DSPy 中描述模型任务的声明式接口。Signature 主要包含三个要素：子任务的简要描述；一个或多个输入字段（Input Fields）；一个或多个输出字段（Output Fields）。例如："context, question -> answer"

该 Signature 定义了一个接收上下文和问题、输出答案的任务。其重点是声明需要完成的文本转换，而非手动规定模型必须使用怎样的提示词。

更完整的声明方式可以写为：

```
import dspyclass AnswerQuestion(dspy.Signature):
"""Answer the question using the given context."""
context = dspy.InputField()
question = dspy.InputField()
answer = dspy.OutputField()
```

6.4 Module：将推理技术封装为程序组件

Module 用于封装特定的模型调用方式或推理策略。典型模块包括：

- `dspy.Predict`：基本预测；
- `dspy.ChainOfThought`：链式思考；
- `dspy.ProgramOfThought`：程序辅助推理。

例如：

```
qa = dspy.Predict("question -> answer")mt = dspy.ChainOfThought(
"english_document -> french_translation")rc = dspy.ProgramOfThought(
"contexts, question -> answer_found: bool")
```

这些模块可以通过不同的 Signature 完成不同任务。

Module 与 Signature 的区别在于：Signature 定义任务接口，Module 负责执行相应的推理过程。

6.5 Adapter 与 Predictor：从程序到 Prompt

DSPy 内部的 Prompt 转换过程。例如：

```
self.generate_query = dspy.ChainOfThought(
"context, question -> query")
```

DSPy 可以根据模块和 Signature 生成结构化 Prompt，其中包含：输入字段说明；输出字段说明；字段格式；推理字段（Reasoning）；最终输出字段（Query）。

在这一过程中：Adapter 负责将结构化字段转换为模型输入，并解析模型输出；Predictor 负责根据模块定义执行模型预测。因此，程序员不必为每次调用手动编写完整 Prompt，而是通过统一接口定义模型行为。

6.6 DSPy 的完整工作流程

DSPy 的开发流程的五个阶段：

1. Collect Dataset：收集输入输出示例，构建优化数据集
2. Write DSPy Program：定义 Signatures、Modules 和信息流
3. Define Validation Logic：确定评价指标、约束和优化器
4. Compile Program：根据数据和指标优化提示词或模型参数
5. Iterate：改进数据、程序结构或验证逻辑

DSPy 的编译过程可抽象为：$P^* = \text{Compile}(P, D, M, O)$

P：原始 DSPy 程序；D：用于优化的数据；M：评价指标；O：优化器；P^*：经过优化的程序。

这里的 Compile 不等同于传统编程语言中的机器码编译，而是针对模型调用行为进行参数搜索和优化。

6.7 Optimizer：自动提示词优化

优化器可以同时调整程序中多个模块的提示词或模型参数，而不是孤立地优化单次模型调用。

其典型过程包括：

1. 使用 Adapter 为各模块构建初始 Prompt；
2. 通过 Rejection Sampling（拒绝采样）生成和筛选模块示例；
3. 利用筛选后的示例更新模块配置；
4. 根据评价指标选择表现更好的程序。

三类典型优化方法

优化方法	DSPy 工具	主要机制
Automatic Few-shot Prompting	BootstrapFewShotWithRandomSearch	自动构建和搜索少样本示例
Instruction Induction	MIPROv2	自动生成和优化指令及示例
Multi-stage Fine-tuning	BootstrapFinetune	利用生成的数据执行微调

MIPROv2 展示了自动优化过程：optimizer = MIPROv2()optimized_program = optimizer.compile(program)

实际使用时还需要根据 DSPy 版本提供模型、评价指标和数据集等参数。自动优化能够显著改善特定任务的程序表现，但提升幅度取决于数据、任务、模型和评价指标。

6.8 DSPy 的实际应用与研究成果

采用相关自然语言编程与优化思想的研究系统，包括：PATH；IReRa；STORM；EDEN；Efficient Agents；ECG-Chat。这些案例说明，DSPy 所代表的程序化优化思想已经应用于不同的语言模型任务和研究系统。

6.9 Natural Language Programming：程序优于单体模型的条件

两项核心观点。

第一，在许多任务中，经过合理设计的程序可以比单一模型更准确、更可控、更透明，甚至更高效。

第二，开发者主要需要定义声明式程序，而不必手动规定每个底层实现细节。

高层优化器可以根据任务数据自动改进 Prompt、模型权重或未来可能出现的其他模型参数形式。

这体现了从手工提示词设计到声明式程序优化的转变。

6.10 Natural Language Optimization：自动优化的三项经验

1. 高质量示例的重要性

对于许多任务，自动构建高质量示例（Bootstrapping Good Demonstrations）是有效的优化方法。其基本思想是通过示例展示期望行为，而不只是通过文字描述规则。

2. 指令优化与条件规则

在高质量示例基础上进一步生成和优化指令，可以改善程序表现。当任务包含较多条件判断或复杂规则时，明确的指令尤其重要。

3. Grounding 与 Credit Assignment

优化过程还需要有效的 Grounding（依据外部信息或评价证据建立可靠约束）和明确的 Credit Assignment（贡献归因）。对于多模块系统，如果最终结果错误，需要判断是哪个模块导致了性能下降，才能进行有针对性的优化。

例如，一个 RAG 系统回答错误可能来自检索失败、上下文选择错误或答案生成错误。仅根据最终答案分数优化所有模块，可能无法准确识别问题来源。

因此，DSPy 的长期价值不仅在于自动生成 Prompt，还在于支持基于数据、指标和模块结构的系统性优化。

总结

技术层次	核心问题	关键技术
推理计算	如何避免重复 Prefill	KV Cache
存储基础设施	如何长期保存与共享缓存	Hierarchical Storage、LMCache
缓存算法	如何复用非前缀上下文	CacheBlend
系统架构	如何提高模型系统的可靠性与可控性	Compound AI Systems
程序抽象	如何减少对手写 Prompt 的依赖	DSPy Signature、Module
自动优化	如何基于数据持续改进系统	DSPy Optimizer、Compiler

从整体上看，本讲体现了两个重要转变：

第一，从重复计算到计算结果复用。 KV Cache、LMCache 和 CacheBlend 将模型内部的计算结果转化为可保存、可共享、可管理和可选择性重计算的资源。

第二，从模型调用到程序化智能体系统。 Compound AI Systems 和 DSPy 将模型能力封装为具有明确接口的模块，并通过数据驱动的优化方法改进系统性能。

两条技术路线共同说明：高效、可靠的智能体不仅依赖模型能力，也依赖模型外部的存储系统、执行架构、模块接口与优化机制。

5205 Introduction to AI

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_6/knowledge/5205-Introduction-to-AI.md`

5205 Introduction of AI

Lecture 1

Part 1

1.1 课程要求

理解人工智能的基本原理与算法 (BFS、神经网络、大语言模型等)。 Understand fundamental principles and algorithms of AI (BFS, neural networks, LLMs, ⋯).

分析与评估不同 AI 方法的优势与局限。 Analyze and evaluate the strengths and weaknesses of various AI methodologies.

将 AI 算法应用于工程、数据分析与优化场景。 Apply AI algorithms to engineering, data-analytic and optimisation scenarios.

设计与实现 AI 技术以解决实际问题 (对应期末项目)。 Design and implement AI techniques to solve practical problems (final project).

1.2 AI 概述：Industry 4.0 的核心引擎

阶段 Stage	时代 Era	核心驱动力 Core Driver	目标 Goal
Industry 1.0	18 世纪末	机械化 Mechanization	机器取代人力
Industry 2.0	19 世纪末	电气化 Electrification	大规模生产
Industry 3.0	20 世纪后半	信息化 Informatization	计算机与互联网
Industry 4.0	当下	智能化 Intellectualization	让日常生活更智能 (by AI)

Merriam-Webster 将 artificial 定义为"人为构造,常以自然事物为模型"。这暗示了 AI 的两条核心属性：由人创造、通过行为而非生物大脑来识别。 Merriam-Webster defines "artificial" as "humanly contrived, often on a natural model". This implies two core attributes: (1) human-made, (2) identified by behaviour,

not biological substrate.

1.3 AI 的三种典型定义 Three Classic Definitions of AI

提出者 / 时期 Author / Era	定义 Definition
McCarthy & Minsky, 1950s	由程序或机器完成的任何任务,如果由人来完成需要运用智能。 Any task performed by a program or machine that, if a human carried out the same activity, would require the human to apply intelligence.
John McCarthy (现代定义 Modern definition)	AI 又称机器智能 (machine intelligence),是与人类和动物自然智能相对的、由机器展示的智能。 Artificial intelligence — sometimes called machine intelligence — is intelligence demonstrated by machines, in contrast to the natural intelligence displayed by humans and other animals.
Two emphasised points	(1) 由人创造 human-made;(2) 通过行为而非生物大脑来识别 identified by behaviour rather than biological brain.

1.4 AI 发展史与代表性里程碑

AI 的发展可划分为两大时代： "深度学习前时代"与"大模型时代"。

Year	Milestone	Notes
1958	Perceptron 感知机	最早的神经网络模型之一
1974	BP 神经网络	反向传播算法使多层网络可训练
1998	CNN 卷积神经网络	LeNet 等开启深度卷积时代
2012	Google Brain / AlexNet	深度学习在 ImageNet 大放异彩；大数据 + HPC + 大存储 + 先进算法
2018	BERT & GPT	进入预训练时代
2022	ChatGPT (GPT-3.5)	大模型时代到来 LLM era
2023	GPT-4	大模型全面突破
2024	Agent 时代	多模态智能体与自主工作流
2025 (Aug)	GPT-5	OpenAI 在 2025 年 8 月发布,作为 NLP 的里程碑

Three supporting pillars 三大支柱：大数据 Big Data + HPC 高性能计算 + 先进算法 Adv. Algorithms。 AI 的三个演化阶段：

阶段	能力	典型例子
ANI 弱人工智能 Artificial Narrow Intelligence	专注于特定任务 Focus on specific tasks	AlphaGo、语音识别、图像识别、机器翻译、AI 导师、自动驾驶等
AGI 通用人工智能 Artificial General Intelligence	具备人类水平的思考、计划、理解与复杂问题求解能力；能从经验中学习、自适应。	尚未实现 Not yet realised
ASI 超级人工智能 Artificial Super Intelligence	比最聪明的人类更聪明的 AI	理论上可发展出医学、纳米技术、自我复制机器人等所有可能技术

1.5 AI 的双重属性：工程与科学

AlphaFold 是 Google DeepMind 提出的蛋白质 3D 结构预测模型,解决了 50 年悬而未决的"蛋白质折叠问题",给定氨基酸序列预测其三维构象。 AlphaFold 作为“AI 同时是科学也是工程”。

视角 Perspective	关注点 Focus	本课侧重
工程 Engineering	构建智能系统解决真实世界问题;研究支持性的机制、算法与表征。 Build intelligent systems to solve real-world problems; study mechanisms, algorithms, representations that make them possible.	主线 Mainstream
科学 Science	理解智能的本质 (人或机器);实现可评估的智能模型;探索算法/表征调整带来的影响。 Understand the nature of intelligence (human or device); implement evaluable models; explore consequences of algorithm/representation choices.	适度覆盖 Covered, esp. learning & agents

1.6 AI 基础:图灵测试与深度学习

1.6.1 图灵测试

1950 年 Alan Turing 提出图灵测试:若一台机器在对话中能让询问者无法区分其与人类,则可认为该机器具有智能。 In 1950 Alan Turing proposed the Turing test: if a machine's conversational responses are indistinguishable from a human's, the machine can be considered intelligent.

“图灵测试关注行为表现而非内部机制。” "The Turing test cares about behaviour, not internals."

1.6.2 深度学习的核心逻辑

A deep neural network is a hierarchy of layers that progressively transforms input data into more abstract representations (e.g., edge → nose → face). The output layer combines those features to make predictions.

深度神经网络由多层组成,每一层把输入数据转换为更抽象的表示 (例如：边缘 → 鼻子 → 整张脸)。输出层把这些特征组合起来完成预测。

1.6.3 规则驱动到数据驱动的范式转变

驱动力 Driver	含义
更好的模型 Better models	Rule-based → Statistical → Deep Learning；以更多参数拟合更复杂的数据。
更强的算力 Better compute	更强大的 RAM / CPU / GPU;分布式计算平台。
更多的数据 More data	海量、可分析、含信息的数据可用 → “大数据”。

"Big data is a collection of data sets so large and complex that it becomes difficult to process using on-hand database management tools or traditional data processing applications."

“大数据是指数据量大到传统数据库与处理工具难以处理的集合。”

1.7 大数据

1.7.1 数据来源

Data fueling model growth comes from everyday human activity — "Nearly every human action leaves a digital trace: medical, traffic, logistics, bioscience, weather, chemistry, education, system, finance, e-commerce, manufacturing, ⋯

1.7.2 4V 特性

维度 Dimension	含义	典型数据
Volume 体量	从 TB 到 ZB 的数据规模需要处理。From terabytes to exabytes to zetabytes.	2015: 8 billion TB;2020: 40 ZB;人均 ~5.2 TB
Velocity 速度	批式、实时、流式数据,从毫秒到秒级响应。Batch, real-time, streaming.	毫秒级响应的实时推荐系统
Variety 多样性	结构化 / 半结构化 / 非结构化:文本、图片、视频、音频等。	Structured, semi-structured, unstructured.
Veracity 真实性	数据的不一致性、不完整性、歧义性、欺骗、近似模型带来的不确定性。	Uncertainty from inconsistency, incompleteness, ambiguity, deception.

1.8 数据挖掘与机器学习

1.8.1 数据挖掘

Data mining discovers patterns and models that are valid, useful, unexpected and understandable. Examples: "milk + bread" often bought together (descriptive); tomorrow's weather (predictive).

数据挖掘旨在发现数据中“有效、有用、出乎意料、可解释”的模式与模型。

典型例子：从超市数据中发现“牛奶 + 面包”常被一起购买 (描述型)；从历史数据预测明日天气 (预测型)。

任务类别	目标	例子
描述型 Descriptive Methods	发现人类可解读的、描述数据的模式	购物篮分析 Milk and Bread
预测型 Predictive Methods	用部分变量预测其他变量的未知值或未来值	天气预测 weather prediction

1.8.2 经典机器学习 vs 深度学习

阶段	特征 Feature	模型 Model
经典机器学习 Classical ML	手动设计特征 (Handcrafted Feature Extraction → Feature Selection)	统计模型 Statistical models
深度学习 Deep Learning	多层感知机自动学习特征 (Automatic Feature Learning)	神经网络 / 多层感知机 Multi-layer Perceptron

Machine learning lets machines learn new things from data. Deep learning, with more complex models, fits large-scale data better.

机器学习 (Machine Learning) 让机器能从数据中学习新东西；深度学习 (Deep Learning) 通过更复杂的模型拟合大规模数据,更适合大数据场景。

1.9 机器学习三大范式

范式 Paradigm	数据特征	任务	典型例子
监督学习 Supervised Learning	有标签 Labeled Data	预测标签	价格预测
无监督学习 Unsupervised Learning	无标签 Unlabeled Data	聚类 / 主题发现	客户分群、主题挖掘
强化学习 Reinforcement Learning	基于奖励 Rewards	寻找好的行为	训练小狗 / 游戏 AI

1.9.1 监督学习：回归与分类

Supervised learning trains on labeled data to predict labels on new samples. Predicting a real-valued target = regression; predicting a discrete category = classification.

监督学习的核心：用有标签数据训练模型，使其能对新样本做出预测。预测目标若是连续值，称为回归 (Regression)；若是离散类别，称为分类 (Classification)。

回归例子 Example：用面积、楼层、位置预测房价 (连续值)。

Predicting house prices from area, floor, location (continuous).

分类例子 Example：用肿瘤大小、患者年龄判断良性/恶性 (离散)。

Predicting benign/malignant from tumour size and age (discrete).

As the number of features grows from 1 to few to many, models move from simple linear fits to high-dimensional complex models — deep learning is the extreme of this trend.

当特征从 1 个到少量再到大量时，模型从简单线性拟合逐步走向高维复杂模型，深度学习正是这种趋势的极致。

1.9.2 无监督学习

In unsupervised learning, the dataset has no labels x(1), ⋯, x(n); The goal is to discover interesting structures or patterns — clustering, topic discovery, dimensionality reduction, etc.

无监督学习的数据集没有标签 x(1), ⋯, x(n)，目标是发现数据中“有趣的”结构或模式，如聚类、主题、降维等。

1.9.3 强化学习

In reinforcement learning, the agent interacts with the environment at discrete steps t = 1, 2, 3, ⋯, T. At each step the agent:

强化学习中，智能体 (Agent) 在离散时间步 t = 1, 2, 3, ⋯, T 与环境 (Environment) 交互。在每一步，智能体: 观察状态 (环境) Observes the state (environment)；产生动作 Produces an action;；获得奖励 Gets the resulting reward；进入下一状态 And produces the next state。

Through a "try → collect feedback → improve policy" loop, the agent progressively learns better behaviour. 通过“尝试策略并收集反馈 → 根据反馈改进策略”的循环，智能体逐步学会更好的行为。

1.10 伦理、法律与隐私

The rise of AI raises a number of unresolved ethical and legal questions:

数据隐私保护 How do we protect data and the use of data in AI? How do we protect data and the use of data in AI?

人类对 AI 的控制 Should the law regulate the level of human control over AI?AI 是否应在某阶段被视为法律主体? Should the law regulate the level of human control over AI? At what stage, if any, should AI be considered a legal person?

AI 决策的责任边界 Should machines be allowed to make decisions that affect whether people are injured (e.g., self-driving cars making life-and-death choices)? Should machines be allowed to make decisions that may injure people (e.g., self-driving cars)?

自主武器 Should we allow AI in autonomous weapons that select and engage targets without human intervention? Should AI be allowed in autonomous weapons that select and engage targets without human intervention?

Lecture 2

Overview

周 Week	主题 Topic	核心内容 Key Content
Week 1	An Introduction	AI 定义 / 三大学习范式 / ANI - AGI - ASI
Week 2	Searching Techniques	无信息搜索（BFS / DFS）→ 启发式搜索（爬山法 / 最佳优先 / A*）→ 元启发式（SA / GA） 先看什么信息都没有时怎么找解（BFS / DFS），再给搜索装上启发式（爬山法 / 最佳优先 / A*），最后把找一个解升级成找最优解，于是引出两大元启发式：模拟退火（Simulated Annealing, SA）与遗传算法（Genetic Algorithm, GA）
Week 3	Evolutionary Computation	EA 总览 / PSO / DE / MA / MOEA / ETO / NAS
Week 4	Artificial Neural Network	神经元模型 / 感知机 / MLP / 反向传播
Week 5	Deep Learning & Applications	深度网络、卷积、循环与注意力

Part 1 Searching

1.1 Definition and Five Elements

Searching is the process of looking for a solution to a problem through a set of states.

搜索是在一组状态中寻找问题解的过程。

要素	原文	理解
当前状态	Current state: the step of where we are	我们现在走到哪一步
目标状态	Goal state: the step where the solution is found	解所在的那一步
动作	Actions: move one state to another	把一个状态搬到另一个状态的操作
代价	Cost: how many steps needed to find the solution.	找到解需要走多少步

要素	原文	理解
解	Solution: a path from the current state to the goal state.	从当前状态到目标状态的一条路径

解是一条路径（path），不是单个状态。这条约定决定了后面所有的搜索算法都在做同一件事，在状态图上从起点长出一棵树，直到某个叶子正好是目标。所以找到目标状态和给出到达目标的路径是两件事，后者需要回溯父节点。

1.2 搜索的重要性

An agent can find a sequence of actions that achieves its goals when no single action is done.

当没有任何单个动作能直接达标时，智能体需要找出一串动作。

Almost every AI problem depends on a search procedure to find the solution.

几乎每一个 AI 问题都依赖一个搜索过程来求解。

1. Check the current state (reach the goal?); 检查当前状态是否已达成目标；
2. Execute allowable actions to move to the next state; 执行允许的动作，移动到下一个状态；
3. Check if the new state is the goal state. 检查新状态是不是目标状态：No 时回到 Search，Yes 时 End。

循环有一个终止条件：课件在流程图旁写得很清楚 —— The process is repeated until a solution is found or the state space is exhausted.（这个过程一直重复，直到找到解，或者状态空间被穷尽）。

算法	前提	特点	说明
Linear Search 线性搜索	数据无序也可用	逐个比较，最坏情况看完全部	无信息搜索（只能一格一格试，等价于 BFS 的机械展开）
Binary Search 二分搜索	数据必须有序	每次砍掉一半，快得多	有信息搜索（有序这个先验就是启发信息，它让每一步都能排除一半）

1.3 图与树 Graph and Tree Representation

节点 Nodes：the states in a searching process 搜索过程中的状态；

边 Edges：transitions between states (actions) 状态之间的转移（也就是动作）。

与或图（And-Or Graph）：多条路都能成功

节点类型	原文	含义
And 节点	a node with all edges emanating from it connected by a curved line	发出的所有边用一条弧线连起来，表示必须全部满足
Or 节点	a node has no line connecting its edges to indicate that any emanating path can be a solution	边之间没有连线，表示任选一条路径即可

旅行商问题（TSP）：访问完所有城市是 And（每个城市都得去），先访问哪一个是 Or（顺序可以任选），这正是组合爆炸的来源。

1.4 搜索方法 Searching Approaches

Uninformed (Blind) Search：Breadth-first Search / Depth-first Search

Informed (Heuristic) Search：Hill Climbing / Best-first Search / A* Algorithm Search / Simulated Annealing / Genetic Algorithm

大类	方法	用不用启发信息	是否保证最优
无信息（盲）搜索	BFS 广度优先	不用	在边权相等时保证最优
无信息（盲）搜索	DFS 深度优先	不用	不保证（p.20 明确说明）
有信息（启发式）	Hill Climbing 爬山法	用 h	不保证（会困在局部最优）
有信息（启发式）	Best-first / Greedy	只用 h	不保证（贪心，p.28 示例）
有信息（启发式）	A*	g 与 h 都用	h 可采纳 / 一致时保证最优
元启发式（随机）	Simulated Annealing	用概率接受更差解	不保证
元启发式（随机）	Genetic Algorithm	用种群与适应度	不保证

A simplified road map of part of Romania（罗马尼亚部分路网简化图）,这张地图就是本讲后面所有实例的舞台：贪心搜索、 A* 全过程，用的都是它（起点 Arad、终点 Bucharest）。

Part 2 Generate & Test

2.1 生成—测试（Generate & Test）与井字棋

最原始的搜索策略 Generate & Test:

Step 1. Generate a possible state 先生成一个可能的状态;

Step 2. Test to see if it is a goal state (found a solution) 再测试它是不是目标状态。

用井字棋（Tic-Tac-Toe）来演示：从初始状态出发，按放 X（cross）或 O（zero）的规则不断生成新状态。

核心提问：How many states should be explored?

两个数：9! = 362,880 数的是对局数（下棋的先后顺序不同，是不同的对局）；3^9 = 19,683 数的是棋盘局面数（只看到棋盘长什么样，不管怎么走到这里的）

2.2 Open 与 Closed

Move downwards, level by level, until goal is reached.（一层一层向下推进，直到抵达目标）。

Open: a list of states that have been generated but whose children have not been examined.

已经生成、但孩子还没被考察过的状态。决定下个展开谁，BFS 用队列（FIFO）实现它，DFS 用栈（LIFO）实现它。

Closed: a list of states already visited.

已经访问过的状态。防止重复展开与死循环，遇到图里的回路时，靠它把回头路挡住。

本讲开头说盲搜索不用任何额外信息，但记忆不算信息，它只是记录自己走过哪里。所以BFS非常吃内存，BFS 的 Open 表要同时存下整整一层的节点；而 DFS 的 Open/递归栈只存当前这一条路径。

Part 3 广度优先搜索

3.1 无信息搜索 Uninformed (Blind) Search

3.1.1 Definition

The only information available is the ability to distinguish goal state from non-goal state. An uninformed (blind) search algorithm uses no information other than the initial state, the search operator and a test for a solution.

唯一能用的信息就是分辨一个状态是不是目标；盲搜索除了初始状态、搜索算子、解的判定以外，不使用其他信息。

The different types of blind searches are characterized by the order in which they expand the nodes. The order can have a dramatic effect on how well the search performs (randomly).

各种盲搜索的唯一区别就是展开节点的顺序，而这个顺序（会对搜索的表现产生巨大的影响）。

换句话说：BFS 与 DFS 的代码几乎相同，差别只在用队列还是用栈。

3.2 广度优先搜索的定义

Expand the tree layer by layer, progressing in depth. Expand the root node first, expand all nodes at level 1 before expanding level 2 OR expand all nodes at level j before expanding nodes at level j + 1.

一层一层地展开，先展开根节点；先展开完第 1 层的全部节点，再去展开第 2 层，或者说，先展开完第 j 层的全部节点，再展开第 j+1 层的节点。

BFS 在边权相等时保证最短：因为它是按层推进的，第 j 层的所有节点都比第 j+1 层的离起点更近。所以第一次遇到目标时，走的一定是步数最少的路径。

3.2.1 广度优先算法

课件把 BFS 画成一张带 Expand node（展开节点）的流程框图。它的文字层只有 Expand node 与 Breadth-first Algorithm 两个标注，但流程本身与 2.5 / 2.7 的两个实例完全一致，可以还原成下面的伪代码：

> current = open.pop(0) ← 从队列头部取出（FIFO）
>
> if child not in open and child not in closed: open.append(child) ← 追加到队尾

📌 「第几步」怎么数：课件 p.13 与 p.19 都写了 State space at the sixth step（第六步时的状态空间），说明课件把「展开一个节点」算作一步，并把每一步的 Open / Closed 内容都列出来。本笔记照此约定。

3.2.2 Example 1：从 A 到 U

课件给的搜索树以 A 为初始状态、U 为目标状态，并逐次标出展开的节点：

3.2.3 Example 2：八数码

Why only one expansion here? 为什么这里只扩展出一个？。

同一个棋盘局面可以由不同动作到达，因此生成时必须先做去重，否则搜索树会膨胀成一棵指数级的树。其余方向的邻居要么不合法、要么已经在 Open 或Closed 里（重复局面）。

3.2.4 Example 3：从 S 走到 G

这里的广度优先只有两步，而后面的深度优先结果 S-A-C-G（3 步），所以证明了后续的 Depth-first search does not find the shortest path（深度优先一般不找最短路径）。

第 3 步 A 的孩子 C 被追加到队尾，队里 B 还在前面，所以下一步展开的是 B 而不是 C；

第 4 步 B 的孩子里 C 已经在 open 里，于是不重复追加，只追加 G；

第 5 步 C 的两个邻居 A 与 G 都已在表里，于是open 只剩 G。

如果把②③的判重去掉，C 会被反复加入，搜索可能永远不结束。

Part 4 深度优先搜索与两者对比

4.1 深度优先搜索的定义

Expand the tree as deep as possible, returning to upper levels when needed. Select a child from left to right. ・ Repeatedly go to the left child, as long as possible. ・ Return to left-over alternatives (higher up) only when needed.

尽量往深里展开，需要时才退回上层；从左到右选一个孩子，只要可能就一直往左孩子走；只有在需要时，才回到（更上层的）那些还没走的分支。

4.2 深度优先算法

与 BFS 一样，课件给的是一张带 Expand node（展开节点）的流程框图，文字层只留下 Depth-first Algorithm 这个标题。按 p.20 的实例还原如下：

> current = open.pop() ← 从栈顶取出（LIFO）
>
> for child in reversed(current.children): ← 倒序压栈，保证从左到右访问

4.3 Example 1：从 A 到 U

同一棵搜索树（初始 A、目标 U），但展开次序完全不同：

对比项	BFS	DFS
初始 / 目标	Initial state: A / Goal state: U	Initial state: A / Goal state: U
展开序列	Expand A → Expand B → Expand C → Expand D → Expand E	Expand A → Expand B → Expand E → Expand K → Expand S → Expand L → Expand T → Expand F → Expand M → Expand C ……
形状	横向铺开，一层层推进	纵向钻下去，一路到底才回头

4.4 Example 2：从 S 走到 G

结论：Solution path: S-A-C-G

In the route example, by breadth-first search, the path is S-B-G; by depth-first search, the path is S-A-C-G. 深度优先一般找不到最短路径。同一个例子里，广度优先给的是 S-B-G，深度优先给的是 S-A-C-G。

4.5 深度优先 vs. 广度优先

They both find a solution if one exists (and if the state isn't too big).

只要解存在（且状态空间不太大），两者都能找到解。

对比维度	深度优先 DFS	广度优先 BFS
主要风险	跑进没有解的极深分支里空耗时间	内存占用极大
课件原文	If the search space contains very deep branches without solution, then depth-first may waste much time in them.	Is very demanding on memory! Must remember the unexpanded nodes at a level, which could become very large as you go deeper into the tree/graph
要记住什么	depth-first remembers just the path 只记当前这一条路径	要记住整整一层的未展开节点，越深越大
最短路径	不保证	边权相等时保证
何时该用	解深、内存紧、只要求「有解」	解浅、要求最短、内存够

空间复杂度：设分支因子为 b、最大深度为 m。DFS 的空间是 O(b · m)（深度每加一层，每层只要记住还剩几个没走）；BFS 的空间是 O(b^d)（d 是解所在深度，因为要把整层都存下来）。在棋盘、地图这类分支因子很大的问题里，b^d 会瞬间爆炸，太过依赖记忆。

Part 5 启发式的有信息搜索： Hill Climbing, Best-first and A*

5.1 启发式搜索

5.1.1 使用场景

AI problem solver employs heuristics in two situations:

AI 求解器会在两种情形下动用启发式。

1. A problem may not have an exact solution because of ambiguities in problem statements or available data. Example: medical diagnosis. 问题可能根本没有精确解，因为题面或数据本身就有歧义，例如医学诊断；
2. A problem may have an exact solution, but the computational cost of finding it may be prohibitive. 问题有精确解，但找它的计算代价高到无法承受。

5.1.2 优势与不足

1. Uninformed (blind) searches are normally very inefficient. Information about the problem can sometimes be used to guide the search more efficiently. 盲搜索通常效率很低；问题自身的信息有时可以用来让搜索更高效。
2. Informed search explores the node that is most "likely" to be the nearest to a goal state. 有信息搜索优先展开那个「最可能」离目标最近的节点。

3. Informed search works successfully in many cases, but its success is not guaranteed. 有信息搜索在很多情况下有效，但成功没有保证。比如，爬山法会 stuck at local optima（卡在局部最优），贪心搜索遇到U 形障碍最优都保不住。

5.2 爬山法（Hill Climbing）

Consider an eager but blind mountain climber: go uphill along the steepest possible path until you can go no further up.

想象一个急于登顶却看不见全山的登山者，沿着最陡的坡一直往上爬，直到再也爬不上去为止。

把这个比喻写成算法，就是四条性质：

1. Hill-climbing expands the current state and evaluates its children. 展开当前状态，评估它的孩子；
2. The best child is selected for further expansion; Neither its siblings nor its parents are retained. 选最好的那个孩子继续展开；它的兄弟节点和父节点都不保留；
3. Because it keeps no history, the algorithm cannot recover from failures of its strategy. 因为它不保留历史，所以一旦策略失败就没法恢复。爬山法与 BFS / DFS 的根本分界。BFS 和 DFS 靠 Open / Closed 记住所有走过的路，所以不会迷路（要么找到解，要么穷尽）；爬山法主动扔掉兄弟与父节点，只带一个 Current 走。也是省内存和极快的代价；
4. Hill climbing performs well if the distance estimate (quality measurement) is locally consistent. That is, the node with the shortest distance estimate will eventually lead to the goal. 只有当距离估计（质量度量）局部一致时爬山法才表现良好，也就是距离估计最小的那个节点最终能通向目标。

Hill climbing may encounter the problem of stuck/trap at local optima. 爬山法可能遭遇卡在局部最优的问题。

5.2.1 爬山法三个困境

Local Maxima（局部极大）：四周都比自己低，但并非整座山的最高点，任何单步移动都变差，算法停住。

Plateaus（高原）：周围一片平，所有邻居的评估值都相同，算法看不出该往哪边走，只能随机游荡。

5.3 最佳优先搜索 Best-first search (or Greedy Search)

Select a node for expansion based on an evaluation function f(n), which is construed as a cost estimate. The node with the lowest evaluation is expanded first. i.e., argmin(f(n))

按评估函数 f(n)（代价估计）来选下一个展开的节点；评估值最小的那个先展开，即argmin(f(n))

启发成分 h(n)：

h(n) = estimated cost of the cheapest path from state at node n to a goal state

h(n) = 从节点 n 所在状态走到某个目标状态的**最便宜路径**的代价估计。

If n is a goal node, then h(n) = 0.

若 n 本身就是目标节点，则 h(n) = 0。

Tips: h(n) = 0 这条约定非常重要：它是后面 A* 最优性讨论的基石。到了目标，剩余代价当然是 0，这条看起来理所当然的性质，正是可采纳性（admissible）与一致性（consistent）定义中反复使用的那一步。

5.3.1 Example：启发函数为直线距离

Try to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly. Thus, it evaluates nodes by using just the heuristic function: f(n) = h(n). We use the Straight-Line Distance heuristic, which we call h_SLD.

优先展开离目标最近的那个节点，理由是它看起来能更快到达解；于是它评估节点时只用启发函数：f(n) = h(n)；使用直线距离启发式，记作 h_SLD。

启发式的具体形态：把离目标有多近量化成一个可以直接查表的数。整张表的来源是地图上的直线距离，也就是飞机能直飞的距离，与后面 A* 里的 g(n)（沿公路实际走的里程）形成对照。

注意 Bucharest = 0：这就是 4.4 中 if n is a goal node, then h(n) = 0 的实例。

5.3.2 贪心搜索与局限

从 Arad 出发、直奔 Bucharest 的过程，每个节点旁边标的就是 h_SLD：

把四步连起来看，贪心的短视一目了然：每一步都只做一件事，在眼前这些候选里挑 h_SLD 最小的那个。它从不回头（不像 A* 还留着 Oradea、Rimnicu Vilcea 在候选里），也从不比较已经走了多远（g 完全没出现）。

U 形障碍

However, it is not optimal. This shows why the algorithm is called "greedy" - at each step it tries to get as close to the goal as it can. If it finds its way around the "U"-shaped obstacle, it will follow the red path. It is also incomplete even in a finite state space, much like depth-first search.

它不是最优的；这也解释了为什么叫贪心，每一步都只想尽量靠近目标。如果它能绕过那个U 形障碍，就会走上红色路径。即使状态空间有限，它也可能不完备，很像深度优先搜索。

蓝线是贪心，红线是最优：贪心会径直朝目标的直线方向冲，一头撞进 U 形的内壁，然后卡在凹底（因为它只会朝看起来更近的方向走，而任何往回走的移动都离目标更远，于是它不敢走）。这与爬山法卡在 Local Maxima 是同一个病根：只看一步、不记历史、不接受暂时变差。模拟退火，正是为了治这个病而设计的。

5.4 A* Search

5.4.1 Definition and Core Idea

A* is an algorithm that use heuristic to guide the search while ensuring that it will compute a path with the minimum cost.

A* 是一种一边用启发式引导搜索、一边保证算出的路径代价最小的算法。必须满足的目标：快（靠启发式）与优（保证最小代价），整节剩下的内容都在解释它怎么把这对矛盾统一起来。

核心公式：A* computes the function f(n) = g(n) + h(n)

h(n) 是 "estimated cost"（估计代价）

The h(n) value guides search toward heuristically promising states. The function h(n) relies upon some heuristic information available from the problem domain. It changes from heuristic to heuristic, and from problem to problem. h(n) 把搜索带向启发式上看更有希望的状态；h(n) 依赖问题领域能提供的启发信息，换一个启发式、换一个问题；而且 A* 的全部理论（可采纳性与一致性）都建立在h 不要估高了这一条假设上。

g(n) 是 "actual cost"（实际代价）

while the g(n) value prevents search from persisting indefinitely on a fruitless search. h(n) in general weights how close node n is to the goal. 而 g(n) 防止搜索在一条没有结果的路上一去不回，h(n) 总体上是在度量节点 n 离目标有多近。

分量	是什么	方向	确定性
g(n)	起点 → n 的累计代价	向后看（回头路已定）	确定（实打实发生过的里程）
h(n)	n → 目标的最便宜路径估计	向前看（还有多少路）	不确定（靠启发式猜）
f(n)	两者之和	既是已付出也是还要付	排序用（每一轮选 f 最小的展开）

解决了贪心搜索的U型障碍：贪心只带 h，所以冲进凹底就出不来；A* 额外背着 g，于是已经绕了多远会不断抬升 f 值，让搜索有理由转向别的分支。

5.4.2 Example 罗马尼亚地图上的 A* 全过程

最终答案：最优路径是 Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest，总代价 418

> Arad → Sibiu = 140 → g(Sibiu) = 140，f = 140 + 253 = 393；
>
> Sibiu → Rimnicu Vilcea = 80 → g = 220，f = 220 + 193 = 413；
>
> Rimnicu Vilcea → Pitesti = 97 → g = 317，f = 317 + 100 = 417；
>
> Pitesti → Bucharest = 101 → g = 418，f = 418 + 0 = 418。
>
> 四段相加：140 + 80 + 97 + 101 = 418 ✓ 与 (f) 完全一致。

为什么 (e) 先找到了 Bucharest = 450，却不停?

因为在 (d) 之后，候选队列里有 Pitesti = 417、Bucharest(经 Fagaras) = 450、Craiova = 526…… 417 < 450，所以下一步必须先展开 Pitesti。展开 Pitesti 之后，它就给出了一个更便宜的 Bucharest = 418。如果换成贪心搜索，早在 (e) 见到 h(Bucharest) = 0 就会停下，拿到 450 这个次优解，这就是 A* 与贪心那 32 英里差距的来源。

5.4.3 Properties of A* 前提

条件一：可采纳性（admissible）：

A* generates an optimal solution if h(n) is admissible (never overestimates the cost to reach the destination node, i.e., the cost it estimates to reach the goal is not higher than the lowest possible cost from the current point in the path).

若 h(n) 是可采纳的，A* 就能给出最优解。可采纳的意思是它从不高估到目标的代价，也就是估计值不高于从当前点出发理论上可能的最低代价。

条件二：一致性（consistent）：

A* generates an optimal solution if h(n) is consistent, i.e., for every node n and for every successor node n' of n: $h(n) \leqslant c(n, n') + h(n')$

若 h(n) 是一致的，A* 也能给出最优解，一致的意思是对每个节点 n 和它的每个后继 n'，都有 $h(n) \leqslant c(n, n') + h(n')$，其中 c(n, n') 是从 n 到 n' 的边代价。一致 ⇒ 可采纳；可采纳 ⇏ 一致，一致是更强的条件。

- If h(n) is consistent, f along any path is non-decreasing. 若 h(n) 一致，则沿任意路径的 f 都是不减的；
- If h(n) is consistent, then h(n) is admissible. 若 h(n) 一致，则 h(n) 一定可采纳。

$h(n) \leqslant c(n, n') + h(n')$，即 $h(n) - h(n') \leqslant c(n, n')$ 。从 n 到目标的估计距离减去从 n' 到目标的估计距离，不能超过 n 到 n' 的实际距离。走一条边，估计值最多只能减少这条边的代价。它保证了沿路径的 f 不会下降。而 f 只可能上升或持平，就意味着一旦某个节点被 A* 展开（放进Closed），就再也不会被更便宜的路由绕回来。

Part 6 从搜索到优化 Optimization

6.1 优化问题

What are the values of x and y such that z is equal to the value at the peak? maximization? / minimization?

x 与 y 取什么值时 z 等于峰值？是最大化？还是最小化？

前面几节的目标是从起点走到目标，答案是一条路径；从这里开始，目标是找一组参数使目标函数取极值，答案是一个点。两者的共同骨架仍然是猜一组值 → 看结果好不好 → 改一改 → 重复。区别在于搜索的好不好是到没到目标，优化的好不好是一个数值。

6.2 Simple Example

y = f(x) = x²，where x ∈ [0, 10]，Find the value of x that gives the minimum of y

y = f(x) = x²，其中 x 限制在区间 [0, 10] 内；求使 y 取最小值的那个 x。

优化问题的三要素：

① 决策变量 x；② 目标函数 f(x) = x²；③ 约束 x ∈ [0, 10]。

答案：f 在 [0, 10] 上单调递增，所以最小值在左端点 x = 0 处取得，y = 0。

6.3 搜索空间扩大后的解

Not practical to evaluate ALL combinations! 把所有组合都评估一遍是不现实的！

A String（一个串）与 Cost function or Fitness function（代价函数或适应度函数）。

概念	为什么重要
A String（串 / 染色体） 对一个候选解的编码	优化算法不直接操作解，而是操作它们的编码
Cost function / Fitness function 给一个串打分的函数	整章唯一能判断好坏的依据，所有方法都靠它比较候选

前面 BFS / DFS 的隐含前提是状态空间可以被穷尽（所以 BFS 敢说找到的一定最短）；当搜索空间大到无法穷举时，完备性就必须让位给在有限代价内拿到足够好的解。接下来两个方法（SA 与 GA）都有一个共同特征：都放弃了保证最优，换来了能在大空间里跑得动。

6.4 传统做法：单点搜索与梯度下降 single point search

Conventional methods: single point search

Optimization problem: Finding a $P_o $where

$P_o = P_o \in S | f(P_o) = \min_{\forall P_i \in S} f(P_i)$

优化问题就是找一个 $P_o $：它属于解空间 S，且它的函数值 $f(P_o) $等于 S 中所有解的函数值里的最小值。这类方法叫 single point search（单点搜索），并标出小标题 Gradient-driven search（梯度驱动的搜索），每次只维护一个当前解，沿着梯度往低处挪。

核心公式：

1. 极值点的必要条件：在极值点处，函数对参数的偏导为零（曲面的切线是水平的）

$\partial f/\partial P = 0$

2. 迭代更新式：下一步的位置 = 当前位置 − 学习率 × 当前点的梯度。

$P_{n+1} = P_n - \alpha \cdot (\partial f/\partial P)|_{P=P_n}$

α is a small positive value - learning rate（α 是一个小的正数，称为学习率）。

负号的含义：梯度指向上升最快的方向，所以取负号就是在往下降最快的方向走。

图上有三支从右侧沿切线方向指向谷底的箭头，沿着切线一步步滑下去；谷底那个点上标着 ∂f / ∂P = 0 ，算法停下来的判据，就是梯度为零。

6.5 梯度下降的三条局限

1. Function must be continuous/differentiable 函数必须连续 / 可微；
2. Local optima 会陷在局部最优；
3. Performance depends on the starting point 表现依赖起始点。

局限	问题	后果	解决
必须可微	现实中大量问题是离散的、组合的（排班、路径、选型），根本没有梯度可用	直接无法使用，这也是 GA 的立足点（GA 只需能算适应度，不需要导数）	GA 只要求能算适应度
局部最优	梯度只告诉附近哪里更低，从不告诉你全局最低在哪	可能停在半山腰的坑里，而它自己无从察觉	SA 用概率接受更差解
依赖起点	起点不同，滑进的那个坑可能完全不同	同一个算法跑十次得到十个不同答案	GA 同时维护一整群解

Part 7 模拟退火 Simulated Annealing

7.1 从物理退火到算法

What is simulated annealing? The main idea is to avoid local minima by accepting moves to worse solutions probabilistically. It is one of the first stochastic local search methods, inspired by the analogy to physical annealing.

模拟退火（SA）的核心思想是通过以一定概率接受更差的解来避开局部极小，是最早的一批随机局部搜索方法之一，灵感来自物理退火的类比。

物理退火（Physical annealing）：

A process for obtaining the minimum energy state of a solid, whereby the melted solid is slowly cooled down until reaching its ground state. A specific schedule is used to cool down the solid very slowly.

退火是让固体达到最低能量状态的过程，把熔化了的固体缓慢冷却，直到它落到基态；并且会用一个特定的降温时间表，把固体降得非常慢。

计算机模拟（Computer simulation）：

The Metropolis algorithm is a simulation algorithm for the physical annealing process that can be applied to solve optimization problems!

Metropolis 算法是对物理退火过程的模拟算法，它可以被用来解优化问题

物理世界	优化世界	对应关系
固体的一个微观状态	一个候选解 s	状态与解
能量 E	代价函数值 f(s)	能量越低越好，代价越小越好
温度 T	控制参数 T	它决定敢不敢接受更差的解
基态（最低能量）	全局最优解	退火的目标
缓慢降温的时间表	降温计划（cooling schedule）	降得越慢，越可能落到真正的基态

Tips：降温速度是 SA 成败的核心，所以慢是关键：如果降温太快，固体会来不及排列整齐就被冻住，停在一个亚稳态，这在优化里就是卡在局部最优。SA 用时间换质量，是它相对梯度下降最大的代价。

SA 可以看作是对 Hill Climbing 的改进。普通 hill climbing 只接受让目标函数变好的移动，因此很容易停在 local optimum；而 SA 在搜索早期温度较高时，允许以一定概率接受更差的解，从而有机会跳出局部最优。随着迭代次数增加，temperature T 逐渐降低，算法接受差解的概率也会下降，搜索行为会越来越接近 hill climbing，最终在低温阶段稳定到较优解附近。

7.2 Metropolis 准则

k_B 是 Boltzmann constant（玻尔兹曼常数），T是 temperature（温度）。

情形	判据	动作
新状态更好或持平	$E_j \leqslant E_i$	无条件接受 s_j
新状态更差	$E_j > E_i$	以概率 $ p = exp(−(E_j − E_i)/(k_B · T)) $接受

差得越多，接受概率越小：ΔE 翻倍，概率是平方级下降；

温度越高，接受概率越大：T → ∞ 时 p → 1（高温时几乎什么都接受）；

温度越低，接受概率越小：T → 0 时 p → 0（低温时退化为纯爬山法，只接受更好的解）。

If the temperature is lowered slow enough, the solid may reach thermal equilibrium at each temperature. 如果降温足够慢，固体可以在每个温度下达到热平衡；在优化里意味着在固定的 T 下反复采样足够多次之后，解会按玻尔兹曼分布停在各个状态上；T 缓慢降到 0 时，这个分布会逐渐集中到全局最优那一点。所以 SA 在理论上有概率收敛到全局最优（前提是降温足够慢）；但实践里足够慢意味着时间长得不可接受，所以工程上都是用它换一个足够好的解，而不是真等它收敛。

7.3 SA 的算法框架

Without loss of generality, minimization problem is assumed.

不失一般性，假定是最小化问题，意味着更好= 代价更小，$ f(s_i) < f(s) $就是新解更优解。

SA 的四步流程：

1. Generate neighboring solutions s_i from S

从 s 的邻域里生成邻近解 s_i;

2. Accept new solution if it is better in cost value, i.e., when f(s_i) < f(s)

如果新解的代价更小，接受它；

3. Otherwise, make the decision whether or not to accept: Yes: Replace current solution with the new (worse) solution / No: Discard the new solution

否则要做一个决定：接受（用这个更差的解替换当前解）或丢弃，SA 唯一能翻过山脊逃出局部最优的手段；

4. Probabilistic decision making based on: Difference between new and old cost values / Control parameter T

这个概率决定由两个量决定：新旧代价之差，以及控制参数 T。

7.4 SA illustration

图中标注	含义
HILL CLIMBING × 3	爬山法的轨迹：一条单调下降、然后平掉的曲线（陷入局部最优后不再改善）
AT INIT_TEMP（初始温度）	SA 早期的行为：上下大幅抖动，差解被频繁接受
AT FINAL_TEMP（终止温度）	SA 后期的行为：抖动收敛成一条单调下降线，形态与爬山法一致
Move accepted with probability = $ e^{(-(ΔC/temp))}$	以该概率接受更差移动，即 Metropolis 准则
Unconditional Acceptance（无条件接受）	更好的移动永远直接接受，不掷骰子

在生成邻域解（neighborhood solution）时，可以随机生成一个邻近解，也可以系统性地生成邻近解，以确保整个邻域空间都被探索。SA 常用 Metropolis criterion 作为接受准则。

接受准则（Acceptance criterion）：若新解 s' 的 cost 不大于当前解 s，则直接接受：f(s') ⩽ f(s)；如果新解更差，即 f(s') > f(s)，则以如下概率接受该较差解：p = exp(−(f(s') − f(s)) / T)。

7.2 用的是能量版：p = exp(−(E_j − E_i)/(k_B · T))，分母里有玻尔兹曼常数 k_B；

7.4 用的是优化版：p = exp(−(f(s') − f(s))/T)，k_B 被吸收进了 T。

两者等价，因为 k_B 只是个常数，工程实现时常把它并到温度里统一调节。

其中 T 是当前温度。可以看到，当 T 较大时，即使 f(s') − f(s) 较大，接受概率仍可能不低，因此算法更偏向随机搜索（exploration，探索）；当 T 逐渐降低时，接受差解的概率趋近于 0，算法更偏向爬山法（exploitation，开发）。也就是说，SA 前期通过偶尔接受坏解扩大搜索范围，后期通过降温逐渐收敛。

Part 8 遗传算法 Genetic Algorithms

8.1 GA Definition

A genetic algorithm is a search-based algorithm based on natural selection, which can be used for solving AI optimization problems.

遗传算法是一种基于自然选择的搜索型算法，可以用来求解 AI 优化问题。

术语	课件原文	中文理解
Population 种群	This is a subset of all the probable solutions that can solve the given problem.	能解决给定问题的全部可能解中的一个子集

术语	课件原文	中文理解
Chromosomes 染色体	A chromosome/string is one of the solutions in the population.	种群中的一个解，也叫串（string）
Gene 基因	This is an element in a chromosome.	染色体里的一个元素（串上的一位）
Fitness function 适应度函数	This is a function that uses a specific input to produce an improved output. The solution is used as the input while the output is in the form of solution suitability.	以解为输入、以解的合适程度为输出的函数，它是 GA 判断好坏的唯一依据
Genetic operators 遗传算子	Genetic operators such as Selection, Crossover, Selection, are used for changing the genetic composition of chromosomes.	用来改变染色体基因组成的操作，课件举的例子是选择与交叉

8.2 GA 的组成

A genetic algorithm repeatedly modifies a population of individual solutions. At each step, the genetic algorithm selects individuals at random from the current population to be parents and uses them to produce the children for the next generation. Over successive generations, the population "evolves" toward an optimal solution.

GA 反复地改造一个由个体解构成的种群。每一步中，它从当前种群里随机挑个体当父母，用它们产生下一代的子女。经过一代又一代，种群会进化向一个最优解。

算子	作用
选择 Selection	从当前种群挑出当父母的个体
交叉 Crossover / 重组 Recombination	把两个父母的基因片段交换，产生子女
变异 Mutation	随机改动个别基因

重复与随机：repeatedly说明 GA 是迭代式的；selects individuals at random（随机挑选个体）说明选择并非确定性，适应度高的个体只是更可能被选中，不是一定被选中。这一层随机性就是 GA 能跳出局部最优的机制来源。

8.3 A Simple Genetic Algorithm

1. 编码 Initial/random designs coded：把现实问题的方案，编码成染色体字符串。
2. 初始种群：生成一批随机候选解，附带各自适应度。
3. 解码 / 评估 / 仿真 Decoding / Evaluation / Simulation

   解码：字符串 → 还原回真实问题方案

   仿真计算，得到适应度分数

> 适应度评估一般在 GA 算法外部完成；仿真计算开销大，是 GA 主要算力消耗来源。

4. 选择 Selection：适应度越高，被选为父代的概率越大。
5. 交叉 Crossover：父代染色体片段交换，产生子代。
6. 变异 Mutation：随机修改某一处基因。
7. 新一代 New generation： 返回第 3 步评估，循环迭代。

8. 出口 Final optimized designs coded：迭代终止，输出编码形式的最优解。

交叉：P_2: 4 0 0 3 0 | 1 6 1 与 P_3: 0 1 6 4 1 | 8 0 1交换分割点后的尾巴得到两条子代。

变异：P_2': 4 0 0 3 0 8 0 1 → P_2'': 4 0 1 3 0 8 0 1，第 3 位基因被修改。

8.4 GA三步逐一演示

GA 的工程实现：GA 原生目标是**最大化适应度 fitness**；工程大多任务是最小化代价 cost，需要转换。

1. fitness = 1/cost
2. fitness = C_{max} − cost
3. fitness = exp (−cost/T)

初始种群：

P_1 : [1, 2, 0, 9, 0, 2, 1, 7], $f(P_1)$ = 5%

P_2 : [4, 0, 0, 3, 0, 1, 6, 1], $f(P_2)$ = 60%

P_3 : [0, 1, 6, 4, 1, 8, 0, 1], $f(P_3)$ = 35%

1. 繁殖 Reproduction（选择）：按照适应度分配后代数量。P_1后代少，P_2生 2 个、P_3生 1 个。

> 仅选择：不改动染色体基因，不需要重新算适应度。

2. 交叉 Crossover：随机选分割点，交换父代片段。

> 交叉只重组已经存在的基因，不会创造全新基因；不需要算适应度。

3. 变异 Mutation：随机翻转 / 修改部分基因。唯一可以生成种群原本不存在的新基因。

> 变异后得到全新子代，进入下一轮循环必须重新评估适应度。

交叉只是重组已有的基因（不引入新基因），而变异引入了搜索空间里原本不存在的新基因。从评估预算角度看，这两步的成本是一样的（都产生新串）

8.5 Basic Operations of a Simple GA：简单 GA 的基本算法

步骤	作用	属于
Make initial population	生成初始种群	只做一次
Evaluate the fitness of all initial chromosomes	给初始种群整体打分	只做一次
Reproduce children from parents (selection)	按适应度选父母、产下一代	每代循环
Apply crossover to some chromosomes	对一部分染色体做交叉	每代循环
Apply mutation to some chromosomes	对一部分染色体做变异	每代循环
Preserve some elite individuals	保留若干精英个体	每代循环
Decode the best chromosome found	解码最优染色体	只在最后

Tips：交叉、变异**不是对全部个体生效**。

交叉：较高概率，0.6 - 0.9，成对操作；变异：每个基因很小概率，典型值 0.02； 这两个概率是 GA 最重要调参旋钮。

8.6 Details of Genetic Algorithms

Constant population size = n：**种群大小 n 全程固定不变。**资源受限的竞争：任何个体的加入，都意味着有另一个个体被挤出。而梯度下降只有一个点，无所谓竞争；GA 有一群点，并且每代都在做谁留下、谁出局的淘汰。

层级从上到下：

1. 种群 Population：一共 n 个个体，总数恒定 n。
2. 个体 / 染色体 Chromosome：共 n 条，每条附带自己适应度f_i。
3. 基因 Gene：每条染色体基因长度全部相等。

种群规模固定意味着：**每新增一个个体，就要淘汰另一个个体**，是资源竞争淘汰机制。 对比梯度下降，梯度下降只有单个搜索点，不存在竞争淘汰。

Constant population size = n 这条约束：种群规模在演化过程中保持不变。这带来一个必须被显式处理的后果，选择时必须填满 n 个名额（谁多生、谁不生、谁被淘汰，要凑够整数），这也正是用精英替换最差的 10% 子女这种做法能成立的前提：替换而不是增殖，所以总数不变。

8.7 Designing a Representation

We have to come up with a method of representing an individual as a genotype. When choosing a representation, we have to bear in mind how genotypes will be evaluated and what genetic operators might be. 我们必须想出一种把个体表示成基因型的方法。选择表示法时，必须同时考虑两件事：基因型将如何被评估，以及遗传算子会是什么。

核心思想：表示形式决定算子怎么写，不同编码，交叉、变异实现完全不一样。

两条设计原则：

1. 需要把实际问题解表达成基因型（染色体串）。
2. 设计编码的时候，同时要想好两件事：怎么评估这个基因型、遗传算子如何实现。

设计维度	选项	影响
基因的取值类型	binary, decimal, real, order-based?（二进制 / 十进制 / 实数 / 基于次序）	决定变异怎么写
问题的性质	numerical or combinatorial?（数值型 / 组合型）	决定交叉怎么写
基因的最小单位	Bit or Binary number（位 或 二进制数）	决定串长与精度

举例不同编码对应的变异实现

1. 二进制编码变异：比特位取反。
2. 基于次序编码（TSP 旅行商）变异：随机交换两个基因。不能简单改数字，否则出现重复城市。
3. 树形编码变异：随机替换树内部某个节点。

> 硬性约束：交叉、变异操作结束，**必须产出合法的染色体解**。

8.8 基因型到表现型：二进制整数编码

$1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 128 + 32 + 2 + 1 = 163$

8 位二进制串 10100011 按权展开求和得到 163，于是基因型 10100011 对应表现型整数 163。

课件把这两个概念并排写成一张对照表：图上标 8 bits（8 位），左边是 Genotype（基因型：编码字符串）、右边是 Phenotype（表现型：现实世界真实数值）。

What if we have 10 parameters? How many bits in total?

如果有 10 个参数呢？总共需要多少位？若每个参数都用 8 位精度，则总串长 10 × 8 = 80 位。

GA 的搜索空间随参数个数指数增长，1 个参数是 $2^8 = 256$ 种组合，10 个参数搜索空间大小就是 $2^{80} \approx 1.2 \times 10^{24}$ 种指数爆炸，**无法穷举全部组合**。

8.9 二进制映射实数

当表现型是实数时，例如用 8 位二进制表示区间 [2.5, 20.5] 上的一个数，则按线性映射把基因型的整数值163折算成实数 x。把整数基因型线性映射到实数区间 [2.5, 20.5]，区间跨度 20.5 − 2.5 = 18。

$x = 2.5 + \frac{163}{255}\times 18=13.9609$

分母 255：保证全 1 严格等于区间上界 20.5；

分母 256：简单工程实现，但最大值达不到区间上界。

8.10 Mutation Operators：变异算子

变异的核心设定：变异通常以很小的概率 p_m 发生，例如每个基因 0.02。

Mutation usually happens with small probability $p_m $(e.g., 0.02) for each gene.

变异的三个条件：

1. At least one mutation operator should allow every part of the search space to be reached. 至少要有一个变异算子，能让搜索空间的每一个部分都被抵达。没有变异，种群里的基因材料只会被重新组合，永远造不出初始种群没有的基因；搜索会退化成一个封闭的组合游戏。
2. The size of mutation is important and should be controllable. 变异的幅度应当可控。幅度太大等于随机重启（丢掉父辈的全部信息），太小则形同没变，所以必须可调。
3. Mutation should produce valid chromosomes. 变异必须产生合法的染色体。例如基于次序的表示里每个城市必须恰好出现一次，变异绝不能造出重复或缺失的城市。

Is mutation critical in GA? 变异在 GA 里是必需的吗？

是，交叉只能重组已有的基因（selection 与 crossover 都不产生新基因），因此变异是 GA 唯一的新基因来源，也是三条要求里第一条（every part of the search space to be reached）的直接依据。交叉负责开发（exploitation，用好现有材料），变异负责探索（exploration，去没去过的地方）。

表示法决定算子：同一个变异操作，在不同表示法下是完全不同的代码：

1. 二进制表示：把某一位取反

例：（1 1 1 1 1 1 1 → 1 1 1 0 1 1 1）；

2. 基于次序的表示（order-based representation）：随机交换两个基因。交换两个基因会让同一个位置上出现重复取值。所以条件三才会专门强调 Crossover should produce valid chromosomes（交叉必须产生

合法染色体）。

例：（7 8 3 4 1 2 6 5 → 7 8 3 4 6 2 1 5）；

3. 树形表示：变异 = 随机把一个节点替换成另一个节点。

8.11 交叉算子

交叉（Crossover）：拿两个父代个体，交换基因片段来生成新子代，核心是混合两份父代的特征。

交叉的两个条件：

1. The child should inherit something from each parent. If this is not the case, then the operator is a mutation operator. 子代必须同时继承来自两个父代的基因。如果子代完全只抄其中一方，这不叫交叉，本质等同于变异。
2. Crossover should produce valid chromosomes. 交叉产生出来的子代染色体，必须是**合法有效的解**。

单点交叉：交换连续一大截基因。

把两条染色体，随机选一个切割位置，互相交换切割点**后面的一整段基因**。

退化情况：如果切割点在**最左端**：子代直接完整复制另一个父代，没有混合双方信息。如果切割点在**最右端**：子代和原来父代一模一样，没有产生任何新组合。 这两种不算有效交叉，代码实现时要重新随机选切割点。

均匀交叉：每一位都可以独立选择来源，基因打散重组更充分。

不再切一整块；生成一条和染色体长度一样的**掩码序列**，掩码每一位决定这一位基因取自哪个父代。

掩码 =0 → 该位置基因拿父代1

掩码 =1 → 该位置基因拿父代2

第3、5、8位是 1，取父代2的C、E、H；剩下位置取父代1。子代2会反过来，掩码0取父代2、掩码1取父代1。

缺点：单点交叉和均匀交叉都很容易把已经搭配得很好的一长串优良基因组合给拆碎。

基于次序表示的重组（用于TSP旅行商这类排列问题）

场景：染色体代表访问城市的顺序，每个数字（城市）只能恰好出现一次。 普通单点/均匀交叉会造出重复城市、丢失城市，直接变成非法解。所以要用专门的次序重组逻辑。

执行步骤：

1. 在父代1上，截取一段连续基因片段，直接原样保留放到子代对应位置。
2. 剩下空出来的位置：从**父代2**拿基因，但跳过已经被截取片段里出现过的基因。
3. 填充顺序：从截取片段的切点往后填，填到染色体末尾就绕回最开头。
4. 两个父代互换角色，再生成第二个子代。

8.12 选择策略 Selection Strategy

核心设计思想有两点：

> We want to have some way to ensure that better individuals have a better chance of being parents than less good individuals. This will give us selection pressure which will drive the population forward. We have to be careful to give less good individuals at least some chance of

> being parents - since they may include some useful genetic material. The most commonly used selection methods include Roulette Wheel Selection, Rank Selection, Tournament Selection etc.

1. 质量好的个体，有更大机会被选为父代。由此产生选择压力，驱动种群不断往更优的方向演化。
2. 但不能完全淘汰掉差的个体。就算表现差，它身上也可能藏有有价值的基因；要给它保留一点点被选中的可能性，避免基因多样性彻底消失。最常用的选择方法有：轮盘赌选择、排名选择、锦标赛选择等。

选择的核心目的：从当前种群里挑选个体当父代，用来做交叉、变异产生下一代。

方法	怎么选	特点
轮盘赌 Roulette Wheel	按适应度占比分配被选概率（本页下一页的算例）	直观，但适应度差异大时会过早收敛
排名 Rank	按适应度的排名而非数值分配概率	抗超级个体适应度再大也只按名次算
锦标赛 Tournament	随机抽 k 个个体，取其中最优秀的	实现简单、参数少，最常用

轮盘赌选择的两大缺陷

$P(x_i)=\frac{f_i}{\sum f_i}$

个体被选中概率和自身适应度**数值大小直接成正比**，想象一个轮盘，适应度越高占的扇区面积越大。

1. **早熟收敛 premature convergence** 如果出现一个适应度遥遥领先的“超级个体”，它被选中的概率会极高。几代之内种群大部分都变成它的后代，种群多样性直接丢失，算法卡在局部最优出不来。
2. **选择压力不足** 如果种群里面所有个体适应度都差不多，那大家被抽中的概率几乎一样。选父代接近随机抽签，演化几乎变成随机游走，很难向着好解推进。

为什么需要排名选择？ 排名选择扔掉原始适应度的绝对数值，只用排序名次。 哪怕某一个个体适应度爆炸式高，它也只能拿到第一名对应的概率，不会一家独大。既保留优胜者优先被选的机制，又规避超级个体垄断种群的风险。除了换排名选择，精英保留（8.13）也是缓解这类问题的常用手段。

8.13 精英保留 Elitism

Re-introduce in the population previous best-so-far (elitism). For example, keep the best 10% of the parent population and re-introduce them to the next generation by replacing the worst 10% child population.

把算法运行到现在为止，历史表现最好的一批个体，重新放回下一代种群。例如，保留父代中最好的 10%，用它们替换掉子代中最差的 10%，从而进入下一代。

Example of a convergence trace（收敛轨迹示例）

无精英保留	开启精英保留
之前找到的优秀解，可能被交叉变异破坏、直接丢失；最优适应度曲线上下震荡波动	历史最优个体一定会被保存。最优适应度曲线单调不降，呈现阶梯上升，很容易观察算法有没有继续进步。

精英保留：只做保底。不会产生任何新基因、不会探索新区域，只管守住已经找到的好结果。

变异Mutation：负责探索，制造之前种群没有的新基因。 两者各司其职，不能互相替代。

8.14 Stopping Criterion：停止准则

三条停止条件：

1. The optimum is reached. 已经达到最优（基本不存在）
2. Limit on CPU resources: maximum number of fitness evaluations. 计算资源上限：适应度评估的最大次数。
3. Limit on the patience: after some generations without improvement. 耐心上限：连续若干代没有改善之后就停。

三条实验纪律：

1. Never draw any conclusion from a single run. 不能只跑一次 GA 就下结论。
2. Use statistical measures (e.g., average, standard deviation). 报告统计指标：平均值、标准差，还可以补充最优、最差、中位数，再配上收敛曲线（最优值随迭代代数变化）。
3. Obtain results from a number of independent runs. 必须跑多次独立重复实验。原因：GA 有大量随机性，初始种群是随机生成。单次运行结果有偶然性，一次跑的好不代表算法本身强。

8.15 GA 的优点与局限

8.15.1 优点

1. 不需要导数信息 梯度下降要求目标函数可微分才能更新参数。GA 不用求导，只要能算出适应度就行。

> 这是 GA 最大优势，特别适合组合优化问题，例如路径规划、排班，这类问题根本求不出导数。

2. 适配多种优化任务 不管是连续数值问题、离散问题，还是多目标优化都可以处理，适用面广。
3. 天然适合并行计算 一个种群里面有很多个体，每个个体算适应度互相之间互不干扰，可以同时跑。

> 如果适应度是仿真模拟，并行可以大幅节省运行时间，工程仿真场景很喜欢用 GA。

4. 结果随迭代持续变好 随着一代代演化，通常能够不断得到质量更高的候选解。

8.15.2 局限

1. 简单可微问题效率低 如果目标函数光滑可求导，梯度下降收敛速度远快于 GA。GA 不是万能算法，它是梯度方法的备选方案，而不是替代品。
2. 不能保证得到全局最优解 如果参数、算子设置不合理，很容易卡在局部最优。就算调得很好，也没有理论保证一定找到全局最优。

> 在搜索空间巨大、没办法全部穷举的场景，GA 目标是有限算力下拿到 “足够好用的解”，不等于一定拿到理论上最好的解。

3. 适应度计算开销巨大 每一代所有个体都要计算适应度。如果适应度需要跑仿真、复杂模拟，算力消耗会非常高。
4. 实现细节影响结果 编码方式、交叉变异概率、选择策略、精英保留这些配置一旦设置错，最终结果会很差。

优点 / 局限	呼应
不需要导数	直接绕过了梯度下降必须可微这一条，这是 GA 能被用在组合优化（排班、路径、选型）上的根本原因

优点 / 局限	呼应
能处理离散与多目标	适应度函数只需能算，不需要连续、不需要可微
并行能力极佳	因为一整代个体彼此独立，可以并行评估，这是 GA 在工程仿真场景里特别受欢迎的原因
不保证最终解质量	在无法穷举的空间里，没有任何方法能保证最优；GA 只是在有限预算内给出一个足够好的解
反复算适应度有计算挑战	一次适应度评估可能就是一次完整的工程仿真
简单问题上效率不高	简单问题（尤其可微的连续问题）应当直接用梯度方法；GA 是没有梯度可用时的备选，不是万能替代

BFS / DFS（无信息、完备但昂贵）→ 爬山法、最佳优先与 A* 算法（引入启发式）→ 模拟退火（用概率接受差解，逃出局部最优）→ 遗传算法（用种群与算子，在无法穷举的空间里做有方向的随机搜索）。

四类方法的共同骨架：猜一个解 → 用某个函数评价它 → 按某个规则改成新解 → 重复；区别只在怎么改与怎么评价。

Part 9 Tutorial 2

9.1 Exercise 1：广度优先搜索

Given the problem-solution graph in Figure 1, perform a breadth-first search starting from the initial state A and ending in the goal state F. If there are two or more nodes of the same preference, then handle the nodes alphabetically.

给定 Figure 1 的问题—解图，从初始状态 A 出发、到目标状态 F 为止，执行一次广度优先搜索。若出现两个或以上优先级相同的节点，则按字母顺序处理。

按同层按字母升序的规则，BFS 逐层展开如下：

步	展开	open（队列）	closed
1	A	[B, C, D]	[A]
2	B	[C, D]	[A, B]
3	C	[D, E]	[A, B, C]
4	D	[E, F]	[A, B, C, D]
5	E	[F]	[A, B, C, D, E]
6	F	—	—

全部展开后才找到：A → B → C → D → E → F

9.2 Exercise 2：深度优先搜索

Given the problem-solution graph in Figure 1, perform a depth-first search starting from the initial state A and ending in the goal state F. If there are two or more nodes of the same preference, then handle the nodes in descending (rather than ascending) alphabetical order.

同样的 Figure 1，从 A 出发、到 F 为止，执行一次深度优先搜索。但这次优先级相同的节点改为按字母降序（而不是升序）处理。Exercise 2 与 Exercise 1 的区别就是descending (rather than ascending)，升序改正降序

按同层按字母降序的规则：

步	展开	孩子按降序	选择	open（栈）
1	A	D > C > B	选 D	[B, C, D]（D 在栈顶）
2	D	F > E	选 F	[B, C, E, F]（F 在栈顶）
3	F	—	F 就是目标 → 结束	—

解路径为 A → D → F，只用了 2 条边，因为降序规则恰好把搜索直接指向了最深处的目标 F。

9.3 Exercise 3：四皇后问题的爬山法

The 4 queens problem consists of a 4 x 4 chessboard with 4 queens. The goal is to place the 4 queens on the chessboard such that no two queens can attack each other. Each queen attacks anything in the same row, in the same column, or in the same diagonal.

在 4×4 棋盘放 4 个皇后。

规则：皇后可以攻击同一行、同一列、同一斜线上的其他皇后。要求任意两个皇后之间不能互相攻击。

好处：直接彻底消除同列冲突，不用再检查列；只需要判断同行、对角线斜线冲突。

表达方式：用数组 $[x_0, x_1, x_2, x_3]$。x_i 代表第i列的皇后所在行号。

邻域操作：交换两列对应的行号。交换之后依旧保持每列 1 个皇后，不会产出非法解。

代价函数 cost

统计互相攻击的皇后对的数量。4 个皇后，两两组合一共 $C_4^2 = 6$ 对。 对第i列、第j列的皇后（$i < j$，行号x_i, x_j）： 发生攻击，满足下面任意一条：

1. 同行：$x_i = x_j$
2. 斜线：$|x_i - x_j| = |i - j|$

> cost = 冲突的皇后对数。 cost = 0 代表完全没有冲突，就是问题的解。

并列候选的抉择规则

当多个邻居的代价 cost 一样好时： 优先选择被交换的两列下标数字最小的那一对。 例：可以交换(x_0, x_1)，也可以交换(x_2, x_3)，选(x_0, x_1)。

爬山法逻辑

1. 当前状态，把所有邻域（全部两列交换）全部枚举，算出每个邻居 cost。
2. 挑选代价最小的邻居作为下一步状态；多个一样优，就用上面下标最小规则破平局。
3. 重复，直到：所有邻居都不能比当前更好；或者直接达到`cost=0`（不可能再优化），算法终止。

完整爬山执行步骤

初始状态：$[3, 1, 2, 0]$，初始 cost=2 含义：第 0 列皇后在 3 行；第 1 列在 1 行；第 2 列在 2 行；第 3 列在 0 行。

第一轮：初始状态生成全部邻居

序号	交换的列对	新状态 (x_0, x_1, x_2, x_3)	cost
1	(x_0, x_1)	(1, 3, 2, 0)	1
2	(x_0, x_2)	(2, 1, 3, 0)	1
3	(x_0, x_3)	(0, 1, 2, 3)	6
4	(x_1, x_2)	(3, 2, 1, 0)	6
5	(x_1, x_3)	(3, 0, 2, 1)	1
6	(x_2, x_3)	(3, 1, 0, 2)	1
初始	（基准）	(3, 1, 2, 0)	2

最优代价是 cost=1，一共有 4 个邻居同时达到。看交换下标：(x_0, x_1)下标最小，所以下一步跳到状态 [1, 3, 2, 0]。

第二轮，以[1, 3, 2, 0]作为当前状态，再枚举全部邻居

序号	交换的列对	新状态 (x_0, x_1, x_2, x_3)	cost
1	(x_0, x_1)	(3, 1, 2, 0)	2
2	(x_0, x_2)	(2, 3, 1, 0)	2
3	(x_0, x_3)	(0, 3, 2, 1)	4
4	(x_1, x_2)	(1, 2, 3, 0)	4
5	(x_1, x_3)	(1, 0, 2, 3)	2
6	(x_2, x_3)	(1, 3, 0, 2)	0 ← 目标!
当前	（基准）	(1, 3, 2, 0)	1

出现 cost=0，已经完全没有冲突，得到四皇后的合法解。 cost 最小只能等于 0，不可能继续优化，爬山算法直接终止。最终解：[1, 3, 0, 2]

9.4 Exercise 4：A* 算法

Perform the A* Algorithm on the following graph. Explicitly write down the queue at each step, where the red figure means the heuristic value, and the black figure is the path cost.

在给定图上执行 A* 算法，并显式写出每一步的队列；图上红色数字是启发值 h，黑色数字是路径代价 g。

i	c（展开谁）	open_list	close_list	本步发生了什么
0	—	[S]	[ ]	初始化：open_list.append(start)
1	S	[A, B, C]	[S]	展开 S → 加入 A、B、C
2	C	[A, B, D]	[S, C]	f 最小者为 C（14）→ 加入 D
3	A	[E, B, D]	[S, C, A]	f 最小者为 A（16）→ 加入 E
4	E	[B, D, F]	[S, C, A, E]	f 最小者为 E（16）→ 加入 F
5	F	[B, D, G]	[S, C, A, E, F]	f 最小者为 F（17）→ 加入 G
6	D	[B, G]	[S, C, A, E, F, D]	f 最小者为 D（18，与 B 并列，课件取 D）；D → F 无改进

i	c（展开谁）	open_list	close_list	本步发生了什么
7	B	[G, D, E]	[S, C, A, F, B]	展开 B → E（f 降到 15）与 D（f 降到 14）被重开，从 close_list 移出
8	D	[G, E]	[S, C, A, F, B, D]	f 最小者为 D（14） → 重新展开；D → F 仍无改进
9	E	[G, F]	[S, C, A, B, D, E]	f 最小者为 E（15） → E → F 使 F（f 降到 16）被重开
10	F	[G]	[S, C, A, B, D, E, F]	f 最小者为 F（16） → F → G 使 G 的 f 降到 18
11	G	—	—	current_node == end → return，算法结束

Lecture 3

part 1 Search&Optimization Recap 搜索优化回顾

1.1 Search

Search 搜索：Agent（智能体）能够构造**动作序列**以达成其目标。

The agent can construct sequences of actions that achieve its goals.

Uninformed (blind) search 无信息（盲目）搜索：除初始状态、搜索算子与解的测试外，不使用任何其他信息。*Uses no information other than the initial state, the search operator, and a test for a solution.*

Informed search 有信息搜索：可访问启发函数 h(n)，用来估计从 n 到解的成本。*Has access to a heuristic function h(n) that estimates the cost of a solution from n.*

**算法 ** Algorithm	**展开准则 ** Expansion Criterion	**公式 ** Formula
Greedy Best-First Search 贪婪最佳优先	展开 h(n) 最小的节点	f(n) = h(n)
A* Search	展开 f(n) 最小的节点	f(n) = g(n) + h(n)

其中 g(n) = 从起点到 n 的实际成本，h(n) = 从 n 到目标的估计成本。

A* 在 h 满足可采纳性（admissible）与一致性（consistent）时最优。

1.2 Optimization

Optimization 优化：通过**最小化成本函数**或**最大化适应度函数**，从一组候选中找出最佳解。*Finding the best solution from a set of candidates by minimizing a cost function or maximizing a fitness function.*

Simulated Annealing (SA) 模拟退火：以**概率方式接受更差的动作**。概率取决于成本差 ΔE 与温度 T：**高温探索（explore）、低温利用（exploit）**，从而跳出局部极小。*Accepts worse moves probabilistically; high T explores, low T exploits. It escapes local minima.*

Genetic Algorithms (GA) 遗传算法：维持一个**种群**，通过**选择（selection）、交叉（crossover）、变异（mutation）**逐代进化出更好的解。*Maintain a population of solutions and apply selection, crossover, and mutation to evolve better solutions over generations.*

Schema Theorem 模式定理：**定义长度短、阶数低、适应度高于平均**的模式（schema）获得指数级增长的试验次数，这些是搜索的**积木块（building blocks）**。*Short defining length, low order, above-average schemata receive exponentially increasing trials.*

part 2** **Evolutionary Algorithms 进化算法

2.1 Definition and Origin

EAs are **population-based **search approaches that are inspired by the principles of natural selection and genetics.

一类基于种群的搜索方法，其灵感来源于自然选择与遗传学原理。

Era	Milestone	Who / What
1950	Alan Turing 提出用自然进化构建智能机器	*“Computing machinery and intelligence”, Mind 49: 433-460, 1950.* 进化驱动生物系统的适应、多样性与创新。
1960s Sprouting Period 萌芽期	Alexander Fraser 尝试模拟遗传系统并研究其进化动力学	Evolutionary Programming 思想萌芽
1980s / 1970s	**Genetic Algorithm 遗传算法**	John Holland 提出模拟自然进化过程来求解优化问题
1990s	**Particle Swarm Optimization 粒子群优化**	James Kennedy （社会心理学家） & Russell Eberhart （电气工程师）提出，模拟 **鸟群飞行行为** 求解优化问题（ 1995 ）
1990s	**Genetic Programming 遗传编程**	John Koza 提出用遗传算法 **进化计算机程序代码**
1990s - 2000s	**Neuroevolution 神经进化**	Kenneth Stanley & Risto Miikkulainen 提出 **NEAT** （ NeuroEvolution of Augmenting Topologies ）算法
2016 Applications in the New Era	Uber & OpenAI 展示进化计算的强大能力	EC 进入新世代工业应用
Now 至今	ETO / NAS 等前沿方向持续发展	与深度学习、迁移学习融合

2.2 The Four-Step Loop

Step	Operation	Description
1	**Initial population** 初始种群	从一个初始种群（一组候选解）开始 Start with an initial population
2	**Evaluation & Selection** 评估与选择	评估并选择出更适应的个体 Evaluations and selection performed to identify fitter individuals
3	**Crossover & Mutation** 交叉与变异	通过交叉和变异进行繁殖（ reproduction ） Crossover and mutation carried out for reproduction
4	**Iterate** 迭代	迭代进化直到满足停止准则 Iteratively evolves until stopping criteria are satisfied

2.3 Three Advantages

EA 的种群移动趋势本身携带**可跨问题迁移的信息**，可以用来引导搜索走向有希望的最优区域。*The population movement patterns carry transferable cross-problem information to guide the search towards promising optimal regions.*

特点：

1. **Strong global search capability 强大的全局搜索能力**

不依赖梯度，适合黑盒问题；

2. **Easy to implement 易于实现**

优化逻辑简单：初始化 → 评估 → 选择 → 交叉变异 → 迭代，易于实现；

3. **Self-adaptation and self-learning 自适应与自学习**

过程信息可复用：一次搜索中往哪里走更好的经验，可以被搬到同类问题上（Part 7 ETO范式）。

Part 3 Particle Swarm Optimization

3.1 Core Idea

PSO is a robust stochastic optimization technique based on the movement and intelligence of swarms.

PSO 是一种鲁棒的随机优化技术，基于群体的运动与智能

PSO 将社会交互（social interaction）的概念应用于问题求解。

术语 Term	含义 Meaning
Swarm 群体	由多个particles（粒子）构成，在搜索空间中移动寻找最优解。
Particle 粒子	N 维空间中的一个点，依据自身飞行经验与其他粒子的飞行经验调整其飞行。
pbest 个体最佳	该粒子自身迄今到达过的最佳位置（ personal best ）。 The best solution achieved so far by that particle.
gbest 全局最佳	该粒子邻域内任意粒子迄今取得的最佳值（ global best ）。 The best value obtained so far by any particle in the neighborhood.

The basic concept of PSO lies in accelerating each particle toward its p-best and g-best locations, with a random weighted acceleration at each time step.

PSO 的基本原理：在每一时间步，以随机加权加速度将每个粒子朝其 pbest 与 gbest 位置加速。

3.2 Update Equation

$Vid = w · Vid + c1 · Rand() · (pid − xid) + c2 · Rand() · (gd − xid) (1) $

$xid = xid + Vid (2) $

1. *w* · *Vid* **惯性项**（inertia）：保持原有运动趋势；
2. *c*1 · *Rand*() · (*pid* − *xid*) **认知项**（cognitive / personal）：向自身历史最佳靠拢；
3. $c2 · Rand() · (gd − xid) $ **社会项**（social）：向群体最佳靠拢。

Symbol	Meaning
V id	每个粒子在每一维上的**速度**
i	粒子索引

Symbol	Meaning
d	维度
w	**惯性权重**
c 1 , c 2	常数，分别对应**认知与社会**分量
Rand()	[0, 1] 之间的随机数
p id	每个粒子的 **最佳位置**
g d	**群体最佳位置**
x id	每个粒子在每一维上的**当前位置**

3.3 Parameters Tuning

Parameter	Range	Effect
惯性权重 w Inertia Weight	[0.5, 1.5]	大w促进**全局搜索(global search);** **小w促进局部搜索（local search)。** 沿运行过程**线性递减**w比固定值表现更好。
c1,c2	[0, 4.0]	若c2>c1：社会信息影响更大; 若c1>c2：粒子更受个体最佳影响而非全局最佳。 c_2 大听社会，c_1 大听自己。
Rand()	[0, 1]	均匀随机数

3.4 Comparison with other EC Techniques

特征 Feature	PSO	GA / EP / ES
选择操作Selection	**没有**选择操作	有选择操作
种群存续	所有粒子**始终保留**在种群中	个体可能被淘汰
交叉/变异crossover / mutation	**不需要**	核心算子
全局vs局部平衡	通过**惯性权重 w** 调节	通过其他机制调节

Tips: PSO 用速度 + 向最佳位置加速代替了 GA 的选择 + 交叉 + 变异

没有个体被淘汰，没有基因重组，全部通过连续的位置更新实现。

3.5 Why Particle?

3.5.1 Particle vs Point

问题：为什么PSO 里的个体叫 particle（粒子），而不是 point（点）

原因：粒子携带位置、速度与加速度，而点只有位置。

正因为粒子带着运动的惯性，它才能把上一次的运动方向保留下来（即公式中的 $w・Vid $惯性项），并在个体记忆（pbest）与群体信息（gbest）的共同作用下不断修正前进方向。

3.5.2 In-Class Demo

Step	Description
初始配置	20个粒子，**随机或规则排布**，并赋一个**很小的****随机初速度**
运动规则	每个粒子同时追踪**pbest（自己的历史最佳）与gbest（群体的历史最佳）**
过程现象	粒子逐渐**漂移到有希望的区域**；公式中的随机项让粒子**不会过早停滞在局部极小**
最终结果	收敛到（近似）全局最优，**全程不需要梯度信息**

Tips:

1. 因为不依赖梯度，PSO 天然适用于**梯度不可得或不可微（gradient-free）**的实际问题
2. 这里的随机成分不是为了增加随机性，而是**防早熟，**它让粒子有概率跳出当前正在聚集的区域，这与模拟退火的按概率接受更差解是同一种思路的两种表达。

Part 4 Differential Evolution 差分进化

4.1 Background

Item	Content
提出者Authors	**Price&Storn**，1995年
定位Positioning	被认为是**实数函数优化**最强大的进化算法之一
出处Paper	*“DifferentialEvolution—asimpleandefficientadaptiveschemeforglobaloptimizationovercontinuousspaces”,1995.*
竞赛成绩 Competition	1996年在Nagoya举行的首届国际进化计算竞赛中获得第3名
性能对比 Performance	在**34函数测试集**上**优于GA和PSO**（Vesterstrom&Thomsen,2004）
持续表现	在IEEECEC系列会议中，对动态、多目标、约束、多模态等各类优化问题持续展现卓越性能

4.2 Main Idea

利用下式生成试验向量（trialvectors,v）:

$v = x_r1 + F \cdot (x_r2 - x_r3)$

其中 x_r1, x_r2, x_r3从当前种群中随机选取，F称为缩放因子（scalingfactor）。

优势：DE的自适应变异引擎，具有随种群收敛自动调节的性质，是DE**参数极少、易用**（few parameter sto be tuned）的根本原因。

解读：DE的差分体现在 $ x_r2,x_r3 $：用两个随机个体之差作为扰动方向与步长。这种扰动幅度不是人为设定的固定步长，而是有当前种群的分散程度决定的。

种群越分散，$(x_r2 - x_r3)$步长越大，偏向探索（exploration）;

种群收敛，$(x_r2 - x_r3)$步长自动减小，自适应地平衡探索与开发（exploitation）。

4.3 DE Algorithm

步骤	伪代码Pseudocode
1	用**均匀随机值**初始化种群X
2	根据适应度函数确定X中各成员的质量
3	**repeat**（重复）
4	对种群中的每个父代Xc:
5	随机选择差分对Xa和Xb
6	计算X′ c=Xc+F・(Xa−Xb)
7	计算X″ c=Crossover(Xc,X′ c)
8	确定X″ c的质量
9	**if **Xc″ 比Xc更好; **then**Xc″ 进入新种群; **else **Xc进入新种群
10	**endfor**；新种群替换旧种群
11	**until**种群收敛或达到代数上限

Tips: DE采用1-to-1**贪婪选择（greedyselection)**，即只有当试验向量 X_c优于父代 $ X_c $时才替换，否则保留父代。优势如下：

1. 种群规模恒定：每个父代对应一个试验向量，种群大小不随迭代变化
2. 适应度不退化：只有变好才接受，因此种群质量单调不降（fitnessneverdegrades）
3. 实现极简：不需要轮盘赌、不需要排序，一次比较即可
4. 与GA的差别：GA（先选父代→再重组）；DE是每个父代都给一次机会（只用进行一个减法和比较）选择与繁殖耦合更紧。

4.4 DE Mutation Scheme 变异方案命名约定

Symbol	Meaning
DE	DifferentialEvolution（差分进化）
x	表示**被扰动向量**的字符串（thevectortobeperturbed）
y	用于扰动x的**差分向量个数**（numberofdifferencevectors）
z	所采用的**交叉类型**（typeofcrossover）

Part5 Memetic Algorithms

5.1 Development in EC

Era	Method
1960s	Evolutionary programming 进化规划
1970s	Genetic algorithm 遗传算法
1970s	Evolutionary strategies 进化策略

Era	Method
1990s	Evolutionary algorithms 进化算法
1990s	Genetic programming 遗传编程
1990s	**Memetical gorithms 模因算法**

5.2 Definition & Metaphor

模因算法是一类同时整合全局搜索（交叉+变异）与局部搜索的进化算法。

MA are evolutionary algorithms that are in corporate global search (crossover+mutation) and local searches simultaneously.

层次	机制	对应算法成分
跨代 Cross-generation	**进化适应** Evolutionary adaptation	遗传算法GA
个体生命周期 Within a lifetime	**个体学习** Individual learning	局部搜索器 problem-specific local searchers

5.3 Preservation of Diversity

局部搜索（local search）：一种使算法能够在**当前个体的邻域**内获得更好解的机制。

Mechanism that enables the algorithm to obtain better solutions within the neighborhood of the current individual.

问题：在MA中使用局部搜索很容易导致多样性丧失（lossofdiversity）因为所有点/个体都被移动到同一个局部最优。

Strategy	Description
限制局部搜索比例	只允许**一小部分种群**进行局部搜索 Allow only a small fraction of the population to undergo local search
使用多种局部搜索器	引入**不同的搜索空间与不同的局部最优** Use multiple local searchers to induce different search spaces and distinct local optima
选择算子去重	在选择算子中**防止重复**个体 Prevent duplicates in the selection operator
新的接受准则	为局部搜索引入新接受准则，如**Boltzmann方法**（类似模拟退火） New acceptance criteria, e.g. Boltzmann method (similar to Simulated Annealing)

5.4 Utilizing Domain Knowledge

GA 可以通过引入问题特定的领域知识而更具针对性。领域知识表现为：

- Specialized crossover 专用交叉算子
- Specialized mutation 专用变异算子
- Problem specific representations 问题特定的表示
- Problem specific local search 问题特定的局部搜索让 EA 在具体任务上表现更好，就催生了模因算法（Memetic algorithms）

5.4.1 General Workflow of MA

步	流程 Step
1	Initialize population 初始化种群
2	Evaluate each candidate 评估每个候选
3	Selection and Recombination 选择与重组
4	Mutation 变异
5	**Improve via local search** 通过局部搜索改进 ← MA 特有
6	回到步骤2，循环（Improve via local search ）

领域知识的注入位置是可选的 Multiple Injection Points

MA的工作流保留了标准进化循环（选择→重组→变异），差别只在第五步（评估之前多一步局部搜索）。但这一步的插入位置并不唯一：

- 可以在变异之后注入；
- 也可以在重组之后、评估之前注入；
- 还可以通过专门的进化算子把领域知识写进搜索过程。

具体插在哪，取决于问题的性质与算力预算。所以MA的本质就是全局评估+局部精炼的循环。

5.5 Performance of EA

可见，早期进化算法在通用性上强，但在特定问题上不如专用算法；加入领域知识（→ MA）后可显著提升针对性。

方法类型	适用面	峰值表现
Generalist 通用方法（纯 EA ）	广	中等但稳定
Specialist 专用方法	窄	很高
EA + Domain Knowledge （ → MA ）	**较广**	**较高** ← 兼顾两者

Tips: MA 是把**本问题**的知识注入，而 Part 7 的 ETO 是把**其他相似问题**的知识注入。两者动机相同，知识来源不同。

Part 6 Multi-objective Evolutionary Algorithms

6.1 Rational and Motivation

1. 许多**现实问题**有多个（可能相互**冲突**的）目标需要优化。
2. 有些问题被**转化（或聚合）**成单目标问题（SOP），从而用单目标优化技术求解。
3. 把多目标问题（MOP）转成 SOP 需要**先验知识**来设定各目标的相对重要性。
4. 用**多目标框架**定义问题更**通用**，并为决策者提供不同目标之间的**权衡（tradeoffs）**。
5. **多目标优化（multi-objective optimization）又称多准则优化（multi-criteria optimization），目标是找到一组向量**，在满足给定约束的同时优化多个目标函数。

6.2 Classification of MOP

对一个 MOP，目标数 M 必须**大于 1**。根据约束可分类为：

不等式约束J	等式约束K	x_i^L	x_i^U	MOP Type
0	0	$-\infty$	∞	**Unconstrained MOP** 无约束
0	0	C_1	C_2	**Bound constrained MOP** 边界约束
> 0	0	x	x	**Inequality constrained MOP** 不等式约束
0	> 0	x	x	**Equality constrained MOP** 等式约束

其中 C1、C2 为常数。为简化，用 S 表示**可行决策空间**（满足约束的区域）。MOP 可重写为：

$minx \in S[f_1(x), f_2(x), \cdots, f_M(x)]$

6.3 Dominance Relation 支配关系

Definition：**解 **$x_1 * * 支配（*dominate*）另一个解* *x_2 $，记作 $ x_1 ≼ x_2$，当且仅当以下两个条件同时成立：

1. x_1在**所有目标**上都不比 x_2差（no worse in all objectives）
2. x_1在**至少一个目标**上严格优于 x_2（strictly better in at least one objective）

$x_1 ≼ x_2 \iff$ $ f_i(x1) ⩽ f_i(x2) $\forall i \in \{1,\cdots,M\}$ $ 且 $\exists j \in 1,...,M | fj(x1) < fj(x2)$

6.4 Three Key Definitions

Definition	内容 Content
非支配集 ** **Non-Dominated Set	在一组解P中，非支配解集P′ 由**不被 P 中任何成员支配**的解组成。 Among a set of solutions, P, the non-dominated set P′ are those not dominated by any member of P.
Pareto 最优集 **Pareto Optimal Set**（决策空间）	在**可行目标空间**中**非支配**的解的集合。 The set of solutions that are non-dominated in the feasible objective space.
Pareto 最优前沿 **Pareto Optimal Front**（目标空间）	在**可行目标空间**中的**非支配解集** 。 The set of non-dominated solutions in the feasible objective space.

Two-Objective Example：两个待最小化目标 f_1 与 f_2，Pareto 最优前沿是**一条最优权衡曲线（trade-off curve）**，位于**不可行区域**（unfeasible region）的边界上。

which one is better? Pareto 前沿上的点相等，所以**互不支配**，无法仅凭目标值判断谁更优，需依赖决策者偏好。

6.5 Pareto Ranking

Why is sharing needed for MO optimization? 为什么多目标优化需要共享机制?

答：Pareto 排序只关注收敛，会让种群聚集到前沿的某一小段；共享函数通过降低拥挤区域个体的适应度，迫使种群散布在整个前沿。

6.6 Mainstream MOEAs

Evolutionary Multi-objective Optimization (EMO)是进化算法用于求解多目标优化问题的应用。MOEA 用于逼近多目标问题的 Pareto 最优前沿。

Algorithm	Full Name
MOGA	Multi-objective Genetic Algorithm 多目标遗传算法
NSGA-II	Non-Dominated Sorting Genetic Algorithm II 非支配排序遗传算法 II
IMOEA	Incrementing Multi-objective Evolutionary Algorithm 增量式多目标进化算法
MOEA/D	Multiobjective Evolutionary Algorithm based on Decomposition 基于分解的多目标进化算法

这些算法主要在以下方面不同：适应度分配方案（fitness assignment scheme）、多样性保持（diversity preservation）、共享与精英策略（sharing and elitism strategies）。

6.7.1 General MOEA Flow

步	流程Step
1	Initialization 初始化： $x_1, x_2, \cdots, x_N$
2	Fitness Evaluation 适应度评估： $f(x_1), f(x_2), \cdots, f(x_N)$
3	Mating Selection 交配选择
4	Recombination / Crossover 重组 / 交叉
5	Mutation 变异（ Perturb the generated offspring solution 扰动生成的子代解）
6	Fitness Evaluation 适应度评估
7	Selection 选择（ Select individual to survive 选择存活的个体）
8	Termination 终止检查（ Check terminated condition ）

6.7.2 Issues&Strengths

EMO needs to address these issues	Why EA is powerful for MO
适应度分配、偏好 （Fitness assignment, preference）	基于种群，能搜索**全局权衡** （global trade-off）
良好散布、均匀分布、最小邻近 （Good spread, uniform distribution, minimum proximity）	鲁棒，适用问题范围广 （Robust and applicable to a wide range of problems）
多目标、多约束 （Many objectives, constraints）	能处理**不连续与多维**问题 （discontinuous and multi-dimensional）
噪声、动态环境、鲁棒优化 （Noise, dynamic landscape, robust optimization…）	

6.6 What to change in Simple GA for MO optimization?

修改适应度计算（Modify the Fitness Computation）：

- 通过Pareto排序强调非支配解 → 促进收敛性（convergence）
- 通过共享函数（sharing function）强调不拥挤的解 → 促进多样性（diversity）

6.8 A House-Buying Example

6.8.1 House Buying as a MOP

用买房来解释什么叫做相互冲突的多个目标：

目标 Objective	冲突 Conflict
地段好坏	地段越好 → 价格越高
价格高低	价格越低 → 地段与房屋条件往往越差
房屋条件	条件越好 → 价格越高

没有唯一最优解，只有不同的权衡（trade-off）。

把这三个目标加权求和成单目标再优化，就必须先知道地段 vs 价格 vs 条件各自的权重，而这恰恰是决策者事先给不出的先验知识（prior knowledge）。

MOEA 的价值：不做加权聚合，直接求出一整组权衡解（Pareto 前沿），把最终选择留给决策者。

6.8.2 Partial Ranking

偏序排序（partial ranking）的两个条件

调整 Adjustment	作用 Effect
偏序排序 Partial ranking	优先保留**非支配解，促进收敛（convergence）**； 非支配个体rank = 1，其余个体 rank=1+**支配它的个体数量**
共享机制 Sharing	优先保留**不拥挤的解** → 促进**多样性（diversity）**

6.8.3 Why Sharing Is Necessary

Why sharing is needed for MO optimization?

只有Pareto排序	加上Sharing共享机制
只强调**收敛（convergence）**	同时强调**多样性（diversity）**
种群会**聚集到Pareto前沿的某一个点或一小段**	通过**降低拥挤区域个体的适应度**，迫使种群**散布在整个前沿**
结果：只拿到一个折中方案	结果：拿到**完整的前沿近似**

偏序排序负责往前沿推，共享机制负责在前沿上散开，两者缺一不可。这也解释了为什么多目标优化比单目标更复杂：单目标只需回答哪个最好，多目标还要回答这一组解分布得够不够开。

6.9 Limitation and Observation of EA

1. EA 中独立的搜索与迭代过程常导致缺乏有效引导、搜索效率低。
2. 当前的进化优化器往往从零开始搜索，不利用求解相似问题的经验。
3. 任务/问题之间的共同信息（common information）若被恰当利用，可有效帮助问题求解。

Sphere 函数优化过程中找到的解，**有可能**被用于帮助更复杂的 **Rastrigin 函数**的优化，这正是知识迁移的切入点，直接引出 Part 7 的 ETO。

Part 7 Evolutionary Transfer Optimization (ETO)

7.1 Motivation and Definition

Motivation: Knowledge Learning and Transfer.

人类具备跨任务学习与迁移知识的能力，机器可以通过微调（fine-tuning）复用已训练的模型来实现知识迁移。

Definition: A paradigm that integrates evolutionary optimizers with knowledge learning and transfer across related domains for better searching performance.

进化迁移优化（Evolutionary Transfer Optimization, ETO）是一种将进化优化器与跨相关领域的知识学习与迁移相结合，以获得更好搜索性能的范式。

EA + KT（Knowledge Transfer）→ 更好的性能 Performance

Benefit	Description
Head start 更好的起点	从更好的初始点出发
Faster convergence speed 更快的收敛速度	缩短优化过程
Closer approximation 更接近的逼近	更接近最优解

适用问题类型：约束优化问题（Constrained）、动态优化问题（Dynamic）、多目标优化问题（Multi-objective）。

7.3 Why ETO?

ETO aims to guide and accelerate the search process in the new problem space by leveraging information and solutions from similar optimization problems.

ETO 的目标：通过利用来自相似优化问题的信息与解，来引导并加速新问题空间中的搜索过程。

维度 Dimension	目标 Goal
Efficiency 效率	提升搜索效率、 **减少计算资源** Enhance search efficiency and reduce computational resource
Effectiveness 有效性	处理**复杂问题** Handle complex problems
Robustness 鲁棒性	提升**适应性与鲁棒性 **Improve adaptability and robustness** **应对**环境变化 **Environments Changing

7.4 Paradigms of ETO ETO范式

范式 Paradigm	知识来源 Knowledge Source	结构示意 Structure
Evolutionary **Sequential Transfer Optimization 进化序列迁移优化	从**先前已求解的源任务**中捕获的知识，支持后续优化 Knowledge captured from previously-solved source tasks	Source Tasks → Knowledge Base → Target Task

范式 Paradigm	知识来源 Knowledge Source	结构示意 Structure
**Evolutionary **Multitask Transfer Optimization 进化多任务迁移优化	在**多个正在进行**的任务间**共享**的知识 Knowledge shared across multiple ongoing tasks	Task 1 / Task 2 / ⋯ → Knowledge Base（同时进行）
**Evolutionary **Multiform Transfer Optimization 进化多形式迁移优化	把**复杂原任务分解**成更简单的替代源任务，积累经验反哺原任务 Knowledge obtained from alternate formulations of the original task	Formulation 1 / Formulation 2 → Knowledge Base → Target Task
**Evolutionary **Dynamic Transfer Optimization 进化动态迁移优化	从**历史信息**中提取的知识，提升当前任务的表现 Knowledge extracted from historical information	Dynamic Tasks 1 → 2 → ⋯ → t-1 → t，共享 Knowledge Base

Tips: 序列 = 过去任务间；多任务 = 并行任务间；多形式 = 同一任务的不同表述间；动态 = 随时间变化的同一任务。

7.5 Key Issues of ETO

设计新的知识学习与迁移方法对 ETO 而言是必要的。核心三问：

Question	Explanation
What to transfer?	选择**具有跨任务共性（latent inter-task communality）的可迁移知识：解（ solutions ）、特征（ features ）、参数（ parameters ） ⋯⋯ 并挑出最有希望**的候选
When to transfer?	需要**自适应的迁移时机**：迁得太频繁 → **负迁移（negative transfer）**； 迁得太少 → 白白损失正向收益。或不迁移No transfer。
How to transfer?	选择合适的迁移方式，如**直接朴素平移（direct naive translation）**、**精细的拓扑变换（fine topological transformation）**等

负迁移（Negative Transfer）

从不相关或弱相关的任务迁移知识，反而拖累当前任务的优化。ETO的关键难点之一就是只迁该迁的；在不该迁时，果断不迁。这也解释了为什么No transfer本身就是When to transfer的一个合法答案。

7.6 ETO Successes

Application	Result
能源系统优化	运营成本降低 **20%**
污染路径问题	污染扩散减少 **50%**
油藏生产优化	收益增加 **700 万美元**
气动设计优化	升阻比提升 **13%**

7.7 ETO benefits &MA

收益 Benefit	课堂说明 Explanation
更好的初始种群（Head start）	不用随机初始化，而是从一个表现更好的初始种群出发→ 稳健的领先优势
更快的收敛 + 更低的计算成本	复用存储下来的历史搜索经验，避免从零开始
更接近全局最优	借助简单问题到复杂问题之间的知识迁移提升解的质量； 对动态环境把历史状态知识迁移到当前已变化但相似的地形上，增强鲁棒性

Dimension	MA 模因算法	ETO 进化迁移优化
知识来源	**本问题自身**搜索过程中的局部搜索知识	**其他相似优化问题**（跨问题域）的信息
类比	一个人自己反复琢磨同一道题	把做过的同类题的解法搬过来
作用范围	单个优化过程内部	**问题与问题之间**

Part 8 Neural Architecture Search (NAS)

8.1 Development of Neural Architecture

Period	Stage	Representative Models
1943~1980	**Fundamentals** 基础	M-P模型、 MLP（多层感知机）
1990s	**Various architectures** 多种架构	LSTM、LeNet-5、RBM、HNN
2012	**Deep architecture** 深度架构	AlexNet
2014 - 2016	**Deeper models** 更深的模型	VGGNet 、ResNet 、DenseNet
2017	**Novel fundamental model** 全新基础模型	Transformer
Now 至今	**Large models** 大模型	BERT、GPT、Llama

8.2 NAS as an Optimization Problem

神经网络架构设计常可被形式化为一个**优化问题**，神经架构搜索（Neural Architecture Search, NAS）正是为求解该问题而提出的。

NAS问题的三大特性

- Bi-level optimization 双层优化：外层搜架构 arch，内层训练权重 w
- Black-block optimization 黑盒优化：无法求梯度/无解析形式
- Expensive optimization 昂贵优化：每次评估都需训练并验证模型

Component	Content
Search space 搜索空间	A（架构空间）
Search method 搜索方法	强化学习Reinforcement Learning、进化算法Evolutionary Algorithm、梯度方法 Gradient-based
Performance evaluation 性能评估	准确率Accuracy、延迟Latency、计算量Flops、模型大小Model size

8.3 MTNAS：Knowledge Transfer Strategy

问题：多任务场景下（图像分类、图像分割、机器翻译），若对每个任务独立运行 NAS（Task1 → NAS、Task2 → NAS、Task3 → NAS），计算开销巨大且无法复用经验。

思路：让多个任务的架构搜索**共享知识，**这正是ETO思想在NAS上的落地。

Mechanism	Description
Knowledge extraction 知识提取	从搜索过程中提取可复用知识
Knowledge reuse 知识复用	提出一种**局部搜索方法**用于知识复用
Transfer probability adaption 迁移概率自适应	对每次搜索过程**自适应调整迁移概率**： divi：来自**搜索过程自身**的有用程度； di：来自**公共档案（common archive）**的有用程度。

迁移概率示意：0.8 / 0.7 / 0.9 / 0.6 / 0.5 / 0.5，随搜索进程动态变化。

8.4 Results & Ablation Study

Experiments	Conclusion
Search Cost on NAS-bench-201	MTNAS至少实现**2×时间缩减**
Performance on Trans-bench-101	可实现 **2 - 10000× 加速**（2-10000x speedup）
Performance on DARTS	能为**多个任务**找到更好的架构（find better architectures）

Conclusion	Value
NAS-Bench-201 基准	仅需现有SOTA方法**一半的评估次数**即可找到全局最优架构
性能vs时间成本	在性能与时间成本上**均优于**对比方法
消融验证的四个模块	**知识共享 / 知识提取 / 知识复用 / 迁移概率自适应，**每个模块都有效
搜索轨迹可视化	证明方法**快速移动到最优架构**，并借助知识共享**逃离局部最优**

8.5 Visualization

Overview of the search space and search process：可视化展示进化搜索过程如何在搜索空间中探索，以及如何**跳出局部最优（Escape from the local optima）**这是本章所有算法（EA / PSO / DE / MA / MOEA / ETO / NAS）的共同主题。

8.6 NAS Loop & MT-NAS Archive 交互架构与共享机制

8.6.1 The Common NAS Loop 通用交互架构

环节 Stage	内容 Content
定义搜索空间	定义**极其庞大**的候选架构空间（ a massive search space ）
应用搜索方法	常用**进化算法**， 也有强化学习、梯度方法
评估采样架构	采样一个架构 → **训练 + 验证** → 得到性能

环节 Stage	内容 Content
反馈指导迭代	把评估结果反馈回去， **指导下一轮搜索**

8.6.2 Two Problems of Single-Task NAS

单任务架构在多任务场景中的困境

1. 成本线性增长：对每个目标任务独立跑一遍 NAS，任务数越多，总开销线性上升；
2. 没有万能架构：由于数据分布漂移（data distribution shifts），不存在一个架构能在所有任务上都最优。

结论：把**知识迁移**引入搜索空间，是降低多任务 NAS 成本的**有希望的方向**。

8.6.3 The Shared Archive Mechanism 共享机制

机制 Mechanism	说明 Explanation
并行搜索 Parallel search	基于EA的**并行搜索范式**，同时为不同任务搜索各自的架构
公共档案 Common archive	设置一个**公共档案**，存放**所有并行搜索过程中**发现的高性能架构
选择性注入 Selective injection	**有选择地**把这些有价值的架构加入**每一个**独立的搜索过程，从而加速整体优化

Tips: ETO 思想在 NAS 上的落地：每个任务各搜各的，但**把别人的好架构放进一个共享仓库**，谁需要谁取。

Part9 Exercise3

9.1 Exercise1：GA 交叉子代适应度计算Crossover

三条染色体 00001, 01000, 11010 用于最大化 f(x, y) = x + y。x 由前 2 位表示，y 由后 3 位表示。第 1 与第 2 条染色体交叉，交叉点在第 4 位与第 5 位之间；第 1 与第 3 条染色体交叉，交叉点在第 2 位与第 3 位之间。请计算子代的适应度值。

注意解码时用二进制

答案汇总：4 个子代适应度分别为 **0, 2, 2, 4**（对应 00000、01001、00010、11001）。注意交叉时**前段取父代 1、后段取父代 2**（反之亦然），不要混淆分段归属。

9.2 Exercise2: Selection (Rank & Fitness Weighting) 选择算子

题目：一个含 4 个个体的二进制 GA，适应度如下（最大化问题）：

Q1：(a) 秩加权选择的概率表

(b) 需选 2 个个体，随机数为 0.15, 0.82

为了确定每个染色体对应的区间，需要计算累积概率：

- 染色体 2: [0, 0.4)，染色体 4: [0.4, 0.7)，染色体 1: [0.7, 0.9)，染色体 3: [0.9, 1.0)

所以选择染色体2和1。

Q2：(a) 适应度加权选择的概率表

适应度总和 = 169 + 576 + 64 + 361 = **1170**

Q2：(b) 需选 2 个个体，随机数为 0.15, 0.82

累积区间：#2 ∈ [0, 0.49)，#4 ∈ [0.49, 0.80)，#1 ∈ [0.80, 0.94)，#3 ∈ [0.94, 1.0)。

Chromosomes 2 and 1（与 Q1 结果相同，但**概率分布完全不同，**注意区分）。

9.3 Exercise3: TSP 变异 + 秩加权选择

题目：两条染色体 ACDBE, ADECB 用于求解旅行商问题，5 个节点的距离矩阵如下。**变异算子**为交换两个变异点的位置。染色体 ACDBE：变异点为第 **2** 与第 **4** 位；染色体 ADECB：变异点为第 **5** 与第 **4** 位。若从父代与子代中按**秩加权选择**选出 2 条染色体，随机数为 0.3, 0.6，会选中哪两条?

路径成本计算 · Cost Calculation

类型	染色体	成本计算 Cost	成本	秩
Parent	ACDBE	AC + CD + DB + BE + EA = 31+15+40+7+10	**103**	2
Parent	ADECB	AD + DE + EC + CB + BA = 8+5+20+9+56	**98**	3
Offspring	ABDCE	AB + BD + DC + CE + EA = 56+40+15+20+10	**141**	1
Offspring	ADEBC	AD + DE + EB + BC + CA = 8+5+7+9+31	**60**	4

Tips: TSP 是**最小化问题**，但 GA 需要最大化适应度。因此**成本越低 → 秩越高**：最优的 ADEBC（60）秩 4，最差的 ABDCE（141）秩 1。

秩加权概率表 · Rank Weighting Probability

Chromosome	Cost	Rank	Probability
ADEBC	60	4	4/(4+3+2+1) = **0.4**
ADECB	98	3	3/(4+3+2+1) = **0.3**
ACDBE	103	2	2/(4+3+2+1) = **0.2**
ABDCE	141	1	1/(4+3+2+1) = **0.1**

累积区间：ADEBC ∈ [0, 0.4)，ADECB ∈ [0.4, 0.7)，ACDBE ∈ [0.7, 0.9)，ABDCE ∈ [0.9, 1.0)。选中 **ADEBC 和 ADECB**。

9.4 Exercise4: 模式定理

题目：考虑繁殖、交叉与变异在遗传算法中的单独与联合作用，推导模式定理以考察种群中各种模式（相似模板）的增长率。第 0 代种群如下：异概率 Pm = 0.01，交叉概率 Pcross = 1。用模式定理估计第 1 代中形如 1**** 的模式（schemata）的期望数量。

因为该模式**定义长度短（d=0）、阶数低（o=1）、适应度高于平均（15 > 12.5）**，所以它获得增长（2 → 2.376）。

Tips:

1. 把 1**** 的 **d(S)** 误算成非 0（只有确定位才计定义长度，单个确定位 → d = 0）
2. 把 fit(S) 当成全体平均（应为**匹配该模式的串**的平均）
3. 忘记乘交叉破坏与变异破坏两个惩罚因子

Part10 Overall

10.1 关于模因算法 On Memetic Algorithms

**"General AI is a good starting point, but adding problem knowledge makes it much stronger." **通用方法是很好的起点，但加入「问题知识」后，它会强得多。

对应知识点：MA = 全局进化搜索 + 问题特定的局部搜索；1990s 的 generalist / specialist / +domain knowledge 三分法。

为什么重要：它一句话概括了 MA 的核心价值，也预示了 ETO 的思路，**引入外部知识。**

10.2 关于进化迁移优化 On ETO

"We human, we do not learn everything from scratch."我们人类，并不是什么都从零学起。

对应知识点：ETO 的核心动机——传统 EA **从零开始搜索**，不复用相似任务的已有经验；而人类会**跨任务迁移知识。为什么重要**：这是理解 ETO「为什么存在」的最直观入口，也是 What / When / How 三问的前提。

10.3 关于遗传算法 On Genetic Algorithms

"Genetic algorithms have been proved to be very powerful in escaping from the local optimum to search for the global maximum."遗传算法已被证明非常擅长逃离局部最优，去搜索全局最大值。

对应知识点：GA 相对传统启发式搜索（A* 等）的核心优势。

注意：是**擅长逃离局部最优，不是保证全局最优。**

10.4 串讲 The Through-line

GA 擅长跳出去（10.3）→ 但通用方法还不够强，要加问题知识（10.1）→ 而知识不只来自本问题，还能来自别的相似问题（10.2）。

这三步正好对应 Part 9（GA 定位）→ Part 5（MA）→ Part 7（ETO）的推进逻辑。

Lecture 4

---

part 1 Neural Network

1.1 Definition

A massively parallel distributed processor

Storing experimental knowledge and making it available for use

神经网络是一个大规模并行分布式处理器，天生具有存储经验知识并使其可用的倾向

Acquire knowledge by the network through a learning process

Store knowledge by the inter-neuron connection strengths known as synaptic weights

知识是通过学习过程获得的

神经元之间的连接强度，即突触权重（synaptic weights），用来存储知识

ANN: **parameterized computational nonlinear** algorithms for numerical data/ signal/ image processing.

人工神经网络是**带参数的、可计算的非线性算法**，用来处理（数值型）数据 / 信号 / 图像

Tips: **parameterized**（带参数 → 所以「学习」= 调参）与 **nonlinear**（非线性 → 所以能拟合真实世界的复杂数据；也让误差面变得非凸，见第 9.3 节的局部极小

1.2 Properties and Capabilities

1.2.1 High computational power

Generalization: producing reasonable outputs for inputs not encountered during training/learning.

Massively parallel distributed structure

1.2.2 Useful properties and capabilities

Nonlinearity: most physical systems are nonlinear.

Adapativity : adapt their **synaptic weights** to changes in the environment.

Fault tolerance: if connecting links are damaged, the response may still be OK.

Tips: 容错的来源是**分布式存储**（distributed nature of information stored in a network），而不是，网络足够大所以坏几个无所谓。

性质 Property	英文原文要点	含义与意义
高计算能力 High computational power	embodied in two core aspects	体现在两点：**泛化 + 大规模并行分布式结构**
泛化 Generalization	Producing reasonable outputs for inputs **not encountered during training**	对**训练时没见过**的输入也能给出合理输出 —— 这才是能用的前提
大规模并行分布式结构	Has a massively parallel distributed structure	信息分布存储在许多单元上，支持大批量信息同时处理
非线性 Nonlinearity	Most physical systems are nonlinear	大多数物理系统是非线性的；线性模型表达不了
自适应性 Adaptivity	built-in capability to adapt their **synaptic weights** to changes in the environment	环境变了就调突触权重 —— 这是「在线学习」的基础
容错性 Fault tolerance	If a neuron or its connecting links are damaged, the overall response **may still be ok**	因为信息是**分布式存储**的，坏掉个别神经元或连接，整体响应仍可接受

1.3 Biological Neural Networks

1.3.1 Biological neurons vs.silicon logic gates

Biological neurons, the basic building blocks of the brain, are slower than silicon logic gates. So it makes up for the slow rate of operation with 2 factors;

**数量巨大: **A huge number of nerve cells(neurons) and interconnections.

**单个神经元的功能远比一个逻辑门复杂: **The function of a biological neuron seems to be much more complex than that of a logic gate.

**1.3.2 生物神经元的结构 **

胞体 cell body：信号处理的核心；

轴突 axon：一根细长的管子（tubular axon），分成多个分支，末端是**终末小球（end bulbs）**，几乎触到其他细胞的树突；

树突 dendrites：大量毛发状分支，形成围绕胞体的极细纤维丛（a very fine filamentary brush）

突触（synapses）的三个要点：

是**终末小球与树突之间的小间隙**（Small gap between an end bulb and a dendrite）；

是神经元之间相互作用（interactions）的**基本结构与功能单位**；

可以对接纳信号的神经元施加兴奋（excitation, active）或抑制（inhibition, inactive）

1.4 Synapse to Weight

生物学习机制和人工学习机制

生物侧 Biological	人工侧 Artificial	对应关系
突触的强度可随激活情况调整（adjustable, activation-dependent strength）	连接权重 w_{kj}	**突触强度 ↔ 权重**：这是全部学习发生的地方
兴奋 / 抑制（excitation / inhibition）	权重的**正 / 负号**	正权重 ≈ 兴奋性连接；负权重 ≈ 抑制性连接
学习与记忆的生物学基础	训练过程（training）改变权重	「学习」= 把权重调到合适的值
神经元的激活与否	激活函数 $\phi(\cdot)$ 的输出	决定这个单元是否「被激活」向下游传递信号

tips：人工神经网络里**唯一会被学习改变的东西，就是连接权重**（以及每个神经元的偏置）。网络结构、激活函数在训练开始前就定死了。反向传播，本质上就是在算每个权重该改多少。

Part 2 Model of a Neuron

2.1 Three Basic Components

A set of synapses or connecting links: characterized by a weight or strength of its own.

一组突触 / 连接：每条连接都有自己的权重或强度

tips：这里的权重是**可调的**，这正是学习发生的地方。

An adder: sum the input signals, weighted by the respective synapses of the neuron, for a linear combiner.

一个加法器：把经过各自突触加权的输入信号求线性组合

An activation function: limit the amplitude of the neuron output, typically[0,1]or[-1,1].

一个激活函数：用来**限制神经元输出的幅度**

2.2 Mathematical Formulation

符号	含义	说明
$x_1, x_2, \cdots, x_m$	输入信号	一共 m 个输入，即输入数据的**维度**（dimension of input data）
w_k1, w_k2, ⋯, w_km	神经元 k 的**突触权重**	每个输入对应一个权重，共 m 个
u_k	线性组合器输出	$u_k = \Sigma\ w_{kj}\ x_j$
b_k	偏置 bias	一个与输入无关的常数项，**移动激活阈值**
$\phi(\cdot)$	激活函数	限制输出幅度、引入非线性
y_k	神经元的输出信号	$y_k = \phi(u_k + b_k)$

2.3 Effect of bias

偏置 b_k 的作用：对 u_k 做一次仿射变换（affine transformation）

2.3.1 定义

v_k = u_k + b_k，这个量被称为**诱导局部场（induced local field）**。

2.3.2 几何含义

b_k 把激活函数的输入**整体平移**，等价于**移动了神经元的激活阈值**（shifts the activation threshold）在Part4.7中体现。

- **三个情形**（slide 10 的三张图）：b_k > 0 曲线左移（更容易被激活）；b_k = 0 不变；b_k < 0 曲线右移（更不容易被激活）
- **一个细节**：当 u_k = 0 时 v_k = b_k ，也就是说 单看偏置就能知道**没有任何输入时这个神经元的起点在哪里**

2.4 Asborbing bias

改写做法：**额外增加一个输入 $x_0 = +1$**（固定输入，fixed input），并令它的权重 w_k0 = b_k。

于是偏置就变成了一条**输入恒为 1 的特殊连接**（图注写作 Fixed Input, x0 = +1），不再需要单独的加法项。

好处：记号统一（少写一个 b_k）、**实现统一**（所有连接一视同仁，权重向量就是一个整体）。

改写前 Before	改写后 After
输入向量 $x = (x_1, \cdots, x_m)^T$	输入向量 $x' = (x_0, x_1, \cdots, x_m)^T$，其中 $x_0 = +1$
权重向量 $w = (w_1, \cdots, w_m)^T$	权重向量 $w' = (w_0, w_1, \cdots, w_m)^T$，其中 $w_0 = b$
y_k = ϕ(Σ_{j=1}^{m} w_kj x_j + b_k)	y_k = ϕ(Σ_{j=0}^{m} w_kj x_j)

Part 3 Activation Functions

3.1 Threshold Function

阈值函数

优点：简单，用符号判断。感知机就是它。

缺点：不可微，在0处不连续，其余位置导数恒为0。

后果：与梯度下降训练不兼容。梯度要么为0，要么不存在。

tips：为什么不能只用阈值函数？阈值函数**不能**用于多层网络的训练。

it adopts a **nonlinear (differentiable)** activation function for each neuron

所以 MLP 与反向传播，每个神经元**必须换成非线性且可微的激活函数**

3.2 Sigmoid and its Derivative

其中 **a 是斜率参数**（slope parameter），且 $a > 0$、$-\infty < v < \infty$。极限值为 **0 和 1**

3.2.1 三条性质

1. **严格单调递增**（Strictly increasing function）
2. **渐近逼近饱和值**（Asymptotically approach the saturation values）
3. **连续且可微**（Continuous & differentiable everywhere）

3.2.2 Example

1. **只要知道神经元的输出 $\phi(v)$，就能直接算出导数，不需要再单独求导 **

反向传播能把梯度算得又快又省的关键

2. **ϕ' 在 $\phi(v) = 0.5$ 时取最大值**（a/4）
3. **ϕ' 在 $\phi(v) \to 0$ 或 1 时趋近于 0** —— **梯度消失**（vanishing gradient）的数学根源

3.3 Common Activation Functions

函数 Function	公式 Formula	输出范围	优点 Advantages	缺点 Drawbacks
Sigmoid 逻辑函数	f(x) = 1/(1+e^{-x})	**(0, 1)**	平滑可微、导数可由输出自表达	**梯度消失 vanishing gradient**
Tanh 双曲正切	f(x) = (e^x − e^{-x})/(e^x + e^{-x})	**(−1, 1)**	**零中心（zero-centered）**，梯度流动比 sigmoid 好	两端仍会饱和
ReLU 修正线性单元	f(x) = max(0, x)	**[0, +∞)**	**计算快**；x > 0 时**没有梯度消失**	x < 0 时出现 **dying ReLU**（神经元永久失活）
Leaky ReLU 带泄漏 ReLU	f(x) = max(a · x, x)，a 为小斜率	**(−∞, +∞)**	用小斜率 a 修好 dying ReLU 问题	需要多调一个超参 a
Softmax	softmax(z_i) = e^{z_i} / Σ_j e^{z_j}	**(0, 1) 且和为 1**	输出可直接当**概率分布**	只用于**分类网络的输出层**（见 4.4）

tips：为什么 Tanh 比 Sigmoid 好？零中心（zero-centered）

sigmoid 的输出恒为正，下一层神经元的输入就恒为正，于是每层梯度的符号高度相关，**参数更新会走出「之」字形（zig-zag）路径**，收敛变慢。tanh 的输出关于 0 对称，这个问题就缓解了。

3.4 Softmax

用途：把一个 **logits 向量**转换成**概率分布**（a probability distribution）

使用位置：**分类网络的输出层**（Output layer of classification networks）

性质：所有输出之和为 **1.0**

3.4.1 Example

logits 为 [2.0, 1.0, 0.5, −0.5]

softmax = [0.5977, 0.2199, 0.1334, 0.0491]，逐项为 **[0.598, 0.220, 0.133, 0.049]**

3.4.2 三条性质

1. 平移不变：softmax(z) = softmax(z + c)

所以，logits 的绝对值不重要，只看差值，所以**logits 不是概率。**

2. 保序：z_i > z_j ⟺ p_i > p_j

所以，取最大概率 = 取最大 logit（argmax 不变）

3. 指数放大了差距：z 差 1 时概率比约 e ≈ 2.72 倍

3.5 How to Choose

使用场景	推荐选择	理由
分类任务的输出层	**Softmax**（多类）/ Sigmoid（二类）	需要把输出解释成概率、且（多类时）和为 1
回归任务的输出层	**恒等函数（线性）**	输出不该被压进 (0,1)，需要实数范围
隐藏层（现代默认）	**ReLU / Leaky ReLU**	计算快、正区间不饱和、缓解梯度消失
隐藏层（需要零中心）	**Tanh**	输出关于 0 对称，收敛路径更直
单神经元分类器（感知机）	**阈值函数**	不需要求导，只需要符号判断

tips:

1. **把 Softmax 用在隐藏层** —— 错。Softmax 只在分类网络的输出层用
2. **ReLU 完全解决梯度消失** —— 不严谨。ReLU 只在 **x > 0** 时梯度恒为 1；**x < 0 时梯度为 0**，这正是 dying ReLU 的成因
3. **Sigmoid 已经不用了** —— 在本讲的语境里不成立：课件后面推导反向传播用的**就是 Sigmoid**（因为它的导数能写成 y(1−y)，形式极简）

Part 4 The Perceptron

4.1 Definition and Structure

4.1.1 Definition

a machine that learns, using examples, to assign input vectors(samples) to different classes.

一台通过样例学习、把输入向量（样本）分配到不同类别的机器。

本质是一个神经元：built around a nonlinear neuron（the hard limiter model）

用的3.1的硬限幅器（阈值函数）

4.1.2 Structure

左边是线性部分（Linear，$v = \Sigma\ w_i x_i + b$），右边是非线性部分（Non-linear，硬限幅）

tips：

1. 它是一个线性分类器（linear classifier）：决策边界是一条直线（二维）/ 一个超平面（高维）
2. 为什么用感知机而不是逻辑回归？因为它不需要求导，是神经网络最早的、也是最简单的形态
3. 线性 + 非线性这个两段式结构是全部神经网络的通用模板：先做加权和（线性），再过激活函数（非线性）。感知机的特殊之处只是它的非线性部分换成了**取符号hard limiter**而已。

4.2 Formulation and Decision Boundary

the equation of the hyperplane (in this example, a straight line) as decision boundary for a two-dimensional, 2-class pattern-classification problem.

用超平面（此例中是一条直线）作为 decision boundary，解决二维的两类模式分类问题

Goal

To correctly classify the set of externally applied stimuli $x_1, x_2,\ldots$ Into one of two classes, C_1 and C_2

把外部施加的刺激 $x_1, x_2, \cdots, x_n$正确分类到两个类别 C_1 与 C_2 之一

Difine

输入向量：$x'(n) = [+1, x_1(n), x_2(n), \cdots, x_m(n)]^T$

权重向量：$w'(n) = [b(n), w_1(n), w_2(n), \cdots, w_m(n)]^T$

线性组合器的输出：$v(n) = w'^T(n) \cdot x'(n)$

1. 方程 $w'^T x' = 0$ 在 **m 维空间**（坐标为 $x_1, x_2, \cdots, x_m$）中定义了一个**超平面（hyperplane）**，它就是两类输入之间的**决策面（decision surface）**
2. 二维时退化成一条**直线**（$w_1x_1 + w_2x_2 + b = 0$）
3. **判决规则**：$w'^T x' \geqslant 0$ → 判为 class 1（例如 y = 1）；$w'^T x' < 0$ → 判为 class 2（例如 y = 0）

Tips: 课件用 $w'^T x' \geqslant 0$ 中**大于等于 0** 归到 class 1。这个≥ 还是 >的约定**必须与题目一致，因为 v = 0 恰好落在决策边界上，两种约定对边界点**的判定结果不同

4.3 How to Select the Weights

方法 Method	做法 How	适用场景	局限
离线计算（off-line calculation）	如果问题**相对简单**，往往可以**从问题的规范直接算出**权重向量	逻辑门这类结构已知的问题	复杂问题算不出来
学习过程（learning procedure）	从给定的（训练）输入—输出向量对（**exemplars**，样例）中确定权重，目标是**对训练向量取得最好的分类**	**本讲重点，也是神经网络的通用做法**	需要数据；收敛性依赖数据性质

Tips: 两种方法的关系是可不可解：离线计算要求把边界先想出来再反推权重；学习过程则是把**确定边界**这一步也交给算法。NAND 门是前者（答案一眼可见），学习算法是后者

4.4 Building NAND Gates——example

用感知机实现 NAND 门。所需的输入—输出关系由真值表与相应图示给出

1. 输入模式（向量）分属两类，在输入空间（平面 (x_1, x_2)）中用两种记号标出；
2. 决策平面是一条直线，方程为 $x_1 + x_2 = 1.5$，即 $\mathbf{-x_1 - x_2 + 1.5 = 0}$；其实只要阈值在 (1,2] 之间都行，**感知机的解不唯一**。
3. 为了让权重向量**指向 y = 1 的那一类**，选取 $w = (1.5, -1, -1)$，即 $w' = [b, w_1, w_2] = [1.5, -1, -1]$；
4. 对每个输入向量计算输出信号 y = 1 if v ⩾ 0 else 0，其中 $v = 1.5 \cdot 1 - 1 \cdot x_1 - 1 \cdot x_2$。

Tips:

线性可分：感知机只能处理能用“直线”一刀切开的问题（所以它搞不定 XOR 异或门）。

权重比例：权重和偏置同比例缩放（如全部翻倍），决策直线的位置不变。

只看符号：分类结果只取决于计算结果是正还是负，而不取决于这个数有多大。感知机只看符号，所有权重同乘一个正数都等价。所以 (1.5,−1,−1) 、(3,−2,−2) 、(15,−10,−10) 都表示同一个 NAND 分类器。

4.5 2-D Example

一个感知机（Perceptron）进行二分类的具体实例。它通过几何图形和数学模型直观地解释了感知机是如何工作的。

1. 分类任务

- 第一类（Class 1）：包含点 $(0,0), (0,1), (1,0), (-1,-1)$；第二类（Class 2）：包含点 $(2.1,0), (0,-2.5), (1.6,-1.6)$。目标是找到一条直线，把这两组点分开。

2. 左侧：几何划分（Geometric Partition）

- 取决策线 $x_1 - x_2 = 2$，即 $x_1 - x_2 - 2 = 0$。对应权重 $w = [+1, -1, -2]$（依次为 x_1、x_2 的权与偏置）
- *求和节点v：计算公式为* $v = (+1 \times -2) + (x_1 \times 1) + (x_2 \times -1)$，*即* $v = x_1 - x_2 - 2$
- 原理：这条线把平面切成两半。 在线的右下方，满足感知机输出 1；在线的左上方，满足感知机输出 0

3. 右侧：感知机模型（Perceptron Model）

这部分展示了硬件或算法层面是如何实现的：

- 固定输入（Fixed Input）：它的权重（Weight）是 -2。这其实就是偏置项（Bias, b）
- 激活函数（Hard limiter）：符号 $\phi(\cdot) $表示，如果算出 $ \ge 0$ 就输出 1，否则输出 0。

4. 总结：Output = 1 if $(x_1 - x_2 - 2) \geqslant 0$, and 0 otherwise.

4.6 Equivalent Perceptrons

等价感知机：无限多组权重

硬阈值感知机（hard limiter / sign）的决策边界只由权重“比例”决定，不由绝对大小决定，所以有无穷多组等价权重

$核心点：感知机输出是阶跃函数，不关心 ∣v∣ 多大，只关心正负 $

$ 所有权重同乘一个正数，边界不变： $(w0,w1,w2) \sim \lambda (w0,w1,w2), \lambda > 0$

4.7 Linear Separability

(a) A pair of linearly separable patterns;

线性可分：**当且仅当（至少）**存在一个权重向量 **w**，使感知机能正确完成分类。

(b) A pair of non-linearly separable patterns.

线性不可分：单层感知机**永远无法**解决 XOR。因为把四个点画在平面上，不可能用**一条直线**把 (0,1)、(1,0) 与 (0,0)、(1,1) 分开，**解不存在**。这个问题直到加上隐藏层才被解决，也是神经网络在 1969 - 1980 年代沉寂的直接原因。

逻辑函数	真值表（输出为 1 的组合）	是否线性可分	一条可用的判定线
AND	仅 (1,1) → 1	**是**	$x_1 + x_2 - 1.5 = 0$
OR	(0,1)、(1,0)、(1,1) → 1	**是**	$x_1 + x_2 - 0.5 = 0$
XOR	(0,1)、(1,0) → 1	**否**	**不存在**这样一条直线

4.8 Beyond Two Dimensions

4.8.1 多维空间中的线性可分

当输入维度达到 3 维或更高时，我们不再使用简单的“直线”来划分数据，而是使用超平面（Hyperplane）。

- 超平面方程：在 m 维空间中，超平面的数学表达式为
- 空间划分：这个超平面将 m 维空间划分为两个区域，分别对应不同的类别，不同区域满足不同方程：

4.8.2 维度挑战

- 二维空间：我们可以通过几何构造（即肉眼观察和作图）轻松判断数据是否线性可分，并找到合适的分类器。
- 三维及更高维：随着维度的增加，直观的几何判断变得不再直接，也无法通过简单的视觉观察来确定分类界限。

4.8.3 核心问题

如果两个类别在数学上是线性可分的，是否存在一种机械化的程序（即算法）来自动获取感知机（Perceptron）的权重 w?

引出了机器学习中著名的**感知机学习算法**，它能够通过迭代自动找到这个超平面的权重。

Part 5 Perceptron Learning

5.1 Setting up the problem

假设感知机的输入变量来自两个**线性可分**的类 C_1 和 C_2。

已知：若 C_1 与 C_2 线性可分，则**存在**权重向量 w^* 使：

$w'^{*T} \cdot x' > 0$ 对所有 $x \in C_1$ … (1)

$w'^{*T} \cdot x' < 0$ 对所有 $x \in C_2$ … (2)

给定训练集 $X = X_1 \cup X_2$（其中 $X_1 \cap X_2 = \varnothing$），且 $X_1 \subseteq$ class C_1、$X_2 \subseteq$ class C_2。

训练问题（Training problem）：寻找一个权重向量 w'^*，使感知机能正确分类训练集 X。注意这个定义里没有最小化误差，**感知机不优化误差**，它只是反复修正直到全部判对（这是**它与后面反向传播最本质的差别**）

5.2 The Learning Algorithm

情形	d	y	(d − y)	更新式
真实属于 C_1，却判成了 2	1	0	**+1**	$w'_k = w'_{k-1} + \eta \cdot x'$ （**朝 x' 方向加**）
真实属于 C_2，却判成了 1	0	1	**−1**	$w'_k = w'_{k-1} - \eta \cdot x'$ （**朝 x' 反方向减**）

感知机学习算法（Perceptron Learning Algorithm, PLA）的数学原理、具体计算步骤以及最终的收敛过程。

5.2.1 感知机权重更新的几何直观

算法的核心逻辑：

- 动态调整：图中展示了决策边界（直线）随着迭代次数 $n = 0, 1, 2, 3$ 的变化。通过不断旋转和移动，直到 $w'^T x' = 0$ 这条线把所有 + 与 − 样本分到两侧。
- w' 是决策面的法向量，样本被漏判为正类时，把 w' 朝这个样本的方向拉（$+\eta x'$） 让这个点在新的法向下投影变正；样本被误判为正类时，把 w' 朝它的反方向推 （$-\eta x'$）。
- η 只影响步长，不影响最终能否收敛（对线性可分数据）。
- 更新量正比于 x' 本身，所以离原点远的样本会造成更大的修正。
- 两种情形合起来就是 $w'_k = w'_{k-1} + \eta(d - y)x'$
- 对于**线性可分**的数据集，感知机学习算法会在**有限次训练轮（epoch）内收敛**到一个有效的决策边界。反之，**对线性不可分的数据（如 XOR），算法会持续振荡、永不收敛。**

```
Algorithm Perceptron;
Start with a randomly chosen weight vector w'₀;
Let k = 1;
while there exist input vectors that are misclassified by w'_{k-1}, do
    Let x' be a misclassified input vector;
    Update the weight vector to w'_k = w'_{k-1} + η(d − y)x'
    where η > 0 and → d = 1 if x belongs to class C₁; d = 0 if x belongs to class C₂
    Increment k
end-while
```

5.3 1-D Worked Example

题目给定的初始条件：

学习率 $\eta = 1$；初始权重 $w'_0 = [-1.5, -2]^T$（依次为偏置 b 与权重 w）

样本呈现顺序：$x' = (1, 0.5)^T, (1, -1)^T, (1, 2)^T, (1, -2)^T$

逐步推演（每一步都写全：算 v → 判 y → 比 d → 决定是否更新）：

迭代	x'	真实 d	$v = w'^T x'$	判出 y	是否更新	更新后的 w'
0	（初始）	—	—	—	—	[−1.5, −2]

迭代	x'	真实 d	$v = w'^{T}x'$	判出 y	是否更新	更新后的 w'
1	(1, 0.5)	1	$-1.5 \cdot 1 + (-2)(0.5) = -2.5$	0	**是**（$0 \neq 1$）	[−0.5, −1.5]
2	(1, −1)	0	$-0.5 \cdot 1 + (-1.5)(-1) = +1.0$	1	**是**（$1 \neq 0$）	[−1.5, −0.5]
3	(1, 2)	1	$-1.5 \cdot 1 + (-0.5)(2) = -2.5$	0	**是**（$0 \neq 1$）	[−0.5, 1.5]
4	(1, −2)	0	$-0.5 \cdot 1 + 1.5 \cdot (-2) = -3.5$	0	**否**（$0 = 0$）	[−0.5, 1.5]

每次更新用了哪条分支：

1. **迭代 1**：$w'_1 = w'_0 + \eta(1 - 0)x' = w'_0 + (1, 0.5)^T = [-0.5, -1.5]^T$（漏判为正类，**加**）；
2. **迭代 2**：$w'_2 = w'_1 + \eta(0 - 1)x' = w'_1 - (1, -1)^T = [-1.5, -0.5]^T$（误判为正类，**减**）；
3. **迭代 3**：$w'_3 = w'_2 + \eta(1 - 0)x' = w'_2 + (1, 2)^T = [-0.5, 1.5]^T$（漏判为正类，**加**）；
4. **迭代 4**：$w'_4 = w'_3$（本轮判对，**不更新**）。

至此完成了**第一个 epoch**。复用训练数据重复上述过程后可以发现，**该算法在这个问题上于第一个 epoch 之后就已经收敛**（即 w'_4 就是解）：$w'_4 = [-0.5, 1.5]^T$

对应判定点为 $x = -b/w = -(-0.5)/1.5 = 1/3$

$w'_1 = [-0.5, -1.5] \rightarrow x = -1/3$（1st）；

$w'_2 = [-1.5, -0.5] \rightarrow x = -3$（2nd）；

$w'_3 = w'_4 = [-0.5, 1.5] \rightarrow x = +1/3$（3rd）。

初始权重 $w'_0 = [-1.5, -2]$ 对应的判定点为 x = −0.75（即图上的 0th）—— 该图的横轴为非等距的自定义刻度，标注位置只作示意，不必按像素间距去反推数值。最终边界 x = 1/3 恰好落在 Class 2 的最大值（−1）与 Class 1 的最小值（0.5）之间，所以它成功分开了两类 。

5.4 2-D Example

有两类模式，分别围绕 (1, 2) 与 (3, 1)，**每类 5 个数据点**

- 用**感知机学习算法**训练；
- **经过 100 个 epoch 之后**，得到的权重向量为：

（自上而下依次为**偏置 **$b = 0.0041$、$w_1 = -0.0349$、$w_2 = 0.0534$）

因此判定线为：$0.0041 - 0.0349 \cdot x_1 + 0.0534 \cdot x_2 = 0$

这个结果怎么读（把它整理成熟悉的斜截式）：

- 移项：$x_2 = (0.0349 \cdot x_1 - 0.0041) / 0.0534 \approx 0.6536 \cdot x_1 - 0.0768$；
- 代入 (1, 2)：$f = 0.0041 - 0.0349 + 0.1068 = +0.076 > 0$，落在正侧；
- 代入 (3, 1)：$f = 0.0041 - 0.1047 + 0.0534 = -0.047 < 0$，落在负侧；
- 两类确实被分到判定线两侧 —— 训练成功。

1. **权重都趋近于 0（0.0041 / −0.0349 / 0.0534）**，说明算法在后期只做极小的修正 ， 感知机在线性可分数据上收敛后，误分样本会越来越少，更新步长自然变小。注意这不是学习率自动衰减，而是错分的样本少了、修正的机会少了。

2. **w_1 是负的、w_2 是正的，与两类样本的位置关系一致**：x_1 越大越可能属于 (3,1) 那一类，x_2 越大越可能属于 (1,2) 那一类。权重的符号直接告诉你哪个特征指向哪一类，这是线性模型唯一可解释的地方，多层网络变黑箱的对照：多层网络的权重就没有这种直观含义了。

这两张图片从网络架构的角度，解释了感知机如何从简单的“单层结构”演变成能够处理复杂问题的“多层结构”。

Part 6 Multilayer Perceptron

6.1 Network Architectures

6.1.1 网络架构的定义

分层网络（Layered Networks）：

1. 单个节点不足以解决许多实际问题（Single node is insufficient for many practical problems）；实践中经常使用大量节点组成的网络；
2. 网络架构（network architecture）定义节点之间如何连接；
3. 分层网络（Layered Networks）：节点被划分成若干子集，每个子集称为一层（layer）；如果 k > j，则没有从第 k 层到第 j 层的连接（No connections lead from layer k to layer j if k > j）；层内连接可能存在（Intra-layer connections may exist）。

层 Layer	名称	说明
Layer 0	输入层 input layer	只负责接收外部数据，不做计算，不计入层数
Layer 1 ⋯ n−1	隐藏层 hidden layer(s)	夹在输入与输出之间，做非线性变换
Layer n	输出层 output layer	产出网络的最终结果

6.2 Single-Layer Feedforward Networks

单层前馈网络

- 计算层：虽然图中有“输入”和“输出”两排节点，输入层（源节点）→ 一层输出神经元，中间没有隐藏层，它被称为“单层网络”。
- 原因：因为输入层的节点只负责分发信号，不进行任何数学运算（不加权、不求和、不激活）。只有输出层的神经元在进行计算 the layer refers to the **output layer of computation nodes**。
- 局限性：正如我们之前讨论的，这种**单层感知机**只能解决线性可分的问题。

6.3 Multilayer Feedforward Networks

多层前馈网络：从单层（只能画直线）到多层（可以画曲线、处理复杂逻辑）

隐藏层（Hidden Layers）概念：

- 隐藏层节点：位于输入层和输出层之间，被称为“隐藏神经元”，是神经网络强大的核心。
- 特征提取：**隐藏层的作用（intervene between the external input and the network output）是介入输入与输出之间，作为中间表达，对原始数据进行加工**。通过多层非线性变换，**网络能够提取出数据中更深层次的特征**。

- 解决复杂问题：通过增加隐藏层，神经网络可以突破线性可分的限制，从而解决像 XOR（异或） 这样单层感知机无法处理的非线性问题，所以隐藏层的加入，使得神经网络具备了模拟任何复杂函数的能力。
- 全连接（Fully Connected）：图中展示了每一层的每个节点都与下一层的所有节点相连，这是最经典的多层感知机（MLP）结构 Neurons in each layer have as their inputs the **output signals of the preceding layer only**。

6.4 Multilayer Perceptrons (MLP)

MLP 是**单层感知机的推广（Generalization of the single-layer perceptron）**，由三部分组成：

1. 一个**输入层**（An input layer）
2. **一个或多个**由**计算节点组成**的**隐藏层**（One or more hidden layers of computation nodes）
3. 一个**输出层**（An output layer of computation nodes）

MLP **一般对每个神经元采用非线性（且可微）的激活函数** it adopts a **nonlinear (differentiable)** activation function for each neuron，例如**逻辑函数（logistic function）**。

其中 $v_j $是神经元 $ j $的**诱导局部场**（加权和 \+ 偏置），$y_j $是该神经元的输出：

$y_j = 1/(1 + exp(-v_j))$

Tips：

为什么必须是可微的？Training Algorithm: Back-Propagation (BP) algorithm。因为训练要靠反向传播，而反向传播要靠链式法则求导，所以激活函数必须可微。

为什么不能就用阈值函数堆多层？因为不可微，梯度传不下去。

这一个因果链串起了阈值函数不可微、必须换 logistic、BP 全部推导。

6.5 The XOR Problem

1. 问题背景

- 任务是**对单位超立方体中的点（取 0 或 1）做分类**；
- **XOR 是非线性可分（nonlinearly separable）的典型**；
- **基本感知机无法解决 XOR 问题，**因为 **XOR 的决策边界不可能是一条直线，不是线性的。**

2. 解决方法

使用一个**两层网络**，每个神经元都用**阈值函数**作为激活函数。

网络含 3 个神经元：**Neuron 1、Neuron 2 在隐藏层，Neuron 3 在输出层。**

Part 7 Back-Propagation

7.1 Training as Optimization

目标：to minimize an energy function representing the instantaneous error.

最小化一个表示「瞬时误差」（instantaneous error）的 energy function（能量函数）

误差函数（能量函数）：

$E = ½ Σ_{j=1}^{n₃} (d_qj − x⁽³⁾{out,j})² = ½ ‖ d_q − x⁽³⁾{out} ‖ ² $

其中 $ d_q $是第 q 个输入模式的期望输出， $x⁽³⁾_out $是 MLP 的实际输出，1/2是为了在求导时抵消平方带来的因子 2。

7.2 The General Learning Rule

权重更新通式：

$\Delta w^{(s)}_{ji} = -\eta^{(s)} \cdot \partial E_q / \partial w^{(s)}_{ji}$

即， $w(n+1) = w(n) - \eta \cdot g(n)$

问题：怎么高效地算出 $\partial E / \partial w$?

解决：链式法则+从输出层往输入层递推，即反向传播。

符号	含义
s = 1, 2, 3	指定网络层（designates the appropriate network layer）
$\eta^{(s)} > 0$	对应层的学习率参数（learning rate parameter），例如取一个常数
∂E_q/∂w	误差对权重的偏导（**反向传播要算的核心量**）

7.3 Output Layer 局部梯度的推导

对输出层的神经元 j：

1. 先用链式法则（By chain rule）把要求的偏导拆成两部分： $\partial E / \partial w^{(3)}_{ji} = (\partial E / \partial v^{(3)}_j) \cdot (\partial v^{(3)}_j / \partial w^{(3)}_{ji})$
2. 第二部分： $v^{(3)}_j = \Sigma_i w^{(3)}_{ji} \cdot x^{(2)}_{out,i}$，所以 $∂v⁽³⁾_j/∂w⁽³⁾ji = x⁽²⁾{out,i} $
3. 也就是前一层（第二层）的输出信号，即进入这个权重的那个输入；
4. 第一部分定义为局部梯度：

输出层的局部梯度：局部梯度 = （期望输出 − 实际输出）× 激活函数的导数，即

$\delta^{(3)}_j = -\partial E / \partial v^{(3)}_j = (d_{qj} - x^{(3)}_{out,j}) \cdot \phi'(v^{(3)}_j)$

最后得到输出层的权重更新规则：

$\Delta w^{(3)}_{ji} = \eta^{(3)} \cdot \delta^{(3)}_j \cdot x^{(2)}_{out,i}$

即 $w^{(3)}_{ji}(k+1) = w^{(3)}_{ji}(k) + \eta^{(3)} \cdot \delta^{(3)}_j \cdot x^{(2)}_{out,i}$

$δ⁽³⁾_j$： δ 被称为该神经元的局部梯度（local gradient），也叫局部误差（local error）。即这个神经元该往哪个方向改、改多猛，误差越大改得越多，激活函数在当前工作点的导数越大改得越多。

7.4 Hidden Layers 局部梯度的递归

原则：隐藏层的神经元没有直接的期望输出，所以它对总误差的贡献是间接的，它通过影响后一层，再影响输出，才作用到误差上。

推导过程（仍然是链式法则，只是展开得更长）：

1. 起点同样是 $\Delta w^{(2)}_{ji} = -\eta^{(2)} \cdot (\partial E / \partial v^{(2)}_j) \cdot (\partial v^{(2)}_j / \partial w^{(2)}_{ji})$；
2. 第二部分依然简单： $\partial v^{(2)}_j / \partial w^{(2)}_{ji} = x^{(1)}_{out,i}$（第一层的输出）；

3. 难点全在 $\partial E/\partial v^{(2)}_j$：要把误差函数展开，用链式法则对它求导，并引入输出层的局部误差来化简。

隐藏层的局部梯度：局部梯度 = 本层激活导数 × （后一层所有神经元的局部梯度，按连接权重加权求和）

$$\delta^{(2)}_j = \phi'(v^{(2)}_j) \cdot \sum_{k=1}^{n_3} \delta^{(3)}_k \cdot w^{(3)}_{kj}$$

对应的权重更新规则：

$$\Delta w^{(2)}_{ji} = \eta^{(2)} \cdot \delta^{(2)}_j \cdot x^{(1)}_{out,i}$$

$$w^{(2)}_{ji}(k+1) = w^{(2)}_{ji}(k) + \eta^{(2)} \cdot \delta^{(2)}_j \cdot x^{(1)}_{out,i}$$

可见何为反向?

输出层的 δ 用的是已知的期望值 d（直接可算）；隐藏层的 δ 用的是后一层已经算好的 δ。所以计算顺序必须从输出层开始，一层一层往回退。也正因为如此，只要激活函数可微，隐藏层有多少层都能用同一个递推式。

7.5 Unified Form and Two Passes

1. 两层的规则放在一起对照，权重更新统一式相同：

$$w^{(s)}_{ji}(k+1) = w^{(s)}_{ji}(k) + \eta^{(s)} \cdot \delta^{(s)}_j \cdot x^{(s-1)}_{out,i}$$

2. 区别：局部梯度的算法

输出层：局部梯度正比于期望输出与实际输出之差；

$$\delta^{(s)}_j = (d_{oj} - x^{(s)}_{out,j}) \cdot \phi'(v^{(s)}_j)$$

隐藏层：局部梯度也可以理解为正比于该神经元期望输出与实际输出之差，但隐藏层神经元的期望输出是未知的，必须用所有相连神经元的误差信号递归估计出来；

$$\delta^{(s)}_j = \phi'(v^{(s)}_j) \cdot \sum_k \delta^{(s+1)}_k \cdot w^{(s+1)}_{kj}$$

Tips: 该式可以推广到任意数量隐藏层的 MLP（Eqn. (4) can be generalized for MLP having an arbitrary number of hidden layers）

反向传播算法的计算（Back-propagation: 2 passes of computation）：

1. 前向传播（Forward pass）：为每个神经元计算函数信号（function signals）。
2. 反向传播（Backward pass）：从输出层开始，递归地为每个神经元计算 δ，从输出层一直回到第一个隐藏层；在每一层，按（5）式的 delta 规则相应地修改突触权重。

Tips：BP 能撑起深度学习的最直接的工程原因：无论你的多层网络有多少个隐藏层，都可以用完全相同的递归方式来更新权重堆层不需要新算法，只需要多循环几次。

7.6 Why Sigmoid Is Convenient

1. 激活函数的核心要求是可微（Differentiability）；常用的是 S 形函数，例如逻辑函数。简化后公式为：

$$\phi'(v) = a \cdot y \cdot (1-y)$$

此时，$\phi'(v)$在 y = 0.5 时取最大值，在 y = 0 或 1 时取最小值 0。

2. **梯度消失：**Sigmoid 饱和 → $\phi' \to 0$ → 梯度消失 → 前面的层学不动

如果某个神经元的输出已经饱和（y 接近 0 或 1），它的 $ $\phi'(v) \approx 0$，于是权重几乎不再更新，这个神经元学不动了。在多层网络里，这种衰减会逐层相乘，越靠前的层梯度越小，最后趋近于 0，这就是深层网络难以训练的经典原因。

3. 两个局部梯度都可以只用输出值表达：

情形	局部梯度表达式（Sigmoid 下）	含义
输出层神经元 j	$\delta_j = a \cdot y_{out,j} \cdot (1 - y_{out,j}) \cdot (d_j - y_{out,j})$	误差 × 导数，导数用输出值表示
隐藏层神经元 j	$\delta_j = a \cdot y_{out,j} \cdot (1 - y_{out,j}) \cdot \sum_k \delta_k \cdot w_{kj}$	后一层误差信号加权求和，再乘本层导数

由于权重改变量正比于导数 $ $\phi'(v)$，对 Sigmoid 激活函数来说，那些函数信号处于中间范围的神经元，权重被改变得最多；局部梯度 $\delta_j $可以在不需要显式知道激活函数的情况下算出来。

7.7 Epoch and Stopping Criteria

7.7.1 Epoch

An epoch = A complete presentation of the entire training set during the learning process. 一个 epoch 就是把整个训练集完整过一遍，学习过程按 epoch 为单位反复进行（epoch-by-epoch），直到满足收敛准则（criterion of convergence）。

7.7.2 三种停止准则

1. 每个 epoch 的均方误差的绝对变化率足够小（Absolute rate of change in mean squared error per epoch is sufficiently small）看的是**误差还在不在动**（一阶差分），哪怕误差还很大，只要不再下降就停；
2. 突触权重与偏置水平稳定下来（Synaptic weights and bias level stabilized）看的是**参数还动不动；**
3. 整个训练集上的均方误差小于某个最小值（Mean squared error over the entire training set is less than some minimum value）看的是**误差够不够小**（绝对水平）。

Tips: 三者可以独立使用，也可以组合。实践中常把1或3配一个**最大 epoch 数**做保险（否则可能永远不满足）。

7.8 The Full Algorithm

用流程图标出五个步骤：Initialization → Presentation of training examples → Forward computation → Backward computation → Is stopping criterion met? → No（回到呈现样本）/ Yes → End

1. **初始化（Initialization）**：初始化**权重与偏置**；
2. **呈现训练样本（Presentation of training examples）**：呈现一个 epoch 的训练样本；**对每个样本**执行前向与反向计算；
3. **前向计算（Forward computation）**：计算**误差信号**；
4. **反向计算（Backward computation）**：计算 **δ 并依据delta rule调整权重**；
5. **迭代（Iteration）**：若**未满足停止准则**，回到第 2、3、4 步。

7.9 Structure vs Decision Regions

Different Nonlinearly Separable Problems

结构 Structure	决策域类型 Types of Decision Regions	XOR 问题	网格状区域 Meshed Regions	最一般形状 Most General
单层 Single-Layer	**由超平面界定的半平面** Half Plane Bounded By Hyperplane	做不到	做不到	只得到半平面
两层 Two-Layer **隐藏层 + 输出层	**凸的开放或闭合区域** Convex Open Or Closed Regions	能做到	部分可以	受限
三层 Three-Layer	**任意形状（复杂度受节点数限制）** Arbitrary (Complexity Limited by No. of Nodes)	能做到	能做到	最一般

单层 → 半平面（一条直线 / 一个超平面）；

两层 → 凸的开放或闭合区域；

三层 → 任意形状，复杂程度只受节点数量限制。

需要注意：一个 MLP 在容量上可能足以表示某个函数（**三层能表示任意形状**），但梯度下降这个过程未必能找到那组权重，因为它在错误的盆地（basin）里下降到谷底就停下了。表达力是网络结构的问题，可学习性是优化过程的问题，两者不是一回事。

Part 8 Limitations of Back-Propagation

编号	缺陷 Deficiency	课件原文 Key Sentence	含义
①	**收敛慢 Slow convergence**	Learning often takes a long time to converge. Complex functions often need **hundreds or thousands of epochs**.	复杂函数要成百上千个 epoch，训练成本高
②	**黑箱 Black box**	It may provide a desired mapping between input and output vectors (x, y) but does not have the information of **why a particular x is mapped to a particular y**.	只能给出映射，学到的只是操作性参数，没有可解释的因果说明
③	**容易陷入局部极小 Local minimum**	Gradient descent approach only guarantees to reduce the total error to a **local minimum** （E may not be reduced to zero）.	误差曲面非凸、无唯一最小点，梯度下降可能困在局部极小；通常只能靠换结构、换初值、加随机扰动（如模拟退火）缓解。
④	**参数全靠经验 Empirical parameters**	The learning (accuracy, speed, and generalization) is highly dependent on a set of learning parameters… Most of them can only be determined **empirically (via experiments)**.	初始权重、学习率、层数与节点数没有理论解，只能试

Tips: 1. 慢、2. 说不清、3. 可能不是最优、4. 全靠调参。

前两条是工程代价，后两条是理论缺口。

第4条同时说明了一件事，神经网络的超参数（hyperparameter）这个概念就是从这儿来的：**权重与偏置是学出来的，层数 / 节点数 / 学习率 / 初始权重是人给的。**

8.1 Long Training Time

1. 现象：复杂函数常常需要成百上千个 epoch才收敛。

2. 为什么慢：每个 epoch 都要对全部训练样本做一次前向 + 反向传播计算；而每次权重更新只挪动很小的一步（步长由学习率 η 决定）。
3. 与第7部分呼应：第 7.3 / 7.4 节的更新式 $w ← w + η · δ · x $里，如果 $ η $太小 → 步数多； $η $太大 → 可能在极小点附近来回震荡甚至发散。这就是第4条参数全靠经验的具体表现。

Tips：没有在这里给出怎么加速（动量、自适应学习率等都不在本讲范围）。本讲只要求记住：收敛慢是 BP 的固有代价，不是实现 bug。

8.2 Black Box

The net is essentially a black box 这个网络本质上就是一个黑箱。

为什么它说不清：

1. 它能给出映射，但不给理由：It may provide a desired mapping between input and output vectors (x, y) but does not have the information of why a particular x is mapped to a particular y.
2. 所以给不出直观的（例如因果的）解释：It thus cannot provide an intuitive (e.g., causal) explanation for the computed result.
3. 根因在于隐藏单元与学到的权重没有语义：This is because the hidden units and the learned weights do not have a semantics.

What can be learned are operational parameters, not general or abstract knowledge of a domain.

能学到的只是操作性参数，而不是某个领域的通用或抽象知识。

operational parameters与abstract knowledge这组对立：一个训练好的网络里存的是一堆数字（权重与偏置），不是如果高频能量集中在 2 号频段，就说明是 Steve这类可以用语言说清楚的规则。这正是医疗、司法等需要高透明度决策的场景不能直接用 BP 网络的原因。

8.3 Local Minima

1. Gradient descent approach only **guarantees to reduce the total error to a local minimum**（E may not be reduced to zero）
2. **Cannot escape from the local minimum error state.**
3. **Not every function that is representable can be learned.**

可表示≠可学习，一个 MLP 在**容量上**可能足以表示某个函数，但**梯度下降这个过程**未必能找到那组权重，因为它在错误的盆地（basin）里下降到谷底就停下了。**表达力是网络结构的问题，可学习性是优化过程的问题，两者不是一回事。**

8.3.1 How bad

depends on the shape of the error surface. Too many alleys/wells will make it easy to be trapped in local minima.

窄巷与坑越多，越容易被困在局部极小

MLP with nonlinear activation functions have complex error surfaces (e.g., plateaus, long valleys etc.) with no single minimum.

含非线性激活函数的 MLP，其误差曲面是复杂的，**深度学习网络是非凸优化，分布山谷山脊，以山谷为例。在山谷震荡，不存在唯一的最小点。**

词汇 Vocabulary	中文	在误差曲面上的含义
plateau	平台区	一大片几乎平坦的地方，梯度≈0 → 走得很慢，像是卡住了
long valley	长谷	一个方向陡、另一个方向平的长条形凹槽 → 在最速下降里会沿谷壁来回弹
alley	窄巷	又窄又深的通道，很容易把人关在里面
well	井 / 坑	局部极小点所在的凹陷处
local minimum	局部极小	比周围都低，但不是全局最低
global minimum	全局最小	整个曲面上真正的最低点 —— 我们想要的

8.3.2 Three Remedies

1. 换网络规模试试：Try nets with different # of hidden layers and hidden units ——它们会带来不同的误差曲面，有些可能比另一些更好（they may lead to different error surfaces; some might be better than others）。
2. 换初始权重试试：Try different initial weights —— 相当于在曲面上换一个出发点（different starting points on the surface）。
3. 用随机扰动强行跳出：Forced escape from local minima by random perturbation（例如 simulated annealing）。

simulated annealing（模拟退火）正是 Week 2 讲过的允许以一定概率接受更差解的方法，Week 3 的进化计算也是同一思路（靠种群与随机变异维持探索）。

Tips：所以本讲最后给出的跳出局部极小的出路，其实是用的之前学的，**不依赖梯度的迭代搜索，梯度负责快，随机搜索负责别被困死。**

8.4 No Theoretical Quality Guarantee

There is no theoretically well-founded way to assess the quality of BP learning:

What is the confidence level one can have for a trained BP net, with the final E (which may or may not be close to zero)?

没有理论上站得住脚的办法去评价BP 学出来的东西有多好。训练完误差 E 可能很小、也可能不接近 0，而你对这个网络的信心该有多大，无法从理论上回答。

问题出在学习参数上：

1. 依赖哪些参数：Initial weights（初始权重）、learning rate（学习率）、# of hidden layers and # of units（隐藏层层数与节点数）…
2. 关键限制：Most of them can only be determined empirically (via experiments). —— 其中绝大多数只能靠实验（经验）来确定。

这一条与第9部分的衔接关系：既然训完不知道好不好，那就必须在训练集之外另拿一份数据去看，于是才有了下一部分的 validation error（验证误差）、underfitting / overfitting 与 learning curves。

Part 9 Overfitting and Underfitting

9.1 Three Regimes of Fit

状态 State	别名 Alias	课件原文 Original Wording	训练误差	验证误差
Underfitting 欠拟合	**High Bias** 高偏差	The model is too simple — **cannot capture the underlying pattern**. Both training and validation errors are high.	高	高（且与训练误差接近）
Good Fit 恰当拟合	**Balanced** 平衡	The model captures the true pattern **without memorizing noise**. Both errors are low and close.	低	低（且与训练误差接近）
Overfitting 过拟合	**High Variance** 高方差	The model is too complex — **memorizes training data, including noise. Low training error but high validation error.**	低	**高**

Tips:

欠拟合 = 都高（训练也学不会）;

恰当拟合 = 都低且接近（学会了规律，没背住噪声）;

过拟合 = 训练低、验证高（把噪声也记下来了，换新数据就露馅）。

9.2 The Bias-Variance Tradeoff

模型复杂度与偏差：讨论模型该多大。

图例 Legend	中文	曲线形状（看图）	随复杂度怎么变
$Bias^2 $(underfitting)	偏差平方（对应欠拟合）	红色，**单调下降**，由约 1.0 降到接近 0	复杂度 ↑ → 偏差 ↓
Variance (overfitting)	方差（对应过拟合）	蓝色，**单调上升**，由接近 0 升到约 5	复杂度 ↑ → 方差 ↑
Total Error	总误差	绿色，**U 形**，最低处约 0.97	先降后升，中间有个最低点
Irreducible Error	不可约误差	灰色虚线，**水平**，约 0.05	与复杂度无关，降不下去

Bias：来自假设过度简化的误差。

Variance：来自对训练数据中微小波动过度敏感的误差。

Goal：找到总误差最小的那个。

★**总误差曲线的最低点 **Optimal Complexity：**落在 Model Complexity ≈ 1.7 的位置，再往左走偏差太大（欠拟合），再往右走方差太大（过拟合）。**

Tips:

1. 三条曲线的关系：总误差 ≈ 偏差² + 方差 + 不可约误差。红色（偏差²）往下走、蓝色（方差）往上走，两者相加再加一条水平的灰线，得到的正是那条绿色的 U 形曲线。这就是权衡（tradeoff）这个词的来源：动复杂度，一边降另一边必然升，只能找平衡点。
2. 注意课件把偏差写成了 Bias^2（偏差的平方），不是 Bias。这不是笔误，标准分解式里偏差那一项本来就是平方项。所以图例读作bias squared，中文说偏差平方。

9.3 Learning Curves as a Diagnostic

讨论训练集大小与误差（训练误差，验证误差）：讨论数据量该多少。

子图 Panel	两条曲线怎么走	诊断结论
Underfitting 欠拟合	红线从约 **0.38** 缓降到约 **0.36**（几乎水平）；蓝线从约 **0.47** 降到约 **0.40**。**两条都高，且彼此贴得很近。**	Both training and validation errors are high and close together. Adding more data does not help-need a more complex model. **加更多数据没用，需要更复杂的模型。**
Good Fit 恰当拟合	红线从约 **0.18** 降到约 **0.09**；蓝线从约 **0.28** 降到约 **0.13**。**两条都低，且随数据量增加而收敛到一起。**	Both errors are low and converge. The model generalizes well to unseen data. **模型对未见数据泛化得好。**
Overfitting 过拟合	红线从约 **0.06** 降到约 **0.015**（非常低）；蓝线只从约 **0.48** 降到约 **0.38**。**训练低、验证高，中间夹着一大片粉色区域。**	Large gap between training (low) and validation (high) error. More training data can help reduce the generalization gap. **加更多训练数据有助于缩小泛化差距。**

Tips：Generalization gap（泛化差距），**它就是训练误差曲线与验证误差曲线之间的面积**。gap 越大 = 过拟合越严重。**过拟合的量化指标就是这个gap有多宽。**

诊断口诀（先用学习曲线判断是哪一种病，再用偏差-方差权衡决定该往哪个方向调）：

1. **两条都高、且挤在一起 → 欠拟合。**
2. **两条都低、且收敛靠近 → 拟合恰当。**
3. **一条低一条高、中间 gap 很大 → 过拟合。**

9.4 Five Remedies for Overfitting and Underfitting

9.4.1 Five Remedies for Overfitting

1. 正则化 Regularization（L1/L2 penalty）：Adds penalty for large weights.

改目标函数：对大权重加惩罚项，逼网络别把权重推到极端值。

2. 随机失活 Dropout：Randomly deactivate neurons during training.

改训练过程：训练时随机关掉一部分神经元，让网络不能依赖某个特定神经元的组合。

3. 早停 Early Stopping：Stop when validation error starts increasing.

改训练过程：验证误差一开始上升就停止训练（对应第 7.7 节的停止准则，但这里用的是验证误差而不是训练误差）。

4. 数据增强 Data Augmentation：Increase training data diversity.

改数据/输入分布：增加训练数据的多样性（对应第 9.3 节加数据能缩小 gap）。

5. 批归一化 Batch Normalization：Normalize layer inputs to stabilize training.

改数据/输入分布：把每层的输入归一化，稳定训练过程。

9.4.2 Five Remedies for Underfitting

1. 增加模型复杂度 Increase Model Complexity：Add more layers or neurons.

加层或加节点。

2. 训练更久 Train Longer：Increase number of epochs.

加 epoch 数。

3. 减弱正则化 Reduce Regularization：Lower L1/L2 penalty or dropout rate.

把 L1/L2 惩罚或 dropout 率调小。

4. 更好的特征工程 Better Feature Engineering：Add more informative features.

补更有信息量的特征。

5. 换更强的结构 Use a Different Architecture：Try a more powerful model class (e.g., deeper network).

换更深的网络等。

Tips：

1. 过拟合要**减容量**（正则化、Dropout、早停）；欠拟合要**加容量**（加层加节点、训练更久、减弱正则化）。**唯一的例外是第4条的两侧不对称**：过拟合那边的数据增强是**造更多数据**，欠拟合这边的特征工程是**把现有数据表达得更好。**一个加量，一个提质，因为两种缺的东西不一样。
2. **减弱正则化这条容易被忽略**：如果你发现模型欠拟合，要先检查一下是不是**正则化开得太狠**、或者**dropout 率设得太高，**这不一定是模型容量的问题，也可能是把它捆得太紧了。
3. 掌握：这是欠拟合还是过拟合、为什么、下一步该加数据还是改模型。

Part 10 Case Study: Voice Recognition

本讲概念 Concept	在这个案例里对应什么
神经元模型与激活函数（Part 2 - 3）	60 个频段值进入输入层；每个隐藏神经元做加权求和 + 激活，输出层同理
MLP 结构（Part 6）	60 - 6 - 2 的前馈网络，一个隐藏层；输入层不计入层数 → 两层网络
前向传播（第 6.6 节）	从 60 维频谱算出 2 个输出值（第 10.3 节的 0.43 / 0.26）
误差函数（第 7.1 节）	输出与 one-hot 目标之差（第 10.4 节的四个误差）
反向传播（第 7.3 - 7.5 节）	按误差反推，调整 60×6（输入→隐藏）+ 6×2（隐藏→输出）共 372 个权重与 8 个偏置
epoch 与停止准则（第 7.7 节）	走完所有训练对= 一个 epoch；重复多次= 多个 epoch
收敛（第 8.2 节）	输出从 (0.43, 0.26) 收敛到 (0.01, 0.99)
过拟合 / 泛化（Part 9）	已知样本与新样本都是 100% → 属于 Good Fit，未见明显过拟合

权重个数怎么来的：输入层 60 → 隐藏层 6，所以输入到隐藏的权重有 60 × 6 = 360 个；隐藏层 6 → 输出层 2，所以隐藏到输出的权重有 6 × 2 = 12 个；合计 372 个权重。偏置每个神经元一个：6 + 2 = 8 个。逐层相乘再相加的算法在第 6.6 节里有对应，也是第7部分推导里 $w^{(s)}_{ji}$ 的两个下标（i 遍历前一层、j 遍历本层）的由来。

Part11 Exercise

11.1 Exercise1: Regression

11.2 Exercise2: Gradient Descent

Lecture 5

Part 0 Introduction

周 Week	主题 Topic	核心内容 Key Content
Week 1	An Introduction AI 导论	AI 定义 / 机器学习三大范式 / ANI-AGI-ASI / 学术技能
Week 2	Searching & Optimization 搜索与优化	BFS / DFS / Hill Climbing / Best-First / A* / SA / GA
Week 3	Evolutionary Computation 进化计算	EA 总览 / PSO / DE / MA / MOEA / ETO / NAS
Week 4	Artificial Neural Network 人工神经网络	神经元模型 / 激活函数 / 感知机 / MLP / 反向传播 / 过拟合引言
Week 5	**Deep Learning & Applications 深度学习与应用**	**CNN 卷积网络 / RNN-LSTM 序列网络 / GNN 图网络 / 生成模型与扩散模型**

本周回答了：同样一套反向传播，为什么堆得更深反而更强？面对图像、序列、图这三种完全不同的数据，网络结构要怎么改？

学习路线：深度学习总览 → CNN → 序列神经网络 → 图神经网络 → 生成模型与扩散模型 → 过拟合与欠拟合。

总结

被装在四种不同的数据接口上的同一个反向传播引擎。

区别在于接口的类型：图像是规则网格，用滑动窗口（卷积）；序列有先后，用递归链（循环）；图没有顺序，用邻居聚合（消息传递）；要造新数据，换成对分布的建模（自编码 / 对抗 / 扩散）。

共同点：猜参数 → 看误差 → 反向传播改参数

本讲的四条支线	解决什么	共同的技术手段
CNN	图像：把局部性 + 平移不变注入模型	权值共享；逐步加深；残差连接
RNN / LSTM	序列：处理变长输入与长程依赖	细胞状态的加性记忆通道
GNN / GCN	图：在无固定顺序的邻居集合上做聚合	邻居消息传递；置换不变
生成模型	造新数据：学数据分布本身	潜在空间正则；对抗博弈；迭代去噪

Part 1 From NN to Deep Learning

1.1 Three Formulas and Levels

1. **单个神经元 Model of a neuron**：每个输入被一个权重缩放、与一个偏置求和，再过一层激活函数。

$v = \Sigma_{i=1}^{m} w_i x_i + b, \quad y = \phi(v)$

2. **感知机 Perceptron**：一个线性分类器。只有两类**线性可分（linearly separable）**时才收敛。

$w_{n+1} = w_n + \eta(d_n - y_n)x_n$

3. **MLP 与反向传播 MLP and backpropagation**：隐藏层去掉了线性限制；输出神经元与隐藏神经元使用**不同的局部梯度**。

$w_{n+1} = w_n - \eta \cdot \partial E/\partial w$

Lecture4 得这些概念分别代表了单元（一个神经元）→ 能力边界（单层只能线性分类）→ 突破办法（多层 + 反向传播），本讲是这些的延伸，讲解层与层的联系。

1.2 Deep Learning

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction.

多个处理层 multiple processing layers 即深，层的数量，不是参数的绝对规模。

学习表示 learn representations 即学，不再是人工设计的，而是从数据里生成的。神经网络是**实现**深度学习的方式之一。

多个抽象层次 multiple levels of abstraction 即层次的递进：边缘—鼻子-脸。

1.3 Two Paradigms

两者的区别在特征提取器能否被训练：上面那张图的红色框圈住的是 hand-crafted Feature Extractor，下面那张图的红色框圈住的是 Trainable Feature Extractor + Trainable Classifier 这个整体。

环节 Stage	传统模型（20 世纪 50 年代起）	深度学习模型
特征来源	Fixed/engineered feature 人工设计、固定不变的特征提取器	Trainable feature extractor **可训练**的特征提取器
分类器	"Simple" Trainable Classifier 可训练的**简单**分类器（并非不可训练）	Trainable Classifier 可训练的分类器
特征端代表方法	HoG（方向梯度直方图）、SIFT（尺度不变特征变换）	不独立存在
分类端代表方法	两段式：人做特征 → 机器学分类 Linear Regression、SVM、Decision Tree、Random Forest	一体式：与特征端**连成一个整体**一起训练
代表网络	无	AlexNet、VGGNet、Inception、ResNet、ViT

1.4 Why is it called Deep?

It is deep if more than one stage of non-linear feature transformation.

只要有**多于一个阶段的非线性特征变换**就算深度。深的含义是表示被逐级复合地变换，与实现上是否有隐藏层、隐藏层有几个没有对应关系。

1.5 The ImageNet Trend

1.6 Why Deep Learning Works

1. 表示能力 Representation Power

已知结论：a shallow network can approximate any function. 一个浅层网络可以逼近任意函数。

代价：this requires many hidden units. 需要极多的隐藏单元。

结论：Depth gives an exponential advantage. 深度带来指数级优势。同样表达力下，深层网络的参数需求远小于浅层。

2. 可优化性 Tractability of Optimization

存在的问题：Lots of local minima. 误差曲面上布满局部极小（Lecture 4 第 8.3.1 ）。

实践：optimization still works. 优化仍然能用。

可行的解决方法：with tricks like variants of adagrad, using relu / leaky relu, dropout, batch norm⋯ —— 自适应学习率（Adagrad 的变体）、ReLU / Leaky ReLU 激活、Dropout、批归一化。

3. 泛化能力 Generalization Power

存在的问题：DNN can "memorize" a large training set (huge capacity) . 深度网络容量巨大，完全有能力把训练集背下来。

实践：Yet they don't overfit (when training data and model size is compatible). 只要训练数据量与模型规模相匹配，它并不因此过拟合。

可行的解决方法：they don't just store the examples. 它学到的不是样本本身。

Tips：表示能力承认浅层也能逼近、但代价指数级；可优化性承认满地局部极小、但工程技巧能压住；泛化承认容量足以背训练集、但实测没有背。

1.7 A Brief History

Part 2 CNN I 卷积神经网络（卷积层与池化层）

2.1 CNN Definition

Convolutional Neural Networks learn a complex representation of visual data using vast amounts of data. They are inspired by the human visual system and learn multiple layers of transformations, which are applied to the top of each other to extract a progressively more sophisticated representation of the input.

CNN 用**大量数据**学习视觉数据的复杂表示；它受**人类视觉系统（局部感受野 + 逐级抽象**）启发，学习**多层变换（每一层都在对上一层的输出做运算）**，这些**变换逐层叠加（至少一个非线性函数）**，从而提取出越来越精细的输入表示

2.2 Convolution Definition 卷积

1. **数学定义：**Convolution is one of the main building blocks of a CNN. The term convolution refers to the mathematical combination of two functions to produce a third function. It merges two sets of information. 卷积是两个函数的数学组合，产生第三个函数；它把两组信息合并。

2. **操作定义**：Convolution is essentially sliding a filter over the input. 卷积本质上就是把一个滤波器/卷积核在输入上滑动。
3. **直觉定义**：A convolution can be thought as looking at a function's surroundings to make better/accurate predictions of its outcome. 卷积可以理解为看一个函数的周围环境，以便对它的结果做出更准的预测。这就意味着，卷积的输出在某个位置上的值，取决于**输入在该位置邻域内的取值**，而不是该位置单独的取值。

2.3 Convolutional Kernels 卷积核

英文术语	中文	课件原句
filter / kernel	滤波器 / 卷积核	these terms are used interchangeably **两个词指的是同一个东西**，可互换使用
feature map	特征图	卷积在输入上滑动后**产生**的输出
sliding the filter over the input	滑动	At every location, a matrix multiplication is performed and sums the result onto the feature map.

Examples:

1. 左图用全 1 的 2×2 卷积核对一张 4×4 的输入做卷积，步长（stride）为 2，得到一张 2×2 的特征图。
2. 右图真实输入图像得到特征图：用的核是 [[−1,−1,−1], [−1,8,−1], [−1,−1,−1]]。特点是中心权重远大于四周、且符号相反，它输出的特征图上只剩边缘，平坦区域被减成接近 0。**卷积核里装什么，特征图上就留下什么。**

2.4 Pooling Layer 池化层

功能角度：The function of pooling is to continuously reduce the dimensionality to reduce the number of parameters and computation in the network. 池化持续降低维度，以减少网络中的参数量与计算量。

动作：The function of pooling is the process of merging. So it's basically for the purpose of reducing the size of the data. 池化是合并的过程，目的是缩小数据尺寸。

Example：input image → convolutional layer → pooling layer → convolutional layer → pooling layer → fully-connected layer，整条链被框在一个标着 CNN 的方框里，最右端输出 car（predicted class）。

Tips：最后一级是 fully-connected layer（全连接层）而不是卷积层：特征提取完成之后，仍要靠全连接层把特征映射到类别。

2.4.1 Max and Average Pooling

Example:

同一张 4×4 输入，2×2 窗口、步长 2，并排演示两种池化。

窗口位置	四个数 Four values	最大池化 Max pooling	平均池化 Average pooling
左上	5, 3, 0, 2	5（最大值）	(5+3+0+2)/4 = 2.5
右上	3, 1, 8, 5	8（最大值）	(3+1+8+5)/4 = 4.25
左下	1, 4, 0, 9	9（最大值）	(1+4+0+9)/4 = 3.5
右下	4, 2, 2, 7	7（最大值）	(4+2+2+7)/4 = 3.75

最大池化输出 [[5, 8], [9, 7]]，平均池化输出 [[2.5, 4.25], [3.5, 3.75]]

Tips：

1. 池化层没有参数：它不是卷积，卷积核里的参数是学出来的，而池化的取最大或取平均是固定的运算，训练过程中不更新任何值。所以池化能降低维度和计算量，却不需要额外的学习容量。
2. 两种池化的分工：最大池化保留窗口内最强的响应，对某个特征在不在这类判断更敏感，也因此对微小位移更宽容；平均池化保留窗口内的整体强度，对这个区域大概有多活跃更敏感。实际网络里最大池化更常见。

2.5 How CNN Works

2.5.1 Two Kinds of Layers

卷积层 Convolution layer：is a feature detector that automatically learns to filter out not needed information from an input by using convolution kernel. 一个特征检测器，靠卷积核自动学出该过滤掉输入里的哪些信息。

池化层 Pooling layers：compute the max or average value of a particular feature over a region of the input data (downsizing of input images). Also, it helps to detect objects in some unusual places and reduces memory size. 在输入的一个区域上求某个特征的最大或平均值（即缩小输入图像）；还有助于检测出现在异常位置的物体，并减少内存占用。

2.5.2 What Each Layer Learns

某一层的卷积核的响应模式：可视化（**边缘** → **纹理** → **部件** → **物体**）

层 Layer	学到的内容	含义
Conv 1	Edge+Blob（边缘与斑点）	最底层：只认边缘朝向与色块
Conv 3	Texture（纹理）	由边缘组合出的重复性图案
Conv 5	Object Parts（物体部件）	开始出现眼睛、轮子、鼻子这类可命名部件
Fc8	Object Classes（物体类别）	最顶层：整类物体（ship、sock、dining table、grocery store）

Part 3 CNN II 四个模型

3.1 Four Milestone Models

eNet 证明卷积可行→ AlexNet 证明卷积能规模化→ ResNet 证明卷积能叠很深→ ViT 证明卷积是不是必须的。前三个模型都在**改进卷积的用法**，第四个模型**取消了卷积**。

对比项	LeNet-5 (1998)	AlexNet (2012)	ResNet (2015)	ViT (2020)
代表成绩	手写数字识别成功落地	ILSVRC 2012 Top-5 错误率 15.3%	ILSVRC 2015 Top-5 错误率 3.57%	JFT-300M 上预训练后超越 CNN
核心创新	卷积 + 池化的基本结构；**权值共享 + 局部连接**	**ReLU + GPU + Dropout + 数据增强**	**残差（跳跃）连接**，学 F(x) = H(x) − x	**图像切成 patch + 纯注意力**，不用卷积
解决的问题	MLP 直连像素导致**参数爆炸、忽略空间结构**	把 CNN **放大到大数据集**（ImageNet 120 万张）	深层网络的**退化问题**与梯度消失/爆炸	CNN 的**归纳偏置**是否必要

对比项	LeNet-5 (1998)	AlexNet (2012)	ResNet (2015)	ViT (2020)
遗留局限	浅、sigmoid/tanh、小数据集	层数仍有限	未提；后由注意力架构接续	需要**超大数据集**，小数据上效率低

3.1.1 LeNet-5 第一个成功的卷积网络

架构：经过 Conv、Pool、Conv、Pool 、FC 、 FC 、Output 两次卷积 + 池化，两次全连接。

输入输出：Input: 32x32 grayscale; Output: 10 digit classes (0-9)

应用：handwritten digit recognition (MNIST, postal codes) 手写数字识别，如邮政编码。

解决的问题：MLPs on raw pixels had too many parameters, ignored spatial structure. 在原始像素上做 MLP 参数量太大，且完全忽略空间结构。

关键洞察：weight sharing + local connectivity drastically reduce parameters. 权值共享 + 局部连接大幅减少参数。

局限：shallow, sigmoid/tanh activations, small datasets. 网络浅、用 sigmoid/tanh 激活、只在小数据集上验证过。

3.1.2 AlexNet 把 CNN 推到大数据集上（进入GPU时代）

成绩：Top-5 error: 15.3% vs. 26.2% (second place, hand-crafted features) .

解决的问题：scaling CNNs to large datasets (ImageNet, 1.2M images) 把 CNN 扩展到大数据集。

四项创新：

1. **ReLU 激活**：ReLU activation (faster training than sigmoid/tanh) 训练比sigmoid/tanh更快。
2. **GPU 训练**：GPU training (2 GPUs, major speedup) 用 2 块 GPU，带来大幅加速。
3. **Dropout 正则化**：Dropout for regularization 用随机失活抑制过拟合。
4. **数据增强**：Data augmentation (cropping, flipping, color jitter) **裁剪、翻转、颜色抖动**。

3.1.3 ResNet 用残差连接把网络做深

成绩：3.57% top-5 error（对比 AlexNet 的 15.3%）。

解决的问题：vanishing/exploding gradients in very deep networks. 极深网络的梯度消失/爆炸。

3.1.3.4 The Degradation Problem 退化问题

现象：deeper networks are harder to train — degradation problem (not overfitting) 更深的网络更难训练，这叫退化问题，而且它不是过拟合。

证据：Adding layers increases training error, contrary to expectation. 加深层数反而让训练误差上升。（过拟合的表现是训练误差下降、验证误差上升）

成因：vanishing/exploding gradients 梯度在深层传播过程中消失或爆炸。

解法：学「残差」而不是学「映射」

做法：Solution: skip (residual) connections 加**跳跃连接（残差连接）**。

改写的目标：Learn residual F(x) = H(x) - x, instead of H(x) directly 不去直接学目标映射 H(x)，而是学**残差 F(x) = H(x) − x**。

更容易学习的原因：Identity mapping is easy to learn: just set F(x) = 0 **恒等映射**：F(x) = 0 。

效果：Enabled networks of 50, 101, even 152 layers 使更深的网络成为可能。

Tips: Lecture 4 讲反向传播时提到过深层网络梯度会消失。ResNet 作为**工程解法，用跳跃连接给梯度提供了一条不经过任何非线性层**的直通路径，即x，使梯度可以直接回传到浅层。

3.1.4 Vision Transformer 把 Transformer 直接用在图像上

apply the Transformer architecture (from NLP) directly to images

把来自 NLP 的 Transformer 架构直接用到图像上。

做法：

1. Split image into fixed-size patches (e.g., 16x16), linearly embed, add position 把图像切成固定大小的小块（如 16×16），做线性嵌入，再加上位置信息。
2. Feed patch embeddings into a standard Transformer encoder 把小块嵌入送进标准 Transformer 编码器。
3. 分类头与预训练：[CLS] token for classification; pretrain on large datasets (JFT-300M) 用 [CLS] 标记做分类；在大规模数据集（JFT-300M）上预训练。

**解决的问题: **CNNs have inductive bias (locality, translation equivariance) CNN 自带归纳偏置：局部性与平移等变。

Tips: CNN 的归纳偏置（局部性、平移等变）在数据有限时是巨大的优势，它注入了自然图像的规律。但数据量足够大时，它限制了模型去发现人类没预设过的模式。

结论：ViT shows that with enough data, a general attention model beats CNNs 数据足够时，让模型自己学，不要预设。

局限：needs huge datasets; less efficient on small data 需要超大数据集，在小数据上效率低。

Part 4 序列神经网络 Sequential Neural Networks

4.1 Sequence Transduction Models

Sequence transduction models refer to a class of models that aim to transform one sequence of data into another sequence. Such as Google's autocomplete feature predicts the rest of the words a user is typing.

序列转换模型指的是一类**把一条数据序列变换成另一条序列**的模型，如谷歌补全词条。

优势：序列到序列的优势是输入和输出的长度可以变化。

4.2 循环神经网络 Recurrent Neural Network

基本形式：input sequence is transformed into output sequence in a one-to-one fashion 输入序列以**一对一（步进节拍：时间步 t 进一个 token、同一时间步出一个 token）**的方式变换为输出序列（每一步进一个、出一个）。

设计目标：Design an architecture that possesses the ability to generate output sequences of variable length, while incorporating contextual relevance and a memory mechanism that leverages prior inputs to influence the current input and output.

1. 能产生任意长度的输出序列；
2. 兼顾上下文相关性；
3. 有一套记忆机制，让先前的输入能影响当前的输入与输出。

Tips: RNN是一对一，而Transformer 也是一步进一个位置、出一个位置，但它的上下文不再靠一条递归链传递，而是靠注意力一次性看全序列。

4.3 RNN 的能力边界 RNN's Breaks Down

A small example where RNN can work perfectly: Prediction of the last word in the sentence: "The clouds are in the sky" 一个 RNN 能完美工作的例子 预测句子的最后一个词。

RNN can't handle situation where the gap between the relevant information and the point where it is needed is very large. (Long-term dependency)

当**相关信息**与**需要用到它的位置**之间的**间隔很大**时，RNN 处理不了。这就是**长程依赖（long-term dependency）**问题。

1. Vanishing and exploding gradients 梯度消失与梯度爆炸。递归链上每一步都要乘一次权重矩阵，经过 t 步以后梯度里就带着权重矩阵的 t 次幂，若**矩阵特征值小于 1 就消失，大于 1 就爆炸。**
2. Limited memory capacity 记忆容量有限。
3. Computational complexity 计算复杂度（逐步递归，无法并行）。

4.4 LSTM

为了解决长期依赖：LSTMs - are a special kind of RNN, capable of learning long-term dependencies. LSTM 是一种**特殊的 RNN**，**能够学习长程依赖**。

结构：All RNNs have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.

所有 RNN 都是一条由重复模块组成的链。标准 RNN 的重复模块极简，例如只有一层 tanh；

LSTM has the same chain-like structure except for the repeating module. LSTM 的**链式结构完全相同**，区别只在重复模块内部。

对照项	标准 RNN 的重复模块	LSTM 的重复模块
结构复杂度	极简单，例如单层 tanh	明显复杂：多个门控与逐点乘法
输入输出	$x_t $进、 h_t 出	$x_t 进，h_t $与 C_t**（细胞状态）**同时维护

左边（标准 RNN）每个方块标 A，方块里只有一条带 tanh 的支路；

右边（LSTM）每个方块标 A，内部有 **四个小的 σ / tanh 单元**以及多条交叉的加减与乘法节点。

4.4.1 How to Work

The core idea behind LSTMs is the cell state, which acts as an information highway running through the entire sequence.

LSTM 的核心是细胞状态（cell state），它像一条贯穿整条序列的信息高速公路。

4.4.2 Three Gates

The cell state is updated and controlled by three main structures called gates: the input gate, the forget gate, and the output gate. These gates regulate the flow of information into, out of, and within the cell state, allowing LSTMs to selectively retain or forget information as needed.

细胞状态由三个称为「门」的结构更新与控制 —— 输入门、遗忘门、输出门。这些门**调节信息流入、流出以及细胞状态内部**的流动，使 LSTM 能按需**选择性地保留或遗忘**信息。

门 Gate	作用	直觉
input gate	把新信息**写进**细胞状态	决定「这一时刻要不要记住新东西」
forget gate	从细胞状态里**丢掉**信息	决定「过去记下的东西还要留多少」
output gate	把细胞状态里的信息**读出**来给输出	决定「这一刻要把记忆的哪一部分说出来」

门的实现方式：

Gates are generally composed out of a sigmoid neural net layer and a pointwise multiplication operation 门一般由**一个 sigmoid 神经网络层 + 一次逐点乘法**组成。

sigmoid 输出在 (0,1)，乘上去就等价于**保留多少比例，**输出 0 是丢，输出 1 是留。

4.4.3 The Four Equations

LSTM 的四个核心方程。 $C_t $是细胞状态， $h_t $是隐藏状态， $x_t $是当前输入。

量	方程	读法
遗忘门	$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$	把**上一时刻的细胞状态、上一时刻的隐藏状态、当前输入**拼在一起，过 σ得到 0 - 1 之间的系数
输入门	$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$	同一组输入，另一套权重，得到「写入多少」的系数
输出门	$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$	注意这里用的是**更新后的** C_t，而不是 C_{t-1}
细胞状态	$C_t = f_t * C_{t-1} + (1 - f_t) * C_t$	旧记忆按 $f_t $的比例保留，新候选记忆 $C_t $按 $1 - f_t $的比例写入

4.5 SNN 问题What Issue Are They Facing?

短板	原文	中文
训练慢	Slow to train (inefficient / not parallelizable) This sequential dependency limits the ability to parallelize computations, as each step must wait for the completion of the previous step.	训练慢、效率低、**无法并行化** 这种**顺序依赖**限制了并行计算能力，**每一步都必须等上一步算完**
梯度问题	Suffer from exploding or vanishing gradients	仍会遭遇梯度爆炸或梯度消失
长程依赖仍受限	Cannot handle very long-term dependencies In Seq-Seq models, decoder only accesses the last hidden state. Early information in sentences can be lost.	仍然处理不了**极长**的依赖 在**序列到序列（Seq-Seq）模型里，解码器只能拿到最后一个隐藏状态**，句子**早期的信息可能丢失**

Tips: 主要问题是**并行性**（能不能同时算）和**信息通道宽度**（所有历史被压进一个固定长度的隐藏状态）。所以设计目标是 **既要能并行，又要能直接看到序列里的任意位置，而不是只透过一个固定大小的窗口。**

Part 5 GNNs I: From Graphs to GNNs

图是什么 → 为什么不能直接用 DNN → GNN 怎么做

5.1 Definition of Graph

A graph represents the relations (edges) between a collection of entities (nodes).

图表示**一组实体（节点）之间的关系（边）**

属性类型	英文原文	中文	例子
节点属性	V Vertex (or node) attributes	顶点 / 节点属性	e.g., node identity, number of neighbors 节点的身份、邻居数量
边属性	E Edge (or link) attributes and directions	边 / 链接属性与方向	e.g., edge identity, edge weight 边的身份、边的权重
全局属性	U Global (or master node) attributes	全局 / 主节点属性	e.g., number of nodes, longest path 节点总数、最长路径

属性分成 V / E / U： 后续所有 GNN 模型的输入清单，任何一个 GNN 层要做的事都可以写成拿 V、E、U 的一部分作为输入，输出更新后的 V、E、U。

5.2 图的应用 Graphs Application

5.2.1 Images and Text as Graphs

图像和文本本身就是规则结构，把它们看成图只是换了一种表示。真正的图数据是下一组那些天生就没有规则网格结构的数据，图像和文本的图结构是规则、稠密、平移不变的，不需要通用的图方法。

5.2.2 Graph-Valued Data in the Wild

Molecules as graphs **分子**：原子是节点，化学键是边。

Social network as graphs **社交网络**：人是节点，关注或互动是边。

Brain network as graphs **脑网络**：脑区是节点，功能或结构连接是边。

Graph data is everywhere around the world !

为什么要专门为图设计网络的动机：如果图数据真的无处不在，那把图塞进只能用于规则网格的 CNN 就会损失大量结构信息。

5.3 图解决的问题 Solving Problems over Graphs

图可以解决的五类问题（6.3.2）

任务 Task	定义	中文
Node Classification	Classifying individual nodes.	**节点分类**：对单个节点分类，对应 $softmax(z_n) $，**逐节点**过 softmax

任务 Task	定义	中文
Graph Classification	Classifying entire graphs.	**图分类**：对整个图分类，对应 $softmax(\Sigma_n z_n)$，**先对所有节点求和**再过 softmax
Node Clustering	Grouping together similar nodes based on connectivity.	**节点聚类**：按**连接关系**把相似节点归为一组
Link Prediction	Predicting missing links.	**链接预测**：预测缺失的边
Influence Maximization	Identifying influential nodes.	**影响力最大化**：找出有影响力的节点

5.4 图数据上用深度神经网络的挑战Challenges of Using Graphs in DNN

为什么不能使用DNN？图的特征对应CNN的前提不成立。

1. **Graph-Structured Data（图结构的数据）**：Neural networks have been traditionally used to operate on fixed-size and/or regular-structured inputs (such as sentences, images and video). 神经网络传统上都用来处理**固定尺寸、规则结构**的输入，例如句子、图像和视频。**图两者都不是。**
2. **Lack of Consistent Structure（缺乏一致结构）**：Graphs are extremely flexible mathematical models; but this means they lack consistent structure across instances. 图是**极其灵活**的数学模型，但这也意味着**不同样本之间结构不一致**：每个图的节点数、连边数、度数分布都可能不同。
3. **Node-Order Equivariance（节点顺序等变）**：Graphs often have no inherent ordering present amongst the nodes. 图的节点之间往往没有固有的顺序。任何 GNN 的输出不能依赖于节点的编号顺序。把输入节点重排一遍，输出的对应关系应该跟着重排，而数值不变。所以Invariance to permutations（置换不变）

Tips：CNN 靠平移不变来适应图像的位移，GNN 靠置换不变来适应图的重新编号。

5.5 图神经网络 What is a Graph Neural Network?

Define:

A GNN is an optimizable transformation on all attributes of the graph (nodes, edges, global-context, connectivity) that preserves graph symmetries.

GNN 是作用在图的全部属性（节点、边、全局上下文、连接关系）上的可优化变换，并且保持图的对称性。

Main Idea:

Pass messages between nodes to refine node (and possibly edge) representation.

在节点之间传递消息，以此来精修节点（可能还有边）的表示。

1. 节点V、边E、全局上下文U、连接关系这四个属性全都要处理。
2. 可优化变换：GNN 不是一套固定规则，不是人为设定，它的参数是通过训练学的（与 CNN、MLP 同源）。
3. 保持图的对称性：节点换编号后结果不能变，这就是要保持的对称性。
4. 传递消息：这是 GNN 的工作方式，也是消息传递网络（Message Passing）每个节点从邻居那里收消息，更新自己的表示，然后进入下一层。

Part 6 GNNs II: Graph Convolution

6.1 单个像素的更新 Update for a Single Pixel

卷积的关键两个步骤：

1. Transform messages individually 每个邻居传来的信息 $ h_i $各自乘上一套权重 W_i，得到 $W_i h_i$。

Tips：在CNN中，每个位置一套独立权重，卷积核可以给每个位置配一套固定的权重 。但图的节点没有固定范围，一个节点的邻居不能确定数量和先后，所以卷积无法在图上使用。

2. Add everything up 把结果全部加起来 $ Σ_i W_i h_i$。

$h_i \in \mathbb{R}^F$ 是某个像素 / 节点的（隐藏层 hidden layer）激活值 activations of a pixel/node。

为了在图上使用，完整的单点更新式：

$$h_4^{(l+1)} = \sigma(W_0^{(l)} h_0^{(l)} + W_1^{(l)} h_1^{(l)} + \cdots + W_8^{(l)} h_8^{(l)})$$

中心节点编号为 4，它周围有 0 - 8 共 9 个位置（3×3 窗口，含自己），每一个位置各自的激活值先乘上各自的权重矩阵 W_i，全部相加，再过激活函数 σ。

6.2 图卷积网络 Graph Convolution Networks (GCNs)

6.2.1 更新规则 The Update Rule

$ h_i^{(l+1)} = σ(h_i^{(l)} W_0^{(l)} + Σ_{j∈N_i} (1/c_ij) h_j^{(l)} W_1^{(l)})$

节点 i 在下一层的表示，等于两部分的加权和:

1. 它自己的表示 $h_i $乘一套权重 W_0;
2. 它的所有邻居 j 的表示 $h_j $各乘一套权重 W_1、再除以一个归一化常数 c_{ij}，然后全部加起来;
3. 最后过一个非线性激活 σ。

符号	说明	中文
σ	σ 加在整体外面	非线性激活函数
$h_i^{(l)}$	节点 i 在第 l 层的表示	注意**自己的信息**单独乘 W_0，不与邻居混在一起
$W_0^{(l)} / W_1^{(l)}$	两套权重矩阵	**只有两套**：一套给自己、一套给邻居
N_i	neighbor indices	节点 i 的**邻居索引集合**
c_{ij}	norm. constant (fixed/trainable)	归一化常数，可以**固定**也可以**可训练**

Tips：关键改动：邻居之间共享权重 Weight sharing over all locations，代价是丢失了邻居的相对方位，所以后来 GAT 等模型要引入注意力权重。

6.2.2.1 希望具备的四条性质 Four Desirable Properties

1. Weight sharing over all locations 在所有的位置上共享权重。
2. Invariance to permutations 对置换不变：重排节点编号，结果不变。

3. Linear complexity O(E) 线性复杂度，与边数 E 成正比。

4. Applicable both in transductive（**直推式**） and inductive（**归纳式**） settings 两种设定都能用。

直推式和归纳式的区分：直推式指训练时已经见过整张图（包括测试节点），只是不知道它们的标签，例如对一个固定的社交网络做节点分类，图是同一张，只是有一部分人没标注；归纳式指测试时要处理训练时没见过的新节点或新图，例如对一张全新的分子做预测。

为什么 GCN 两者都能用：因为它的更新规则只依赖邻居的现场表示与学好的共享权重。前者可以现场算出来，后者是训练阶段留下的参数；所以遇到新节点时不需要重新训练整个模型。

6.2.3 谱方法 The Spectral View

Spectral-based methods can be also interpreted as spatial-based methods.

基于谱的方法也可以被解释为基于空间的方法。

$ H^{(l+1)} = \delta (D^{(-1/2)} Â D^{(-1/2)} H^{(l)} \theta)$

含义：把整层所有节点的表示堆成矩阵 H，先左乘一个**归一化后的邻接矩阵** $ D^{(-1/2)} Â D^{(-1/2)} $（相当于聚合邻居），再乘可学习参数 θ，最后过一个非线性函数 δ。

记号	课件说明	中文
A	A: Adjacency matrix	邻接矩阵
D	D: Degree matrix	度矩阵（对角阵，记录每个节点的邻居数）
Â	Â: A + I	带自环的邻接矩阵（加上单位阵 I，把自己也算作自己的邻居） 即更新规则里自己那一项 $h_i W_0$（把自己当成邻居）
D	D: D + I	带自环的度矩阵
θ	θ: learnable parameter	**可学习参数**
δ	δ: non-linear activation function	**非线性激活函数**

空间式更好理解（在邻居之间传消息），谱式更好实现（一次矩阵乘法算完一整层）。

6.3 GNNs应用 GNNs for Application

6.3.1 输入与逐层更新 Input and Layer Update

环节	课件内容	中文
输入 1	Feature matrix $X \in \mathbb{R}^{(N \times E)}$	特征矩阵 X，形状 N×E
输入 2	preprocessed adjacency matrix $\hat{A}$	预处理过的邻接矩阵$\hat{A}$
逐层更新	$H^{(l+1)} = \sigma(\hat{A}H^{(l)}W^{(l)})$	每一层做一次邻接聚合 → 线性变换 → 非线性激活
起始状态	$X = H^{(0)}$	输入特征矩阵就是第 0 层的表示
结束状态	$Z = H^{(N)}$	跑完 N 层之后得到的最终表示 Z
层间激活	图上每个隐层旁标注 ReLU 与 $\sigma(\cdot)$	每层的非线性激活，与普通神经网络一致

6.3.2 三类下游任务 Three Readout Heads

任务	输出头
节点分类 Node classification	$softmax(z_n) $对**每个节点**的表示过 softmax
图分类 Graph classification	$softmax(\Sigma_n z_n)$ **先对所有节点求和**，再过 softmax
链接预测 Link prediction	$p(A_ij) = \sigma(z_i^T z_j) $ 两个节点表示的内积再过 sigmoid

区别是读哪里：**节点分类**读单个节点的向量；**图分类**读整张图的汇总（所以要先求和，把 N 个节点压成 1 个向量）；**链接预测**读两个节点的配对（所以用内积来衡量它们像不像应该相连）

回到 5.3 节的五类任务 节点分类与图分类对应前两类，链接预测对应第四类；节点聚类与影响力最大化在本页没有给出式子。

6.4 GNN vs CNN

对比维度	CNN（Part 2 - 3）	GNN / GCN（Part 5 - 6）
数据形态	规则网格（图像）	不规则的图
输入尺寸	固定（如 224×224×3）	可变（节点数与边数都随样本变）
样本间结构	所有样本结构相同	**每个样本结构都可能不同**
节点顺序	有固定朝向（宽、高）	**无固有顺序**
要保的性质	**平移等变 / 不变**	**置换不变**
权重套数	每个相对位置一套（3×3 核 = 9 套）	**只有两套**：W_0（*自己*） + W_1（*邻居*）
聚合方式	在固定窗口内做加权求和	在**邻居集合**$N_i $上求和，除以归一化常数 c_{ij}
复杂度	与图像尺寸、核大小相关	O(E)，与**边数**成正比
引入的归纳偏置	**局部性 + 平移等变**	**置换等变 + 邻居聚合**

Tips: ViT 质疑的是CNN 的归纳偏置是否必需（3.1.4）；而 GNN 不是去掉归纳偏置，而是换一套更贴合图数据的归纳偏置（置换不变 + 邻居聚合）。两条路线的共同问题：当数据本身没有规则结构时，我们该把什么结构免费注入模型？

Part 7 Generative Models I 生成问题（一）

前三个部分（CNN / RNN / GNN）的共同点：输入一段已有的数据（图像 / 序列 / 图），输出一个**判别结果**。从本部分开始换任务：输入一批样本，输出同一分布下的新样本。这是机器学习里一个完全不同的建模目标。不处理已经存在的数据的部分，而是造出新的数据。

7.1 判别模型与生成模型 Discriminative vs Generative

In Machine Learning, one can clearly distinguish between the two modelling types:

1. Classifying an image as a dog or a cat falls under Discriminative Modelling,
2. Producing a realistic dog or a cat image is a Generative Modelling problem

机器学习可以清楚地区分两种建模类型：把一张图判成狗还是猫，属于判别式建模；生成一张逼真的狗或猫的图片，是生成式建模问题。判别模型学的是**边界**，生成模型学的是**分布**。

对比维度	判别模型 Discriminative	生成模型 Generative
要回答的问题	**这是哪一类?**	**这类数据长什么样?**
学的是什么	条件分布 $p(y	x)$：给定输入 x，输出的标签概率
输出	一个标签（或一组概率）	一个**新样本**

7.2 Goal and Three Ingredients

7.2.1 Goal

Goal: Given training samples, generate new samples from the same distribution.

给定训练样本，**从同一分布中生成新的样本**。

In other words, generative models try to learn a model p_model(x) similar to p_data(x).

换句话说，生成模型试图学到一个 p_model(x)，让它**接近真实的数据分布** p_data(x)。

7.2.2 Three Ingredients

1. 一个模型 Model：we specify its parametric form, e.g., neural networks, etc. 指定它的参数化形式，例如神经网络。
2. 一个学习目标 Learning objective：e.g., maximum likelihood 例如最大似然。
3. 一个优化算法：optimization algorithm to achieve the learning objective 一个用来达成该学习目标的算法。

Tips: 生成模型的三要素与神经网络框架一致：定结构 → 定损失 → 用梯度下降优化，区别在于生成模型的第二项，如何确定学习目标，因为 p_data(x) 是一个无法直接写出的高维分布，所以测量模型离真实分布有多远要另想办法，所以引出了VAE、GAN、Diffusion 三条路线的分歧点。

7.3 自编码器Autoencoders

结构：Autoencoders are neural network architectures that consist of an encoder（编码器**）** and a decoder,（解码器） which work together to learn a **lower-dimensional representation** of input data and **reconstruct** the data from the representation.

自编码器由与解码器组成，两者合作学习输入的低维表示，并从该表示重建数据。

核心任务：With the lower-dimensional representation z, the fundamental task of an autoencoder is to reconstruct its input x as accurately as possible. 用低维表示 z，**尽可能准确地重建输入 x**。

过程：Through a process of encoding and decoding, the autoencoder learns to map the input data to a compressed representation and then decode this representation back to the original input space.

把输入映射到**压缩表示**，再解码回**原始输入空间**。

7.3.1 数据通路与损失函数 Data Flow and Loss

环节	记号	含义
输入	x	原始数据
编码器	E（encoder, x → z）	**降维**：把 x 压成 z

环节	记号	含义
潜在编码	z	压缩后的低维表示
解码器 / 生成器	G（decoder/generator, z → x）	把 z 还原回数据空间
重建	x̂	重建出的输出
损失	L	重建误差，用均方误差衡量

均方误差（Mean Squared Error, MSE） 把每个样本原始值与其重建值之差的平方求平均。n 是变量的个数（where n is the number of variables）。

$$L = (1/n)\,\Sigma_{i=1}^{n}(x^i - G(E(x^i)))^2$$

1. The encoder network is for dimension reduction, just like PCA 编码器的作用就是降维，和 PCA 类似；
2. Latent code: the compressed low dimensional representation of the input data 潜在编码：输入数据的压缩低维表示。

7.3.2 自编码器的三条局限 Why it Cannot Generate

1. **训练目标里没有生成这一项。**损失函数 L 只要求 x̂ 尽量等于 x，也就是输入什么就还回来什么。一个完美重建的自编码器，它的输出空间里没有新样本的位置。
2. **潜在空间 z 没有被规定形状。**编码器可以学会把每个训练样本映射到潜在空间里一个孤立的点，只要解码器能把这些点，还原回来，损失就是 0。这时在潜在空间里随便取一个新点 z，解码器完全不知道该输出什么，因为它从没见过那个位置。
3. 问题的根源：它**只学了点对点的映射**，没学空间的结构，缺少一个可以安全采样的潜在空间。 这正是下一节 VAE 要解决的事。

7.4 变分自编码器 Variational Autoencoders (VAEs)

7.4.1 两条关键改动 Two Key Changes

改动一：Encode the latent features in a meaningful space, e.g., Gaussian distribution. 把潜在特征编码到一个有意义的空间里，例如高斯分布。

改动二：Sample from the meaningful latent space to generate data. 从这个有意义的潜在空间中采样来生成数据。

Example：Assume training data x is generated from the **Gaussian distribution** of latent representation z

Intuition: x is an image, z is latent factors used to generate x.

假设训练数据 x 是由潜在表示 z 的**高斯分布**生成的。**直觉**：x 是一张图像，z 是**用来生成 x 的潜在因子**。

In a nutshell, a VAE is an autoencoder whose encoding distribution is **regularized** during the training in order to ensure that its latent space has good properties allowing us to generate some new data.

简言之，VAE 就是在训练过程中对编码分布加正则的自编码器，目的是让潜在空间具备允许我们生成新数据的良好性质。

7.4.2 两个概率分布与损失 Two Distributions, Two Losses

项	记号	读法
编码（推断）	q(z|x) Inference (encoder)	给定数据 x，推断潜在变量 z 的分布
解码（生成）	p(x|z) generation (decode)	给定潜在变量 z，生成数据 x 的分布
重建损失	L_{MSE}	重建得像不像
正则损失	L_{kl}	**KL 散度**：把编码分布拉向标准高斯，让潜在空间有形状
总损失（核心）	$L_{total} = L_{MSE} + L_{kl}$	**两项相加**：既要重建得准，又要潜在空间规则

$L_{total} = L_{MSE} + L_{kl}$ 这一行是 VAE 的核心。根据 7.3.2 的三条局限：只有 L_{MSE}（即普通自编码器）时，潜在空间可以是任意形状、点与点只有映射；加上 L_{kl} 之后，编码器被迫把每个样本编码成**一个高斯分布**而不是一个点，于是整个潜在空间被铺满，**随便取一个点都能解码出合理的数据**。

Example：先验采样 → 解码生成的完整路径

从 Latent features: z 经 Decoder 得到 Data: x，标注为 Generate x ~ p_θ (x|z)；而 z 是 Sample from true prior z ~ p_θ (z)（从真实先验中采样）。生成时不再需要编码器，只需要从先验里取一个 z 然后解码。

7.5 GAN 的提出动机 Why GANs

问题：VAEs define density function with latent z of Gaussian distribution. However, it is non-trivial（不容易） to model complex, high-dimensional training data with an **explicit distribution**.

VAE 用高斯分布的潜在变量 z 来定义密度函数。**但是**，要**用一个显式的分布**去建模复杂的、高维的训练数据，并不是一件容易的事。

7.5.1 GAN 的两条路线选择 Two Design Choices

1. 放弃显式密度：GANs belong to "implicit density"（隐式密度）group, where any explicit density function of samples did not provide. GAN 属于隐式密度这一组，它不提供样本的显式密度函数。

即 VAE 的路线是写出密度，然后去逼近它；GAN 的路线是写不出密度，那就绕过它，既然无法显式写出 p_data(x) 长什么样，那就让一张网络（判别器）来隐式地衡量 这像不像真数据，另一张网络（生成器）去把这个衡量推翻。

2. 用博弈论做动机：Motivated by game theory.

GAN 的代价是训练变得不稳定，因为它本质上是一个对抗性的博弈过程，而不是一个单纯的 误差下降。

玩家	课件原文	中文
生成器 Generator	Generator tries to generate "fake" samples from its model (tries to fool the discriminator)	生成器试图从自己的模型里**造出假样本**，目标是骗过判别器
判别器 Discriminator	Discriminator tries to distinguish "fake" and "real" samples	判别器试图**区分假样本**与真样本

Part 8 Generative Models II 生成问题（二）GANs and Diffusion Models

8.1 GAN 的目标函数 The Minimax Objective

Minimax objective function（极小极大目标函数）与 How to train (Alternative between)（怎么训练：在两者之间交替）。

极小极大目标：

生成器参数 θg想最小化、判别器参数 θd想最大化同一个目标函数；它由两项组成，**真数据的对数和**与**假数据的1减去判别结果的对数和**。

记号	中文	说明
$D_{\theta d}(x)$	判别器对真样本 x 的输出	判别器认为 x 为真的概率
$G_{\theta g}(z)$	生成器由噪声 z 造出的假样本	从先验 p(z) 采样 z，生成假样本
第一项 $E_{x\ p_{data}} logD(x)$	真数据的损失	用红框标出，注为 Loss for real data x
第二项 $E_{z\ p(z)} log(1 - D(G(z)))$	假数据的损失	图上用蓝框标出，注为 Loss for fake data x

8.1.1 两个网络 How It Works

判别器 D：D tries to maximize the cost such that D(x) close to 1 (for real x) and D(G(z)) close to 0 (fake)

让真样本的输出趋近 1、假样本的输出趋近 0。

生成器 G：G tries to minimize the cost such that D(G(z)) is close to 1 (try to make generated samples real-looking as much as possible to fool D) 让假样本的判别输出也趋近 1，即尽可能把假样本做得像真的去骗过 D。

8.1.2 交替训练 Alternate Between

固定 G，训练 D：Fix G, D maximize the cost (Gradient ascent on discriminator) 判别器做梯度上升。

固定 D，训练 G：Fix D, G minimize the cost (Gradient descent on generator) 生成器做梯度下降。

Tips：交替训练的问题：

普通的神经网络训练有一个静止的目标：损失函数在训练过程中形状不变（数据固定）。而 GAN 里，判别器每更新一次，生成器的损失函数就变了一次，这是博弈式建模的固有性质。

8.1.3 GAN Lab

GAN Lab: Play with Generative Adversarial Networks in Your Browser! (poloclub.github.io)

GAN 的交替训练过程很难用静态文字讲清楚，生成器与判别器的决策边界在二维平面上一来一回地移动，这是 GAN Lab 最直观的地方。看生成器的样本点如何一步步逼近真实分布、判别器的分界线如何反复摆动。观察交替训练和不稳定性。

8.3 GAN 的三个问题与扩散模型 Problems of GANs

问题 Problem	原文	中文
梯度消失 Vanishing Gradients	when the discriminator doesn't provide enough information for the generator to make process.	当判别器不能提供足够的信息让生成器取得进展时出现
模式崩塌 Mode Collapse	This is when generator starts producing the same output (or a small set of output) over and over again.	生成器开始反复产出同一个输出（或很小的几个输出）
难以训练 Hard to train	GANs are very sensitive to hyperparameters.	GAN 对超参数非常敏感

GAN问题共同根源：判别器的设计。梯度消失来自判别器太强，模式崩塌来自生成器找到了骗过判别器的捷径，难训练来自两者需要保持脆弱的平衡。

解决：**Diffusion Models**: Emerging as powerful generative models, outperforming other deep generative models, such as GANs.

扩散模型正在成为强大的生成模型，表现超过了 GAN 等其他深度生成模型。扩散模型的做法是它不训练判别器，而是训练一个网络去拟合一个明确的、已知的加噪过程。

8.4 扩散模型是什么 What Are Diffusion Models

灵感来源：Inspired by non-equilibrium thermodynamics to model the data distribution, diffusion models can describe the probability distribution of each step in the ink diffusion process.

受非平衡热力学启发，把一滴墨水滴进水里，它会逐步散开，最后完全均匀，这个过程是随机、不可逆的。扩散模型的做法是**反向学**这个过程：从完全均匀的噪声出发，一步步收拢回一滴墨水。

Diffusion models are a type of probabilistic generative model that transforms noise into a representative data sample. 是一类概率生成模型，它把噪声变换成一个有代表性的数据样本。

8.5 前向加噪与反向去噪 Forward and Reverse Processes

Diffusion model aims to learn the reverse of noise generation procedure

扩散模型的目标是学习 噪声生成过程 的反过程。

具体而言，正向的加噪过程是我们自己定义的、完全已知的（每一步加多少高斯噪声固定的），所以模型只需要学怎么反向走一步，这是一个有明确监督信号的学习问题，不需要另一个网络来判别。

8.5.1 两个方向 Two Directions

方向	原文	中文
前向步 Forward step	(Iteratively) Add noise to the original sample The sample x_0 converges to the complete noise x_T (which is the Markov chain)	（迭代地）给原始样本加噪声 样本 $x_0 $收敛到完全的噪声 x_T；这个过程是一条马尔可夫链
反向步 Reverse step	Recover the original sample from the noise Note that it is generation procedure	从噪声中恢复出原始样本 注意：这一步就是 生成 过程

8.5.2 训练目标 Training Objective

To train a Diffusion Model, the objective is to find the reverse Markov transitions that **maximize the likelihood of the training data.**

训练扩散模型的目标，是找到那些能**最大化训练数据似然**的反向马尔可夫转移。

左边是交叉熵损失（负的、模型对真实样本的对数似然），右边是它的一个上界：分子$ q(x_{1:T}|x_0) *是我们自己定义的前向加噪分布*，*分母*p_θ (x_{0:T}) $是要学的反向生成分布。

8.6 一个网络处理所有时间步 One Network for All Timesteps

For diffusion models, it is common to use a single network for all timesteps. Something UNet-like, which is very common in denoising and image-to-image problems.

对扩散模型来说，常见做法是用一个网络处理所有时间步。这个网络大致是 U-Net 形态，U-Net 在去噪与图像到图像类问题里非常常见。

8.6.1 训练时的六个步骤 The Training Process

1. Sampling a random timestep t for each training sample within the batch (e.g. images)

为 batch 里的每个训练样本各采样一个随机时间步 t。

2. Adding Gaussian noise by using the closed-form formula, according to their timesteps t

按各自的 t，用闭式公式加上高斯噪声。

3. Converting the timesteps into embeddings for feeding the U-Net or similar models (or other family of models)

把时间步转成嵌入向量，喂给 U-Net 或同类模型。

4. Using the images with noise and time embeddings as input for predicting the noise present in the images

用带噪图像 + 时间嵌入作为输入，去预测图中的噪声。

5. Comparing the predicted noise to the actual noise for calculating the loss function

把预测噪声与真实噪声比较，算出损失。

6. Updating the Diffusion Model parameters via backpropagation using the loss function

用损失做反向传播，更新参数。

三张训练进度图：分成 EPOCH 1、EPOCH 2、EPOCH k 三栏，每栏里列出 TIME EMBEDDINGS（时间步嵌入）与 NOISY IMAGES（带噪图像）的配对。EPOCH 1 里是 t = 14 / 35 / 6，EPOCH 2 里是 t = 78 / 153 / 721，EPOCH k 里是 t = 64 / 4 / 9。这组数字要传达的是** 每个样本的 t 都是随机抽的** 同一个样本在不同 epoch 会配到不同的噪声强度，这样网络才能学会所有时间步。

U-Net 结构图：Conv+BatchNormalization+ReLU（红色块）、Pooling operation（黄色块）、Upsampling Layer（蓝色块）、Skip-Connection（箭头）。整图呈 U 形：左侧向下经过多次池化（Layer1 → Layer2 → Layer3 → Layer4），底部最窄，右侧向上经过多次上采样，并有跨过底部的跳跃连接把两侧同层连起来，就是 U-Net 名字的由来，也是它能在去噪任务里保细节的原因。

8.7 采样过程 The Sampling Process

与训练的唯一区别：For sampling new images, the difference lies in that we don't have an input image.

生成新图像时，区别在于我们手上没有输入图像。

起点：We sample random Gaussian noise and define how much steps of noise (T) to take for generating the new images. 我们从随机高斯噪声出发，并规定要走的噪声步数 T。

每一步做什么：In each step, the Diffusion Model predict the whole noise present in the image, taking as input the current timestep. Then, it removes just a fraction of this predicted noise. —— 每一步里，模型以当前时间步为输入，预测整幅图中的噪声，然后只去掉其中的一小部分。

Tips: 为什么不一次把预测出来的噪声全减掉？因为网络在时间步 t 上的预测只在 t 这个噪声强度下可靠。一次性减完会直接跳到 x_0，但那个 $x_0 $是在 噪声还很多 的假设下估出来的，误差很大。逐步减去一小部分，等价于“走一小步、重新估一次、再走一小步”每一步都在新的噪声强度下重新用网络判断，误差不会累积放大。

结果：We obtain our image generation result after T inference steps. 经过 T 次推理步后得到最终生成结果。

两段伪流程对应图：

1. Sample Gaussian noise t = T = 1000、X_1000 = N(0, I)，箭头标注 sample，指向一张纯噪声图；

2. Iteratively denoise the image 下面画了多组 噪声图 → U-Net → pred noise 的迭代。

关键等式 $X_{999} = X_{1000} - prednoise$、 $X_2 = X_1 - prednoise$、 $X_1 = X_2 - prednoise$，最后得到干净图像。

这些等式传达的正是减掉预测噪声这个动作，两者合起来看：预测的是全部噪声，但每次只减一部分。

8.8 扩散模型小结 Summary

生成质量更好：Generates better data than VAEs and GANs. Diffusion models learn true distribution through step-by-step learning of Gaussian noise instead of raw data. This makes learning objectives simpler and the generation quality better. 扩散模型逐步学习高斯噪声而不是直接学原始数据，这让学习目标更简单、生成质量更好。

更容易训练：Easier to train. Diffusion models are easier to train than GANs and do not suffer from mode collapse due to their inherent diffusion process. 比 GAN 更容易训练，且由于其内在的扩散过程，不会遭遇模式崩塌。

In conclusion, Diffusion Models became a new effective paradigm for image generation and manipulation. By combining the principles of diffusion processes with deep learning techniques, these models offer new avenues for generating high-quality images.

总而言之，扩散模型已经成为图像生成与图像编辑的一种新的有效范式。它把扩散过程的原理与深度学习技术结合，为生成高质量图像提供了新的路径。

8.9 四类生成模型 Four Generative Paradigms

对比项	Autoencoder（AE）	VAE	GAN	Diffusion
核心思路	编码降维 + 解码重建	在 AE 上给潜在空间加正则	生成器与判别器对抗博弈	学加噪的反过程
损失函数	$L = (1/n)\Sigma(x^i - G(E(x^i)))^2$	$L_{total} = L_{MSE} + L_{kl}$	$min_\theta g max_\theta d ElogD(x) + Elog(1 - D(G(z)))$	$L_{CE} = -Elogp_\theta(x_0)$（及其变分上界）
能否生成新样本	不能	能（从先验采样 z 再解码）	能（生成器直接造）	能（从纯噪声迭代去噪）
潜在空间	无约束，可能长成任意形状	被正则成高斯，可安全采样	隐式密度，无显式 z 空间要求	噪声强度 t 充当连续控制变量

对比项	Autoencoder（AE）	VAE	GAN	Diffusion
主要毛病	无法生成	显式建模高维分布很难	梯度消失 / 模式崩塌 / 难训练	采样步数多（T = 1000）

AE 只会重建 → VAE 加正则让潜在空间可用 → GAN 绕过显式密度改用对抗 → Diffusion 放弃对抗改学一个已知的加噪过程。每一次转折都是在解决上一代的某个具体毛病：AE 的问题是不能生成，VAE 的问题是显式密度难写，GAN 的问题是训练不稳定，而 Diffusion 用有确定答案的回归问题换掉了没有确定答案的博弈问题。

Part 9 Overfitting and Underfitting 过拟合与欠拟合

9.1 三种拟合状态 Three Fitting Regimes

状态	定义	中文	图上特征
欠拟合 Underfitting（高偏差 High Bias）	Model too simple - cannot capture the underlying pattern. Both training and validation errors are high.	模型太简单，抓不住潜在规律；训练误差与验证误差都高	红色直线，几乎穿过数据的整体趋势，但完全跟不上那个弯
良好拟合 Good Fit（平衡 Balanced）	The model captures the true pattern without memorizing noise. Both errors are low and close.	抓住了真实规律而没有记住噪声；两个误差都低且接近	绿色曲线，与灰色虚线几乎重合
过拟合 Overfitting（高方差 High Variance）	Model too complex - memorizes training data including noise. Low training error but high validation error.	模型太复杂，把训练数据连噪声一起背下来；训练误差低，但验证误差高	蓝色曲线，穿过每一个黑点，在黑点之间剧烈扭曲

9.2 偏差-方差权衡 Bias-Variance Tradeoff

曲线	走向	含义
偏差平方 Bias² (underfitting)	随复杂度单调下降	Bias: Error from oversimplified assumptions. → Underfitting 来自假设过于简化的误差，对应欠拟合
方差 Variance (overfitting)	随复杂度单调上升	Variance: Error from sensitivity to small fluctuations in training data. → Overfitting 来自对训练数据微小波动过于敏感的误差，对应过拟合
总误差 Total Error	先降后升，呈 U 形	偏差与方差之和；最低点就是最优复杂度
不可约误差 Irreducible Error	一条水平虚线	与模型复杂度无关、降不下去的那部分误差

Find the sweet spot with minimum total error. 即找到总误差最小的那个最佳点(Optimal Complexity)。

在复杂度 0 处，偏差平方约 1.0、方差约 0.05、总误差约 1.05；随着复杂度增加到约 2，偏差降到约 0.7、方差升到约 0.3；到复杂度 10 时，偏差接近 0，但方差升到约 5。所以总误差的最低点并不在偏差最小或方差最小处，而在两者交叉附近。

权衡 Tradeoff: 降低偏差要靠更复杂的模型，而更复杂的模型必然带来更高的方差，两者是同一个动作的两面，不可能同时降。这与 1.6 节 的第三条（泛化）直接相关：Yet they don't overfit (when training data and model size is compatible). 只要训练数据量与模型规模相匹配，它并不因此过拟合。

9.3 诊断模型性能的学习曲线 Learning Curves

状态	定义	两曲线的形态	图上读数
欠拟合	Both training and validation errors are high and close together. Adding more data does not help-need a more complex model.	两条线都很高且互相贴近，且随数据量增加几乎不下降	训练约 0.38 → 0.36；验证约 0.47 → 0.42
良好拟合	Both errors are low and converge. The model generalizes well to unseen data.	两条线都低、且随着数据增多收敛到一起	训练约 0.19 → 0.09；验证约 0.28 → 0.13
过拟合	Large gap between training (low) and validation (high) error. More training data can help reduce the generalization gap.	两条线之间差距很大；图例把这段差距标为 Generalization gap，并用粉色阴影涂出	训练约 0.08 → 0.02；验证约 0.49 → 0.38

More training data can help reduce the generalization gap. 更多的训练数据可以缩小泛化差距。这与欠拟合的状态：Adding more data does not help（加数据没用）形成对照。

注意图上过拟合曲线在横轴右端仍然没有收敛，这正是在说数据还不够。

欠拟合：两条曲线都高且贴在一起，加数据只是让它们更贴，误差下不去 → 该换更复杂的模型。

过拟合：两条曲线差距大，加数据会让验证误差往下追训练误差 → 该去收集更多数据。

9.4 五条对策 How to Address Overfitting and Underfitting

这两类问题需要完全相反的动作。

序号	对过拟合（缓解）	对欠拟合（增强）
1	Regularization (L1/L2 penalty) , adds penalty for large weights. （对大的权重**加惩罚**）	Increase Model Complexity, Add more layers or neurons. （加层或**加神经元**）
2	Dropout, randomly deactivate neurons during training. （训练时随机失活一部分神经元）	Train Longer, Increase number of epochs. （增加训练轮数）
3	Early Stopping, Stop when validation error starts increasing. （早停：验证误差开始上升就停）	Reduce Regularization, Lower L1/L2 penalty or dropout rate.（降低惩罚项或失活率）
4	Data Augmentation, Increase training data diversity. （增加训练数据的**多样性**）	Better Feature Engineering, Add more informative features. （加更有**信息量**的特征）
5	Batch Normalization, Normalize layer inputs to stabilize training.（归一化每层输入以稳定训练）	Use a Different Architecture, try a more powerful model class (e.g., deeper network). （换更强的模型）

Part 10 Tutorial 5 Reinforcement Learning

Learning Objectives

1. 描述通过交互与延迟奖励来学习 describe learning through interaction and delayed rewards。
2. 把序列决策问题形式化成一个马尔可夫决策过程 formulate a sequential decision problem as a Markov decision process。
3. 区分策略、回报、状态价值与动作价值 distinguish policies, returns, and state and action values。

4. 解释贝尔曼最优方程与 Q-learning 更新式 explain the Bellman optimality equation and the Q-learning update。
5. 说出表格型 Q-learning 的主要假设与局限 state the main assumptions and limitations of tabular Q-learning。

10.1 Introduction: Reward Now or Later

近处有一颗小星星，远处还有一颗更大的，但要多走一步。智能体该选哪条路?

眼前收益 vs 长期收益。后面 p.9「等待的价值」会把这个问题算成具体数字。

注意插图本身不含技术信息（一个机器人在岔路口，左近右远）—— 本讲真正给出答案的是折扣因子 γ：γ 大就愿意等，γ 小就只顾眼前。

10.2 Reinforcement Learning

Reinforcement learning (RL) studies how an agent learns to choose actions through interaction with an environment to maximize expected cumulative reward.

强化学习研究智能体如何通过与环境交互来学会选择动作，以最大化期望累积奖励。

Actions can change both the immediate reward and future states. 动作会同时改变即时奖励与未来状态。

Feedback is typically a scalar reward, not the correct action for each state. 反馈是一个标量奖励，而不是每个状态下的正确动作，与监督学习的区别。

The agent must balance using what it has learned with gathering new information. 必须在使用已学到的信息与获取新信息之间权衡。

三类决策问题示例：robot control（机器人控制）、resource allocation（资源分配）、game playing（博弈）。

10.3 The Agent - Environment Interaction

在时刻t，智能体观察S_t、选择 A_t、先把动作做出去，环境才给出反馈，于是收到奖励R_{t+1} 与下一个状态S_{t+1}。

策略 policy：一个从状态到动作的映射，策略$\pi(a|s)$规定在状态 s 下如何选动作（给每个动作一个被选中的概率）。

轨迹 trajectory：训练数据，一条轨迹就是状态、动作、奖励交替出现的序列。

回合 episode：一条样本的结束，一个回合在终止状态（terminal state）结束；而持续性任务没有自然终点。LM的区别在于：RL 的样本不是别人给的，是智能体自己走出来的。An episode ends in a terminal state; Continuing tasks have no natural endpoint.

10.4 序列决策的挑战

问题1：Delayed consequences: an action may have little immediate reward yet improve later outcomes

延迟后果：某个动作即时奖励很少，却改善了后续结果。需要回报 G（把未来奖励折现加总），因为只看即时奖励就分不出好坏；

问题2：Uncertainty：the same action may lead to different next states or rewards

不确定性：同一个动作可能导向不同的下一状态或奖励。需要期望值 $E[・] $与转移概率 $p(s', r|s, a)$

问题3：Exploration：information about unfamiliar actions is acquired by trying them

探索：关于陌生动作的信息，只能靠去试才能获得。需要$ ε -greedy $这类行为策略

The objective is therefore to optimize a sequence of decisions, rather than each reward in isolation. 要优化的是一连串决策，而不是孤立的每一个奖励。

10.5 马尔可夫决策过程 MDP

Markov property: given the current state and action, the distribution of the next state and reward does not depend on the earlier history.

给定当前状态与动作时，下一状态与奖励的分布不依赖于更早的历史。

状态与可行动作 states S and available actions A(s)。可行动作依赖当前状态。

转移与奖励律： $p(s', r|s, a) = Pr(S_{t+1} = s', R_{t+1} = r|S_t = s, A_t = a)$。在 s 做 a 之后，下一状态是 s′ 且奖励是 r 的概率。

折扣因子：a discount factor $\gamma \in [0,1)$ for the discounted setting。

Tips：The state must therefore contain the information needed to predict the consequences of actions. If only partial observations are available, an MDP state may not be directly observed.

马尔可夫决策的工程依据：如果你的状态表示漏掉了预测所必需的信息，那它就不是一个合格的状态。只能拿到部分观测时，MDP 里的状态可能根本观测不到。

10.6 Return, Value, and Bellman Equations 回报、价值与贝尔曼方程

10.6.1 回报与学习目标 Return and the Learning Objective

The return from time t is the discounted sum of future rewards：从时刻 t 出发的回报（return），是未来奖励的折现和。 γ^k 是第 k 步之后那笔奖励的折扣权重。

$G_t = \Sigma_{k=0}^{\infty} \gamma^k R_{t+k+1}, 0 \leqslant \gamma < 1$

回合制任务：For an episodic task, the sum stops at the terminal time. 求和到终止时刻就停。

γ 的作用： γ controls how strongly later rewards contribute to the objective. γ 控制越晚的奖励对目标的贡献有多强。

$\gamma \in [0,1)$ 这个区间是为了保证求和收敛。如果奖励有界（比如 |R| ⩽ R_max），那么 $ |G_t| ⩽ R_max / (1 − γ)$ 越接近 1，这个上界越松，长期视野越长。 $\gamma = 0$ 时 $ G_t = R_{t+1}$，智能体只看眼前一步。

学习目标：The goal is to find a policy maximizing expected return，写成 $\pi^* \in argmax_{\pi} E[G_0]$（对某个指定的初始状态分布取期望）。

奖励设计也是问题的一部分：The objective depends on how rewards are defined; reward design is part of the problem formulation.

10.6.2 Example：The Value of Waiting

Suppose one action ends the episode with reward 2. Another yields 0 now and rewards 5 on the following step, then ends the episode.

路线	课件写法	含义
立即收益 Immediate reward	$G_t = 2$	做完这一步回合就结束，直接拿 2
延迟收益 Delayed reward	$G_t = 0 + \gamma(5)$	第一步拿 0，第二步拿 5，但第二步的 5 要乘 γ 折现

At $\gamma = 0.9$, the delayed route returns 4.5 and is preferable. At $\gamma = 0.3$, its return is 1.5, so the immediate route is preferable.

$\gamma = 0.9$ 时延迟路线的回报是 4.5（$0.9 \times 5 = 4.5 > 2$），优于立即收益；$\gamma = 0.3$ 时回报是 1.5（$0.3 \times 5 = 1.5 < 2$），立即收益更好。

γ 不是多重视未来这种含糊的说法，它直接改变最优决策。同一个环境、同一组奖励，只换了 γ，答案就从等翻成不等。取舍的分界点是 $\gamma \times 5 = 2$，即 $\gamma = 0.4$。$\gamma > 0.4$ 等，$\gamma < 0.4$ 不等，$\gamma = 0.4$ 两条路线完全相同（都是 2）。这个临界 γ 是判断这类题最快的方法。

10.6.3 状态价值与动作价值 State and Action Values

$v_\pi(s) = E_\pi[G_t \mid S_t = s]$

$q_\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$

$v_\pi(s)$：evaluates a state when actions follow π 评估在状态 s，之后都按策略 π 选动作时的期望回报。

$q_\pi(s, a)$：evaluates first taking a in s, then following π 评估在 s 先做 a，之后才按 π 走时的期望回报。因为选动作需要一个对每个候选动作打分的函数，而 v 给不出这个分数。

最优动作价值：对所有策略取最大，得到真正的最优动作价值。

$q^*(s,a) = \max_\pi q_\pi(s,a)$

在有限 MDP 里，对 q* 逐状态取 argmax 的策略就是最优策略。

10.7 从回报到贝尔曼期望方程 From Return to Bellman Expectation

10.7.1 Splitting the Return 把回报拆成现在 + 将来

The return separates into the next reward and the remaining return:

$G_t = R_{t+1} + \gamma G_{t+1}$

从今天看的回报 = 下一步的奖励 + 从下一步看的回报再折一次现。

1. 第一步，引入的是环境的不确定性（$p(s', r|s, a)$）：先对可能的下一状态与奖励取条件 condition on the possible next state and reward (s′, r)。
2. 第二步，引入的是策略的不确定性（$\pi(a'|s')$）：选择下一动作时按策略 π 走 follow policy π when choosing the next action a′。
3. 最后对所有这些结果取平均 average over these outcomes。

10.7.2 The Bellman Expectation Equation 贝尔曼期望方程

$q_\pi(s, a) = \Sigma_{s', r} p(s', r|s, a)[r + \gamma \Sigma_{a'} \pi(a'|s') q_\pi(s', a')]$

This equation separates action value into:

1. the immediate reward;

2. the discounted value of subsequent decisions under π.

方程	对应上一步	说明
$Σ_{s′,r} p(s′,r	s,a) [⋯]$	第 1 步 + 第 3 步
r	第 1 步	括号里的即时奖励部分
$γ Σ_{a′} π(a′	s′) q_π(s′,a′)$	第 2 步

意义：它把一个无限长的期望变成了一步 + 同一个量。右边又出现了 $q_π$ 自己（bootstrapping）：用下一状态的价值来定义当前状态的价值。正因为有这个自指结构，才能迭代求解，而不必把无穷级数真的展开。

At the terminal next state, the continuation value is zero. 若下一状态是终止状态，后续价值那一项为 0（此时只剩即时奖励）。

10.8 从策略评估到最优控制 From Policy Evaluation to Optimal Control

From policy evaluation to optimal control, The Bellman optimality equation replaces the policy average with the best available next action:

唯一改动是把平均换成取最大（最好的动作），把评估换成了优化

贝尔曼最优方程：

$$ q*(s,a) = Σ_{s′,r} p(s′,r | s,a) [r + γ max_{a′} q*(s′,a′)]$$

对比项	贝尔曼期望方程	贝尔曼最优方程
评估对象	$q_π$ 给定策略 π 的价值	< br > $q*$最优价值
下一动作	$Σ_{a′} π(a′ | s′) ·$ 按策略取加权平均	$(⋯)$
用途	策略评估：算这个 π 有多好	最优控制：算最好能有多好
对应的算法路线	迭代策略评估等评估类方法	Q-learning 等最优控制方法

Dynamic programming can use this equation when the transition and reward law is known. 动态规划（DP）能用这条方程，前提是已知 $p(s′,r|s,a)$；

Q-learning estimates q* from sampled transitions $(s,a,r,s′)$ without requiring that law. Q-learning 从采到的样本估计 q*，不需要知道这个定律。

10.9 From Bellman Optimality to Q-learning

10.9.1 三个替换 Three Substitutions

贝尔曼最优方程，需要对所有可能的环境输出结果求期望：$r + \gamma \max_{a'} q_*(s', a')$。

但环境模型未知的时候，智能体只能采样观察到单条状态转移元组 (s, a, r, s')。

因此需要做三处替换来得到可实际更新的目标：

1. Replace the unknown q_* with the current estimate Q. 把真实最优价值函数 q_* 替换成模型当前的估计值 Q。自举（bootstrap）
2. Use the observed reward and the next state to form a one - step target. 利用真实观测得到的奖励 r 和下一状态 s'，构造一步更新目标。 随机近似（用单个样本代替整体期望）

3. Move $Q(s,a)$ partway toward that target. 不再做完整平均，将 $Q(s,a)$ 向上面构造的目标值挪动一部分。步长 / 学习率控制更新幅度

10.9.2 The Q-learning Update

$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha_t(S_t,A_t)[R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a') - Q(S_t,A_t)]$

步长： $\alpha_t(S_t,A_t) \in (0,1]$ is the step size for the updated pair. α 是针对被更新那一对的步长，取值在(0,1]。

The bracketed quantity is the temporal-difference (TD) error. 方括号整块就是时序差分误差。

终止状态：If $S_{t+1} $is terminal, the target is R_{t+1}: there is no future-value term. 若下一步是终止状态，目标就是 R_{t+1}，没有未来价值项。

更新式	作用
$R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a')$ TD 目标（one-step target）	用实际观测 + 当前估计拼出来的这一步应该是多少
$R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a') - Q(S_t,A_t)$ TD 误差 δ	目标比现值高多少；乘上 α 就是要修正的量
α_t 步长 / 学习率	一次修正走多远：$\alpha = 1$ 直接跳到目标，α 小则慢慢挪

α 的写法是 $\alpha_t(S_t,A_t)$ 而不是一个常数 α。这提示步长允许随状态动作对而不同，甚至允许随迭代轮次变化。

10.9.3 Example： Q-learning by Numbers

给定	值
当前估计 $Q(s,a)$	3
观测到的奖励 r	2
下一状态的最优估计 $\max_{a'} Q(s',a')$	4
折扣因子 γ	0.9
步长 α	0.5

第 1 步，先算 TD 目标： $ r + \gamma \max_{a'} Q(s',a') = 2 + 0.9(4) = 5.6。$

第 2 步，更新估计： $ Q_{new}(s,a) = 3 + 0.5(5.6 - 3) = 4.3。$

10.10 TD 误差 How to Interpret the TD Error

Define the one-step target and error as:

$ Y_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a')$

$ \delta_t = Y_t - Q(S_t,A_t)$

$Y_t $是一步目标， $ \delta_t $是TD 误差 = 目标 - 当前估计。

课件的读法：更新把$Q(S_t,A_t) *按比例*\alpha_t$ 朝样本目标$ Y_t $移动。

A positive δ_t means the sample target exceeds the current estimate; A negative value means it falls below it. δ_t为正表示样本目标高于当前估计；为负表示低于。

A single TD error is noisy, so its sign alone does not establish that the true action value is under-or overestimated. 单次 TD 误差是有噪声的，所以只看它一次的正负，并不能断定真实动作价值被低估或高估了。

10.11 Exploration and Exploitation： ε-greedy

以概率 ε_t随机选一个可行动作（探索）；其余概率下在 $argmax_a Q(S_t, a)$ 里选（利用）。

探索的作用：Exploration supplies information about less-visited actions. 提供较少被访问动作的信息。

利用的作用：Exploitation uses the currently highest-valued action. 使用当前估值最高的动作。

衰减必须有下限意义上的保障：A decay schedule must still provide sufficient visits to state - action pairs for tabular convergence claims to apply. 衰减方案仍须保证状态动作对获得足够的访问次数。

10.12 Quick Check: ε-greedy Behavior

一般式： $Pr(贪心动作) = (1 - \varepsilon) + \varepsilon \cdot (1/n)$，其中 n 是可行动作数（本题 n = 2）。当 n 很大时，探索带来的这 ε /n 就接近于 0，这也说明探索在大动作空间里更稀释。

10.13 Why Q-learning Is Off-Policy

> 核心现象： 智能体用 ε - *greedy*这种带探索的**行为策略去产生样本轨迹**； 但是更新目标里面用 $\mathbf{max_{a'}\ Q(S_{t+1}, a')}$，瞄准的是**完全贪心的目标策略**。
>
> 关键定义： The policy generating experience need not match the policy whose value is being learned. 生成样本的行为策略 ≠ 我们正在学习价值的目标策略 → 这就是 off - policy（离策略）。

off - policy ≠ 不需要探索 虽然更新不依赖行为策略，但依然要有充足探索。如果很多(s, a)从来没被访问，Q 得不到修正，max 就取不到真实最优值。

10.14 Convergence of Tabular Q-learning 表格型 Q - Learning 收敛条件

前提：有限折扣 MDP，奖励有界。两组**充分条件**（满足就一定收敛；不满足不代表一定发散，只是定理不作保证）

1. 每一个相关状态 - 动作对(s, a)被**无穷多次更新**。
2. 步长序列满足：

$\Sigma_n \alpha_n(s, a) = \infty, \quad \Sigma_n \alpha_n(s, a)^2 < \infty$

含义拆解：

$\Sigma\ \alpha_n = \infty$：总步长要足够大，保证参数可以走到目标，不会中途卡住。

$\Sigma\ \alpha_n^2 < \infty$：步长后期快速变小，压制样本噪声，不会持续震荡。

> 经典可行例子：$\boldsymbol{\alpha_n=\dfrac1n}$

两种不满足充分条件的实践做法：

1. **固定步长**：$\Sigma\,\alpha = \infty$成立，但$\Sigma\,\alpha^2 = \infty$，第二条不满足。工程上大量使用，但没有该理论收敛保证。
2. ε衰减到 0，部分(s, a)再也不会被采样：第一条无穷次访问不满足。

在全部充分条件成立下：**表格 Q - Learning 以概率 1 收敛到真实最优动作价值**q_*。

10.15 适用范围与局限（每一条对应一类后续算法）

1. **内存爆炸**：Q 表内存大小正比于状态 - 动作对总数，无法处理巨大、连续状态空间。

A Q-table requires memory proportional to the number of state - action pairs and is unsuitable for very large or continuous spaces.

2. **稀疏 / 延迟奖励**：很难做探索和信用分配。

Sparse or delayed rewards can make exploration and credit assignment difficult.

3. **部分可观测 POMDP**：需要额外记录历史信息或者更强模型。

Partial observability requires a representation of relevant history or a richer model.

4. **函数逼近（神经网络如 DQN）**：可以跨状态泛化，**但是表格 Q - Learning 那一套收敛保证不会自动继承过来。**

Function approximation can generalize across states, but tabular convergence guarantees do not automatically carry over.

> 这四个局限催生三大方向：近似价值函数、策略梯度、基于模型强化学习。

The agent may behave according to an exploratory policy, such as ε-greedy, while the update targets the greedy next action through $max_{a'}\ Q(S_{t+1}, a')$.

智能体可能按探索性策略行动，但更新式瞄准的是贪心动作，与正在执行的策略无关。

Thus the policy generating experience need not match the policy whose value is being learned.

产生经验的策略，不必等于被评估价值的那个策略。这就是 off-policy（离策略）的含义。

This does not remove the need for adequate exploration. 但 off-policy 不等于不需要探索。行为策略可以随便与可以不去试是两件事。如果没有足够探索，Q 里那些没被访问过的状态动作对就永远得不到修正，max 也就取不到真正的最大。

Overall

1. RL optimizes expected long-term reward through interaction. RL 通过交互优化期望长期奖励。
2. An MDP formalizes states, actions, rewards, and transition dynamics. MDP 把状态、动作、奖励与转移动态形式化。
3. Bellman equations relate current values to future values. 贝尔曼方程把当前价值与未来价值联系起来（这是能迭代求解的根源）。

4. Q-learning uses sampled transitions to estimate optimal action values. Q-learning 用采样的转移估计最优动作价值。
5. Reliable learning depends on exploration, suitable step sizes, and the assumptions of the chosen model. 可靠的学习依赖探索、合适的步长、以及所选模型的假设三者齐备。

目标 → 形式化 → 方程 → 算法 → 边界。G_t 怎么定义 → MDP 三要素 → 贝尔曼期望 vs 最优（就差一个 max）→ 更新式的四块（现值、奖励、折现后继、步长）→ 收敛条件与四条局限。

5434 Big Data Computing 1

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_6/knowledge/5434-Big-Data-Computing-1.md`

5434 Big Data Computing1

Lecture 4: Hadoop and MapReduce

part 0 Navigation

Hierarchical Structure

层级	课件原文	内容	与本课程的关系
第 1 层	**Data** 数据	Images, text documents, social networks, spatial data (GPS), time series data, transaction networks, gene networks, ⋯	Week 1 - 3 处理过的对象
第 2 层	**Data analytics** 数据分析	Clustering, Community detection, Association rules, Classification, Neural Networks, Graph algorithms ⋯	**Week 1 - 3 的内容全在这一层**
第 3 层	**Computational infrastructures** 计算基础设施	Hadoop, MapReduce, Spark, Hive, RDBMS, KV database, Graph database, ⋯	**本讲与下一讲的内容**
第 4 层	**Applications** 应用	FinTech, Smart City, Bioinformatics, Social Media, Tourism, Transportation, ⋯⋯	最终落点

本节主讲：**当数据装不下一台机器时，把存储和计算都拆开、并把计算搬到数据旁边去。**这就是Hadoop的工程实现，MapReduce 是它的计算模型，整讲都在回答“为什么必须这样拆、拆了之后要额外解决哪些问题”。

全篇脉络:

100 PB 装不下单机

多线程只能解决效率（存储与内存仍无解） 跨机器并行会 lost update（读-改-写非原子） 加锁能保正确但降低并行度（reduced parallelism） 改用“各写各的局部表 + 最后合并”——不需要共享可写状态 这条思路就是计算向数据并拢MapReduce（move operations near the data） 实现为分治法Partition/Combine = Map / Shuffle and Sort / Reduce → 用户只写两个函数，运行时自动并行、容错、均衡、限制通信

板块	内容
Hadoop · Introduction	大数据四 V、两个核心关切、Hadoop 的定义与四个核心组件、历史、里程碑、设计目标
Hadoop · Ecosystem	四个核心模块 + 九个相关项目
MapReduce · Introduction	出身、收益、Google 的用途、典型应用
MapReduce · Motivations	词频统计算例、单机瓶颈、丢失更新、加锁代价、把计算搬到数据旁、分治法
MapReduce · Technical Ideas	并行编程的六类难题、负载均衡、MapReduce 隐藏了什么、三条设计立场、工作流全景
**MapReduce ** · Workflow and Programming	Map / Reduce 抽象、伪代码逐行解读、数据分区、流水线、可解问题、使用步骤、局限

part 1 Hadoop Introduction

1.1 background

大数据4V：Volume, Velocity, Variety, Veracity（体量、速度、多样性、真实性）;

数据经济：We are in a data/knowledge economy 我们处在数据/知识经济之中;

1. 数据是资产：Data is an important asset to any organization 数据对任何组织都是重要资产，需要存储。
2. 知识发现 Knowledge Discovery：基因组（速度）、奥巴马竞选（个体化）、Netflix 纸牌屋（内容决策）

1.2 Two Main Concerns

How to process such a huge amount of data in a distributed system?

处理：并行度与同步——Hadoop MapReduce

How to store such a huge amount of data in a distributed system?

存储：容量与可靠性——Hadoop Distributed File System (HDFS)

1.3 Definition, Goal and Components

1.3.1 Definition

1. **Hadoop**: an open-source software framework that supports **data-intensive distributed applications**, licensed under the **Apache v2** license.

data-intensive：数据密集而不是计算密集，所以瓶颈在 I/O 与存储，不在 CPU，把计算搬到数据旁边。

Apache v2 license**：**允许商用、允许修改与再分发。Hadoop 低成本的法律基础。

2. **HDFS **：A distributed file system that runs on large clusters of commodity machines. 运行在大型商用机器集群上的分布式文件系统。
3. **YARN **：A framework for job scheduling and cluster resource management. 作业调度与集群资源管理的框架。

1.3.2 Goal

Goals: to solve the problems about process and storage of huge amount of data解决.大规模数据的处理与存储问题。

1. 抽象并简化存储与处理

Abstract and facilitate the storage and processing of large and/or rapidly growing data sets. 抽象并简化大型（体量大）以及快速增长（在线扩容）的数据集的存储与处理。

2. 两类数据都要支持结构化与**非结构化**数据。这是 HDFS 与 HBase 并存的原因。
3. **简单的编程模型 Simple programming models**。
4. 高可扩展与高可用 **High scalability and availability** 。

open-source implementation of frameworks for reliable, scalable, distributed computing and data storage. 可靠（reliable）、可扩展（scalable）、分布式（distributed）的计算与数据存储。

5. 便宜、冗余少的商用硬件 **Use commodity hardware with little redundancy**

a flexible and highly-available architecture for large scale computation and data processing in a network of commodity hardware. 硬件前提: 面向商用硬件网络的、灵活且高可用的架构。

6. 容错 **Fault-tolerance**

这是用廉价硬件的必要补偿：硬件不可靠，软件必须可靠。

7. 把计算搬向数据 **Move computation rather than data**

移动计算而不是移动数据。数据量大、移动成本高，所以把程序送到数据所在节点执行。

因果链	内容
成本 → 容错	用 **commodity (cheap!) hardware with little redundancy** ⇒ 硬件会坏 ⇒ 必须有 **Fault-tolerance** ⇒ 由软件层（HDFS 副本 + 任务重试）承担
数据大 → 移动计算	**data-intensive** 且数据**rapidly growing** ⇒ 移动数据的 I/O 成本极高 ⇒ 必须 **Move computation rather than data** ⇒ 把操作搬到数据旁

1.3.3 Components

组件	职责	解决的问题
Hadoop MapReduce	a system for parallel processing datasets 并行处理数据集的系统	处理
Hadoop Distributed File System	a system for distributed dataset storage 分布式数据集存储系统	存储
Yet Another Resource Negotiator（Hadoop YARN）	a framework for job scheduling and cluster resource management 作业调度框架和集群资源管理	支撑前两者
Hadoop Common	Common utilities that support Hadoop 支撑 Hadoop 的通用工具	支撑前三者

1.4 Hadoop and MapReduce

1. MapReduce是一个计算模型（a Computational Model/Program）

2. Apache Hadoop是MapReduce这一思想的开源实现（an open-source implementation of the MapReduce idea）

MapReduce这个思想不只Hadoop一个实现，它是 Google 提出的编程模型；Hadoop 只是Apache基金会把它工程化、开源化的产物。It is not a program, but an idea。

1.5 Question and founders

1. The Question: "How to process big data with reasonable cost and time?" 如何以合理的成本与时间处理大数据?

传统方案用**高端服务器**换时间（scale up），成本极高；

Hadoop选择用**大量廉价机器**换时间（scale out），把可靠性问题交给软件层（fault-tolerance）。

2. Google search engines

1998年的Google页面复杂，但2013年的Google界面极简，全部复杂度转移到后端。15 年间前端极简化、后端能力膨胀。用户看到的越简单，背后的分布式计算需求越大，所以需要Hadoop。

3. 从论文到 Apache

Nutch（2002，Doug Cutting + Mike Cafarella）→ GoogleFileSystem论文（2003，存）→ MapReduce论文（2004，算）→ Yahoo 资助 → 2008 移交 Apache

1.6 Google Origins

2003 存：GFS论文 → HDFS。HDFS 不是 Google 的实现，而是这篇论文的开源复现。

2004 算：MapReduce论文 → Hadoop MapReduce。这是本讲的主角。

2006 结构化存储：Bigtable 论文 → Apache HBase。解决结构化数据的分布式存储。

part 2 Hadoop Economy

2.1 Four Hadoop Core Modules

模块	课件定义原文	关键词
Hadoop Common	A set of components and interfaces for distributed file systems and general I/O	components and interfaces（组件与接口）、general I/O（通用 I/O）
Hadoop Distributed File System (HDFS)	A distributed file system that runs on large clusters of commodity machines	runs on large clusters（跑在集群上）、commodity machines（商用机器）
Hadoop MapReduce	A distributed data processing model and execution environment that runs on large clusters of commodity machines	**model and execution environment**（模型**与执行环境**）
Hadoop YARN	A framework for job scheduling and cluster resource management	job scheduling（作业调度）、cluster resource management（集群资源管理）

2.2 Nine Hadoop-Related Projects

项目	课件定义原文	中文与归类
Ambari	A tool for provisioning, managing, and monitoring Hadoop clusters	集群**配置 / 管理 / 监控**工具 → 运维类
Avro	A serialization system for cross-language RPC, and persistent data storage	**跨语言 RPC** 的序列化系统、持久化数据存储 → 数据格式类
Cassandra	A scalable multi-master database with no single points of failure	可扩展的 **multi-master** 数据库，**无单点故障** → NoSQL 类
Chukwa	A data collection system for managing large distributed systems	大型分布式系统的**数据采集**系统 → 采集类
HBase	A distributed, column-oriented database	分布式**列式**数据库 → 数据库类（对应 2006 Bigtable）
Hive	A distributed data warehouse provides a query language based on SQL	分布式**数据仓库**，提供基于 **SQL** 的查询语言 → 数仓类
Mahout	A Scalable machine learning and data mining library	可扩展的**机器学习与数据挖掘**库 → 算法类
Pig	A data flow language and execution environment for exploring huge datasets	**数据流语言**及其执行环境，用于探索海量数据集 → 语言类
ZooKeeper	A distributed, highly available coordination service	分布式**高可用协调服务** → 协调类

Tips:

1. **HBase vs Hive：HBase 是数据库**（列式，直接存取行/列）；Hive 是**数据仓库**（用类 SQL 语言做批式查询）。一个是存储引擎，一个是查询层。
2. **Pig vs Hive：两者都是用更简单的语言写MapReduce，但 Pig 是数据流语言**（过程式），Hive 是**类 SQL**（声明式）。
3. **Cassandra vs HBase：**Cassandra 的特点是multi-master + no single points of failure（无中心节点，任一节点可写）；HBase的定位是 column-oriented（列式）。

part 3 MapReduce Introduction

3.1 Definition

1. MapReduce is a programming model Google has used successfully for processing its "big-data" sets. 它是一个Google 成功用于处理其大数据集的编程模型。
2. A framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. 它是一个允许用简单编程模型在计算机集群间分布式处理大数据集的框架。
3. It is not a program, but an idea. MapReduce 是方法论层面的东西，不是一份可以下载运行的软件。
4. Hadoop MapReduce is an implementation of MapReduce. Hadoop MapReduce 是它的实现。

MapReduce 是什么？它是 Google 提出的编程模型（不是程序），是允许在计算机集群上分布式处理大数据集的框架；Hadoop MapReduce 是它的开源实现，其底层运行时自动完成并行化、容错与通信。

3.2 Function

课件原文	含义	对应 Part IV
The underlying runtime system automatically parallelizes the computation across large-scale clusters of machines	底层运行时**自动**在大型集群上把计算并行化	automatic parallelization and distribution
The underlying system also handles machine failures	运行时负责**处理机器故障**	fault-tolerance、assume failures are common
…efficient communications, and performance issues	以及**高效通信**与**性能问题**	网络与磁盘传输优化

**Tips: **

1. 自动化：这些在传统并行编程里必须手写的部分（调度、同步、容错、通信），MapReduce 全部自动化。
2. runtime system：它说明MapReduce包括一个在背后干活的系统（runtime）。用户只写Map和Reduce两个函数，剩下的交给runtime。

3.3 Benefits

含义	机制
突破存储与计算的传统限制	把两者都拆到集群上
以廉价商用硬件为平台	commodity (cheap!) hardware
提供从1到5000台服务器的线性可扩展性（scale out）	scale out：加机器即扩容
低成本、开源软件	Apache v2 license

Tips:

linear scalability线性可扩展性：**性能随机器数量近似线性提升，**2 倍机器约 2 倍吞吐。这个性质的前提是任务之间足够独立（**数据量大而处理独立：索引、聚类、报表、影像、语言模型、备份，每一项都是逐条/逐片处理**，条目之间不需要互相等待），这正是MapReduce Pipeline要处理的问题（限制进程间通信量）。

3.4 Scale Out & Scale Up

维度	scale out（横向扩展）	scale up（纵向扩展）
做法	增加**机器数量**（1 → 5000 台）	提升**单机性能**（换更强的服务器）
成本曲线	线性、可用廉价硬件	非线性、须高端专有硬件
上限	理论上可持续加机器	受单机物理上限限制
原文	**A large number of commodity low-end servers is preferred over a small number of high-end servers**	—
代价	必须解决**故障**与**通信**问题	成本高

3.5 Application

应用	中文	为什么适合 MapReduce
distributed pattern-based searching	分布式模式检索	逐条记录独立匹配模式，天然可并行
distributed sorting	分布式排序	分片局部排序 + 归并（Shuffle and Sort 的核心操作）；也是2008基准测试的题目
web access log stats	网站访问日志统计	日志逐行处理 + 按 key 聚合（与词频统计同构）
inverted index construction	倒排索引构建	词 → 文档列表的聚合，Map 输出 (词, 文档)，Reduce 汇总（**典型的两阶段结构**）
document clustering	文档聚类	Google News
statistical machine translation	统计机器翻译	需要大规模语料上的n-gram统计

**Tips: **

1. **倒排索引为什么是标准范例：**它是**结构上最贴合 MapReduce 的问题，**Map 阶段逐文档产出 (term, docID) 对，Shuffle 按 term 分组，Reduce 阶段把同一 term 的所有 docID 合成列表。与词频统计（Map 出 (word,1)、Reduce 求和）只差累加与收集的区别。

part 4 MapReduce Technical Ideas

本节思路：100 PB 装不下 → 多线程只解决效率 （efficiency）→ 跨机器并行会丢更新（lose update）→ 加锁代价高 （lock solution）→ 改用局部状态 + 最后合并（MapReduce机制和分治法）→ 这就是 MapReduce 的起点

4.1 Example：Word Count

问题：为什么统计一堆文档里面的词频单机做不了，MapReduce能做?

MapReduce 能默写 输入：文档集合；输出：<term, frequency> 列表

1. 结构最简：只需「拆分 → 计数 → 合并」三步，没有迭代、没有共享状态、没有先后依赖。典型解题四步正是从这里抽象出来的。
2. 可扩展性最强：把 100 PB 文档切成 N 片，每片独立计数，最后把 N 张局部表按词相加即可，这就是 Separated data / Separated counters图的全部内容。
3. 与其他任务同构：倒排索引、访问日志统计、n-gram 统计都是同一个形状，因此这个词频统计的解法可以原样迁移。

4.2 Other Applications

应用	是什么	与词频表的关系
Word cloud 词云	按词频决定字号的可视化	**直接**用频次作字号权重
Inverted index 倒排索引	从文档 → 词翻转为词 → 文档列表	把频次换成文档 id 列表，聚合结构相同
TF-IDF	词频 × 逆文档频率，衡量词对文档的代表性	**TF 就是词频**；IDF 需要该词出现在多少篇文档中，这也是一次按词聚合

应用	是什么	与词频表的关系
Google Trend	热门查询趋势报表	与Google 用例直接对应，是查询日志的词频统计

1. 四个应用的共同点：都需要先按词聚合。词频表就是这个聚合的结果，因此它是这些任务的公共前置步骤。
2. TF-IDF 的特殊地位：两个因子（TF 与 IDF）都能由一次 MapReduce 式的按词聚合算出。
3. 倒排索引的计算量：在 Google 的规模下，倒排索引的构建正是 MapReduce 的发源场景。

Lecture2 中的 TF-IDF 与余弦相似度是用已经统计好的词频向量算的；本章说明那些词频向量本身也需要一个可扩展的方式来算。

4.3 ** **Three Single Machine Problems

词频统计在单机上的三个瓶颈及MapReduce的解决：

瓶颈	具体含义	MapReduce 的解法
Efficiency issue 效率问题	单机串行扫描 100 PB 需要天文数字的时间；CPU 与磁盘带宽是上限	**并行化：**多 worker 同时处理不同分片
Storage issue 存储问题	一台机器装不下 100 PB；即使装下也无法可靠维护	**HDFS：**切块 + 多副本存在集群上
Memory issue 内存问题	词表本身可能大到装不进内存（不同词的个数也可以是上亿级）	**局部聚合：**每个 worker 只维护自己分片的局部计数表，最后再合并

三个问题的独立性：三者不能互相替代，加 CPU 不能解决存储（数据还是装不下）；加磁盘不能解决效率（单机 I/O 带宽有限）；加内存不能解决前者（100 PB 远超任何单机内存上限）。必须同时换三套方案。

4.4 For Efficiency Issues

4.4.1 Naive Solution by Multiple Threading

用多线程来应对效率问题（注意：本质是单机内的优化，所以只提效率，不能扩容存储与内存，后面必须走分布式）。

朴素方案：一份 Data collection 输入，连接到n 个计数器线程并行处理。单核串行变成多核并行，CPU 利用率提升，**效率问题的单机部分**被缓解。

弊端：1. 数据仍要**全部在一台机器上**（存储问题未解决）；2. 所有线程**读同一份数据**，I/O 带宽成为新瓶颈；3. 各线程若共享一张全局计数表，就埋下了丢失更新的隐患。

4.4.2 The Worker: Generalize to a larger scope

Worker: a basic instance to undertake the tasks. 承担任务的基本执行单元 · A worker can be a thread or a processor. · Workers can be in a single machine or multiple machines.

Worker 的实现形态	所在位置	对应场景
一个**线程**（thread）	单台机器内	多线程方案
一个**处理器**（processor）	多台机器上	真正的分布式计算

Tips: 实际上，所有并发问题（同步、共享状态）在单机与多机上是同一类问题，不必重新发明一套问题描述。

4.4.3 Distributed Computing

课件原文	含义
A cluster of machines/nodes 一个机器/节点集群	分布式计算的最小前提：多台机器组成集群
Data is **split** into several parts and **stored** in distributed machines 数据被切成若干部分存放在分布式机器上	数据**先切分再分布**（这就是 HDFS 的角色）
Multiple workers work together simultaneously 多个 worker 同时协同工作	并行不是可选项，而是定义的一部分

Tips: This requires careful programming. Otherwise, we risk getting incorrect results! 因为并行的风险不只是慢，而是**答案****错，并行与正确性之间存在张力。**并行计算不会自动得到正确结果，需要谨慎的编程（careful programming）**

4.4.4 Lost Update

4.4.4.1 Lost Update Problem

为什么并行的风险是错呢?

- **场景描述**：介绍了一个全局数组“count”，存储在中心节点，并可由两个工作者访问。用红色文本强调潜在错误。假设两个工作者正在处理相同的术语“apple”。指出这个问题不仅存在于分布式系统中，也适用于本地多线程。
- 一个标记为“工作者”的框中包含5步伪代码算法：“读-改-写”循环
- **时间线**：**工作者1（蓝色柱）**：执行“读”操作，然后进行“写”操作；**工作者2（粉色柱）**：在工作者1的读取后执行“读取”，在工作者1的写入后进行“写入”。
- **结果**：数据库中的最终值为1，正确应为2，一个增量操作被覆盖，实际丢失了。
- **原因：**Worker 2 的那一次读发生得太早，读到了0，写发生得太晚。**根因是「读-改-写」不是原子操作，**中间有可被交错的时间窗口。
- **为什么两个 worker 同时处理同一个词：这正是词频统计的常态。因此这个冲突不是极端情况，而是必然发生。**

4.4.4.2 Locking-based Solution

又如何解决呢?

A locking protocol may be used to ensure correctness. A worker must hold a lock on "count[t]" before using Read/Write.

- **执行调度表**：将“工作者1”和“工作者2”分成两列。
- **工作者1列**：“Lock (count["apple"])” & “Read (count["apple"])”
- **工作者2列**：显示工作者1持有锁时的情况：
    1. “Wait for Lock(count["apple"])”用红色文本书写。
    2. 一个省略号（“...”）表示工作者2被阻塞并在等待。
- **锁机制的工作原理：**当工作者1获得对特定资源（在此为“apple”的计数）的锁时，工作者2必须等待该锁被释放，这也体现了上文提到的“降低并行性”的概念。

- **缺点**：1. 复杂性增加：实现代码变得复杂。2. 运行时开销高：导致“降低并行性”。

4.4.4.3 The Cost of Solution

加锁这种方式的弊端

代价	含义	后果
Complicate implementation	使实现变复杂	程序员必须自己管理锁的获取、释放、异常路径，这与Simple programming models（只写map和reduce代码）目标背道而驰
Expensive runtime overhead: reduced parallelism	运行开销昂贵：并行度下降	这是更致命的一条，为了正确性付出的是性能，而性能正是当初并行的目的

1. 排队意味着串行化，并行度直接掉下来。
2. **锁的粒度问题：**若锁的粒度是单个词，不同词的worker仍可并行；但若共享表很大，锁管理的开销本身也会成为负担。
3. **与三个瓶颈的关系：**加锁只解决正确性问题，对 Storage issue 与 Memory issue 完全无解，还额外拖慢了效率。

4.4.5 The Core Idea of MapReduce

Q：How to ensure correctness without or with less usage of locking?

加锁有很多问题，如何少用or不用锁呢?

A：引入Core Idea：move operations near the data

把操作移向数据，再用局部状态代替共享状态。把加锁的全局表换成各写各的局部表最后并行。

1. Let each worker use its local storage (hard disk) to store a local array "count"

每个 worker 有一份**自己的**局部计数表（存在本机磁盘上）

2. Each worker can only update its local array

worker **只写自己的表**，绝不碰别人的

3. When workers finish their tasks, they send their local arrays to the central node

全部完成后，把局部表**上交中心节点**

Cost：引入了一次**额外的通信**（上交局部表）与一次**额外的合并步骤**（把局部表相加）。这个代价可接受，因为合并发生在**所有 worker 完成后**，可以批处理，不打断并行。

4.5 For Memory and Storage Issues

4.5.1 Example: Peta-Scale Data

Peta-scale data is commonly distributed from data that can be easily stored or processed on a single machine.

大尺度数据通常来自那些可以在单台机器上轻松存储或处理的数据

主题	与本节的关系
两个核心问题（处理和存储）	提出单机服务不了全部数据

主题	与本节的关系
100 PetaBytes + 三个瓶颈	给出具体体量与三条困难
集群与数据切分	给出物理形态（机器集群 + 切分后的数据）
Peta 级数据天然分布式（本节）	把分布式从方案确认为前提
分治法 Divide and Conquer	给出处理前提（分布式）下的方法论

Peta 级数据无法在单机上存储/处理，因此必须是分布式的。

4.5.2 Divide and Conquer 分治法

方法论层面的答案：**分治法**

标签	含义	MapReduce 术语	阶段
Partition	把一个巨大工作拆成三份子工作，分给三个 worker 各自计算，得到三个局部结果，最后合成 Result	输入切分（Split）+ 任务分配	Map 之前（由运行时完成）
各 worker 独立计算		Map + 本地聚合	Map 阶段
（中间的通信）		Shuffle and Sort	Map 之后、Reduce 之前
Combine	把分与合正式命名为两个阶段	Reduce	Reduce 合并阶段

1. **split 与 merge 成对出现：**每条边上都有 **split / merge**（split 用粉色箭头向下，merge 用蓝色箭头向上），在后面的内容中把这一页的两条边正式命名为 **Partition**（对应 split）与 **Combine**（对应 merge）。
2. **递归结构：从 1 个 Problem 到 2 个 Subproblem 再到 4 个 Compute Subproblem，是逐层二分。**实际的 MapReduce 分多少层都可以。
3. **compute 的位置：计算发生在叶层**（Compute Subproblem），中间层只负责**拆与合**。这对应 MapReduce 的两个阶段：Map 在叶层干活，Reduce 负责合并。

分治法与前面各节的关系

前面讲的问题	分治法给出的答复
数据装不下	把 Problem **split** 成多个 Subproblem，每个都能放进一台机器
两个 worker 会冲突	让各 Subproblem **独立计算**，不共享可写状态
加锁拖慢	只在 **merge** 阶段通信，中间过程无需同步
结果怎么合	**merge** 就是把各局部结果合并的一步

Tips: 分治法不保证正确：分治法给出的是结构，不是正确性保证。任务怎么分给 worker？部分结果怎么合并？worker 死了怎么办？这些问题分治法自身并不回答。

part 5 MapReduce Workflow and Programming

分治法（Partition/Combine）→ 六类难题 → 负载均衡 → 自动化与三条设计立场 → 五段工作流 → 四步解题范式

5.1 Basic Workflow of MapReduce

5.1.1 Input and Output

1. 输入是一个目录或数据集，不是单个文件：Peta-scale data collection意味着输入本身就是一个许多文件的集合，切分粒度在运行时决定。

2. 输出是<key, value> 对的列表：词频表“hadoop 1”就是 key=hadoop、value=1。这个形态是Map与Reduce的接口。

3. 输入输出是一对多与多对一的关系：合并步骤的通信量远小于输入数据量，输入很大（Peta 级）、输出很小（一张词频表）。The output usually contains a few pairs compared to input。

5.1.2 DIvide and Conquer

MapReduce的最简形式：Split → Parser → Counter → 汇总

有Separated data and counters 独立的数据和计数器（数据分治+局部数组）

级	节点	职责	MapReduce 对应
第 1 级	**Split 1 ⋯ Split n**	把数据切成 n 份（分开的数据）	**Split：**输入分片
第 2 级	**Parser 1 ⋯ Parser n**	逐份解析（切词）	**Map：**把记录变换成 <key, value> 对
第 3 级	**Counter 1 ⋯ Counter n**	各自统计本份的局部计数（分开的计数器）	**Map 的本地状态 / Combine：**局部聚合
汇总	（虚线箭头汇入右侧一张表）	把 n 张局部表合并	**Reduce：**按 key 合并

**Tips: **

1. **Parser 与 Counter 完全可并行：**第 i 份数据的解析与计数只依赖第 i 份数据，与其他份无关，任务之间足够独立（linear scalability的前提）。
2. **汇总必须等待：**右侧那张表要等所有 Counter 都完成才能给出最终结果，**All Map should be completed before Reduce starts**（所有 Map 必须在 Reduce 开始前完成）。

5.2 Challenges

给出一个具体的集群规格，在这样的集群上写程序要考虑什么？这些问题正是 MapReduce 运行时系统要自动处理的清单。

1. How do we assign work tasks to workers?

怎么把工作分配给 worker？调度问题。

2. What if we have more work tasks than workers?

任务比 worker 多怎么办？需要任务队列与排队机制。

3. What if workers need to share partial results?

worker 之间需要交换部分结果怎么办？通信问题。

4. How do we aggregate partial results?

怎么汇总部分结果？合并问题。

5. How do we know all the workers have finished?

怎么知道所有 worker 都完成了？同步/终止检测问题。

6. What if workers die?worker

死了怎么办？容错问题（Assume failures are common）。

类别	对应问题	MapReduce 的处理
调度与负载	1、2	**5.4 load balancing**；任务数可配置
通信与同步	3、5	**Limit the amount of communication**；Map 全部完成后才进 Reduce
结果合并	4	**Shuffle and Sort + Reduce**
容错	6	**handling machine failures；Assume failures are common**

5.3 Managing Multiple Workers

不知道 worker **运行的顺序**。不知道 worker **何时互相打断**。不知道 worker **何时需要交换部分结果**。不知道 worker **访问共享数据的顺序**。（5.4补充工作量不可知）

4.4.2 节把 Worker 抽象为一个执行单元；这一节说明这种执行单元的**时间行为不可预测**，这正是管理困难的全部来源。

工具	操作	用途
Semaphores 信号量	Semaphores (lock, unlock)	互斥：保证同一时刻只有一个 worker 进入临界区
Conditional variables 条件变量	Conditional variables (wait, notify, broadcast)	同步：让 worker 在条件不满足时等待，条件成立时被唤醒

broadcast 的含义：notify 唤醒一个等待者，broadcast 唤醒所有等待者。

5.3.1 工具管理共享状态后仍然存在的问题

Still, lots of problems: e.g., Deadlock. Cause of problems with resource access: They need to access shared resources.

死锁 Deadlock：加锁后多个 worker 互相等待对方持有的锁，谁也无法推进。

根因：They need to access shared resources 问题的根源是**worker仍然需要访问共享资源**。所以改用局部状态（如果不共享，这些工具与问题都不需要）。

5.4 Load Balancing

MapReduce的**调度目标**：让所有 worker 同时结束

Q 问题：如何优化worker运行和等待访问共享资源？原方案里Worker2承担了两个大任务，成为最慢的一环。总时间由最慢的worker决定，空闲的算力被浪费。

A 改进的办法：把Task4从Worker2移到Worker3，三条时间线的长度变得接近，最慢的那条变短，总时间随之缩短。图右侧用两条竖直红线标出Time Saved。

P 前提条件：这种移动任务只有在**任务之间互相独立**时才可能，与5.1.2节Split / Parser / Counter 同一前提。

Tips：任务的实际耗时取决于数据量，程序员事先无法精确预知。

所以Load balancing直接列为MapReduce运行时替用户完成的工作之一，因为手工做这件事需要预知任务耗时，而在集群上这是做不到的，所以MapReduce把这个自动化5.4.1。

5.4 Problem Types

类别	课件原文清单	解读
Fundamental issues 基本问题	Scheduling, data distribution, synchronization, inter-process communication, robustness, fault tolerance, …	调度、数据分布、同步、进程间通信、健壮性、容错——**与前两节的六个问题同源**
Architectural issues 体系结构问题	Flynn's taxonomy (SIMD, MIMD, etc.), network topology, bisection bandwidth, cache coherence, …	Flynn 分类（SIMD/MIMD）、网络拓扑、对分带宽、缓存一致性——**偏硬件与体系结构**
Common problems 常见故障	Livelock, deadlock, data starvation, priority inversion, … dining philosophers, sleeping barbers, cigarette smokers, … Load balancing,…	活锁、死锁、数据饥饿、优先级反转；以及经典的哲学家就餐、理发师睡觉、吸烟者问题——**并发编程的经典病**
Different programming constructs 编程构件	Mutexes, conditional variables, barriers, … masters/slaves, producers/consumers, work queues, …	互斥量、条件变量、屏障；以及主从模式、生产者/消费者、工作队列——**实现这些机制的工具与模式**

拓展：

Flynn 分类的两个缩写

1. SIMD：Single Instruction Multiple Data 单指令多数据。同一条指令作用在一批数据上（如 GPU 的向量运算）。
2. MIMD：Multiple Instruction Multiple Data 多指令多数据。各处理单元执行不同的指令流（Hadoop 集群属于这一类）。

5.4.1 Reason and Solution

**Problems arise from: **Access to shared resources (e.g., data), Communication between workers (e.g., to exchange state).

两个根因：**访问共享资源；worker 之间需要通信。**

**MapReduce: **Automate most of these for you.

MapReduce 把这些大部分自动化，把复杂性从程序员转移到运行时。从而实现了Hadoop的目标之一：Simple programming models。

注意是most而不是 all：MapReduce 不解决所有并行编程问题，只自动化了它面向的那一类（批式、读多写少的大数据处理）。

5.5 MapReduce Features

Hide messy details in MapReduce **runtime library**

关键是 **runtime library** 复杂性被搬进**运行时库**，因此用户只需写Map与Reduce函数。

Features	中文	对应5.2的问题
automatic parallelization and distribution	自动**并行**化与**分发**	怎么把任务分给 worker（问题 1）
load balancing	负载均衡	Load imbalance（问题2）

Features	中文	对应5.2的问题
network and disk transfer optimization	**网络与磁盘**传输优化	worker之间怎么交换结果（问题 3）+ 节的通信虚线
handling machine failures	机器故障处理	worker 死了/终止怎么办（问题 5，6）+ 节的容错
inherently designed for batch and high throughput processing jobs	天生面向批式与高吞吐的作业	说明它的适用范围，不是通用并行框架 Note that MapReduce is not designed to solve all kinds of problems. 它是**有适用边界**的框架

三位一体：**scale out → failures common → co-location**

1. Scale "out", not "up" **横向扩展**而不是纵向扩展。

A large number of commodity low-end servers is preferred over a small number of high-end servers. 更倾向于使用大量低端的商品服务器，而不是少量高端服务器。

out 必然导致 failures：服务器数量一多，按概率就有机器在坏。选择廉价硬件就是选择接受故障，所以假设故障是常态。

2. Assume failures are common 假设故障是常态。

Fault-tolerant service must cope with failures **without impacting the quality of service. **容错服务必须能够应对故障，而不影响服务质量。

failures 必须靠 co-location 缓解：如果计算与存储分离，机器一坏就要跨网络重传数据，代价极高；把 processors与storage放在一起，坏了一台只需把它的任务在别处就地重跑（因为那里也有同一份数据的副本，HDFS备份机制）。

3. Moving processing to data 把处理移向数据。

MapReduce assumes an architecture where processors and storage are co-located. MapReduce 假设处理器和存储是共址的架构。

processors and storage are co-located：前提是MapReduce 假设每一台机器既是数据节点也是计算节点，这是计算移到数据的物理实现，并与**HDFS配合保证数据在本地备份**，使 co-location 的假设在发生故障后依然成立。

「without impacting the quality of service」的分量 ・容错不只是「不崩」，而是服务质量不下降——用户不应感知到某台机器坏了。这是一个较强的要求，靠的是副本与任务重试。

5.6 MapReduce Workflow

Split → Map（local write）→ Shuffle and Sort（remote read, sort）→ Reduce（write）→ Output File

1. Map的定义

Map: a mapping that is responsible for dividing the data（划分数据） and transforming the original data into key-value pairs（把原始数据变换成键值对）.

2. Shuffle and Sort 的定义

Shuffle and Sort: the process of further organizing（组织） and delivering（传递） the Map output to the Reduce.

The output of the Map must be sorted（排序） and segmented（分段/分组）,

then passed（传递） to the corresponding Reduce

Communication occurs（唯一的跨网络通信发生）.

3. Reduce 的定义

Reduce: a merge（合并） that processes the **values** with the same key and then outputs to the final result.

段	元素	说明
Input Data	Split 0 / Split 1 / Split 2 三个红框，带一条 read 箭头	输入数据被切成三片；read表示本地读取
Map	extract something you care about from each record 从记录提取需要的信息	3个worker 并行做 Map；输出本地写出local write（不跨网络）
Shuffle	remote read, sort	跨网络读取各 Map 的输出并排序，这是唯一的跨节点通信段
Reduce	aggregate, summarize, filter, or transform 聚合、汇总、过滤或变换	2个worker做 Reduce，把结果写出
Output Data	Output File 0 / Output File 1	Reduce 的输出文件；两个 Reduce ⇒ 两个输出文件

Tips:

1. local write与remote read： Map 的输出是本地写（local write），Shuffle 是远程读（remote read）。跨网络的数据移动只发生在 Shuffle 这一步。
2. Map 任务数与 Reduce 任务数可以不同。
3. Shuffle 的两个动作：remote read（去各 Map 节点把属于自己 key 的数据取回来）+ sort（把取回的数据按 key 排序，使同 key 的值相邻）。这两个动作合称 Shuffle and Sort，即将映射的数据进行组织和分组。
4. Map 是从记录中抽出（record → 若干键值对，一对多）；Reduce 是聚合/汇总/过滤/变换（同 key 的值集合 → 结果，多对一）。方向相反。

5.7 Typical Problem Solving by MapReduce

Read（读文档集）→ Map（每条记录产出 (term, docID)）→ Shuffle（按 term 分组）→ Reduce（收集同 term 的 docID 成列表）→ Write

5.7.1 MapReduce 单词计数过程

如何通过 MapReduce 计算三个文件中单词的频率。

- **流程图:**
  - **输入:** 来自三个文件的原始文本。
  - **Spliting 分割:** 文件被分成独立的块。
  - **Mapping 映射:** 每个块被拆分为键值对。
  - **Shulffing and Sorting 洗牌与排序:** 这些对被重组，使相同的键聚集在一起。
  - **Reducing 归约:** 对每个键的值进行求和（例如，“Bear, 2”）。
  - **最终结果:** 每个单词的汇总计数。

5.7.2 Map抽象

- **输入:** 接受一个键值对，其中键引用输入值，值是待操作的数据集。
- **评估:** 对每个输入值应用用户定义的函数，可能需要解析。
- **输出:** 生成一组新的键值对，其数据类型可以与输入不同。

5.7.3 Reduce抽象

- **输入:** 接受中间的键值对。
- **过程:** 系统在调用“reduce”函数之前执行“洗牌和排序”。所有共享相同键的对被分组在一起，迭代器将这些值提供给函数。
- **功能:** 其目的是聚合、汇总、过滤或转换数据。
- **输出:** 生成最终的键值对。输出通常比输入要小得多，因为它对数据进行了压缩（例如，对每个特定关键词的值求和）。

5.7.4 编程模型

- **Map():** 程序员指定一个函数，该函数接受输入数据并生成一组中间的键值对。
- **Reduce():** 程序员指定一个函数，该函数接受中间的键值对并将其合并，以生成一组合并的输出值。

part 6 Tutorial: Hadoop

6.1 Pseudo-Code

MapReduce 模型中用于“词频统计”（WordCount）的两个核心函数的伪代码：

1. **Map 函数**：处理输入数据，将文本行拆分为单个单词（Token），并为每个单词输出一个键值对，键是单词，值是1。
2. **Reduce 函数**：汇总处理结果，接收每个单词对应的所有计数值，进行累加求和，输出该单词及其总出现次数。

6.2 Challenge and Solution

管理多个工作单元（Workers）时面临的挑战及相应的解决方案：

面临的挑战：工作单元的执行顺序不可预测，工作单元之间可能会相互干扰，难以确定工作单元何时需要交换中间结果，对共享数据的访问顺序不确定。

解决方案：限制工作单元立即访问共享数据，防止工作单元相互干扰，将操作移至数据附近执行，在工作单元完成后进行批量更新。

6.3 MapReduce: Data Partition

问题：数据在什么时候、由谁切分？

答案：在加载时就分好，由 HDFS 负责

数据分区与存储 (Data Partitioning & HDFS)：

事实	含义	解决的问题
加载时即分布	数据在写入时就已经分布在集群各节点上，不是处理时才切分	解释了为什么Map能本地读取
切成chunks	文件被切成块，每块由不同节点管理	决定了Split 1…Split n的颗粒度，数据切分大小由chunks决定
每块多副本	Each chunk is replicated across several machines	Assume failures are common 副本是容错的物理基础

6.3 MapReduce Pipeline

1. **限制通信**：旨在减少进程间的通信量。
2. **独立处理**：每个记录由 Mapper 独立处理，互不干扰。
3. **映射阶段 (Mapping)**：各节点对本地预加载的数据进行映射处理。
4. **洗牌阶段 (Shuffle)**：节点间交换 Mapper 生成的中间数据。
5. **归约阶段 (Reducing)**：Reducer 将不同 Mapper 的结果合并，生成最终输出并存储在本地。

6.4 Classes of Problems

MapReduce 的适用场景（可并行化问题）：基准测试：数据密集型计算，如排序；Google 应用：用于词频统计、AdWords、PageRank 和数据索引；简单算法：如 grep（文本搜索）、文本索引和反向索引；其他行业应用：贝叶斯分类、Facebook 的人口统计分析、金融服务分析以及天文学中的高斯分析。

6.5 MapReduce steps

阶段	课件原文	要点
判断适用性	Determine if the problem is parallelizable and solvable using MapReduce	两个条件：可并行 + 可用 MapReduce 解
检查数据形态	Is the data Write Once Read Many (WORM)?	WORM 数据一次写入、多次读取，是并行的前提（无需锁）
设计与实现	Design and implement Mapper class and Reducer class	两个函数
配置、加载与运行	Configure the application (job) as to the number of mappers and reducers (tasks), input and output streams；Load the data；Launch the job and monitor	任务数可配置；数据加载；提交并监控

6.6 Limitation and Summary of MapReduce

- **局限性**：

    1. 难以处理共享全局状态，依赖共享状态的算法在 MapReduce 中难以实现。

    Online learning：在线学习每来一个样本就更新一次模型参数，下一个样本的处理依赖上一次更新后的参数。这意味着：样本之间不独立；参数是共享可写状态。

    model parameters as shared global state：共享全局状态，只要有全局可变的模型，就无法用纯 MapReduce 表达。

    2. 不适合小数据集，该框架主要为大规模数据的批量处理而优化。

要处理小数据集，框架必须改造。原因是MapReduce 的整套机制（切分、分发、Shuffle、容错）都有固定开销，数据量小的时候这些开销占比过高。

batch 是设计目标而非缺陷。specially optimized for batch operations，这是它的设计选择。batch的另一面就是不擅长低延迟。

Lecture 5 HDFS and Spark

1. HDFS 动机：分布式系统的存储挑战 → 故障概率算式 → HDFS 的八条需求 → 文件系统定义与四个动机问题 → 两页设计目标。这是问题定义。
2. HDFS 技术细节（一）：块与副本、抽象拓扑、NameNode / DataNode / 2NN 的架构、两层物理部署、元数据与 FsImage + EditLog、DataNode 职责。这是静态结构。
3. HDFS 技术细节（二）：副本放置策略、机架与带宽、心跳与故障检测、复制引擎、2NN 的合并机制与恢复上限。这是容错机制。
4. HDFS 技术细节（三）：读操作七步、写操作七步、四个带宽层级、网络拓扑与距离定义、命令行接口。这是动态行为。
5. Hadoop 作业调度：JobTracker / TaskTracker、Master-Slave 架构、七步调度流程、十步执行流程、Map/Reduce 数量、负载均衡、冗余执行、局部性优化。这是任务派发。
6. Spark 动机：MapReduce 的五条短板、Spark 的六项能力、内存 vs 磁盘共享、Daytona Gray 排序竞赛、Spark 组件栈。这是换引擎的理由。
7. Spark 技术细节：RDD 三个字母的含义、Transformation vs Action、窄变换与宽变换（shuffle）、DataFrame 与血统、DAG 与 stage 划分、Action 的语义。这是新引擎的原理。
8. Spark 应用与边界：八类用例、五个公司案例、三条不适用场景、全讲总结。这是落地与边界。

Part 1 HDFS

1.1 Motivation

1.1.1 Big data storage: Challenging

1. Data volumes are massive 数据量巨大
2. Reliability of storing PBs of data is challenging 规模化存储的可靠性
3. All kinds of failures: Disk / Hardware / Network Failures 故障种类多（磁盘故障、硬件故障、网络故障）

1.1.2 Failures are Common

两条设计的前提：

1. Single machine cannot serve all the data（单机服务不了全部数据）
2. Failures are common in commodity hardware, **Failure is expected, rather than exceptional**（商用硬件上故障很常见，事故不是例外）

在这里假设了独立同分布的算式隐含每块盘独立以 0.01 的概率故障。真实系统里同机架、同批次的盘会相关失效，那时的概率会更高。

1.2 Why HDFS?

HDFS: a distributed（分布式：文件跨机器存放） file system（文件系统）to store files with fault-tolerance（自带容错） 一个带容错能力的分布式文件系统

原文	解读
Need to process **huge datasets on large clusters of computers**	要在**大规模机器集群**上处理超大数据集
Very **expensive to build reliability into each application**	**把可靠性做进每个应用里太贵，**所以可靠性要下沉到基础设施层
Nodes fail every day **Failure is expected, rather than exceptional** The number of nodes in a cluster is **not constant**	节点**每天都在坏** 故障是**预期之内**，不是例外 集群节点数是**变化的，**扩容、下线、故障替换都要能在线完成
Need a **common infrastructure** **Efficient, reliable, easy to use** **Open Source, Apache Licence**	需要一个**公共基础设施**，而不是每个应用各做一套 三个品质要求：高效、可靠、易用 开源、Apache 许可

1.3 File System

系统文件：In computing, file system (often abbreviated to fs) is a method and data structure that the operating system uses to control how data is stored and retrieved.

两个组成：文件系统 = method（方法） + data structure（数据结构）。前者是组织规则，后者是磁盘上的实际布局。

职责范围：由操作系统用来控制数据存（stored）与取（retrieved）的方式。所以文件系统本来就是一个 OS 概念。

HDFS 的特殊之处：HDFS 是同一个抽象（方法 + 数据结构、存 + 取）在多机上的实现，所以它叫 distributed file system，而不是换了一个全新的东西。

1.5 Motivation Question

四个动机问题与 HDFS 的对策

问题	HDFS 的对策	对应章节
Problem 1: Data is too big to store on one machine.	Store the data on multiple machines 把数据存到多台机器上	块与副本
Problem 2: High-end machines are too expensive	Run on commodity hardware 跑在商用硬件上	物理部署
Problem 3: Many commodity hardware will fail occasionally	intelligent enough to handle hardware failure 足够聪明能处理硬件故障	心跳与故障检测
Problem 4: How to handle data on failed hardware	data replication 数据复制	副本放置

1.6 Goals of HDFS

原文	解读
Very Large Distributed File System (Scalability) 10K nodes, 100 million files, 10PB	非常大的分布式文件系统，括号里点明对应的是可扩展性 三个具体刻度：1 万节点、1 亿文件、10 PB 数据
Assumes Commodity Hardware Files are replicated to handle hardware failure Detect failures and recover from them (Failure tolerant)	假设跑在商用硬件上 用副本应对硬件故障 检测 + 恢复两件事都要做，合起来才叫容错
Optimized for Batch Processing (High Throughput) Data locations exposed so that computations can move to where data resides Provides very high aggregate bandwidth	为批处理优化，追求高吞吐，不是低延迟 暴露数据位置，好让计算搬到数据所在处（局部性优化） 提供非常高的聚合带宽，聚合指整个集群合起来的总带宽，是靠多机并行 + 就近读得到的

Tips:

1. High Throughput 吞吐量（throughput）与延迟（latency）是一对权衡。批处理场景要的是单位时间处理的数据量，允许单个任务慢。
2. It is not for low latency access. 凡是要求毫秒级随机读写的场景（在线事务、交互式查询）都不该用HDFS。
3. WORM: Write Once, Read Many. 一次写入、多次读取，无需互斥锁。本讲把它升格为HDFS 的语义前提。。

Part 2 HDFS Technical Details

2.1 Data Organization: Blocks and Replicas

文件不是整体存放，而是被切成固定大小的块，每块再存多份副本。这两件事解释了 HDFS 的容量、容错与负载均衡。

原文	解读
HDFS is a block-structured file system	与普通文件系统最大的结构性差别是HDFS 是块结构的文件系统
Individual files are broken into blocks (chunks) of a fixed block size (64MB by default)	文件被切成固定大小的块，默认 64MB。固定大小让寻址变成算术，若块长可变，就必须维护一张索引表才能定位第 k 块。
Blocks are stored across a cluster of machines	块散在集群上 ≠ 块被切开：一个块整体存在一台机器上（不做条带化）。文件级才是分布式的，所以一台机器坏了丢的是若干整块，而不是每个文件的一小片。
Each copy of the block is called replica	每一份块的拷贝叫 replica（副本）

Block size and replicas are configurable.

Block size 与 Replicas 都可配，所以 64MB 与默认 3 副本都只是默认值，不是固定约束。

2.1.1 Why Block?

问题：既然已经有文件了，为什么还要多一层块？（为什么需要块抽象？）

1. 超出单盘容量：File can be larger than a single disk 文件可以比任何一块磁盘都大。把文件切成块后，各块落在不同机器上，文件总大小就不再受单盘限制。

2. 固定大小便于管理：Block is of fixed size, easy to manage and manipulate 块大小固定，于是管理与操作都变成等长单元的算术，无需处理变长边界。
3. 易复制且细粒度均衡：Easy to replicate and do more fine-grained load balancing 块是复制的基本单位，也是负载均衡的基本单位：某台机器忙，就把它的块搬到别处。

2.2 HDFS's Abstract Topology

抽象拓扑（只有角色与连线）

角色	职责
HDFS Client	发起操作的客户端；与 NameNode 之间标注 FS / namespace / meta ops
NameNode	接收客户端的文件系统 / 命名空间 / 元数据操作；向下辐射到所有 DataNode
Secondary NameNode	与 NameNode 之间是灰色双向箭头，标注 Namespace backup（命名空间备份）
DataNode	每个 DataNode 内部画了 3 个彩色小方块（表示持有的块）；向下连到磁盘图标

② 图上的两条横向说明带 ·

中间黄色横带：Heartbeats, balancing, replication etc.. 底部黄色横带：Nodes write to local disk

Heartbeats（心跳，存活检测）、balancing（均衡，搬块）、replication（复制，补副本）这三件事都在 NameNode 与 DataNode 之间发生。

Nodes write to local disk 数据写在本地磁盘上。这是Move computation rather than data能成立的前提：数据既然在本地盘，把计算送过来比把数据送出去便宜。

Data serving（DataNode → 磁盘，表示实际的数据服务）与 Namespace backup（NameNode → 2NN）。

考点：NameNode 只有一个（中心）、DataNode 有很多个（5 个是示意）、数据落在本地磁盘。

2.3 HDFS Architecture

组件：命名空间、NameNode（NN）、DataNode（DN）、Secondary NameNode（2NN），以及各自的语言与版本归属。

三处名词的层级：三者不构成主从复制关系。

NameNode（1 个） 管理命名空间；DataNode（N 个，通常一机一个） 存实际数据；Secondary NameNode（1 个） 帮 NameNode 合并元数据。

2.3.1 命名空间与 NameNode

HDFS exposes a file system namespace and allows user data to be stored in files.

命名空间：HDFS 对外暴露一个文件系统命名空间，用户数据以文件形式存放。接口仍然是我们熟悉的目录/文件。

The HDFS cluster consists of a NameNode (NN), a master node that manages the file system namespace and regulates access to files.

NameNode：NN 是主节点（master node），负责管理命名空间并管制对文件的访问（manage是有哪些文件，regulate是谁能动）。

2.3.2 DataNode

There are a number of DataNodes (DN), usually one per machine in a cluster. The DataNodes manage storage attached to the machines that they run on.

DataNode 数量、位置、管理范围：通常一台机器一个。每个 DN 只管自己这台机器上的存储(data locality)。

2.3.3 Secondary NameNode

原文	说明
Another node runs Secondary NameNode (2NN) to serve as a helper of NN	2NN 是 NameNode 的助手（helper）
Proposed in Hadoop 1.x	Hadoop 1.x 引入（Hadoop 2.x 换成 CheckpointNode）
NOT a complete backup of NN	不是 NameNode 的完整备份
Written in Java, it also supports Python and Ruby	主体 Java，另外支持 Python 与 Ruby

2NN 不是 NameNode 的完整备份（NOT a complete backup of NN），2NN 在合并 FsImage 与 EditLog，加速重启。

2.4 HDFS Physical Deployment

把架构图落到机房布局：两层交换机、机架、每架节点数，并给出 NN/DN/JT/TT 在机架上的实际分布。物理部署（机房、机架、交换机）：

两层架构

Aggregation switch（汇聚交换机）连到 Rack switch（机架交换机），每个机架交换机再连若干 Node（节点，每个带 Disk 图标）。

Ps. Nodes are commodity PCs（节点是商用 PC）；30-40 nodes/rack（每机架 30 - 40 个节点）。

机架内外的带宽差

同机架便宜、跨机架贵：Rack switch 内部（同机架）带宽高；Rack switch ↔ Aggregation switch（跨机架）带宽低。

机架	特殊角色	普通节点	说明
Rack 1	Name Node	DN + TT × 4	NameNode 与第一批数据节点同架
Rack 2	Job Tracker	DN + TT × 4	JobTracker（作业调度）在第 2 架
Rack 3	Secondary NN	DN + TT × 4	2NN 在第 3 架，刻意与 NameNode 分开 因为 NameNode 是单点故障，如果 2NN 与 NN 同架，一次机架级断电就同时失去两者。物理隔离是冗余的一部分。
Rack 4	Client	DN + TT × 4	客户端所在机架
Rack N	—	DN + TT × 5	省略号表示中间还有若干个同构机架

HDFS: NameNode, DataNode MapReduce: JobTracker, TaskTracker

World：向下连到两个顶层 switch，代表集群之外的外部网络。

2.4 NameNode

2.4.1 Functions of NameNode

1. Manages File System Namespace (maintains and manages the file system metadata): Maps a file name to a set of blocks; Maps a block to the DataNodes where it resides.

管理文件系统命名空间，即维护与管理元数据，两条映射，文件名 → 块集合的映射（一个文件由哪些块组成）；块 → 所在 DataNode 集合的映射（每块在哪几台机器上）

2. Cluster Configuration Management 集群配置管理（有哪些节点、机架归属等）
3. Replication Engine for Blocks 块的复制引擎（检测失败并补副本）

Tips：两条映射合起来才构成读一个文件的完整寻址链：文件名 → 块列表 → 每块的位置。

NameNode 不存数据：五项功能里没有一项是存文件内容。

为什么叫中心而不是主节点复制：2.3.1 把NN 叫 master node，但 HDFS 的 master 与一般主从复制的 master 不同，它管的是元数据，不是数据本身。

2.4.2 Types of metadata

1. List of files 文件清单（有哪些、在哪里）

2. List of Blocks for each file 每个文件的块清单

3. List of DataNodes for each block 每个块的 DataNode 清单

4. File attributes, e.g., creation time, replication factor 文件属性，如创建时间、副本因子等属性。

2.4.3 Metadata in Memory 元数据放内存

The entire metadata is in main memory. Memory read/write is faster than disk read/write

为什么放内存：内存读写比磁盘快。元数据每次操作都要查，放磁盘会成为瓶颈。

代价：Data in memory will be lost if the machine shuts down 机器一关，内存里的元数据全丢。

于是需要持久化：In addition to metadata in memory, we need a persistent copy of metadata 所以除了内存中的元数据，还需要一份持久化副本。所以 引入FsImage 与 EditLog。

2.4.3.1 FsImage and EditLog

内存元数据怎么落盘。答案是一对文件——只读的快照 FsImage + 记录增量的 EditLog，以及「重启时合并」这一套固定流程。

文件	功能	性质	大小
FsImage	The persistent record of the metadata stored on the local disk is called FsImage	只读（Read-only）的元数据持久化记录	随集群增长，大
EditLog	Records modifications to FsImage — file creations, file deletions, etc.	记录对 FsImage 的修改（建文件、删文件等）本质是所有会改变元数据的操作。	Stored in disk. (smaller)，较小

分工：FsImage 本身不是备份，它是一份截至某时刻的元数据快照；EditLog 也不是备份，它是快照之后的增量。只有两者合起来（FsImage + EditLog）才等于当前元数据状态。

Metadata in memory will be modified as well 内存元数据也会同步改：内存里的那份会同步更新，所以服务期间的查询走内存，不走磁盘。

2.4.3.2 Current status and Restart

current status: FsImage + EditLog Restart 1. NameNode loads FsImage in memory, applies modifications logged into EditLog, and generates a new FsImage and empty EditLog to replace the old ones. 2. Start service.

1. 回放合并：把 FsImage 载入内存 → 逐条应用 EditLog 的修改 → 生成新的 FsImage 与一个空的 EditLog，替换掉旧的。合并后 EditLog 归零，重新开始记录。

2. 启动服务Start service：元数据就绪后才对外提供服务。

3. 为什么这样设计：若不合并，EditLog 会无限增长，每次重启都要回放全部历史修改，重启时间越来越长。合并把它压成一份快照，重启时间与上次合并点而非集群总历史相关。

4. 但这个合并谁来做：第 23 页只讲了 NameNode 自己重启时的合并。第二十节会讲 2NN 如何把这个合并从 NameNode 的重启路径里挪出去——这就是 2NN 存在的全部理由。

2.5 DataNode

讲完名称（中心）节点，转向干活的一方。DataNode 有五件事：存块、存块的元数据、对外服务、定期汇报、参与流水线转发。

2.5.1 Functions of DataNode

原文	解读
Where HDFS stores the actual data；A Machine	实际数据存放处；物理上就是一台机器
Stores data blocks in the local file system	DataNode 不直接操作裸磁盘，而是把块写成本地文件
Stores metadata of a block	同时存块自身的元数据（注意：是块的元数据，不是文件系统元数据）
Serves data and metadata to Clients	向客户端提供数据与元数据，读操作的数据流直接从 DataNode 走
定期 BlockReport + 心跳 + 流水线转发	

Tips: NameNode 的元数据 = 文件名/块/位置的全局映射；DataNode 的块元数据 = 这一块自己的信息（校验和、长度等）。两者的范围完全不同。

2.5.2 Block Report 心跳

Block Report：Periodically（3s/time） sends a report of all existing blocks（本机**现有全部块**的清单） to the NameNode (heartbeat)

汇报：因为 NameNode 才持有块，即DataNode 的映射。这份映射必须由 DataNode 主动上报才能维持准确，NameNode 不主动扫描。

2.5.3 Facilitates Pipelining of Data

Forwards data to other specified DataNodes

转发：链式传递，写一个块时，客户端只把数据发给第一台 DataNode，由它转发给下一台。

流水线：若客户端给三台 DN 各发一份，客户端的出口带宽就是瓶颈。流水线让每台 DN 收一份、转一份，客户端只需一份出口带宽。

Part 3 DataNodes and NameNode Illustration

如果坏了副本放哪、怎么发现坏了、怎么补，以及中心节点自己坏了的上限在哪里。

3.1 DataNodes and NameNode Illustration

NameNode 只存元数据，DataNode 存块，文件到块的映射关系就写在元数据里。

1. NameNode：Stores metadata only 只存元数据（文件名 → 块号列表）

foo → 1, 2, 4：文件 /user/aaron/foo 由块 1、块 2、块 4 三块组成

2. DataNodes: Store blocks from files 存块

图 1 持 {1, 2, 4, 5} 共 4 块、图 2 持 {2, 3, 5} 共 3 块、图 3 持 {1, 3, 4} 共 3 块，本例中 10 个块位置对应 5 个块，所以每块恰好 2 份副本。

Tips：foo → 1,2,4 与副本的关系：元数据里只记录哪些块，不记录每块有几份副本（副本数由 replication factor 属性控制）。所以 foo 的块 1 与块 4 在图1和3 各一份、块 2 在图 1和2 各一份，元数据侧不体现。

3.2 Racks and Network Bandwidth

把同机架便宜、跨机架贵写成前提，并指出机架这个物理概念在 HDFS 里是被显式建模的。五条关于机架的事实：

	原文	解读
1	Many racks in a cluster	一个集群有很多机架
2	Communication between racks is through switches	跨机架通信要经过交换机，交换机就是带宽瓶颈的位置
3	Network bandwidth between machines on the same rack is greater than those on different racks	同机架带宽 > 跨机架带宽（成本模型的基石）
4	NameNode determines the rack ID for each DataNode	NameNode 为每个 DataNode 判定机架 ID，机架归属是元数据的一部分
5	Racks play an important role in block replication	机架在块复制中扮演重要角色（副本放置的逻辑：既然跨机架贵，副本就不能全放一个机架。）

Tips：

4 中的 机架 不是网络层自动知道的，而是 HDFS 主动建模的：管理员设计机架映射/感知（rack awareness），NameNode 据此计算距离、决定副本落点、决定该读哪份副本。

3.3 HDFS Data Replication: Illustration

块 2 在 Node A、B、D 三处被标成红色，即同一个块的三份副本分别落在哪。所以红色 = 同一块的多份副本。

总数守恒：5 个块 × 3 份 = 15 个块位置；5 个节点 × 3 个格位 = 15 个格位。两边相等，说明示意图是恰好铺满的设计。

副本不在同一节点：任何一块的 3 份副本分布在不同节点上。这是最直观的容错要求：一台机器坏了，该块仍有两份可用。

3.4 Replication：Block Placement

HDFS Rack-aware replica placement

机架策略（本地/同机架 → 不同机架 → 同远程机架的另一节点）

1. To put one replica on the local machine if the writer is on a DataNode, otherwise on a random DataNode in the same rack as that of the writer.

第一份副本的两种情形（在 DN 上 / 不在 DN 上）：若写入方本身在某个 DataNode 上，第一份就放在本地机器上，本地优先让写操作变成本地磁盘写，省掉一次跨网络传输。否则，放于与写入方同一机架的某个随机 DataNode。

2. Another on a node in a different (remote) rack

第二份放另一个（远程）机架上的节点。

3. The last one on a different node in the same remote rack

第三份放在相同的远程机架里的另一台不同节点的机器上。

Ps. If the replication factor is greater than 3, the placement of the 4th and following replicas is determined randomly.

超过 3 份怎么办，第 4 份起随机放置。默认副本因子 3，前 3 份有策略，后面的不管。

To reduce communication costs, clients read from nearest replicas

降低通信开销，读最近（按网络拓扑算距离）的那份副本。

策略意图：

1. 至少跨 2 个机架，机架断电不会丢块；
2. 最多跨 2 个机架，避免跨机架写入的带宽开销。

3.5 Necessity for Replication

三个故障场景（节点级 → 数据级 → 设备级）

1. A Datanode may become unavailable

节点整机不可用：一台 DataNode 可能整机不可用（宕机、断电、网络隔离）。

2. A replica may become corrupted

副本被损坏：副本内容可能损坏（静默数据损坏 bit rot）。注意这里说的是 replica 而非 node。若只有节点故障，多副本就够了；但副本损坏意味着副本本身不可信，所以必须有校验机制来判断哪份副本是好的，所以需要后续的BlockReport 与校验和设计。

3. A hard disk on a Datanode may fail

磁盘故障：DataNode 上的某块硬盘可能坏掉（一机多盘时，坏盘不等于整机不可用）。

3.6 心跳机制Heartbeats

复制解决坏了还能读，本节解决怎么知道坏了。

机制：DataNode 定期报平安，NameNode 超时就判死。

课件原文	解读
DataNodes send heartbeat to the NameNode	DataNode → NameNode 发送心跳（DN 主动上报，不是 NN 去探）
Once every 3 seconds	heartbeat 周期 = 3 秒，唯一的具体时间数字
The Namenode receives a Heartbeat and a BlockReport from each DataNode in the cluster	NN 从每个 DataNode 收到心跳 + BlockReport两样东西；Heartbeat = 我还活着（存活信号）；BlockReport = 我有哪些块（内容清单）
BlockReport contains a list of all the blocks on a Datanode	BlockReport 的内容 = 该 DataNode 上全部块的清单
NameNode uses heartbeats to detect DataNode failure	NN 用心跳来判断 DataNode 故障

3.7 复制引擎 Replication Engine

1. NameNode detects DataNode failures. Data blocks on the failed Datanode are replicated on other Datanodes

NameNode 检测到 DataNode 故障。补副本：失效节点上的块在其他 DataNode 上重新复制。

2. Choose new DataNodes for new replicas

选新落点：为新副本选新的 DataNode。选新落点时的约束：Balance disk usage（均衡磁盘使用率）与 Balance communication traffic to DataNodes（均衡到各 DataNode 的通信流量）。

两个均衡目标的区别：磁盘均衡是容量维度（别让某台机器塞满）；通信均衡是带宽维度（别让某台机器成为热点）。

补副本 vs lecture4 重跑任务：第 4 讲的容错是重算（任务重跑）；本讲的容错是补数据（复制块）。同一思想在存储层与计算层各有一个实现。

3.8 NameNode 故障

代次	方案	说明
共性	唯一的单点故障 A single point of failure	NameNode 只有一个，它挂了整个 HDFS 不可用
Hadoop 1.x	Secondary NameNode (2NN): Hadoop 1.x	1.x 的缓解方案（第 19 - 20 节展开）
Hadoop 2.x	CheckpointNode (CN): Hadoop 2.x	2.x 改名为 CheckpointNode

两代方案名字不同、性质相似：2NN 与 CN 都不是 NameNode 的活备份（NOT a complete backup），它们做的是元数据检查点的合并。

3.8 Accelerate NameNode Restart

澄清一个最常见的误解：2NN 的第一职责不是备份，而是加速重启。它会合并 FsImage 与 EditLog，并周期性把结果回传。

3.8.1 五步工作流程

copy → merge → upload → purge → 目的是加速重启

没有 2NN（Original 原始版本）

NN 重启时，必须：加载旧 FsImage → 重放整条完整 EditLog，日志越长，重启越慢。

搭配 2NN 的版本

2NN 已经提前做完 Checkpoint，生成了合并后的新 FsImage。 NN 重启时，只需要加载 2NN 生成的最新 FsImage，只重放 Checkpoint 之后新增的一小段 EditLog（绿色增量），大幅缩短启动时间。2NN 本地保存一份最新 FsImage 副本。

1. 2NN copies FsImage and EditLog from NameNode

取文件：2NN 从 NameNode 拷贝 FsImage 与 EditLog。2NN 是读取方，不是被写入方，它不可能实时同步。

2. 2NN merges FsImage and EditLogs periodically

合并：2NN 周期性合并两者（2NN 的核心动作）。

3. 2NN uploads new FsImage to the NameNode

回传：把新的 FsImage 上传回 NameNode。

4. EditLog on NameNode is purged

清理：NameNode 上的 EditLog 被清空（因为它已经合并进新 FsImage 了）。

5. It is designed to accelerate the speed of NameNode's restarting

目的：设计目的是加速 NameNode 重启。如果 EditLog 很大，重启会耗很多时间。2NN 把回放长日志这件事提前、离线做掉了，NameNode 重启时只需载入一份新鲜的快照。

检查点（checkpoint）：2NN 做的事情本质是检查点合并器，把快照 + 增量压成新快照 + 空增量，NameNode 之外执行，于是 NameNode 重启变快。

3.8.2 备份能力与限制

Additional features: It can provide a copy if NameNode fails. Not the latest one.

NN 挂了可以用 2NN 上的 FsImage 临时顶替，所以 2NN 常被误认为备份。2NN 只在每次合并的时点持有快照，两次合并之间的修改（新产生的 EditLog）它没有。所以丢掉的是上次合并到故障时刻这段更新。所以，这不是完整备份（NOT a complete backup of NN），数据是滞后的，两页合起来才是完整答案。

易错点：2NN 的 Secondary 不是第二主节点，而是次要的、辅助的。它不接管 NameNode 的在线服务，也不参与读写路径。

3.9 2NN Works

F = **FsImage**（文件系统镜像，NameNode 的持久化元数据快照）

E = **EditLog**（编辑日志，记录后续所有文件系统操作的增量日志）

1. NameNode(NN)

   NN 初始：FsImage (F) + 空 EditLog (E)

   客户端操作不断写入 EditLog(E)（红色代表新增日志），FsImage 保持不变

   此时：FsImage 是旧快照，所有新改动存在 EditLog 里

2. SecondaryNameNode(2NN) 周期性触发Checkpoint检查点操作：

   拉取（fetch）NN 当前的 FsImage + EditLog

合并（merge）：把 EditLog 的所有操作应用到 FsImage，生成新的 FsImage

把合并完成的最新 FsImage 保存在 2NN 本地

再将新 FsImage 传回 NameNode，NN 替换旧 FsImage，并清空自己的 EditLog

2NN 不是 NN 热备，只定期帮 NN 做 FsImage+EditLog 合并，生成最新快照，减轻 NN 重启加载压力

3.10 NameNode Recovery

场景：NameNode 在两次 2NN Checkpoint 之间崩溃

1. NN 宕机时，最新的增量修改（绿色部分）只存在 NN 本机 EditLog，还没同步合并到 2NN
2. 2NN 手里，只有上一次 Checkpoint 完成的 FsImage 快照（缺少崩溃前这一段新操作）
3. 故障恢复：

   把 2NN 上保存的 FsImage 副本拿出来，作为恢复的基础镜像

   缺点：Checkpoint 之后、宕机之前的新增元数据（绿色更新）会丢失

重点考点：

1. 2NN ≠ NameNode 备用节点，**不**能在 NN 宕机后自动切换接管，只是定期做 Checkpoint、生成 FsImage 备份。
2. Checkpoint：2NN 拉取 FsImage+EditLog，回放日志合并生成新镜像。
3. NN 正常重启：有 2NN 可以减少需要回放的 EditLog，加快启动。
4. NN 崩溃恢复：2NN 不能做到实时热备份，只能恢复到最近一次 2NN checkpoint 时刻，两次 checkpoint 之间的元数据更新丢失。

Part 4 HDFS 读和写操作

4.1 HDFS Architecture (Read and Write Path)

Read 路径：读走同机架的 DataNode；

Replication 路径：跨机架复制

1. NameNode（NN）

- 管理**所有元数据**：文件名、目录、权限、Block编号、Block所在DN位置
- **不存储业务数据**，只存索引信息
- 接收客户端RPC请求，调度读写，**不参与数据传输**

2. DataNode（DN）

- 真正**存储文件Block真实数据**
- 定时向NN心跳、汇报块状态
- 负责读写数据、副本存储、Pipeline传输

3. Client 客户端

- 负责文件拆分Block、组装数据、容错重连、流水线控制
- Client 到 NameNode，标注 Metadata ops，只读写不传数据
- Client 到 DataNode，标注 Write，真正的直接流，读时 Client 直接向 DataNode 取，写时 Client 发起流水线；

4.3 HDFS读操作

步骤1：Client connects to NN to request data read.

Client发起 open() 请求。客户端想要读取 HDFS 文件，首先通过 RPC 向 NameNode 发送 open 请求，询问该文件的元数据信息。

步骤2：NN tells the client where (i.e., DN) to find the data blocks.

NameNode 返回 Block 位置列表。NN 查询自己内存中的元数据，返回：

- 该文件包含哪些 Block（Block1、Block2、Block3…）
- 每个 Block 的所有副本分别位于哪些 DataNode 节点
- 返回的列表已经经过机架感知排序（就近优先）

重点：NN 只返回位置，不传输任何数据！

步骤3：Client reads blocks directly from data nodes (without going through NN)

Client 主动连接最优的 DataNode。客户端根据 NN 返回的有序 DN 列表，优先连接距离最近、负载最低的 DN，建立 Socket 连接。

步骤4：DN 以 Packet 数据包形式返回数据

DataNode 从本地磁盘读取 Block 数据，拆分为 Packet（默认64KB），逐包发送给 Client。

Client 持续接收、缓存、拼接数据。

步骤5：n case of node failures, client connects to another node that serves the missing block

读取失败自动容错（核心考点）如果当前 DN 掉线、卡顿、读取出错：

- 客户端立刻断开当前连接
- 从同 Block 的其他副本节点中选择下一个最优 DN
- 从失败的 Packet 位置继续读取，不重头读整个块

步骤6：一个 Block 读完，读取下一个 Block

当前 Block 的全部 Packet 接收完毕，客户端继续读取文件的下一个 Block，重复上述流程。

步骤7：全部读取完成，执行 close()

文件所有 Block 读取完毕，客户端关闭数据流、释放连接，读操作结束。

1. 客户端连接 NameNode，请求读数据。

2. NN 告诉客户端到哪个 DataNode 去找这些块。

3. 客户端直接从 DataNode 读块，不经过 NN。

4. I节点故障时，客户端改连另一台持有该块的节点。

4.2.1 两个问题

Why does HDFS choose such a design for reading?

Why not ask clients to read blocks through NN?

原因	解读
Prevent NN from being the bottleneck of the cluster	防止 NameNode 成为集群瓶颈，所有数据若都过它，带宽立刻打满
Allow HDFS to scale to large number of concurrent clients	让 HDFS 能扩展到大量并发客户端，瓶颈一旦在中心，并发度就被锁死
Spread the data traffic across the cluster	让数据流量分散到整个集群，而不是集中到一个节点出口

4.2.2 机架感知机制

HDFS 将集群网络抽象为三层树状拓扑：数据中心 → 机架 → 节点，读取副本优先级（速度从快到慢）：

1. 本节点读取（最快，无网络开销）
2. 同机架不同节点（同交换机，带宽高、延迟低）
3. 同机房不同机架（跨交换机，速度下降）
4. 跨机房（最慢、延迟最高）

作用：减少跨机架流量、提升读取速度、降低集群网络压力

4.2.3 读操作核心特点

数据不经过 NameNode，避免 NN 瓶颈

控制流走 NN，数据流直连 DN

天然支持容错，副本自动切换

支持顺序读、大文件吞吐极强，不适合随机读

4.3 HDFS 写操作

步骤1：Client 向 NN 发送 create() 创建文件请求。客户端想要上传文件，先 RPC 请求 NameNode 创建空文件。

步骤2：NN 执行权限、路径、状态校验，NN 检查：用户权限是否合法；文件路径是否已存在；父目录是否存在；集群是否可写。校验通过后，NN 在元数据中生成一个空文件（此时文件对用户可见，但未完成）。

步骤3：NN 分配 Pipeline 节点列表（副本节点策略），NN 根据副本机制（默认3副本）智能选择一组 DN，形成流水线顺序：放置策略（必考）：

1. 副本1：写入客户端所在节点（就近写入，速度最快）
2. 副本2：写入同机架不同节点（防单节点故障）
3. 副本3：写入不同机架节点（防机架整体断电/故障）

NN 将有序的 DN 列表返回给 Client。

步骤4：Client 构建 Pipeline 流水线。客户端根据返回的 DN 顺序，依次和 DN1、DN2、DN3 建立链式 Socket 通道，形成流水线。

步骤5：客户端切分 Packet，链式传输写入。客户端将文件数据切分为 Packet（默认64KB），先发给第一个 DN：

- DN1 收到 Packet → 落盘保存 → 立刻转发给 DN2
- DN2 收到 Packet → 落盘保存 → 立刻转发给 DN3
- DN3 收到 Packet → 落盘保存（最后节点，不转发）

整条流水线并行写多副本，极大提升写入效率

步骤6：反向 ACK 确认机制（核心重难点）。每一个 Packet 写完后，ACK 从后往前逐级返回：

- DN3 写完 → 发 ACK 给 DN2
- DN2 收到后 → 发 ACK 给 DN1
- DN1 收到后 → 最终 ACK 传回 Client

规则：必须收到整条流水线的 ACK，才算当前 Packet 写入成功，才会继续发送下一个 Packet。

步骤7：Block 写满，继续写下一个 Block。当一个 Block 写满（默认128M）：

- Client 关闭当前 Block
- 重新向 NN 请求新的 DN 节点列表
- 开启新 Pipeline，写下一个 Block

步骤8：文件全部写完，Client 发送 complete() 完成写入。文件所有 Block 写完后：

- Client 调用 close()
- 向 NN 发送 complete 信号
- NN 将文件状态从“正在写入”改为“已完成”

此时文件正式对外可读。

4.3.1 写操作容错机制

流水线中任意一个 DN 故障（掉线、卡死、磁盘坏）：

1. Client 立刻检测到 ACK 超时/失败
2. 自动剔除故障 DN
3. 用剩余健康 DN 重建新 Pipeline
4. 从上一个成功 ACK 的 Packet 位置继续写入，不重头写整个文件
5. 后续 NN 会自动补充副本，保证副本数量达标

考点：HDFS 写过程支持断点续写、动态重建流水线

4.3.2 写操作核心特性

- 控制流走 NN，数据流走 Pipeline
- 采用链式流水线批量写副本，效率高
- 严格 ACK 校验，保证数据强一致性

- 写入过程文件处于“未完成状态”，不可读
- 支持动态容错、节点剔除、续写

4.4 HDFS 读写操作

对比维度	读操作 Read	写操作 Write
NN 作用	只返回 Block 位置列表，不参与数据传输	创建文件、分配DN、最后标记文件完成
数据流向	DN → Client 单点直连	Client→DN1→DN2→DN3 流水线链式传输
副本策略	就近读（机架感知择优）	跨机架三副本策略，保证容灾
容错方式	节点失败，切换同块其他副本继续读	节点失败，重建Pipeline，剔除坏节点续写
数据包单位	Packet	Packet，带反向ACK确认
文件状态	文件必须已完成	写入中文件临时不可读，complete后才可见

1. 为什么 HDFS 读写数据不经过 NameNode?

为了避免 NameNode 成为集群性能瓶颈，分离控制流和数据流，让大量读写流量分散到各个 DataNode，支持高并发、高吞吐，提升集群扩展性。

2. HDFS 三副本放置策略是什么?

第一副本存客户端当前节点，第二副本存同机架不同节点，第三副本存不同机架节点。兼顾写入速度和容灾能力，防止单节点、单机架故障导致数据丢失。

3. 什么是 Pipeline 流水线写入?

客户端将数据 Packet 依次传递给多个 DataNode，链式完成多副本写入，并逐级返回ACK确认。实现多副本并行写入，大幅提升写入效率，同时保证数据一致性。

4. 读操作机架感知的作用?

根据网络拓扑选择距离最近的副本节点读取数据，减少跨机架网络传输，降低延迟、节省带宽、提升读取性能。

Part 5 Hadoop’s Job Scheduling 作业调度

本文讲解的是 MapReduce v1（MRv1），即 Hadoop 1.x 时代的经典版本。采用Master/Slave 主从架构，核心组件为 JobTracker + TaskTracker。

该版本存在致命架构缺陷，在 Hadoop 2.x 后被 YARN 完全替代，是大数据面试、考试、入门的核心基础知识点。

核心定位：MapReduce 是 Hadoop 生态圈中的分布式离线计算框架，负责对海量HDFS存储数据进行分片、并行计算、汇总输出，主打高容错、高可靠、可横向扩展。

5.1 JobTracker & TaskTracker 作业调度

HDFS 解决了数据放哪，这一节转向任务派给谁。

5.1 MRv1 整体集群架构

MRv1 架构严格分为Master 主节点 和 Slave 从节点，同时依赖 HDFS 的 NameNode、DataNode 实现数据存储，全程是计算与存储绑定的设计思路。

与 HDFS 的镜像关系：NameNode : DataNode ≈ JobTracker : TaskTracker。前者管元数据与任务，后者管块与执行。

5.1.1 Master 主节点核心进程

Master 节点仅部署 JobTracker（JT） 一个计算调度进程，同时集群的 NameNode 通常也部署在主节点，负责文件系统管理。

JobTracker = central control node，负责 split 与 send；

TaskTracker = worker node 上接受 Map / Reduce / Shuffle 任务的进程。

YARN 改进此结构（out of scope）。

JobTracker 核心职责（全局调度中心）

A central control node runs JobTracker (JT). JobTracker splits up data into smaller tasks("Map") and sends it to the TaskTracker process in each node

把数据切成更小的任务 Map，再派发给各节点的 TaskTracker 进程。

- 作业接收与注册：专门接收客户端提交的完整 MapReduce Job 作业，分配全局唯一的 JobID，初始化作业运行环境。
- 作业拆分与分片规划：解析用户提交的作业配置、输入路径、计算逻辑，自动将一个大 Job 拆分为若干个 MapTask 和若干个 ReduceTask，同时根据文件大小生成输入分片（InputSplit）。
- 任务调度与分配：集群唯一的任务决策者，实时感知所有 Slave 节点的资源空闲状态、数据存储位置，精准分配 Map/Reduce 任务，实现负载均衡和数据本地性优化。
- 集群状态监控：维持与所有 TaskTracker 的心跳通信，实时监控节点存活状态、资源占用、任务运行进度。
- 故障判定与重试：检测节点故障、任务运行失败、任务卡顿等异常，自动触发任务重试、节点拉黑、作业容错机制。
- 作业进度管理与结果汇总：全局统计作业运行进度，记录任务运行日志，作业完成后更新最终状态，通知客户端获取结果。

5.1.2 Slave 从节点

所有从节点同时部署两个进程，分别负责计算执行和数据存储，是真正干活的节点：

1. TaskTracker（TT）计算执行代理

Worker Nodes also run TaskTracker (TT). A TaskTracker is a node in the cluster that accepts tasks — Map, Reduce, and Shuffle operations — from a JobTracker

在工作节点上运行 TaskTracker。集群里接受 JobTracker 派发任务的节点。

每个 Slave 节点有一个 TaskTracker，是 JobTracker 的从属工作进程，无自主调度权限，只服从主节点指令。

- 心跳上报：默认每3秒向 JobTracker 发送一次心跳，上报节点CPU、内存、空闲槽位、正在运行的任务状态等信息。
- 任务接收与执行：接收 JobTracker 下发的 MapTask、ReduceTask，独立启动子JVM运行任务，避免任务之间相互干扰。
- 资源管控：管理当前节点的任务运行槽位（Map槽位、Reduce槽位），限制单节点最大并发任务数，防止资源耗尽。

- 状态反馈：实时向 JobTracker 汇报任务启动、运行、完成、失败、卡顿等状态。

2. DataNode（DN）数据存储节点

负责存储 HDFS 数据块，保存所有待计算的原始数据、任务中间临时数据，为本地性调度提供数据支撑。计算任务优先读取本节点 DataNode 的数据，减少网络传输。

5.1.3 Client 客户端

用户程序入口，无需常驻集群，仅负责作业提交、资源上传、进度查询、结果获取，核心操作：

- 封装 Job 配置信息（输入输出路径、Map/Reduce逻辑、任务参数）；
- 向 JobTracker 申请 JobID；
- 将作业Jar包、配置文件、依赖资源上传至HDFS临时目录；
- 正式提交作业，轮询查询作业运行进度，作业结束后输出结果。

5.2 MRv1 完整作业执行流程

整个流程分为 作业准备阶段、任务调度阶段、任务运行阶段、作业收尾阶段 四大阶段，全程自动化执行。

阶段1：客户端作业准备

1. 用户编写 MapReduce 程序，设置作业参数，打包为Jar文件；
2. Client 连接 JobTracker，请求分配唯一 JobID；
3. Client 在 HDFS 中创建该作业的专属临时目录，上传Jar包、配置文件、切片信息等资源；
4. Client 正式向 JobTracker 提交完整作业。

阶段2：JobTracker 作业初始化

1. JobTracker 接收作业，校验配置合法性，初始化作业运行上下文；
2. JobTracker 主动请求 NameNode，查询本次作业所有输入文件的 数据块存储位置、副本分布节点；
3. 根据文件大小、Block大小，自动切分 InputSplit 输入分片，一个分片对应一个MapTask，确定Map任务总数；
4. 初始化 ReduceTask 数量，创建任务调度队列，等待节点资源空闲。

阶段3：任务调度与下发

1. 所有 TaskTracker 持续向 JobTracker 发送心跳，上报空闲资源；
2. JobTracker 基于数据本地性优先、节点负载均衡原则，筛选合适的TaskTracker；
3. JobTracker 通过心跳响应，将 MapTask/ReduceTask 分配给目标节点；
4. TaskTracker 接收任务指令，从HDFS下载作业Jar包和配置资源。

阶段4：任务运行执行

1. TaskTracker 为每个任务启动独立子JVM进程（核心隔离机制，单个任务崩溃不影响节点和其他任务）；
2. 子JVM 运行 MapTask：读取本地HDFS数据块，执行Map逻辑，输出局部中间数据；
3. 所有MapTask执行完成后，启动Shuffle洗牌阶段，对数据分区、排序、合并；

4. 运行ReduceTask：拉取对应分区的Map中间数据，聚合汇总，执行Reduce业务逻辑；
5. 任务运行中持续向TaskTracker反馈状态，TT同步上报JobTracker。

阶段5：作业收尾

1. 当所有MapTask、ReduceTask全部执行成功，JobTracker 标记作业为完成状态；
2. 清理HDFS中的作业临时资源、日志缓存；
3. 客户端轮询获取作业完成状态，读取HDFS中的最终输出结果；
4. 若存在任务失败，触发重试机制，重试次数耗尽则标记作业失败。

5.3 MapTask与ReduceTask数量确定规则

5.3.1 MapTask 数量规则

MapTask 是数据分片并行处理的核心，数量直接决定作业并行度。

- 默认规则：MapTask 数量 = 输入文件的 HDFS Block 块数量。Hadoop 默认Block块大小128M，一个1G文件会被分为8个Block，默认启动8个MapTask。
- 核心原理：一个 InputSplit 对应一个 MapTask，分片大小默认等于Block大小，最大限度贴合数据存储粒度，避免跨块读取。
- 手动调整方式：可通过修改最小分片大小、最大分片大小，主动增加或减少MapTask数量，适配小文件、大文件场景。
- 优化原则：小文件过多会产生大量小MapTask，造成资源浪费，需提前合并小文件；超大文件可适当调大分片，减少任务数量。

5.3.2 ReduceTask 数量规则

- 默认值：默认ReduceTask数量为1，所有数据汇总到一个Reduce任务中处理。
- 常规生产配置：手动设置，数量远小于MapTask数量，通常根据Key的种类、数据量分配。
- 性能风险：ReduceTask数量过少，会导致单个Reduce任务处理海量数据，成为作业瓶颈；数量过多会产生大量小输出文件，增加文件系统压力。
- 特殊场景：无需聚合汇总的任务，可设置ReduceTask数量为0，只执行Map阶段，提升效率。

5.4核心优化机制（底层原理）

5.4.1 数据本地性优化（Locality Optimization）

核心思想：移动计算，不移动数据。大数据场景下，磁盘IO速度远大于网络传输速度，跨节点传输数据是最大性能瓶颈。

三层调度优先级（从高到低）

1. 本地节点优先：将Map任务调度到「存储该数据块的Slave节点」，直接读取本地磁盘数据，无网络开销；
2. 同机架优先：若数据所在节点繁忙，调度到同一机架的空闲节点，机架内网络传输速度快、延迟低；
3. 跨机架调度：无本地、同机架资源时，才调度到其他机架节点，产生跨网络传输。

价值：极大降低集群网络IO压力，是MapReduce海量数据处理高效运行的核心保障。

5.4.2 推测执行机制（Speculative Execution / 冗余执行）

问题背景

集群中部分节点会因磁盘老化、CPU负载高、网络卡顿、资源抢占等问题，运行速度远慢于其他节点。单个慢任务会拖慢整个作业的完成时间，形成木桶效应。

机制原理：JobTracker 实时对比同批次所有任务的运行进度，当检测到某一个任务进度远低于平均进度、长期卡顿，不会无限等待，而是主动在空闲节点启动一个相同的备份任务（冗余任务）。

执行规则：谁先执行完成，就采用谁的结果，后完成的任务直接被强制终止，释放资源。

适用与禁用场景

- 适用：离线批量计算、只读数据任务；
- 禁用：会写入数据、非幂等性任务，避免重复写入导致数据错乱。

5.4.3 负载均衡机制

JobTracker 调度时不仅看数据本地性，还会实时统计每个TaskTracker的当前任务负载、空闲槽位、资源使用率，避免任务集中分配到少数节点。

通过动态分配任务，让所有Slave节点的任务执行时长、资源占用尽量均衡，杜绝部分节点过载、部分节点空闲的问题，最大化利用集群整体算力。

5.4.4 任务 JVM 复用机制

MRv1 默认每个任务新建 JVM，开销大。支持开启 JVM 复用，同一作业的多个小任务可复用同一个 JVM，减少启停开销，极大提升小任务集群效率。

5.5 故障恢复机制

MapReduce v1 具备完善的容错机制，可应对绝大多数节点、任务异常，保障集群稳定运行。

5.5.1 TaskTracker（Slave节点）故障

故障判定：JobTracker 依靠心跳机制判定节点故障：默认心跳超时时间为10次心跳无响应（约30秒），超时则直接判定该TaskTracker宕机。

恢复策略

- JobTracker 立即清除该节点的所有任务运行记录；
- 该节点上正在运行的未完成MapTask、ReduceTask，全部重新调度到其他空闲节点执行；
- 若该节点频繁故障、卡顿，JobTracker 会将其加入集群黑名单，不再分配任务，人工排查修复后可解除黑名单；
- 已完成的任务结果无需重跑，仅重试未完成的任务，最大限度节省资源。

5.5.2 单个Task任务故障

故障场景：任务代码异常、内存溢出、数据读取失败、子JVM崩溃等。

恢复策略

- TaskTracker 检测到任务失败，立即上报JobTracker；

- JobTracker 自动重新分配该任务，更换节点重试；
- 重试次数限制：默认最大重试4次，超过次数仍失败，则判定作业失败，终止任务并打印错误日志。

5.5.3 JobTracker（Master节点）故障

MRv1 最大的架构短板：JobTracker 单点故障（SPOF）。

- 整个集群只有一个JobTracker，无备用节点、无故障转移机制；
- 一旦JobTracker进程挂掉、主节点宕机，整个集群调度彻底瘫痪：所有正在运行的作业终止、新作业无法提交、节点全部失联；
- 恢复方式：仅能重启JobTracker进程，但重启后所有正在运行的作业全部丢失，需要重新提交执行，无法断点续跑。

核心结论：这是MRv1被YARN彻底替代的最根本原因。YARN将JobTracker的职责拆分，解决了单点故障问题，同时实现资源与任务解耦。

5.6 MRv1 架构优缺点

核心优点

- 架构简单清晰，主从分工明确，入门门槛低，易于运维；
- 容错机制完善，节点、任务故障可自动恢复，适合海量离线数据处理；
- 数据本地性优化优秀，极大规避网络IO瓶颈，计算效率高；
- 任务粒度细，单任务崩溃不影响全局，集群稳定性较好。

致命缺点（核心考点）

- 单点故障问题：JobTracker 单节点运行，宕机即集群瘫痪，无高可用；YARN later , Hadoop improved this structure. (out of scope) 后续版本的 Hadoop 用 YARN 改进了这个结构；
- 资源耦合严重：计算资源、任务调度、作业管理全部由JobTracker承担，压力极大；
- 扩展性差：集群节点数量过多时，JobTracker 心跳处理、任务调度压力过载，性能急剧下降；
- 资源利用率低：任务资源固定分配，无法动态调整，容易出现资源闲置或资源不足。

Part 6 Spark Motivation

6.1 MapReduce的缺陷

1. Force your data processing into Map and Reduce

强制将所有数据处理逻辑限定在 Map 和 Reduce 两个固定阶段。

扩展知识点：MapReduce 编程模型极度固化，无论业务简单或复杂，都必须拆分为映射、归约两个步骤。面对多表关联、多层嵌套计算、复杂统计逻辑时，代码冗余度极高，开发效率很低，灵活性极差。

2. Not good for jobs with a shared global state

不支持带有全局共享状态的计算任务。

扩展知识点：MapReduce 是无状态计算模型，不同任务、不同阶段之间无法直接共享内存变量。对于需要累计参数、迭代更新状态的场景（机器学习、图计算），无法直接实现，只能通过磁盘文件进行中转。

3. Read and write to Disk before and after Map and Reduce (stateless machine)

Map、Reduce 阶段前后均需要磁盘读写，属于无状态计算机制。

扩展知识点：这是 MapReduce 最大的性能瓶颈。每一轮计算的中间结果都会强制落地到 HDFS 磁盘，下一轮计算再重新读取。磁盘IO速度远低于内存读写，大量重复读写造成极大资源浪费。stateless 无状态指集群节点不会保留任何中间计算数据。

4. Not efficient for iterative tasks, i.e. Machine Learning

迭代式计算任务效率极低，典型场景为机器学习。

扩展知识点：机器学习、深度学习、图算法都需要成千上万次迭代运算。MapReduce 每次迭代都要落盘、读盘，迭代次数越多，IO 累积开销越大，几乎无法支撑大规模迭代计算。

5. Only for Batch processing

仅支持离线批处理计算。

扩展知识点：MapReduce 只能处理已经完全存储完毕的静态数据，无法处理实时增量数据，不适合实时业务场景。

6. Does not support interactivity, streaming data

不支持交互式查询与流式数据处理。

扩展知识点：MapReduce 每次提交任务都需要完整调度执行，无法即时返回查询结果，不适合数据分析人员交互式探索数据，也无法处理源源不断的实时流数据。

核心总结：MapReduce 痛点集中在：模型死板、磁盘IO频繁、不支持迭代计算、仅离线批处理、无交互、无实时能力，这也是 Spark 诞生的核心原因。

6.2 Apache Spark

1. A new general framework, which solves many of the shortcomings of MapReduce

全新的通用分布式计算框架，解决了 MapReduce 的绝大多数缺陷。Spark 2009年诞生，核心设计理念是**内存计算**，将中间计算结果缓存至内存，规避 MapReduce 频繁磁盘读写的短板，大幅提升计算速度。

2. It leverages the Hadoop ecosystem, e.g., HDFS, YARN, HBase, S3

完全兼容并复用 Hadoop 生态，支持 HDFS、YARN、HBase、S3 等主流组件。Spark **不替代 Hadoop**，只替代 MapReduce 计算引擎。Spark 可以直接部署在现有 Hadoop 集群上，沿用原有存储和资源调度体系，迁移成本极低，生态兼容性极强。

3. Has around 30 efficient distributed operations

内置约30种高效分布式数据操作算子，包含 join、filter、flatMap、groupByKey、reduceByKey、sortByKey、collect、count 等。MapReduce 仅有 Map、Reduce 两个基础算子，而 Spark 提供丰富的开箱即用算子，复杂的数据清洗、分组、关联、排序逻辑只需少量代码实现，开发效率成倍提升。

4. In-memory caching of data for iterative, graph, and machine learning algorithms

支持数据内存缓存，适配迭代计算、图计算、机器学习算法场景。可将高频使用的中间数据集缓存于集群内存，多次计算可重复复用数据，无需重复读取磁盘，是 Spark 迭代任务秒杀 MapReduce 的核心原因。

5. Native Scala, Java, Python, and R support

原生支持 Scala、Java、Python、R 四种开发语言。适配不同岗位需求，后端高性能开发优先使用 Scala，数据分析人员常用 PySpark，统计分析可使用 SparkR，受众更广。

6. Supports interactive shells for exploratory data analysis

支持交互式命令行，可用于探索性数据分析。通过 spark-shell、pyspark 可实现逐行代码执行、即时出结果，适合临时数据分析、代码调试、数据探索，完美弥补 MapReduce 无法交互的缺陷。

7. Spark API is extremely simple to use

Spark API 简洁易用，封装度高。高层API屏蔽了分布式底层细节，开发者无需关注集群调度、节点通信、数据分片，专注业务逻辑即可。

Spark 与 MapReduce 核心机制对比

方案	流程	每轮之间
Hadoop: Use Disk for Data Sharing	HDFS read → Iteration1 → HDFS Write → HDFS read → Iteration2 → HDFS Write	落盘（每次交接都是一次 HDFS 读 + 一次 HDFS 写）
Spark: In-Memory Data Sharing	HDFS read → Iteration1 →（内存）→ Iteration2 →（内存）	留在内存（只有最开始一次 HDFS read）

Hadoop MapReduce：依靠磁盘共享中间数据，每轮计算结束强制落盘，迭代任务反复读写磁盘，IO开销巨大。

Spark：依靠内存共享中间数据，计算结果常驻内存，多轮迭代直接读取内存数据，跳过磁盘IO，性能大幅提升。

官方排序测试结论：Spark 仅用 1/10 的集群节点，实现 3 倍于 MapReduce 的计算速度，单机资源利用率远高于 MapReduce。

6.3 The Spark Stack 组件栈

Apache Spark supports data analysis, machine learning, graphs, streaming data, etc. It can read/write from a range of data types and allows development in multiple languages.

Apache Spark 支持数据分析、机器学习、图计算、流式数据处理等多类场景，支持读写多种数据源，兼容多语言开发。Spark 是**一站式全域大数据计算平台**，并非单一计算引擎，覆盖离线、实时、机器学习、图计算全场景。

6.3.1 Spark 分层架构（由上至下）

1. 应用层：支持 Scala、Java、Python、R、SQL 多语言开发

2. 高层API层：DataFrames、ML Pipelines 结构化、机器学习流水线API

3. 核心组件层：Spark SQL（结构化查询）、Spark Streaming（实时流计算）、MLlib（机器学习库）、GraphX（图计算库），所有组件均基于内核构建

4. 底层内核：Spark Core，负责任务调度、内存管理、容错机制、RDD 核心抽象

5. 数据源层：兼容 HDFS、Hive、HBase、MySQL、S3、流式数据、JSON、高性能分布式文件系统等几乎所有存储介质

Part 7 Spark Technical Details

新引擎的原理层：一个统一抽象（RDD）、两类算子、一条依赖图（DAG），以及它们如何决定快慢。

7.1 RDD 弹性分布式数据集

Resilient Distributed Datasets (RDDs) are a fundamental data structure of Spark. It is an immutable distributed collection of objects.

RDD（弹性分布式数据集）是 Spark 的基础核心数据结构，是一组不可变的分布式对象集合。RDD 是 Spark 所有功能的底层基石，Spark SQL、Streaming、MLlib 高层 API 底层全部依赖 RDD 实现。immutable 不可变是 RDD 的核心特性，创建后无法修改，只能通过算子生成新 RDD，保证数据安全、支持容错。

7.1.1 三大核心特性

1. Resilient - capable of rebuilding data on failure

弹性，在节点故障时可自动重建丢失的数据。依靠**血缘依赖（Lineage）**记录所有计算过程，无需备份数据，故障后只需重放上游算子即可恢复数据，轻量化容错。

2. Distributed partition data across various nodes in a cluster

分布式，数据被切分为多个分区，分散在集群的不同节点。Partition 分区是 Spark 的最小并行计算单元，分区数量决定并行度，分区越多，并行计算能力越强。

3. Dataset - a collection of partitioned data with values

数据集，用于存储被分区的真实业务数据。

All the different processing components in Spark share the same abstraction: RDD. RDD is created by parallelizing a collection or reading a file.

Spark 的所有计算组件共用 RDD 抽象，RDD 可通过并行化本地集合、读取外部文件两种方式创建。日常开发两种常用的创建方式：本地集合并行化 sc.parallelize()、外部文件读取 sc.textFile()，覆盖绝大多数数据初始化场景。

RDD vs HDFS 块的对照

HDFS 块：物理存在的、多副本的、可寻址的数据单元。

RDD：逻辑上的、不可变的、可重算的数据集合。RDD 是分布在 HDFS 之上的逻辑视图。

7.2 Spark 的两大算子

Spark Operations = TRANSFORMATIONS + ACTIONS;

Action 给出 non-RDD values（判据）、触发 DAG 执行、将结果返回程序或写入外部存储。Action 才触发计算，那么调试时看一个 Spark 程序是否真的运行，就看有没有 Action，只写一串 Transformation 的程序什么都不会发生。

7.2.1 Transformation 转换算子

Transformation 是 RDD → RDD 的函数，且是惰性的（不会立刻执行）。因为变换是惰性的，Spark 就能在真正执行前优化整条链（合并、剪枝、决定分区）

Spark Transformation is a function that **produces a new RDD** from the existing RDDs. It takes RDD as input and produces one or more RDDs as output.

转换算子是以已有 RDD 为输入，生成新 RDD 的函数操作。转换算子具备**懒加载特性**，调用后不会立即执行计算，仅记录血缘依赖、构建 DAG 执行图，无任务调度、无资源消耗。同时天然支持容错，可链式连续调用。

窄依赖 Narrow Transformation

Each input partition will contribute to only one output partition.

每一个输入分区仅对应一个输出分区。一对一数据依赖，无需跨节点网络传输（无 Shuffle），计算速度快、开销低。常见算子：map、filter、flatMap。

宽依赖 Wide Transformation

Input partitions that contribute to many output partitions require a shuffle.

输入分区数据会分发到多个输出分区，需要 Shuffle 洗牌操作。一对多数据依赖，需要跨节点网络传输数据，是 Spark 的最大性能开销。Shuffle 的目的是将相同 Key 的数据汇聚到同一分区，完成分组、聚合。常见算子：groupByKey、reduceByKey、join、sortByKey。

7.2.2 Action 行动算子

Action is performed when we need actual dataset results; it does not form a new RDD and triggers DAG execution.

行动算子用于获取最终计算结果，不会生成新 RDD，同时触发 DAG 完整执行。行动算子返回非 RDD 结果（数值、列表、文件），是**真正触发集群计算**的唯一入口。所有懒加载的转换操作，都会在遇到 Action 后统一执行。结果可返回到 Driver 端或写入外部存储。

三个副作用：

1. 触发 DAG 执行 Triggers the execution of the DAG

2. 返回结果给程序 Returns the results to the program

3. 或者把数据写入 HDFS 或文件 Or writes the data to HDFS or to a file

看返回值：返回 RDD ⇒ Transformation；返回非 RDD（数字、列表、写文件）⇒ Action。Transformation 产生新 RDD，Action 不产生。这是两者的形式差别；语义差别是记录 vs 求值。

Action 是惰性的终点： Action 触发执行。所以一个 Spark 程序里，只有 Action 会出现真正的计算。

极简区分口诀：转换懒加载、只构图不运算；行动触发执行、出结果、跑任务。

7.3 DataFrames & Spark SQL

DataFrames are distributed datasets organized in named columns, similar to a relational database or a Pandas DataFrame, immutable, and track lineage.

DataFrame 是按命名列组织的分布式数据集，结构类似关系型数据库表、Pandas 表格，具备不可变性与血缘追踪能力。Pandas DataFrame 是单机、仅能处理小数据；Spark DataFrame 是分布式、支持 TB/PB 级大数据。和 RDD 一致，不可变、可容错、支持分布式并行计算。

DataFrame 构建方式

Construct from files, existing DFs, or a parallelizing Python collection.

可通过读取文件、转换已有 DataFrame、并行化本地集合三种方式构建。

1. 读文件 Read from file(s)

2. 从已有 DataFrame 变换 Transforming an existing DF (Spark or Pandas)

3. 把 Python 的 list 并行化 Parallelizing a Python collection list。

三种方式对应三个数据来源 ・外部文件 / 已有的表 / 内存里的 Python 对象。第二条特别实用：可以把小规模的 Pandas DataFrame 接进来。

RDD 与 DataFrame 对比扩展

RDD 是底层接口，DataFrame 建在它之上，并且多了模式与优化。RDD 是底层低级 API，无数据结构描述，Spark 无法自动优化，需要手动优化代码。

DataFrame 基于 RDD 构建，自带 Schema 表结构（列名、数据类型），Spark 可通过 **Catalyst 优化器**自动优化执行计划，运行速度远快于原生 RDD，是企业开发主流用法。

原文	层级 / 含义
RDDs provide a low-level interface into Spark	RDD 是面向 Spark 的底层接口
DataFrames are built on top of the RDDs and the core Spark API	DataFrame 建立在 RDD 与 Spark Core API 之上
DataFrames have a schema	DataFrame 有模式（schema）即列名与类型
DataFrames are cached and optimized by Spark	DataFrame 被 Spark 缓存并优化

7.4 Spark DAG 有向无环图

DAG consists of vertices (RDDs) and edges (operations). Action triggers DAG submitted to scheduler and splits into stages.

DAG 由顶点（RDD）和边（算子操作）组成，调用 Action 后，DAG 提交调度器并切分为多个 Stage 执行。DAG 完整记录数据计算血缘与依赖关系。

Stage 划分核心规则：**遇到宽依赖 Shuffle 就切分 Stage**，单个 Stage 内部全部为高效窄依赖操作。集群根据 Stage、Task 完成分布式并行计算，故障时可根据 DAG 血缘重算恢复数据。

边清单：A→B、B→C、B→D、B→E、C→E、S→D、D→E、E→F

顶点 = RDD，边 = 操作（Transformation）；Action 触发 DAG 提交给 DAG Scheduler，再由它切成 stages；会算一张给定 DAG 的源点 / 汇点 / 汇合点与边数。

Part 8 Spark Application

8.1 Spark's Main Use Cases 主要用例

三类用例：时序类（Streaming、Batch、Interactive）、算法类（Machine Learning、Graph、Spatial/GIS）、数据组织类（Data Warehousing、Exploratory Data Analysis）。

1. 流处理 Streaming Data 流式数据。Spark Streaming

2. 机器学习 Machine Learning 机器学习。MLlib

3. 交互式分析 Interactive Analysis 交互式分析。

4. 数据仓库 Data Warehousing 数据仓库。Spark SQL

5. 批处理 Batch Processing 批处理（与 MapReduce 重叠的那一块）。

6. 探索性数据分析 Exploratory Data Analysis 探索性数据分析（EDA）。

7. 图数据分析 Graph Data Analysis 图数据分析。GraphX

8. 空间（GIS）数据分析 Spatial (GIS) Data Analysis 空间/GIS 数据分析。

8.2 Spark in the Real World

公司	行业	核心技术组合	业务目标
Uber	出行	Kafka + Spark Streaming + HDFS	实时 ETL：原始事件 → 结构化数据
Pinterest	图片社交	Spark ETL + Spark Streaming	实时推荐相关 Pin、推断购买与旅行意向
Conviva	流媒体视频	Spark	优化视频流 / 管理直播流量，降低 churn

公司	行业	核心技术组合	业务目标
Capital One	金融	Spark + data science algorithms	理解客户、欺诈侦测
Netflix	流媒体	Spark	观看习惯洞察 + 影片推荐 + 内容创作

8.3 Spark的边界

三条限制

Even though Spark is versatile, that doesn't mean Spark's in-memory capabilities are the best fit for all use cases

原文	解读
For many simple use cases Apache MapReduce might be a more appropriate choice	对许多简单用例，Apache MapReduce 可能更合适
Spark users are required to know that memory they have is sufficient for a dataset	用户必须自己确认内存够放这个数据集
Spark was not designed as a multi-user environment	Spark 不是为多用户环境设计的

局限 2 + 局限 3：Spark 把中间数据放在共享的内存里，内存是独占且有限的资源，既要求单次数据集能装下（局限 2），又要求没有别的用户来抢（局限 3） 局限 1：通用框架有额外开销，简单任务的调度开销可能大于收益。

8.4 Summary

块	课件原文
Hadoop (HDFS, MapReduce)	Provides an easy solution for processing Big Data；Brings a paradigm shift in programming distributed system 为大数据处理提供简单的解决方案；带来了分布式系统编程的范式转变。
Spark	Has extended MapReduce for memory computations；for streaming, interactive, iterative and machine learning tasks 为内存计算扩展了 MapReduce；四类任务：流式 / 交互式 / 迭代式 / 机器学习。
Changing the World	Made data processing cheaper and more efficient and scalable；It is the foundation of many other tools and software 让数据处理变得更便宜、更高效、更可扩展；是许多其他工具与软件的基础。

Lecture 6 神经网络

Part 1 Neural Network：神经网络

问题定义：神经网络的沉浮史 → 图像识别 → 二分类 vs 多分类 → 车辆检测算例（2500 / 7500 / ≈300 万特征）→ 线性假设的两条不足。

线性假设需要手工造组合特征，而组合数会爆炸。

1.1 神经网络 Neural Networks for Classification

Neural Networks for Classification are very powerful, but have been dumped for decades because of their complexity and large computational resources

神经网络被弃用，因为复杂性和算力需求赶不上。 Resurrected a few years ago because of cheaper CPU, RAM, GPU, large clusters with many machines, high-speed networks, and many more training examples (big data). Now, like Deep Learning.

复兴原因：算力条件（更便宜的CPU / RAM / GPU 处理器、更大的集群large clusters with many machines、更快的网络 high-speed network）；数据条件（样本更多、就是大数据many more training examples, big data）。

深度学习是神经网络复兴后的名字，不是另起炉灶的新方法。神经网络是方法，深度学习是它复兴后的名字，图像识别是它最典型的任务。

1.2 Classification

对比项	二分类	多分类
标签写法	Giraffe / Not Giraffe	Giraffe / Car / Cat
正类个数	1 个正类，其余全归为负类	k 个互斥类别，每张图恰好属于 1 个
输出层	1 个输出单元（sigmoid → 概率）	k 个输出单元+ Softmax
真值 y 的形式	标量 0 或 1	one-hot 向量，只有 1 维是 1
用到的损失	交叉熵的 k = 2 特例	交叉熵/ 均方误差

1.2.1 Binary Classification

只有是 / 不是两种结果，可以用一个概率表示。三张图里只有长颈鹿是 Giraffe，另外两张（车、猫）都被归为 Not Giraffe。正类 1 个、负类归成一堆。

1.2.2 Multi-class Classification

类别数从 2 变成 3，输出层就不能再用一个概率表示。需要多个输出单元，每个类别一个三个互斥标签，同一组图换成 Giraffe / Car / Cat 三个互斥类别，每张图只属于其中一个。

1.2.3 Example: Car Detection

如果把像素两两相乘当特征，会有 300 万个特征。所谓识别，就是从这堆数字里判断它是不是车，输入是数值，不是概念。

灰度图每像素 1 个数 ⇒ 50 × 50 = 2500；RGB 每像素 3 个数（R/G/B）⇒ 2500 × 3 = 7500。

x 把 50×50 的图按顺序摊平成一个 2500 维列向量，图像从此就是普通的特征向量，可以喂给任何学习算法。

二次特征的量级 ・把像素两两乘积 $x_i \times x_j$ 也当特征，个数 ≈ $n^2 / 2 = 2500^2 / 2 \approx 312$ 万，所以课件写 ≈ 3 million features。

1.3 Motivation

动机总结：复杂度爆炸，所以换方法

1. Need non-linear decision boundary

需求一：非线性边界，右上角的图：x_1 - x_2 平面上蓝色空心圈与红色叉无法用一条直线分开。

2. Need complicated hypothesis with different combinations of features

需求二：复杂的组合假设，算式里除了一次项 $\theta_1 x_1$、$\theta_2 x_2$，还出现了特征两两相乘的项，$ g(\theta^T X) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_k x_1 x_2 + \theta_{k+1} x_1 x_3 + \cdots)$

The computation complexity is extremely large. Hence we have to take another method, called Neural Network.

线性假设不够 → 手工加特征组合 → 组合数爆炸（≈3 million）→ 算不动 → 改用神经网络自动学特征。

Part 2 Perceptron：感知机

最小单元。大脑启发 → 线性神经元的阈值与偏置 → Sigmoid 激活 → 感知机四部件 → AND / OR 的权重解 → XNOR 线性不可分 → 两层网络解决。

2.1 Models and Perceptron

2.1.1 Models

It learns new features from input features. 模型自动学特征 Its architecture is based on our brain structure. 架构来源于人的神经网络，但并不模拟生物机制 The axon terminal（轴突末梢）of one neuron connects with the dendrite（树突）of another neuron, which consists of quite a complicated neural network.

2.1.2 Perceptron: The Artificial Neuron

$x_i $are inputs and $a = h(x; \theta)$ is the output.

> 感知机 = 把输入向量 x 映射成输出 a 的函数 h(x; θ)，θ 是要学、要逼近最优的那个参数组
>
> x is an input vector. In this example, x = (x_1, x_2, x_3) θ is the parameters that result in the best function approximation 需要学习的最佳参数

Node in orange represents the body of neuron.

2.2 A Simple Linear Neuron 线性神经元：阈值与偏置

运算：加权和 + 阈值判定 把偏置并进权重

输出结果是 1 或 0 两种，没有中间值。逻辑神经元是光滑的 sigmoid。

三种等价写法：阈值式（$\geqslant -b$）⇔ 偏置式（$+b \geqslant 0$）⇔ 内积式（$wx \geqslant 0$）。

2.3 Logistic Neuron 与 Sigmoid 激活

把阶跃换成光滑的 sigmoid，输出就变成 (0,1) 区间里的数，可当概率用。

2.3.1 Linear vs Logistic

Logistic Neuron: Sigmoid, Non-linear Activation Function Linear：直接输出

$h(x; \theta) = wx$

Logistic：经过激活函数 f（= sigmoid），输出被压进 (0, 1)

$h(x; \theta) = f(wx) = 1/(1 + e^{(-wx)})$

2.3.2 Perceptron four parts

一个感知机的四个部件：输入 → 参数 → 加权和 → 激活 → 输出

部件	说明
Inputs	输入 x_i（可以是像素矩阵拉直后的向量）
Parameters (weights and biases)	权重 $w_1 \cdots w_n$ 与偏置 b（ = w_0）
Weighted Sum	$\Sigma\ w_i x_i$ 或内积 wx
Activation Functions	sigmoid（也可换成别的非线性函数）

Tips：sigmoid 是非线性激活（Non-linear Activation Function）。如果激活也是线性的，多层叠加仍等价于一层线性模型，非线性是多层有用的前提。

2.4 特征学习 Feature Learning:

一个神经元能学出什么：AND 与 OR 都是线性可分问题，各由一组权重实现。

2.4.1 AND：权重已知的算例

先算 $ z = w₁x₁ + w₂x₂ + w₀$，再取 g(z)。

2.4.2 OR：同样的形式，权重留白

把三个权重都留给读者自己解。

一组可行解：取 $w_1 = w_2 = 20$、$w_0 = -10$，符合 OR 真值表。

与 AND 的差别：AND 的 $w_0 = -30$，OR 的 $w_0 = -10$；两条权重都是 20。偏置决定门槛有多高。

2.4.3 线性假设 Linear Hypothesis

两者都是线性可分问题，一个单神经元就能解决。

衔接：线性不可分的反例，XNOR。

2.5 Non-Linear and Multi-Layer NN

问题：XNOR 一条直线分不开，加一层隐藏层就能分开，多层的价值在这里被具体化。

解决： Multi-Layer NN Model $ x₁, x₂ → a₁, a₂ → h$

a_1 的偏置 −30； $x_1 \to a_1 = 20$，$x_2 \to a_1 = 20$ a_2 的偏置 +10； $x_1 \to a_2 = -20$，$x_2 \to a_2 = -20$

$ $a_1 \to h = 20$，$a_2 \to h = 20$$；h 的偏置 −10

第一层学出 AND 与 NOR 两个中间特征，a_1 算的是 AND（只有 (1,1) 为 1），a_2 算的是 NOR（只有 (0,0) 为 1）。

第二层再把这两个新特征 OR 起来 → 正好是 XNOR。

a_1 列 = 0,0,0,1 是 AND；a_2 列 = 1,0,0,0 是 NOR；h 列 = 1,0,0,1 是 XNOR。

Part 3 多层神经网络与前向计算

模型的语言：三种网络形状 → $a_i^{(j)}$ / $w_ki^{(j)}$ ；

符号约定 → 一层的标量式与矩阵式 → 网络 = 函数复合。

3.1 网络形状与符号约定 Neural Network and Notation

1. input layer (4) , hidden layer (5/3/2) , output layer (1)

四层网络；三层隐藏层说明深就是把隐藏层一路叠下去

2. Input layer (3) , hidden layer (4) , output layer (2)

输出 2 个单元，适用于二分类或多标签任务

3. input layer (3) , hidden layer 1 (4) , hidden layer 2 (4) , output layer (1)

两个隐藏层 + 单输出，适用于二分类

3.2 Model Representation

符号	读法要点
$a_i^{(j)}$ 第 j 层第 i 个神经元（同时也是它的输出值）	上标 j 是层号，下标 i 是层内序号；神经元与它的输出共用同一符号
$w_ki^{(j)}$ 从 $a_i^{(j-1)}$ 连到 $a_k^{(j)}$ 的权重	第一个下标 k 是去哪，第二个下标 i 是从哪来
X 输入向量	$(1, x_1, x_2, x_3)^T$，第一维恒为 1（偏置项）
$a^{(j)}$ 第 j 层的输出向量	$(1, a_1^{(j)}, \cdots)^T$，同样第一维为 1
θ 全部参数的统称	每层各有一份：$\theta^{(1)}$、$\theta^{(2)}$

前向传播

1. 逐个神经元写 Connections between Input Layer and Hidden Layer 1:

$$ a_1^{(1)} = f^{(1)}(w_{10}^{(1)} + w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3) $$ $$ a_2^{(1)} = f^{(1)}(w_{20}^{(1)} + w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{23}^{(1)}x_3)$$ $$ a_3^{(1)} = f^{(1)}(w_{30}^{(1)} + w_{31}^{(1)}x_1 + w_{32}^{(1)}x_2 + w_{33}^{(1)}x_3)$$

三条式子结构相同：一个偏置 + 三个加权输入，外面套同一个激活 $f^{(1)}$。三条并列，就是三个隐藏神经元各自的计算

$w_{10}^{(1)}/w_{20}^{(1)}/w_{30}^{(1)}$即 bias，三个偏置和 b 是一回事，只是写成了这个形式。

2. 合并成矩阵：Rewrite it by matrix calculation

$a^{(1)} = f^{(1)}(W^{(1)}x)$

Function $f^{(1)}$ applies to each component in a vector $f^{(1)}$

逐分量作用，因为 sigmoid 一次只能算一个数，向量上的用法是对每个分量分别算

$W^{(1)}$ 的三行对应三个隐藏神经元，四列对应 (偏置1, x_1, x_2, x_3)；矩阵乘法的结果就是一个 3 维向量。

3. 逐层复合：Layer Composition

把隐藏层 1 → 输出层也写成矩阵式，然后把两层叠起来，网络的最终假设就是各层函数的复合。

Connections between Hidden Layer 1 and Output Layer:

$a_1^{(2)} = f^{(2)}(w_{10}^{(2)} + w_{11}^{(2)}a_1^{(1)} + w_{12}^{(2)}a_2^{(1)} + w_{13}^{(2)}a_3^{(1)})$

对比上一步，把输入从 x 换成 $a^{(1)}$：上一层的输出就是下一层的输入。bias 同样存在。

$ $a^{(2)} = f^{(2)}(W^{(2)} a^{(1)})$

Function $f^{(2)}$ applies to each component in a vector

输出层只有一个神经元，所以结果是一个标量（写成 1 维向量）。

第一层算出来的 $a^{(1)}$ 只有三个数，进第二层前前面补一个 1，才能让第二层的偏置项进得来。每往下一层都要补一次。叠起来：整个网络是一个复合函数

$a^{(2)} = f^{(2)}(W^{(2)}a^{(1)}) = h^{(2)}(a^{(1)}; \theta^{(2)})$ $ $a^{(1)} = f^{(1)}(W^{(1)}x) = h^{(1)}(x ; \theta^{(1)})$ $ $a^{(2)} = h^{(2)}(h^{(1)}(x ; \theta^{(1)}) ; \theta^{(2)}) = h(x ; \theta)$

Each layer is a function. 每一层 = 一个函数 $h^{(l)}(x; \theta^{(l)})$ The final value is a function composition from layer functions. The latter one will be the outer one.

整个网络 = 各层函数依次复合，训练网络就是拟合一个复合函数，不是拟合 k 个独立函数；后面的层写在外层；全网络只有一组参数 θ，梯度下降是对整体 θ 做的。

三层结构被压缩成$h(x; \theta) = h^{(2)}(h^{(1)}(x))$。深：再叠一层就是再复合一个函数。

Part 4 多分类输出与 Softmax

多分类要多个输出单元，还要一层归一化才能当概率用。

4.1 多分类 Multi-class Classification

输出向量之和不是 1，不能直接当概率。

Multiple output units: One-vs-all 每个输出单元 负责 回答 是 不是 这一类。输出单元数 = 类别数 k。

one-hot：真值 $y^{(i)}$ 是四个向量之一，只有一个分量为 1，其余为 0。

问题：课件给的输出向量不是概率分布。单看这组数，既不能求和为 1、也不好直接比较。如果按取最大（argmax）判断，最大的是 0.3，对应第 4 类 Truck。

解决：把 [0.2, 0.1, 0.05, 0.3] 这种原始输出归一化成概率向量，这就是 Softmax。原始输出的各分量之和不一定为 1，需要 Softmax 才能当概率用。

4.2 Softmax Function

Softmax 是输出层的激活函数：接收任意实数向量，输出一个和为 1、每项非负的概率向量。

Softmax is an activation function to normalize the outputs. Given a real vector a ∈ R^k (Classification with k classes)

$ s_j(a) = e^{(a_j)} / Σ_{i=1..k} e^{(a_i)}$

性质	数学形式	作用
非负	$< br > e^{(a_j)} > 0$	分子是正数，输出恒为正
归一	$\Sigma_j s_j(a) = 1$	分母是所有 e^(a_i) 之和 ⇒ 输出和为 1

性质	数学形式	作用
放大最大项	$a_j \gg a_i \Rightarrow s_j(a) \to 1$	指数把差距放大，明显的类更明显
可当概率	s(a) 是 probability vector	固定搭配：softmax输出后配交叉熵Cross-entropy loss，直接与 one-hot 真值比较

Softmax 是激活函数（activation function），作用位置是输出层；它本身不带参数，只是归一化。

Part 5 代价函数与梯度下降

5.1 代价函数：Cost Function

作用：衡量模型预测结果到底有多差

符号	含义	对应
$y = (y_1, \cdots, y_k)$	ground truth，one-hot 向量（ $y_j = 1$ 当且仅当属于第 j 类）	四个 4 维向量
$h(x; \theta)$	probability prediction，第 j 个分量给出第 j 类的预测概率	s(a)，即 Softmax 之后的输出
$h(x; \theta)_j$	第 j 类的预测概率（下标取分量）	0.2 / 0.1 / 0.05 / 0.3
cost function	评价预测有多差的函数	Cross-entropy loss and mean squared error cost

对于k分类任务： 真实标签$y = [y_1, y_2 \cdots y_k]$

one-hot 编码：样本属于类别 j 则$y_j = 1$，其余全部 = 0；

模型预测输出向量$h(x; \theta)$，$[h(x; \theta)]_j$ 代表模型预测该样本属于第 j 类的概率

不同任务可以选不同代价函数，h(x; θ) 与 y 的区别是 y 是硬标签（只有 0/1），h(x; θ) 是软概率（0 到 1 之间的实数）。代价函数就是衡量这两者的差距。

different scenarios：不同场景用不同代价函数，交叉熵与均方误差不是替代关系，而是并列可选项。

y 是真值（one-hot）、h(x; θ) 是预测概率向量、J(θ) = L(y, h) 是代价。

5.1.1 交叉熵损失：Cross-Entropy Loss

分类问题的常用代价：只惩罚真值的预测概率，预测得越不自信，损失越大。

$L(y,h)=-\sum_{j=1}^k y_j\log_2[h(x;\theta)]_j=-y^T\log_2 h(x;\theta)$

- y 只有 1 个位置是 1，其余为 0，公式简化：$L = -\log$ (模型预测真实类别的概率)
- 如果模型给真实类别的预测概率很低 → 损失 L 很大； 训练目标：调整参数 θ，尽量让损失变小。

预测低的 Loss越大，越不自信，罚得越狠，而且是指数级的严厉。

算例：若softmax [0.2584, 0.2338, 0.2224, 0.2855]，则 $L = -\log_2 0.2855 = 1.8083$ bit。

Change θ to keep cost as low as possible 改变参数使代价最小，即使用梯度下降。

5.1.2 均方误差损失：Mean Squared Error Cost

$L(y,h)=\frac1k\sum_{j=1}^k\big([h(x;\theta)]_j-y_j\big)^2$

- 预测值 h 和真实 y 越接近，损失 L 越小
- 分类任务也可以用，但不如交叉熵好用（梯度容易消失）

同一个任务可以换一把「尺子」：均方误差衡量预测向量与真值向量之间的欧氏距离，本页与上一页构成两种并列选择。

对比项	交叉熵	均方误差
形式	$-\Sigma_j y_j log_2 h_j$	$(1/k)\,\Sigma_j (h_j - y_j)^2$
用到 y 的几维	只有 1 维（因为 y 里只有一维是 1）	全部 k 维（每维都参与平方）
对自信的错误的惩罚	极严厉（log 在 0 附近发散）	有上界（最大 1，因为 h_j、y_j ∈ [0,1]）
梯度表现	误差越大梯度越大，收敛快	预测接近 0 或 1 时梯度会变小（饱和）

算例：真类为第 4 类，h = [0.2584, 0.2338, 0.2224, 0.2855]，各项平方和为 0.6813，除以 k = 4 得 L = 0.1703。

两种损失区别：交叉熵只取真值那一维、对自信的错误惩罚更狠；均方误差用全部 k 维、惩罚有上界。

5.2 梯度下降：Gradient Descent

一阶导数迭代优化算法，用来找损失函数的局部最小值

Gradient descent is a first-order iterative optimization algorithm for finding a local minimum of a differentiable function. The idea is to take repeated steps in the opposite direction of the gradient of the function at the current point, because this is the direction of steepest descent.

核心思想：沿着**梯度的反方向**一步步更新参数，直到走到（局部）最小值。

$x = x_0 - \alpha\left.\frac{\partial f(x)}{\partial x}\right|_{x=x_0}$

- αlearning rate 学习率，控制每一步走多大。太大可能越过最低点甚至发散；太小则收敛很慢。
- $∂f/∂x $当前的导数（陡峭程度与方向）。
- 减号表示往梯度反方向走，梯度指向上升最快的方向，所以要下降就走反方向。

神经网络参数更新公式：

$w_i = w_i - \alpha \frac{\partial J(\theta)}{\partial w_i}$

每一步用参数的梯度，减去学习率 × 梯度，更新权重 w，降低代价 J

局限： local minimum

Part 6 反向传播：误差传回

链式法则 + 中转量复用 + 反转的工作流 + 梯度乘权重。

6.1 Chain Rules

神经网络是多层嵌套复合函数，求梯度必须用链式法则：

若 $z = f(y), y = g(x)$

$\frac{\partial z}{\partial x}=\frac{\partial z}{\partial y}\cdot\frac{\partial y}{\partial x}$

例子：$z=\frac12(x-5)^2$，令$y=x-5,\ z=\frac12 y^2$ $z'(x) = f'(y) \cdot g'(x) = y \cdot 1 = x - 5$

6.2 单层网络及梯度拆解 Single Network and Gradient Decomposition

$a = h(x; \theta) = f(z) = f(w_0 + w_1 x_1)$

$z = w_0 + w_1 x_1$

$J(\theta) = L(y, a)$

- f 激活函数；a 输出预测；J损失函数；
- 目标：求权重w_1, w_0的梯度，用梯度下降更新 ；
- 链式展开：

$\frac{\partial J}{\partial w_i}=\frac{\partial J}{\partial a}\cdot\frac{\partial a}{\partial z}\cdot\frac{\partial z}{\partial w_i}$

1. $\frac{\partial J}{\partial a}$：代价函数对输出 a 求导，选定的损失函数决定；
2. $\frac{\partial a}{\partial z}$：激活函数 f 的导数，选定的激活函数决定；
3. $\frac{\partial z}{\partial w_i}$：线性部分$z = w_0 + w_1 x_1$对权重求导

6.3 两层网络

两层网络（每层 1 个神经元，引出反向传播）

$a^{(2)} = f(w_{21} a^{(1)} + w_{20}) = f(w_{21} f(w_{11} x_1 + w_{10}) + w_{20})$

w_{10}, w_{20}是 bias 偏置

网络链路：

$x_1 \to z^{(1)} \to a^{(1)} \to z^{(2)} \to a^{(2)} \to J$

继续链式展开：

$\frac{\partial J}{\partial w_{11}}=\frac{\partial J}{\partial a^{(2)}}\frac{\partial a^{(2)}}{\partial z^{(2)}}\frac{\partial z^{(2)}}{\partial a^{(1)}}\frac{\partial a^{(1)}}{\partial z^{(1)}}\frac{\partial z^{(1)}}{\partial w_{11}}$

节点	关系	图中标注
$z^{(1)}$	由 x_1 与权重 w_{10}、w_{11} 汇入	第一层的加权和
$a^{(1)}$	由 $z^{(1)}$ 经过外层函数得到	第一层的输出（下一层的输入）
$z^{(2)}$	由 $a^{(1)}$ 与权重 w_{20}、w_{21} 汇入	第二层的加权和
$a^{(2)}$	由 $z^{(2)}$ 得到，即 prediction	网络的最终预测
J	由 $a^{(2)}$ 与真值 y 得到	代价

6.4 反向传播：Backward Propagation

核心：从输出层往输入层反向算梯度，复用已经算好的结果 $\delta J/\delta z$，避免重复计算。

正向传播：从左往右，输入→计算每层z, a，得到预测与损失；

反向传播：从右往左（输出层往回）逐层求梯度。梯度反向流动，乘权重，一层一层往前传。

1. 第 L 层神经元先算出$\frac{\partial J}{\partial z^{(L)}}$
2. 用这个值，计算本层权重梯度 $\frac{\partial J}{\partial w_{Li}}$
3. 再乘以本层权重，把梯度传递给上一层 (L-1)：$\frac{\partial J}{\partial a^{(L-1)}}=\frac{\partial J}{\partial z^{(L)}}w_{L1}$
4. 上一层继续计算$\frac{\partial J}{\partial z^{(L-1)}}$，不断向前传递

拓展：一层多个神经元

公式形式不变，需要对后一层所有神经元求和：

$\frac{\partial J(\theta)}{\partial a_k^{(L-1)}}=\sum_{j=1}^{n_L}\frac{\partial J(\theta)}{\partial a_j^{(L)}}\frac{\partial a_j^{(L)}}{\partial z_j^{(L)}}w_{jk}^{(L)}$

n_L = L 层神经元数量

每一层把 $\delta J/\delta a$ 这个信号传给上一层，上一层再补上自己那段激活导数继续往下传，逐层回传。

量	单神经元	一层多神经元
本层加权和	$z = w_0 + w_1 a^{(L-1)}$	$z_j^{(l)} = w_j0^{(l)} + \Sigma_k w_jk^{(l)} a_k^{(L-1)}$
本层权重梯度	$\delta J/\delta w_L i = (\delta J/\delta z)(\delta z/\delta w_L i)$	$\delta J/\delta w_j k^{(l)} = (\delta J/\delta a_j^{(l)})(\delta a_j^{(l)}/\delta z_j^{(l)})(\delta z_j^{(l)}/\delta w_j k^{(l)})$
回传给上一层	$\delta J/\delta a^{(L-1)} = (\delta J/\delta z^{(l)}) \cdot w_L 1$	$\delta J/\delta a_k^{(L-1)} = \Sigma_j (\delta J/\delta a_j^{(l)})(\delta a_j^{(l)}/\delta z_j^{(l)}) \cdot w_j k^{(l)}$
是否需要求和（二者区别）	不需要（只有一个上层神经元）	需要，求和范围 1…n_L

6.5 多条训练记录的训练循环：Multiple Training Records

把单样本的前向 → 算代价 → 反向 → 更新（Forward → Calculate Cost → Backward → Update weights）放进数据集循环，构成完整的训练流程。

训练集 $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$，对每一条样本：

1. 前向传播 Forward Propagation：从左向右，逐层算出所有层$a_j^{(L)}$
2. 计算当前样本的代价 $J(\theta)$
3. 反向传播 Backward Propagation：从右向左，逐层算出$\frac{\partial J}{\partial z_j^{(L)}}$和所有权重梯度$\frac{\partial J}{\partial w_{jk}^{(L)}}$
4. 更新后的 θ（用梯度下降）：梯度下降更新全部权重，循环迭代直到损失收敛

Part 7 从神经网络到深度学习

层数堆上去带来的问题：错误率、层次化特征。

7.1 深度学习

浅层前馈网络 vs 深度神经网络

Non-deep feedforward NN（浅层前馈网络）：单个隐藏层，特征提取能力弱，依赖人工设计特征，泛化能力差。

Deep neural network（深度神经网络）：⩾2 个隐藏层，多层堆叠，分层自动特征学习，无需人工设计特征。

> 定义：深度 = 隐藏层数量，层数越多，网络越深

ImageNet：深度带来性能飞跃（Revolution of Depth）

网络越深，图像识别错误率越低；ResNet 解决了深层网络训练难的问题。

深层的好处：层次化特征

- 浅层：学习底层基础特征。线条、边缘、色块、纹理
- 中层：学习局部组合特征。轮廓、部件（眼睛、鼻子、纹理块）
- 深层：学习高层语义特征。完整物体、人脸、场景、抽象概念

深度学习的本质：通过多层嵌套，自动从数据中学习从简单到复杂的特征层级。

7.2 Modern Applications

7.2.1 Generative Pre-trained Transformer（文本）

GPT 的整体流水线：Input → Input Embedding（旁边并入 Positional Encoding）→ Dropout → Transformer Block Layer 1 → Layer 2 → ⋯ → Transformer Block Layer L → LayerNorm → Linear → Softmax → Output。

Transformer Block 是重复单元，堆叠 L 层。单个Transformer Block的内部：LayerNorm → Linear（分出三路 → 就是 Q/K/V）→ 多头注意力（三路各自走 MatMul / Mask / Softmax / Dropout，图上标 Head1 / Head2 / Head3）→ Linear（合并各头）→ Dropout → 残差相加 → LayerNorm → Linear → GELU → Linear → Dropout → 残差相加 → 输出。

残差连接：一处在注意力之后、一处在 GELU 前馈之后。

7.2.2 扩散模型：Diffusion Model（文生图）

扩散模型是生成模型：输入一段文字，输出一张图。

两代并排是为了展示迭代速度：同一类模型在一年内效果明显提升。

生成模型的另一条技术路线是 GAN，两者都在回答如何让网络生成新数据。

Part 8 卷积神经网络

网络过深会出现：梯度消失/梯度爆炸 + 模型退化（层数增加，训练精度反而下降）

ResNet 创新：残差连接（shortcut），允许梯度直接跨层传递，避免链式累乘导致的梯度消失，支持超深层网络训练。

全连接在图像上要 10 亿个参数；局部连接 + 参数共享把这件事变得可行。

8.1 全连接处理图像：Fully Connected on Images

深度网络中 1000×1000 的图像接 1000 个隐藏单元，需要 10 亿个参数。

Spatial correlation is local（空间相关性是局部的：一个像素只和它附近的像素相关）;

Better to put resources elsewhere!（别把资源全砸在每个输入连每个单元上）。

若将1000×1000图像拉直为一维向量输入全连接网络：

1. **参数量爆炸**：产生数亿甚至十亿级参数，算力消耗极大、极易过拟合
2. **丢失空间信息**：二维图像的位置、相邻相关性被彻底破坏
3. **特征冗余**：全连接层认为每个像素独立，忽略图像局部相关性（相邻像素高度关联，远距离像素无关）

8.2 局部连接与参数共享：Reduce Connection to Local Regions

局部连接 + 参数共享 = 卷积

减少参数的两种做法：每个神经元只看图像的一小块（局部连接），同一组参数在图像各处重复使用（参数共享）。

1. 局部连接：不再让神经元接收整张图输入，每个神经元只感知图像局部小区域。大幅减少连接数与参数量，贴合图像局部相关性规律。
2. 参数共享：图像的基础特征（边缘、纹理、角点）具有**平移不变性**：左上角的边缘特征和右下角的边缘特征原理一致。同一个卷积核（Filter/Kernel）在整张图像滑动，全程复用同一套权重参数，极大压缩参数量，这是CNN高效的根本原因。

Convolutions with learned kernels(filter)。学出来的卷积核就是那组共享的小矩阵。

8.3 CNN：Convolutional Neural Network

Convolutional Neural Network (CNN) is proposed to reduce the number of parameters, preserve the image layout information, and make the network deeper 减少参数（局部连接 + 参数共享）、保存图像的布局信息（不是拉直成向量，行列结构保留）、使网络更深（参数省了，就有余力加层）。

Key Components in CNN

流水线：输入图像 → 卷积层（提特征）→ 激活层（非线性）→ 池化层（降维）→ 重复堆叠 → 拉直Flatten → 全连接层 → 分类输出

Input → Convolution → Pooling → Fully Connected → Output。

阶段	包含的层	作用
Feature Extraction	Convolution → Pooling	从像素里逐层抽取特征
Classification	Fully Connected → Output	把抽出的特征映射到类别

8.4 卷积层：Convolution Layer

操作规则：卷积核在特征图上按固定步幅滑动，每滑动一个位置，执行对应位置像素相乘，全部求和，输出一个特征值，最终生成全新特征图。

超参数详解

Kernel Size 卷积核尺寸：常用3×3、5×5，尺寸越大感受野越大

Stride 步幅：每次滑动的像素数，stride越大输出特征图越小

Padding 填充：图像边缘补0，保留边缘特征、控制输出尺寸

卷积核数量：一个卷积核提取一种特征，多个卷积核可同时提取边缘、纹理、色块等多类特征，输出多通道特征图

8.5 池化层：Pooling Layer

降采样：把特征图变小以减少计算量，常见做法是取最大值或取平均值。

作用：

1. Makes the representations smaller and more manageable 让表示更小更易管理；
2. Operates over each activation map independently 逐特征图独立处理；
3. Max pooling and average pooling are widely used max / average 两种最常用；
4. Pooling can speed up the training process 尺寸减半，后面各层的计算量与参数量大幅下降。

两种池化方式对比

最大池化 Max Pooling（最常用）：取窗口内最大值，保留最强特征，抗干扰能力强

平均池化 Average Pooling：取窗口平均值，保留整体特征，但容易模糊细节

关键区别：卷积层有可学习参数，池化层无任何可学习参数，只是固定的统计操作。

例：max pool 2×2、stride 2

Part 9 GAN：生成对抗网络

和Diffusion Model 扩散模型一样，都在回答如何让网络生成新数据。

两个网络、一个损失、方向相反：从纳什均衡到极小极大博弈。

9.1 纳什均衡：Nash Equilibrium

In game theory, the Nash equilibrium, is a set of strategies that players act out, with the property that no player benefits from changing their strategy. Intuitively, if any given player were told the strategies of all their opponents, they still would choose to retain their original strategy.

纳什均衡来自博弈论，是一组策略集合：博弈里所有玩家选定策略之后，任何单独一个玩家单方面改变自己的策略，都不会获得更好的收益。这是 GAN 对抗训练的核心思想。

Nash showed that there is a Nash equilibrium for every finite game, a finite number of players in which each player can choose from finitely many pure strategies

纳什定理：任何有限博弈，一定存在纳什均衡。优先博弈是玩家数量有限，每个玩家可选的纯策略数量也是有限的。 When it reaches Nash Equilibrium, if any competitors change their current strategies, they will get a worse result.

结论：达到纳什均衡后，任意竞争者单方面改变策略，结果会变差。

GAN 里，Generator 生成器、Discriminator 判别器，就是博弈里的两个玩家，训练目标就是到达纳什均衡。

第 62 页先补一段博弈论：GAN 的对抗训练，目标就是让两个网络走到纳什均衡。

9.2 GAN Main Idea and Framework

GAN 包含两个独立神经网络，对抗式同步训练：

1. Generator takes in random noise and returns a fake image.
2. Discriminator takes in both real and fake images and returns probabilities with 1 representing authenticity and 0 fake
3. One neural network, called the generator, generates new data instances
4. One neural network, called the discriminator, evaluates them for authenticity; i.e. decides whether each image is real.

角色	输入	输出	目标
Generator（G）	random noise（随机噪声）	fake image（图中生成了一个手写的 8）	造出以假乱真的样本
Discriminator（D）	both real and fake images	probabilities：1 = 真，0 = 假	分辨真伪

训练循环： 生成器造假图 → 判别器判断真假 → 两者互相迭代对抗，共同提升能力。

真实样本x：从真实数据集采样，送入判别器D，$D(x)$希望越接近 1 越好（识别为真）

噪声z输入生成器G，得到伪造样本$G(z)$送入判别器；

D希望$D(G(z))$尽量接近 0（识别出这是假图）；

G希望$D(G(z))$尽量接近 1（欺骗 D，让 D 以为是真图）

D 和 G 玩极小极大对抗游戏，目标最终达到纳什均衡，联合训练得到强大的生成器和判别器

9.3 极小极大博弈：Min-Max Game

损失函数：

$$L = E_x[\log(D(x))] + E_z[\log(1 - D(G(z)))]$$

符号拆解：

1. $D(x)$：判别器评估真实数据x是真样本的概率
2. $E_x[\cdots]$：在全部真实数据上求数学期望
3. $G(z)$：生成器根据噪声z输出伪造样本
4. $D(G(z))$：判别器评估生成器造的假样本是真样本的概率
5. $E_z[\cdots]$：在全部随机噪声输入上求期望

博弈目标：

- 判别器 D：最大化这个 Loss，尽可能分清真假
- 生成器 G：最小化这个 Loss，尽可能让判别器分不清真假

minimax（极小极大）博弈：D 最大化损失，G 最小化损失，二者对抗。

9.4 GAN 图像合成

GAN 能生成多个类别、逼真度较高的图像。

第 67 页回到粗版大纲 ・三条：Neural Networks / Deep Neural Networks / Overfitting and Underfitting——与第 2 页同一版本。位置在 GAN 之后、Overfitting and Underfitting 之前，标志进入全讲最后一块。

考点 ・ 能列举 GAN 的典型应用场景（图像合成 / 生成）；并知道它与扩散模型同属生成模型两条路线。

Part 10 Overfitting and Underfitting：过拟合与欠拟合

属于机器学习基础问题，GAN 训练也同样会遇到过拟合问题。欠拟合、过拟合是机器学习算法的通用问题，不只在分类任务中出现，在回归任务中同样存在。

10.1 Underfitting 欠拟合

左图	中图	右图
$\theta_0 + \theta_1 x$	$\theta_0 + \theta_1 x + \theta_2 x^2$	$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$
High Bias（Underfitting）	Low Bias, Low Variance（Goodfitting）	High Variance（Overfitting）
一条直线，明显拟合不上	一条平缓抛物线，贴合趋势	一条蛇形曲线，穿过每个点

模型拟合能力太弱，连训练集的数据规律都学不好。

特点：高偏差 High Bias，低方差 Low Variance

原因：模型太简单，假设过于简化，无法捕捉数据内在模式。

例子：用一条直线去拟合带曲线趋势的数据

理想模型：Low Bias + Low Variance，既能学到真实规律，又不会被噪声干扰（$\theta_0 + \theta_1 x + \theta_2 x^2$）

10.2 Overfitting 过拟合

概念	课件原话	中文	什么时候出现
Overfitting	fit the training set very well, but fail to generalize to new examples (high variance)	训练集拟合得很好，但泛化失败	特征太多（too many features）/ 模型太复杂
Bias	error due to overly simplistic assumptions in the learning algorithm	由假设过于简单带来的误差	模型太简单 ⇒ 欠拟合 ⇒ high bias
Variance	error due to the model’s sensitivity to fluctuations in the training data	由对训练数据波动过于敏感带来的误差	模型太复杂 ⇒ 过拟合 ⇒ high variance

模型能力太强，把训练集里的噪声、随机波动也当成规律记住了（曲线）。

在训练集上效果极好，但是泛化能力差，在新的测试样本上表现很差 → 高方差 High Variance

原因：特征数量过多，模型复杂度过高，过度记住训练集的细节。

10.3 Model Complexity 复杂度曲线

训练误差与测试误差画成两条曲线，给出诊断规则。

② 复杂度曲线：训练误差与测试误差（第 72 页）· Underfitting and Overfitting

图中元素	内容
横轴	model complexity，分成 low / medium / high 三档
纵轴	prediction error (1 − prediction accuracy)
实线	Training data 训练误差
虚线	Test data 测试误差
标号 1 / 2 / 3	训练误差上的三个点，分别在 low / medium / high 复杂度
标号 4 / 5 / 6	测试误差上的三个点，分别在 low / medium / high 复杂度

两条曲线之间的间距就是泛化误差。

训练误差高 + 测试误差高 ⇒ 欠拟合（high bias）;

训练误差很低 + 测试误差回升 ⇒ 过拟合（high variance）;

测试误差最低处 ⇒ 好模型。

10.4 Ways to Address

欠拟合 = More data + More powerful model；加（加数据、加模型能力）

过拟合 = Feature Selection + Regularization。减或约束（减特征、压参数）

欠拟合

1. More data
2. A more powerful model. e.g., Neural Networks, rather than simple linear regression

过拟合

手段	做法	适用场景
Feature Selection（特征选择）	Manual selecting which features to keep (by domain knowledge) 手工挑，靠领域知识	Okay esp. when some features are useless.（有些特征完全没用时最合适）
Regularization（正则化）	Keep all features, but reduce their influence by giving smaller values to the parameter θ_i 保留全部特征，但把 θ_i 压小	Okay when many features, each of which contributes a bit to predicting y（特征很多、每个都贡献一点点时最合适）

5434 Big Data Computing 2

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_6/knowledge/5434-Big-Data-Computing-2.md`

5434 Big Data Computing2

Lecture 7 Graph Analytics

本讲是七讲里第一次处理「关系数据」：前几讲的数据都能排成一张表，本讲的信息藏在连接方式里。

七个部分的分工

1. Part 1 图与应用：六类真实网络→ 定义加权 / 有向两种分类 → 邻居与度数 → 路径 → 完全图 → 三种存储模型。这是语言与表示。
2. Part 2 图遍历：BFS与 DFS→ 两页伪代码 → 两张算例。这是所有全局算法的发动机。
3. Part 3 中心性（上）：度中心性（数邻居）→ 接近中心性（到所有人的距离之和取倒数，C(2) = 1/9）→ 算法要对每个顶点各跑一次 BFS。这是局部与几何两种「重要」。
4. Part 4 PageRank：全局重要性 → 随机游走与阻尼因子 d → 迭代更新公式 → 两轮手工迭代算例（n = 4、d = 0.8，结果写成/12 的分数）。这是被重要者指向才重要。
5. Part 5 最短距离与 Dijkstra：最优子结构观察 → 伪代码与松弛 → 五顶点六帧算例（δ(s,t) = 8）→ 归纳法正确性证明（可选）。这是加权图上的最短路。
6. Part 6 顶点相似度：链路预测问题 → Jaccard 相似度（共同邻居 ÷ 邻居并集）。这是用结构猜缺失的边。
7. Part 7 社区与聚类：连通分量（无向）、SCC 与 WCC（有向）、局部聚类系数（无向 1/3 / 有向 1/6）与平均聚类系数。这是把图切开看结构。

Part 1 Graphs and Applications

1.1 社交网络与用户/ 物品图 Social Networks / User-Item Graphs

两者的结构差别：边是不是跨类型的。

1. Social Networks：节点是实体，边是关系

节点（顶点）：users、images 等 all kinds of entities（各种实体）。

边：friendship、followship 等 all kinds of relationships（各种关系）。

任务	中文	在问什么
Community detection	社区发现	把图上抱团的节点群找出来（连通分量与聚类系数）
Link prediction	链路预测	预测两个节点之间将不会不会连上边（Jaccard 相似度就是它的工具）
Product/Ad advertisement	商品/广告投放	按图上的邻近关系决定推给谁
Node classification	节点分类	给节点打标签（例如机器人账号、高价值用户）

1.2 User-Item Graphs：二部图（Bipartite Graph）

两类：users（用户）与 items（物品），这是二部图的第一条特征。

边：ratings / purchasing / reviewing ，边只能连接两类之间的节点，这是二部图的第二条特征。

典型任务：Recommendation systems（推荐系统）。

二部图与普通图的关键差别

对比项	社交网络（普通图）	用户 - 物品图（二部图）
顶点类型	同一种（人）	两种（用户 / 物品）
边的方向性	通常是无向的（好友是互相的）	常有方向（谁买了谁）
边能不能出现在同类之间	可以（人 - 人）	不可以（只有用户 - 物品）
典型分析	社区发现、链路预测	推荐、二部图投影

二部图不是有两种节点的图这么简单，关键是边只存在于两类之间。若允许用户 - 用户边，它就不再是二部图。

考点 ： 看到 Bipartite 就要条件反射地想起两类顶点 + 边只跨类；看到 Recommendation systems 就要想到它是用户 - 物品图上的任务。

1.3 应用：Web / 交易 / 电信 / 航空网络

1. Web Graphs

这是有向图最经典的原型：PageRank（Part 3）就是在这个图上定义的。

顶点Vertices: webpages（网页）;

边 Edges: hyperlinks（超链接），且有向，A 链到 B 不代表 B 链到 A。

2. Transaction Networks

典型任务：欺诈检测（Fraud detection），异常的资金环、异常的中转链在图上比在一张流水表里明显得多。

顶点：账户 / 商户 / 支付平台。

边：一笔转账或支付。

3. Telecommunication Networks

典型任务：社区发现（把通话圈子切出来）、关键节点识别（去掉哪个节点会切断网络）。

顶点：电话 / 手机 / 基站。

边：一次通话或一条链路。

4. Airline Networks

典型任务：枢纽识别（Hub identification，中心性度量）、连通性分析（连通分量）。

顶点：机场（或城市）。

边：一条直飞航线。

四类网络的共同结构

网络	顶点 Vertices	边 Edges	有向?	本讲会用到的工具
Social Networks	users, images, etc	friendship, fellowship, etc	多为无向	连通分量、聚类系数
User-Item Graphs	users, items	ratings, purchasing, reviewing …	常有向	Jaccard 相似度（推荐）

网络	顶点 Vertices	边 Edges	有向?	本讲会用到的工具
Web Graphs	webpages	hyperlinks	有向	PageRank
Transaction Networks	账户 / 商户	一笔支付（Third-party payments）	有向	社区发现
Telecommunication Networks	电话 / 基站	通话 / 链路	可无向	社区发现、连通分量
Airline Networks	机场	直飞航线	多为无向	中心性、连通性

1.4 图的数学定义

Graph model G = (V, E)

V is a set of vertices; E is a set of edges; each edge is for a **pair** of vertices

G = (V, E) 图 = 顶点集合 + 边集合；V 是 vertex（顶点）的集合，E 是 edge（边）的集合。

边的定义：each edge is for a pair of vertices 每条边对应一对顶点。

Tips：定义里说的是 pair（一对），所以单独一个顶点不构成边；但单独一个顶点可以构成一个连通分量。

Example:

项	取值	读法
顶点集合 V	{1, 2, 3, 4}	4 个顶点，编号 1 - 4
边集合 E	{(1,2), (2,3), (3,4), (2,4)}	4 条边
顶点 1 的邻居	{2}	只连到 2
顶点 2 的邻居	{1, 3, 4}	度数为 3（本图度数最大的顶点）
顶点 3 的邻居	{2, 4}	度数为 2
顶点 4 的邻居	{2, 3}	度数为 2

1.4.1 Weighted vs Unweighted / Directed vs Undirected

有向 ≠ 加权，两者是正交的

类型	定义	例子
Unweighted graph（无权图）	An edge does not have any weight 边不带任何权重	Facebook friend relationship（好友关系）
Weighted graph（加权图）	An edge has a weight 边带权重	road segment: length（路段：长度）；communication link: bandwidth（通信链路：带宽）
Undirected graph（无向图）	An edge can be traveled in both ways 一条边两个方向都能走	Facebook friend relationship（加好友是双向的）
Directed graph（有向图）	An edge can be traveled in one way 一条边只能单向走	Twitter follower relationship（单向关注）

1.4.2 邻居与度数 Neighbors and Degree of a Vertex

度数（degree）是图上最基础的局部指标。无向图只有一个度数；有向图拆成入度与出度两个。

1. 无向图：邻居集合与度数

N(v) 表示顶点 v 的邻居集合。上图里 N(2) = {1, 3, 4}，所以顶点 2 的度数是 3。

等价说法：无向图中，度数 = 邻居个数 = 该顶点上的边数。

无向度数 = 入度 + 出度

2. 有向图：入邻居 / 出邻居，入度 / 出度

1.5 路径 Paths

A path is a finite or infinite sequence of edges which joins a sequence of vertices. In most definitions, edges and vertices are **distinct**.

简单路径（simple path）是一串边，把一串顶点依次接起来。路径的长度是边的条数（不是顶点个数）。边不重复、顶点也不重复。

对比项	无向图	有向图
可达是否对称	对称：a 能到 b ⇒ b 能到 a	不对称：4 → 1 可能不存在路径
最短路径	两端之间的最短路	从一个点到另一个点的最短路（方向固定）
连通性概念	连通分量 CC（Part 6.1）	强连通 SCC / 弱连通 WCC

无向图上的路径

1<->2 is a path with length 1；1<->2<->3 is a path with length 2；This is the shortest path between 1 and 3

结论：1<->2<->3 是顶点 1 与 3 之间的最短路径（the shortest path between 1 and 3）因为 1 与 3 之间没有直接相连的边，绕 2 是唯一的两步走法。

有向图上的路径

1->2->3->4 is a path with length 3；1->2->4 is a path with length 2

This is the shortest path from 1 to 4. There is no path from 4 to 1

1.6 完全图与稠密度: Complete Graph

完全图是稠密度的标尺：它给出任意 n 个顶点最多能有多少条边，于是实际边数 ÷ 完全图边数就成了密度的定义。

A graph in which each vertex is connected to every other vertex is called a complete graph.

Densest possible graph

How many edges are in an undirected complete graph with n vertices? —— n(n-1)/2

How many edges are in a directed complete graph with n nodes? —— n(n-1)

类型	边数公式	怎么来的
无向完全图	n(n−1)/2	每个顶点连其余 n−1 个 ⇒ 有向意义上的 n(n−1) 条有序对；无向边 (u,v) 与 (v,u) 是同一条，故除以 2

类型	边数公式	怎么来的
有向完全图	n(n−1)	每个顶点有 n−1 条出边，共 n 个顶点，且 (u→v) 与 (v→u) 是两条不同的边，不除 2

两个用途

Real-world graphs are rarely complete.

Complete graphs are usually used as substructures to decide the density of a graph or a community in a graph

现实中很罕见，真实网络通常是稀疏的，边数远小于 n(n−1)/2。

可以当作分母用，完全图是密度的参照系：图（或社区）的稠密度 = 实际边数 ÷ 完全图边数。

局部聚类系数的分母正是v 的邻居们两两相连最多能有几条边，也就是邻居集合上的完全图边数。

1.7 三种存储模型：Graph Models

四个顶点：V = {1, 2, 3, 4}。

四条边：(1,2) = 1.2、(1,3) = 4.3、(2,3) = 0.8、(3,4) = 2.6，注意这是加权无向图（邻接矩阵对称）。

度数：顶点 1 度 2、顶点 2 度 2、顶点 3 度 3、顶点 4 度 1（4 是唯一的叶子）。

三种存储模型

模型	形状	一句话
边表 Edge List	一张 (V1, V2, Weight) 的表，4 行	最省地方；要查某点的邻居必须扫全表
邻接矩阵 Adjacency Matrix	4 × 4 的方阵 A	查邻居 O(1)；但空间是O(n²)，稀疏图极浪费
邻接表 Adjacency Lists	每行一个顶点，后面跟 邻居(权重)	空间 O(n + m)；遍历邻居最常用

邻接矩阵（完整数值）

对称性：矩阵关于主对角线对称（A[i][j] = A[j][i]），是无向图（有向图一般不对称）。对角线全是 0，说明没有自环。

行和 = 度数（加权图中的强度）：第 1 行 0+1.2+4.3+0 = 5.5；第 2 行 0+1.2+0.8+0 = 2.0；第 3 行 0+4.3+0.8+2.6 = 7.7；第4 行 0+0+2.6+0 = 2.6。

所有行和相加 = 所有边权之和的 2 倍：5.5 + 2.0 + 7.7 + 2.6 = 17.8，而边权和 = 1.2 + 4.3 + 0.8 + 2.6 = 8.9，17.8 = 2 × 8.9

邻接表（完整数值）

顶点 1 的邻居是 2（权重 1.2）和 3（权重 4.3）。每个顶点只出现一次作为行首，所以共 4 行对应 4 个顶点。2 + 2 + 3 + 1 = 8 条，正好是无向边数 4 的 2 倍，因为每条无向边在两端各登记一次。

问题	边表	邻接矩阵	邻接表
查两点间是否有边	扫全表 O(m)	O(1)	扫该点邻居 O(deg)
列出某点的所有邻居	扫全表 O(m)	扫一行 O(n)	O(deg) 最优
空间	O(m) 最省	O(n²) 最费	O(n + m)
适合	存储 / 交换	稠密图、需要矩阵运算	稀疏图的算法（BFS/DFS/PageRank）

Part 2 Graph Traversal Methods：图遍历

起点：从源顶点 s（source vertex）出发。换一个 s，整个遍历的次序就全变了，遍历是关于 s 的，不是关于图的。

覆盖：要走到图中的每一个顶点，一个都不能漏。

次序：按某个一定的顺序访问。BFS 与 DFS 的差别只在这个顺序上，其余（起点、覆盖、只访问一次）两条完全一样。

不重复：每个点只访问一次（visited 一次）。前提：图里有环，不做标记就会绕圈不停。

对比项	BFS（广度优先）	DFS（深度优先）
推进策略	先近后远：把离 s 近的点全走完，才碰远一层的点	能深就深：沿当前顶点一直往下钻，钻不动了才回头
生活比喻	在水里投一颗石子，波纹一圈圈往外扩	走迷宫：一路往前，撞墙才退回来换岔路
课件原话	Visit the vertices level by level	Visit a vertex "deeper" in the graph whenever possible
被谁用到	无权图最短路、社交网络的分层扩散	连通性、环检测、拓扑排序、强连通分量

判断遍历序列题：给一张图与起点 s，要求写出 BFS 序列或 DFS 序列。BFS 的答案唯一（按层）；DFS 的答案依赖邻接表顺序，题目必须指定「按顶点编号从小到大」或给定邻接表，否则答案不唯一。

2.1 广度优先搜索 Breadth-First Search

给定图 $G = (V, E)$ 和源点 s

按距离分层遍历：先访问离 s 更近的所有顶点，再访问更远的顶点，一层一层向外扩展（level by level）

- 遍历轨迹会生成一棵 BFS 树（breadth-first tree），树是无环连通图
- 数据结构：FIFO 队列 Queue（先进先出）

Tips:

规则：离 s 近的顶点没走完，绝不碰离 s 远的顶点，**所以 BFS 拿到了最短路径**。

必须有一个起点 s，以及图 G = (V, E)。

访问序列天然是按层分块 (level by level) 的：第 0 层 {s}、第 1 层、第 2 层……

A tree is a connected graph containing no cycles.

两个条件：连通（任意两点之间都有路）；无环（不存在回到自己的回路）。

副产品：广度优先树，无环 + 连通 → 任取一个点当根，父子关系不会出现一个点有两个爸爸的情形。同一个原图，可以生成**不同的 BFS 树**（邻接点遍历顺序不同）。但无论哪棵树，各顶点的 d 值完全相同。

BFS 森林: 如果图不连通，从 s 出发走不到所有点。此时对每个未访问的顶点再起一次 BFS，得到的若干棵树组成森林。

顶点属性

$u.p$：parent，u 的父节点

$u.d$：源点 s 到 u 的距离（无权图：两点之间最少边的数量）未访问顶点初始距离： ∞（无穷大）

n 个顶点的树恰好有 n − 1 条边。因为每次发现一个新顶点，恰好新增一条树边，从 1 个点长到 n 个点。

用途	产出什么	成立条件
求最短路	从 s 到每个顶点的路径本身（沿 u.p 一路回溯到 s）	仅无权图（或边权全相等）
求最短距离	从 s 到每个顶点的边数 u.d	仅无权图；加权图要用 Dijkstra
检测环	遍历时若遇到一条边指向已访问且不是父亲的顶点，说明有环	无向图需额外排除回到父亲这条边
构造 BFS 树 / 森林	树 = 从单个 s 出发；森林 = 对每个未访问顶点各跑一次	任意图（有向无向皆可）

2.1.1 BFS Example：用队列一层层推

算法思想：

1. 源点距离设为 0，入队
2. 不断从队首取出节点u，遍历它所有邻接节点v
3. 如果v没有被访问（$v.d = \infty$）：更新距离、记录父节点，入队

性质：出队节点的距离一定是升序排列

矛盾点：边数少的那条（直达）反而贵一倍多。BFS 只看边数，看不见权重，于是给出错误答案。当所有边权相等（例如全为 1）时，边数最少与权重和最小重合，BFS 就是加权图上的特殊正确情形。

正确做法：加权图的单源最短路要用 Dijkstra 算法，它的松弛（relaxation）操作专门处理绕远路但更便宜的情形。

高频判断题：加权图上能否用 BFS 求最短路？不能。

复杂度： $O(V + E)$，每个顶点最多入队出队一次（$O(V)$）；每个顶点的邻接表恰好扫一次（$O(E)$）。合计 $O(V + E)$。前提是用邻接表存储，若用邻接矩阵则是 $O(V^2)$。

把队列换成栈会发生什么？答案：退化成 DFS。

2.2 深度优先搜索： Depth-First Search

DFS 与 BFS 的差别只是把队列换成递归栈，但行为完全不同：BFS 像水波扩散，DFS 像走迷宫，沿着一条路一直钻，钻到没新点可走才回头。

核心定义：给定源点 s，尽可能往深处走，走到底走不通再回溯（backtrack），像走迷宫。隐式生成一棵 DFS 树（depth-first tree）；实现方式是递归（本质是调用栈 LIFO 后进先出）

顶点属性

u.p：parent 父节点

u.depth：节点在 DFS 树里的深度；未访问顶点初始深度：∞

deeper = 离起点更远的树的层。只要当前顶点还有未被访问的邻居，就立刻钻过去，不先把同层邻居都走完。

implicitly（隐式）：BFS 的树是显式存的（每个点存 u.p 并连边画出来）；DFS 的树是递归调用栈自己形成的，算法本身不维护一棵树对象。

算法思想：

1. 初始化所有节点深度无穷，源点深度 = 0，调用访问函数
2. DFS-VISIT(u)：遍历 u 的所有邻接点 v
3. v 未访问，则更新深度、父节点，**递归访问 v**

4. 递归触底之后，执行回溯 backtrack

细节	BFS	DFS
初始化范围	$G.V - s$（除源点）	G.V（全部顶点，含源点）
源点赋值	$s.d \leftarrow 0; s.p \leftarrow NIL$（连父亲一起置空）	只写 s.depth ← 0（父亲在初始化那步已置 NIL）
新点处理	标记 d、记父亲、入队（稍后处理）	标记 depth、记父亲、立刻递归（马上处理）
判重字段	v.d = ∞	v.depth = ∞

2.2.1 DFS Example：递归

课件例子（起点 node 1）

遍历顺序：1, 2, 3, 4, 6, 5, 7 步骤：

1. 访问 1 → 访问 2 →访问 3 →访问 4（到底）
2. 回溯回到 3，回溯回到 2
3. 访问 2 的另一个邻点 6 →访问 5（到底）
4. 回溯回到 6，回溯回到 2，回溯回到 1
5. 访问 1 剩下邻点 7

同一个原图，可以生成**不同的 DFS 树**（邻接点遍历顺序不同）

2.3 BFS VS DFS

	BFS 广度优先	DFS 深度优先
核心思路	一层一层向外扩散，就近优先	一路往深处钻，走不动回溯
底层数据结构	队列 Queue（FIFO）	递归调用栈 Stack（LIFO）
适用场景	无权图最短路径，找最近节点	拓扑排序、找环、连通分量、迷宫搜索
遍历特点	按到起点距离从小到大访问	先一条路走到黑，再回头探索其他分支
树生成	BFS 树	DFS 树
树的形式	显式构造	隐式形成
层号字段	u.d（distance，等于最短路）	u.depth（只是树深）
实现要点	while 循环 + if v.d = ∞	for 循环 + if v.depth = ∞ + 递归调用

1. BFS **只适用于无权图最短路径**，带权图 BFS 失效
2. BFS、DFS 树都不唯一，取决于邻接表遍历邻节点的先后顺序
3. BFS 距离 = 源点到该点最少边数；DFS 的 depth 只是树里深度，**不是原图最短距离**
4. BFS：队列；DFS 递归版本靠系统栈，迭代版需要手动栈

Part 3 Centrality Measure

前置基础：BFS 用于无权图最短路径；Degree、Closeness 多用于无向图；PageRank 用于有向图（网页超链接）

核心目标：衡量网络中每个节点的重要程度（how central/important a vertex is）

三种度量

度量	特点	只看什么	计算代价
Degree Centrality	邻居数量多	局部：只看自己的度数	最便宜，读一遍边即可
Closeness Centrality	到其他所有顶点的距离之和小	全局几何：需要所有点对距离	贵：每个顶点各跑一次BFS
PageRank	被重要的顶点指向	全局迭代：靠迭代求解	较贵：多轮迭代直至收敛

3.1 度中心性：Degree Centrality

Degree centrality uses the vertex degree. Higher degrees means more central. It only considers local direct neighbors, ignoring global network structure.

> 节点的度 = 和该节点直接相连的边数量 Rule：The higher the degree, the more central the node is. 度越大，节点越重要、越处于网络中心。

核心思想：只看直接相邻的邻居（local 局部信息），不考虑整个网络的全局结构。

- 节点拥有越多直接相连的邻居 → 度越高 → 度中心性越高
- 在社交网络例子：一个人好友数量越多，度中心性越高

优点：计算最简单，开销极低，一眼就能算出；直观易懂。

缺点：只看局部，忽略全局信息 只统计直接相连节点，完全不考虑邻居本身是否重要，也不考虑节点到远处节点的距离。

> 例子：一个节点有 5 个普通邻居；另一个节点只有 2 个邻居，但这 2 个是网络超级核心节点。
>
> Degree Centrality 会判定前者更重要，但真实影响力后者更强。
>
> 无法区分边的方向（有向图要拆成入度中心性 in-degree / 出度中心性 out-degree）

适用场景

快速粗略筛选网络中连接多的节点；简单无向网络。比如，社交网络找关键人物、Web 找权威页面、传播场景找阻断谁最有效。能高效地把信息传遍全图的顶点，就是最中心的顶点。

度最高的顶点一定是最中心的吗？不一定。度中心性只数邻居，不看邻居是谁；PageRank 恰恰相反，看的是邻居的质量。反例：页面 E 的入链比 C 多得多，PageRank 却只有 C 的四分之一。

3.2 接近中心性：Closeness Centrality

Closeness centrality $C(v) = 1/\Sigma\, dist(v, u)$. We run BFS for each node to get shortest paths in unweighted graph. Nodes with smaller total distance have higher closeness. It is a global measure. It works only for connected unweighted graphs.

度中心性只看直接连着几个人；接近中心性看从我开始，到全图所有人，加起来要走多少条边，越小越中心。

$C(v)=\frac{1}{\sum_{u} dist(v,u)}$

- $dist(v,u)$：v 到 u 的最短路径距离（无权图：边的条数）
- $\Sigma_u dist(v,u)$：节点 v 到图中所有其他节点 u的最短路径距离之和（总距离）
- 总距离 $\Sigma dist(v,u)$ 越小 → $C(v)$ 越大
- High closeness centrality：这个节点离网络里所有其他节点都很近，信息可以快速传播到全网。

例题：vertex 2 到 1,3,4,5,6,7 的最短距离依次：1,1,2,2,1,2 Sum = 1+1+2+2+1+2 = 9 → $C(2) = 1/9$

顶点 v	到其余 6 个点的距离	距离之和	C(v)
1	1, 2, 3, 3, 2, 1	12	1/12 ≈ 0.0833
2	1, 1, 2, 2, 1, 2	9	1/9 ≈ 0.1111（最大）
3	2, 1, 1, 3, 2, 3	12	1/12 ≈ 0.0833
4	3, 2, 1, 4, 3, 4	17	1/17 ≈ 0.0588（最小）
5	3, 2, 3, 4, 1, 2	15	1/15 ≈ 0.0667
6	2, 1, 2, 3, 1, 1	10	1/10 = 0.1
7	1, 2, 3, 4, 2, 1	13	1/13 ≈ 0.0769

排序 C(2) = 1/9 > C(6) = 1/10 > C(1) = C(3) = 1/12 > C(7) = 1/13 > C(5) = 1/15 > C(4) = 1/17。

算法

Input: Unweighted graph G For each vertex v in graph:

1. Run BFS starting from v
2. Record shortest path distance $dist(v,u)$ from v to every other vertex u
3. Sum all distances, compute $C(v) = \frac{1}{\sum dist(v,u)}$ Return vertex v' with maximum $C(v')$

关键点：BFS 是用来计算无权图最短路径的工具，Closeness Centrality 依赖 BFS，对每个顶点各跑一次 BFS。对 n 个顶点各跑一次，总代价 O(V・(V + E))。稀疏图（E ≈ V）时约为 $O(V^2)$。而度中心性只要 O(V + E)，一次遍历就够。接近中心性贵了整整一个因子 V。

优点：全局指标：考虑节点到网络全部节点的距离，不只是直接邻居；

衡量信息传播速度：closeness 高的节点，消息传递到全网更快。

缺点：只适用于无权连通图。如果图不连通，部分节点之间没有路径，距离无穷大，公式失效；

计算开销更大：每个节点都要单独跑一遍 BFS。n 个节点，复杂度$O(n(n+m))$，大图很慢；

不区分邻居的重要性，只看距离长短。

问题：度中心性为什么不足以刻画重要性？只看数量、不看质量，且只覆盖一步邻域

适用场景：连通、无权无向网络；找网络里 “到所有人最近” 的枢纽节点（交通枢纽、信息中转站）

3.3 Google PageRank

PageRank is a global importance metric for directed graphs, originally for web search. It can be interpreted by random walk. With probability d, surfer follows out-links; with 1 − d, jumps to a random page. We iteratively update PR values. A page is important if many high-quality pages link to it. Sum of all PR = 1.

PageRank：有向图上的全局节点重要性指标，谷歌早期搜索引擎用来给网页排序。

Vertices = web pages；Directed edges = hyperlinks（超链接 A→B：网页 A 链接到网页 B）

度中心性与接近中心性都是在图上量距离，它们看不出邻居的质量。PageRank 的解决思路是把这件事做成一个递归，PageRank 量的是整个图范围内的重要性，而且这个重要性是被其他顶点投票投出来的。

核心思想

A webpage has high PageRank if:

1. Many other pages link to it（大量网页指向它）；AND/OR
2. The pages that link to it themselves have high PageRank value（指向它的网页本身是高权重权威页面）

课件例子：Page C has fewer incoming links than page E, but C’s PageRank is higher. Because the page linking to C has very high value. 不是单纯数入链数量，入链的质量 > 入链数量

3.3.1 Random Walk 随机游走模型

想象一个随机上网的人（random surfer /random walker）：

d = damping factor 阻尼因子，经典取值 $d = 0.85$，课件例题用$d = 0.8$

1. With probability d：顺着当前网页的超链接，随机选择一个出邻居页面跳转
2. With probability (1 − d)：不再点击链接，随机跳转到图中任意一个网页

$PR(v)$ = the probability that this random walker lands on page v 在稳态下，随机游走者停留在页面 v 的概率。

All PageRank values in a graph sum to 1。整张图所有节点 PR 之和 = 1。

阻尼因子1 − d作用： 解决悬挂节点（dangling node，没有出链接的页面），防止游走卡死；同时模拟用户不想继续点链接，新开网页的行为。

3.3.2 PageRank 迭代更新公式

$PR^t(v) = \frac{1-d}{n} + d \cdot \Big(\sum_{v'\in N_{in}(v)} \frac{PR^{t-1}(v')}{|N_{out}(v')|}\Big)$

- n：total number of nodes in graph 总节点数
- $N_{in}(v)$：in-neighbors of v，指向 v 的节点集合（入邻居）
- $|N_{out}(v')|$：out-degree of node v' ，节点v' 的出度（v' 向外链接多少个页面）
- $PR^{t-1}(v')$：上一轮（t-1 迭代）节点 v' 的 PageRank 值
- $\frac{1-d}{n}$：随机跳转项。1 − d这部分概率，平均分给所有 n 个节点

迭代完整流程

1. Initialization (Iteration 0)： 所有节点初始 PR 均等分配：$PR^0(v) = \frac1n$
2. Iteratively update PR： 不断用上面公式计算新的PR^t，每一轮用上一轮 t-1 的 PR 结果
3. Convergence 收敛判断： 重复迭代，直到所有节点 PR 值变化很小，基本稳定，停止迭代。
4. 性质：每一轮迭代，全部节点 PR 总和始终等于 1

例子（n=4，d=0.8）

Iteration0（初始化）：$PR(v_1)=PR(v_2)=PR(v_3)=PR(v_4)=\frac14$

Iteration 2：把 Iteration1 算出的$\frac{1.4}{12},\frac{2.6}{12},\frac{4.2}{12},\frac{3.8}{12}$作为旧值，再次代入公式，继续更新。反复迭代直到稳定。

5. 优点：有向图全局指标；不只看链接数量，重视链接来源的质量；

抗简单作弊：单纯大量垃圾页面链接你的页面，提升 PR 效果有限，因为垃圾页面自身 PR 很低。

6. 缺点：需要多次迭代计算，开销大；

是概率稳态值，依赖阻尼因子参数 d 的选择；

原始 PageRank 不考虑网页内容，只看链接结构。

3.4 三种测量总结

指标	图类型	核心信息来源	计算方法	核心思想
Degree Centrality 度中心性	无向图（有向图可拆入 / 出度）Local 局部	直接相连邻居数量	统计节点度数	连接越多邻居，越重要
Closeness Centrality 接近中心性	连通无权无向图 Global 全局	到所有节点最短路径总和	每个节点 BFS 求最短距离，公式 $C(v) = \Sigma 1/dist(v,u)$	离全网所有节点越近，越重要
PageRank	**有向图** Global 全局	入链数量 + 入链页面自身权重	随机游走模型，迭代更新公式，阻尼因子 d	被高质量页面链接，则节点重要

1. BFS 的定位：BFS 用来求无权图最短路径，是计算 Closeness Centrality 的底层算法；PageRank不使用BFS，靠迭代。
2. Closeness Centrality：图必须连通；不连通图，部分 dist 无穷大，无法计算。
3. PageRank 公式分母：$|N_{out}(v')|$ 是源节点 v' 的出度，不是目标节点 v 的度。
4. 阻尼因子d：0.85 是经典值；$1-d$是随机跳转概率，均匀分给全部节点。
5. PageRank 所有节点 PR 之和恒等于 1；初始化全部等于$1/n$。
6. 区分：Degree：数邻居多少；Closeness：算距离远近；PageRank：看谁链接，重视链接来源质量（有向）。

Part 4 Shortest Distance

加权图上 BFS 失效，靠**松弛**逐个把顶点的最短路确定下来；正确性的根基是最优子结构 + 非负权。

基础概念

Problem：给定图，求源点 source s到图中所有其他顶点的最短距离。 与接近中心性不同，那里要跑 n 次单源最短路，这里只跑一次。

Application：地图导航、路线规划。

$\delta(u, v)$：u 与 v 之间真实最短路径距离

前提假设：图所有边权重 NON-NEGATIVE（非负权重！Dijkstra 不能处理负权边）

4.1 Optimal Substucture 最优子结构

If vertex u lies on the shortest path from s to v, then the subpath $s \to \cdots \to u$ must be the shortest path from s to u. 最短路的最短路仍然是最短路。

推论：$\boldsymbol{\delta(s, u) \leqslant \delta(s, v)}$

距离单调递增：最短路上的点，沿着路径走只会越来越远（因为非负权重）。

记号	读法	含义
δ (u, v)	delta	真实的最短距离（true shortest distance）是想求但还不知道的那个真值
d(s, v)	d	算法当前维护的估计值（用 p.45 的伪代码）是个会不断变小的上界
w(u, p)	w	边的权重（weight）课件假设 w ⩾ 0（非负）

解题思路：先求解离源点更近节点的最短距离，利用已经算出的近点结果，求解更远节点。

Dijkstra 为什么可以按距离从小到大逐个确定顶点？因为最优子结构 + 非负权，所有更远的顶点的距离一定不小于当前顶点，所以当前最小的 d 不可能再被改进。

4.2 Dijkstra Algorithm 伪代码

Input: 带非负权图 $G(V, E)$，源点s

1. 初始化：对所有顶点 $v \in V$，$d(s, v) = \infty$；源点 $d(s, s) = 0$
2. 集合 Q：保存全部 $(v, d(s, v))$ 键值对（优先队列 / 最小堆，每次取出距离最小节点）
3. 集合 S：空集合，存放**已经确定最短距离**的节点
4. while $Q \neq \varnothing$:
    1. 从 Q 取出拥有最小$d(s, u)$的元组$(u, d(s, u))$，从 Q 删除
    2. 将u加入 S：$S = S \cup \{u\}$
    3. 遍历u的所有邻居p：if $d(s, p) > d(s, u) + w(u, p)$则$d(s, p) = d(s, u) + w(u, p)$，并更新p的距离值。

Relaxation（松弛操作）：尝试用经过 u 的新路径，更新 p 的最短距离。只有新路径更短才更新。

Q 的实现	取最小的代价	总复杂度
线性数组（扫一遍找最小）	O(V)	$O(V^2 + E)$
二叉堆 / 优先队列	O(log V)	O((V + E) log V)
斐波那契堆	O(1) 摊还	O(E + V log V)

4.2.1 Example

结论：对所有自然数 n 成立，算法正确。

操作轨迹：四个距离之和 8 + 5 + 9 + 7 = 29

帧	取出的顶点（Q 中最小）	本轮发生的松弛	取出后的 d 值（s,t,x,y,z）
(a)	—（初始）	—	0, ∞, ∞, ∞, ∞
(b)	s（0）	s→t: ∞ → 10；s→y: ∞ → 5	0, 10, ∞, 5, ∞
(c)	y（5）	y→t: 10 → 8；y→x: ∞ → 14；y→z: ∞ → 7	0, 8, 14, 5, 7
(d)	z（7）	z→x: 14 → 13；z→s: 0 < 14，不改	0, 8, 13, 5, 7
(e)	t（8）	t→x: 13 → 9；t→y: 5 < 10，不改	0, 8, 9, 5, 7
(f)	x（9）	x→z: 7 < 13，不改；Q 空 → 结束	0, 8, 9, 5, 7

对比：BFS vs Dijkstra

BFS 无权图最短路径，边权重 = 1，队列，不处理权重。是加权最短路在等权特例下的退化形式。BFS 只用于无权图（边数最小）；

Dijkstra 非负加权图单源最短路径，最小优先队列，处理任意⩾0 边权。若把所有边权都设成 1，结果与 BFS 完全一致，是 BFS 的加权推广。Dijkstra 用于边权非负的加权图（权重和最小）

> Dijkstra 限制：不能存在负权重边；负权图要用 Bellman-Ford

4.2.2 Dijkstra 正确性证明

命题：算法结束时，对所有顶点v，$d(s, v) = \delta(s, v)$（预估距离 = 真实最短距离）

步骤：按取出次序做归纳；基例是源点；归纳步在最短路里取一条S → Q的跨界边(x, y)；用 δ(s,y) ⩽ δ(s,v) ⩽ d(s,v) ⩽ d(s,y) 夹逼，得到 d(s,v) = δ(s,v)。

基础 Base case：第 1 个取出节点是源点s，$d(s, s) = 0 = \delta(s, s)$，成立。

归纳假设 Induction hypothesis：前$n-1$个从 Q 移除的节点，都满足$d = \delta$

归纳步骤 Inductive step：第n个取出节点v，证明$d(s, v) = \delta(s, v)$

1. 松弛操作只会**减小**$d(s, v)$；**一旦**$d(s, v) = \delta(s, v)$，值不会再改变。
2. 当节点v被从 Q 取出的时候，它的$d(s, v)$已经等于真实最短距离$\delta(s, v)$，不会有更短路径。
3. 利用最优子结构：最短路径上的子路径本身也是最短路径。

步	内容	原理
归纳起点	n = 1 时取出的是源点 s，d(s,s) = δ(s,s) = 0，成立	初始化的定义
归纳假设	假设前 n − 1 个被取出的顶点都满足 d = δ	归纳法
找一条跨界的边	在第 n 个顶点 v 的真实最短路上，取第一条起点在 S、终点在 Q的边(x, y)	S 非空（至少含 s）、Q 含 v，所以这样的跨界边一定存在
夹逼（三个不等式）	由观察得 δ(s,y) ⩽ δ(s,v)；由v 是Q 中最小得 d(s,v) ⩽ d(s,y)；由第一部分得 δ(s,v) ⩽ d(s,v)	最优子结构 + 贪心选择 + 单调性

步	内容	原理
夹逼（收口）	三式串成 $\delta(s,y) \leqslant \delta(s,v) \leqslant d(s,v) \leqslant d(s,y)$；由归纳假设 $d(s,y) = \delta(s,y)$，两端相等 → 中间全部相等 → $\delta(s,v) = d(s,v)$	归纳假设把链条锁死

Tips:

跨界的边：要证明 d(s,v) 已经降到 $\delta(s,v)$，得说明在 v 之前一定已经有一条路径把正确的值送过来了。这条路径就是 s → ⋯ → x → y → ⋯ → v，其中 x 已在 S（已确定）、y 还在 Q。

夹逼：把不等式链条把四个量串在一起，两端相等就逼出中间全部相等。

$d(s,y) = \delta(s,x) + w(x,y) = \delta(s,y)$ ： x 被取出时（由归纳假设 $d(s,x) = \delta(s,x)$）它一定松弛过自己的邻居 y，把 y 的估计值设成了 $\delta(s,x) + w(x,y)$；而这条路径本身是最短路的一段，所以这个值就是 $\delta(s,y)$。即最短路的一部分必须是最短路。

Part 5 Vertex Similarity/ Proximity Measures 链路预测

5.1 链路预测 Link Prediction

目标：预测两个节点v_1, v_4之间未来会不会产生边，或者评估两个节点之间存在边的可能性。

核心思路：计算节点之间的相似度 similarity / 邻近度 proximity。

5.2 雅卡尔相似度 Jaccard Similarity （无向图）

又称定点相似度：共同邻居 ÷ 邻居并集，一个 0 到 1 的数，用来预测这条边该不该存在。

链路预测为什么可以用相似度？因为真实网络有同质性（homophily），相似的人更容易产生联系，所以邻居重合度高的一对顶点未来连边的可能性更大。但相似度只是启发式，不是必然，最终要看预测任务的评价指标。

$Jaccard(v_1,v_4)=\frac{|N(v_1)\cap N(v_4)|}{|N(v_1)\cup N(v_4)|}$

- $N(v)$：节点v的一阶邻居集合
- 分子：两个节点公共邻居的数量（交集大小）
- 分母：两个节点所有邻居总数（并集大小）
- 含义：Jaccard 越高，两个节点共享越多共同邻居，越有可能产生新边。常用于社交网络好友推荐。

例子：v1 邻居 {v2,v3}, v4 邻居 {v2,v5} 交集$N(v1) \cap N(v4) = \{v2\}$，并集$\{v2, v3, v5\}$ $Jaccard = \frac13$

顶点对	交集	并集	Jaccard	是否已有边
(v_1, v_2)	$\{v_3\}$ → 1	$\{v_1,v_2,v_3,v_4\}$ → 4	1/4 = 0.25	已有
(v_1, v_3)	$\{v_2\}$ → 1	$\{v_1,v_2,v_3,v_4\}$ → 4	1/4 = 0.25	已有
(v_1, v_4)	$\{v_2, v_3\}$ → 2	$\{v_2, v_3\}$ → 2	1（最大）	没有 → 预测它就对了
(v_2, v_3)	$\{v_1, v_4\}$ → 2	$\{v_1,v_2,v_3,v_4\}$ → 4	1/2 = 0.5	已有
(v_2, v_4)	$\{v_3\}$ → 1	$\{v_1,v_2,v_3,v_4\}$ → 4	1/4 = 0.25	已有
(v_3, v_4)	$\{v_2\}$ → 1	$\{v_1,v_2,v_3,v_4\}$ → 4	1/4 = 0.25	已有

Part 6 Community and Clustering Concepts：社区与聚类

6.1 Connected Components 连通分量

6.1.1 Undirected Graph 无向图：Connected Components (CC)

定义： A connected component (CC) is a maximal subgraph in which each pair of nodes is connected via paths. 连通分量 CC 是极大连通子图：子图内部任意两点之间都存在路径；不能再加入更多节点而保持连通。

- 单个孤立节点本身也可以是一个连通分量。

寻找无向图连通分量算法（BFS/DFS）

Steps:

1. 全部顶点初始标记为未访问 not visited。
2. 重复直到所有节点都被访问：挑选一个未访问节点v，从v开始运行 DFS 或者 BFS 遍历；遍历过程中把所有可达节点标记为 visited；本次 DFS/BFS 遍历结束，得到一个连通分量。

为什么 DFS 和 BFS 都可以？两者都能从一点出发走到所有可达点，而 CC 只关心可达不可达，不关心访问次序。

外层循环跑几次？跑的次数正好等于 CC 的个数。从几个起点出发过，就知道图被切成了几块。

复杂度：合计O(V + E)，与一次遍历同阶。每次遍历把一批顶点标记一次，所有顶点总共只被访问一次、所有边也只被检查一次（无向图每条边检查两次）。

6.1.2 Directed Graph 有向图：SCC & WCC

1. Strongly Connected Components (SCC) 强连通分量

A strongly connected component (SCC) of a directed graph is a maximal strongly connected subgraph. Within SCC, there exists a path in both directions between every pair of vertices.

强连通分量 SCC：有向图的极大强连通子图。分量内部任意一对节点u, v，同时存在 u→v 和 v→u 双向路径。

- 单个孤立节点自身也可以是 SCC。

课件例题 SCC 结果：$\{1, 2\}, \{3\}, \{4, 5, 8\}, \{6\}, \{7\}$

1 和 2 互相可达，构成 SCC；3 只能单向进出，自己单独 SCC；4,5,8 互相可达。

6 只能被 3 指向，无法返回；7 指向 6，无法回来，各自独立 SCC。

2. Weakly Connected Components (WCC) 弱连通分量

Treat directed graph as undirected graph (ignore edge directions). The connected components of this undirected graph = WCC of original directed graph.

把有向图忽略边方向，当成无向图看待，得到的无向连通分量就是原图的弱连通分量 WCC。

Example

例题 WCC 结果：$\{1, 2, 3, 6, 7\}, \{4, 5, 8\}$ 只要不看箭头，节点之间能连通，就属于同一个 WCC，不要求双向路径。

对比项	SCC（强连通）	WCC（弱连通）
用什么图算	保持有向	先转成无向
判据	任意一对顶点双向可达	任意一对顶点至少单向可达
要求强度	强	弱
数量关系	个数 ⩾ WCC 个数	个数 ⩽ SCC 个数
典型用途	找封闭的圈层（如互相转发的朋友圈）	找同一批人/同一个话题（不关心方向）

SCC 是否一定是 WCC？是（前提是它作为子图还有意义）；

怎么求 WCC？把有向边当无向边，求连通分量即可。

6.2 Local Clustering Coefficient 局部聚类系数 & 平均聚类系数

衡量网络三角闭合程度（朋友之间互相认识的程度，社交网络经典指标）

6.2.1 Undirected graph 无向图

Local Clustering Coefficient C_v

$C_v=\frac{\text{number of edges between neighbors of }v}{\text{max possible edges among neighbors of }v} =\frac{\text{actual edges between neighbors}}{\frac{|N(v)|(|N(v)|-1)}{2}}$

分子：节点v的邻居们之间真实存在的边数量；

分母：v的邻居集合如果构成完全图，理论上最多能拥有多少边（组合公式 $C_k^2=\frac{k(k-1)}{2}$，k 是邻居数量$|N(v)|$）

课件例子：$C_2=\frac{2\times2}{4\times3}=\frac13$

顶点	分子（邻居间的边数）	分母 k(k−1)/2	C_v
v_1	v_2 - v_3 → 1	2 × 1 / 2 = 1	1（邻居连着）
v_2	v_1 - v_3、v_4 - v_5 → 2	4 × 3 / 2 = 6	1/3 ≈ 0.333（课件给的）
v_3	v_1 - v_2 → 1	2 × 1 / 2 = 1	1
v_4	v_2 - v_5 → 1	2 × 1 / 2 = 1	1
v_5	v_2 - v_4 → 1	2 × 1 / 2 = 1	1

含义：$C_v \in [0,1]$

$C_v = 1$：v 的所有邻居两两之间全部相连，邻居构成完全图；

$C_v = 0$：v 的邻居之间没有任何边，没有三角结构。

6.2.2 Directed graph 有向图 Local Clustering Coefficient

邻居集合取入邻居∪出邻居 $N_{in}(v) \cup N_{out}(v)$

$C_v=\frac{\text{edges between nodes in }N_{in}(v)\cup N_{out}(v)}{|N_{in}(v)\cup N_{out}(v)|\cdot(|N_{in}(v)\cup N_{out}(v)|-1)}$

课件例子：$C_2=\frac{2}{4\times3}=\frac16$

顶点	k	分子（并集内的有向边）	分母 k(k−1)	C_v
v_1	2	$v_3 \to v_2 \to 1$	2 × 1 = 2	1/2
v_2	4	$v_3 \to v_1$、$v_4 \to v_5 \to 2$	4 × 3 = 12	1/6（课件给的）
v_3	2	$v_1 \to v_2 \to 1$	2 × 1 = 2	1/2
v_4	2	$v_2 \to v_5 \to 1$	2 × 1 = 2	1/2
v_5	2	$v_2 \to v_4 \to 1$	2 × 1 = 2	1/2

6.2.3 Graph Average Clustering Coefficient 图平均聚类系数

整张图的平均聚类系数 = 图里所有节点的局部聚类系数的算术平均值

$C_{avg}=\frac1n\sum_{v\in V}C_v$

用来描述整个网络的整体聚集特性：高平均聚类系数代表网络有大量三角形、小团体结构（例如社交网络）。

Lecture 8 Big Data Storage 大数据存储

Part 1 数据类型与存储系统选择 Types of Data

数据的组织结构与更新频率直接影响存储系统的设计。课件从两个相互独立的维度对数据进行分类：结构化程度（Structured / Unstructured）与变化频率（Dynamic / Static），并通过四象限模型说明不同数据类型的存储需求。

1.1 按结构化程度分类

类型	定义与特征	典型示例
Structured Data（结构化数据）	具有预定义的数据模型，便于存储、处理、检索和管理，通常以表格形式组织	关系型数据（Relational Data）
Unstructured Data（非结构化数据）	不具有便于直接查询和处理的统一数据模型，通常需要额外解析或处理	文本、音频、视频等原始文件

非结构化数据并非完全没有结构。 例如，音频文件仍然具有编码格式及相关元数据。两类数据的核心区别在于其组织形式是否便于直接检索与处理，而非是否存在任何内部结构。

1.2 按变化频率分类

类型	定义	典型示例
Dynamic Data（动态数据）	内容更新相对频繁	办公文档、金融数据库中的交易记录
Static Data（静态数据）	内容相对稳定，更新频率较低	MRI、CT 医学影像

动态与静态描述的是数据内容的变化频率，而非访问频率。静态数据也可能被频繁读取。

1.3 数据分类四象限 Data Classification Quadrants

Unstructured + Dynamic 动态非结构化，例如：媒体制作、eCAD、mCAD、办公文档

Unstructured + Static 静态非结构化，例如：媒体归档、广播内容、医学影像

Structured + Dynamic 动态结构化，例如：交易系统、ERP、CRM

Structured + Static 静态结构化，例如：BI（商业智能）、Data Warehousing（数据仓库）

横向：Dynamic → Static；纵向：Unstructured → Structured

分类的主要目的是辅助选择合适的存储方案（Storage Solution）：

- 关系型数据库（Relational Databases）：通常用于结构化数据，特别是需要事务管理的业务系统。
- 文件系统或 NoSQL 数据库：可用于静态非结构化数据的存储。

Part 2 关系型数据库与 ACID 事务

2.1 事务 Transaction

RDBMS（Relational Database Management System） 即关系数据库管理系统。

事务是数据库执行操作的逻辑单元，可以包含多个底层操作（Low-level Tasks），用于将相关的数据修改作为一个整体进行管理。例如，账户 A 向账户 B 转账金额 x： $\begin{aligned} A'&=A-x\ B'&=B+x \end{aligned}$，转账完成后应满足： $A'+B'=A+B$。

虽然转账涉及多个数据库操作，但这些操作必须共同满足业务约束。因此，RDBMS 通过 ACID 属性保障事务的正确执行。

2.2 ACID 四大属性

属性	定义	核心要求
Atomicity（原子性）	事务中的所有操作被视为不可分割的整体	全部成功或全部失败
Consistency（一致性）	事务开始和结束时，数据库均满足完整性约束	保持数据合法性
Isolation（隔离性）	并发事务不能访问其他事务未完成的中间状态	避免不当的并发干扰
Durability（持久性）	事务一旦提交，其修改必须持久保存	系统故障后仍保留已提交数据

Atomicity： 若转账过程中扣款成功但入账失败，整个事务必须回滚，不能保留部分修改。

Consistency： 转账前后应保持资金总额不变，且数据库约束仍然成立。

Isolation： 一个事务不应读取其他事务尚未提交的中间修改。具体并发行为由数据库隔离级别决定。

Durability： 已提交的事务在系统故障后仍应能够恢复。

保障机制包括：Transaction Logs（事务日志）、Backups（备份）

需要区分：ACID 的 Consistency 指事务对数据完整性约束的维护，不等同于后续 CAP 定理中的分布式副本一致性。

Part 3 数据库横向扩展 Scaling Out Databases

随着数据规模与访问需求增长，单台数据库服务器可能受到存储容量、计算资源和吞吐能力的限制。

数据库扩展主要有两种方式：

- Vertical Scaling（纵向扩展）：提高单台服务器的 CPU、内存或存储性能。
- Horizontal Scaling（横向扩展）：增加服务器数量，通过多台机器并行处理工作负载。

重点讨论横向扩展，主要技术包括 Sharding（数据分片） 和 Replication（数据复制）。

3.1 Amdahl 定律：并行加速的理论上限

Amdahl's Law 用于描述程序中不可并行部分对整体加速效果的限制。

设：

- T_1：程序串行执行时间。
- T_p：使用 p 个处理器并行执行的时间。
- s：程序中不可并行部分所占比例。
- $1 - s$：程序中可并行部分所占比例。
- p：处理器或机器数量。

在理想情况下，不可并行部分耗时保持不变，可并行部分由 p 个处理器均匀分担：

$T_p=sT_1+\frac{(1-s)T_1}{p} $

因此，加速比（Speedup）为：

${ S(p)=\frac{T_1}{T_p} =\frac{1}{s+\frac{1-s}{p}} } $

课件算例：80% 的程序可以并行，使用 4 个处理器。

已知：$s = 0.2, 1 - s = 0.8, p = 4$

代入公式：$S(4)=\frac{1}{0.2+\frac{0.8}{4}} =\frac{1}{0.4} ={2.5} $

因此，4 个处理器只能实现理想情况下的 2.5 倍加速，而非 4 倍。

若串行执行总耗时为 100 个时间单位，则：

$T_1=20+80=100 $

$T_4=20+\frac{80}{4}=40 $

$S(4)=\frac{100}{40}=2.5 $

进一步，当处理器数量趋于无穷大时：

${ \lim_{p\to\infty}S(p)=\frac{1}{s} } $

本例的理论最大加速比为：

$S_{\max}=\frac{1}{0.2}=5$

结论：不可并行部分决定整体加速比的上限，增加处理器数量无法消除串行执行的限制。

3.2 理想并行与实际并行

Amdahl 定律建立在理想条件下，假设任务能够均匀分配，且忽略通信与调度开销。

实际并行系统还受到以下因素影响：

限制因素	含义	影响
Communication Overhead（通信开销）	处理器或服务器之间的数据传输、同步与协调	增加执行时间
Workload Imbalance（负载不均衡）	不同处理器承担的任务量不同	整体完成时间受最慢任务限制

对于必须等待所有并行任务完成的程序，可将实际执行时间近似表示为：

$T_p^{\text{actual}} \approx sT_1+\frac{(1-s)T_1}{p} +T_{\text{communication}} +T_{\text{imbalance}} $

其中后两项是对课件理想模型的补充，分别表示额外通信成本与负载不均衡造成的延迟。

因此，实际加速比通常低于理想加速比。

提高并行效率需要遵循三项原则：

1. Maximize Parallelizable Fraction：提高可并行部分比例，减小 s。
2. Balance Workload：均衡各处理器的工作负载，减少等待。
3. Minimize Communication Time：降低节点之间的数据传输与同步开销。

3.3 数据分片 Data Sharding

数据分片是将大型数据集划分为多个部分（Shards / Chunks），并分布到不同机器上，以支持并发和并行访问。

课件示例将一个大型文件划分为六个数据块：

由于 Chunk 2、Chunk 4 和 Chunk 6 位于不同机器，因此可以同时访问。

分片的主要作用包括：

- 将数据存储需求分散到多台机器。
- 提高并行读取和处理能力。
- 在适当的分片策略下，提高整体吞吐量。

分片的主要挑战包括跨分片查询、数据分布不均与跨节点事务协调。

3.4 数据复制 Data Replication

数据复制是将相同数据保存到多个服务器上，形成多个副本（Replicas）。

三项作用：

- Avoid Performance Bottlenecks：将访问请求分散到不同服务器，缓解性能瓶颈。
- Avoid Single Points of Failure：单个服务器故障时，可由其他副本继续提供服务。
- Enhance Scalability and Availability：提高系统的可扩展性与可用性。

分片与复制的核心区别如下：

对比	Sharding	Replication
基本操作	拆分数据	复制数据
节点数据	不同节点保存不同部分	多个节点保存相同数据
主要目的	扩展容量、支持并行处理	提高容错能力、可用性与读吞吐量

对比	Sharding	Replication
主要挑战	跨分片访问与负载均衡	副本同步与一致性

两种技术可以结合使用：将数据划分为多个分片，并为每个分片建立多个副本。

Part 4 分布式一致性与两阶段提交 2PC

4.1 数据复制带来的一致性问题

当同一份数据分布在多个服务器上时，数据修改必须在相关副本之间协调。

例如，电商系统将银行数据库复制到两台服务器。如果某次账户余额更新只在一台服务器上完成，而另一台尚未同步，则不同服务器可能返回不同余额。

因此，数据复制虽然提高了可用性与扩展性，但也引入了 Consistency Challenge（一致性挑战）。

Two-Phase Commit Protocol（两阶段提交协议，2PC） 说明如何协调分布式事务的提交或中止，以维护原子性和一致性。

4.2 两阶段提交协议 Two-Phase Commit Protocol

2PC 包含两类角色：

- Coordinator（协调者）：组织投票并决定全局事务的提交或中止。
- Participants（参与者）：执行本地事务操作，并根据协调者的最终决定提交或回滚。

协议分为两个阶段。

Phase I: Voting / Prepare（投票或准备阶段）

Coordinator: 发送 VOTE_REQUEST

Participant 1: 准备事务并投票

Participant 2: 准备事务并投票

Participant 3: 准备事务并投票

返回 VOTE_COMMIT 或 VOTE_ABORT

执行过程：

1. 协调者向所有参与者发送 `VOTE_REQUEST`。
2. 各参与者执行可撤销的本地事务操作，并检查是否能够提交。
3. 能够提交的参与者返回 `VOTE_COMMIT`；否则返回中止投票。
4. 已准备提交的参与者需要保留必要的事务状态与锁，等待第二阶段的最终决定。

全局决策规则为： $\text{Decision}= \begin{cases} \text{Commit}, & \text{所有参与者均投赞成票}\\ \text{Abort}, & \text{否则} \end{cases} $

Phase II：Commit / Abort（提交或中止阶段）

若所有参与者均同意提交：

1. 协调者发送 `GLOBAL_COMMIT`。
2. 参与者执行 `LOCAL_COMMIT`，正式提交本地修改。
3. 参与者释放事务锁，并完成相应的提交确认。

若存在参与者无法提交：

1. 协调者决定全局中止（Global Abort）。
2. 参与者回滚尚未提交的本地事务。
3. 释放相关锁与资源。

需要区分两个阶段的操作：

第一阶段	第二阶段
准备并检查事务能否提交	执行全局提交或中止决定
修改仍可撤销	提交后成为正式持久化结果
参与者可能持有锁	事务完成后释放锁
不产生最终提交决定	完成最终事务决策

4.3 2PC 的局限性

2PC 通过协调多个参与者的事务结果，实现跨节点的原子提交，但需要额外的通信与等待。

其主要代价包括：

- 锁持有时间增加：参与者在准备阶段完成后，通常必须等待全局决定才能释放相关锁。
- 通信开销增加：协调者需要与多个参与者交换消息。
- 阻塞风险（Blocking）：协调者在关键阶段发生故障时，已准备的参与者可能无法独立决定提交还是回滚。
- 可扩展性受限：参与者数量增加时，协调成本和资源占用可能上升。

因此，严格一致性（Strict Consistency）的实现可能限制系统的可扩展性（Scalability）。

2PC 主要保障分布式事务的原子提交，并不意味着仅凭 2PC 就能解决所有副本一致性或网络分区问题。

4.4 小结

本部分内容围绕数据库从单机存储向分布式存储扩展时面临的问题展开，其核心逻辑为：

1. Data Classification 根据数据结构和变化频率选择存储系统
2. RDBMS & ACID 通过事务保证数据库操作的正确性
3. Horizontal Scaling 通过并行处理、分片和复制扩展系统能力
4. Consistency Challenge 分布式数据更新引入跨节点协调问题
5. Two-Phase Commit 通过协调提交保障分布式事务原子性，但增加通信与阻塞成本

概念对照

概念	主要解决的问题
Data Classification	如何根据数据特征选择存储方案
ACID	如何保障事务的正确性与可靠性
Amdahl's Law	并行处理的理论加速上限
Sharding	如何分散数据和处理负载
Replication	如何提高可用性和容错能力
2PC	如何协调多个节点共同提交或中止事务

结论： 横向扩展通过增加机器提高系统的存储与处理能力，但分片和复制使数据管理从单节点操作转变为跨节点协调。为了维护分布式事务的原子性与一致性，系统必须承担额外的通信、同步与等待成本。这也为后续 CAP Theorem、BASE Properties 和 NoSQL 数据库的讨论建立了基础。

Part 5 CAP 定理与 BASE 属性 CAP Theorem & BASE Properties

随着数据库从单机架构扩展到分布式架构，数据复制与跨节点通信使一致性问题更加复杂。CAP 定理揭示了网络分区发生时，一致性与可用性之间的根本权衡；BASE 则提供了一种适用于部分高可用分布式系统的宽松一致性设计思路。

5.1 CAP 定理 CAP Theorem

CAP 定理指出：当分布式系统发生网络分区时，无法同时保证强一致性（Consistency）和可用性（Availability）。

CAP 包含三个属性：

属性	定义	核心要求
C — Consistency（一致性）	每次读取都返回最近一次成功写入的数据，或者返回错误	不返回过期数据
A — Availability（可用性）	每个发送到正常运行节点的请求最终都获得非错误响应	请求能够得到服务
P — Partition Tolerance（分区容错性）	节点之间发生网络通信故障时，系统仍能继续运行	容忍网络分区

CAP 中的 Consistency 与 ACID 中的 Consistency 不同：

- ACID Consistency：事务执行前后满足数据库完整性约束。
- CAP Consistency：强调分布式读写操作的单副本一致性，通常用线性一致性（Linearizability）描述。

CAP 的关键并非一般情况下只能拥有三个属性中的两个，而是网络分区发生时，必须在 C 和 A 之间作出选择。

5.2 网络分区下的 CP 与 AP

假设两个节点 N_1 和 N_2 保存同一数据的副本，网络故障导致两者无法通信。

此时，如果客户端向其中一个节点发起更新操作，系统面临两种策略：

策略	CP（Consistency + Partition Tolerance）	AP（Availability + Partition Tolerance）
核心目标	优先保证一致性	优先保证可用性

策略	CP（Consistency + Partition Tolerance）	AP（Availability + Partition Tolerance）
分区期间	拒绝或延迟无法安全执行的操作	尽可能继续响应请求
主要代价	部分请求不可用	可能读取旧数据或出现副本差异
适用倾向	对数据正确性要求较高的业务	对持续服务要求较高的业务

CP： 当无法确认其他副本的状态时，系统可能拒绝操作，以避免违反一致性要求。

AP： 系统允许分区两侧继续处理请求，但可能产生暂时不一致，需要在网络恢复后进行同步或冲突解决。

需要注意，AP 并不意味着所有请求必然成功，也不意味着系统完全放弃一致性；CP 同样不意味着所有操作都不可用。

5.3 一致性权衡

Google、Amazon 等企业在构建大规模数据库时，需要特别重视 24/7 Availability（全天候可用性），因为短时间停机也可能造成显著的业务损失。当数据库横向扩展至数千台服务器时，节点故障和网络故障的可能性增加。因此，系统必须考虑如何在网络分区下维持服务。

部分大规模系统选择优先保障可用性和分区容错性，并放宽严格一致性要求。

这种设计体现了以下权衡： $\text{Consistency Strictness} \quad\longleftrightarrow\quad \text{Availability / Scalability} $

一致性要求越严格，跨节点同步与协调成本通常越高；适当放宽一致性要求，则可能减少协调开销并改善可用性与扩展能力。Good-enough Consistency（足够的一致性）：一致性级别应根据应用需求确定，而非所有系统都采用最严格的一致性模型。

例如，银行账户余额通常需要较强的一致性保障，而社交平台的点赞数量可以在一定时间内允许副本存在差异。

这并非严格一致性必然低效，而是分布式环境中的一致性保障通常需要额外的通信和协调。

5.4 从 ACID 到 BASE

传统关系型数据库通常强调 ACID 事务，而部分面向大规模分布式应用的 NoSQL 系统采用 BASE 思路，以更宽松的一致性要求换取可用性与扩展能力。

BASE 可视为在一致性要求上更加宽松的设计思想，但并不是 ACID 的直接替代品。

BASE 属性	含义	说明
Basically Available（基本可用）	系统在大多数情况下能够提供服务	允许部分功能降级或响应延迟
Soft State（软状态）	数据状态可能在没有新的外部输入时发生变化	副本同步等内部过程会改变系统状态
Eventually Consistent（最终一致性）	在停止更新且同步能够持续完成的条件下，各副本最终收敛到一致状态	允许暂时不一致

最终一致性可以用以下形式表示：

设 $R_i(t)$ 表示第 i 个副本在时刻 t 的数据状态。如果系统停止接收新写入，且更新能够可靠传播，则：

$\exists T, \forall t \geqslant T: R_1(t) = R_2(t) = \cdots = R_n(t)$，该表达式表示在满足相应条件时，所有副本最终达到相同状态。

5.5 ACID 与 BASE 对比

维度	ACID	BASE
设计重点	事务正确性与可靠性	可用性与宽松一致性
一致性要求	保证事务维护完整性约束	允许副本暂时不一致
状态变化	强调事务边界及其约束	允许内部异步同步引起状态变化
典型应用	金融交易、订单事务	社交互动、内容分发、部分分布式存储系统

ACID 与 BASE 并非完全互斥。现代分布式数据库可以同时支持 ACID 事务、复制与不同级别的一致性保证。

本部分逻辑：网络分区使 C 与 A 的冲突不可避免，而 BASE 为部分优先可用性的系统提供了最终一致性的设计方向。

Part 6 NoSQL 数据库概述 Introduction to NoSQL

6.1 NoSQL 的产生背景与定义

传统关系型数据库（RDBMS）长期是企业业务系统的主流存储方案。然而，Web 应用快速发展，特别是 Facebook、X 等社交平台带来的数据量和访问量激增，对数据库提出了新的要求：

- 海量数据存储与高并发访问。
- 跨多台服务器的水平扩展。
- 对半结构化及非结构化数据的灵活支持。
- 在分布式环境下维持较高的可用性。

传统 RDBMS 的事务协调、固定模式以及部分跨节点操作可能增加大规模分布式部署的复杂度，由此推动了 NoSQL 数据库的发展。

NoSQL（Not Only SQL） 表示“不仅仅是 SQL”，泛指一类不局限于传统关系模型的数据库系统。

其名称发展经历两个阶段：

- 1998 年：Carlo Strozzi 使用 NoSQL 命名其基于文件的数据库。
- 2009 年：Eric Evans 在讨论开源分布式数据库的活动中重新推广这一术语。

NoSQL 的核心思想不是否定关系型数据库，而是针对关系型数据库不适合解决的特定问题，采用不同的数据模型与系统架构。

6.2 NoSQL 的主要特征

NoSQL 数据库通常具有以下特征：

特征	说明
Non-relational	采用非关系型数据模型
Flexible Schema	不要求所有记录遵循统一、严格的固定模式
Horizontal Scalability	通过增加节点扩展存储和处理能力
Partitioning	支持将数据分布到多个节点

特征	说明
Replication	通过数据副本提高容错能力与可用性
High Write Performance	部分系统针对大规模写入进行优化
Fast Access	针对特定访问模式提供快速读写
Open-source Ecosystem	存在大量开源实现，可降低软件许可成本

通过分片与复制，部分 NoSQL 系统能够在节点故障后重新分配任务或使用其他副本，避免单点故障（Single Point of Failure）。

但这些优势并非所有 NoSQL 产品都具备，具体性能取决于数据库实现、数据模型及工作负载。

6.3 NoSQL 的局限性

相较于传统关系型数据库，部分 NoSQL 系统在复杂关系查询、统一查询接口和事务保障方面存在限制。

1. 关系型操作支持有限

部分 NoSQL 数据库不直接支持跨整个分布式集群的复杂关系操作，例如：

- `JOIN`：多表连接。
- `GROUP BY`：分组聚合。
- `ORDER BY`：排序。
- 跨分区参照完整性约束（Referential Integrity Constraints）。

这些操作可能需要在应用层实现，或借助额外的查询与计算组件。

2. 缺少统一的声明式查询语言

关系型数据库普遍使用 SQL，而不同 NoSQL 数据库的查询接口差异较大，因此可能增加开发与迁移成本。

3. 事务保障可能较弱

部分 NoSQL 系统放宽 ACID 约束，以减少分布式协调成本，但这也意味着开发者可能需要自行处理部分一致性与并发问题。

现代 NoSQL 数据库已有较大改进，例如 MongoDB 支持多文档事务，Neo4j 支持 ACID，因此不能将“不支持事务”视为 NoSQL 的普遍特征。

6.4 NoSQL 的主要分类

按照数据模型，NoSQL 数据库主要分为四类：

类别	数据组织方式	课件代表产品
Key-Value（键值型）	通过唯一键访问对应值	DynamoDB、Voldemort、Scalaris
Column-based（列族型）	通过列族组织相关数据	Bigtable、Cassandra、HBase
Document-based（文档型）	以 JSON 等文档结构存储对象	MongoDB、CouchDB
Graph-based（图数据库）	以节点和边表示实体及关系	Neo4j、InfoGrid

主要区别：数据如何组织，以及数据库针对哪类查询操作进行优化。

早期应用案例：Google Bigtable、Amazon Dynamo、LinkedIn Voldemort，以及使用 Cassandra、HBase 等技术的互联网服务。这些案例说明 NoSQL 技术在大规模互联网应用中的发展背景，并不代表相关企业当前的完整技术架构。

Part 7 四类 NoSQL 数据库 Various NoSQL Databases

四类 NoSQL 数据库的根本差异在于数据模型及其访问方式：键值型强调按键快速访问，列族型强调稀疏宽表与相关列的组织，文档型支持复杂嵌套对象，图数据库则面向实体之间的关联关系。

7.1 键值型数据库 Key-Value Stores

1. 数据模型与分布式架构

键值型数据库将数据组织为键值对集合： $D={(k_1,v_1),(k_2,v_2),\ldots,(k_n,v_n)} $

其中 k_i 是用于定位数据的键（Key），v_i 是对应的值（Value）。

该模型结构简单，适合海量数据存储、高并发访问及水平扩展。

以 Amazon Dynamo 系列设计为背景，介绍了 Dynamo Ring Partitioning and Replication（环形分区与复制）：

- 通过键的哈希值确定数据分布位置。
- 将不同键的数据分配到不同节点。
- 为数据建立多个副本，提高容错能力。

其基本思想是通过键分布数据，而不是依赖单台服务器保存完整数据集。

2. DynamoDB 的复合主键

DynamoDB 示例采用复合主键（Composite Primary Key），由两部分构成：

- Partition Key（分区键）：决定数据所属的逻辑分区。
- Sort Key（排序键）：在同一分区键下区分并排序不同记录。

两者组合形成唯一主键： ${PK=(\text{Partition Key},\text{Sort Key})} $

例如：

Partition Key	Sort Key	Attributes
ProductID = 1	Book ID	Odyssey、Homer、1871
ProductID = 2	Album ID	6 Partitas、Bach
ProductID = 2	Album ID : Track ID	Partita No. 1
ProductID = 3	Movie ID	The Kid、Chaplin

不同记录可以具有相同的分区键，但完整主键组合必须唯一。

DynamoDB 还允许不同 Item（数据项）具有不同的非主键属性，因此不要求所有记录遵循完全相同的模式。DynamoDB 也支持仅使用 Partition Key 的简单主键，并非所有表都必须采用复合主键。

3. 基本操作接口 APIs

API	功能
get(key)	根据键获取对应值
put(key, value)	创建或更新键值对
delete(key)	删除键及其对应值
execute(key, operation, parameters)	对指定键对应的数据结构执行操作

其中 `execute` 可以用于操作 List、Set、Map 等数据结构，具体支持范围取决于数据库实现。

4. 优缺点

优点	局限性
数据模型简单	难以直接表达复杂对象关系
按键访问速度快	复杂条件查询能力可能有限
易于按键进行水平分区	跨键关联操作可能需要应用层实现
可通过复制实现容错	分布式副本可能带来一致性问题
适合高并发访问	不适合所有复杂分析型查询

将 Eventual Consistency（最终一致性）列为键值系统的常见特征，但不同产品支持的一致性级别并不相同。

5. 代表产品

产品	开发者或来源	数据模型与查询特点
SimpleDB	Amazon	键关联属性集合，支持受限 SQL 风格查询及 GetAttributes、PutAttributes 等操作
Redis	Salvatore Sanfilippo 最初开发	支持 String、List、Set、Sorted Set、Hash 等数据结构，并提供相应操作原语
Dynamo	Amazon	分布式键值架构，强调简单的 Get/Put 操作及上下文相关的版本处理
Voldemort	LinkedIn	分布式键值存储，接口设计与 Dynamo 类似

其中需要区分 Dynamo（Amazon 的分布式存储设计） 与 DynamoDB（Amazon 的托管数据库服务），二者并非同一产品。

7.2 列族型数据库 Column-Family Stores

1. 行式与列式存储

传统关系型数据库通常采用行式存储（Row-oriented Storage），将同一记录的不同属性组织在一起。

列式存储（Column-oriented Storage）则将相同列的数据集中组织，以便读取指定列的数据。

列式组织的主要优势是 Data Locality（数据局部性）：当查询仅涉及部分列时，数据库可以减少不必要的数据读取；相关数据集中存放，也有利于批量访问。

Bigtable、HBase、Cassandra 等属于宽列或列族数据库（Wide-column Stores），其物理组织并不完全等同于分析型数据库中的纯列式存储。

2. 列族数据模型

列族型数据库通过 Column Family（列族）组织相关列。

课件将其抽象为： Column Family = (*key*, *value*)

其中： $value={\text{Related Columns}} $

其主要特点包括：

- 一个列族包含若干逻辑相关的列。
- 不同记录可以拥有不同数量的列，适合稀疏数据（Sparse Data）。
- 数据通常按照行键、列名或其他存储顺序组织。
- 通过数据局部性优化特定的范围读取与列访问。

例如，同一个用户表中，有的用户记录包含电话号码，有的用户记录包含地址，而不必为每条记录保存完全相同的属性集合。

3. 代表产品

产品	开发者或来源	数据模型	查询特点
Bigtable	Google	基于行键、列与时间戳组织数据	支持行、列及时间戳范围选择
HBase	Apache	基于 Bigtable 思想的列族模型	Java API、Shell 操作
Hypertable	Hypertable 项目	类似 Bigtable	HQL（Hypertable Query Language）
Cassandra	Apache，最初由 Facebook 开发	分区键、聚簇列及列组织	按键查询、范围查询及指定列访问
PNUTS	Yahoo	支持哈希或有序表及灵活模式	主键查询、范围查询、排序及 Top-k 等操作

PNUTS 属于较早期的研究系统，表中的产品特征主要用于理解不同数据库模型的设计差异。

7.3 文档型数据库 Document Stores

1. 数据模型

文档型数据库以文档（Document）作为基本存储单位，能够表示比简单键值对更复杂的数据对象。

其逻辑结构为： $D = \{d_1, d_2, \cdots, d_n\}$

每个文档可以包含多个字段、数组以及嵌套对象。

常见文档格式包括：

- JSON（JavaScript Object Notation）
- XML（Extensible Markup Language）
- 其他半结构化格式（Semi-structured Formats）

文档模型能够表达对象、记录、结构体、列表、数组、映射和布尔值等结构。日期等类型通常由具体数据库的扩展格式支持，例如 MongoDB 的 BSON。

2. 嵌套文档示例

以下示例展示了如何在单个文档中表示个人信息、子女及联系电话：

文档数据库可以直接保存这类嵌套结构，而不必将其全部拆分为多个关系表。

其优势在于：

- 能够自然表示复杂对象。
- 支持嵌套对象与数组。
- 允许不同文档具有不同字段。
- 可以通过文档 ID 或字段进行查询。

3. 文档型与列族型的区别

对比	Document-based	Column-family
基本单位	Document	Row / Column Family
数据结构	支持多层嵌套	以行键和列族组织
模式	灵活、可变	通常需要定义列族或表结构
查询特点	文档字段、嵌套属性查询	行键、分区键及列范围查询
适用场景	用户档案、内容管理、复杂对象	海量稀疏数据、时序或大规模分布式写入

课件将列族模型描述为相对固定的一至两层字典结构，而文档模型允许任意复杂的嵌套文档。这是对典型数据模型的概括，不代表所有列族数据库都无法保存嵌套值。

4. 代表产品

产品	开发者或来源	数据模型与查询特点
MongoDB	10gen（后更名 MongoDB Inc.）	文档存储于 Collection，每个文档具有唯一 _id，支持字段查询、逻辑条件、更新和删除
Couchbase	Couchbase	以 JSON 文档存储数据，支持键访问及文档查询；课件介绍了 Views、JavaScript 和 MapReduce 等机制

文档数据库特别适合对象结构复杂、字段变化较频繁的应用，但对于高度复杂的跨实体关系，仍可能需要额外的关联查询或数据建模设计。

7.4 图数据库 Graph Databases

1. 数据模型

图数据库重点表示数据之间的连接关系（Interconnectivity），其设计受到图论（Graph Theory）的启发。

图可以表示为： ${G=(V,E)} $

其中：

- *V*：节点集合（Vertices / Nodes），表示实体。
- *E*：边集合（Edges），表示实体之间的关系。

在属性图模型（Property Graph）中，节点与边都可以具有属性（Properties）。

例如，社交网络中的用户可以表示为节点，关注关系可以表示为有向边。节点表示实体，边表示实体间的连接关系。

图数据库主要具有以下特征：

- 节点可以包含 ID 及其他属性。
- 边可以包含标签（Labels）、角色（Roles）及属性。
- 节点与边均可通过键值对表示属性。
- 不同产品提供不同的查询语言与访问接口。
- 适合邻居查询、路径搜索和多跳关系遍历。

图数据库尤其适用于关系结构复杂的数据，例如社交网络、知识图谱、推荐系统及欺诈检测。

2. 图查询

图数据库擅长执行 Graph-like Queries，例如：

- 邻居查询（Neighbor Query）
- 路径查询（Path Query）
- 最短路径（Shortest Path）
- 多跳关系遍历（Multi-hop Traversal）

例如，寻找两个节点之间的最短路径： $d(u,v)=\min_{p\in P(u,v)}|p|$。其中 $P(u, v)$ 表示从节点 u 到节点 v 的所有路径集合，$|p|$ 表示路径中的边数。该公式适用于无权图的最短路径定义。

3. 代表产品

产品	技术与来源	特征
JanusGraph	Apache 生态相关集成；主要使用 Java	分布式图数据库，支持大规模图分析与查询，并可集成 Spark、Hadoop 等平台
Neo4j	属性图数据库	支持 ACID 事务、图查询与高可用部署，提供可视化图探索工具
FlockDB	Twitter 开发的图存储系统	面向大规模社交关系的存储与访问
Pregel	Google 图计算模型	面向大规模图计算，不属于典型的事务型图数据库
InfoGrid	图数据相关平台	以图结构组织和访问相互关联的数据

Neo4j 产品表还列出了 Java、Python、Go、Ruby、PHP、R、C/C++ 等语言生态。需要区分数据库本身的实现语言与其客户端支持语言。

7.5 四类 NoSQL 数据库对比

维度	Key-Value	Column-family	Document	Graph
核心模型	Key → Value	行键 + 列族	Document	Nodes + Edges
主要访问方式	按键访问	按键及列范围访问	按文档字段查询	沿关系遍历
结构复杂度	较低	中等	支持复杂嵌套	支持复杂关系
主要优势	简单、高吞吐、易分区	稀疏宽表、大规模数据访问	灵活模式、对象表达	关系查询与路径计算
主要局限	复杂查询较弱	查询受数据模型影响较大	跨文档关联可能复杂	大规模全局分析可能需要专用计算框架

维度	Key-Value	Column-family	Document	Graph
典型场景	缓存、会话、键值查询	日志、时序数据、海量记录	用户档案、内容管理	社交网络、知识图谱

数**据库选**型的核心依据

不同 NoSQL 模型并非简单的性能高低关系，而是分别针对不同的数据组织和访问模式进行优化。

- 主要通过唯一键读取数据：优先考虑 Key-Value。
- 需要管理海量稀疏记录、按分区键或列范围访问：考虑 Column-family。
- 数据包含复杂嵌套对象，且字段结构经常变化：考虑 Document。
- 主要查询实体之间的多跳关系或路径：考虑 Graph。

选型时还需结合一致性要求、事务能力、数据规模、读写比例及运维成本进行综合判断。

7.6 本章知识总结

本章围绕大规模数据存储的架构演进展开，其核心逻辑为：

Data Types 数据结构与访问需求决定存储模型

Scaling & Replication 横向扩展提高系统容量，但引入分布式协调问题

CAP & BASE 网络分区下权衡一致性与可用性

NoSQL 采用不同数据模型满足特定规模与查询需求

本章需要重点掌握三个层面的区别：

1. ACID 与 BASE：前者是事务正确性的属性集合，后者是强调基本可用和最终一致性的分布式设计思路。
2. CAP 中的 C 与 ACID 中的 C：前者关注分布式操作的一致性，后者关注事务对数据库约束的维护。
3. 四类 NoSQL：分类依据是数据模型，而不是是否支持 SQL、是否支持 ACID 或是否使用分布式架构。

结论：NoSQL 的价值并非取代关系型数据库，而是通过不同的数据模型、分区机制和一致性策略，为特定的大规模数据存储与访问需求提供更合适的解决方案。

5583 Agentic Computing 1

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_6/knowledge/5583-Agentic-Computing-1.md`

5583 Agentic Computing1

Lecture 4 Deep Reinforcement Learning II & Introduction to Prompt

Engineering

深度强化学习进阶与提示工程导论

本讲主要包括三个部分：深度强化学习的进阶方法、现代 LLM 智能体的信息获取机制，以及强化学习策略训练实践。

整体知识体系可归纳为六个相互衔接的主题：

1. 强化学习基础回顾：智能体与环境的交互、价值函数、DQN 与策略梯度。
2. 强化学习中的奖励与探索问题：稀疏奖励、奖励塑形、课程学习。
3. 深度强化学习的典型应用：AlphaGo 系列、多智能体强化学习。
4. 现代 LLM 智能体与知识边界：预训练知识的来源、覆盖范围、质量与时效性。
5. 联网搜索与信息可靠性：搜索触发、来源选择、信息过滤与多步搜索。
6. 深度研究与自主任务执行：研究规划、多源评估、迭代检索、证据整合与 Agentic AI。

Part 1 强化学习基础

1.1 强化学习的基本框架

强化学习（Reinforcement Learning, RL）是一种通过与环境交互、利用奖励反馈优化决策策略的机器学习方法。其核心目标是学习能够最大化长期期望回报的策略，而非仅优化单步奖励。

强化学习系统包含以下基本要素：

要素	符号	含义
Agent（智能体）	—	与环境交互并作出决策的学习主体
Environment（环境）	—	智能体执行动作并获得反馈的外部系统
State（状态）	\(s_t\)	环境在时刻 \(t\) 的状态
Observation（观测）	\(o_t\)	智能体能够感知的环境信息
Action（动作）	\(a_t\)	智能体在当前状态下采取的行为
Reward（奖励）	\(r_t\)	环境对动作结果提供的数值反馈
Policy（策略）	\(\pi\)	从状态或观测到动作或动作概率的映射

智能体与环境的交互过程可以表示为：

每一时间步的交互流程为：

1. 智能体接收当前状态或观测。
2. 根据策略 π 选择动作 a_t。
3. 环境执行状态转移，并返回奖励与下一状态。
4. 智能体根据反馈更新策略，重复交互过程。

以电子游戏为例，游戏画面构成观测，移动或射击构成动作，得分构成奖励，游戏模拟器则充当环境。

回报（Return） 是一个回合（Episode）内获得的累计折扣奖励。

$R=\sum_{t=1}^{T}\gamma^t r_t $

- T：回合长度。
- r_t：第 t 步的即时奖励。
- $\gamma \in [0,1]$：折扣因子（Discount Factor），控制未来奖励的重要程度。

一般而言，γ 越接近 1，智能体越重视长期收益；γ 越小，智能体越偏向近期收益。

强化学习的优化目标为：$\pi^* = \arg\max_\pi \mathbb{E}_\pi[R]$，即寻找能够最大化期望累计回报的最优策略。

1.2 状态价值函数与动作价值函数

强化学习通过价值函数（Value Function）评估不同状态或动作的长期收益，主要包括两种形式。

1. 状态价值函数（State-Value Function）

状态价值函数 $V^\pi(s)$ 衡量：从状态 s 出发，持续遵循策略 π 时能够获得的期望回报。

${ V^\pi(s)= \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty}\gamma^k r_{t+k} \mid s_t=s \right] }$

例如，在网格导航任务中，靠近目标位置的状态通常具有较高价值，而靠近危险区域的状态通常具有较低价值。

2. 动作价值函数（Action-Value Function）

动作价值函数 $Q^\pi(s,a)$ 衡量：在状态 s 下首先执行动作 a，之后遵循策略 π 时的期望回报。

${ Q^\pi(s,a)= \mathbb{E}*{\pi} \left[\sum*{k=0}^{\infty}\gamma^k r_{t+k} \mid s_t=s,a_t=a \right] } $

两者的区别在于：

- $V^\pi(s)$ 评价状态本身的长期价值。
- $Q^\pi(s,a)$ 评价特定状态下执行某一动作的长期价值。

对于离散动作空间，若已经获得最优动作价值函数 $Q^*(s,a)$，则最优动作可由以下规则确定：

$pi^*(s) = \arg\max_a Q^*(s,a)$

课件中的机器人导航示例：

由于向右的 Q 值最高，因此贪心策略选择向右移动。

需要注意，Q 值表示期望回报，而非动作概率，因此可以为负，也不要求归一化。

1.3 深度 Q 网络与策略梯度

深度强化学习（Deep Reinforcement Learning, DRL）利用深度神经网络近似价值函数或策略函数，从而处理高维状态空间，例如游戏图像、机器人传感器数据等。

两类方法：

1. 深度 Q 网络（Deep Q-Network, DQN）

DQN 属于基于价值的方法（Value-Based Methods），利用深度神经网络估计不同动作的 Q 值：

$Q(s, a; \theta) \approx Q^*(s, a)$

其中，θ 表示神经网络参数。其动作选择规则为：$\pi(s) = \arg\max_a Q(s, a; \theta)$

课件中的游戏示例：

由于 $Q(s, \text{Fire}) = 10$ 最大，DQN 选择射击。随后智能体执行动作，观察下一状态及奖励，继续更新网络。

2. 策略梯度（Policy Gradient）

策略梯度属于基于策略的方法（Policy-Based Methods），不通过 Q 值间接选择动作，而是直接学习参数化策略：$\pi_\theta(a \mid s) = P(a \mid s; \theta)$

神经网络（Actor）输出动作概率分布，再根据该分布采样：$a_t \sim \pi_\theta(\cdot \mid s_t)$

尽管 Fire 的概率最高，策略仍可能采样到 Move right。课件展示的采样结果就是 Move right。

对于离散动作空间，动作概率必须满足：$\Sigma_{a \in \mathcal{A}} \pi_\theta(a \mid s) = 1$

对于连续动作空间，策略也可以采用概率分布表示，例如：$pi_\theta(\cdot \mid s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta^2(s))$

以高斯分布为例，展示了根据均值、方差生成动作分布，再采样得到连续动作（如速度 0.4 m/s）的过程。

两类方法的核心区别如下：

维度	DQN	Policy Gradient
方法类别	Value-Based	Policy-Based
网络输出	动作价值 $Q(s, a)$	动作概率 $pi_\theta(a \mid s)$
动作选择	选择最大 Q 值动作	根据策略分布采样
输出约束	Q 值可正可负	概率非负且总和为 1
主要优化对象	价值函数	策略函数

Actor-Critic 方法：Actor-Critic 结合价值估计与策略优化

- Actor：学习策略，负责动作选择。
- Critic：估计价值，为 Actor 的更新提供评价信号。

Part 2 稀疏奖励、奖励塑形与课程学习

2.1 稀疏奖励问题 Sparse Rewards

稀疏奖励是强化学习中的重要困难，指智能体在大多数交互步骤中无法获得具有充分指导意义的奖励，或者奖励仅在任务完成后才出现。

任务	奖励特点	主要困难
围棋（Go）	胜负奖励可能仅在终局产生	奖励延迟，难以评价中间落子
机械臂套环（Ring-on-Peg）	成功套环时才获得有效奖励	随机探索很难达到目标状态

稀疏奖励主要造成两方面的问题：

- 探索效率低：智能体难以通过随机尝试发现成功路径。

- 信用分配困难（Credit Assignment）：最终奖励难以准确反映此前各个动作的贡献。

例如，在机械臂套环任务中，若只有成功套环时奖励为 1，其余时刻奖励为 0，则可表示为：

$r_t= \begin{cases} 1,&\text{成功完成套环}\ 0,&\text{其他情况} \end{cases} $

当成功概率极低时，大量训练轨迹可能完全没有正奖励，导致策略难以获得有效改进。

2.2 奖励塑形 Reward Shaping

奖励塑形通过引入中间奖励，为智能体提供更密集的学习信号，缓解稀疏奖励问题。其基本思想是在原始任务奖励之外，对接近目标、完成中间步骤等行为提供反馈，同时保留任务完成时的主要奖励。

一般可写为： $r'_t = r_t + F(s_t, a_t, s_{t+1})$

- r_t：原始环境奖励。
- F：额外设计的塑形奖励。
- r'_t：用于学习的调整后奖励。

例如，在目标导航任务中，可以奖励智能体缩短与目标的距离： $F_t=\lambda(d_t-d_{t+1}) $

其中 d_t 表示当前位置与目标之间的距离， λ 为奖励权重。

当 $d_{t+1} < d_t$ 时，智能体获得正向反馈。

奖励塑形的风险与改进：不合理的中间奖励可能使智能体追求代理目标，而非真正完成任务。例如，若持续奖励移动行为，智能体可能反复移动以获取奖励。

一种具有理论保证的设计方式是基于势函数的奖励塑形（Potential-Based Reward Shaping）：

$F(s_t, s_{t+1}) = \gamma \Phi(s_{t+1}) - \Phi(s_t)$

其中 $\Phi(s)$ 为状态势函数。在适当的终止条件等假设下，该形式能够保持原始任务的最优策略不变。

2.3 ViZDoom 奖励塑形案例

以 ViZDoom 第一人称射击竞赛说明复杂环境中的奖励设计。任务目标不仅是击败敌人，还需要考虑生存、弹药、移动及资源获取等因素。因此，奖励函数由多个部分组成：

$r_t = r_{\text{living}} + r_{\text{health}} + r_{\text{ammo}} + r_{\text{pickup}} + r_{\text{distance}} + \cdots$

其中，living 是课件中对持续存活状态施加的每动作惩罚，不应误解为单纯奖励生存时间。

奖励塑形的**核心价值**在于将复杂任务的最终目标转化为训练过程中更容易观察的反馈信号。

2.4 课程学习 Curriculum Learning

课程学习通过将复杂任务组织为一系列难度逐渐增加的子任务，使智能体能够从较容易的问题开始学习，再逐步迁移到目标任务。

其一般过程为： $\mathcal{T}_1 \to \mathcal{T}_2 \to \cdots \to \mathcal{T}_n$

其中： $\text{Difficulty}(\mathcal{T}_1) < \text{Difficulty}(\mathcal{T}_2) < \cdots < \text{Difficulty}(\mathcal{T}_n)$

以机械臂套环任务介绍反向课程学习（Reverse Curriculum Learning）。传统训练直接从随机初始位置开始，机械臂很难偶然完成套环，因此难以获得有效奖励。

反向课程学习则从接近目标的位置开始训练：阶段 1 靠近目标 低难度、阶段 2 扩大起点范围、阶段 3 随机初始位置 高难度。通过逐步扩大初始状态范围，智能体可以在已有能力的基础上学习更困难的任务。

奖励塑形与课程学习的区别如下：

维度	Reward Shaping	Curriculum Learning
核心思想	增加中间奖励	调整任务训练顺序
主要改变	奖励函数	任务或初始状态分布
主要作用	增强反馈信号	降低早期探索难度
典型应用	ViZDoom 奖励设计	机械臂反向课程学习

两者可以结合使用，以进一步提高复杂强化学习任务的训练效率。

Part 3 深度强化学习的典型应用

3.1 AlphaGo：监督学习与自我对弈

围棋是深度强化学习的典型应用场景，具有以下特点：

- 状态空间极大：19×19 围棋的合法局面数量远超可观测宇宙中的原子估计数量（约 10^{80}）。
- 奖励稀疏且延迟：胜负通常需要在对局结束后才能确定。
- 长期决策依赖显著：单步落子的价值需要结合后续多步变化判断。

围棋的复杂性使得简单穷举所有可能局面不可行。AlphaGo 通过结合神经网络、搜索与强化学习应对上述问题。

其中有两个学习阶段：

阶段一：基于人类棋谱的监督学习（Supervised Learning）

利用专家对局数据训练策略网络，学习在给定棋局下模仿专家落子： $pi_\theta(a \mid s)$

其中 s 为棋盘状态，a 为落子位置。

可以使用交叉熵损失进行训练：

$\mathcal L_{\mathrm{SL}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log \pi_\theta(a_i^{*}\mid s_i)$

其中 a_i^* 为专家在状态 s_i 下选择的动作。

监督学习可以快速获得具有基本棋力的策略，但其局限在于人类棋谱无法覆盖全部可能的棋盘状态。

阶段二：基于自我对弈的强化学习（RL Self-Play）

在监督学习策略的基础上，通过智能体之间的自我对弈生成新的训练经验，并利用胜负结果继续优化策略。

该阶段不再局限于模仿人类行为，而是通过自主探索改进决策质量。

因此，两阶段学习分别解决：

- 监督学习：提供有效的初始策略，降低训练难度。
- 强化学习：突破专家数据覆盖范围，提高策略表现。

蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）

AlphaGo 还利用 MCTS 在决策时探索未来可能的落子序列，并结合策略网络与价值网络提高搜索效率。

其中：

- 策略网络提供候选动作的先验信息。
- 价值网络估计棋盘局面的胜率或价值。
- MCTS 在有限计算预算内搜索具有较高潜力的决策路径。

3.2 AlphaGo、AlphaZero 与 MuZero 的演进

三代系统展示强化学习从依赖人类经验逐步走向自主学习与规划的发展过程。

系统	人类棋谱依赖	环境知识	核心特点
AlphaGo	使用专家棋谱进行监督预训练	已知围棋规则	监督学习、强化学习与树搜索结合
AlphaZero	不依赖专家棋谱	已知游戏规则	从自我对弈中学习，可应用于围棋、国际象棋和将棋
MuZero	不依赖专家棋谱	不要求预先提供完整环境动态模型	学习用于规划的环境模型

AlphaGo 人类棋谱 + 自我对弈、AlphaZero 从零开始自我对弈、MuZero 学习环境模型并规划

三代系统的主要区别在于学习所依赖的先验知识，减少对人类示范及预先提供的环境动态知识的依赖。

AlphaGo 利用人类专家数据进行初始化；AlphaZero 通过自我对弈学习策略；MuZero 则进一步学习用于规划的环境动态表示。

需要区分：MuZero 不需要预先提供完整的环境转移模型，但仍然需要可交互的环境、动作接口以及奖励等任务信息，并非完全不需要任何规则或环境知识。

3.3 多智能体强化学习 Multi-Agent Reinforcement Learning

多智能体强化学习（MARL）研究多个智能体在共享环境中同时学习与决策的问题。

与单智能体强化学习相比，MARL 的关键特点是一个智能体的决策结果不仅取决于自身动作，还取决于其他智能体的行为。

对于包含 N 个智能体的系统，联合动作可以表示为： $\mathbf{a}_t = (a_t^1, a_t^2, \cdots, a_t^N)$

环境状态转移依赖所有智能体的联合动作： $P(s_{t+1} \mid s_t, a_t^1, \cdots, a_t^N)$

每个智能体 i 的目标仍然是最大化自身期望回报：

$\max_{\pi_i} \mathbb E_{\pi_i,\pi_{-i}} \left[\sum_{t=0}^{T}\gamma^t r_t^i \right] $

其中 π_{-i} 表示其他智能体的策略集合。

以多智能体 Hide and Seek 环境作为示例，说明多个智能体能够在同一环境中相互影响、学习和行动。

根据奖励目标的关系，多智能体系统可分为三种类型：

类型	目标关系	课件示例
Cooperative（合作）	共享目标与团队收益	仓库机器人协同配送
Competitive（竞争）	目标相互对立	网球比赛中的双方选手

类型	目标关系	课件示例
Mixed（混合）	同时存在合作与竞争	足球比赛中队内合作、队间竞争

奖励设计决定了智能体之间的行为关系。例如，网球比赛中的胜负奖励使双方成为竞争者，而仓库配送任务中的共享奖励则鼓励合作。

3.4 多智能体独立学习的困难

每个智能体可以独立采用 DQN 或策略梯度进行训练，但这种方式存在两项主要困难。

1. 环境非平稳性（Non-Stationarity）

在单智能体强化学习中，通常假设环境转移规律相对稳定。但在多智能体环境中，其他智能体也会不断更新策略。因此，从某个智能体的视角看，即使自身状态和动作不变，下一状态及奖励的分布也可能随着其他智能体的策略变化而改变：

$P(s_{t+1} \mid s_t, a_t^i; \pi_{-i})$

当 π_{-i} 不断变化时，该转移分布也随之变化。

这会导致：

- 已学习的有效动作可能失效。
- 历史经验可能与当前环境不一致。
- 策略优化过程更难稳定收敛。

2. 团队信用分配问题（Multi-Agent Credit Assignment）

在合作任务中，多个智能体可能共享一个团队奖励：

$R_{\mathrm{team}} = \sum_{t=0}^{T}\gamma^t r_t^{\mathrm{team}}$

但团队奖励无法直接指出每个智能体对最终结果的具体贡献。

例如，多个机器人共同完成配送后获得奖励，但无法仅根据最终得分判断哪个机器人的动作最关键。

一种常见解决思路是 Centralized Training with Decentralized Execution：

- 训练阶段：Critic 可以利用多个智能体的联合状态、动作等信息进行价值评估。
- 执行阶段：每个 Actor 根据自身可获得的观测独立选择动作。

CTDE 可以缓解部分非平稳性与信用分配问题，但不能保证完全消除这些困难。

Part 4 现代 LLM 智能体及信息来源

4.1 智能体架构的转变

以传统 AI Agent 与现代 AI Agent 的对比，引出基于大语言模型的智能体。

传统强化学习智能体主要通过环境交互获得奖励，再利用梯度更新模型参数。现代 LLM 智能体则以预训练语言模型为核心，结合记忆、自然语言指令、工具调用与反馈进行任务执行。

维度	传统 RL Agent	现代 LLM Agent
核心模型	策略网络、价值网络	大语言模型
决策依据	状态、价值估计、奖励	语言上下文、预训练知识、外部信息
主要学习机制	奖励驱动的参数优化	预训练与后训练，推理时可结合反馈调整行动
反馈利用	更新策略参数	可通过反思、记忆与重新规划调整后续步骤
外部能力	环境交互	搜索、工具、记忆及其他系统

两种不同的反馈机制：

- 传统 RL 中的梯度下降通常用于更新模型参数。
- LLM Agent 的反思推理通常是在推理过程中利用反馈调整后续行动，不必然修改模型参数。

两类智能体并非完全互斥。现代 LLM 也可以通过强化学习进行后训练，LLM Agent 也可以使用价值评估或奖励机制。

整门课程划分为四个阶段：

阶段	课程	核心内容
Agent Foundations	L1 - L3	AI Agent 概述、多模态嵌入、神经网络、Transformer、经典 RL、深度 RL
Agent Interaction	L4 - L7	Prompt Engineering、Chain of Thought、工具增强智能体、Retrieval、RAG、Grounding 与 Verification
Agent Systems	L8 - L10	单智能体与多智能体系统、Codex、Skills、具身智能体、Vision-Language-Action 模型
Integration	L11 - L13	系统评估、前沿研究与项目实践

本讲是 Agent Interaction 阶段的起点，重点从传统 RL 决策机制过渡到 LLM 智能体的信息获取与任务执行机制。

4.2 大语言模型的知识边界 Knowledge Boundary of LLMs

4.2.1 预训练知识的来源

大语言模型通过大规模预训练语料学习语言模式、事实关联及其他知识表示。

主要预训练知识来源包括：

数据来源	代表平台或资源
Internet Forums（网络论坛）	Reddit、Quora
Books（书籍）	Google Books、Open Library
Encyclopedias（百科）	Wikipedia、Wikidata
News Sites（新闻网站）	The New York Times、BBC
Academic Articles（学术论文）	arXiv、PubMed

这些来源共同构成模型可能接触的知识基础，但并不意味着每个模型都实际使用了上述所有平台的数据。

LLM 的知识能力受到训练数据的覆盖范围、质量、频率和时间范围等因素影响。

4.2.2 预训练知识的覆盖与频率差异

Pretraining data shapes the agent's internal knowledge structure.

预训练数据的分布会影响模型内部形成的知识表示。不同主题在训练数据中的出现频率并不相同。

知识类别	课件示例	数据分布特点
高频知识	Cooking、Celebrities、Movies	相关语料较丰富
相对低频知识	Cantonese、Quasars	相关语料相对有限
非公开知识	Secret / Proprietary Data	可能未包含在可访问训练语料中

通常，某个主题在训练数据中出现得越频繁，模型就越有机会学习与其相关的语言和事实模式。

但知识频率并不等同于知识正确性。高频错误同样可能被模型学习，而低频知识也不一定无法被准确掌握。

此外，模型可能在缺乏充分依据时生成表面合理但事实上不正确的内容，即产生幻觉（Hallucination）。

4.2.3 对不完美自然语言的理解能力

LLM 对自然语言具有一定鲁棒性，可以处理拼写错误、缩写和不完整表达。

- 非标准输入：can u cook egs in microwav
- 标准表达：Can you cook eggs in the microwave?

模型能够识别两者表达的基本意图，并给出关于微波炉加热鸡蛋的安全提示，例如某些加热方式可能导致鸡蛋爆裂。

这一能力说明，LLM 不必依赖严格的关键词匹配或固定命令格式，即可从自然语言中推断用户意图。但自然语言容错不意味着模型能够可靠识别所有错误前提，因此仍需要事实验证。

4.2.4 训练数据的质量问题

训练数据中可能存在的三类错误：

1. Typos（拼写错误）：例如 `cookin egs` 等不规范表达。
2. Misconceptions（错误观念）：未经验证或不符合事实的观点。
3. Outdated Information（过时信息）：例如将北京 2022 年冬奥会描述为尚未举行的事件。

这些问题可能影响模型输出的准确性与可靠性。

从知识形成过程看，模型主要通过学习数据中的统计规律获得能力，而非对每条训练信息进行独立的事实核验。

因此，训练数据中的错误、偏差和过时内容可能反映在模型生成的答案中。

4.2.5 知识截止日期 Knowledge Cutoff

预训练知识具有时间边界。

课件以 GPT-5.4 为例，将 2025 年 8 月 31 日标记为其知识截止日期，用于说明模型无法仅依靠该截止日期之前的预训练数据掌握之后发生的新事件。这一日期是课件示例，并不代表所有模型具有相同的知识截止时间。

知识截止日期带来的问题包括：无法仅依靠预训练知识获得最新新闻、无法保证掌握当前营业信息、政策及价格、无法确认训练截止日期之后发生的事件、可能沿用已经失效的历史事实。

因此，预训练知识适合提供相对稳定的基础知识，但不能替代实时信息检索。

4.3 联网搜索与信息可靠性 Web Search

4.3.1 联网搜索的适用场景

联网搜索（Web Search）使 LLM Agent 能够在推理过程中获取外部信息，以弥补预训练知识的局限。

问题分为两类：预训练知识和 Web Search

联网搜索的主要触发条件可概括为：

- Real-time：需要当前或近期信息。
- Location-specific：需要特定地理位置的信息。
- Niche：涉及相对小众、训练数据可能覆盖不足的知识。

例如，查询香港红磡附近的健身房时，需要结合近期用户评价、健身房官网及地理位置信息。而对于长洲抢包山比赛，搜索可以补充活动背景、规则和具体安排。

4.3.2 联网搜索的触发机制

两种搜索触发方式：

1. 模型自主触发（Model-Initiated Search）

模型根据用户问题判断是否需要外部信息。

例如，当问题涉及实时、本地或小众知识时，模型可以主动选择搜索。

2. 用户显式触发（User-Initiated Search）

用户明确要求模型使用网络搜索，或在界面中主动启用搜索功能。

两种方式的区别主要在于工具调用决策由模型还是用户发起。

4.3.3 搜索来源的可靠性与时效性

联网搜索可以获取外部信息，但并不保证搜索结果天然准确。两个风险：

1. 来源可靠性差异（Source Reliability）

搜索结果可能来自：社交媒体、博客、网络论坛、百科平台、政府机构、专业组织、学术研究。

不同来源的可靠性、证据质量和适用范围存在差异。2025 年 LLM 常见引用域名统计中，Reddit、Wikipedia、YouTube、Google、Yelp 等平台占据较高位置，说明模型可能大量引用热门网络来源。但引用频率并不能直接证明来源可靠。

2. 来源时效性问题（Source Freshness）

以美国内华达州 Henderson 的跑步地点查询为例。模型检索到一条发布于 2003 年的跑步论坛帖子，但该信息距离查询时间已有约 23 年，相关场地可能已经关闭。

这说明：成功检索到信息，不代表该信息仍然有效。对于营业状态、交通路线、设施开放情况等动态信息，必须核查发布时间、更新日期和当前官方公告。

4.3.4 来源引导 Source Guidance

来源引导是提示工程中控制搜索质量的重要方法。

普通提示：How safe are gray-market peptides?

来源约束提示：How safe are gray-market peptides? Use official health organizations and rigorous scientific studies.

第二种提示明确要求使用官方卫生机构与严谨科学研究，能够引导搜索过程优先考虑更符合证据要求的信息来源。提示不仅决定搜索主题，也可以影响搜索来源的选择与信息过滤规则。

需要注意，权威机构和学术论文也应结合具体问题评估其适用性、发布时间与证据质量，而不能仅根据机构名称判断结论一定正确。

4.3.5 联网搜索的多步骤工作流

课件将 Web Search 描述为一个包含检索、筛选和综合的多步骤过程：

用户提示中的主要信息包括：

- Topic：研究主题。
- Task：需要完成的任务。
- Target Sources：目标信息来源。
- Desired Depth：期望的信息深度。

搜索系统据此执行检索、扫描结果、过滤不相关内容、汇总信息和生成回答。

其中，信息过滤不仅要考虑主题相关性，还应结合可靠性、时效性和来源质量。

案例：黄山徒步

该案例说明，复杂度适中的搜索任务可以分解为多个具体信息需求，再将结果整合为可执行的建议。

4.3.7 搜索引擎与 AI 模型的选择

传统搜索引擎与 LLM 在信息处理方式上具有不同优势。

搜索引擎（Search Engine）	AI 模型（AI Model）
快速浏览多个搜索结果	综合多个来源的信息
定位具体网站或页面	解释复杂概念
查看信息原始形式	比较不同来源的观点
适合明确的导航和查找任务	适合解释、比较与综合任务

两个典型例子：

搜索 2013 Honda Civic air filter：更适合通过搜索引擎定位 The Home Depot、Walmart 等商品页面；查询维生素补充剂的利弊：更适合利用 AI 综合 WHO、FDA、EMA 等机构的相关信息。

因此，搜索引擎更适合获取原始页面与定位资源，而 AI 模型更适合跨来源的信息解释与综合。两者可以结合使用。

4.4 Deep Research 与 Agentic AI

4.4.1 深度研究的基本概念

深度研究（Deep Research）是一种面向复杂问题的多步骤信息获取与分析工作流。

与普通联网搜索相比，其主要特点是：

1. Plan a Research Task：将复杂目标分解为多个研究子任务。
2. Read and Evaluate Sources：阅读并评估不同来源的内容。
3. Synthesize Findings with Citations：综合研究结果，并提供可追溯的引用。

深度研究通常需要多次搜索、阅读、评估和迭代，而不是执行一次检索后直接生成答案。

4.4.2 深度研究的迭代工作流

课件展示的完整流程为：

1. Plan Research Task 研究规划与子问题分解
2. Web Search 多次检索；Read Sources 阅读资料
3. Evaluate Sources 判断相关性、可靠性与充分性
4. More Needed 返回检索阶段补充证据；Sufficient 进入结果综合阶段
5. Synthesize → Draft → Add Citations 综合证据、撰写报告并添加引用
6. Final Answer with Sources

深度研究的关键在于评估结果是否充分，并据此决定继续检索还是生成最终报告。

这构成一个基于证据充分性的反馈循环，而不是固定执行一次的线性流程。

Agentic AI：模型能够在给定任务范围内，自主决定下一步需要执行的操作。

这种自主性体现在：

- 根据任务目标分解研究步骤。
- 判断不同子任务需要哪些信息来源。
- 在执行过程中决定下一步搜索或阅读内容。
- 整合检索结果并生成报告。

但自主执行并不等于不需要用户控制。用户仍然可以通过任务范围、约束条件及必要的确认步骤限制智能体行为。

4.5 Web Search 与 Deep Research 的区别

维度	Web Search	Deep Research
问题类型	通常是单个聚焦问题	多个相互关联的问题
任务复杂度	相对较低	相对较高
来源数量	通常少量来源	可能几十条或更多
典型耗时	几秒至几分钟	几分钟或更长
处理过程	检索、过滤、总结	规划、检索、阅读、评估、迭代、综合
主要产出	聚焦的事实或答案	带证据的综合分析报告
案例一	寻找附近高评分健身房	研究每日步数对长期健康的影响
案例二	查询迪拜本周天气	分析天气对迪拜旅游业的影响

两者的主要区别不是是否使用搜索工具，而是研究任务的复杂度、证据整合要求及是否需要多轮迭代。

4.6 智能体信息来源的综合比较

现代 LLM 智能体的三种信息获取方式对比：

维度	Pretrained Knowledge	Web Search	Deep Research
信息来源	模型内部参数知识	外部检索结果	多轮外部检索与分析
检索来源数量	无	通常少量	可能几十条以上
信息时效性	受训练数据限制	取决于来源	取决于来源
典型耗时	秒级	秒至分钟级	分钟级或更长
主要用途	定义、常识、基础解释	实时、本地、小众信息	复杂问题的多源综合
课件示例	手机掉入汤中如何处理	寻找附近健身房	每日步数与长期健康关系

三种方式不是相互排斥的独立技术，而是可以组合使用的信息获取机制。

例如，深度研究通常同时利用：

- 预训练知识理解任务、生成研究计划。
- Web Search 获取外部证据。
- LLM 的推理与综合能力整理资料。
- 引用机制提高结果的可追溯性。

因此，现代 LLM Agent 的能力不仅取决于模型自身掌握多少知识，也取决于其能否正确判断信息需求、选择工具、评估来源并综合证据。

4.7 本讲核心知识总结

本讲的内容可以归纳为两条主要技术路线。

第一条：从奖励驱动的学习到复杂环境中的自主决策。

强化学习通过奖励优化策略，DQN 和策略梯度分别代表基于价值与基于策略的典型方法。

当任务具有稀疏奖励时，可以通过奖励塑形和课程学习改善训练条件。AlphaGo 系列展示了监督学习、自我对弈、搜索与模型学习的结合；多智能体强化学习则进一步引入环境非平稳性与团队信用分配问题。

第二条：从预训练知识到外部信息增强的智能体。

LLM 具备广泛的预训练知识，但受到数据覆盖、知识频率、数据质量及知识截止日期等限制。

联网搜索通过外部信息补充模型知识，同时引入来源可靠性和时效性问题。深度研究进一步将信息获取组织为规划、检索、评估、迭代与综合的完整工作流。

两条路线共同体现了智能体研究的核心问题：如何使智能体在不确定环境中利用反馈与信息持续改进决策。

Part 5 思维链推理与问题分解

本部分讨论如何通过提示工程（Prompt Engineering）提高大语言模型处理复杂任务的能力。其核心思想是：将复杂任务分解为可管理的中间步骤，并根据任务性质选择语言推理、程序计算或外部工具。

主要涉及三种方法：

1. 思维链推理（Chain-of-Thought, CoT）：通过中间推理步骤解决复杂问题。
2. 程序辅助语言模型（Program-Aided Language Models, PAL）：利用语言模型生成程序，再由程序执行精确计算。
3. 问题分解与提示链（Problem Decomposition & Prompt Chains）：将复杂任务拆分为多个子任务，通过模型与工具的多轮调用完成。

5.1 思维链提示 Chain-of-Thought Prompting

5.1.1 基本原理

思维链提示是一种通过引导模型生成中间推理步骤来完成复杂任务的提示技术；传统提示通常要求模型直接输出最终答案，而思维链提示则要求模型先完成必要的中间计算或逻辑分析，再得出结论。

1. 以网球数量计算为例：Roger 原本有 5 个网球，又购买了 2 罐网球，每罐有 3 个。他现在共有多少个网球?

标准提示（Standard Prompting）可能直接得到答案，而思维链提示会提供分步求解。

2. 通过苹果数量问题说明两种方法的区别：某食堂原有 23 个苹果，使用 20 个制作午餐，之后又购买 6 个。

标准提示示例错误地回答为 27，而思维链提示按照步骤求解。

显式分解问题可以帮助模型处理多步骤计算，降低遗漏条件或错误组合运算的风险。但需要注意，思维链并不保证结果正确。模型生成的推理步骤仍可能存在逻辑错误或计算错误，因此不能仅凭推理过程完整就认定结论可靠。

5.1.2 Few-shot CoT 与 Zero-shot CoT

思维链提示主要有两种实现方式。

比较维度	Few-shot CoT	Zero-shot CoT
提示方式	提供包含中间步骤的完整示例	直接使用推理指令
是否需要示例	需要	不需要
主要机制	上下文学习（In-context Learning）	通过指令诱导模型进行分步推理
提示长度	相对较长	相对简短

1. Few-shot Chain-of-Thought

通过在提示中提供若干完整的解题示例，使模型学习任务的推理结构。

其形式可以表示为： $P = \{(x_1, r_1, y_1), \cdots, (x_k, r_k, y_k), x_{\text{new}}\}$

- x_i：示例问题。
- r_i：示例中的中间推理步骤。
- y_i：示例的最终答案。
- x_{new}：需要解决的新问题。

模型根据上下文中的示例，生成新问题的求解步骤及答案。这种方法属于上下文学习，不需要更新模型参数。

2. Zero-shot Chain-of-Thought

即使没有提供完整的推理示例，模型也可能通过简单指令生成中间推理步骤。

典型指令为：Let's think step by step.

该研究涉及 PaLM 和 InstructGPT 等模型。模型能够在缺少示例的情况下产生推理步骤，可能与预训练数据中包含的推理范例，以及系统指令等因素有关。

CoT 的适用边界

思维链更适合多步骤推理、逻辑分析和复杂问题求解，而不是所有任务都必须生成详细推理文本。对于简单事实查询，直接回答通常更高效；对于精确计算，则应优先使用程序验证。

5.2 程序辅助语言模型 Program-Aided Language Models, PAL

5.2.1 基本思想

程序辅助语言模型（PAL）通过结合自然语言理解与程序执行，提高复杂计算任务的可靠性。

核心原则：由语言模型理解问题并生成程序，由程序解释器执行计算并返回结果。

传统语言模型直接生成计算结果，可能产生算术错误。PAL 将自然语言推理与数值计算分离，使计算过程能够通过确定性程序执行。

整体流程：自然语言问题（Question）；LLM：理解问题并生成程序；Python Interpreter：执行程序；计算结果（Answer）。

5.2.2 PAL 的实现方式

两项关键方法：

1. 在提示中提供结合自然语言与代码的示例。
2. 由 LLM 生成程序，并使用 Python 解释器执行程序以获得答案。

5.2.3 PAL 的优势与局限

PAL 的主要优势包括：

- 计算准确性：使用程序执行算术运算，减少语言模型直接计算产生的错误。
- 可复现性：相同输入和相同程序通常能够得到相同结果。
- 可验证性：可以检查生成的代码、变量定义及计算逻辑。
- 工具扩展性：可进一步使用 Python 库处理统计分析、符号计算及数据可视化任务。

需要区分两类错误： 最终结果错误 = 问题形式化错误 ∨ 程序执行或计算错误

这一表达式用于说明错误来源，而不是概率公式。即使程序能够正确执行，如果模型错误理解题意、遗漏条件或生成错误公式，最终结果仍然可能不正确。

因此，PAL 的可靠性依赖于两个环节：

1. 语言模型是否正确地将问题转化为程序。
2. 程序是否正确执行并产生预期结果。

5.3 问题分解与提示链 Problem Decomposition and Prompt Chains

5.3.1 问题分解

问题分解是指将复杂任务划分为多个规模较小、目标明确的子问题，并逐步解决。

典型工作流（重复执行）： $\text{提出子问题} \rightarrow \text{搜索引擎检索} \rightarrow \text{获取中间答案} \rightarrow \text{继续提问} $

例如，问题为：Theodore Haeker 和 Harry Vaughan Watkins 谁活得更久？

模型首先需要分别查询两人的死亡年龄，再比较结果；另一示例涉及查询当前最高的木制格构塔及其建成年份。模型首先通过搜索确定目标建筑为 Gliwice Radio Tower，随后继续查询其建成年份，得到 1935 年。

复杂信息查询往往需要利用前一步的检索结果来确定下一步的检索目标。通过 Few-shot Examples 示范如何提出子问题、调用搜索工具以及根据检索结果继续追问。

5.3.2 提示链 Prompt Chains

提示链是将多次语言模型调用与外部函数调用按照一定顺序连接起来的工作流。

5.3.3 思维链与提示链的区别

维度	思维链（CoT）	提示链（Prompt Chains）
核心对象	推理过程	系统执行流程
主要形式	中间推理步骤	多次模型或工具调用
是否必须调用工具	否	否，但可以集成工具
中间结果	推理步骤或中间答案	结构化数据、文本或工具结果
主要作用	提升复杂推理能力	提高复杂任务的可管理性

两者可以结合使用：提示链中的某个模型调用可以采用思维链提示，而整个任务由多个调用共同完成。

总结： CoT 解决如何组织推理，PAL 解决如何借助程序计算，Problem Decomposition 解决如何拆分任务，Prompt Chains 则将这些步骤组织为可执行的工作流。

Part 6 多模态人工智能与内容生成 Multimodal AI and Content Generation

多模态人工智能能够处理不同形式的信息，包括文本、图像、音频和视频。

本部分主要讨论多模态输入输出、视觉理解的局限、图像提示工程以及生成模型的基本原理。

6.1 多模态输入与输出 Multimodal Inputs and Outputs

6.1.1 多模态系统的基本结构

传统文本模型主要处理自然语言，而多模态模型可以同时接收不同类型的信息，并生成一种或多种形式的输出。

多模态任务可以抽象为： $Y=f_{\theta}(X_{\text{text}},X_{\text{image}},X_{\text{audio}}) $

其中，f_θ 表示多模态模型，X 表示不同模态的输入，Y 表示输出结果。

6.1.2 多模态生成的价值

课件通过两个示例说明生成式 AI 在视觉表达方面的应用：

1. 从设计概念到可视化效果

通过生成图像，可以将蛋糕设计构想转化为具体的视觉参考，从而提高设计方案的可沟通性。

2. 生成现实中难以拍摄的场景

例如，生成一个人缩小到昆虫大小的画面。相较于静态图像，视频还包含时间维度上的变化，包括人物动作、物体运动以及场景演化。

因此，视频生成不仅需要保证单帧画面的合理性，还需要维持连续帧之间的一致性。

6.2 多模态理解的局限与图像提示策略

6.2.1 细粒度视觉识别的挑战

尽管多模态模型能够理解图像中的主要对象，但在识别外观高度相似的物体时，仍可能发生错误。

课件中的健身器械案例：图像整体语义理解能力不等于细粒度视觉辨别能力。对于具有相似外观的设备，模型可能忽略决定其功能的局部结构。

实际应用中，应重点关注物体的结构、连接方式、操作部件和功能差异，而不应仅依赖整体外观。

6.2.2 图像与文本的联合推理

课件以餐厅收据为例，说明多模态模型需要同时完成图像信息提取、文本条件匹配和数值计算。

这一案例反映出多模态任务可能同时包含三种误差来源：

1. 图像识别错误，例如读取了错误的商品价格。
2. 语义匹配错误，例如未正确识别用户消费的商品。
3. 数值计算错误，例如错误计算税费或合计金额。

因此，对于涉及票据、报表等结构化信息的任务，应将信息提取与数值计算分开处理，并对最终结果进行复核。

6.2.3 图像提示的实践原则

课件提出以下建议：

1. 利用图像文字识别能力：模型可以读取图像中的基本文字，例如收据、标牌和截图。
2. 注意视觉细节遗漏：对关键数字、细小文字和相似物体进行额外检查。
3. 必要时提供多张图像：使用不同角度、不同区域或不同时间的图像补充信息。
4. 明确每张图像的作用：说明各图提供的具体信息，减少模型对图像关系的错误理解。

其中，第四点尤为重要。例如，分析某个设备时，可以明确规定：图一展示设备整体结构、图二展示控制面板、图三展示接口与连接方式。这种方式能够提高输入信息的组织程度，使模型更容易围绕任务目标提取有效信息。

6.3 多模态生成的能力、时间与成本

6.3.1 多模态生成技术的发展

课件通过不同阶段的生成结果说明图像、视频和语音生成能力的发展。

视频生成方面，对比了：Imagen Video（2022）、Seedance 系列视频生成模型

语音生成方面，对比了早期与近期的 AI 语音生成效果。这些示例主要说明多媒体生成在视觉质量、动作表现和语音自然度等方面取得了显著进展。

6.3.2 不同模态的生成成本

课件按照典型生成时间与成本，从低到高给出以下排列： Text < Speech < Images < Video

这里的 C 表示课件中的典型成本趋势，而非适用于所有模型的严格不等式。

实际生成成本还取决于模型、输出长度、分辨率、采样设置及服务定价。

在工程实践中，可以先使用低成本的文本交互确定需求，再进行图像生成或视频生成，从而减少不必要的高成本调用。

6.4 图像生成提示工程 Image Generation Prompt Engineering

6.4.1 图像提示的核心组成

课件建议通过 AI 辅助编写图像生成提示，将简短需求扩展为包含具体视觉信息的描述。

例如，原始需求为：生成一只猫在夜晚秘密经营咖啡店的图像。

扩展后的提示包含三个主要组成部分。

组成	英文	主要内容
场景	Setting	时间、地点、环境、背景
角色细节	Character Details	主体外观、服装、动作
氛围与风格	Mood / Style	情绪、艺术风格、画面表现

例如：

- Setting：夜间咖啡店、温暖的室内环境。
- Character Details：橘猫、围裙、制作浓缩咖啡。
- Mood / Style：奇幻、温馨、卡通风格。

这一结构能够将抽象创意转化为更具体的视觉生成要求。

6.4.2 使用明确的视觉风格词汇

课件列举了四种典型风格：相比于“让图片更好看”等模糊要求，明确指定视觉风格更容易形成可执行的生成约束。此外，课件还建议让 AI 描述参考图像，将参考图中的构图、光线、色彩与风格转化为文字提示。

图像生成提示的扩展结构

在课件三个组成部分的基础上，可以进一步加入构图与限制条件：

- S：Setting，场景。

- *C*：Character Details，主体细节。
- *M*：Mood / Style，氛围与风格。
- *K*：Composition，构图与镜头。
- *R*：Restrictions，限制条件。

6.5 图像生成模型的基本原理

6.5.1 自回归生成与扩散生成

两种生成机制：

1. 自回归模型（Autoregressive Models）

自回归语言模型通常根据已有 token 序列，逐个预测后续 token。其联合概率可以表示为：

$P(x_1,\ldots,x_T) = \prod_{t=1}^{T}P(x_t\mid x_1,\ldots,x_{t-1})$

其中，x_t 表示第 t 个 token。

2. 扩散模型（Diffusion Models）

扩散模型通过多个步骤，将噪声逐渐转化为具有结构的图像。简化的反向生成过程为：

$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$

x_T：初始噪声；x_t：中间去噪状态；x_0：最终生成图像。

对于文本条件图像生成，模型根据文本提示对应的条件表示引导去噪过程，使最终图像符合用户描述。

简化流程：Text Prompt → Text Understanding → Latent Representation → Image Generation

文本理解模块负责处理提示中的词语及其关系，图像生成模块根据条件信息生成视觉内容。需要注意，扩散模型并非唯一的图像生成方法，实际系统也可能使用其他生成架构。

6.5.2 从低维潜在分布生成图像

课件进一步介绍了生成模型的一般思想：通过生成器，将低维潜在变量映射到高维图像空间。

设潜在变量为：$z \sim p_z(z)$

其中，p_z 可以是标准正态分布：$z \sim \mathcal{N}(0, I)$

生成器执行映射：$y = G_\theta(z)$

- z：低维潜在向量。
- G_θ：生成器。
- y：生成的高维图像表示。

生成器通过学习，将简单潜在分布映射为复杂的数据分布，从而生成具有不同外观与特征的图像。

6.5.3 生成对抗网络 Generative Adversarial Networks, GAN

GAN 由两个网络组成：

- Generator（生成器）：生成尽可能逼真的图像，以欺骗判别器。

- Discriminator（判别器）：判断输入图像来自真实训练数据还是生成器。

训练过程可以理解为生成器与判别器之间的对抗博弈。

其经典目标函数为： $\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log (1 - D(G(z)))]$

- $D(x)$：判别器认为真实图像 x 为真的概率。
- $G(z)$：生成器根据潜在向量 z 生成的图像。
- $D(G(z))$：判别器认为生成图像为真的概率。

生成器希望生成图像难以被判别器识别为伪造，判别器则希望提高真假识别能力。在反复训练过程中，生成器逐渐学习真实图像的数据分布。

GAN 与扩散模型的主要区别

维度	GAN	Diffusion Model
核心机制	生成器与判别器对抗训练	学习逐步去噪
生成过程	通常一次生成映射	通过多个去噪步骤生成
典型训练特点	对抗优化	去噪目标优化
主要挑战	训练不稳定、模式崩溃	采样计算成本等

Part 7 使用 AI 进行数据分析 Data Analysis with AI

本部分讨论如何结合语言模型、代码执行工具与数据可视化完成数据分析任务。

核心原则：模型负责理解分析目标、组织流程和解释结果，代码负责数据处理、数值计算及绘图。

7.1 AI 数据分析的基本流程

三个步骤：

1. Ask a clear question about your data：明确数据分析问题。
2. Use code to calculate and plot：使用代码计算并绘制图表。
3. Review the resulting insights：审查分析结果与结论。

这一流程强调，数据分析不能仅依赖模型生成的文字解释，而应建立在实际数据计算和结果验证的基础上。

7.1.1 明确分析目标

分析问题决定了需要使用的数据字段、统计指标和可视化方式。

例如：我的跑步配速和距离如何变化？该问题至少涉及时间、跑步距离和配速三个变量。

7.1.2 使用程序计算与可视化

课件展示了个人跑步记录的分析案例。

7.1.3 审查分析结论

数据分析完成后，应检查：

- 指标定义和单位是否正确。
- 统计结果是否与原始数据一致。
- 图表是否正确反映数据变化。
- 模型给出的解释是否具有数据依据。

尤其需要区分相关关系与因果关系，不能仅根据趋势同时变化就认定存在因果联系。

7.2 代码执行作为智能体工具

7.2.1 AI 工具的基本类型

课件将代码执行与其他工具放在同一个智能体框架中。

其中，代码执行工具主要用于需要精确计算或生成图表的任务。

7.2.2 工具调用与推理循环

智能体可以根据当前上下文判断是否需要使用工具，并将工具返回结果重新纳入后续处理。

这一流程将语言模型的推理能力与外部工具的执行能力结合起来。

7.2.3 商业销售数据分析

以小型企业销售数据为例，展示 AI 如何通过代码执行完成月度分析。

数据包含：Date：日期；Orders：订单数量；AvgOrderValue：平均订单金额；Revenue：营业收入

基本关系为： $R_i = N_i \times A_i$

其中，R_i 表示第 i 天营业收入，N_i 表示订单数，A_i 表示平均订单金额。

月度营业收入为： $R_{\text{month}}=\sum_{i=1}^{n}R_i $

模型可以通过代码计算月度营业收入、生成图表并识别异常值。

该案例展示了 AI 从自然语言问题到可执行数据分析流程的转换。

7.3 从数据中提取洞察 Getting Insights from Data

课件以饮品销售数据为例，要求识别销售变化最大的饮品并绘制图表。

模型执行的主要步骤为：

1. Inspecting the data：检查数据。
2. Calculating monthly changes：计算月度变化。
3. Identifying outstanding drinks：识别变化显著的饮品。
4. Plotting selected drinks：绘制选定饮品的趋势图。

图表同时使用 Others (avg) 作为其他饮品的平均销售基线，并标注 Spring Promo、Summer Push 和 Fall Launch 三个营销阶段。

销售变化的计算：设某饮品在月份 t 的销售额为 S_t。

绝对变化量： $\Delta S_t = S_t - S_{t-1}$

相对变化率： $r_t=\frac{S_t-S_{t-1}}{S_{t-1}}\times100\%$

其中，变化率仅在 $S_{t-1} \neq 0$ 时可直接使用。

绝对变化量反映实际销售额的变化规模，相对变化率反映相对于原有销售水平的变化幅度。

可视化不仅是展示数据，更应突出与分析问题相关的变化、异常和对照关系。

Part 8 人工智能的能力与责任 AI Capabilities and Responsibility

8.1 生成式 AI 的双重用途

随着生成式 AI 能力增强，技术应用不仅需要考虑功能实现，还需要考虑安全性、授权和潜在社会影响。

课件通过 AI 语音生成说明同一种技术既可以支持合理应用，也可能被用于欺诈。

应用场景	具体用途	风险性质
Podcast Editing	修复播客中的口误或音频问题	正当编辑用途
Character Voice Generation	为虚拟角色生成逼真的语音	创意内容制作
Voice Impersonation	模仿亲属声音实施诈骗	身份冒充与欺诈

语音生成本身并不能决定应用是否安全，关键取决于具体使用方式、是否获得授权以及是否具有误导或欺诈目的。

8.1.1 负责任使用 AI 的原则

在实际系统中，可以从以下方面降低风险：

- 知情同意：使用真实人物声音或形象时，应获得必要授权。
- 透明性：在适当场景中标明内容由 AI 生成或修改。
- 隐私保护：避免未经授权收集或处理个人敏感数据。
- 身份验证：对涉及资金、账户及身份的高风险请求进行独立验证。
- 人工监督：对可能产生重大影响的输出设置审核机制。

这一部分与前文的多模态生成形成对应关系：模型生成能力越强，越需要将安全约束纳入实际应用设计。

Part 9 使用 AI 构建应用 Building Apps with AI

本部分介绍如何通过自然语言提示，让 AI 辅助生成交互式应用程序。

课件强调将 AI 的代码生成能力与明确的应用需求结合起来，并通过简单项目逐步积累开发经验。

9.1 AI 辅助应用开发

AI 可以将用户描述的功能需求转换为程序代码，并组合界面、交互逻辑与算法。

应用类型包括：Game App：游戏应用、Study Timer：学习计时器、Quiz App：测验应用、Habit Tracker：习惯追踪应用、Expense Splitter：费用分摊工具。

应用开发的关键不只是让模型生成代码，还包括准确描述需求并验证生成结果。

9.2 AI 应用提示的三个基本要素

三个基本模块组织应用开发提示。

模块	英文	核心问题
Goal	What to create	要开发什么应用?
Input	What users provide	用户如何提供输入或操作?
Output	What the app shows	应用应该显示什么结果?

三者共同构成应用需求的基本描述框架。

9.2.1 示例：寻路游戏（Pathfinding Game）

用户希望构建一个可以放置障碍物与目标位置，并模拟路径搜索的游戏。

其需求可以拆分为：

- Goal：构建交互式寻路游戏。
- Input：用户在画布上设置障碍物、起点和终点。
- Output：显示搜索到的路径，并通过动画演示。

模型使用单个 HTML 文件实现交互式游戏，并使用 A* 算法进行路径搜索。

A* 算法的评价函数为： $f(n) = g(n) + h(n)$

- $g(n)$：从起点到当前节点 n 的已知路径代价。
- $h(n)$：从当前节点到目标节点的估计代价。
- $f(n)$：用于选择搜索节点的估计总代价。

当启发函数满足相应条件时，A* 可以找到最优路径。

9.2.2 示例：烟花模拟器（Fireworks Simulator）

用户要求：构建一个交互式烟花应用，点击屏幕后从底部发射烟花并产生爆炸效果。

其需求结构为：

- Goal：生成烟花模拟器。
- Input：用户点击或触摸屏幕。
- Output：显示彩色烟花的发射、爆炸与消散动画。

课件中的模型使用 HTML5 Canvas 与 JavaScript 实现烟花动画，并引入重力、摩擦和渐隐拖尾等效果。该案例说明，用户可以先描述应用目标与交互行为，再由模型选择适当的实现技术。但对于重要功能，仍需要在提示中明确约束，避免模型自行补充不符合预期的设计。

9.3 应用需求描述与验证

9.3.1 学习计时器（Study Timer）

课件要求创建一个包含以下功能的学习计时器：25 分钟专注时间。5 分钟休息时间。Start、Pause 和 Reset 控制。倒计时结束时显示完成消息。提供 10 秒测试模式，便于课堂验证完整流程。

该任务不仅定义了应用功能，还包含明确的测试要求。从软件工程角度看，测试模式可以帮助开发者快速验证状态转换、计时逻辑及完成反馈是否正常。

9.3.2 Flappy Bird 游戏

课件要求根据参考图片生成 Flappy Bird 游戏。

与学习计时器不同，该任务没有提供完整的文字规格，因此需要从参考图像和游戏规则中补充：游戏目标。玩家操作方式。障碍物与碰撞规则。得分机制。游戏结束条件。

这一案例说明，AI 辅助开发并不能消除需求分析的重要性。即使模型能够生成代码，仍需要明确预期行为与验收标准。

9.4 应用复杂度与开发策略

不同应用的实现难度存在显著差异，应优先从简单项目开始。

简单应用通常具有明确的规则、有限的状态以及较少的外部依赖。

复杂应用则可能涉及：网络通信与服务器。多用户状态同步。实时模型调用。响应延迟。API 成本。异常处理与系统可靠性。

因此，合理的开发策略是先完成最小可行产品（Minimum Viable Product, MVP），再逐步加入复杂功能。

Part 10 Take-home Points and Exercises

10.1 本讲核心知识总结

课件最后将整讲内容归纳为四个方面。

第一，借鉴人类学习策略提高智能体性能。

课程学习（Curriculum Learning）与奖励塑形（Reward Shaping）等策略可以改善智能体的训练效果。

第二，将预训练知识与外部搜索结合。

LLM 智能体能够利用已有知识，并通过搜索与研究工具获取当前、小众或复杂的信息。

第三，通过思维链与问题分解处理复杂任务。

将复杂问题转化为中间步骤，并在必要时结合程序计算与外部工具，提高任务处理的可验证性。

第四，利用提示工程完成实际应用。

通过明确目标、输入、输出和约束，AI 可以辅助完成图像生成、代码编写及简单应用开发。

从智能体能力提升到实际应用构建的整体逻辑：

$\text{学习与优化} \rightarrow \text{知识与工具增强} \rightarrow \text{推理与任务分解} \rightarrow \text{多模态生成与应用开发} $

10.2 课堂练习

练习一：DQN 的贪婪动作选择

问题：采用贪婪策略时，DQN 选择什么动作？

A. Estimated Q-value 最大的动作

B. Policy Sampling Probability 最大的动作

C. 上一次即时奖励最大的动作

正确答案：A。

DQN 根据估计的动作价值选择动作： $ a^*=\arg\max_a Q(s,a)$

练习二：应用提示的三个组成部分

问题：哪个提示同时包含应用目标、用户输入与输出？

A. 输入账单和人数，显示每个人应支付的金额。

B. 创建一个设计美观且实用的学生应用。

C. 创建一个使用最先进的 AI 的账单应用。

正确答案：A。同时明确了费用分摊目标、账单与人数输入，以及每人应付金额的输出。

练习三：模型推理详细但计算错误

问题：AI 给出了详细推理过程，但最终总额错误，应如何处理？

A. 因为推理详细而接受答案。

B. 检查单位并使用代码重新计算。

C. 要求模型提供更长的解释。

正确答案：B。详细推理不等于正确计算。对于可形式化的数值任务，应通过代码执行与结果复核提高可靠性。

建立一套完整的方法体系：通过推理与分解理解任务，通过代码与工具执行任务，通过多模态技术扩展输入输出形式，最终将这些能力组合为可验证、可交互的 AI 应用。

5583 Agentic Computing 2

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_6/knowledge/5583-Agentic-Computing-2.md`

5583 Agentic Computing2

Lecture 5 Prompt Engineering II

Part 1 Brainstorming

1.1 Roadmap

Brainstorming 上下文 + 选项 + 迭代

Context Management 上下文管理：上下文窗口里到底装了什么、什么是相关的、什么是干扰的。

In-Context Learning 上下文学习：不改参数，只靠提示里的示例改变模型行为。

Reasoning & Sycophancy 推理与谄媚：模型怎么想多步问题，以及它为什么会顺着你的话讲。

Writing & Revision 写作与修订：拆解 AI 味的来源，用大纲、逐段编辑、评分标尺来改进产出。

Tutorial: Prompt Engineering Practice 提示工程实践。

1.2 Modern Agent

Recap: from classical RL agents to modern LLM agents

从经典强化学习智能体过渡到以语言模型为核心的智能体。

Recap: several key concepts in prompt engineering

复习提示工程的几个关键概念，为第二部分的深入做铺垫。

Recap: GPT with next-token prediction

下一词预测是后面为什么采样能带来多样性的机制前提。

1.3 How people use GPT

AI 最常被用来做的是写和给建议，而创意发散只占很小一块分，这是一项被低估、但值得专门练的能力。

Writing：Editing text / Personal communication / Translation / Summarization / Writing fiction。

Practical Guidance：Learning / How-to advice / Health, fitness, beauty / Creative ideation。

其余并列大类：Finding info. / Technical help / Multimedia / Self-expression / Other。

1.4 Brainstorming with an AI agent

三个可复用动作

1. 上下文具体化：Providing more context 提供更多上下文，把可量化条件一次给全。
2. 限定输出形态：Give 3 plans 明确要三个方案而不是一段建议。
3. 反馈式迭代：Iterating multiple rounds（多轮迭代）I can’t stick to these 把问题说成我做不到而不是你的方案不好

$p(x_{t+1}|x_{1:t}, W)$

From next-token prediction $p(x_{t+1}|x_{1:t}, W)$ to varied responses through sampling。

W 是模型参数（权重），$x_{1:t}$是已经生成的前缀，x_{t+1}是下一个 token。

模型输出的不是一个词，而是一个概率分布；从分布里采样（sampling）而不是取最大，就会得到不同结果。

Sampling can produce different responses 同一句提示问两遍，答案可能不同，原因就在这里。

注意 W 在这句里是固定不变的：多样性来自采样过程，不是来自模型更新。

1.5 Recap: GPT with next-token prediction

Given the previous tokens, the model tries to predict the next token。

训练目标是自回归（autoregressive）：给定前 t 个 token，最大化第 t+1 个 token 的似然。所以模型的一切行为，顺着你说、写 AI 味的句子、给出常见答案，都源于预测下一个最可能的 token这一件事。

理解了这个目标，才能理解为什么 prompt 有这么大影响（改变的是条件分布）。

1.6 How to encourage creativity?

给出上下文（决定在哪个区域采样）和多个选项（决定采几个点）

1. 只给上下文、不要多个选项。模型给一个看起来很合理的方案，没有比较的余地；
2. 只要多个选项、不给上下文。模型可能给的是分布中心的三个变体。

1.7 Brainstorming is iterative

三轮对话的节奏：发散 → 收敛 → 落地

第一轮给约束（4 人 / 10 分钟 / 下周五），产出 3 个结构不同的方案；

第二轮给反馈（不能线下见面 / 有人紧张 / 只有 20 分钟线上），产出 3 个针对新约束的方案；

第三轮做选择（Plan 4 sounds workable）并要求展开到日程级别。

注意第三轮的关键动作：先明确说保留什么（Keep the shared outline and short rehearsal），再要求细化，这样模型不会在细化过程中把前两轮谈定的东西改掉。

1.8 头脑风暴流水线　Brainstorming recipe

把前三节收成一条流水线，并给出AI 当思考伙伴的完整定义。

Context（上下文） 说清楚约束与现状；

Options（选项）一次要多个，不要一个；

Iteration（迭代）对选项给反馈，让模型在新约束下重出；

High-quality ideas（高质量想法）是这条链路末端的产物，不是第一步就能直接要到的。

Part 2 Context Management

2.1 上下文 Context

Definition：All the text and files the model uses to generate its response 模型用来生成回答的全部文本与文件。

没有上下文，即通用模型。问 Pros and cons of studying physics vs. zoology，若模型一无所知，只能给一页通用的利弊清单，它不知道你的成绩、时间表、职业倾向。但补上上下文即可得到定制回答，应用于Career assessment results、High school schedule、Course choice。

2.1.1 AI 的上下文 AI model’s context

AI context 系统提示在底部，对话历史往上叠，窗口自下而上包含：

组成部分	原文	说明
System prompt	Current date, model name and capabilities, general instructions（restrictions）	当前日期、模型名与能力、通用指令
Tool definitions	Available tools (like web search) and how to use them	可用工具（如联网搜索）及其用法
User prompt	Build workout plan for: 38 yo, beginner, dumbbells, 15 mins/day	
AI response	3 options: full-body, upper - lower split, low-impact strength	

Tips: The conversation is part of the context 对话窗口本身就是上下文的一部分

为什么同一句话在不同对话里效果不同？ User prompt 是叠加在 Chat history 之上被处理的，进入模型的完整输入不同，输出自然不同。

相关性：这条信息是否会影响本次任务的目标输出。

1. 反例：Irrelevant context can become distracting and may lead the AI to generate a worse answer 不相关的上下文会成为干扰，可能让 AI 给出更差的答案。
2. 解决：Start a new chat to reduce influence from the current conversation. 开新对话之后，Chat history 被清空，只剩 System prompt + Tool definitions + 本轮提问。想要一个不受前文影响的判断，就换一个干净上下文

2.2 提示 Prompt

2.2.1 用户提示 vs. 系统提示 User prompts vs. system prompts

角色	说明
system	A system prompt sets persistent and high-level instructions（系统提示设定的是持久、高层的指令，位置在最前面、对整段对话持续生效） 规定怎么回答：任务范围、输出格式、语言、风格、限制。
user	the user’s message（用户的消息） 规定回答什么：本次的具体输入。

2.2.2 聊天提示 Chat Prompts

The series of messages is turned into a string that uses special tags.

多条消息会被转成一个带特殊标记的字符串。

Tips：模型看到的始终是一段连续文本，system / user / assistant 的分界靠特殊标记（special tokens）表示。

2.3 系统提示 System Prompt

The intended answer follows the system instruction。系统提示不是建议，而是条件化输出分布的那个条件

Example：Anthropic Claude

1. 行为规范型

System prompts can contain many different rules describing how the model should behave. Such as Knowledge cutoff（知识截止时间）、Helps with prompting techniques（协助提示技巧）、Step-by-step reasoning（逐步推理）.

2. 安全与风格型

The system prompt can also be used to guide the model toward safer behavior and help it avoid harmful or inappropriate responses. Such as Avoid harmful requests（拒绝有害请求）、AI “self-esteem”（模型的自我评价/语气倾向）.

https://console.groq.com/playground 一个可以直接改系统提示、立刻看输出变化的在线试验台。

Example: basic English and proficient English

对照实验：同一个问题，只改系统提示里对语言水平的要求。用基础英语与熟练英语要求作为系统提示，对比同一请求得到的回答。

2.3.1 Functions of System Prompt

1. Can help to control the model’s behavior 帮助控制模型行为；
2. Specify the set of tasks 指定任务集合；
3. Desired properties of model responses 规定回答应有的属性（如语言）；
4. Output formatting 输出格式；
5. Condition the model’s output distribution 对模型的输出分布施加条件；

2.3.2 Downsides of System Prompt

Question：If two prompts mean exactly the same thing but differ slightly in formatting (e.g., spacing), should the model perform similarly?

两句语义完全相同、只有格式（如空格）不同的提示，模型的表现在理想情况下应该一样?

Answer：Prompt formatting matters in seemingly arbitrary ways

实验：用同一条提示的六种写法做对照，横轴是 Task Accuracy（任务准确率，0 到 1）。语义相同的几种格式，准确率从 0.036 跨到 0.804。

现象：刻度两端标注了极值：最低 0.036、最高 0.804，中间那段横条被标注为 Performance Spread Among Plausible Formats（各种看起来都合理的格式之间的性能跨度）。

结论：Even tiny irrelevant formatting changes can cause large performance differences 即使极微小、看似无关的格式变化，也能造成很大的性能差异。

1. No universal “best” format : format sensitivity depends on the model, task and prompt. 不存在通用的最佳格式：格式敏感性取决于模型、任务和提示本身。
2. For serious systems, test robustness and use structured interfaces：对正式系统而言，要测鲁棒性，并使用结构化接口（而不是把关键信息塞进自由文本的格式里赌稳定性）。

Part 3 In-Context Learning

3.1 上下文学习 In-context learning

An LLM exhibits a capability using the text supplied in its input context, without updating its parameters. 不改参数，只靠输入里的文本，模型就表现出新能力。

For complex tasks, ICL provides useful behavioral specifications!

对复杂任务而言，上下文学习提供了一种行为规格说明书。

Advantages：（cost）

Easy: use task examples without collecting a large task-specific training set.

容易：不必为一个任务收集大规模专用训练集；

No extra training: use one model for several tasks without extra training.

无需额外训练：一个模型跑多个任务；

Deployable: reuse one model across applications.

易部署：一个模型复用到多个应用。

3.1.1 Strategy 1: Text completion 文本补全（不给指令）

After pre-training, we have an autoregressive language model.

预训练之后，我们得到一个自回归语言模型

The language model can complete text: — 语言模型可以补全文本。公式写作 $x_{1:t} \to x_{t+1:T}$，即用前 t 个 token 生成后面的 token。

Prompt: "When a dog sees a squirrel, it will usually"

Output: "bark a lot, especially at close range.", "start to chase after it. When it does⋯"

3.1.2 Strategy 2: Zero-shot 指令提示

We can prompt the model to perform a specific task by providing instruction. Called “zero-shot” because we don’t provide (x, y) examples in the prompt.

通过给出指令，可以提示模型执行某个具体任务。之所以叫 zero-shot，是因为提示里没有提供任何 (x, y) 示例。公式 f(x; instruction) 里的两个部分：x 是输入、instruction 是指令，二者一起决定输出。

Considerations（可靠性 / 格式敏感性问题）

1. 输出格式不好对付 The model’s generated output formatting can be varied and tricky to deal with.
2. 表现随措辞变化 Performance can vary based on the prompt wording.

3. 基础模型可能不会听指令或只支持特殊格式 Base models may not be trained for instruction following, or trained only to support specific formats

3.1.3 Strategy 3: Few-shot （Instruction + examples）

Additionally provide (x, y) examples in the prompt 在提示里额外提供 (x, y) 示例。

公式中 K 是示例个数，x_test 是待预测的新输入，y_test 是输出。

Examples help the model understand the task format and expected outputs without any parameter updates. Referred to as “in-context learning”示例帮助模型理解任务格式与期望输出，且不涉及任何参数更新——这就叫上下文学习。它传递的不是知识，而是行为要求（格式、风格、推理颗粒度）。

Example: MATH dataset: a few-shot prompt used in Llemma

用 Llemma 模型在 MATH 数学题数据集上的 few-shot 提示原文。

Conclusion: ICL 擅长教行为（如用比喻解释），而不擅长塞知识。

3.1.4 Extension: Many-Shot In-Context Learning

变体	原文	说明了什么
ICL	input, reference output	标准做法：每个示例都带人写的或标注的参考答案
Unsupervised ICL	inputs + format guidance	只给输入和格式说明，不给参考答案
Reinforced ICL	model-generated outputs filtered for correctness	用模型自己生成的输出，过滤掉不正确的再当示例

In some experiments, inputs with format guidance outperform full demonstrations. 在某些实验里，输入 + 格式说明 的效果优于完整的示例。

原因：示例的两个作用要分开看，定位任务（靠输入+格式就能做到）与示范风格/颗粒度（往往需要完整示例）。如果提示效果不好，可以试着拆开：先给格式说明看够不够，不够再加完整示例。

3.2 示例数量与模型的影响 Effects of sample numbers and models

把示例数量从 2^0 一路加到 2^{10}，看四个模型的翻译准确率怎么变。

图例模型	Bemba 方向的大致走势	Kurdish 方向的大致走势
Gemini 1.5 Pro	整体最高，从 ~32% 一路上升到 ~48%	起始 ~39%，缓慢升到 ~43%
Claude-3-Opus	起始 ~35.5%，几乎持平，末端 ~39%	起始 ~38%，中间略降到 ~37.5%，末端回到 ~39%
GPT-4-Turbo	起始 ~26%，升到 ~32%	全程最低，约 ~31% 上下
Gemini 1.5 Flash	起始 ~24%，升幅最大，末端 ~40%	起始 ~39%，升到 ~42%

Ability to use many examples varies by model

能用多少示例这件事本身因模型而异。示例越多越好并不成立，收益取决于模型是否有能力把这些示例用起来。

3.3 不同任务的表现不同 Effects of different tasks

同一批模型、同一套示例数量，换任务之后曲线形状会变。不存在通用的最佳示例数量

Different tasks do not all follow the same pattern. 不同任务并不都遵循同一种规律。

3.4 上下文学习的建议 Tips for in-context learning

1. What ICL is good for

“Illustrate your explanations with examples, thought experiments, or metaphors.” and “Avoid being heavy-handed or repetitive.”

要求用例子、思想实验或比喻来讲解；避免说教或重复。

共同点：这两条要求的都是风格与做法（behavior），不是事实与知识（knowledge）。

2. What to be careful about

Knowledge tokens leave less room for conversation. 塞知识会挤占对话空间；

Long contexts make inference expensive. 长上下文让推理更贵；

New information must fit in the context window. 新信息必须塞得进上下文窗口。

Part 4 推理 Reasoning & Sycophancy

AI reasoning: using context and intermediate computation to solve multi-step problems. 推理即使用上下文与中间计算来解决多步问题。

4.1 长思考示例 Long thinking example

Example：购车比价场景，说明推理在智能体里指的是什么。

输入材料：Insurance plans（保险方案）、Spec sheets（规格表）、Car loan quotes（车贷报价）。

用户请求：What are the tradeoffs for each car? Read everything and **think hard before answering（显式要求先思考再回答）**

模型思考时做的四件事：Reading documentation / Searching for customer reviews / Defining evaluation criteria / Generating report。

最终输出：Here’s a detailed review of the tradeoffs among these three cars…

4.2 推理工作流程 How reasoning works

Example：行程规划，推理是反复循环的回路。

The reasoning process may involve repeatedly gathering information and thinking about it. 推理过程可能涉及反复地收集信息并加以思考。

4.3 AI长任务发展 AI progress on long-running tasks has exploded

4.4 鼓励推理 Encouraging reasoning

1. Use reasoning controls 使用推理控制
2. Give the model hard tasks 给模型难题。简单问题上，模型不需要长推理就能给出分布中心的答案

Example：Design a 12-month plan for a 4-person startup with limited cash为一家现金有限的 4 人创业公司设计 12 个月计划。

给模型的两样东西：Real job tasks（真实工作任务）与 All the context someone would need to complete the task（完成该任务所需的全部上下文）；

在旁标注提示语：Check your reasoning.（检查你的推理）。

4.5 AI 推理的四条建议 Tips for reasoning with AI

1. use the best models available 前提（模型能力）：使用可得的最强模型；
2. Give them as much context as needed 开关（推理预算）：给足所需的上下文；
3. Give them hard tasks 给难题；
4. Use reasoning settings when available 有推理设置就用。

4.6 AI 过度同意 Sycophancy

Example：

问法一：Don’t you think remote work is better than office work?

回答一：Yes, remote work offers many advantages…

问法二：Is it true that office work is more productive?

回答二：Yes, office environments provide strong benefits…

两次提问指向互相对立的结论，而模型两次都同意。

How you phrase the question may influence the agent’s response. 如何措辞提问，会影响智能体的回答。

两个问法都有共同的诱导特征：Don’t you think…（你不觉得……吗）与 Is it true that…（……是真的吗）都是预设了答案的引导式提问。

4.6.1 产生原因 Why does sycophancy happen?

Models are trained to be helpful assistants using human feedback, which can reinforce sycophancy 模型通过人类反馈被训练成有用的助手，而这可能强化谄媚

问题：How can model responses better match human preferences? 模型回答怎样才能更好地匹配人类偏好?

答案：Learn from human feedback (RLHF) 从人类反馈中学习（RLHF）。

逻辑链：人类偏好 → 标注者倾向给顺着我的回答高分 → RLHF 放大这个倾向 → 模型学会先同意。

4.6.2 难以察觉的陷阱 Sycophancy feels helpful but degrades quality

Example：

1. 明显

用户：I’m really proud of this essay, what do you think?

模型：Your essay is excellent! You clearly put in a lot of effort…

2. 难以察觉

用户：Analyze this data and find all the positive measures of performance this quarter.

模型：The data clearly shows revenue growth, strong retention, improving margins…

What bias does this prompt introduce?（这个提示引入了什么偏差？）

第一个例子里，用户的倾向写在情绪里；第二个例子里，倾向藏在任务定义里 find all the positive measures。模型在第二种情况下并没有违反指令，所以不会触发任何警报；但分析结论会系统性地偏乐观。

4.6.3 解决方案 Combating sycophancy

课件给出三条并列动作。

1. Neutral framing 中性框架。Avoid giving hints as to the answer you want to hear

换度量词：bad / good → to what extent（把是非题改成程度题）；

换来源：Do you agree → What does current research say（把立场题改成证据题）；

换动词：reduce / improve → compare（把方向题改成对比题）。

必要时再加 if at all / if any 这类让步短语。

引导式提问　Leading prompt	中性提问　Neutral prompt
Aren’t carbon taxes bad for small businesses?	To what extent, if at all, do carbon taxes affect small businesses?
Do you agree that AI will create a lot of jobs?	What does current research say about AI’s effect on jobs?
Doesn’t remote work reduce worker productivity?	How does productivity compare between remote and in-office work?

2. Explicitly ask for objective critique 明确要求客观批评。

Example：Be critical. Assume this needs improvement 它一定还有改进空间。

3. Start a new chat for a fresh opinion 开新对话以获得不受影响的意见。

Part 5 Writing & Revision

5.1 AI 味产生原因 What makes AI slop?

AI slop：AI content that looks good, but lacks substance (Vague, empty-sounding text) 看起来不错、但没有实质内容的 AI 产出。

AI 味 的两个可观测特征：破折号的使用频率，以及句子结构的固定套路。

四类可辨认的语言模式：

模式	例句	特征
Overused words（过度用词）	“This is a nuanced issue. Let’s delve into the implications…”	偏爱 nuanced、delve 这类词
Lists of three（排比）	“Clear, concise, and compelling”	强行凑三个并列形容词/短语

模式	例句	特征
Vague praise（空洞的赞美）	“This is a robustly structured and highly insightful paper…”	用高级形容词但不说具体好在哪
Not x but y（强行转折）	“It’s not just about speed—it’s about availability”	用一个否定-转折句式制造虚假深度

智能体可能影响人类 AI agent may influence human speech behaviours

例如，用 delve 这个词在播客与演讲里的使用频率，展示 AI 语言向人类口语的渗透。

5.2 去 AI 味

5.2.1 技巧一：用渐进式大纲写作 Write via progressive outlining

研究 → 3 个大纲 → 选一个并调整结构 → 展开成要点 → 局部补充 → 生成全文

1. 起点：给研究任务，不要直接要文章

不要一上来就让它做完整深度研究然后出稿。用户的原话是：

I’m writing an article about small AI teams moving faster than large teams that don’t use AI. Research **evidence for and against this hypothesis**.

2. 要三个结构不同的选项

Create 3 outline options. Include a counterarguments section. Also, here are some stories from teams I work with.

3. 选定后改结构，不动内容

Let’s use Option 1 and keep all the stories, but move the thesis after Story 1.,. I also want to add a historical analogy: Pixar created Toy Story in the 90s.

4. 大纲确认后才要正文

Expand each heading into bullet points based on the research.

5. 继续局部修改，最后才生成全文

Explain more about the startup. Tie in the Pixar story with the thesis.

Now, generate the text for this article.

5.2.2 技巧二：逐段编辑 Editing piece by piece

把改一篇文章拆成反复改一句/一段。

1. 先要几种不同的说法

Everyone talks about “human-level AI,” but what does that actually mean? Smarter chatbots? Automated office work?，用户要求 Brainstorm a few different ways to say this:

风格	改写
Punchy（有力/短促）	（课件在本列展示的是同一段的另一种紧凑写法）

风格	改写
Visionary（有远见式）	The public thinks achieving AGI means computers will be as smart as people. For decades, we’ve imagined AGI as machines that can reason and create like humans do.
Conversational（口语化）	A lot of people hear “AGI” and assume it means computers as smart as humans that can do most desk jobs.

2. 逐段推进，而不是整篇重写

5.2.3 技巧三：评分标尺 Well-written rubrics

把客观一点这个模糊要求，换成一个有分项、有分值的评分表。

1. Specify important criteria（明确重要标准）+ 2. Develop instructions for evaluating criteria（为每条标准写评价细则）+ Point system（分数体系）。

评价维度	分值	课件给出的评分细则示例
Characters	25 points	Every named character has a goal (10 points) / Conflict between two characters’ goals (10 points) / At least one character makes an irreversible decision (5 points)
Plot	25 points	（同上结构，细则按剧情维度展开）
World building	25 points	（同上结构）
Writing craft	25 points	（同上结构）

反面对照：同样有标尺，但写法不对，分数会被抬高。

1. Scoring early can anchor judgment 先给总分，后面的分项就会被调去凑这个总分
2. Vague categories invite subjectivity 没有细则，模型只能靠语感，而语感会偏向夸

对比项	5.9 节（可用）	5.10 节（会抬分）
分项细则	有（每条标准可判定、带小分值）	无（只有维度名）
给总分的位置	最后 sum the scores at the end（先分项、后求和）	最前 score it out of 100（先定总分）
自我指涉	无	story I wrote（暗示作者身份）
结果	65 / 100，指出人物目标无冲突	分数被抬高

5.2.4 技巧四：评分标尺 Cross-model review

换一个模型来评，避免自己评自己。

同样带标尺 + 分项打分 + 求总分，差别在于打分者不是写作者，切断自我一致性。

不同模型有不同的训练数据与偏好，能暴露出某一个模型特有的宽容，这正是一种鲁棒性测试。

Part 6 Summary

6.1 Summary: instruction prompts and in-context examples

提示的三种基本型态：

6.2 In-class exercises

第 1 题　中性框架

题干：You want to compare productivity in remote and office work. The prompt: Don't you agree remote work is better? invites agreement. Which replacement best reduces this leading framing?

- A. Explain why remote work improves productivity more than office work.
- B. Be critical: explain why office work is more productive than remote work.
- C. Compare productivity in both settings using evidence and its limitations.

第 2 题　上下文管理

题干：Your workout chat contains 15 minutes/day, dumbbells, no squats. You now want a plan for your mother, who has 30 minutes/day and no equipment. Which request manages context most clearly?

- A. In the same chat, ask only to make my plan longer for my mother.
- B. Copy the entire old chat and ask Adapt this for my mother.
- C. Start a new chat with her available time, equipment and preferences.

第 3 题　上下文学习的机制

题干：A model follows 4 examples, then loses the pattern in a fresh chat without them. Best explanation?

- A. The new chat reset the newly trained weights.
- B. The rule was stored but hidden in the new chat.
- C. The examples guided inference without training.

第 4 题　评分标尺

题干：AI gives your essay 92/100, then fits vague category scores to that total. Best prompt revision?

- A. Define criteria, score with evidence, then total.
- B. Keep the total and explain each category in detail.
- C. Ask for a stricter overall score before categories.

Scoring early can anchor judgment（先给总分成为锚点）与 Vague categories invite subjectivity（含糊分类引入主观性）。应该先定义标准、每项带证据打分、最后才求和。

Lecture 6: Retrieval-Augmented Generation (RAG)

Part 0 Orientation

0.1 Roadmap

本讲围绕一个具体的工程问题展开：当大语言模型自身掌握的信息不足以回答问题时，如何在生成答案之前，从外部知识源中检索出相关证据，并利用这些证据提高回答质量？将这个问题拆成五个部分。

Part	英文标题	中文主题	主要问题
I	Why RAG?	为什么需要 RAG	为什么不能只依赖模型参数中的知识?
II	RAG Architecture	RAG 架构	检索器、知识库与生成模型如何协作?
III	Typical Retrieval Methods	典型检索方法	如何从大量文档中找出相关信息?
IV	Retrieval Evaluation	检索评估	如何判断检索器是否找对了内容?
V	Tutorial: RAG in practice	RAG 实践	如何实现一个可运行的检索增强问答系统?

上一讲讨论如何通过 Prompt、Context Management、Reasoning 等方法控制模型的行为。本讲进一步追问：如果模型需要的上下文本身不存在于当前提示中，那么这些上下文从哪里来?

这就是 RAG 需要解决的问题。

两讲的关系可以概括为：

- Prompt Engineering：如何组织和使用已经拥有的信息。
- RAG：如何根据问题找到需要的信息，再将其组织进提示。
- Generation：模型如何利用提供的信息进行推理和回答。

需要注意，RAG 不等于 Prompt Engineering，但 RAG 通常会把检索结果放入提示，因此两者在实际系统中密切相关。

Part 1 Why RAG?

1.1 Three Ways of LLM Obtaining Information

How Does an LLM Obtain Information? LLM 获取信息的三种方式。

维度	参数知识	上下文信息	网页信息
来源	训练数据	当前输入	外部网络
是否依赖实时获取	否	不一定	通常需要
是否可以使用私有知识	取决于训练情况	可以	取决于授权与访问
主要限制	可能过时、不完整	上下文容量有限	质量、可靠性和安全性不一
是否必然属于 RAG	否	否	否

有外部信息，不一定就是 RAG。如果用户手动复制一段文章给模型，这属于上下文提供。**如果系统根据用户问题自动搜索知识库、选择相关片段，再将片段提供给模型，这才符合典型的 RAG 工作方式。**

1. **Stored in parameters** 数据经过训练过程，使模型获得大量通用知识。

Training Data（E-books, Books, Web pages, Articles） → Training → LLM

例如：为什么酒店周末更贵？模型可以根据已经学习到的供需关系，给出解释。但模型参数中的知识有一个重要限制：知识不会因为现实世界发生变化就自动更新。

知识截止时间 Knowledge cutoff 不意味着模型一定不知道此后的所有事实，也不意味着截止时间之前的事实都掌握得准确。它表示模型的预训练知识不能保证覆盖之后的新信息。

2. **Provided in context **模型除了使用已有知识，还可以利用当前输入中提供的信息

Chat history / Your documents / Notes → LLM

例如，用户给模型一段课程说明：Week 6 covers Retrieval-Augmented Generation. 然后询问：What is covered in Week 6? 即使模型原本不知道这门课程，课程内容出现在当前上下文中，也可以根据提供的文本回答。

3. **Found on the web** 通过网络获取

Search → Web pages → LLM

例如：What happened in Hong Kong last weekend? 模型可以通过外部搜索系统获得相关网页内容，再利用这些内容作答。但需要强调：

LLM 本身不一定具备联网能力。网页搜索通常由模型外部的工具或系统完成。

1.2 The Core Problem and Idea

Tips：为什么不能把所有外部知识都塞进上下文?

The problem with large external sources. Too much to put into context.

当知识源很大时，无法简单地把全部内容都放入模型输入。即使模型支持较长的上下文，也不代表把全部数据塞进去就是合理方案。主要原因包括：

1. 上下文窗口存在容量限制。
2. 输入越长，通常消耗越多计算资源与费用。
3. 大量无关内容可能干扰模型识别关键证据。
4. 每次重复发送完整知识库会产生额外开销。
5. 知识持续更新时，维护完整上下文十分困难。

解决方案：Retrieval-Augmented Generation (RAG).

核心流程：Large knowledge source → Retriever → Relevant information → LLM

不把所有知识交给模型，而是先找到与当前问题最相关的部分。LLM 可以从参数、上下文和外部工具获得信息。当外部知识规模很大时，RAG 通过检索选择相关内容，使模型能够在有限上下文中使用更有针对性的证据。

1.3 General Knowledge Answers

问题一：

Why are hotels more expensive on weekends? 为什么酒店在周末更贵?

More people travel, so there is more competition for rooms. 更多人出行，因此酒店房间的竞争更激烈。

可以直接回答，因为主要询问一般性的经济规律，模型在预训练过程中接触过大量类似知识，因此通常能够给出合理解释。其逻辑为：周末出行需求增加 → 酒店房间供给相对有限 → 供需关系变化 → 房价可能上涨。

如果一个问题能够依靠模型已有知识可靠回答，额外检索未必带来收益。

问题二：

What is Python for loop? 通常不需要专门搜索网页。

How does our company's internal Python framework implement task retries? 就可能需要检索公司代码库。

这类问题通常不需要外部检索，但模型仍可能进行一定程度的内部推理。

检索与推理不是同一个概念。

- Retrieval：寻找所需事实或证据。
- Reasoning：根据已有事实进行分析和推导。

是否需要 RAG，首先取决于回答问题是否需要模型当前尚未掌握的外部信息。一个问题可能不需要检索，但需要复杂推理；也可能需要检索，却只需要简单地提取事实。

1.4 Recent Information and Thinking Answers

Why were HK hotels so expensive last weekend? 为什么香港酒店上周末那么贵?

需要知道特定时间、特定地点发生了什么。这涉及两个不同的问题。

问题一：Knowledge Gap（知识缺口）模型没有掌握相关事实。

问题二：Reasoning Gap（推理缺口）模型掌握了事实，但不能正确推导结论。

RAG 主要针对第一个问题，通过外部检索补充事实。但 RAG 并不能保证第二个问题自动得到解决。

推理不能代替事实，如果模型没有核实就给出具体事件名称，就可能产生没有证据支持的回答。因此，合理的处理方式是先检索近期信息，再分析可能的原因。

时间一致性问题

还要检查 last weekend 对应的实际日期。如果问题发生在黄金周之后较长的时间，不能自动把黄金周作为那个周末的原因。RAG 系统不仅需要找到相关关键词，还需要验证检索结果的时间范围是否与问题一致。这属于工程中的 Temporal Relevance（时间相关性）。

总结

General knowledge gives us a starting point, but it is not enough to explain particular events.

通用知识为解释提供起点，但不足以解释具体事件。

Retrieval provides time-specific and more relevant evidence.

检索能够提供具有时间针对性、更加相关的证据。

1.5 RAG 的两个关键步骤

Two Critical Steps in RAG

1. Retrieval：Collecting useful information.

检索：收集有用信息。

2. Generation：Reasoning over that information and responding.

生成：基于这些信息进行推理并作答。

第一类问题：不需要专门收集外部信息，模型可直接依据已有知识回答。

第二类问题：需要先调查特定事件，再综合证据生成回答。

Retrieval 主要解决：应该把哪些信息交给模型?

Generation 主要解决：已经拿到这些信息，应该如何回答?

这是 RAG 系统最重要的职责划分。

假设用户询问：Week 6 的 tutorial 在哪间教室?

可能出现两种失败：检索失败，知识库中存在教室信息，但检索器没有找到；生成失败，检索器已经找到正确教室信息，但模型在回答时使用了错误的教室号。两种失败表面上都表现为回答错误，但修复方式完全不同。前者需要改善检索。后者需要改善提示、生成模型或答案验证。

1.6 Key Idea in RAG

LLMs store extensive information → LLMs can also benefit from an efficient retrieval phase.

LLM 已经存储了大量信息，但仍然能够从高效检索中进一步受益。RAG 通过高效检索弥补模型在近期信息和专业知识上的不足，使已有知识与外部证据相互补充。

两类需要外部补充的信息：Very recent information. Specialized knowledge.

1. Very recent information：近期信息

例如：最新发布的软件版本；今天的产品库存；最近发生的事件；最新的公司政策。

这些信息可能在模型训练之后才出现。因此，即使模型具有很强的语言理解能力，也不意味着它知道这些新事实。

2. Specialized knowledge：专业知识

例如：企业内部代码结构；某个医学研究领域的最新文献；特定项目的技术规范；专业法律文档。

这些知识可能很少出现在通用训练数据中，也可能受到访问限制。RAG 可以让模型在回答相关问题时使用经过授权的专业材料。

Great answer OR Not so good：模型在拥有足够相关知识时，可能给出较好的回答；面对近期或专业知识缺口时，如果没有适当的外部证据，回答质量可能下降。

OR 表示知识库是可插拔的：回答酒店问题需要网页，回答中国菜谱需要菜谱库，回答课程问题需要课程文档。工程含义是RAG 系统的检索器要对着一个特定的知识库来配。换库就要重建索引。

RAG 不是替代 LLM：LLM 已有大量知识，但在需要的时候可以通过检索获得额外帮助。因此，RAG 应被理解为增强机制，而不是完全替代模型原有能力。

1.7 检索器的工作流 The Retriever Workflow

Step 1: Search for a knowledge base of trusted information 在可信知识库中搜索信息。

knowledge base 知识库是用于保存和组织可检索信息的外部数据集合。它可以包括：课程文档、企业内部手册、项目源代码、产品资料、专业文献。

Retriever 负责接收查询、选择相关内容，查询与检索结果

Knowledge Base 负责存储可供检索的外部知识。检索器位于知识库上方，通过双向箭头表示查询与结果返回。

Trusted information 可信信息，检索系统不能只考虑内容是否相关，还要考虑信息来源是否适合被使用。但可信来源也不保证其中每一条内容都准确、完整或及时。

Step 2: Finds the most relevant information and shares it with the LLM 找到最相关的信息，并提供给 LLM。

第一步：寻找；第二步：传递。

Step 3: Improves generation 改善生成结果。

这是检索器存在的最终目的，但它描述的是预期收益，不是绝对保证。如果检索结果不相关、过时或错误，也可能降低生成质量。

Relevant 不等于 Correct

Retriever 的职责是从知识库中找出相关信息，但相关性、正确性和是否足以回答问题需要进一步区分。

Relevant：相关。表示某段文本与问题在词语或语义上有联系，相关性不等于可回答性，检索到不等于已经核实。

Correct：正确。表示信息本身准确，且能够支持所需结论。

1.8 RAG Definition

RAG combines a pretrained LLM with external knowledge retrieved at inference time to support generation.

RAG 将预训练语言模型与推理时检索到的外部知识结合起来，以支持答案生成。

RAG 的关键特征是在推理阶段检索外部知识，并利用检索结果支持生成，而不是要求模型事先记住全部知识。

Pretrained LLM：预训练语言模型

表示系统已经拥有一个经过训练的模型。它可以具备：语言理解能力、通用知识、文本生成能力、一定的推理能力。RAG 通常不需要从头训练一个新的语言模型。

External knowledge：外部知识

课件明确列举: Local documents, knowledge base, web search, APIs ... 即本地文档、知识库、网页搜索和 API 等。

这些来源有一个共同点：信息不必全部编码在语言模型参数中。

Retrieved at inference time：推理时检索

Inference time 指模型实际处理用户请求、生成回答的阶段。在典型 RAG 中，系统会根据当前问题，从外部来源寻找相关信息。这与训练阶段学习知识不同。

To support generation：支持生成

检索结果并不是最终答案。它们是用于支持答案生成的外部上下文。如果问题要求进一步解释或比较，模型还可以在证据基础上组织更复杂的回答。

RAG 与 Fine-tuning 的区别

维度	RAG	Fine-tuning
主要方式	推理时提供外部知识	通过训练更新模型参数
知识更新	更新知识库和索引	通常需要额外训练
适合任务	事实查询、动态知识、引用	行为适配、格式、特定任务能力
是否必然改变 LLM 权重	否	是
能否组合使用	可以	可以

实际系统可以同时使用 RAG 和微调。先微调模型，使其更擅长按照企业规范回答；再通过 RAG 检索最新企业文档。

某些知识广泛存在于公共数据中，而另一些知识很少出现，甚至根本不公开。因此，不能假设通用模型能够可靠掌握所有专业或私有信息。不过，互联网出现频率只是一个说明性维度，不能直接等同于模型对该知识的掌握程度。

1.9 Why RAG? Code Generation Example

RAG is important when we want to adapt an off-the-shelf LLM to a specific application.

当我们希望将现成的 LLM 应用于某个具体场景时，RAG 非常有用。

RAG 能让通用代码模型利用项目专属知识，提高代码生成和代码问答的针对性。

Off-the-shelf LLM

表示现成、可直接使用的模型。例如，一个通用代码模型可能知道：Python 语法、SQL 查询、REST API、常见软件设计模式。但它不一定知道：你的项目有哪些模块、项目采用什么内部约定、哪些函数已经实现、哪些接口刚刚发生变化。

Use your codebase as a knowledge base

把自己的代码库作为知识库。例如，代码库中存在：

```
class DatabaseManager:
def connect_to_db(self):
return self.connection
```

以及：

```
def process_user_data(user_input):
validator = CustomValidator()
return validator.validate(user_input)
```

这些函数体现了项目特定的结构与约定。

如果用户要求：请按照当前项目的规范实现一个新的数据库访问函数。通用模型可能生成语法正确、但不符合项目规范的代码。而 RAG 可以先检索：

- 现有数据库管理类。
- 项目使用的验证器。
- 已有错误处理方式。
- 相似函数的实现。

再把这些内容交给模型。

RAG 如何改善代码生成?

其作用可以分为三层。第一层：识别已有接口，避免模型凭空编造函数名称；第二层：学习项目约定，让生成代码更符合项目现有结构；第三层：利用最新代码，减少依赖已经过时的接口说明。

代码场景的三个典型缺口&为什么代码是最典型的 RAG 场景

缺口	表现	RAG 的作用
私有性	项目代码未公开	检索内部代码库
时效性	代码持续修改	获取较新版本
专业性	存在自定义模块与约定	提供项目特定上下文

公共语料足够丰富，加检索的收益很小。这也是代码库问答非常适合 RAG 的原因。

1. 完全私有（网上没有第二个一样的项目）；
2. 频繁变动（每天都可能改函数签名）；
3. 命名独特（自研模块名、内部约定，模型不可能见过）。

工程延伸：代码检索的特殊性

代码不仅包含自然语言，还包含大量精确符号：函数名称、类名称、文件路径、参数、依赖关系。

因此，代码 RAG 不能只依赖宽泛的语义相似性。

例如：`connect_to_db`与`create_database_connection`语义可能相近，但它们不一定是同一个真实接口。实际工程中通常需要兼顾精确匹配和语义匹配。

1.10 RAG for Companies 企业应用

企业 RAG 的价值不仅在于使用私有知识，还在于根据权限与时效要求，安全地提供相关业务信息。

1. Chatbots：企业客服机器人

Know about your company's products, current inventory, and common troubleshooting steps.

了解公司的产品、当前库存和常见故障排查步骤。

例如：Product A 现在还有库存吗？模型仅凭预训练知识无法可靠知道实时库存。企业 RAG 系统可以查询经过授权的库存数据，再生成回答。

又例如：我们公司的设备出现 Error 103 应该怎么处理？系统可以检索内部故障手册，找到对应的处理步骤。

2. AI Assistants：企业内部助手

Access and use private company data through internal or local services.

通过内部或本地服务访问和使用企业私有数据。

例如：检索内部制度、查询项目文档、总结企业报告、回答员工操作流程问题。

3. 三段式流程：Company knowledge → Retrieve evidence → Answer with context

即：企业知识 → 检索证据 → 结合上下文回答。

这个流程表明：企业知识不需要全部预先存储在模型参数中，系统可以根据用户问题按需获取。

工程延伸：权限控制

企业场景除了相关性，还必须考虑权限。假设企业有两类文档：普通员工可访问的产品说明和仅管理层可访问的财务报告。即使财务报告与用户问题高度相关，也不意味着检索器可以把它交给所有员工。因此，企业 RAG 需要满足：

也就是说，可被用户检索到的文档必须处于该用户的授权范围内。权限过滤应在向模型暴露内容之前执行，而不是仅要求模型在生成时自行保密。

工程延伸：知识更新

current inventory 信息的动态性。如果库存数据库每分钟更新，但 RAG 索引每天才更新一次，回答仍可能过时。因此，需要区分：数据源是否更新、检索索引是否更新、检索结果是否来自最新版本。

1.11 RAG for Web Search

Web-based RAG allows an LLM to retrieve the latest information from the Web and use it as external context when generating an answer.

基于网页的 RAG 允许 LLM 从网络检索最新信息，并将其作为生成答案时的外部上下文。

网页 RAG 让模型能够利用外部网页证据生成回答，并通过来源引用提高可核查性，价值包括时效性、覆盖范围和可追溯性。

Search the web → **Retrieve passages** → Answer with sources

搜索网页 → 检索相关段落 → 附带来源回答。

1.12 RAG for Specialized Knowledge

专业领域 RAG 的核心价值在于按需利用专业资料，并使回答能够接受更严格的证据核查。

1. Healthcare：医疗

Medical journals and authorized patient records. 医学期刊与经过授权的患者记录。

医疗领域具有几个特点：专业术语密集、研究持续更新、患者资料涉及隐私、错误信息可能造成严重后果。因此，医疗 RAG 需要特别重视证据质量和授权范围。

例如：医生希望查找某项临床研究的结论、系统可以检索医学文献，提供相关段落与来源。但不能仅因为系统使用了 RAG，就认为它的结论已经得到医学验证。

2. Legal Research：法律研究

知识来源：Case files and legal reference documents. 案件文件和法律参考文档。

法律知识具有：强烈的司法辖区差异、条文版本变化、具体案件事实依赖、较高的引用与核查要求。

例如：用户询问某条法律规定的适用条件。系统不仅需要找到相关条文，还需要确认其版本、司法辖区和适用时间。

两个领域的共同特征

维度	Healthcare	Legal Research
专业性	高	高
时效要求	医学研究持续更新	法律条文与判例变化
私有数据	患者记录	案件文件
来源要求	医学文献与授权记录	法律文件与权威资料

维度	Healthcare	Legal Research
主要风险	医疗误导、隐私泄露	法律误导、保密风险

工程延伸：高风险场景为什么需要更严格验证？

普通问答系统可能只关注：回答是否看起来合理？

高风险系统还需要检查：检索来源是否可靠、证据是否支持回答、信息是否过时、用户是否有访问权限、是否存在重要遗漏、是否需要专业人员复核。需要强调，具体法律合规义务取决于地区、用途和适用法规，不能直接从课件推出某个检索指标是法定合规要求。

1.13 Personalized RAG

More software includes personal assistants. These tools need context. Relevant context can improve results.

越来越多软件包含个人助理。这些工具需要上下文。相关上下文可以改善结果。

个性化 RAG 使通用模型能够在获得授权的前提下，利用与用户相关的动态信息提供更贴合具体情境的回答。

个人数据的四种来源

- Calendar Events：日历事件。
- Emails：电子邮件。
- Contacts：联系人。
- Text Messages：文本消息。

模型的通用能力负责理解与总结，用户数据提供具体事实。

与企业 RAG 相比，个性化 RAG 的数据具有：强个人相关性、高频更新、多来源分散、权限敏感、时间关联明显。

工程延伸：个性化不等于无限制访问

个人助理不应该因为拥有某个数据连接，就无条件读取全部个人数据。系统应根据具体任务和用户授权确定访问范围。例如，回答日程问题通常不需要读取所有私人邮件。这涉及最小权限原则：只访问完成当前任务所需的数据。

维度	企业 RAG	个性化 RAG
知识主体	企业	个人
数据来源	内部文档、业务系统	日历、邮件、消息、联系人
权限依据	企业角色和访问控制	用户授权与服务权限
典型任务	企业问答、内部知识助手	日程查询、个人信息整理
共同机制	检索相关数据并用于生成	检索相关数据并用于生成

Part 2 RAG Architecture

2.1 The RAG Architecture RAG 的整体架构

上路由（Direct LLM use） Prompt 直接进 LLM，得到 Response。这是没有 RAG 的基线。

下路由（RAG System） Prompt 先被 Routes it to the retriever（路由给检索器），再由 Retriever 从 Knowledge Base 取回 Relevant Documents。

1. Direct LLM use：直接使用语言模型。

用户提供 Prompt，LLM 根据已有参数知识和输入上下文生成 Response。

例如：Why were Hong Kong hotels expensive last weekend?

如果没有外部检索，模型只能利用当前已知的信息回答。

这会带来一个风险：模型可能掌握一般性的酒店供需规律，却不知道特定周末香港发生了什么。因此，直接调用模型并非一定错误，而是可能缺少回答特定问题所需的外部证据。

2. RAG System：先检索外部知识，再支持模型生成。

Prompt → Retriever → Relevant Documents

同时：Retriever ↔ Knowledge Base

组件	中文	职责
Retriever	检索器	接收查询，并判断知识库中哪些文档与问题相关
Knowledge Base	知识库	存储外部知识，例如课程资料、公司文档、网页或代码
Relevant Documents	相关文档	检索阶段的输出，后续会被用于构造模型输入

Routes it to the retriever：将问题路由给检索器

这里的 Route 指的是数据流向，而不一定意味着系统已经实现了一个复杂的智能路由算法。

原笔记写：是否检索由系统决定，不由用户决定。

需要调整为：在图示的 RAG 路径中，系统负责把 Prompt 传递给 Retriever；但课件没有进一步说明是否存在动态判断需要检索还是直接回答的路由器。

在实际系统中，可以有不同设计：

- Always Retrieve：每个问题都执行检索。
- Conditional Retrieval：根据问题决定是否检索。
- User-triggered Retrieval：由用户操作触发检索。

两条路径的真正区别

RAG 的核心区别在于：生成模型能够使用根据当前问题检索到的外部证据。

- q：用户问题。
- K：知识库。
- R：检索过程。
- D_q：与问题相关的检索结果。
- G：生成模型。
- A：最终回答。

2.2 Building the Augmented Prompt 构造增强提示

RAG 的增强不是直接改变模型参数，而是在生成前把检索证据组织进模型输入。

Why were Hong Kong hotels expensive last weekend? 为什么香港酒店上周末那么贵?

1. 从 Relevant Documents 到 Augmented Prompt

Prompt → Retriever → Relevant Documents → Augmented Prompt

这意味着检索结果不能只停留在检索器中。它们必须以某种方式提供给生成模型。通常做法是把原始问题、检索片段和任务指令组织成新的 Prompt。

其中：

- P_{aug}：Augmented Prompt。
- I：Instructions，任务指令。
- $C_{\text{retrieved}}$：检索得到的上下文。
- Q：原始问题。

加号在这里表示文本或结构化消息的组合，而不是数值相加。

2. 提示示例

Answer the hotel question using the passages below. ... 使用下面提供的段落回答酒店问题。

这句话包含三个功能：

第一：明确任务。模型需要回答酒店价格问题。

第二：指明信息来源。模型应关注下面提供的段落。

第三：建立问题与证据的联系。检索结果不是独立展示的资料，而是用于支持当前回答的上下文。

using the passages below 不严格等于 using only the passages below。前者要求使用给定段落，后者才明确限制只能使用给定段落。因此，不能仅凭这一句认定课件已经要求模型完全禁止使用其他知识。

一个完整的增强提示示例

```
Task:
Answer the user's question using the provided context.
If the context is insufficient, explain the uncertainty.

Retrieved Context:
[Document 1]
National Day holiday weekend.

[Document 2]
Higher visitor demand expected.

User Question:
Why were Hong Kong hotels expensive last weekend?
```

模型可以据此回答：香港酒店价格较高可能与国庆假期期间的游客需求增加有关，但要确认具体涨价幅度及原因，还需要酒店价格和入住率等数据。这种回答将外部证据与已有经济知识结合，同时保留了证据不足时的不确定性。

Augmented Prompt 不等于普通 Prompt

维度	Original Prompt	Augmented Prompt
用户问题	包含	包含
外部检索证据	通常没有	包含
任务指令	可以包含	通常包含
内容长度	相对较短	通常更长
知识来源	模型已有知识和用户输入	额外加入检索结果

工程延伸：为什么 Prompt Construction 很重要?

即使检索器找到了正确文档，如果提示组织不当，模型也可能无法正确利用。例如，存在以下问题：证据与问题混在一起，难以区分；多份文档之间没有来源标记；文档包含相互矛盾的信息；无关片段占据大量上下文；检索文本包含恶意指令。因此，实际 RAG 系统通常需要专门的 Prompt Construction 模块。它负责把检索结果整理为模型容易理解、可追溯且安全的输入结构。

2.3 Generating the Response 生成最终回答

RAG 通过检索增强提示后再生成答案，但这种增强通常需要额外的系统计算与时间成本。

National Day holiday weekend; higher visitor demand expected. 国庆假期周末；预计游客需求较高。

这段上下文提供了两类信息：

- 时间背景：National Day holiday weekend。
- 需求背景：Higher visitor demand expected。

但它并没有直接提供酒店价格数据。因此，模型可以据此提出合理解释，却不能仅凭这段信息确认某家酒店的涨价幅度。

Added Latency：额外延迟

这是 RAG 的重要工程代价。相较于直接调用模型，RAG 通常增加了：

1. 检索查询时间。
2. 检索结果处理时间。
3. 增强提示构造时间。
4. 可能增加的模型输入处理时间。

RAG 通常引入额外的检索开销，但不能保证在所有实现、所有请求下都比直接生成更慢。

工程延伸：如何降低 RAG 延迟?

方法	工作原理	潜在代价
缓存	复用已有检索结果	可能出现过期数据
调整 Top-K	减少进入模型的片段	可能遗漏证据
优化索引	提高检索效率	需要索引维护
并行检索	同时查询多个来源	增加资源使用
压缩上下文	减少输入 token	可能丢失信息

关键词检索并非在所有场景下都比向量检索快。速度取决于索引结构、规模、硬件和具体实现。

2.4 Advantages of RAG RAG 的五项优势

RAG 的优势不仅在于提高回答质量，还包括知识更新、来源追溯和模块化维护；但这些优势都是有条件的，而不是自动保证。

课件原文	中文解释	核心机制
Adds missing knowledge	补充缺失知识	外部知识弥补模型知识缺口
Updates knowledge without retraining	不重新训练即可更新知识	更新数据源与检索索引
Supports source citation	支持来源引用	保留检索结果的来源信息
Separates retrieval from generation	检索与生成分离	模块化设计
Helps alleviate hallucinations	有助于缓解幻觉	提供与问题相关的证据

1. Adds missing knowledge：补充缺失知识

例如，模型不知道某个企业的最新产品规格。RAG 可以从企业知识库检索规格文档，再用于回答。

它解决的是知识不足的问题。但前提是外部知识源确实包含所需内容，而且检索器能够找到。

2. Updates knowledge without retraining：不重训即可更新知识

传统模型的参数知识通常依赖训练过程。RAG 可以在不修改生成模型参数的情况下，更新外部知识。

例如：旧文档：Product A warranty: 12 months

新文档：Product A warranty: 24 months

系统更新数据源和必要的索引后，就可以检索新的保修政策。但这不意味着 RAG 自动拥有最新知识。如果索引没有同步更新，仍可能检索旧内容。此外，并非每次更新都必须完整重建索引；一些系统支持增量更新。

3. Supports source citation：支持来源引用

RAG 可以保留文档的来源信息，例如：文件名称、URL、文档 ID、页码、片段位置。回答时可以展示引用，帮助用户核查。但要区分：能够引用来源与引用正确支持答案。前者是系统功能，后者需要额外验证。

4. Separates retrieval from generation：检索与生成分离

这是重要的工程优势。当回答错误时，可以分别检查：

- Retriever 是否找到了正确证据?
- Generator 是否正确使用了证据?

例如，检索结果正确但回答错误，问题可能出在生成阶段。反过来，如果检索器根本没有找到关键文档，单纯修改生成提示可能无法解决根本问题。这种模块化设计使系统更容易测试、替换和优化。

5. Helps alleviate hallucinations：缓解幻觉

检索证据能够降低模型在缺少信息时随意补充事实的风险。但它不能彻底消除幻觉。因为模型仍可能：

- 误读证据。
- 引用不相关片段。
- 混合不同来源的事实。
- 将推测表述成确定结论。

- 使用错误或过期的检索内容。

2.5 Recap: Autoregressive Generation 回顾自回归生成

New tokens depend on the tokens already generated. 新生成的 token 依赖此前已经生成的 token。

模型并不是一次性直接输出整个答案，而是逐步生成 token。

Sampling：采样

With sampling, the same prompt can produce different completions.

使用采样时，相同的 Prompt 可以产生不同的续写。这两个续写都可能符合语法与语义。因此，模型生成具有概率性，而不必然是固定输出。

Sampling 与 Greedy Decoding 的区别

Greedy Decoding：每一步选择概率最大的 token。

Sampling：按照概率分布采样 token。

采样能够提高输出多样性，但也可能引入不稳定性。

为什么自回归机制与幻觉有关?

自回归模型的基本目标是生成符合其学习到的语言分布的后续 token。但语言上的合理性不等于事实正确性。

例如：A marathon doubled hotel prices.

这句话在语法上完全成立，也符合某种可能的经济因果关系。但它是否真实发生，需要额外证据。因此，自回归生成机制本身不保证输出事实正确；事实核查需要依赖模型能力、提示策略或额外验证机制。

2.6 Hallucinations in LLM Outputs LLM 输出中的幻觉

Factuality 关注回答是否符合现实事实；Faithfulness 关注回答是否受到给定上下文的支持。

Generated content that is factually incorrect or unfaithful to the supplied context (unsupported).

生成内容在事实上不正确，或者不忠实于给定上下文、缺乏上下文支持。

1. Factuality：事实性

Does the claim agree with verifiable real-world facts? 这个说法是否符合可验证的现实世界事实?

例如：A marathon doubled hotel prices. 一场马拉松使酒店价格翻倍。

判断事实性时，需要检查：

- 是否确实发生了马拉松?
- 酒店价格是否真的翻倍?
- 是否存在支持两者因果关系的证据?

如果缺少这些验证，就不能认定该说法为真。

2. Faithfulness to Context：对上下文的忠实度

Do the supplied passages support the claim? 提供的段落是否支持这一说法?

假设检索内容只有：National Day holiday weekend. Higher visitor demand expected.

但模型回答：A marathon doubled hotel prices. 这段回答没有得到给定上下文的支持。

因为上下文没有提到：

- 马拉松。
- 酒店价格翻倍。
- 马拉松与价格之间的因果关系。

因此，它属于 unsupported answer（缺乏证据支持的答案）。

3. Factuality 与 Faithfulness 的区别

	Faithful：有上下文支持	**Unfaithful：无上下文支持**
Factual：事实正确	事实正确且有证据支持	事实正确，但不是由给定证据支持
Non-factual：事实错误	忠实复述了错误资料	事实错误且无证据支持

RAG 能改善哪个维度?

RAG 能为模型提供外部证据，从而帮助改善 Faithfulness。同时，如果外部证据准确、及时，RAG 也可能改善 Factuality。但两者都不是自动保证的。因此，更准确的结论是：RAG 可以同时帮助事实正确性和上下文忠实度，但必须结合检索质量、来源质量和生成行为进行判断。

2.7 Grounding Responses with RAG 用 RAG 为回答提供依据

RAG 通过在生成前提供相关证据，使模型有机会给出更加有依据的回答faithfulness，但证据使用是否正确factuality仍需验证。

Relevant context “grounds” the LLM's responses. RAG adds evidence before the model answers.

相关上下文为模型回答提供依据。RAG 在模型回答之前加入证据。

Grounding 让模型回答与可识别的外部证据建立联系。

用户问：Why were Hong Kong hotels expensive last weekend? 没有证据时，模型可能提出各种猜测。

加入相关证据后：National Day holiday weekend. Higher visitor demand expected.

模型就可以围绕假期需求解释酒店价格。

Grounding 的三个环节

1. Retrieval 找到与问题相关的证据
2. Context Construction 把证据组织进模型输入
3. Evidence-aware Generation 要求模型参考证据生成答案

Grounding 与 Citation 不一样

Grounding 关注：回答是否由证据支持?

Citation 关注：回答是否标出了来源?

例如：香港酒店价格上涨是因为国庆假期。[Source 1]

即使附有 Source 1，也必须检查 Source 1 是否真的支持这个因果结论。

因此：有引用不等于有充分证据支持。

Grounding 是否保证不会幻觉?

不能。模型可能忽略部分证据、错误理解检索结果、引入证据之外的细节、将相关性误认为因果关系。

因此，Grounding 是降低无依据生成风险的机制，不是正确性的数学保证。

2.8 RAG 的计算限制

上下文窗口与计算资源都有限，因此 RAG 必须在信息覆盖范围和上下文成本之间进行选择。

1. Higher Computational Cost：更高计算成本

Longer prompts require more computation and memory.

更长的 Prompt 通常需要更多计算与内存资源。

Long contexts can still increase latency.

长上下文仍然可能增加延迟。

例如：输入 2,000 tokens 与输入 200,000 tokens，模型需要处理的上下文规模明显不同。即使模型支持长上下文，也不代表处理成本可以忽略。

2. Context Window Limit：上下文窗口限制

长上下文通常增加计算与内存压力，但具体增长方式取决于模型架构、推理阶段和实现。

Prompt and answer share a limited context budget.

Prompt 与 Answer 共享有限的上下文预算。

Capacity depends on the model and configuration.

为什么检索比全部输入更合理?

假设知识库包含 10,000 个片段。用户的问题只需要其中 5 个。全部输入意味着大量无关内容进入上下文。选择性检索则只需要返回相关片段。这样可以降低上下文占用。

但也引入新问题：如果检索器没有选对片段怎么办? 这正是后续 Retriever Tradeoffs 和 Retrieval Evaluation 要解决的问题。

2.9 Example

How can I find some recipes to make Shanghai-style baozi at home? 怎样找到在家制作上海风味包子的食谱?

图书馆拥有大量书籍，但用户不可能逐本阅读。因此，需要一种高效的查找机制。

组成部分	课件原文	作用
Collection	Books on many topics	收藏不同主题的书
Organization	Different shelves and sections	按书架与区域组织
Search	A librarian helps you find the best sections or books	图书管理员帮助查找

图书馆	RAG 系统	工程作用
Books	Documents	存储原始知识
Shelves and sections	Index	组织可检索信息

图书馆	RAG 系统	工程作用
Librarian	Retriever	根据问题查找文档

检索系统不仅需要信息集合，还需要组织信息与查找信息的机制。

Organization：索引

如果文档没有组织结构，查找可能非常低效。索引可以帮助系统快速定位候选内容。

索引是提升检索效率的重要工具，但不是所有检索算法在逻辑上都必须依赖索引。

不同索引的组织方式不同。常见索引包括：

- Inverted Index：倒排索引。
- Vector Index：向量索引。
- Metadata Index：元数据索引。

Search：检索

用户提出问题后，Retriever 根据查询寻找相关文档。

例如：

问题：How can I make Shanghai-style baozi?

检索器可能优先返回：Baozi Basics

而不是：Shanghai History

What's the Role of a Librarian?

1. Understands the meaning of your question 理解问题的含义。

例如：How can I make Shanghai-style baozi?

系统需要识别用户希望寻找的是制作方法，而不是上海的历史介绍。

在检索系统中，这通常涉及 Query Understanding（查询理解）。

2. Identifies the right shelves to search 确定应该查找哪些书架。
3. Eventually finds relevant books 最终找到相关书籍。

图书管理员行为	检索系统概念
理解问题	Query Understanding / Representation
定位书架	Candidate Selection / Filtering
找到相关书籍	Retrieval / Ranking

需要注意，课件是在用人类图书管理员进行概念类比。它并不意味着所有 Retriever 都必须严格分成三个独立模块。例如，一个向量检索器可以直接把查询编码为向量，再进行近邻搜索。查询表示与候选选择可能在同一套检索过程中完成。

优秀的检索器需要根据问题选择合适的候选内容，并将相关结果返回给后续生成阶段。

2.10 Ranking Retrieved Documents 检索文档排序

检索器不仅需要找到候选文档，还需要对它们排序；但检索分数代表算法定义的匹配程度，不等于事实正确性或可回答概率。

How can I find some recipes to make Shanghai-style baozi at home? 检索器返回四个结果。

Ranking 是将候选文档按相关性得分排序。检索器通常希望相关文档的得分高于不相关文档。

检索器需要考虑：What does the prompt mean? 问题表达了什么含义？以及 What documents in the knowledge base are similar? 知识库中哪些文档与问题相似？这两个问题体现了查询理解与文档匹配。但它们不能严格对应为两条互斥的技术路线。关键词检索也需要查询处理，语义检索也需要计算相似度。

检索分数是不是概率？不是必然。

某篇文档即使与问题存在部分相关性，也不一定包含回答问题所需的信息。

相似度较高，不代表文档足以回答问题。

2.11 Retriever Tradeoffs 检索器的取舍

检索结果过少可能遗漏证据，过多可能引入噪声。系统需要通过指标和实际测试选择合适的检索配置。

检索系统的核心权衡：应该返回多少文档？

1. Relevance vs Irrelevance: Need to return relevant documents and withhold irrelevant ones.

应该返回相关文档，并尽量排除不相关文档。这看起来很简单，但实际并不容易。因为检索器无法保证每个评分都完全符合真实相关性。

2. Return Every Document? Irrelevant documents waste the context window.

如果返回全部文档：优点：可能减少遗漏。缺点：大量不相关文档占用上下文窗口。

3. Return the Single Highest Ranked Document?

如果只返回排名第一的文档：优点：上下文更短，输入成本更低。缺点：可能遗漏其他有价值的信息。

4. Top-K 的基本思想

Top-K 表示返回评分最高的 K 个结果。K 越大，通常能够覆盖更多候选信息，但也可能增加噪声和上下文成本。

5. Precision 与 Recall 的权衡

Precision：表示返回的结果中，有多少是真正相关的。

Recall：表示全部相关文档中，有多少被成功找回。

6. No Perfect Solution：没有完美方案

No perfect solution: Scoring and monitoring. Change settings to find what works.

通过评分、监控和调整配置，寻找适合实际任务的方案。这意味着 Top-K 并不存在一个对所有系统都最好的固定值。

7. 工程延伸：为什么不能只追求 Recall?

如果为了找回全部相关文档而返回大量候选内容，Recall 可能上升，但无关片段也可能增加。

这会导致：上下文成本上升、模型更难定位关键证据、可能出现证据冲突、回答质量不一定提高。

因此，检索系统的目标不是简单地让 K 越大越好，而是在覆盖率、精确率、延迟和生成质量之间取得平衡。

2.12 RAG 的改进

RAG 建立在成熟的信息检索技术之上，并通过语言模型将检索结果转化为面向用户的生成式回答。

信息检索并不是随着 LLM 和 RAG 才出现的技术。

1. Search Engine：搜索引擎

Retrieves relevant webpages. 检索相关网页。

2. Database：数据库

Retrieves relevant tables and rows. 检索相关的数据表和数据行。数据库根据查询条件返回符合要求的记录。

需要注意，数据库中的精确条件查询与搜索引擎的相关性排序并不完全相同，但两者都涉及从大量信息中定位所需内容。

3. Information Retrieval 是成熟领域

Information retrieval was already mature before LLMs, and ideas from this field underlie the design of retrievers and RAG systems.

在 LLM 出现之前，信息检索已经是一个成熟领域，而该领域的思想构成了 Retriever 和 RAG 系统设计的重要基础。

RAG 继承了哪些经典技术?

技术	主要作用	与 RAG 的关系
TF-IDF	词项重要性加权	关键词检索
BM25	文档相关性排序	稀疏检索
Inverted Index	快速定位包含词项的文档	检索索引
Cosine Similarity	比较向量方向相似性	稀疏与稠密向量检索
Precision	衡量返回结果的准确程度	检索评估
Recall	衡量相关结果覆盖程度	检索评估
MAP / MRR	衡量排序检索表现	排序评估

需要区分，向量相似度方法可以与经典信息检索技术结合，但现代神经嵌入模型的发展也引入了新的检索能力。

RAG 真正增加了什么?

传统搜索引擎主要返回：相关文档或网页。

RAG 在此基础上进一步执行：将检索结果提供给生成模型，让模型综合信息生成自然语言答案。

RAG 的创新重点不是从零发明检索，而是将检索机制与语言生成系统结合。

Part 3 Typical Retrieval Methods 典型检索方法

3.1 The Retrieval Challenge 检索的挑战

Retriever Goal: find documents in the knowledge base that can help an LLM respond to a prompt.

从知识库中找到能够帮助 LLM 回答用户问题的文档。

角色	课件定义	实际困难
Prompts	Unstructured conversation	用户表达自由、口语化
Retriever	Find relevant documents quickly	必须同时兼顾相关性与速度
Documents	Many formats written for humans	文档格式复杂、表达不统一

1. 用户问题不一定与文档使用相同的词。

例如，用户问：How can I make baozi at home? 文档可能写：Use flour and yeast for Shanghai-style baozi dough. 用户关心的是制作方法，但文档不一定包含完整的提问表达。

2. 检索命中不等于文档能够回答问题。

如果文档写的是：Shanghai baozi shops offer home delivery. 它虽然包含 baozi 和 home，却没有提供制作包子的方法。因此需要区分两个概念：

- Lexical Match（词面匹配）：查询词是否出现在文档中。
- Relevance（相关性）：文档是否真正有助于回答问题。

这正是后面引入语义检索的原因。

3.2 检索器内部

How can we efficiently search the knowledge base?

这里的 efficiently（高效）不仅是运行速度快，还意味着能够从大量文档中优先找出相关内容。

Retriever 本身并不负责生成最终答案，而是为 LLM 提供更有用的上下文。

因此，检索质量会直接影响后续生成质量：

- 检索遗漏关键文档，LLM 可能缺少必要信息。
- 检索返回大量无关文档，有限的上下文窗口可能被噪声占用。
- 检索返回高相关文档，LLM 更容易生成有依据的回答。

3.3 Two Search Approaches 两种搜索范式

	Keyword Search	Semantic Search
中文	关键词检索	语义检索
核心依据	Exact words	Similar meaning
关注的问题	词是否相同?	意思是否接近?
典型方法	TF-IDF、BM25	Embeddings
主要优势	精确词匹配	同义表达匹配
主要局限	难处理同义词	可能忽略精确标识符

查询：How can I make baozi?

文档 A：Instructions for preparing steamed buns.

文档 B：Baozi shops provide home delivery.

关键词检索可能更偏向 B，因为它直接包含 baozi。语义检索则可能更偏向 A，因为 preparing steamed buns 与制作包子在含义上更接近。两种方法的差异，不是有没有向量，而是向量表示的是什么。

关键词检索用向量表示词项出现情况；语义检索用模型学习出的向量表示文本含义。

3.4 Search Techniques 检索技术总览

三个数量概念：

1. 候选文档数量：检索器最初召回的文档。
2. 过滤后数量：满足元数据约束的文档。
3. Top-K 数量：最终返回给 RAG 后续环节的最高排名文档。

后面还会引入 RRF（Reciprocal Rank Fusion），用于把关键词检索和语义检索得到的两个排名列表融合成一个。

3.5 Metadata Filtering 元数据过滤

元数据（Metadata）指描述文档属性的信息，而不一定是文档正文。

Metadata Filtering 的核心不是计算相关性，而是判断文档是否满足硬性条件。

例如，用户要求只查询 2024 年 6 - 7 月的文章，即使一篇 2023 年的文章内容高度相关，也应当被排除。

优点与局限

Benefits（收益）	Limits（局限）
Easy to understand and debug　容易理解与调试	Does not interpret document content　不解读文档内容
Fast, mature implementations　实现快且成熟	Does not rank by relevance　不按相关性排序
Enforces exact metadata conditions　严格执行元数据条件	Usually paired with a search method　通常需与检索方法搭配

- 第一条收益 debug 很实际，过滤是布尔逻辑，出错了可以直接看出是哪条条件写错，不像打分那样难定位。
- 第一条限制是根本性的，它只读元数据，不读正文，所以无法判断内容是否切题。
- 第二条限制让它无法独自完成检索，它只能缩小范围，不能提供排序，最终还是需要一个排序器。
- 第三条既是限制也是用法说明，通常与检索方法搭配把它的定位钉死为预处理，而非独立方案。

其中最重要的区别是：Filtering（过滤）不等于 Ranking（排序）。假设有三篇文档：

文档	发布日期符合要求?	内容相关性
Doc A	是	高
Doc B	是	低
Doc C	否	很高

元数据过滤会保留 A、B，排除 C。但它不会自动判断 A 比 B 更相关。这需要关键词检索或语义检索等排序方法。因此，Metadata Filtering 通常作为检索流水线中的约束机制，与搜索算法配合使用。

3.7 Typical Retrieval Methods 典型检索方法

课件接下来正式进入三个主要方法：

1. Keyword Search：Bag of Words、TF-IDF、BM25。
2. Semantic Search：Embedding、Vector Similarity、Positive and Negative Training Pairs。
3. Hybrid Search：结合关键词与语义检索结果。

这三种方法并非完全替代关系，而是逐步解决不同的问题。关键词检索回答的是哪些词匹配，语义检索回答的是哪些含义接近，混合检索则尝试综合两种证据。

3.8 Introduction to Keyword Search

关键词检索可以找到字面相似的文档，但词项重合不等于内容相关。

Match words in the query to words in documents. Shared terms provide a simple basis for ranking.

关键词检索通过统计查询与文档之间的共有词项来衡量相关性。给这些共有词项打分。

1. BoW 把文本转换为词频统计
2. Sparse Vector 把词频表示成向量
3. TF 衡量词在文档中出现的频率
4. IDF 衡量词在整个文档集合中的罕见程度
5. BM25 进一步控制词频饱和与文档长度影响

这一条技术路线的逻辑是：

最初只判断词有没有出现，随后考虑出现多少次，再考虑这个词是否常见，最后处理重复词带来的过度加分以及文档长短不公平的问题。

Example：包子查询中的关键词匹配

用户的问题是：How can I make baozi at home?

关键词检索首先寻找共有词

把问题拆开，可以得到：`how`、`can`、`I`、`make`、`baozi`、`at`、`home`

这些词并不具有相同的信息价值。

Doc 1 包含 baozi，并提供了制作面团的信息。

Doc 2 包含 baozi 和 home，但讨论的是外送。

Doc 3 包含 Baozi，却主要介绍餐厅。

因此，包含更多相同词项，并不一定意味着文档更能回答问题。

后续关键词检索的三个问题：

1. 不同关键词是否应该具有相同权重？
2. 同一个词出现很多次，是否应该持续增加得分？
3. 长文档比短文档包含更多关键词，是否就应该排名更高？

接下来的 BoW、TF-IDF 和 BM25，就是逐步解决这些问题。

3.8.1 Bag of Words 词袋模型

Bag of Words represents text by counting how often each word appears. Word order is ignored, only word presence and frequency matter.

词袋模型通过统计每个词出现的次数来表示文本。

模型只关心有哪些词，以及每个词出现了多少次，而不关心词的排列顺序。

BoW 本质上是一种词项统计表示，而不是完整的语义表示。

词袋模型为什么仍然有用

虽然它丢失词序，却带来两个重要优势。

第一，文本可以转换为数字。

第二，不同长度、不同内容的文本，可以映射到同一个词汇空间中进行比较。

这为后面的稀疏向量、词项—文档矩阵和相关性评分建立了基础。

3.8.2 稀疏向量 Sparse Vectors

词频表可以表示成一个向量。

构造词汇表，按照课件展示的词项顺序，可以整理为：

Term	Value	Term	Value	Term	Value
making	1	baozi	2	without	1
a	1	steamer	1	rice	0
flour	0	dough	0	tea	0
bowl	0	water	0	salt	0
soup	0	yeast	0	filling	0

向量长度为 15，因为词汇表有 15 个词项。只有 5 个位置非零。

One vocabulary term per position; most entries are zero.

Sparse 即每个位置对应词汇表中的一个词，而大部分位置都是 0。

例如，一个真实知识库可能有几十万个不同词项，但一篇短文只包含其中几百个。因此，其向量绝大多数维度都是 0。这就形成了高维稀疏向量。

稀疏向量与稠密向量的区别

	Sparse Vector	Dense Vector
中文	稀疏向量	稠密向量
典型用途	关键词检索	语义检索
维度含义	对应具体词项	学习得到的特征维度
数值分布	大量为 0	通常多数非零
代表方法	BoW、TF-IDF	Embedding Models

稀疏向量的每一维通常可以解释为一个具体词项；语义嵌入向量的单个维度通常没有这种直接可读的含义。

工程上为什么不直接存储所有 0

如果一个向量有 100,000 维，只有 100 个非零值，那么直接存储全部数值非常浪费空间。实际系统可以只存储：非零元素所在的位置、对应的数值。

在真实检索系统中，倒排索引（Inverted Index）通常比直接逐篇比较完整稀疏向量更适合关键词搜索。

3.8.3 The Term-Document (TD) Matrix 词项—文档矩阵

如果知识库有很多篇文档，如何把它们组织到同一个数据结构里？

答案是 Term-Document Matrix，即词项—文档矩阵。

矩阵的基本结构

- 行：Terms（词项）。
- 列：Document Vectors（文档向量）。
- 单元格：某个词项在某篇文档中的匹配或词频信息。

其中：

- t_i：第 i 个词项。
- d_j：第 j 篇文档。
- M_{ij}：该词项在该文档中的出现次数。

从矩阵到排序

这个过程可以概括为：文本 → 词频向量 → 文档矩阵 → 查询匹配 → 相关性得分 → 排序。

课件后面的 TF-IDF 仍然使用这个结构，只是把矩阵中的简单词频替换为带权重的词频。

因此，TF-IDF 不是推翻词袋模型，而是在已有的词项表示基础上改进权重。

Scoring a Keyword Match 计算相关性

规则：每匹配一个查询词项，就加 1 分。

这个公式对应的是 Binary Term Matching（二值词项匹配）。

它只判断词项是否出现，不考虑出现次数。

问题一：常见词与罕见词同权。a 命中一次也 +1，baozi 命中一次也 +1，但显然后者更有信息量，归一化解决。

问题二：长文档天然占优。文档越长，越可能包含查询里的词，命中数自然更高，词重要性（IDF）解决。

Scoring All Query Terms 对全部查询词打分

在当前规则下：

- `baozi` 匹配，加 1 分。
- a 匹配，也加 1 分。

但从信息价值来看，两者显然不同。`baozi` 能帮助识别文档主题。a 几乎不能帮助区分哪些文档与制作包子相关。不同词项是否应该具有不同的重要性？后面的 IDF 就是为此设计的。

3.8.4 Frequency-Based Scoring

前面的简单匹配规则只考虑词有没有出现。那么一个查询词在文档中出现很多次，是否意味着文档更相关?

课件定义:

TF 的优势与问题

TF 比简单的二值匹配更细致，因为它考虑了重复出现的词项。但它存在一个明显问题：较长的文档往往天然包含更多词，因此也更容易获得较高的原始 TF 分数。

例如，一篇 10,000 字的文章，可能比一篇 100 字的文章包含更多次 baozi。但这不代表前者一定更相关。

所以，课件下一页引入长度归一化。

3.8.5 Normalized Term Frequency 归一化词频

Issue: Raw counts can favor longer documents.

原始词频可能偏向长文档。

排名发生了逆转

Raw TF：Document 2 更高；Normalized TF：Document 1 更高

这说明：仅仅改变评分中的长度处理方式，就可能改变最终的检索排序。

归一化是否完全解决了长度问题?

没有。简单地除以文档长度，可能过度奖励非常短的文档。例如，一篇只有 5 个词的文档，其中 3 个是查询词，它的归一化分数可能很高。但这篇文档不一定包含足够的信息来回答问题。

这也是后面 BM25 不直接使用简单的 $TF/|d|$，而采用更灵活的长度归一化机制的原因之一。

3.8.6 TF-IDF：词频与逆文档频率

前面已经解决了两个问题：词是否出现。词出现多少次。

现在还剩一个重要问题：不同词的重要性是否应该相同?

Basic TF scoring treats all words equally, whether they're common filler words or rare, meaningful terms.

基础 TF 评分把所有词都看得一样重要，不论它是常见词还是具有区分性的关键词。

例如：`the`：非常常见；`baozi`：相对少见，并且具有明确主题信息。因此，引入 TF-IDF。

注意，TF-IDF 有多种实际实现形式，TF 可以使用原始次数、对数缩放或归一化形式，IDF 也可以进行平滑处理。这里先严格按照课件给出的基本公式理解。

为什么要使用逆文档频率

假设有 100 篇文档。`the` 出现在全部 100 篇中。`baozi` 只出现在 5 篇中。

`the` 几乎不能帮助检索器区分文档，因为每篇都有。`baozi` 则更具有区分性，因为只有少量文档包含它。

所以一个词越少见，它在区分文档方面通常越有价值。这就是 IDF 的基本思想。

3.8.6.1 Calculating Inverse Document Frequency 计算逆文档频率

第一步：统计文档频率

第二步：取倒数

第三步：取自然对数

为什么还要取对数？如果直接使用 N/df极罕见词可能得到非常大的权重。

例如：出现在 10 篇文档中：倒数为 10；出现在 1 篇文档中：倒数为 100。两者相差 10 倍。

取对数后，差距被压缩。因此，对数变换既保留了罕见词权重更高的趋势，又避免权重增长得过快。

3.8.6.2 Applying TF-IDF Weights

3.8.6.3 Ranking with TF-IDF 用 TF-IDF 排序

Rare keywords can contribute more to the score than common words

罕见关键词对分数的贡献可以超过常见词

加了 TF 之后，同一个词的得分变化，但词与词之间的相对高低仍由 IDF 主导

TF-IDF 解决了什么

相比原始词频，TF-IDF 能够降低大量普通词的影响。

例如，一篇文档反复出现 `a`、`the`、`and`，不应仅仅因为这些词出现得越多就获得高相关性。

TF-IDF 仍然存在什么问题

TF-IDF 并不是完整的相关性理解模型，仍然依赖词项重合。

例如：查询：How to prepare steamed buns? 文档：Instructions for making baozi.

即使两个文本语义非常接近，如果没有合适的词形处理或词项扩展，传统 TF-IDF 也可能无法有效匹配。

此外，原始 TF-IDF 中的词频可能持续增长，长文档长度问题也需要额外处理。

这两个问题正是 BM25 重点改进的方向。

3.8.7 From TF-IDF to BM25

BM25 refines keyword scoring with saturation and length normalization.

BM25 通过词频饱和与文档长度归一化，进一步改进关键词相关性评分。

两个核心概念：

1. Term Frequency Saturation（词频饱和）

一个词出现 20 次，不应该必然获得出现 10 次时的两倍得分。

2. Document Length Normalization（文档长度归一化）

长文档包含更多词，但不应该仅仅因为篇幅更长而获得不合理的排名优势。

这两点构成 BM25 相比简单 TF-IDF 的主要改进。

Modern RAG systems often use BM25 to refine keyword scoring.

BM25 是现代检索系统中广泛使用的关键词排序方法。

3.8.7.1 BM25 Scoring

BM25 全称 Best Matching 25，来自 1990 年代的 Okapi 检索系统，是 TREC 评测里迭代出的第 25 版公式。它不是从理论推导出来的，而是在大量评测中调出来的经验公式。这也解释了为什么它带有 k_1 和 b 两个需要人工设定的参数，它们是评测调优的产物。

BM25 Tunable Parameters BM25 可调参数

维度	Raw TF-IDF	BM25
词频增长	10 次 = X → 20 次 = 2X（线性）	20 次 → Score < 2X（饱和）
短文档	Short Doc = Higher Score	Short Doc = Less Penalty
长文档	Long Doc = Lower Score	Long Doc = Tunable Penalty
惩罚性质	Fixed length penalty　固定	Tunable Penalty　可调（由 b 控制）

Two Improvements in BM25 BM25 的两项改进

第一项：词频饱和

- 简单线性词频评分：rice 出现 10 次得到 X，出现 20 次得到 2X。
- BM25：出现 20 次的得分通常小于出现 10 次得分的两倍。

原因是重复出现的词会产生递减收益。例如，一篇文档已经多次讨论 rice，再增加一次 rice，提供的额外相关性证据可能并不多。

第二项：文档长度归一化

- 简单按长度归一化可能形成固定比例的长度惩罚。
- BM25 使用可调节的长度惩罚。

这让检索器可以适应不同类型的文档集合。例如：新闻标题：文本普遍较短。学术论文：文本较长。RAG 文本块：长度可能较为接近。这些集合不一定适合完全相同的长度惩罚。

工程上的重要细节

如果 RAG 系统已经把长文档切成长度相近的 Chunk，BM25 的长度归一化效果可能与直接检索完整文档时不同。

因为此时长度比例接近 1。这意味着，Chunking 策略会间接影响 BM25 的评分行为。

饱和为什么符合直觉？一篇文档里出现 10 次 rice，已经足够说明它在讲米饭；出现 20 次，信息量并没有翻倍。如果一篇文档反复刷同一个词，多半是垃圾内容或关键词堆砌，不应该无限加分。饱和机制正是对这种噪声的防御，它是边际效用递减在检索打分里的实现。

TF-IDF vs. BM25

维度	TF-IDF	BM25
词频处理	基本形式可线性增长	饱和函数
词项罕见程度	IDF	IDF
文档长度	依赖具体实现，可另行归一化	内置可调长度归一化
参数	基础公式较简单	(k_1, b)
工程应用	经典检索基线	常用生产级关键词排序方法
语义理解	不直接理解语义	不直接理解语义

BM25 比基础 TF-IDF 更灵活，不代表它在任何数据集上都必然更好。最终效果仍然需要通过检索评估指标和实际任务测试判断。

工程实现上

默认选 BM25：成本相近、质量更好、生态成熟

用 TF-IDF：语料很小、需要极简实现、或需要一个能看懂每一步的教学基线——Tutorial 用的就是 TF-IDF，因为 13 个分块、345 个词项的规模下，BM25 的两个参数反而增加理解负担。

3.8.8 关键词检索小结

To match a prompt according to how frequently the prompt’ s keywords appear in each document

按提示关键词在各文档中出现的频次来匹配提示

1. TF-IDF 的三项加权： Keyword rarity（关键词罕见度）、Term frequency（词频）、Document length（文档长度）。
2. BM25（最常用）的两项改进：Document length normalization 与 Term Frequency Saturation。
3. 流程四个动词 ： Sparse Vectors → Scored → Ranked（稀疏向量 → 打分 → 排序）
4. What are the pros and cons?——Why Meaning Matters?

3.9 Semantic Search and Embeddings

3.9.1 语义的重要性

关键词检索的两类系统性失败：

失败一：同义词（Synonyms）

失败后果：文档里写 glad，查询里写 happy，共有词为零，分数为零。

失败二：同形异义（Same spelling, different meaning）

失败后果：查询里的 Python（编程语言）与文档里的 Python（蟒蛇）字面相同，会被判为高度相关。

现象	方向	关键词检索的表现	语义检索的期望表现
Synonyms　同义词	漏检	happy vs glad 分数为 0	两者向量接近，能命中
Polysemy　同形异义	误检	Python 语言 vs 蟒蛇被当作同一回事	由上下文决定向量位置，可区分

关键词检索的全部信息就是字面是否相同，它没有别的线索可用，所以这两类错误是方法层面的，不是参数层面的。 要解决它们的问题，需要引入embedding，让表示本身携带语义。

3.9.2 关键词检索与语义检索

Both approaches represent the query and documents as vectors.

关键词检索和语义检索都可以把查询与文档表示为向量。两种方法都把查询与文档表示成向量，最大的区别不在于是否使用向量，而在于向量表示什么。

1. Keyword Search：A sparse lexical space 稀疏词汇空间

关键词检索统计词汇表中各个位置上的词。例如：每一维通常对应一个具体词项。如果词汇表包含 50,000 个词，向量就可能具有 50,000 个维度。大部分维度为零。

2. Semantic Search：语义嵌入空间

语义检索使用 Embedding Model，将文本转换为能够表达学习到的语义特征的向量。

维度	Keyword Search	Semantic Search
构造方式	Count words in vocabulary positions 在词表位置上计数	Use an embedding model to represent meaning 用嵌入模型表示含义
空间名称	A sparse lexical space（稀疏的词法空间）	A semantic (embedding) space（语义/嵌入空间）
表示依据	词项出现与频率	学习得到的语义特征
维度特点	词表有多大就有多少维，绝大多数为 0	维度固定（如 384），几乎每维都有值
典型向量	Sparse Vector	Dense Vector
可解释性	高：分数可拆到具体词项	低：分数是整体相似度
核心方法	BoW、TF-IDF、BM25	Embedding Model
主要优势	精确词项匹配	语义相似性匹配
主要局限	同义词与上下文理解	可能忽略精确词项差异

两种检索方式都能使用向量，但关键词向量表示词项统计，语义向量表示模型学习到的文本特征。

- 共同点比差异更重要，既然两者都产出向量，那么相似度计算方式、排序方式、top-k 截断方式都可以共用。
- 差异只在怎么造这个向量，这正是混合检索可行的技术前提。

3.9.3 Word Embedding Space 词嵌入空间

Embedding Different Text Units 嵌入不同的文本单元

嵌入可以作用在三个粒度上，各有各的模型。

文本单元	模型	课件示例
Word　词	Embedding Model	cat ・ happy
Sentence　句子	Sentence Embedding Model	The weather is nice ・ What a lovely day
Document　文档	Document Embedding Model	（课件以空框示意，未给示例句）

分块（chunking）本质上是在选粒度。块太小，一个事实会与它的解释被切开；块太大，一个向量要概括太多主题，相似度会被稀释，模型要处理的信息越多。一个句子需要处理语序与修饰关系；一篇文档还要处理长距离依赖。

Sentence Embedding Example 句子嵌入示例

三句话、三个向量：意思接近的两句在空间里靠得近

1. He spoke softly in class.
2. He whispered quietly during class.
3. Her daughter brightened the gloomy day.

第一句和第二句：spoke softly、whispered quietly 表达的动作和语境相近；第三句则表达了不同的内容。

- 课件结论句 — Once we project these sentences into an embedding space, we can use vector similarity to measure the semantic meaning.

示意：A、B 方向更接近，余弦相似度更高；C 与它们的方向差异更大。

3.10 语义检索流程 Semantic Search

Documents/Prompt → Embedding Model → Vector Space

- 第一步，用 Embedding Model 把 Documents 全部转成向量，放进 Vector Space。
- 第二步，用同一个模型把 Prompt 转成向量，也放进 Vector Space。
- 第三步，Rank by distance and return the closest documents（按距离排序，返回最近的文档）。

距离近 = 语义近。

欧氏距离（L2）：越小越相似。

余弦相似度：越大越相似（值域 -1 到 1，非负向量时为 0 到 1）。

必须是同一模型，换模型意味着两个向量不在同一个空间，比较结果失去意义。

3.11 Embedding Models Learn from?

前面已经知道：相似的文本应该在向量空间中更接近。

但这引出了一个新问题：模型怎么知道哪些文本相似，哪些文本不相似?

课件使用三个例子：Hello、Good morning!、Noisy trombone

Hello 和 Good morning! 都属于问候语。

Noisy trombone 则表示一种不同的概念。

1. 向量位置不是天然正确的

在训练前，模型并不会天然拥有一个符合任务目标的语义空间。课件通过散点图展示了不同文本在空间中的位置。模型需要利用训练数据和损失函数，逐步调整参数，让空间结构更符合训练目标。

2. 训练的三个基本组成部分：

Training Data（训练数据）告诉模型哪些文本相似，哪些文本不同。

Architecture（模型结构）负责将文本映射为向量。

Loss Function（损失函数）衡量当前向量表示是否符合训练目标。

训练过程不断更新模型参数：

3.12 Positive and Negative Training Pairs 正样本对与负样本对

样本类型	课件例子	含义	训练动作
Positive pair　正样本对	Good Morning ↔ Hello	Similar meaning	拉近两个向量
Negative pair　负样本对	Good Morning ↔ Noisy trombone	Different meanings	推远两个向量

为什么正负样本需要同时存在

如果只训练正样本，模型可能只学会让某些相似文本靠近，却缺乏足够的区分能力。

负样本提供了对照：不仅要知道什么是相似的，还要知道什么是不相似的。

这种利用相似与不相似关系学习表示的方法，是 Contrastive Learning（对比学习）的核心思想。

3.13 Training data

Compile massive training dataset of positive and negative pairs

构建大规模正负样本训练数据集

为什么需要大量训练数据

单独一个正样本对：`Good Morning` — `Hello`只能告诉模型这两个表达比较相似。但真实语言具有大量不同的语义关系。例如：问候语、同义词、主题相似、问题与答案、描述与标题。

因此，模型需要从大量样本中学习更一般化的表示能力。

样本关系与训练目标

在语义检索中，一个正样本对不一定是两句完全相同意思的话。

Query：How do I make baozi?

Positive Document：Steps for preparing steamed buns at home.

它们不是同一句话，但文档可以回答查询，因此在检索任务中可以被视为相关样本。

这说明：检索模型学习的不仅可以是语言意义上的相似性，也可以是查询与文档之间的任务相关性。

这是对课件正负样本概念的检索场景的延伸。

3.14 Further Reading

3.14.1 Contrastive Loss

公式说明正负样本是如何被同时拉近与推远的

Contrastive Loss 通过比较正负样本的相对相似度，引导模型学习具有区分能力的嵌入空间。

3.14.2 Updating the Embedding Model

上一页定义了训练目标。这一页展示模型如何通过重复训练改善向量空间。

把训练过程画成一个循环：Embed（嵌入当前样本）→ Score（按损失函数打分）→ Evaluate（评估误差）→ Update（更新模型参数）

训练集上的正负样本分离并不能自动保证检索效果良好。实际还需要在独立评估数据上验证模型的泛化能力。

3.14.3 High-Dimensional Embeddings

More dimensions mean more room to form clusters and capture nuanced relationships.

更多维度为模型表示复杂关系提供了更大的空间。高维表示可以帮助模型学习这些复杂因素之间的关系。不过，这并不意味着每一个维度都对应某个明确、独立、可解释的语义属性。

高维空间仍然遵循相同原则

The same principles hold. Close vectors, similar meanings.

即使嵌入向量有数百甚至数千维，基本思想仍然相同：在模型采用的相似度度量下，语义相关的文本应该具有较高相似度。

高维不一定更好

增加维度可能提升表示容量，但也可能增加：向量存储成本、检索计算开销、索引内存需求。

因此，Embedding Dimension 需要在表示能力、检索效果和系统成本之间取得平衡。

3.15 嵌入模型要点

课件原文	含义
Vector locations are arbitrary before training.	训练之前，向量位置没有意义
Training data and constrative loss form the learning	学习由训练数据与对比损失共同塑造
compare vectors from the same embedding model.	只有同一模型产出的向量才能互相比较
Pretrained models can be used directly for retrieval.	预训练模型可以直接拿来做检索，无需自己训练

1. Vector locations are arbitrary before training

训练之前，向量的位置并不天然对应所需的语义结构。通过训练，模型逐步学习有用的表示。

2. Training data and contrastive loss from learning

课件强调训练数据与 Contrastive Loss 的共同作用。训练数据定义哪些文本应该接近、哪些应该区分。损失函数把这些关系转换为可以优化的数学目标。

3. Compare vectors from the same embedding model

工程原则：不同嵌入模型即使输出相同维度的向量，也不代表这些向量位于同一个兼容空间中。虽然维度相同，但如果两个模型独立训练，直接比较其输出通常没有可靠的语义意义。对于 RAG 系统，文档和查询通常需要使用同一个嵌入模型，或者使用经过联合训练、明确兼容的查询与文档编码器。

4. Pretrained models can be used directly for retrieval.

预训练模型可以直接用于检索，也就是说，并不是每个 RAG 项目都需要从头训练一个 Embedding Model。可以使用已有的预训练嵌入模型，为查询和知识库文档生成向量。这显著降低了语义检索系统的构建门槛。

这一部分从关键词检索的局限出发，解释了语义检索的完整逻辑。

核心问题	对应概念	解决思路
不同词能否表达相同意思?	Semantic Similarity	比较语义而非仅比较词项
文本如何变成数字?	Embedding Model	映射到向量空间
哪些文本更相关?	Vector Similarity	比较查询与文档向量
模型如何知道相似关系?	Positive / Negative Pairs	提供训练监督
如何训练嵌入空间?	Contrastive Loss	拉近正样本、区分负样本
如何改善模型?	Embed - Score - Update - Evaluate	反复优化
如何用于实际 RAG?	Pretrained Embeddings	生成向量并检索

3.16 Comparing Retrieval Methods 比较各种检索方法

方法	课件原文	中文
Metadata filtering	Applies strict criteria such as permissions or region.	施加严格判据，如权限或地区
Keyword search	Matches exact terms and provides a strong lexical baseline.	匹配精确词项，提供强的词法基线
Semantic search	Finds similar meanings even when wording differs.	即使措辞不同也能找到相近含义

这三种方法并不互相排斥。实际系统可以先使用元数据约束候选范围，再利用关键词和语义检索找到相关内容。

3.17 Combining Ranked Lists 合并排序列表

混合检索的目标：Lists combined to form single ranking 把多个排序列表合成一个排序。

为什么两个结果列表不同?

因为两种检索方法使用不同的相关性标准。

Keyword Search 可能认为 Document A 最相关，因为它包含更多精确匹配的关键词。

Semantic Search 则可能认为 Document C 最相关，因为它在嵌入空间中更接近查询。

注意，两个列表也存在重叠，例如 Document B 和 Document C。

这意味着某些文档同时得到了两种检索方法的支持。

合并不是简单拼接

假设直接把两份列表连接起来，就可能出现两个问题：

- 同一篇文档重复出现。
- 两个列表之间缺少统一的相关性顺序。

融合要解决的核心难题：分数不可比

关键词检索的分数是 TF-IDF/BM25 值（无上界，量纲依赖于语料）；语义检索的分数是余弦相似度（通常在 0 - 1）。两者的数值不能直接相加，加出来的结果没有任何意义。因此，Hybrid Search 需要一个 Ranking Fusion（排序融合）机制，丢掉分数、只用排名。 Reciprocal Rank Fusion，正是用于解决这个问题。

3.18 Reciprocal Rank Fusion 倒数排名融合

Reciprocal Rank Fusion，简称 RRF。是一种根据文档在多个检索结果列表中的排名，计算综合得分的方法。与直接合并 BM25 分数和向量相似度不同，RRF 主要使用排名位置。

1. Rewards documents for being highly ranked on each list 按每个列表里的排名高低给分，排名越高奖励越多
2. Optional weights balance keyword and semantic rankings 可选权重用来平衡两路排序的比重

RRF 的基本思想

- 排名越靠前，贡献越大。
- 可以使用权重平衡关键词和语义排名。

- 当 $k = 0$ 时，第 1 名得 1 分，第 2 名得 0.5 分。可以看到，排名越靠后，得分越低。当 k 增大时，靠前和靠后排名之间的贡献差距会变得相对平缓。课件没有指定生产系统应该使用的固定 k 值；在实际实现中，该参数可以根据系统需求设置。
- 各个排名列表的得分相加，决定最终排名。

核心逻辑是：一篇文档在多个列表中排名越靠前，最终的融合得分通常越高。

An RRF Calculation

3.19 Weighting Semantic and Keyword Search 语义与关键词的加权

除了排名融合的方式，还有一个更直接的控制手段：给两路结果加权重 β。

- β = 0.8 的情形 — Semantic Search 占 80%，Keyword Search 占 20%。
- β = 0.7 的情形 — Semantic Search 占 70%，Keyword Search 占 30%。
- β 调的是比重 ——直接决定两路贡献的相对大小，作用在加权求和式的融合里。
- RRF 的 k / c 调的是排名差的压缩程度 ——它不改变两路谁更重要，只改变排名靠前这件事的收益有多陡。

β	Semantic Search	Keyword Search	适用倾向
0.8	80%	20%	问题措辞灵活、依赖语义理解
0.7	70%	30%	含精确串（编号、型号、专名），需要字面匹配

3.20 Returning the Top K Documents 返回前 K 篇文档

前面已经完成：Keyword Search、Semantic Search、Metadata Filtering、Ranking Fusion

接下来，系统需要决定：最终应该把多少篇文档返回给 LLM?

课件给出的答案是：Return the “top_k” highest-ranked documents. 即返回排名最高的 K 篇文档。

区分：课件重点讲的是检索结果如何融合并返回 Top-K，图中的完整 LLM 生成流程是与前面 RAG 架构内容的衔接。

为什么不返回全部文档

如果把检索到的所有文档都交给 LLM，可能带来：更高的上下文 Token 消耗、更多无关信息、更长的输入处理时间、更高的生成成本。因此，Top-K 的作用是限制最终进入后续处理阶段的候选文档数量。

K 越大越好吗

不一定。较小的 K 可能减少无关内容，但也可能遗漏有用信息。较大的 K 可以覆盖更多候选文档，但也可能引入噪声。因此，K 的选择需要结合检索质量、上下文预算和具体任务。

Hybrid RAG 的最终结论

Hybrid RAG search is still a mainstream and effective design. 混合检索仍然是一种主流且有效的 RAG 检索设计。

它不是简单地选择关键词检索或语义检索其中之一，而是将不同方法的优势组合起来。

总结

三种技术各自负责什么

技术	核心作用	典型方法
Metadata Filtering	严格约束候选文档	字段条件筛选
Keyword Search	精确词项相关性	BM25
Semantic Search	语义相关性	Embedding + Vector Similarity
Ranking Fusion	合并不同检索结果	RRF
Top-K Selection	选择最终返回的文档	排序截取

五个核心问题

1. 为什么需要 Retrieval？ 因为 LLM 需要从外部知识库获取与用户问题相关的信息。
2. BM25 为什么比简单词频更有效？ 因为它结合了词项重要性、词频饱和与文档长度归一化。
3. Embedding 为什么能用于语义检索？ 因为模型学习了可比较的向量表示，使相关文本具有较高的向量相似度。
4. RRF 为什么适合 Hybrid Search？ 因为它根据排名融合结果，不需要直接比较不同检索器的原始分数。
5. Top-K 的作用是什么？ 从融合后的排序中选出有限数量的高排名文档，供后续 RAG 流程使用。

核心结论：高质量 RAG 检索不只依赖一种匹配方式，而是通过关键词匹配、语义相似性、元数据约束与排序融合，共同找到最适合回答用户问题的文档。

Part 4 Retrieval Evaluation 检索评估

Part 3 解决了一个问题：如何从知识库中找到相关文档？ Keyword Search、BM25、Semantic Search、Hybrid Search，以及 RRF 排序融合。但找到文档并不代表检索效果一定好。

例如，两个 Retriever 都返回了 10 篇文档：

- Retriever A 返回了 8 篇相关文档。
- Retriever B 返回了 5 篇相关文档。

我们可能认为 A 更好，但如果 A 把真正重要的文档排在最后，而 B 把它们排在最前面，情况就没那么简单。

因此，Part 4 要解决三个核心问题：

1. Precision： 返回的文档有多少是相关的?
2. Recall： 所有相关文档中，有多少被成功找回?
3. MAP： 相关文档是否排在检索结果的前面?

4.1 Retrieval Quality Metrics 检索质量指标

三要素：查询、排序结果、标准答案。

课件元素	含义
Query: The specific information needs being evaluated	被评估的那个具体信息需求
Ranked results: Documents returned in ranked order	按排名返回的文档列表
Ground truth: Documents labeled relevant or irrelevant	被标注为「相关 / 不相关」的文档

Ground truth 是评估的前提，没有人工标注的相关性判断，任何指标都算不出来。常见来源：人工标注（最准、最贵）、用户行为日志（点击/停留，规模大但有噪声）、合成标注（用模型生成，最便宜但可能有偏）。

To evaluate a retriever, you need relevant judgments. 评估检索器需要相关性判断，指的是一批查询、每个查询一批标注，不是单点结论。这一步的成本常被低估，标注需要人工，规模越大成本越高，但它是所有指标的地基。

4.2 Precision and Recall 精确率与召回率

指标	公式	课件追问
Precision　精确率	Relevant retrieved / total retrieved	How many returned documents are relevant?
Recall　召回率	Relevant retrieved / total relevant	How many relevant documents were returned?

Precision：你找回来的东西，有多少是对的?

Recall：所有对的东西，你找回来了多少?

4.2.1 The Precision - Recall Trade-off 精确率与召回率的取舍

运行	Retrieved	Relevant	Precision	Recall
First Run	Retrieved: 12 Documents	Relevant: 8 Documents	8/12 = 66.7%	8/10 = 80%
Second Run	Retrieved: 15 Documents	Relevant: 9 Documents	9/15 = 60%	9/10 = 90%

提高一个必然牺牲另一个，为什么会发生这种变化?

第二次检索多返回了 3 篇文档，但只有 1 篇是相关的。

因此：Recall 提高，因为多找回了一篇相关文档；Precision 降低，因为额外引入了两篇无关文档。

Precision penalizes returning irrelevant documents. Precision 惩罚返回无关文档。

Recall penalizes leaving out relevant documents. Recall 惩罚遗漏相关文档。

如果检索器能更准确地找到相关文档，两者也可能同时提高。

4.3 Evaluating the Top K Results 评估前 K 个结果

1. Retrieval metrics are influenced by how many documents the retriever returns（指标受返回数量影响）。
2. Metrics are discussed in terms of top-k documents（指标都以 top-k 的方式讨论）。

返回得越多，Precision 的分母越大（一般会被拉低），Recall 的分子越大（一般会被抬高）。

为什么 K 会影响指标? Retrieval metrics are influenced by how many documents the retriever returns.

因为 K 改变了被评估的文档集合。当 K 增大时：

- Recall@K 不会下降，因为已经找到的相关文档仍然保留。
- Precision@K 可能上升，也可能下降，取决于新增文档是否相关。

实际应用应该如何选择 K? Evaluate at a cutoff that matches the number of results your application uses.

评估时使用的 K，应该与实际应用使用的结果数量一致。

例如，如果 RAG 系统只把前 5 篇文档交给 LLM，那么 Precision@5、Recall@5 就比只报告 Precision@100 更贴近实际使用情况。

4.3.1 Precision and Recall at K 前 K 个结果的精确率与召回率

同一个排序列表，换个截断位置，指标就变了。

截断	课件标注	计算	结果
Precision@5	2 out of 5	前 5 位命中 2 个相关	40%
Precision@10	6 out of 10	前 10 位命中 6 个相关	60%

Precision@K 不一定随着 K 增大而下降。

Evaluate at a cutoff that matches the number of results your application uses.

如果后面新增的文档中有较高比例的相关文档，Precision@K 反而可能上升。

4.4 Mean Average Precision 平均精度均值

前面的 Precision@K 只关心前 K 篇文档里有多少篇相关。但它没有充分体现相关文档具体排在什么位置。

例如：

- Retriever A：相关文档排在第 1、2、3 位。
- Retriever B：相关文档排在第 4、5、6 位。

假设只有这 3 篇相关文档，两者在 K=6 时，虽然两个指标相同，但显然 A 把相关文档排得更靠前。这就是 Average Precision（AP）要解决的问题。MAP 解决的是光看 Precision@k 看不出排序好坏这件事。

MAP averages Average Precision (AP) across queries. MAP 把各查询的 AP 再做平均。

AP 步骤

1. 找出所有相关文档所在的排名。
2. 将这些排名位置对应的 Precision@K 相加。
3. 除以全部相关文档的数量。

4.5 Using Retrieval Metrics 检索指标怎么用

指标	课件定位	回答的问题
Recall@K	Coverage of the relevant documents	“Did we retrieve the relevant information?”
Precision 与 MAP	Result relevance and ranking quality	“How much of what we retrieved is relevant?” / “Are the relevant results ranked near the top?”

不同指标与实际检索系统的调优目标

1. Recall@K：Coverage

Coverage of the relevant documents. 相关文档的覆盖程度。

核心问题：Did we retrieve the relevant information? 如果 Recall@K 很低，说明检索系统遗漏了大量相关文档。

可能的改进方向包括：

- 增加候选召回数量。
- 引入 Semantic Search 补充关键词检索。
- 调整过于严格的非权限类过滤条件。

但权限过滤不能为了提高 Recall 而随意放宽。

2. Precision：Result Relevance

How much of what we retrieved is relevant? Precision 衡量返回结果中相关文档所占的比例。

如果 Precision 很低，说明检索结果包含较多无关内容。

可能的改进方向：

- 调整 Top-K。
- 改进关键词与语义检索的匹配策略。
- 使用更有效的相关性排序或过滤方法。

3. MAP：Ranking Quality

Are the relevant results ranked near the top? MAP 关注相关文档是否排在前面。

如果系统 Recall 较高，但 MAP 较低，可能意味着：相关文档虽然已经被检索出来，却排在较靠后的位置。

这时可以考虑调整：

- BM25 参数。
- Embedding Model。
- Hybrid Search 的权重 β。
- RRF 的融合设置。

这些属于基于指标的工程调优思路，并非课件保证有效的固定操作。

4. 指标与调优方向总结

指标	主要关注	偏低时可能的问题
Recall@K	覆盖率	遗漏相关文档
Precision@K	结果纯度	无关文档太多
MAP	排序质量	相关文档排得靠后

为什么要在同一组查询上比较?

Compare settings on the same labeled query set.

如果 Retriever A 使用一组简单查询，而 Retriever B 使用另一组困难查询，那么两者的指标无法公平比较。

正确的实验方法是：固定相同的 Query Set 和 Ground Truth，再比较不同的检索设置。通过比较，可以判断哪一种设置更符合应用需求。评估指标的意义不只是给检索器打分，更重要的是帮助选择和改进检索配置。

4.6 全讲要点

1. Knowledge beyond training

RAG can provide private, recent or specialized information.

RAG 可以补充模型训练之外的私有、近期或专业信息。

2. Retrieval before generation

Relevant documents ground the answer in supplied evidence.

先检索相关文档，再基于检索证据生成回答。

3. Retrieval methods

Keywords match terms, embeddings match meaning, and filters enforce criteria.

关键词匹配词项，Embedding 匹配语义，Metadata Filter 执行条件约束。

4. Evaluation guidance

Use recall and precision to compare retrieval settings.

使用召回率与精确率等指标比较不同检索设置。

为什么需要外部知识 → 如何把外部知识提供给 LLM → 如何找到相关知识 → 如何判断找得好不好。

Lecture 7 Retrieval Optimization and Agentic RAG Systems 检索优化与智能体 RAG

Part 1 Scalable Retrieval 可扩展检索

1.1 RAG 工作流与检索方法回顾

1.1.1 RAG Workflow

传统 LLM 与 RAG 的主要区别在于是否引入外部知识检索。

- Direct LLM Use： Prompt → LLM → Response。模型直接利用自身参数中存储的知识生成答案。
- RAG System： Prompt → Retriever → Relevant Documents → Augmented Prompt → LLM → Response。检索器从外部知识库（Knowledge Base）获取相关文档，将其与原始问题组合为增强提示，再交给 LLM 生成答案。

RAG 将外部知识引入生成过程，因此检索质量与检索效率都会影响最终系统表现。

1.1.2 三种典型检索方法 Typical Retrieval Methods

方法	核心原理	特点
Keyword Search（关键词检索）	根据查询与文档之间的词项匹配进行检索	依赖词汇匹配

方法	核心原理	特点
Semantic Search（语义检索）	将查询和文档编码为向量，根据语义相似度检索	可以匹配不同措辞表达的相近含义
Hybrid Search（混合检索）	结合关键词检索与语义检索结果	将两路结果融合为统一排序

关键词与语义检索分别返回 50 篇文档，经过元数据过滤后分别剩余 35 篇和 30 篇，再融合为单一排序。

其中，Metadata Filter（元数据过滤） 根据文档属性进一步筛选候选结果；Hybrid Ranking（混合排序） 则将不同检索路径的结果整合，而不是简单地分别返回两组列表。

1.1.3 语义检索流程 Semantic Search

语义检索以向量表示为基础，主要包括三个阶段：

1. Embedding Model（嵌入模型）： 将文档和查询分别编码为向量，使其能够在统一的向量空间中进行比较。
2. Vector Space（向量空间）： 语义相近的文档和查询通常具有更接近的向量表示。
3. Rank by Distance（距离排序）： 根据向量距离或相似度对文档排序，返回与查询最接近的结果。

这一流程构成后续 KNN、ANN、NSW 和 HNSW 的共同基础。不同方法的主要区别不在于是否使用向量，而在于如何高效定位距离查询向量最近的候选文档。

1.2 精确向量检索与规模挑战 KNN & Scaling Challenges

1.2.1 KNN：精确最近邻检索

KNN（k-Nearest Neighbors，k 近邻）通过比较查询向量与所有文档向量之间的距离，确定最近的 k 个文档。检索流程：

1. Embed documents and the query： 将文档和查询编码为向量。
2. Compare with every document： 计算查询向量与每个文档向量的距离。
3. Rank by distance： 按距离从近到远排序。
4. Return the nearest k documents： 返回距离最近的 k 个文档。

在完整扫描的前提下，KNN 能够找到全局最近邻，但必须对整个文档集合进行距离计算。

设知识库包含 N 个文档向量，每次查询需要：N次向量距离比较。因此，距离计算的时间复杂度为：O(N)

这里的线性复杂度针对距离比较阶段；完整检索还包含编码和结果选择等操作。

1.2.2 规模扩展问题 Scaling Challenges

随着知识库规模扩大，精确扫描的比较次数同步线性增长。

示意图：仅表示距离比较次数与文档数量的线性关系。

文档向量可以提前计算并存储，但每次新查询仍需要与全部文档向量比较。因此，主要的扩展性瓶颈是逐一比较全部文档，而非文档向量的初始编码。

1.3 近似最近邻检索 Approximate Nearest Neighbors, ANN

ANN 的核心思想是通过预先建立的索引（Index）缩小搜索范围，只访问部分候选向量，而不对所有文档执行完整扫描。三个基本特征：

- Searches fewer candidates using an index： 利用索引减少候选文档数量。
- Trades exactness for speed： 通过放弃严格的最近邻保证来提高检索速度。
- The closest result is not guaranteed： 检索结果可能不是全局最近邻。

ANN 是一类近似检索方法，而 NSW、HNSW 是其中基于邻近图（Proximity Graph）的具体实现。

KNN 与 ANN 对比

比较维度	KNN（精确扫描）	ANN
搜索范围	所有文档向量	部分候选向量
索引	不依赖近似索引	利用预建索引
最近邻保证	保证全局最近邻	不保证全局最近邻
大规模检索	线性扫描成本较高	通常显著更快
主要取舍	精确性优先	速度与召回率平衡

ANN 的性能需要同时考虑两个维度：

- Search Speed（检索速度）： 完成一次查询所需的时间或计算量。
- Recall（召回率）： 近似检索找到的结果与真实相关结果或精确最近邻结果的重合程度，具体取决于评估定义。

因此，ANN 的优化目标不是单纯追求最快，而是在可接受的召回率下减少检索成本。

1.4 可导航小世界图 Navigable Small World, NSW

NSW 是一种基于图结构的 ANN 方法，通过建立文档向量之间的邻近关系，将全局扫描转化为局部图导航。

1.4.1 邻近图的构建 Proximity Graph Construction

邻近图由节点（Nodes）和边（Edges）构成：

- Node： 代表一个文档向量。
- Edge： 表示两个向量之间建立的邻近连接。

构建过程包括：

1. Compute distances between all document vectors： 计算文档向量之间的距离，确定邻近关系。
2. Connect each node to its nearest neighbors： 将每个节点与其附近的若干节点连接。
3. Insert document vectors incrementally： 支持增量插入新的文档向量。
4. Traverse the graph along edges： 查询时沿图中相邻节点之间的边进行搜索。

前面三个要点描述索引构建与维护，第四个要点描述索引在查询阶段的使用。

邻近图的构建具有一定计算成本，但可以预先完成，并在后续多次查询中重复使用。

1.4.2 NSW 检索算法：贪心游走 Greedy Walk

NSW 的检索过程，四个阶段。

阶段一：选择入口（Entry Point）

从图中随机选择一个节点作为初始候选节点。该节点不需要接近查询向量。

阶段二：比较相邻节点（Neighbor Comparison）

计算当前节点及其相邻节点与查询向量的距离，只考虑当前节点直接连接的邻居。

若存在距离查询更近的邻居，则沿对应的边移动，并将其更新为当前候选节点。

阶段三：重复局部搜索（Iterative Search）

在新的候选节点上重复相同操作：

$v_{t+1} = \arg\min_{u \in N(v_t)} d(u, q)$

其中：

- q：查询向量（Query Vector）
- v_t：当前候选节点
- $N(v_t)$：当前节点的邻居集合
- $d(u, q)$：邻居节点与查询向量之间的距离

只有当所选邻居比当前节点更接近查询时，才执行移动。

阶段四：终止搜索（Stopping Condition）

当所有相邻节点都不比当前节点更接近查询向量时，终止当前贪心游走。

$d(v_t, q) \leqslant d(u, q), \quad \forall u \in N(v_t)$

此时得到的是局部最优（Local Optimum），不一定是全局最近邻（Global Nearest Neighbor）。

1.4.3 NSW 的优势与局限

NSW 通过沿图边访问候选节点，通常能够以更少的距离比较找到接近查询的文档，并可用于寻找多个近邻。但由于贪心搜索只考虑当前节点的直接邻居，搜索可能在局部最优位置停止，即使图中其他区域存在更接近查询的节点。

因此，NSW 与精确 KNN 的根本区别是：

KNN 比较全部向量以确定全局最近邻；NSW 通过邻近图进行局部导航，以较少的比较次数寻找近似最近邻。

1.5 分层可导航小世界图 Hierarchical Navigable Small World, HNSW

HNSW 在 NSW 邻近图的基础上引入层级结构（Hierarchy），通过不同密度的图实现从粗粒度定位到细粒度检索。

1.5.1 HNSW 的层级结构

HNSW 由稀疏上层、中间层和稠密底层组成。

层级	结构特征	检索任务
Sparse Upper Layer	节点数量少、图稀疏	Coarse Navigation：粗粒度导航
Intermediate Layer	节点数量增加	Narrow the Search：缩小搜索范围
Dense Base Layer	包含全部文档向量	Local Refinement：局部精细检索

Start with fewer nodes. Refine the search as the layers get denser.

其核心思想是从节点较少的上层开始搜索，再逐层进入更密集的图，逐步提高定位精度。

1.5.2 分层索引构建 Building the Hierarchy

课件以包含 1,000 个文档向量的数据集为例，构建三层邻近图：

上层节点更少，底层包含全部文档向量。图中数量为课件示例，并非固定配置。

- Layer 3： 随机选择 10 个向量构建邻近图，用于最高层的快速导航。
- Layer 2： 随机选择 100 个向量构建邻近图，用于中间层搜索。
- Layer 1： 包含全部 1,000 个向量，用于最终检索。

上层节点也是下层节点的一部分，使搜索能够在不同层级之间连续进行。课件采用的 10、100、1,000 是用于说明稀疏到稠密结构的示例规模。

1.5.3 HNSW 检索流程 Top-down Search

HNSW 采用自顶向下（Top-down）的搜索方式。

第一阶段：Layer 3 — Coarse Navigation

从最高层随机选择一个入口节点，在稀疏邻近图中搜索距离查询较近的候选节点。由于上层节点较少，可以通过较少的比较完成初步定位（Make Big Jumps Early）。

第二阶段：Layer 2 — Intermediate Search

以上一层找到的最佳候选节点作为本层的起点，在节点数量更多的邻近图中继续搜索，进一步缩小与查询向量的距离。

第三阶段：Layer 1 — Local Refinement

以 Layer 2 的最佳候选作为起点，在包含全部文档向量的底层图中执行最终搜索，得到近似最近邻结果。

这一过程的关键是：

$\text{Best Candidate}_{L3} \rightarrow \text{Entry Point}_{L2}$

$\text{Best Candidate}_{L2} \rightarrow \text{Entry Point}_{L1}$

每一层都继承上一层的搜索结果，而不是重新随机选择入口。

因此，HNSW 与 NSW 使用相似的邻居比较机制，但 HNSW 通过多层图组织搜索过程，减少在大规模底层图中进行无效导航的成本。

1.5.4 检索复杂度与性能 Complexity & Performance

KNN 的线性增长 vs HNSW 的近似对数增长

方法	检索增长趋势	主要原因
KNN	$O(N)$	与全部文档向量比较
HNSW	通常近似 $O(\log N)$	利用稀疏上层导航和逐层细化减少候选访问

这里的 $O(\log N)$ 是课件对 HNSW 常见检索表现的近似描述，不是所有数据分布、索引配置和召回率要求下都成立的严格保证。

HNSW 的实际检索速度还取决于：

- Index Settings（索引设置）： 影响图结构与搜索范围。
- Recall Targets（召回率目标）： 对结果准确性的要求越高，通常需要检查更多候选节点。

因此，HNSW 能够显著改善大规模向量检索的扩展性，但仍需要在速度、召回率与索引构建成本之间进行权衡。

1.6 KNN、NSW 与 HNSW 综合比较

维度	KNN	NSW	HNSW
检索类型	精确检索	近似检索	近似检索
数据结构	向量集合	单层邻近图	多层邻近图
搜索策略	全量距离比较	单层贪心游走	自顶向下逐层搜索
入口节点	不需要	随机图节点	最高层入口节点
候选范围	全部向量	当前节点及邻居	各层的局部候选
最近邻保证	有	无	无
扩展性	线性扫描	减少距离比较	通常近似对数增长
索引构建成本	无需建立近似图索引	需要预建邻近图	需要预建多层邻近图
主要局限	大规模查询成本高	可能陷入局部最优	索引复杂，性能受参数和召回目标影响

1.7 总结

本部分介绍的优化路径可以归纳为三个层次：

1. KNN → ANN：减少比较数量。 精确 KNN 对所有文档向量进行距离计算，查询成本随知识库规模线性增长。ANN 通过索引缩小候选范围，以牺牲全局最近邻保证换取更高的检索效率。
2. ANN → NSW：通过图结构组织候选搜索。 NSW 将文档向量组织为邻近图，查询时沿连接边进行局部贪心搜索，从而避免全量扫描，但可能因局部最优而错过全局最近邻。
3. NSW → HNSW：通过层级结构进一步优化导航。 HNSW 将邻近图划分为从稀疏到稠密的多个层级，先在上层快速定位，再在下层精细检索，通常能够获得更好的大规模检索效率。

最终结论： ANN 的优势来自预计算索引与减少在线距离比较。其代价是更高的索引构建成本，以及检索速度与召回率之间的权衡。

从系统执行阶段来看：

$\underbrace{ \text{Document Vectors} \rightarrow \text{Build Index} }_{\text{Offline Index Construction}}$

$\underbrace{ \text{Query Vector} \rightarrow \text{Visit Candidates} \rightarrow \text{Top-}k }_{\text{Online Retrieval}}$

将较高的计算成本前移至离线索引构建阶段，使在线查询不再需要遍历全部文档向量，是本部分可扩展检索技术的共同设计思路。

Part 2 分块、查询改写与检索编码器

Chunking, Query Rewriting and Retrieval Encoders

本部分围绕 RAG 检索质量的三个核心问题展开：文档如何组织、查询如何表达，以及查询与文档如何匹配。

- Chunking（文档分块）：确定知识库中的基本检索单元，平衡信息完整性、相关性与存储成本。
- Query Rewriting（查询改写）：消除用户输入中的歧义与冗余，使查询更适合检索。
- Retrieval Encoders（检索编码器）：通过不同的编码与交互机制，平衡检索精度、速度及存储开销。

三者分别作用于文档侧、查询侧和匹配阶段，共同决定最终检索证据的质量。

2.1 文档分块 Document Chunking

2.1.1 分块的必要性

Chunking 是将长文档划分为较小的文本单元的过程。其核心目标不是单纯缩短文本，而是使每个检索单元包含足够的信息，同时尽量减少无关内容。

分块的作用

目的	说明
Respect embedding token limits	避免文档超过嵌入模型的输入长度限制
Improve retrieval relevance	减少不同主题之间的语义干扰，提高检索相关性
Send focused context to the LLM	向生成模型提供聚焦且充分的证据，减少上下文窗口浪费

不分块的问题（Indexing without Chunking）

假设知识库包含 1,000 本书，每本书整体编码为一个向量。这种方式虽然减少了向量数量，却存在两个主要问题。

首先，整本书可能包含多个主题，而单个向量只能表达文档的整体语义，难以准确表示具体章节或段落。不同主题的信息混合后，可能降低特定查询的匹配效果。

其次，即使检索到相关书籍，也可能需要将大量无关内容送入 LLM，占用有限的上下文窗口。

因此，分块的关键是将检索粒度由整个文档细化为更有意义的局部内容。

2.1.2 分块粒度与大小权衡

文档可以按不同粒度划分：

Book → Page → Paragraph → Sentence

随着粒度减小，检索单元通常更加具体，但单个片段包含的上下文也会减少。

分块大小	主要问题	对检索的影响
Too large（过大）	包含多个主题和大量无关内容	语义混杂，匹配不够精确
Too small（过小）	缺少必要上下文	语义不完整，匹配能力减弱
Just right（适中）	保留回答问题所需的信息	兼顾相关性与信息充分性

Useful evidence contains enough information to answer the question without large amounts of unrelated material.

分块质量的核心标准是有效证据应包含足以回答问题的信息，同时避免大量无关内容。

因此，最佳分块大小并非固定值，而取决于文档结构、查询需求和模型输入限制。

2.1.3 主要分块方法

1. 固定长度分块（Fixed-size Chunking）

按照预设字符数划分文档，不考虑文本语义或结构。假设每块包含 $C = 250$ 个字符，且没有重叠，则第 k 个块的字符范围为：

其中，N 表示文档总字符数。

优点：实现简单、计算成本低、块大小可预测，适合作为分块策略的基线。

缺点：切分位置可能位于句子或概念内部，破坏语义完整性。

2. 重叠分块（Overlapping Chunking）

在固定长度分块的基础上，使相邻文本块共享部分内容，以减少边界处的上下文损失。设：

- C：Chunk Size，块长度。
- O：Overlap Size，重叠长度。
- S：Stride，分块步长。

三者关系为：

$ S=C-O $

重叠比例为：

$r=\frac{O}{C} $

课件采用：

$C=250,\quad O=25 $

因此：

$S = 250 - 25 = 225$

$r=\frac{25}{250}=10% $

Chunk	字符范围	与前一块的重叠
Chunk 1	1 - 250	无
Chunk 2	226 - 475	226 - 250
Chunk 3	451 - 700	451 - 475

对于从 1 开始计数的字符位置，第 k 个块的起点为：

${Start}_k=1+(k-1)(C-O) $

重叠分块能够保留边界附近的上下文，但也增加了向量数量、存储成本及检索结果的冗余。

忽略最后一个不足整块的片段，对于较长文档，重叠引起的存储量放大比例近似为：

$Storage Factor\approx\frac{C}{C-O} $

在课件示例中：

$\frac{250}{225}\approx1.111$

即相较于不重叠分块，存储的文本量约增加 11.1%。

3. 递归字符切分（Recursive Character Splitting）

递归字符切分优先利用文档的自然结构划分文本，而不是直接按照固定字符位置截断。

典型的分隔符优先级可以表示为：

Paragraph → Line → Sentence → Character

处理过程是先使用较高层级的分隔符；如果某个片段仍超过长度限制，再使用更细粒度的分隔符继续切分。

主要特点：

- 尽可能保留段落和句子的完整性。
- 不同文本块的长度可以不同。
- 相较于固定长度分块，更符合文档的自然结构。

4. 结构感知分块（Structure-aware Chunking）

对于具有明确结构的文档，应优先利用其原有结构信息确定分块边界。

文档类型	分块依据
HTML	标题标签、段落标签，如 <h1>、<p>
Python	函数定义、类结构
Markdown	标题层级、段落结构
普通文本	换行、段落或递归字符分隔符

例如，Python 代码按函数定义进行分块，可以保留函数签名、参数和实现逻辑之间的联系，避免将一个完整函数切割成无法独立理解的片段。

关键原则：分块应尽可能遵循文档已有的结构边界。

2.1.4 分块边界与语义完整性

Chunk Boundaries Can Change Meaning 分块不仅影响检索精度，还可能改变文本的原始含义。

That night she dreamed, as she did often, that she was finally an Olympic champion.

原句表达的是“她梦见自己成为奥运冠军”，并非真实发生的事件。

如果分块后仅保留：She was finally an Olympic champion. 则可能被误解为已经发生的事实。

相反，保留：She dreamed … She was finally an Olympic champion. 可以维持原始语义。

因此，分块应尽量保留影响事实判断的限定信息，例如时间、否定、条件、假设以及主语与谓语之间的关系。

这一问题也是引入语义分块与 LLM 分块的重要原因。

2.1.5 语义分块（Semantic Chunking）

语义分块根据相邻文本的语义相似程度确定边界，而非使用固定字符长度。其基本流程为：

1. 将当前文本块与下一句分别编码为向量。
2. 计算二者的语义距离。
3. 根据预设阈值判断是否合并。
4. 逐句重复，直至处理完整篇文档。

课件使用 Cosine Distance（余弦距离） 作为判定依据。

首先计算余弦相似度：

$\operatorname{sim}(\mathbf a,\mathbf b) = \frac{\mathbf a\cdot\mathbf b} {|\mathbf a||\mathbf b|} $

再定义余弦距离：

$d_{\cos}(\mathbf a,\mathbf b) = 1-\operatorname{sim}(\mathbf a,\mathbf b) $

其中：

- $\mathbf{a}$：当前 Chunk 的语义向量。
- $\mathbf{b}$：下一句的语义向量。
- τ：预设距离阈值。

简化判定规则为：

$\begin{cases} d_{\cos}(\mathbf a,\mathbf b)<\tau &\Rightarrow \text{Add to current chunk}\[4pt] d_{\cos}(\mathbf a,\mathbf b)\geq\tau &\Rightarrow \text{Start new chunk} \end{cases} $

即：

- 距离低于阈值：语义相近，合并。
- 距离达到或超过阈值：语义差异较大，创建新块。

注意这里使用的是 Distance，不是 Similarity，因此数值越小表示语义越接近。

以加拿大的枫糖浆、山脉与自然景观为例，说明语义分块可以将相关句子组织在一起，不必按照固定字符位置截断。

优点：能够保留相关语义，减少不合理的文本边界。

缺点：需要额外的 Embedding 和 Similarity/Distance Computation，增加预处理时间与计算成本。

Evaluate whether better retrieval justifies the extra computation.

因此，语义分块是否值得采用，应根据实际检索质量提升与计算成本进行评估。

2.1.6 基于 LLM 的分块 LLM-based Chunking

LLM-based Chunking 直接利用语言模型理解文档内容，并按照指定规则生成文本块。

与语义分块依赖向量距离不同，LLM 可以根据自然语言指令识别概念、主题转换和内容之间的逻辑关系。

典型指令包括：

- 将同一概念的相关内容保留在同一块中。
- 当文档开始讨论新主题时，创建新块。
- 尽量保留每个片段的语义完整性。

处理流程为：

$\text{Document} \xrightarrow{\text{LLM}} {C_1,C_2,\ldots,C_n} $

其中 C_i 表示模型生成的第 i 个文本块。这种方法能够进行更复杂的语义判断，但需要额外的 LLM 推理成本。

上下文感知增强（Context-aware Enrichment）

除了决定分块边界，LLM 还可以为已有的 Chunk 生成简短的上下文描述。

课件示例：

Original Fragment	Added Context
Reflections on Launching Our First App	App-launch title
After six months … We launched our first app	Launch announcement
The biggest lessons were around user feedback	Lessons learned
Thanks to Daniel Townsend	Contributors

其核心思想是：为缺乏上下文的文本片段补充必要的主题或角色信息，使其在检索时更容易被正确理解。

例如，一个只包含人名的致谢片段，如果缺乏文档背景，可能难以匹配“项目贡献者”相关查询。增加 Contributors 标签后，可以改善其语义表达。

上下文增强不要求必须采用 LLM 分块，也可以与固定长度、递归或语义分块结合使用。

成本发生在哪个阶段?

阶段	操作	成本特点
Before queries arrive	LLM 为 Chunk 添加上下文，并建立索引	增加离线预处理成本
When a query arrives	检索增强后的 Chunk，再生成答案	不需要为添加 Chunk 上下文额外调用 LLM

因此，上下文增强的主要成本发生在索引构建阶段，而不是每次查询时。

需要注意，这并不代表整个 RAG 查询过程没有 LLM 调用；生成最终答案仍然可能需要调用 LLM。

2.1.7 分块策略的综合比较

方法	切分依据	主要优势	主要局限
Fixed-size	字符数量	简单、高效、适合作为基线	容易破坏语义
Overlapping	固定长度 + 重叠区域	减少边界信息丢失	增加存储与冗余
Recursive	自然分隔符	较好保留文本结构	块长度不一致
Structure-aware	文档结构标记	保留逻辑结构	依赖文档格式
Semantic	向量语义距离	保留相关主题	额外 Embedding 成本

方法	切分依据	主要优势	主要局限
LLM-based	模型语义理解	更灵活的概念划分	推理成本与复杂度较高

选型建议：先建立简单基线，再根据检索效果决定是否引入更复杂的方法。

固定长度分块适合作为初始方案；语义分块和 LLM 分块可能改善效果，但应通过实验验证收益。上下文增强则可以与不同分块策略结合使用。

2.2 查询改写 Query Rewriting

分块优化的是文档侧，而查询改写优化的是用户输入侧。

其目标是将不明确、冗余或不适合检索的自然语言输入转化为更清晰的查询，同时保留用户的原始意图。

2.2.1 查询改写的必要性

用户在对话中经常使用省略、指代或非正式表达。

课件示例：Hey, can you tell me about that climate thing we talked about?

that climate thing 缺少明确的主题信息。如果检索器无法获取之前的对话内容，就难以确定用户实际需要检索什么。

因此，Query Rewriting 需要利用已有对话上下文补全指代；如果上下文仍不足，则应要求用户澄清，而不是自行猜测主题。

2.2.2 查询改写的主要操作

课件以医学文档检索为例。

原始查询：My dog yanked hard on the leash. Three days later, my shoulder is still numb and my fingers are all pins and needles.

改写后：Sudden shoulder traction; shoulder numbness and finger tingling for three days. Potential causes?

改写过程主要包括四项操作：

操作	英文术语	目的
消除歧义	Clarify ambiguous phrases	明确指代和模糊表达
术语规范化	Use domain terminology	使查询与专业文档用语一致
同义词扩展	Add synonyms	增加不同表述之间的匹配机会
删除冗余信息	Remove distracting information	减少无关信息对检索的干扰

在示例中，pins and needles 被规范化为 finger tingling，同时保留了症状持续三天这一重要信息。

查询改写的关键约束是：Preserve the user's meaning（保留用户原意）。

改写可以改变表达方式，但不应添加未经用户提供或上下文支持的事实，也不能改变用户真正想要解决的问题。

2.2.3 查询改写的工作流程

$Q_{\text{original}} \xrightarrow{\text{Query Rewriter (LLM)}} Q_{\text{optimized}} \xrightarrow{\text{Retriever}} D_{\text{relevant}}$

其中：

- Q_{original}：用户原始查询。
- $Q_{\text{optimized}}$：改写后的检索查询。
- D_{relevant}：检索到的相关文档。

查询改写发生在检索之前，属于在线处理阶段。

2.2.4 收益与代价

Potential Benefits	Trade-offs
Clearer queries：查询表达更清晰	Extra LLM latency and cost：增加调用延迟与费用
Better retrieval relevance：提高检索相关性	Risk of changing user intent：可能偏离用户原意

查询改写特别适用于对话式检索、指代较多的输入以及口语表达与专业文档用词差异较大的场景。

对于已经明确、简洁且包含准确术语的查询，额外改写不一定带来明显收益。

因此，是否启用 Query Rewriting 同样需要考虑检索质量与额外在线成本之间的权衡。

2.3 检索编码器 Retrieval Encoders

检索编码器决定查询与文档如何被表示和比较。

三种主要架构：Bi-encoder、Cross-encoder 和 ColBERT。

三者的区别在于编码方式、交互粒度以及计算与存储成本。

2.3.1 双编码器 Bi-encoder

Bi-encoder 将 Query 和 Document 分别编码为独立的语义向量。

$\mathbf q=E_q(Q) $

$\mathbf{d}_i = E_d(D_i)$

其中，E_q 与 E_d 分别表示查询编码器和文档编码器，两者可以共享权重。

随后，通过向量相似度计算相关性：

$s(Q,D_i)= \frac{\mathbf q\cdot\mathbf d_i} {|\mathbf q||\mathbf d_i|} $

对于大规模知识库，通常使用 ANN（Approximate Nearest Neighbor）快速搜索相近向量。

**关键特点 **

Precomputed Document Vectors（文档向量预计算）：文档可以在查询到来之前完成编码，并存储在向量数据库中。

因此：

- Offline：编码全部文档并建立索引。
- Online：仅编码新查询，再执行 ANN 搜索。

这种机制避免了每次查询都重新处理全部文档，具有较高的检索速度和可扩展性。

但由于 Query 与 Document 独立编码，模型在编码过程中无法直接建立两者之间的细粒度 Token 交互。

适用场景：大规模语义检索和第一阶段候选文档召回。

2.3.2 交叉编码器（Cross-encoder）

Cross-encoder 将 Query 与 Document 拼接后共同输入模型，直接生成相关性分数。

$s(Q, D_i) = f_{\text{cross}}(Q, D_i)$

与 Bi-encoder 不同，Cross-encoder 可以在编码过程中建模查询与文档之间的上下文交互。

课件示例：Query：Great places to eat in New York

Document	Relevance Score
Manhattan restaurants	0.7
New York subway	0.12
Brooklyn food scene	0.85

因此，排序结果为：$D_{\text{Brooklyn}} > D_{\text{Manhattan}} > D_{\text{Subway}} $

Cross-encoder 能够结合查询意图与文档语义进行更细致的相关性判断，而不只是匹配共同出现的词语。

优势

- 能直接建模 Query - Document 的上下文交互。
- 通常可以提供比 Bi-encoder 更精细的相关性评分。
- 适合对初步检索结果进行重新排序。

局限

Cross-encoder 必须针对每一个 Query - Document Pair 单独计算分数。

若候选文档数量为 N，则一次查询需要进行：$N_{\text{pair evaluations}} = N$次模型评分。

此外，相关性分数依赖当前 Query，因此不能像 Bi-encoder 的文档向量一样提前独立计算。

当文档规模达到百万级或十亿级时，直接使用 Cross-encoder 对全部文档评分通常成本过高。

适用场景：对较小候选集进行 Reranking（重排序），而非大规模初始检索。

2.3.3 ColBERT：Token-level Matching

ColBERT 采用 Late Interaction（后期交互）机制，在 Bi-encoder 的独立编码与 Cross-encoder 的联合交互之间取得平衡。核心思想是：分别编码 Query 和 Document，但保留每个 Token 的向量，而不是将整个文本压缩成单一向量。

假设查询包含 m 个 Token，文档包含 n 个 Token，则：

$Q={\mathbf q_1,\mathbf q_2,\ldots,\mathbf q_m} $

$D={\mathbf d_1,\mathbf d_2,\ldots,\mathbf d_n} $

其中：

- $\mathbf{q}_i$：查询中第 i 个 Token 的上下文向量。
- $\mathbf{d}_j$：文档中第 j 个 Token 的上下文向量。

与 Bi-encoder 每个文档只生成一个向量不同，ColBERT 为文档中的多个 Token 分别生成向量。

2.3.4 Token 相似度矩阵

ColBERT 首先计算每个 Query Token 与各个 Document Token 的相似度。

$S_{ij} = \text{sim}(\mathbf{q}_i, \mathbf{d}_j)$

如果采用点积作为相似度函数：

$ S_{ij}=\mathbf q_i^\top\mathbf d_j $

对于已归一化的向量，点积等价于余弦相似度。

所有 Token 两两比较后形成相似度矩阵：

$\mathbf S= \begin{bmatrix} S_{11}&S_{12}&\cdots&S_{1n}\\ S_{21}&S_{22}&\cdots&S_{2n}\\ \vdots&\vdots&\ddots&\vdots\\ S_{m1}&S_{m2}&\cdots&S_{mn} \end{bmatrix}$

矩阵维度为：$\mathbf S\in\mathbb R^{m\times n} $。每一行对应一个 Query Token，每一列对应一个 Document Token。

例如，查询中的 eat 可能与文档中的 cuisine 具有较高的语义相似度，即使二者并非相同单词。

这使 ColBERT 能够捕获比整篇文档向量更细粒度的语义匹配关系。

2.3.5 MaxSim Scoring

MaxSim 是 ColBERT 的核心评分机制。对于每一个 Query Token，找到与其最相似的 Document Token，并将这些最大相似度相加。

其公式为：

${ \operatorname{MaxSim}(Q,D) = \sum_{i=1}^{m} \max_{1\leq j\leq n} \operatorname{sim}(\mathbf q_i,\mathbf d_j) }$

如果使用点积：

$\operatorname{MaxSim}(Q,D) = \sum_{i=1}^{m} \max_{1\leq j\leq n} \mathbf q_i^\top\mathbf d_j $

公式包含两个运算：

1. Max：对每个 Query Token，在所有 Document Token 中寻找最佳匹配。
2. Sum：将所有 Query Token 的最佳匹配分数相加，得到文档总分。

课件示例中，各查询 Token 的最大相似度为：

Query Token	最大相似度
Great	0.3
places	0.3
to	0.2
eat	0.7
in	0.3
New	0.8
York	0.8

因此，Token 相似度列的求和结果：

$\begin{aligned} \operatorname{MaxSim}(Q,D) &=0.3+0.3+0.2+0.7={3.4} \end{aligned} $

需要注意：MaxSim 分数不是概率，也不要求落在 [0, 1] 区间内。 它是多个 Token 匹配分数的累加结果。

2.3.6 ColBERT 的存储与性能权衡

Bi-encoder 通常为每篇文档存储一个向量：$ D_i\rightarrow\mathbf d_i $

ColBERT 则为文档中的多个 Token 存储向量：$D_i \rightarrow \{\mathbf{d}_{i1}, \mathbf{d}_{i2}, \cdots, \mathbf{d}_{in}\}$

因此，ColBERT 需要更大的向量存储空间。

Pros	Cons
结合独立编码的可扩展性与 Token 级交互	需要存储大量 Token 向量
支持较细粒度的语义匹配	索引体积和存储成本较高
可用于实时或近实时检索	评分过程比单向量相似度更复杂

ColBERT 的优势在于：文档 Token 向量可以预计算，同时保留查询到来后进行细粒度匹配的能力。

因此，它在检索质量、速度和存储之间提供了一种折中方案。

2.4 三种检索编码器的综合比较

比较维度	Bi-encoder	Cross-encoder	ColBERT
编码方式	Query 与 Document 分别编码	Query 与 Document 联合编码	分别编码，保留 Token 向量
表示粒度	文档级向量	Query - Document Pair	Token 级向量
评分方法	向量相似度	模型直接预测相关性	MaxSim
交互机制	独立编码后比较	编码过程中交互	编码后 Token 级交互
检索质量	较好	通常较高	通常接近 Cross-encoder
检索速度	快	慢	中等
向量存储需求	较低	较低	较高
文档预计算	支持	无法预计算查询相关分数	支持
典型用途	大规模初始检索	候选结果重排序	细粒度语义检索

三种架构的本质区别

Bi-encoder：整体向量匹配

$s_{\text{bi}}(Q, D) = \text{sim}\left(E_q(Q), E_d(D)\right)$

Cross-encoder：联合编码评分

$s_{\text{cross}}(Q,D) = f_{\text{cross}}(Q,D) $

ColBERT：Token 级最大相似度求和

$s_{\text{ColBERT}}(Q,D) = \sum_{i=1}^{m} \max_{j} \operatorname{sim}(\mathbf q_i,\mathbf d_j)$

三种方法分别代表不同的检索设计取向：

- Bi-encoder 优先考虑检索效率和可扩展性。
- Cross-encoder 优先考虑精细的相关性判断。
- ColBERT 在保留文档预计算能力的同时，引入 Token 级交互以改善匹配质量。

2.5 本章总结

本部分的知识点可以归纳为三个相互衔接的层次。

层次	核心问题	主要技术	优化目标
Document Processing	文档应如何组织?	Fixed-size、Overlap、Recursive、Semantic、LLM Chunking	语义完整性与证据相关性
Query Processing	用户查询应如何表达?	Query Rewriting	消除歧义、规范术语、保留用户意图
Retrieval Matching	如何判断查询与文档是否相关?	Bi-encoder、Cross-encoder、ColBERT	匹配质量、速度与存储成本

完整流程可以概括为：

$\text{Documents} \xrightarrow{\text{Chunking}} \text{Chunks} \xrightarrow{\text{Encoding}} \text{Index}$

查询阶段：

$\text{User Query} \xrightarrow{\text{Query Rewriting}} \text{Optimized Query} \xrightarrow{\text{Retrieval}} \text{Relevant Chunks} $

最终：

$\text{Relevant Chunks}+\text{Query} \xrightarrow{\text{LLM}} \text{Answer} $

本部分的核心结论是：RAG 检索质量不仅取决于搜索算法，还取决于检索单元的设计、查询表达的质量，以及编码器对语义相关性的建模能力。

在实际系统中，应以简单且可扩展的方案建立基线，再通过实验判断语义分块、查询改写和更复杂的编码架构是否带来足以抵消额外成本的性能提升。

Part 3 Advanced RAG Architectures 进阶 RAG 架构

从优化单一检索模块，扩展到优化 RAG 系统的知识表示、检索决策、生成质量与整体工作流。

传统 RAG 通常采用固定的 Retrieve → Generate（检索 → 生成） 流程，但存在三个主要问题：

- 知识表示受限：传统文本 RAG 难以充分处理图像、图表和复杂 PDF 版式。
- 检索与生成缺乏自我判断：系统可能执行不必要的检索，使用不相关的证据，或生成缺乏证据支持的答案。
- 控制流程缺乏灵活性：固定的线性流程难以处理需要条件分支、反复修正或多任务并行的复杂问题。

因此，本章围绕三个方向展开：

方向	核心技术	主要解决的问题
Multimodal and PDF RAG	多模态嵌入、视觉 Patch 检索	如何检索非纯文本知识
Self-RAG	Reflection Tokens、自适应检索与自我批判	何时检索、证据是否相关、生成是否有依据

方向	核心技术	主要解决的问题
Agentic RAG Workflows	Routing、Evaluation、Iteration、Parallelism	如何组织复杂的检索与生成任务

3.1 Multimodal and PDF RAG 多模态与 PDF RAG

3.1.1 Multimodal RAG：多模态检索增强生成

传统 RAG 主要针对文本进行检索，而实际知识库可能同时包含文本（Text）、图像（Images），以及音频（Audio）和视频（Video）；多模态 RAG（Multimodal RAG）通过多模态嵌入模型（Multimodal Embedding Model），使不同模态的信息能够在统一的语义空间中表示与检索。

核心要求：Match the retriever and generator to the modalities you use.

即检索器（Retriever）和生成器（Generator）都必须支持系统所使用的模态。

- Retriever：能够处理相应模态并检索相关内容。
- Generator：能够理解检索返回的多模态信息，并据此生成回答。

例如，即使检索器成功找到了相关图像，如果生成模型无法理解图像内容，系统仍然无法充分利用这部分证据。

3.1.2 Multimodal Embedding Model：共享嵌入空间

多模态嵌入模型的关键是将文本和图像映射到 Shared Embedding Space（共享嵌入空间）。

对于文本 t 和图像 I，分别进行编码：

$\mathbf{e}_t = f_{\text{text}}(t)$

$\mathbf{e}_I = f_{\text{image}}(I)$

其中：

- f_{text}：文本编码函数。
- f_{image}：图像编码函数。
- $\mathbf{e}_t, \mathbf{e}_I$：处于共享向量空间中的表示。

共享空间的目标是使语义相近的内容具有相近的向量表示，而不受原始模态限制。

例如，课件展示了两个语义聚类：

语义类别	输入示例	向量空间中的关系
Dogs	“dog”“puppy”、狗的图像	语义相近，聚集在同一区域
Trees	“tree”、树的图像	聚集在另一区域

因此，文本查询可以检索图像，图像查询也可以检索相关文本。

跨模态相似度可用余弦相似度表示：

$$\operatorname{sim}(t,I) = \frac{\mathbf{e}_t^\top \mathbf{e}_I}{\|\mathbf{e}_t\|\|\mathbf{e}_I\|}$$

该公式是对共享向量空间检索原理的数学表达，课件本页没有单独给出此公式。

3.1.3 Multimodal Vector Retrieval：多模态向量检索

多模态向量检索同时处理知识库内容和用户查询，多模态查询与知识库通过共享向量空间实现语义匹配

完整流程可以概括为：

1. 将多模态知识库中的文本、图像编码为向量。
2. 使用兼容的多模态编码模型处理用户查询。
3. 在共享向量空间中进行相似度匹配，返回相关内容。
4. 将检索结果提供给支持相应模态的生成模型。

与传统 RAG 的根本区别：传统文本 RAG 主要比较文本语义，多模态 RAG 则能够进行跨模态语义匹配。

3.1.4 PDF RAG：基于视觉 Patch 的文档检索

PDF 和 Slides（幻灯片）通常具有较高的信息密度，可能同时包含：Text：正文文本、Captions：图注、Charts：图表、Images：图像。

传统方法依赖复杂的文档检测和版面解析算法（Sophisticated Detection Algorithms）

PDF RAG 采用 将PDF 页面切分为视觉 Patch 后分别向量化 (Split into patches, then vectorize)

主要优势：保留原始文档的视觉内容（Preserve Visual Content），包括文字、图形及版式信息，减少仅依赖文本提取造成的信息损失。

PDF RAG 与 ColBERT 的联系

Each prompt token finds the most similar patch in a document, and the maximum similarity scores are summed.

PDF RAG 的评分方式与 ColBERT 相似：设查询包含 m 个 Token，文档被切分为 n 个视觉 Patch：

$ Q={\mathbf q_1,\ldots,\mathbf q_m} $ $D = \{\mathbf{p}_1, \cdots, \mathbf{p}_n\}$

则其 MaxSim 风格的相关性评分可表示为：

${ S(Q,D)= \sum_{i=1}^{m} \max_{1\le j\le n} \operatorname{sim}(\mathbf q_i,\mathbf p_j) }$

其中：

- $\mathbf{q}_i$：第 i 个查询 Token 的向量。
- $\mathbf{p}_j$：第 j 个文档 Patch 的向量。
- max：为每个查询 Token 选取最相似的 Patch。
- Σ：汇总所有查询 Token 的最大相似度。

比较维度	ColBERT	PDF RAG
文档表示	多个 Token 向量	多个视觉 Patch 向量
匹配方式	Query Token 对 Document Token	Query Token 对 Document Patch
评分机制	MaxSim + Sum	MaxSim + Sum
主要特点	细粒度文本匹配	保留文档视觉细节

PDF RAG 的代价

一份文档需要存储多个 Patch 向量，因此会带来：

- Larger Vector Index：向量数量增加，索引和存储成本上升。
- Higher Matching Cost：需要比较更多向量，增加检索计算开销。

因此，PDF RAG 需要在视觉信息保留、检索质量、存储成本和匹配成本之间进行权衡。

3.2 Self-RAG：Retrieval and Self-Critique 检索与自我批判

3.2.1 Simple RAG 的有效性与局限

在介绍 Self-RAG 前，需要明确传统 RAG 的优势及其不足。

1. Effectiveness of Simple RAG

不同知识流行度下检索增强的准确率趋势，研究展示的主要趋势是：

- Rare（冷门知识）：检索带来的准确率提升最大。
- Neutral（中等流行度知识）：检索仍然具有明显收益。
- Popular（热门知识）：检索收益可能较小，甚至出现负面影响。

原因：语言模型对常见知识通常具有较充分的参数化记忆（Parametric Memory），而冷门知识更依赖外部的非参数化记忆（Non-parametric Memory）。

结论：Retrieval improves accuracy most on rare questions in this study.

需要注意，这是课件引用研究中的实验结论，并不意味着检索对所有冷门问题都一定有效。

2. Limitations of Simple RAG

传统 RAG 主要存在以下四类局限：

局限	含义	影响
Unnecessary Retrieval	对无需外部知识的问题仍执行检索	增加不必要的计算成本
Unhelpful Context	检索结果不相关或具有误导性	模型可能被错误上下文干扰
Unsupported Citations	引用来源不一定支持生成的结论	降低答案的可验证性
Increased Latency	检索内容增加输入上下文长度	增加编码和推理延迟

说明无效检索的危害的典型案例

User Query: Who is Sam Altman?

Retrieved Context: Samuel Adams (Sam Adams) was an American statesman, political philosopher and a founding father of the U.S.

Incorrect Response: He was a U.S. founding father.

错误产生的原因是检索结果中的 Samuel Adams 与查询对象 Sam Altman 不一致，而生成模型没有识别这种不相关性，直接采纳了错误证据。

这一案例说明：检索结果并不必然提高答案质量；检索相关性与证据支持性必须得到验证。

3.2.2 Self-RAG 的核心机制: Reflection Tokens

传统 RAG 通常使用固定的两阶段流程：

$\text{Input}\rightarrow\text{Retrieve}\rightarrow\text{Generate} $

Self-RAG（Self-Reflective Retrieval-Augmented Generation）引入可学习的反思 Token（Learned Reflection Tokens），使模型能够在检索和生成过程中进行自主判断。

三个判断：

Reflection Token	核心问题	判断阶段	作用
Retrieve?	Is retrieval needed?	检索之前	判断是否需要外部知识
Relevant?	Are documents relevant?	检索之后	判断检索证据是否相关
Supported?	Is generation supported?	生成之后	判断生成内容是否受到证据支持

这三个判断分别对应：

1. Retrieval Decision：是否检索。
2. Evidence Relevance：证据是否相关。
3. Generation Grounding：生成内容是否有依据。

与固定检索流程相比，Self-RAG 的关键变化是将检索与证据评估纳入模型学习的决策过程。

3.2.3 Self-RAG Control Flow：自适应检索与生成

这三个判断共同构成 Self-RAG 的自适应检索与自我批判机制：

第一阶段：检索需求判断

模型根据问题决定是否检索。如果已有知识足以回答，则可以直接生成；否则调用检索器获取外部证据。

第二阶段：证据相关性判断

检索完成后，模型评估返回文档是否与问题相关，避免将无关内容直接纳入生成过程。

第三阶段：生成支持度判断

生成答案后，模型评估回答中的陈述是否得到检索证据支持。

3.2.4 Relevance vs. Support：相关性与支持度

Self-RAG 不仅关注检索到的内容是否相关，还关注生成的具体陈述是否得到证据支持。

两者的区别是：

- Relevance：内容是否与用户问题相关。
- Support：检索证据是否足以支持生成的陈述?

相关内容不一定能充分支持生成的结论。

课件案例: Scientific Literature Synthesis

用户请求: Suggest papers showing how LLMs help scientists synthesize scientific literature.

模型检索到以下两类证据：

- OpenScholar is a retrieval-augmented LM designed to synthesize literature.
- GPT-4o has been shown to generate new research ideas.

在此基础上，模型生成不同陈述并评估其支持程度。

Generated Statement	Relevance	Support
OpenScholar is an LM for literature synthesis.	Relevant	Fully supported
Studies show GPT-4o can help scientists with idea generation and literature synthesis.	Relevant	Partially supported

第一条陈述与检索证据一致，因此被评为 Fully supported。

第二条陈述虽然与科研任务相关，但证据仅支持 GPT-4o 能够帮助产生研究想法，并未充分证明其具有文献综合能力，因此被评为 Partially supported。

两个支持度评分（用于体现候选陈述的证据支持程度）：

OpenScholar 陈述：0.9 Fully supported

GPT-4o 陈述：0.5 Partially supported

Self-RAG 的关键意义：将检索需求、证据相关性和生成支持性转化为模型可学习、可评估的判断，从而提高答案的可靠性。

3.3 Agentic RAG Workflows 智能体 RAG 工作流

3.3.1 From LLM to Agent System：从单模型到智能体系统

传统单模型调用通常采用：

$ \text{Prompt}\rightarrow\text{LLM}\rightarrow\text{Response} $

这种结构适合相对简单的任务，但对于需要规划、检索、评估和多步骤处理的复杂任务，单次调用可能难以完成全部工作。而Agent System（智能体系统）通过将复杂任务分解为一系列步骤与决策，并结合外部工具完成任务。

课件展示了三类典型子任务：

- Planning LLM：规划任务步骤。
- Research LLM：执行信息检索和研究。
- Writing LLM：整合信息并生成最终回答。

智能体系统可以访问外部工具，包括代码工具、数据库和互联网资源。

其主要特点是 a series of steps and decisions（由多个步骤和决策组成），而不是依赖单次模型调用。

3.3.2 Example Agentic RAG System

Agentic RAG：路由、证据评估、重新检索与回答生成

完整逻辑为：

1. Router LLM 判断问题是否需要检索。
2. 如果不需要检索，直接生成回答。

3. 如果需要检索，则执行 Retrieve。
4. Evaluate Evidence 判断当前证据是否充分。
5. 如果证据不足，返回检索阶段再次检索；如果充分，则生成带引用的回答（Generate + Cite）。
6. 输出最终 Response。

这个系统同时包含条件分支与迭代机制，说明 Agentic RAG 并不局限于单一工作流模式。

3.3.3 Agentic Systems as Flowcharts：流程图式系统设计

智能体系统可以表示为流程图（Flowchart），由不同的节点和连接关系组成。每个 LLM 节点完成一个特定任务，并将输出传递给下一节点。

Different LLMs for different tasks. 不同任务可以采用不同规模和能力的模型：

任务	推荐模型类型	设计目的
Routing	Lightweight Models	低成本完成路径选择
Evaluation	Lightweight Models	高效判断结果质量
Response Generation	Larger Models	提高最终回答质量
Citations	Specialized Models	处理引用与证据对应关系

因此，Agentic RAG 的设计不仅涉及模型能力，还包括任务分解、模型分配、流程控制和外部工具调用。

3.3.4 Four Workflow Patterns：四种典型工作流

四种基本工作流：

1. Sequential Workflow：顺序工作流

信息按照固定顺序依次经过多个 LLM 调用，前一步的输出成为后一步的输入。

示例为 Q&A Chatbot：

$\text{Parse} \rightarrow\text{Rewrite} \rightarrow\text{Retrieve} \rightarrow\text{Generate with Citations} $

特点：结构简单、执行顺序明确，适用于步骤固定且存在前后依赖关系的任务。

其局限是缺少动态决策和自动回退机制，前一阶段的错误可能传递至后续阶段。

2. Conditional Workflow：条件工作流

条件工作流通过 Router 判断输入应进入哪条处理路径。

例如：

- Knowledge needed：先检索外部知识，再生成回答。
- No retrieval needed：跳过检索，直接回答。

特点：根据输入动态选择执行路径，可以减少不必要的检索开销，并降低无关上下文带来的干扰。

其主要风险是 Router 的判断错误可能导致系统选择不合适的处理路径。

3. Iterative Workflow：迭代工作流

迭代工作流允许系统根据评估结果返回之前的步骤，重新执行任务。

课件使用 AI Coding 作为示例：

$\text{Generate Code} \rightarrow\text{Run Tests} \rightarrow\text{Identify Errors} \rightarrow\text{Revise Code} \rightarrow\text{Test Again} $

系统包含两个主要角色：

- Writer：生成候选结果。
- Evaluator：评估结果并提供 Feedback。

当评估未通过时，Feedback 返回 Writer，触发下一轮修改。

终止条件：

$\text{Stop}= \begin{cases} \text{Accepted}, & \text{结果满足要求}\ \text{Iteration Limit}, & \text{达到最大迭代次数} \end{cases} $

设置最大迭代次数 N 可以避免无限循环。

特点：适用于需要多轮验证和改进的任务，例如代码生成、答案修订和证据补充。

4. Parallel Workflow：并行工作流

并行工作流将任务分解为多个相互独立的子任务，同时执行，再将结果整合为最终回答。

核心组件包括：

- Orchestrator：拆分任务并协调执行。
- LLM Agents / Router：执行独立子任务或负责分配任务。
- Synthesizer：整合多个子任务的结果。

课件示例为 AI Travel Planner（AI 旅行规划系统），可以分别处理航班、住宿和景点信息，再整合为完整行程。

并行执行的前提是 Independent Subtasks（子任务相互独立）。若子任务之间存在数据依赖，则不能简单并行。对于 n 个独立子任务，其顺序执行时间可近似表示为：

$T_{\text{seq}}=\sum_{i=1}^{n}T_i $

并行执行时间可近似表示为：

$T_{\text{parallel}} \approx \max(T_1,T_2,\ldots,T_n)+T_{\text{overhead}} $

其中 T_{overhead} 包含编排、通信和结果合成开销。

特点：通过同时执行独立任务降低整体等待时间，但需要额外的任务协调与结果合成机制。

3.3.5 四种工作流综合比较

Workflow	控制流特征	核心组件	适用场景
Sequential	线性执行	多个顺序 LLM 调用	固定步骤的问答处理
Conditional	根据条件选择分支	Router	按需检索
Iterative	评估后回退并重新执行	Writer + Evaluator	代码修正、答案优化
Parallel	独立子任务同时执行	Orchestrator + Synthesizer	多来源研究、旅行规划

四种工作流并非互斥关系，而是可以组合使用的控制结构。

例如，一个完整的 Agentic RAG 系统可以先通过 Router 判断是否需要检索，再并行查询多个知识源，随后评估证据充分性；若证据不足，则重新检索，最终生成带引用的回答。

3.3.6 Implementing Agentic Systems：工程实现

三项实现原则：

第一，简单系统可以手动实现。

对于规模较小、步骤明确的 Agentic System，可以直接编写工作流逻辑，无需引入复杂框架。

第二，复杂系统可以使用工具、库或平台。

当系统包含较多模型调用、条件分支、循环及并行任务时，可以考虑使用专门的工具、库或平台进行管理。

第三，不同工作流模式可以组合。

实际智能体系统通常综合使用顺序、条件、迭代和并行结构，以满足不同任务需求。

3.4 本章总结

本章的三个进阶方向分别从知识表示、模型决策和系统控制流三个层面改进传统 RAG。

层面	技术	核心改进
Knowledge Representation	Multimodal RAG	通过共享嵌入空间支持跨模态检索
Document Retrieval	PDF RAG	将视觉 Patch 向量化，保留文档图表和版式
Retrieval Decision	Self-RAG	使用 Reflection Tokens 判断是否检索
Evidence Verification	Self-RAG	判断证据相关性与生成支持度
System Orchestration	Agentic RAG	将复杂任务拆分为多个 LLM 节点
Workflow Control	Sequential / Conditional / Iterative / Parallel	支持顺序执行、动态分支、循环修正与并行处理

1. Multimodal RAG 的基础是 Shared Embedding Space，使不同模态的信息能够进行语义匹配。
2. PDF RAG 通过视觉 Patch 保留文档内容，并使用类似 ColBERT 的 MaxSim 评分机制，但需要承担更高的向量存储和匹配成本。
3. Simple RAG 对冷门知识问题具有明显收益，但不必要的检索、无关上下文、错误引用及更长的输入上下文可能降低系统表现。
4. Self-RAG 通过 Retrieve?、Relevant? 和 Supported? 三类反思判断，实现自适应检索与证据支持性评估。
5. Agentic RAG 将复杂任务组织为由多个模型节点及外部工具构成的工作流，不同节点可以采用不同能力与成本的模型。
6. Four Workflow Patterns 分别提供线性执行、条件路由、迭代反馈和并行执行机制，并且可以组合使用。

结论： 进阶 RAG 的目标不只是检索更多信息，而是使系统能够处理更丰富的知识形式、判断何时及如何使用证据，并通过灵活的工作流提高答案质量与执行效率。

Part 4 RAG Systems Evaluation RAG 系统评估

本章核心： RAG 系统的评估不仅需要衡量检索结果和生成答案的质量，还需要考察系统在真实生产环境中的响应延迟、吞吐能力与可扩展性，并通过组件级性能分析定位和优化瓶颈。

本章围绕三个方面展开：

1. Evaluation Challenges：真实用户请求的不可预测性，以及高并发环境下的性能压力。
2. Performance and Quality Metrics：从性能和质量两个维度评估完整 RAG 系统。
3. Evaluation and Optimization：通过延迟分析、模型优化以及自动化与人工评估改进系统。

4.1 Evaluation Challenges RAG 系统评估的挑战

与固定测试集上的离线检索评估不同，生产环境中的 RAG 系统需要持续处理复杂、多样且不断增长的用户请求，因此面临两项主要挑战。

4.1.1 Unpredictability of Prompts：用户输入的不可预测性

真实用户的请求具有创造性（Creative）和不可预测性（Unpredictable），即使经过严格测试，也无法预先覆盖系统可能接收到的所有问题。

课件通过 Google AI Overview 对异常问题的回答说明这一风险：当用户询问每天应该吃多少石头时，系统可能将不可靠的检索内容转化为具有误导性的建议。

这反映了 RAG 系统的两个问题：

- 有限的测试覆盖率：离线测试集无法穷举真实用户的输入类型。
- 较高的生产错误成本：不准确或缺乏证据支持的回答可能在实际应用中造成严重影响。

因此，离线测试表现良好并不意味着系统在生产环境中始终可靠，还需要持续评估真实请求及其生成结果。

4.1.2 Scaling Performance：系统性能的可扩展性

随着用户数量和请求流量增加，RAG 系统需要处理更高的计算负载。

课件强调三个方面：

- Latency and Load：流量增长会增加系统负载，并可能导致响应延迟上升。
- Memory and Compute Costs：请求数量增加会提高内存与计算资源需求。
- Scalability：系统需要在扩大服务规模的同时维持较高的性能。

两个挑战分别对应不同的评估重点：

评估挑战	核心问题	主要关注
Prompt Unpredictability	能否可靠处理未知或异常输入	回答质量与可靠性
Scaling Performance	能否在请求增长时保持服务能力	延迟、吞吐与资源开销

结论： RAG 系统评估必须同时覆盖 Quality（质量）和 Performance（性能），仅衡量检索准确率不足以反映系统的整体表现。

4.2 Performance and Quality Metrics 性能与质量指标

课件将系统评估划分为两个维度：

- Performance：How efficiently does it run? 衡量系统运行效率。
- Quality：How useful are the results? 衡量检索与生成结果的有效性。

四项核心指标如下：

评估维度	Metric	定义	评估对象
Performance	Latency	完成一次请求所需的时间	响应速度
Performance	Throughput	单位时间内处理的请求数量	系统处理能力
Quality	Relevance	检索文档是否与用户问题相关	检索质量
Quality	Faithfulness	生成答案是否得到证据支持	生成可靠性

4.2.1 Performance Metrics：性能指标

1. Latency：响应延迟

Latency 表示系统从接收用户请求到返回响应所经历的时间。

对于第 i 个请求： $ L_i=t_{\text{response},i}-t_{\text{request},i} $

其中，$t_{\text{request},i}$ 为请求到达时间，$t_{\text{response},i}$ 为响应完成时间。

若统计 N 个请求，其平均延迟为： ${ L_{\text{avg}}=\frac{1}{N}\sum_{i=1}^{N}L_i }$，延迟越低，通常意味着用户等待时间越短。

2. Throughput：系统吞吐量

Throughput 表示系统在单位时间内能够处理的请求数量，常以 Requests per Second（RPS）衡量。

${ \text{Throughput}=\frac{N_{\text{completed}}}{\Delta t} }$

其中：

- $N_{\text{completed}}$：统计时间内完成的请求数。
- Δt：统计时间长度。

吞吐量越高，说明系统在给定条件下能够处理更多请求。

Latency 与 Throughput 的区别： Latency 衡量单个请求的完成时间，Throughput 衡量系统整体的处理能力。二者需要结合分析，高吞吐量并不必然意味着低延迟。

4.2.2 Quality Metrics：质量指标

1. Relevance：检索相关性

Relevance 衡量检索返回的文档是否与用户问题相关。

其评估对象是： $ \text{Query}\longleftrightarrow\text{Retrieved Documents}$

这一指标与前面介绍的检索评估指标相衔接，例如：

- Precision@K：前 K 个结果中相关文档的比例。

- Recall@K：前 K 个结果覆盖全部相关文档的比例。
- MRR：首个相关文档的排名质量。

这些指标用于量化不同方面的检索相关性。

2. Faithfulness：生成忠实度

Faithfulness 衡量模型生成的陈述是否能够得到提供给模型的证据支持。

其评估对象是： Generated Response ↔ Retrieved Evidence

例如，若检索文档明确记载某航天任务有两名宇航员，而模型回答三名，即使文档检索正确，生成结果仍存在 Faithfulness 问题。

因此：

- Relevance 关注是否检索到了正确的信息。
- Faithfulness 关注生成答案是否正确使用了这些信息。

二者相互关联，但不能互相替代。高检索相关性并不保证生成答案具有高忠实度。

4.3 System Latency Analysis and Optimization 系统延迟分析与优化

4.3.1 End-to-End Latency：端到端延迟

RAG 系统通常采用以下处理流程：

端到端延迟（End-to-End Latency）表示从用户提交问题到系统返回完整响应的总耗时。

对于顺序执行的 RAG 流程，可以将其分解为：${ T_{\text{total}} = T_{\text{query}} + T_{\text{retrieval}} + T_{\text{generation}} + T_{\text{other}} } $

其中：

- T_{query}：查询处理时间。
- $T_{\text{retrieval}}$：检索耗时。
- $T_{\text{generation}}$：LLM 生成耗时。
- T_{other}：其他处理、通信等开销。

两个测量层次：

测量层次	指标	作用
System-Level	End-to-End Latency	判断系统整体响应是否存在性能问题
Component-Level	Retriever Timing / Generation Timing	定位延迟主要来自哪个组件

核心原则： 端到端延迟用于发现整体性能问题，组件级测量用于定位具体瓶颈。

4.3.2 Latency Bottleneck：延迟瓶颈

典型 RAG 流程： Query Processing、Retrieval 和 LLM Generation 三部分，Transformer Inference 往往是主要延迟来源。

课件的主要结论是：

- Transformer Inference can dominate latency.
- LLM Calls are often the main bottleneck.
- Retrieval and database operations are typically faster.

但这一结论并非适用于所有配置。如果检索阶段包含高成本的重排序或复杂计算，Retriever 也可能成为主要瓶颈。因此，优化应当以实际测量结果为依据，而不是预先假定某一组件最慢。

4.3.3 Reducing LLM Latency：降低生成延迟

当组件级测量表明 LLM 推理是主要瓶颈时，课件提出两种优化方法。

1. Smaller or Quantized Models：使用小模型或量化模型

- Smaller Model：采用参数规模较小的模型，减少推理计算需求。
- Quantized Model：采用低精度数值表示模型参数，降低存储与计算资源需求。

两种方法都可能降低推理成本，但实际加速效果及输出质量变化需要通过 Benchmark 验证。

2. Route by Task Complexity：根据任务复杂度进行模型路由

Router 根据任务复杂程度选择不同模型：

- 简单任务交由较小的模型处理，以减少推理成本。
- 复杂任务交由能力更强的模型处理，以维持回答质量。

这一策略通过差异化分配计算资源，在效率和质量之间进行权衡。

Benchmark the Speedup：评估加速效果

优化完成后，需要比较优化前后的推理耗时。

加速比（Speedup）可以表示为： ${ \text{Speedup} = \frac{T_{\text{before}}}{T_{\text{after}}} }$

例如，加速比为 2 表示优化后完成相同任务所需的时间为原来的一半。

但性能优化不能只关注加速比，还应检查 Relevance 与 Faithfulness 是否受到影响。

因此，完整的优化过程是： ${ \text{Measure} \rightarrow \text{Locate Bottleneck} \rightarrow \text{Optimize} \rightarrow \text{Benchmark} } $

即先测量系统表现，再定位瓶颈，针对性实施优化，最后验证性能和质量变化。

4.4 Evaluation Methods 系统评估方法

除直接测量性能与质量指标外，区分两种评估器：Code-Based Evaluators 与 Human Feedback Evaluators。

4.4.1 Code-Based Evaluators：基于代码的评估器

Code-Based Evaluators 使用程序自动检查系统运行情况和可明确判定的输出要求。

典型检查包括：

- Track Requests per Second：统计每秒请求处理数量，衡量吞吐量。
- Check for Valid JSON Output：检查输出是否符合合法 JSON 格式。

- Low Evaluation Cost：自动化评估通常成本较低，执行简单。

其优势是便于自动化、成本低，适合频繁执行的系统检查。

但传统规则型代码评估器通常难以直接判断开放式回答的语义正确性或证据支持程度。

4.4.2 Human Feedback Evaluators：人工反馈评估器

Human Feedback Evaluators 通过用户或人工评审人员评价系统输出。

三种方式：

- Thumbs Up / Down Ratings：通过点赞或点踩收集总体评价。
- Detailed Text Feedback：收集具体文字反馈，识别回答存在的问题。
- Manual Quality Assessments：通过人工审查判断回答的相关性、正确性与证据支持程度。

人工评估能够发现代码规则难以识别的语义和质量问题，但需要更多时间和人力成本。

4.4.3 两类评估方法的比较

比较维度	Code-Based Evaluators	Human Feedback Evaluators
成本	较低	较高
自动化程度	高	较低
评估重点	吞吐量、输出格式等可程序化检查的内容	用户体验、语义质量、证据支持
主要优势	简单、高效、可重复执行	能发现代码检查遗漏的问题
主要局限	难以覆盖复杂语义问题	成本高，难以全面覆盖所有请求

结论： 两种方法具有互补性。代码评估器适合大规模自动化检查，人工反馈评估器则用于识别难以通过固定规则判断的质量问题。

4.5 本章总结

技术方向	关键方法	核心权衡
Scalable Retrieval	ANN、HNSW	检索速度与 Recall
Chunking and Query Processing	Chunking、Query Rewriting	上下文保留与检索有效性
Ranking	Cross-Encoder、ColBERT	排序质量与计算、存储成本
Advanced RAG	Multimodal RAG、Self-RAG、Agentic Workflows	系统能力与架构复杂度
System Evaluation	Relevance、Faithfulness、Latency	结果质量与运行效率

五项核心原则

1. Scalable Vector Search

使用 Approximate Nearest Neighbor（ANN）提高大规模向量检索速度，但需要权衡速度与检索召回率。

2. Effective Chunking and Query Processing

通过合理分块和查询改写提高检索有效性，同时保留原始上下文及用户意图。

3. Improved Ranking

利用 Cross-Encoder 或 ColBERT 改善排序质量，但需要考虑额外的计算开销和向量存储成本。

4. Advanced RAG Architectures

根据任务需求引入多模态证据、Self-RAG 自我批判机制和 Agentic Workflows，提高系统对复杂任务的处理能力。

5. Measurement-Driven Optimization

同时评估 Retrieval Quality、Answer Faithfulness 和 End-to-End Latency，并针对实际测得的性能瓶颈实施优化。

核心关系：

- Relevance 与 Faithfulness 分别衡量检索证据的相关性和生成内容的证据支持程度。
- Latency 与 Throughput 分别衡量单次请求响应时间和系统单位时间处理能力。
- End-to-End Latency 与 Component-Level Timing 分别用于识别整体性能问题和定位具体瓶颈。
- Code-Based 与 Human Feedback Evaluators 分别提供低成本自动化检查和更深入的人工质量判断。

最终，RAG 系统的优化应遵循 Evaluation → Bottleneck Identification → Targeted Optimization → Benchmarking 的过程，在检索质量、生成可靠性、响应速度和资源成本之间取得平衡。

4.6 In-Class Exercise

题目	正确答案	核心依据
Q1. ColBERT 的 Token-Level Representation 相比每篇文档一个向量的主要权衡是什么?	A	更细粒度的匹配能力，但需要存储更多向量
Q2. HNSW 为什么先搜索稀疏的上层，再进入密集的底层?	C	先进行大范围导航，再逐层缩小范围，完成局部精细搜索
Q3. 从精确搜索切换到 ANN 的主要权衡是什么?	B	减少距离比较次数，提高速度，但可能错过真正的最近邻

$\text{ColBERT}: \quad \text{Matching Detail} \leftrightarrow \text{Vector Storage} $

$\text{HNSW}: \quad \text{Coarse Navigation} \rightarrow \text{Local Refinement} $

$ \text{ANN}: \quad \text{Search Speed} \leftrightarrow \text{Retrieval Recall} $

其中，ColBERT 的多向量表示不同于 Cross-Encoder 对查询与文档进行联合编码；HNSW 的上层用于导航，并非删除不相关文档；ANN 减少的是搜索时的比较次数，并不必然减少存储文档的数量或向量维度。

5583 Agentic Computing 3

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_6/knowledge/5583-Agentic-Computing-3.md`

5583 Agentic Computing3

Lecture 8 智能体系统设计 Design of Agentic Systems

本讲围绕智能体系统（Agentic Systems）的设计展开，重点讨论如何从单一语言模型构建具有自主决策、工具调用、任务规划和反馈修正能力的智能体系统。

课程内容按照以下逻辑组织：

1. 智能体系统基础（Agentic System Foundations）：理解 AI Model、AI System、Agent 和 Multi-Agent System 的区别，掌握智能体的组成及自主性程度。
2. 智能体工作流设计（Agentic Workflow Design）：理解顺序、并行、迭代和条件工作流，学习任务分解与组件选择。
3. 智能体推理与决策（Agentic Reasoning and Decision-Making）：研究智能体如何利用感知、记忆、目标与约束选择行动。
4. 自反思与反馈优化（Self-Reflection and Feedback）：理解内部反思、外部反馈及其在代码生成、SQL 查询等任务中的应用。

Part 1 智能体系统基础 Agentic System Foundations

1.1 从语言模型到智能体系统

人工智能应用可以按照系统能力与组织形式，区分为四个层次：

AI Model → AI System → Agent → Multi-Agent System

从基础模型能力到自主行动、再到多智能体协作的系统演进

1.1.1 四个层次

层次	核心定义	主要特征
AI Model	提供基础计算与智能能力的模型	生成文本、理解信息、进行预测
AI System	将模型能力集成到应用程序中	包含模型、界面、数据及业务逻辑
Agent	能够围绕目标自主选择行动的系统	具有决策、规划、工具调用和反馈能力
Multi-Agent System	由多个具有决策能力的智能体组成的系统	支持智能体之间的交互、分工与协作

1. AI Model：基础模型

模型提供计算能力，但通常不直接负责完整的业务流程。例如，LLM 可以根据输入生成文本。

这一形式描述了单次模型调用，但并不包含外部工具、任务执行或环境反馈。

2. AI System：人工智能系统

AI System 将模型能力封装为能够完成实际任务的应用。例如，智能客服系统可能包含：LLM：理解用户问题并生成回复、数据库：保存用户信息与订单记录、API：查询物流状态、用户界面：展示交互结果。

因此，AI System 的重点是将多个组件组织为完整应用，而不一定具备自主规划能力。

3. Agent：智能体

Agent 不仅生成信息，还能够根据目标与环境状态决定下一步行动。

典型行为包括：判断是否需要检索信息、选择合适的外部工具、将复杂任务分解为多个步骤、根据执行结果调整后续行动、判断任务是否已经完成。

智能体的关键特征是 Goal-directed Action（目标导向的行动）。

4. Multi-Agent System：多智能体系统

多智能体系统由多个能够独立作出一定决策的智能体组成。不同智能体可以承担不同职责，例如：Research Agent：负责资料检索、Analysis Agent：负责信息分析、Writing Agent：负责内容生成、Review Agent：负责质量检查。

多个智能体通过通信、任务分配和结果整合共同完成复杂任务。需要注意，增加智能体数量不一定提高系统性能。多智能体协作也可能引入通信开销、协调冲突及错误传播。

1.2 智能体的核心组成 Components of an Agent

将智能体能力划分为核心能力（Core Capabilities）与附加能力（Additional Capabilities）。

1.2.1 核心能力

核心能力	英文	作用
感知	Perception	获取并理解环境或用户输入
自主行动	Agency	根据目标选择并执行行动
推理与规划	Reasoning and Planning	分析当前状态并组织行动步骤
记忆	Memory	保存和利用历史信息
反馈与学习	Feedback / Learning	根据执行结果调整行为

1. Perception：感知

感知是智能体获取环境信息的过程。

输入可以来自用户文本、文件、数据库、搜索结果或外部系统。

感知不仅意味着接收数据，还包括从输入中提取与当前任务相关的信息。

2. Agency：自主行动

Agency 指智能体根据目标采取行动的能力。

与单纯生成文本不同，具有自主行动能力的智能体可以决定是否调用工具、执行操作或进入下一阶段。

3. Reasoning and Planning：推理与规划

推理用于判断当前状态和行动选择，规划用于组织实现目标所需的步骤。

两者密切相关，但侧重点不同：

- Reasoning：当前应该做什么，以及为什么。
- Planning：完成目标需要哪些步骤，以及这些步骤如何安排。

4. Memory：记忆

记忆使智能体能够利用先前的信息，而不是在每次调用时重新开始。

可以进一步区分为：

- 短期记忆：当前任务的上下文、中间结果和工具反馈。
- 长期记忆：历史经验、用户偏好或持久化知识。

这种区分是对课件内容的扩展，具体实现可能采用上下文窗口、数据库或向量检索等方式。

5. Feedback / Learning：反馈与学习

反馈机制使智能体能够判断当前行动是否达到预期。

例如，代码执行失败后，智能体可以读取错误信息、修改代码并重新执行。

这里的学习不一定意味着更新模型参数，也可以表现为任务执行过程中的策略调整。

1.2.2 附加能力

附加能力	英文	作用
多模态	Multimodality	处理文本、图像、音频等不同模态
通信	Communication	与用户、工具或其他智能体交换信息
具身性	Embodiment	通过物理或虚拟环境中的行动与外界交互

具身智能体（Embodied Agent）可以通过传感器感知环境，并通过执行器采取行动。例如，机器人能够观察物体位置、规划移动路径并执行抓取操作。

1.2.3 智能体的基本运行循环

综合上述能力，可以将智能体抽象为：

1. Perception 获取用户输入与环境状态
2. Reasoning & Planning 结合目标、约束与记忆制定行动
3. Action 生成回复或调用外部工具
4. Feedback & Memory Update 检查结果并更新任务状态

未完成时继续循环；满足完成条件后终止。这一循环是智能体区别于单次模型调用的重要特征。

1.3 智能体自主性程度 Degrees of Agent Autonomy

1.3.1 自主性是连续谱

与其争论某个系统是否属于真正的 Agent，不如承认系统存在不同程度的 Agentic Behavior（智能体行为）。因此，智能体不应简单地被划分为“是 Agent”或“不是 Agent”，而应根据其自主决策程度进行分析。

低自主性：固定流程、固定工具、有限决策

半自主性：能够选择行动，但工具集合预定义

高自主性：自主规划较多步骤，可能动态创建工具

1.3.2 三种自主性水平

维度	低自主性	半自主性	高自主性
工作流程	所有步骤预先确定	部分步骤由模型决定	多数步骤可自主决定
工具使用	工具调用硬编码	从预定义工具中选择	可能动态创建新工具
决策能力	主要局限于文本生成	能够选择部分行动	能够自主规划与调整
灵活性	较低	中等	较高
可预测性	通常较高	中等	相对较低

高自主性系统具有更大的灵活性，但也需要更严格的权限控制、执行监测与结果验证。

1.3.3 自主性系统比较及原则

方案 A：低自主性系统

User Task → LLM → Web Search → Web Fetch → LLM → Essay

1. LLM 生成搜索查询。
2. 系统执行网页搜索。
3. 系统抓取网页内容。
4. LLM 根据资料撰写文章。

流程基本预先确定，LLM 对整体流程的修改空间有限。

方案 B：更高自主性系统

Task → LLM → Research → LLM → Fetch Sources → Draft → Reflect → Revise

该系统能够从预定义工具集合中选择工具，包括：Web Search、News、arXiv、Web Fetch、PDF to Text

模型先决定如何搜索，再选择和获取资料，随后生成初稿并进行反思修改。

还明确包含获取五个最佳来源（Fetch 5 Best Sources）的步骤。

两种系统的主要区别不是使用了多少个模型，而是模型对行动选择、工具使用和任务流程具有多大控制权。

自主性设计原则

应根据任务特点选择适当的自主性程度。

对于流程明确、风险较高且需要严格审计的任务，固定工作流可能更合适；对于开放性研究、探索性任务或动态环境，较高自主性可能提供更强的适应能力。

因此，系统设计目标不是最大化自主性，而是在灵活性、可靠性、成本与可控性之间取得平衡。

Part 2 智能体工作流设计 Agentic Workflow Design

智能体工作流是指将模型推理、工具调用、条件判断和反馈修正组织为完整任务执行过程的方法。

合理的工作流设计可以提高任务完成质量、执行效率和系统可维护性。

2.1 四种基本工作流模式

四种常见模式：Sequential、Parallel、Iterative 和 Conditional。

模式	核心机制	主要优势	典型场景
Sequential	依次执行	依赖关系清晰	多阶段数据处理
Parallel	独立任务同时执行	缩短执行时间	多来源检索
Iterative	根据反馈反复修正	改善输出质量	代码调试、文章修改
Conditional	根据条件选择路径	提高流程灵活性	RAG 路由、异常处理

2.1.1 顺序工作流 Sequential Workflow

顺序工作流要求各步骤依次执行，后续步骤通常依赖前一步的输出。

例如： Extract Information → Analyze → Generate Report

适用于存在明确依赖关系的任务。若三个步骤执行时间分别为 T_A, T_B, T_C，忽略额外调度开销，则：

$$T_{\text{sequential}} = T_A + T_B + T_C$$

2.1.2 并行工作流 Parallel Workflow

并行工作流将能够独立执行的任务分配到不同分支。

例如，研究报告可以同时执行多个独立的信息检索任务，再将检索结果整合为报告。

在任务真正独立、资源允许且忽略调度开销的情况下： $T_{\text{parallel}} \approx \max(T_1, T_2, \cdots, T_n)$

相较于串行执行： $T_{\text{sequential}}=\sum_{i=1}^{n}T_i $，并行执行能够降低等待时间。

但如果任务存在数据依赖、资源冲突或副作用，就不能简单地并行执行。

2.1.3 迭代工作流 Iterative Workflow

迭代工作流通过反复检查与修正来改善结果。

$\text{Draft} \rightarrow \text{Review} \rightarrow \begin{cases} \text{Done}, & \text{通过检查}\\ \text{Revise}\rightarrow\text{Review}, & \text{未通过} \end{cases}$

典型应用包括：文章撰写与修改、代码生成与调试、数据查询与错误修正、复杂任务的多轮优化。

迭代工作流的核心是利用反馈改善输出，而不是仅增加模型调用次数。

2.1.4 条件工作流 Conditional Workflow

条件工作流根据当前状态或判断结果选择不同执行路径。

$\text{Check} \rightarrow \begin{cases} \text{Route A}, & C=\text{True}\\ \text{Route B}, & C=\text{False} \end{cases}$

例如：

- 如果已有知识足以回答问题，则直接生成答案。
- 如果需要外部证据，则先进行检索。

条件工作流可以避免不必要的工具调用，也可以根据任务类型选择不同的处理方式。

2.2 智能体工作流的主要优势

三项关键优势：

1. 潜在的性能提升（Potentially Better Performance）

复杂任务可以经过多轮推理、检查和修改，而不必依赖单次模型生成。

例如： $\text{Draft} \rightarrow \text{Review} \rightarrow \text{Revise} $

但多轮执行并不保证性能一定提升。如果反馈质量较差，迭代也可能引入新的错误。

2. 通过并行执行提高效率（Faster Execution Through Parallel Work）

当多个任务相互独立时，可以同时执行，从而降低整体延迟。

例如，三个检索任务可以并行运行，完成后再整合结果。

3. 模块化（Modularity）

模块化设计使系统能够独立更新工具和模型。

例如：替换搜索 API、增加数据库查询工具、将原有 LLM 替换为其他模型、单独优化评估模块。

模块化能够降低系统维护与扩展的复杂度。

2.3 任务分解与组件选择 Task Decomposition

2.3.1 任务分解的基本原则

任务分解是将复杂任务划分为能够由模型、工具或二者组合完成的子任务。

两个核心问题：

1. 一个 LLM、一个工具，或二者组合能否完成当前步骤？
2. 如果不能，是否需要进一步分解，或引入人工参与（Human-in-the-Loop）？

一个合理的子任务应具有明确的输入、输出和完成标准。任务分解过粗可能导致模型无法稳定完成任务；分解过细则可能增加调用次数、延迟和系统复杂度。

2.3.2 客户邮件处理案例

课件中的客户 Susan Jones 发送了一封投诉邮件：客户订购了蓝色 KitchenPro 搅拌机，订单号为 #8847，但实际收到的是烤面包机。客户希望在女儿周末生日派对前收到正确商品。系统需要处理三个子任务。

第一步：信息提取。从邮件中提取结构化信息；

第二步：订单查询。模型根据订单号调用数据库工具，核实订单记录与实际履约状态；

第三步：生成回复并执行操作。模型根据核实的信息撰写回复，再通过邮件工具发送。

需要注意，课件描述的是“撰写并发送回复”的流程，并未说明系统已经确认能够补发商品。因此，不应在缺少订单查询结果时擅自承诺具体补救措施。

在实际业务系统中，涉及退款、补发或其他具有业务后果的操作时，可以增加权限校验或人工审批。

2.4 智能体系统的基本构建模块 Building Blocks

将可用组件分为 Models 和 Tools 两大类。

类别	组件	典型用途
Models	LLMs	文本生成、工具调用、信息提取
Models	Other AI Modules	文本转语音、图像分析
Tools	APIs	网页搜索、实时数据获取、发送邮件、日历查询
Tools	Information Retrieval	数据库查询、文档检索、RAG
Tools	Code Execution	基础计算、数据分析

模型主要负责语义理解、推理、决策及内容生成。

工具主要负责访问外部资源、执行确定性操作或与实际系统交互。

例如，查询订单时：

- LLM 负责理解用户请求并生成查询参数。
- 数据库工具负责执行查询并返回真实记录。
- LLM 再根据返回结果组织后续行动。

选择组件时应考虑：任务是否需要实时信息、是否涉及精确计算、是否需要读取或修改外部数据、是否要求结果可验证、是否具有安全或权限限制。

例如，涉及数值计算时应优先使用代码执行；涉及当前订单状态时应调用数据库，而不能仅依赖模型预训练知识。

Part 3 智能体推理与决策 Agentic Reasoning and Decision-Making

3.1 推理与决策的基本机制

智能体推理：智能体利用可用信息，决定下一步应该采取什么行动，以实现目标。

推理过程依赖三个主要信息来源：

1. Perception / Grounding：当前感知与外部事实。
2. Memory：历史信息与已有状态。
3. Goals / Constraints：任务目标与约束条件。

这些信息共同支持智能体的决策与行动。

可以抽象为： $a_t = \pi(o_t, m_t, g, c)$

a_t：时刻 t 选择的行动、o_t：当前观测信息、m_t：记忆或历史状态、g：任务目标、c：约束条件、π：智能体的决策策略。

3.1.1 推理的主要作用

推理主要完成三项任务：

1. 理解当前状态与任务目标

识别当前已经掌握的信息、尚未完成的工作以及最终目标。

2. 在多个候选行动之间作出选择

例如，面对用户查询，智能体可能选择：直接回答、调用搜索工具、查询数据库、请求补充信息。

3. 评估权衡、证据与约束

行动选择不仅取决于能否完成任务，还应考虑：证据是否充分、工具是否可用、时间与计算成本、安全性与权限、用户提出的约束条件。

3.1.2 推理的三个主要应用场景

场景	英文	作用
反思	Reflection	诊断问题并修正输出
工具使用	Tool Use	决定何时以及如何调用工具
规划	Planning	组织多步骤行动与依赖关系

这三种能力相互联系。

规划决定整体任务结构，工具使用负责执行具体操作，反思则利用结果反馈修正执行过程。

Part 4 自反思与反馈优化 Self-Reflection and Feedback

自反思（Self-Reflection）是指智能体对已有输出进行检查，识别其中的问题，并根据评价结果生成改进版本。

两种反思方式：基于模型评价的内部反思、基于工具执行结果的外部反馈反思。

4.1 基于模型评价的自反思

4.1.1 基本流程

Task → LLM Draft → Code V1 → LLM Review → Code V2

首先，模型根据任务生成初始代码；随后，模型检查代码是否符合用户要求，并生成修订版本。

例如：def do_task(args): ...

经过反思后生成：def do_task_v2(args): ...

其中，第二版代码应根据审查结果修正第一版存在的问题，而不只是改变函数名称。

4.1.2 反思模型的配置

Reviewer 可以使用不同的 LLM，例如专门用于推理的模型。因此，生成器与审查器不一定必须是同一个模型。

可以采用： $y_1=G(x)$ ， $f=R(x,y_1)$， $y_2=G(x,y_1,f)$

G：生成模型、R：审查模型、f：反馈信息、y_1：初始输出、y_2：修订输出。

使用不同模型可能提供互补的评价能力，但并不能保证审查结果一定正确。

4.2 基于外部反馈的自反思

4.2.1 基本原理

内部反思主要依赖模型对已有结果的判断，而外部反馈利用实际执行结果指导修改。

课件展示的流程为： Task → Draft → Code V1 → Execute → Output / Errors → Revise → Code V2

与内部反思相比，外部反馈提供了来自执行环境的可观察信息。

例如，程序执行后返回：SyntaxError: unterminated string literal (detected at line 1)

该错误表明代码中存在未正确结束的字符串字面量。

模型可以利用错误类型及行号定位问题，修改代码并重新执行。

4.2.2 内部反思与外部反馈的区别

维度	内部反思	外部反馈
反馈来源	模型自身或审查模型	代码解释器、测试工具、数据库等
主要依据	语义理解、规则与评价标准	实际输出、错误信息、测试结果
优势	不一定需要执行环境	具有更直接的可观察依据
局限	可能重复原有错误	反馈质量取决于测试与执行条件
典型场景	文本审查、需求检查	代码调试、SQL 验证

外部反馈的价值在于将模型的主观判断与实际执行结果结合起来。

但程序能够正常执行，并不意味着逻辑一定正确。因此，除检查运行错误外，还需要测试输出是否符合预期。

4.3 反思提示的设计原则 Reflection Prompt Design

课件指出，一个有效的反思提示应明确三个要素：

1. Review Action：需要执行什么审查操作。
2. Review Criteria：依据哪些标准进行检查。
3. Revised Output：最终需要输出什么修订结果。

案例一：域名筛选

课件要求模型检查之前生成的域名建议。

评价标准包括：是否容易发音、是否有利于口头传播、是否可能在其他语言中具有负面含义。

最终只输出符合标准的域名候选列表。

该案例说明，反思不一定意味着修改已有文本，也可以通过评价与筛选减少不合格候选项。

案例二：邮件优化

课件要求模型审查邮件初稿。

评价标准包括：语气是否专业、是否存在粗鲁或不敏感的表达、事实是否准确、日期是否准确、承诺是否准确。

审查完成后，模型应输出下一版本的完整邮件。

这一案例说明，反思提示应针对具体任务设置明确标准，而不能仅使用“改进一下”之类的模糊要求。

4.4 反思机制的实现与评估 Implementing and Evaluating Reflection

4.4.1 SQL 查询修正流程

课件以销售数据查询为例，说明如何通过反思提高查询准确性。

用户问题为：2025 年 5 月共售出了多少件商品?

系统执行流程：User Question 查询 2025 年 5 月商品销量

LLM：Generate SQL V1 生成初始 SQL 查询

LLM：Review V1 and Write SQL V2 审查并修正查询

Execute SQL V2 执行修订后的 SQL

LLM：Generate Final Answer 根据查询结果回答

课件展示的是先审查 SQL V1，再执行 SQL V2 的流程；并未展示 V1 的具体 SQL 语句或错误原因，因此不能据此断定第一版究竟遗漏了哪个字段或筛选条件。

4.4.2 反思前后的实验结果

测试问题	Ground Truth	No Reflection	With Reflection
2025 年 5 月售出多少件商品?	1201	980	1201
最昂贵的商品是什么?	Airflow sneaker	Airflow sneaker	Airflow sneaker
经营多少种款式?	14	14	14

- Ground Truth：标准答案。
- No Reflection：未使用反思机制的结果。
- With Reflection：使用反思机制后的结果。

在第一个问题中，未使用反思机制时得到 980，与标准答案相差： $E_{\text{before}} = |980 - 1201| = 221$

使用反思机制后： $E_{\text{after}}=|1201-1201|=0$

若计算相对误差： $\text{Relative Error} = \frac{|980-1201|}{1201}\times100\% \approx18.40\%$

反思使该示例中的相对误差从约 18.40% 降至 0%，另外两个问题在反思前后均正确。

因此，在课件展示的三个测试问题中： $\text{Accuracy}_{\text{before}}=\frac23\approx66.67\%$， $\text{Accuracy}_{\text{after}}=\frac33=100%$

这只是三个示例构成的小规模比较，不能直接推断反思机制在所有任务中的整体准确率。

4.4.3 反思机制的适用条件与局限

反思通常更适合以下任务：存在明确评价标准、可以获取外部反馈、允许多次生成与修改、错误能够通过审查或测试被发现。例如，代码生成、SQL 查询、结构化数据处理及文档审查都可以采用反思机制。

但反思也具有局限：

1. 增加计算与调用成本

反思通常需要额外的模型调用，甚至多次工具执行。

若初始生成成本为 C_g，审查成本为 C_r，执行验证成本为 C_e，则： $C_{\text{total}} = C_g + C_r + C_e$

2. 不保证每次修改都正确

审查模型可能遗漏错误，也可能错误地修改原本正确的内容。

3. 需要合理的终止条件

如果系统持续反思而没有明确停止标准，可能造成无效循环。

可以设置：最大迭代次数、评价分数阈值、测试通过条件、成本或时间预算。

例如，令第 k 次修订结果的质量分数为 Q_k，当： $Q_k \geqslant \tau$ 或达到最大迭代次数 $K_{\max}$ 时终止。

其中， τ 为预设质量阈值。

Part 5 Agentic RAG 与智能体工作流的综合应用

本讲开头回顾了 Agentic RAG System 与 RAG Workflow。它们可以作为前述智能体概念的具体应用。

5.1 RAG 的基本工作流

RAG（Retrieval-Augmented Generation）通过检索外部知识，为语言模型生成答案提供依据。

课件将完整流程划分为两个阶段。

5.1.1 离线准备 Offline Preparation

离线阶段负责构建可检索知识库。

主要步骤包括： $\text{Raw Documents} \rightarrow \text{Preprocessing} \rightarrow \text{Chunking} \rightarrow \text{Embedding / Indexing} \rightarrow \text{Knowledge Base}$

1. 原始数据准备

知识来源可以包括：PDF 文档、图像、视频、其他多模态数据。

多模态 RAG 可以通过适当的模型或预处理方法，将不同模态的信息转换为可检索表示。

2. 文档预处理

清理与规范化文档内容，使其适合后续检索。

3. 文档分块（Chunking）

课件列举三种方式：Fixed-size Chunking：固定长度分块、Semantic Chunking：语义分块、Context-aware Chunking：上下文感知分块。分块策略影响检索单元的语义完整性和相关性。

4. 向量化与索引

通过 Embedding 将文档片段转换为向量，并建立索引。ANN 与 HNSW 等近似最近邻检索技术能够支持大规模向量搜索。

5.1.2 在线推理 Online Inference

在线阶段从用户查询开始，经过检索、证据选择和答案生成。

$ \text{Query} \rightarrow \text{Query Processing} \rightarrow \text{Retriever} \rightarrow \text{Candidate Documents} \rightarrow \text{Final Evidence} \rightarrow \text{Generation} \rightarrow \text{Response} $

关键步骤包括：

1. 查询处理（Query Processing）

可以采用查询解析、查询改写或可选的 HyDE 方法改善检索输入。

2. 检索（Retrieval）

课件列举的检索方法包括：Bi-encoder：双编码器检索、ColBERT：基于后期交互的检索方法、ANN / HNSW：近似最近邻检索。

3. 证据筛选（Evidence Selection）

从候选文档中选择与问题最相关的证据，减少无关信息进入生成阶段。

4. 增强提示与生成（Augmented Prompt & Generation）

将用户问题与检索证据共同提供给模型，生成有依据的回答。Self-RAG 与 Agentic Workflow 可以参与检索和生成过程，通过判断是否需要更多证据来调整系统行为。

5.2 Agentic RAG 的决策与反馈机制

普通 RAG 往往采用相对固定的检索与生成流程，而 Agentic RAG 可以增加路由、证据评估与迭代检索。

其关键在于三个决策点。

1. 是否需要检索：Router LLM 判断当前问题能否直接回答，还是需要外部知识。
2. 检索证据是否充分：Evidence Evaluation 模块判断检索结果能否支持回答。如果证据不足，则重新检索。
3. 是否能够生成有依据的答案：当证据充分时，模型生成答案并引用相应来源。

这体现了智能体工作流中的条件分支与迭代反馈。

5.3 RAG 系统的主要优化方向

不同技术对应的优化目标：

技术	主要优化目标
Chunking	提高检索相关性
Bi-encoder / ColBERT	提高匹配质量
Query Rewriting / Parsing	改善搜索输入
Multimodal RAG	支持更丰富的证据类型
ANN / HNSW	提高大规模检索速度
Self-RAG	增强自适应控制与可靠性
Evaluation / Security	提高鲁棒性与部署准备程度

这些技术并非互相替代，而是分别作用于数据准备、检索、证据选择、生成控制与系统保障等不同环节。

Part 6 智能体的工具调用与动态决策 Tool Use and ReAct

前文已经介绍智能体的基本组成、工作流设计、推理与决策机制，以及自反思能力。本部分进一步讨论智能体如何选择和执行工具、如何根据外部反馈动态调整决策，以及如何通过代码执行扩展智能体的能力。

6.1 工具调用的决策机制 Tool-Use Decision Making

6.1.1 工具可用性与工具必要性

智能体能够访问某个工具，并不意味着每次任务都需要调用该工具。

Having access to a tool doesn't mean the agent always needs to use it.

工具调用涉及两个不同的问题：

1. Tool Availability（工具可用性）：系统提供了哪些工具？
2. Tool Necessity（工具必要性）：当前任务是否需要使用某个工具？

例如，系统提供 `get_current_time` 工具时：

用户问题	是否调用工具	原因
What time is it?	是	当前时间属于动态外部信息
How much caffeine is in green tea?	通常不需要	可以利用已有知识给出一般性估计

对于第一个问题，执行流程为： $\text{Query} \rightarrow \text{LLM} \rightarrow \texttt{get_current_time} \rightarrow \text{Tool Result} \rightarrow \text{LLM} \rightarrow \text{Answer} $

课件中的工具返回 `15:20:45`，模型将其整理为 `It's 3:20 p.m.`对于第二个问题，模型可以直接回答：一杯绿茶通常含有约 25 ~ 50 mg 咖啡因。具体含量取决于茶叶种类、冲泡时间和用量。

这说明工具调用是智能体根据任务需求作出的独立决策，而不是固定执行步骤。

5.1.2 工具选择的判断依据

工具选择通常需要考虑以下因素：

- 信息时效性：是否需要当前时间、实时价格或最新记录。
- 外部依赖：是否需要访问数据库、文件或其他系统。
- 计算复杂度：是否需要精确计算或程序执行。
- 知识充分性：模型已有信息是否足以支持回答。
- 执行必要性：任务是否要求实际操作，而非仅生成文本。

可以将工具调用决策抽象为： $a_t = \pi(q_t, s_t, \mathcal{T})$

q_t：当前任务或查询；s_t：当前系统状态；$\mathcal{T}$：可用工具集合；a_t：选择的行动，包括调用工具或直接回答。

5.2 基于代码的工具调用 Code-Based Tool Use

工具可以通过普通编程函数定义，再向 LLM 提供其名称、功能描述和参数信息。

使用 Python 的 `datetime` 模块定义时间查询工具

- `datetime.now()`：获取运行环境的当前本地时间。
- `strftime("%H:%M:%S")`：将时间格式化为字符串。
- Docstring：描述函数的用途，帮助模型理解工具能力。

随后，课件使用 `aisuite` 将工具提供给模型：在该示例中，`aisuite` 可以根据 Python 函数信息生成模型可理解的工具描述，使模型能够判断是否调用 `get_current_time`。

需要区分两个层次：模型负责决定调用什么工具，运行环境负责真正执行工具。

因此： $\text{Tool Calling} = \text{Tool Selection} + \text{Argument Generation} + \text{Tool Execution} + \text{Result Processing} $

工具执行通常需要由运行时程序完成。模型本身生成工具调用请求，并不等于已经执行相应操作。

5.3 工具之间的依赖关系 Dependencies Between Tools

复杂任务往往需要连续调用多个工具，而后续调用可能依赖前一次调用的结果。

日历预约案例

用户请求：Find a free slot on Thursday in my calendar and make an appointment with Alice.

系统提供三个工具：`check_calendar, make_appointment, delete_appointment`

$ \text{User Request} \rightarrow \texttt{check_calendar} \rightarrow \text{Available Slots} \rightarrow \texttt{make_appointment} \rightarrow \text{Confirmation}$

1. LLM 选择 `check_calendar`。
2. 工具返回周四 `15:00`、`16:00`、`18:00` 可用。
3. LLM 选择 `make_appointment`。
4. 工具返回预约创建成功。
5. LLM 确认预约时间为周四 `15:00`。

数据依赖与执行顺序

预约工具需要使用日历查询得到的可用时间，因此： `check_calendar` $\prec$ `make_appointment`

其中，$\prec$ 表示前置依赖关系。

类似地，只有预约工具确认成功后，系统才能向用户报告预约已经完成。

因此： $\text{Successful Booking} \prec \text{Success Confirmation} $

这一案例说明：工具执行顺序应由数据依赖与实际执行结果决定，而不能仅根据预先生成的步骤机械执行。

5.4 ReAct：推理与行动的协同 Reasoning and Acting

ReAct 是一种将推理、行动和环境观察结合起来的智能体执行方法。

ReAct 包含三个核心环节：

Reason: 分析当前状态并选择下一步行动

Act: 调用工具或执行操作

Observe: 读取工具返回结果或环境反馈

利用新观察更新状态，继续下一轮推理

在时刻 t，智能体根据当前状态选择行动： $ a_t=\pi(s_t)$

执行行动后，环境返回观察： $ o_{t+1}=\operatorname{Execute}(a_t)$

系统利用观察更新状态： $s_{t+1}=\operatorname{Update}(s_t,a_t,o_{t+1}) $

随后根据更新后的状态决定下一步行动： $a_{t+1} = \pi(s_{t+1})$

智能体的决策不是一次性完成，而是能够随着新信息的出现不断调整。

Tool Calling、ReAct 与 Reflection 的区别

概念	主要作用	核心问题
Tool Calling	提供执行外部行动的接口	如何调用工具?
ReAct	结合推理、行动与观察	如何根据新信息调整下一步行动?
Reflection	检查并修正已有结果	如何发现和纠正错误?

三者并不互斥。ReAct 可以使用工具调用执行行动，也可以在循环内部加入 Reflection 来检查中间结果。

ReAct 与其他提示方法的比较

引用 ReAct 论文中的 HotpotQA 示例，对比四种方法：

方法	主要特征	局限
Standard	直接生成答案	缺少显式推理与外部验证
CoT（Reason Only）	通过多步推理生成答案	可能基于错误知识持续推理
Act-Only	调用工具获取信息	缺少明确的推理指导
ReAct（Reason + Act）	交替进行推理、工具调用和观察	需要管理循环、工具成本与终止条件

示例问题：除了 Apple Remote 之外，最初设计用于交互的程序还可以通过什么设备控制?

课件中的 ReAct 示例通过搜索逐步发现：Apple Remote 与 Front Row 有关、Front Row 是 Apple 的媒体中心软件、Front Row 也可以通过键盘功能键控制。

最终得到正确答案：Keyboard function keys（键盘功能键）。

这一案例体现了 ReAct 的优势：通过外部检索纠正不完整或错误的初始判断。

5.5 为什么需要协同推理与行动?

新证据可以改变下一步行动

课件继续使用日历预约案例，展示了环境状态可能发生变化。

初次查询显示 `15:00` 可用，但预约执行时该时间已被占用。

因此，系统不能继续按照原计划执行，而应根据错误反馈重新查询并选择其他时间。

其决策过程为： Plan → Act → Observe Conflict → Replan → Act Again

这也是 ReAct 相较于固定工作流的重要优势：能够处理动态环境中的执行失败与状态变化。

新证据可以改变最终答案

以校园 AI 使用政策为例。

任务：Write an evidence-backed summary of the campus rules on AI use.

执行过程分为三个阶段：

Search：初步检索，搜索结果声称学校允许使用 AI。

Read：阅读权威来源，进一步阅读官方页面后发现，允许使用 AI 存在附加条件：必须遵守具体作业规定。

Update：更新答案，最终回答应缩小原有结论的适用范围，明确引用官方条件，并在缺少具体作业规则时建议用户向相关负责人确认。

这一案例说明，ReAct 不仅可以改变工具调用顺序，也可以根据新证据修正最终结论。

5.6 ReAct 的最小实现与终止机制

有界执行循环 Bounded Loop

ReAct 的三种终止结果

每次循环都会消耗时间，也可能产生新的外部影响，因此必须设置停止规则。

1. Finish：正常完成

当任务成功条件满足，且结果具有充分证据支持时，终止。

2. Recover：有限恢复

当出现暂时性错误时，可以在固定预算内重试。

例如：网络请求暂时失败、数据库连接超时、日历预约发生时间冲突。

3. Escalate or Stop：升级或停止

当出现以下情况时，应停止自动执行：缺少必要权限、证据不足、已耗尽执行预算、达到最大迭代次数。

可以定义： $N_{steps} \leqslant N_{max}$以及 $C_{total} \leqslant C_{max}$

其中，N_{max} 是最大执行步数，C_{max} 是允许的最大执行成本。

需要特别注意，达到步数上限并不意味着任务已经成功。系统应明确报告未完成状态，而不是生成缺乏依据的成功结论。

5.7 代码生成与执行能力 Code Generation and Execution

5.7.1 通过代码执行扩展模型能力

LLM 可以生成代码，再交由外部运行环境执行。

课件以计算 $\sqrt{2}$ 为例： $ \sqrt{2}\approx1.4142135623730951$

系统提示要求模型将 Python 代码放入特定标签：

Write code to solve the user's query. Return your answer as Python code delimited with <execute_python> and </execute_python> tags.

模型生成：import mathprint(math.sqrt(2))

执行结果：1.4142135623730951

模型读取执行结果后，生成自然语言回答。

整体流程为： $ \text{Question} \rightarrow \text{LLM Generates Code} \rightarrow \text{Execute Code} \rightarrow \text{Result} \rightarrow \text{LLM Answer}$

与直接依赖模型生成数值相比，代码执行可以为确定性计算提供更可靠的结果。

5.7.2 生成代码的安全执行

直接执行 LLM 生成的代码可能产生严重风险。例如，错误生成的删除命令可能删除项目目录中的 Python 文件。因此，代码执行需要受控环境。

Sandbox（沙箱） 是一种隔离执行机制，用于限制代码可以访问的文件、资源和系统操作。

两个相关工具：Docker：容器化运行环境、E2B：面向代码执行的沙箱服务。

需要注意，Docker 容器并不天然等于安全沙箱。安全性还取决于实际配置，例如文件挂载、网络权限、用户权限和资源限制。

实际系统通常需要控制：文件系统访问权限、网络访问权限、CPU 与内存使用、执行超时时间、系统命令权限、敏感数据访问。

安全原则：模型可以生成代码，但代码的实际执行必须受到运行环境的独立约束。

Part 6 智能体规划与结构化执行 Planning and Structured Execution

6.1 规划能力的定义

Planning 是将复杂任务组织为多个有序行动的能力。

与单次工具调用相比，规划更关注：完成任务需要哪些步骤、每个步骤应该使用什么工具、步骤之间是否存在依赖、如何利用前一步结果执行后续步骤。

规划可以表示为： $P = (a_1, a_2, \cdots, a_n)$。其中， a_i 表示第 i 个行动。

若行动之间存在依赖关系，则必须满足相应的执行顺序。

6.2 客户服务智能体的规划案例

用户询问：Do you have any round sunglasses in stock that are under $100?

完成当前任务只需要前三个工具。合理规划为：

1. 使用 `get_item_descriptions` 查找圆形太阳镜。
2. 使用 `check_inventory` 判断候选商品是否有库存。
3. 使用 `get_item_price` 检查有库存商品是否低于或等于 100 美元。

整体依赖关系为： $\text{Product Search} \rightarrow \text{Inventory Check} \rightarrow \text{Price Check} $

只有满足商品形状、库存和价格三个条件，才能给出符合要求的推荐。

Example：邮件助手中的行动依赖

Reply to that email invitation from Bob about dinner in New York and tell him I'll attend. Then archive his email.

可用工具包括：`search_email, send_email, move_email, delete_email`

执行计划为：

1. 搜索 Bob 发送的、提及纽约晚餐的邀请邮件。
2. 使用 `send_email` 回复并确认出席。
3. 使用 `move_email` 将原邮件移动到归档文件夹。

依赖关系为： $ \text{Search Email} \prec \text{Send Reply} \prec \text{Archive Email} $

归档操作应在回复成功后进行，以免任务尚未完成就改变邮件状态。

说明：规划不仅是列出步骤，还必须明确步骤之间的依赖关系与完成条件。

6.4 使用 JSON 表示结构化计划

JSON（JavaScript Object Notation）是一种文本形式的结构化数据交换格式。它能够让模型生成的计划被程序解析、验证和执行。JSON 要求对象键名和字符串值使用双引号。

JSON 序列化与反序列化

Python 使用 `json` 模块进行转换：

- `json.dumps()`：将 Python 对象序列化为 JSON 字符串。
- `json.loads()`：将 JSON 字符串解析为 Python 对象。

转换过程为： $\text{Python Object} \xrightarrow{\texttt{json.dumps()}} \text{JSON String} $, $\text{JSON String} \xrightarrow{\texttt{json.loads()}} \text{Python Object}$

解析 JSON 只完成数据转换，并不会自动执行其中描述的操作。

将行动计划表示为 JSON

要求每个计划步骤包含四个字段：

`step`：步骤编号；`description`：步骤描述；`tool`：工具名称；`args`：工具参数。

这种结构使程序能够识别工具名称、参数和执行顺序。但其中的 `"results from step 1"` 只是描述性占位符。实际执行时，运行环境需要将其解析为第一步返回的真实数据。

因此，结构化计划执行至少包含： Generate Plan → Parse JSON → Validate Plan → Resolve Dependencies → Execute Tools

对于更复杂的系统，还可以增加参数类型验证、工具白名单及异常处理机制。

6.5 多工具规划的复杂性

咖啡店交易数据说明：当工具数量和任务类型增加时，规划难度也会提高。

Find the month with the highest hot-chocolate revenue.

多工具组合问题

这里假设每条记录代表一件商品交易，且 `price` 是该交易的实际收入；若存在数量、退款或折扣字段，还需要调整计算方法。

多工具规划的主要困难

两个核心问题：

- More combinations：工具组合数量增加。
- More edge cases：边界情况增加。

例如，系统需要处理：日期格式不一致、商品名称存在不同写法、数据包含缺失值、查询条件涉及多个字段、工具之间存在复杂依赖。

随着工具数量增加，模型不仅需要选择正确的工具，还必须决定调用顺序和参数传递方式。

通过代码执行简化复杂规划

对于复杂的数据分析任务，可以让 LLM 生成完整代码，再由运行环境执行。

“查询最近五笔交易金额”

- `pd.read_csv()`：读取交易数据。
- `pd.to_datetime()`：将日期转换为时间类型。
- `sort_values()`：按日期降序排列。
- `head(5)`：选取最近五条记录。
- `to_list()`：将金额转换为列表。

与连续调用多个细粒度工具相比，代码执行能够将筛选、排序和聚合等操作组合为完整程序。但这并不意味着代码执行一定优于工具调用。

代码执行需要额外考虑：代码正确性、执行环境安全、文件访问权限、异常处理、结果验证。

因此，适合采用哪种方式取决于任务复杂度、工具能力与执行风险。

6.6 单智能体能力总结

核心能力：

能力	示例	主要作用
Reflection	修改邮件、审查代码	检查并改善已有输出
Tool Use	查询当前时间、创建预约	获取外部信息并执行行动
ReAct	预约冲突后重新查询日历	根据新观察调整行动
Planning	查询商品、库存与价格	组织多步骤任务及依赖关系

这四项能力可以组合在同一个智能体中。然而，单个智能体不一定适合所有复杂任务。某些任务需要多个具有不同专业能力或评价标准的决策角色，因此引出多智能体系统。

Part 7 多智能体系统基础 Multi-Agent Fundamentals

7.1 多智能体系统的定义

多智能体系统（Multi-Agent System, MAS）由多个能够进行决策的智能体组成，这些智能体在共享任务或环境中相互作用。其核心不是简单增加 LLM 数量，而是引入多个具有独立职责、局部状态和决策能力的实体。

要素	含义
Agents	多个能够基于观察和历史信息选择行动的智能体

要素	含义
Goals	智能体可能具有共享、不同或相互竞争的目标
Environment	多个智能体共同观察和改变的环境
Interaction	智能体通过行动、消息或共享状态相互影响

某些复杂任务包含多个互补的专业角色。多个智能体能够将不同知识、工具和职责分配给专门的角色。但并非所有任务都需要多智能体架构。只有当新增智能体能够解决具体任务限制时，才具有引入价值。

7.2 多智能体协作案例

一家商店希望推广售价为 HK$299 的圆形、哑光黑色太阳镜。

目标输出：一张宣传图片、一段不超过 50 个英文单词的宣传文案。系统设置三个专业智能体。

1. Researcher Agent

负责：选择信息来源、检查产品相关声明、确认宣传内容是否有证据支持。

输出：已核实的产品事实、来源说明。

2. Designer Agent

负责：选择视觉风格、设计并修改宣传图片、确保图片符合任务要求。

输出：与营销需求一致的宣传图片。

3. Writer Agent

负责：撰写宣传文案、根据反馈修改内容、控制文案长度。

输出：清晰且不超过 50 个单词的宣传文案。

三个智能体围绕同一营销目标工作，但分别承担事实核查、视觉设计和文本创作。

7.3 采用多智能体系统的五个理由

1. 专业化能力（Specialized Expertise）

不同智能体可以具有不同知识、工具或职责。

例如：Researcher 侧重事实与证据、Designer 侧重视觉表达、Writer 侧重语言与传播效果。

专业化的核心是能力分工，而不是简单地将同一任务重复交给多个模型。

2. 独立并行工作（Independent Parallel Work）

当任务之间不存在必要依赖时，可以并行执行。

例如： $\text{Approved Facts} \rightarrow \begin{cases} \text{Visual Design}\\ \text{Copywriting} \end{cases} \rightarrow \text{Campaign}$

设计与文案可以在共享已核实事实的基础上同时进行，再整合为最终宣传素材。

需要区分：Specialization 关注不同智能体具备什么能力、Parallelism 关注任务之间是否存在执行依赖。专业化不意味着一定能够并行执行。

3. 上下文隔离（Context Isolation）

每个智能体可以维护独立工作上下文，减少无关信息干扰。

Local Context每个角色只保留完成自身任务所需的证据和历史信息。

Shared Constraints 多个角色共享统一的任务要求和已核实事实。

Handoff 智能体之间传递结论、来源和未解决问题。

例如：Researcher 保存详细检索记录、Designer 主要接收产品视觉要求与已确认事实、Writer 主要接收营销目标、产品事实和长度限制。这样可以减少上下文冗余，同时保持关键约束的一致性。

4. 独立验证（Independent Verification）

独立验证用于检查已有结果是否满足证据、约束与推理要求。

例如，Writer 生成：These sunglasses are the most popular product this summer.

如果缺乏销量或市场调查数据，这一声明可能没有充分依据。独立审查智能体可以质疑原有假设，并要求删除或修改未经证实的表述。因此，验证不仅检查文本是否通顺，更重要的是检查结论是否受到证据支持。

5. 多视角评价（Multiple Perspectives）

不同智能体可以采用不同评价标准。

例如：Researcher：事实准确性、Designer：视觉质量、Writer：语言清晰度与传播效果。

最终决策需要在不同标准之间进行权衡。

需要特别区分：Independent Verification 主要关注结果是否正确、有依据；Multiple Perspectives 主要关注不同评价标准之间的平衡。

7.4 多智能体系统的目标与成功标准

Completing individual subtasks does not mean the overall task is complete.

即使所有智能体分别完成自己的工作，也不代表整体任务已经成功。

因此，成功标准需要在三个层次上定义。

层次	核心目标	评价重点
Task Level	交付可使用的最终成果	输出完整性、任务约束
Agent Level	完成分配的职责	单个贡献的质量与完整性
Interaction Level	可靠整合多个贡献	一致性、问题解决、终止条件

例如，太阳镜营销任务中：

- Researcher 必须提供可靠产品信息。
- Designer 必须生成符合要求的图片。
- Writer 必须提供合格文案。
- 整体系统必须保证图片与文案一致，并符合最终交付要求。

逻辑关系：$\text{Task Success} \Rightarrow \text{Required Outputs Valid} \land \text{Constraints Satisfied} \land \text{Integration Valid} $

单个智能体完成任务只是整体成功的必要组成部分，不一定构成充分条件。

7.5 单个智能体角色的定义

为了使多智能体系统具有明确的职责边界，课件提出六个角色描述字段。

字段	需要回答的问题
Name	如何标识该智能体?
Inputs	可以使用哪些证据和上下文?
Responsibilities	负责哪些决策?
Tools	允许使用哪些工具和行动?
Output	必须交付什么结果?
Goal	应追求什么目标?

一个清晰的角色定义应将职责与可检查的行动和输出联系起来。这种结构有助于明确智能体的职责边界，避免多个角色重复执行相同任务或遗漏关键工作。

7.6 多智能体目标关系 Goal Relationships

多个智能体之间的目标关系可以分为三类。

1. 合作型（Cooperative）

多个智能体通过共享成果共同受益。

主要协调问题包括：贡献分配、信息共享、联合成功标准。

例如，Researcher、Designer 和 Writer 共同完成营销活动。

2. 竞争型（Competitive）

一个智能体的成功可能以其他智能体的利益受损为代价。

主要协调问题包括：策略选择、资源分配、信息访问。

例如，多个竞价智能体竞争有限资源。

3. 混合目标型（Mixed-Motive）

智能体在部分目标上合作，在其他目标上存在冲突。

主要协调问题包括：谈判、局部激励协调、共同目标与个体目标的平衡。

三种关系的区别如下：

目标关系	主要特征	协调重点
Cooperative	共享成果	合作与信息共享
Competitive	利益可能相互冲突	策略与资源竞争
Mixed-Motive	部分合作、部分竞争	谈判与激励协调

多智能体系统的交互机制应根据目标关系设计，而不能默认所有智能体始终具有完全一致的利益。

7.7 多智能体交互、记忆

社会模拟中的生成式智能体 Generative Agents

该研究使用多个生成式智能体模拟虚拟社区中的日常行为。每个智能体拥有自己的日程、记忆和计划，并通过行动和对话影响其他智能体。

研究关注的问题包括：智能体是否具有共同目标？每个智能体能够观察到什么？一个智能体如何影响其他智能体？不同智能体之间有哪些差异？

社会模拟强调：多智能体系统中的整体行为可以由多个个体的局部行动与交互共同产生。

7.7.1 生成式智能体的记忆与交互机制

1. Memory Stream：记忆流

智能体将观察到的事件、经历和交互记录保存到记忆中。

例如：与其他角色的对话、曾经执行的行动、环境中发生的事件、已经制定的计划。

2. Memory Retrieval：记忆检索

智能体并非每次都读取全部历史记录，而是检索与当前情境相关的记忆。

检索结果用于支持当前决策。

3. Reflection：反思

反思能够将具体经历整合为更高层次的理解。

例如，智能体可能从多次互动中总结其他角色的习惯或偏好。

4. Planning：规划

智能体根据当前目标、环境状态和相关记忆制定行动计划。

5. Social Interaction：社会交互

智能体的行动和对话会改变共享环境，并成为其他智能体的新观察。

因此，系统形成持续循环： Perceive → Store Memory → Retrieve → Reflect / Plan → Act → New Observations

记忆使智能体能够保持行为连续性，交互则使多个智能体之间产生动态影响。

7.7.2 局部观察与上下文 Local Observations

多个智能体可以共享同一个环境，但不一定拥有相同的信息。

1. World State：世界状态。环境包括其他智能体及其行动。
2. Local Observation：局部观察。每个智能体只能获取自身可访问的信息。
3. Private Context：私人上下文。不同智能体可以拥有不同的记忆、信念和计划。

假设真实环境状态为 s_t，智能体 i 的观察为： $o_t^i = O_i(s_t)$

其中，O_i 是智能体 i 的观察函数。

即使处于同一环境，也可能存在： $o_t^i\neq o_t^j $

因此，两个智能体可能基于不同信息作出不同决策。

这种现象称为 Partial Observability（部分可观测性）。

7.8 多智能体系统的主要挑战

多智能体系统由于包含多个决策主体，会引入信息、协调和学习方面的复杂性。

7.8.1 部分可观测性 Partial Observability

不同智能体掌握的信息不完整，可能导致不兼容的决策。

例如，一个智能体已经修改任务状态，但另一个智能体尚未收到更新，仍然按照旧状态执行。

因此，系统需要合理设计状态共享、消息传递和信息同步机制。

7.8.2 规模扩展 Scale

随着智能体数量增加，系统需要处理更多：消息路由、工具调用、协调过程、结果整合、计算与通信成本。

更多智能体不一定意味着更高效率。

7.8.3 策略依赖 Policy Dependence

一个智能体行动的价值可能取决于其他智能体的行为。

例如，在竞争资源分配的任务中，某个智能体是否应该立即行动，可能取决于其他智能体是否也在竞争相同资源。

可以表示为： $ U_i=U_i(a_i,a_{-i})$

- U_i：智能体 i 的效用。
- a_i：智能体 i 的行动。
- a_{-i}：其他智能体的联合行动。

因此，智能体不能总是独立评估自身行动的价值。

7.8.4 非平稳性 Non-Stationarity

当多个智能体同时学习或调整策略时，环境对于某个智能体而言可能不断变化。

例如，智能体 j 的策略从： $ \pi_j^{(t)} $，更新为： $ \pi_j^{(t+1)} $

那么即使智能体 i 策略没有变化，它面对的交互环境也可能发生改变。

这会增加策略学习、行为预测和系统稳定性维护的难度。

7.9 综合总结

从单智能体到多智能体，系统设计关注点发生了变化。

维度	单智能体系统	多智能体系统
决策主体	一个主要决策实体	多个决策实体
任务组织	内部规划与工具调用	角色分工与协同执行

维度	单智能体系统	多智能体系统
信息管理	单一主要工作上下文	局部上下文与共享状态
反馈机制	工具结果、自反思	个体反馈与智能体间交互
主要挑战	工具选择、规划、执行安全	协调、通信、信息不完整、策略依赖
成功标准	完成整体任务	个体贡献与整体协作均满足要求

Tool Use → ReAct → Planning → Multi-Agent Coordination

其中：

- Tool Use 使智能体能够获取外部信息并执行行动。
- ReAct 使智能体能够根据新观察动态调整决策。
- Planning 使智能体能够组织多步骤任务及其依赖关系。
- Multi-Agent Coordination 使多个具有不同职责和局部信息的智能体能够共同完成任务。

这些能力之间并不是简单的替代关系，而是可以相互组合。

最终，智能体系统设计的核心问题是：如何将模型、工具、推理、规划、记忆和交互机制组织为一个能够可靠完成任务的系统，同时控制执行成本、风险和协调复杂性。

Part 8 多智能体系统设计 Designing Multi-Agent Systems

前文介绍了多智能体系统的基本概念、角色分工、目标关系、局部观察和交互机制。

本章进一步讨论如何将这些概念转化为可实施的系统架构，重点解决任务如何分配、智能体如何协调、工作流如何组织、信息如何交换，以及系统如何安全终止和评估等问题。

多智能体系统设计并非简单地增加智能体数量，而是根据任务依赖、控制需求和协调成本，构建具有明确职责、执行流程和通信规则的协作系统。

8.1 多智能体系统的核心设计维度 Design Decisions

多智能体系统设计包含四个相互关联的维度。

设计维度	核心问题	主要设计内容
Task and Roles（任务与角色）	谁负责每项决策和输出？	任务分解、职责分配、权限边界
Coordination（协调机制）	谁负责分配工作和解决冲突？	集中式、去中心化、混合式协调
Workflow / Architecture（工作流与架构）	智能体、控制逻辑和状态如何组织？	顺序、并行、迭代及图结构工作流
Communication（通信机制）	哪些信息需要传递，在哪里保存？	消息格式、路由、共享状态、版本控制

四个维度分别解决不同层次的问题：

- 任务与角色决定系统中有哪些工作，以及由谁负责。
- 协调机制决定决策权和任务分配权如何组织。
- 工作流与架构决定各项任务如何随时间执行。
- 通信机制决定智能体之间如何交换信息、共享状态和传递结果。

区分：协调模型描述的是谁作出决策，执行工作流描述的是任务以什么顺序执行。两者相关，但并不等同。

8.2 任务分解、职责分配与角色契约

8.2.1 任务分解与分配 Task Decomposition and Allocation

任务分解（Decomposition）是将总体目标拆分为具体输出和子任务，并识别它们的依赖关系与所需输入。

任务分配（Allocation）则是将这些子任务交给具有相应能力的智能体，并明确责任归属和交付条件。

二者的区别如下：

维度	Decomposition	Allocation
核心问题	需要完成哪些工作？	由谁完成这些工作？
主要内容	子任务、输入、输出、依赖	能力匹配、责任归属、交付条件
主要产物	任务分解结构	角色与任务映射关系

任务分解时应明确每个子任务的输入、输出及前置条件，避免将存在数据依赖的任务错误地安排为并行执行。

8.2.2 角色边界、工具与权限 Role Boundaries, Tools and Permissions

角色契约（Role Contract）不仅需要说明智能体应当完成什么工作，还必须明确它被允许执行哪些操作。

角色边界分为三个方面：

1. Decision Ownership：决策归属

明确智能体能够独立作出哪些决策，以及哪些决策需要其他角色或人工确认。

2. Tool Access：工具访问权限

明确智能体能够访问哪些数据，以及允许执行哪些系统操作。

3. Escalation Boundary：升级处理边界

明确哪些不确定情况、冲突或高风险操作必须提交上级协调者或人工处理。

这一原则体现了最小权限设计思想（Principle of Least Privilege）：每个智能体只应获得完成其职责所必需的操作权限。

8.2.3 智能体之间的交接契约 Handoff Contracts

Handoff 指一个智能体完成阶段性工作后，将结果及必要上下文传递给另一个智能体。

有效的交接不应只包含“任务已完成”的通知，而应提供能够支持下一步决策的可用产物。

四项交接内容：

组成部分	应包含的信息
Content	结论、结果及支持证据
Traceability	信息来源和产物版本
Status	已验证、不确定或等待审查
Open Questions	尚未解决的问题以及下一责任方

这种结构能够减少信息丢失，并使后续智能体判断哪些内容可以直接使用，哪些内容需要进一步验证。

8.3 多智能体协调模型 Coordination Models

协调机制决定任务分配、决策和结果整合发生在系统的什么位置。

三种主要模式：集中式、去中心化和混合式。

1. 集中式协调（Centralized Coordination）

集中式架构由一个 Manager Agent 负责分配任务、协调执行并整合结果。

其典型流程为： Manager → Task Assignment → Worker Execution → Manager Integration

优势是责任明确、便于统一规划和结果整合。

主要代价是 Manager Bottleneck（管理者瓶颈）：随着任务数量增加，中央管理者可能承担过多通信、决策和整合工作。

2. 去中心化协调（Decentralized Coordination）

去中心化架构中，各智能体可以直接与其他智能体交换信息，并在本地作出决策。

其特点包括：

- 没有唯一的中央决策者。
- 智能体能够根据局部状态自主协商。
- 信息与任务可以在多个节点之间流动。

这种模式适合规模较小、需要灵活交流的协作团队。但由于缺少统一控制者，系统更难达成全局一致决策，可能出现重复工作、冲突或信息不一致。

3. 混合式协调（Hybrid Coordination）

混合式架构结合集中式的全局规划和去中心化的局部自主性。

例如： $\text{Global Manager} \rightarrow \begin{cases} \text{Team A Coordinator}\ \text{Team B Coordinator} \end{cases} \rightarrow \text{Local Workers} $

上层负责整体目标、资源分配和跨团队协调，下层负责具体任务执行与局部决策。

这种模式可以降低中央管理者的直接负担，但需要设计额外的层级通信和责任边界。

三种模式的对比如下：

模型	决策方式	主要优势	主要代价
Centralized	中央管理者分配与整合	全局控制清晰	中央瓶颈
Decentralized	智能体自主协调	灵活性高	全局一致性较难
Hybrid	全局规划与局部自治结合	兼顾控制与自主性	架构复杂度增加

8.4 执行工作流与系统架构 Execution Workflows and Architecture

协调模型回答“谁负责决策”，执行工作流回答“任务如何随时间组织”。

8.4.1 顺序、并行与迭代工作流

课件介绍三种基本执行模式。

对于顺序工作流，若任务耗时为 $T_1, T_2, \cdots, T_n$，忽略调度开销： $ T_{\text{sequential}}=\sum_{i=1}^{n}T_i $

对于相互独立且同时启动的并行任务： $T_{\text{parallel}} \approx \max(T_1, T_2, \cdots, T_n)$

实际系统还需要考虑调度、通信与结果整合开销。

迭代工作流则需要明确退出条件，避免无界循环。

8.4.2 对等网络与层级架构 Peer Networks and Hierarchies

智能体之间的连接方式也会影响通信成本与协调效率。

Peer Network（对等网络）

智能体之间可以直接通信，具有较高灵活性，但需要消息路由规则和通信预算。

若 n 个智能体形成完全连接的无向网络，潜在连接数量为： $E=\frac{n(n-1)}{2} $

因此，完全连接网络的连接规模为： $E = O(n^2)$

这里描述的是潜在连接数量，不代表每次任务都必须发生这么多次通信。

Hierarchical Architecture（层级架构）

智能体被组织为多个局部团队，由上层协调者处理跨团队信息交换。

这种方式能够减少直接跨团队通信，但增加了信息传递层级。

因此，对等网络与层级架构的主要权衡是： $\text{Communication Flexibility} \quad\text{vs.}\quad \text{Coordination Overhead} $

8.4.3 黑板架构 Blackboard Architecture

黑板架构允许多个智能体通过共享工作空间进行协作，而不必为每项任务建立直接通信连接。

共享工作空间可以保存：Evidence：证据、Requests：任务请求、Drafts：草稿、Priorities：优先级、Status：执行状态。智能体读取共享状态，并向其中提交自己的工作成果。

黑板架构需要解决三个问题。

1. Shared Workspace：共享工作空间

确保各智能体能够访问相关任务状态和中间结果。

2. Contributions and Control：贡献与控制

通过调度或控制策略决定下一步应由哪个智能体作出贡献。

3. Consistency：一致性

对共享产物进行版本管理，并处理多个智能体同时写入时的冲突。

例如，Researcher 更新了产品事实后，Writer 应基于最新已核实版本修改文案，而不是继续使用旧信息。

因此，共享可见性不等于自动实现协调。黑板架构仍需要明确调度、权限和一致性规则。

8.4.4 图结构与混合工作流 Graph-Based and Hybrid Workflows

图结构工作流将执行节点、数据依赖和路由条件显式表示出来。

可以抽象为有向图： $G = (V, E)$

- V：执行节点集合。
- E：节点之间的依赖关系或状态转移。

组成部分	作用
Node Types	定义 Agent、模型调用、工具、函数和人工输入等节点
Routes	定义分支、汇合与循环的条件
Guards	检查权限、状态、合法转移和执行限制

例如，内容生成工作流可以设计为：这种设计的优势是控制流明确，便于验证任务状态、执行条件和终止规则。

8.5 系统架构选择 Architecture Selection

课件根据任务结构、控制需求和协调成本，比较了五类架构模式。

架构模式	适用情况	主要权衡
Pipeline	依赖关系稳定的任务	上游错误容易向下游传播
Centralized	需要统一任务分解与结果整合	管理者瓶颈
Peer Network	小规模团队需要灵活交流	消息与协调成本
Blackboard	多个智能体共同维护动态状态	一致性与调度
Graph	存在明确分支、循环和混合节点	控制流设计与维护成本

架构选择应优先分析任务结构，而不是直接选择最复杂的模式。

例如：

- 稳定的数据处理流程适合 Pipeline。
- 需要统一分工的研究团队适合 Centralized。
- 小规模自主协商任务适合 Peer Network。
- 需要持续共享中间成果的任务适合 Blackboard。
- 包含条件分支、人工审批和重试循环的任务适合 Graph。

需要注意，这些架构并不完全互斥。实际系统可以采用集中式协调，同时通过图结构定义工作流，并使用共享状态保存执行结果。

8.6 多智能体通信设计 Communication Design

通信不仅是信息传输机制，也是影响智能体后续决策的重要组成部分。

一个消息可能改变另一个智能体的知识状态，从而改变其下一步行动。

通信设计的四个方面

设计方面	核心问题
Message Format	传递什么结果、请求或证据?
Routing and Turn-Taking	谁接收消息，谁下一步行动?
Shared State	使用哪个产物及其版本?
Termination	消息是否触发继续、完成或升级?

通信系统必须能够识别消息发送者、接收者、任务上下文、执行状态以及后续处理责任。

结构化消息模式（Message Schema）

这一结构使系统能够明确：Writer 请求 Researcher 检查第二版文案中的“今年夏天最受欢迎”声明是否有证据支持。

消息验证机制

要求在消息接收和回复后进行不同层次的验证。

1. Structure Validation：结构验证

检查必需字段是否存在，字段类型是否合法。

例如，`task_id` 应为有效任务标识，`recipient` 应为可识别的智能体。

2. Permission and State Validation：权限与状态验证

检查：发送者是否有权发起该请求、接收者是否允许执行指定操作、当前任务状态是否允许该转移、产物版本是否仍然有效。

3. Evidence Validation：证据验证

当 Researcher 返回结果后，需要进一步判断来源是否支持原有声明。

因此： Valid Message ≠ Valid Claim，结构正确、权限合法的消息，其内容仍可能缺乏事实依据。

8.7 直接消息与共享状态 Direct Messaging and Shared State

智能体可以通过直接消息或共享产物交换信息。

1. 直接消息（Direct Messaging）

直接消息具有明确的发送者和接收者，通常用于：

- 请求其他智能体执行操作。
- 传递特定任务结果。
- 追踪消息送达与回复责任。

例如： $\text{Writer} \xrightarrow{\text{Evidence Request}} \text{Researcher}$

这种机制适合需要明确责任归属的任务交接。

2. 共享状态（Shared State）

共享状态允许多个智能体读取或更新共同的任务产物。

系统需要管理：Artifact Version：产物版本、Access Control：访问权限、Write Ownership：写入责任、Consistency：数据一致性。

例如，多个智能体可以共享同一份产品事实文件，但必须保证 Writer 和 Designer 使用的是一致且经过确认的版本。

两种机制的区别

维度	Direct Messaging	Shared State
交互方式	向特定接收者发送消息	读取或修改共同产物
适用场景	明确请求、任务交接	持续共享信息与中间结果
关键管理问题	路由、送达、回复责任	版本、访问权限、写入冲突

两种机制可以结合使用：通过直接消息通知任务变化，通过共享状态保存完整产物。

8.8 在 Prompt 中描述协作智能体

智能体需要知道系统中有哪些其他角色，以及什么时候应该与它们通信。

因此，应在角色指令中包含协作智能体的描述。

1. Capability（能力）

智能体需要了解其他角色具备哪些工具和权限，以便将请求发送给合适的对象。

2. Knowledge（知识）

智能体需要判断其他角色已经掌握哪些证据、信息和产物版本，避免基于过期或缺失的信息进行协作。

3. Objective（目标）

智能体需要了解其他角色的职责和优先级，以预测其响应方式并协调共同目标。

例如，Writer 在发现宣传声明缺乏证据时，应知道 Researcher 负责事实核查，并将具体声明及文案版本发送给 Researcher，而不是自行假定声明成立。

这种设计有助于减少角色之间的通信歧义，但实际权限仍应由运行时系统独立验证，而不能仅依赖 Prompt 约束。

8.9 多智能体系统的终止、恢复与升级机制

多智能体系统需要明确区分正常完成、错误恢复、替代执行、升级处理和强制停止。

五种执行结果：

结果	触发条件	系统响应
Done	满足验收标准	返回经过验证的结果
Retry	可恢复的暂时性错误	在固定限制内重试
Fallback	存在获得授权的替代方案	使用替代方案并记录变化
Escalate	存在无法解决的冲突或不确定性	提交相关状态，请求进一步决策
Stop	达到预算、权限或安全限制	停止执行并报告未完成工作

Retry 与 Fallback 的区别

Retry 表示在相同或修正后的条件下再次尝试原操作。例如，网络请求暂时失败后重新请求。

Fallback 表示使用另一种被允许的执行方式。例如，主要数据服务不可用时，改用经过授权的备用数据源。

前者主要针对可恢复错误，后者主要针对可替代执行路径。

Escalate 与 Stop 的区别

Escalate 表示自动决策无法继续，需要其他角色或人工介入。

Stop 表示执行必须终止，例如预算耗尽、权限不足或安全约束被触发。

二者都可能导致当前自动流程暂停，但 Escalate 强调移交决策责任，Stop 强调终止当前执行。

有界执行与停止条件

多智能体系统可以同时设置步数、成本和时间限制：

$N_{\text{steps}} \leqslant N_{\max}$，$C_{\text{total}}\leq C_{\max} $，$T_{\text{elapsed}} \leqslant T_{\max}$

- $N_{\max}$：最大执行步数。
- $C_{\max}$：最大允许成本。
- $T_{\max}$：最大执行时间。

达到限制后，系统应返回实际完成状态，并记录尚未完成的任务，而不是将停止误判为成功。

8.9 多智能体系统评估与风险规划 Evaluation and Risk Planning

多智能体系统的评估不能只检查最终输出，还应考虑执行成本、响应时间以及系统在异常情况下的行为。

1. 质量与价值（Quality and Value）

应将多智能体系统与单智能体基线进行比较。

主要指标包括：Quality：任务完成质量、Cost：模型调用、工具使用和协调成本、Latency：任务完成延迟。

如果多智能体系统增加了大量成本，却没有带来明显的质量改善，就需要重新评估其架构合理性。

2. 失败与预算测试（Failure and Budgets）

除了正常任务测试，还应验证系统在异常情况下的行为，包括：权限不足、工具执行失败、证据缺失、智能体之间出现冲突、重试次数耗尽、资源预算不足、需要人工介入。

评估重点不仅是系统能否成功完成任务，还包括它能否在无法完成任务时安全、明确地终止。

多智能体系统设计检查清单（Design Review Checklist）

在正式实施前，应检查以下五个方面。

检查维度	关键问题
Tasks	需要交付什么？如何判断任务成功？
Roles	谁负责各项职责？允许执行哪些操作？
Coordination / Architecture	谁负责分配任务、解决冲突和整合结果？
Messages	哪些信息需要传递？是否需要结构与版本验证？
Stop and Recover	何时完成、重试、升级或停止？

这五项检查共同确保系统的任务、职责、控制、状态和完成规则在实施前已经明确。

8.10 本章总结

本章从系统工程角度讨论了多智能体系统的设计方法，核心结论如下。

1. 任务分解与角色分配是设计起点。 系统需要先明确子任务、依赖关系、输入输出和责任归属，再决定需要哪些智能体。
2. 协调模型与执行工作流属于不同设计维度。 集中式、去中心化和混合式描述决策权的组织方式；顺序、并行和迭代工作流描述任务的时间执行关系。
3. 系统架构必须与任务结构匹配。 Pipeline、Peer Network、Blackboard 和 Graph 等架构具有不同的适用条件与协调成本，不存在对所有任务都最优的单一架构。
4. 通信是多智能体决策机制的一部分。 消息格式、路由规则、共享状态、版本管理和证据验证会直接影响智能体的后续行动。
5. 终止机制与评估机制不可缺少。 系统必须明确完成、重试、替代、升级和停止条件，并综合评价质量、成本、延迟及异常处理能力。

其中，单智能体已经能够完成工具调用、规划、反思和动态决策。多智能体系统则进一步通过专业化分工、信息交换和协作控制处理复杂任务。

因此，多智能体设计的核心不是增加智能体数量，而是通过合理的角色划分、协调架构、执行工作流和通信规则，提高整体任务的完成质量与可靠性。

8.11 课堂练习 In-Class Exercises

Q1：工具观察结果与当前计划矛盾时，ReAct 允许智能体做什么?

正确答案：A. Use observation to revise the next action.

ReAct 允许智能体根据新观察修改下一步行动，而不是忽略新信息或每次都重新启动整个任务。

Q2：两个智能体共享同一环境，但收到不同观察结果，意味着什么?

正确答案：A. Each may need to decide using incomplete information.

共享环境不意味着具有相同观察。不同智能体可能只能访问环境状态的不同部分，因此需要基于局部信息进行决策。

Q3：Researcher 表示“Research done”，还应增加什么信息才能使交接对 Writer 更有用?

正确答案：A. The findings, their sources, and unresolved questions.

有效交接需要包含结论、来源、验证状态和未解决问题，使后续智能体能够直接利用研究成果并判断其可靠性。

5584 LLM for Agent 1

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_6/knowledge/5584-LLM-for-Agent-1.md`

5584 LLM for Agent1

Lecture 1 大语言模型与智能体导论 Course Overview

Part 1 智能体与大语言模型的基本概念

1.1 智能体（Agent）的定义

An “intelligent” system that interacts with some “environment”.

能够与环境进行交互的智能系统。

这一定义包含三个基本要素：

- Agent：负责决策与行动的主体；
- Environment：Agent 所处并与之交互的环境；
- Interaction：通过观察与行动形成的交互过程。

Agent 通过 Observation（观察）获取环境信息，并通过 Action（行动）影响环境。

由此，Agent 的基本运行模式可概括为：Observe → Decide → Act → Observe

与一次性生成答案的系统相比，Agent 更强调持续交互、状态更新与行动反馈。

1.2 Agent 的环境类型

课件根据交互环境将 Agent 分为三类。

环境类型	典型系统	主要特征
Physical Environment	Robot、Autonomous Car	通过传感器感知物理世界，执行实际动作
Digital Environment	DQN for Atari、AlphaGo、Siri	通过数字接口观察环境并执行操作
Humans as the Environment	Chatbot	通过语言与人交互，人的回应构成反馈

PPT p.9 还展示了自动驾驶汽车、AlphaGo、ChatGPT、Dota 2、Alexa 和机械臂等示例，说明 Agent 并不限于聊天机器人，而是一个涵盖物理和数字交互系统的广义概念。

课件同时指出，Agent 的定义取决于如何界定 Intelligence（智能）和 Environment（环境），而这些概念会随着技术发展发生变化。

1.3 大语言模型（LLM）的定义与学习机制

Large Language Model 可以从 Language Model 与 Large 两个方面理解。

Language Model（语言模型） 的核心任务是对语言序列进行建模，并根据输入生成文本。

Large（大规模） 主要体现在两个维度：

1. Massive Number of Parameters：具有大量模型参数；
2. Huge Amounts of Training Data：利用大规模数据进行训练。

LLM 通常采用 Transformer、Mamba 等模型架构，通过训练数据调整模型权重。给出的训练数据来源包括互联网、书籍和代码。

从概率建模角度，语言模型可以表示为：$P(x_1,x_2,\ldots,x_T) = \prod_{t=1}^{T} P(x_t\mid x_1,\ldots,x_{t-1})$。其中，$x_t$ 表示第 t 个 Token。对于自回归语言模型，其核心目标是根据历史 Token 预测下一个 Token。

1.4 LLM 的能力与局限性

课件将 LLM 的主要能力归纳为五类。

能力	英文术语	典型应用
文本生成	Text Generation	邮件、文章、诗歌、代码
机器翻译	Translation	多语言文本转换
文本摘要	Summarization	提取长文档关键信息
问题回答	Question Answering	根据知识或上下文回答问题
对话系统	Conversational AI	聊天机器人、虚拟助手

但 LLM 仍然存在以下问题：

局限性	英文术语	具体表现
幻觉	Hallucination	生成看似合理但不正确的信息
偏见	Bias	反映训练数据中的偏见
实时信息不足	Real-time Information	无法仅凭静态权重获取训练后的新信息
隐私问题	Privacy	训练数据可能包含个人信息
安全问题	Security	可能受到恶意输入操纵

这些局限性构成了引入工具、记忆、检索、验证和外部执行机制的重要动机。

1.5 LLM Agent 与普通 LLM、AI Workflow 的区别

PPT p.15 从、及三个角度定义 AI Agent。

外部行为：从外部看，Agent 是能够自主运行的实体；

内部结构：从内部看，它是包含模型调用、通常也包含工具调用的程序。

比较维度	LLM	AI Workflow	LLM Agent
核心功能	语言生成与推理	按预设流程执行任务	自主决策与执行
执行路径	主要由调用方式决定	通常预先定义	可以根据反馈动态调整
环境交互	需依赖外部系统	由流程设计决定	是核心能力
工具使用	需外部封装	按预设步骤调用	可根据任务选择调用
自主性	较低	受流程限制	相对较高

系统能力：Autonomous and can “morph”。其中，Morph 指 Agent 能够在运行过程中根据环境反馈改变执行路径，而不是始终遵循固定步骤。

因此，LLM Agent 与 AI Workflow 的核心区别不在于是否使用 LLM，而在于系统是否能够自主决定后续行动。

1.6 LLM Agent 的核心组成

1. Planning：规划。规划主要包含两个方面

Subgoal and Decomposition（子目标分解）

将复杂任务拆解为较小、可管理的子任务，提高任务执行的可操作性。

例如：$G \rightarrow \{g_1, g_2, \cdots, g_n\}$。其中，$G$ 为整体目标，g_i 为子目标。

Reflection and Refinement（反思与改进）

Agent 对过去的行动和结果进行评估，通过自我批评和反思修正后续行为。

2. Memory：记忆。Memory 分为短期记忆与长期记忆。

类型	Short-term Memory	Long-term Memory
存储位置	当前上下文	外部存储系统
主要机制	In-context Learning	Vector Store、Retrieval
生命周期	通常受当前会话或上下文限制	可以跨较长时间保留
主要作用	保持当前任务状态	保存历史信息并按需召回

长期记忆通常通过外部向量存储和快速检索实现。 “infinite” 强调其可扩展性，而非真正无限的物理存储容量。

3. Tool Use：工具使用。工具使用使 Agent 能够通过外部 API 获取模型权重中没有的信息或能力。主要包括：

- Current Information：访问当前信息；
- Code Execution：执行代码与计算；
- Proprietary Information：访问专有数据；
- External APIs：调用日历、搜索、计算器等服务。

工具使用的本质是将 LLM 的能力从文本生成扩展到外部系统操作。

4. Action：行动

Action 是 Agent 将规划结果转化为实际操作的阶段。行动既可以发生在数字环境中，也可以作用于物理机器人或设备。行动的有效执行依赖于规划、记忆与工具使用，并通过环境反馈形成新的决策循环。

1.7 LLM Agent 的独特优势

- Language Mastery：能够理解和生成自然语言，支持人机交互。
- Decision-making：具备一定推理与决策能力，能够处理复杂任务。
- Flexibility：可适配不同任务与应用场景。

- Collaborative Interactions：能够与人类或其他 Agent 协作。

LLM 是 Agent 的重要认知组件，但 Agent 是包含规划、记忆、工具与行动机制的完整交互系统。

Part 2 智能体的发展历史：从规则系统到生成式 Agent

核心问题串联智能体的发展历史：How does a machine decide what to do next?

机器如何决定下一步应该执行什么行动？智能体的发展可以划分为三个阶段：

阶段	时间	核心范式
Symbolic & Rule-Based	1950s - 1980s	基于符号表示和人工规则
Formalization & Learning	1990s - 2010s	智能体形式化、搜索与机器学习
Generative & Agentic	2017 - Present	基于大模型的推理、行动与工具协作

2.1 符号主义与规则驱动阶段

1. Turing Test：智能的行为判据（1950）

Alan Turing 在 1950 年提出一个重要问题：Can machines think?

由于“思考”难以直接定义，Turing 将问题转化为 Imitation Game（模仿游戏），通过机器在交互中的可观察行为判断其智能表现。

这一思想为后续智能体研究提供了重要启示：可以通过系统行为而非内部主观体验评价智能。

2. Dartmouth Workshop：人工智能学科的建立（1956）

1956 年的 Dartmouth Summer Research Project 被视为人工智能学科的重要起点。

John McCarthy 提出了 Artificial Intelligence 这一名称，相关研究者包括 Marvin Minsky、Nathaniel Rochester 和 Claude Shannon。

这一阶段确立了人工智能作为独立研究领域的基本方向。

3. ELIZA：基于模式匹配的对话系统（1966）

ELIZA 通过关键词识别、模式匹配和文本替换生成对话回复，其中 DOCTOR 脚本模拟心理咨询式对话。其主要特点是：

- 使用预先定义的文本匹配规则；
- 不具备真正的领域知识或任务规划能力；
- 仍然能够使部分用户产生系统理解自己的印象。

这一现象被称为 ELIZA Effect。

Fluent ≠ Understanding（语言流畅不等于真正理解）。

4. MYCIN：基于规则的专家系统（1970s）

MYCIN 使用约 600 条人工编写的规则，为感染性疾病相关治疗提供抗生素建议。课件指出，其建议约有 65% 被评价为可接受，与 Stanford 专家教师的表现相当，但系统并未实际用于临床。

其主要限制不仅在于推理准确率，还在于实际系统集成：相关事实需要人工输入，难以自然融入真实工作流程。

5. BDI：信念—愿望—意图模型（1987）

BDI（Belief - Desire - Intention）通过三个概念描述智能体的决策状态。

组成	含义	对应问题
Belief	智能体认为当前世界是什么状态	What is true?
Desire	智能体希望达到什么目标	What does it want?
Intention	智能体决定执行什么计划	What does it commit to?

BDI 将 Agent 的内部状态与决策过程联系起来。

这一思想与现代 LLM Agent 的任务状态、目标管理和计划执行具有概念上的连续性。

2.2 智能体形式化与机器学习阶段

1. Russell & Norvig：Agent 的形式化定义（1995）

Perceiving its environment through sensors, acting upon it through actuators.

智能体通过 Sensors（传感器）感知环境，并通过 Actuators（执行器）作用于环境。

在现代 LLM Agent 中，可以建立如下对应关系：

经典 Agent	LLM Agent
Sensors	Context Window、工具返回信息
Percepts	用户输入、观察结果
Decision-making	LLM 推理与规划
Actuators	Tool Calls、API Calls
Actions	工具执行、系统操作

因此，LLM Agent 并未改变经典 Agent 的基本交互结构，而是改变了感知、决策和行动的实现方式。

2. Deep Blue：专用任务中的超人类表现（1997）

IBM Deep Blue 在 1997 年以 3.5 : 2.5 击败国际象棋世界冠军 Garry Kasparov。Deep Blue 主要依靠高性能搜索和人工设计的评估机制，而非现代意义上的通用学习系统。

其意义在于证明：机器可以在特定任务中达到超越人类专家的水平，但这种能力不一定能够迁移到其他任务。

3. LSTM：序列建模与长期依赖（1997）

LSTM（Long Short-Term Memory）由 Hochreiter 和 Schmidhuber 提出，通过门控机制控制信息的写入、保留和输出。

其核心包括：Forget Gate：遗忘门；Input Gate：输入门；Output Gate：输出门；Cell State：单元状态。

LSTM 的主要贡献是改善传统循环神经网络在长序列学习中的梯度传播与长期依赖问题。

课件将其与 Deep Blue 对比：Deep Blue 展示了专用搜索系统的能力，而 LSTM 推动了通用序列学习的发展。

4. Siri 与 Alexa：广泛部署但能力受限（2011 - 2014）

Siri 于 2011 年推出，Alexa 于 2014 年推出。这类早期语音助手能够完成语音识别、意图匹配和预设任务，但存在以下限制：

- 主要依赖 Intent Matching（意图匹配），缺乏复杂推理；
- 交互通常以单轮任务为主；
- 每项技能需要人工设计；
- 难以处理开发者未预先定义的新任务。

其核心局限是缺乏对未知任务的广泛泛化能力。

2.3 生成式与智能体化阶段

1. Transformer：注意力机制与规模化训练（2017）

Vaswani 等人在 Attention Is All You Need 中提出 Transformer。其核心特征是使用注意力机制进行序列建模，摆脱传统循环结构对逐步计算的依赖。

标准 Scaled Dot-Product Attention 表示为： $\operatorname{Attention}(Q,K,V) = \operatorname{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right)V $

Transformer 支持训练阶段对序列进行高度并行化处理，为大规模语言模型训练提供了重要基础。

2. ReAct：推理与行动交替执行（2022）

ReAct（Reasoning and Acting）将模型推理与环境行动结合。其典型过程为：

Thought: 分析当前问题，决定需要什么信息

Action: 调用搜索、计算器或其他外部工具

Observation: 接收工具返回的环境反馈

Update / Finish: 根据观察结果继续推理、调整行动或结束任务

ReAct 的重要特点是允许 Agent 在任务执行过程中根据观察结果调整计划。

3. Function Calling：结构化工具调用（2023）

Function Calling 将工具使用转化为具有明确参数规范的接口调用。模型负责生成工具名称和调用参数，外部程序负责执行工具并返回结果。

因此，模型提出调用请求，实际工具由执行环境运行。这一机制减少了人工解析非结构化 Prompt 输出的需求，提高了工具调用的可控性。

4. MCP 与 A2A：智能体基础设施标准化（2024 - 2026）

MCP（Model Context Protocol）用于标准化模型应用与外部数据、工具之间的连接。MCP 可以减少原本需要针对多个模型与多个工具分别开发的定制集成。

若存在 N 个模型应用和 M 个工具，完全定制集成在最简单的计数模型下可能需要：$ C_{\text{custom}}=N\times M $，统一协议可以通过标准接口减少重复适配工作，但实际集成复杂度仍取决于具体系统。

A2A（Agent-to-Agent）则面向不同 Agent 之间的通信与协作。

课件指出，MCP 于 2024 年 11 月出现，随后在 2025 年获得更多厂商支持；2025 - 2026 年编码智能体逐渐成为重要应用方向。同时引用 Gartner 对智能体项目取消风险的预测，强调智能体落地的核心挑战逐渐从能力演示转向系统可靠性。

2.4 三个阶段的本质变化

The shift is not intelligence but agency — who decides the next step, the programmer or the model?

三个阶段最重要的区别在于决策权的变化：

- 规则系统：程序员预先定义规则和执行路径；
- 学习系统：模型学习部分决策机制，但通常针对特定任务；
- LLM Agent：模型可以根据任务目标和反馈动态决定后续行动。

因此，智能体技术的发展不仅表现为模型能力增强，更表现为行动决策权从固定程序逐步转向具备自主性的模型系统。

Part 3 大语言模型的发展与规模化能力

3.1 从单任务模型到通用语言模型

传统机器学习通常针对不同任务分别训练模型，例如文本分类、翻译、摘要和问答。

这一范式存在两个主要问题：

1. 每个任务可能需要单独训练和维护模型；
2. 每个任务通常需要大量标注数据。

大语言模型改变了这一范式：通过大规模预训练学习通用语言表示，再利用任务描述或少量示例完成不同任务。

语言模型的优势：通过学习预测下一个 Token 或句子，使一个模型能够执行多种任务，而非仅完成单一任务。

3.2 BERT 与 GPT：表示学习和生成学习

BERT 和 GPT 所代表的两条技术路线：

比较维度	BERT	GPT
核心方向	Representation	Generation
主要架构	Encoder-only	Decoder-only
典型预训练目标	Masked Language Modeling	Autoregressive Language Modeling
优势任务	分类、语义理解	文本生成、续写
上下文建模	双向上下文	因果注意力

BERT 侧重学习输入文本的上下文表示，而 GPT 侧重根据已有文本预测后续 Token。

两者均属于预训练语言模型，但其训练目标和注意力结构不同。

3.3 GPT-2：从任务专用模型到通用模型

GPT-2 的重要研究方向是探索语言模型对未见任务（Unseen Tasks）的泛化能力。

传统任务专用模型可表示为：$P(y \mid x)$，其中，x 为任务输入，y 为输出。

通用模型进一步将任务描述纳入条件：$P(y\mid x,\tau)$，*其中*，τ 表示 Task Specification（任务规范）。

例如，任务描述可以直接使用自然语言：Translate this French text to English.

这样，模型不仅根据输入内容生成结果，还可以根据任务说明调整生成行为。

这一转变使任务描述成为模型输入的一部分，为后续 Zero-shot 和 Few-shot Learning 提供了基础。

3.4 Zero-shot Learning 与规模化训练

GPT-2 的研究动机之一是减少对每个新任务进行独立微调的依赖。

构建具有广泛泛化能力的语言模型需要两个重要条件：

1. Diverse Training Data（多样化训练数据）

训练数据覆盖多种语言、主题、任务和表达形式，有助于模型学习可迁移的知识与模式。

2. Scaling Model Capacity and Data（扩展模型容量与数据）

增加模型参数量和训练数据规模，可以改善语言建模能力，并可能提高下游任务表现。

Perplexity（困惑度）说明模型规模化的效果：

对于长度为 T 的 Token 序列，困惑度定义为：$\operatorname{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^{T} \log P(x_t\mid x_{<t}) \right) $。困惑度越低，通常表示模型对测试序列的概率预测越准确。但较低的困惑度并不必然意味着模型在所有下游任务中表现更好。

3.5 GPT-3 与模型规模扩展

对比了 GPT-2 和 GPT-3 的参数规模，单位：十亿参数（Billion Parameters）

GPT-2 的最大版本约为 1.5 Billion Parameters，而 GPT-3 为 175 Billion Parameters。

规模扩展使研究者能够进一步探索大模型在少样本学习、推理和复杂任务中的能力变化。

3.6 Emergent Abilities：涌现能力

Emergent Abilities（涌现能力）：某些能力在较小模型中不明显或不存在，但在更大模型中表现出来。例如，GPT-3 在仅接受文本训练的情况下，能够完成部分加减法任务。

1. 不同能力可能在不同模型规模下出现；
2. 模型规模并非涌现能力的唯一决定因素；
3. 模型架构、训练数据与训练方法同样可能影响能力表现。

引用 BIG-Bench 的部分实验结果：在 14 项任务上，LaMDA 137B 和 GPT-3 175B 的表现接近随机水平，而参数量更小的 PaLM 62B 达到高于随机的表现。

这说明：$\text{Model Performance} \neq f(\text{Parameter Count Only}) $，模型表现受到多个因素共同影响。

涌现能力的具体判定还与评价指标有关。某些看似突然出现的能力，也可能与离散评价标准有关，因此不能简单认为参数规模达到某个固定阈值就必然产生某种能力。

3.7 LLM 的基本特征

将大语言模型概括为具有大量参数、能够通过 Prompting 完成多种任务的语言模型。

课件采用超过 1B 参数作为介绍性规模描述，并列举 GPT、Llama 2、Gemini、PaLM 和 Mixtral 等模型。

这一规模描述并不是严格、普遍适用的 LLM 分类标准。

本章的核心结论是：语言模型从针对特定任务训练，逐渐发展为通过大规模预训练获得通用能力，并通过自然语言任务描述适配不同任务。

Part 4 大语言模型架构与注意力机制

4.1 三种 Transformer 架构

将主流语言模型架构划分为 Encoder-only、Decoder-only 和 Encoder - Decoder 三类。

架构	代表模型	预训练目标	主要应用
Encoder-only	BERT	Masked Language Modeling	文本分类、语义理解
Decoder-only	GPT	Autoregressive Language Modeling	文本生成、通用语言建模
Encoder - Decoder	T5、T0	Masked Span Prediction	机器翻译、摘要、文本转换

1. Encoder-only：掩码语言建模

Masked Language Modeling（MLM）随机遮盖输入中的部分 Token，要求模型根据上下文预测被遮盖内容。

设被遮盖的位置集合为 $\mathcal{M}$，则其训练目标可表示为：$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log P_\theta(x_i \mid x_{\setminus \mathcal{M}})$

由于 Encoder 可以利用双向上下文，因此适合学习文本的语义表示。

2. Decoder-only：自回归语言建模

Decoder-only 模型根据历史 Token 预测下一个 Token。

其训练损失为：$ \mathcal L_{\text{AR}} = -\sum_{t=1}^{T} \log P_\theta(x_t\mid x_{<t})$这种结构适合逐 Token 生成文本，是 GPT 和 LLaMA 等模型采用的主要架构。 Decoder-only 架构具有训练稳定、收敛较快以及预训练后泛化能力较强等优势。

3. Encoder - Decoder：序列到序列建模

Encoder - Decoder 架构使用 Encoder 表示输入，Decoder 根据输入表示生成输出。

其条件生成过程可表示为：$P(y\mid x) = \prod_{t=1}^{T_y} P(y_t\mid y_{<t},x)$，$x$：输入序列；$y$：输出序列；$T_y$：输出序列长度。

这种架构适合机器翻译、摘要等输入输出之间存在明确映射关系的任务。

4.2 T5/T0：Masked Span Prediction

Masked Span Prediction（片段掩码预测），其过程包括：

1. 从输入序列中选择连续 Token 片段；
2. 使用特殊标记替换这些片段；
3. 将处理后的序列输入 Encoder；
4. 由 Decoder 生成被遮盖的片段。

与 BERT 的 Token 级掩码相比，Span Prediction 侧重对连续文本片段进行重建。

这种目标能够使模型学习较长范围的语义结构，并适用于文本生成任务。

4.3 Attention Patterns：注意力模式

不同架构中的注意力连接方式：

1. Bidirectional Attention（双向注意力）

Token 可以访问序列中的其他 Token，适合上下文理解。

2. Causal Attention（因果注意力）

第 t 个 Token 只能访问自身及此前的 Token，避免生成时访问未来信息。

对于 Causal Attention，可以定义注意力掩码：$M_{ij} = \begin{cases} 0,&j\le i\\ -\infty,&j>i \end{cases}$

注意力计算为：$\operatorname{Attention}(Q,K,V) = \operatorname{softmax} \left(\frac{QK^\top}{\sqrt{d_k}}+M \right)V $

在 Encoder - Decoder 架构中，Encoder 通常对输入进行双向注意力建模，而 Decoder 在生成输出时使用因果注意力，并通过 Cross-Attention 访问 Encoder 的表示。

还给出了 Decoder-only 与 Encoder - Decoder 架构的经验性能比较，指出在其引用的相近配置实验中 Decoder-only 表现更优。这并不意味着 Decoder-only 在所有任务中都优于 Encoder - Decoder。

4.4 LLaMA 2 的架构改进

以 LLaMA 2 为例，介绍了 Decoder-only Transformer 的重要设计变化。

技术	传统设计	LLaMA 2 相关设计	目的
Normalization Position	Post-Norm	Pre-Norm	改善训练稳定性
Normalization Type	LayerNorm	RMSNorm	简化归一化并提高效率
Activation	ReLU	SwiGLU	增强非线性表示能力
Position Encoding	Absolute/Relative PE	RoPE	建模位置信息
Attention	Multi-Head Attention	Grouped-Query Attention（部分配置）	减少 KV 存储与推理开销

1. Pre-Norm

Post-Norm 通常在残差连接后执行归一化：$y = \mathrm{Norm}(x + F(x))$

Pre-Norm 则在子层计算前进行归一化：$y = x + F(\mathrm{Norm}(x))$

Pre-Norm 有助于改善深层 Transformer 的梯度传播。

2. RMSNorm

RMSNorm 通过均方根进行归一化：$\operatorname{RMS}(x) = \sqrt{ \frac{1}{d}\sum_{i=1}^{d}x_i^2+\epsilon }$，$\operatorname{RMSNorm}(x) = \frac{x}{\operatorname{RMS}(x)} \odot\gamma $

与 LayerNorm 不同，RMSNorm 不显式减去均值。

3. SwiGLU

SwiGLU 是一种门控激活机制，可写为：$ \operatorname{SwiGLU}(x) = \operatorname{Swish}(xW_1)\odot(xW_2)$，*其中*：$\operatorname{Swish}(z)=z\sigma(z)$

SwiGLU 通过门控乘积提高前馈网络的表达能力。

4. RoPE

RoPE（Rotary Position Embedding）通过旋转变换将位置信息融入 Query 和 Key。

其重要特点是使注意力计算能够反映 Token 之间的相对位置信息。

5. Grouped-Query Attention

Grouped-Query Attention（GQA）允许多个 Query Heads 共享较少数量的 Key/Value Heads。

设 Query Head 数量为 H_q，KV Head 数量为 H_{kv}，则在其他条件相同时，KV Cache 的规模与 H_{kv} 成正比。

相对于相同数量 Query/KV Heads 的普通多头注意力，KV 存储比例约为：$\frac{M_{\text{GQA}}}{M_{\text{MHA}}} = \frac{H_{kv}}{H_q}$

GQA 因而能够减少 KV Cache 占用及相关内存带宽需求。

Part 5 大语言模型的训练与对齐

5.1 Decoder-only LLM 的训练流程

以 LLaMA 2 为例，将大语言模型的训练划分为三个主要阶段：

Stage 1 Autoregressive Pre-training：通过大规模语料学习语言建模能力与通用知识

Stage 2 Supervised Fine-tuning（SFT）：利用指令与目标回答训练模型遵循任务要求

Stage 3 Safety / RLHF：利用人类偏好反馈进一步优化模型行为

这三个阶段分别解决不同问题：

- Pre-training：模型如何学习通用语言能力？
- SFT：模型如何理解并执行人类指令？
- RLHF：模型如何使输出更符合人类偏好和安全要求？

1. Autoregressive Pre-training：自回归预训练

预训练的核心目标是根据历史 Token 预测下一个 Token。

设训练序列为：$X = (x_1, x_2, \cdots, x_T)$，其最大似然训练目标为：$\max_\theta \sum_{t=1}^{T} \log P_\theta(x_t\mid x_{<t})$ 。*等价地，最小化负对数似然损失*：$\mathcal L_{\text{pretrain}} = -\sum_{t=1}^{T} \log P_\theta(x_t\mid x_{<t})$，LLaMA 2 的预训练使用了约 3 Trillion Tokens 的大规模语料。预训练主要使模型获得语言模式、知识和一定的任务泛化能力，但并不直接保证其能够遵循复杂指令。

2. Supervised Fine-tuning：监督微调

SFT 使用带有标准输出的任务样本进一步训练预训练模型。

典型数据形式为：$D_{\text{SFT}} = \{(I_i, X_i, Y_i)\}_{i=1}^{N}$

- I_i：Instruction，任务指令；X_i：Input，任务输入；Y_i：Target Output，期望输出。

SFT 的训练目标为：$\mathcal L_{\text{SFT}} = -\sum_{i=1}^{N} \log P_\theta(Y_i\mid I_i,X_i)$

在 Token 层面，通常只对目标回答部分计算损失，而将指令和输入作为条件上下文。

SFT 与 Pre-training 的区别

比较维度	Pre-training	SFT
数据形式	大规模自然语言语料	指令、输入与目标输出
预测目标	自回归预测语料 Token	预测目标回答 Token
主要目的	学习通用语言知识与能力	学习任务遵循与输出行为
数据要求	大规模预训练语料	高质量指令样本
主要结果	Base Model	Instruction-tuned Model

PPT 还指出，SFT 可以通过训练数据引导模型输出的有用性（Helpfulness）和安全性（Safety）。

5.2 Instruction Tuning：指令微调

Instruction Tuning 通过多种自然语言指令训练模型，使其能够理解不同任务要求。

与只针对某个固定任务进行微调不同，指令微调通常利用多任务数据，提高模型对新任务描述的泛化能力。

因此：Pre-training 主要学习语言与知识分布，Instruction Tuning 主要学习如何按照指令使用这些能力。

5.3 Alignment：模型对齐问题

Unsafe Outputs（不安全输出）及 Alignment Problem（对齐问题）。

模型可能生成：

- 包含偏见、歧视或不当表达的文本；
- 可能造成直接危害的信息；
- 不符合人类偏好、价值或安全要求的内容。

因此，模型训练不仅需要提高任务准确率，还需要使其输出符合适当的人类偏好和行为约束。

两类主要方法：

1. SFT：通过高质量示例学习期望行为；
2. RLHF：通过人类偏好反馈进一步优化输出。

5.4 RLHF：基于人类反馈的强化学习

RLHF（Reinforcement Learning from Human Feedback）通常包含三个核心步骤。

1. 收集人类偏好数据

对于同一个 Prompt，模型生成多个候选回答，由人工标注者比较其质量。

偏好数据可以表示为： $ D_{\text{pref}} = {(x,y^+,y^-)}$

- x：输入 Prompt；
- y^+：更受偏好的回答；
- y^-：较不受偏好的回答。

2. 训练奖励模型（Reward Model）

奖励模型学习根据输入和回答输出一个偏好分数：$ r_\phi(x,y)$

一种常见的成对偏好训练目标为：$\mathcal{L}_{\text{RM}} = -\mathbb{E}[\log \sigma(r_\phi(x, y^+) - r_\phi(x, y^-))]$

3. 利用强化学习更新语言模型

将语言模型视为策略：$\pi_\theta(y \mid x)$，通过策略优化，使模型生成更高奖励的回答。

典型优化目标可表示为：$\max_\theta \mathbb{E}_{x,y\sim\pi_\theta}[r_\phi(x,y) - \beta D_{\text{KL}}(\pi_\theta(\cdot \mid x) \,/\!/\, \pi_{\text{ref}}(\cdot \mid x))]$

- r_ϕ：奖励模型；π_θ：待优化策略；π_{ref}：参考策略；D_{KL}：KL 散度；β：约束强度。

KL 项用于限制策略相对于参考模型发生过大的变化。

以 LLaMA 2 为例介绍了基于奖励模型和策略梯度的 RLHF 训练思想。

5.5 RLHF 的局限与对抗鲁棒性

完成 RLHF 和 Safety Tuning 并不意味着模型永远不会生成有害输出。主要原因包括：

- 不可能穷尽所有潜在有害输出；
- 训练数据无法覆盖所有异常输入；
- 攻击者可能通过对抗性输入诱导模型违反预期行为。

这一问题属于 Adversarial Robustness（对抗鲁棒性）。

课件还给出了讲授者对 Claude、ChatGPT 和 Llama 安全表现的经验性观察。该观察不构成统一测试条件下的定量排名。因此，模型对齐应被视为持续改进过程，而不是一次训练即可彻底解决的问题。

Part 6 大语言模型推理与提示工程

6.1 Prompting 与 Prompt Engineering

Prompt 是在推理阶段提供给模型的自然语言任务描述。Prompt 可以非常简短，也可以包含任务背景、输入数据、输出格式和具体约束。

Prompt Engineering（提示工程）则是设计、选择和优化 Prompt，以提高模型在特定任务中的表现。

基本原则：Be as specific and descriptive as possible. 尽可能明确、具体地描述任务要求。

提示工程可以采用人工方式，也可以通过自动方法完成，例如 Prompt 改写、Prefix Tuning 等。

6.2 Zero-shot、One-shot 与 Few-shot Prompting

根据提供的任务示例数量，Prompting 可以划分为三种基本形式。

方法	示例数量	主要特点
Zero-shot	0	只提供任务说明
One-shot	1	提供一个参考示例
Few-shot	少量	通过多个示例展示任务模式

1. Zero-shot Prompting

不提供示例，直接要求模型完成任务。

```
Classify the sentiment of this sentence:
"The service was excellent."
```

2. One-shot Prompting

提供一个示例，引导模型理解输入输出格式。

```
Text: "The food was terrible."
Sentiment: Negative

Text: "The service was excellent."
Sentiment:
```

3. Few-shot Prompting

提供多个输入输出示例，使模型根据示例推断任务规则。这属于 In-context Learning（上下文学习）。

重要特点：模型在推理过程中利用上下文示例调整行为，通常不需要更新模型参数。

可以将其表示为： $P(y \mid x, D_{\text{demo}})$，其中，$D_{\text{demo}}$ 表示上下文中提供的示例集合。

6.3 Chain-of-Thought Prompting

Chain-of-Thought（CoT）Prompting 通过引导模型进行分步推理，提高其处理多步骤问题的能力。

其核心思想是将复杂任务分解为若干中间推理步骤，而不是直接生成最终答案。

例如： $ \text{Problem} \rightarrow \text{Intermediate Reasoning} \rightarrow \text{Answer} $

CoT 尤其适用于数学、逻辑和多步骤推理任务。

需要注意，模型生成的推理文本并不必然反映其真实内部计算过程，也不能保证最终答案正确。

6.4 Full Fine-tuning 与 Prompting 方法的选择

四种适配方法：

方法	主要优势	主要限制
Full Fine-tuning	可以获得较强的任务专用性能	需要较多标注数据与训练资源
Few-shot	需要较少任务专用数据	示例选择与任务适配存在挑战
One-shot	接近自然语言示范方式	单个示例可能不足以明确任务
Zero-shot	最方便，无需任务示例	任务可能存在歧义，性能不稳定

Full Fine-tuning 通常需要约 10^3 到 10^5 甚至更多标注样本，但这不是所有微调任务的固定要求。不同方法之间存在性能、数据需求和使用便利性的权衡。

通常而言，增加任务专用训练和监督有助于改善特定任务表现，但也会增加数据准备、训练和维护成本。

6.5 Parameter-Efficient Fine-tuning

Parameter-Efficient Fine-tuning（PEFT，参数高效微调）其目标是在不更新全部模型参数的情况下，使预训练模型适应新任务。主要思路包括：

- 冻结基础语言模型，仅训练部分参数；
- 添加外部 Adapter 模块；
- 使用 Low-Rank Adaptation（LoRA）。

LoRA 的基本原理

对于预训练权重矩阵：$W_0 \in \mathbb{R}^{d \times k}$，LoRA 将权重更新表示为低秩矩阵乘积：$ \Delta W=BA$,

其中： $B\in\mathbb R^{d\times r}, A\in\mathbb R^{r\times k}$ ，且：$r\ll\min(d,k)$

更新后的线性变换为：$h=(W_0+BA)x $，训练时冻结 W_0，只更新 A 和 B。

原始全参数更新需要训练：$N_{\text{full}} = dk$个参数，而 LoRA 只需：$N_{\text{LoRA}} = r(d + k)$

因此参数比例为：$ \frac{N_{\text{LoRA}}}{N_{\text{full}}} = \frac{r(d+k)}{dk} $，当 r 较小时，可以显著减少可训练参数量。

Part 7 LLM 评估、多模态与开放挑战

7.1 Evaluating LLMs：大模型评估

LLM 评估具有较高复杂性，应尽可能在不同数据集和任务上进行测试。

原因：LLM 通常具有多任务能力，单一数据集难以全面反映其性能。

因此，评估时需要考虑：不同任务类型；不同数据分布；模型输出质量；不同应用场景下的可靠性。

7.2 Multimodal LLMs：多模态大语言模型

传统 LLM 主要处理文本，而多模态模型进一步引入图像、音频等信息。两个目标：

1. 使模型能够通过视觉和听觉理解世界；
2. 使模型能够执行 Cross-modal Reasoning and Learning（跨模态推理与学习）。

两条引入多模态能力的路线：

路线	代表模型	主要思想
Pre-training	Gemini	在预训练阶段引入多模态学习
Instruction Tuning	AudioGPT、Flamingo	通过多模态组件或任务适配建立跨模态能力

7.3 Open Challenges：大模型的开放挑战

1. New Capabilities：新能力

主要包括：Multimodal：多模态理解与生成；Multilingual：多语言能力；More Complex Tasks：更复杂任务的处理能力。

2. Performance：性能与可靠性

主要包括：Reduce Hallucinations：减少幻觉；Improve Alignment with Human Preference：改善人类偏好对齐；Increase Context Length Efficiently：高效扩展上下文长度；Improve Data, Training Strategy and Model Architecture：优化数据、训练策略和模型架构。

3. Efficiency：效率

主要关注：计算成本；训练与推理时间；资金与资源消耗；GPU、TPU、HPU 等计算架构。

这些挑战表明，LLM 的研究目标不仅是提高模型能力，还包括降低部署成本、改善可靠性和扩展实际应用范围。

Part 8 总结

本讲的知识逻辑可以归纳为以下六个层次。

知识层次	核心内容	关键结论
Agent 概念	Agent - Environment 交互	智能体通过观察与行动形成闭环
LLM Agent 架构	Planning、Memory、Tool Use、Action	Agent 是完整系统，而非单一模型
智能体历史	Rules → Learning → Generative Agents	决策机制从固定规则转向模型驱动
LLM 技术基础	Transformer、GPT、Scaling	大规模预训练支持多任务泛化
模型训练与适配	Pre-training、SFT、RLHF、Prompting、LoRA	不同方法解决不同层面的能力适配问题
未来发展	Evaluation、Multimodal、Reliability、Efficiency	实际部署需要同时考虑能力、可靠性和成本

概念对	关键区别
LLM vs. LLM Agent	语言生成模型 vs. 能够自主交互和行动的系统
Agent vs. Workflow	动态决策执行路径 vs. 主要遵循预设流程
Short-term vs. Long-term Memory	上下文内信息 vs. 外部持久化信息
Encoder-only vs. Decoder-only	双向表示学习 vs. 自回归生成
Pre-training vs. SFT	学习通用语言能力 vs. 学习指令遵循
SFT vs. RLHF	学习标准回答 vs. 优化偏好与行为
Fine-tuning vs. Prompting	更新模型参数 vs. 通过输入上下文适配任务
Full Fine-tuning vs. LoRA	更新大量或全部参数 vs. 训练低秩增量参数

一条完整的技术演进逻辑：

$$ \text{Rule-based AI} \rightarrow \text{Learning-based AI} \rightarrow \text{Large Language Models} \rightarrow \text{LLM Agents} $$

传统规则系统依赖程序员预先定义行为，机器学习使系统能够从数据中学习决策模式，大语言模型进一步通过大规模预训练获得通用语言能力，而 LLM Agent 则将语言能力与规划、记忆、工具和行动机制结合，使系统能够在环境中持续完成任务。

LLM 提供语言理解与推理基础，Agent 架构赋予模型与环境交互和自主执行任务的能力，而可靠的智能体系统还需要训练、对齐、工具、评估及计算基础设施的共同支持。

Lecture 2 Anatomy of an LLM Agent

Part 1 Agent

The engineering question: what is actually inside?

Every design choice we make is a trade-off, not an upgrade 我们做的每一个设计选择都是权衡，不是升级；

Goal: by the end, you can draw an agent, then critique one.

目标：可以绘制一个 agent 并且评判它

Each part answers one hole — and one property of our task. For every option, we ask two things: what it costs, and what it buys you. There is no "best" column — only a fit for your environment.

每一块回答一个漏洞，也回答任务的一个属性；对每个选项都要问两件事：它花什么代价，它买到什么；没有最优这一列，只有与你的环境的匹配度。

Tips：为什么 Fine-tune 出现在 Memory 里而不是 Reasoning 里?

记忆的差别不在存哪儿，而在存什么。微调被归入记忆，因为它的本质是把知识写进权重，是一种持久化手段，而不是一种推理方式。RAG 把知识放在外部文档里、Files 放在文件系统里、Fine-tune 放在参数里，三者都是记住东西的不同落点。

1.1 Agent 的能力和风险

1. 十万行 C 编译器

Sixteen agents built a 100,000-line C compiler in two weeks—able to compile the Linux kernel. 十六个智能体在两周内造出了一个十万行的 C 编译器，能编译 Linux 内核。

规模：十万行、十万行级别的代码是可以由一群 agent + 一个任务队列推进的。

协调：协作的成本被显式地写进了工作流，主体错位 / 责任推诿 要解决的问题；

可观察：agent 的工作必须留下可被人类读懂的痕迹，否则无法验收、复现。

2. 邮件清理失控

不可逆动作：trash EVERYTHING in inbox older than Feb 15 是一个破坏性批量操作，而它是在没有被要求的情况下自行拟定并执行的。

用户的否决晚且无效：agent 的循环并没有把人类否决当成一个全局中断信号

无法中断：除了杀掉宿主机上的进程，它没有任何层内的停止机制。

3. Flask 应用排障

其动作序列是推理闭环：终端里连续尝试启动应用、报错（应用侧自己识别出来的）、采纳自己的建议，换端口重启、右侧浏览器里出现应用页面（ngrok 暴露的公网地址）。识别了错误、提出了处置方案、执行了方案、然后验证了结果。

Part 2 Agent 的组成

2.1 四个组件与环境

组件	连接关系
Perception System	接收 Input；由 Environment 反向喂入
Reasoning System	接收 Perception 的输出；与 Memory 双向连接
Memory System	与 Reasoning 双向连接（读 + 写）
Execution System	接收 Reasoning 的输出；动作打到 Environment
Environment	接收 Execution 的动作，再把结果送回 Perception

数据是单线流动的： Perception → Reasoning → Execution

Memory 是唯一的双向块：推理既能读记忆也能写记忆，这解释了为什么 p.11 的画法里 Memory 被单独标成 read + write；

闭环靠环境完成：Execution 的动作改变 Environment，Environment 的变化再被 Perception 读到。

2.2 Running Example

TASK: Go through my inbox for this week, find every expense-claim email, put the amounts in a spreadsheet, highlight anything over ¥5000 in red, and email the summary to Finance.

翻一遍我这一周的收件箱，找出所有报销申请邮件，把金额填进一张表格，把超过 ¥5000 的标红，再把汇总邮件发给财务。

任务属性	对应的子系统	难处
Email count unknown up front 邮件数量事先未知	Reasoning（unknown number of steps）	无法预先写死步数，固定流程从第一步就错
One amount is inside an image 有一笔金额在图片里	Perception（the amount in the image）	文本感知看不见它，必须上多模态或工具
It runs again next week 下周还要再跑一次	Memory（it runs again next week）	要记住上次怎么做的，否则每周从零开始
The last step is irreversible 最后一步不可撤销	Execution（sending mail is not undoable）	发邮件不可回滚，误发就是真实的失误

2.3 Agent 必要条件

对照项	Workflow 工作流	Agent 智能体
策略来源	Follows a plan the designer wrote 遵循设计者写好的计划	Generates its own strategy 自己生成策略
执行方式	Fixed sequence of steps 固定的步骤序列	Acts on environmental feedback 依据环境反馈行动
出错时	Cannot re-plan when it hits an error 碰到错误无法重新规划	Recovers from mistakes, re-plans 能从错误中恢复并重新规划

对照项	Workflow 工作流	Agent 智能体
适用场景	Great for predictable, repetitive tasks. 适合可预测、重复的任务	Needed for unpredictable environments. 不可预测的环境才需要

Adding tools to an LLM does not make an agent; it makes a workflow.

给 LLM 加上工具，并不会把它变成 Agent，只会把它变成一条工作流。

调用了工具是能力层面的描述：模型能访问外部世界了。

是否是 Agent 是架构层面的描述：控制流由模型在运行时决定，且它能根据反馈改变接下来的步骤。

2.3.1 五种workflow 模式

模式	定义	控制流由谁决定
Prompt chaining 提示链	Fixed order, one LLM call per step 固定顺序，每步一次调用	设计者（顺序写死）
Routing 路由	Classify, then dispatch to a specialist promptly 先分类，再分派给专门的提示	设计者（分派表写死）
Parallelization 并行化	Sectioning into subtasks, or voting on one 切成子任务并行，或对同一任务投票	设计者（切分方式写死）
Orchestrator - workers 编排者 - 工作者	Central LLM splits work at run time 中心 LLM 在运行时拆分工作	模型（运行时决定）
Evaluator - optimizer 评估者 - 优化者	Generate, score, rewrite 生成、打分、重写	模型（循环次数运行时定）

Orchestrator - workers is the first pattern whose step count is unknown until run time. 编排者 - 工作者是第一个在运行时才确定的模式；

The criterion is not "does it have tools?" — it is who decides the control flow. 判据不是它有没有工具，而是谁决定控制流。

2.3.2 Workflow 优先

Agents cost more, run slower, and fail in more ways. Use one only when you cannot enumerate the steps. Agent 更贵、更慢、失败方式更多；只有当步骤无法枚举时才用它；

Your default assumption is that an agent is the more advanced choice. It is the most expensive one. 你的默认假设是Agent 是更先进的选择，但它其实是更昂贵的那个选择。

Tips：Workflow（主干）+ Agent 混合架构才是常态。

能写出且清单稳定就用 workflow，会得到更低的成本、更短的延迟、更可预测的失败；写不出来（因为数量/分支/环境状态视运行时而定）就在那一个节点上嵌一个 agent 模块。

2.4 The Loop and Exit 闭环与退出条件

1. Exit 的三行判定： Success? Budget exhausted? Loop detected?

任务真的完成了吗？这三条里最难验证的一条；

预算（步数 / token / 时间 / 金钱）花完了吗？

是不是在原地打转？

2. 任务与六步

Task: Reply to the unread email from Accounting.

动作	中文	对应章节
Perceive	看见：截图 + 可访问性树	Part 3 感知
Retrieve	回想：上次我是怎么回这类邮件的?	Part 5 记忆
Reason	分解任务，挑下一步	Part 4 推理
Act	行动：click(126)	Part 6 执行
Observe	观察：屏幕变成了新状态	回到 Part 3（重新感知）
Reflect	反思：刚才那步成了吗？继续还是停	Part 4 推理、反思

Part 3 Perception 感知子系统

3.1 Four Options

方案	定义	代价
Text 文本	The environment describes itself in words. 环境用文字描述自己	Cheapest — blind to anything unwritten. 最便宜，但对没被写下来的东西表达不了
Multimodal 多模态	Pixels via VLM / MM-LLM. Sees everything, including images 通过 VLM / MM-LLM 直接看像素，能看到包括图片在内的一切	costs the most 成本最高
Structured 结构化	Node tree, DOM, JSON rows. Exact semantics, cheap 节点树、DOM、JSON 行；语义精确且便宜	but only what the tree exposes. 但只能拿到树暴露出来的东西
Tool-based 工具化	Call something to find out. 调用某个东西去查明，成本可控	Fidelity depends entirely on the tool you wrote. 保真度完全取决于你自己写的那个工具

维度	A 截图	B 可访问性树	C IMAP（JSON）
成本	~1,100 - 1,600 tokens	~400 tokens	~120 tokens
看到图片附件	能（缩略图）	不能	不能（只有文件名）
拿到精确金额	不能（需 OCR）	能（value: "5000"）	能（字段精确）
看懂视觉标记	能（红旗）	不能	不能（丢颜色标记）
界面重做后	失效	失效（除非树重新标好）	稳定

感知方案的选择会直接决定任务可行或不可行。

3.2 方案一：Text-Based Perception 文本感知

The environment writes text straight into the prompt, no perception module at all, Cheapest option by far

环境把文本直接写进 prompt；根本没有感知模块；是所有方案里最便宜的；

代表环境： ALFWORLD / TEXTWORLD；

适用：Chat, text games, text-driven simulations（对话、文字游戏、文字驱动的模拟）；

局限： Blind to anything not already in words（对任何尚未被写成文字的东西都不知道）。

不可迁移： Much agent research runs in text environments — so its conclusions may not hold on a GUI.

大量 Agent 研究是在文本环境里做的，所以它们的结论未必在图形界面上成立。

优势：把感知问题彻底消掉了，环境写好了文本，可以专心研究推理与控制流，不受感知噪声干扰。文本环境是隔离变量的实验台，代价是外部效度下降。

3.3 方案二：Multimodal Perception 多模态感知

适用：在没有任何 API、只剩一块屏幕的时候。Essential when there is no API — only a screen.

左半段叫 Multimodal Understanding（多模态理解），右半段叫 Multimodal Generation（多模态生成）。感知只用到左半段，视觉信号要经过专用编码器 → 投影层 → 主干三跳才能进入语言空间，而且每一跳都有信息损失与计算开销，所以模型看得见但数不准。

Still weak at precise spatial relationships and accurate counting — without external aid.

在没有外部辅助时，仍弱于精确的空间关系与准确的计数。

Seeing an image is not the same as measuring it.

看见一张图，不等于测量一张图。

3.3.1 Set-of-Mark

Draw a numbered box on every clickable element, then let the model name a number.

给每一个可点击元素画一个带编号的框，然后让模型说出一个编号。

收益：Improves counting（改善计数）、Cuts hallucination（减少幻觉）。

Cheaper than retraining the whole MM-LLM. 比重新训练整个多模态模型便宜。

改数据/加训练（贵、慢、还需可复现）与改输入形式（便宜、即插即用）。

Tips：文本环境是把感知问题消掉，结构化数据是把坐标问题消掉，ACI 是把点击问题消掉。这些都在改任务形式，而不是加大模型。

It converts an open-ended spatial problem into a closed-form, multiple-choice one.

它把一个开放式空间问题，转换成了一个封闭式的选择题。这是工程技巧之一：当模型在某类任务上系统性变弱时，先想办法把任务形式换掉，而不是换一个更大的模型。

3.4 方案三：Structured Data 结构化数据

Names, not coordinates. No pixel guessing. 用名字，不用坐标；不用猜像素。

三种结构化来源：

- Accessibility tree 每一个 UI 元素，带角色与状态（every UI element, with role and state）；
- HTML / DOM 元素描述与截图并排放；

> 结构化感知与截图不是二选一，而是互补
>
> A（截图）能看见红色标记，但算不出精确金额；B（树）有精确金额，但看不见任何图形与颜色。所以最稳的组合是结构化做数值与定位 + 截图做视觉属性判断。虽然组合能提高保真度，但代价是上下文被撑爆。

- 健壮性： Survives a re-skin; breaks on a re-label.

> 换皮（re-skin）：只改配色、字号、圆角，元素的名字与角色没变。所以可访问性树里读到的东西不变，agent 照常工作；
>
> 换标签（re-label）：把按钮文字从 Reply 改成 Respond、或把 textfield 的标签改掉。agent 依赖的锚点消失了，于是它崩掉了。
>
> 实践价值：如果长期维护一个结构化感知的 agent，要把标签稳定性当成接口契约来管，而不是当成实现细节。界面换皮它能活，标签一改它就崩。

来源	适用场景
Playwright accessibility snapshot	Web 自动化场景下的可访问性快照
Chrome DevTools Protocol (CDP)	直接通过浏览器调试协议取 DOM 与可访问性信息
Android UI Automator	Android 端的界面元素枚举

Precise semantics, cheap tokens — but only if the tree exists and is well-labeled.

语义精确、token 便宜，前提是这棵树存在，而且标注良好。

3.5 方案四： Tool-based perception 工具化感知

工具化感知：The agent calls something to find out what it cannot see. Agent 调用某个东西，去查明它自己看不见的事。

四类典型工具：

工具类型	说明	在报销任务里的对应
Web search	Facts past the training cutoff 训练截止之后才发生的事实	查某个供应商是否真实存在
Domain APIs	Weather, markets, literature 天气、行情、文献	对接财务系统的报销接口
OCR / sensors	A wrapper that returns JSON — our receipt 一个返回 JSON 的封装，即收据	读出图片里的那笔金额
Code execution	Parse logs, run statistics 解析日志、跑统计	解析邮件原始报文、做金额汇总

感知与执行共用同一个工具调用机制，只是意图不同，一个是为了获取信息，一个是为了改动信息。

它意味着怎么写工具，对感知层的工具同样适用（返回字段要精简、错误信息要写给模型看），也意味着感知与执行在架构上可以复用同一套基础设施。

四种失效的情况：

失效项	说明	中文
Hallucination	reports what is absent	幻觉：报告出屏幕上根本不存在的东西（最严重）
Latency	vision + calls add up	延迟：视觉处理与工具调用的耗时是累加的（时序错导致状态漂移）
Context blowup	screenshot + DOM	上下文爆炸：截图加 DOM 一起塞进上下文
Cost	fidelity is not free	成本：保真度不是免费的

解决思路：先修会静默出错的，再修慢的，最后优化贵的。因为静默错误会被后续所有步骤继承，而后两者至少是可观测的。

Part 4 Reasoning 推理子系统

4.1 推理的职责

职责	课件定义	中文
Decompose	Break a big task into steps you can actually execute	分解：把大任务拆成你真的能执行的步骤
Choose	Generate several candidate plans, then pick one	选择：生成若干候选计划，然后挑一个
Reflect	Judge what happened and revise the plan	反思：判断发生了什么，并修订计划

靠提示词：能力存在于输入里。每次调用都要把请一步步写进上下文，占 token，而且不稳定；

靠训练：能力存在于权重里。推理过程被内化进了模型，不再需要外部提示去激活它。

4.2 CoT 与分解的时机

1. 基础对比：标准提示词 vs CoT 提示词

   标准 Prompt：直接输出答案，复杂算术题容易直接答错。

   CoT（Chain-of-Thought，思维链）：让模型分步写出思考过程再输出答案，显著提升推理正确率。

2. CoT 真正的贡献：推理精度可以靠推理时的计算量换取，而不是只靠模型参数量、训练数据集。

这就是缩放的第二根坐标轴。

> 轴一（训练时缩放）：把参数变大、把训练数据变多。这是 2020 年前后的主线；
>
> 轴二（推理时缩放）：让模型在回答前多花算力（多想几步、多采样几条路径、多搜索几棵分支）。这是 CoT 打开的；
>
> 技巧会过时，轴不会，全都建立在推理时算力可以换准确率这个前提上。

3. 两层视角看待 CoT

   作为提示词技巧：如今已经过时，现代推理大模型内部原生就会做分步思考，不需要人工写这个 prompt。

   作为思想范式：衍生出整套 “思考模型” 的研究方向。

4. 任务拆解的两种策略：先全部拆解 / 边做边拆

   Decomposition first 前置拆解（Plan-and-Solve、ReWOO）：任务开始前一次性把完整计划全部定好。
   缺陷：遇到未知数量子任务（例如不确定邮件数量、附件），计划第一步就出错，后续全部连锁失败。

Interleaved 交错式拆解（CoT、ReAct）：一次只展开 1~2 个步骤，边执行边规划。适合子任务数量未知的场景，无法预先固定总步数。

策略	两种做法
(a) Sequential Decomposition-Planning 全部先规划，再逐步执行	Decomposition first —— Plan everything up front; 代表：Plan-and-Solve, ReWOO
(b) Interleaved Decomposition-Planning 交错：规划一步、执行一步	Interleaved —— Reveal one or two steps at a time; 代表：CoT, ReAct
(c) DPPM 动态的、逐步演化的规划	规划本身随执行展开

Tips：当任务元素的数量与形态在运行前可知时，先规划是更高效的（一次想清楚，减少往返）；当元素数量未知时，交错式（CoT / ReAct）才成立。先规划 = 把分解挪到设计期，交错 = 把分解留在运行时。

4.3 推理结构

4.3.1 链式、树式、图式推理（Chain / Tree / Graph）

1. IO Prompt 输入输出：输入直接到输出，没有中间思考。
2. CoT 思维链：单条线性思考路径，从头到尾一条思路。
3. CoT-SC（Self-Consistency 自一致性）：采样多条独立 CoT 路径，多数投票选出最终答案，消除采样噪声。仅适用于离散答案。
4. ToT Tree of Thoughts 思维树：每一步生成多个候选思路，用 BFS/DFS 搜索思路分支；24 点游戏中，CoT 只有 4% 成功率，ToT 可达 74%，因为 24 点拥有完美算术校验器。
5. GoT Graph of Thoughts 思维图：思路可以合并、互相转换，不止树形分支。

全称	课件定义	结构特征
Chain of Thought	One line of thought, start to finish 一条思路，从头走到尾	线：单一路径，不能回溯
Tree of Thought	Several thoughts per step, searched by BFS or DFS 每一步生成若干想法，用 BFS 或 DFS 搜索	树：可分叉、可回溯，但分支之间不能合并
Graph of Thought	Thoughts can merge and be transformed 想法可以合并与变换	图：允许汇聚与改写，结构最灵活

Tips:

从 CoT 到 ToT 到 GoT，每一步都在增加结构自由度：线不能回头，树可以回头但不能合并，图既可以合并也可以变换。

但是自由度增加带来两个直接后果：需要搜索与评估的机制和计算开销上升，所以要根据情况选择合适的推理结构。

ToT 的优势依赖于存在可靠评估器（下一节）这个前提，而不是树形结构本身更聪明。

4.3.2 评估器

CoT-SC：多数投票，零额外代价，仅离散答案可用。

ToT / GoT：大模型自行打分评估每一步分支；开销大，而且模型评判本身会有噪声。

LLM-MCTS/RAP：需要模拟推演到终点，或者价值评估模型。

核心结论：所有搜索类推理方法都必须配备评估器 evaluator。劣质的评估器反而会让搜索结果更差，因为会自信地选错分支。分支带来的计算开销是线性增长，评估器的可靠性才是瓶颈。

三种评估器按可靠性排序

算术 / 单元测试 / 环境反馈的特点是可靠，代价是只适用于特定任务；

多数投票（CoT-SC） 的特点是免费，但要求答案是离散的，且多数不等于正确；

让同一个模型给自己的步骤打分的特点是最通用但最不可靠（同一个模型的两个错误容易同源）。

4.4 ReAct：工作的循环

Same question, four strategies. Watch what Act-only lacks, and where CoT hallucinates.

同一个问题，四种策略；看只有行动的那一版缺了什么，以及 CoT 在哪里产生幻觉。

Action space = environment actions + language 动作空间 = 环境动作 + 语言（语言本身也是一个动作）；

Thoughts are free — they change nothing outside the model. 思考是免费的， 它们不改变模型之外的任何东西；因为它意味着 Thought 是唯一可以在不产生副作用的前提下试错的环节。这正是 ReAct 相对于直接行动的安全边界：能想清楚的事，就不要用动作去试。

example：邮件事故中的 agent 直接执行了 trash EVERYTHING，中间没有任何 Thought 环节去问这个动作可逆吗。把 Thought 加回来，就有了一个能承载风险判断的位置。

Observations are the only ground truth 观察是唯一的真实来源。

影响：推理与行动不能分家。不用再在先想清楚再动手和边做边想之间二选一，ReAct 证明了两者必须在同一个循环里交替。

4.4.1 一个 while 循环

In engineering terms, an agent is this while loop.

用工程语言说，一个 Agent 就是这个 while 循环。Agent 不是一种新架构或者新模型，它是一个模型 + 工具字典 + 显式的停止条件的循环。所以多数 agent 框架的差别，都是在这三样东西上加约束或加调度。

性质	对应代码
Stop conditions must be written explicitly. 三个停止条件必须被显式写出来	range(MAX_STEPS) / out.done / detect_loop(...)
Messages grows monotonically. 消息列表是单调增长的。它保证了模型能看到完整历史（可归因），但也意味着上下文只会越来越长	messages += [out.raw, f"Observation: {obs}"]
The model never touches the system. 模型从不直接触碰系统。一个安全边界：模型只产出要调用哪个工具、参数是什么，真正的执行发生在你自己的代码里。	真正的系统调用发生在 TOOLS[out.tool](**out.args)
TOOLS is the action space. TOOLS 就是动作空间，拦截危险	动作集合 = 这个字典的键集合

4.5 Reflection

4.5.1 四个动作 Four Moves

Reflection: Learning from mistakes without touching weights. 不碰权重地从不犯错中的学习。

Reflection is verbal reinforcement learning: the feedback is written in language and put back into the context — no gradient step.

反思的机制：语言上的强化学习，反馈被写成语言，再放回上下文里，没有梯度更新。

动作	回答的问题
Self-evaluation 自我评估	Compare what happened to what was expected 把实际发生的和期望的比一比
Error analysis 错误归因	Bad prompt? wrong tool? logic slip? world changed? 是提示写坏了？工具选错了？逻辑漏了？还是环境变了？
Correction 纠正	Revise the plan, the tool use, or the memory 修订计划、工具用法或记忆
Goal-driven 目标驱动	Reflect on efficiency too, not just on failure 不只在失败时反思，也要反思效率。把反思从纠错升级成优化

4.5.2 Reflection Loop 反思循环

循环的组件与功能：

Actor：Generates thoughts and actions from state + memory

行动者：依据状态与记忆生成思考与动作

Evaluator：Scores the trajectory: —the exact match, heuristic, or an LLM

评估者：给整条轨迹打分，可以是精确匹配、启发式规则，或者另一个 LLM

Self-reflection model：Turns a pass/fail signal into specific written advice

自反思模型：把一个通过/失败的信号，转成具体的书面建议

Tips：

1. Reflections enter working memory — up to three;

反思不是越多越好：反思内容本身也要占上下文，而上下文是有限的会进入工作记忆，最多累积三条，所以反思必须写得精炼、可行动；

2. A heuristic stops runs that loop or hallucinate

一个启发式规则会中止那些打转或产生幻觉的运行；

3. Stuck: A loop repeating the same action with the same result is the definition of stuck.

同一个动作在不同环境下可能是合理的，但如果动作和结果都一样，那就一定是在原地打转。

4.5.3 Feedback and Self-correct

With no external signal, asking a model to review its own reasoning lowers accuracy. It has no reliable way to tell "I was wrong" from "I was right" — so it changes correct answers too.

在没有外部信号的情况下，让模型复查自己的推理，会降低准确率。它没有可靠的办法区分我刚才是错的或对的，所以它会把正确的答案也改掉。验证器是反思的前提，要执行反思，先要能验证。没有验证手段时，宁可不要反思环节。

Reflection works when there is a judge（有裁判时才有效）	And fails when there is not（没有裁判时失效）
✓ HumanEval — unit tests run（单元测试会真的跑起来）	✗ Pure math reasoning — no referee（纯数学推理，没有裁判）
✓ ALFWorld — the environment replies（环境会回话）	✗ Open-ended writing（开放式写作）
✓ Game of 24 — arithmetic checks（算术可以检查）	✗ "Does this look right to you?"（「你觉得这个对吗？」）

三档反思：

最强：可执行的检查 unit tests、算术、编译、schema 校验。它们给出的是客观通过与失败；

中等：环境回复 ALFWorld 类的交互环境会如实告诉你动作成功没有；

最弱且常被误用：让模型自己说你觉得这样对吗。

4.5.4 事前反思 Reflect Before

This is why anticipatory reflection matters: it asks the model to imagine a failure, not to judge one. It therefore sidesteps the previous slide's problem entirely — no referee required.

事前反思的意义在于：它要求模型想象一个失败，而不是判断一个失败。因此它完全绕过了上一页的难题，不需要裁判。判断需要一个外部标准，而想象不需要。模型不需要知道我刚才做对了没有，只需要知道有哪几种做法可能出问题。

Example：事前方式：按 RePrompting 的形式，在发邮件给财务这一步之前，反思表格是否已经写完并核对过？unmet → 先回去补表格，不要发信；按 Devil's advocate 的形式，如果这封汇总邮件发错了，会是怎么错的？可能的答案：漏了几封、金额读错、发错收件人。

不可逆的动作最值得在事前做一次失败想象，事后修复的成本是无穷大。

两种反思（事前与事后）不可替换：事前反思负责减少失败（省下那一步），事后反思负责从失败中学习（改进后续尝试）；事前不需要裁判，事后需要。

4.6 六种推理策略

Option	What it buys	What it costs	Needs a verifier?
Single call	Cheapest, lowest latency	No recovery at all	No 它们不搜索、不修订，没有选择这一步
CoT	Accuracy for compute	More tokens per answer	
CoT-SC	Cancels sampling noise	n× the calls	Discrete answers only 只在答案离散时可用多数投票代替裁判
ToT / GoT	Explores wide spaces	Branch cost + scoring cost	Yes — critical 坏评估器让搜索比不搜索更差
ReAct	Grounds each step in	Context grows every turn	The environment is one 环境即是评估器，观察是唯一的真实来源
Reflexion	Improves across attempts	Wasted failing runs	Yes — or it degrades 没有反馈时自纠正会降低准确率

Part 5 Meomory 记忆子系统

5.1 借用人类的记忆地图

人类三级记忆映射到 LLM 系统的三个部件：

人类记忆层级与LLM的映射	说明
Sensory→ embeddings	Raw input turned into vectors 原始输入被转成向量
Short-term→ context window	Finite — about 4 chunks in humans 人类的短时记忆容量极小但会主动遗忘，而上下文窗口容量更大且不主动遗忘
Long-term→ external store	Vector database, queried at retrieval time 外部存储：向量数据库，在检索时被查询

5.2 RAG 的两个阶段

5.2.1 RAG 的两个阶段

检索增强生成的两段流水线：Query → Retrieve → Augment。

Retrieve：vector search over docs（在文档上做向量检索），

Augment：docs + query → LLM（文档与查询一起交给 LLM）。

What you gain	What you pay
Reaches data the model never trained on 能触达模型从未训练过的数据	Retrieval quality caps everything after it 检索质量封死了它之后的一切上限。后面的生成模型再强，也无法弥补前面检索漏掉的内容。如果正确的文档没被检索出来，模型没有任何机会用它；而如果检索出来的是无关文档，模型还会被带偏。
Answers become traceable to a source 答案可以追溯到来源	An indexing pipeline to maintain 多了一条需要维护的索引流水线。反思需要裁判。在 RAG 系统里，答案里的每句话能不能在检索到的文档里找到依据本身就是一个可用的裁判。
Cuts hallucination on internal facts 减少内部事实上的幻觉	Retrieved text competes for context space 检索回来的文本会和别的内容抢上下文空间

5.2.2 排序策略

Filling the context is not the same as using the context. "Bigger context window" is not a memory strategy. This is the bridge to context engineering.

把上下文填满，和把上下文用起来，是两件事，给的信息越多 ≠ 答案越好；更大的上下文窗口不是一种记忆策略，是为了上下文工程。

排序策略

既然中间位置会掉分，那么检索结果的排序顺序本身就成了一种要设计的策略。

把最可能相关的放两端，把最不确定的放中间，注意，这与按相似度从高到低排是不同的策略。

人类的短时记忆靠遗忘来腾空间，而上下文窗口靠位置来分配注意力。

5.2.2.1 CoALA 分类法

Classify by content type, not by storage location — Retrieval strategy follows content.

按内容类型分类，而不是按存储位置分类；检索策略要跟着内容走，而四类内容的存取方式不同：

1. Working Memory 工作记忆

   含义：当前这一轮的上下文，Agent 当下正在处理的信息，对应 LLM 上下文窗口

   例子：this run's 7 emails（本次任务里的 7 封邮件）

2. Episodic Memory 情景记忆

   含义：过去特定的经历、事件轨迹，类似个人日记，记录 “发生了什么”，带时间 / 场景

   例子：last week Sato's amount was in a PDF（上周 Sato 的数据在一份 PDF 里）

3. Semantic Memory 语义记忆

   含义：关于世界的客观事实、静态知识，知识库 / RAG 库，抽象事实，不绑定具体场景

   例子：Finance is finance@...（财务相关的定义 / 客观信息）

4. Procedural Memory 程序记忆

   含义：怎么做一件事，流程、方法、技能，做事的步骤、代码 / 操作模板

   例子：build the sheet before filling it（先建表格，再填数据）

Agent 整体架构

三大长时记忆：Procedural / Semantic / Episodic，都支持 Retrieval（检索读到工作记忆） + Learning（学习写入保存）

Working Memory（工作记忆）是中央工作台：所有检索出来的信息在这里做推理、反思（Reasoning）

Decision Procedure（决策模块）：LLM 解析 Prompt、检索记忆，输出 Action；Agent 通过动作和三类环境交互：Dialogue 对话、Physical 物理世界、Digital 数字环境

CoALA 核心亮点

很多早期 Agent 架构是按数据库位置区分记忆（比如：向量库 A、文件存储 B）； CoALA 提出：按信息本身是什么类型来分类。同一个存储数据库里，可以同时存放 episodic、semantic 内容；检索策略要适配内容类型，而不是适配存储介质。

CoALA 是一套LLM 智能体的认知架构分类框架，把 Agent 记忆拆成工作记忆 + 情景记忆 + 语义记忆 + 程序记忆，并且定义了 Agent 观察 - 规划 - 评估 - 选择 - 执行的决策循环，用来统一梳理、设计、对比各种 LLM Agent。

5.2.2.2 四种记忆方式

工程实现的四种落地方式：

方式	定义	中文
Embodied	Baked into the weights by fine-tuning. Expensive; often impossible on closed models.	写进权重：靠微调写进参数。 昂贵；在闭源模型上常常做不到
RAG	Retrieve documents, then answer them. Citable — the usual default.	检索文档，再据此回答。 可引用——通常的默认选择
SQL database	Structured rows via text-to-SQL. Wins when the answer is a number on a table.	结构化行：通过 text-to-SQL 查询。 当答案就是表里的一个数时，这种方式胜出

方式	定义	中文
Files 主力	Let the agent write notes into the filesystem and read them back next session.	文件：让 agent 把笔记写进文件系统，下次会话再读回来

适用情况：

答案是一个数、且数据在表里 → SQL database；

答案是一段可引用的陈述，且语料庞大 → RAG；

要跨会话累积经验、且希望人类能读懂 → Files；

要让能力成为模型的本能、且愿意付出训练成本 → Embodied（微调）。

Tips：Files是记忆的主力，因为RAG 需要嵌入模型、向量库、分块策略、重建索引，每一层都是一个可能失效也可能需要调优的环节；而文件系统只需要 write 和 read；并且有这三个优点：

- No indexing pipeline to maintain 不需要维护索引流水线；
- Naturally auditable — you can read it 天然可审计，你可以直接读它；
- The agent organises its own structure 结构由 agent 自己组织。

5.3 存储什么信息

核心原则：不要无脑保存原始日志，要提取可复用的抽象知识

WORTH KEEPING（值得保存），对应 CoALA 四类记忆：

1. semantic 语义记忆：客观规则、事实
2. semantic 语义记忆：静态事实信息
3. procedural 程序记忆：做事流程、操作方法
4. episodic 情景记忆：特定事件、过往经历

NOT WORTH KEEPING（不值得保存）

例子：click(412,286) 这种底层原始动作坐标

问题：环境 / 页面布局一变，这条记录直接失效；Agent 还会自信地继续使用，造成错误误导后续任务。

抽象层级决定可迁移性

AGENT WORKFLOW MEMORY 智能体工作流记忆

> 从大量具体任务轨迹（trajectories）里提炼出可复用的工作流程，而不是直接存储整条原始轨迹 记忆的价值取决于抽象程度：你存的内容越具体，就越难迁移到新场景。

5.4 记忆实效

Context window 上下文窗口限制

问题：LLM 一次性能装载的内容有硬性上限（hard ceiling）

常规解法：截断文本（truncate）或者做摘要（summarize）

缺陷：摘要会丢失后续可能需要的细节

Duplication 重复冗余

问题：大量几乎一模一样的记忆记录不断堆积

解决方案： 把多次相似任务浓缩合并成一套通用 plan（计划）或者只保存计数，而不是重复保存完整副本

Part 6 Execute 执行子系统

6.1 为 Agent 设计工具的四条规则

给 Agent 设计工具，是项目里收益最高的事情；

The model understands your tool only through its names and its error text. Both are prompt engineering, whether you meant them to be or not.

模型只能通过工具的名字和它的错误文本来理解你的工具。这两样东西都是提示工程。

4 条设计准则

1. Split by task, not by endpoint 按任务拆分工具，而不是按 API 端点拆分。把列举 + 过滤合在一起，代价是每次多一次模型往返，收益是模型不必自己写过滤逻辑，而筛选逻辑正是最容易出错的地方。
   - ✅ find_expense_emails(week)（面向任务）
   - ❌ list_emails() + filter yourself（底层原始接口，让模型自己过滤，容易出错）
2. Optimise returns for tokens 返回结果要做 token 优化。返回越冗长，关键信息越容易被淹没。
   - ✅ 返回精简字段：{id, subject, amount}
   - ❌ 返回完整原始 MIME 邮件正文（token 开销巨大）
3. Write errors for the model 为模型编写可读错误信息。把错误处理从模型自己推理变成了接口直接告知。
   - ✅ sheet is read-only, call unlock_sheet first（自然语言，告诉 Agent 该怎么做）
   - ❌ HTTP 403（机器码，LLM 很难理解含义）
4. Name parameters self-evidently 参数命名自解释。这在有多个可选工具时，用参数名区分工具。
   - ✅ recipient_email
   - ❌ arg2（无意义变量名）

第三点的核心结论：能自建接口，就不要让 Agent 去点击像素坐标 。

例子 SWE-agent：直接操作像素模拟点击（human tools）成功率≈3%；专门构建的 Agent 友好接口 ACI 成功率 12.5% 同模型，仅仅更换接口，性能大幅提升 推荐给 Agent 的环境组件：带行号文件查看器、限制窗口大小、每次编辑后自动运行 linter 并返回错误。

它与 Set-of-Mark 是同一种思路的两个应用场景：SoM 把空间定位变成报编号，ACI 把编辑代码变成按行编辑 + 自动检查；

它与 规则一（按任务切分工具）也是同一个方向：把模型不擅长的低层操作，封装成模型擅长的形式。

6.2 MCP: Model Context Protocol 模型上下文协议

标准化的 Agent 工具通信底层协议（A client - server protocol），Client-Server 架构

问题：M 个 Agent，N 个工具，需要 **M×N** 套独立适配适配器

解决：Agent 实现一次 MCP，工具实现一次 MCP，直接互通，只需要 M+N 份适配代码

MCP 对外暴露三类东西：

1. tools：可调用工具
2. resources：可读资源
3. prompts：可复用提示模板

Tool discovery and invocation become one interface, not per-framework glue.

作用：工具发现、工具调用统一成一套接口，不用为每个框架单独写 glue 代码。一个协议同时覆盖了感知、执行、推理三个子系统的接口需求。

暴露的东西	说明
工具 tools callable	可调用的工具（四条规则）
资源 resources readable	可读取的资源（感知与执行共用同一套机制）
提示 prompts reusable templates	可复用的提示模板（推理策略，提示形态的封装）

6.3 Execution Breaks

三个根源问题

1. Latency：Vision and physical action run on different clocks

延迟：视觉感知和物理动作的时钟不同步

2. Error propagation：Failure can start in perception, planning or execution

错误传播：错误可以起源于感知、规划或者执行任意一环，逐级放大

3. State sync：The agent's belief must match the real screenAgent

状态同步：内心相信的状态，必须和真实屏幕 / 环境状态保持一致

经典 debug 难题：Agent 点错按钮外部现象完全一样，但是根源分 3 种：看错屏幕 / 规划错步骤 / 正确规划但执行出错 。所以必须保存轨迹日志 trajectory logs 记录每一步：观察 observation、思考 thought、动作 action、结果 result 没有日志，无法区分错误类型，不能定位 bug

Part 7 Failure and Evaluation 失败与评估

7.1 Classical Wrong

四大子系统及检验的典型故障

Subsystem	Typical failure	Task 例子
Perception 感知	GUI 定位识别，读错内容	把 ¥5.000 识别成 ¥5000
Reasoning 推理	级联规划、死循环	计划处理 5 封邮件，实际找到 7 封，计划崩了
Memory 记忆	过时信息、检索污染	调用旧的财务邮箱地址发邮件
Execution 执行	不可逆操作，状态漂移	发送不完整的摘要

Subsystem	Typical failure	Task 例子
Verification 校验	没人检查结果	Agent 说 “完成”，漏掉 2 封邮件

7.2 Benchmark Half-Life 基准测试的半衰期（OSWorld）

OSWorld：369 个真实电脑操作任务

- 2024 原始论文：12.2%
- 2025 中期：≈42.9%
- 2026 OSWorld-Verified：≈85%，超过人类基线

思考题：

1. 模型真的学会操作电脑了？还是 benchmark 被刷饱和了？

OSWorld-Verified 修复了评估脚本，基线本身变了，数字不能直接横向对比

2. 成功率是唯一指标吗？

不是。一个正确率 85% 但要花两三倍时间的系统，在很多场景里依然是不可用的。比如，顶尖 agent 的步数是人类轨迹的 1.4 - 2.7 倍。做对了，但慢到无法部署。Goal-driven（Reflect on efficiency too, not just on failure）效率是与正确性并列的一等指标。

3. 排行榜数字为什么不一致？

它说明在 agent 评测里，模型不是唯一的变量。脚手架、最大步骤限制、重试次数不同，结果差异很大

7.3 MAST: 14 Ways a Multi-Agent System Fails 多智能体 14 类故障

分析 7 个多智能体框架真实失败轨迹，故障分为三大类：

1. System Design Issues 系统设计问题（41.8%）
2. Inter-Agent Misalignment Agent 之间对齐问题（36.9%）
3. Task Verification 任务校验问题（21.3%）

7.4 Prompt Injection 提示注入

案例：邮件正文里面嵌入系统指令

核心问题：Agent 无法区分用户指令 和 数据文本里面的指令；两者会进入同一个上下文窗口。这不是简单 bug，是架构本身结构性缺陷。

触发条件（三者同时满足就危险）

1. 读取不可信外部内容
2. 有权限访问敏感数据
3. 可以对外通信

缓解方案（**没有完美方案**）

- 标记不可信内容，降低它的权重

- 不可逆操作必须人工确认
- 动作空间限制，邮件只能发给白名单邮箱
- 把读取不可信内容的逻辑隔离在低权限子 Agent 里面

Lecture 3 Reasoning in LLMs (1)

Part 0

问题：怎么在不改模型权重的前提下，让它把答案算对？

答案：把一次性输出改成先花算力想，再输出，想的思维链（chain-of-thought）既是草稿纸，也是算力的计量单位。但是代价是想得越多，token 烧得越多，用算力换准确率。

方法一：写得更长

让模型生成更长的思维链。依次给例子（few-shot CoT）、给指令（zero-shot CoT）、让模型自己造例子（analogical prompting）、让模型自己改提示词（prompt optimization）、把问题拆成小问（least-to-most）、让模型自己组推理结构（Self-Discover）。

方法二：挑得更好

既然一次生成可能错，那就生成很多条，然后挑。依次是投票（self-consistency）、按执行结果聚类（AlphaCode）、让模型自己投票（universal self-consistency）、训练一个打分器（ORM / PRM）、把打分器插进搜索过程（Tree-of-Thought）。

Core idea:

1. trigger the LLM to generate long chain-of-thought (CoT)

触发模型写长思维链。

2. variable computation of the thought process adapting to tasks of different difficulty levels

思维链的意义是让计算量随题目难度变化：难题多写几步，简单题少写几步。

3. The best practice to interact with LLMs evolves over time

鼓励长思考 + 支持任务需要的推理策略。

4. When LLM self-evaluation works well: search in the partial solution space can help

当模型能可靠自评时，在半成品上搜索比生成完整答案再挑更省、也更准。

Part I Introduction of Reasoning

1.1 Reasoning Models

The trick didn't make the model bigger, it was just that model thought longer

这次突破不是把模型做大，而是让它想得更久。后面所有技术（CoT、自洽性、树搜索）都在同一个方向上发力，即延长或拓宽想的那一段，而不是换一个更大的模型。

Cost: Not Free

O3 achieved 87.5% accuracy on ARC-AGI, with >$1k test-time cost to solve each task. Other existing LLMs achieved <25% accuracy if no special inference-time techniques were used.

o3 在 ARC-AGI 上达到 87.5% 的准确率，但每解一题要花超过 1,000 美元的测试时算力。 不用任何推理时间技巧的话，其他模型都在 25% 以下。图中 o1 low 正好压在 25% 这条线上，KAGGLE SOTA 也在这附近。

OpenAI o1

The visible answer is only the tip of the whole picture of the iceberg.

能看到的回答只是冰山一角，大部分工作发生在你永远看不到的 token 里。o1 那一代不给原始推理轨迹，所以不能拿它做调试目标，只能拿最终的答案对错来评估。所以需要 verifier 这类外部打分器来替代读模型的思考。

Gemini 2.0

Gemini 2.0 Flash Thinking demo: generating long thought before the solution

先长思考、再短答案，把这个确认为产品层面的标准形态，而不是本模型独家做法。这一代模型（以及 Claude、Gemini 的后续版本）允许你看到完整内部思考，o1 则不给权限。

1.2 Four Approaches to trigger CoT

Core idea: trigger the LLM to generate long chain-of-thought (CoT)

Approaches to trigger CoT generation:

1. Few-shot CoT prompting 少样本思维链提示：纯推理时间手段，给几个带推理过程的例子即可
2. Instruction prompting 指令提示：纯推理时间手段，一句话触发（如 Let's think step by step）
3. Instruction tuning 指令微调：要改模型权重，属于训练侧
4. Reinforcement learning 强化学习：要改模型权重，属于训练侧

Tips:

前两条不动权重：模型拿到更长的上下文，自己写完再答。这就是 inference-time techniques。

后两条要动权重：把先推理再回答这个模式直接写在参数里，不需要人在提示里触发。

本质都在回答同一个问题：scaling token budget（放大 token 预算），即怎么把花在思考上的 token 变多、并且花得值。

Part 2 Basic Prompting Techniques

2.1 从标准提示到思维链 From Standard Prompting to CoT

2.1.1 Background: Standard Prompting

标准少样本提示（standard prompting）的局限：standard few-shot exemplars only provide information on the final solution format, but not the rationale to derive the solution. 只教了格式，没教理由。

环节	课件原文
题目	Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
Exemplar	A: The answer is 11.

环节	课件原文
真实待解题目	Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
Model Output（模型输出）	A: The answer is 27. （答错）

Before the advancement in post-training techniques, standard prompting performance is poor on reasoning benchmarks. 在后训练技术尚未发展的阶段，标准提示被定义为表现差。

2.1.2 Development：CoT 思维链（把思考写进示例）

两层机制：CoT本质是一种提示写法

1. The model can imitate the written procedure, not just from the format。

模仿。模型现在有一个可模仿的程序，而不只是格式。

2. Each step gets to build on the last one instead of everything happening, just a single pass

把中间结果写进上下文。每一步的中间结果都变成下一句的输入，误差不会在一次前向传播里累积。

2.2 思维链的规模效应 CoT Scales

CoT performance improves more significantly with the increase in model size. Better models benefit more with CoT generation. A drastic improvement in reasoning performance when the model reaches a certain scale.

模型越大，思维链带来的提升越显著。更好的模型从思维链中获益更多。规模跨过某个门槛时，推理表现出现陡增。

Note: these experiments used pretrained-only LLMs. Recent post-trained LLMs might have different scaling curves, but the main conclusions still hold.

这些实验用的是仅预训练的模型，近期的后训练模型曲线可能不同，例如，仅预训练的 base 模型和后训练过的 instruct 模型，同一条提示下的表现差别很大。但更强的模型，多花思考时间更划算，这个总的结论不变。

Tips: 鼓励模型写思维链的前提：要有足够的推理能力。

2.3 零样本思维链 Zero-shot CoT

核心思路：直接在问题后面加一句提示：Let's think step by step，不需要提前给示例（exemplars），就能让大模型自动生成思维链推理过程。

对比四种范式：

1. Few-shot：给少量问答示例，直接输出答案，没有分步推理
2. Few-shot-CoT：给少量带分步推理的示例，再提问
3. Zero-shot：不给示例，直接提问，模型直接输出答案
4. Zero-shot-CoT：不给示例，只加一句 “一步步思考”，诱导模型自己做分步推理

结论：Let's think step by step 可以无示例触发 CoT 推理。

Zero-shot-CoT 在绝大多数任务上显著优于普通 zero-shot：

- 算术任务：SingleEq、AddSub、MultiArith、GSM8K、AQUA、SVAMP 全部上涨
- 常识推理：CommonSenseQA 略降，但 StrategyQA 大幅提升
- 其他推理：Date Understanding、Shuffled Objects、Last Letter、Coin Flip 都明显提升

短板：

Zero-shot-CoT 效果依然弱于 Few-shot-CoT

- Zero-Shot-CoT：78.7 / 40.7
- Few-Shot-CoT(8 samples)：93.0 / 48.7

问题：如何提升 CoT 性能，又不用人工标注推理示例?

人工写 CoT 示例成本很高，希望免除人工标注。

实验结果

The reasoning ability was already sitting on the model, and that prompt was just unlocking it.

推理能力本来就在模型里，提示词只是把已有的能力解锁出来。

2.4 Analogical Prompting 类比提示

2.4.1 Core Idea

Analogical prompting: instruct the LLM to generate exemplars. Prompt the LLM to **first recall relevant exemplars**, before solving the test problem.

让大模型自己先生成相关例题（exemplars），再去解决当前问题

1. 指令：先回忆、生成和当前题目**相关的例题**（Relevant exemplars）
2. 再使用生成的例题，求解原始问题

灵感来自人类类比推理（Polya《How to Solve It》）： 人遇到新问题时，不会每次都有人给示范；而是主动回忆过去相似的解题经验，借助旧问题的思路解决新问题。

2.4.2 Advantage

Exemplars are **self-generated by LLMs, **no manual labeling.

1. 例题由模型自己生成，不需要人工标注，节省标注成本

Exemplars are **tailored **to individual problems.

2. 生成的例题针对当前题目量身定制（tailored），匹配度更高

示例

题目：求四个顶点(-2,2), (2,-2), (-2,-6), (-6,-2)构成正方形的面积

1. 模型先自己生成相关例题：边长为 5 的正方形，面积 = 5^2=25
2. 再套用这个解题思路，计算原题：求边长→计算面积，得到答案 32

2.4.3 整体逻辑链

1. 普通 Zero-shot：直接答题，推理弱

2. Zero-shot-CoT：加 Let's think step by step，不用样例，提升推理，但上限不如 Few-shot-CoT

3. Few-shot-CoT 效果好，但需要人工手写推理样例，成本高

Issue：zero-shot CoT performance is still worse than a few-shot CoT. How to improve CoT performance w/o manually labeling exemplars?

Answer：Analogical Prompting：让模型自主生成类比样例，模拟人类回忆相似题的思维，不用人工标注，去逼近 Few-shot-CoT 的效果

tips：为什么少样本更强：手工写的示例携带了额外信息。 they will carry extra information and they will show the model what good reasoning looks like，示例不只是格式，它示范了好的推理长什么样。

为什么这不令人满意：手工标注示例很贵。要覆盖各类题型，就得写很多例子。

Humans are not explicitly given demonstrations every time for a new task, Instead, humans intrinsically recall from past relevant experience 人每接一个新任务，并不会有人先给你演示一遍，人会从过去的相关经验里主动回忆。

2.5 Example：竞赛编程实例

1. 提示词：先讲算法再写教程

Besides exemplars, the LLM can also self-generate high-level knowledge. The generated knowledge complements the problems with broader insights

除了示例，模型还能自己生成高层知识，生成的知识用更宽的视野补足具体题目。它是一个已经解完的题，能直接照着做，但换一道题可能就不适用。

知识抽象：它是前缀积算法这类可迁移的方法论，适用范围宽，但单看它不会写代码。所以提示词要求两者都给，在两个抽象层级上同时覆盖。

2. 模型输出：算法知识 + 三道例题

Those examples are very concrete and specific, and knowledge is abstract and knowledge can be transferred better. It gives you the better result and coverage at two levels

Algorithms（识别算法）→ Tutorial（讲清算法）→ Example Problems（造三道相关题并给出代码）→ 最后才解原题。

自造知识：完整的算法描述 + 复杂度分析（O(n)）+ 可运行代码。

3. 最终作答

Example: Codeforces (solving the initial problem)

2.6 类比提示的结果

1. 结果总览：全面占优

Analogical prompting outperforms 0-shot CoT and manual few-shot CoT

类比提示优于零样本思维链，也优于人工少样本思维链。

Tips：为什么数学题涨的更优

For the mathematics problem, you have the definitions, you have the formula, you have all the concepts. And models should learn the concept first and then adapt that concept to specific questions。

数学任务的解法依赖可显式陈述的概念与公式，所以先唤起知识、再套用这条路收益最大。代码与时序推理虽然也涨，但它们更依赖具体模式，抽象知识能帮的忙相对少。

2. 越强的模型越会类比

Weaker LLMs benefit less from analogical prompting, though it does not hurt zero-shot performance

弱模型从类比提示中获益较少，但至少不会伤害零样本表现。 With stronger LLMs, analogical prompting outperforms CoT with manually-designed or retrieved exemplars

强模型下，类比提示胜过人工设计或检索得到的示例。

The generated CoT is more tailored to the underlying LLM

模型生成的思维链更贴合它自己。人写的例子是按人类的思路组织的，模型未必按那个顺序思考。模型自己生成的推理，用的就是它自己的计算路径，所以更容易被后续生成复用。

2.7 指令 Instructions Work

Current LLMs are sensitive to prompt design 当前模型对提示词设计非常敏感。

There is no clear principle of how to write optimal prompts 不存在怎么写最优提示的明确原则。

2.8 Prompt Engineering By LLM

2.8.1 Roles and Pipeline

这是**LLM 自动优化提示词**的一套方法，也叫元提示（meta-prompt）优化，让大模型自己生成、打分、迭代改进 prompt，不需要人工手写 prompt。

1. Proposal generation: leverage the LLM to generate initial instructions.

LLM 作为推理模型（Inference Model）：输入 prompt + 样例，让模型执行任务，输出答案，用来验证这个 prompt 好不好。

2. Scoring each instruction based on the prediction correctness of a small set of problems.

LLM 作为打分模型（Scoring Model）：给每一条候选 prompt 打分：用对数似然 log-likelihood，看这个指令能不能在少量测试样例上，预测出正确答案 Log Probability。

1. 高分候选：保留，继续迭代
2. 低分候选：直接丢弃
3. LLM 作为重采样模型（Resampling Model，可选）：在不改变语义的前提下，改写现有的高分 prompt，生成更多变体候选，扩大搜索空间。

流程：生成候选 prompt → 打分 → 计算概率分数 → 保留高分 prompt → （可选）重采样生成语义相近的候选 prompt

> 两个核心步骤：
>
> - 候选生成：让 LLM 生成一批初始指令 prompt
> - 指令打分：在小的测试集上，评估每个 prompt 的预测正确性

Tips：这里的评分标准是模型自己的对数概率，不是人类认为的质量。所以最好的提示词对人类来说往往很奇怪。

2.8.2 进阶：迭代改进 Prompt

让 LLM 利用历史优化轨迹（sorted (solution, score) pairs），不断生成更好的新提示词。

优化器 Optimizer：the LLM to propose a new instruction given old ones and task exemplars 接收旧 prompt、任务样例，提出新的指令。

gradient free optimization（无梯度优化）：这个循环类似强化学习或微调，但差别在于这里没有梯度，也没有 GPU。梯度就是模型读一张排行榜。模型从 (solution, score) 对里看出哪类写法分高，然后照着写一条新的。

评估器 Evaluator：the LLM to evaluate the accuracy of an instruction 评估一条 prompt 在任务上的准确率。

整体流程闭环：

1. 优化器 LLM 读取元提示（meta-prompt），包含：任务描述 + 历史（prompt,分数）配对
2. 优化器（LLM as optimizer）生成候选 prompt，交给模型推理，产出任务答案
3. 目标函数评估器（objective function evaluator）对生成的解做客观评分
4. 把新的（prompt,分数）存入 meta-prompt，再次喂回优化器，循环迭代
5. 迭代结束，返回得分最高的 prompt 作为最终方案（return top solutions when finish），本轮生成的新候选（generated solutions）

2.9 Prompt Optimization

问题：提示词写法没有理论指引，只能靠人试

第一版解法：让模型生成候选、按对数概率打分、保留高分并同义改写。

第二版解法：把历史成绩排序后喂回模型当梯度，让它写更好的。

元提示：排行榜 + 带占位符的示例 + 行动指令。

结果：从 60.8% 起步，跑到 80.2%，比人写的最强基线高 8.4 个点，追平少样本思维链。

形态：随步数上升后走平；打分器的选择直接影响上限。

代价：得到的提示词对人类来说往往很难看，且不一定可迁移。

2.9.1 元提示 Meta-prompt：给优化器看的指令

Meta-prompt 就是用来让 LLM 生成更好 prompt 的 prompt，例子是 GSM8K 数学题任务： 里面包含两部分：

1. 历史候选 prompt + 对应的分数：按分数排序，让模型看到哪些 prompt 好、哪些差
2. 任务样例 exemplars：展示这个任务的输入输出格式（GSM8K 数学应用题）

> 元提示要求 LLM：写出和旧版本不一样、尽可能高分的新提示词。

2.9.2 GSM8K 实验结果

基线：经典Let's think step by step.（CoT，71.8% 准确率）

初始 prompt：Let's solve the problem. 只有 60.8%

自动优化得到的最好 prompt：Take a deep breath and work on this problem step-by-step.，**80.2%，**比原版 step-by-step 高约 8%，达到 PaLM2 论文里 few-shot CoT 的水平。

不同优化器（PaLM、GPT3.5、GPT4）生成的 prompt 效果略有差别。

2.9.3 优化曲线图

横轴 = 优化迭代步数，纵轴 = 训练集准确率

多个任务（GSM8K、BBH 的 movie 推荐、时序序列）都有相同规律： 准确率随迭代步数上升，之后进入平台期（plateau，不再明显上涨）

不同打分器（PaLM2-L /text-bison）、不同任务，收敛速度不一样；但趋势一致：前期快速提升。准确率随优化步数上升，然后趋于平稳。

Part 3 推理策略与结构

3.1 CoT Reasoning Strategies

核心思路：把难题拆成多个简单子问题，逐个解决，逐步推导最终答案，解决大模型的组合泛化（compositional generalization）难题（测试样本比训练更长、结构更复杂）

variable computation：Chain-of-thought prompting: variable computation of the thought process, adapting to tasks of different difficulty levels. More complex questions -> more reasoning steps in the chain-of-thought

思维链的本质是可变计算量，推理过程的长度随任务难度自动伸缩。越复杂的问题，思维链里的推理步数越多。

Reasoning strategies enabled by CoT：Decomposition、Planning

思维链使能了一些推理策略：分解、规划，等等。

We can explicitly instruct the LLM with the desired reasoning strategies for problem solving

我们可以显式地指定要让模型用哪种推理策略。与其期待模型自己挑策略，不如把策略写进提示，或者让模型自己写一份策略。

接下来三种方法的共同动机：least-to-most 指定的是分解这个策略，dynamic least-to-most 给分解配上了例子检索，Self-Discover 则是让模型从一堆候选策略里自己挑并组装。

3.2 由易到难 Least-to-Most Prompting

3.2.1 用分解做泛化

easy-to-hard generalization via decomposition

两大阶段

Stage1: Problem Reduction

问题拆解，让大模型把原始复杂问题，自动拆分成一系列更简单的子问题。例子：Amy 滑滑梯题目，原问题拆出子问题：“一趟行程需要多久？”

Stage2: Sequentially Solve Subquestions

顺序求解子问题，逐个回答子问题；子问题的答案传给下一个子问题。上一个子问题的答案，追加进 prompt，作为上下文，用来解决下一个更难的子问题，直到算出最终答案。

3.2.2 Example 1：SCAN 数据集

SCAN: translating synthetic natural language commands into action sequences.

SCAN（把自然语言命令翻译成动作序列） 的任务是把合成自然语言指令翻译成动作序列。

Length split: action sequences in the test set are longer than training samples. These splits are designed specifically to break the models.

长度切分：测试集的动作序列比训练样本更长，切分是专门设计来打断模型泛化的。

Least-to-most prompting can achieve nearly perfect test accuracy with 0.1% training samples as exemplars

只用 0.1% 的训练样本做示例，就能达到接近满分的测试准确率。

结论：

SCAN 难点：**测试集动作序列长度 > 训练集**，考验组合泛化能力

普通 prompt 和原始 CoT 在长序列上失效。text-davinci-002 与 code-davinci-001 在 Chain-of-Thought 一列都是 0.0 说明：思维链在长度切分上完全失效，比标准提示还差。普通思维链必须在一次生成内完成从易到难的泛化。

而 Least-to-Most 把同一个基准从 16.7 推到 99.7（code-davinci-002）。同一模型、同一数据，只换提示结构。L2M 只用 0.1% 训练样例，就能在 SCAN 上拿到极高准确率，因为每个子问题都很短，不再依赖长序列示例。

算例二：

3.2.3 Example 2：自然语言到 SPARQL

把自然语言，如英语问题翻译成 SPARQL 查询

Examples in CFQ (Compositional Freebase Questions) benchmark

> 例 1：Question: Did M1 star M2 , star M3 , and star a art director and editor of M0? SPARQL: SELECT count(*) WHERE { ?x0 edited M0 . ?x0 art directed M0 . M1 starred ?x0 . M1 starred M2 . M1 starred M3 }
>
> 例 2：Question: What was produced by a art director that M1 and M2 employed? SPARQL: SELECT DISTINCT WHERE { ?x0 produced by ?x1 . ?x1 a art director . M0 employed ?x1 . M1 employed ?x1 }

- Challenge: more complicated grammar with a larger vocabulary

挑战在于语法更复杂、词表更大。

- A single prompt might not be enough to cover all grammar rules

单个提示覆盖不了所有语法规则。示例是有限的，而语法规则的空间很大。

- Decomposition enables customized prompt for each subproblem

分解不只是让推理变简单，还让每个子问题都能配一份定制提示。

Tips：SCAN 的困难是长度（序列更长）；CFQ 的困难是语法覆盖（more complicated grammar with a larger vocabulary）。

3.3 动态 Dynamic least-to-most prompting

基础 L2M 的升级版本，在拆解之外，增加动态检索样例 完整三步：

1. Problem Reduction：句法解析，把复杂问句拆成子问题
2. Dynamically Select Exemplars 从样例池（Exemplar Pool）里，根据当前子问题，检索匹配的相关示例，不是固定一套 few-shot 样例，每个子问题用专属样例。Retrieve using constituent from decomposition 触发检索的键是分解出来的成分，所以分解驱动、增强了检索。
3. Sequentially Solve Subproblems：带着检索到的样例，依次求解子问题，自底向上拼装，拼接得到最终答案。

静态版用的是同一批固定示例，所有子问题共用一个提示模板。

动态版用的是示例池 + 按需检索，每个子问题可能配到不同的示例。

3.3.1 Example：自然语言到 SPARQL

CFQ 难点：复杂语法、大词汇量；单一固定 prompt 很难覆盖全部语法规则。

Dynamic L2M（Ours）平均准确率 95.0%，超过所有全监督训练基线模型（T5、LeAR 等）。不仅拆分问题，还给每一个子问题动态找最适合的示例。

右侧曲线图：随着样例池增大，Dynamic L2M 始终 > CoT > Vanilla Few-shot，差距明显。

3.4 Self-Discover

核心思想：instruct the LLM to compose reasoning structures for each task without human demonstrations

让 LLM 自己发现这个任务专属的推理结构，不需要人工写示范样例，定位不是又一个更强的提示词，而是一个按任务生成提示结构的元方法。

分两阶段：

1. Task-level：Discover Reasoning Structure

输入任务描述 + 基础原子推理模块（Atomic Reasoning Modules），让模型生成任务专属推理框架（key-value 结构，是解题的步骤）。例如颜色物体计数任务：推理结构会定义：每种物品类型、每种颜色物品数量等中间问题。

2. Instance-level：Solve Problems with discovered structure

差别在于它加上了 Task Instance（具体题目实例），并在解码时把值填进去（Fill in the Values based on Keys during decoding），套用上一步生成好的推理结构，填充各个 key 的值，得到最终答案。

Tips：Different tasks, they have different reasoning structures 不同任务有不同的推理结构。比如数学题适合分解，但换一类任务就不一定。比如，社交推理题需要换视角（change your perspective）、调试类问题需要验证假设（test some hypothesis）

实验效果

- Self-Discover 对比直接输出答案：BBH 提升 + 11%，T4D+39%，MATH+5.5%
- Self-Discover 对比原始 CoT：BBH+7%，T4D+29%，MATH+8.5%

> 区别对比：
>
> - CoT：直接写一步一步自然语言思考
> - L2M：拆分子问题，顺序求解
> - Self-Discover：先为任务规划一套结构化推理模板，再用模板解题

3.5 Summary

1. CoT：可变计算量，思考过程长度自适应不同难度题目

分支一 让模型写得长：给例子→ 给指令→ 让模型自造例子→ 让模型自写提示。

分支二 让模型按结构写：显式指定策略（分解）→ 分解 + 检索→ 模型自组结构。

分支三 生成多个再挑：Part 4。

2. 推理场景下提升 CoT 效果的几种方案： Few-shot prompt，附带标注好的思考样例；直接用指令 prompt 触发模型做 CoT；让 LLM自动设计 prompt。
3. 提示策略的核心原则（不会过时）：复杂任务鼓励更长的 CoT 推理、提示策略要适配任务所需要的推理方式、和大模型交互的最佳实践一直在迭代更新。

方法	核心	特点
CoT	直接生成连续自然语言思考	简单，但是长组合任务容易崩
Least-to-Most	拆成子问题，顺序求解，带上前序答案	解决组合泛化
Dynamic L2M	L2M + 为每个子问题动态检索样例	CFQ 这类语义解析任务效果极强
Self-Discover	先生成任务专属结构化推理框架，再解题	不需要人工写 demo

Part 4 Search and Selection Methods

原版 Self-Consistency（数学，需要答案抽取） → 局限：无法用于开放式自由文本 → USC 通用自洽：不用抽取答案，开放任务可用 → 升级：不靠单纯一致性投票，训练 ORM/PRM 验证器，PRM 更强 → 局限：以上所有，都要等完整答案生成完才选择，浪费单步信息 → ToT 思维树：分步生成 + 分步评估，在中间 partial solution 上做树搜索（24 点例子）

核心主题：不再只采样 1 条 CoT 推理路径；生成多条候选推理分支，再挑选最优答案，用来修复单条生成里的错误。

4.1 挑战与新思路

之前 CoT、L2M 都是只生成单条推理路径（greedy 解码），存在缺陷：限制模型只能输出一个解法，一旦中间一步推理错，最终答案直接出错。

Idea: Exploring multiple branches allows the LLM to recover from mistakes in a single generation, generate multiple candidate solutions per problem, generate multiple potential next reasoning steps given the current (partial) thought.

新思路（搜索与选择）：探索多个分支，让模型有机会从单次生成的错误中恢复。对同一个问题，生成多条候选完整解；给定当前不完整的思路，生成多个可能的下一步。

Challenge: how to select the best response from multiple candidates? In most cases, we do not have an oracle scorer at inference time

核心挑战：如何从一堆候选里选出最好的回答？推理时（inference）我们没有打分器（oracle scorer），没法直接知道哪个答案是真值。

对比项	Part I - III（提示技术）	Part IV（搜索与选择）
核心动作	让一条思维链写得更好、更长	生成多条候选，然后挑一条
算力花在	单条链的长度（token budget）	候选的数量（sample budget）
新出现的问题	怎么写提示	怎么在没有标准答案的情况下挑
代表方法	CoT / analogical / least-to-most / Self-Discover	self-consistency / AlphaCode / USC / ORM · PRM / ToT
算力换准确率	纵向投入（深度），把一条链写长	横向投入（广度），做法是写很多条

4.2 Self-Consistency 自一致性

Self-Consistency: select the response with the most consistent final answer.

核心思想：对同一个问题，采样多条不同的 CoT 推理路径，统计最终答案的众数（majority vote，出现次数最多的最终答案）作为输出。

Note: the selection is only based on the final answer. The reasoning paths do not need to be the same across different sampled responses.

重点：只看最终答案是否相同，中间推理过程可以完全不一样。只要多条不同思考路径收敛到同一个答案，就采信这个答案。

数学理解：marginalize out reasoning paths（把所有推理路径边缘化，只聚合最终结果）

实验结论

一致性 ≈ 准确度，一致性越高（越多采样得到同一个答案），答案正确概率越高。

1. 在数学数据集（AddSub, MultiArith, GSM8K 等）上，所有模型（UL2, LaMDA, PaLM, GPT, Code-davinci）加上 Self-Consistency 之后，准确率全部明显上涨。
2. 性能随采样路径数量增加而提升：采样越多推理路径，效果越好；一直上升，逐渐趋于平缓。
3. Sample-and-Rank：选 log 概率最高的那条回答；Self-Consistency 远好于单纯基于概率排序。

采样多样性

1. 温度（temperature）/ 核采样（nucleus sampling），能产出更多样推理路径 → SC 效果大幅提升；
2. Self-Consistency（基于采样的自洽性）随样本数持续提升，保持提示词、改采样，差异来自模型随机性。
3. Ensemble（多 prompt 集成）的做法是改提示词、保持贪心解码，所以它们之间差异来自问法不同。同样的样本量，Self-Consistency 改采样比改提示词涨得多。所以改写提示词只是换个入口，模型内部走的还是同一条高概率路径；而高温采样是真的在探索不同的推理路径。
4. Beam search：保留解码过程概率最高的 top-k 路径，多样性差，SC 提升有限。束越宽，取最好的那条，分数反而越低，因为束搜索保留的是概率最高的一批路径，而它们彼此高度相似。所以对这些路径投票，等于对同一批几乎一样的答案投票。

4.2.1 Example：AlphaCode 按执行结果聚类

任务背景：Competitive programming（Codeforces）

题目：长文本描述 + 少量样例输入输出；代码不仅要过样例，还必须通过隐藏测试用例。

本质是自洽性思想跨域迁移，生成多个独立样本 → 按行为一致性分组 → 选取多数收敛的结果。

底层原理是多个独立采样样本收敛到相同行为，通常代表这个解是正确的。

前提是提交预算，可以生成海量候选程序（Sample budget 采样预算），但最终能提交到评测平台的候选数量有硬性上限（submission budget 提交预算）。采样可以很多，但提交名额很少。

AlphaCode 一致性聚类流程：

1. 基础过滤：筛掉无法通过题目给出样例测试用例的代码。

- 缺陷：剩下的代码依然有可能在隐藏测试集（held-out test cases）上出错。

2. 方案：训练模型只生成新的测试输入，不生成输出。

- 原因：没有真值标签（ ground truth ），没法生成标准答案；生成输入远比生成正确判题输出简单。

3. 执行与聚类： 将所有采样出来的候选代码，在全部生成的测试输入上运行；把在所有测试输入上输出完全相同的代码划分到同一个簇。从规模最大的 10 个簇中，每个簇挑选 1 条代码提交（竞赛提交次数有限，多选不同簇，相当于押多个不同解法，提高命中正确代码的概率）。

   核心假设：all programs in the same cluster are semantically equivalent if the generated inputs are diverse and of high quality 如果生成的测试输入足够多样、质量足够高，同一个簇里的代码语义等价。

> 对比曲线：filtering（只过滤掉过不了样例的代码）有提升；filtering + clustering 额外再涨性能；但依然和 oracle upper bound 有差距

结论：

1. 单纯疯狂增加采样预算（生成几百万代码）没用。价值来自在提交预算限制下的筛选策略。
2. 提交预算约束，迫使模型做多样性选择：优先挑选行为不同的方案，而不是单纯挑 “模型打分最高” 的方案。
3. 整套方法强依赖测试输入质量：输入多样性不足，会导致语义完全不同的代码碰巧输出一样，被错误分到同一簇。输入生成模型的好坏，直接决定筛选可靠性。
4. Clustering provides additional performance gain over filtering only. Still a gap from the oracle selection, the selection is far from solved. 聚类相比单纯过滤还有额外收益，但距离完美的神谕选择仍存在差距。选择问题远没有被解决。

> 性能提升来自算法流程设计，不是靠放大模型规模。小模型也能靠这套搜索选择方法拿到显著提升。

4.2.2 Self-Consistency 的局限

Limitation of self-consistency decoding: requires an answer extraction process

核心限制：原版自洽解码，必须有答案抽取（answer extraction）的步骤。在自由开放式生成场景中（文本摘要、开放式问答、故事写作） 两段合格的摘要，表达意思可以相同，但文字句子完全不一样，没办法直接比对。

底层逻辑：投票的前提，是能把每一条模型输出，压缩成一个可以相互比对的标签。 所以前提就是要能够提取、比对答案。

> 本质理解： 自洽流水线隐含要求：存在映射函数，把一段长文本映射成等价类标签

数学任务映射：提取末尾答案数字。

AlphaCode 代码任务映射：程序在整套测试输入上的输出向量 自由文本生成缺少这个映射。

> 这个限制，引出Universal Self-Consistency（USC）：直接让大模型自己完成一致性挑选，绕开答案抽取。

4.3 Universal Self-Consistency（通用自一致性）

4.3.1 流程

Question → LLM → Responses（多条）→ 拼进 Universal Self Consistency prompt → 再交给 LLM → Selected response

1. LLM 生成多条回答（候选 responses）
2. 把所有候选全部塞回 prompt 交给 LLM，指令：基于多数共识，选出最一致的回答。
3. LLM 自己充当选择器，选出最终输出

4.3.2 特点

Universal Self-Consistency (USC) improves performance on open-ended generation (summarization, QA), where the original self-consistency is inapplicable.

USC matches self-consistency performance in math reasoning and coding.

USC does not require answer extraction and code execution.

USC performance is bounded by the long-context capability.

- 开放式生成上有提升（摘要、问答），原本自洽性根本用不了的地方，现在能用了。
- 数学与代码上持平。
- 不需要答案抽取、不需要代码执行 ，工程上更省事，部署成本低。原方法（写程序抽答案）的优点是确定、便宜、可验证；新方法（让模型选）的优点是通用，代价是吃上下文且不确定。
- 受长上下文能力限制（候选很多时 prompt 会很长）。

4.4 LLM 排序器 / 验证器

核心思想：不再只靠 “多数投票一致性”，单独训练一个奖励模型（verifier/reward model）给候选解打分，比单纯一致性选择更强。

4.4.1 三步流水线

1. Train generator：训练生成器，输入问题Q_i，生成解S_i;
2. Generate and label：对每个问题生成 100 个候选解，打标签；

3. Train Verifier：训练验证器，输入问题 + 候选解，输出该解好坏的奖励分数 Y_i^j。

4.4.2 两种 LLM 验证器 / 奖励模型

ORM（结果监督）：在完整解的层面做验证，只看最终答案对不对。

PRM（过程监督）：对每一步推理 step 单独验证，监控推理中间步骤对错，比 ORM 更强。

扩展性不同：PRM 随样本数增加涨得更好，而 ORM 更早走平。

过程监督能抓住理由错但答案蒙对的解进行去噪：the step level scoring just removes the noise. It catches a solution that got lucky with the right answer for a broken reason

基于 LLM 的验证器，性能超过基于一致性的选择方法

1. PRM 随着样本量增加，扩展性更好（scales better with more samples）
2. 性能高度依赖验证器本身质量 highly dependent on the verifier quality
3. 同一个验证器不一定能跨任务泛化 might not generalize across tasks

4.5 Tree of Thoughts

问题：到目前为止的所有方法都是完整回答全部生成完毕之后，再做选择，没有充分利用单步的打分。

引出 Tree of Thoughts（ToT，思维树）：LLM + tree search，不再一次性生成完整答案，在局部部分解（partial solutions）就评估、搜索，优先探索更有前景的分支。

4 种范式：

1. IO Prompting 输入直接输出；
2. CoT Prompting 思维链；
3. CoT-SC 思维链 + 自洽多数投票；
4. ToT Tree of Thoughts 树搜索，在每一步分支做评估剪枝。

核心区别：CoT/SC 是整条推理路径一次性生成；ToT 是分步展开、分步评估，像搜索树。

前提：需要一个好的 LLM，加上好的提示设计，来做自评估。

这是整个搜索过程的地基：剪枝的依据就是模型自己的判断。如果自评估不准，被剪掉的可能是正确答案，最后搜到的是错的那一支。

Example

1. **Thought generation 思路生成**：提示大模型提出下一步可行的思考分支；
2. **Thought evaluation 思路评估**：提示大模型评估当前这个中间状态有没有希望（promising /impossible）。

评估分支好坏：LLM 多次投票，选多数结果作为选择，在候选中间状态里挑最优分支。生成多个 plans → 展开成不同段落方案 → LLM 多次投票，选出最优 state。

> b 代表每一步保留最好的 b 个分支（breadth，搜索宽度），b 越大效果越好。
>
> 右图：success rate 随访问节点数量变化，ToT（绿色虚线）显著高于 CoT、IO。

1. 采用 BFS 广度优先搜索的 ToT，在 token 预算限制下扩展性远好于普通 Prompt、CoT；

2. 可以结合更高级搜索算法，如 MCTS 蒙特卡洛树搜索；
3. ToT 依赖：能力足够的 LLM + 用于自我评估的提示词设计。

总结

1. 推理时计算量可以通过在解空间采样多条分支来扩容（inference-time compute，推理时算力）；
2. Consistency-based selection 基于一致性的选择：简单、有效、通用的核心原则

   Self-Consistency：边缘化多条推理路径，基于最终答案投票选择；

   代码生成：基于执行一致性做重排序（AlphaCode 聚类）
3. 当 LLM 的自我评估靠谱时，训练验证器（ORM / PRM），或依赖模型自评估做树搜索（ToT）。

顺序本身就是成本顺序：越往后越贵、也越强。

Lecture 4 Reasoning in LLMs (2)

Part 1 两种思维模式

1.1 快思考与慢思考

对比项	System 1 Thinking	System 2 Thinking
中文名	系统 1 快思考	系统 2 慢思考
课件给的特征词	Instinctive / Quick / Automatic Little/No Effort / Emotional / Un Conscious	Slower / Rational / Complex Decisions Conscious / More Logical / Effortful
例子	2 + 2	17 x 24
读法	不用动脑、答案自己冒出来	要动脑、要一步一步算

两个例子的差别不是难度，是是否需要中间步骤。前者靠记忆直接取出答案（factual recall），后者必须经过推理，需要多步中间生成（multi-step generation）。

1.1.1 System 1

早期 LLM 只有系统 1

要素	课件原文	读法
结论	Prompting and Fine-Tuning can (and only) yield a (better) System 1 !	提示与微调能做的只是把系统 1 练得更好，做不出系统 2
归因	as they were developed mainly based on the so-called “Deep Learning Hypothesis”	深度学习假说

Deep Learning Hypothesis

神经元最快每秒 100 次；人做感知只需 0.1 秒，最多只跑了 10 层；所以一个有 10 层的神经网络就够了。系统 1 本身极强：语言流畅度、格式遵循、常识补全、模式匹配，全在系统 1 的范围内。

Human perception is fast 限定的是感知（perception），不是推理。也就是说这个假说不是解释慢思考。

如果模型只按这个假说设计，那它天然就是系统 1，一次前向传播，10 层左右，出来就交答案。

1.1.2 System 2

Example：下棋

系统	课件原文	场景
System 1	generates the proposals (used in speed chess)	生成候选着法，用于快棋
System 2	Need to “Think” more / Reason, e.g. keep track of the tree (used in competitions)	在脑子里维护整棵搜索树，用于正式比赛

两个系统是流水线：先由系统 1 靠直觉砍掉大部分着法（否则搜索空间爆炸），再由系统 2 在剩下的候选上算到底。

1.2 Reasoning Definition

Reasoning is decomposing a potentially complex task into simpler subtasks the LM can solve more easily by itself or using tools. There exist various ways to decompose into subtasks, such as recursion or iteration. In that sense, reasoning is akin to planning

1. 推理的动作是分解，不是变聪明（所以能拆 ≠ 拆对了）
2. 子任务要能被模型自己、或借助工具（Agent 接口）解决。
3. 分解的两种组织方式是递归与迭代。递归 = 子任务还是同类问题；迭代 = 一轮一轮改。
4. 推理近似于规划（即Planning）。

For LLMs, “Reasoning” is commonly defined as the process of answering questions that require complex, multi-step generation with intermediate steps. As opposed to answering single-step Factual-recall type of questions

区分的不是答案，是过程的存在与否。能不能写出中间步骤，才是推理的判据。把推理从一个能力概念变成了一个输出形态概念。后面的所有技术（提示、搜索、RL）之所以能起作用，正是因为它们都在改变输出形态，让模型多写中间步骤，而不是让模型变聪明。

1.3 Example

层次	课件标注	在这一页对应什么
第一层	Intermediate reasoning chain that is not explicitly shown to the user	模型内部想了什么，用户看不到（Reasoned about last digit of powers for a couple of seconds）
第二层	Intermediate reasoning steps that are shown as part of the answer	写进答案里的推导过程，用户能看到，能检查、能训练、能投票
结论	The main answer	最终答案 3，只是整段输出里最后一行

Part 2 LLM 推理能力争议

2.1 推理动机 Three Motivation of Reasoning

动机	原文	与人对照
提升少样本学习能力 Few-shot Learning	“Humans (even a small child) can learn from relatively few examples because Human can “Reason.””	from Dr. Denny Zhou of Google DeepMind
提升分布外鲁棒性 Out-of-distribution Robustness	Humans possess the ability to generalize beyond familiar scenarios.	人会迁移到没见过的场景
提供可解释性 Explainability	Humans are capable of providing explanations for their decisions or predictions, whereas machines (especially deep neural networks) are often considered “black box” within limited explainability.	人能解释自己为什么这么判断

三个动机的权重不一样。前两个是性能诉求（更少数据、更强泛化）；最后是信任诉求。

2.2 推理能力的争议

2.2.1 LLM Can Reasoning

证据	原文
1.成绩证据（分数高）：在推理任务上高分	High performance on various tasks requiring reasoning
2.机制证据（存在有效手段）：能用思维链逐步推理	The ability to reason step-by-step with chain-of-thought prompting
3.行为证据（表现出类人偏差）：会表现出类人的内容效应	The reflection of human-like content effects reasoning

Tips：

CoT：被拿来当它有一点推理能力的证据，但 CoT 本身就是提示技巧。

Content Effects：人在推理时会被题目内容（而非结构）影响，比如同一道逻辑题换成现实场景就更容易做错。模型也表现出类似偏差，这被当作像人一样推理的旁证。但另一个角度是会被人无关内容带偏，也许说明的是它只是在做模式匹配，不是真的在推理。

2.2.2 LLM Can not Reasoning

But there are also observations that suggest LLMs may not be capable of reasoning:

观察	原文
1.复杂推理仍然失败	LLMs still struggle with tasks that require complex reasoning
2.对某些术语的出现频率敏感 对训练语料里出现频率不同的数字，准确率会系统性变化，这更像查表而不是计算	The performance of LLMs has been found to be sensitive to the frequency of certain terms
3.关联不上自己记住的信息 模型记住了某条信息，却在需要它的时候用不上，记忆与调用是断开的	Language models have been found to struggle with associating relevant information that they have memorized

观察	原文
4.未解的核心疑问 写长上下文只是往上下文里塞了更多相关文本，让下一个 token 更容易猜对	“It is not yet fully understood whether the LM is really reasoning, or simply producing a larger context that increases the likelihood of correctly predicting the missing tokens.”
5.可用测试集自查	Misguided Attention: a suite of testing prompts 公开的测试提示词集
6.有些错误形态不像懂推理的人会犯的	Some mistakes made by LLMs seems to show they do not actually know reasoning

Example：错误答案范例

题面	模型的答案	正确答案
how many r’s are there in the word "strawberry"?	There are two "r"s in the word "strawberry."	3 个 r（strawbe-r-r-y 中有三个 r）
9.11 and 9.9, which one is larger?	9.11 is larger than 9.9.	9.9 更大（11 > 9 的直觉被误用）

错误的归因：

1. 分词：tokenizer 可能被切成错误的token
2. 训练语料：日期类型的统计强度可能盖过了小数比较。

三条具体的缺陷

硬伤	课件原文
易被无关上下文带偏	LLMs can be easily distracted by Irrelevant Context/ Content
前提顺序会影响结果（但不应该）	Premise Order matters in LLM “Reasoning”, but they **should not be （前提条件的顺序在逻辑上不影响结论，所以模型对顺序敏感本身就是反逻辑的）！**
没有 Oracle 就无法可靠自我纠错	LLMs cannot self-correct reliably without an oracle: while allowing LLMs to review self-generated responses can sometimes help to correct inaccurate answers, **such reviews may also change correct answers into incorrect ones（双向影响：自我检查既可能把错的改对，也可能把对的改错）**.

2.2.2.1 无关信息干扰 Distracted by Irrelevant Context

Example 1：加入无关上下文的干扰信息。模型把不该进算式的数字拉进了算式。所以这不是计算能力问题，是取舍问题。

位置	课件原文
输入	Lucy has $65 in the bank. She made a $15 deposit and then followed by a $4 withdrawal. Maria’s monthly rent is $10（无关信息）. What is Lucy’s bank balance?
模型输出	Lucy’s bank balance is $65 + $15 - $4 - $10（被加入） = $66. The answer is $66.

Example 2：加入不相干但无害的信息，仅起到稀释作用。说明模型的取舍依赖提示。

把原题拆散，在中间反复插入 The sky is blue. 与 The grass is green.

加入一点提示：Ignore irrelevant context.

模型输出变成：Lucy has $65 in the bank. After making a $15 deposit and then a $4 withdrawal, her bank balance is $76. Maria’s monthly rent is not relevant to this question, so it can be ignored. The answer is $76. 答对了，而且模型还主动说明了为什么忽略那一句。

2.2.2.2 前提顺序改变 Premise Order matters

改动：He loses 10 bills while getting home. 这一句从第 2 句挪到了倒数第 2 句。逻辑上不该有影响：丢 10 张钞票这件事发生在哪个时点，都会让张数减少 10。但挪动之后，减 10 被放到了最后一步，而前面多算了一步 triples，于是基数完全不同。

模型给出的完整推导：

两次推导都把丢 10 张放在不同位置，而两种放法在题目文字里都说得通。原顺序下丢钞票发生在付账单之前，基数变成 40；重排后发生在最后，基数变成 300。

Tips：这种错误有两种解读。读法一是模型对语序敏感，并不是真推理；读法二是这个题面本身歧义，两个答案都对。

Part 3 三条教它推理的路径

3.1 三条路径总览

Make（让它做）→ Lead（引导它做）→ Force（强制它做）

类型	课件原文	改变什么
Type 1	Using some Prompting techniques for the LLM, e.g. Chain-of-Thought (CoT) prompting	让模型一次写得更长（提示技巧）
Type 2	Make LLM perform Search and Selection from Multiple Candidates => Increase the WIDTH to explore solution space	加宽：生成多条候选再挑
Type 3	Make LLM perform Iterative self-improvement => Increase the DEPTH to reach the final solution	加深：在同一条回答上反复改

3.2 加深的动机 Iterative self-improvement

。论证很紧：先说「」，再说「」，最后指出缺的是什么。

问题：

1. Even humans often make (sometimes trivial) mistakes at first thought 人也会错
2. Although sampling multiple solutions (i.e. increase the width of exploration space) can **reduce mistakes **by considering more than one single prediction. 加宽能减少错误但不够

原因：**NO feedback loop to correct the mistakes after a complete solution is generated. **没有反馈回路。加宽只是并行多生成几份，每份都是一次成型，写完就定稿，中途没有修正机会。

解决：Under the Inference-time self-improvement approach, LLM iteratively improves its own response to the given task. 在回答的中间过程上反复迭代。

3.2.1 迭代自省 Self-Refinement

Basic idea: prompt an LM to critique/revise its own outputs iteratively.

元素	作用
Input	原始输入，喂给模型
Model M	在 Feedback 和 Refine 两步 Self-Refine 同一个模型既做生成也做评审
Feedback	让 M 先给自己的输出写评语
Refine	让 M 拿着评语改上一版输出
回环	改完再评、评完再改，形成循环（全部反馈来自模型自己）

Self-Refine vs Reflexion

Self-Refine：同一模型自评自改，强调不需要额外训练、不需要外部信号。

Reflextion：把反馈写成文字存起来（verbal reinforcement learning），强调跨回合（trajectory）积累经验，主要在 Agent 任务上。

共同点：都把反馈表达成自然语言而不是梯度；都是迭代的。

3.2.2 Reflection：三个任务、五个阶段

(a) Task → (b) Trajectory → (c) Evaluation → (d) Reflection → (e) Next Trajectory。

阶段	任务
(a) Task	任务描述
(b) Trajectory	第一轮的动作与观察（Action: / Obs:）
(c) Evaluation	这一轮好不好的判据，内部启发式还是外部环境奖励
(d) Reflection	把失败原因写成一段自然语言（课件的配色规则见下）
(e) Next Trajectory	带着反思重来一遍的动作与观察

任务	判据来源（c）	第一轮的结果	反思后
1. Decision making	内部启发式: Rule/LM Heuristic: Hallucination.	动作全都 Nothing happens.（选错了抽屉/灶眼）	换成 stoveburner 2，成功把锅放进台面
2. Programming	外部但自造: Self-generated unit tests fail	返回 'No'，判据错了（只比总数、不看顺序）	改为检查嵌套结构 check(S1) or check(S2)
3. Reasoning	外部环境奖励: Environment Binary Reward: 0	答 novelist, screenwriter（假设两人职业集合相同）	改为 novelist（只取真正的交集）

三列的判据强度是递减的：列 3 有真正的环境奖励（0/1），列 2 是自造单测，列 1 只是语言模型的启发式判断（Hallucination）。所以，判据越硬，反思越有用。

LLM generates feedback on its output. Use external evaluation (oracle) when available.

LLM self-refines its output given both internal feedback and external evaluation.

3.3 边界条件 Token Budget

3.3.1 Good External Evaluation (Oracle 预言机)

Self-reflection and Self-refinement work ONLY WHEN Good External Evaluation (Oracle) is available

图表特征：

1. 全部上涨，自省有效。
2. ALFWorld 是平行上升后拉开差距（说明反思的增益是逐轮累积的）；HotPotQA 是一开始就分叉（ReAct + Reflexion 从 0.30 一路到 0.55，而两条不带反思的曲线几乎是水平线）。
3. ReAct + Reflexion 比 CoT + Reflexion 高（0.55 vs 0.40）：说明反思作用在被执行过的动作轨迹上，比作用在纯文本推理上更有效，因为动作产生了环境反馈。

3.3.2 Without Oracle 自我纠错掉分

图表特征：

1. 六个格子全部走平或下跌，且基本是单调下降。
2. GPT-4 在 HotpotQA 上是 49.0 → 49.0 → 43.0：第一轮没动，第二轮掉 6 个点。说明改坏不是立刻发生的，多改一轮多一分风险。
3. GPT-3.5 在 CommonSenseQA：75.8 → 38.1，几乎砍半。常识类问题没有统一判据，模型的自评最容易翻车。
4. 第 1 轮要 3 次调用、第 2 轮要 5 次，掉分的同时还多花了近 5 倍算力。

Self-correction without oracle feedback hurts reasoning performance, LLMs can wrongly judge the correctness of its predictions。

3.4 利用预算 Utilize the Token Budget

Giving AI additional test-time computing can greatly improve performance. However, previous demonstrations are mainly limited to specific tasks.

推理时算力可以提升表现，但在特定任务时有限。

如何更好地利用算力?

手段	中文	类型	判对错的依据
N answers … find the most common one（自洽性 / USC）	获得 N 个答案，然后找出其中最常见的那个答案（即“自我一致性”或“普遍自我一致性”）	Type 2 加宽	答案之间的一致性，不需要外部判据
Best-of-N with a Learned Verifier	使用“学习式验证器”来进行“N 人比”的竞赛	Type 2 加宽 + 验证器	训练出来的验证器（因为模型自评不可靠）
revise and correct it’s own responses iteratively	利用大语言模型来反复修改和纠正自己的回答	Type 3 加深	模型自己，所以只在有 Oracle 时可靠

All these techniques induce the use of more token budget

手段不同，代价的性质相同。

3.5 验证器搜索与训练

3.5.1 三种验证器搜索：Best-of-N / Beam / Lookahead

> ⊞ = Apply Verifier（虚线方框：在这个位置调用验证器）
>
> ● = Full Solution（实心圆：完整解答）
>
> ○ = Intermediate solution step（空心圆：中间步骤）
>
> ● = Selected by verifier（绿色：被验证器选中，不等于正确答案，涨分并不是满分）
>
> ● = Rejected by verifier（红色：被验证器否决）

策略	标注	结构
Best-of-N	Generate N full solutions with the verifier → Select the best final answer using the verifier 使用验证器生成N个完整解 → 使用验证器选择最佳最终答案	完全并行：一次生成 N 份完整答案，全部交给验证器打分，取最高。只在终点判。
Beam Search	Select the top-N samples using a PRM → Select the best final answer using the verifier 使用PRM选择前N个样本 → 使用验证器选择最佳最终答案	分层并行：每步保留 top-N 条中间步骤，逐层展开。每一步都判。
Lookahead Search	Beam search, but at each step select the top-N candidates to roll out based on the PRM value at the end of the current step 采用波束搜索法，但在每个步骤中根据当前步骤结束时的PRM值，选择前N个候选解进行进一步处理	带回传的串行：先试跑（Rollout “Lookahead” step），把分数 Propagate PRM values back to previous step，再 Continue Search from the top-N options。先往前多看一步再回来判。

3.5.2 训练过程验证器 Training a Process Based Verifier

For search, the verifier should score each step in a solution. Prior work used human labels of per-step correctness for supervision. This work instead follows the approach to learn value function.

在搜索过程中，验证者需要对解决方案中的每一步进行评分：之前的研究使用了人类对每步正确性的标注来进行监督。而本项研究则采用一个价值函数。

要素	说明
Problem	首一四次多项式，已知三个根，求 p(0) + p(4)
(a) Outcome Annotation	$y_S = 0 $只看最终答案，整条链判 0
(b) Process Annotation	$y_{s1}^{SE} = 2/3$；$y_{s1}^{HE} = 1$，对 s1 这一步单独给分
图例	s1 是第 1 步； $s_{1,1} $是第 1 个完整解的第 1 步

SE 与 HE 是两个不同的估计量： y^{SE}是统计估计，3 次试跑 2 次成功； y^{HE}是硬估计，取多数结果。

3.3.2 说明模型自评不可靠；3.5.2说明不要用模型自评，用一个单独训练的 value function。

3.5.3 搜索方法对比

All verifier search methods outperform the majority baseline. 所有验证搜索方法都优于大多数基线方法。

Majority 是基线，即 self-consistency（多条答案取多数票），准确率最低，证明加了验证器之后，同样的预算能拿到更高分数。但验证器也不能解出所有题。

Beam search performs best at small budgets. At large budgets, Best-of-N performs similarly to beam search. 在预算有限的情况下，波束搜索效果最好；在预算充足的情况下，Best-of-N方法与波束搜索效果相当。

在图片中$M := sqrt(N) $与 $M := 4 $的区别是 beam 的宽度策略：前者随总预算 N 动态取 √N，后者固定为 4。图上两种策略的曲线在中段交叉，说明没有一种宽度策略全程占优。

Lookahead generally underperforms due to the high cost of lookahead rollouts. 由于“预看”技术的实施成本较高，其性能通常不如其他方法。

Lookahead 把预算花在试跑上，而试跑本身不产出最终答案。同样的 token 预算下，纯并行的 Best-of-N 更划算。

Part 4 Post-training Techniques & DeepSeek-R1

4.1 后训练：四条路径

4.1.1 从推理时算力转向训练时算力

背景：没有 Oracle 的自省会掉分，而一个有 Oracle 的场景在真实任务里很少见。如果审查能力不能靠模型自省得到，那就只能把它训进去。本讲的关注点从推理时怎么多花算力切换到训练时怎么改权重。

4.1.2 Reasoning is one of the Specialized Skills

四阶段图说明推理模型在 LLM 生态里的位置。专精 ≠ 替代。

阶段	标注	产物
STAGE 1: BUILDING	构建阶段	Building an LLM
STAGE 2: PRE-TRAINING	预训练阶段	Foundation model（基座模型）
STAGE 3: FINE-TUNING (“POST-TRAINING”)	微调 / 通用后训练	Classifier（分类器）、Personal assistant（个人助理）
STAGE 4: MORE SPECIALIZATION	进一步专精	Web search LLMs / RAGs / Topic chatbots / Multimodal LLMs / Code assistants / Reasoning models / Agents / Distilled & cheap models

Reasoning是八大 Specilized skill 的方向之一，这背后存在不可避免的权衡（inevitable trade-offs）。得到的是省去全部标注数据与SFT阶段；代价是模型输出可读性下降、语言一致性变差。

4.1.3 Four Ways & Two Compute

序号	英文	中文	归类	代表
1	Inference-time compute scaling	推理阶段算力扩展	test-time compute	上一讲全部内容

序号	英文	中文	归类	代表
2	Pure RL (e.g., DeepSeek R1 Zero)	纯强化学习（如 DeepSeek R1 Zero）	train-time compute	DeepSeek-R1-Zero（纯 RL，无 SFT）
3	RL + SFT (e.g., DeepSeek R1)	强化学习 + 监督微调（如 DeepSeek R1）	train-time compute	DeepSeek-R1（先 SFT 再 RL）
4	Pure SFT (e.g. s1, distilled DeepSeek R1)	纯监督微调（如 s1、蒸馏版 DeepSeek R1）	train-time compute	s1、蒸馏出来的小模型

第 1 条不改权重（推理时算力），第 2 - 4 条改权重（训练时算力）。训练算力与测试算力可以互相替换（$log10(test) = -1.2 \cdot log10(train) + ...$）。所以这四条路本质上是在同一条替换曲线上选点。

4.2 推理阶段算力扩展

Realizing LLM Reasoning via Inference-Time Scaling

4.2.1 Core Idea

Core Idea: Trigger the LLM to generate one or more LONG Chain-of-Thought (CoT) Path of Exploration via one or more of the following: 触发 LLM 生成一条或多条**长思维链（CoT）探索路径**，可通过以下一种或多种方式实现：

四个手段	属于哪类	动的是什么
Few-Shot CoT Prompting	提示（Type 1）	给几道带过程的例题
Instruction Prompting	提示（Type 1）	给一句指令，如一步一步想
Instruction Tuning	训练（SFT）	改权重：用带指令的数据微调
Reinforcement Learning	训练（RL）	改权重：用奖励信号优化

Few-shot CoT 的机制：用 token 预算换正确率的直接体现。不是教模型算术，是教它先写过程再给答案的格式。

4.2.2 RL Optimized COT

RL optimized Chain of Thought can be a highly effective way to scaling test-time compute.

RL 强化学习 optimized Chain of Thought 与写长思维链的区别：前者是用一个奖励信号把怎么想训进权重，后者只是提示模型这一次写长一点。前者是训练阶段算力，后者是推理阶段算力。

DeepSeek-R1: **Incentivizing** Reasoning Capability in LLMs via Reinforcement Learning

Tips：是 Incentivizing，不是 Teaching 或 Training。这与 不给任何监督数据 是同一件事的两种表述，能力不是教会的，是被奖励信号激励出来的。

4.3 SFT

Augments input-output examples with instructions, which enables instruction-tuned models to generalize more easily to new tasks.

通过指令扩充输入输出样例，让经过指令微调模型能够更容易泛化到新任务：在训练样本上额外配上任务指令（指令 + 输入→输出），模型学习 “按照指令完成任务”，而不是只记住固定样例，因此泛化到没见过的新任务。

例 1 的答案是开放文本（翻译），例 2 的答案是封闭标签（情感三选一）。这两个案例说明 instruction-tuned models 的通用性：同一套数据格式能覆盖两类任务。

限制：要做推理的 SFT，期望输出里就得包含完整的思考过程。但它只能模仿数据里出现过的过程。要得到数据里没有的新推理路径，就需要的强化学习 RL。

4.4 R1 Family and DeepSeek Training Paradigm

4.4.1 R1 Family (DeepSeek-R1-Zero vs DeepSeek-R1)

对比项	DeepSeek-R1-Zero	DeepSeek-R1
训练方式	large-scale reinforcement learning (RL)，without supervised fine-tuning (SFT) as a preliminary step	incorporating a multi-stage training and cold-start data before RL
有先做 SFT 吗	没有（Zero 的含义） 零监督数据（without supervised fine-tuning）	有（**cold-start data** 就是 SFT 数据） cold-start data 即是一小批人工整理、格式友好的推理数据，用来在 RL 之前把输出格式改正。
问题	poor readability, and language mixing （可读性差，思维链写得读者看不懂；语言混杂，中英文混在一段里，都不影响正确率，只影响可用性）	针对上面两个问题而设计
类型	Pure RL	RL + SFT
意义	不给任何人工标注，纯靠 RL 也能长出推理能力	R1 是在DeepSeek-R1-Zero上，解决可用性问题

4.4.2 DeepSeek Training Paradigm 训练范式全景图

区域	作用
起点	整个家族的共同起点
冷启动主线	GRPO / Rule-based Reward (Accuracy, Formatting) + CoT Language Consistency Reward
虚线捷径	虚线绕过全部 SFT 与数据生产，纯 RL
数据生产	用 DS-V3 as judge 筛数据
三个数据集	Reasoning Data 来自上面的筛选用；Non-Reasoning Data 由 DeepSeek-V3 (671B/37B Activated) 经 CoT Prompting 产出
R1 支路	最终产出 DeepSeek-R1
蒸馏	标注 Distillation，底座为 Qwen2.5-Math-1.5B / Qwen2.5-Math-7B / Qwen2.5 14B / Qwen2.5 32B / Llama-3.3-70B-Instruct / Llama-3.1-8B

4.5 R1-Zero：纯 GRPO 训练

4.5.1 起点：MoE base

DeepSeek-V3 (DeepSeek-AI 24) is used as the starting checkpoint model.

从 V3 的权重出发，而不是从零训练或从头预训练。整个 R1 家族都在 V3 的骨架上做后训练。

DeepSeek-V3 is a Transformer and Mixture of Experts (MoE) based model.

主干是 Transformer，前馈层是 MoE 结构。

MoE 结构的作用：R1 系列能在合理成本下做大规模 RL，RL 要反复前向，激活参数量直接决定训练成本，若 671B 是总参数量，37B 是每次前向实际激活的参数量，比值约 18:1。而MoE 的作用就是节省计算量（不是存储量，总参数量在显存中）。

4.5.2 MoE：Mixture of Experts

MoE：将 Transformer 中原有的稠密前馈网络（FFN）层替换为稀疏 Switch FFN 层（浅蓝色）。该层对序列中的各个 token 独立运算，每个 token 只路由到一个专家。

例子：两个 token（下方x_1=“More”、x_2=“Parameters”）路由过程（实线），二者被分配至 4 个 FFN 专家；路由模块为每个 token 独立选择专家。说明 the router independently routes each token，路由是逐 token 做的，同一个句子里的不同词可以去不同专家。Switch FFN 层最终输出**被选中 FFN 的结果乘以路由门控值**（虚线），说明 the selected FFN multiplied by the router gate value，所以专家的输出要乘上它的权重再相加。

标准 Transformer 块（左边）Add + Normalize → Switching FFN Layer → Add + Normalize → Self-Attention。

MoE 展开（右边）：原本一个 Switching FFN Layer 的位置，展开成 FFN1 - FFN4 四个专家加一个 Router。
MoE 的组成

部分	原文	读法
稀疏层	Sparse MoE layers instead of dense feed-forward network (FFN) layers.	把稠密 FFN 换成多个专家 FFN，每个 token 只走其中少数几个
路由器	Routers determines which tokens are sent to which expert.	一个路由器（ Router）决定每个 token 派给哪个专家 但路由是按 token 相似度学出来的，不一定对应人类理解的领域

4.5.3 只用 GRPO：DeepSeek- R1-Zero

Develop reasoning capabilities without any supervised data, via reinforcement learning finetuning. The Group Relative Policy Optimization (GRPO) algorithm was utilized to fine-tune the V3 base model.

通过强化学习微调，在不使用任何监督数据的前提下，开发模型的推理能力。研究采用GRPO 强化学习算法（Group Relative Policy Optimization，组相对策略优化）对 V3 基座模型进行微调。从 DeepSeek-V3 Base 到DeepSeek-R1-Zero只经过一个模块（面向推理的强化学习 Reasoning Oriented RL），全程没有任何SFT阶段。

所以 R1-Zero的缺陷 poor readability, and language mixing（可读性差、语言混杂）。来源于奖励函数只约束答案正确性和输出格式，省掉了全部标注数据与 SFT 阶段，代价是可读性与语言一致性，不负责文本表达质量。

Part 5 强化学习与 DeepSeek-R1

Part 4 讲为什么要转向训练，并给出了 DeepSeek-R1 家族的训练流水线。这一部分讲流水线里最关键的算法：GRPO。讲完算法，再看它训出来的 R1-Zero 长什么样、代价是什么，最后看 R1 怎么把代价补上。

5.1 Reinforcement Learning

Element	动物行为学	在 RL 里的角色
State	一轮扔木棍（的现场状态）	当前的局面，策略据此决定动作

Element	动物行为学	在 RL 里的角色
Action	狗朝木棍跑过去	当下做出的选择
Reward	一块零食	打分：动作好不好的数值反馈
Policy	狗自己的行为习惯	从状态到动作的映射，就是要学的东西
Objective	尽可能多拿零食	长期找到策略，使奖励最大，而不是答对 让模型在没有标注数据的情况下自己发现推理行为

> State: a round of stick throwing.
>
> Action: the dog moving toward the stick.
>
> Reward: a treat.
>
> Policy defines how an agent behaves in a certain state. How to assign the reward.
>
> The objective is to find the policy that maximizes the reward.

5.1.1 Five Elements of RL

一个逐项映射表，把语言模型还原成一个标准 RL 问题，从而让通用 RL 算法（PPO、GRPO）可以直接搬过来用。

RL 概念	LLM 对应物	工程含义
Agent	the LLM itself 模型本体	智能体就是模型，没有额外的控制器
State	the initial input 初始输入	整段提示就是初始状态；注意它不含模型已生成的内容 在同一道题上采样多次，初始状态完全相同、只有动作序列不同
Action	next token prediction 预测下一个 token	动作空间 = 词表；一个回答是一串动作
Policy	**defines the behavior of the LLM, define the word distribution**	策略就是词分布；训练就是改这个分布 所以 RL 不需要另建一个策略网络，直接微调那 671B 权重。
Reward	a numerical value to identify the correctness of the chosen action	整段回答结束后给一个奖励，不是逐 token 的密集奖励 奖励稀疏是 R1-Zero 需要自己的长思维链的根本原因：没有中间引导，模型只能靠整段结果的好坏去反推哪一步做对了。

5.2 GRPO

5.2.1 GRPO and PPO Algorithms

GRPO algorithm is based on the Proximal Policy Optimization (PPO) algorithm.

GRPO算法基于近端策略优化（PPO）算法

The Reward model is a trained model that assigns rewards.

奖励模型：是一个经过训练、用于分配奖励的模型。

**The value model **estimates the long-term expected future reward from a given state.

价值模型：根据给定状态，预估长期的期望未来奖励。虽然GRPO理论上源自PPO，但**GRPO训练时不再使用价值模型**，这是它和原版PPO最核心的差异。

PPO：q → Policy Model → o → Reference/Reward Model → r → v → A，GAE 把 r 与 v 合并；

GRPO：$r_1⋯r_G *直接进GroupComputation出*A_1⋯A_G$，整条链上没有 v。

模块	中文	作用	状态
Policy Model	策略模型	要训练的主体：给定 q 产生输出 o	两法皆有 Trained Model
Reference Model	参考模型	冻结的旧策略，用来算 KL 散度、防止跑偏	两法皆有 Frozen Model
Reward Model	奖励模型	训练出来的模型，给输出打分 r	两法皆有 Frozen Model
Value Model	价值模型	估计从当前状态出发，未来长期期望回报 v	只有 PPO Trained Model
GAE	广义优势估计	用 r 与 v 一起算优势 A	只有 PPO
Group Computation	组内计算	只靠 $r_1⋯r_G $ 直接算优势 A_i	只有 GRPO

Tips：PPO 价值模型。PPO 算优势需要知道这一步比平均水平好多少，而那个平均水平就是 v。在传统 RL 里，v 由价值网络估出来。但 LLM 场景在每个状态评估多个动作，让这件事变得特别贵。

5.2.2 Three Changes

GRPO generates multiple responses at once and generates multiple rewards. Group computation decides the advantage of the response based on other responses in the group. The value model is removed since it’ s expensive to evaluate multiple actions every state.

实现	改动	换来什么
一次生成多个回答	无（新增采样），得到 o_1 ⋯ o_G	有了一组可以互相比较
用组内比较定优势	把 GAE 换成 Group Computation	以平均分为基线，高于评价就是好动作，不需要价值模型就能算出优势
删掉价值模型	删掉 Value Model 与 v	显著降低训练资源占用，因为 LLM 的每个状态都要评估整个词表的动作

5.2.3 Objective Function 目标函数与优势函数

公式里的符号	读法	要点
$q $~ $ P(Q)$	从问题分布里采一个问题	For every question (q)
$o_i ~ $\pi_{\theta_{old}}$(O	q)$	用旧策略在同一问题上采 G 个回答
r_i < br>	第 i 个回答拿到的奖励	来自 Reward Model
A_i	第 i 个回答的优势，同一道题必定有一部分A_i > 0、一部分$A_i < 0 $这就是组相对（Group Relative）	Group Computation 的输出，组内标准化
$\pi_\theta / \pi_{\theta_{old}}$	新旧策略的概率比	Scale the reward by the ratio
$clip(1 - \varepsilon, 1 + \varepsilon)$	把比值截断在 $1 ± \varepsilon $内，clip 限制单步更新的幅度（conservative updates）	or clipped ratio (conservative updates)
$\beta \cdot D_{KL}(\pi_\theta \| \pi_{ref})$	与参考模型的 KL 散度惩罚，限制整体偏离参考模型的距离，防止把模型训崩。	Maintain divergence from the ref model

5.2.4 Reward Modeling in GRPO

$r_i $的来历：奖励不靠训练一个奖励模型，而是靠规则直接算出来。

rule-based reward system 规则奖励系统的两个类型：

奖励类型	原文	怎么判定	例子
准确率奖励 Accuracy rewards	Ensures correct responses	答案对不对，可程序化验证	math（比对最终答案）、LeetCode（跑测试用例）
格式奖励 Format rewards	enforces the model to put its thinking process between <think> and </think>tags	思考过程有没有写在指定标签里	<think> 包裹思考、标签外放最终答案

rule-based 规则奖励意味着不需要训练奖励模型、不需要人类标注，只要有一个能判对错的程序就行。但适用范围是数学与编程这种可以判对错的情况，开放写作、对话这类任务用这套奖励训不出来。

5.3 R1-Zero

5.3.1 Naturally more Thinking

训练集回复的长度在RL过程中自然（没有任何奖励，是为了拿准确率奖励而被迫发生的作用）变长了，花费了更多时间，解决了更多思考任务（但不一定都是对的）。

The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time.

阴影带（波动范围）随训练推进而变宽，说明不同题目之间的长度差异在拉大，模型学会了按难度分配思考量。

5.3.2 DeepSeek outperforms Open AI

Comparison of DeepSeek-R1-Zero and OpenAI o1 models on reasoning-related benchmarks.

self-consistency（多采样投票）：上一讲是在推理时手动投票，这一讲训练出来的模型天生适合投票。

pass@1 = 只给一次机会，看答对的概率；

cons@64 = 采 64 次后投票，看多数票是否正确；

赢的三处：AIME 2024 cons@64 拿到 86.7，比 o1-mini（80.0）与 o1-0912（83.3）都高；MATH-500 拿到 95.9，是全表最高（o1-0912 94.8）；GPQA Diamond 的 73.3 明显高于 o1-mini 的 60.0。

输的两处：LiveCodeBench 只有 50.0，低于 o1-mini 的 53.8（差了 3.8）；CodeForces 的 1444 距离 1820 / 1843 差得更远。

AIME 的分裂：pass@1 上 R1-Zero 的 71.0 低于 o1-0912 的 74.4；但 cons@64 上 86.7 反超 83.3。一次答对不如它，64 次投票就超过它。说明它的单次输出方差更大、探索范围更宽，而投票能把这份宽度兑现成准确率。

5.3.3 Aha Moment

The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

模型自己回头检查并推翻了自己的推导。

Wait, wait. Wait. Anthropomorphic tone（拟人化的语气）。模型不是在报告一个计算结果，而是在表达一种惊讶，但模型并不具有人的意识。

That’s an aha moment I can **flag** here：模型把自己发现的问题打了一个**标记**。在它自己的语境里，这是一条待办事项；对我们来说，这是一个突发的检查机制。这是无监督的，并且规则奖励里不会自我检查，所以自我纠错不是被奖励直接鼓励的，而是想得更久的自然产物。

5.4 R1-Zero 的两个代价和 R1

第 56 页是转折页：R1-Zero 的能力很惊人，但它不能直接交付。这一页给出两条毛病，以及 R1 的动机。

代价	中文	具体表现	从哪来
poor readability	可读性差	思考过程写法随意，人读不懂、也不好当成品输出	奖励只对答案对错与标签格式负责，没人管可读性
language mixing	语言混杂	同一段思考里多种语言混着写	训练语料与奖励都没约束语言一致性

两个代价都是奖励设计的直接后果。Accuracy rewards 管对错，Format rewards 管标签，readability 与 language consistency 都不在奖励里。因为trade-off 的一般规律，想让模型保住的品质，要么写进奖励，要么写进冷启动数据。R1 选择的是后者，用 cold-start data 把人喜欢的样子先灌进去，再用 RL 在此基础上得到推理能力。

R1 的产生：To make reasoning processes more readable（目的）we explore DeepSeek-R1, a method that utilizes RL with human-friendly cold-start data：cold-start data （手段）。

区别：R1-Zero 从零冷启动，R1 从人写的数据冷启动。

5.5 DeepSeek-R1 Steps

5.5.1 Step1 启动

起点回到同一个 V3 Base，但中间插了一段 SFT。

Start again from the V3 Base model.

- Perform supervised fine-tuning (SFT) for 2 epochs.
- Perform Reinforcement Learning (RL) to improve helpfulness and harmlessness.

步骤	原文	做了什么	与 R1-Zero 的差别
回到起点	Start again from the V3 Base model.	推倒重来，从 V3 Base 重新开始	相同 R1-Zero 也是从 V3 Base 出发
监督微调	Perform supervised fine-tuning (SFT) for 2 epochs.	用人写的数据（冷启动）做 SFT，跑 2 个 epoch	不同 修复了poor readability and language mixing R1-Zero without any supervised data
强化学习	Perform Reinforcement Learning (RL) to improve helpfulness and harmlessness.	再做 RL，目标扩到有用与无害（not correctness）	目标不同 实现了helpfulness and harmlessness R1-Zero 的 RL 只管推理正确性

R1-Zero：V3 Base → RL（纯 RL，无 SFT）。证明推理可以无监督涌现，代价是可读性与语言一致性。

R1：V3 Base → SFT (2 epochs, cold start) → RL。用一小段人写数据把人的偏好固定在起点上，再用 RL 长推理。

两者共享同一个基座、同一种算法（GRPO）、同一套奖励思路；唯一的不同是要不要先给人写的数据。

5.5.2 Step2 SFT 数据

那 2 个 epoch 用的 80 万条数据是怎么造出来的？两路并行、一冷一热。

左边这条分支，产出60万条推理数据：**模型自己生成推理答案，再筛选合格样本**。用的是拒绝采样（Rejection Sampling）：先生成一大堆候选回答，只留下检验合格的。判分有两套机制，一套是规则校验，另一套直接拿DeepSeek V3模型当裁判打分。

右边这条分支，产出20万条非推理数据，由两部分合并而来：一部分是现成的DeepSeek-V3监督微调数据集；另一部分是用DeepSeek-V3（671B，激活37B）做思维链提示（CoT Prompting）生成的数据。这20万条是用来保住模型的通用能力。如果80万训练数据全都是推理题，模型写作文、日常聊天、事实问答这类基础能力就会变差。加入20万非推理样本，相当于保留25%的基础能力盘。

细节：**CoT语言一致性奖励**。专门针对R1-Zero出现的语言混杂问题，直接新增一项奖励约束。模型身上的问题，如果没有对应的奖励目标，就只能靠奖励函数或者训练数据去修正。

冷启动长思维链数据（Cold Start Long CoT Data）只有几千条，而后面SFT训练用到80万条数据。也就是说：**人工只需要提供几千条种子数据，剩下绝大部分数据，全部靠模型自举自动生成**，这是这套流水线节省人工标注成本最核心的地方。

为什么训练的时候，推理数据和非推理数据要混合在一起？只拿推理数据训练模型，会造成通用能力下滑。推理只是八大专精能力中的一项，能力之间必然存在取舍权衡。资源全部倾斜到推理，别的能力就会下降。 所以80万训练样本采用60万推理能力样本 + 20万基础能力样本的组合。 这也区分了R1-Zero和R1：R1-Zero单纯用来证明模型推理潜力，R1才是能正常对话、落地使用的成品模型。

5.5.3 Step3 Distillation

普通蒸馏（权重蒸馏）：大模型直接教小模型；老师模型（R1）和学生模型同时跑。学生要模仿老师每一步输出的概率、中间层特征。老师模型本身要在线参与训练。缺点：师生模型架构最好接近，很难跨家族，比如Qwen很难直接蒸馏给Llama。

R1的数据蒸馏：大模型只出题写答案，把这些题目+标准答案打包成数据集，拿这个数据集去训一堆别的小模型。

第一步：先用R1生成一大堆高质量问题+ R1 写的完整推理答案，保存成80万条数据。 第二步：R1模型本身可以关掉，不再参与训练。拿这堆数据，直接给Qwen、Llama各种基座做普通SFT（有监督微调）。 图中箭头是从数据集指向各个基座，没有箭头从R1模型连到学生模型。所以用数据蒸馏，不是权重蒸馏。

6个学生基座： Qwen2.5-Math-1.5B、Qwen2.5-Math-7B（数学专精小模型） Qwen2.5 14B、Qwen2.5 32B（通用Qwen） Llama3.3-70B-Instruct、Llama3.1-8B（Llama家族）

同一个80万数据集，丢给上面全部6种模型，全都能训。 说明：**这套推理数据集包容性很强，不挑模型。**

	数据蒸馏（R1）	权重蒸馏（传统）
老师做什么	提前写好大量问答样本，之后老师模型就不用了	训练全程老师模型都要运行
学习素材	文本数据集（问题+答案）	模型输出分布/中间特征
能不能跨家族（Qwen→Llama）	可以	一般不行
训练方式	普通SFT	蒸馏损失训练
应用场景	**跨模型家族、快速批量训一堆不同基座、不想让大模型一直在线** → 优先**数据蒸馏**	**学生模型和老师模型输出行为、概率分布高度一致，师生架构相同** → 用**权重蒸馏**

5.6 R1 的成绩与路线对比

5.6.1 R1 Matches Open AI O1

六个基准、五个模型的柱状图，读法要按组而不是按柱。

核心结论：**推理专项训练不是全方位变强，是能力定向偏移。** 模型在数学、代码这类推理任务上的成绩大幅上涨；但是它原本的通用知识能力基本保持不变，不会明显提升。

> 底层逻辑，呼应前面课件提到的 trade-off 不可避免的取舍： 训练资源会定向优化目标能力（数学、代码推理），但不会额外增益通用知识。通用能力是基座原生自带的，推理 RL 只改造推理相关的行为，不改动已有的知识库。

不是说推理训练**损害**MMLU，这里是**基本不动**。和 R1-Zero 不一样，R1-Zero 是只做 GRPO，没有混合非推理数据，会出现可读性、语言混杂的副作用；而 R1 训练混入了 200k 非推理样本，守住了通用能力。

经过 R1 数据蒸馏后的 32B 开源模型，在大部分评测基准上，性能可以胜过一些闭源的小模型。

5.6.2 RL vs SFT+RL

> R1-Zero：只做GRPO强化学习，**没有SFT冷启动**；
>
> R1：先做2轮epoch的冷启动SFT（80万混合数据：60万推理+20万非推理），再做RL。 **多加这2个epoch的SFT，换来哪些收益、哪些损失**

逐行拆解变化（R1-Zero → R1）

1. **LiveCodeBench（代码基准）** R1-Zero=50.0 → R1=65.9，**+15.9** 全表里涨幅最大。代码正是纯RL（R1-Zero）最薄弱的环节，冷启动SFT把这块短板补上。
2. **AIME 2024（高难度数学竞赛）** R1-Zero=71.0 → R1=79.8，**+8.8** R1-Zero原本71.0，低于o1-0912（74.4）；经过SFT后反超，拉到79.8。
3. **CodeForces（算法代码竞赛评分）** R1-Zero=1444 → R1=2029 直接超过 o1-0912(1843)、o1-mini(1820)。 R1-Zero代码能力明显弱于o1，经过SFT+RL之后，代码能力实现反超。对应前面课件提到R1-Zero代码短板的问题。
4. **GPQA Diamond（生物/物理/化学专家科学题）** R1-Zero=73.3 → R1=71.5，**-1.8，唯一下降项** 增加SFT混合数据之后，专家级科学问答分数轻微下跌。

> 原因：SFT数据集里没有足够多这类高端科学题目，属于前面提到的能力取舍（trade-off）。虽然混入20万非推理数据用来兜底通用能力，但无法完全消除副作用。

核心问题：2个epoch冷启动SFT到底价值多少？冷启动SFT**不是全方面均匀变强**。它重点补齐纯GRPO（R1-Zero）的短板（代码、竞赛数学），但会在部分不在训练数据覆盖范围内的任务上，出现轻微性能下滑。 而80万数据里搭配20万非推理样本，目的就是**把这类能力衰减控制到很小**，但无法完全消除（GPQA这里的-1.8就是残留代价）。

> 代码（LiveCodeBench）：**+15.9百分点**（最大增益）
>
> AIME竞赛数学：**+8.8百分点**，实现反超o1-0912
>
> GPQA专家科学题：**-1.8百分点**，小幅下降

5.7 本讲总结

1. **DeepSeek-R1 matches OpenAI o1 in complex reasoning tasks.**

> R1 在复杂推理任务上追平 OpenAI o1。
>
> 用词是 mtach ，不是outperform 。

定位：**结果层**，证明开源权重模型，推理能力可以对齐闭源前沿模型o1。

2. **DeepSeek-R1-zero achieves strong logical reasoning without supervised fine-tuning.**

> R1-Zero 在不使用监督微调SFT的条件下，获得很强的逻辑推理能力。
>
> R1-Zero有短板，LiveCodeBench、CodeForces分数明显落后。

定位：**方法层**，证明推理能力可以脱离人工标注数据，只用RL训练出来。这是R1-Zero最核心的贡献：**纯强化学习GRPO，不靠人工标注SFT，推理能力可以自己涌现。**

3. **Distillation enables smaller models to inherit advanced reasoning capabilities from DeepSeek-R1.**

> 蒸馏技术可以让小模型继承R1的高级推理能力。
>
> 工程落地价值：671B超大模型训练、部署成本极高，绝大多数团队无法使用。借助**数据蒸馏**，把R1学到的推理能力迁移到1.5B~70B中小模型，低成本可用。

定位：**落地层**，把前面两项科研成果变成行业能实际使用的模型。

Lecture3 和 Lecture4 串联对比

Lecture3（推理1）：提示与搜索，不动模型权重 Self-consistency、ToT、least-to-most、ORM/PRM。 思路：模型权重不变，**推理时靠提示词引导模型拉长思考过程。**

Lecture4（推理2）：训练，修改模型参数 GRPO + 规则奖励 + 冷启动SFT + 蒸馏 思路：在训练阶段，直接把深度思考的能力**固化进模型参数里面。**

共同目标：让模型作答前充分思考。

差别： Lecture3：**当场靠提示临时唤起思考**； Lecture4：**在训练阶段，把思考能力写入参数。** R1证明：后者不仅可行，甚至可以完全不用人工标注数据。

下节预告：Next Course: Planning for LLM Agents

推理和规划是递进关系：推理负责想明白每一步怎么做；规划负责把一连串动作编排好，调用工具、接收环境反馈、失败后重试。 对应之前Lecture2的智能体四子系统：感知 / 推理 / 记忆 / 执行。

Reasoning（本讲）：**单次问题，想出中间推理步骤**（解决思考内容质量）

Planning（下一讲）：**智能体，把多步推理组织成一套可执行动作序列**

Lecture 5 Planning in LLM Agents

1. 为什么需要计划什么是计划：反应式智能体做长任务时，会以三种方式翻车；一个工作定义，以及它最轻量的形态思维链（Part I）。

3. 为什么难：推理错一步可以重来，在真实世界里做错一步往往不行（Part II）。

4. 四股压力：模块化、反馈、长时程与控制，分别逼出不同的解法（Part III - V）。

5. 经典规划遇上 LLM：把 1970 年代就成型的 PDDL 拿出来试（Part VI - VIII）。

Part 1 Planning

1.1 Example：反应式智能体的长任务

Buy the lab's new GPU workstation: compare configurations and prices across vendor sites, check compatibility against the cluster requirements on the internal wiki, and file the purchase requisition.

假设问题：为实验室购买新的 GPU 工作站，比较各厂商网站上的配置与价格，对照内部 wiki 上的集群要求核验兼容性，然后提交采购申请。让一个反应式（reactive）智能体去完成一件长任务。所谓反应式，就是每步只看当前上下文、不做显式计划。

1. Four independent searches run as one thread in one context.

架构问题：比价（信息收集）四次相互独立的搜索，但被串进同一条线程、同一份上下文里跑。

后续解决：把规划器与执行器分开。

2. The four sites it chose all come back and none is great.

信息问题：核兼容（约束检查）它挑的四家网站都返回了，但没有一家让人满意。

后续解决：一个回头改条件或改计划的信号。

3. It picks one anyway and files the requisition.

控制问题：提交申请（不可撤销的动作）它还是挑了一家，直接把申请提交了。

后续解决：计划作为安全边界。

1.1.1 Four Problems

结构	English	在工作站算例里的体现
并行搜索	parallel searches	Vendor A / B / C / D / E 五路互不依赖，可同时进行
依赖	dependencies	五路搜索 → Compatibility check (wiki) → Requisition
迟到信息	information that arrives later in the task	兼容性检查结果要到中途才知道（箭头标注 arrives later）
不可撤销	a step that cannot be undone	提交采购申请这一步标注 irreversible

The graph itself also changes: when none of the four vendors fits, a fifth search has to be added. 图本身还会变：四家都不合适时，必须临时加进第五次搜索。

维度	要回答的问题
价值 Value	When is planning useful? When can it hurt?　规划什么时候有用？什么时候反而有害？
表示 Representation	How is a plan represented?　计划用什么形式表示？
承诺 Commitment	When is a plan made, and what may change it?　计划在什么时候做出？什么可以让它改变？
监督 Oversight	How/when should a person weigh in?　人应该在什么时机、以什么方式介入？

1.2 Plan Mode

1.2.1 Application

Cursor, Claude Code, and Codex all introduced a Plan Mode: write a persistent plan 写一份持久化的计划（落在文件里，不随对话消失）, have a human verify it 让人审这份计划, carry it out (possibly with sub-agents, in parallel) 可以用子智能体并行做.

好处	说明
更多算力 more compute	允许模型多想一步，而不是边想边做
更好的上下文 better context	先调查一遍，再带着全局信息动手
一个外部产物 an external artifact	计划是文件，可以被 review、被 diff、被复用
分解 decomposition	把大任务切成小块，每块可独立验证
人的批准 the human approval	把不可撤销的那一步交给人类把关

1.2.2 Five Steps

步骤	做什么
1. read-only investigation	只读调查：先看代码库，不改任何东西
2.clarifying questions	澄清提问：把不清楚的需求问出来
3.written plan	写出计划：形成一份可读的文档
4.human review	人工评审：由人来确认或打回
5.execution	执行：按计划动手改代码

1.3 Definition

A plan is an explicit representation of intended future behavior: the actions or subgoals an agent will attempt, with any ordering or dependencies among them.

计划是对意图中的未来行为的一种显式表示：智能体将要尝试的动作或子目标，以及它们之间的顺序与依赖关系。

1. 它面向本任务　A plan is for this task.（对照：skill 是跨任务复用的指导）。Plans can be more specific than skills.　计划可以比技能更具体。
2. 它组织未来的工作　A plan organizes future work. It may specify actions or subgoals, along with their ordering and dependencies.
3. 它是提案，不是保证　A plan is a proposal, not a guarantee.It can be inspected and revised: by the user, another model, or interactions with the environment.

对照项	计划 Plan	技能 Skill
适用范围	为这一个任务而做	跨任务复用的通用指导
具体程度	Plans can be more specific than skills 可以更具体	偏抽象，留出实现空间
生命周期	任务结束即作废	长期保留、反复使用
可否被修订	a proposal, not a guarantee 可被检查与修订 修订来源：用户（by the user）、另一个模型（another model）、与环境的交互（interactions with the environment）。	通常不随单次任务改动

1.3 Chain-of-thought

最轻量的一种计划：思维链（chain-of-thought）

CoT 是说出来给自己听，同一个模型，同一次生成，说着说着就答了。

Plan 是写下来给别人看，写完之后，别人（人、另一个模型、环境）可以插手。

1. Provides extra compute and a scratchpad for the model to refer back to　提供了额外算力，以及一块模型可以回头查阅的草稿。
2. Plan and solution arrive in one model generation.　计划与解答在同一次生成里一起出来。
3. Nothing explicitly inspects the plan. 没有任何外部环节去检查这份计划，因为它不能被检查与修订。
4. The CoT can be trained to increase the probability of a correct answer (STaR, Zelikman et al.; DeepSeek R1).　思维链可以被训练，用来提高答对的概率。

Example：Let's think step by step. OR Let's first understand the problem and devise a plan to solve the problem. Then, let's carry out the plan and solve the problem step by step.

1.4 Decompose

1.4.1 Steps and Function

Stage 1: Decompose Question into Subquestions. 把原问题拆成若干子问题

Stage 2: Sequentially Solve Subquestions. 按顺序逐个解答，并把答案向前传递

Function:

1. Models struggled at answering complex questions but were better at answering sub-questions.

模型在复杂问题上吃力，但在子问题上表现更好。

2. Answering sub-questions requires less context.

回答子问题需要的上下文更少。

3. Improvements grow with problem length.

问题越长，这种改进越明显。阶段一写出的子问题是可检查的中间产物，人看一眼就知道拆得对不对。同样，如果阶段一把问题拆错了，回答质量就会下降。后续解决方案是自适应递归分解，让委派本身成为一个可训练的动作，模型自己学什么时候该往下拆。

1.4.2 modularity 模块化

A decomposer generates sub-tasks and routes each sub-task to a dedicated handler.

一个分解器产生子任务，并把每个子任务路由到专门处理它的处理器。

Modularity allows improving handler models independently: via in-context demonstrations or training.

模块化让每个处理器模型可以被独立改进，用上下文示例或直接训练。

Programmatic/structured control: we'll see this later on with RAO, RLM, and MACU.

程序化 / 结构化的控制：后文会在 RAO、RLM、MACU 再见到。

Part 2 Reasoning vs Acting

2.1 Versus

Acting means plans must handle feasibility, consequences, information, and recovery.

一旦要行动，计划就必须同时处理四件事：可行性（feasibility）、后果（consequences）、信息（information）、恢复（recovery）。对应后面章节的规划压力：可行性→模块化，信息→反馈，后果→控制，恢复→长时程。

推理步只改变不产生新信息（changes only text），行动可以揭发信息（can reveal information），这是两者唯一一处行动比推理更有利的地方。

对比项	一次 CoT 推理步 A CoT reasoning step	世界中的一个动作 An action in the world
改动了什么	changes only text 只改动文本	changes the state the agent meets next 改变智能体下一步面对的状态
能否撤回	can be undone by writing more 再写几句就能推翻	can be irreversible: submit, send 可能是不可逆的：提交、发送
信息可见性	every intermediate is visible 每一个中间步骤都可见	can reveal information 可以揭示新信息（原本不知道的）
失败方式	（写错了重写）	can fail （可能失败）
为什么有用	plans are useful if the model can condition on them better 只要模型能更好地以计划为条件	（同样的计划，在这里要面对可行性、后果、信息与恢复）

2.2 Classical Planning 经典规划：状态、动作与前置条件

在 LLM 出现之前，规划是一个有严格定义的问题。

积木世界（blocks world）五个要素：状态、动作、前置条件与效果、初始状态与目标、计划。

A state is a set of predicates; actions have preconditions and effects.

状态是一组谓词的集合；动作带有前置条件与效果。

A valid sequence of actions satisfies every action's preconditions.

一个合法的动作序列，是每一步的前置条件都被满足的序列。

Planning is a search problem: find a sequence of valid actions that reach the goal.

规划就是一个搜索问题：找一条能到达目标的合法动作序列。

步	动作	使合法的前置条件	效果
1	unstack(blue, orange)	blue 在 orange 上面、且 blue 是干净的、手是空的	变成 holding(blue)，orange 变干净
2	put-down(blue)	手上有 blue	blue 落到桌上、手变空
3	pick-up(orange)	orange 在桌上、干净、手空	变成 holding(orange)
4	stack(orange, blue)	手上是 orange、blue 干净	得到目标 on(orange, blue)

2.3 Changed hard point

经典规划里，世界的模型（状态、前置条件、效果）是显式写出来的，所以世界模型对不对是可验证的；LLM 把这些全放进权重里，写一个看起来合理的计划变容易了，判断它是否真的理解世界反而变难了。

1. With LLMs everything is implicit! 有了 LLM，一切都变成隐式的。
2. For an LLM agent, writing a plausible plan is easier, and knowing whether its model of the world is correct is harder. 对 LLM 智能体来说，写出一个看起来合理的计划更容易了，而判断它对世界的建模是否正确更难了。
3. So agent design uses less deep search, and more checking the world: observation, verification, and recovery from errors. 所以智能体设计的重心从深搜转向检验世界：观察、验证、从错误中恢复。
4. But search, and action applicability, are still useful concepts 但搜索与动作的适用性仍然是有用的概念，只是不再需要人手工写出来。

对比项	经典规划	LLM 智能体
世界模型	显式写出（谓词 + 前置条件 + 效果）	everything is implicit，藏在权重里
最难的一步	搜索：在巨大状态空间里找到可行序列	判断：判断模型的界模型到底对不对
主要手段	深度搜索（A*、启发式）	observation, verification, and recovery from errors
搜索还需要吗	需要，是主体	仍然有用，但不再是设计的重心

2.4 Plan Representations

Formal plans are checkable; language plans are general. Programs are in-between.

形式化计划是可检查的，语言计划是通用的，程序在两者之间。

表示	可检查性	通用性	示例
形式化计划（formal）	高：能自动验证每一步是否合法	低：需要预先写出域模型	PDDL
语言计划（language）	低：只有人能读	高：什么任务都能描述	CoT、自然语言 plan
程序（program）	中：能跑、能读，但受 API 限制	中：受可用 API 与程序结构限制	Policy Code

Perception APIs（感知 API，绿色）：把画面变成符号，例如 detect_objects(...)。

Control APIs（控制 API，橙色）：把符号变成动作，例如 pick_place(obj0, obj1)。

这段代码是程序即计划的典型：它把 找空碗 → 拼出待堆列表 → 逐个摆放 写成了确定的控制流，因此可以被读、被 diff、被改，而不是像 CoT 那样只在一次生成里存在。

2.5 RePlan while Acting

1. 开发者预先定义工作流（Developer-defined workflow，无智能体 Agentless） 人直接写死固定流程：只能按写好的供应商列表挨个检查。 特点：逻辑固定，完全不能临场变通。
2. 先规划、再执行智能体（Plan-then-execute agent） 大模型先一次性做好完整规划（决定要查哪些供应商），规划敲定之后，再从头到尾执行，执行中途不再改计划。特点：速度更快、成本更低、结果更容易预测。
3. 自适应智能体（Adaptive agent，RAO/MACU） 边跑边看反馈，遇到意外可以重新规划。 例子：连续 4 个供应商都失败了，模型拿到这个执行反馈之后，主动去找新的供应商。特点：当执行过程里发现了新信息、突发状况，模型可以随之调整方案。原则：只有当临场反馈带来的收益，值得多花的成本和额外复杂度时，才用边执行边重规划。

Part 3 Planning Pressures I：模块化

3.1 Workflow

案例场景：采购 GPU 工作站。一共 4 个核心压力：模块化、反馈、人工管控、长任务视野。

MODULARITY 模块化

Searches carried out separately. 各个供应商的查询任务可以拆成独立小块分开执行。

任务拆成：查询供应商 A、B、C、D，每一步搜索都是独立模块。

好处：可以单独执行、单独失败，不用把整个流程推倒重来。

FEEDBACK 反馈（replan 重规划）

None fits, so revise the plan. 查完 A/B/C/D 之后，发现所有供应商的配置全都不符合要求 → 拿到这个反馈，修改原有计划，再去查找新的供应商。

核心：执行结果返回信息，触发重新规划。

CONTROL 管控 / 人工审核

A person approves before the agent submits. 智能体不能直接决定，提交采购申请之前，必须经过人来审批确认。

作用：给智能体加一道人工安全阀，防止模型做出错误采购。

LONG HORIZON 长视野 / 长任务跨度

Requirements and progress persist across the whole task. 整个采购任务周期很长，从头到尾，需求信息、已经查到的进度都要持续保存，不能中途丢失。

例子：一开始定的 GPU 性能要求，在后面新增供应商检索、人工审批环节都要记住。

Workflow

1. 并行 / 依次独立搜索 A、B、C、D 四个供应商（模块化）
2. 核对所有配置，发现全都不满足需求（拿到反馈）
3. 智能体重新规划，去寻找新供应商
4. 找到合适配置后，提交给人审批（人工管控）
5. 全程记住采购需求、历史查找记录（长视野）

6. 审批通过，提交采购申请

3.3 Solution 1: Planner and Executor

3.3.1 Core Idea

核心思想：把智能体拆成**规划器 (Planner)** 和**执行器 (Executor)** 两个独立模块，各司其职，可以用不同模型、不同训练数据。

Example：找到最便宜的 4K 显示器

1. 视觉观察（Vision-Only Observation） 拿到网页截图画面，任务目标：找最便宜 4k 显示器
2. Planner 规划器（用 GPT-4o） 能看懂画面，但不擅长输出像素坐标。只负责用自然语言描述下一步动作：

在页面顶部搜索框，输入 “4k monitor”

3. Grounding 定位 单独询问：页面顶部搜索框对应的像素坐标是多少？得到坐标
4. Executor 执行器（7B 小模型，单独训练） 把规划器的文字指令，翻译成屏幕像素操作、输入文字，真正操作网页。

Tips：规划需要的是页面上有什么元素、它承担什么角色，接地（grounding）需要的是这个元素在哪个像素。

训练数据：规划可以从动作轨迹里学，接地可以从点击坐标里学。

代价：规划器必须把动作描述得足够清楚，否则执行器接不上接口。

3.3.2 Training the Planner

1. Grounded Plan Generation 用一个更强的教师大模型（Teacher LLM），把已经成功的任务轨迹，切分、标注，生成规划步骤。
2. Synthetic Plan Expansion 基于少量种子规划数据（Seed Plan Data），让教师模型扩充生成更多查询 + 规划样本。 最后拿这些标注好的轨迹，微调规划器模型、执行器模型。

3.3.3 Multi Agent Systems and Four Fixed Roles

流转逻辑： Reproducer （复现 bug，写出复现代码）成功 → 交给 Fault Localizer （定位代码错误位置）；Reproducer 失败 → 直接交给 Editor （修改代码）； Editor 改完 → Verifier （验证 bug 是否修好），验证成功则结束。但给代码任务预先分配固定角色的多智能体方案，效果很有限，每加一次交接，就多一次有损压缩。

两个主要缺点：

1. 验证器（verifier）只负责验证，它本身不能去定位 bug，没办法回头检查自己的判断。角色不能临时切换。
2. 智能体之间靠简短报告交接任务，丢失很多细节上下文，下一个智能体缺少关键信息，容易出错。

Overall

规划器 + 执行器分离：拆分职责，大模型做思考规划，小模型做底层像素 / 动作执行，是有效的模块化方案

但固定角色多智能体（写死分工） 有明显缺陷：角色不能灵活切换、交接报告丢失上下文，代码任务里效果一般。

3.4 Solution 2: SayCan

模块化的另一个用法：不拆角色，而是给每个技能一个现在能不能做的概率，再拿它给计划重打分。

思路：把经典规划的前置条件换成概率

Recall classic planning: actions have pre-conditions. Learn a value function for each robotic skill, which predicts probability of completing it. Multiply this value with the probability of the LLM producing the plan.

经典规划中动作是有前置条件的，为每一个机器人技能学一个价值函数，预测它被成功完成的概率。把这个价值乘上 LLM 给出该动作的概率。

Example：左列是 LLM 的对数似然（负数，越接近 0 表示模型越想选它）；

右列是 价值函数给的成功概率。

图上左侧的提示例子是 How would you put an apple on the table?

模型的回答 I would: 1. ___；重打分之后变成 I would: 1. Find an apple, 2. ___。

Part 4 Planning Pressures II：反馈

1. 重规划：执行中收到环境观测，遇到障碍动态修改计划
2. 思考vs行动：光堆token思考有上限，和环境交互更重要
3. 过度思考：想太多（分析瘫痪）反而降低解决问题的成功率
4. 思考vs探查：拿不准时，优先探查环境获取真实信息，不要盲猜
5. CTA置信校准：行动前显式评估不确定性，让模型识别自己不知道的地方，主动测试

4.1 Environmental feedback

Example：煮土豆，然后丢进回收垃圾桶

执行一步、拿到环境观测，遇到障碍就动态改方案，就是环境反馈驱动重规划。

1. t=0：LLM规划器一次性生成初始高层计划：找到土豆→拿起土豆→放进回收桶
2. t=5：智能体在环境里执行，观察反馈：找不到土豆，但看到冰箱。拿到这个新观测，模型重新修改规划：打开冰箱、拿土豆……
3. t=20：继续执行，再次收到反馈：找不到回收桶，但看到普通垃圾桶，再次重规划，改用普通垃圾桶存放土豆。

4.2 Thinking vs Acting

不要光靠空想推理，多和环境交互拿到真实观测更有用。

横轴：每条任务轨迹消耗token数量（越多=思考越多）；纵轴：任务成功率

3条策略：

1. Per-Step Budget Forcing：每一步强制固定token预算
2. Per-Step Best-of-N：每一步采样多条候选，挑最好一条
3. Interaction Scaling：增加和环境交互次数 结论： 单纯增加token、疯狂长思考，收益会慢慢饱和；和环境交互获取真实信息，能更好提升成功率。

4.3 Overthinking

模型的过度思考分数越高，能解决的问题比例反而越低，想太多会降低任务完成率。

Classical Example：

1. Analysis paralysis 分析瘫痪：无限拆解、反复分析，迟迟不下手行动
2. Rogue action chains 失控动作链 / premature disengagement：跑偏到无关步骤，或者中途直接放弃任务

4.4 Thinking vs Looking

不要仅凭内部猜测直接行动；拿不准的时候，优先探查环境获取真实信息，而不是在脑子里空想。

任务：找到价格最大的商品ID

Guess-and-Go（猜了直接做）：模型凭99%的自信直接写代码读取文件，赌文件格式是csv。一旦猜错直接失败。

Deliberate Testing（先探查测试）：模型意识到自己只有50%把握，先执行探查指令，看一眼文件真实格式，拿到真实信息之后再写最终代码。

置信度就是决策信号

两边的模型能力一样、任务难度一样，差别只在一件事：它对文件格式的置信度不同（99% vs 50%）。99% 时就直走（Guess-and-Go）；50% 时就先去环境里取信息（Deliberate Testing）。同一个模型可以按置信度切换策略，而先测那一路并不比猜那一路上算力更多。

4.5 CTA：Calibrate-Then-Act 行动前校准置信度

Baseline 基线方案（普通Prompt / RL智能体）

模型隐式地判断自己的置信度：靠模型内部知识、RL训练学到的统计，不会显式写出概率，直接选动作。

CTA：在做决策前，显式估算先验概率，把概率信息放进Prompt，模型基于概率做推理：

1. 先预估先验概率 $\hat{p}(Z|x)$（比如文件分隔符是;的概率0.85）
2. 模型读到这个概率，知道“我不能百分百确定”，从而主动安排测试去验证。

Part 5 Planning Pressures III：长时间任务

1. 长视界失败：长任务中**单点错误会连锁传递**，越往后影响越大
2. 自条件效应 self-conditioning：模型会拿自己之前的错误作为上下文继续推理，导致后续准确率持续下降；大模型能跑更长任务，但也更容易出现这个问题
3. 递归自适应拆解：难题自动委派子智能体递归拆分，委派动作本身可以训练

5.1 Vending-Bench 长时程失败

长时序任务的错误会不断累积传递

Example：Vending-Bench 自动售货机业务模拟

任务：定价、接单、补货；最长 222 个模拟天数，最多 2000 条消息（约 25M token）

核心痛点：长序列任务里，一个临时的小错误，会一路影响后面所有决策，越错越严重。故障连锁流程：

1. Premature restock 过早补货：环境提示商品暂时缺货，模型误判成 “已经发货”
2. Wrong inference 错误推理：把 “现在缺货” 直接推导成 “生意失败”，不再等待到货通知
3. Error persists 错误持续放大：后续产生罚金，最终触发业务终止、欺诈警告

Tips：并非是坏计划，如果是计划坏，那么换一份更好的计划就能解决；但这里坏的不是计划，而是每执行一步就把自己的历史当作事实这个机制。哪怕开局计划完美，第一个小错误依然会被继承下去。

5.2 Execution horizon 执行视野

1. 左图：单步准确率一点点提升，放到长任务上，任务总长度会指数级上涨
2. 右图：模型会基于自己之前的错误继续决策（self-conditioning 自条件效应）

绿色线：理想预期的准确率；红色线：真实观测，越往后准确率持续下滑

结论：更高的单步准确率，能支持更长的任务；但自条件效应，会让模型后续步骤因为前面的错误而越做越差。

Better step accuracy extends the horizon, while self-conditioning can make later steps less accurate.
单步更准能把视野拉长，而自我条件化会让后面的步更不准。

5.3 Decomposing adaptively and recursively 自适应递归任务拆解

Model size and execution horizon 模型大小与执行视野

左图：模型参数量越大，能够跑完的任务长度越长（Qwen3、Gemma3 都呈现这个趋势）

右图：大模型更容易出现自条件效应。同样的诱导错误率下，越大的模型，后续轮次准确率掉得更厉害。

思考题：能不能用强化学习解决这个自条件带来的长任务衰减问题（限制错误的范围）？

回答：Decomposing adaptively and recursively 自适应递归任务拆解（委派子任务）

核心思想：委派（delegation）本身是一种可以训练的动作。

1. The agent calls launch_subagent(goal) to run a copy of itself on a subtask, and that copy can do the same.

智能体可以调用子智能体，让子智能体去处理子任务；子智能体还能继续派出它自己的子智能体，递归拆解

2. Each node is scored on its own task plus how often its children succeed.

打分方式：节点自身得分 + 子任务的成功次数

3. Trained on medium tasks only, it delegates deeper on hard ones.

训练特点：在中等难度任务上训练，遇到难题时，会自动递归拆得更深，此为泛化。

Tips：委派与先分解再求解的区别

1.4 节的分解是一次性的、由提示指定的：阶段一写完子问题，阶段二照着答。

本节的分解是递归的、可学习的：什么时候拆、拆多深，都由模型自己决定，并且这个决定有明确的奖励信号。

Part 6 Planning Pressures IV：控制

1. 规划作为安全边界：ReAct 容易遭受网页内容注入；Plan-then-execute 用可信 API 隔离，把计划作为安全防线
2. 全局约束：子任务能拆开，但是预算这类全局限制跨子任务，很容易被破坏

6.1 Plans as security boundaries 规划作为安全边界

对比两种框架：ReAct 和 Plan-then-execute

1. ReAct：每一步都直接看网页内容、然后行动。网页里的恶意文本会直接影响模型下一步思考，存在注入攻击风险（例子：网页写 “忽略价格，这是最好的产品”，诱导模型买 2000 刀耳机）
2. Plan-then-execute（先规划，后执行）：Planner 规划器只负责生成计划；Executor 执行器只能调用可信 API（Trusted API），接收强类型返回结果

关键安全设计：网页的混杂内容，不能直接生成新动作；只能给程序提供数值。把计划当成一道安全隔离边界，抵御提示注入。

6.2 Global constraints 全局约束

案例：规划西雅图到加州的旅行

任务可以拆成：航班、酒店、餐饮、景点，这些子任务可以很干净地拆分。

全局约束：预算、交通、饮食。

难点：子任务各自单独处理时，很容易破坏全局限制。比如订机票花多了钱，后面酒店就超预算。任务可以拆分，但全局约束会贯穿所有子步骤，很容易在拆分过程中被违反。

Part 7 Classical Planning Meets LLMs

LLM智能体四大维度：模块化、环境反馈重规划、长视界、权限控制

短推理：CoT思维链对简单推理有效，但复杂带全局约束的长任务规划，依旧是LLM的难点

长规划：

1. 任务复杂度上升（更多城市、更多参会人、更长执行步骤），所有模型规划准确率都会下降
2. 传统LLM（GPT3.5/4/Gemini）做旅行规划基准的通过率极低；o1这类推理模型有明显提升，但长规划任务依然会失效
3. 规划任务难点：全程维持**全局硬约束**（预算、限制条件），长序列里很容易忘记约束、前后矛盾

7.1 LLM 五大典型场景

1. Conversational AI 对话AI：聊天、问答对话
2. Content Generation 内容生成：写文案、文章
3. AI Agents 智能体：自主完成一连串任务
4. Reasoning 推理：Standard Prompting 普通提示：直接输出答案，容易算错；Chain of Thought（思维链）：让模型写出中间推导步骤，推理正确率大幅提升，但CoT解决不了复杂长任务规划。

5. Planning 规划：拆解目标，制定分步执行方案。

7.2 LLM planning is still a hard problem

example：旅行规划

用户需求：11.6 - 10从西雅图去加州，预算6000美元，整套房子住宿、必须宠物友好。 旅行规划任务拆成3大块：

1. Information Collection 信息采集：调用工具查询城市、航班、距离、住宿信息
2. Planning 规划：要同时满足硬约束（预算、房型、宠物友好）+常识约束
3. Delivery Plan 输出完整行程方案

难点： 旅行规划是带大量全局硬约束的长任务，需要多次调用工具、全程守住预算/条件；LLM很容易顾此失彼，这就是LLM现在短板。

哪怕是GPT4-Turbo、Mistral、Gemini这类当时最强模型，直接做旅行规划的最终通过率很低。并且用SOTA大模型，硬约束长行程规划依然很难；o1推理模型相比传统GPT4有明显提升，但没完全解决。

Two-stage：先调用工具，再做规划

Sole-planning：纯模型自己规划，不用真实工具。Final Pass Rate，GPT4-Turbo也只有0.6%，非常差。

Ground-truth tool use（使用真实、准确的工具返回结果再规划），DirectGPT-4-Turbo能到4.4%，但依旧很低。

o1-mini、o1-preview 相比GPT4系列，在旅行规划基准上大幅提升，但依旧不完美，随着约束变复杂，依然会翻车。

结论：

1. Trip planning 旅行规划：横轴=要去的城市数量。城市越多（任务越长），所有模型准确率一路下跌。
2. Meeting planning 会议规划：横轴=参会人数。人数越多，规划成功率快速下降。
3. Mystery Blocksworld 积木世界规划：横轴=计划的步骤长度。计划越长，正确率断崖下跌。
4. o1系列（LRM大推理模型）整体显著优于普通LLM，但计划一旦很长，照样掉点。

Part 8 Three Solutions

1. 路线 1 Scaling Law：单纯堆大模型、数据、算力。昂贵，很难单独搞定复杂约束规划。
2. 路线 2 Hybrid 混合系统（LLM+Solver）：LLM 理解自然语言，Solver 负责严格约束优化（如 MILP）。可以保证方案满足全部硬约束，旅行规划任务效果好。
3. 路线 3 符号涌现 + PDDL：LLM 翻译成 PDDL 规划语言，交给传统符号规划器做启发式搜索。

   评估智能体用 APEC 四个维度：准确率、主动性、高效、可信度；可以用 Agent-as-Judge 做自动评估。

8.1 三条路线的总览

路线	概括
Option One: Scaling Law	继续堆数据、堆算力、堆参数。
Option Two: Hybrid System	把 LLM 和符号求解器混合在一起。

路线	概括
Option Three: Emerging Symbolic Structure from Neural network	等模型自己产生符号结构。

区别：符号结构从哪里来。 路线一是它从规模里自发长出来（converge to anything symbolic）；路线二由人事先写死（求解器、约束、API）；路线三是它从神经网络的表示里产生出来。

8.2 前两条路线：缩放定律 vs 混合系统

8.2.1 Scaling Law 缩放定律

思路：堆更多数据、更多算力、更大的模型。

图曲线：参数量越大、算力（FLOPs）越高，训练损失不断下降。

缺点：成本极其昂贵，并且只靠堆规模性价比差，规划问题很难只靠大模型搞定。

8.2.2 Hybrid Systems 混合系统（大模型 + 求解器 Solver）

三种混合架构：

1. LLM 调用工具 → Solver 负责计算求解
2. Solver 产出数据，用来训练 LLM
3. End2End 端到端：深度模型 + 求解器融合在一起，联合训练

核心思想：LLM 擅长自然语言理解；Solver 擅长严格约束、组合优化。二者互补。

Example：TTG（To - the - Globe）

语言驱动的有保证旅行规划（LLM + MILP 整数规划求解器）

TTG 的思路：不让大模型做 “规划求解” 这件难事，LLM 只做翻译，求解交给 MILP 混合整数线性规划符号求解器。

把自然语言的旅行需求翻译成形式化描述，再交给混合整数线性规划求解器。

1. 用户输入：自然语言旅行需求。

> 示例：西雅图去加州，11.6 - 11.10，预算 6000 美元，整套房间、宠物友好住宿。

2. Instruction Translator 指令翻译器（微调后的 LLM）。

> 任务：自然语言 → 符号 JSON 规格描述

提取出发地、目的地、时间窗口、预算、住宿偏好；读取外部数据库，把候选航班、酒店信息填充进符号结构；

3. 送入 MILP 混合整数线性规划求解器

拿到全部符号约束，数学求解，输出数学保证最优、并且 100% 满足所有约束条件的行程方案。

> Guaranteed（有保证）的含义：求解器数学上保证解满足全部约束，不是 LLM 猜出来的。

4. 输出层：把求解器输出的符号最优解，再转成人类可读的自然语言行程给用户。

> LLM 翻译：2.508s（整个系统瓶颈）；
>
> MILP 求解总共仅：0.575s 大部分时间耗在 “人话转符号”，求解器本身算得很快。

局限

1. 依赖数据库内的航班、酒店候选集合；数据库之外的选项处理不了。
2. 全部压力落在 LLM 翻译模块，翻译出错，整套方案失效。
3. 问题必须可以写成 MILP 数学形式，不是任意规划问题都可以套用。

TTG（To - the - Globe）是 LLM+MILP 的神经符号混合旅行规划系统。大模型仅将用户自然语言需求翻译成符号规格，由 MILP 求解器输出满足全部约束的最优行程；正确性保证来自求解器，但系统风险来自 LLM 的翻译错误。

核心痛点：LLM 自己很难处理一大堆约束，但组合求解器可以严格满足约束。数据库里存放航班、酒店真实信息，求解器能保证方案满足所有硬约束，不会出现 LLM 忘记预算这类幻觉。

实验结果（人工评估 NPS 净推荐值）

三个维度用户满意度：完全满足需求：+40.0；性价比：+35.1；高效性：+36.9 。说明这套 LLM+MILP 混合方案，用户体验很好。

8.3 Multi-round Dialogs 多轮对话

用户经常有**隐藏约束**（没主动说，需要主动挖掘，比如无障碍需求、舱位偏好）。

The user has hidden constraints. How to figure them out? → Proactively ask!

智能体不能等用户一次性说完，要主动提问，挖出隐藏条件。

例子：普通 Llama 3.1-8B 准确率 0.231；APEC-Travel 智能体 0.279，更强。

8.4 APEC Agent Constitution 智能体评估 4 大指标

评估智能体不能只看最后答案好不好，**过程也要打分**

1. **Accuracy 准确率**：最终方案质量，能不能答对问题

The quality of the final solution that the agent provides (e.g., number of questions that are answered correctly). 最终解的质量，例如答对的问题数

2. **Proactivity 主动性**：能不能主动收集必要信息，主动提问挖掘隐藏约束

Whether the agent proactively collects useful information to solve the task. Such information may be public or private, vague or precise, explicitly provided or inferred from requests. 是否主动收集有用信息；这些信息可能是公开或私有的、含糊或精确的、明说的或从请求里推出来的

3. **Efficiency 高效性**：用最少轮对话、最少工具调用完成任务

Whether the agent can achieve its goal with a minimal number of interactions (e.g., number of questions asked, API calls and tool uses). 用最少的交互次数达成目标，例如提问数、API 调用数、工具使用次数

4. **Credibility 可信度**：幻觉、前后矛盾的多少，结果是否稳定可靠

The reliability with which agents achieve positive outcomes (e.g., amount of hallucination and inconsistency). 稳定达成好结果的能力，例如幻觉与不一致的量

用Agent Constitute 微调模型

Iteration2 第二轮微调效果最好： 总智能体分数 12.95，整体准确率、关键约束准确率、效率、主动性都明显上涨。 证明：用 APEC 这套评估指标做微调，可以实实在在提升智能体能力。

8.5 Judge Agent by Agent

评估方案演进：

1. LLM-as-a-Judge（普通大模型做裁判）
2. Agent-as-a-Judge（智能体当裁判）
3. Human-as-a-Judge 人类评估 Agent 裁判本身可以调用工具、做规划，比单纯 LLM 更适合评估复杂智能体任务。

左：LLM-as-a-Judge（对话任务 Conversation Task）

任务：If the FED buys bonds in the secondary market： (A) the money supply will increase. (B) the money supply will decrease. (C) the money supply will not be affected. (D) the discount rate would be affected. Output the answer only.

LLM A 答：The Federal Reserve buys bonds in the secondary market to increase the money supply.

LLM B 答：(A) the money supply will increase.

Judge LLM 判词：B is better because it gives the correct option directly, following the required format of the question.

Compare-based Judge 做的是相对判断：只回答A 和 B 谁更好，并给出理由（gives the correct option directly, following the required format）。

Metrics-based Judge 做的是绝对打分：两个模型 Accuracy 都是 10/10，差别只在 Instruct-Following（5/10 对 10/10）。A 的答案内容正确，但没按只输出答案的格式要求写，这一条只有指标裁判看得出来。

中：Agent-as-a-Judge（智能体任务 Agentic Task）

任务：Hi! Please follow the instruction and set up the script from the blog https://www.factsmachine.ai/p/hidden-in-plain-sight to generate 1080p images with hidden text "FUTURE," in src/visualize.py. Save them in results/ and verify the text is embedded.

Agent A 与 Agent B 各自有一个 Workspace；中间是共享的 Environment。两侧的 Act 箭头都指向 Environment。

Judge Agent 的判词：B is better because it runs and generates the required files, fulfilling the user's request. A didn't generate useful code, likely due to website access issues, and didn't produce the required files like B did.下面同样分 Compare-based Judge 与 Metrics-based Judge 两层。

右：Human-as-a-Judge（智能体任务 Agentic Task）

右栏的任务与工作区布局和中栏完全相同，只是 Judge Agent 换回了 User。

8.6 第三条路线：PDDL 规划域定义语言

问题：大模型在训练后，能不能自发涌现出符号化的结构（Emerging Symbolic Structure）？

没有给答案。这条路线的想法：不外接 Solver，希望 LLM 自己学会类似符号规划的能力，直接做规划。

引出规划领域标准语言：PDDL Planning Domain Definition Language，规划域定义语言

TTG 和 PDDL 的异同

相同：都是LLM 翻译成符号描述，交给现成外部求解器求解范式。

区别：TTG 用MILP 混合整数线性规划，适合组合优化、预算最优这类连续 + 离散混合问题；PDDL 多用于经典动作规划（积木世界、机器人动作序列）。

8.6.1 PDDL 基础概念

Planning Domain Definition Language：不承诺任何方案，只提出一个问题。

1. **Precondition 前提条件**：执行这个动作必须满足的状态，条件不满足动作不能执行

must hold to ensure the action can be executed；defines the states in which the action can be executed；action is applicable in state s if the preconditions are satisfied by s

2. **Effect 效果**：动作执行之后，世界状态发生什么变化

represent the effects of action on the world；defines the result of executing the action

Add list：动作之后新增成立的事实 (the fluents in) the positive literals in the action's effects　例：{At(p, to)}

Delete list：动作之后不再成立的事实 (the fluents in) the negative literals in the action's effects　例：{At(p, from)}

3. **状态更新公式**：执行动作，删掉 Delete 里的内容，加上 Add 里的内容。

Result of action a in state s: $ RESULT(s,a) ≝ (s \ DEL(a)) ∪ ADD(a)$

例子：Fly 飞机动作，Fly(P1, SFO, JFK) ⟹ remove At(P1, SFO), add At(P1, JFK)。

前提：飞机在 SFO；效果：飞机不在 SFO，出现在 JFK。

8.6.2 Example

Action schema 动作模板

前提：p 是飞机，from/to 是机场，飞机 p 在 from 机场

效果：飞机不在 from，飞机出现在 to

Action instantiation 动作实例化：把 p、from、to 填成具体值，比如 Fly (P1,SFO,JFK)

8.6.3 求解：启发式搜索 Best-First Search

规划本质就是状态空间搜索，PDDL 用来描述状态、动作、目标，搜索算法找到动作序列。

g：从起点走到当前节点的实际代价

h：启发函数，预估当前节点到目标的剩余距离

w：调节的是g和h之间的比例

f = g + w · h：用已知代价加上一个被缩放过的猜测

w=1 → A * 搜索：h 满足可采纳性时能给出最优解；

w>1 → WA（加权 A）：wA* 更偏向 h，更快但不再保证最优；

w → ∞ 时 GBFS 只看 h、完全不管已经花了多少代价，最快也最不稳。

w 趋向无穷大 → GBFS 贪心最佳优先搜索

8.7 用 LLM 解决 PDDL 规划的三类范式

自然语言规划问题，交给 LLM，有 3 大类做法：

1. LLM as search components generator：大模型生成搜索组件代码，交给黑盒搜索算法执行
2. LLM as policy generator：大模型生成领域专用规划策略代码，直接输出动作
3. LLM as PDDL generator（最常用）：LLM 把自然语言问题翻译成 PDDL 符号描述，再调用现成经典规划器（off-the-shelf planner）求解

角色	LLM 产出什么	谁来执行	交付
搜索组件生成器 LLM as search components generator	Search components code (black box model) 搜索组件的代码（把模型当黑盒）	Search algorithm 搜索算法	交代码：LLM 写出一段搜索程序的组件（启发函数、剪枝规则之类），搜索本身还是由算法跑。
策略生成器 LLM as policy generator	Policy code (domain-specific planner) 策略代码（领域专用规划器）	代码本身	交策略：LLM 直接写一个领域专用规划器，规划能力被固化进代码里。这一档对模型的要求比第一档高。模型在这条路上不被信任到能自己搜，只被用来生成搜索用的零件。
PDDL 生成器 LLM as PDDL generator	High level planning model (PDDL) 高层规划模型（PDDL 描述）	Off-the-shelf planner 现成的经典规划器	交规格：LLM 只把自然语言问题翻译成 PDDL 描述，求解交给 Off-the-shelf planner。这一档对模型的信任度最低，也最可靠。

5584 LLM for Agent 2

Archive source: `prompt_appendix/task3_v1/llm_output/iteration_6/knowledge/5584-LLM-for-Agent-2.md`

5584 LLM for Agent2

Lecture 6 Tool Use for LLM Agents

章节	内容
1 Capabilities	tools add retrieval, execution, generation, and application actions. 工具给模型加上检索、执行、生成与应用动作
2 Mechanics	schemas enter the prompt; model protocols emit calls; harnesses validate, dispatch, and match results by ID. schema 进提示、模型按各自协议吐调用、harness 校验并派发、按 ID 配对结果

章节	内容
3 Constraints	JSON Schema and grammar masks guarantee form—not semantic correctness. JSON Schema 与语法掩码只保证形式合法，不保证语义正确
4 Interfaces	Direct REST calls and MCP are alternative ways to expose capabilities and manage credentials. 直连 REST 与 MCP 是暴露能力、管理凭据的两条路
5 Systems	orchestrate dependencies and concurrency; evaluate selection, arguments, outcomes, cost, and operational failures. 编排依赖与并发，并评估选择、参数、结果、成本与运行故障

本讲的三个阶段

第一阶段（Part 1 - 2）：工具是什么、调用怎么走 —— 能力与机制。

第二阶段（Part 3 - 4）：怎么保证调用能用、外部能力怎么接 —— 约束与接口。

第三阶段（Part 5 - 6）：多个调用怎么协同、怎么度量 —— 系统与评估。

Part 1 Capabilities 工具的能力

1.1 Tool Definition

A tool is an interface through which a language model can invoke an external computer program.

工具是一个接口（interface），语言模型通过这个接口去调用一个外部计算机程序。接口说明它是一份契约，不是一段能力与外部说明程序不在模型参数里，模型无法凭自身参数完成这件事。

The model proposes that external software decides whether and how to execute it.

责任分界线：模型只提议，无法绕过 harness 直接产生副作用；是否执行、如何执行，由外部软件决定。

1.2 Benefits of Tools

动词	中文	说明
扩展 Extend	Access information or actions unavailable in the model's parameters. 访达到模型参数里根本没有的信息或动作	Extend 解决的是能力边界：模型参数里没有今天 PolyU 发了什么公告，再大的模型也答不出来
分担 Facilitate	Delegate operations external programs perform more reliably or efficiently. 把外部程序做得更可靠或更高效的操作交给它们	解决的是可靠性：18 + 23 + 41 + 54 的平均数模型其实会算，但会算错；交给程序算是为了让错误率从偶尔降到几乎不可能。

1.3 Example of Tools 典型工具

Search 检索：把模型不知道变成去查一下。

Calculator 计算器：把模型算不准变成交给确定性的程序算。

Browser 浏览器：把读到静态文本变成在网页上操作。

Python 代码解释器：一个能表达任意控制流的通用工具。

API 外部服务接口：把私有数据与业务动作接进来。

Need	Example tool	Benefit
Current information　当前信息	Search	Fresh evidence　新鲜证据
Exact computation　精确计算	Calculator or Python	Reliable execution　可靠执行
Private state　私有状态	Account API	Authorized application data　经授权的应用数据
External action　外部动作	Browser or API	Changes the environment　改变环境

工具并不是越多越好，把 30 个工具一次性塞进提示，会同时抬高四类代价：提示更长（延迟与成本）、候选更多（选错工具的概率上升）、权限面更大（风险）。

Call tools only when their benefit outweighs latency, cost, failure, and risk. Start with the abilities we want, then define a narrow interface for each kind of interaction.

只有当工具的收益超过延迟、成本、失败与风险时，才去调用它。从想要的能力出发，为每一类交互定义一个窄接口（接口窄，则参数少、校验简单、权限小）。

- latency 延迟：一次工具往返要多久？串行四次就是四倍等待。
- cost 成本：检索调用、API 调用通常按次计费，长任务里会累积。
- failure 失败：工具可能超时、限流、返回畸形数据，harness 要有兜底。
- risk 风险：对外部动作类工具（External action），副作用不可撤销。

第一类：harness 控制动作 finish(text)

Return the final text and stop the agent loop. This is a harness control action, not an external program.

智能体循环需要一个出口。返回最终文本并停止智能体循环。这是一个 harness 控制动作，不是外部程序。

第二、三类：检索、计算与生成

三类工具的共同骨架，三者的返回类型不同：

检索类返回的是文本证据（Three matching news items...）；计算类返回的是一个确定性数值（34.0）；生成类返回的是一个文件引用（generated_image.png）。

信息检索 search_web(query)

Retrieve information from the web. 从网上检索信息。

代码执行 execute_code(code)

Run a program in an isolated environment and return its output. 在隔离环境里跑一个程序并返回输出。

图像生成 generate_image(query)

Generate an image from a textual description. 从文字描述生成一张图。

第四类：应用专属工具 create_grocery_cart(items)

前三类工具是通用的（谁都能用），第四类是专属于某个应用的。

Expose an application-specific capability through one typed call.

用一次带类型的调用暴露一个应用专属的能力。

Tips：重点：带类型

好处：模型不需要读懂返回内容，只要按字段取用；下游代码也不需要解析自然语言，可以直接断言 items == 3、total == 31.42。这也是生成时约束的东西：JSON Schema 管住了输入的形式，而工具返回结构的约定管住了输出的形式

第五类：代码作为元工具 execute_code(code)

前四类工具都是一个工具做一件事，而代码是工具之上的工具，所以叫 meta-tool（元工具）。

Code combines control flow, libraries, and multiple operations into a single action.

代码把控制流、库与多个操作组合进一个动作里。control flow 是循环与分支（面包题里的连减）、libraries 是外部库（pandas）、multiple operations 是多次操作的编排（视觉问答三步）。

1.4 程序化组合：同一任务的两种智能体

左边：传统 LLM Agent（Text / JSON as Action，典型就是 ReAct）

Action = JSON 结构化文本调用工具，一次 Action 只能调用 1 个工具

流程：`Think → JSON Action调用单个API → Environment返回结果 → 重复循环`

任务：计算 4 个国家手机 CodeAct 1 的美元总价，找出最便宜国家

1. 第一轮：`lookup_rates(country=Germany)` → 拿到汇率、税率
2. 第二轮：`lookup_phone_price(model="CodeAct 1", country=Germany)` → 当地售价
3. 第三轮：`convert_and_tax(price, rate, tax)` → 换算 + 计税
4. 还要单独调用`estimate_shipping_cost`、`estimate_final_price`
5. 然后重复全部步骤，逐个国家（USA/Japan/India）循环调用 缺点：

- 需要非常多轮 LLM 调用、多步交互，每一步只能执行单一工具调用
- 大量重复的 Think-Action-Observe 循环，token 消耗高、速度慢
- 循环逻辑、批量计算、`min()`这种聚合操作都无法一步完成

右边：CodeAct Agent（Code as Action，代码作为动作）

Action = 一段可执行 Python 代码，单段代码内可以连续调用多个 API、写循环、保存中间变量、调用内置函数

优势（图中标注）

1. Fewer Actions Required：更少动作 / 更少 LLM 轮次。一次性写循环遍历全部 4 国，一次性完成全部 API 调用、计算、比较，不需要一步一步逐个工具交互
2. Control & Data Flow of Code Simplifies Complex Operations：代码自带循环、变量存储、分支逻辑，天然支持复杂流程
3. Re-use `min` Function from Existing Software Infrastructures(Python library)：直接复用 Python 内置函数，不用让 LLM 手写比较逻辑

	Text/JSON Action Agent (ReAct)	CodeAct (Code as Action)
Action type	JSON / 结构化文本	可执行 Python 代码
单次 Action 能力	只能调用单个工具	可以调用多个工具 + 循环 / 变量 / 内置函数
Interaction rounds	Many（多轮）	Much fewer（少轮次）

	Text/JSON Action Agent (ReAct)	CodeAct (Code as Action)
Strength	简单任务，易解析	复杂计算、批量任务、多步骤逻辑
Weakness	长任务下大量交互，容易出错	需要代码沙箱执行环境，有安全风险

CodeAct：LLM Agent 把动作表达为可执行代码，而不是 JSON 工具调用。

论文结论：代码作为动作，表达能力更强、减少交互轮次，处理多步计算类任务表现更好。

代码的两面性：动作空间宽带来的三个结果

维度	内容	对策
强 Expressive	Loops · variables · libraries 循环、变量、库 表达能力来自动作空间宽	Good for multi-step data manipulation and composition.
难管 Harder to constrain	Broad action space 动作空间宽 难约束也来自动作空间宽	Validation is less precise than for a narrow typed function.
危险 Higher impact	Files · network · processes 文件、网络、进程 高影响同样来自动作空间宽	Use sandboxing, resource limits, permissions, and audit logs.

四条对策：

sandboxing（沙箱）把执行环境隔离，让副作用出不了笼子。

resource limits（资源限额）限制 CPU、内存、时长，防住一个死循环拖垮整个 agent。

permissions（权限）决定这段代码能碰什么，是最小权限原则的落地。

audit logs（审计日志）出事后能回答它到底做了什么，这也是合规的前提。

Part 2 Mechanics 工具的机制

工具调用本质上就是**特殊标记（tokens）**，是大模型生成文本序列的一部分。

2.1 工具调用作为 token 序列

LLM 生成有两种输出分支：

1. Text continuation（普通文本续写）：正常自然语言回答，例：The weather is sunny.
2. Tool continuation（工具调用续写）：模型输出特殊控制标记，触发工具调用，例：<tool_call> get_weather(.)

模型不区分 “回答文字” 和 “调用工具”，两者都是模型生成的 token。 当模型输出工具标记，外部执行框架（harness）捕获到这个动作，去执行工具。

messages become model input

API 的多轮消息（system /user/assistant）不能直接喂给模型，需要通过 chat template 序列化为一整条 token 输入序列。

- API messages：按 role 组织（system、user、assistant）

- Chat template：加上特殊分隔符标签（如 Qwen 的<|im_start|>、<|im_end|>）拼接成一串文本，作为模型输入。

2.2 Tools add structure to the request 给请求增加结构化信息

Tool definition：提前把工具的 JSON schema 传给模型。定义工具名称、参数类型、必填字段（比如get_weather需要city字符串参数）

API request：调用 chat completion 接口时，除了messages，额外传入tools=[tool_schema]

框架把消息 + 工具定义一起送入模板，暴露给模型阅读。

Definition in, tool call out

Prompt 内部包含<tools> ... </tools>，里面是全部工具的 JSON 定义。 模型读到工具定义后，生成带标记的工具调用输出：<tool_call> {name, arguments} </tool_call> API 会给这个调用分配唯一call_id。

2.3 Same tool call, different model protocols

同一工具调用，不同模型有不同协议格式

One schema; model-specific serialization and parsing.

模型	协议名	形态特征
Qwen 3.8	Function + parameter tags	层层嵌套的标签：函数名与参数名都做成标签
Mistral Small 4	Control token + call ID	一个控制 token 加一段 JSON（JSON 里带 id）
DeepSeek V3.2	DSML invocation block	一套 DSML 标签，参数上还带类型标注

工具的底层 schema 是统一的，但是不同模型家族，使用不同特殊 token / 标签序列化工具调用。

重点：每个模型有自己专属的控制 token 格式。Agent 框架必须为不同模型写对应的 parser，把模型输出的文本解析成统一的{name, args}结构。

2.4 OpenHands 工具调度流程

对照项	初始化阶段	每次调用阶段
图上编号	1 Register at initialization	2 Dispatch each model call
发生几次	一次	每次模型调用一次
主要动作	resolve_tool(spec, state)	Parse → Lookup → Validate → Execute → Return
产出	tools_map[name]	ObservationEvent(call_id)
单次代价	高：解析 schema、构造类型、准备执行器	低：查表、校验、执行

两大阶段：

Register at initialization（初始化注册阶段）

1. 输入 Tool spec（工具描述，如 BashTool）
2. resolve_tool(spec, state)：解析、配置工具
3. 存入tools_map[name]：字典映射，工具名 → 工具定义 + 执行器 executor

Dispatch each model call（每一次模型输出后的调度）

1. Parse：解析模型输出，得到{name, args}
2. Lookup：在tools_map查找对应工具定义
3. Validate：校验参数合法性（参数类型、必填项）
4. Execute：调用 executor 执行工具 action
5. Return：封装结果为ObservationEvent(call_id)返回

步骤	表示	作用	失败后果
Parse	{ name, args }	从模型输出里抽出函数名与参数	The harness cannot parse it.
Lookup	tools_map[name]	按名字查阶段一注册好的表	查不到就是未知工具
Validate	action_from_arguments(args)	把裸参数转成强类型 Action	类型或必填项不符就校验失败
Execute	executor(action)	真正执行，产生副作用	抛异常则变成一次失败结果
Return	ObservationEvent(call_id)	把结果打包成事件，带上call_id	不带 ID 就无法配对

2.5 通过 call ID 返回工具结果

结果里为什么一定要带 call_id?

用来绑定工具调用和返回结果，支持并行多工具调用（parallel tool calls），保证结果对应正确的调用。

一次模型响应可能带多个调用，harness 可能并发执行它们，完成先后取决于各工具快慢，而不是发出顺序。如果结果按第几个回来配对，一旦乱序就会把 A 的结果填给了 B，而且这种错误不会报错，只会让模型拿到错的数据继续往下推理。只有 ID 能让结果回到正确的调用上。

Example：若同时发出两个调用（例如查两个城市的天气），harness 可能并发执行，谁先完成取决于网络与工具耗时。

没有 ID：只能假设先发的先回，一旦乱序就把 A 的结果填进了 B。这种错误不会抛异常，只会让模型拿着错的数据继续推理，这是最难排查的一类 bug。

有 ID：无论谁先回来，都能放回正确的位置。

2.6 小结

工具调用完整流程：

1. 系统将工具 schema 渲染进 prompt，放在<tools>标签内送入模型。
2. 模型读取工具定义，按照自身专属协议输出<tool_call>（携带工具名、参数、唯一call_id）。
3. 外部执行框架 harness 处理： Parse：解析模型输出；Lookup：查询注册好的工具映射表；Validate：校验参数合法性；Execute：运行外部工具；Return：封装工具返回内容。
4. 工具返回tool 角色消息：role="tool"，tool_call_id与之前调用的 id 匹配，携带结果content。
5. 把工具结果放回对话上下文，模型读取观测结果，生成最终自然语言回答。

完整一轮工具交互链路：

1. assistant：tool_call（发起调用，带 call_id）
2. tool：tool-result（返回观测，匹配 call_id）

3. assistant：最终文本回答

机制要点：call_id 的作用是绑定调用和返回结果，支持并行工具调用；工具调用本质是模型生成的特殊 token，不同模型协议标签格式不一样。

易错：

1. 模型不知道自己的 call ID；
2. tool call 不是函数调用，是一段字符串；
3. 结果靠 tool_call_id 配对，不靠顺序（including parallel calls）。

Part 3 Constrained Decoding 工具的约束：只保证形式合法

JSON Schema 校验工具调用的结构合法性；PDA 用来处理 JSON 嵌套；Token Masking 在解码阶段提前阻止模型输出非法 token，减少 malformed tool calls。

事后校验与事前约束的分工

做法	对应	时机	代价
事后校验	The harness cannot parse it.	整段生成完之后	整段输出作废，需要重试，多一次完整往返
事前约束	Constrained Decoding	生成的每一步	每步多一次掩码计算（CPU 开销）

解码层保证结构上能解析：畸形括号这类问题。

harness 保证业务上能用：Validate（action_from_arguments）。

但要注意约束的射程：受约束解码消除的是形式类错误。{"city":"PittsburghX"} 是合法 JSON、类型也对，城市名却是错的，那已经越出形式约束的范围，要靠Values 甚至更外层的业务校验。

3.1 Malformed tool calls 格式错误的工具调用

模型输出语法破损的 JSON，例如少闭合大括号，harness无法解析，直接调用失败。

Tool-call constraints 工具调用的四层约束（后一层以前一层为前提）

约束	约束	作用	例子
Syntax	valid JSON	合法 JSON	缺一个 } 就不是 valid JSON
Shape	expected fields	字段符合预期	name, arguments 等必填 key 存在
Types	city is a string	参数类型匹配 schema	city 必须是字符串 string
Values	值在允许范围内	值在允许范围内	units 只能是`C`或`F`

Constraints ensure form—not truth, permissions, tool choice, or task success.

约束只保证格式合法，不能保证工具返回结果是真的，也不能保证选对工具。

四类约束执行时机

约束	解码时（掩码能做吗）	派发时（harness 做什么）
Syntax	能：字节级语法掩码	解析失败即拒：The harness cannot parse it.

约束	解码时（掩码能做吗）	派发时（harness 做什么）
Shape	能：required 编译进文法	缺字段则 Validate 失败
Types	能："type": "string" 进文法	action_from_arguments 里做类型转换与检查
Values	能：enum 限制候选取值	业务层可再校验一次
truth / permissions / tool choice / task success	不能	需要额外机制：接口与凭据、评估

Tips：加了 JSON Schema 就绝对安全吗?

真伪要靠工具自身返回真实数据，以及额外的交叉验证；权限要靠 harness 的白名单与凭据边界；选对工具要靠评估里的 selection 指标。任务成功要靠端到端的结果评测。

3.2 JSON Schema

Vocabulary for describing JSON data. Definite Types, required fields, allowed values

JSON Schema 是一套用来描述 JSON 数据结构的机器可读规则，定义字段类型、必填字段、允许枚举值（enum）、禁止额外字段。

示例：city:string，units: enum ["C","F"]，required:["city"]，additionalProperties:false

Schema-valid vs schema-invalid 是否可以推导出来derives

Valid：{"city":"Pgh","units":"F"} 结构、类型、取值全部匹配 schema

Invalid：{"city":"Pgh","units":"K"} JSON 语法没问题，但K不在允许枚举集合里，schema 校验失败

1. JSON valid：文本语法能被 JSON 解析器读
2. Schema valid：不仅语法对，数据结构、类型、取值完全符合预先定义的 schema

3.3 A pushdown automaton** **PDA，下推自动机

PDA = **有限状态控制 Finite control + 栈 Stack memory**

3.3.1 两个部件

部件	原句	记住什么	记不住什么
Finite control 有限状态控制器	The state records the local position in a grammar rule.	当前文法规则内部走到哪一步（expect value / inside value / expect separator）	嵌套层级有多深
Stack memory 栈存储	The stack remembers arbitrarily deep nesting.	每一层嵌套对应的结构类型（object /array），支持任意深度 arbitrarily deep	当前这条文法规则内部的局部位置

State tracks current rule; stack remembers where nested rules return.

状态跟踪当前规则；栈记住嵌套结束后要返回的上层规则。栈可以无限增长，有限状态做不到。

栈为空 = 所有括号配平。

底层原理： 纯有限状态机（正则表达式）无法表达任意深度的配对括号，只能处理固定深度嵌套。JSON 存在无上限嵌套，所以必须使用 PDA。

3.3.2 文法检查 Grammar checking happens below tokenization

语法检查运行在tokenization 底层，字节层面

- Token 边界 ≠ 语法边界：单个 token 可能跨多个语法终结符，或者在文法中间截断。不能把 token 边界当成语法边界。
- 流程：预先编译文法 → 构建字节级 PDA；对每一个候选 token，解码为字节，送入 PDA 校验。

示例：PDA 栈演算

1. 读取 `{` → push `object`，栈：`[object]`（深度 1）
2. 读取 `[` → push `array`，栈：`[object, array]`（深度 2，array 在栈顶）
3. 读取 `]` → pop array，栈：`[object]`（深度 1）
4. 读取 `}` → pop object，栈：empty（深度 0）

最终栈为空：括号配平； 缺`}`（malformed 例子）：栈停留在`object`，不为空，直接判定语法错误。仅机械栈操作，不需要理解语义。

3.3.3 PDA 能力边界

对接前面 4 类 tool-call 约束：Syntax / Shape / Types / Values

PDA 可以校验：

- Syntax：括号、引号配对（JSON 配平，本职）
- Shape：必填字段、合法字段名
- Values 里的有限枚举（`units ∈ {"C","F"}`）

上面这些都可以写成上下文无关文法的产生式

PDA **无法校验**：跨字段业务语义、外部事实知识

例：`units="C"` 时 city 必须是加拿大城市，需要外部知识库，不属于文法范畴。

Constraints ensure form—not truth, permissions, tool choice, or task success.

约束只保证**形式合法**，不能保证事实真伪、权限、工具选择、任务是否成功。

3.4 Token masking 令牌掩码

问题：非法 token 判定依赖 PDA 栈状态，如果每次生成 token 都完整重算，开销很高。

缓存掩码流水线思路：大部分 token 查表缓存，少数上下文依赖的 token 现场实时计算。

1. **Parse the prefix**

prefix = 已经生成输出的字节序列。persistent PDA stacks持久化栈，不需要每次从头重建栈，直接在已有栈状态上，用已经输出的字节向前推进状态。

解决：长序列每次重建栈带来的平方复杂度开销。

2. **Fetch cached decisions **

栈顶对应文法节点，索引预计算好的缓存表，直接查询上下文无关 token（context-independent tokens）是否合法。核心操作就是查表，速度很快。

对应图：Retrieve token mask cache from stack top / Adaptive Token Mask Cache

3. **Check the minority **

只对少量上下文依赖 token（context-dependent tokens），拿完整 PDA 栈现场校验。

union results across matching stacks：存在多分支、多条并行栈状态时，把多条栈得到的合法性结果做并集。

对应图：Context-dependent tokens (minority，黄色窄条)

4. **Mask and continue **

合并查表 + 现场校验得到的全部判定结果；对模型 logits 施加 token 掩码；采样选出一个 token；更新 PDA 栈。更新后的栈，交给下一轮Parse the prefix使用，四步形成闭环。

对应图：Complete Token Mask，输出 Sampled Token

三种实现策略取舍

策略	代价	缺点
全部缓存	按栈状态 × 词表 V预计算	栈状态组合爆炸，缓存表容量不可行
全部现场计算	每一个候选 token 都完整跑 PDA	词表 V 上万，CPU 计算开销巨大，拖慢 GPU 推理
缓存 + 少数现场计算	预缓存大部分，仅少数 context-dependent 实时校验	需要处理分支多栈、结果并集逻辑

定性代价模型

词表大小V； m = 上下文无关、可以查表的 token 占比； $1 - m$ = 少数需要现场计算的 token 占比。

- 全现算：每一步成本正比于 V
- 缓存方案：每一步成本 ≈ 一次查表 + 正比于 $(1 - m)V$ 的现场校验

加速幅度近似由m决定；m越大，加速效果越好。

课件只定性：成本随 minority 占比线性下降，不给出具体数值。

配图：绿色（允许）+ 红色（拒绝）占绝大部分词表；黄色上下文依赖 token 只是很小一部分，m很大。

persistent 持久栈的关键意义：

如果不使用 persistent 栈，每次生成新 token，把全部 prefix 字节从头重新跑一遍 PDA。总开销随生成长度平方增长，越长越慢；Persistent 栈只对新增字节推进栈状态，单步是常数复杂度。 所以这套四步流水线不只是用来生成掩码，还解决了长文本生成过程中掩码计算越来越慢的性能隐患。

LLM 推理输出 logits，token mask 是 0/1 掩码：

- 1：允许采样该 token；0：屏蔽，logits 置−∞，softmax 后概率归零
- 流程：LLM 输出原始 logits → 乘 mask 得到 masked logits → softmax 得到概率分布 → sampler 采样 token

3.5 Cached token-mask generation 带缓存的掩码生成，加速方案

1. Parse the prefix：用已经生成的字节，推进持久化 PDA 栈
2. Fetch cached decisions：栈顶节点直接读取预计算缓存，大部分 token 是上下文无关的，直接查表
3. Check the minority：少数上下文依赖的候选 token，用完整 PDA 栈实时校验
4. Mask and continue：合并掩码，屏蔽非法 token，采样新 token，更新 PDA 栈

Case study：加速来源

约束只管形式，加速靠缓存与重叠

1. **Caching**：绝大多数 token 是上下文无关，直接读取预计算 mask，不用实时跑 PDA（大头提速）
2. **Persistent stacks**：持久化 PDA 栈，支持分支、多候选路径
3. **CPU/GPU overlap**：CPU 做语法 / 掩码计算，和 GPU LLM 推理并行，不阻塞模型生成

设计	图上	含义
栈的持久性	Maintained by persistent execution stacks	栈跨步保留，不做每步重建（persistent）
栈的分支	Matching Stack States 里同时列出 [0]<array> 与 [3]<str>	不确定走哪条分支时同时维护多条栈，最后取并集（union results）
token 级查表	Retrieve token mask cache from stack top	查表的键是栈顶节点（Node ）
与推理重叠	Overlapping with LLM engine	CPU 侧的掩码计算与 GPU 侧的推理并行，不是串行等

每步总成本 ≈ cache 查询 + minority × 现场判定

Caching handles most tokens; 大多数 token 查缓存;

persistent stacks support branching; 多栈并行，使不确定走哪条分支不至于卡住;

CPU work overlaps GPU inference. 把 CPU 的工作与 LLM engine 重叠。

多数段直接查缓存（Context-independent: Fetched from cache (fast, majority)）

少数段运行时判定（Context-dependent: Checked at runtime (slow, minority)）

Part 4 REST API 工具接口

4.1 Library API

4.1.1 API 的本质

Python Library API 是程序内部暴露给其他代码使用的编程接口。

Caller and function share one process. Python supplies the name, arguments, types, and return type.

Python 库函数是带类型的编程接口，不是网络服务。示例 get_weather，调用方和函数共享同一个进程、共享内存地址空间。 Python 直接提供函数名、参数、类型、返回类型，直接传内存对象，不需要地址、协议、序列化、认证。

4.1.2 局限与前提

局限：不能远程调用，而进程内调用靠共享地址空间，跨网络没有共享内存。

所以有跨网络调用硬性需求：

1. `address`地址：远端服务的位置
2. `protocol`协议：两端交互报文约定
3. `serialization`序列化：内存对象编码为网络字节流
4. `authentication`认证：验证调用者身份

缺件	进程内为什么不需要	跨网络为什么必须有
address 地址	同一进程里，函数名就是入口	服务在另一台机器上，得先知道它在哪里
protocol 协议	Python 的调用约定就是协议	两端可能不是同一种语言，要约定报文格式与交互次序
serialization 序列化	参数就是内存里的对象，直接传引用	网络只能传字节，对象必须先编码成可传输的形态
authentication 认证	同进程内没有谁在调用这个问题	任何到达端点的请求都必须先证明自己是谁

Pydantic BaseModel 定义返回结构，这套类型标注会一路传递：Python 类型 → JSON 响应 → OpenAPI JSON Schema → MCP 工具 schema。

4.1.2 缺点

不能直接远程调用：进程内部可以直接使用 Python 函数调用机制。但如果调用方在另一台计算机上，就无法直接使用同一个内存地址空间。因此，远程调用必须解决四个问题：

问题	进程内部	跨网络调用
Address	通过函数引用找到代码	必须知道远程服务地址
Protocol	Python 调用约定	必须约定通信协议
Serialization	直接传递 Python 对象	需要编码为可传输数据
Authentication	由进程和应用权限控制	通常需要验证远程调用者

其中，Serialization（序列化）尤其重要。

例如，Python 对象：

```
Weather(    summary="Sunny",    temperature=75.2)
```

通过网络传输时，可以表示为 JSON：

```
{
  "summary": "Sunny",
  "temperature": 75.2
}
```

这里 JSON 是一种数据表示格式，不是网络协议本身。

本节结论：Library API 是 typed programming interface，而不是 network service。

4.2 REST APIs：通过 HTTP 暴露函数

4.2.1 REST API 的核心思想

REST API 使外部客户端可以通过 HTTP 请求访问服务端提供的资源和操作。

在课程案例中，我们希望将：get_weather("Pittsburgh", "F")

转换成远程请求：GET /weather?city=Pittsburgh&units=F

它们实现类似的业务功能，但使用不同的调用方式。

4.2.2 FastAPI 如何实现这一转换?

REST APIs expose functions over HTTP

把库函数挂到一个 URL 上，让它可以被一次 HTTP 请求触发。用 FastAPI 把本地库函数绑定到 HTTP 端点，对外提供网络调用能力。

元素	代码
GET /weather	@app.get("/weather", ...) HTTP 方法与路径
city =Pittsburgh	city: str 这个参数，以查询参数形式传进来（课件原样多印了一个空格）
units F 与孤立的 =	units: Literal["C", "F"] = "F" 参数名、取值、默认值被版式拆散了
Header: X-API-Key	APIKeyHeader(name="X-API-Key") 凭据走 HTTP 头，不走 URL
Response: Weather as JSON	-> Weather 返回类型被序列化成 JSON

注意：`verify(key)` 是验证函数。

第一：创建 HTTP Endpoint

- `GET`：HTTP Method。
- `/weather`：Endpoint Path。
- `get_weather`：OpenAPI 中的 Operation ID。

特别注意：Python 路由函数名是 `weather`，但 Operation ID 被显式指定为 `get_weather`。这让后面 OpenAPI 与 MCP 工具映射可以使用一致的操作名称。不过，`operationId` 并不要求必须与底层 Python 函数同名。

第二：解析请求参数

city: strunits: Literal["C", "F"] = "F"，FastAPI 根据参数声明解析 Query Parameters。

请求示例：/weather?city=Pittsburgh&units=F

第三：获取 API Key 并认证

key_header = APIKeyHeader(name="X-API-Key")

X-API-Key 放在 HTTP header；FastAPI 只取出 key，业务代码 verify (key) 负责校验合法性。

客户端在 HTTP Header 中提交：X-API-Key: YOUR_API_KEY，然后 key: Annotated[str, Security(key_header)] 表示从指定的安全依赖中取得 Key。

FastAPI 负责提取该凭证；示例中的 `verify(key)` 负责判断它是否有效。

- 声明类型：str 这个值是一个字符串。

- 声明来源：Security(key_header) 它不从查询串来，而是由 APIKeyHeader(name="X-API-Key") 从 HTTP 头里取。
- 声明职责：函数体里 verify(key) 负责校验，框架不替你做这一步，FastAPI 只管把值取出来，管不管用是你的事。

第四：执行函数并返回 JSON

请求：GET /weather，city/units放在 query 参数

返回：Python Weather 对象序列化为 JSON

verify(key)return get_weather(city, units) 服务器先检查凭证，再调用 Python 库函数。

返回的 `Weather` 对象由 FastAPI 序列化为 JSON 响应。

FastAPI binds the library function to a REST endpoint and documents its API key header.

FastAPI 将库函数绑定到 REST 端点，并且自动生成包含 API 密钥头的机器可读文档。即端点 + 说明书。

4.3 OpenAPI：API 的机器可读说明书

4.3.1 OpenAPI 的作用

有了 REST API，客户端已经可以调用天气服务。但 Agent 仍然需要知道：有哪些 Endpoint？使用 GET 还是 POST？参数叫什么？参数是否必填？数据类型是什么？如何认证？OpenAPI 就是用来描述这些信息的规范。

4.3.2 OpenAPI 的结构

OpenAPI describes the HTTP operation; embedded JSON Schema describes its values.

- OpenAPI 负责：HTTP 路径、请求方法、参数位置、安全方案
- JSON Schema 负责：类型、枚举、必填、默认值
- required 默认 false；in="query" 只是参数位置，**不等于必填**，二者相互独立。
- OpenAPI 是发布物，是框架自动生成的产物。

三部分：

1. Operation

表示有一个 `GET /weather` 操作，其标识为 `get_weather`。

2. Parameters

OpenAPI 字段	含义
name	参数名称
in	参数的位置
required	参数是否必填
schema.type	参数类型
schema.enum	允许的取值
schema.default	默认值

`in: query` 表示参数来自 URL 查询字符串，并不代表该参数必填。

3. Security

- 认证方案是 API Key。
- API Key 位于 HTTP Header。
- Header 名称是 `X-API-Key`。

4.3.3 OpenAPI 与 JSON Schema 的区别

OpenAPI	JSON Schema
描述 HTTP API	描述 JSON 数据结构
HTTP Method	数据类型
URL Path	枚举值
参数位置	对象属性
认证方案	数据约束
请求和响应结构	数据格式与验证规则

OpenAPI 可以在参数和请求、响应模型中使用 JSON Schema。

因此，课程中有一条重要的数据结构演变路线：Python Type Hints / Pydantic；FastAPI Response & Validation；OpenAPI + JSON Schema；MCP Tool Input Schema。这解释了为什么已有的 OpenAPI 描述可以被 FastMCP 复用。

4.4 curl 直连

4.4.1 curl 的作用

`curl` 是命令行 HTTP 客户端。

参数	含义
-sS	减少输出，同时保留错误信息
-G	将数据参数添加到 URL Query
--data-urlencode	URL 编码参数
-H	设置 HTTP Header
$WEATHER_API_KEY	从 Shell 环境变量读取密钥

Agent 可以根据 OpenAPI 文档构造并执行这个命令。

4.4.2 Agent 的三步调用过程

三个步骤及失败模式

步骤	说明	典型失败
1 Read the API description	Select the operation and required parameters. 读说明书，挑出要调哪个操作、需要哪些参数	读的是过期的文档，调用了一个已经不存在的操作或改了名的参数

步骤	说明	典型失败
2 Construct the command	The shell expands the environment variable; curl sends HTTP. shell 展开环境变量，curl 负责把 HTTP 发出去	环境变量没导出（展开成空串），请求带着空凭据打到服务端
3 Limit the credential	A coding agent with shell access may also read or misuse that secret. 能进 shell 的智能体也能读到、甚至滥用密钥	凭据权限过大（不只读天气，还能改别的资源），一旦泄露损失放大

Step 1 Read the API Description

读取 OpenAPI，找到正确的 Endpoint、参数和认证方式。

Step 2 Construct the Command

根据接口要求构造 curl 命令。

Shell 展开环境变量，curl 发送 HTTP 请求。

Step 3 Limit the Credential

尽量限制 API Key 的权限、有效期和使用范围。

为什么需要第三步？因为运行 curl 的 Shell 环境需要取得 API Key。如果 Agent 能在这个环境中任意执行 Shell 命令，就可能读取或滥用该凭证。

Tips：

与MCP路线的区别：同一个需求，两条路线的第一反应

REST 直连：怎么把命令写对。凭据问题被当成一个注意点，写在 Limit the credential。

MCP ： 怎么让凭据不进入会执行代码的那一侧。所以它先引入服务端，再谈工具清单。

4.4.3 安全问题的本质

存在凭证暴露风险：`WEATHER_API_KEY` 可由 Shell 访问。

这里并不是说 curl 不安全，而是说如果 Agent 的执行环境同时拥有广泛的 Shell 权限和敏感凭证，安全边界就比较弱。这为下一章 MCP 的凭证隔离设计提供了动机。

4.5 本章小结

本章建立了下面这条链路：

Python Library → FastAPI → REST API → OpenAPI → curl / HTTP Client

它解决了三个核心问题：

1. 如何让内部函数能够被远程调用。
2. 如何让机器理解 API 的调用方式。
3. Agent 如何直接调用 HTTP API，以及这种方式可能带来的凭证风险。

三步流程：

1. Read the API description：读取 OpenAPI 文档，选择操作、确定参数
2. Construct the command：shell 展开环境变量，curl 发送 HTTP 请求

3. Limit the credential：密钥暴露在 shell 环境，一旦泄露风险高；只能做最小权限限制，无法完全隐藏。限制它的最小权限、短期有效、随时可撤销。

Direct REST access is simple, but the agent's shell receives the API credential.

核心结论：直接访问 REST 简单，但智能体所在 shell 会拿到 API 凭据，存在密钥泄露风险。

MCP 诞生的核心动机：不让密钥落到 shell/agent 一侧，把凭据放在服务端。

六组对照（Part4&5）：

关键词	必须理解的内容
Library API → REST API	从进程内部调用变成网络调用
FastAPI → OpenAPI	从 HTTP 服务变成机器可读的接口描述
OpenAPI → FastMCP	从 REST 操作变成 MCP 工具
MCP Host → Client → Server	AI 应用与外部工具的通信架构
tools/list → tools/call	先发现工具，再执行工具
MCP Key → Upstream Key	客户端认证与上游认证分离

Part 5 Model Context Protocol

5.1 MCP 基础概念

核心问题：假设有 N 个 AI 应用和 M 个外部工具，如果每个 AI 应用都需要单独适配每个工具，可能产生 N × M 套集成工作。例如：Without MCP：不同 AI 应用分别适配不同工具；MCP 通过标准化工具接口减少重复适配。

即在没有MCP之前，连接 N 个大模型 (LLM) 和 M 个外部工具 / 数据源，是 N×M 的全连接问题。每一个 LLM 都要单独对接每一个工具，对接成本随数量爆炸式增长。

解决：With MCP：使用统一协议进行工具集成，课程使用 N→1→M 来描述这一思想。但要注意，它表示的是统一接口标准，而不是所有工具必须经过同一个中央服务器。实际系统可以包含多个 MCP Server。

即引入 MCP，MCP 将 LLM 和外部工具之间的通信标准化，变成 N→1→M 的星形架构：所有 LLM 只对接 MCP 中间层，MCP 再统一对接所有外部工具；大幅减少集成工作量。

MCP 的主要作用是：

- 统一工具发现方式。
- 统一工具调用方式。
- 减少重复集成工作。
- 支持不同类型的外部能力。
- 允许通过服务端隔离上游凭证。

MCP 不是 REST API 的必要替代品。

对于只需要调用少量 HTTP API 的简单系统，直接使用 REST 也完全合理。

5.2 MCP 的架构：Host、Client、Server

架构：客户端 - 服务器模型（Client-Server）

三个角色

角色	本质	作用
host	一个应用／进程	承载模型循环与用户界面，持有多个client
client	一条连接的连接器	只负责把 host 的意思翻译成 MCP 报文并送出去
server	一个能力提供方	把外部的工具与数据源按 MCP 暴露出来

host 并不是服务端：host 与 server 方向正好相反：host 在客户端一侧，server 是提供能力的一方。MCP is an alternative agent-facing interface 里的 agent-facing，指的正是 host 这一侧看到的东西，接口是给 host 里的智能体用的。

- **MCP Client**：需要调用工具 / 读取数据的 Agent（嵌入在 AI 主机 / LLM 应用内部）
- **MCP Server**：对外暴露外部工具、数据源的服务，包装底层能力给 LLM 调用
- **MCP Host**：承载 LLM + MCP Client 的 AI 应用程序（如 Claude、DeepSeek 这类 AI 应用）
- **Without MCP**：LLMs 和工具两两直连，网状结构；With MCP：所有 LLM 连接 MCP 中间节点，MCP 再连接全部外部工具

Tips：工具（tool）是可以被调用的动作，资源（resource）是可以被读取的数据。

5.2.1 MCP Host

Host 是承载 AI 应用的整体环境，它可以管理多个 MCP Client。例如：

```
AI Application
├── LLM
├── MCP Client A
└── MCP Client B
```

5.2.2 MCP Client

MCP Client 负责：与 Server 建立连接；发现工具；发送工具调用请求；接收工具结果。需要注意，MCP Client 不等于 LLM。LLM 负责推理和工具选择，Client 负责实际协议通信。

5.2.3 MCP Server

MCP Server 负责暴露外部能力。

例如：文件系统工具；数据库查询工具；GitHub 操作工具；REST API 封装工具；MCP Server 可以部署在本地，也可以部署在远程。

5.3 MCP的两种传输方式

MCP is an alternative agent-facing interface—not that is not a requirement for calling REST APIs.

MCP 是一个面向智能体的替代接口，它并不是调用 REST API 的必要条件。

MCP Host（AI 应用）可以同时创建多个 MCP Client，分别对接不同类型 MCP 服务：

1. Client A → LOCAL: STDIO Filesystem server（本地文件系统 MCP 服务）

2. Client B → REMOTE: HTTP Sentry server（远程 HTTP 的 Sentry 服务）

MCP 是面向 Agent 的接口方案；不是调用 REST API 的必需替代品，是一套上层封装协议。

元素	层级
MCP HOST	宿主机：跑 AI 应用的整体
AI application	宿主机里的应用：AI 应用本身
Client A / Client B	客户端：host 内部为每一个服务端各开一个客户端
LOCAL: STDIO + Filesystem server	服务端与传输：本地服务端，走 STDIO（标准输入输出）
REMOTE: HTTP + Sentry server	服务端与传输：远程服务端，走 HTTP

host 与 client 之间是一对多的关系。host 里同时挂着多个客户端，每个客户端各管一条连接，互不干扰。而 client 与服务端之间是一对一的。

stdio 是 Standard Input / Standard Output。

典型特点：通过本地进程的标准输入输出通信；通常不需要网络端口；适合本地文件、开发工具等能力。

HTTP Transport 用于通过 HTTP 进行 MCP 通信。

典型特点：可以连接远程服务器；支持网络部署；可以结合 HTTPS 和认证机制。

重点：MCP 定义上层交互协议，stdio 和 HTTP 是承载消息的传输方式。

同一个 Host 可以同时连接使用不同 Transport 的 MCP Server。

传输	配对例子	它在哪一侧	适合什么
LOCAL：STDIO	Filesystem server	同一台机器上	本地文件、本地进程这类不出机器的能力
REMOTE：HTTP	Sentry server	远端	第三方服务、SaaS 平台这类跨网络的能力

传输方式不是偏好，是被能力的位置决定的。比如Filesystem server（文件系统服务端）要读的是本机文件，把它放到远端反而多绕一圈，所以走STDIO，而 Sentry server（Sentry 是一个 SaaS 错误监控平台）本体就在网上，所以走 HTTP。

所以，同一个 host 里两种传输可以并存，这正是 client-server model 的价值：接口统一，底下的连接方式可以不同。

5.4 MCP 完整调用流程（时序图）

对应课程时序图的十个步骤：

1. User 向 MCP Host 输入 Query。
2. LLM 根据问题选择工具。
3. 模型输出工具调用信息。
4. Host 根据策略决定是否需要用户审批。
5. MCP Client 向 MCP Server 发送请求。
6. MCP Server 执行工具或调用第三方 API。
7. 底层工具产生结果。
8. MCP Server 将结果返回 Client。

9. Host 将工具结果提供给 LLM。
10. LLM 生成最终自然语言回答。

注意，用户审批是可能出现的安全控制，不是每次调用都必须发生。

5.4.1 两个关键 MCP 方法

工具发现：tools/list，用于获取服务器提供的工具及其 Schema。

工具调用：tools/call，用于指定工具名称并传入参数。

例如：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "city": "Pittsburgh",
      "units": "F"
    }
  }
}
```

5.5 MCP 三元素的关系

MCP is an alternative agent-facing interface—notn't a requirement for calling REST APIs.

MCP 是一个面向智能体的替代接口，它并不是调用 REST API 的必要条件。

MCP Host（AI 应用）可以同时创建多个 MCP Client，分别对接不同类型 MCP 服务：

1. Client A → LOCAL: STDIO Filesystem server（本地文件系统 MCP 服务）
2. Client B → REMOTE: HTTP Sentry server（远程 HTTP 的 Sentry 服务）

MCP 是面向 Agent 的接口方案；不是调用 REST API 的必需替代品，是一套上层封装协议。

元素	层级
MCP HOST	宿主机：跑 AI 应用的整体
AI application	宿主机里的应用：AI 应用本身
Client A / Client B	客户端：host 内部为每一个服务端各开一个客户端
LOCAL: STDIO + Filesystem server	服务端与传输：本地服务端，走 STDIO（标准输入输出）
REMOTE: HTTP + Sentry server	服务端与传输：远程服务端，走 HTTP

host 与 client 之间是一对多的关系。host 里同时挂着多个客户端，每个客户端各管一条连接，互不干扰。而 client 与服务端之间是一对一的。

5.6 FastMCP：将 OpenAPI 映射成 MCP Tools

5.6.1 FastMCP

如果已经有 REST API 和 OpenAPI 文档，就不一定需要重新编写一套工具定义。x

默认行为：FastMCP 会把 OpenAPI 里每一条路由，自动变成一个可被 LLM 发现的 MCP 工具。

执行逻辑：

1. 使用 `httpx.get()` 获取 OpenAPI 文档。
2. 解析文档中的 API 描述。
3. 创建用于访问原始 REST API 的 HTTP Client。
4. 通过 `FastMCP.from_openapi()` 构建 MCP Server。
5. 将 OpenAPI 操作映射为 MCP 能力。

5.6.2 自动映射的优点

- 复用已有 OpenAPI 描述。
- 减少重复编写工具 Schema。
- 降低 REST API 与工具定义不一致的风险。
- 适合快速接入已有服务。

5.6.3 自动映射的局限

问题一：工具数量可能过多

如果 OpenAPI 包含上百个操作，自动映射可能产生大量 MCP 工具。这会增加工具发现、上下文管理、模型选择和权限管理的复杂度。同时放大四类代价：延迟（latency）；成本（token / 计费 cost）；失败概率（failure rate）；安全风险（risk）

问题二：接口名称不一定适合 LLM

仅做结构层面映射，不做语义优化 映射只看 OpenAPI 结构，不理解业务语义。

接口命名清晰（如GET /weather）：自动生成的工具 get_weather 效果很好；

接口命名模糊、语义不清：转换后的工具，LLM 很难判断什么时候该调用，选择准确率下降。

适合场景：API 接口数量少、接口命名清晰规范的场景，自动映射性价比很高。 当接口数量庞大时，自动映射只是第一步，生成工具列表之后，必须额外增加一层筛选、分组能力，精简暴露给 LLM 的工具集合。

例如：GET /weather，转换成：get_weather 含义非常清晰。但如果原始 API 命名不规范：POST /v2/action/process 即使成功转换，LLM 也未必能判断什么时候应该调用它。

因此，实际工程中可能需要：

- 筛选工具。
- 改善工具名称和描述。
- 按业务场景分组。
- 对敏感操作设置权限。

注意：具体默认映射行为取决于 FastMCP 的版本与配置，不能笼统认为任何版本都会把所有路由无条件暴露为工具。

5.7 Credential Separation：独立凭证机制

5.7.1 独立凭证的作用

直接 REST 调用的问题

Agent 的执行环境需要取得上游 API Key。

使用 MCP 后

	MCP_API_KEY	UPSTREAM_API_KEY
使用者	MCP Client	MCP Server
验证方	MCP Server	REST API Server
保护对象	MCP 服务入口	上游 API
是否需要交给模型	不需要	不需要

5.7.2 Server 端代码

核心思想：MCP 客户端密钥 和 上游第三方 API 密钥互相隔离，MCP Server 作为中间代理，做凭证中转。

MCP Server 启动：两套密钥

- UPSTREAM_API_KEY：用来调用上游第三方 API（天气服务）
- MCP_API_KEY：用来保护 MCP 服务端点，供 MCP 客户端接入

> 教学演示使用静态 token；生产环境推荐 JWT 或者 OAuth。

代码可以分成三个逻辑模块：

模块 A：上游 HTTP Client，使用 `UPSTREAM_API_KEY` 访问原始天气 API。

upstream = httpx.AsyncClient(...)

模块 B：MCP Authentication，验证 MCP Client 提交的凭证。

mcp_auth = StaticTokenVerifier(...)

模块 C：MCP Server，把工具映射、上游 HTTP Client 和 MCP 认证机制组合起来。

mcp = FastMCP.from_openapi(...)

5.7.3 Client 端代码

MCP Client 侧：只持有 MCP 密钥

凭证代理（Credential brokering）

- Agent/Harness 只持有 MCP key，不知道上游 API 密钥
- MCP Server 负责：验证 MCP 客户端身份 → 授权 → 使用独立的上游密钥去调用第三方 API
- 优势：大模型本身看不到任何上游密钥，密钥不会暴露给 LLM，安全隔离。

两个关键调用：

await client.list_tools()，发现工具。

await client.call_tool(...)，执行工具。

整个过程中，Client 不需要获取 `UPSTREAM_API_KEY`。这就是 Credential Brokering（凭证代理）。

5.7.4 安全边界

Separate credentials allow the MCP server to mediate access rather than forward its upstream secret.

独立凭证让 MCP 服务器充当访问中介，而不是把上游密钥转发出去。

Agent 不知道上游 API Key不等于Agent 完全接触不到任何凭证。MCP Client 仍然持有 MCP_API_KEY。如果 Agent 拥有对该客户端环境的广泛访问权限，这个凭证也可能暴露。

所以，MCP 可以把上游凭证隔离在 Server 端，但 MCP 本身并不自动保证所有凭证安全。生产环境还需要身份认证、细粒度授权、最小权限、密钥轮换和审计等措施。

两套凭据：upstream 与 MCP

变量	保护的是	读法
UPSTREAM_API_KEY	MCP 服务端 → 天气 API 这一段	上游凭据
MCP_API_KEY	客户端 → MCP 服务端这一段	入口凭据

UPSTREAM_API_KEY authorizes API calls; MCP_API_KEY protects the MCP endpoint.

UPSTREAM_API_KEY 授权对上游 API 的调用；MCP_API_KEY 保护 MCP 端点本身。

前者是 authorizes（授权对外发起调用），后者是 protects（保护自家入口）。

5.8 REST API 与 MCP 的最终对比

维度	REST API / OpenAPI	MCP
Shared	名称、描述、JSON Schema	名称、描述、JSON Schema
Discovery	获取 OpenAPI 文档	tools/list
Invocation	HTTP Method + Path + Parameters	tools/call
Authentication	Client 向 API 认证	Client 向 MCP 认证，上游认证可以独立
Scope	描述 HTTP 操作	Tools、Resources、Prompts 等

重点一：Shared Schemas

REST API 和 MCP 可以共享结构化输入的 Schema 信息。

因此，FastMCP 才能利用 OpenAPI 生成工具定义。

重点二：不同的 Discovery

REST API：GET /openapi.json，REST 客户端通过 API 描述了解操作。

MCP：tools/list，MCP Client 通过协议发现 Server 当前提供的工具。

重点三：不同的 Invocation

REST：GET /weather?city=Pittsburgh&units=F，REST 使用 HTTP 方法、路径和参数。

MCP：tools/call(name="get_weather", arguments={...})，MCP 使用统一的工具调用接口。

重点四：不同的 Authentication Boundary

直接 REST 调用：Client → REST API

MCP 代理方案：Client → MCP Server → REST API

后者可以把客户端认证与上游认证分开。但这种凭证隔离是一种可采用的架构设计，不是 MCP 独有或自动提供的安全保证。

重点五：不同的 Scope

OpenAPI 主要描述 HTTP API。MCP 除了 Tools，还支持 Resources 和 Prompts 等能力。

MCP 能力	主要用途
Tools	执行操作
Resources	提供上下文数据
Prompts	提供可复用的提示模板

Part 6 Multiple Tool Calls：串行、并行与依赖

前面我们学习了 Agent 如何发现和调用一个外部工具，现在要解决新的问题：当 Agent 需要调用多个工具时，应该一个接一个执行，还是同时执行？如何减少延迟，同时保证执行结果正确？

五个递进的知识模块：

1. Serial Tool Calls：为什么串行调用很慢？
2. Parallel Tool Calls：并行如何减少模型生成轮数和等待时间？
3. Dependencies：哪些工具可以并行，哪些必须串行？
4. Concurrent Execution：如何用 `asyncio.gather()` 实现并发？
5. Call ID & Result Matching：多个结果如何正确对应原始调用？

6.1 Serial Tool Calls：串行调用的代价

6.1.1 串行工具调用

Serial Tool Calls 指多个工具按照先后顺序执行。

在最基本的串行模式中，Agent 每次生成一个工具调用，等待工具执行完成，再根据返回结果继续生成下一次调用。

课程使用三个工具作为例子：

- Weather：查询天气。
- Calendar：读取日历。
- Flight：查询航班。

串行执行流程如下：

6.1.2 四次 Generate

一次工具调用通常经历：

在课件的串行模式中：

- 第一次 Generate：决定调用 Weather。
- 第二次 Generate：读取天气结果，决定调用 Calendar。
- 第三次 Generate：读取日历结果，决定调用 Flight。
- 第四次 Generate：读取航班结果，生成最终回答。

因此，这个例子中：3 次串行工具调用需要 4 次模型生成。如果每轮只调用一个工具，并且每次结果返回后都重新调用模型，那么：

这是一种特定串行执行模式下的公式，不代表所有多工具系统都必须这样运行。

6.1.3 串行调用的代价

串行调用的延迟来自两个部分。

1. Model Generation Cost 模型生成本身需要时间。
2. Tool Waiting Cost 模型需要等待工具完成，才能继续下一步。

因此，仅仅优化工具速度还不够。如果三个工具都能瞬间完成，仍然需要四次模型生成，总计 3.2 秒。

串行代价的三层结构

- 生成不可重叠：四次生成各 0.8s，串行时只能排队，合计 3.2s。
- 等待不可隐藏：工具执行时模型在空等，那 3.2s 里没有任何计算在推进。
- 失败会放大：链条上任何一环变慢或出错，后面全部顺延；串行的链条越长，尾部风险越大。

6.2 Parallel Tool Calls：并行工具调用

6.2.1 并行的核心思想

如果多个工具互不依赖，Agent 可以在同一次模型生成中提出多个工具调用，再由执行环境并发运行它们。

例如：用户希望获得天气、日历和航班信息。这三个查询在输入已经确定、且彼此独立的情况下，可以同时执行。

6.2.2 两次 Generate

串行模式：Generate → Weather；Generate → Calendar；Generate → Flight；Generate → Final Answer

并行模式：Generate → [Weather, Calendar, Flight]→ Collect Results；Generate → Final Answer

因此，课件中的并行模式只有两个模型生成阶段，中间不再需要为每个独立工具分别进行一次模型生成：

1. 第一次 Generate：同时产生三个工具调用。
2. 第二次 Generate：读取全部工具结果，生成最终回答。

6.2.3 并行总耗时的计算

在课件的理想化模型中：

因为三个工具同时执行，最终需要等待的是耗时最长的那个工具 max()。

例如：Weather：1.2s；Calendar：0.9s；Flight：1.1s

三个工具一起开始，Calendar 最先完成，Flight 随后完成，Weather 最后完成。

这个公式假设工具能够同时启动、没有额外调度开销，而且模型生成与工具等待没有其他重叠。

6.2.4 串行与并行性能对比

所以并行优化有两个来源：减少模型生成轮数 + 重叠工具等待时间。

需要特别注意：并行没有让 Weather 从 1.2 秒变成更短，它只是让 Weather、Calendar 和 Flight 的执行时间互相重叠。

6.2.5 并行的性能瓶颈

并行总耗时由最慢的分支决定。如果三个工具耗时分别是：Weather: 1.2s；Calendar: 0.9s；Flight: 1.1s。那么优先优化 Calendar，可能对整体完成时间没有帮助，因为 Weather 仍然需要 1.2 秒。只有降低最长分支，或改变关键路径，才可能进一步降低这批调用的完成时间。

结论：并行的价值不是让单个工具变快，而是减少串行往返，并让独立工具的等待时间重叠。

6.3 Dependencies 并行条件

Parallelize only independent, safely concurrent calls.

即：只并行那些相互独立、并且能够安全并发执行的调用。

6.3.1 可以并行: Independent Calls

Weather for Pittsburgh; Read today's calendar; Look up flight status

如果三个工具所需的输入已经确定，它们不需要等待彼此的输出。

6.3.2 必须串行: Dependent Calls

Find customer ID; Use ID to fetch orders; Refund selected order

因为第二步需要第一步返回的 `customer_id`，第三步又需要前面查询并选择出的订单信息。在第一步完成之前，第二步所需的输入还不存在。因此，它们存在 Data Dependency（数据依赖）。

6.3.3 三种常见依赖

- 数据依赖：后一步的参数来自前一步的返回值（ID → orders）。
- 控制依赖：后一步是否执行取决于前一步的结果（查不到客户，就不该发起退款）。
- 副作用依赖：后一步改变的状态必须建立在前一步已提交的状态之上（先退款、再记账）。

类型	定义	示例
Data Dependency	后一步需要前一步的输出	获取客户 ID → 查询订单
Control Dependency	后一步是否执行取决于前一步结果	查询是否有库存 → 决定是否下单
Side-effect Dependency	多个操作会影响共享状态或执行顺序	更新余额 → 记录交易

第三种尤其重要，因为没有数据依赖，不代表并发一定安全。

例如：Tool A: Withdraw $100; Tool B: Withdraw $200

两个调用可能都知道账户 ID 和金额，不需要彼此提供参数。但如果同时修改同一账户余额，且底层系统没有正确处理并发，就可能产生 Race Condition（竞态条件）。因此，并行判断至少需要检查：

- 输入是否已经全部确定。
- 一个调用是否依赖另一个调用的结果。
- 多个调用是否会修改共享状态。
- 底层服务是否支持安全并发。

6.3.4 真实任务通常是混合执行

不是所有任务都必须完全串行或完全并行。

在这个例子中：

- Weather 可以与 Find Customer ID 并行。
- Fetch Orders 必须等待 Customer ID。
- Refund Order 必须等待订单查询与必要确认。

这种结构可以理解为一个有向依赖图（Dependency Graph）。工具调用的执行顺序由依赖关系决定，而不是简单地由工具数量决定。

6.4 Concurrent Execution：使用 asyncio 实现并发

前面解释了什么时候应该并行，现在看代码如何实现。课程使用 Python 的 `asyncio.gather()`。

6.4.1 课程代码

```
import asyncioimport json
async def execute(call):    //定义一个 Coroutine Function（协程函数）。
调用它会得到协程对象，可以由 asyncio 事件循环调度执行。
args = json.loads(call.arguments)    //解析参数,假设模型返回的参数是 JSON 字符串
value = await TOOLS[call.name](**args)    //根据名称选择工具,通过工具名称查找对应函数,传递参数并等待工具结果
return call.call_id,  //保留 Call ID
valueresults = await asyncio.gather(
*(execute(call) for call in function_calls),
return_exceptions=True,
)
```

这里假设：

- `TOOLS` 是工具名称到异步函数的映射。
- `function_calls` 包含本轮需要执行的工具调用。
- 每个 `call` 有 `name`、`arguments` 和 `call_id`。

6.5 asyncio.gather()：一次调度多个协程

6.5.1 gather 的作用

```
results = await asyncio.gather(
*(execute(call) for call in function_calls),    //为每个工具调用创建一个协程
//* 将这些协程展开，作为多个位置参数传给 gather()
return_exceptions=True,)
```

`asyncio.gather()` 可以并发运行多个可等待对象，并等待它们完成。

6.5.2 并发完成顺序可能不同

假设同时执行：

`asyncio.gather()` 返回的结果顺序与输入的协程顺序一致，而不是与实际完成顺序一致。并发执行时，谁先跑完谁先回来，这个顺序是不确定的。工具响应时间不同、网络抖动，都会改变完成次序，结果乱序完成时，要靠 ID 把它们对回原调用。

例如，输入顺序为：[execute(weather), execute(calendar), execute(flight),]，那么 `gather()` 返回的结果仍然按照：[weather_result, calendar_result, flight_result,] 排列，即使 Calendar 最先完成。

因此，不能根据位置匹配 gather 返回结果，对于同一次 `gather()` 调用，只要保留原始输入顺序，位置对应关系是有保证的。但在完整 Agent 系统中，仍然应该保留 Call ID，因为工具结果需要与协议层的原始调用关联，不能依赖偶然的执行或消息顺序。

6.5.3 Call ID

例如：return call.call_id, value

可以产生：[("call_001", weather_result), ("call_002", calendar_result), ("call_003", flight_result),]

三个主要作用：

- Result Matching：把结果与原始工具调用配对。
- Traceability：在日志中追踪某次调用。
- Error Handling：将错误与具体调用关联。

在采用工具调用协议的系统中，返回结果通常需要携带与原始调用对应的标识符。

6.5.4 return_exceptions=True 的作用

return_exceptions=True

表示某个协程抛出异常时，`gather()` 将该异常作为对应位置的结果返回，而不是立即把它作为整个 `gather()` 的异常抛出。

例如：[weather_result, TimeoutError(), flight_result,]

这样，Calendar 失败不一定阻止其他两个工具完成。但异常对象不会自动变成成功的工具结果。执行环境仍然需要识别异常，并构造对应的错误响应。

6.6 总结

维度	Serial Tool Calls	Parallel Tool Calls
执行方式	依次执行	同时发起独立调用
模型生成	可能需要多轮	可以在一轮生成中提出多个调用
工具等待	耗时累加	主要由最慢工具决定
依赖处理	自然满足先后依赖	必须提前检查依赖
执行实现	顺序 await	asyncio.gather() 等
结果配对	使用 Call ID	同样需要 Call ID
课件耗时	6.4s	2.8s

其中：

- N：工具数量。
- G：每次模型生成耗时。
- T_i：第 i 个工具的执行时间。

1. Serial Calls：多个工具依次执行，会累加模型生成时间和工具等待时间。
2. Parallel Calls：独立工具可以在同一轮生成中提出，并发执行以降低延迟。
3. Dependencies：只有相互独立、并发安全的调用才能同时执行。
4. Concurrent Execution：Python 可以使用 `asyncio.gather()` 并发运行异步工具。
5. Call ID：工具结果必须能与原始调用正确对应，尤其是在异步和并发系统中。

这章最核心的思想是：优化多工具调用，不是盲目地把所有工具同时运行，而是根据依赖关系，在保证正确性的前提下，尽量减少不必要的模型生成轮数和等待时间。

它与前面 REST API、MCP 两章的关系也很清晰：

- REST API 解决如何通过网络调用工具。
- MCP 解决 Agent 如何标准化地发现和调用工具。
- Multiple Tool Calls 解决 Agent 如何高效、正确地组织多个工具调用。

Part 7 Evaluating Tool Use

前一章解决的是如何让 Agent 高效、正确地组织多个工具调用？这一章进一步讨论我们如何判断一个 Agent 的工具使用能力到底好不好？评估不能只看最终答案是否正确，还要检查它有没有选对工具、传对参数、安排好调用顺序，以及在真实运行环境中是否足够快、稳定、安全。

本章知识框架

1. Benchmark：考什么？BFCL 的四类测试场景
2. Evaluation Levels：在哪一层判断对错？Selection → Arguments → Trajectory → Task
3. Operational Quality：真实运行表现如何？Efficiency · Reliability · Safety
4. Failure Attribution：错误来自哪里？Model · Harness · Provider / Tool
5. Production Evaluation：真实数据怎么看？Provider Tool-call Error Rates
6. Layered System：整讲知识 Capabilities → Mechanics → Constraints → Interfaces → Systems

7.1 BFCL：工具调用能力的四类测试

Berkeley Function Calling Leaderboard 是用于评估大语言模型 Function Calling 能力的基准测试。

核心问题：模型能不能在不同场景下正确使用工具？

课件将测试场景分为四类：

7.1.1 Single Turn：单轮工具调用

Single Turn 评估模型在一次交互中生成正确工具调用的能力。

例如：

Simple: 调用单个工具

get_weather(city="Paris")

Multiple: 一次生成多个工具调用

get_weather(city="Paris")

get_calendar(date="today")

Parallel: 同一工具的多个独立调用

get_weather(city="Paris")

get_weather(city="London")

get_weather(city="Tokyo")

Multiple-parallel: 多种工具的多个并行调用

get_weather(city="Paris")

get_weather(city="London")

get_calendar(date="today")

get_flight(flight_no="AF123")

这里需要区分：

Single Turn 中的 multiple 和 parallel 都涉及多个调用，但 parallel 特别强调同类调用的独立并发场景；multiple-parallel 则组合了多工具和多次调用。这与 Part 6 的 Parallel Tool Calls 直接对应。

7.1.2 Multi-turn：多轮工具调用

Multi-turn 评估模型在多个交互步骤中能否持续正确使用工具。

课件列出四个子任务。

1. Base：基本多轮任务

评估模型能否利用上一轮工具返回的信息继续执行。例如：

```
User: 帮我查一下客户订单
Agent → find_customer()
Tool  → customer_id
Agent → get_orders(customer_id)
Tool  → order_list
Agent → Final Answer
```

2. Missing Function：缺少必要工具

例如用户要求：“帮我给客户退款。”但系统只提供：

```
TOOLS = {
"get_customer": ...,
"get_orders": ...
}
```

没有退款工具。模型不应该编造一个不存在的 `refund_order()` 调用。

3. Missing Parameter：缺少必要参数

例如：

```
get_flight_status(flight_number)
```

用户只说：“帮我查一下航班状态。”却没有提供航班号，也没有其他信息可以确定航班。正确行为通常是请求补充必要信息，而不是随意编造航班号。

4. Long Context：长上下文

在很长的对话中，测试模型是否仍能正确找到工具、参数和之前的任务信息。

7.1.3 Agentic：智能体场景

课件给出两个方向：

- Web Search：使用搜索工具获取外部信息。
- Memory：使用记忆信息支持后续任务。

相比简单的函数调用，这类测试更接近 Agent 的实际工作方式。

例如，一个 Agent 可能需要搜索信息、读取结果、决定下一步，再综合完成用户目标。

7.1.4 Robustness：鲁棒性

Robustness 关注的是：当输入不理想时，模型是否仍然可靠？

两个测试方向

1. Hallucination Measurement 测试模型是否会编造工具、参数或不存在的结果。
2. Format Sensitivity 测试模型对格式变化是否敏感。

例如同一个工具调用需求，分别用 JSON、自然语言、不同排列顺序或不同格式描述，模型的表现是否会发生明显变化。这与 Constraints 的内容相关：格式合法不代表语义正确，格式变化也可能影响模型选择。

7.2 Evaluate the Whole Stack：四层评估体系

一次工具调用究竟应该从哪些层面判断正确性？四个层次：

7.2.1 Selection：选择工具

第一层检查 Tool Selection。

例如用户说：“查询巴黎天气。”

系统提供：get_weather(city); get_calendar(date); book_flight(destination)

模型应该选择：get_weather(city="Paris")，如果调用了 `get_calendar()`，就是选错工具；如果本应调用工具却没有调用，就是 Missing Call如果调用了不必要的工具，就是 Extra Call。

常见评估指标：

- Precision：模型发出的调用中，有多少是应该发出的。
- Recall：应该发出的调用中，有多少被模型正确发出。

其中 TP 是正确调用，FP 是多余或错误调用，FN 是遗漏调用。实际评测还需要明确如何匹配预测调用与标准调用。

7.2.2 Arguments：检查参数

第二层检查 Tool Arguments，即使工具选对，参数仍然可能错误。

例如：get_weather(city="London")，用户要求的是巴黎的天气，这个调用可能满足 JSON Schema，但 `city` 的语义值错误。

两个检查方向：

- AST Match：比较工具调用的结构。
- Schema Match：检查参数结构与字段是否符合要求。

需要注意：Schema Match 主要保证结构合法，不能单独保证参数值符合用户意图。

7.2.3 Trajectory：调用顺序

第三层评估调用轨迹（Trajectory），这与 Part 6的 Dependencies 直接对应。

例如：Correct Trajectory；Find Customer ID；Fetch Orders；Refund Selected Order

如果模型先退款，再查询订单，就违反了依赖关系，即使每个工具本身正确，整个执行序列也可能错误。

课件使用 Sequence Success 作为示例指标，并指出这一层通常需要执行或观察工具交互过程。

7.2.4 Task：最终目标

第四层检查 End-to-end Task Success，这是最接近用户真实体验的指标。

例如：用户要求：“帮我查询明天的航班状态。”

Agent:

1. 正确选择航班工具。
2. 正确传入航班号。
3. 正确执行查询。
4. 却在最终回答中说错了航班时间。

前三步可能都通过，但 Task 仍然失败。课件给出的典型失败是：Plausible Wrong Answer：看起来合理，但实际上错误的答案。它尤其难发现，因为错误可能不是格式问题，也不是工具执行异常，而是模型对结果的理解或使用出了问题。

7.3 Operational Quality：运行质量的三个维度

7.3.1 Efficiency：效率

A correct agent can still be unusably slow or expensive.

即使 Agent 能正确完成任务，也可能因为延迟过高或成本过高而无法实际使用。

四个指标分别是：

指标	关注什么
Latency	完成任务需要多久
Calls	调用了多少次工具

指标	关注什么
Tokens	消耗了多少 Token
Cost	完成任务的实际成本

Part 6 的串行与并行对比就是典型的 Efficiency 问题。串行耗时 6.4 秒，并行耗时 2.8 秒。虽然工具查询结果可能完全相同，但并行执行改善了响应延迟。

7.3.2 Reliability：可靠性

Reliability 关注系统在真实运行条件下能否稳定完成任务。

Timeouts 工具没有在规定时间内返回结果。

Retries 请求失败后重新尝试。

Partial Failure 一批调用中部分成功、部分失败。

例如三个并发工具：

Weather → Success Calendar → Timeout Flight → Success

系统需要决定：是否重试 Calendar？是否可以利用已经成功的两个结果？是否向用户说明部分信息不可用？

这与上一章 `asyncio.gather(return_exceptions=True)` 的异常处理直接相关。

7.3.3 Safety：安全性

Safety 关注 Agent 的工具调用是否符合权限和安全约束。

课件列出：

- Policy：政策与规则约束。
- Permissions：权限控制。
- Side Effects：执行操作带来的状态变化。

例如：

```
delete_file(path)
transfer_money(amount)
send_email(recipient, content)
```

这些工具不仅查询信息，还可能产生真实影响。所以除了判断调用是否正确，还需要评估调用是否经过授权、是否符合安全规则，以及执行后是否造成不必要的副作用。

7.3.4 三个维度之间的权衡

真实系统中，这三个维度可能互相影响。

例如：

- 增加重试可能提高 Reliability，但增加 Latency 和 Cost。
- 增加权限校验可能提高 Safety，但增加处理时间。
- 减少工具调用可能提高 Efficiency，但如果省略必要的检查，也可能降低任务成功率。

因此，生产环境不能只追求单一指标。

本节结论：Agent 的质量不仅取决于是否正确，还取决于是否高效、可靠、安全。

7.4 Operational Tool Failures：三类故障来源

当一次工具调用失败时，错误究竟来自模型、执行框架，还是外部工具？

课件将故障分为三个来源。

7.4.1 Model：模型侧错误

模型侧主要负责决定调用什么工具、传递什么参数，以及如何理解返回结果。

三种典型错误：Wrong Tool 模型选错工具；Bad Arguments 工具正确，但参数不正确；Ignores Result 工具已经返回结果，但模型忽略结果，仍然生成错误答案。

例如：

```
Tool result:
Flight delayed by 2 hours
Agent final answer:
The flight is on time.
```

这里工具执行成功，错误发生在模型对结果的使用阶段。

7.4.2 Harness：执行框架错误

Harness 是模型与工具之间的编排执行层，主要负责解析、验证、派发调用，以及将结果匹配回原始调用。

四种故障：Parse 无法正确解析模型生成的工具调用内容；Auth 调用时出现认证或授权问题；Validation 工具参数不符合 Schema 或其他校验规则；ID Mismatch 工具返回结果与原始 Call ID 配对错误。最后一种直接呼应 Part 7的并发执行。

例如：

```
call_001 → Weather
call_002 → Calendar
```

如果 Harness 把 Calendar 的结果错误关联到 `call_001`，就会导致工具调用与结果不一致。

7.4.3 Provider or Tool：供应商或工具侧错误

即使模型和 Harness 都正确，外部服务仍然可能失败。

Timeout 服务响应超时；Rate Limit 调用频率超过服务限制；Execution Error 工具执行时出现错误。例如：

```
Model:
Correct tool + correct arguments
Harness:
Valid request + correct dispatch
Provider:
HTTP 429 Too Many Requests
```

这种情况下，失败不是模型选错了工具，而是外部服务限流。

7.4.4 故障的不同处理

同样是 Task Failure，不同来源需要不同处理方式。

错误来源	典型改进方向
Model	改善提示词、工具描述、模型能力或训练
Harness	修复解析、验证、授权与结果配对逻辑
Provider / Tool	超时控制、退避重试、熔断、降级

Production evaluation needs benchmark accuracy and operational failure rates.

生产级评估既需要 Benchmark Accuracy，也需要 Operational Failure Rates。仅仅知道模型在测试集上的工具选择准确率，并不能说明整个 Agent 系统在真实环境中有多可靠。

7.5 Provider Tool-call Error Rates：真实错误率数据

Tool Call Error Rate by Provider 不同供应商的工具调用错误率。

图表按时间展示供应商侧的错误率变化，观察外部工具调用在运行中发生错误的情况。

图表纵轴标注为 Error Rate (%)，但截图中的悬浮读数没有单独显示单位。这里保留截图中的原始数值，不额外推断统计口径。

Cloudflare 的 15.1 明显高于其余可见读数，是这张截图中的突出异常值。

不过，单凭截图不能判断异常的具体原因，也不能据此评价供应商长期的整体可靠性。

总结

第一，供应商错误率是随时间变化的。同一服务可能在不同日期出现不同的错误水平。

第二，不同供应商的运行稳定性可能不同。即使模型使用完全相同的工具调用策略，外部服务的失败也可能影响最终任务成功率。

第三，评估需要区分模型能力与运行环境。BFCL 测试的是模型工具调用能力的不同方面。Provider Error Rate 则提供运行环境可靠性的另一类证据。这两类指标应该结合分析，而不是互相替代。

7.6 Tool Use Is a Layered System

Tool Use for LLM Agents 五层内容：

1. Capabilities 说明工具为模型扩展了哪些能力。Retrieval · Execution · Generation · Application Actions
2. Mechanics 说明工具定义如何进入模型上下文，模型如何产生调用，Harness 如何执行并返回结果。Schemas · Model Protocols · Harness · Call IDs
3. Constraints 说明 JSON Schema 和 Grammar Masks 可以约束输出形式，但不能保证语义正确。JSON Schema · Grammar Masks
4. Interfaces 说明 REST API 与 MCP 如何向 Agent 暴露外部能力。Direct REST Calls · MCP

对应：Part 6：如何组织多个工具调用，优化执行效率。

5. Systems 则负责把前面所有部分组织成完整的系统：管理依赖、并发执行、评估正确性、控制成本并处理运行故障。Dependencies · Concurrency · Evaluation · Failures

对应：Part 7：如何评估这些工具调用及其最终结果。

7.7 总结

假设用户要求 Agent 查询航班信息。

Agent 调用了航班查询工具，最后给出航班状态。

我们可以依次检查：

1. Selection：是否选择了航班工具，而不是天气工具？
2. Arguments：航班号、日期等参数是否正确？
3. Trajectory：是否按照必要的依赖顺序执行？
4. Task：最终航班信息是否正确？
5. Efficiency：是否用了过多时间、Token 或工具调用？
6. Reliability：超时、限流或部分失败时能否妥善处理？
7. Safety：是否遵守权限与安全约束？
8. Failure Attribution：如果失败，究竟是模型、Harness，还是外部服务的问题？

这就是Evaluate the whole stack 的真正含义：不仅评估模型生成了什么调用，还要评估整个调用过程以及最终结果。

Lecture 7 Agent Memory and RAG

Part 1 RAG 的研究动机与基本原理

1.1 大语言模型的局限性

大语言模型（Large Language Models, LLMs）依靠大规模训练数据和模型参数获得语言理解与生成能力，但仍存在四类主要问题。

挑战	定义与表现
Hallucination（幻觉）	生成不准确、无意义或缺乏事实依据的内容
Domain-specific Knowledge & Expertise（领域知识不足）	缺乏医学、法律、金融等专业领域的知识与推理经验
Privacy（隐私）	在训练、部署及调用过程中可能产生敏感信息泄露与数据安全风险
Inconsistency（不一致性）	不同调用中回答质量不稳定，可能出现相互矛盾或与问题无关的内容

其中，幻觉和领域知识不足是引入外部检索的重要动机，但 RAG 并不能自动消除隐私风险或保证生成结果完全一致。

垂直领域中的幻觉问题

斯坦福研究人员发表于 Journal of Legal Analysis（2024）的论文：Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models。研究指出，LLM 在回答可核验的联邦法院案件问题时，可能产生较高比例的法律幻觉。在特定法律问题上，幻觉率可达到 69% - 88%。图中的 58% - 88% 则是所展示模型的平均幻觉率范围，两种表述不应混为同一个统计口径。

这一结果说明：语言流畅性与事实可靠性并不等价，通用模型的能力也不能直接代表其专业领域能力。

1.2 参数化知识与非参数化知识

课件用 Big Model + Big Training Data 概括 LLM 的能力来源，并提出两种知识存储方式。

比较维度	Parametric Knowledge	Non-parametric Knowledge
中文	参数化知识	非参数化知识
存储位置	模型参数	外部文档、数据库、索引
获取方式	模型前向推理	Information Retrieval（IR）
更新方式	通常需要训练或参数更新	更新外部知识库及索引
优势	无需每次显式检索	易更新、可追溯、可扩展
局限	知识更新成本高，难以覆盖全部长尾知识	依赖检索质量与外部数据

参数化知识通过训练过程隐式编码于模型权重 θ 中，模型基于输入 x 生成回答： $P_\theta(y\mid x) $

其中 x 表示输入，y 表示生成结果。非参数化知识则通过外部语料库 $\mathcal{D}$ 存储，并在需要时执行检索：

$\mathcal D_q=\operatorname{Retrieve}(q,\mathcal D) $

其中 q 为查询，$\mathcal{D}_q$ 为检索得到的相关文档集合。

两者并非相互替代：LLM 提供语言理解与生成能力，外部知识库提供可动态访问的信息。

1.3 Retrieval-Augmented Generation（RAG）

1. LLM 无法将全部知识，尤其是长尾知识（Long-tail Knowledge），完整编码进参数。
2. 训练结束后新增的信息与专业领域知识，也不一定能够被模型准确获取。

重新训练或微调（Re-training / Fine-tuning）可以改变模型能力，但通常需要较高的计算与数据成本，并不适合所有知识更新需求。Retrieval-Augmented Generation（RAG）通过将外部检索结果引入生成过程，使模型能够利用参数之外的信息完成任务。

其基本过程为：

常见的输入级 RAG 可形式化为： $y\sim P_\theta(y\mid q,\mathcal D_q) $

案例：询问 2023 年女足世界杯冠军。未使用 RAG 的旧模型可能由于知识截止时间而无法回答；使用 RAG 后，模型通过外部信息获得事实，回答 Spain。

传统 LLM 与 RA-LLM =可以类比为：闭卷考试与开卷考试

- Close-book Exam：仅依赖模型参数中已有的知识。
- Open-book Exam：允许检索外部参考资料，再生成答案。

其关系可以概括为： ${\text{RA-LLM}=\text{LLM Generation}+\text{Information Retrieval}} $

课件将训练数据与外部知识库作如下对比：

Data for Training LLMs	External Knowledge Base
Low quality	High-quality knowledge
General	Specialized knowledge

Data for Training LLMs	External Knowledge Base
Fixed	Scalable
Hard to update	Easy-updated

训练数据也可以具有高质量和专业性，外部知识库同样可能包含错误或过时信息。RAG 的主要优势在于知识可更新、可检索和可扩展，但其最终可靠性仍取决于知识库质量、检索准确率与生成模型的证据使用能力。

Part 2 RA-LLM 的研究体系与标准架构

2.1 RA-LLM Research Taxonomy

从 Architecture、Learning 和 Application Tasks 三个维度建立 RA-LLM 的研究分类。

Architecture：架构维度

三种 Augmentation 方式的主要区别是检索知识进入生成模型的位置。

1. Input-layer Integration（输入层集成）

将检索文本与查询拼接，作为模型输入。课件后续介绍的 RAG 与 In-Context RALM 均采用这种思路。

2. Intermediate-layer Integration（中间层集成）

在模型内部表示或中间计算过程中引入检索信息。

3. Output-layer Integration（输出层集成）

在输出阶段结合检索证据，例如通过输出概率或候选结果的融合影响最终预测。

Learning：学习维度

训练方式	含义
Independent	检索器与生成器分别训练
Sequential	按阶段依次训练不同组件
Joint	对检索与生成进行联合优化

Application Tasks：应用维度

- NLP：Question Answering、Fact Verification
- Downstream：Recommendations、Software Engineering
- Domain-specific：Science、Finance

三维分类分别回答：系统如何构建、组件如何学习、技术应用于哪些任务。

2.2 RAG 技术演进（2019 - 2024）

课件通过时间轴展示四条研究路线：

技术路线	代表模型或方法
RAG Framework / Pipeline	kNN-LM、REALM、RAG、FiD、RETRO、Atlas、Self-RAG、In-Context RALM

技术路线	代表模型或方法
RAG Learning	REALM、RAG、EMDR2、RETRO、Atlas、Self-RAG
Retriever Learning	DPR、Contriever、EPR、CEIL、UDR、RA-DIT
Pre-/Post-Retrieval Techniques	SPALM、Re2G、HyDE、Query2doc、RECOMP、QueryRewriter

其中，REALM、RAG 等早期工作奠定了神经检索与语言模型结合的基础；后续研究逐渐扩展至检索器训练、检索前后处理以及更复杂的自适应检索流程。

需要注意，同一研究可能同时属于多个类别。例如 RAG 既是一种架构，也涉及检索与生成的学习机制。

2.3 RA-LLM Standard Pipeline

标准 RA-LLM 由两个必要组件和两个可选模块组成：

Retrieval 和 Generation 是必要组件（Major Components），Pre-retrieval 与 Post-retrieval 属于可选的辅助模块（Affiliated Modules）。Integration 表示检索与生成之间的信息融合关系，而不是额外的独立处理模块。

2.4 两种典型的检索增强模型

1. RAG：DPR + BART

- Retrieval：Dense Passage Retriever（DPR）+ Maximum Inner Product Search（MIPS）
- Generation：BART
- Integration：将检索段落分别与问题拼接

对于查询 q 和检索文档 z，模型可按文档概率对生成结果进行边缘化：

$P(y \mid q) = \Sigma_{z \in \text{TopK}(q)} P_\eta(z \mid q) P_\theta(y \mid q, z)$

其中：

- $P_\eta(z \mid q)$：检索器选择文档 z 的概率。
- $P_\theta(y \mid q, z)$：生成器在查询与文档条件下生成答案的概率。

该式对应经典 RAG-Sequence 的文档级边缘化形式，是对课件架构图的理论补充；不同 RAG 变体的概率融合方式可能不同。RAG 可以通过查询编码器 q_η 和生成器 p_θ 的梯度反向传播进行端到端学习。

2. In-Context RALM：BM25 / BERT / Contriever + GPT-series

In-Context RALM 使用 BM25、BERT 或 Contriever 等检索方法，并将检索段落前置到模型输入中。

$x'=\operatorname{Concat}(d_1,\ldots,d_k,q) $

$y\sim P_\theta(y\mid x') $

其中 $d_1, \cdots, d_k$ 为检索文档。

与前述经典 RAG 模型相比，这种方式通常不要求对生成模型进行端到端训练，而是通过上下文提示利用检索证据。世界杯扩军案例：模型可以从检索文本中获取“2026 年世界杯扩军至 48 支球队”的信息，而无需将该知识重新写入模型参数。

比较	经典 RAG	In-Context RALM
检索器	DPR + MIPS	BM25 / BERT / Contriever
生成器	BART	GPT-series
集成方式	问题与各检索段落分别结合	将检索内容前置到输入
学习机制	可端到端联合训练	通常直接利用预训练 LLM
主要特点	显式建模检索文档与生成概率	实现简洁，适用于黑盒 LLM

Part 3 Retriever：检索器的分类、原理与训练

检索器（Retriever）负责从知识库中筛选与查询相关的信息，是 RAG 系统中决定证据质量的关键组件。不同检索器不仅影响 Retrieval Performance，也会进一步影响生成模型的输出质量。

3.1 检索器的两个分类维度

分类依据	类型	核心区别
Relevance Measurement	Sparse / Dense	如何表示文本并计算相关性
Retriever Learning	Task-specific / General-purpose	如何获得检索能力、适应什么任务

Sparse 与 Dense 是表示和评分方式的区别；Task-specific 与 General-purpose 则是训练目标与适用范围的区别。

不同检索器对生成质量的影响

课件使用 Perplexity（困惑度）衡量不同检索器辅助语言模型时的生成效果。

$ \operatorname{PPL}(x) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t\mid x_{<t}) \right) $

PPL 越低，表示模型对测试文本的预测越好。

课件实验数据：不同检索器下的 PPL。困惑度越低越好。两种模型规模下，BM25 均取得最低 PPL。

实验结果表明：

- 对 GPT-2 117M，BM25 的 PPL 为 30.2，优于 No Retrieval 的 37.8。
- 对 GPT-2 1.5B，BM25 的 PPL 为 16.7，优于 No Retrieval 的 19.9。
- BERT 的表现略差于不检索，说明加入检索不一定改善生成质量。
- 大模型整体 PPL 更低，但不同检索方法之间的差距有所缩小。

这一实验说明，检索器的选择应依据下游任务效果，而不能仅依据是否采用了更复杂的神经网络。

3.2 Sparse Retrieval 与 Dense Retrieval

两者的根本区别在于文本表示方式与相关性计算机制。

Sparse Retrieval（稀疏检索）

稀疏检索通过词项及其统计特征表示文档，通常在高维词汇空间中进行匹配。

基本过程： $\text{Document} \rightarrow \text{Tokenization} \rightarrow \text{Inverted Index} \rightarrow \text{Relevance Scoring} $

代表方法：

1. TF-IDF 词频（Term Frequency）与逆文档频率（Inverse Document Frequency）相结合：

$\operatorname{TFIDF}(t,d) = \operatorname{TF}(t,d)\operatorname{IDF}(t) $

常见 IDF 形式为：

$\operatorname{IDF}(t) = \log\frac{N}{\operatorname{DF}(t)}$

其中 N 是文档总数，DF (t) 是包含词项 t 的文档数量。

2. BM25 在词项匹配基础上加入词频饱和与文档长度归一化。

一种常见表达为：

$\operatorname{BM25}(q,d) = \sum_{t\in q} \operatorname{IDF}(t) \frac{ f(t,d)(k_1+1) }{ f(t,d)+k_1 \left(1-b+b\frac{|d|}{\operatorname{avgdl}} \right) }$

其中：

- $f(t, d)$：词项在文档中的频率。
- k_1：控制词频饱和程度。
- b：控制文档长度归一化。
- $|d|$：文档长度。
- avgdl：语料库平均文档长度。

课件强调 Sparse Retrievers 的三个特点：

- Feasible to apply：易于实现和部署。
- High efficiency：检索效率较高。
- Fine performance：在许多任务上具有良好效果。

Dense Retrieval（稠密检索）

稠密检索通过神经编码器将查询与文档映射为低维连续向量 $\mathbf q=E_Q(q) $， $\mathbf{d} = E_D(d)$。再通过向量相似度计算相关性：$s(q, d) = \mathbf{q}^\top\mathbf{d}$。对于使用归一化向量的系统，也可采用余弦相似度。

其基本过程： $ \text{Query / Document} \rightarrow \text{Embedding} \rightarrow \text{Vector Index} \rightarrow \text{Similarity Search}$

代表方法是 DPR 和 Contriever。

Dense Retrievers 的主要特点包括：

- Allowing Fine-tuning：可以针对特定任务微调。
- Better Adaptation：能够学习更适合任务的语义表示。
- Customizable for More Retrieval Goals：可以针对不同检索目标定制。

综合比较

维度	Sparse Retrieval	Dense Retrieval
文本表示	高维稀疏词项向量	低维稠密语义向量
主要依据	词项匹配及统计权重	向量语义相似性
典型方法	TF-IDF、BM25	DPR、Contriever
索引	倒排索引	向量索引
训练要求	通常无需神经网络训练	通常依赖预训练编码器
优势	高效、可解释、适合精确关键词	语义匹配、可微调、迁移能力
局限	容易受到词汇不匹配影响	训练和索引成本较高，可能遗漏精确实体

两类检索器可以通过 Hybrid Retrieval 结合，在词项匹配与语义匹配之间形成互补。

3.3 Dense Passage Retriever（DPR）

DPR 是面向 Question Answering（QA）的有监督稠密检索器。

双编码器结构（Dual Encoder）

DPR 使用两个独立的 BERT 编码器： $\mathbf q=E_Q(q) $， $\mathbf{d} = E_D(d)$

其中 E_Q 负责查询编码，E_P 负责段落编码，两者不共享参数。

使用 Inner Product Similarity： $\operatorname{sim}(q,p) = E_Q(q)^\top E_P(p)$

检索时对所有候选段落进行评分，选择得分最高的 Top-K 段落： $\operatorname{TopK}(q) = \underset{p\in\mathcal D}{\operatorname{TopK}} \operatorname{sim}(q,p)$

有监督训练数据

DPR 使用 Question-Passage Sets： $\mathcal{D} = \{(q_i, p_i^+, p_{i,1}^-, \cdots, p_{i,n}^-)\}_{i=1}^m$

其中：

- q_i：查询。
- p_i^+：相关段落（Positive Passage）。
- $p_{i,j}^-$：不相关段落（Negative Passage）。
- m：训练样本数量。

对比学习目标

$\mathcal L_i = -\log \frac{ e^{\operatorname{sim}(q_i,p_i^+)} }{ e^{\operatorname{sim}(q_i,p_i^+)} + \sum_{j=1}^{n} e^{\operatorname{sim}(q_i,p_{i,j}^-)} }$

该目标通过增大正例相似度、降低负例相对得分，使检索器能够区分相关与不相关段落。

In-batch Negatives（批内负例）

在一个训练 Batch 中，其他查询的正例段落可作为当前查询的负例。

例如： $(q_1, p_1^+), (q_2, p_2^+), \cdots, (q_B, p_B^+)$

对于 q_1，p_1^+ 是正例，其余段落可作为负例候选。这种方法减少了额外负例采样成本，但存在负例过于简单、难度不足，或错误地将相关段落当成负例的问题。因此，Negative Sample Selection 是影响 DPR 训练效果的重要因素。

3.4 Contriever：无监督通用稠密检索器

Contriever 采用无监督对比学习构建通用检索器。

与 DPR 不同，Contriever 对查询和文档使用共享编码器： $\mathbf{q} = E(q), \mathbf{d} = E(d)$

其相似度仍可通过内积计算： $s(q, d) = E(q)^\top E(d)$

无监督对比学习

Contriever 不要求人工标注的 Question-Passage Sets，而是从未对齐文档（Unaligned Documents）中构建对比学习样本。

对比目标以正样本 k_+、负样本 k_i 和温度参数 τ 为基础。

标准 InfoNCE 损失可写为： $\mathcal L(q,k_+) = -\log \frac{ \exp(s(q,k_+)/\tau) }{ \exp(s(q,k_+)/\tau) + \sum_{i=1}^{K} \exp(s(q,k_i)/\tau) } $

温度系数 τ 控制 Softmax 分布的尖锐程度。

DPR 与 Contriever 对比

比较	DPR	Contriever
定位	Task-specific	General-purpose
训练	Supervised	Unsupervised
编码器	Two Independent BERTs	Shared BERT Encoder
数据	Question-Passage Pairs	Unaligned Documents
目标	特定 QA 任务表现	通用语义检索能力
迁移	可能依赖目标任务适配	更适合零样本与少样本迁移

无监督预训练可能改善 Zero-shot Transfer 和 Few-shot Learning 场景下的鲁棒性，但并不意味着在所有任务上都优于监督检索器。

3.5 DPR 与 Contriever 的性能比较

两组实验比较检索器性能。

1. End-to-end QA：Exact Match Accuracy

Exact Match（EM）表示生成答案与标准答案完全匹配的比例： $\operatorname{EM} = \frac{1}{N} \sum_{i=1}^{N} \mathbb I(\hat y_i=y_i)$

Single 与 Multi 是课件表格中的训练设定标签，不应理解为单轮和多轮问答。

从结果可以观察到：

- DPR（Single）在 NQ 上达到 41.5，高于 BM25 的 32.6。
- DPR 在 TriviaQA 上达到 56.8，高于 BM25 的 52.4。
- BM25 在 SQuAD 上仍具有优势。

- BM25+DPR 的组合并非在所有任务上取得最高分。
- Multi 设定在部分任务上提升显著，但也可能降低其他数据集的表现。

因此，稠密检索通常具有较强的语义匹配能力，但不存在对所有数据集都最优的检索器。

2. 检索 Recall@K

$R@K = \frac{ \text{Top-}K\text{ 中被成功覆盖的相关目标数} }{ \text{全部相关目标数} }$

在开放域 QA 检索评估中，Recall@K 主要反映正确证据是否被前 K 个结果覆盖。

还列出了 Inverse Cloze Task、Masked Salient Spans 等无监督预训练方法，用于比较不同预训练目标对检索效果的影响。从 NQ 数据看，Contriever 的 R@20 与 R@100 均超过 BM25，但仍低于监督训练的 DPR。这说明无监督检索器具有竞争力，而任务相关监督信号仍可能带来额外收益。

DPR 与 Contriever 均已广泛应用于 RAG / RA-LLM 系统：

- DPR：RAG、FiD、RETRO、EPR、UDR 等。
- Contriever：Self-RAG、Atlas、RAVEN 等。

3.6 面向 In-Context Learning 的检索器

此前的检索器主要寻找事实证据，而面向 In-Context Learning（ICL）的检索器主要寻找能够帮助模型完成当前任务的提示或示例。

Prompt Retriever

Prompt Retriever 从候选提示中选择适合当前输入的 Prompt。

结构包括： Training Data → Unsupervised Retriever → Candidate Prompts

再通过 Scoring LM 评价候选提示的质量。

模型还可以使用 Utterance Encoder E_X 与 Prompt Encoder E_P，通过对比学习区分：

- Positive Examples
- Hard Negative Examples

Exemplar Retriever（CEIL）

CEIL 通过检索适合当前输入的 Exemplars（示例），优化上下文学习效果。

其核心过程是：

1. 创建训练数据（Creating Training Data）。
2. 建立示例与输入之间的关联。
3. 使用 DPR Retriever 产生检索得分。
4. 使用语言模型计算示例对任务表现的影响。
5. 通过 Alignment Loss 对齐检索器与语言模型的偏好。

与传统证据检索不同，CEIL 更关注：哪些示例能够使语言模型在当前任务上取得更好的表现？

3.7 Search Engine as Retriever

传统 RAG 通常使用预先建立的知识库，但这种方式存在三类局限：

- 实时网页内容难以及时更新到本地知识库。
- 预定义数据库可能受到存储规模限制。
- 无法直接利用 Google、Bing 等成熟搜索引擎长期优化的排序能力。

因此，可以使用 Search Engine 作为外部检索器。多轮搜索过程：

$x \rightarrow q_1 \rightarrow D_{q_1}$， $D_{q_1} \rightarrow y_2 \rightarrow q_3 \rightarrow D_{q_3}$

例如，回答一个人物教育经历的问题时，模型可以先检索人物背景，再检索就读大学，最后检索所获学位。

这类方式允许模型依据中间结果继续搜索，体现了检索与生成交替进行的过程。

但网页搜索并不自动保证信息可信，仍需处理来源可靠性、检索噪声与事实验证问题。

Part 4 Pre-Retrieval Techniques：检索前优化

检索前优化（Pre-retrieval）发生在检索器执行之前，其目标是提高查询与检索系统之间的适配程度。

三种主要方法：

方法	主要操作	解决的问题
Query Rewriting	改写查询	原始表述不适合检索
Query Decomposition	分解查询	复杂问题包含多个信息需求
Query Expansion	扩展查询	查询缺少相关关键词或语义信息

此外，课件将 HyDE 作为独立技术介绍，它通过生成假设文档改变检索输入。

4.1 Query Rewriting 查询改写

查询改写通过语言模型将原始问题转换为更适合检索的表达形式。

$q' = f_{\text{rewrite}}(q)$

其中 q 是原始查询，q' 是改写后的查询。

三种流程：

方法	流程
Direct Retrieval	Input → Retriever → Documents → LLM Reader
LLM Rewriter	Input → Black-box LLM Rewriter → Web Search → Reader
Trainable Rewriter	Input → Small PrLM Rewriter → Web Search → Reader

Trainable Rewriter 进一步通过 Reward 信号优化改写器，使生成的查询更有利于检索命中和下游回答。

查询改写与分解的联系

课件示例：What profession do Nicholas Ray and Elia Kazan have in common? 比较两个人的共同职业。

可以改写为：Nicholas Ray profession, Elia Kazan profession。检索到两个人的职业信息后，再综合得到共同职业 Director。同时体现了 Query Decomposition：将复杂问题拆成多个相对独立的检索子任务。

实验效果

课件在 HotpotQA、AmbigNQ、PopQA 三个数据集上比较了不同改写方式。

其中 F1 用于衡量预测答案与标准答案之间的匹配程度。

对于常见的抽取式问答 Token-level F1： $F_1=\frac{2PR}{P+R}$

实验显示，查询改写在多个 QA 数据集上有效，但可训练改写器并非始终优于 LLM Rewriter。例如 PopQA 上后者的 F1 为 49.74，略高于前者的 49.51。同时，HotpotQA 的普通 Retrieve-then-read 表现甚至低于 Direct，说明引入检索不必然提升答案质量，查询适配与证据质量同样重要。

4.2 HyDE：Hypothetical Document Embeddings

HyDE 的核心思想：先生成一篇可能回答问题的假设文档，再利用该文档的语义表示检索真实文档。

传统检索： $q \rightarrow \text{Retrieve}(q)$

HyDE： $q \xrightarrow{\text{LLM}} d' \xrightarrow{\text{Encoder}} E(d') \xrightarrow{\text{Search}} \mathcal D_q$

其中 d' 是 Hypothetical Document，即模型生成的假设文档。

为什么生成假设文档?

自然语言问题与目标文档在表达形式上可能存在较大差异。例如，问题可能只有几个词，而目标文档是包含背景、术语与详细描述的长段落。HyDE 先生成一段具有文档形式的文本，再执行 Document-to-Document Similarity Search，以减小查询与目标文档之间的表示差异。

课件示例涉及：

- 智齿拔除所需时间。
- COVID-19 对心理健康的影响。
- 韩语表达的领域问题。

这些例子说明，HyDE 可以根据问题类型生成不同风格的假设文档。

关键限制

HyDE 生成的文档不一定事实正确。它的用途是辅助检索，而不是直接充当答案。因此必须区分：

$\text{Hypothetical Document}\neq\text{Verified Evidence} $

其主要风险是生成内容偏离问题，导致检索方向错误；同时也增加了一次生成模型调用的延迟与成本。

4.3 Query Expansion 查询扩展

Query Expansion 不直接替换原查询，而是为其补充相关信息。

Query2doc 说明了通过 LLM 生成文档来扩展查询的方法。

核心公式： $q^+ = \text{concat}(q, [SEP], d')$

其中：

- q 原始查询。
- $\acute{d}$ LLM 生成的扩展文档。
- [*SEP*]：分隔符。
- q^{+}扩展后的查询。

与 HyDE 的区别在于，Query Expansion 保留原始查询，并追加生成内容。

实验比较： MS MARCO dev 与 TREC DL 19 上的 Query2doc 的表现。

- MRR@10：前 10 个结果中第一个相关结果的倒数排名均值。
- R@50、R@1k：前 50、1000 个结果的召回表现。
- nDCG@10：考虑相关性等级与排名位置的归一化折损累积增益。

Query2doc 在两类检索器上均有收益。BM25 的 nDCG@10 从 51.2 提升至 66.2，增加 15.0；DPR 从 64.7 提升至 68.7，增加 4.0。

三种检索前技术对比

技术	对查询的处理	主要特点
Query Rewriting	重新表达查询	提高检索适配性
HyDE	生成假设文档并以其向量检索	缩小问题与文档的语义表示差异
Query Expansion	原查询 + 生成文档	补充关键词与语义信息

Part 5 Post-Retrieval Techniques：检索后优化

检索后优化（Post-retrieval）发生在检索器返回候选结果之后，主要解决结果排序不佳、内容冗余、证据错误等问题。四种重点方法：

- Re-ranking：重新排序候选文档。
- Compression：压缩冗余检索内容。
- Correction：检测并纠正不可靠的检索结果。
- Refiner：抽取与查询相关的原文证据。

5.1 Re-ranking：检索结果重排

初始检索通常追求较高召回率与较低计算成本，但初检排序未必足够准确。

此外，BM25 与神经检索器采用不同的评分机制，其原始分数通常无法直接比较。

因此，Re2G 使用 Reranker 对候选文档进行统一评分。

两阶段检索

普通 RAG 直接使用初检的 Top-K 结果；Re2G 则先收集较大的 Top-N 候选集合，再通过重排器筛选出最终的 Top-K 文档。

BERT Cross-Encoder 重排模型

Interaction Model Based on Sequence-pair Classification.

其输入结构为： $[\mathrm{CLS}]\ q\ [\mathrm{SEP}]\ d\ [\mathrm{SEP}] $。查询和文档被共同输入到 BERT 中，使两者能够在注意力层中发生交互。

1. 取 [CLS] 对应的输出表示： $\mathbf h_{\mathrm{CLS}} = \operatorname{BERT}(q,d)_{\mathrm{CLS}} $；
2. 相关性分类概率：通过线性层与 Softmax $P(r \mid q,d) = \mathrm{Softmax}(W\mathbf{h}_{\mathrm{CLS}} + b)$，其中 $r \in \{\mathrm{yes}, \mathrm{no}\}$；
3. 重排分数： $s_{\mathrm{rerank}}(q,d) = P(\mathrm{yes} \mid q,d)$。

Cross-Encoder 能够对查询与文档进行细粒度交互，但通常需要逐对计算，因此成本高于使用预计算文档向量的双编码器，所以系统通常先执行高效召回，再对少量候选进行精排。

5.2 Retrieved Result Compression：检索结果压缩

检索文档可能包含大量与查询无关的信息，直接输入 LLM 会增加 Token 成本，也可能干扰模型识别关键证据。

Retrieved Result Compression 通过压缩器将长文档转换为更简洁的上下文。

$\mathcal D_q \xrightarrow{\text{Compressor}} c_q$，其中 c_q 是压缩后的上下文。

RECOMP 推理流程为： Retrieve → Compress → Prepend → Black-box LM

**Compressor Learning Objectives **压缩器需要同时满足三个目标：

目标	含义
Concise	尽可能简洁，降低输入成本
Effective	保留有助于下游任务的信息
Faithful	忠实于原始检索内容，不引入虚构事实

这三个目标之间存在权衡：过度压缩可能遗漏关键证据，而仅追求信息完整又无法有效减少 Token。

RECOMP 案例：以日产 Xterra 停产年份为例 When did they stop making the Nissan Xterra?

方法	Context Tokens	Answer	结果
No Retrieval	0	2010	错误
RALM	749	2015	正确
RECOMP	58	2015	正确

$\text{Token Reduction} = \frac{749-58}{749}\times100% \approx92.3% $

压缩比为： $\text{Compression Ratio} = \frac{749}{58} \approx12.91$

说明 RECOMP 在显著减少上下文长度的同时，保留了回答问题所需的关键事实。

5.3 Corrective RAG（CRAG）：纠正式检索增强生成

传统 RAG 往往假设检索结果可直接作为生成依据，但检索器可能返回不相关、错误或存在歧义的文档。

Corrective RAG 引入 Retrieval Evaluator，对检索结果进行评估，并根据结果质量选择不同的知识处理策略。

基本控制流程

****Question → Retrieve → Grade

若检索结果可靠，则直接使用相关证据；若发现不相关或不可靠文档，则

Rewrite Query → Web Search → Answer

两个问答案例对比：

- Henry Feilden 的职业问题：检索到正确人物信息，回答 Politician。
- Death of a Batman 的编剧问题：检索结果混入其他 Batman 作品信息，模型错误回答 Hamm。

该案例说明，错误检索结果可能诱导生成器给出看似合理但实际错误的答案。

CRAG 的三种检索结果状态

完整架构中，Retrieval Evaluator 将结果分为：

状态	处理策略	生成上下文
Correct	Knowledge Refinement	$x + k_{\text{in}}$
Ambiguous	Refinement + Knowledge Searching	$x + k_{\text{in}} + k_{\text{ex}}$
Incorrect	Knowledge Searching	$< br > x + k_{\text{ex}}$

其中：

- x：用户查询。
- k_{in}：从原始检索结果中提炼的知识。
- k_{ex}：通过外部搜索获得的补充知识。

这里的内部知识指初始检索语料中精炼出的内容，并不等同于模型参数化知识。

Knowledge Refinement

对原始检索文档进行： $ \text{Decompose} \rightarrow \text{Filter} \rightarrow \text{Recompose}$

即先将文档分解为细粒度片段，再筛除无关信息，最后重组为更可靠的上下文。

Knowledge Searching

当初始检索结果不可靠时： $x \xrightarrow{\text{Rewrite}} q' \xrightarrow{\text{Web Search}} \{k_1,\ldots,k_n\} \xrightarrow{\text{Select}} k_{\mathrm{ex}}$

包含 Death of a Batman; screenwriter; Wikipedia 的查询作为示例，说明外部搜索可以通过更明确的查询重新寻找证据。

CRAG 的核心贡献是将检索结果质量评估引入生成流程，避免模型无条件信任初检文档。

5.4 Refiner：相关证据抽取与结构化

Refiner 使用单个 Decoder-only LLM，从检索文档中自适应抽取与查询相关的原文内容，并保留必要上下文。

Adaptively extract query-relevant contents verbatim along with the necessary context. 其中 Verbatim 强调尽可能忠实地抽取原始文本，而非自由改写或生成新的事实。

Refiner 的处理流程

$ (q,\mathcal D_q) \xrightarrow{\text{Refiner}} \mathcal C_q$

$ \mathcal C_q \xrightarrow{\text{Downstream LLM}} y$

其中 c_q 为筛选和结构化后的证据。

课件案例

用户询问：Are North Marian High School and Seoul High School both located in the same country?

检索结果包含四个学校的信息。课件中的问题使用 North Marian，而检索结果使用 North Marion，存在拼写不一致。该差异也提示，实体名称匹配错误可能影响证据筛选。

Refiner 通过 Extract and Structure，保留两段关键证据：

1.1 Seoul High School：位于 South Korea。

1.2 North Marion High School：位于 United States。

两种结果：

- Without Refiner：模型受到无关学校信息干扰，错误判断两所学校位于同一国家。
- With Refiner：模型依据筛选后的证据，正确判断两所学校位于不同国家。

Refiner 与 Compression 的区别

比较	Compression	Refiner
核心操作	压缩长文档	抽取查询相关内容
主要目标	降低 Token 成本	减少无关证据干扰
内容处理	可以进行摘要式改写	强调 Verbatim Extraction
输出	精简摘要或压缩上下文	结构化相关证据
代表方法	RECOMP	Refiner

两者都属于检索后优化，但Compression 主要解决上下文冗长，Refiner 更强调证据相关性与保真性。

5.5 RAG 技术体系的综合比较与总结

整个 RAG 系统可以按照信息流分为三个优化阶段：

1. Pre-Retrieval 优化查询表达

Rewriting · Decomposition · HyDE · Expansion

2. Retrieval 从知识库中召回候选证据

BM25 · DPR · Contriever · Search Engines

3. Post-Retrieval 优化检索结果的质量与内容

Re-ranking · Compression · Correction · Refiner

Generation 综合查询与证据生成答案

关键技术对照

技术	处理对象	核心机制	主要作用
BM25	查询、文档	稀疏词项匹配	高效关键词检索
DPR	查询、段落	双编码器、有监督对比学习	提高特定 QA 任务的检索能力

技术	处理对象	核心机制	主要作用
Contriever	查询、文档	共享编码器、无监督对比学习	提高通用语义检索能力
Prompt Retriever	提示模板	候选提示评分与学习	选择有效提示
CEIL	上下文示例	检索得分与 LM 得分对齐	优化示例选择
Query Rewriting	查询	重新表达信息需求	改善查询适配性
HyDE	查询	生成假设文档后检索	改善查询与文档的表示匹配
Query Expansion	查询	原查询与生成内容拼接	扩大检索覆盖范围
Re2G	候选文档	交互式重排	改善候选排序
RECOMP	检索文档	内容压缩	减少 Token 与冗余信息
Corrective RAG	检索结果	质量评估、精炼与外部搜索	修正不可靠证据
Refiner	检索文档	相关内容抽取与结构化	减少证据噪声

重要概念辨析

1. Sparse Retrieval 与 Dense Retrieval

前者依靠词项统计匹配，后者依靠稠密向量的语义相似性。两者可以互补，并非简单的技术替代关系。

2. DPR 与 Contriever

DPR 使用两个独立编码器，通过有监督问答数据学习；Contriever 使用共享编码器，通过无监督对比学习获得通用检索能力。

3. Query Rewriting、HyDE 与 Query Expansion

Rewriting 重新表达查询；HyDE 生成假设文档作为检索输入；Expansion 保留原查询并追加生成内容。

4. Retriever 与 Reranker

Retriever 面向大规模语料进行高效召回，Reranker 对较少候选进行精细评分。两者通常构成粗排与精排的两阶段架构。

5. RECOMP、CRAG 与 Refiner

RECOMP 主要处理上下文冗余，CRAG 主要处理证据质量不可靠，Refiner 主要处理相关证据的抽取与组织。

RAG 的效果不仅取决于是否连接外部知识库，还取决于整条信息处理链的质量。

可以将系统性优化目标概括为：

$${ \begin{aligned} \text{RAG Quality}\ \longleftarrow;&\text{Query Quality}+\text{Retrieval Quality}+\text{Evidence Quality}+\text{Generation Quality} \end{aligned} } $$

LLM 的参数化知识存在局限，因此引入外部检索；检索器决定能够找到哪些信息，检索前优化改善信息需求的表达，检索后优化提高证据的相关性、可靠性与紧凑性，生成器最终利用这些证据完成任务。

Part 6 Agent Memory 智能体记忆

本章围绕智能体如何存储、检索、更新和利用历史信息展开。

核心问题：如何突破大语言模型有限上下文窗口与静态参数知识的限制，使智能体能够在持续交互中积累经验、形成知识、复用技能，并实现长期学习。

本章的知识体系可以归纳为五个层次：

1. 记忆的必要性：LLM 的参数记忆与上下文记忆存在局限，传统 RAG 不能完全替代智能体记忆。
2. 记忆的认知基础：借鉴人类长期记忆的海马体索引理论、模式分离与模式补全机制。
3. 记忆的内容分类：情景记忆（Episodic）、语义记忆（Semantic）和程序性记忆（Procedural）。
4. 记忆的实现与学习：从外部文本存储到参数微调、提示优化、代码库改进及自我反思。
5. 记忆的管理机制：有限上下文下的外部记忆读写、召回、更新与冲突消解。

6.1 智能体记忆的动机与基本概念

6.1.1 从人类持续学习到智能体记忆

人类及多数动物能够通过持续经历形成长期记忆，而传统 LLM 的知识主要来自预训练阶段。模型完成训练后，参数通常保持固定，新的交互信息不会自动转化为永久知识。

课件通过神经突触变化说明生物记忆的形成过程：

- 短期记忆（Short-term Memory）：与短暂的神经活动及突触功能变化相关。
- 长期记忆（Long-term Memory）：涉及更持久的突触结构变化和神经连接调整。

这一类比揭示了智能体设计的基本要求：不仅需要在当前任务中利用信息，还应能够将重要经验保存下来，供后续任务使用。

然而，直接通过修改 LLM 参数实现持续学习存在困难。

1. 灾难性遗忘（Catastrophic Forgetting）

模型学习新知识时，参数更新可能破坏原有知识，导致旧任务性能下降。

2. 涟漪效应（Ripple Effects）

知识之间存在关联。修改一项事实后，模型可能无法同步更新相关知识。

例如，将某人物的国籍由国家 A 修改为国家 B，可能还需要同步更新其所属国家、语言及其他关联事实。

3. 瞬时学习（Transient Learning）

新知识可能仅在当前上下文中有效，或者在后续参数更新中无法稳定保留。

因此，智能体需要一种能够持续存储、组织和调用信息的记忆机制，而不能完全依赖参数更新。

6.1.2 参数记忆与非参数记忆

LLM 的知识存储可以分为两种主要形式。

维度	参数记忆（Parametric Memory）	非参数记忆（Non-parametric Memory）
存储位置	模型权重	外部数据库、文档、记忆库
知识更新	通常需要训练或编辑参数	可以直接修改外部数据
信息访问	通过模型推理隐式调用	通过检索显式获取
持久性	随模型权重保存	由外部存储系统保证

维度	参数记忆（Parametric Memory）	非参数记忆（Non-parametric Memory）
主要局限	更新成本、遗忘、知识冲突	检索误差、信息质量、上下文限制

非参数记忆能够提供额外证据，但并不保证模型正确使用这些证据。

课件研究，说明 LLM 在外部证据与参数知识冲突时，可能接受具有连贯性和说服力的外部信息。

实验包含两种情况：

- Correct Memory：模型原有知识正确，但外部信息提供错误事实。
- Wrong Memory：模型原有知识错误，而外部信息提供正确事实。

例如，模型知道法国首都是巴黎，但外部文本声称首都是尼姆（Nîmes）。模型可能受到外部文本影响而改变答案。这一现象具有双重含义：

- 外部记忆可以帮助纠正模型的过时或错误知识。
- 外部记忆也可能引入错误信息，导致模型放弃原本正确的知识。

因此，记忆系统不仅需要检索能力，还需要证据可信度判断、知识冲突检测和更新控制。

6.1.3 短期记忆与长期记忆

智能体通过与环境交互形成行动—观察循环： $ \text{Reasoning} \rightarrow \text{Action} \rightarrow \text{Observation} \rightarrow \text{Reasoning}$

在这一过程中，记忆负责保存过去的观察、行动和推理结果。

特征	短期记忆（STM）	长期记忆（LTM）
主要载体	当前上下文窗口	外部存储或持久化模型能力
写入方式	追加当前对话与工具结果	显式写入、更新或学习
容量	受上下文长度限制	可扩展
生命周期	通常局限于当前任务或会话	可以跨任务、跨会话保留
主要功能	支持即时推理	支持经验积累和知识复用

课件强调，短期记忆通常具有三个特点：

- Append-only：交互内容不断追加。
- Limited context：上下文容量有限。
- Limited attention：即使信息仍处于上下文中，模型也不一定能够有效关注。

长期记忆则强调 Read and Write，即既能检索已有信息，也能写入新的经验、知识与技能。

需要注意：短期记忆和长期记忆的区别主要是生命周期与使用机制，并不等同于是否使用向量数据库。

6.2 RAG 与 Agent Memory 的关系

6.2.1 传统 RAG 的局限

检索增强生成（Retrieval-Augmented Generation, RAG）通过检索外部知识，为模型生成答案提供参考。

其基本过程为： $q \xrightarrow{\text{Retriever}} D_q \xrightarrow{\text{LLM}} y$

其中：

- q：用户查询。
- D_q：检索得到的相关文档。
- y：最终生成结果。

传统 RAG 的知识库通常具有只读特征： Retrieve $(q) \rightarrow D_q$。这种知识库为 Read-only LTM（只读长期记忆）。因为传统检索语料往往由外部人员编写，例如维基百科编辑者，而非智能体自身产生。

由此带来两方面限制：

1. 经验来源限制：外部知识库主要保存他人的知识和经验，未必适合当前智能体的特定任务。
2. 存储结构限制：文档最初可能面向人类阅读，而不是为智能体的高效检索和推理设计。

相比之下，Agent Memory 需要支持自主写入： Write $(m_t, M_t) \rightarrow M_{t+1}$

其中，M_t 为当前记忆状态，m_t 为新产生的记忆内容。

因此，Agent Memory 不仅需要 Retrieve，还需要 Write、Update 和 Delete 等操作。

6.2.2 为什么传统 RAG 不总是有效

课件使用一个关于斯坦福大学教授与阿尔茨海默病神经科学研究的问答示例，说明传统 RAG 在复杂关联检索中的不足。传统 RAG 通常根据查询与文档之间的相似度返回相关片段，但复杂问题可能要求模型跨多个事实建立关联。

例如： $\text{Stanford Professor} \rightarrow \text{Research Area} \rightarrow \text{Alzheimer's Disease} $

如果相关信息分散在不同文档中，仅依靠单次相似度检索可能无法恢复完整关系。通过人类记忆进行对比，强调人类可以借助已建立的关联，从部分线索恢复相关信息。

由此引出智能体记忆的两个关键要求：

- 关联组织：不仅存储独立文本，还要建立信息之间的联系。
- 关联检索：能够沿已有关系逐步查找，而不是只执行一次相似度匹配。

这也为后续的记忆索引、遍历式检索与推理增强检索提供了动机。

6.3 人类长期记忆的认知基础

6.3.1 海马体索引理论

海马体索引理论（Hippocampal Indexing Theory），作为智能体长期记忆设计的认知启发。

- 新皮层（Neocortex）：参与存储分布式记忆表征。
- 海马体（Hippocampus）：形成指向这些表征及其关联的索引。

记忆可以涉及人物、事件、地点等不同信息。这一理论说明，长期记忆不一定需要将所有内容存储在同一个位置，而可以采用： $\text{Memory Index} \rightarrow \text{Stored Representations} $。索引负责定位和关联，具体记忆内容则分布存储。

6.3.2 模式分离与模式补全

海马体索引机制支持两种重要的记忆能力。

1. 模式分离（Pattern Separation）

将相似经历区分为不同记忆，避免信息相互混淆。例如，两次发生在同一家餐厅的会议虽然地点相同，但时间、参与者和讨论内容不同，系统应将其区分为不同事件。

课件将这一机制与新皮层及海马旁区域（Parahippocampus）联系起来。

2. 模式补全（Pattern Completion）

利用部分线索和已有关联恢复较完整的记忆。例如，用户只提到某次项目会议的地点，系统就可能根据关联找到会议时间、参与者和决策结果。

课件指出，这一机制主要与海马体，特别是 CA3 区域有关。

机制	目标	对 Agent Memory 的启发
Pattern Separation	区分相似记忆	去重、消歧、避免记忆混合
Pattern Completion	从局部线索恢复完整信息	关联检索、多跳检索、上下文恢复

这两种机制相互补充：模式分离提高记忆的可区分性，模式补全提高记忆的可恢复性。

6.4 智能体长期记忆的分类

课件借鉴人类长期记忆体系，将 Agent Memory 按内容分为三类：

记忆类型	英文	存储内容	代表系统
情景记忆	Episodic Memory	经历、事件、交互记录	Generative Agents
语义记忆	Semantic Memory	事实、概念、归纳知识	Generative Agents
程序性记忆	Procedural Memory	技能、操作方法、可执行程序	Voyager

这一分类基于记忆内容，而非底层实现方式。例如，三类记忆都可以使用文本数据库或向量数据库保存，但其语义和用途不同。

6.4.1 情景记忆：Episodic Memory

情景记忆保存智能体经历过的具体事件。

其基本形式为： $m_i=(t_i,e_i,c_i) $

- t_i：事件发生时间。
- e_i：事件内容。
- c_i：事件相关上下文。

该表达式是对课件中事件流结构的抽象，并非课件规定的固定数据格式。课件以 Generative Agents 为例，展示了事件记忆的写入与读取机制。

写入：Append-only Event Streams。

智能体持续记录观察到的事件，形成 Memory Stream（记忆流）。例如：某角色正在计划情人节聚会、某角色邀请朋友参加活动、某角色正在研究聚会安排。这些记录按照时间顺序追加。

读取：Heuristic Retrieval

由于事件流可能非常庞大，系统需要根据启发式评分选择相关记忆。

三个评分因素：

1. Recency：近期性。
2. Importance：重要性。
3. Relevance：相关性。

可以将综合评分概括为： $S(m_i, q) = \alpha R_{\text{recency}}(m_i) + \beta R_{\text{importance}}(m_i) + \gamma R_{\text{relevance}}(m_i, q)$

其中 α, β, γ 为权重。

示例中，系统根据“现在最期待什么”这一问题，从大量事件中检索与情人节聚会有关的记忆，进而生成回答。

因此，情景记忆的核心机制是： Event Recording → Memory Retrieval → Contextual Response

6.4.2 语义记忆：Semantic Memory

语义记忆保存从经历中抽象出的稳定知识，而不是单个事件本身。

Write: LLM reasoning over events 即通过 LLM 对多个事件进行推理、归纳和反思，形成更高层次的知识。

例如，智能体观察到：

- Klaus 正在阅读研究文章。
- Klaus 正在做笔记。
- Klaus 正在与图书管理员讨论研究材料。

系统可以将这些观察归纳为：Klaus Mueller is dedicated to research.

这种由具体观察形成抽象结论的过程，可以表示为： $\{e_1,e_2,\ldots,e_n\} \xrightarrow{\text{Reflection}} k$

其中 k 表示归纳得到的语义知识。

语义记忆的读取方式同样是 Retrieval，但检索对象从具体事件扩展为抽象事实和经验总结。

情景记忆与语义记忆的主要区别：

- 情景记忆回答“发生过什么”。
- 语义记忆回答“从这些经历中知道了什么”。

二者可以相互转化：事件不断积累，经由反思形成语义知识。

6.4.3 程序性记忆：Procedural Memory

程序性记忆保存可重复使用的技能或操作程序。课件使用 Voyager 展示程序性记忆的实现。

Voyager 的三个核心组成部分是：

1. Automatic Curriculum（自动课程）：根据当前能力与探索进度生成新的任务。
2. Iterative Prompting Mechanism（迭代提示机制）：结合执行反馈和错误信息反复改进程序。
3. Skill Library（技能库）：保存已验证的代码技能，供后续任务复用。

例如，智能体可能逐步掌握：Mine Wood Log：采集木材、Make Crafting Table：制作工作台、Craft Stone Sword：制作石剑、Make Furnace：制作熔炉、Combat Zombie：对抗僵尸。

其执行过程为： $\text{Task} \rightarrow \text{Skill Retrieval} \rightarrow \text{Code Execution} \rightarrow \text{Feedback} \rightarrow \text{Skill Update} $

课件强调：

- Write: Code-based skills：将技能保存为代码。
- Read: Embedding retrieval：通过向量检索找到相关技能。

当执行失败时，系统根据环境反馈与错误信息修正代码；当技能通过自我验证后，将其加入技能库。

与情景记忆相比，程序性记忆的重点不是记录过去，而是将成功经验转化为未来可直接执行的能力。

6.5 长期记忆的实现机制

6.5.1 基础实现：文本存储与检索

课件将最简单的长期记忆实现称为 Naïve Implementation。它包含两个操作：

Write 将新的经验追加到文本语料库： $M_{t+1} = M_t \cup \{m_t\}$

Read 使用现成检索器获取相关记忆： $M_q = \text{Retrieve}(q, M)$

例如：BM25、Neural Embeddings（神经向量表示）。

这种实现简单、易于扩展，但主要依赖文本相似性，可能难以处理复杂的关联关系。

6.5.2 改进方向：关联遍历与索引增强

两类更复杂的实现方式：

1. Online Retrieval by Traversal

通过遍历已有关系进行在线检索。与直接返回相似文本不同，这类方法允许系统沿记忆之间的关联逐步查找。

例如： $m_1 \rightarrow m_2 \rightarrow m_3$

这与前文模式补全及复杂关系检索的需求相对应。

2. Index Augmentation with Reasoning/Referral

在写入阶段利用推理或引用关系增强记忆索引。

例如，一条新记忆除了保存文本内容，还可以建立与已有事件、人物或知识的关联。

这意味着记忆写入不再只是追加文本，还包括组织和维护索引结构。

两类方法分别改善：读取阶段的关联探索能力、写入阶段的知识组织能力。

6.6 从记忆到学习：长期能力积累的统一视角

将长期记忆进一步扩展为智能体学习的统一框架。这里的 Memory 不仅指外部文本数据库，还包括能够跨任务保留的模型参数、提示策略、程序和反思经验。

6.6.1 参数微调 Fine-tuning Model Weights

最直接的学习方式是更新模型权重： $\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$

其中：

- θ_t：当前模型参数。
- η：学习率。
- $\mathcal{L}$：训练损失函数。

课件展示了通过多任务问题、多种推理方法和工具调用轨迹生成训练数据的过程。

涉及的方法包括：Chain-of-Thought（CoT）、ReAct、Reflexion。

系统保留正确的执行轨迹，删除错误轨迹，并将有效轨迹重新格式化用于微调。例如，模型可以学习在不同任务中隐式选择适合的推理方法，而不是始终采用固定策略。这种方式将经验写入参数记忆，但可能存在训练成本和灾难性遗忘问题。

6.6.2 跨任务提示优化 Prompt Optimization

第二种方式不直接修改模型参数，而是优化提示指令。一个提示优化过程：

$P_{t+1} = \text{Optimize}(P_t, \mathcal{F}_t)$

- P_t：当前提示。
- $\mathcal{F}_t$：评估反馈。
- P_{t+1}：改进后的提示。

例如，系统针对一组语言任务生成多个候选指令，通过评分模型评估其效果，再保留表现更好的指令。

课件中的图表比较了不同提示优化方法在未见任务上的表现，说明提示优化可以提高跨任务泛化能力。

这类长期记忆保存的是如何指导模型完成任务的策略。

6.6.3 改进智能体自身代码库

第三种方式是将学习成果保存到代码库。

以 SWE-agent 的 Agent - Computer Interface 为例，展示智能体与计算机环境的交互。

智能体可以：Navigate Repository：浏览代码仓库、Search Files：搜索文件、View Files：查看文件、Edit Lines：编辑代码。

环境通过终端、文件系统等接口返回反馈。与参数微调不同，这种方式通过修改外部程序改变系统的后续行为。因此，代码库可以成为一种长期能力存储载体。

6.6.4 保存事件与示例供未来检索

第四种方式是将执行经验保存为可检索的示例。

基本过程为： $\text{Experience} \rightarrow \text{Memory Storage} \rightarrow \text{Future Retrieval}$

其典型实现就是前文 Generative Agents 的 Memory Stream。

这种方式无需改变模型参数，能够通过持续积累历史事件支持后续决策。

6.6.5 自我反思 Self-Reflection

第五种方式是将失败经验和改进建议写入后续提示。

课件使用 Reflexion 来说明这一机制。

基本流程为： $\text{Query} \rightarrow \text{LLM} \rightarrow \text{Action} \rightarrow \text{Environment} \rightarrow \text{Reward} $

随后系统根据行动轨迹和奖励进行反思： $r_t = \text{Reflect}(\tau_t, R_t)$

- τ_t：执行轨迹。
- R_t：环境反馈或奖励。
- r_t：生成的反思内容。

反思内容被加入后续提示，指导下一次尝试。

其特点是：

- 不一定需要更新模型权重。
- 将失败经验转化为自然语言反馈。
- 通过多轮尝试改善后续行为。

6.6.6 多种学习方式的组合

课件最后以 BetterTogether 为例，强调不同学习机制可以组合使用。

例如，交替进行提示优化和参数微调：

$P_{t+1} = \text{OptimizePrompt}(P_t, \theta_t)$

$\theta_{t+1} = \text{FineTune}(\theta_t, P_{t+1})$

核心思想：提示与参数可以协同优化，而不是必须单独选择一种方式。

五种学习机制：

学习方式	主要更新对象	保留的长期能力
Fine-tuning	模型参数	模型内部知识与行为模式
Prompt Optimization	提示指令	任务执行策略
Codebase Improvement	代码库	可执行技能与程序
Experience Storage	记忆库	历史事件与示例
Self-Reflection	反思记录或提示上下文	错误经验与改进建议

这些方法共同说明：智能体长期学习不局限于参数训练，也可以通过外部持久化信息和可复用行为实现。

6.7 外部记忆管理

6.7.1 有限上下文下的记忆架构

即使拥有长期记忆，LLM 在单次推理中仍受到上下文窗口限制。以 MemGPT 风格的外部记忆架构为例，展示了有限上下文与外部存储之间的协作。主要组成部分如下：

组件	作用
System Instructions	保存系统级指令，通常只读
Working Context	保存当前工作信息，可读写
FIFO Queue	按先进先出方式管理近期消息
Output Buffer	为模型输出保留空间
Archival Storage	长期归档存储
Recall Storage	保存可召回的历史交互
Function Executor	执行记忆读写工具
Queue Manager	管理消息队列与上下文

其中，FIFO（First In, First Out）表示先进先出。当上下文窗口接近容量限制时，系统需要决定哪些信息继续保留在工作上下文中，哪些信息移至外部存储。

其基本关系为： Working Context ↔ External Memory

6.7.2 记忆读写作为工具调用

Reads and writes are function calls the agent issues itself. 记忆的读取与写入由智能体主动发起工具调用完成。

例如：recall_storage.search("six flags")

智能体可以根据当前任务检索历史记录，并将结果重新引入上下文。

这一机制具有三个重要特征：

1. 自主性：智能体决定何时检索或写入记忆。
2. 持久性：信息不依赖当前上下文窗口长期存在。
3. 选择性：只将当前任务需要的信息加载进上下文。

因此，外部记忆管理的核心不只是增加存储容量，还包括控制信息何时进入模型的工作上下文。

6.8 记忆冲突与动态更新

6.8.1 记忆更新的必要性

长期记忆可能包含过时、重复或相互矛盾的信息。

例如：

- 旧记忆：用户目前居住在城市 A。
- 新消息：用户已经搬到城市 B。

如果系统仅追加新记录而不处理旧信息，后续检索可能同时返回两个冲突事实。

因此，长期记忆需要动态更新机制。

6.8.2 记忆提取与更新流程

以 mem0 风格的记忆管理流程为例，将其分为两个阶段。

第一阶段：Extraction Phase

系统读取：当前消息、最近交互记录、已有摘要。

通过 LLM 提取新的候选记忆。

$M_{\text{new}} = \text{Extract}(H_t)$

其中 H_t 表示当前可用的交互历史。

第二阶段：Update Phase

系统从已有记忆中检索与新记忆最相似的若干条：

$M_{\text{similar}} = \text{TopK}(M_{\text{new}}, M)$

然后根据新旧信息之间的关系选择更新操作。

6.8.3 四种记忆更新操作

操作	触发条件	处理方式
ADD	不存在相似记忆	当新信息尚未被保存时，创建新的记忆记录
UPDATE	新消息修改已有事实	当新信息改变已有事实时，修改对应记忆
DELETE	新消息否定已有事实	当新信息明确否定已有事实时，删除不再成立的记忆
NOOP	信息已存在或无关	当信息重复、无关或无需保存时，不执行写入

例如：旧记忆：用户正在学习课程 A；新消息：用户已经完成课程 A。系统应更新学习状态，而不是保留两个相互矛盾的当前状态。

这四种操作共同构成长期记忆的基本维护机制： $M_{t+1} = \text{UpdateMemory}(M_t, m_t, o_t)$

其中 $o_t\in {\text{ADD},\text{UPDATE},\text{DELETE},\text{NOOP}} $

需要区分：这是一种对课件更新流程的形式化概括，实际系统还可能涉及时间戳、来源和版本管理等机制。

6.9 智能体记忆的整体架构与总结

6.9.1 外部环境与内部记忆

External Environment 与 Internal Memory 有什么区别？二者都可以通过工具访问，但功能不同。

维度	外部环境（External Environment）	内部记忆（Internal Memory）
核心功能	提供交互对象与行动反馈	保存信息、经验和技能
典型操作	Action / Observation	Read / Write
状态变化	行动可能改变环境	写入或学习改变记忆
主要作用	执行任务、感知世界	支持推理、检索和长期学习

例如，智能体访问网页属于与外部环境交互；将网页中的重要事实保存到长期记忆，则属于记忆写入。

二者并非绝对独立：数据库、文件系统等技术载体既可能承载环境状态，也可能承载智能体记忆。区别主要在于其在系统中的功能。

6.9.2 Agent Memory 的完整工作循环

综合本章内容，智能体的持续学习过程可以抽象为：

这一过程包含两个相互关联的循环：

任务执行循环 Reason → Act → Observe

记忆学习循环 Experience → Store → Retrieve → Reflect → Update

前者使智能体能够完成当前任务，后者使智能体能够积累经验并改善未来表现。

6.9.3 本章核心知识

核心问题	对应概念或技术
为什么需要记忆?	有限上下文、静态知识、灾难性遗忘
为什么传统 RAG 不够?	只读知识库、缺少自主写入、关联检索不足
人类记忆提供什么启发?	海马体索引、模式分离、模式补全
记忆按内容如何分类?	Episodic / Semantic / Procedural
如何读取和写入记忆?	Retrieval、Append、Index Augmentation
如何实现长期学习?	Fine-tuning、Prompt Optimization、Codebase Improvement、Experience Storage、Reflexion
如何管理有限上下文?	Working Context、FIFO Queue、Archival Storage、Recall Storage
如何处理记忆冲突?	ADD、UPDATE、DELETE、NOOP

6.10 本章总结

Agent Memory 的本质是构建一种能够跨任务持续保留和利用信息的机制。

本章的逻辑可以归纳为三个层次。

第一，记忆是对 LLM 固有局限的补充。 参数知识难以实时更新，上下文窗口无法保存全部历史信息，传统 RAG 又主要依赖外部只读知识。因此，智能体需要能够自主读写的长期记忆。

第二，长期记忆不仅是存储，还涉及知识组织和能力形成。 情景记忆记录经历，语义记忆归纳知识，程序性记忆保存技能。通过检索、反思和关联机制，智能体能够将过去的经历转化为当前任务所需的信息与能力。

第三，记忆系统需要动态管理。 记忆的价值取决于能否准确检索、合理更新、处理冲突，并与当前推理过程有效结合。长期学习还可以通过参数微调、提示优化、代码改进和反思等多种方式实现。

最终，智能体的能力不再只由预训练模型决定，而由模型、外部记忆、工具和环境交互共同形成。

$$\text{Agent Capability} = f(\text{Model}, \text{Memory}, \text{Tools}, \text{Environment})$$

Lecture 8 Agent Infrastructure 智能体基础设施

本讲围绕智能体系统的基础设施展开，重点研究两个相互关联的问题：

1. 推理基础设施（Inference Infrastructure）：如何通过 KV Cache 复用、分层存储和选择性重计算，降低长上下文推理的延迟与成本。
2. 智能体软件架构（Agent Software Architecture）：如何利用复合 AI 系统（Compound AI Systems）与 DSPy，将单一模型调用转化为可组合、可控制、可优化的程序。

两部分分别从计算资源管理与程序结构设计出发，提高智能体系统的效率、可靠性和可维护性。

Part1 长上下文推理与 KV Cache

1.1 长上下文推理的机遇与挑战

长上下文推理（Long-context Inference）使大语言模型能够在单次请求中处理大量信息，包括文档、书籍、新闻、代码仓库、对话历史、视频与会议记录等。其核心价值不仅是扩大上下文窗口（Context Window），还在于使模型能够利用更多外部信息完成复杂任务。

上下文长度增加会带来三方面的问题，这三种代价共同限制了长上下文推理的大规模部署。

1. 推理延迟：输入越长，响应时间越长

LLM 推理通常分为两个阶段：

- Prefill（预填充）：处理输入上下文，计算各层的注意力中间状态。
- Decode（解码）：基于已有状态，自回归地逐个生成输出 Token。

总推理时间可以表示为： $T_{\text{total}} = T_{\text{prefill}} + T_{\text{decode}} + T_{\text{other}}$

其中，T_{other} 包括排队、调度、数据传输等开销。

对于标准全注意力 Transformer，长度为 L 的输入，其 Prefill 计算复杂度可近似表示为： $C_{\text{prefill}} = O(Ld^2 + L^2d)$

- L：输入 Token 数量；
- d：隐藏状态维度；
- $O(Ld^2)$：线性投影与前馈网络的主要计算项；
- $O(L^2d)$：标准注意力计算项。

因此，Prefill 的计算成本并非在所有情况下都严格随上下文长度线性增长。对于标准全注意力，注意力部分具有二次复杂度。

2. 推理成本：GPU 计算与资源消耗增加

长上下文需要更多 GPU 计算资源，并增加 KV Cache 的存储需求。GPU 成本随使用量和上下文长度增加。实际成本不仅取决于 Token 数量，还与模型规模、并发程度、硬件利用率和缓存命中率有关。

3. 输出质量：长上下文中的长尾知识可能难以被准确利用

长上下文并不保证模型能够有效利用其中的全部信息。随着上下文长度增加，模型可能难以准确定位、整合和利用部分信息，尤其是长尾知识（Long-tail Knowledge）。

因此，上下文容量（Context Capacity）不等于有效信息利用能力（Effective Context Utilization）；长上下文优化既需要解决计算效率问题，也需要关注信息利用质量。

1.2 KV Cache 的原理

KV Cache（Key-Value Cache）是 Transformer 自回归推理中的一种中间状态缓存机制。

在注意力计算中，输入隐藏状态 X 通过线性变换生成： $Q = XW_Q, K = XW_K, V = XW_V$

注意力输出为： $\operatorname{Attention}(Q,K,V) = \operatorname{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right)V$

- Q（Query）：当前 Token 用于查询其他 Token 的表示；
- K（Key）：用于与 Query 计算匹配程度；
- V（Value）：用于生成注意力加权结果。

在因果注意力（Causal Attention）中，历史 Token 不依赖未来 Token。因此，生成新 Token 时，已有 Token 的 K、V 可以在相同计算条件下复用。

若已经生成 t 个 Token，则： $K_{\text{cache}}^{(t)} = [K_1, K_2, \cdots, K_t]$，$V_{\text{cache}}^{(t)} = [V_1,V_2,\ldots,V_t] $

生成第 $t+1$ 个 Token 时，只需计算新的 K、V，并更新缓存：$K_{\text{cache}}^{(t+1)} = [K_{\text{cache}}^{(t)}; K_{t+1}]$， $V_{\text{cache}}^{(t+1)} = [V_{\text{cache}}^{(t)}; V_{t+1}]$

KV Cache 避免了对历史 Token 的重复 K、V 计算，但新 Token 仍需对历史缓存执行注意力操作，因此不能将其理解为消除了全部注意力计算。

KV Cache 的基本流程

该流程说明：无论 KV 是重新计算还是从缓存加载，只要缓存有效且计算语义一致，后续生成阶段就能够使用它。

1.3 Prefill 瓶颈与推理性能指标

将长上下文推理的主要瓶颈定位于 Prefill 阶段，并重点使用两个指标评估性能。

1. Time to First Token（TTFT，首 Token 延迟）

指从提交请求到产生第一个输出 Token 的时间。

$TTFT = T_{\text{queue}} + T_{\text{prefill}} + T_{\text{first decode}} + T_{\text{overhead}}$

长上下文通常会显著增加 T_{prefill}，从而提高 TTFT。

2. Inter-Token Latency（ITL，Token 间延迟）

指相邻两个输出 Token 之间的时间间隔：$ITL_i=t_{i+1}-t_i$ ，*平均ITL可表示为*：$\overline{ITL} = \frac{1}{N-1} \sum_{i=1}^{N-1}(t_{i+1}-t_i) $

TTFT 衡量开始响应的速度，ITL 衡量持续生成的速度。

二者并不等价：减少 Prefill 时间可能显著改善 TTFT，但不一定提高 Decode 阶段的生成速度。

1.4 跨请求 KV Cache 复用

传统 KV Cache 主要用于同一请求内部的自回归解码，而 PPT 进一步提出跨请求、跨推理引擎的缓存复用。

其基础是：Contexts are often shared across queries.

多个请求可能共享相同的上下文，例如：

- Book：针对同一本书进行多次问答；
- News：对相同新闻材料执行不同分析；
- Docs：企业内部多个用户访问相同文档；
- Chat History：多轮对话重复包含历史消息。

若每个请求都独立执行 Prefill，就会对共享内容进行重复计算。

You Only Prefill Once（YOPO）

当一个推理引擎已经完成某段上下文的 Prefill 后，其他引擎应尽可能复用已有 KV Cache，而非重新计算。

设共享上下文的 Prefill 成本为 C_p，共有 N 次请求，不使用跨请求缓存时：$C_{\text{no-cache}} = NC_p$。理想情况下，仅首次请求需要计算：$C_{\text{cache}} = C_p$，理论上可节省：$S=1-\frac{1}{N} $。

实际系统还需要考虑缓存查找、加载、传输和维护开销，因此并不能达到完全零成本的复用。

结论： KV Cache 从单次推理的计算优化机制，进一步发展为跨请求共享的计算结果。其价值在于减少重复 Prefill，但要实现大规模复用，还必须解决存储容量、数据传输和缓存管理问题。

Part 2 KV Cache 的分层存储与跨设备管理

2.1 KV Cache 的存储容量问题

为了最大化 KV Cache 复用，系统需要存储更多缓存，并延长其保留时间。

理想情况下，每份 KV Cache 应当保存至最后一次复用完成。但由于未来请求无法准确预测，实际系统需要根据访问频率、存储成本和容量限制制定缓存保留与淘汰策略。

KV Cache 的存储量计算

对于具有 N_{layers} 层的 Transformer，KV Cache 的理论存储量为：$M_{\text{KV}} = 2LN_{\text{layers}}N_{\text{kv-heads}}d_{\text{head}}b$

- L：缓存的 Token 数量；
- N_{layers}：Transformer 层数；
- $N_{\text{kv-heads}}$：每层 KV Head 数量；
- d_{head}：每个 Head 的维度；
- b：每个元素的存储字节数；
- 系数 2：分别对应 Key 和 Value。

因此， $M_{\text{KV}}\propto L $即 KV Cache 大小与上下文长度线性相关。在其他条件相同时，增加上下文长度会等比例增加 KV Cache 容量。

对不同模型的 KV Cache 大小进行了比较，说明即使将整张 GPU 显存全部用于缓存，也只能保存有限数量的长文档。以 Llama 3 70B 为例，按图中约 300 GB / 10^6 Token 估算，100K Token 的 KV Cache 约为：$M_{\text{100K}} = 300\times\frac{100000}{1000000} = 30\text{ GB} $，若 H100 显存容量为 80 GB，理论上最多保存： $N_{\text{docs}} = \left\lfloor\frac{80}{30}\right\rfloor = 2$。

这尚未计入模型权重、运行时工作区和其他显存占用。

因此，仅依赖 GPU 显存无法满足大规模、长期的 KV Cache 复用需求。

2.2 分层存储架构 Hierarchical Storage

PPT 提出了 KV Cache 存储层次的扩展路线：

GPU Memory: 最快、容量有限，服务于当前推理

CPU RAM: 提供更大容量，支持 GPU - CPU Offloading

SSD: 持久化与大规模本地缓存

S3 / Remote Storage: 远程、跨节点及跨云共享

越往下，通常容量越大、单位容量成本越低，但访问延迟和传输代价越高。

技术演进顺序为： $\text{GPU (2024)} \Rightarrow \text{CPU RAM (2025 Q1)} \Rightarrow \text{SSD} \Rightarrow \text{S3 / Cross-cloud} $分。

分层存储的基本策略是将高频访问的热数据保留在高速存储中，将低频访问的冷数据迁移到容量更大、成本更低的存储介质中。

2.3 计算成本与存储成本的权衡

Compute (Prefill) Cost ≫ Storage (KV Cache) Cost，重新执行 Prefill 的成本通常显著高于保存 KV Cache 的成本。

需要注意，图中采用两种不同的计费单位：Compute：每百万 Token 的 Prefill 成本；Storage：每百万 Token 每小时的 KV Cache 保存成本。因此，比较时必须考虑缓存保存时间。

设：C_p：一次 Prefill 的计算成本；C_s：每小时保存对应 KV Cache 的成本；T：缓存保存时长；R：保存期间发生的有效复用次数；C_l：每次加载缓存的成本。

缓存的净收益为：$B = RC_p - (TC_s + RC_l)$，当$RC_p>TC_s+RC_l $缓存才具有正向经济收益。

进一步得到： $R(C_p - C_l) > TC_s$，在 $C_p > C_l$ 的前提下，复用次数的盈亏平衡阈值为： $R> \frac{TC_s}{C_p-C_l}$

这属于基于课件成本比较的进一步推导，并非 PPT 直接给出的公式。

2.4 三种 KV Cache 管理机制

传统缓存方式与三种跨 GPU 管理机制对比：

机制	核心操作	主要目的
Traditional KV Cache	缓存仅存在于单次请求和单个引擎中	加速同一请求的解码
CPU Offloading	GPU → CPU	扩展缓存容量与生命周期
Prefix Caching	在不同请求、不同引擎间共享前缀 KV	避免重复计算相同前缀
PD Disaggregation	在 Prefill 与 Decode 引擎之间传输 KV	将两类推理负载分离

CPU Offloading（CPU 卸载）

将 KV Cache 从 GPU 显存迁移到 CPU 内存，在需要时重新加载，从而降低 GPU 显存压力。

Prefix Caching（前缀缓存）

当多个请求具有相同的 Token 前缀时，可以复用该前缀的 KV Cache。

例如：$P_1 = [A, B, C, Q_1]$，$P_2 = [A, B, C, Q_2]$，两个请求均可以复用 $[A, B, C]$ 对应的 KV Cache。

但前缀缓存通常不能直接复用出现在不同前文之后的非前缀片段。这一限制将在第四章进一步讨论。

Prefill-Decode Disaggregation（预填充与解码分离）

将 Prefill 和 Decode 部署到不同推理引擎，由 Prefill 引擎计算 KV Cache，再传输给 Decode 引擎继续生成。其目标是根据两种阶段不同的计算和内存访问特征分别优化资源配置。

三种机制并非互斥，而是可以在同一推理系统中组合使用。

Part 3 LMCache：独立的 KV Cache 基础设施

3.1 LMCache 的设计目标与架构

LMCache 是位于推理引擎与存储后端之间的独立 KV 缓存层，旨在为不同推理引擎提供统一的 KV Cache 管理能力。

四种基本缓存操作语义：

1. GPU - GPU Transfer：在不同 GPU 之间传输 KV Cache。
2. GPU - CPU Offloading：将 KV Cache 从 GPU 卸载到 CPU。
3. CPU - CPU Transfer：在不同 CPU 内存空间之间传输缓存。
4. Storage Offloading：将 KV Cache 卸载到持久化或远程存储。

该架构将缓存管理从推理引擎内部抽离，使缓存能够具有独立的生命周期，并被多个引擎共享。

3.2 LMCache 的三项核心挑战

挑战	问题来源	LMCache 的解决方向
缓存 I/O 效率低	分页内存管理、L2/L3 存储访问和 I/O 干扰推理	优化传输路径与缓存操作
难以适应推理引擎演进	vLLM、SGLang 的调度器与内存分配器不断变化	通过稳定接口降低耦合
缺少交互式管理能力	运维系统无法直接控制缓存	提供实时查询、迁移和管理 API

新模型频繁发布，可能采用不同的内存分配和缓存布局。因此，如果外部缓存系统深度依赖推理引擎内部实现，就会产生较高的适配成本。LMCache 的设计目标是在推理引擎持续变化的情况下，尽量保持缓存层的独立性。

3.3 LMCache 的性能优化技术

技术优化可以整合为三个主要方向：

1. 计算与 I/O 重叠（Compute - I/O Overlapping）

通过异步执行，使 KV Cache 加载或卸载与 GPU 计算同时进行。

相关技术包括：

- Layer-wise Pipelining：按 Transformer 层进行流水线传输；
- Request Async & Prefetch：异步请求与预取；
- KV Cache Proxy：通过缓存代理协调数据访问。

不采用重叠执行时： $T_{\text{serial}}=T_c+T_{io}$ ； *理想完全重叠时*：T_{\text{overlap}} \approx \max(T_c,T_{io})$

因此理论上节省的时间为： $\Delta T = T_c + T_{io} - \max(T_c, T_{io})$。其中， T_c 表示计算时间， T_{io} 表示数据传输时间。

实际系统仍存在流水线启动、同步与资源竞争开销。

2. 批量操作（Batched Operations）

GPU 内存中的最佳计算粒度可能小于网络与存储设备适合处理的数据粒度。如果直接传输大量小型缓存块，会产生频繁的操作开销。因此，LMCache 通过批量操作提高 I/O 效率，包括：

- Configurable Chunk Size：可配置缓存块大小；
- Customized CUDA Kernel：定制 CUDA 内核；
- Batched KV Cache Store/Load：批量缓存读写；
- GPU Transfer Buffering：GPU 传输缓冲。

设每次 I/O 固定开销为 t_0，单次数据传输量为 s，带宽为 B，则： $T_{\text{IO}}=t_0+\frac{s}{B}$。当大量小数据分别传输时，固定开销会被重复支付。通过合并操作，可以减少 I/O 调用次数。

3. 最小化数据复制（Minimum Data Copy）

主要包括：

- Zero Copy：在 PD 分离、远程存储或磁盘传输中尽量减少额外的数据复制；
- Eviction-based Offloading：根据缓存淘汰策略决定哪些缓存需要迁移到较低层存储。

其共同目标是减少不必要的内存复制、带宽消耗和 GPU 资源占用。

3.4 KVConnector：推理引擎与缓存层解耦

KVConnector 为推理引擎和外部缓存层定义相对稳定的交互边界，使 LMCache 不必直接依赖 vLLM 内部调度器或内存分配器的所有实现细节。七个接口如下：

API	功能
build_connector_meta	构建请求与缓存加载相关的元数据
start_load_kv	启动从连接器缓冲区加载 KV Cache
wait_for_layer_load	检查或等待指定层的 KV 加载完成
save_kv_layer	将某层 KV 从 vLLM 分页缓冲区保存至连接器缓冲区
wait_for_save	等待缓存保存操作完成
get_num_new_matched_tokens	获取远程缓存中新匹配的 Token 数量
update_state_after_alloc	临时缓冲区分配后更新连接器状态

这些接口可以归纳为四种功能：元数据维护、缓存加载、缓存保存和状态同步。

其核心价值是实现推理计算与缓存管理的接口解耦。

3.5 LMCache Controller：交互式缓存管理 API

与 KVConnector 主要服务于推理引擎内部交互不同，LMCache Controller 面向用户、管理员和编排系统（Orchestrators）提供缓存控制能力。六个管理 API：

API	功能	典型应用
Lookup	根据 Token 列表查询缓存	检查缓存命中
Clear	清除缓存	释放资源、缓存维护
Pin	持久化缓存或设置 TTL	保留高价值缓存
Move	将缓存迁移到其他位置	跨节点迁移、负载调度
Compress	压缩 KV Cache	减少存储与传输开销
CheckFinish	检查非阻塞控制操作是否完成	异步操作状态管理

其中，TTL（Time to Live）表示缓存的有效保留时间。

KVConnector 与 Controller 的区别在于：KVConnector 解决推理引擎如何加载和保存 KV；Controller 解决外部系统如何查询、控制和管理 KV。

3.6 LMCache 的性能实验与复用特征

1. KV Cache Offloading 性能

实验比较了 LMCache、原生 vLLM（Naive vLLM）以及两种商业方案。TTFT 随 QPS（Queries Per Second，每秒请求数）的变化；ITL 随 QPS 的变化。图中 LMCache 在较高请求负载下仍保持较低的 TTFT 和 ITL，体现了缓存复用与卸载机制在重复上下文工作负载中的优势。

这一实验结果依赖具体模型、硬件、请求分布和缓存命中情况，不能直接推断所有场景都具有相同的性能提升。

2. 实际 KV Cache 复用率

右侧 CDF（Cumulative Distribution Function，累积分布函数）图展示了不同缓存复用率的分布。

其重要结论是：缓存复用具有明显的不均衡性，少量高复用用户或工作负载能够产生较高的缓存收益。

这意味着缓存管理策略应当关注复用频率，而不只是缓存大小。

3.7 实际部署经验与技术趋势

KV Cache 基础设施发展的技术趋势。

1. 存储 I/O 性能持续提升：$100\text{ MB/s (S3)} \rightarrow 1\text{ GB/s (S3 Express One)} \rightarrow \sim10\text{ GB/s (CAIOS)} $
2. 新型注意力技术可以减少 KV Cache 的存储需求。
3. 系统需要综合考虑缓存压缩、远程数据传输、缓存复用价值和存储层次，而非单纯追求更大的缓存容量。

核心结论是：KV Cache 的价值不仅取决于是否能够缓存，还取决于是否能够以足够低的代价保存、定位、传输和复用。

Part 4 CacheBlend：非前缀 KV Cache 的选择性重计算

4.1 Prefix Caching 的局限性

传统前缀缓存适用于多个请求共享相同 Token 前缀的场景，但在 RAG（Retrieval-Augmented Generation）中，检索结果往往由多个独立文档块组成。

例如： $P=[D_1,D_2,D_3,Q] $。其中 D_i 表示检索文档块，Q 表示用户查询。

假设 D_2 的 KV Cache 是在其单独作为输入时生成的，那么将其放在 D_1 后面，并不能保证缓存仍然有效。

原因：KV Cache 不仅与当前 Token 内容有关，还与其前文上下文有关。

因此，相同文本片段在不同上下文位置上可能产生不同的 KV 表示。

简单拼接缓存为何会降低质量？

两个文档块分别包含 Messi 和 Ronaldo 的进球信息，问题要求比较两者的进球数。

完整的 Prefill 可以在 Transformer 的后续层中建立跨文档的信息交互。

但如果直接拼接分别计算得到的文档 KV Cache：$KV_{\text{concat}} = [KV(D_1); KV(D_2)]$ 就可能缺失在完整上下文中应当发生的跨文档注意力交互。

Missing Cross-Attention（缺失跨片段注意力），这里的 Cross-Attention 是指不同文档片段之间的注意力交互，并非特指 Encoder - Decoder 架构中的 Cross-Attention 模块。

对于完整上下文，第二个文档的某层隐藏状态可能依赖第一个文档：$H_{D_2}^{(l)} = f_l(H_{D_1}^{(l-1)}, H_{D_2}^{(l-1)})$，而独立计算 D_2 时，这种依赖不存在。

因此：$KV(D_1 \oplus D_2) \neq KV(D_1) \oplus KV(D_2)$

一般情况下，不能通过简单拼接独立 KV Cache 来获得完整上下文 Prefill 的结果。

4.3 计算效率与生成质量的权衡

三种缓存策略：

策略	计算量	质量特点
Full Prefill	高	保留完整上下文交互
Reusing All KV Caches	低	可能缺失跨片段注意力
CacheBlend	介于两者之间	通过选择性重计算恢复关键交互

CacheBlend 的核心思想：Selective Prefill / Selective Recompute（选择性预填充或选择性重计算）。并非重新计算所有 Token，而是在 Transformer 的不同层中选择部分重要 Token，重新计算其 KV 表示，其余 Token 继续复用已有缓存。

4.4 CacheBlend 的分层重计算机制

完整 Prefill 对所有层、所有相关 Token 重新计算 KV；CacheBlend 则只重计算每层的一部分 Token。

设第 l 层需要重计算的 Token 集合为：$S_l\subseteq{1,2,\ldots,L}$ ，*那么*：KV_i^{(l)} = \begin{cases} KV_{i,\text{recomputed}}^{(l)}, & i\in S_l\ KV_{i,\text{cached}}^{(l)}, & i\notin S_l \end{cases}$

不同层可以选择不同的 Token 集合： $S_1\neq S_2\neq\cdots\neq S_N $

这是因为不同层的隐藏状态和注意力交互需求并不相同。

其基本目标是尽量减少重计算量，同时恢复对生成质量影响较大的跨片段交互。

需要注意，课件展示了分层选择性重计算的机制，但没有在该页完整展开 Token 选择算法的数学细节。

4.5 CacheBlend 的性能实验

实验比较了完整 KV 重计算、仅前缀缓存、直接 KV 复用和 CacheBlend。

1. 响应延迟

使用 F1 Score 与 TTFT 评估不同方案的质量和延迟。CacheBlend 在保持较高问答质量的同时，实现了约 3× 的响应速度提升。

如果基线延迟为 *T*，速度提高三倍意味着：$T_{\text{CacheBlend}}\approx\frac{T}{3}$，对应的延迟降低比例约为： $1-\frac{1}{3} = 66.7\%$

2. 吞吐量

使用平均请求到达率与 TTFT 的关系展示系统吞吐能力。图中 CacheBlend 相较完整重计算可实现约 3× 的吞吐量提升。吞吐量提升的主要来源是减少重复 Prefill 计算，使相同硬件能够处理更多请求。

4.6 KV Cache 作为 AI-Native Data

将 KV Cache 提升到更一般的概念：AI-Native Data（AI 原生数据）

含义：文本、对话、文档等上下文在 LLM 内部会被转换为可供后续计算使用的 KV 表示。与原始文本相比，KV Cache 保存了模型在特定计算条件下形成的中间表示。

三项价值：

- 可复用性：避免重复计算，降低延迟与 GPU 成本。课件给出 5 - 10× 响应速度提升的潜力。
- 注意力信息表示：KV Cache 与模型注意力计算直接相关。
- 上下文操作能力：支持更智能的上下文检索、压缩与融合。

但 KV Cache 不能脱离模型结构、位置编码、上下文依赖和缓存格式等兼容性条件任意复用。

结论： Prefix Caching 解决相同前缀的重复计算问题，CacheBlend 则针对非前缀文档片段，通过少量重计算恢复必要的上下文交互，实现计算效率与生成质量之间的平衡。

Part 5 复合 AI 系统 Compound AI Systems

5.1 从单体模型到模块化 AI 系统

传统 AI 应用通常以单个大语言模型（Monolithic LM）为核心，通过一次或少量模型调用完成复杂任务。

这种架构存在三个主要问题：

- 控制困难（Hard to Control）：模型内部的推理与决策过程难以进行精细控制。
- 调试困难（Hard to Debug）：输出错误时，难以确定错误来源。
- 改进困难（Hard to Improve）：修改提示词或更换模型可能影响整个系统的行为。

Compound AI Systems：使用语言模型作为专门化组件构建的模块化程序（Modular Programs）。

其核心思想是将复杂任务拆解为多个具有明确职责的模块，通过程序控制模块之间的信息流和执行顺序。

5.2 Compound AI Systems 的五项优势

通过 RAG、多跳检索和组合式报告生成等例子，归纳了复合 AI 系统的五项优势：

优势	核心含义
Quality（质量）	组合职责更明确的模型能力，提高系统可靠性
Control（控制）	利用工具和程序逻辑控制执行过程，并逐步改进
Transparency（透明性）	调试执行轨迹，向用户提供结果来源
Efficiency（效率）	使用较小模型，将知识和控制流交由外部组件处理
Inference-time Scaling（推理时扩展）	在推理阶段系统性搜索更优输出

1. Quality：模块化提高输出质量

通过任务分解，可以为不同子任务配置适合的模型或工具。

例如，报告生成系统可以分别完成信息检索、内容规划、段落生成和结果检查，避免让单一模型承担所有职责。

2. Control：显式控制任务执行

多跳 RAG（Multi-Hop RAG）可以根据中间结果执行下一轮检索。

这种机制允许系统通过工具获取外部证据，并根据任务需要调整执行路径。

3. Transparency：可追踪与可解释

系统能够记录检索结果、工具调用、模块输入输出和中间决策。

这些记录既有助于定位错误，也可以为用户提供结果来源（Attribution）。

4. Efficiency：降低系统成本

复合系统可以将知识存储、检索和流程控制从模型参数中分离出来。

因此，部分任务可以使用较小的模型完成，而不必完全依赖大规模模型。

5. Inference-time Scaling：推理时扩展

系统可以在推理阶段投入更多计算资源，例如生成多个候选答案、进行搜索或验证。

其目标不是简单增加 Token 数，而是通过结构化搜索提高获得高质量输出的概率。

5.3 Agentic Frameworks 的三种类型

智能体框架：

类型	代表框架	主要关注点
Agent Development	LangGraph、AutoGen、Crew	会话恢复、低代码、精细控制、并发、流式执行
Data-Driven Optimization	DSPy、TextGrad、Trace	结构化配置、结构化日志、优化算法
Holistic Framework	TapeAgents	同时支持状态管理、结构化日志和数据驱动优化

其中，Agent Development 框架主要解决智能体如何执行任务，Data-Driven Optimization 框架主要解决智能体如何通过数据改进性能。

TapeAgents 试图结合两者，将智能体表示为可恢复的模块化状态机（Resumable Modular State Machine），并通过结构化日志支持后续优化、微调数据生成和日志复用。

5.4 现有复合系统的 Stringly-Typed 问题

虽然 Compound AI Systems 在架构上具有模块化特征，但很多系统仍然高度依赖人工编写的 Prompt 字符串。

传统 Prompt 往往混合五种不同职责：

职责	对应抽象	作用
输入输出行为	Signature	定义模块要完成什么任务
推理策略	Predictor	定义如何执行模型推理
输入输出格式转换	Adapter	构建模型输入并解析输出
目标与约束	Metrics / Assertions	定义评价标准和行为限制
参数优化	Optimizer	优化提示词或模型权重

当这些职责都被写入同一个字符串时，系统架构就会与具体 Prompt 的表达方式耦合。这被称为 Stringly-Typed 问题，即系统过度依赖缺乏明确结构约束的字符串。其后果是修改 Prompt 可能影响多个模块；模型迁移成本较高；优化目标与程序逻辑难以分离；系统难以自动优化。

进一步的研究方向：将 Compound AI Systems 抽象为由自然语言类型化模块组成、且能够学习和优化自身行为的程序。**这构成了 DSPy 的设计动机。**

Part 6 DSPy：从 Prompt Engineering 到 Language Model Programming

6.1 DSPy 的定义与设计理念

DSPy（Declarative Self-improving Language Programs in Python）是由 Stanford NLP 研究人员开发的语言模型编程框架。其核心理念为Programming over Prompting（以编程代替直接操作提示词）。

传统方法主要依赖人工设计 Prompt，而 DSPy 将任务定义、执行策略与参数优化分离。

具体而言：

- 程序员定义任务的输入输出和模块关系；
- 框架负责将程序结构转换为模型可以执行的提示；
- 优化器根据数据和评价指标改进提示词或模型参数。

因此，DSPy 并不是取消 Prompt，而是将 Prompt 从人工维护的主要对象转变为可自动生成和优化的实现细节。

6.2 DSPy 的三个核心抽象

DSPy 的核心概念：

传统方法	DSPy 抽象
Hand-written Prompts	Signatures
Prompting Techniques / Chains	Modules
Manual Prompt Engineering	Optimizers + DSPy Compiler

这三个抽象分别回答三个问题：

- Signature：任务是什么？
- Module：采用什么推理策略？
- Optimizer：如何根据评价结果改进程序？

6.3 Signature：声明式输入输出规范

Signature 是 DSPy 中描述模型任务的声明式接口。Signature 主要包含三个要素：子任务的简要描述；一个或多个输入字段（Input Fields）；一个或多个输出字段（Output Fields）。例如："context, question -> answer"

该 Signature 定义了一个接收上下文和问题、输出答案的任务。其重点是声明需要完成的文本转换，而非手动规定模型必须使用怎样的提示词。

更完整的声明方式可以写为：

```
import dspyclass AnswerQuestion(dspy.Signature):
"""Answer the question using the given context."""
context = dspy.InputField()
question = dspy.InputField()
answer = dspy.OutputField()
```

6.4 Module：将推理技术封装为程序组件

Module 用于封装特定的模型调用方式或推理策略。典型模块包括：

- `dspy.Predict`：基本预测；
- `dspy.ChainOfThought`：链式思考；
- `dspy.ProgramOfThought`：程序辅助推理。

例如：

```
qa = dspy.Predict("question -> answer")mt = dspy.ChainOfThought(
"english_document -> french_translation")rc = dspy.ProgramOfThought(
"contexts, question -> answer_found: bool")
```

这些模块可以通过不同的 Signature 完成不同任务。

Module 与 Signature 的区别在于：Signature 定义任务接口，Module 负责执行相应的推理过程。

6.5 Adapter 与 Predictor：从程序到 Prompt

DSPy 内部的 Prompt 转换过程。例如：

```
self.generate_query = dspy.ChainOfThought(
"context, question -> query")
```

DSPy 可以根据模块和 Signature 生成结构化 Prompt，其中包含：输入字段说明；输出字段说明；字段格式；推理字段（Reasoning）；最终输出字段（Query）。

在这一过程中：Adapter 负责将结构化字段转换为模型输入，并解析模型输出；Predictor 负责根据模块定义执行模型预测。因此，程序员不必为每次调用手动编写完整 Prompt，而是通过统一接口定义模型行为。

6.6 DSPy 的完整工作流程

DSPy 的开发流程的五个阶段：

1. Collect Dataset：收集输入输出示例，构建优化数据集
2. Write DSPy Program：定义 Signatures、Modules 和信息流
3. Define Validation Logic：确定评价指标、约束和优化器
4. Compile Program：根据数据和指标优化提示词或模型参数
5. Iterate：改进数据、程序结构或验证逻辑

DSPy 的编译过程可抽象为：$P^* = \text{Compile}(P, D, M, O)$

P：原始 DSPy 程序；D：用于优化的数据；M：评价指标；O：优化器；P^*：经过优化的程序。

这里的 Compile 不等同于传统编程语言中的机器码编译，而是针对模型调用行为进行参数搜索和优化。

6.7 Optimizer：自动提示词优化

优化器可以同时调整程序中多个模块的提示词或模型参数，而不是孤立地优化单次模型调用。

其典型过程包括：

1. 使用 Adapter 为各模块构建初始 Prompt；
2. 通过 Rejection Sampling（拒绝采样）生成和筛选模块示例；
3. 利用筛选后的示例更新模块配置；
4. 根据评价指标选择表现更好的程序。

三类典型优化方法

优化方法	DSPy 工具	主要机制
Automatic Few-shot Prompting	BootstrapFewShotWithRandomSearch	自动构建和搜索少样本示例
Instruction Induction	MIPROv2	自动生成和优化指令及示例
Multi-stage Fine-tuning	BootstrapFinetune	利用生成的数据执行微调

MIPROv2 展示了自动优化过程：optimizer = MIPROv2()optimized_program = optimizer.compile(program)

实际使用时还需要根据 DSPy 版本提供模型、评价指标和数据集等参数。自动优化能够显著改善特定任务的程序表现，但提升幅度取决于数据、任务、模型和评价指标。

6.8 DSPy 的实际应用与研究成果

采用相关自然语言编程与优化思想的研究系统，包括：PATH；IReRa；STORM；EDEN；Efficient Agents；ECG-Chat。这些案例说明，DSPy 所代表的程序化优化思想已经应用于不同的语言模型任务和研究系统。

6.9 Natural Language Programming：程序优于单体模型的条件

两项核心观点。

第一，在许多任务中，经过合理设计的程序可以比单一模型更准确、更可控、更透明，甚至更高效。

第二，开发者主要需要定义声明式程序，而不必手动规定每个底层实现细节。

高层优化器可以根据任务数据自动改进 Prompt、模型权重或未来可能出现的其他模型参数形式。

这体现了从手工提示词设计到声明式程序优化的转变。

6.10 Natural Language Optimization：自动优化的三项经验

1. 高质量示例的重要性

对于许多任务，自动构建高质量示例（Bootstrapping Good Demonstrations）是有效的优化方法。其基本思想是通过示例展示期望行为，而不只是通过文字描述规则。

2. 指令优化与条件规则

在高质量示例基础上进一步生成和优化指令，可以改善程序表现。当任务包含较多条件判断或复杂规则时，明确的指令尤其重要。

3. Grounding 与 Credit Assignment

优化过程还需要有效的 Grounding（依据外部信息或评价证据建立可靠约束）和明确的 Credit Assignment（贡献归因）。对于多模块系统，如果最终结果错误，需要判断是哪个模块导致了性能下降，才能进行有针对性的优化。

例如，一个 RAG 系统回答错误可能来自检索失败、上下文选择错误或答案生成错误。仅根据最终答案分数优化所有模块，可能无法准确识别问题来源。

因此，DSPy 的长期价值不仅在于自动生成 Prompt，还在于支持基于数据、指标和模块结构的系统性优化。

总结

技术层次	核心问题	关键技术
推理计算	如何避免重复 Prefill	KV Cache
存储基础设施	如何长期保存与共享缓存	Hierarchical Storage、LMCache
缓存算法	如何复用非前缀上下文	CacheBlend
系统架构	如何提高模型系统的可靠性与可控性	Compound AI Systems
程序抽象	如何减少对手写 Prompt 的依赖	DSPy Signature、Module
自动优化	如何基于数据持续改进系统	DSPy Optimizer、Compiler

从整体上看，本讲体现了两个重要转变：

第一，从重复计算到计算结果复用。 KV Cache、LMCache 和 CacheBlend 将模型内部的计算结果转化为可保存、可共享、可管理和可选择性重计算的资源。

第二，从模型调用到程序化智能体系统。 Compound AI Systems 和 DSPy 将模型能力封装为具有明确接口的模块，并通过数据驱动的优化方法改进系统性能。

两条技术路线共同说明：高效、可靠的智能体不仅依赖模型能力，也依赖模型外部的存储系统、执行架构、模块接口与优化机制。

---

reference patterns

Archive source: `prompt_appendix/task3_v1/reference_patterns.md`

Reference Pattern Notes for V1

Only component structure, information hierarchy, and interaction patterns were considered. Reference authors' words, identity, images, and personal information were not copied.

Reference	Pattern observed	Adaptation to Aria's site
al-folio	Clean academic information structure and explicit section hierarchy.	Keep Aria's long-scroll sections easy to scan; avoid decorative controls competing with the content.
Shaoyan Liu's academic homepage	Compact academic homepage with news/publication groupings, filters, and accessible media viewing.	Present the four projects through a compact index plus detailed story cards; reuse existing site interactions and content.
Jiya Batra's art portfolio	Image-led gallery organization, categories, and direct viewing actions.	Keep the V0-owned carousel thumbnails selectable and preserve image/video previews in Life; no unrelated imagery was introduced.
Michael Kolesidis's portfolio	Deliberate, responsive microinteractions around page elements.	Use restrained hover/focus and selected-state feedback only; do not reproduce its deliberately colorful early-web aesthetic or its content.
Scrapbook Portfolio	Multi-page journal organization, original hand-drawn SVG doodles, and small scroll interactions.	Best additional match for Aria's diary feeling: use the page-sequence and restrained hand-drawn annotation ideas, while preserving the watercolor sky and avoiding decorative clutter.
Photography Wall	Uneven masonry photo wall with a keyboard-operated lightbox.	A possible model for organizing Aria's own Life photos and videos into a more image-led gallery with accessible next/previous viewing. Use only her existing photos and videos.
Korbinian Möller's personal website template	Compact researcher/developer site with section navigation, grouped publications, and detail modals.	Useful for making academic/project details scannable and inspectable without adding more full-size tiles. Keep the content and visual tone personal to Aria.
Hitesh Nandakumar's portfolio	Static single-page portfolio with project write-ups, visible capabilities, keyboard-friendly navigation, and reduced-motion support.	Useful as an implementation reference for clear case-study content and accessible vanilla-JS behavior; keep the sky-journal art direction.

Decisions after screenshot review

- The chapter rail originally collided with the carousel thumbnails. It now sits in document flow after the hero, in a single glass strip with dividers.
- Repeated standalone boxes made the Home and Projects index areas feel like a dashboard. Both indices now use typography and separators; the project stories themselves retain the paper-card treatment.
- Artwork remains the focus of the Home hero. Existing V0 assets are used for the hero and thumbnails; no stock image was added.
- These refinements and their tested screenshot output are recorded in `icl_prompt_revision3.md`, `change_log.md`, and the `V1-R4-` evidence files.

V1 iteration 4 application

- `Photography Wall` informed the image-first, uneven arrangement and keyboard-operable viewing; V1 keeps its own native preview dialog rather than copying a reference implementation.
- `Scrapbook Portfolio` informed the use of small, original SVG margin notes. The new sprig and flight-path sketches were authored for this site, use its muted palette, and are decorative only.
- The seven existing photos and two videos remain the only Life media; the art direction, copy, and identity remain Aria's.

---

technique1 comparison

Archive source: `prompt_appendix/task3_v1/technique1_comparison.md`

Task 3 · Technique 1 Results — V0 vs V1

Lineage and protocol

V1 was copied directly from frozen `coursework/website/V0/` and then changed under `coursework/website/V1/`. The V0 code and `coursework/MANIFEST.sha256` were not edited. V0 and V1 were both run through the same `coursework/evaluation/scripts/test_portfolio_e2e.cjs` at a 1512 × 982 desktop viewport in Google Chrome headless on macOS, over local HTTP. The four rubric definitions, weights, scoring anchors, and AI-Judge prompt remain unchanged. Final V1 is published at https://liuxinyue84-sketch.github.io/V1/; root V0 remains at https://liuxinyue84-sketch.github.io/.

Outcome

The expanded shared script recorded 23/25 passing checks for V0 and 25/25 for V1. Both versions passed navigation, carousel, education/project disclosure, filtering, saved-state persistence, media preview, music control, guestbook empty validation, and no application-console-error checks.

V1 passed both tested unsupported-query phrasings; unchanged V0 failed the MapReduce relevance case and the added paraphrased unsupported query. The meaningful differences were:

- **V0 input:** "How does MapReduce work?" The leading excerpt discussed neutral framing and prompt wording, not MapReduce. The stored raw output is `coursework/evaluation/evidence/V0/V0-e2e-results.json`.
- **V1 input:** the same question. The leading excerpt is from "6.5 MapReduce steps" in *Big Data Computing · Part 1*, and the answer cites that note. The raw output is `coursework/evaluation/evidence/V1/V1-e2e-results.json`.
- **V0 boundary failure:** "How do I bake sourdough bread?" returned an unrelated data-mining excerpt because "bread" was a shared corpus term. This is recorded as a V0 failure, not silently removed from the test.
- **V1 correction:** a non-topic query must now match at least two original content terms and at least 50% of them before an excerpt is returned. The two tested sourdough phrasings both receive the no-evidence fallback in V1. This is a targeted lexical threshold improvement, not comprehensive abstention reliability.

Screenshots: V0 and V1 desktop captures are under `coursework/evaluation/evidence/V0/` and `coursework/evaluation/evidence/V1/`. The V1 full Highlights capture also shows the board, preserved course-note interface, cited MapReduce excerpt, and no-evidence response.

Human score estimate (provisional)

These are transparent rubric-based human estimates, not AI-Judge results. V0's existing preliminary self-assessment is 76/100 (4, 4, 4, 3). Using the same rubric after the common run, the provisional V1 estimate is 80/100 (4, 4, 4, 4): Content Completeness 4, Navigation & Task Completion 4, Readability & Visual Consistency 4, Interaction Robustness 4. The weighted total is `(4/5×30)+(4/5×25)+(4/5×25)+(4/5×20) = 80/100`.

The 4/5 interaction score reflects the MapReduce relevance improvement and passing tested interactions, while a valid guestbook submission and length-limit handling remain untested. The 4/5 visual score reflects the coherent desktop screenshot but does not claim a contrast or mobile audit. The content score remains 4 because V1 preserves V0 facts and improves contribution visibility, but source-by-source CV/PS fact verification is outside this browser run. The single rubric point increase is supported by the concrete retrieval change and test evidence, not by the aggregate alone.

AI-Judge comparison remains pending. The coursework requires a new blank AI-Judge conversation containing only the fixed scoring instructions and V1 evidence. This task has not fabricated or inferred an AI score from the human assessment. Use the prepared evidence manifest in `ai_judge_evidence.md`; preserve the Judge's response verbatim in a separate raw-output file.

How the few-shot examples affected V1

1. The education-card demonstration led to a dotted pinboard container, a hand-drawn chronology connector, bundled V0 photos, and keyboard-operable education disclosures.
2. The contribution-first example led to a separate "MY CONTRIBUTION" label around the unchanged V0 project-contribution strings.
3. The MapReduce example led to a topic profile and heading boost in the existing browser-local BM25 retriever. It changed which stored passage ranked first; it did not generate a new answer.

4. The unsupported-question example led to an evidence-overlap check and explicit no-evidence response. A live paraphrase exposed a false positive, so a V1-only minimum term-overlap correction was added and tested against unchanged V0. Broader unknown-question coverage remains untested.

Trade-offs and remaining issues

- The pegboard makes education, skills, and project contributions easier to scan but adds vertical length to Highlights. The board is more illustrative and less compact than V0's two-column baseline.
- The retrieval fix is a small transparent lexical/topic layer. It only models three example topics (MapReduce, RAG, agent planning), depends on curated topic profiles, and is not semantic retrieval or answer generation.
- The portfolio remains static HTML/CSS/JS; local favorites and guestbook entries remain browser-local.
- V1 is deployed at `/V1/`; the root V0 URL remains untouched. The final threshold correction is deployed; the published HTTPS run passed 25/25 after the test harness replaced a fixed carousel delay with a state-based wait. The first 24/25 live run remains archived as an initial test attempt.
- The old implicit favicon 404 remains in both codebases; it is reported separately because it is a browser-default request and not a missing application asset.
- A fresh-session AI-Judge result, valid guestbook submission, length-limit test, and contrast audit still need to be added before the Task 3 submission package is complete.

Structural revision 2 — response to observed weakness

The first V1 did improve the Highlights board and RAG behavior, but it left the Home/Projects interaction and DOM patterns too close to V0. The user identified that as insufficient visible evidence of Few-Shot Learning. That feedback is retained in `change_log.md`; it was treated as a failed design iteration, not omitted.

V1-R2 applies a second explicit set of structural few-shot examples from the same frozen V0-derived V1 branch. The Home now has a numbered chapter rail and selectable image-thumbnail carousel; Projects has an anchored story index, an ordered list of project stories, separately labeled purpose/contribution, and a staggered paper-atlas layout. Interaction remains tied to existing project records, filters, expansions, and local saves. New E2E assertions verify these features.

The shared desktop protocol was expanded to 29 checks and run on both versions. V1-R2 passes 29/29; frozen V0 passes 27/29, with the same two known RAG limitations (MapReduce ranking and a paraphrased unsupported query). These E2E counts are behavior evidence, not replacements for the fixed weighted rubric. They should not be converted into rubric points without the Task 2 evaluator process.

Evidence: `coursework/evaluation/evidence/V1/V1-R2-E2E-01_home_desktop.png`, `V1-R2-E2E-13_projects_full_desktop.png`, and `V1-R2-e2e-results.json`; matched frozen-baseline evidence is under `coursework/evaluation/evidence/V0/` with the `V0-R2-` prefix. The V0 manifest is checked after the revision. Detailed generated files and prompt examples are in `llm_output/iteration_2/`, `icl_prompt_revision2.md`, and `few_shot_examples_revision2.md`.

Iteration 6 — screenshot-led interaction clarity

The user reported unclear numeric-only navigation, carousel controls obscuring the image, inoperative education disclosures, and a neighboring project appearing to expand with the selected project. V1 now labels chapter destinations using the existing V0 section names and labels project-index links with their existing project titles. The image controls sit below the image. Both education entries expand from the title or photo and display a short action hint; the V1 CSS state selector now matches the V1 card class. Project articles use intrinsic heights so adjacent cards stay compact when one opens.

Local results at 1512×982: V1 35/35; V0 32/34, with its two known RAG limitations. Published V1 also passes 35/35 at `https://liuxinyue84-sketch.github.io/V1/?v=6379505`. The 48-group text contract and the V1 shell checks pass with zero differences locally and online. New screenshots record labeled navigation, image controls below the photo, both education disclosures open, and one project open beside a compact neighbor. Raw outputs are `evaluation/evidence/V1/V1-R7-*`, `V1-PAGES-R7-*`, and `evaluation/evidence/V0/V0-R7-*`; the user feedback and exact correction scope are preserved in `icl_prompt_revision6.md`.

The user then identified two further issues from screenshots: the Home thumbnail control and chapter rail collided, and repeated tile cards felt unattractive. The final V1 refinement moves the rail into normal flow below the hero and converts both navigation indices into restrained editorial strips. The final matched visual/interaction evidence is in `coursework/evaluation/evidence/V1/V1-R4-*`; the final local E2E run passed 29/29. V1-R3/R4 feedback, revised few-shot prompt, reference mappings, and complete source captures are preserved in `change_log.md`, `icl_prompt_revision3.md`, `reference_patterns.md`, and `llm_output/iteration_3/`. These run counts are supplementary behavioral evidence and do not replace the fixed weighted rubric.

Iteration 5 — strict copy contract and component architecture

The user later noted that the prior iteration still looked too close to V0. In response, V1 was rebuilt around materially different component shells: a journal-cover Home, education-journey/skill-ledger Highlights, indexed semantic project stories, letter/media-atlas Life, and open-postcard Guestbook. The reviewed few-shot examples informed these structures and interactions; no reference personal copy or assets were used. This implementation remains independently derived from frozen V0.

The V0-to-V1 content contract compares 46 text groups plus hero-image alternative text and the complete project-story fields. It found **zero text differences**. Eleven expected V1 component-shell selectors are present while absent from V0, providing structural evidence beyond color changes. Full raw contract output: `evaluation/evidence/V1/V1-R6-content-contract.json`; live-page contract output: `evaluation/evidence/V1/V1-PAGES-R6-content-contract.json`.

The same desktop E2E protocol at 1512×982 gives local V1 33/33 and published V1 33/33; unchanged V0 gives 30/32, with only the known MapReduce first-passage relevance and paraphrased unsupported-query abstention failures. These are behavioral checks rather than weighted rubric scores. Screenshots cover Home, Highlights, Projects, Life, and Guestbook; see `evaluation/evidence/V1/V1-R6-*` and `V1-PAGES-R6-*`. The public build commit is `c80de98` at `https://liuxinyue84-sketch.github.io/V1/?v=c80de98`.

---

ai judge prompt v2

Archive source: `prompt_appendix/task3_v2_evidence/ai_judge_prompt_v2.md`

AI Judge Prompt — Apply in a New, Clean Session

> You are an independent evaluator assessing a personal portfolio website against the rubric and evidence supplied below. Evaluate only the supplied materials. Do not assume that source code works merely because an implementation is present. Distinguish observed test results from source-code claims and note when evidence is insufficient.
>
> **Do not infer or invent missing behavior.** If a task was not tested, state that limitation. Do not use outside information about the owner. Do not use or request any prior human assessment. Produce an independent assessment from this prompt and its attached evidence only.

Rubric

Score each criterion from 0 to 5. Scores 0, 3, and 5 correspond to the anchors below; integer scores 1, 2, or 4 may be used for performance between anchors. Calculate the weighted total as `Σ (criterion score ÷ 5 × criterion weight)` on a 0 - 100 scale.

1. Content Completeness & Clear Contribution — 30%

Measures whether relevant profile, education, skills, projects, and contact/personal content are present and coherent, and whether project entries clearly explain the owner's contribution.

- **0:** Most expected sections are absent/empty; work or contribution is uninterpretable; major contradictions or unusable content.
- **3:** Several sections/projects are present, but important content is missing, repetitive, vague, or does not distinguish the owner's contribution.
- **5:** Relevant sections are complete and coherent; project contributions are clear; no material gap prevents understanding the portfolio.

2. Navigation & Task Completion — 25%

Measures whether visitors can find the main sections and complete representative tasks such as opening project details, filtering/saving projects, exploring media, using the course-note interface, and submitting a guestbook entry.

- **0:** Main navigation is broken or essential content/tasks are inaccessible; most tasks fail.
- **3:** Main sections are reachable, but an important task is confusing, incomplete, or fails in ordinary use.
- **5:** Main sections are easy to find and representative tasks complete through clear controls with understandable feedback.

3. Readability & Visual Consistency — 25%

Measures text legibility, hierarchy, spacing, responsive presentation, and visual consistency across sections and interaction states.

- **0:** Text/controls are largely unreadable or layout is broken/inconsistent enough to prevent comprehension.
- **3:** Overall style is recognizable and much content is readable, but notable inconsistency, small/low-contrast text, clutter, or layout issues hinder use.
- **5:** Clear hierarchy and comfortable reading; coherent styling; tested responsive layouts and sufficient contrast keep content and controls usable.

4. Interaction Robustness — 20%

Measures whether controls respond correctly, state persists where promised, empty/unavailable/error cases are handled clearly, and no misleading success state is presented.

- **0:** Core interactions fail or cause unrecoverable errors; state is corrupted or actions receive no meaningful response.
- **3:** Basic interactions work in common cases, but meaningful edge cases or environment constraints cause failures, unclear feedback, or unreliable state.
- **5:** Core interactions work repeatedly in supported environments; state is clear and appropriately persistent; errors and restrictions receive useful feedback.

Required output

Return valid JSON first, with this schema:

```
{
  "version": "V2-RAG",
  "criterion_scores": [
    {"criterion": "Content Completeness & Clear Contribution", "weight_percent": 30, "score_0_to_5": 0,
        "evidence_ids": [], "rationale": "", "confidence": "low|medium|high"},
    {"criterion": "Navigation & Task Completion", "weight_percent": 25, "score_0_to_5": 0, "evidence_ids": [],
        "rationale": "", "confidence": "low|medium|high"},
    {"criterion": "Readability & Visual Consistency", "weight_percent": 25, "score_0_to_5": 0, "evidence_ids": [],
        "rationale": "", "confidence": "low|medium|high"},
    {"criterion": "Interaction Robustness", "weight_percent": 20, "score_0_to_5": 0, "evidence_ids": [], "rationale":
        "", "confidence": "low|medium|high"}
  ],
  "weighted_total_0_to_100": 0,
  "limitations": []
}
```

After the JSON, provide a short evidence-based explanation. Cite evidence IDs for every claim. Identify unsupported tasks explicitly. Do not modify, round away, or omit any criterion score. Do not include any human score or human opinion.

Evidence supplied

The clean-session Judge should receive only this prompt, `rubric.md`, and the following V2-RAG evidence. Do not provide `human_v2_assessment.md`, `v0_v2rag_analysis.md`, or any human score/commentary. The current evidence below was captured or verified on 2026-10-27/28; older

screenshots and browser output are preserved under `iterations/` and describe earlier interfaces or reported failures.

- **E1 — Website screenshot:** `evidence/V2-RAG-01_retrieved-answer-desktop.png` (clear RAG query, concise English answer, citations, manually opened cleaned reference materials).
- **E2 — Abstention screenshot:** `evidence/V2-RAG-02_insufficient-materials-desktop.png` (vague query and exact fallback).
- **E3 — Browser E2E record:** `e2e_results_v2.json` (nine current desktop checks, no console errors, no external requests).
- **E3a — Agent-definition screenshot:** `evidence/V2-RAG-03_agent-definition-desktop.png` (repaired English answer cites the agent definition instead of GNN material, with references closed by default).
- **E3b — CNN-significance screenshot:** `evidence/V2-RAG-04_cnn-significance-desktop.png` (exact query, cited explanation and expandable references).
- **E3c — MCP-definition screenshot:** `evidence/V2-RAG-05_mcp-definition-desktop.png` (MCP definition/architecture/transport answer and retained references).
- **E4 — Query/retrieval archive:** `query_runs_v2.json` (queries, candidates, passage text, source metadata, complete assembled prompts, page answers). Treat this as implementation/test evidence, not independent proof of correct semantic retrieval.
- **E4a — Generic pipeline assertions:** `relevance_regression_v2.json` documents ten shared invariants applied to a 28-query corpus. It tests semantic branching, relevant-only citations, preservation of every BM25 hit, visible output, clean English answers, answer shape, coherent explanatory prose, disclosure, and shortcuts. It contains no query-specific expected answer. These are script results, not independent browser proof.
- **E5 — Website evidence:** V2 `website/index.html`, `website/projects.js`, `website/rag-v2.js`, and `website/rag-knowledge.js`. Source code supports implementation claims only; it does not prove behavior by itself.
- **E6 — Frozen V0 baseline evidence for comparison:** `../../../coursework/website/V0/` and the V0 evaluator evidence listed in `../../../coursework/evaluation/ai_raw_output.md`. Only provide these items if the Judge is explicitly asked to compare versions; do not include human scores or assessments.

Clean Judge run metadata (complete after an independent session)

- Model/service:
- New blank session used: Yes / No
- Date/time (Asia/Hong_Kong):
- Evidence IDs sent:
- Raw output file/hash:

---

answer card cleanup log

Archive source: `prompt_appendix/task3_v2_evidence/answer_card_cleanup_log.md`

V2-RAG Answer Card and Reference Disclosure Revision — 2026-10-24

Request and baseline

The user prohibited three families of filler text, required an independently readable English answer with one opening sentence and 2 - 4 concise cited points, requested filtering of irrelevant recalled passages from the answer while preserving them in the expandable evidence area, and specified the title “View Retrieved Reference Materials.” The previous implementation and query archive are preserved in `iterations/2026-10-24-before-answer-card-cleanup/`.

Code changes

- `website/index.html`: renamed the native, default-closed `details/summary` disclosure to **View Retrieved Reference Materials**.
- `website/rag-v2.js`: removed the visitor-facing inability/redirect fallback. The generic evidence selector now discards malformed or irrelevant English fragments and requires enough distinct supported facts to form an opening sentence plus at least two points. If lexical candidates lack answer-bearing evidence, they are retained in the internal `candidatePassages` trace but are not promoted to `retrievedPassages`; the final reference set is empty and the exact no-material message is displayed. This is a deterministic evidence gate, not live translation or LLM generation.
- Curated source-backed summaries cover the newly tested convolution question in addition to the existing course topics. Each answer point ends in the IDs of passages it uses. Irrelevant ranked passages remain visible in the reference disclosure when the overall retrieval is answerable; they are excluded from the Answer text.
- `cleanNote` removes fenced-code markers, inline LaTeX blocks, strikethrough/Markdown decoration, and table separator rows before visitor rendering. The disclosure can scroll and shows complete cleaned passages rather than truncating them mid-sentence.
- `website/style.css`: softened the Answer panel with generous rounded corners, translucent pale blue-white surfaces, and more comfortable text size and line spacing.

Verification boundary

At this revision, the fixed set contained 24 deterministic queries, including `What is convolution?`. The answer-generation retrieval gate described above later caused a false abstention for MCP and was removed; see `mcp_answer_gate_repair_log.md`. Browser E2E/screenshots and independent AI Judge results remain pending. Frozen V0 and V1 were not edited.

---

answer consistency and language guard

Archive source: `prompt_appendix/task3_v2_evidence/answer_consistency_and_language_guard.md`

Answer Consistency and Language Guard

Date: 2026-10-27 Scope: `website/rag-v2.js` only. The frozen V0 source, the knowledge corpus, the page markup and the styling were not modified.

Reported gaps

Two problems remained after the previous iteration:

1. Some answers still quoted the raw classroom sentence — an English statement from the lecture followed by its Chinese rendering, both inside one answer, or a Chinese answer carrying the English original of the note.
2. The order of the stages in a process answer was not stable: the same question could return mapping/shuffling/reducing, or a reflection-token list from a neighbouring section.

Diagnosis

Both gaps came from the same place: the answer was assembled from *any* retrieved section whose statements happened to score well, in whatever order the scoring produced.

Query	Evidence of the defect
`What is retrieval-augmented generation and how does it use retrieved context?`	The answer ended with `Retrieve? decides before retrieval whether external knowledge is needed at all. [R3]` — a Self-RAG reflection token, from a section about a *different* concept.
`How does self-attention work?`	The same Self-RAG section answered the question, because the token `self` of `self-attention` matched `Self-RAG` in a heading.
`cnn的开创性意义`	Bullets carried `操作定义：Convolution is essentially sliding a filter over the input. 卷积本质上就是…` — the English original and its translation in one statement.
`How do I evaluate tool use by an LLM agent?`	The answer was ordered by claim role instead of by the section, so the five layers appeared in an arbitrary order.
`What are the limitations of RAG?`	`what are …` was read as a definition question, so the answer led with the definition of RAG rather than a limitation.

General repairs

All of the following are rules over the query and the corpus. None of them mentions a specific question, and none of them contains an expected answer.

1. **Concept-variant guard** (`isVariant`). A heading that opens with a restrictive modifier (`Self-RAG`, `Multimodal RAG`, `传统 RAG`, `多智能体`, `Multi-turn`) is about a variant of the concept, not about the concept. Such sections stay in the retrieved evidence but lose their claims in definition answers; the exception is a `limitation` or `comparison` question, where the variant section is often exactly the evidence that is wanted.

2. **Head-token rule.** In a noun phrase the head is the last content word, so `self-attention` is about attention. A restrictive modifier alone can no longer establish that a heading is relevant.
3. **One-section reading.** The section that supplied the lead is read forward in source order, so a process answer keeps the order of the notes and its bullets cite the same section. Other sections only fill a shortfall, and only from the same part of the same lecture.
4. **Bilingual clause filter** (`zhOnly`). Notes are bilingual; a Chinese answer keeps only the Chinese clause of each sentence, and a quoted English original is cut at the point where its translation starts.
5. **Sentence shape filters.** Table debris (`a · b ⇒ c`), diagram arrows, course-meta commentary (全讲, 本讲), back-references (前者, 后者, 该) and illustration openers (例如, 示例) cannot lead an answer.
6. **Intent read from the noun, not the opening words.** `what are the limitations of X` is a limitation question; `what is X and how does it work` is still a definition question.
7. **Test-harness fidelity.** `scripts/run_queries.cjs` now renders `innerText` the way a browser does: inline nodes (a citation span) stay on the line of their parent, block nodes start a new line. The answer-shape invariant had been counting each citation span as its own line, so it measured the stub rather than the page; with the stub corrected, the invariant checks the real shape — one statement per line, one to five statements.

Verification

Check	Command	Result
Syntax	`node -c website/rag-v2.js`	passed
28-query corpus	`node scripts/run_queries.cjs`	every query answers or abstains with the exact sentence
Ten generic invariants	`node scripts/relevance_regression.cjs`	`{"passed":true,"queries":28,"genericRules":10}`
Desktop Chrome QA	`node scripts/e2e_v2.cjs`	`{"passed":true,"testCount":9,"errors":[],"externalRequests":[]}`

The browser run re-captured the five evidence screenshots in `evidence/`, and `query_runs_v2.json` holds the current complete record for all 28 queries.

Behaviour after the repair:

```
What is retrieval-augmented generation and how does it use retrieved context?
  RAG combines a pretrained language model with external knowledge retrieved at
  inference time to support answer generation. [R4]
  · The retriever searches the external source for the current question instead
    of relying on knowledge stored in the model parameters. [R4]
  · The retrieved passages are grounded context for generation rather than the
    final answer, and the model itself is usually not retrained. [R4]
  · The external source can be local documents, a knowledge base, web search or
    an API. [R4]
  · RAG supplies knowledge at inference time while fine-tuning changes the model
    parameters, so updating knowledge means updating the index rather than
    retraining. [R4]

How does MapReduce divide and combine work?
  A MapReduce job runs in three stages, mapping, shuffling and reducing…
  · Mapping: each node applies its mapper to the data it has already loaded
    locally. [R4]
  · Shuffling: nodes exchange the intermediate data produced by the mappers. [R4]
  · Reducing: reducers merge the results of the different mappers… [R4]
```

Limits

- `HDFS memory是什么` is a malformed question (two unrelated concepts); the answer stays readable but mixes two sections of the memory material.
- `overfitting和underfitting的对比` and `举几个GAN的例子` fall back to the Chinese sentence extractor, which is weaker than the prepared English evidence layer: the contrast and the examples are present, but the lead sentence is not the most summary-like sentence of the section.
- This is still a static GitHub Pages demo: no backend, no live language model. The English wording comes from a prepared evidence layer keyed by section, and the Chinese wording is quoted from the notes.

---

bare concept and process pipeline fix

Archive source: `prompt_appendix/task3_v2_evidence/bare_concept_and_process_pipeline_fix.md`

Generic Bare-Concept and Process-Answer Repair

Date: 2026-10-27

Reported failure

The short query `rag` retrieved four passages but produced an answer about a research timeline and agent applications. The screenshot supplied by the user showed that the selected answer did not define RAG, even though the visitor asked for the concept itself.

Root cause

This was not only a RAG-specific defect. Three shared pipeline rules were incomplete:

1. A short concept-only query was not consistently treated as a definition request.
2. The title reranker could only reorder the four BM25 results and did not strongly prefer a concise definition or overview unit during initial retrieval.
3. Once a title-qualified chunk entered answer composition, a sentence could be selected merely because the chunk title was relevant; the sentence itself did not always contain local evidence for the query.

Process questions had a second general weakness: evidence markers such as `stage`, `pipeline`, `execute`, `call`, `return`, `mapping`, `shuffle`, and `reduce` were not recognised consistently. Application-oriented long titles could also outrank shorter concept units.

Query-independent repair

The same rules now apply to every query:

- A query containing one to three subject tokens and no explicit intent is classified as a definition request.
- Definition/overview titles receive a retrieval boost; advanced, comparison and variant titles receive a penalty.
- Long, highly specific titles receive a small structural penalty so that concise concept units are preferred when other relevance evidence is similar.
- Process evidence recognises stage, pipeline, execution, call/return and map/shuffle/reduce language in both English and Chinese.
- A title admits a chunk, but each selected answer sentence must independently contain the queried subject or valid evidence for the requested non-definition intent.
- Heading fragments ending in a colon are excluded from the visitor answer.
- Retrieved noise is still retained in the folded reference panel for inspection; it is not silently deleted.

No fixed answer text and no query-specific answer branch were added for RAG, Transformer, MapReduce, tool use, GNN, MCP, back-propagation or gradient descent.

Cross-topic verification

The deterministic browser logic was executed through `scripts/run_queries.cjs` after the change.

Query	Observed post-repair behaviour
`rag`	Selects `RAG Definition` and explains that RAG combines a pretrained LLM with external knowledge retrieved at inference time.
`GNN`	Selects `What is a Graph Neural Network?` and explains the graph transformation and message-passing idea.
`Transformer`	No longer abstains; uses the Transformer Block unit and describes repeated blocks, Q/K/V attention, feed-forward processing and residual connections.
`How does MapReduce work?`	Returns process evidence including Shuffle and Reduce rather than a body-only keyword match.
`How do agents use tools?`	Explains how tool definitions enter context, how calls are produced, and how the harness executes and returns results.
反向传播	Continues to explain reverse gradient computation.
梯度下降	Continues to explain updates in the negative-gradient direction.

The fixed 28-query suite produced no blank answer cards. Vague and out-of-scope queries continued to use the explicit insufficient-material response.

Verification boundary

JavaScript syntax checking and deterministic query execution passed using the bundled Node.js runtime. Direct browser automation was attempted, but local browser control was disabled in the current Work settings. Therefore, the user should refresh `http://127.0.0.1:5584/#highlights` and

capture the final visual evidence manually; this document does not claim a new automated screenshot.

Remaining limitations

This is a static, extractive RAG demonstration without a live LLM. It can select and combine source sentences, but it cannot reliably rewrite every bilingual lecture fragment into polished English. Ambiguous one-word queries may still choose a specialised course meaning when several disciplines use the same term. These limitations should be disclosed in the V0 - V2 analysis rather than hidden.

---

base concept title reranking

Archive source: `prompt_appendix/task3_v2_evidence/base_concept_title_reranking.md`

Base-concept title reranking (2026-10-27)

Reported failure

For the query `GNN`, BM25 correctly retrieved both `What is a Graph Neural Network?` and `GNN vs CNN`, but the answer used the comparison/inductive-bias material instead of the definition. The original screenshot and pre-fix output are retained under `iterations/2026-10-27-before-base-concept-score/`.

Cause

The title gate decided whether a chunk was eligible, but the subsequent ordering still relied mainly on retrieval and semantic scores. A narrower comparison passage could therefore outrank a base-concept definition.

Repair requested and implemented

After title gating, each eligible unit receives a secondary `baseConceptScore` computed only from its existing title text; no knowledge-base field was added.

- `+10`: titles containing `What is`, `Definition`, 定义, 是什么, `Introduction to`, `Overview`, or 基础.
- `−10`: titles containing 归纳偏置, `bias`, `compare`, `vs`, 对比, `tradeoff`, `advanced`, or 变体.

The final `answerScore` is `semanticScore + baseConceptScore`. Eligible units are reordered by this score, and sentence extraction uses the same score. All BM25 results remain unchanged and visible in the folded evidence panel.

Result

For `GNN`, `[R1] What is a Graph Neural Network?` now receives `baseConceptScore = +10` and supplies the definition, optimizable-transformation explanation, and message-passing description. `[R3] GNN vs CNN` receives `baseConceptScore = −10`, remains visible evidence, but no longer leads the answer. The complete post-fix trace is in `manual_base_concept_score_probe.json`. The 28-query, ten-rule generic regression suite remains passing.

---

bilingual evidence repair log

Archive source: `prompt_appendix/task3_v2_evidence/bilingual_evidence_repair_log.md`

Bilingual Evidence Repair Log

Date: 2026-10-25
Version: V2-RAG only; the frozen V0 source was not modified.

Reported behaviour

The query `attention` retrieved four reference materials. The first retrieved chunk directly explained bidirectional attention, causal attention, attention masks and encoder-decoder cross-attention, but the Answer card displayed:

```
Insufficient reference materials to answer this question.
```

Root cause

Retrieval and semantic relevance filtering succeeded. All four recalled chunks were preserved and the relevance layer accepted the directly explanatory chunks. The later evidence-extraction stage, however, required complete English predicate sentences. Because the strongest course-note evidence was mainly Chinese prose with English technical terms, it was discarded before answer composition. The final abstention therefore represented an answer-generation failure rather than a retrieval failure.

General repair

`website/rag-v2.js` now contains a source-grounded bilingual evidence normalisation stage before the existing English evidence filter. The stage:

1. reads explanatory structures from the retrieved source text;
2. converts supported bilingual note statements into concise academic English;
3. retains the original reference number for every converted claim;
4. does not inspect or branch on a particular user query;
5. adds no fact that is absent from the cited chunk.

The BM25 retrieval logic, folded reference panel, evidence cleaning, citation rules and zero-evidence fallback are unchanged.

Verification

- JavaScript syntax check: passed.
- Fixed 28-query evaluation corpus: passed.
- Generic regression rules: 10/10 passed.
- Manual generic probes: `attention`, `attention是什么`, `How does attention work?`, and `Transformer 原理` all produced cited answers.
- Browser check at `http://127.0.0.1:5584/#highlights`: passed. The query `attention` displayed one lead sentence and four cited explanatory points; the reference disclosure remained closed by default.
- Browser console/debug prompt content was not rendered.

The complete manual probe is preserved in `manual_bilingual_evidence_probe.json`. The pre-change source and test scripts are preserved in `iterations/2026-10-25-before-bilingual-evidence-fallback/`.

Remaining limitation

This is a deterministic static GitHub Pages demonstration, not a live LLM. Its bilingual normalisation can only reorganise structures represented by the local rules and source corpus. Unsupported source phrasing may still cause a cautious abstention, even when a human reader could infer a partial answer.

---

clause provenance validation log

Archive source: `prompt_appendix/task3_v2_evidence/clause_provenance_validation_log.md`

Clause and Formula Provenance Validation

Reported defect

A diffusion-model answer cited a reconstruction loss, `L = (1/n) sum (x^i - G(E(x^i)))^2`, taken from a chunk that compared several generative-model families. The chunk was broadly relevant to generative models, but that individual formula belonged to another model column. Chunk-level relevance alone was therefore insufficient.

Generic repair

The retrieval output and chunk-level semantic filter remain unchanged. A second evidence-level check now applies to every candidate clause and formula:

- A prose clause must overlap the query subject, be an explanatory/anaphoric clause from a directly matching topic, or be explanatory evidence from an accepted exercise solution.
- A formula is inspected in its local source window. It is retained only when that local context contains the query subject/acronym, when an accepted exercise solution directly supplies it, or when a single-formula directly matching chunk provides it.
- If a Markdown table contains several formulas, column ownership is considered ambiguous. No formula is extracted from that table unless reliable column attribution exists; the prose evidence can still be used.
- Every rendered sentence retains the reference number of the exact chunk from which its text or formula was extracted.

These rules contain no branch for diffusion models, VAEs, GANs, DQN, CNNs, or any other named topic.

Evidence

- `manual_clause_provenance_probe.json` records a non-Q-number diffusion query. Its answer uses diffusion-process evidence and does not contain the reconstruction loss.
- `manual_exercise_solution_probe.json` confirms that a worked exercise's single arg-max formula remains available and correctly cited.
- `relevance_regression_v2.json` now includes the shared `formula-provenance` invariant. The 28-query corpus passes all ten generic invariants.

This is deterministic script evidence. Current browser E2E and independent AI-Judge verification remain pending.

---

cnn significance repair log

Archive source: `prompt_appendix/task3_v2_evidence/cnn_significance_repair_log.md`

V2-RAG CNN Significance Retrieval Repair — 2026-10-24

Reported failure and preserved evidence

User query: `cnn的开创性意义.`
Before repair: no reference passages and the exact no-material fallback, as shown in the user's screenshot at `iterations/2026-10-24-before-cnn-retrieval-fix/cnn-no-answer-user-screenshot.png`. The pre-repair retrieval ranked AlexNet first at score 5.2767 but gave it only 1/6

query-token coverage (0.1667), below the 0.24 reliability threshold, so it discarded all candidates. Query tokenization included 的开, 开创, 创性, and 性意, although the notes explain the concept using different wording.

The pre-repair V2 script, test runner, regression script, and query archive are preserved under `iterations/2026-10-24-before-cnn-retrieval-fix/`. Frozen V0 and V1 were not edited.

Actual course-note evidence

The knowledge base already contained relevant material in `knowledge/5205-Introduction-to-AI.md`:

- **LeNet-5, "the first successful convolutional network"**: raw-pixel MLPs had too many parameters and ignored spatial structure; weight sharing and local connectivity drastically reduce parameters; LeNet-5 was applied to handwritten-digit recognition.
- **AlexNet milestone**: scaling CNNs to ImageNet-scale datasets with GPU training, ReLU, dropout, and data augmentation.
- **CNN Definition**: stacked transformations learn progressively more sophisticated representations of visual input.

These claims support an answer about CNNs' pioneering significance without outside facts.

Changes

1. In `website/rag-v2.js`, recognize CNN "significance / importance / impact / 开创性意义" as one concept query. Remove the Chinese question phrase before tokenizing; expand with source-attested technical terms (`weight sharing`, `local connectivity`, `spatial`, `image recognition`). Apply a focused ranking boost only to the Introduction to AI sections whose original paragraphs actually contain those terms.
2. The resulting retrieved passages are LeNet-5 `[R1]`, AlexNet `[R2]`, and CNN Definition `[R3]`. The visible English answer summarizes the precise contribution: fewer parameters while preserving image structure, a handwriting-recognition demonstration, subsequent large-dataset scaling, and layered visual features.
3. Add Q19 exact user query and two paraphrases Q20/Q21 to the deterministic archive and regression suite. The browser E2E script now includes the exact typed query and a pending repaired-answer screenshot.

Verification and limits

The deterministic suite passes **21/21** cases. Q19 - Q21 each retrieve three relevant passages and return the same cited English explanation. The original no-answer screenshot is retained as failure evidence; no post-repair browser screenshot or browser E2E result has been claimed. The same lexical retrieval still has limits on unsupported paraphrases and concepts; these three checks demonstrate this specific repair, not universal semantic understanding.

---

coherent answer generation log

Archive source: `prompt_appendix/task3_v2_evidence/coherent_answer_generation_log.md`

Coherent Answer Generation Revision

Requirement

The shared RAG pipeline must not render `The notes identify ...`, terminology lists, or raw table fragments as an answer. Every supported answer must use one explanatory lead sentence and two to four complete explanatory points, with citations at sentence ends. This rule must apply uniformly and must not introduce topic-specific branches.

Implementation

The semantic relevance stage and BM25 retrieval remain unchanged. The shared `englishEvidence()` and `composeAnswer()` stages now:

1. remove Chinese remnants, markup, questions, instruction-like text, URLs, malformed formula fragments, and label-only rows;
2. accept only English clauses containing a grammatical predicate and at least five words;
3. convert only general bilingual-table syntax, such as `Subject · Collecting ...`, into the grammatical form `Subject involves collecting ...`;
4. reject capitalized mid-line table commands such as `Map Reduce` when they are not predicates;
5. deduplicate overlapping evidence;
6. require at least three supported sentences before rendering an answer;
7. render one lead plus two to four bullet points, each ending in its supporting `[R#]` citation;
8. use the exact insufficient-material response when the retrieved material cannot support a coherent English explanation.

No course topic, model name, query phrase, Q-number, or fixed answer appears in these generation rules.

Evaluation

The historical 28-query corpus is retained unchanged. The generic regression suite now applies ten shared invariants, including a new `coherent-explanatory-prose` rule that rejects filler templates and terminology-only lines. All 28 queries pass the ten generic invariants. This is script-level evidence; current browser E2E and independent AI-Judge assessment remain pending.

---

component relevance log

Archive source: `prompt_appendix/task3_v2_evidence/component_relevance_log.md`

Parent Concept and Component Relevance

Reported behavior

The query `Transformer原理` recalled a KV Cache passage containing Q/K/V roles, causal attention, and cache reuse, but the answer abstained because the English evidence generator required direct query-level prose. The recalled passage was valid component evidence for the parent concept.

Relevance rule

The semantic filter now supports a lightweight parent-component relation. A recalled passage may be relevant when:

1. the query names a parent concept;
2. the passage's own topic/body is directly about a recognized component of that concept; and
3. the evidence still passes clause-level provenance checks before it can be cited.

For Transformer, the current component vocabulary covers QKV, multi-head attention, KV Cache, and residual connections. This relation changes relevance only; it contains no fixed answer text.

Answer construction

The same source-grounded answer generator remains in use. A general bilingual-structure normalizer converts explanatory source statements into grammatical English while retaining their exact reference:

- a `subject is ... mechanism` sentence becomes an English mechanism statement;
- labeled Query/Key/Value role descriptions become complete explanatory sentences;
- causal-attention reuse statements remain attached to the same chunk;
- formulas still require local subject agreement and formula provenance validation.

No Transformer-specific answer paragraph is stored in the implementation. The rendered content is assembled from the recalled passage.

Evidence

`manual_component_relevance_probe.json` records a non-Q-number `Transformer原理` probe. The single KV Cache reference is accepted, and the output contains a lead mechanism sentence plus four explanatory component points, all cited `[R1]`. The existing 28-query corpus continues to pass all ten generic invariants. Browser E2E and independent AI-Judge verification remain pending.

compound query title gate fix

Archive source: `prompt_appendix/task3_v2_evidence/compound_query_title_gate_fix.md`

Compound-query title-gate repair (2026-10-27)

Reported failure

The query `What is retrieval-augmented generation and how does it use retrieved context?` retrieved four clearly relevant RAG units but returned `Insufficient reference materials to answer this question.` The user screenshot and complete pre-fix query result are preserved under `iterations/2026-10-27-before-compound-query-fix/`.

Cause

The title gate treated both the main concept and the secondary process clause as title subject tokens. A title such as `RAG Definition` could not contain terms from `how does it use retrieved context`, so every retrieved unit failed the title-coverage threshold. This was a relevance-filter false negative, not a knowledge-base or BM25 retrieval failure.

Generic repair

1. Split compound questions only when a conjunction introduces a new `how`, `why`, `what`, `where`, or `when` clause.
2. Use the first clause to identify the title-matching subject.
3. Keep the full question available as the requested answer intent.
4. Infer an acronym only from the main subject clause.
5. Prefer a base definition title over a prefixed or suffixed variant such as `Self-RAG` when the query asks for the base concept.

No fixed RAG answer was added. The same compound-query and base-topic ordering rules apply to every query.

Result

The exact reported query now selects `RAG Definition` as the highest-priority unit and returns a cited explanation beginning with the definition of RAG and its use of external knowledge at inference time. The complete post-fix output is stored in `manual_compound_query_probe.json`. The fixed 28-query suite continues to pass all ten generic invariants.

deployment record

Archive source: `prompt_appendix/task3_v2_evidence/deployment_record.md`

V2-RAG GitHub Pages Deployment Record

- Deployment date: 2026-10-28 (Asia/Hong_Kong)
- Public URL: https://liuxinyue84-sketch.github.io/V2/
- Repository: https://github.com/liuxinyue84-sketch/liuxinyue84-sketch.github.io
- Branch: `main`
- Commit: `0091834fdb54ac56fe47b72cfb29b049ab7582d6`
- Commit message: `Deploy Aria Sky Journal V2 RAG`
- GitHub Pages workflow run: https://github.com/liuxinyue84-sketch/liuxinyue84-sketch.github.io/actions/runs/40682186055
- Workflow conclusion: `success`

Scope protection

The V2 source was deployed as the new `/V2/` directory. Existing root files serving V0 and the existing `/V1/` directory were not modified by the deployment commit.

Post-deployment checks

Fresh public V2 RAG E2E run — 2026-10-28

The public URL was tested with the bundled Playwright/Chrome runner at a 1512x1000 desktop viewport. All 9 tests passed with zero browser/console errors: RAG preset retrieval, MapReduce ordering, memory summary, vague-query abstention, out-of-scope abstention, tool-use retrieval, Chinese agent definition, CNN significance, and MCP definition. The run also verified closed-by-default reference disclosure, cleaned reference text, citation consistency, no prompt leakage, and no failed assertions. Evidence is stored in `e2e_results_v2.json` and five V2 screenshots in `evidence/`.

A second public run covered non-RAG portfolio paths. Home, all four navigation destinations, education disclosure, four projects, project detail expansion, research filtering, save/remove state, Life media count, photo preview, and Guestbook form all passed at the same viewport. The result is archived in `nonrag_public_e2e.json`; the only console message was a favicon 404.

The following public resources were requested after the Pages workflow completed:

Resource	Result
`/V2/`	HTTP 200; HTML title `Aria's Sky Journal` and `Explore my notes` marker present
`/V2/rag-v2.js`	HTTP 200; JavaScript served
`/V2/rag-knowledge.js`	HTTP 200; 4,920,462 bytes served

Resource	Result
`/V2/assets/sky.webp`	HTTP 200; WebP served

The remote `main` ref resolved to the same deployment commit recorded above.

---

exercise solution relevance log

Archive source: `prompt_appendix/task3_v2_evidence/exercise_solution_relevance_log.md`

Exercise and Solution Relevance Rule

Requirement

A recalled chunk must not be rejected merely because it comes from a classroom exercise, quiz, multiple-choice question, or worked solution. If the explanatory text contains a queried concept's definition, mechanism, formula, decision rule, role, or supported result, the chunk is valid evidence. A bare question or option list remains noise.

Generic implementation

The shared `assessChunk()` function now detects exercise-source markers in the topic or section path and separately detects explanatory-solution evidence in the passage. The evidence vocabulary is general: answer/solution/explanation markers, causal or definitional language, estimation and approximation language, selection/maximization language, and formula notation. The query subject or acronym must still occur in the body or topic. This rule contains no model-specific query branch.

Exercise explanations receive the reason label `exercise solution contains direct explanatory evidence`. All recalled chunks remain visible in the folded reference panel; only chunks that pass the shared relevance rule can be cited.

The shared evidence generator also preserves short mathematical expressions from worked solutions. It renders formulas as complete explanatory sentences, such as a decision rule that maximizes a stated expression, instead of exposing raw Markdown/LaTeX.

Manual generic probe

`manual_exercise_solution_probe.json` records a manual probe asking how a value-based deep-reinforcement-learning method chooses an action. The exercise solution is accepted as relevant evidence, its arg-max rule is cited, and supporting value-function material is combined into the answer. The probe is not appended as a new Q-number and does not add a topic-specific expected-answer test.

The existing 28-query corpus still passes all nine generic invariants. Browser E2E and independent AI-Judge verification remain pending.

---

generic semantic pipeline refactor

Archive source: `prompt_appendix/task3_v2_evidence/generic_semantic_pipeline_refactor.md`

Generic Semantic RAG Answer Pipeline Refactor

Trigger

Manual testing exposed a structural defect: the earlier static demo added a topic-specific answer branch whenever a new wording failed. This produced brittle behavior, duplicated logic, and blank Answer cards for unrecognized queries even when BM25 had recalled useful material.

Preserved behavior

- The existing BM25 keyword retrieval and its four-hit limit remain the retrieval layer.
- Every recalled hit remains visible as its original `R1` - `R4` entry in the default-closed **View Retrieved Reference Materials** panel.
- The UI, reference cleaning, citation format, and exact insufficient-material message remain unchanged.
- The pre-refactor implementation and query output are preserved in `iterations/2026-10-24-before-generic-semantic-pipeline/`.

Shared pipeline

Every query now follows the same code path:

1. `queryProfile()` detects a general information need (definition, process, reason, example, comparison, limitation, or general) and removes question phrasing from the subject tokens.
2. `assessChunk()` compares the subject with each recalled chunk's topic and body, checks whether the passage contains evidence appropriate to the general intent, penalizes navigation/outline text, and labels the chunk relevant or noise.
3. `filterRelevantChunks()` keeps semantic decisions separate from retrieval. Noise remains visible in the reference panel but cannot support the answer.
4. `englishEvidence()` extracts clean English evidence from every relevant chunk, removes markup, instruction-like source text, questions, duplicates, and malformed fragments.
5. `composeAnswer()` selects up to five distinct cited lines. It contains no topic names, query-specific branches, or fixed answers.
6. When relevant evidence is absent, the Answer card always displays `Insufficient reference materials to answer this question.` It never remains blank.

Evaluation change

The 28 accumulated queries are retained as an evaluation corpus and historical evidence. `relevance_regression.cjs` no longer contains a separate expected sentence for each Q-number. It applies eight shared invariants to the complete corpus:

- visitor controls remain unchanged;
- the reference disclosure remains present;
- every query produces visible output;
- the relevant/zero-relevant branch is consistent;
- every citation points to a chunk accepted by the shared semantic filter;
- all BM25 hits remain in the evidence panel;
- visitor output contains no Chinese answer text, prompt/debug material, or internal KB identifiers;
- each answer contains one lead and no more than four cited points.

Current deterministic result: **28 evaluation queries passed all 10 generic rules**. This is script-level evidence, not browser E2E or independent AI-Judge evidence.

---

knowledge base manifest

Archive source: `prompt_appendix/task3_v2_evidence/knowledge_base_manifest.md`

V2-RAG Knowledge Base Manifest

The browser corpus at `website/rag-knowledge.js` is generated by `scripts/build_knowledge_base.py` from frozen V0 materials. Each record contains an ID, topic title, section path, original Markdown paragraph, and source metadata. Source paragraphs are retained in full; the retrieval layer filters non-informative image-only/very short records without deleting them from the corpus.

- 8 lecture-note Markdown files in `website/knowledge/`
- 9,811 paragraph records from the lecture notes
- 20 supplemental records from four project profiles in frozen V0 `projects.js`
- **9,831 total records** across 9 distinct source files
- The research-paper profile carries the DOI `https://doi.org/10.3390/fi17080353`; its descriptive text is inherited from V0 `projects.js`.

The full serialized JavaScript knowledge object is the canonical corpus archive. `query_runs_v2.json` records the passages actually retrieved for every test query, their BM25 scores, assembled prompts, and displayed answers. Rebuild with:

```
python3 scripts/build_knowledge_base.py
```

The source notes and project records are preserved unchanged in the V2 website copy. V0 itself was not modified; hashes captured before work are recorded in `v0_baseline_sha256.txt`.

---

linked answer fix log

Archive source: `prompt_appendix/task3_v2_evidence/linked_answer_fix_log.md`

V2-RAG Retrieval/Answer Linkage Repair — 2026-10-24

User-reported defects

1. A query could display one or more retrieved reference passages while the answer area said `Insufficient reference materials to answer this question.` The retrieval stage and the small curated-answer catalog had separate success conditions. For example, **"How do agents use tools?"** retrieved four passages, while the former answer function returned `null` and displayed the no-material message. **"What are the limitations of RAG?"** retrieved one relevant passage and produced the same contradiction.
2. The visitor shortcut row still contained a Chinese `Agent 是什么?` test button. The tool-use shortcut also used a label different from the requested "How do agents use tools?".

The preceding implementation, scripts, and query output are retained under `iterations/2026-10-24-before-linked-answer-fix/`. Frozen V0 and V1 were not edited.

Repair

- `website/rag-v2.js`: `renderAnswer` now selects the exact no-material sentence **only when `hits.length === 0`**. Every nonempty retrieval gets a source-cited English response. Source-checked summaries cover the common topics; other queries use matching English evidence sentences from the retrieved passages, and Chinese-only passages receive a cautious English source description. This is deterministic formatting, not a live LLM call.
- `website/rag-v2.js`: the internal assembled prompt template now states the same zero-hit-only abstention rule. It remains in code and evaluation records, never in visible HTML.
- `website/rag-v2.js`: added supported summaries for agent tool use, simple-RAG limitations, diffusion sampling, and reinforcement learning. Broad `memory` results summarize both meanings with `[R1]` and `[R2]` citations instead of only posing a follow-up question.
- `website/index.html`: the shortcut row is exactly **What is RAG?**, **How does MapReduce work?**, **How do agents use tools?**. The `agent是什么` case remains available in the test archive and typed form, but no Chinese debug shortcut appears on the page.
- `scripts/relevance_regression.cjs`: each query now enforces the invariant `retrieved passages > 0 ⇒ cited answer without the no-material sentence`; `retrieved passages = 0 ⇒ exact no-material sentence`. Four new cases test the reported contradiction and cross-source citations. `scripts/e2e_v2.cjs` checks the same rule and shortcut labels when a browser rerun is available.

Current verification and boundary

The deterministic suite passes **18/18** cases. In archived results, the tool-use query has four passages and a cited English answer; the RAG-limitations query has one passage and a cited English answer. The zero-hit vague and tiramisu queries show only the exact no-material sentence. The V0 baseline hashes remain unchanged.

The fallback summary is intentionally conservative because GitHub Pages cannot run a private server-side LLM. It may describe a retrieved passage rather than fully answer an arbitrary question; it never invents unsupported details or claims there are no materials when materials are displayed. Revised browser screenshots, browser E2E, human reassessment, and the independent AI Judge result are still pending and must not be inferred from the deterministic test output.

---

mcp answer gate repair log

Archive source: `prompt_appendix/task3_v2_evidence/mcp_answer_gate_repair_log.md`

V2-RAG MCP Answer-Gate Repair — 2026-10-24

Reported regression

`What is MCP?` correctly ranked four course-note sections: MCP basic concepts, Host - Client - Server architecture, the relationship among those elements, and transport methods. A later answer-quality gate nevertheless called `composeAnswer()` inside `retrieve()` and replaced all four hits with an empty array when the mixed Chinese/English notes did not yield three generic English evidence sentences. The visitor therefore received the no-material response despite valid retrieval.

The pre-repair source and test files are preserved under `iterations/2026-10-24-before-removing-answer-gate/`.

Repair

- Removed the answer-generation gate from `retrieve()`. Retrieval is again decided by BM25 evidence strength and query coverage; answer formatting no longer erases valid hits.
- Added a source-checked MCP summary derived from the recalled notes. It explains MCP as a standardized agent-facing interface, its integration purpose, the Host - Client - Server separation, and stdio/HTTP transports with `[R1]`, `[R2]`, and `[R4]` citations.
- Added Q25 `What is MCP?` to the query archive and regression suite.

Verification boundary

The first repair passed **25/25** cases: Q25 retained four recalled passages and produced one opening sentence plus three cited points. A later manual test exposed a second wording-specific failure: `MCP的作用` retrieved four passages but left the Answer card blank because the composer did not recognize the Chinese role intent and the MCP concept section was not guaranteed to rank first. The retriever now expands and boosts MCP concept, architecture, and transport terms; the composer can also answer from a valid architecture passage. Q26 preserves this exact reported query. The deterministic suite now passes **26/26** cases. This is script-level evidence; revised browser E2E/screenshots and independent AI Judge output remain pending. Frozen V0 and V1 were not edited.

---

multi source answer log

Archive source: `prompt_appendix/task3_v2_evidence/multi_source_answer_log.md`

V2-RAG Multi-Source Answer Revision — 2026-10-24

Request and prior behavior

The user required a self-contained, substantive English answer that examines every retrieved passage, integrates relevant information, and cites every passage actually used. The prior V2 answer catalog often cited only `[R1]` even when four references appeared. Its generic fallback could say "Open the cited reference for its detailed explanation." That did not answer the visitor's question.

The prior script and 21-query archive are retained in `iterations/2026-10-24-before-multi-source-answer/`. Frozen V0 and V1 were not edited.

Implementation

- `website/rag-v2.js` now inspects all retrieved paragraphs when preparing the answer. Source-checked supplementary claims enrich the existing curated explanations. A passage is cited only if its text supports the claim and it is relevant to the question; an unrelated high-ranked passage is not forced into the answer merely to show `[R4]`.
- The generic extractive fallback selects distinct English evidence from different retrieved passages. It no longer contains an instruction to open references. If none of the available passages provides a reliable English clause, the interface states that limitation rather than inventing a translation. This remains a static deterministic demonstration, not real LLM generation.
- The internally archived RAG prompt now explicitly asks for multi-passage synthesis and precise citations. It remains absent from the visitor page.
- Added two regression queries beyond the original 21: `What is overfitting?` and `How does breadth-first search work?`. Their cited answers combine definition/variance evidence and

queue/optimality/example evidence respectively.

Deterministic evidence

`query_runs_v2.json` stores every query, all retrieved original paragraphs, the assembled internal prompt, and the visitor answer. `relevance_regression_v2.json` records **23/23** passing deterministic checks. The checks reject the banned placeholder wording, citations to nonexistent `[R#]` items, non-English output, and the nonempty-references/insufficient-answer contradiction. In particular, Q01 uses `[R1][R3][R4]`, Q02 uses `[R1][R2][R4]`, Q17 uses `[R1][R2][R3][R4]`, and Q19 - Q21 use `[R1][R2][R3]`.

These are code-level checks, not browser E2E or an independent AI Judge. The multi-source answer strategy is bounded by the fixed course-note corpus and deterministic templates. It cannot guarantee an accurate fluent synthesis for every arbitrary Chinese-only query; this limitation is exposed rather than masked with fabricated facts.

---

query run metadata

Archive source: `prompt_appendix/task3_v2_evidence/query_run_metadata.md`

Query Run Metadata

- Version: V2-RAG
- Query cases: 25 (original six, eight agent/GNN relevance regressions, four retrieval/answer linkage cases, three CNN significance cases, three multi-source/answer-format cases, one MCP answer-gate regression)
- Structured knowledge records: 9831
- Method: client-side BM25 retrieval over neighboring source paragraphs + cited deterministic English summaries enriched from relevant recalled passages, exact no-material response only for zero hits
- Evidence detail: `query_runs_v2.json` stores exact queries, tokenization, ranked candidates, retrieved original paragraphs, source metadata, complete assembled prompts, page answer text, cleaned visitor reference text, and whether references were open by default.
- `relevance_regression_v2.json` applies ten shared invariants to all 28 archived evaluation queries. It checks output totality, relevant/irrelevant branching, relevant-only citations, preservation of all BM25 evidence, visitor-output cleanliness, answer shape, and the unchanged controls/disclosure. It contains no query-specific answer assertions.
- Revised browser check: pending; `e2e_results_v2.json` contains the pending marker. The initial browser result and screenshots are preserved under `iterations/2026-10-24-before-ui-simplification/`.

---

query set

Archive source: `prompt_appendix/task3_v2_evidence/query_set.md`

V2-RAG Test Query Set

Queries cover explicit concepts, a broad but relevant query, a vague query, and a completely out-of-scope request. Exact retrievals, BM25 scores, assembled prompt text, and page answer strings are archived in `query_runs_v2.json`.

ID	Category	Exact query	Expected behavior
Q01	Clear topic	What is retrieval-augmented generation and how does it use retrieved context?	Retrieve RAG definition and cite sources
Q02	Clear topic	How does MapReduce divide and combine work?	Retrieve MapReduce pipeline evidence
Q03	Clear topic	How do I evaluate tool use by an LLM agent?	Retrieve tool-selection/evaluation notes
Q04	Ambiguous across courses	What did we discuss about memory?	Summarize both retrieved meanings with citations and invite a narrower question
Q05	Vague	Tell me about that thing from class.	Abstain with the required exact message
Q06	Out of scope	What is the best recipe for tiramisu?	Abstain with the required exact message
Q07	Reported failure retest	agent是什么	Cite the agent definition; never answer with GNN material
Q08	Chinese paraphrase	什么是智能体	Cite the agent definition
Q09	English paraphrase	What is an agent?	Cite the agent definition
Q10	Chinese definition	什么是RAG	Cite the RAG definition
Q11	Confounder control	什么是图神经网络	Cite the GNN definition; keep it distinct from agent
Q12	Chinese definition	什么是MapReduce	Cite an introductory MapReduce definition
Q13	Qualified memory	agent memory是什么	Cite an agent-memory section
Q14	Qualified memory	HDFS memory是什么	Cite the HDFS metadata-in-memory section
Q15	Suggested tool use	How do agents use tools?	Show a source-backed tool-use answer, matching the visitor shortcut
Q16	Retrieved but previously contradictory	What are the limitations of RAG?	Summarize the cited study’ s limits; do not show the no-material message alongside a source
Q17	Cross-source process	How do diffusion models generate images?	Use the retrieved sampling passage with its actual citation number
Q18	Cross-source definition	What is reinforcement learning?	Use the retrieved agent/environment/reward passage with its actual citation number
Q19	User-reported zero-recall failure	cnn的开创性意义	Retrieve CNN milestone evidence and explain local connectivity, weight sharing, LeNet and AlexNet
Q20	Chinese CNN paraphrase	CNN的意义	Preserve the same evidence-backed CNN significance answer

ID	Category	Exact query	Expected behavior
Q21	English CNN paraphrase	Why were CNNs important for image recognition?	Preserve the same evidence-backed CNN significance answer
Q22	Multi-source concept	What is overfitting?	Integrate the definition and bias - variance note; cite both relevant sources, omit unrelated hits
Q23	Multi-source algorithm	How does breadth-first search work?	Explain queue and layers, equal-edge-weight optimality, and the route example with their source IDs
Q24	Independent-answer format	What is convolution?	Give a source-backed definition plus operation and CNN-layer points; avoid raw fragments or redirection text
Q25	Reported answer-gate regression	What is MCP?	Preserve all four relevant recalled passages and summarize concept, architecture, and transport with matching citations
Q26	Reported Chinese MCP-role regression	MCP的作用	Retrieve the MCP concept, architecture, and transport notes and produce a substantive cited English answer
Q27	Reported fitting-comparison regression	overfitting和underfitting的对比	Exclude outline/GAN noise and contrast model capacity, bias, variance, and generalization
Q28	Reported GAN-example regression	举几个GAN的例子	Retrieve image-synthesis and handwritten-digit evidence and produce a cited answer

The first six queries were retained from the initial evaluation set. Q07 - Q14 are relevance regressions for the reported agent/GNN mismatch. Q15 - Q18 test the rule that a nonempty retrieval must yield a cited English answer. Q19 - Q21 test the user-reported CNN zero-recall failure and two paraphrases. Q22 - Q24 check substantive, independent answers beyond the initial curated topics. Q25 checks that answer formatting cannot erase valid MCP retrievals, Q26 checks the Chinese “MCP role” wording, and Q27 checks the reported overfitting/underfitting comparison that previously retrieved outline noise and left the Answer card blank. The visitor interface exposes exactly three English shortcuts: “What is RAG?”, “How does MapReduce work?”, and “How do agents use tools?”. The reported typed queries remain in the regression suite. The prior browser run is preserved under `iterations/2026-10-24-before-ui-simplification/`; revised browser E2E is pending.

---

rag prompt template

Archive source: `prompt_appendix/task3_v2_evidence/rag_prompt_template.md`

V2-RAG Prompt Template

This is the exact prompt structure assembled internally for each query. `{question}` is replaced with the visitor’s text. `{retrieved_materials}` contains ranked excerpts with `[R1]` - `[R4]`, topics, and source metadata, or the no-match marker. The completed prompts are archived in `query_runs_v2.json`; the visitor page does not display them.

```
SYSTEM: You are a course-notes assistant. Read every retrieved passage before answering. Integrate distinct, relevant facts into a concise academic English explanation with clear points. Cite every passage actually used as [R1], [R2], etc.; do not cite unrelated material or invent facts. The answer must be understandable without opening the references. Never replace a substantive answer with a direction to inspect a reference. Only when no passages are retrieved, say exactly: "Insufficient reference materials to answer this question."

USER QUESTION:
{question}

RETRIEVED REFERENCE MATERIALS:
{retrieved_materials}

TASK:
Synthesize the supported answer from all relevant retrieved passages, with precise source citations.
```

Execution boundary: this template is assembled and archived for coursework demonstration. No remote model receives it. The page’ s answer uses deterministic source-checked English summaries and evidence selection, not live LLM generation.

---

relevance fix log

Archive source: `prompt_appendix/task3_v2_evidence/relevance_fix_log.md`

V2-RAG Relevance Repair — 2026-10-24

Reported failure

User test/query: `agent是什么` in the V2 question form.
Observed before repair: the page cited `KB-1296`, “Part 5 GNNs I: From Graphs to GNNs”, and displayed “图是什么 → 为什么不能直接用 DNN → GNN 怎么做” as the answer. The user screenshot is preserved unchanged in `iterations/2026-10-24-before-relevance-fix/agent-wrong-answer-user-screenshot.png`.
Why this was wrong: the passage did not define an agent. It was selected because the tokenizer treated Chinese interrogative bigrams such as 是什 and 什么 as high-value search terms, while the short paragraph and repeated heading/path text distorted BM25 ranking. The response stage then quoted the first ranked hit without checking whether it actually answered the question.

Repair steps and code changes

1. In `website/rag-v2.js`, remove Chinese question phrasing before tokenizing the subject. Filter English stopwords before suffix normalization, so `does` does not become a false content term `doe`.
2. Group neighboring short original paragraphs from the same source section into retrieval units. The canonical `rag-knowledge.js` corpus remains unchanged at 9,831 records; each returned unit keeps its original record IDs and exact source text. Image-only Markdown paragraphs are excluded as answer evidence.
3. Weight matches in original paragraph bodies more heavily than title matches, and require body evidence. Apply definition-section boosts only when the section also matches the queried

subject. Distinguish a singular agent definition from a multi-agent-system definition. Add small bilingual concept aliases only when they help a focused definition question.

4. For explicitly qualified questions, require both core terms when there are two, such as `agent memory` or `HDFS memory`. For the broad `memory` question, show both agent-memory and computer-memory sources and ask which topic the visitor means instead of asserting that the HDFS meaning is intended.
5. When the user asks for a MapReduce process or tool-use evaluation, rank passages that actually describe the process or evaluation dimensions. The visible answer still quotes the leading source; no external LLM or semantic embedding model was added.

These are repairs to V2-RAG retrieval and answer selection, not a new technique and not changes to frozen V0 or V1. The pre-repair JS, test set, query output, and screenshot are preserved under `iterations/2026-10-24-before-relevance-fix/`.

The visitor page also adds an `Agent 是什么？` example question so the reported scenario can be retried directly.

After-repair observations

Query	Top source after repair	Visible behavior
`agent是什么`	`knowledge/5584-LLM-for-Agent-1.md`, “1.1 智能体（Agent）的定义”	Quotes an intelligent system interacting with its environment and cites `[R1]`.
什么是智能体; `What is an agent?`	Same agent-definition section	Same grounded definition, not GNN.
什么是图神经网络	GNN definition section	Keeps the GNN answer associated with a GNN question.
`How does MapReduce divide and combine work?`	“6.3 MapReduce Pipeline”	Excerpt includes Mapper and Reducer stages.
`How do I evaluate tool use by an LLM agent?`	“Part 7 Evaluating Tool Use”	Excerpt mentions tool selection, arguments, order, and operational quality.
`What did we discuss about memory?`	Agent-memory and computer-memory references	Requests clarification rather than choosing a meaning silently.
Vague and unrelated queries	No reliable source	Preserves the exact insufficiency fallback.

The exact 14 queries, candidate rankings, source passages, assembled prompts, and page strings are archived in `query_runs_v2.json`. `relevance_regression_v2.json` records 14 passing deterministic semantic assertions; the assertions verify source relevance and critical answer terms, not just that some text was returned.

Verification boundary and remaining risks

JavaScript syntax and the 14 deterministic checks passed. Frozen V0 file hashes were rechecked. A browser E2E rerun and screenshot of the repaired query are still pending because browser launch/local server access was blocked in this execution environment; `e2e_results_v2.json` remains a pending marker. Do not call the user screenshot a post-repair result.

The system is still lexical BM25 plus deterministic extractive rendering. It cannot guarantee relevance for every paraphrase or synthesize across sources like a real LLM-backed RAG system. Ambiguity and low-confidence matches should be treated conservatively in the report.

---

retrieval iteration log

Archive source: `prompt_appendix/task3_v2_evidence/retrieval_iteration_log.md`

Retrieval Iteration Log

These implementation trials are retained as development evidence. Retrieval scores are internal BM25 scores, not rubric scores.

1. **Initial unfiltered ranking:** Q02 (“How does MapReduce divide and combine work?”) ranked a `Tips:`-only fragment first (13.2068), ahead of a useful MapReduce passage. The indexer was changed to skip very short and image-only records during ranking while retaining every original record in the corpus. The final top result is a MapReduce pipeline passage (17.0853).
2. **Vague-query boundary:** Q05 (“Tell me about that thing from class.”) initially matched generic `class` fragments. Query normalization now removes generic terms after normalization/stemming. Q05 now has no meaningful query tokens and returns the exact abstention sentence. This is a lexical safeguard, not semantic intent detection.
3. **RAG definition ranking:** Q01 initially ranked an example passage (“National Day holiday weekend”) under a prompt-construction heading above the definition. Phrase-aware expansion adds the course acronym `RAG` for the explicit term “retrieval-augmented generation” and weights that expanded concept. The final top result is the note titled “RAG Definition” (25.1334), and the example remains visible as a lower-ranked reference (18.4772 in the final expanded-query run).
4. **Display cleanup:** Source Markdown remains verbatim in retrieved-reference cards; the answer display removes Markdown escape punctuation for legibility. The visible answer cites `[R1]`.

Known residual: keyword retrieval can still return adjacent examples or table rows that match terms but do not fully answer a natural-language question. The static extractive answer does not reason over multiple sources.

---

rubric

Archive source: `prompt_appendix/task3_v2_evidence/rubric.md`

Evaluation Rubric — Aria’s Sky Journal

Scope. This rubric is version-agnostic and must be applied unchanged to V0, V1, and V2. Each criterion is scored from 0 to 5. Intermediate integer scores (1, 2, and 4) may be used where performance falls between the stated anchors. The weighted total is calculated as `Σ (criterion score ÷ 5 × criterion weight)` and reported on a 0 - 100 scale.

Criterion	Weight
Content Completeness & Clear Contribution	30%
Navigation & Task Completion	25%
Readability & Visual Consistency	25%
Interaction Robustness	20%
Total	**100%**

1. Content Completeness & Clear Contribution — 30%

Definition. Measures whether the portfolio presents the expected personal, educational, skill, project, and contact content; whether project entries explain the owner’s role or contribution; and whether claims are specific enough for visitors to understand the work. Assess presence, internal consistency, and clarity, not whether the evaluator personally agrees with the claims.

Why this criterion matters. A portfolio is useful only when visitors can understand who the owner is, what they have done, and what their contribution was. Missing or ambiguous content weakens the site’s purpose even when its appearance is polished.

- **0 — Not met:** Most expected sections are absent or empty; project work is missing or gives no interpretable account of the owner’s role; major content is contradictory or unusable.
- **3 — Partially met:** Several expected sections and projects are present, but one or more important areas are missing, repetitive, vague, or do not clearly distinguish the owner’s contribution.
- **5 — Well met:** Relevant profile, education, skills, projects, and contact/personal content are present and coherent; project entries clearly state the owner’s contribution; no material content gaps prevent visitors from understanding the portfolio.

2. Navigation & Task Completion — 25%

Definition. Measures whether visitors can locate the main sections and complete representative tasks, including opening project details, filtering or saving projects, exploring the life gallery, using the course-note interface, and submitting a guestbook entry.

Why this criterion matters. Portfolio content must be findable and its promised actions must be discoverable. Navigation and task completion evaluate the site as a usable experience rather than as a static collection of sections.

- **0 — Not met:** Main navigation is broken or essential content cannot be reached; most representative tasks cannot be completed.
- **3 — Partially met:** Main sections are reachable, but one or more important tasks are confusing, incomplete, or fail under ordinary use; visitors need avoidable workarounds.
- **5 — Well met:** Main sections are easy to find and representative tasks can be completed through clear controls with understandable feedback.

3. Readability & Visual Consistency — 25%

Definition. Measures text legibility, hierarchy, spacing, responsive presentation, and consistency of the visual language across sections and interactive states. Evaluate whether visual styling supports comprehension and whether contrast and sizing remain usable.

Why this criterion matters. The site’ s visual identity is central to the portfolio, but decoration should not obstruct reading or interaction. This criterion balances visual coherence with practical legibility.

- **0 — Not met:** Text or controls are largely unreadable; layout is broken or inconsistent enough to prevent comprehension; major overflow or contrast problems dominate.
- **3 — Partially met:** The overall design is recognizable and much content is readable, but notable inconsistencies, small/low-contrast text, clutter, or responsive/layout issues hinder some content.
- **5 — Well met:** Clear type hierarchy and comfortable reading; coherent styling across sections; responsive layouts and sufficient contrast keep text and controls accessible in tested viewports.

4. Interaction Robustness — 20%

Definition. Measures whether interactive controls respond correctly and consistently, preserve state where promised, handle invalid/empty/unavailable conditions gracefully, and avoid uncaught errors or misleading success states.

Why this criterion matters. A visually convincing interface can still fail users if controls break, saved state disappears unexpectedly, or browser restrictions produce opaque errors. Robustness is especially relevant to a static site with media, local storage, and client-side retrieval.

- **0 — Not met:** Core interactions fail or cause unrecoverable errors; state is corrupted or user actions provide no meaningful response.
- **3 — Partially met:** Basic interactions work in common cases, but meaningful edge cases or environment constraints produce failures, unclear feedback, or unreliable state.
- **5 — Well met:** Core interactions work repeatedly in supported environments; state changes are clear and appropriately persistent; errors, empty states, and browser restrictions are handled with useful feedback.

Scoring procedure

1. Use the same task list, viewport sizes, browser settings, and evidence capture method for each version.
2. Separate observed behavior from source-code inference. Do not award a behavior as tested merely because code appears to implement it.

3. For each score, retain at least one evidence item and the exact task/prompt that elicited it.
4. Calculate the weighted score using the formula above. Preserve criterion-level scores; the aggregate must not replace qualitative evidence.
5. Apply the same protocol to all versions. Record browser, operating system, viewport, date, and whether the test used `file://` or HTTP.

---

title led relevance gate

Archive source: `prompt_appendix/task3_v2_evidence/title_led_relevance_gate.md`

Title-led relevance gate repair (2026-10-26)

Problem observed

The second offline translation model improved terminology consistency but did not produce portfolio-quality English for long lecture paragraphs. More importantly, translation did not address retrieval noise: a passage could mention a query term incidentally while its main subject was different. `Gradient descent` and `clustering` exposed this distinction.

General repair

The BM25 retrieval stage is unchanged. It still returns up to four ranked passages, and every retrieved passage remains visible in the folded **View Retrieved Reference Materials** panel for assignment evidence.

Answer eligibility is now decided by a shared title-led gate:

1. The knowledge-unit title must directly match most of the query subject tokens; a one-token coincidence is not enough for a longer subject.
2. A short acronym written in the title may match a full-form query when the acronym is derived automatically from that query.
3. A title naming a recognized component may support its parent concept, but body-only component mentions do not pass the gate.
4. A passage that only mentions the term inside its body remains retrieval evidence but cannot support the answer.
5. If no title-qualified passage yields a concise explanatory extract, the page returns exactly `Insufficient reference materials to answer this question.`

No query-specific answer text was added. All queries use the same retrieval, title gate, extract selection, citation, and abstention path.

Observed checks

- 梯度下降: four BM25 passages were preserved, but none had a title directly about gradient descent; the Answer correctly abstained.
- 反向传播: the titled unit `反向传播: Backward Propagation` passed and produced cited extracts from `[R2]`.
- `attention`: `Attention Patterns: 注意力模式` passed and produced cited extracts from `[R1]`.
- `Transformer原理`: the titled component `KV Cache 的原理` passed the existing parent/component rule and produced cited extracts from `[R1]`.
- An unrelated tiramisu query retrieved zero passages and correctly abstained.

The fixed 28-query generic regression suite passed all ten invariants. Browser QA over `http://127.0.0.1:5584/#highlights` confirmed both the abstention branch and the supported-answer branch. The reference disclosure remained collapsed and showed all four retrieved passages.

Trade-off

This conservative gate favors precision and traceability over recall. A useful paragraph whose title is broad may be excluded even when its body contains an answer. This is intentional for the static demonstration because it avoids presenting body-level keyword noise as a grounded answer. The source language is preserved instead of applying low-quality automatic translation.

---

ui simplification log

Archive source: `prompt_appendix/task3_v2_evidence/ui_simplification_log.md`

V2-RAG Front-End Simplification — 2026-10-24

User feedback and reason for change

The user asked that assembled prompts not appear on the website and that the visitor-facing page remain concise and accurate. The preceding review identified overlapping navigation, repeated empty-state text, oversized results, evaluator jargon in the public UI, and the misleading “Generated Answer” label. This is a presentation correction within V2-RAG; it is not a new course technique, a new baseline, or a change to V0/V1.

Source changes

- `website/index.html`: removed the visible prompt disclosure and verbose technical intro; replaced test-category labels with three natural example questions; renamed the answer panel

“Answer from sources” because the corpus includes course and project material; hid result panels until the first query; retained separate source and answer sections.

- `website/rag-v2.js`: continues to assemble the complete prompt and return it in `runQuery()` for the coursework trace, but no longer writes it into the DOM. The visible answer now quotes only the first retrieved note excerpt and cites only `[R1]`, avoiding implied support from unrelated lower-ranked results. Original source paragraphs remain available in expandable reference cards. The empty state is shorter.
- `website/style.css`: removed the prompt-panel styles and tall empty boxes; allowed the two result panels to size independently; reserved layout space for the sticky navigation and increased section anchor offset.
- `scripts/e2e_v2.cjs`: updated browser checks for the new UI. It tests that the prompt is absent from the DOM while prompt assembly remains available to the archive script.

Preserved evidence

The pre-change source files, E2E script, test JSON, query archive, screenshots, and assessment documents are preserved in `evaluation/iterations/2026-10-24-before-ui-simplification/`. The old screenshots in `evaluation/evidence/` are marked `_PRE_UI`; they must not be submitted as screenshots of the revised UI.

Verification and limits

- Passed: JavaScript syntax checks; six deterministic query runs; assembled prompts remain in `query_runs_v2.json`; frozen V0 file hashes match `v0_baseline_sha256.txt`; visitor HTML contains no prompt disclosure or “Generated Answer” label.
- Browser rerun is **pending**. This turn's shell sandbox rejected binding a local HTTP server (`PermissionError: Operation not permitted`) and Chrome headless exited during launch. The in-app browser also rejected a local `file:` URL under its URL policy. No revised screenshot or browser E2E result is claimed.
- Before using the revised V2 in the final report or independent AI Judge, rerun `scripts/e2e_v2.cjs` in a browser-enabled environment and refresh the screenshots. Keep the earlier evidence as iteration history.

Remaining limitations

The answer is deterministic extraction, not live LLM generation. Lexical retrieval can select a passage that is relevant to a keyword but only partially answers a broad question, for example the archived broad “memory” query. The updated UI labels the excerpt honestly and does not imply that every retrieved result supports the answer.

---

v0 v2rag analysis

Archive source: `prompt_appendix/task3_v2_evidence/v0_v2rag_analysis.md`

V0 vs V2-RAG Analysis

> **Evidence status, 2026-10-24:** This comparison and the preliminary V2 scores describe the first V2-RAG UI. Later changes removed prompt display and culminated in a generic semantic relevance and answer pipeline shared by every query. All BM25 hits remain visible, while only chunks accepted by the shared filter may support an answer. Earlier query-specific answer implementations are preserved under `iterations/` as failed attempts. Browser screenshots/E2E and a new human/AI assessment of the current V2 remain pending. Do not use historical screenshots or scores as proof of the revised version. See `generic_semantic_pipeline_refactor.md` and the dated repair logs.

Version lineage and technique

V2-RAG is a copy of frozen `coursework/website/V0/`; it was not created from V1. Hashes of V0 `index.html`, `style.css`, `rag.js`, and `knowledge-data.js` were captured before editing in `v0_baseline_sha256.txt` and are verified unchanged. Only the V2 course-note question module was redesigned, with its own structured JS corpus and browser-side RAG demo. The V0 version remains untouched.

The formal technique is Retrieval-Augmented Generation (RAG): retrieve passages from an external-to-the-query knowledge corpus, combine the original question with retrieved passages in a grounded prompt, then format a cited response. Given GitHub Pages' static-only hosting and the explicit no-key/no-external-LLM constraint, the final response step is deterministic extractive rendering rather than a real LLM call. This is an educational workflow simulation, not a production RAG service.

Rubric comparison

The fixed Task 2 rubric was used for both columns. V0's 76/100 is the existing preliminary human score, not an AI-Judge score. V2's score is a preliminary human assessment based on this iteration's evidence and unchanged V0 evidence for unaffected areas. The independent V2 AI-Judge output is pending and is not fabricated here.

Criterion	Weight	V0 preliminary	V2-RAG human	Change
Content Completeness & Clear Contribution	30%	4/5	4/5	0
Navigation & Task Completion	25%	4/5	4/5	0
Readability & Visual Consistency	25%	4/5	4/5	0
Interaction Robustness	20%	3/5	3/5	0
Weighted total	**100%**	**76/100**	**76/100**	**0**

The overall score is unchanged because this rubric evaluates the whole portfolio. The non-RAG modules were deliberately inherited from V0, and whole-site guestbook/audio/state edge cases have not been rerun for V2. This is not evidence that the RAG task itself did not improve.

Observable RAG changes and evidence

- **V0 behavior:** V0’s `rag.js` retrieves up to six BM25 chunks, then displays the first excerpt as the answer; its live V0 record notes weak leading relevance for a MapReduce query and a file-loading restriction that had required the embedded knowledge data.
- **First V2 behavior:** the visitor saw original retrieved passages and citations separately from the answer and could inspect the exact assembled prompt. The revised UI keeps separate source and excerpt sections, but the complete prompt is now retained only in code/archive, not shown on the page. `rag-v2.js` applies BM25 over 9,831 structured paragraphs from eight course notes and four existing V0 project profiles.
- **Historical runtime evidence:** six desktop browser tests passed before the UI revision. The result is preserved in `iterations/2026-10-24-before-ui-simplification/e2e_results_v2.json`. The revised UI's deterministic query archive is `query_runs_v2.json`; browser E2E remains pending.
- **Historical screenshot descriptions:** The pre-revision E1 and E2 screenshots are preserved under `iterations/2026-10-24-before-ui-simplification/` and as `_PRE_UI` files in `evidence/`. They do not show the revised interface.

Improvements, trade-offs, limitations

Improvement: Retrieval evidence, prompt construction, source attribution, and abstention behavior are observable and repeatable. Visitors can verify the original passage instead of receiving an unexplained excerpt. The structured corpus covers course notes and pre-existing project/paper-profile content.

Trade-offs: Keeping everything client-side makes the demo self-contained and avoids exposing API credentials, but downloads a multi-megabyte corpus and performs retrieval in the browser. BM25 is lexical; bilingual bigrams and a small phrase expansion help, but do not provide semantic understanding. The response quotes/extracts from the top passage instead of synthesizing multiple passages with a real model.

Static-site constraint: GitHub Pages has no backend proxy in this implementation. No real LLM API was called and no API key is embedded. The revised page labels its response “Answer from sources” and states that it quotes an excerpt rather than claiming live AI generation.

Remaining defects / failed attempts: An early Q02 ranking selected a `Tips:`-only fragment; a vague Q05 initially matched generic `class` text; and Q01 first ranked a “National Day holiday weekend” example above the RAG definition. The recorded fixes filter non-informative records at ranking time, remove generic query terms after normalization, and expand/weight the explicit RAG concept. See `retrieval_iteration_log.md`. Residual false positives can still occur for broad lexical queries, and this demo is not a substitute for semantic retrieval or model-generated answers.

Later relevance repair: The user independently found a false positive for `agent是什么`: the answer cited a GNN passage because interrogative bigrams outweighed the actual subject. The repair removes question phrasing, groups short source paragraphs, emphasizes body evidence, and asks for clarification on the ambiguous `memory` query. The before/after evidence and test limitations are in `relevance_fix_log.md`. The original V0 - V2 scores above have not been recalculated from the revised browser UI.

Earlier visitor UI revision: Answers used concise, source-checked English summaries for supported course-note topics with `[R1]` citations. The `Retrieved Reference Materials` panel became a native disclosure closed by default; displayed snippets were cleaned of Markdown/LaTeX debris

and KB record IDs. At that stage, an unsupported query could still trigger the no-material message despite showing a retrieved passage; the assembled prompt and raw records remained in the evaluation archive, not in the visible page. This was a static RAG workflow demonstration without live model generation.

Subsequent linkage repair: The prior answer catalog could return the no-material message even when retrieval displayed passages. Now the no-material sentence is exclusive to zero-hit retrieval. Nonempty results yield a cited English summary, using controlled topic summaries or cautious extraction from the retrieved evidence. The Chinese debug shortcut was removed, leaving exactly the three requested English shortcuts. See `linked_answer_fix_log.md`.

CNN significance repair: The user's exact question `cnn的开创性意义` originally yielded zero passages because the Chinese significance phrase became low-coverage bigrams; this was a retrieval failure despite relevant course notes. The updated retrieval maps that intent to source-attested CNN terms and retrieves LeNet-5, AlexNet, and the CNN definition. Its English answer explains local connectivity, weight sharing, digit recognition, and larger-scale image learning with matching citations. At that revision, the deterministic suite passed 21/21 cases, including two CNN paraphrases; browser E2E and the revised screenshot are still pending. See `cnn_significance_repair_log.md`.

Later multi-source answer revision: The prior generic answer could direct visitors to open a reference, and many curated answers summarized only `[R1]`. The revised deterministic composer inspects every retrieved passage, adds distinct supported claims from relevant sources, and avoids citing unrelated matches. For example, the RAG explanation uses `[R1][R3][R4]`, the diffusion explanation uses all four references, and the CNN significance answer uses `[R1][R2][R3]`. Two new uncovered-topic cases check overfitting and breadth-first search. At that revision, the suite passed 23/23 script assertions, including citation validity and banned-phrase checks; this did not constitute browser or Judge verification. See `multi_source_answer_log.md`.

Answer card cleanup revision: The revised visitor output has an opening sentence and 2 - 4 cited points; unrelated recalled passages can remain in the default-closed **View Retrieved Reference Materials** section without appearing in the answer. Weak lexical candidates that lacked enough answer-bearing evidence remained only in the internal candidate trace, so the final reference set was empty and the exact no-material response was used. The banned redirection/inability phrases were removed, and cleaned reference passages were no longer truncated mid-sentence. At that revision, the suite passed 24/24 deterministic assertions. Its strict answer-bearing gate was subsequently removed after the MCP regression described below. See `answer_card_cleanup_log.md`.

MCP repairs: The answer-quality gate first caused a regression: `What is MCP?` ranked four correct note sections but erased them because the mixed-language text did not pass a generic English-sentence threshold. Retrieval no longer calls the answer composer or clears valid hits. A later test showed that `MCP的作用` retrieved four passages but rendered a blank Answer card because its Chinese intent was not mapped to the MCP composer. MCP-specific ranking and bilingual intent handling now produce a source-checked summary covering the standardized interface, Host - Client - Server roles, and stdio/HTTP transports. The current suite passes 26/26 deterministic assertions; browser and independent Judge checks remain pending. See `mcp_answer_gate_repair_log.md`.

Current generic pipeline revision: The topic-by-topic answer catalog described above was identified as the underlying design defect and removed from the current source. Every query now uses the same intent profile, chunk relevance assessment, noise separation, English evidence extraction, deduplication, citation, and abstention rules. The 28 accumulated questions are an evaluation corpus rather than 28 hard-coded expectations; all pass eight shared pipeline invariants. See `generic_semantic_pipeline_refactor.md`. Browser and independent Judge checks remain pending.

Screenshot capture and Judge evidence

The previous desktop screenshots are marked `_PRE_UI` in `evaluation/evidence/` and copied to `evaluation/iterations/2026-10-24-before-ui-simplification/`. Capture the revised RAG panel after an explicit query and after a no-match query before final comparison; keep the desktop viewport and record browser/version, viewport, date, query, retrieved sources, and answer. Then use `ai_judge_prompt_v2.md` in a new blank Judge session with only the rubric and current website evidence. Save its complete response unchanged in `ai_judge_raw_output.md`.

---

v2 v0 lineage check

Archive source: `prompt_appendix/task3_v2_evidence/v2_v0_lineage_check.md`

V2 lineage check — is V2 built on V0?

Requirement. V2 must be built on the frozen V0 portfolio, not on V1 and not as a standalone site.

Checked: 2026-10-27 (Asia/Hong_Kong), against `coursework/website/V0/` and `task3/v2/website/`.

1. V0 was not modified

`v0_baseline_sha256.txt` was captured before V2 work. Recomputing the four hashes today returns the recorded values, so the baseline the V2 copy came from is intact:

File	Recorded = current
`index.html`	`653c9c6d…2f42e`
`style.css`	`028549f6…16d7`
`rag.js`	`375b5f2e…c803`
`knowledge-data.js`	`217ce13b…f42e`

2. What V2 changed relative to V0

File	Result	Detail
`app.js`	identical	byte-for-byte
`projects.js`	identical	byte-for-byte
`assets/`	identical	recursive compare, no differences
`knowledge/`	identical	recursive compare, no differences

File	Result	Detail
`index.html`	3 hunks	the course-note card, the footer version label (`V0 · 2026` → `Aria's Sky Journal · 2026`), and the two script tags that swap `knowledge-data.js/rag.js` for `rag-knowledge.js/rag-v2.js`
`style.css`	one appended block	45 added lines, every selector under the `/* V2-RAG coursework module */` block is `.rag-v2-*` (plus one `@media` containing only `.rag-v2-*`); no V0 rule is edited

The V2 page therefore contains every V0 section, control and asset unchanged, and adds one redesigned module.

3. Consequence recorded after the check

Two global overrides had been appended while building the card — `.nav-bar{margin-bottom:0}` and `.section{scroll-margin-top:120px}`. Both changed page-wide spacing relative to V0 (the first pushed all content 53 px down). They were removed on 2026-10-27 so that the only visible difference from V0 is the course-note module. Rechecked afterwards: the sticky nav ends at 66 px and the card starts at 329 px, so no overlap; page height dropped from 5,420 px to 5,367 px, which matches V0's spacing.

4. Verification after the change

- `node scripts/run_queries.cjs` → 28 queries, answers re-archived to `query_runs_v2.json`.
- `node scripts/relevance_regression.cjs` → `{"passed":true,"queries":28,"genericRules":10}`.
- `node scripts/e2e_v2.cjs` → `{"passed":true,"testCount":9,"errors":[],"externalRequests":[]}`, five evidence screenshots re-captured from the same run.

5. Scope note

V1 is a separate branch from the same frozen V0 (`coursework/website/V1/`), so its Home chapter rail, thumbnail carousel and project story index are not part of V2. Under the stated requirement this is correct: V2 is measured against V0 with one technique applied, and importing V1's layout would confound that comparison.

---

visitor answer cleanup log

Archive source: `prompt_appendix/task3_v2_evidence/visitor_answer_cleanup_log.md`

V2-RAG Visitor Answer Cleanup — 2026-10-24

Trigger and preserved state

The user requested a visitor-facing interface without internal prompts, agent/developer notes, raw KB identifiers, or pasted Markdown/LaTeX. References should be a native `details/summary` disclosure closed by default; the answer should be concise, fluent English with clear source citations.

The immediately preceding V2 website files, test scripts, and query output are preserved in `iterations/2026-10-24-before-visitor-answer-cleanup/`. The frozen V0 was not edited.

Code changes

File	Change	Reason
`website/index.html`	Answer appears before a native `details` panel called “Retrieved Reference Materials”; the panel is closed by default. Removed the visible implementation note and narrowed the question invitation to selected course-note ideas.	Give ordinary visitors a clean, accurate interface.
`website/rag-v2.js`	Reset the disclosure to closed for every query. Clean displayed snippets: remove Markdown images/links, presentation markers, excess table syntax, and LaTeX command debris; divide text into paragraphs and bullets; omit internal `KB-` IDs. Full raw source and prompt remain in the archived query run.	Keep references checkable without flooding the page or showing source-code artifacts.
`website/rag-v2.js`	Replace the direct “closest excerpt” answer with bounded, hand-checked English summaries for supported agent, RAG, GNN, MapReduce, tool-evaluation and memory topics. Each claim is tied to the top retrieved note and cited as `[R1]`; an unsupported or mismatched topic abstains.	A lexical hit is not automatically a valid answer.
`website/style.css`	Single-column answer-first layout; compact disclosure and scrollable opened references.	Keep the result area readable.
`scripts/run_queries.cjs`, `scripts/relevance_regression.cjs`, `scripts/e2e_v2.cjs`	Archive visitor-facing reference text/default disclosure state; assert English content, citations, closed disclosure, and absence of raw prompt/KB/LaTeX markers.	Make the user-facing requirements repeatably testable.

Verification and limits

The 14 deterministic query cases pass after this revision. The reported `agent是什么` query now yields an English agent definition with `[R1]`, while the GNN query yields a separate GNN answer. A broad `memory` query requests clarification; vague and unrelated questions return the exact insufficiency

message. The test archive contains the retrieved original passage, complete assembled prompt, page answer, cleaned visitor reference text, and default disclosure state for each query.

The English answers are curated deterministic summaries, **not** live LLM generation. They deliberately cover a limited set of course-note topics; related retrieval results outside that set can still lead to an insufficiency response. A new browser E2E run and screenshots of this exact visitor UI are pending; prior screenshots must not be represented as post-revision evidence. The unchanged Task 2 rubric and AI Judge prompt will be used only after new browser evidence is captured.

---

rubric

Archive source: `evaluation/rubric.md`

Evaluation Rubric — Aria’s Sky Journal

Scope. This rubric is version-agnostic and must be applied unchanged to V0, V1, and V2. Each criterion is scored from 0 to 5. Intermediate integer scores (1, 2, and 4) may be used where performance falls between the stated anchors. The weighted total is calculated as `Σ (criterion score ÷ 5 × criterion weight)` and reported on a 0 - 100 scale.

Criterion	Weight
Content Completeness & Clear Contribution	30%
Navigation & Task Completion	25%
Readability & Visual Consistency	25%
Interaction Robustness	20%
Total	**100%**

1. Content Completeness & Clear Contribution — 30%

Definition. Measures whether the portfolio presents the expected personal, educational, skill, project, and contact content; whether project entries explain the owner’s role or contribution; and whether claims are specific enough for visitors to understand the work. Assess presence, internal consistency, and clarity, not whether the evaluator personally agrees with the claims.

Why this criterion matters. A portfolio is useful only when visitors can understand who the owner is, what they have done, and what their contribution was. Missing or ambiguous content weakens the site’s purpose even when its appearance is polished.

- **0 — Not met:** Most expected sections are absent or empty; project work is missing or gives no interpretable account of the owner’s role; major content is contradictory or unusable.
- **3 — Partially met:** Several expected sections and projects are present, but one or more important areas are missing, repetitive, vague, or do not clearly distinguish the owner’s

contribution.

- **5 — Well met:** Relevant profile, education, skills, projects, and contact/personal content are present and coherent; project entries clearly state the owner's contribution; no material content gaps prevent visitors from understanding the portfolio.

2. Navigation & Task Completion — 25%

Definition. Measures whether visitors can locate the main sections and complete representative tasks, including opening project details, filtering or saving projects, exploring the life gallery, using the course-note interface, and submitting a guestbook entry.

Why this criterion matters. Portfolio content must be findable and its promised actions must be discoverable. Navigation and task completion evaluate the site as a usable experience rather than as a static collection of sections.

- **0 — Not met:** Main navigation is broken or essential content cannot be reached; most representative tasks cannot be completed.
- **3 — Partially met:** Main sections are reachable, but one or more important tasks are confusing, incomplete, or fail under ordinary use; visitors need avoidable workarounds.
- **5 — Well met:** Main sections are easy to find and representative tasks can be completed through clear controls with understandable feedback.

3. Readability & Visual Consistency — 25%

Definition. Measures text legibility, hierarchy, spacing, responsive presentation, and consistency of the visual language across sections and interactive states. Evaluate whether visual styling supports comprehension and whether contrast and sizing remain usable.

Why this criterion matters. The site's visual identity is central to the portfolio, but decoration should not obstruct reading or interaction. This criterion balances visual coherence with practical legibility.

- **0 — Not met:** Text or controls are largely unreadable; layout is broken or inconsistent enough to prevent comprehension; major overflow or contrast problems dominate.
- **3 — Partially met:** The overall design is recognizable and much content is readable, but notable inconsistencies, small/low-contrast text, clutter, or responsive/layout issues hinder some content.
- **5 — Well met:** Clear type hierarchy and comfortable reading; coherent styling across sections; responsive layouts and sufficient contrast keep text and controls accessible in tested viewports.

4. Interaction Robustness — 20%

Definition. Measures whether interactive controls respond correctly and consistently, preserve state where promised, handle invalid/empty/unavailable conditions gracefully, and avoid uncaught errors or misleading success states.

Why this criterion matters. A visually convincing interface can still fail users if controls break, saved state disappears unexpectedly, or browser restrictions produce opaque errors. Robustness is especially relevant to a static site with media, local storage, and client-side retrieval.

- **0 — Not met:** Core interactions fail or cause unrecoverable errors; state is corrupted or user actions provide no meaningful response.
- **3 — Partially met:** Basic interactions work in common cases, but meaningful edge cases or environment constraints produce failures, unclear feedback, or unreliable state.
- **5 — Well met:** Core interactions work repeatedly in supported environments; state changes are clear and appropriately persistent; errors, empty states, and browser restrictions are handled with useful feedback.

Scoring procedure

1. Use the same task list, viewport sizes, browser settings, and evidence capture method for each version.
2. Separate observed behavior from source-code inference. Do not award a behavior as tested merely because code appears to implement it.
3. For each score, retain at least one evidence item and the exact task/prompt that elicited it.
4. Calculate the weighted score using the formula above. Preserve criterion-level scores; the aggregate must not replace qualitative evidence.
5. Apply the same protocol to all versions. Record browser, operating system, viewport, date, and whether the test used `file://` or HTTP.

---

ai judge prompt

Archive source: `evaluation/ai_judge_prompt.md`

AI Judge Prompt — Apply in a New, Clean Session

> You are an independent evaluator assessing a personal portfolio website against the rubric and evidence supplied below. Evaluate only the supplied materials. Do not assume that source code works merely because an implementation is present. Distinguish observed test results from source-code claims and note when evidence is insufficient.
>
> **Do not infer or invent missing behavior.** If a task was not tested, state that limitation. Do not use outside information about the owner. Do not use or request any prior human assessment. Produce an independent assessment from this prompt and its attached evidence only.
>
> ## Rubric
>
> Score each criterion from 0 to 5. Scores 0, 3, and 5 correspond to the anchors below; integer scores 1, 2, or 4 may be used for performance between anchors. Calculate the weighted total as `Σ (criterion score ÷ 5 × criterion weight)` on a 0 - 100 scale.

1. Content Completeness & Clear Contribution — 30%

Measures whether relevant profile, education, skills, projects, and contact/personal content are present and coherent, and whether project entries clearly explain the owner's contribution.

- **0:** Most expected sections are absent/empty; work or contribution is uninterpretable; major contradictions or unusable content.
- **3:** Several sections/projects are present, but important content is missing, repetitive, vague, or does not distinguish the owner's contribution.
- **5:** Relevant sections are complete and coherent; project contributions are clear; no material gap prevents understanding the portfolio.

2. Navigation & Task Completion — 25%

Measures whether visitors can find the main sections and complete representative tasks such as opening project details, filtering/saving projects, exploring media, using the course-note interface, and submitting a guestbook entry.

- **0:** Main navigation is broken or essential content/tasks are inaccessible; most tasks fail.
- **3:** Main sections are reachable, but an important task is confusing, incomplete, or fails in ordinary use.
- **5:** Main sections are easy to find and representative tasks complete through clear controls with understandable feedback.

3. Readability & Visual Consistency — 25%

Measures text legibility, hierarchy, spacing, responsive presentation, and visual consistency across sections and interaction states.

- **0:** Text/controls are largely unreadable or layout is broken/inconsistent enough to prevent comprehension.
- **3:** Overall style is recognizable and much content is readable, but notable inconsistency, small/low-contrast text, clutter, or layout issues hinder use.
- **5:** Clear hierarchy and comfortable reading; coherent styling; tested responsive layouts and sufficient contrast keep content and controls usable.

4. Interaction Robustness — 20%

Measures whether controls respond correctly, state persists where promised, empty/unavailable/error cases are handled clearly, and no misleading success state is presented.

- **0:** Core interactions fail or cause unrecoverable errors; state is corrupted or actions receive no meaningful response.
- **3:** Basic interactions work in common cases, but meaningful edge cases or environment constraints cause failures, unclear feedback, or unreliable state.
- **5:** Core interactions work repeatedly in supported environments; state is clear and appropriately persistent; errors and restrictions receive useful feedback.

Required output

Return valid JSON first, with this schema:

```
{
  "version": "V0",
  "criterion_scores": [
    {"criterion": "Content Completeness & Clear Contribution", "weight_percent": 30, "score_0_to_5": 0,
        "evidence_ids": [], "rationale": "", "confidence": "low|medium|high"},
    {"criterion": "Navigation & Task Completion", "weight_percent": 25, "score_0_to_5": 0, "evidence_ids": [],
        "rationale": "", "confidence": "low|medium|high"},
    {"criterion": "Readability & Visual Consistency", "weight_percent": 25, "score_0_to_5": 0, "evidence_ids": [],
        "rationale": "", "confidence": "low|medium|high"},
    {"criterion": "Interaction Robustness", "weight_percent": 20, "score_0_to_5": 0, "evidence_ids": [], "rationale":
        "", "confidence": "low|medium|high"}
  ],
  "weighted_total_0_to_100": 0,
  "limitations": []
}
```

After the JSON, provide a short evidence-based explanation. Cite evidence IDs for every claim. Identify unsupported tasks explicitly. Do not modify, round away, or omit any criterion score. Do not include any human score or human opinion.

Evidence supplied

Attach the evidence package listed in `evaluation/ai_raw_output.md` under “Evidence supplied to the clean Judge session”. The evaluator should receive rubric, screenshots, relevant source excerpts, and dated test logs only. Do not include human score sheets or human commentary.

---

ai raw output v0

Archive source: `evaluation/ai_raw_output_v0.md`

AI Judge — Evidence Manifest and Raw Output Slot

Clean-session protocol

Open a new, blank AI session. Submit only the full text of `ai_judge_prompt.md` and the evidence listed below. Do not include `human_self_assessment.md`, human scores, the comparison document, or the weakness analysis. Save the complete Judge response verbatim in the raw-output block below; do not edit its wording or JSON. Record the model/service and date separately as metadata.

Evidence supplied to the clean Judge session

Current evidence inventory:

ID	Evidence	Current state	Judge use
E1	Frozen V0 `index.html` excerpt: section IDs, navigation, RAG panel, media, guestbook, footer	Available at `../website/V0/index.html`; excerpts must be copied into a session evidence bundle	Structure/content claims only
E2	Frozen V0 `projects.js`	Available at `../website/V0/projects.js`	Project titles, purposes, stated contributions
E3	Frozen V0 `app.js` excerpts: project filters/details/saves, gallery/media, audio, guestbook	Available at `../website/V0/app.js`; excerpts must be copied into a session evidence bundle	Implementation claims only; not proof of runtime success
E4	Frozen V0 `rag.js` and `knowledge-data.js` behavior	Available at `../website/V0/rag.js` and `../website/V0/knowledge-data.js`	Retrieval method and source behavior
E5	Screenshot of the RAG section supplied in the conversation	User-supplied screenshot is visible in conversation history; a standalone evidence copy is not currently in this folder	Single-view visual evidence; screenshot predates/does not prove current repaired behavior
E6	Dated desktop end-to-end task log for navigation, filters, persistence, RAG, guestbook, media/audio	**Available**	See `evidence/V0/2026-10-23_live_e2e_test.md`; valid guestbook submission and RAG no-match remain untested
E7	Current full-page desktop screenshots	**Not available**	Must be captured before broad visual/usability claims

Before running Judge, create a dated evidence bundle under `evaluation/evidence/V0/` and record exactly which evidence IDs were sent. Exclude this human-assessment document and all human score values.

Raw AI Judge output (paste verbatim below)

Status: Completed — independent clean-session V0 Judge evaluation.


```
{
  "version": "V0",
  "criterion_scores": [
    {
      "criterion": "Content Completeness & Clear Contribution",
      "weight_percent": 30,
      "score_0_to_5": 5,
      "evidence_ids": ["E1", "E2"],
      "rationale": "The portfolio includes coherent home, academic highlights, skills, course-note/RAG, projects,
life/media, guestbook, and contact-oriented content. All four project entries state a purpose and explicitly identify the
owner's contribution.",
      "confidence": "high"
    },
    {
      "criterion": "Navigation & Task Completion",
      "weight_percent": 25,
      "score_0_to_5": 4,
      "evidence_ids": ["E1", "E3", "E6"],
      "rationale": "Navigation, carousel, education/project details, project filters, saved-project persistence, media
previews, audio controls, and empty guestbook validation passed the dated desktop tests. The MapReduce RAG query produced a
poor leading result, and valid guestbook submission was not tested.",
      "confidence": "high"
    },
    {
      "criterion": "Readability & Visual Consistency",
```

```
      "weight_percent": 25,
      "score_0_to_5": 3,
      "evidence_ids": ["E1", "E5", "E6", "E7"],
      "rationale": "The supplied homepage screenshot and source show a recognizable, coherent visual system with structured
sections and readable content. However, only homepage visual evidence is available; current full-page, responsive, contrast,
and interaction-state presentation were not comprehensively tested, so a top score is unsupported.",
      "confidence": "medium"
    },
    {
      "criterion": "Interaction Robustness",
      "weight_percent": 20,
      "score_0_to_5": 4,
      "evidence_ids": ["E3", "E4", "E6"],
      "rationale": "The dated E2E log confirms working navigation, filtering, local save persistence across reload, media
dialogs, audio play/pause, empty-form validation, and no final console errors. Robustness is reduced by the observed RAG
relevance failure for MapReduce, browser autoplay restrictions, and untested valid guestbook submission, length limits, and
RAG no-match behavior.",
      "confidence": "high"
    }
  ],
  "weighted_total_0_to_100": 81,
  "limitations": [
    "No current full-page screenshot evidence was supplied.",
    "Responsive layouts and accessibility/contrast were not fully audited.",
    "A valid guestbook submission and message-length handling were not tested.",
    "A RAG no-match query was not tested.",
    "The MapReduce query showed a retrieval-ranking/relevance problem.",
    "Audio autoplay was blocked by browser policy until user interaction."
  ]
}

Evidence-based explanation: E1 and E2 support complete portfolio sections and explicit project contributions. E6 confirms
that the main navigation and most representative interactions passed on desktop, while also recording the weak MapReduce
retrieval result and the untested valid guestbook path. E5/E6 show only limited visual evidence, and E7 is unavailable, so
visual scoring is conservative. E3/E4 demonstrate implemented interaction and retrieval behavior, but source code alone
cannot replace runtime evidence.
```

Run metadata (not part of the Judge output)

- Model/service: independent clean-session Judge agent
- Session created as a new blank conversation: Yes
- Date and time (timezone): 2026-10-28, Asia/Hong_Kong
- Evidence IDs supplied: E1, E2, E3, E4, E5, E6, E7 from the V0 manifest
- Operator: Codex
- Raw output filename/hash: see `judge_output_hashes.sha256`

---

ai raw output v1

Archive source: `evaluation/ai_raw_output_v1.md`

V1 AI Judge Raw Output

Clean-session metadata

- Model/service: independent clean-session Judge agent
- Session created as a new blank conversation: Yes
- Date and time (timezone): 2026-10-28, Asia/Hong_Kong

- Evidence supplied: fixed rubric/prompt, V1 Iteration 12 source, V1-R12 E2E output, V1-R12 content-parity output, V1 screenshots and deployment evidence
- Human scores supplied to Judge: No

Raw response (verbatim)

{ "version": "V1", "criterion_scores": [{ "criterion": "Content Completeness & Clear Contribution", "weight_percent": 30, "score_0_to_5": 5, "evidence_ids": ["V1-R12-content-parity", "E2E-04a", "E2E-04e", "E2E-04b", "E2E-04c", "E2E-04d"], "rationale": "The supplied source contains coherent Home, education, skills, course notes, four project stories, Life/media, and Guestbook sections. Project entries explicitly state purpose and the owner's contribution; the R12 parity record reports 48 compared copy groups with zero differences and zero structure errors.", "confidence": "high" }, { "criterion": "Navigation & Task Completion", "weight_percent": 25, "score_0_to_5": 4, "evidence_ids": ["E2E-01", "E2E-02", "E2E-02b", "E2E-03", "E2E-04c", "E2E-05a", "E2E-05b", "E2E-05c", "E2E-06a", "E2E-06b", "E2E-08", "E2E-09", "E2E-10a", "E2E-11"], "rationale": "Desktop navigation, carousel, education/project details, project index, filters, saved projects, media, video, music, and empty guestbook handling all passed the R12 E2E record. Score is below 5 because a valid guestbook submission—the rubric's representative task—was explicitly not tested.", "confidence": "high" }, { "criterion": "Readability & Visual Consistency", "weight_percent": 25, "score_0_to_5": 4, "evidence_ids": ["V1-R12-E2E-01_home_full_desktop", "V1-R12-E2E-12_highlights_full_desktop", "V1-R12-E2E-13_projects_full_desktop", "V1-R12-E2E-14_life_full_desktop", "V1-R12-E2E-15_guestbook_full_desktop", "V1-R12-E2E-16_education_details_open_desktop", "V1-R12-skills-open_414", "V1-R12-skills-open_820", "V1-R12-skills-open_1024"], "rationale": "The supplied desktop and breakpoint screenshots show a coherent journal-like visual system, readable hierarchy, spacing, and consistent controls across major sections and open states. Score is below 5 because the evidence does not include a formal contrast audit or comprehensive mobile-layout verification.", "confidence": "medium" }, { "criterion": "Interaction Robustness", "weight_percent": 20, "score_0_to_5": 4, "evidence_ids": ["V1-R12-e2e-results", "E2E-06b", "E2E-07a", "E2E-07b", "E2E-07c", "E2E-08e", "E2E-08f", "E2E-09", "E2E-10a", "E2E-11", "E2E-12"], "rationale": "The dated R12 record reports 35/35 desktop checks passed with no page or console errors, including saved-state reload persistence, relevant and no-evidence RAG responses, keyboard media preview/Escape, video controls, music play/pause, and empty-input handling. Score is below 5 because valid guestbook submission, length-limit behavior, and broader responsive/error cases remain untested.", "confidence": "high" }], "weighted_total_0_to_100": 85, "limitations": ["Only the supplied evidence was used.", "A valid guestbook message was not submitted; only empty submission behavior was tested.", "Guestbook length-limit behavior was not tested.", "No formal contrast audit was supplied.", "Comprehensive mobile layout verification was not supplied.", "The R12 E2E evidence is desktop-focused; the listed breakpoint skill screenshots do not establish full-site responsive correctness."] }

Evidence-based explanation: The latest R12 record reports 35/35 local desktop checks passed, zero console errors, successful deployment, and content parity with 48 copy groups and zero differences/structure errors. The screenshots show a coherent, readable journal-style layout across Home, Highlights, Projects, Life, Guestbook, and open states. The principal evidence gaps are the untested valid guestbook flow, untested length limits, lack of a formal contrast audit, and incomplete mobile verification; therefore the score is 85 rather than full marks.

---

ai raw output v2 rerun

Archive source: `evaluation/ai_raw_output_v2_rerun.md`

V2-RAG AI Judge Raw Output — Rerun 2026-10-28

Clean-session metadata

- Model/service: independent clean-session Judge agent
- Human scores supplied: No
- Additional evidence: `e2e_results_v2.json` and `nonrag_public_e2e.json`

Raw response (verbatim)

{ "version": "V2-RAG", "criterion_scores": [{ "criterion": "Content Completeness & Clear Contribution", "weight_percent": 30, "score_0_to_5": 5, "evidence_ids": ["E5", "NR1", "NR-projects", "NR-detail"], "rationale": "The V2 portfolio contains profile, education, academic highlights, skills, four projects, life/personal content, media, and guestbook sections. Project records include explicit contribution statements, and the public E2E confirms four projects are present and one expands successfully.", "confidence": "high" }, { "criterion": "Navigation & Task Completion", "weight_percent": 25, "score_0_to_5": 5, "evidence_ids": ["E3", "E4", "NR-highlights", "NR-projects", "NR-life", "NR-guestbook", "NR-edu", "NR-detail", "NR-filter", "NR-save", "NR-media"], "rationale": "The latest public evidence reports all nine RAG cases passing, including retrieval, citations, abstention, Chinese agent retrieval, CNN retrieval, and MCP retrieval. The non-RAG public E2E reports successful navigation to all main sections, education disclosure, project expansion, filtering, saving, life media, photo preview, and guestbook form availability.", "confidence": "high" }, { "criterion": "Readability & Visual Consistency", "weight_percent": 25, "score_0_to_5": 4, "evidence_ids": ["E1", "E2", "E3", "E4", "E5"], "rationale": "The supplied V2 screenshots show clear hierarchy, readable controls and answer cards, consistent visual styling, understandable citation presentation, and closed-by-default reference panels. Evidence is limited to a 1512x1000 desktop viewport; responsive behavior and accessibility across other viewports are not independently demonstrated.", "confidence": "medium" }, { "criterion": "Interaction Robustness", "weight_percent": 20, "score_0_to_5": 4, "evidence_ids": ["E3", "E4", "E5", "NR1", "NR-detail", "NR-filter", "NR-save", "NR-media", "NR-guestbook"], "rationale": "The public RAG E2E reports 9/9 cases passing with no console errors, including relevant retrieval, exact abstention, citations, and multiple language/query types. The non-RAG E2E reports all listed interaction checks passing. However, the non-RAG run records one 404 resource error, and the supplied checks do not independently establish persistence across reloads, repeated stress behavior, or responsive interaction behavior.", "confidence": "high" }], "weighted_total_0_to_100": 91, "limitations": ["The latest evidence uses Google Chrome headless at a 1512x1000 desktop viewport; other viewport sizes, mobile layout, and assistive-technology behavior are not tested.", "The non-RAG public E2E records one 404 resource error despite all listed functional checks passing; its source and user impact are not identified.", "The E2E evidence verifies the listed representative tasks but does not establish arbitrary-query semantic correctness beyond the tested cases and generic regression invariants.", "Persistence across a full reload or a new browser session is not directly demonstrated by the supplied E2E records.", "Source code supports implementation claims but is not, by itself, independent runtime proof."] }

The latest deployed RAG evidence is materially stronger than the earlier pending record: `e2e_results_v2.json` is dated 2026-10-28, uses the public GitHub Pages URL, reports 9/9 cases, and has no console errors. The non-RAG public E2E separately reports successful navigation, education disclosure, project details, filtering, saving, life media/photo preview, and guestbook-form

checks. The desktop screenshots support readability and citation/reference presentation, while the recorded 404 and lack of mobile/reload testing justify retaining 4/5 for robustness and readability rather than awarding the maximum.

2026 10 23 live e2e test

Archive source: `evaluation/evidence/V0/2026-10-23_live_e2e_test.md`

V0 Live End-to-End Test Log

Date: 2026-10-23 (Asia/Hong_Kong)
Target: https://liuxinyue84-sketch.github.io/
Repository: liuxinyue84-sketch/liuxinyue84-sketch.github.io
Deployed commit: `fffbb35f956ed8d73f36853c0fd2a42a19ea9378`
Version integrity: the 32 files in `coursework/website/V0/` and `outputs/Aria_Sky_Journal_V0/` have matching SHA-256 manifests; the published commit was prepared from that frozen V0 directory. No V0 source was edited for this test.

Environment and initial state

- Browser: Google Chrome on macOS.
- Desktop viewport: 1512 px wide (browser screenshot viewport).
- Route: public GitHub Pages HTTPS root.
- Initial project favorites: `Saved 0`.
- Initial guestbook: empty ("No notes yet").
- Test state cleanup: favorite toggled back off and verified after reload (`Saved 0`); no guestbook message submitted; browser viewport reset and returned to the homepage.

Test results

ID	Action	Observed result	Status
E2E-01	Open the deployed root URL	Page title `Aria's Sky Journal` loaded; the hand-drawn sky background and hero image rendered. GitHub Pages returned HTTP 200. The checked homepage images had non-zero natural dimensions.	Pass
E2E-02	Click Home, Highlights, Projects, Life, and Guestbook navigation links	Each link updated the URL hash to its matching section.	Pass
E2E-03	Advance the hero carousel and select a scene	Scene label, selected indicator, and slide counter updated among the three scenes.	Pass

ID	Action	Observed result	Status
E2E-04	Expand education and a project detail	Education disclosures revealed course/award information; the project detail control expanded the selected project's supporting content.	Pass
E2E-05	Select Research, System Builds, and All project filters	Research showed 2 stories, System Builds showed 2 stories, and All showed 4 stories.	Pass
E2E-06	Favorite a project, open Saved, reload, then undo the test favorite	Saved changed from 0 to 1; the Saved view showed the selected research project; the favorite remained after reload. It was then removed and a second reload confirmed `Saved 0`.	Pass
E2E-07a	Ask "What is retrieval-augmented generation?" in the RAG panel	The site searched its packaged lecture notes and returned a relevant RAG workflow explanation with a course/section source. The interface reported 1,359 passages from 8 notes when indexing the corpus.	Pass
E2E-07b	Ask "How does MapReduce work?"	The leading excerpt was about neutral framing/sycophancy, which was not a good answer to the question. A secondary source did mention MapReduce limitations. This exposes a retrieval-ranking/relevance weakness.	Partial / issue found
E2E-08	Open and close a photo preview and a video preview	Photo preview opened in a dialog and closed. The video preview loaded the hosted MP4 (`readyState=1`) with controls and began playback when opened; it was closed after checking.	Pass
E2E-09	Observe initial music state, then click play and pause	Chrome reported that autoplay was blocked on page load. A direct click started the Schumann audio (`paused=false`, `readyState=4`); clicking again paused it.	Pass with browser-policy limitation
E2E-10	Click Sign the guestbook with both fields empty	No note was added; the empty state remained. A valid message was not submitted to avoid leaving a test entry in the browser's local storage.	Pass for empty validation; valid submission not tested
E2E-11	Inspect browser console	No JavaScript console errors were present at the final check.	Pass

Summary and limits

The published baseline is reachable, its main navigation and representative portfolio interactions work, and its favorite state persists locally as designed. The main observed product limitation is RAG retrieval relevance for the MapReduce query. Browser autoplay restrictions require a visitor gesture before background audio can play. Valid guestbook submission, length-limit handling, a RAG no-match query, and a full accessibility/contrast audit were not tested in this run.

Screenshot evidence

- `E2E-01_desktop-home_2026-10-23.png` — desktop capture of the published homepage, showing the Home hero and carousel scene 03/03. PNG, 3024 × 1718 pixels; SHA-256 `a9ce31bf24878d9b3e5a6afaccb06b688164ca121ff3456577ca881df0c76e33`.

This image documents the homepage appearance only; it is not visual evidence for every interaction in the test table. The deployed frozen V0 source was not changed during the test.

V1 E2E test log

Archive source: `evaluation/evidence/V1/V1_E2E_test_log.md`

V1 Desktop End-to-End Test Log

Version: V1, derived from frozen V0 using the recorded Few-Shot Learning prompt.
Date: 2026-10-23 20:20 Asia/Hong_Kong (2026-10-23 12:20 UTC).
Browser: Google Chrome headless via Playwright; same runner for V0 and V1.
OS: macOS. **Viewport:** 1512 × 982 CSS px.
Route: local HTTP, `http://127.0.0.1:5584/`; final V1 is also published at `https://liuxinyue84-sketch.github.io/V1/`.
Starting state: isolated fresh browser context; no saved favorites and no guestbook entries.

All 25 assertions passed. Exact test inputs and observed outputs are preserved in `V1-e2e-results.json`.

Test	Input/action	Result
E2E-01	Open site	HTTP 200; title and homepage load.
E2E-02	Navigate Home, Highlights, Projects, Life, Guestbook	All five anchors match their sections.
E2E-03	Advance hero carousel	Counter and scene label advance to slide 2/3.
E2E-04	Expand education and project details	Both disclosures expose their original details.
E2E-05	Select Research, System Builds, All	Counts are 2, 2, and 4.
E2E-06	Save a project, open Saved, reload, remove test favorite	Saved count persists across reload and returns to zero.
E2E-07a	Ask “How does MapReduce work?”	Leading excerpt is from “MapReduce steps” in Big Data Computing · Part 1; this corrects the irrelevant V0 leading excerpt.
E2E-07b	Ask “How do I bake a sourdough loaf?”	Returns a no-evidence message rather than a course-note passage.
E2E-07c	Ask the paraphrased unsupported query “How do I bake sourdough bread?”	Returns the no-evidence message after the V1 overlap threshold was tightened; this added case failed on unchanged V0.
E2E-08	Open and close a photo	Dialog opens and closes.
E2E-09	Open and close the first video	MP4 metadata loads and controls are present.

Test	Input/action	Result
E2E-10a	Click music control to play, then pause	Both user-controlled states work in the isolated browser.
E2E-11	Submit the guestbook form empty	No message is created; the empty state remains. A valid message was not submitted.
E2E-12	Inspect page and console errors	No uncaught application errors. Chrome's implicit `/favicon.ico` 404 is recorded separately.

Screenshots

- `V1-E2E-01_home_desktop.png` — desktop Home view.
- `V1-E2E-12_highlights_desktop.png` — Highlights viewport.
- `V1-E2E-12_highlights_full_desktop.png` — full Highlights section and tested MapReduce/no-evidence conversation.

Limits

The suite was also run against the deployed HTTPS URL. The first live run recorded one carousel assertion failure because the runner waited a fixed 500 ms; it passed on the same page after the runner was changed to wait for the actual slide counter. The untouched first-run JSON/screenshots are retained with the `V1-PAGES-initial-` prefix. The final public run passed 25/25; its raw JSON and three screenshots use the `V1-PAGES-RETEST-` prefix. A deployed V1 tab was also inspected in Chrome. A valid guestbook message, length-limit behavior, contrast audit, and mobile layout are not tested. The script's favorite is stored only in its isolated browser context and cleared before exit.

Published-site run

Date: 2026-10-23 20:20 Asia/Hong_Kong (2026-10-23 12:20 UTC). **Route:** `https://liuxinyue84-sketch.github.io/V1/`. **Result:** 25/25 assertions passed with the common runner. Raw evidence: `V1-PAGES-RETEST-e2e-results.json`; screenshots: `V1-PAGES-RETEST-E2E-01_home_desktop.png`, `V1-PAGES-RETEST-E2E-12_highlights_desktop.png`, and `V1-PAGES-RETEST-E2E-12_highlights_full_desktop.png`. The initial timing-sensitive 24/25 result is retained separately with its screenshots; no application errors occurred.

---

test prompts and feedback index

Archive source: `evaluation/test_prompts_and_feedback_index.md`

Task 2 Test Prompts and Feedback Index

This index points to the actual saved test operations, query prompts, outputs, and feedback. It does not reconstruct missing historical interactions.

V0 baseline

- **Dated public browser operations and observations:** `evidence/V0/2026-10-23_live_e2e_test.md`.
- **Captured browser evidence:** `evidence/V0/E2E-01_desktop-home_2026-10-23.png`.
- **Specific RAG test prompts and observed outcomes:** the dated log records “What is retrieval-augmented generation?” as relevant, and “How does MapReduce work?” as having a weak/irrelevant leading excerpt. It also records the empty Guestbook submission, music autoplay restriction, and untested valid Guestbook submission.
- **AI Judge evidence used for V0:** listed in `ai_raw_output_v0.md`. The raw Judge response is preserved; provenance discrepancies are disclosed in `record_corrections.md`.

V1 Few-Shot version

- **Earlier version-level test prompts and feedback:** `evidence/V1/V1_E2E_test_log.md`.
- **Latest Iteration 12 E2E checks:** `evidence/V1/V1-R12-e2e-results.json`.
- **Content-preservation comparison:** `evidence/V1/V1-R12-content-parity.json`.
- **Screenshots submitted with the R12 evidence:** files named `V1-R12-*.png` in `evidence/V1/`.
- **Few-Shot prompt iterations and examples:** `../prompt_appendix/task3_v1/icl_prompt*.md` and `few_shot_examples*.md`; they are included in `../prompt_appendix.md`.

V2 RAG version

- **RAG query set and expected behavior:** `../prompt_appendix/task3_v2_evidence/query_set.md`.
- **Exact per-query retrieval candidates, assembled prompt, answer and displayed references:** `../prompt_appendix/task3_v2_evidence/query_runs_v2.json` and `title_gate_query_results.json`.
- **Latest deployed RAG E2E result:** `evidence/V2/e2e_results_v2.json` (9/9 recorded cases passed).
- **Public non-RAG browser checks:** `evidence/V2/nonrag_public_e2e.json`.
- **Current screenshots:** PNG files in `evidence/V2/`; older pre-UI screenshots are retained separately under `prompt_appendix/task3_v2_evidence/iterations/` and labeled historical.
- **AI Judge output based on the latest evidence:** `ai_raw_output_v2_rerun.md`.

Feedback and repair history

V1 design feedback, Few-Shot prompt revisions, and implementation changes are in `../prompt_appendix/task3_v1/`. V2 user-reported failures and shared-pipeline repairs are recorded as dated/named Markdown logs under `../prompt_appendix/task3_v2_evidence/`; those records are compiled into `../prompt_appendix.md`. Exact test outputs remain in the JSON files listed above.

---

record corrections

Archive source: `evaluation/record_corrections.md`

Evaluation Record Corrections and Provenance Notes

These notes clarify inconsistencies found while assembling the submission. Raw AI Judge outputs are preserved verbatim and are not silently rewritten.

V0 evidence files

The dated log `evidence/V0/2026-10-23_live_e2e_test.md` and its homepage screenshot were recovered from the retained project evaluation archive and copied into this submission. The log is an actual contemporaneous record; it was not recreated. It describes the 2026-10-23 public V0 test and explicitly records the weak MapReduce result and untested valid Guestbook submission.

The V0 raw Judge file's evidence manifest states that E7 full-page screenshots were unavailable, but the raw Judge JSON lists E7 among the evidence IDs for readability. The historical Judge output itself cannot establish that those screenshots were supplied. The current package does not retroactively claim they were. The raw output is retained unchanged for auditability.

Score and output references

`comparison_and_limitations.md` now uses the official Task 2 human totals (V0 76, V1 85, V2 86) and the corresponding AI Judge totals (V0 81, V1 85, latest V2 rerun 91). The earlier V2 score is not used in the current table. The supplementary five-dimension totals (V0 79, V1 90, V2 91) remain separately labeled in `supplementary_human_assessment.md`.

The latest V2 AI raw output is `ai_raw_output_v2_rerun.md`. Previous comparison text that pointed to a missing generic `ai_raw_output.md` or an old V2 path has been corrected.

Missing historical data

V0 and V1 supplementary five-dimension example-level notes were not separately recorded. The blank templates were removed; no historical tests or feedback have been invented. The official four-criterion human assessment files contain their own evidence narratives.

Companion artifacts

- Complete static websites: `website/V0/`, `website/V1/`, `website/V2/`.
- V1 full generated source snapshots: `prompt_appendix/task3_v1/llm_output/`.
- Exact V2 retrieval runs and regression JSON: `prompt_appendix/task3_v2_evidence/`.
- Consolidated Task 2/3 evidence by version: `evaluation/evidence/V0/`, `evaluation/evidence/V1/`, and `evaluation/evidence/V2/`.
- V2 iteration screenshots and earlier-state captures: `prompt_appendix/task3_v2_evidence/evidence/` and `iterations/`.

These companion files are part of the submission directory and preserve full code, machine-readable outputs and visual evidence.